
PROGRAMMING IN PYTHON

assist. Sorin Milutinovici



Contents

1	Introduction	7
2	Execution Architecture: From C Processes to Python Runtime Semantics	8
2.1	Operating Systems, Programs, and Process Mechanics	9
2.1.1	Foundations of Execution Architecture	10
2.1.2	The Traditional Compilation Pipeline (The C Blueprint)	15
2.1.3	Program Loading and Execution Mechanics	50
2.1.4	Memory Layout in Bare-Metal Compiled Processes	60
2.1.5	Operating System Resource Management	74
2.2	Process Virtual Machines and Interpreted Runtimes	92
2.2.1	From Native Executables to Runtime-Hosted Programs	92
2.2.2	Abstracting the Hardware Layer: The Process VM Blueprint	107
2.2.3	Comparative Runtime Case Studies	127
2.3	Architectural Roadmaps of Modern Procedural Extensions	147
2.4	Architectural Roadmaps of Modern Procedural Extensions	147
2.4.1	Functions as Runtime Values and Saved Execution Context	148
2.4.2	Suspended Execution and Runtime-Managed Control Flow	182
3	Introduction to the Python Language Architecture	220
3.1	Historical Context and Design Evolution	220
3.1.1	Chronological Timeline: The Genesis, Origins, and Formal Release of Python (Guido van Rossum, 1989–1991)	220
3.1.2	Design Philosophy and Tensions: Reading “The Zen of Python” Critically	222
3.1.3	Why Python Spread: Scripting, Web Frameworks, Scientific Computing, Research Workflows, and Package Ecosystem Expansion	224
3.2	Python Implementations and the Role of CPython	227
3.2.1	Python as a Language vs. CPython as the Reference Implementation	227
3.2.2	CPython, PyPy, Jython, IronPython, and Alternative Runtime Strategies	230
3.2.3	Python Compilers, Accelerators, and Translators: Cython, Nuitka, Numba, MyPyC, and Related Tools	233
3.2.4	The Practical Consequence: Python Code Can Have Different Execution Backends but One Dominant Reference Runtime	237
3.3	The CPython Reference Implementation as Python’s Concrete Runtime	239
3.3.1	From Chapter 1’s Runtime Model to CPython’s Concrete Implementation	239
3.3.2	From Chapter 1’s Runtime Model to CPython’s Concrete Implementation	239
3.3.3	The Concrete Internal C Structure of the CPython Runtime	242
3.3.4	Parsing and Compiling Python Source: From .py Files to AST, Code Objects, Bytecode, and .pyc Marshaled Disk Artifacts	248
3.3.5	Code Objects, Frame Objects, and Namespace Dictionaries	253
3.3.6	The CPython Interpreter Loop as the Concrete Execution Site of Python Bytecode	259
3.4	CPython Memory Management Dynamics and Object Topologies	265
3.4.1	The Base Object Framework	265
3.4.2	The Heap Subsystem and Object Allocation	274

3.4.3	Introspection of Core Object Properties	299
3.5	Comparative Syntax and Language Paradigm Divergence	314
3.5.1	Visual Comparison: Python Code Shape vs. C-Style Braced Code	314
3.5.2	Architectural Case Study: High-Level Python Managed Code Realities vs. Bare-Metal Procedural C Coding	318
3.5.3	Code Blocks: From Explicit Braces (<code>{}</code>) to Indentation Tokens and Lexer-Level Whitespace Rules	322
3.6	Data Type Architecture and Runtime Type Systems	327
3.6.1	Classifying Type Systems: Static vs. Dynamic Variable Typing, Strong vs. Weak Enforcement Boundaries	328
3.6.2	Python's Type Boundary: Dynamic Names with Strong Runtime Type Constraint Verification	333
3.6.3	Names, Variables, and Object References: Why Python Variables Are Not C Boxes	338
3.6.4	Memory Storage Classifications: Built-in Single-Value Objects vs. Composite Reference Collections	345
3.6.5	Mutability and Identity: Immutable Objects, Mutable Objects, Shared References, and <code>is</code> vs. <code>==</code>	351
3.6.6	Object Interfaces: Dunder Methods, Internal Slots, Operators, Construction, and Representation	360
4	Variables and Singular Data Types	369
4.1	Identifiers and Syntax Rules for Variables	369
4.1.1	Legal and Structural Lexer Constraints	370
4.1.2	Stylistic Coding Conventions	378
4.2	Assignment and Name Binding	384
4.2.1	Simple Assignment: Creating or Rebinding a Name	384
4.2.2	Multiple Assignment and Unpacking	390
4.2.3	Augmented Assignment	400
4.2.4	Deleting Names	408
4.3	Singular Data Domains and Operator Precedence	415
4.3.1	The Integer Representation Architecture (<code>int</code>)	415
4.3.2	The Floating-Point Representation Architecture (<code>float</code>)	425
4.3.3	The Complex Representation Architecture (<code>complex</code>)	432
4.3.4	The Boolean Representation Architecture (<code>bool</code>)	439
4.3.5	The Null-Sentinel Object (<code>None</code>)	447
4.3.6	Numerical Operator Architecture and Precedence Graphs	454
4.4	Runtime Typing Consequences in Singular Operations	463
4.4.1	Dynamic Typing in Assignment	463
4.4.2	Strong Typing in Operations	469
4.4.3	Structural Summary	479
5	Sequential Component Data Architectures (Strings, Lists, and Tuples)	484
5.1	The General Sequence Model in Python	484
5.1.1	Ordered Component Storage and Positional Access	484
5.1.2	Sequence Operators and Shared Behaviors	493
5.2	Text Encoding Systems, Lexical Escape Maps, and Character Representation	503
5.2.1	Character Interoperability Standards	503
5.2.2	Native Conversion and Boundary Escape Sequences	512
5.2.3	Extended Universal Codepoint Escape Access	520
5.3	CPython String Memory Realities (<code>str</code>)	525
5.3.1	Structural Abstraction of the Unicode Standard	525
5.3.2	CPython Memory Optimization Architecture (PEP 393)	532
5.3.3	Structural Extraction and Evaluation	540
5.3.4	String Sequence Operations	548

5.4	Dynamic Pointer Arrays: The Python List Architecture (list)	555
5.4.1	The Memory Layout Paradigm Contrast	556
5.4.2	CPython Dynamic Scaling Mechanics	561
5.4.3	List Mutation Operations	567
5.4.4	Deep vs. Shallow Structural Cloning	579
5.5	Fixed Structural Contiguity: The Tuple Architecture (tuple)	587
5.5.1	Tuple Syntax and Structural Role	587
5.5.2	Immutable Sequence Invariants	595
5.5.3	CPython Allocation Optimizations	600
6	Control Flow Graphs, Lazy Traversal, and Exception Routing	608
6.1	From Linear Source Text to Control Flow Graphs	608
6.1.1	Statements, Basic Blocks, and Execution Edges	608
6.2	Code Block Syntax and Indentation Semantics	615
6.2.1	The Philosophy of Syntactic Whitespace	615
6.2.2	Comparative Paradigm Analysis	622
6.2.3	Empty Blocks and Structural Placeholders	626
6.3	Conditional Statements and Decision Trees	630
6.3.1	Syntax of Branching Control Graphs	630
6.3.2	Logic Evaluation Architecture	638
6.4	Arithmetic Sequence Descriptors: The range() Object	647
6.4.1	The Lazy Evaluation Invariant	647
6.4.2	Sequence Behavior Metrics	652
6.5	Iterative Loops and the Iterator Protocol	659
6.5.1	Indefinite Iteration: The while Loop	660
6.5.2	Definite Iteration: The for Loop	665
6.5.3	Iterable vs. Iterator	673
6.5.4	Interrupting Execution Flow Subsystems	681
6.5.5	Loop- else Completion Semantics	687
6.6	Lazy Traversal Objects and Iteration Helpers	694
6.6.1	Enumerated Traversal	694
6.6.2	Parallel Traversal	698
6.6.3	Transforming and Filtering Iteration Streams	704
6.6.4	Materialization Boundaries	711
6.7	Exceptional Control Flow and Error Routing	718
6.7.1	The Architecture of Non-Local Jumps	718
6.7.2	Exception Class Hierarchies and Handler Selection	725
6.7.3	Exception Handling Infrastructure	732
6.7.4	Exception Handling Infrastructure	743
7	Hash-Based Collections and Associative Mappings	753
7.1	Hash Tables as Non-Sequential Component Access Structures	753
7.1.1	From Positional Access to Hash-Based Access	753
7.1.2	The Hash and Equality Contract	759
7.2	Unordered Unique Domains: The Set Architecture (set)	769
7.2.1	Mathematical Foundations and Structural Syntax	769
7.2.2	Constraints of Element Ingestion and Runtime Engines	776
7.2.3	Core Set Mutation Operations	785
7.2.4	Immutable Set Variants	792
7.3	Set Mathematical Operators and Mutator Methods	797
7.3.1	Fundamental Set Calculations	797
7.3.2	In-Place Memory Mutation	803
7.3.3	Structural Relationship Evaluation	810
7.4	Dictionaries (dict) — Key-Value Associative Mappings	815

7.4.1	Foundations of Associative Mapping	815
7.4.2	Dictionary Construction Patterns	825
7.4.3	CPython Architectural Evolution	833
7.4.4	Ordering Guarantees and Misconceptions	842
7.4.5	Querying, Manipulating, and Mutating Dictionary States	847
8	Function Execution Mechanics, Lexical Scopes, and Advanced Control Architecture	863
8.1	Anatomy and Definition of Functions	863
8.1.1	Structural Syntax and Function Object Creation	864
8.1.2	Return Semantics and Frame Termination	874
8.2	Function Call Mechanics and Parameter Binding	890
8.2.1	Memory Semantics of Parameter Passing	890
8.2.2	Function Signature Architecture	904
8.3	Flexible Parametrization Systems	924
8.3.1	Variadic Positional Parameters	924
8.3.2	Variadic Keyword Parameters	935
8.3.3	Architectural Pitfalls of Argument Evaluation	951
8.4	Namespaces and Variable Scope Resolution	964
8.4.1	The LEGB Rule Invariant	964
8.4.2	Mutating External Scopes	978
8.5	Runtime Scope Mechanics and Variable Binding Analysis	992
8.5.1	Anatomy of Scope Failures	993
8.5.2	Compile-Time Scope Disambiguation	1006
8.6	First-Class Functions, Closures, and Function Transformation	1026
8.6.1	First-Class Citizens and Higher-Order Functions	1026
8.6.2	Lexical Closures	1042
8.6.3	Decorators	1057
8.7	Non-Preemptive Multitasking: Generators and Coroutines	1072
8.7.1	Lazy Stream Evaluation: Generators	1072
8.7.2	Generator Delegation	1087
8.7.3	Bidirectional Data Flow Pipelines: Generator-Based Coroutines	1099
8.7.4	Native Coroutine Bridge	1114
9	Input/Output Architecture, File Streams, and External Data Boundaries	1128
9.1	The I/O Boundary: From Runtime Objects to External Systems	1128
9.2	The I/O Boundary: From Runtime Objects to External Systems	1128
9.2.1	Programs as Data Consumers and Data Producers	1129
9.2.2	The Operating System Mediation Layer	1133
9.3	File Opening, Closing, and Resource Lifetime	1136
9.3.1	The <code>open()</code> Function and File Object Construction	1137
9.3.2	Resource Management Invariants	1140
9.4	Text File Reading and Writing	1144
9.4.1	Reading Textual Data	1145
9.4.2	Writing Textual Data	1148
9.4.3	Encoding and Decoding Boundaries	1152
9.5	Binary File Reading and Writing	1157
9.5.1	Byte-Oriented Data Streams	1157
9.5.2	Binary Access Patterns	1162
9.6	Filesystem Path Architecture	1167
9.6.1	Paths as Structured Filesystem References	1167
9.6.2	Filesystem Inspection and Manipulation	1171
9.7	Structured Data Interchange	1177
9.7.1	JSON Data Boundaries	1177
9.7.2	CSV Tabular Data	1187

9.8	Error Handling in I/O Operations	1187
9.8.1	Expected Failure Modes	1187
9.8.2	Defensive I/O Patterns	1191
9.9	Practical External Data Pipelines	1196
9.9.1	Streaming Large Inputs	1196
9.9.2	Transforming External Data	1200
List of Figures		1206
Bibliography		1207

Chapter 1

Introduction

Python is a magnificent dynamic illusion layered over a massive, hyper-optimized C application. It is the ultimate "glue" language, but an absolute nightmare for hardware cache efficiency. It trades CPU cycles and memory density wholesale for human iteration speed. For an engineer raised on bare metal, looking under Python's hood reveals a fascinating paradox: a language that presents a clean, minimalist facade to the programmer, while forcing the underlying runtime framework to work furiously behind the scenes to maintain that illusion.

Chapter 2

Execution Architecture: From C Processes to Python Runtime Semantics

A Python course can begin with Python syntax, but that approach leaves the student with a dangerous illusion: that programming is mainly the act of arranging visible language constructs. It is not. Programming is the act of describing computations that must eventually be represented, loaded, scheduled, executed, interrupted, resumed, and stored inside a real machine. A student who only learns the visible syntax of Python may become comfortable writing small scripts, but will often remain unable to explain why variables behave differently from C variables, why function calls create frames, why lists store references rather than raw values, why exceptions can unwind several calls at once, why generators can resume after yielding, or why asynchronous programs can handle many waiting tasks without using many CPU cores.

This chapter therefore starts below Python. It begins with the native execution model, using C as the reference architecture. This is not a detour. It is the baseline. C shows the traditional path clearly: source code becomes object code, object code becomes an executable, the operating system loader maps that executable into memory, the process receives a stack and heap, and the CPU executes machine instructions from the text segment. Once this model is understood, Python can be explained honestly: not as magic, not as “just interpreted”, and not as a loose collection of convenient syntax rules, but as a runtime-hosted language executed inside a native interpreter process.

The analogy is simple. One may learn to move a car in a parking lot without understanding roads, intersections, lanes, bridges, roundabouts, traffic lights, and right-of-way rules. But that person is not yet a competent driver. In the same way, one may learn to print strings, write loops, and define functions without understanding processes, memory, frames, objects, bytecode, and runtime scheduling. That person can operate the surface of the language, but cannot yet understand the execution environment in which real programs live.

The first section of the chapter establishes the native process model. It explains what a program is, what a process is, what the operating system kernel does, how compilation transforms C source code into executable form, how loading creates an active virtual address space, and how memory is divided into text, data, BSS, heap, and stack regions. This gives the student a concrete mental model for compiled execution: where instructions live, where variables live, how function calls use stack frames, why heap allocation must be managed, and why the operating system remains in control even when the program is native machine code.

The second section introduces process virtual machines and interpreted runtimes. This is where the Python execution model becomes visible. When a Python file is run, the operating system does not load that file as a native executable. It loads the CPython executable. The Python file is input to a runtime. CPython parses it, compiles it to bytecode, creates code objects and frames, allocates Python objects, and executes virtual instructions through an interpreter loop. This section also compares CPython with the JVM, V8, PHP, and Bash, because the word *interpreted* is too vague to be useful unless we separate source interpretation, bytecode interpretation, opcode execution, JIT compilation, host-provided event loops, and command dispatch.

The third section studies modern procedural extensions: callbacks, anonymous functions, closures,

generators, coroutines, and asynchronous execution. These constructs are placed here because they only make full sense after the runtime model is clear. A closure requires preserved lexical state beyond an ordinary stack frame. A generator requires suspended execution state. A coroutine requires controlled resumption. An asynchronous task requires the runtime or event loop to remember what should continue after an external event becomes ready. These are not just elegant language features. They are consequences of treating code, frames, continuations, and waiting operations as runtime-managed structures.

The chapter is therefore laid out as a bridge. It moves from native execution to runtime-hosted execution, and then from ordinary procedures to runtime-managed control flow. This order is deliberate. Python's variable model cannot be explained well before the object model. The object model cannot be explained well before the heap and references. Python frames cannot be explained well before native stack frames. Python bytecode cannot be explained well before machine code and instruction streams. Async execution cannot be explained well before scheduling, blocking, and suspended state. The chapter builds these dependencies in the order in which they become intelligible.

The purpose is not to turn a Python book into an operating systems book or a compiler construction book. The purpose is to give Python the execution context it deserves. A serious programming language is not only its syntax. It is a complete execution architecture. Python becomes much easier to understand when seen as a managed runtime layered above the native process model inherited from systems such as C.

This approach also protects the student from misleading simplifications. Python variables are not C boxes. Python assignment is not memory copying. Python functions are not only named source blocks. Python exceptions are not only error messages. Python generators are not lists. Python concurrency is not automatically parallelism. Each of these facts becomes clear only when the underlying architecture is visible.

For that reason, this chapter is not preliminary decoration. It is the foundation of the course. A student who understands this chapter will read the rest of Python differently. Syntax will no longer be memorized as isolated rules. It will be interpreted as surface notation for deeper mechanisms: objects, bindings, frames, bytecode, namespaces, managed memory, and runtime-controlled execution.

2.1 Operating Systems, Programs, and Process Mechanics

Before Python can be understood as a programming language, it is useful to understand what happens underneath any language when code is executed. A program does not run in isolation, directly on the physical machine, simply because a programmer wrote instructions in a source file. Between source code and execution there are several layers: executable files, operating-system loaders, process memory layouts, virtual address spaces, kernel scheduling, and hardware-supported resource isolation. These layers form the execution environment in which both C programs and Python programs eventually run.

This section uses the traditional C execution model as the reference point. C is useful here because it exposes the native path clearly: source code is transformed through preprocessing, compilation, assembly, and linking into a binary executable; the operating system loads that executable; the loader maps code and data into virtual memory; the process receives a stack, heap, registers, arguments, and resources; and the CPU begins executing machine instructions from the program's text segment. This path gives us a concrete model of what a native program is and how it becomes an active process.

The main distinction developed in this section is the distinction between a *program* and a *process*. A program is a stored blueprint: a file or collection of files containing instructions, data, and metadata. A process is the running instance created from that blueprint. The process has its own virtual address space, its own stack, heap, mapped code regions, open resources, and scheduling identity. This distinction is essential because Python code will later be shown not as a native executable in the ordinary C sense, but as input executed inside an already running interpreter process.

We will also examine the role of the operating system kernel. The kernel is the privileged layer that creates processes, maps memory, protects address spaces, handles traps and system calls, schedules CPU time, and mediates access to files, devices, and other resources. Even a C program compiled

to machine code does not own the machine. It runs inside a boundary created and enforced by the operating system. Its pointers are process-virtual addresses, its files are kernel-managed resources, and its execution can be suspended and resumed by the scheduler.

The memory layout of a native process is another central theme. We will separate the text segment, data segment, BSS, heap, and stack. These regions explain where machine instructions live, where global and static variables are stored, where dynamic allocations are created, and how function calls produce stack frames. This is the model that C programmers usually approach directly through pointers, arrays, `malloc()`, `free()`, function calls, and local variables.

Finally, we will connect process mechanics to multitasking. Modern operating systems do not run only one program until completion. They time-slice CPU resources, suspend and resume processes or threads, switch execution contexts, and distribute work across multiple cores. This explains the difference between processes and threads, between isolation and shared memory, and between operating-system scheduling and runtime-level scheduling. That last transition prepares the ground for later Python concepts such as generators, coroutines, event loops, and asynchronous execution.

The purpose of this section is not to turn the chapter into an operating-systems course. The purpose is to establish the execution model that Python will modify, abstract, and partially hide. Once the native C model is clear, the Python model becomes easier to describe precisely: the operating system runs the CPython executable as a normal native process, while the user's Python source file is parsed, compiled to bytecode, and executed inside that process by the Python runtime.

2.1.1 Foundations of Execution Architecture

What is a Program? The Static Binary Blueprint and Executable Formats (ELF on Linux, PE/COFF on Windows)

A program is a stored description of future execution. Before it runs, it is not yet an active computation. It is only data placed somewhere in persistent storage: on a disk, inside an archive, inside a package, or inside another executable container. In the native compiled model, such as the model normally used by C, the final program is usually an executable binary file. This file contains machine instructions, static data, metadata, relocation information, and structural descriptions that tell the operating system how the file must be loaded into memory.

This distinction is important. A program is not the same thing as a process. A program is the static object. A process is the active execution instance created from that object. The same executable file can be launched many times, producing many independent processes. Each process receives its own execution state: virtual memory mappings, stack, heap, open resources, CPU register state, and scheduling identity. The executable file is therefore only the blueprint from which the operating system constructs a running process.

In C, after preprocessing, parsing, compilation, assembly, and linking, the final result is commonly a binary executable. This binary is not just a flat sequence of CPU instructions. It has a formal file format. The operating system loader must be able to read this format, validate it, and map its parts into the future process address space. Two major executable format families are especially relevant:

- **ELF** (*Executable and Linkable Format*), used mainly on Linux and other Unix-like systems;
- **PE/COFF** (*Portable Executable / Common Object File Format*), used on Windows.

These formats solve the same basic problem: they describe how a compiled program is organized so that the operating system can load it and the processor can execute it. The file must contain enough information for the loader to answer questions such as:

- What processor architecture is this program compiled for?
- Where is the machine code stored inside the file?
- Which parts of the file must become readable, writable, or executable memory?
- What dynamic libraries are required?

- Where is the initial entry point of execution?
- What symbols or relocations must be resolved before execution can begin?

In an ELF executable, for example, the file contains an ELF header, program headers, sections, and segments. Sections are mainly useful during linking and analysis. Segments are especially important during execution because they describe what the operating system must map into memory. A typical executable contains a code area, often called the `text` segment, together with areas for initialized data, uninitialized data, read-only constants, dynamic linking information, and other runtime metadata.

On Windows, PE/COFF plays a similar role. A Windows executable file, usually ending in `.exe` or `.dll`, contains headers and tables that describe code sections, data sections, imported libraries, exported symbols, relocation information, and the program entry point. The exact structure is different from ELF, but the purpose is the same: the file is a structured contract between the compiler toolchain, the operating system loader, and the processor architecture.

The processor cannot execute an ELF file or a PE file as a whole. The file format is not itself the instruction stream. Instead, the operating system reads the file, interprets its metadata, maps selected parts into virtual memory, prepares the stack and other process structures, resolves required dynamic dependencies, and then transfers control to the program's entry point. Only after this loading phase do the machine instructions begin to execute.

This also explains why a native executable is tied to both an operating system format and a processor instruction set. A Linux ELF binary compiled for x86-64 cannot normally be launched directly as a Windows PE executable. A Windows PE executable compiled for x86-64 cannot directly run on an ARM processor without a compatibility or emulation layer. A native program must match both sides of the execution environment:

- the **operating system**, because the loader must understand the executable format and system call conventions;
- the **processor architecture**, because the machine instructions must belong to the processor's instruction set.

This native model is the reference point for understanding why Python behaves differently. A Python source file, such as `script.py`, is also a program in the broad sense: it is a stored description of future execution. However, it is not normally an executable binary containing native machine instructions for the processor. The operating system does not load `script.py` in the same way it loads a C executable. Instead, the operating system loads the Python interpreter executable, such as `python` or `python.exe`. That interpreter is the native program. The Python source file becomes input data for that running interpreter process.

Therefore, in the C model, the programmer's source code is transformed into the native executable that the operating system loads. In the Python model, the interpreter is the native executable, while the user's Python file is interpreted, compiled to bytecode, and executed inside the runtime environment created by that interpreter. This difference is one of the central architectural transitions between C programming and Python programming.

What is a Process? A Running Program with Its Own Virtual Address Space

A process is a running instance of a program. If the program is the static blueprint stored on disk, the process is the active object created when the operating system loads that blueprint and gives it execution resources. A process is therefore not just the executable file copied into memory. It is a complete runtime container: it has memory mappings, an execution state, open resources, scheduling information, and a relationship with the operating system kernel.

The same program file can produce many processes. If the user starts the same executable three times, the disk file is still one file, but the operating system creates three separate processes. Each process has its own identity, usually represented by a process identifier, or PID. Each process also has its own execution state. One instance may be waiting for keyboard input, another may be writing

to a file, and another may already be finished. They all come from the same program, but they are different runtime entities.

The most important property of a modern process is its **virtual address space**. From the point of view of the running program, memory appears as a large private address range. The program uses addresses as if it owned this memory directly. In reality, the operating system and the memory management unit (MMU) translate those virtual addresses into physical memory locations or into other backing storage mechanisms.

This gives every process an isolated view of memory. One process cannot normally read or write the memory of another process just by using an address. The address `0x100000` inside one process does not mean the same physical location as `0x100000` inside another process. The same virtual address can exist in many processes at the same time, but the operating system maps it differently for each one.

This isolation is one of the main differences between a simple program image and a real process. The executable file contains descriptions of code and data. The process contains mapped memory regions created from those descriptions, plus additional runtime regions created by the operating system and by the runtime libraries. A typical process address space contains regions such as:

- a **text** or code region, containing executable machine instructions;
- read-only constant data;
- initialized global and static data;
- uninitialized global and static data, often called BSS;
- the **heap**, used for dynamic allocation;
- one or more **stacks**, used for function calls and local execution state;
- memory-mapped libraries;
- kernel-provided or runtime-provided auxiliary regions.

In a C program, this structure is visible through concepts such as global variables, local variables, pointers, `malloc()`, `free()`, and function call stack frames. A global variable may live in the data or BSS region. A local automatic variable usually lives in a stack frame. A dynamically allocated object usually lives on the heap. A pointer stores an address inside the process virtual address space.

The processor does not execute an entire process at once. At any exact moment, a hardware thread executes one instruction stream. For a process to run, the operating system scheduler assigns CPU time to it. The process then executes for a while, may be interrupted, may block while waiting for input or output, or may voluntarily give control back to the kernel through a system call.

To suspend and later resume a process, the operating system must preserve its execution context. This includes, among other things, the current instruction pointer, stack pointer, register values, scheduling state, and memory mapping information. When the scheduler later chooses the process again, this saved state is restored, and execution continues as if the process had not stopped. This save-and-restore operation is part of what makes multitasking possible.

A process also owns or references external resources. These resources are not only memory. A process may have:

- open file descriptors or file handles;
- network sockets;
- environment variables;
- command-line arguments;
- loaded shared libraries;

- child processes;
- security credentials and permissions.

These resources are managed by the operating system. The process can request operations on them, but it normally does so through system calls or library wrappers around system calls. User code does not directly control the disk controller, network card, or physical memory. It asks the kernel to perform controlled operations.

This explains why the process is an operating-system object, not only a programming-language object. A C function, a Python function, or a Java method exists inside some runtime model, but the process itself is created and supervised by the operating system. The process is the boundary at which the operating system gives a running program memory isolation, resource ownership, and scheduling identity.

The process model also helps clarify what happens when Python code is executed. When the command

```
python script.py
```

is launched, the operating system does not create a process directly from `script.py`. The file `script.py` is not normally a native executable file. Instead, the operating system creates a process from the native Python interpreter executable: `python` on Linux or `python.exe` on Windows. That interpreter process then opens the Python source file, parses it, compiles it to bytecode, and executes it inside the CPython runtime.

Therefore, in the Python case, there are two different levels that must not be confused. At the operating-system level, the running process is the CPython interpreter process. At the language level, the user's Python program is being executed inside that process. The Python program has its own Python-level objects, frames, namespaces, and bytecode execution state, but all of these live inside the memory and resource boundary of the interpreter process created by the operating system.

This is the key transition from C to Python. In C, the programmer's source code is compiled and linked into the native executable that becomes the process. In Python, the interpreter is the native executable that becomes the process, while the user's source file becomes input to a managed runtime already running inside that process.

The Role of the Operating System Kernel: Supervisor Mode, Hardware Traps, System Calls, and Resource Isolation

The operating system kernel is the privileged core of the operating system. It is the software layer that stands between ordinary programs and the physical machine. A user program does not normally control the processor, memory, disk, keyboard, network card, or display directly. Instead, it runs under restrictions imposed by the kernel and must request privileged operations through controlled entry points.

Modern processors usually separate execution into at least two privilege levels. The exact names differ across architectures, but the conceptual distinction is stable:

- **user mode**, where ordinary application code runs with restricted permissions;
- **kernel mode** or **supervisor mode**, where the operating system kernel runs with privileged access to hardware and process management structures.

A C program, a Python interpreter process, a browser, or a text editor normally runs in user mode. User-mode code cannot directly modify page tables, program the disk controller, access arbitrary physical memory, or take control of another process. These restrictions are not just software conventions. They are enforced by the processor and the memory management unit.

The kernel runs in supervisor mode because it must perform operations that affect the whole system. It creates processes, schedules CPU time, maps virtual memory, handles files, communicates with devices, enforces permissions, and responds to hardware events. If every program could perform

these operations directly, one incorrect pointer, one malicious write, or one infinite loop could damage the whole machine.

The transition from user mode to kernel mode happens through controlled mechanisms. One of the most important mechanisms is the **system call**. A system call is a request made by a user program to the kernel. For example, when a program wants to open a file, allocate more memory, write to the console, create a process, or send data through a socket, it eventually performs a system call.

From the programmer's point of view, this request is often hidden behind a library function. In C, a call such as `printf()` may eventually lead to a lower-level write operation. In Python, a call such as `print()` or `open()` eventually reaches operating-system services through the CPython runtime and the C standard library or platform APIs. The programmer does not normally write the raw system call instruction directly.

A system call requires a controlled change of privilege level. The processor must stop executing ordinary user code and enter kernel code at a known, valid entry point. This transition is not a normal function call. A normal function call stays inside the same privilege level and the same program address space. A system call crosses the boundary between the user program and the operating system kernel.

This boundary crossing is usually implemented using a special processor instruction or a similar architecture-specific mechanism. Conceptually, the program places a request number and arguments in agreed locations, then triggers the transition. The processor switches into supervisor mode and transfers control to the kernel. The kernel validates the request, checks permissions, performs the operation if allowed, places a result or error code where the user program can retrieve it, and then returns control to user mode.

Another important mechanism is the **hardware trap**. A trap is a controlled transfer of execution to the kernel caused by a special event. Some traps are intentionally triggered by programs, such as system calls. Others are caused by exceptional conditions during execution. Examples include division by zero, invalid memory access, illegal instructions, page faults, and breakpoints used by debuggers.

For example, if a C program dereferences an invalid pointer, the processor may detect that the virtual address is not mapped or is not accessible with the requested permissions. The processor then traps into the kernel. The kernel examines the situation. If the access is invalid, it may terminate the process and report a segmentation fault or access violation. If the access corresponds to a valid page that must be loaded or mapped, the kernel may repair the situation and resume the process.

This trap mechanism is central to virtual memory. A process believes it is using a private address space, but the mapping from virtual addresses to physical memory is controlled by the kernel and the MMU. When a process touches a memory page that is not currently available, the hardware can trap into the kernel. The kernel then decides whether the access is legal, whether the page should be loaded, or whether the process must be stopped.

The kernel therefore enforces **resource isolation**. Each process receives the illusion of private memory and controlled access to external resources. One process cannot normally overwrite another process's stack, corrupt another process's heap, close another process's files, or steal another process's network connection. The kernel maintains tables that describe which resources belong to which process and what operations are allowed.

Resource isolation applies to several major areas:

- **memory isolation:** each process receives its own virtual address space;
- **CPU scheduling:** the kernel decides which process or thread runs and for how long;
- **file access:** the kernel checks permissions and manages file handles or descriptors;
- **device access:** programs use kernel-mediated interfaces instead of controlling hardware directly;
- **inter-process boundaries:** communication between processes must use approved mechanisms such as pipes, sockets, shared memory, or signals.

This model also explains why a process can fail without necessarily destroying the whole system. If a user program crashes, the kernel can tear down that process, reclaim its memory, close its resources, and continue running other processes. The process boundary acts as a containment boundary.

Coming from C, this distinction is essential. C gives the programmer direct access to addresses inside the current process. A pointer can refer to memory in the process address space, and incorrect pointer use can corrupt that process. But this does not mean the C program has direct access to all physical memory. Its pointers are still interpreted inside a virtual address space controlled by the operating system. The kernel and MMU decide which addresses are valid for that process.

The same operating-system boundary exists when Python code runs. Python code executes inside the CPython interpreter process. The Python program may create objects, lists, dictionaries, frames, and bytecode-level execution state, but all of these live inside the virtual address space of the interpreter process. When Python code opens a file, prints to the terminal, reads from the keyboard, or creates a network connection, the operation eventually crosses into the kernel through system calls made by the runtime or its libraries.

Thus, the kernel is not merely background software. It is the authority that transforms a static executable into a protected process, gives that process memory and resources, controls transitions to hardware services, and prevents ordinary programs from damaging each other. The programming language runtime, whether C's runtime library or CPython's interpreter, operates inside the boundaries that the kernel creates and enforces.

2.1.2 The Traditional Compilation Pipeline (The C Blueprint)

Core Definition, Intent, and Objectives of Compilation vs. Interpretation

Compilation and interpretation are two different strategies for moving from a human-written program to executed behavior. They are often presented as opposites, but in real language implementations the boundary is not absolute. Many systems combine both mechanisms. A language may compile source code into an intermediate form and then interpret that intermediate form. Another implementation of the same language may compile parts of the program into native machine code at runtime. Therefore, the important distinction is not the marketing label of the language, but the concrete execution path used by a specific implementation.

In the strict native sense, **compilation** means transforming source code into machine code before the program is executed. The compiler reads a program written in a high-level language and produces another program, usually closer to the hardware. In the C model, the final result of the complete toolchain is normally a native executable file. That executable contains machine instructions for a specific processor architecture and structural metadata for a specific operating system executable format.

In a broader sense, compilation means translation from one representation to another. A compiler may translate C source code into assembly, Java source code into JVM bytecode, Python source code into CPython bytecode, or TypeScript source code into JavaScript. The target does not have to be physical machine code. What matters is that the source program is transformed into another representation before that representation is executed or consumed by another stage.

Interpretation means that a runtime program directly drives the execution of another program representation. The interpreter reads source code, bytecode, an abstract syntax tree, or another internal structure, and performs the operations described by it. The interpreter itself is the native executable process. The user program is data consumed by that interpreter process.

This distinction is essential when comparing C and Python. In the usual C model, the programmer's source code is transformed into the native executable that the operating system loads. In the usual Python model, the operating system loads the Python interpreter executable. The user's `.py` file is then read, parsed, compiled to bytecode, and executed inside the interpreter process.

The goal of the traditional C compilation pipeline is not only to produce something that can run. It has several objectives:

- **translation**: convert high-level C constructs into lower-level machine instructions;
- **validation**: detect lexical, syntactic, semantic, and type errors before execution;
- **optimization**: rewrite the program representation to improve speed, memory use, or instruction layout;

- **target adaptation:** generate code for a specific processor architecture and operating system ABI;
- **composition:** combine separately compiled translation units and external libraries into one executable or library;
- **early binding:** resolve many names, types, offsets, and layout decisions before the program starts running.

The C model is especially important because it exposes the difference between source-level structure and runtime structure. A C file written by the programmer is not directly executed by the processor. It passes through a chain of transformations. Preprocessing expands macros and includes header declarations. Lexical analysis divides the text into tokens. Parsing organizes those tokens according to grammar rules. Semantic analysis attaches meaning to names, types, declarations, and scopes. Later phases lower the program into intermediate representations, optimize it, generate assembly, assemble object files, and link those object files into an executable.

The result of this pipeline is a native binary that can be loaded by the operating system. Once loaded, the processor executes machine instructions directly. The C runtime may still prepare the environment and call `main()`, but the user's program has already been transformed into native code. At runtime, there is no general C interpreter reading the original C source line by line.

An interpreted runtime works differently. In a simple interpreter, the runtime reads a program representation and executes operations as it goes. It may parse the source repeatedly, walk an abstract syntax tree, execute bytecode instructions, or combine several techniques. The key idea is that the runtime remains an active mediator between the program representation and the machine.

Python, specifically CPython, is a hybrid case. Python source code is not normally converted into a standalone native executable. However, CPython does compile Python source into bytecode. That bytecode is then interpreted by the CPython virtual machine. So the question "Is Python compiled or interpreted?" is badly formed if it expects only one answer. In CPython, Python is compiled to bytecode and then that bytecode is interpreted.

The difference is visible in the ownership of execution. In C, the compiled program becomes the native executable process. In Python, the interpreter is the native executable process, and the Python program is executed inside that process. This is why Python can offer features that are more difficult in traditional C: dynamic type inspection, runtime code execution, interactive evaluation, automatic memory management, and flexible object models. These features are not magical. They exist because a large runtime system remains present while the user program is executing.

Compilation and interpretation therefore represent two different placements of work. Compilation performs much of the analysis and transformation before execution begins. Interpretation keeps more decisions inside the runtime system while execution is happening. C emphasizes early transformation into native machine behavior. Python emphasizes runtime mediation through objects, bytecode, frames, namespaces, and managed memory.

Moving from C to Python, this is the first major architectural shift. The C programmer is used to thinking about a source file becoming an executable. The Python programmer must instead think about source code being consumed by an already-running runtime engine. The operating system runs CPython; CPython runs the Python program.

The Front-End Translation Phase

The front-end phase is the part of a compiler that understands the source language. In the C compilation model, the front end is responsible for reading C source text and transforming it into a structured internal representation that later compiler stages can analyze, optimize, and lower toward machine code. The front end is therefore the boundary between human-written source code and compiler-owned data structures.

This phase does not yet produce the final executable program. It does not yet decide the exact machine instructions that will run on the processor. Its main responsibility is to answer a different question: *what does this source code mean according to the rules of the programming language?*

For C, the front end must understand C syntax, C declarations, C type rules, C scope rules, C preprocessing conventions, and the structure of C translation units. A compiler front end for another language would be different. A Python front end, a Java front end, and a C front end do not accept the same grammar, do not build the same semantic information, and do not enforce the same compile-time rules. This is why the front end is usually described as the language-specific part of the compiler.

The front-end phase normally includes several connected operations:

- construction of the full source text that will be compiled;
- lexical scanning, where raw characters are grouped into tokens;
- syntactic analysis, where tokens are organized according to grammar rules;
- abstract syntax tree construction;
- semantic analysis, where names, declarations, types, and scopes are checked;
- construction of symbol tables and other internal compiler metadata.

The first important point is that source code is not understood as plain text. The compiler does not treat the program as a sequence of characters with vague meaning. It progressively transforms that text into increasingly structured representations. A fragment such as

```
x = a + 3;
```

is first just a sequence of characters. After lexical analysis, it becomes a sequence of tokens: identifier, assignment operator, identifier, addition operator, integer literal, semicolon. After parsing, those tokens become a structured expression and assignment statement. After semantic analysis, the compiler can attach information such as which declared object `x` refers to, which declared object `a` refers to, and whether the operation is type-correct.

The second important point is that the front end separates language meaning from hardware execution. A C expression such as

```
a + b * c
```

does not immediately become CPU instructions. First, the front end must determine its structure. According to C precedence rules, multiplication binds more tightly than addition, so the expression means:

```
a + (b * c)
```

not:

```
(a + b) * c
```

This distinction is not a machine-code issue yet. It is a source-language issue. The front end resolves it before the back end ever begins instruction selection.

The third important point is that the front end detects many classes of errors before execution. If the source file contains invalid tokens, malformed statements, undeclared identifiers, incompatible types, or impossible declarations, the compiler can reject the program before producing an executable. In C, this early rejection is especially important because the native executable will later run directly as machine code inside a process. The compiler must eliminate many invalid programs before they reach the execution stage.

However, not all errors can be caught in the front end. The front end may determine that an expression is syntactically and semantically valid, but the program may still fail at runtime. For example, pointer misuse, invalid memory access, division by zero in some situations, failed file operations, or

logic errors may still occur after the program has been compiled. The front end validates the source language structure, not the complete future behavior of the running process.

The output of the front end is usually not machine code. It is an internal representation of the program. One of the most important such representations is the abstract syntax tree, or AST. The AST removes many surface details of the source text and keeps the structural meaning of the program. Parentheses, separators, formatting, comments, and exact textual layout may disappear or become secondary. What remains is a tree-like structure representing declarations, expressions, statements, function definitions, control-flow constructs, and other source-level entities.

After the AST is built and checked, later compiler stages may transform it into lower-level intermediate representations. These representations are more convenient for optimization and code generation. For example, nested expressions may be broken into simpler operations, control flow may be made explicit, and temporary values may be introduced. This lowering process prepares the program for the middle-end and back-end stages.

This front-end model also helps explain Python. CPython also reads source text, tokenizes it, parses it, builds an AST, performs symbol-table analysis, and compiles the result into bytecode. The difference is not that C has a front end and Python does not. The difference is what happens after the front-end-like stages. In the traditional C pipeline, the program continues toward native object files, linking, and executable machine code. In CPython, the program continues toward Python bytecode executed by the CPython virtual machine.

Therefore, the front-end translation phase is the first major act of making source code executable. It does not yet execute the program. It transforms raw text into a structured, validated, language-aware representation that the rest of the compiler or runtime can process.

Source Code Ingestion and Translation Unit Construction

The first concrete step in the C compilation pipeline is source code ingestion. At this stage, the compiler driver receives one or more source files and prepares them for the language-specific front-end stages. A C program written by the programmer is stored as ordinary text, usually in files ending in `.c` and accompanied by header files ending in `.h`. The compiler cannot analyze the program as an abstract object until these text files are read, decoded as character streams, and organized into the compilation unit that will be processed.

In C terminology, the central object produced at this early stage is the **translation unit**. A translation unit is not simply the original `.c` file as written by the programmer. It is the result of taking one source file and applying preprocessing operations to it, especially header inclusion and macro expansion. Conceptually, the translation unit is the complete source body that the compiler proper will analyze after preprocessing.

For example, a small C file may contain:

```
#include <stdio.h>

int main(void) {
    printf("Hello, World!\n");
    return 0;
}
```

The programmer sees only a few lines. However, the line

```
#include <stdio.h>
```

instructs the preprocessor to include declarations from the standard input/output header. The compiler front end does not type-check the call to `printf()` by guessing what `printf` means. It needs a declaration describing that function. That declaration is made available through the included header. After preprocessing, the source presented to later compiler stages is much larger than the original visible file.

This is why source code ingestion in C is not only file reading. It also begins the process of constructing the source context in which names, declarations, macros, and included files become visible. A single `.c` file may depend on many headers. Those headers may include other headers. Conditional preprocessor directives may include or exclude parts of the source depending on macro definitions, platform settings, compiler flags, or build configuration.

The translation unit is therefore a boundary of independent compilation. In traditional C compilation, each `.c` file is usually compiled separately into an object file. If a project contains files such as

```
main.c
math_utils.c
io_utils.c
```

then the compiler usually constructs and compiles three separate translation units. Each translation unit produces one relocatable object file, such as:

```
main.o
math_utils.o
io_utils.o
```

These object files are later combined by the linker. This separate-compilation model is one of the reasons C programs can be built incrementally. If only `io_utils.c` changes, the build system may recompile only that translation unit and then relink the final executable.

The translation unit model also explains why header files are not normally compiled by themselves. A header file is usually not an independent program body. It is a reusable source fragment containing declarations, macros, type definitions, inline functions, constants, and other interface information. It becomes part of a translation unit when it is included by a `.c` file.

This model has important consequences. If the same header is included by many source files, its contents are inserted into many translation units. This is why C headers often use include guards or `#pragma once` to prevent repeated inclusion inside the same translation unit:

```
#ifndef MY_HEADER_H
#define MY_HEADER_H

/* declarations */

#endif
```

Without such protection, repeated inclusion could produce duplicate declarations, conflicting definitions, or unnecessarily large source representations.

At this stage, the compiler is still far from producing machine code. It is preparing the textual input so that the front end can perform lexical analysis, parsing, and semantic analysis. However, the decisions made during ingestion and translation unit construction already shape everything that follows. The set of included declarations determines which names are visible. Macro expansion may change the actual token stream that the compiler sees. Conditional compilation may cause different code to exist on Linux, Windows, or another platform.

This is also an important contrast with Python. A Python file is normally executed as a module body by the Python runtime. It may import other modules, but importing does not paste their source text into the importing file in the C preprocessor sense. In C, `#include` is a textual inclusion mechanism that helps construct one translation unit. In Python, `import` is a runtime/module-loading mechanism that binds names to module objects. The two operations may look similar to beginners because both reuse external code, but architecturally they are very different.

Thus, source code ingestion and translation unit construction form the first structural boundary in the C compilation pipeline. The compiler starts from programmer-written text files, expands the source context through preprocessing rules, and produces the complete source body that later phases can tokenize, parse, analyze, optimize, and eventually lower toward native execution.

Preprocessing Subsystems: Macro Expansion, Conditional Directives, and Header Inclusion

In the C compilation pipeline, preprocessing is the phase that transforms the source text before the normal compiler front end performs lexical analysis, parsing, and semantic analysis. The preprocessor works on a source file as text and token streams, not yet as a fully understood C program. Its purpose is to construct the effective source body that the compiler will actually analyze.

This is a major difference between what the programmer writes and what the compiler receives. A short C source file may expand into a much larger translation unit because of included headers, macro definitions, conditional compilation branches, and platform-specific declarations. The compiler proper does not usually analyze only the exact visible text written in the `.c` file. It analyzes the source after preprocessing.

The three most important preprocessing mechanisms are:

- **header inclusion**, through directives such as `#include`;
- **macro expansion**, through directives such as `#define`;
- **conditional compilation**, through directives such as `#if`, `#ifdef`, `#ifndef`, `#else`, and `#endif`.

Header inclusion is the mechanism through which C source files gain access to declarations written elsewhere. For example:

```
#include <stdio.h>
```

does not import a module in the Python sense. It instructs the preprocessor to insert the contents of the selected header into the translation unit. This is why a C file can call `printf()` after including `stdio.h`: the header provides the declaration that lets the compiler understand the function's existence and signature.

This mechanism is textual and structural. The header file becomes part of the source body that is compiled. If a header includes another header, that second header may also become part of the same translation unit. For this reason, C translation units can become very large even when the visible `.c` file is small.

Header inclusion also explains why include guards are needed. If the same header is included multiple times, the same declarations may be inserted multiple times into the same translation unit. To prevent this, headers commonly use a pattern such as:

```
#ifndef VECTOR_H
#define VECTOR_H

/* declarations */

#endif
```

The first inclusion defines the guard macro. Later inclusions see that the macro already exists and skip the guarded body. Many compilers also support:

```
#pragma once
```

as a simpler implementation-specific protection mechanism.

Macro expansion is another major preprocessor operation. A macro is a symbolic replacement rule. A simple object-like macro may be written as:

```
#define BUFFER_SIZE 1024
```

After this definition, appearances of `BUFFER_SIZE` in the source can be replaced by `1024` before compilation continues. The compiler proper will normally see the expanded value, not the original macro name.

Macros can also be function-like:

```
#define SQUARE(x) ((x) * (x))
```

A later expression such as:

```
SQUARE(a + 1)
```

may expand into:

```
((a + 1) * (a + 1))
```

This looks similar to a function call, but it is not one. A macro does not create a call frame, does not have normal parameter variables, and does not obey all the evaluation rules of a real function. It is a source-level replacement mechanism. Because of this, macros can be powerful but dangerous. A badly written macro can evaluate an argument multiple times, ignore type safety, or produce unexpected precedence behavior.

For example:

```
#define DOUBLE(x) x + x
```

used as:

```
int y = 3 * DOUBLE(4);
```

does not become:

```
int y = 3 * 8;
```

It becomes:

```
int y = 3 * 4 + 4;
```

because macro expansion is textual/token replacement before normal expression parsing. This is why defensive macro definitions often use parentheses around both parameters and the whole replacement expression.

Conditional compilation allows parts of the source to exist or disappear depending on preprocessor conditions. For example:

```
#ifdef _WIN32
    /* Windows-specific code */
#else
    /* POSIX/Linux-specific code */
#endif
```

Only one branch becomes part of the translation unit, depending on which macros are defined. This makes it possible for the same codebase to support multiple platforms, architectures, compiler versions, or build modes. Debug code, optional features, and operating-system-specific behavior are often controlled this way.

For example:

```
#ifdef DEBUG
    printf("Debug value: %d\n", value);
#endif
```

If `DEBUG` is defined, the debugging statement is included in the compiled source. If it is not defined, that code is removed before the compiler properly analyzes the program. This is not the same as a runtime `if` statement. With conditional compilation, the excluded branch is not compiled at all.

This distinction matters. In C, preprocessor conditions operate before parsing and semantic analysis. A branch removed by the preprocessor may contain code that would not even compile for the current platform. This is commonly used to separate platform-specific declarations and system calls.

Preprocessing can be observed directly with common compiler options. With GCC or Clang, the command:

```
gcc -E hello.c
```

runs preprocessing and prints the expanded source instead of producing an object file. This output can be surprisingly large because all included declarations and macro expansions have already been inserted.

Preprocessing therefore performs an early source-construction role. It does not understand the full meaning of the C program in the same way the compiler front end does. It does not build the final AST, does not perform full type checking, and does not generate machine code. Instead, it prepares the source text that those later phases will receive.

This is also one of the clearest differences between C and Python. C has a separate preprocessing language whose directives are executed before compilation. Python does not have an equivalent textual preprocessor in the normal language pipeline. Python's `import` statement does not paste source text into the current file. It loads or reuses module objects at runtime through the import system. Python constants, functions, and classes are ordinary runtime objects, not preprocessor replacement rules.

Thus, the C preprocessor belongs to the native compilation architecture. It constructs the translation unit by including files, expanding macros, and selecting conditional source branches before the actual compiler front end performs language-level analysis. It is one of the reasons C compilation is powerful, portable, and efficient, but also one of the reasons C code can be difficult to reason about when the visible source differs substantially from the source actually compiled.

Lexical Scanning and Tokenization Analysis

After preprocessing constructs the effective source text of a translation unit, the compiler must stop treating the program as an undifferentiated stream of characters. The next step is **lexical scanning**, also called **tokenization**. During this phase, the compiler reads the source character by character and groups characters into meaningful units called **tokens**.

A token is a classified fragment of source code. It is not only the text itself, but the text together with its role in the language. For example, in the expression

```
count = count + 1;
```

the compiler does not reason about the individual characters `c`, `o`, `u`, `n`, `t`, space, `=`, and so on. It first transforms the character stream into a sequence similar to:

```
IDENTIFIER(count)
ASSIGNMENT_OPERATOR(=)
IDENTIFIER(count)
PLUS_OPERATOR(+)
INTEGER_LITERAL(1)
SEMICOLON(;
```

This token stream is easier for later stages to process. The parser does not need to know how many characters were used to write an identifier or how many spaces appeared between two operators. It receives already-classified language units and checks whether they appear in an order allowed by the grammar.

Lexical scanning is therefore the first stage where the source text begins to acquire formal structure. The scanner recognizes categories such as:

- **keywords**, such as `int`, `return`, `if`, `else`, `while`, and `for`;
- **identifiers**, such as variable names, function names, and type names;
- **literals**, such as integer constants, floating-point constants, character constants, and string literals;
- **operators**, such as `+`, `-`, `*`, `/`, `=`, `==`, `++`, `->`, and `&&`;
- **punctuation and separators**, such as parentheses, braces, brackets, commas, and semicolons.

For example, the word `return` is not treated as an ordinary identifier in C. The lexical scanner classifies it as a keyword token. A programmer cannot use `return` as the name of a variable because the scanner has already assigned it a reserved syntactic role. By contrast, a word such as `total` is not reserved, so it can be classified as an identifier.

Whitespace also receives special treatment. In C, spaces, tabs, and newlines usually separate tokens but do not become important tokens themselves. The following two fragments are lexically equivalent in most relevant ways:

```
x = a + b;
```

and

```
x=a+b;
```

Both produce essentially the same token sequence. This is one of the important differences between C and Python. In C, block structure is marked with braces and statement endings are marked with semicolons. Whitespace is usually not part of the syntactic structure. In Python, indentation is significant and can generate structural tokens such as `INDENT` and `DEDENT`. Therefore, tokenization is language-specific.

Comments are also handled during or around lexical scanning. In C, comments are not meaningful program instructions. A comment such as

```
/* this is a comment */
```

or

```
// this is also a comment
```

is removed or ignored before the parser receives the token stream. Comments may help human readers, but they do not produce executable behavior.

The scanner must also handle ambiguity between tokens. For example, when it sees the characters

```
==
```

it must produce one equality-comparison token, not two assignment tokens. Similarly, the sequence

```
++
```

is the increment operator, not two separate plus operators. Lexical scanners typically follow a longest-match rule: when several token interpretations are possible, the scanner consumes the longest valid token.

This matters in cases such as:

```
a+++b
```

A scanner may group this as:

```
IDENTIFIER(a)
INCREMENT_OPERATOR(++ )
PLUS_OPERATOR(+ )
IDENTIFIER(b)
```

rather than as:

```
IDENTIFIER(a)
PLUS_OPERATOR(+ )
PLUS_OPERATOR(+ )
PLUS_OPERATOR(+ )
IDENTIFIER(b)
```

The resulting token stream may still later produce a syntax or semantic error, but the lexical grouping follows the language's tokenization rules.

Lexical analysis can also detect certain errors immediately. If the scanner encounters a character sequence that cannot form any valid token, it can stop compilation and report a lexical error. Examples include malformed numeric literals, unterminated string literals, invalid escape sequences, or illegal characters in the source file.

For example:

```
char *s = "hello;
```

contains an unterminated string literal. The scanner cannot correctly form the string token, so compilation cannot proceed normally.

However, lexical scanning does not determine whether the complete program makes sense. It can identify that `x`, `=`, `5`, and `;` are valid tokens, but it does not decide whether `x` has been declared, whether the assignment is type-correct, or whether the statement appears in a valid location. Those are responsibilities of later stages: parsing and semantic analysis.

The separation between tokenization and parsing is important. Tokenization answers the question:

What are the meaningful pieces of this source text?

Parsing answers a different question:

Do these pieces form valid grammatical structures?

Semantic analysis then asks:

What do these structures mean, and are they valid according to the language's type, scope, and declaration rules?

In the C compilation pipeline, lexical scanning is therefore a necessary narrowing step. The compiler starts with raw text, removes irrelevant layout and comments, classifies meaningful fragments, and produces a token stream. That token stream becomes the input for syntactic analysis, where the compiler begins constructing the grammatical structure of the program.

Syntactic Analysis: Concrete Syntax Tree Validation and Grammar Rules

After lexical scanning has transformed raw source text into a sequence of tokens, the compiler must determine whether those tokens form valid language structures. This phase is called **syntactic analysis**, or **parsing**. Its purpose is to check the token stream against the grammar of the programming language.

A grammar defines which token arrangements are legal. Lexical analysis can tell the compiler that a sequence contains identifiers, operators, literals, parentheses, braces, and semicolons. Parsing decides whether those tokens are arranged as valid declarations, expressions, statements, function definitions, loops, conditionals, and blocks.

For example, after tokenization, the statement


```
x = a + 3;
```

may be represented approximately as:

```
IDENTIFIER(x)
ASSIGNMENT_OPERATOR(=)
IDENTIFIER(a)
PLUS_OPERATOR(+)
INTEGER_LITERAL(3)
SEMICOLON(;
```

The parser must decide whether this sequence is a valid C statement. It recognizes that the sequence has the shape of an assignment statement: an assignment target on the left, an assignment operator, an expression on the right, and a semicolon marking the end of the statement.

Parsing is not concerned only with whether individual tokens are valid. All tokens in the following fragment may be individually legal:

```
x = + ; 3 a
```

However, the sequence does not form a valid C statement. The scanner can recognize the pieces, but the parser rejects their arrangement. This is the difference between lexical validity and syntactic validity.

Syntactic analysis also enforces structural rules. In C, a block must be enclosed in braces:

```
if (x > 0) {
    y = 1;
}
```

The parser recognizes the keyword `if`, the condition enclosed in parentheses, and the body enclosed in braces. If one of these structural elements is missing, the parser cannot build a valid structure:

```
if x > 0 {
    y = 1;
}
```

This may look readable to a human, but it is not valid C syntax because the condition is not enclosed in parentheses. The error is syntactic, not lexical. The tokens themselves are recognizable, but they do not satisfy the grammar rule for an `if` statement in C.

A parser usually builds a tree-like structure while checking the grammar. One possible form is a **concrete syntax tree**, also called a **parse tree**. A concrete syntax tree represents the grammatical structure of the source code very closely. It contains many details that come directly from the grammar: parentheses, separators, statement delimiters, and intermediate grammar categories.

For example, the expression

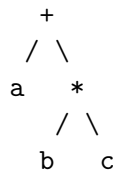
```
a + b * c
```

is not parsed as a flat list of five tokens. The grammar gives multiplication higher precedence than addition, so the parser builds a structure equivalent to:

```
a + (b * c)
```

The tree shape is important because it determines the meaning of the expression. The compiler must know that `b * c` is computed as a subexpression before the addition is applied.

A simplified tree representation could be imagined as:



This tree is not yet machine code. It is a structural representation of the expression according to the language grammar. Later stages may transform it into an abstract syntax tree, intermediate representation, and eventually instructions.

The phrase **concrete syntax tree validation** emphasizes that, at this point, the compiler is still checking whether the source text follows the explicit grammar of the language. In C, this includes rules such as:

- statements must often end with semicolons;
- code blocks are delimited by braces;
- function calls use parentheses after the function name;
- array indexing uses square brackets;
- operators must appear in grammatically valid positions;
- declarations must have valid declarator structure;
- control statements such as **if**, **while**, and **for** must follow their required forms.

For example:

```
int x = 5
```

is lexically valid, but syntactically incomplete in C because the semicolon is missing. The parser expects the declaration statement to end with `;`. Since it does not find it, it reports a syntax error.

Similarly:

```
while (x < 10)
    x++;
}
```

contains a closing brace without a matching opening brace for the loop body. The parser detects that the block delimiters do not match the grammar structure.

Syntactic analysis must also handle nested structures. In a fragment such as:

```
if (x > 0) {
    while (y < 10) {
        y++;
    }
}
```

the parser must correctly represent that the **while** statement is inside the **if** block, and that the increment statement is inside the **while** block. This hierarchy is not a decoration. It determines the control flow of the program.

This is where C differs strongly from Python. In C, braces and semicolons define much of the visible syntax. Indentation may help humans read the code, but it does not define the grammar. The following C code is badly formatted but syntactically valid:

```
if (x > 0) { y = 1; }
```

Python makes a different design choice. Python's parser depends on indentation tokens generated by the tokenizer. A Python block is not marked by braces, but by indentation structure. Therefore, in Python, layout is part of the syntactic structure, not merely visual style.

Parsing can detect many errors, but it still does not fully understand every meaning of the program. For example, the following C statement may be syntactically valid:

```
x = y + 3;
```

The parser can recognize it as a valid assignment statement. However, it does not by itself decide whether `x` and `y` were declared, whether their types are compatible, or whether the assignment is semantically valid. Those checks belong to semantic analysis.

Thus, syntactic analysis answers the question:

Do the tokens form valid structures according to the grammar of the language?

It does not yet answer the full question:

Do those structures make sense according to declarations, types, scopes, and runtime constraints?

This separation is important because the compiler pipeline proceeds by layers. Lexical analysis turns characters into tokens. Syntactic analysis turns tokens into grammatical structures. Semantic analysis attaches language meaning to those structures. Later stages lower and optimize those structures for execution.

In the C blueprint, syntactic analysis is one of the main places where the compiler moves from source text toward executable structure. The result is no longer just a list of tokens. It is a hierarchical representation of the program's grammatical form, ready to be simplified into an abstract syntax tree and enriched with semantic information.

Constructing the Abstract Syntax Tree (AST) Mapping

After the parser has verified that the token stream follows the grammar of the language, the compiler usually constructs a more compact and more useful representation of the program: the **abstract syntax tree**, or **AST**. The AST is a tree-shaped structure that preserves the meaningful organization of the source code while removing many surface-level syntactic details.

A concrete syntax tree is close to the grammar. It may contain nodes for parentheses, separators, punctuation, grammar helper rules, and other structural details that are necessary for parsing but not necessarily useful for later compilation. The abstract syntax tree keeps the important program operations: assignments, expressions, declarations, function definitions, control-flow structures, operators, literals, and name references.

For example, the expression

```
a + b * c
```

may be written as a flat line of text, but its meaning is hierarchical. Multiplication has higher precedence than addition, so the expression means:

```
a + (b * c)
```

The AST records this structure directly. A simplified AST-like representation could be imagined as:

```
BinaryOperation(+)
  left: Name(a)
  right: BinaryOperation(*)
    left: Name(b)
    right: Name(c)
```

The tree shape now expresses the real evaluation structure. The addition node has two children. The left child is the name `a`. The right child is another operation, `b * c`. This structure is more useful to the compiler than the original character sequence because the compiler no longer has to repeatedly re-interpret precedence and grouping rules.

The word *abstract* means that the tree does not preserve every textual detail. For example, the expressions

```
a + (b * c)
```

and

```
a+b*c
```

may produce essentially the same AST. The whitespace is gone. The optional parentheses around `b * c` may also disappear because the same grouping is already implied by operator precedence. The AST stores the meaning of the expression, not the exact formatting of the source file.

The AST also represents statements and larger program structures. For example, a C statement such as

```
if (x > 0) {
    y = 1;
} else {
    y = -1;
}
```

can be represented abstractly as an `if` node with three main parts:

- a condition expression: `x > 0`;
- a body executed when the condition is true;
- an alternative body executed when the condition is false.

A simplified AST-like representation could be:

```
IfStatement
  condition: BinaryOperation(>)
    left: Name(x)
    right: IntegerLiteral(0)
  then_body:
    Assignment
      target: Name(y)
      value: IntegerLiteral(1)
  else_body:
    Assignment
      target: Name(y)
      value: UnaryOperation(-)
        operand: IntegerLiteral(1)
```

This representation is still not machine code. It is also not yet necessarily the final intermediate representation used for optimization. It is a language-level structural map of the program. It tells the compiler what constructs exist and how they are nested.

The AST is useful because later compiler stages can walk the tree. To *walk* the tree means to visit its nodes in an organized way. The compiler can inspect declarations, check expressions, attach type information, find assignment targets, detect invalid constructs, lower complex expressions into simpler operations, or generate another representation from the tree.

In the C compilation pipeline, the AST is closely connected to semantic analysis. The parser can build a structural tree, but that tree still needs meaning attached to it. For example, a node named `Name(x)` is not enough. The compiler must eventually determine which declaration of `x` this node refers to, what type it has, where it is stored, whether it can be assigned to, and whether it is visible in the current scope.

Consider:

```
x = y + 1;
```

The AST can represent the assignment structure:

Assignment

```
target: Name(x)
value: BinaryOperation(+)
      left: Name(y)
      right: IntegerLiteral(1)
```

However, the AST alone does not guarantee that `x` and `y` are valid names. Semantic analysis must connect these name nodes to declarations. If `y` was never declared in the current or enclosing scope, the compiler must report an error.

This separation between structure and meaning is important. The AST says:

This program contains an assignment whose right side is an addition.

Semantic analysis later asks:

Are the names valid? Are the types compatible? Is the assignment allowed?

The AST can also be transformed. Later stages may lower the AST into a representation that makes control flow and data movement more explicit. For example, a complex expression may be broken into temporary values. A loop may be represented as jumps and labels in an intermediate representation. Function calls may be prepared according to calling conventions. These transformations move the program away from source-language structure and closer to executable behavior.

This is why the AST should not be confused with the final intermediate representation. An AST is usually still source-shaped. It resembles the programmer's language constructs. An intermediate representation used in the middle end is often more operation-shaped. It is designed for optimization, data-flow analysis, and eventual code generation.

The same idea appears in Python. CPython also parses source code and builds an AST before compiling it to bytecode. For example, a Python expression such as

```
a + b * c
```

must also be understood as a tree of operations, not as a flat string. CPython then compiles this tree into bytecode instructions for the CPython virtual machine. The destination differs from C, but the need for structured source representation remains.

This comparison is useful for students moving from C to Python. C uses an AST as part of a pipeline that normally continues toward native machine code. Python uses an AST as part of a pipeline that continues toward bytecode executed by a runtime interpreter. In both cases, the source text must first become a structured internal object before it can be analyzed or executed.

The abstract syntax tree is therefore the first major representation in which the program becomes visibly organized as computation. It is no longer raw text and no longer just a token stream. It is a tree of program constructs: expressions inside statements, statements inside blocks, blocks inside functions, and functions inside translation units or modules. This tree becomes the foundation for semantic analysis, lowering, optimization, and eventual execution.

Semantic Analysis: Symbol Tables, Type Information, Scope Rules, and Meaning Attached to Syntax Nodes

After syntactic analysis has built a valid structural representation of the program, the compiler must determine whether that structure is meaningful according to the rules of the language. This phase is called **semantic analysis**. The parser has already answered the question:

Are the tokens arranged in a grammatically valid way?

Semantic analysis answers a deeper question:

Do those grammatical structures make sense as a program?

For example, the statement

```
x = y + 1;
```

may be syntactically valid C. The parser can recognize it as an assignment statement with an expression on the right side. However, the compiler still needs to know whether `x` and `y` were declared, what their types are, whether `x` is assignable, and whether adding `1` to `y` is allowed. These are not purely syntactic questions. They are semantic questions.

Semantic analysis attaches meaning to the nodes of the abstract syntax tree. A node such as

`Name(x)`

is not useful enough by itself. The compiler must connect that name to a declaration. If the source file contains:

```
int x;
```

then later uses of `x` must be connected to that declared object. The compiler must record that `x` has type `int`, belongs to a certain scope, and may be used in certain operations.

To do this, the compiler builds and consults **symbol tables**. A symbol table is an internal compiler data structure that maps names to information about those names. For a C compiler, this information may include:

- the name of a variable, function, type, structure, or label;
- the type associated with that name;
- the scope where the name is visible;
- whether the name has internal or external linkage;
- whether storage is automatic, static, global, or dynamically determined elsewhere;
- whether the name refers to a declaration or a definition;
- source-location information used for diagnostics.

Consider the following program fragment:

```
int total = 0;

int add(int x, int y) {
    int result = x + y;
    return result;
}
```

The compiler must build entries for `total`, `add`, `x`, `y`, and `result`. These names do not all live in the same scope. The name `total` belongs to file or global scope. The name `add` also belongs to a broader scope. The names `x`, `y`, and `result` belong to the function body scope. A use of `x` inside `add` must refer to the parameter `x`, not to some unrelated name outside the function.

This is where **scope rules** become important. A scope is a region of the program where a name is visible and has a particular meaning. In C, block structure affects scope. A name declared inside a block is normally visible only inside that block and its nested blocks. For example:

```
int x = 1;

void f(void) {
    int x = 2;
}
```

There are two variables named `x`. They are not the same variable. The inner declaration shadows the outer one inside the function body. The parser can see two declarations with the same text name, but semantic analysis determines which declaration each use of `x` refers to.

Semantic analysis also checks **type information**. C is statically typed, so many type constraints are checked before the program runs. For example:

```
int x;
char *s;

x = s;
```

This is syntactically valid: an assignment target, an assignment operator, and an expression. But semantically, it is suspicious or invalid depending on the exact conversion rules and compiler settings, because `x` is an integer object and `s` is a pointer to character. The compiler must know the type of both sides before it can judge the assignment.

Function calls are also checked semantically. In:

```
printf("value = %d\n", x);
```

the compiler must know that `printf` is a function name, that it has been declared, and that it can be called. For ordinary functions with fixed signatures, the compiler can check the number and types of arguments. For variadic functions such as `printf`, some checks are weaker, but the function must still be known as a callable symbol.

Semantic analysis can detect errors such as:

- use of undeclared identifiers;
- incompatible assignments;
- invalid operands for operators;
- incorrect number of function arguments;
- returning a value from a `void` function;
- failing to return a value from a non-`void` function in some contexts;
- duplicate declarations in the same scope;
- invalid access to structure or union members;
- misuse of labels, `break`, `continue`, or `return`.

For example:

```
int main(void) {  
    y = 10;  
    return 0;  
}
```

The assignment statement is syntactically valid. However, if `y` has not been declared, semantic analysis rejects the program because the name has no known meaning in the current scope.

Similarly:

```
int main(void) {  
    break;  
    return 0;  
}
```

The token `break` is valid, and a `break` statement has valid syntax. But it is semantically invalid outside a loop or `switch`. The parser can recognize the statement; semantic analysis determines that it appears in a context where it is not allowed.

This phase is also where the compiler begins to enrich the AST. The original AST may contain a node for a binary addition operation:

```
BinaryOperation(+)  
  left: Name(a)  
  right: Name(b)
```

After semantic analysis, the compiler may know that `a` and `b` are both integers, or that one is a floating-point value and the other must be converted, or that the operation is invalid. The tree is no longer only structural. It now carries language meaning.

Semantic analysis also prepares later stages. The middle end and back end need information such as object sizes, alignments, storage classes, function signatures, and type conversions. The compiler cannot correctly generate code for an expression if it does not know whether the operation is integer addition, floating-point addition, pointer arithmetic, or something invalid.

For example, the source symbol `+` does not always map to the same machine-level operation. Adding two integers, adding two floating-point values, and adding an integer offset to a pointer are different semantic cases. The parser only sees the `+` token. Semantic analysis determines what kind of addition is meant.

This is an important distinction for students moving from C to Python. In C, much semantic information is fixed before execution. The compiler determines many name bindings, types, object layouts, and valid operations at compile time. In Python, the situation is different. Python also parses source code, builds an AST, and performs symbol-table analysis, but many value-level decisions are delayed until runtime.

For example, in Python:

```
x = y + 1
```

the compiler can determine that `x` is assigned in the current scope and that `y` is read. However, it does not necessarily know at compile time what object `y` will refer to when the statement executes. The meaning of `y + 1` is resolved at runtime by looking up the object bound to `y`, inspecting its type, and dispatching the addition operation through Python's object protocol.

Therefore, both C and Python perform structural analysis before execution, but they place semantic responsibility in different places. C pushes much more type and declaration meaning into the compile-time pipeline. Python keeps more meaning in the runtime system. This is one reason Python can support flexible rebinding and dynamic object behavior, while C can produce efficient native code with many decisions already resolved.

Semantic analysis is therefore the phase where the program stops being merely valid syntax and becomes a set of meaningful language operations. Names are connected to declarations or scope decisions. Types are checked. Context restrictions are enforced. Syntax nodes are enriched with information that later stages need in order to lower, optimize, or execute the program.

AST vs Intermediate Representation: Tree Structure, Lowering, and Optimization-Friendly Forms

The abstract syntax tree is an important internal representation, but it is usually not the final representation used for optimization and code generation. The AST is still close to the source language. It preserves the programmer's high-level constructs: expressions, statements, declarations, function definitions, loops, conditionals, and blocks. This makes it excellent for representing what the source code says, but not always ideal for deciding how the program should be optimized or translated into machine instructions.

An **intermediate representation**, usually abbreviated as **IR**, is a later internal form designed to be easier for the compiler to analyze, transform, and eventually lower into target-specific code. The IR is normally less concerned with preserving source-language shape and more concerned with making computation explicit.

The distinction can be summarized as follows:

- the **AST** is source-shaped;
- the **IR** is operation-shaped;
- the AST preserves language structure;
- the IR exposes computation, data movement, and control flow more directly.

For example, consider the expression:

```
x = a + b * c;
```

An AST representation keeps the hierarchical expression structure:

Assignment

```
target: Name(x)
value: BinaryOperation(+)
  left: Name(a)
  right: BinaryOperation(*)
    left: Name(b)
    right: Name(c)
```

This tree clearly shows that multiplication happens inside the right branch of the addition. It is a correct structural map of the expression. However, for optimization and code generation, the compiler may prefer a flatter sequence of simpler operations:

```
t1 = b * c
t2 = a + t1
x = t2
```

This second representation is no longer a source-shaped tree. It is closer to a list of elementary computational steps. Temporary values have been introduced. The nested expression has been broken into explicit operations. This transformation is part of what is often called **lowering**.

Lowering means converting a high-level representation into a lower-level representation. It does not necessarily mean generating machine code immediately. It means replacing complex source-level constructs with simpler internal forms that are easier to analyze and transform. A compiler may perform several lowering stages before reaching assembly or machine code.

For example, a high-level loop such as:

```
for (i = 0; i < n; i++) {
  sum += a[i];
}
```

may be represented in the AST as a **for** statement node with initialization, condition, update, and body components. A lower-level IR may instead represent it with labels, jumps, comparisons, loads, stores, and arithmetic operations:

```
i = 0
loop_start:
    if i >= n goto loop_end
    t1 = a[i]
    sum = sum + t1
    i = i + 1
    goto loop_start
loop_end:
```

This form is less elegant as source code, but much more explicit as execution structure. The control flow is visible. The loop back-edge is visible. The exit condition is visible. The memory load from the array is visible. These are exactly the kinds of details needed by optimization passes.

Intermediate representations are useful because many optimizations are easier when the program is expressed as simple operations. A compiler can inspect the IR and perform transformations such as:

- removing computations whose results are never used;
- moving repeated computations out of loops;
- replacing expensive operations with cheaper equivalent operations;
- simplifying constant expressions;
- eliminating unreachable code;
- analyzing which variables are alive at each point;
- deciding which values should be kept in CPU registers;
- preparing memory accesses for efficient instruction selection.

These optimizations would be harder to perform directly on raw source text. They would also be awkward to perform on a source-shaped AST, because the AST still contains high-level language constructs that hide the exact operational structure.

The IR also helps separate compiler responsibilities. The front end is language-specific. It knows C syntax, C declarations, C scope rules, and C type rules. The back end is target-specific. It knows x86-64, ARM, RISC-V, or another machine architecture. The middle end sits between them and works on an intermediate form that is less tied to the source language and less tied to the final processor.

This separation is one reason modern compilers can support multiple languages and multiple targets. A C front end, C++ front end, or Rust front end may lower source programs into a common family of intermediate representations. Then target back ends can generate code for several processor architectures. The IR becomes a shared internal language used by the compiler infrastructure.

The AST and the IR therefore answer different questions. The AST asks:

What is the structural meaning of the source program?

The IR asks:

What explicit operations must be performed, and how can they be transformed before final code generation?

This distinction also appears in Python, although the final destination is different. CPython parses Python source code into an AST, then compiles that AST into bytecode instructions for the CPython virtual machine. Python bytecode is not native machine code, but it is already lower-level than the AST. It is a sequence of virtual instructions that the interpreter loop can execute.

For example, a Python expression such as:

```
x = a + b
```

is first understood structurally as an assignment whose right side is an addition. Later, CPython compiles it into bytecode operations that load names, perform binary addition, and store the result. The AST is the source-level structure. The bytecode is the executable instruction stream for the virtual machine.

Thus, moving from AST to IR is one of the main architectural steps in compilation. The source program stops being represented as nested language constructs and starts being represented as explicit computational operations. This makes optimization possible, prepares the program for code generation or bytecode generation, and creates a cleaner boundary between source-language analysis and execution-oriented transformation.

The Middle-End Phase: Target-Independent Intermediate Representation (IR) Optimization Passes

After the front end has parsed the source program, built an abstract syntax tree, and attached semantic information to it, the compiler can begin transforming the program into forms that are less tied to the original source language. This is the role of the **middle end**. The middle end works mainly on intermediate representations, or IRs, and applies transformations that improve or simplify the program before final machine-code generation.

The middle end is called *middle* because it sits between two specialized compiler regions. The front end is source-language-specific. It knows C syntax, C declarations, C type rules, C scopes, and C semantic constraints. The back end is target-machine-specific. It knows processor registers, instruction sets, calling conventions, object file formats, and platform-specific details. The middle end is designed to be less dependent on both sides. It operates on a more general internal form of the program.

This is why middle-end optimizations are often called **target-independent**. They are not completely detached from all hardware concerns, but they are not written specifically for one instruction set such as x86-64 or ARM. They work on general facts about computation: values, assignments, branches, loops, memory loads, stores, function calls, and data dependencies.

For example, consider the following C fragment:

```
int x = 10 * 20;
int y = x + 0;
```

The front end can determine that this code is syntactically and semantically valid. The middle end can then simplify it. The expression `10 * 20` can be computed at compile time. The addition of zero does not change the value. A simplified representation may become equivalent to:

```
int x = 200;
int y = x;
```

This kind of transformation is not about choosing a specific CPU instruction. It is about recognizing that the program contains unnecessary computation.

One common optimization is **constant folding**. If an expression contains only values known at compile time, the compiler can compute the result during compilation instead of leaving the computation for runtime. For example:

```
int seconds = 60 * 60 * 24;
```

can be transformed into:

```
int seconds = 86400;
```

Another common optimization is **dead code elimination**. If the compiler can prove that some operation has no effect on the final behavior of the program, it can remove it. For example:

```
int x = 5;
x = 6;
printf("%d\n", x);
```

The first assignment to `x` is useless if `x` is overwritten before being read. The middle end can remove the dead assignment and preserve the visible result.

A related optimization is **unreachable code elimination**. If control flow can never reach a certain block, the compiler can remove that block. For example:

```
return 0;
printf("This cannot run\n");
```

The statement after `return` cannot execute in ordinary control flow. It is therefore unreachable.

The middle end also analyzes loops. Loops are important because small inefficiencies inside a loop may be repeated many times. A transformation called **loop-invariant code motion** moves computations out of a loop when their result does not change during the loop. For example:

```
for (int i = 0; i < n; i++) {
    a[i] = b[i] * (x + y);
}
```

If `x` and `y` do not change inside the loop, the expression `x + y` can be computed once before the loop:

```
int t = x + y;
for (int i = 0; i < n; i++) {
    a[i] = b[i] * t;
}
```

The program has the same meaning, but repeated work has been reduced.

Another important class of optimization is based on removing repeated calculations. If the same expression appears multiple times and none of its inputs change, the compiler may compute it once and reuse the result. This is called **common subexpression elimination**. For example:

```
z = (a * b) + (a * b);
```

may be transformed internally into:

```
t = a * b;
z = t + t;
```

The middle end can also perform **copy propagation**. If one variable is only a copy of another, later uses may be rewritten to remove the unnecessary copy. For example:

```
x = y;
z = x + 1;
```

may become:

```
z = y + 1;
```

if the transformation is safe.

These transformations require analysis. The compiler must know which values are used, where they are defined, whether memory may have changed, whether a function call may produce side effects, and whether changing the order of operations would change the program's observable behavior.

Optimization is not random rewriting. It is constrained rewriting based on what the compiler can prove.

This is why intermediate representations often make control flow explicit. A program may be divided into **basic blocks**. A basic block is a straight-line sequence of operations with one entry point and one exit point. Branches, loops, and jumps connect these blocks into a control-flow graph. This representation allows the compiler to reason about which operations can execute before or after other operations.

For example, an `if` statement is not only a source-level block. In IR form, it becomes conditional control flow: one branch for the true case, another branch for the false case, and possibly a later join point. A loop becomes a group of blocks with a backward edge. This graph structure makes optimization more precise than working directly on source text.

Many modern compilers also use a representation called **static single assignment**, or SSA form. In SSA form, each variable-like temporary is assigned exactly once. If different control-flow paths produce different values, special merge operations are inserted. The purpose is not to make the program more readable for humans, but to make data dependencies easier for the compiler to analyze. SSA form helps optimizations determine where values come from and where they are used.

The middle end must also preserve the language's observable behavior. A compiler is allowed to change the internal form of the program only if the transformed program behaves the same under the language rules. It may remove unused computations, but it must not remove computations that produce visible effects. It may simplify arithmetic, but only where the simplification is valid according to the language's rules about overflow, floating-point behavior, memory access, and side effects.

This matters especially in C because some operations have undefined behavior, implementation-defined behavior, or machine-dependent consequences. The optimizer may use the C language rules aggressively. If a program relies on behavior that the C standard does not guarantee, optimization can produce surprising results. This is not the optimizer being arbitrary; it is the optimizer assuming that the program obeys the formal rules of the language.

The output of the middle end is still not usually final executable code. It is an optimized or simplified intermediate representation. The program is now easier for the back end to translate into target-specific instructions. The back end will decide how to use registers, which machine instructions to emit, how to follow calling conventions, and how to produce object code for a specific platform.

This distinction is useful when comparing C and Python. In a traditional C compiler, the middle end is a major place where performance is recovered before the program becomes native machine code. CPython has a much smaller optimization story in its normal execution path. CPython compiles Python source into bytecode, and it may perform some simple optimizations, but it does not usually run a large native-code middle-end optimizer like an ahead-of-time C compiler.

This difference follows from the execution model. C tries to resolve and optimize as much as possible before execution. CPython keeps much more behavior dynamic: names are looked up at runtime, object types are known at runtime, operations dispatch through object protocols, and many decisions depend on the exact objects present during execution. That flexibility limits what can be safely optimized before running the program.

The middle-end phase therefore represents the compiler's main target-independent reasoning stage. The program has already been understood as valid source. It has not yet become target-specific machine code. Between those points, the compiler rewrites the program into simpler, faster, or more explicit forms while preserving its meaning. This is one of the main architectural reasons native compiled languages such as C can achieve high execution speed: much of the work of analysis and transformation happens before the process ever starts running.

The Back-End Code Generation Phase

After the middle end has transformed and optimized the program's intermediate representation, the compiler must finally move toward the target machine. This stage is called the **back end**. The back end is the part of the compiler that understands the processor architecture, the operating system ABI, the calling convention, register layout, instruction set, object file format, and platform-specific execution requirements.

The front end answers the question:

What does the source program mean?

The middle end answers the question:

How can this meaning be simplified, optimized, and represented as explicit computation?

The back end answers a different question:

How can this computation be expressed as instructions for a specific machine and operating system target?

This distinction is essential. The middle end may know that the program contains an addition operation, a conditional branch, a loop, a memory load, or a function call. But the physical processor does not execute abstract operations. It executes instructions from a specific instruction set architecture, such as x86-64, ARM64, RISC-V, or another ISA. The back end is responsible for translating intermediate operations into that target instruction language.

For example, an intermediate representation may contain an operation such as:

```
t1 = a + b
```

At the IR level, this only means that two values must be added and the result stored in a temporary. The back end must decide where `a` and `b` are located, whether they are in registers or memory, which machine instruction can perform the addition, where the result should be placed, and whether additional load or store instructions are necessary.

A simplified assembly-like result might look like:

```
mov eax, DWORD PTR [a]
add eax, DWORD PTR [b]
mov DWORD PTR [t1], eax
```

This example is not the only possible output. A better register allocation may avoid some memory traffic. A different processor may use different instruction names and different register rules. The exact emitted instructions depend on the target architecture and on the compiler's back-end decisions.

One major responsibility of the back end is **instruction selection**. Instruction selection means choosing real machine instructions that implement the operations present in the intermediate representation. A single IR operation may map to one machine instruction, several instructions, or a specialized instruction available only on some processors.

For example, one architecture may have a direct instruction for a certain arithmetic operation, while another may require several simpler instructions. Some processors have specialized vector instructions for operating on multiple values at once. Others may have special instructions for address calculation, bit operations, floating-point arithmetic, or conditional moves. The back end must know which instructions are available and when using them is legal or efficient.

Another important responsibility is **register allocation**. Processors have a limited number of registers. Registers are small, very fast storage locations inside the CPU. Intermediate representations may contain many temporary values, often more than the number of hardware registers. The back end must decide which values should live in registers and which must be placed temporarily in memory.

If there are not enough registers, some values must be **spilled**. Spilling means storing a value from a register into memory so that the register can be reused for another value. Later, the spilled value may need to be loaded back. Spilling is sometimes necessary, but it can reduce performance because memory access is slower than register access.

The back end must also handle the **calling convention**. A calling convention is a set of rules that defines how functions call each other at the machine level. It specifies details such as:

- which registers are used for function arguments;

- which registers are used for return values;
- which registers the caller must save;
- which registers the callee must preserve;
- how stack frames are created and destroyed;
- how additional arguments are passed on the stack;
- how structure values, floating-point values, and variadic arguments are handled.

This matters because separately compiled functions must agree on how calls work. A function compiled from one source file must be callable from another source file. A C function must also be able to call functions from libraries. Without a stable calling convention, compiled code from different files or libraries could not cooperate safely.

For example, when compiling a function call such as:

```
result = add(x, y);
```

the back end must generate code that places `x` and `y` in the correct argument locations, transfers control to the function `add`, receives the return value from the expected location, and continues execution afterward. This is no longer an abstract language operation. It is a concrete machine-level protocol.

The back end also constructs function prologues and epilogues. A **function prologue** is the instruction sequence at the beginning of a function that prepares the stack frame, saves required registers, and reserves space for local data if needed. A **function epilogue** reverses this work before returning to the caller.

A simplified function shape may look like:

```
function_start:
    push rbp
    mov rbp, rsp
    sub rsp, 32

    ; function body

    mov rsp, rbp
    pop rbp
    ret
```

Real compilers may omit or change this structure depending on optimization settings, debugging requirements, calling convention details, and target architecture. However, the general purpose remains the same: the back end must produce machine-level structure that allows function calls, local storage, returns, and stack unwinding to work correctly.

The back end must also emit correct memory accesses. In C, objects may live in global storage, stack frames, registers, or dynamically allocated heap memory. The compiler must generate instructions that access these objects through the correct addresses and offsets. A local variable may be located at an offset from the stack pointer or frame pointer. A global variable may require relocation information. A structure field may require an offset from the beginning of the structure object. An array element may require base-address calculation plus index scaling.

For example, an expression such as:

```
a[i]
```

is not a primitive CPU concept. The back end must compute the address of element `i`. If each element has size four bytes, the address may be computed as:

```
base_address_of_a + i * 4
```

Then the generated machine instruction can load from or store to that calculated address.

The back end is also responsible for target-specific optimization. The middle end performs optimizations that are mostly independent of the final processor. The back end can perform optimizations that depend on the instruction set and hardware model. These may include instruction scheduling, use of specialized addressing modes, branch optimization, register allocation improvements, vector instruction selection, and removal of redundant machine-level moves.

At the end of the back-end phase, the compiler usually emits assembly language or directly emits object code. Assembly language is a textual representation of machine instructions. It is still readable by humans, but it is already very close to the processor's real instruction set. A later assembler can translate this assembly into binary object code.

For example, a very small C function such as:

```
int add(int a, int b) {
    return a + b;
}
```

may become assembly resembling:

```
add:
    mov eax, edi
    add eax, esi
    ret
```

This assembly is target-specific. The register names, instruction names, and calling convention assumptions belong to a particular architecture and platform ABI. Another architecture would express the same function differently.

The back end therefore marks the transition from compiler-internal computation to platform-specific executable material. Before this phase, the program was represented in forms designed for analysis and transformation. After this phase, the program has been translated into instructions and data layouts that belong to a particular execution environment.

This is one of the major differences between traditional C compilation and CPython execution. In the C pipeline, the back end eventually produces native machine instructions before the final executable is run. In CPython, the user's Python source is normally compiled to bytecode for a virtual machine, not to native processor instructions. The native machine instructions being executed belong mostly to the CPython interpreter executable itself, not to the user's Python file.

Thus, the back-end code generation phase is the stage where the C program finally becomes machine-oriented. It maps optimized intermediate operations onto the real processor model, chooses instructions, assigns registers, follows calling conventions, prepares stack behavior, and emits assembly or object code. It is the final compiler-controlled step before assembly, object-file generation, and linking produce a loadable native executable.

Target Machine Architecture Mapping and ISA Instruction / Assembly Emission

Once the compiler has an optimized intermediate representation, it must express that representation in the language of a real processor. This step is called **target machine architecture mapping**. The compiler no longer works only with abstract operations such as addition, comparison, branch, function call, load, or store. It must choose concrete instructions that exist in the instruction set architecture of the target machine.

An **instruction set architecture**, or **ISA**, is the formal set of operations that a processor family understands. It defines the available machine instructions, registers, addressing modes, data sizes, control-transfer mechanisms, and rules for how instructions interact with memory and processor state. Examples include x86-64, ARM64, RISC-V, and others.

This means that the same C source code may produce different assembly depending on the target architecture. A simple function such as:


```
int add(int a, int b) {  
    return a + b;  
}
```

may compile into one sequence of instructions on x86-64 and a different sequence on ARM64. The high-level meaning is the same, but the emitted instruction stream is architecture-specific.

For example, on an x86-64-like target, the emitted assembly may resemble:

```
add:  
    mov eax, edi  
    add eax, esi  
    ret
```

Here, the function arguments are assumed to arrive in specific registers according to the platform calling convention. The value from `edi` is copied into `eax`, the value from `esi` is added, and the result is returned through `eax`. The exact register names and rules belong to the target platform.

On another architecture, the same function may be expressed differently:

```
add:  
    add w0, w0, w1  
    ret
```

The operation is still addition, but the register names, instruction format, and calling convention are different. The compiler back end must know these rules in order to emit correct code.

This is why machine code is not portable in the same way source code is. The C source code describes the computation abstractly. The emitted assembly describes the computation for one processor architecture. A binary compiled for x86-64 cannot normally run directly on ARM64, because the instruction bytes mean different things or may not be valid instructions at all.

During architecture mapping, the compiler must solve several problems at once:

- choose machine instructions for each intermediate operation;
- decide which values should be held in hardware registers;
- decide when values must be loaded from memory or stored back to memory;
- respect the platform calling convention;
- emit correct branch and jump instructions;
- represent memory addresses using the addressing modes supported by the processor;
- prepare relocatable references for symbols whose final address is not yet known.

The mapping is not always one-to-one. One intermediate operation may become one machine instruction, several instructions, or no instruction at all if it was optimized away. Conversely, one machine instruction may perform work that corresponds to several abstract operations. Some architectures support complex addressing modes that combine base addresses, indexes, scaling factors, and offsets inside a single instruction.

For example, an array access such as:

```
x = a[i];
```

requires the compiler to compute the address of element `i`. If each element has four bytes, the conceptual address is:

```
address_of_a + i * 4
```

On an architecture with suitable addressing modes, this calculation may be folded directly into a memory-load instruction. On another architecture, the compiler may need to emit separate instructions for multiplication, address addition, and memory loading.

Assembly language is the human-readable textual form of these target instructions. It is still much closer to the machine than C source code. An assembly file contains instruction mnemonics, register names, labels, directives, and symbolic references. It is not yet necessarily the final binary object file, but it is already target-specific.

For example, the C statement:

```
x = y + 1;
```

may become assembly resembling:

```
mov eax, DWORD PTR [y]
add eax, 1
mov DWORD PTR [x], eax
```

This says, in target-level terms: load the value of `y` into a register, add one, and store the result into `x`. A real compiler may emit different instructions depending on optimization level, register allocation, and whether `x` and `y` are local variables, global variables, or temporary values.

The compiler also emits labels and branch instructions for control flow. A high-level statement such as:

```
if (x > 0) {
    y = 1;
} else {
    y = -1;
}
```

must become comparisons and jumps. A simplified assembly shape may look like:

```
cmp DWORD PTR [x], 0
jle else_block
mov DWORD PTR [y], 1
jmp end_if

else_block:
mov DWORD PTR [y], -1

end_if:
```

The high-level conditional structure has disappeared. What remains is a comparison, a conditional jump, an unconditional jump, and labeled instruction regions. This is the machine-level form of branching control flow.

Loops undergo a similar transformation. A `while` or `for` loop becomes labels, comparisons, conditional jumps, and backward jumps. The source-level idea of a loop is translated into explicit control transfer inside the instruction stream.

Function calls are also architecture-specific. The compiler must place arguments where the calling convention requires them, issue a call instruction, and retrieve the result from the expected return location. It must also preserve registers according to the caller/callee-save rules. These details are not visible in the C source code, but they are required for separately compiled functions to work together.

In addition to instructions, the compiler emits assembly directives. Directives are not CPU instructions. They tell the assembler how to organize the output. They may define sections, align data, declare symbols, reserve storage, or mark functions as global. For example, an assembly file may contain directives for the text section, data section, read-only constants, debugging information, and symbol visibility.

This emitted assembly is usually passed to an assembler. The assembler converts textual assembly into binary machine code inside a relocatable object file. Symbolic names may still remain unresolved at this point. For example, if one source file calls a function defined in another source file, the assembler can emit a placeholder relocation record. The linker will later resolve the final address.

This architecture-specific phase is one of the clearest differences between traditional C compilation and CPython execution. In C, the user's source program is lowered all the way to target ISA instructions before the final executable runs. In CPython, the user's Python source is normally compiled only to Python bytecode. That bytecode is not an instruction stream for the physical processor. It is an instruction stream for the CPython virtual machine. The physical processor executes the native instructions of the CPython interpreter, and that interpreter executes the Python bytecode.

Thus, target machine architecture mapping is the point where the C program becomes tied to a specific hardware and operating-system environment. The compiler transforms abstract computation into assembly instructions, register usage, memory addressing, branch labels, calling-convention behavior, and relocatable symbols. After this phase, the program is no longer merely a language-level description. It has become a target-specific instruction plan ready for assembly and linking.

The Assembler Layer and Relocatable Machine Object File Generation (.o / .obj Format)

After the compiler back end emits assembly or directly prepares machine-level instructions, the next layer in the native compilation pipeline is the **assembler**. The assembler transforms assembly language into binary machine code and packages that binary code into a **relocatable object file**. On Unix-like systems, these files commonly use the extension `.o`. On Windows, they commonly use the extension `.obj`.

Assembly language is already target-specific. It contains instruction names, register names, labels, directives, and symbolic references that belong to a particular processor architecture and platform. However, assembly is still textual. The processor cannot execute assembly text directly. The assembler must convert each assembly instruction into the corresponding binary instruction encoding understood by the processor.

For example, an assembly fragment such as:

```
mov eax, edi
add eax, esi
ret
```

is readable by humans, but the CPU ultimately executes binary instruction bytes. The assembler is the tool that performs this translation from symbolic instruction mnemonics into encoded machine instructions.

The assembler also handles assembly directives. Directives are commands for the assembler itself, not instructions for the CPU. They may describe where code and data should be placed, how symbols should be marked, how memory should be aligned, or which sections should be created. For example, an assembly file may contain directives that place instructions into a text section, constants into a read-only data section, and global variables into a data or BSS section.

The output of the assembler is not usually the final executable. It is a **relocatable object file**. This object file contains machine code, but that machine code is not yet fully placed into its final executable memory layout. Some addresses are still unknown. Some symbol references may point to functions or variables defined in other object files or external libraries. These unresolved relationships are left for the linker.

This is why the file is called *relocatable*. The code and data inside it can later be moved into their final locations by the linker. The assembler produces machine code together with metadata that tells the linker what still needs to be resolved.

A relocatable object file usually contains several important elements:

- machine-code sections, such as executable instruction bytes;
- data sections, such as initialized global or static data;

- BSS-like descriptions for uninitialized storage;
- symbol tables describing names defined or referenced by the object file;
- relocation records describing places where addresses must be fixed later;
- optional debugging information;
- platform-specific metadata required by the object file format.

Consider a project with two C source files:

```
/* main.c */
int add(int a, int b);

int main(void) {
    return add(2, 3);
}

/* math_utils.c */
int add(int a, int b) {
    return a + b;
}
```

If these files are compiled separately, the result may be:

```
main.o
math_utils.o
```

When compiling `main.c`, the compiler knows that a function named `add` exists because it has seen the declaration:

```
int add(int a, int b);
```

However, the body of `add` is not inside `main.c`. Therefore, the object file `main.o` may contain a call instruction that refers symbolically to `add`, but the final address of `add` is not yet known. The assembler records this unresolved reference so the linker can fix it later.

The object file `math_utils.o`, on the other hand, defines the symbol `add`. During linking, the linker connects the unresolved reference in `main.o` to the definition in `math_utils.o`. This is the next stage of the pipeline.

Relocation records are central to this process. A relocation record says, in effect:

At this location in the machine code or data, an address or offset must be adjusted later when the final location of a symbol is known.

Without relocation records, separately compiled files could not reliably refer to each other. The assembler cannot know the final memory position of every function and global object because the final executable has not yet been composed. The linker performs that composition.

The symbol table is another essential part of the object file. It records names that the object file defines and names that it expects someone else to provide. Symbols may represent functions, global variables, static objects, section labels, or other linkable entities. Some symbols are local to the object file. Others are externally visible and can be used by the linker to connect multiple object files.

For example, an object file may contain:

- a definition of `main`;
- an unresolved reference to `printf`;

- local labels used for internal jumps;
- metadata describing where each symbol appears.

The assembler therefore does more than translate instruction names into bytes. It also creates a structured binary container that preserves enough information for later linking. The object file is a bridge between compiled machine code and final executable construction.

This also explains why object files are not normally executed directly. A relocatable object file contains machine instructions, but it is incomplete as a process image. It may not have a final entry point layout. It may contain unresolved external references. Its sections have not yet been combined with sections from other object files and libraries. Its relocations have not yet been applied. The operating system loader expects an executable or shared library format, not an arbitrary unresolved object file.

On Linux and many Unix-like systems, relocatable object files commonly use the ELF format. ELF is flexible enough to represent object files, executables, shared libraries, and core dumps. On Windows, object files use the COFF part of the PE/COFF family. The exact binary structures differ, but the architectural role is the same: the object file stores compiled code and linkable metadata before final executable construction.

The object-file stage is especially important for large C programs. Separate compilation allows each source file to be compiled independently. This means a large project does not always need to be fully recompiled after every small change. If only one source file changes, the build system may rebuild only its corresponding object file and then relink the final executable.

This model also clarifies the contrast with Python. A normal Python source file is not assembled into a relocatable native object file. CPython compiles Python source into bytecode objects and may store them in `.pyc` files, but these are not machine-code object files in the C sense. A `.pyc` file is an artifact for the Python virtual machine, not a relocatable object file for the native linker.

Therefore, the assembler layer is a specifically native-code construction stage. It takes target-specific assembly and produces relocatable machine object files. These files contain real machine instructions, but they are not yet complete executables. They still need linking, symbol resolution, relocation, and executable-format composition before the operating system can load them as running processes.

The Static Linking Phase: Symbol Resolution, External Relocations, and Executable Binary Composition

After each source file has been compiled and assembled, the build process usually contains several relocatable object files. Each object file contains machine code and data, but it is not normally a complete executable program. Some names may still be unresolved, some addresses may still be placeholders, and code from different translation units has not yet been combined into one loadable binary. The phase that performs this composition is called **linking**.

In the traditional C pipeline, linking is the stage that takes object files and libraries and constructs the final executable or shared library. If compilation transforms source code into object code, linking connects those object files into a coherent binary program.

For example, a simple project may contain three source files:

```
main.c
math_utils.c
io_utils.c
```

After separate compilation and assembly, the project may contain:

```
main.o
math_utils.o
io_utils.o
```

Each object file contains the compiled result of one translation unit. However, `main.o` may call a function defined in `math_utils.o`. It may also call functions from the C standard library, such as `printf`. At the object-file stage, those calls may still refer to symbolic names rather than final addresses. The linker is responsible for resolving those names.

The first major responsibility of the linker is **symbol resolution**. A symbol is a named entity known to the object-file and linking system. Symbols may represent functions, global variables, static storage, labels, or externally visible objects. One object file may define a symbol, while another object file may refer to it.

Consider:

```
/* main.c */
int add(int a, int b);

int main(void) {
    return add(2, 3);
}
```

and:

```
/* math_utils.c */
int add(int a, int b) {
    return a + b;
}
```

When `main.c` is compiled, the compiler knows that a function named `add` exists because it has seen the declaration. But it does not see the function body. Therefore, `main.o` contains an unresolved reference to the symbol `add`. The file `math_utils.o` contains the definition of that symbol. During linking, the linker matches the reference in `main.o` with the definition in `math_utils.o`.

If the linker cannot find a definition for a required external symbol, linking fails. This is the source of common errors such as “undefined reference” on Unix-like toolchains or “unresolved external symbol” on Windows toolchains. Such an error usually means that the compiler accepted the source file, but the linker could not find the machine-code implementation of something that was declared or called.

For example:

```
int missing_function(void);

int main(void) {
    return missing_function();
}
```

may compile into an object file, because the declaration tells the compiler that `missing_function` is callable. But if no object file or library provides the actual definition, the linker cannot produce a complete executable.

The second major responsibility of the linker is **relocation**. During assembly, object files contain sections of code and data whose final positions are not yet known. A function in one object file may be placed before or after functions from other object files in the final executable. Global variables may be arranged into final data sections. Libraries may be inserted or referenced. Therefore, many addresses cannot be finalized at assembly time.

A relocation record tells the linker that some location in the object file must be adjusted when the final address of a symbol becomes known. Conceptually, it says:

At this position in the code or data, insert or adjust the address or offset of this symbol.

For example, a call instruction may need the address of a function. A global variable access may need the address of a data object. A pointer constant may need to be initialized with the address of

another object. The assembler cannot always fill in these values, so it leaves relocation information for the linker.

The linker then assigns final locations to sections and symbols. Once it knows where everything will be placed, it patches the necessary addresses and offsets. This is why the object files are called relocatable: their contents can be moved and fixed up during final composition.

The third major responsibility of the linker is **binary composition**. The linker combines sections from multiple object files into the larger sections and segments of the final executable. Code sections are collected into executable regions. Read-only constants are collected into read-only regions. Initialized global data is collected into writable data regions. Uninitialized data descriptions are collected into BSS-like regions. The final binary must follow the executable format expected by the operating system, such as ELF on Linux or PE/COFF on Windows.

The linker also chooses or records the program entry point. In a C program, the programmer usually thinks of `main()` as the starting point. At the native executable level, the actual entry point is usually a runtime startup function supplied by the C runtime. That startup code prepares the process-level runtime environment and then calls `main()`. The linker must arrange the executable so that the operating system loader can transfer control to the correct native entry point.

Linking can be **static** or **dynamic**. In static linking, library code needed by the program may be copied into the final executable. The resulting executable contains more of the machine code it needs at runtime. In dynamic linking, the executable records dependencies on shared libraries that will be loaded or mapped when the program starts. This section focuses on the static linking idea: the linker composes object files and required library material into one final executable image as much as possible before runtime.

Static linking has advantages and disadvantages. It can produce an executable that depends less on external libraries being present on the target system. However, it can also produce larger binaries, duplicate library code across multiple programs, and make security or bug-fix updates harder because the library code has been copied into the executable.

Dynamic linking takes a different approach. Instead of copying all library code into the executable, the executable contains information describing which shared libraries it needs. On Linux, these are often `.so` files. On Windows, they are often `.dll` files. The operating system loader and dynamic linker then arrange for those libraries to be available when the process starts or when they are first needed.

Even when dynamic libraries are used, the static linking phase still creates the final executable file structure. It records imported symbols, dynamic library dependencies, relocation information, and the metadata needed by the loader. Therefore, linking remains necessary even when the executable does not contain every piece of library code inside itself.

The linking phase is also where separate compilation becomes one program. The compiler can process each `.c` file independently, but the program as a whole may contain calls and global references across many files. The linker connects those independently compiled pieces. This is why C projects can be divided into many source files without requiring the compiler to reprocess the entire project every time.

The final result of linking is an executable binary or a library binary. For a normal executable, the file now has the structure needed by the operating system loader. It contains executable code, data regions, symbol and relocation information as needed, dependency metadata, and an entry point. It is no longer just a collection of independent object files. It is a binary composition that can be loaded into a process.

This stage also clarifies another difference between C and Python. A normal Python program does not pass through a native linker that combines its source files into one executable machine-code binary. Python modules are loaded through the import system at runtime. CPython may compile individual modules into bytecode files such as `.pyc`, but these are not linked together like C object files. They remain runtime artifacts consumed by the interpreter.

However, Python can still interact with native linking indirectly. CPython itself is a native executable produced by a C compiler and linker. Many Python extension modules, such as native modules used by numerical or system libraries, are compiled and linked as shared libraries. When Python imports such an extension, it loads native code that was produced by a traditional compilation and

linking pipeline.

Thus, static linking is the phase where separately compiled native fragments become one executable binary structure. It resolves symbols, applies relocations, combines code and data sections, connects external dependencies, and produces a file that the operating system loader can transform into a running process.

Dynamic Linking: Shared Libraries, Runtime Loading, and External Native Dependencies

Static linking is not the only way to build a native executable. Many programs do not contain all required library code inside the executable file itself. Instead, they depend on external binary libraries that are loaded when the program starts or while it is already running. This mechanism is called **dynamic linking**.

A dynamically linked executable contains its own code and data, but it also contains metadata describing external libraries and external symbols that must be resolved. On Linux and other Unix-like systems, dynamically loaded shared libraries usually have the extension `.so`, meaning *shared object*. On Windows, they usually have the extension `.dll`, meaning *dynamic-link library*. On macOS, similar libraries often use `.dylib`.

The main idea is simple: instead of copying library code into every executable that uses it, the operating system can map one shared library into the address spaces of multiple processes. Each process sees the library inside its own virtual address space, but the physical memory pages containing read-only machine code may be shared by several processes. This reduces disk usage, memory usage, and duplication.

For example, a C program that calls `printf()` does not normally contain the full implementation of `printf()` inside its own source file. The function is provided by the C standard library implementation. During compilation, the compiler only needs a declaration that tells it how `printf()` may be called. During linking and loading, the actual binary implementation must be connected.

In a dynamically linked program, the final executable records that it needs a shared library containing the implementation of `printf()` and other runtime functions. The exact name depends on the platform and C runtime. On Linux, this may involve `libc.so`. On Windows, it may involve runtime DLLs such as the Microsoft C runtime libraries. The executable does not necessarily contain the full code of these libraries. It contains references to them.

Dynamic linking therefore divides the linking job into two moments:

- **link time**, when the executable is created and its external library dependencies are recorded;
- **load time** or **runtime**, when the operating system loader and dynamic linker locate shared libraries, map them into memory, and resolve required symbols.

At link time, the linker may verify that required symbols are expected to exist in some library. It also writes dependency metadata into the executable. In ELF executables, this information is stored in dynamic sections and related tables. In PE executables on Windows, similar information is stored in import tables. These structures tell the loader which libraries and imported functions are needed.

At load time, when the user starts the executable, the operating system does not only map the executable file itself. It also invokes dynamic loading machinery. The dynamic loader locates the required shared libraries, maps their segments into the process virtual address space, applies required relocations, and connects imported symbols to actual addresses.

This process is still controlled. The executable cannot simply jump to an external function whose address it does not know. The loader and dynamic linker must make sure that the needed library is present, compatible, and mapped into memory. Then references from the executable to the library can be made usable.

A simplified view looks like this:

```
program executable
  needs: libc.so
  imports: printf, malloc, free
```



```
loader / dynamic linker
    finds libc.so
    maps libc.so into process memory
    resolves printf, malloc, free
    patches or prepares call targets

running process
    executable code + shared library code
```

Dynamic linking also supports shared libraries written by the programmer. A project can be split into an executable and one or more shared libraries. The executable may provide the main program logic, while reusable functionality is placed in libraries. Other programs may load the same libraries.

This is common in large software systems. Graphical toolkits, database clients, compression libraries, cryptography libraries, numerical libraries, image processing libraries, and system APIs are often provided as shared libraries. Programs call into these libraries instead of embedding all their code directly.

Dynamic linking has several advantages:

- multiple programs can share the same library code;
- executable files can be smaller;
- libraries can sometimes be updated without rebuilding every dependent program;
- plugins and extension modules can be loaded at runtime;
- common system functionality can be centralized in shared components.

However, dynamic linking also introduces new failure modes. A program may fail to start if a required shared library is missing. It may fail if the library version is incompatible. It may load a different library than expected because of search-path rules. It may behave differently on two machines if their installed libraries differ. These problems are not source-code syntax errors. They are runtime dependency and loading errors.

For example, on Linux, an executable may fail with an error similar to:

```
error while loading shared libraries: libexample.so:
cannot open shared object file: No such file or directory
```

This means the executable exists and may be structurally valid, but one of its dynamic dependencies cannot be found by the loader.

On Windows, similar problems appear when a required DLL is missing or incompatible. The program may fail to start, or it may report that a specific procedure entry point could not be found. This usually means that the executable found a DLL with the expected name, but not with the expected exported function or compatible version.

Dynamic linking also depends on symbol resolution. A shared library can export symbols, and an executable or another library can import them. A symbol may represent a function, a global variable, or another linkable entity. The dynamic linker connects imports to exports. This is similar in purpose to static symbol resolution, but some of the work is delayed until load time or runtime.

There are several possible binding strategies. Some symbols may be resolved when the program starts. Others may be resolved lazily, only when a function is first called. Lazy binding can reduce startup work, but it means some resolution errors may appear later, when the symbol is first needed. The exact behavior depends on platform, linker settings, loader behavior, and security configuration.

Dynamic libraries can also be loaded explicitly by a program at runtime. In C on POSIX-like systems, this can be done through functions such as `dlopen()` and `dlsym()`. On Windows, similar behavior is available through functions such as `LoadLibrary()` and `GetProcAddress()`. This allows

plugin systems: the main program can load optional components whose names or locations are not known at compile time.

This mechanism is important for understanding Python. CPython itself is a native executable produced by the C compilation and linking pipeline. It uses operating-system services and may depend on shared libraries. In addition, many Python packages contain native extension modules. These modules are not ordinary `.py` files. They are compiled shared libraries loaded into the Python interpreter process.

On Linux, a Python extension module is often a shared object file with a name ending in something like `.so`. On Windows, Python extension modules often use the `.pyd` extension, but structurally they are DLL-like native libraries. When Python imports such a module, CPython does not interpret Python source code from that file. It asks the operating system to load native binary code into the interpreter process.

This is why installing Python packages can sometimes fail for reasons that look more like C/C++ build problems than Python problems. A package may need a compiler. It may need header files. It may need external shared libraries. It may need a binary wheel compiled for the correct Python version, operating system, and processor architecture. If any of these native dependency boundaries are wrong, the Python import may fail even though the Python syntax is correct.

For example, numerical packages often contain C, C++, or Fortran extensions. Image processing libraries may depend on native codecs. Database libraries may depend on client libraries. Cryptography packages may depend on low-level native cryptographic implementations. From Python code, the import may look simple:

```
import numpy
```

but behind that statement there may be compiled extension modules and dynamic native dependencies loaded into the CPython process.

Dynamic linking therefore extends the native execution model beyond the main executable file. A running process is not necessarily built from one binary alone. It may be composed from the executable plus many shared libraries mapped into the same virtual address space. Some are required at startup. Others may be loaded later. Together, they form the native code environment in which the process runs.

For C programmers moving to Python, this explains an important practical reality. Python source code is executed by the Python runtime, but the Python runtime itself is native software, and many Python packages cross back into the native linking world. The high-level Python program may still depend on ELF shared objects, PE/DLL libraries, ABI compatibility, compiler toolchains, and operating-system loader behavior.

Thus, dynamic linking is the mechanism that allows native programs to depend on external binary components without embedding all of them into the executable. It records dependencies, loads shared libraries, resolves external symbols, maps code and data into process memory, and enables reusable native components. It is also the bridge through which Python programs often reach high-performance native code while still presenting a high-level Python interface.

2.1.3 Program Loading and Execution Mechanics

The Lifecycle of Transformation: From Storage Binary Block to Active Virtual Memory Process

After compilation, assembly, and linking, the program exists as an executable binary file stored on a persistent medium. It may be an ELF executable on Linux, a PE executable on Windows, or another platform-specific executable format. At this stage, the program is still not running. It is a structured block of stored data. It contains machine instructions, data sections, metadata, headers, symbol information, relocation information, dynamic dependency descriptions, and an entry point description, but none of its instructions are currently being executed by the processor.

The transition from stored executable file to active process begins when the user, shell, script, desktop environment, or another process requests execution. On a Unix-like system, this request

eventually reaches a system call such as `execve()`. On Windows, a related role is performed through process-creation APIs such as `CreateProcess()`. The exact interface differs, but the architectural event is the same: user-space code asks the operating system kernel to create or replace a process image using an executable file.

The operating system does not simply copy the whole executable file into memory and start reading it from the first byte. An executable file is not a flat instruction tape. It is a formatted binary object that must be interpreted by the loader. The loader reads the executable headers, checks that the file is valid for the current system, determines which parts must be mapped into memory, prepares the initial process state, and finally transfers control to the program's entry point.

The lifecycle can be simplified as the following transformation chain:

```
executable file on disk
      |
      | kernel / loader reads executable format
      v
validated executable image description
      |
      | virtual memory mappings are created
      v
process virtual address space
      |
      | stack, arguments, environment, registers prepared
      v
initial execution context
      |
      | control transferred to entry point
      v
running process
```

The first major operation is validation. The kernel or loader must determine whether the file is actually an executable of a supported format. It checks the executable header, the target architecture, permissions, required loader behavior, and sometimes security constraints. A Linux system cannot normally execute a Windows PE file directly because the loader does not treat it as a native ELF executable. A processor cannot execute instructions compiled for an incompatible architecture unless an emulation or translation layer is involved.

Once the executable is accepted, the operating system creates or replaces a process image. In the common Unix `fork()` followed by `exec()` model, `fork()` creates a new process by duplicating the current one, and `exec()` replaces that process's memory image with the new executable. In other systems or APIs, process creation and executable loading may be combined into one operation. Regardless of the interface, the result is that the operating system constructs a new runtime container for the program.

This runtime container is the process. The process receives a virtual address space. The executable's loadable regions are mapped into that address space according to the executable format. Code regions are usually mapped as readable and executable but not writable. Data regions are usually mapped as readable and writable. Read-only constants may be mapped as readable but not writable. The heap and stack are not simply copied from the executable file; they are runtime memory regions created for the process.

This is the point where the static structure of the executable becomes an active memory layout. The binary file may describe that a certain segment must appear at a certain virtual address range with certain permissions. The loader uses that description to create virtual memory mappings. These mappings connect regions of the process address space to file-backed pages, anonymous memory pages, or other operating-system-managed memory objects.

The transformation is not necessarily eager. Modern operating systems usually do not copy every byte of the executable into physical memory immediately. Instead, they create mappings. Pages may

be loaded from disk only when they are first touched. This is possible because virtual memory and page faults allow the operating system to delay physical loading until actual access. From the program's point of view, the memory appears available. Internally, the kernel may populate it lazily.

The loader also prepares memory that does not come directly from the executable file. The stack must be created. Command-line arguments must be placed in a form that startup code can read. Environment variables must be made available. Auxiliary information may be written for the runtime or dynamic linker. The initial stack is therefore not an arbitrary empty region. It is a carefully prepared runtime structure.

If the executable depends on shared libraries, the dynamic linking machinery also becomes part of the lifecycle. The executable may request libraries such as the C runtime library or other shared objects. These libraries must be located, validated, mapped into the process address space, and connected to the executable through symbol resolution and relocation mechanisms. The process that begins execution is often not only the main executable; it is the main executable plus a set of mapped shared libraries.

After memory mappings are prepared, the operating system initializes the execution context. The processor must be given a starting instruction address. The stack pointer must point to the initial stack. Other registers may be initialized according to the platform ABI. The process must also be registered with the scheduler so that it can receive CPU time.

The final step is control handover. The kernel returns from its loading path not to the caller's ordinary next instruction, but into the newly prepared execution context. The instruction pointer is set to the executable's entry point or to a dynamic loader entry point that will eventually reach the program's startup code. At that moment, the stored binary has become an active running process.

For a C program, this does not usually mean that the first executed user-level instruction is the first instruction inside `main()`. The native entry point normally belongs to runtime startup code. That startup code prepares the C runtime environment, initializes required runtime structures, handles constructors or initialization arrays if needed, prepares arguments, and then calls `main()`. When `main()` returns, control usually goes back to runtime cleanup code, which then terminates the process through an operating-system call.

This distinction matters because `main()` is the source-level entry convention of C programs, not necessarily the first native instruction executed after loading. The operating system loader works with executable-format entry points. The C runtime then bridges from that low-level entry point to the programmer's `main()` function.

The same lifecycle also explains what happens when Python code is launched from a command line such as:

```
python script.py
```

The operating system does not transform `script.py` into a native process. Instead, it loads the executable binary named `python` or `python.exe`. That executable goes through the native loading lifecycle described above. Once the CPython process is active, the interpreter opens `script.py`, parses it, compiles it to bytecode, creates Python-level execution frames, and executes the program inside the interpreter process.

Thus, there are two transformation layers in the Python case. At the operating-system layer, the CPython interpreter executable is transformed from storage binary into active process. At the language-runtime layer, the Python source file is transformed into bytecode and executed by the interpreter. Confusing these two layers leads to the mistaken idea that the operating system directly runs Python source code.

The lifecycle from stored binary to running process is therefore the operating-system side of execution. It is the stage where a passive executable file becomes an isolated virtual memory environment with code mappings, data mappings, stack, heap potential, loaded libraries, register state, and scheduler identity. Only after this transformation does the processor begin executing the program's instructions.

The OS Loader Subsystem: VMA Memory Mapping, Page Table Initialization, and Stack Argument Injection

The operating system loader is the component responsible for turning the executable file's loading description into a real process memory image. The executable file already contains information about which regions must be loaded, which permissions they require, where execution should begin, and which dynamic dependencies may be needed. The loader interprets this information and builds the initial virtual memory structure of the process.

The loader does not treat the executable as a simple byte array. It reads the executable format and identifies its loadable regions. In ELF, these are described mainly by program headers. In PE/COFF, similar information is described through PE headers and section tables. The loader uses this metadata to decide which parts of the file become memory regions in the process virtual address space.

On Linux, these memory regions are commonly described as **virtual memory areas**, or VMAs. A VMA is a continuous range of virtual addresses with common properties: readable, writable, executable, private, shared, file-backed, anonymous, and so on. A process is not just given one large block of memory. Its address space is built from many mapped regions, each with a specific role.

A simplified process memory map may contain regions such as:

- executable code mapped from the program file;
- read-only constants mapped from the program file;
- initialized writable data mapped from the program file;
- uninitialized data backed by zero-filled memory;
- shared libraries mapped into the same process;
- the process heap;
- the main thread stack;
- special kernel or runtime helper mappings.

Each region has permissions. The code region is normally executable and readable, but not writable. Writable data is normally readable and writable, but not executable. These permissions are part of process isolation and security. If a program tries to write into a read-only code segment, the processor and operating system can detect the violation and raise a memory protection fault.

The loader creates mappings, not merely copies. A file-backed mapping connects a virtual address range to bytes stored inside the executable or shared library file. The physical memory page may not be loaded immediately. Instead, the operating system may wait until the process first accesses that page. When access occurs, a page fault brings the required page into physical memory. This lazy loading avoids unnecessary work and is one reason large programs can start without reading every byte of every mapped file immediately.

The loader also initializes anonymous memory regions. Anonymous memory is memory that does not directly come from a file. The stack is anonymous. Heap pages are anonymous when first created. The BSS region, which represents uninitialized global or static storage, is usually backed by zero-filled memory rather than by explicit bytes stored in the executable. The executable describes the size of this region, but the file does not need to physically contain thousands or millions of zero bytes.

After deciding what virtual memory regions must exist, the operating system must make address translation possible. This requires page table initialization. A page table is a data structure used by the processor's memory management unit to translate virtual addresses into physical memory locations or into faults that the kernel can handle.

From the program's point of view, an instruction may access an address such as:

0x7fffffffdc00

The processor does not treat this as a physical RAM address directly. It asks the memory management unit to translate it through the page tables associated with the current process. If the page table says the address is valid and has the requested permission, the access continues. If the page is not present, or if the requested access violates permissions, the processor traps into the kernel.

This is how the operating system gives each process its own private address space. Two processes may both contain a virtual address such as `0x400000`, but their page tables may map that address to different physical pages. The virtual address is meaningful only inside the address space of the current process.

The page tables do not have to map every possible virtual address. Most of the address space is usually unmapped. If a program dereferences a pointer into unmapped memory, the page-table lookup fails and the kernel receives a trap. If the access is invalid, the process may be terminated with an error such as a segmentation fault or access violation.

The loader must also prepare the initial stack. The stack is not empty when user code begins. It contains startup information required by the runtime. In a C program, `main()` commonly receives arguments in the form:

```
int main(int argc, char *argv[])
```

The values `argc` and `argv` do not appear by magic. They are constructed from command-line arguments supplied when the program was launched. The loader and runtime startup code cooperate to place this information into memory and make it available in the form expected by the C runtime.

The initial stack usually contains information such as:

- command-line argument strings;
- pointers to those argument strings;
- the argument count;
- environment variable strings;
- pointers to environment variables;
- auxiliary platform information used by the runtime or dynamic loader.

A simplified view is:

```
initial stack
  argc
  argv[0] -> program name string
  argv[1] -> first command-line argument
  argv[2] -> second command-line argument
  ...
  envp[0] -> first environment string
  envp[1] -> second environment string
  ...
  auxiliary runtime data
```

The exact layout depends on the operating system and ABI, but the principle is stable: the process starts with an already prepared stack containing the data needed by the startup code. The programmer sees this through `argc`, `argv`, environment access functions, or higher-level runtime abstractions.

This stack preparation is called **stack argument injection** in the sense that the operating system constructs the initial stack content before the user-level program begins normal execution. The future process does not read command-line arguments from nowhere. They are copied or mapped into the process address space by the loader path.

The loader also prepares dynamic libraries when needed. If the executable depends on shared libraries, the dynamic loader maps those libraries into the same process address space. Each library

receives its own code and data mappings. Some relocations may be applied immediately. Some symbol resolutions may be delayed depending on lazy binding behavior. The important point is that the final process image usually contains more than the main executable file.

Once mappings are established, page tables are prepared, dynamic dependencies are handled, and the initial stack is built, the process has the memory structure required to run. The loader can then prepare the initial register state. The instruction pointer must point to the correct entry location. The stack pointer must point to the top of the prepared stack. Other registers must satisfy the platform's startup convention.

Only after these steps can the process begin executing user-mode instructions. The executable file has now been transformed into a process memory image: code is mapped, data is mapped, the stack exists, arguments exist, libraries may be present, page tables define valid address translation, and the processor has a starting execution context.

This also clarifies the Python case. When the command

```
python script.py
```

is launched, the loader performs these operations for the native Python interpreter executable. The process memory image belongs to CPython. The argument `script.py` is placed among the command-line arguments available to that interpreter process. CPython later reads that file as input, compiles it to bytecode, and executes it inside its own runtime. The loader does not map the Python source file as executable machine code. It maps and starts the interpreter executable.

Therefore, the loader subsystem is the concrete operating-system machinery that builds a process from an executable. It creates virtual memory areas, initializes page-table mappings, prepares file-backed and anonymous memory regions, constructs the initial stack, injects command-line and environment data, maps shared libraries when required, and sets the initial execution context. Without this loading phase, an executable file would remain only a passive binary object on storage.

Relocation Invariants and Control Handover to the Hardware Thread Execution Context

After the executable and its required libraries have been mapped into the process virtual address space, the loader must make sure that all address-dependent references are valid. This is the role of relocation. Relocation is the process of adjusting addresses, offsets, pointers, and symbolic references so that code and data refer to their real runtime locations inside the process.

A compiled program often contains references to functions, global variables, constants, jump targets, tables, and external library symbols. At compile time, many of these references cannot be known as final absolute virtual addresses. The compiler and assembler may know that a function named `printf` is needed, or that a global variable named `counter` exists, but they may not know the final address where that function or variable will appear after linking and loading.

Some address decisions are made during static linking. The linker combines object files, resolves many symbols, assigns section layouts, and patches many references. However, not everything can always be fixed before execution. Dynamically linked libraries may be mapped at addresses chosen when the process starts. Security mechanisms such as address space layout randomization, commonly called ASLR, intentionally vary load addresses between executions. Shared libraries may also be loaded in different regions depending on what else is already mapped into the process.

Therefore, the executable may still contain relocation entries. A relocation entry tells the loader or dynamic linker that a particular location in code or data must be adjusted once the final runtime address of a symbol or segment is known. Conceptually, a relocation entry says:

When the process image is created, compute the real address or offset of this target and write the corrected value here.

For example, if a global pointer must point to a global object, the executable may contain a placeholder value. During loading, the dynamic linker computes the actual virtual address of that object and writes the correct pointer value. If a function call targets a symbol in a shared library, the

dynamic linker must connect that call path to the loaded library function. If a table contains addresses of functions, those addresses may need to be fixed before the program uses the table.

Relocation must preserve several invariants. An invariant is a condition that must remain true for execution to be correct. Important relocation invariants include:

- every required external symbol must resolve to a valid definition;
- every relocated address must point into a valid mapped region;
- executable code must not refer to stale link-time addresses if the runtime load address differs;
- relocation must respect memory permissions, such as read-only code and writable data;
- the final process image must be internally consistent before control is given to user-level startup code.

If these invariants are not satisfied, execution cannot safely begin. A missing shared library, an unresolved external symbol, an incompatible binary interface, or an invalid relocation can prevent the program from starting. In such cases, the loader reports an error before the program reaches its normal entry point.

Modern systems try to reduce the amount of relocation needed at load time. One important technique is **position-independent code**, often abbreviated as PIC. Position-independent code is generated so that it can execute correctly regardless of the exact virtual address where it is loaded. Instead of relying heavily on fixed absolute addresses, it uses relative addressing, tables, and indirection mechanisms. Shared libraries commonly use this style because the same library may be mapped into different address regions in different processes.

Dynamic linking often uses tables for external references. In ELF systems, mechanisms such as the Global Offset Table and Procedure Linkage Table are used to support access to external data and functions. The exact details are platform-specific, but the idea is simple: rather than hard-coding every external address directly into instructions, the program uses runtime-filled tables or stubs that point to the correct target after loading. Windows PE executables use related import table mechanisms to connect imported functions from DLLs.

Relocation also interacts with memory protection. Code pages are normally mapped as executable and read-only. Data pages are normally writable but not executable. If relocation requires writing corrected addresses into memory, the loader must do this in regions where such writing is allowed, or temporarily adjust permissions in a controlled way. After relocation, some regions may become read-only again. This reduces the possibility that a program or attacker can modify executable code or critical linking structures at runtime.

Once relocation and dynamic symbol resolution have reached the required state, the process image is ready for execution. The loader has mapped executable segments, prepared data segments, created the stack, injected arguments and environment information, mapped shared libraries, and fixed required addresses. The remaining task is to hand control to the initial execution context.

A running program is not started by a vague command such as “begin running this file”. The processor needs concrete register values. Most importantly, it needs an instruction pointer value. The instruction pointer is the register that tells the processor where the next instruction is located. On x86-64 this role is associated with RIP. Other architectures have their own names for the same concept.

The loader prepares the initial hardware thread context. A hardware thread context includes the instruction pointer, stack pointer, processor mode, and other register state required to begin execution. The stack pointer must point to the prepared initial stack. The instruction pointer must point to the executable entry location or to a dynamic loader entry location that will later transfer control to the program startup code.

This control handover is not the same as an ordinary function call from user code. Before the program starts, there is no caller frame from the program itself. The kernel or loader path constructs the process state and then arranges for the processor to resume execution in user mode at the selected entry address. From that moment onward, the process executes its first user-mode instruction.

The selected entry address is usually not the C function `main()` directly. In a normal C program, the executable entry point usually belongs to runtime startup code. That startup code is responsible for preparing the C runtime environment. It may initialize runtime structures, process initialization arrays, prepare standard library state, arrange arguments in the form expected by `main()`, and then call `main()`.

A simplified handover chain looks like this:

```
kernel / loader
    prepares process memory and registers
        |
        v
native executable entry point
    |
    v
C runtime startup code
    |
    v
main(argc, argv, envp)
    |
    v
runtime cleanup and process termination
```

This is why `main()` should be understood as the conventional source-level entry point of a C program, not necessarily as the first instruction executed in the process. The operating system loader works with executable entry points. The C runtime bridges from that low-level entry point to the programmer's `main()` function.

The same model applies when launching Python, but with another layer added. When the command `python script.py`

is executed, the loader prepares the process image for the native CPython executable. Relocations are applied to the interpreter executable and its native shared libraries. The instruction pointer is handed to CPython's native startup code. CPython initializes its runtime, reads the command-line argument `script.py`, opens that file, parses it, compiles it to bytecode, and executes the bytecode inside the interpreter.

Thus, the hardware thread begins by executing native machine instructions from the Python interpreter, not instructions from the Python source file. The Python program's execution begins later, inside the already-running interpreter process.

Relocation and control handover are therefore the final steps that convert a prepared process image into actual CPU execution. Relocation guarantees that all address-dependent structures refer to valid runtime locations. Control handover installs the initial register state and gives the processor an instruction address from which execution can begin. At that moment, the executable stops being only a mapped binary image and becomes an actively executing process.

Native Entry Points: Loader Handover, C Runtime Startup, and the Call to `main()`

When an executable file is loaded into memory, execution does not begin in an arbitrary place. The executable format contains an **entry point**: a virtual address that tells the loader where the first user-mode instruction of the loaded program image should be executed. After memory mappings, relocations, stack preparation, shared library loading, and register initialization are complete, the loader transfers control to this entry point.

This entry point is a native machine-code address. It belongs to the executable image or, in dynamically linked cases, sometimes first to dynamic loader startup code. It is not chosen by scanning the source file for a function named `main`. The operating system loader does not understand C source-level conventions. It understands executable metadata, virtual addresses, memory permissions, and processor state.

This distinction is important because C programmers often say that “a C program starts at `main()`.” This is true at the source-code programming model level, but it is not exactly true at the native loading level. The operating system does not usually call `main()` directly. The operating system transfers control to the executable’s native entry point. That entry point normally belongs to the **C runtime startup code**. The C runtime then prepares the environment required by the C program and eventually calls `main()`.

A simplified execution chain looks like this:

```
operating system loader
    |
    v
native executable entry point
    |
    v
C runtime startup code
    |
    v
main(argc, argv, envp)
    |
    v
return from main
    |
    v
C runtime cleanup
    |
    v
process termination system call
```

The C runtime startup code is a small but important layer between the loader and the programmer’s function. Its exact implementation depends on the operating system, compiler, standard library, and executable format. On Linux systems using common C toolchains, startup code is commonly associated with symbols such as `_start`. On Windows, the corresponding startup path is different, but the architectural idea is the same: there is lower-level startup code before the source-level `main()` function runs.

The startup code performs several initialization tasks. It must arrange the command-line arguments into the form expected by C. It must prepare the C standard library environment. It may initialize global constructors in C++ or initialization arrays in C toolchains. It may prepare thread-local storage, runtime bookkeeping, environment access, standard streams, and other runtime details. Only after these tasks are ready does it call the programmer’s entry function.

In ordinary hosted C programs, that function is usually written as one of the following forms:

```
int main(void)
```

or:

```
int main(int argc, char *argv[])
```

The first form means the program does not explicitly receive command-line arguments. The second form receives an argument count and an array of pointers to argument strings. Some environments also expose environment variables through an additional parameter or through library functions.

The values passed to `main()` are prepared before the function is called. For example:

```
int main(int argc, char *argv[]) {
    return 0;
}
```

does not receive `argc` and `argv` from the compiler as compile-time constants. They are runtime values derived from the process startup state. The loader and startup code cooperate to make the command-line arguments available inside the process memory. The C runtime then passes them to `main()` according to the platform's calling convention.

The call to `main()` is therefore a normal function call made by runtime startup code, not a magical hardware event. By the time `main()` begins executing, the process already exists. Its virtual address space has already been built. Its stack already exists. Its executable and libraries have already been mapped. Required relocations have already been handled. The hardware thread is already running user-mode instructions.

The return from `main()` is also not the immediate destruction of the process by hardware. When `main()` returns an integer, control returns to the C runtime startup layer. That layer can perform cleanup tasks and then terminate the process through the operating system. A statement such as:

```
return 0;
```

inside `main()` conventionally signals successful program termination. A nonzero return value conventionally signals some kind of error or abnormal result. The exact interpretation is made by the surrounding environment, such as the shell or parent process.

A program can also terminate without returning normally from `main()`. It may call `exit()`, abort because of a fatal error, receive a signal, trigger an unhandled hardware fault, or be killed by the operating system or another process with sufficient permission. In these cases, the ordinary return path through `main()` may not occur.

This layered startup model explains why the source-level view and the operating-system view are different. At source level, `main()` is the central beginning of the programmer's logic. At executable level, the real first instruction is whatever address the executable header marks as the entry point. At runtime level, the C runtime startup code bridges these two worlds.

The same distinction becomes essential when comparing C and Python. When a command such as `python script.py`

is executed, the operating system loader starts the native executable `python` or `python.exe`. That executable has its own native entry point and its own runtime startup path, because CPython itself is written largely in C and is built as a native program. The loader does not call a `main()` function inside `script.py`. The file `script.py` is not a native executable and has no operating-system entry point.

Instead, CPython's native process starts first. Its C-level startup code initializes the interpreter runtime, processes command-line arguments, prepares internal memory management, initializes built-in modules and import machinery, and then treats `script.py` as input to the Python runtime. The Python source file is then parsed, compiled to bytecode, and executed inside the interpreter process.

Therefore, there are two different meanings of "program start":

- in native C execution, program startup begins with loader handover to the executable entry point and then proceeds through C runtime startup to `main()`;
- in Python execution, the operating system starts the CPython executable, and the user's Python file starts only later as interpreted runtime input.

This is the final piece of the native loading model. The loader prepares a process and jumps to a native entry point. Runtime startup code prepares language-level execution. Only then does the familiar programmer-defined entry function, such as C's `main()`, begin. Understanding this separation prevents a common confusion: `main()` is the beginning of the programmer's C logic, not the first event in the life of the process.

2.1.4 Memory Layout in Bare-Metal Compiled Processes

The Text Segment: Read-Only Machine Instructions and Instruction Pointer Tracking

When a native executable is loaded into a process, one of the most important memory regions created by the loader is the **text segment**. Despite the name, this segment does not contain human-readable text. It contains the compiled machine instructions of the program. These are the binary instructions that the processor can fetch, decode, and execute.

In the C compilation model, the programmer writes source code such as:

```
int add(int a, int b) {
    return a + b;
}
```

After compilation, assembly, and linking, this function becomes a sequence of machine instructions. Those instructions are stored in the executable file. When the operating system loader creates the process, it maps the executable instruction region into the process virtual address space. This mapped instruction region is commonly called the text segment or code segment.

A simplified process memory map may look like this:

```
high addresses
+-----+
| stack          |
+-----+
| shared libraries |
+-----+
| heap           |
+-----+
| BSS            |
+-----+
| data           |
+-----+
| text / code    |
+-----+
low addresses
```

The exact address ranges are platform-dependent, and modern systems may randomize them for security. The conceptual division is still useful: the text segment stores executable code, while other regions store data, dynamic allocations, or stack frames.

The text segment is normally mapped as **readable and executable**, but not writable. This means the processor may read instructions from it and execute them, but ordinary program instructions should not modify it. This protection is deliberate. If program code were freely writable, then accidental memory corruption or malicious input could modify the instruction stream itself. Marking code pages as non-writable helps enforce a separation between code and data.

This permission model is part of the operating system and hardware memory-protection system. The loader maps pages with permission bits. The memory management unit checks those permissions during execution. If the program tries to write into a read-only code page, the processor traps into the kernel, and the kernel usually terminates the process with a memory protection error.

For example, a C pointer error may accidentally produce an address inside the text segment. If the program attempts to write through that pointer, the write will fail because the mapped page does not allow writing. This is not a C language rule alone. It is enforced by the process virtual memory system.

The processor executes code from the text segment by following the **instruction pointer**. The instruction pointer is a CPU register that stores the virtual address of the next instruction to execute. On x86-64, this register is usually called **RIP**. Other architectures use different names, but the role is the same.

Execution can be imagined as a repeated hardware cycle:

1. read the instruction located at the address stored in the instruction pointer;
2. decode the instruction;
3. execute the instruction;
4. update the instruction pointer to the next instruction, or to a different address if the instruction changes control flow.

For straight-line code, the instruction pointer normally moves forward from one instruction to the next. For example:

```
instruction at address A
instruction at address A + n
instruction at address A + n + m
```

The exact increment depends on instruction length. Some architectures have fixed-length instructions. Others, such as x86-64, have variable-length instructions. The processor still maintains the same conceptual behavior: the instruction pointer identifies the next instruction.

Control-flow instructions modify this normal movement. A conditional branch, unconditional jump, function call, or return changes the instruction pointer. For example, a C `if` statement does not remain an abstract source-level branch after compilation. It becomes comparisons and jumps. The processor follows one path or another by loading a different instruction address into the instruction pointer.

A simplified assembly shape for a condition may look like this:

```
cmp eax, 0
jle else_block
mov ebx, 1
jmp end_if

else_block:
mov ebx, -1

end_if:
```

The source-level idea of an `if/else` structure becomes instruction-pointer redirection. If the comparison result satisfies the jump condition, the instruction pointer moves to `else_block`. Otherwise, execution continues into the next instruction.

Function calls also operate through instruction-pointer changes. When one function calls another, the processor must jump to the callee's code while remembering where to return afterward. In many architectures, the return address is placed on the stack or in a special register. When the callee finishes, a return instruction restores the instruction pointer to the saved return address.

Thus, the text segment and the stack cooperate during function calls:

- the **text segment** contains the machine instructions of both caller and callee;
- the **stack** stores return addresses and frame-related execution state;
- the **instruction pointer** moves through the text segment according to calls, branches, and returns.

For example:

```
main:
    call add
    ...

add:
    ...
    ret
```

The `call` instruction changes the instruction pointer to the address of `add`. It also records the address of the instruction after the call. The `ret` instruction later uses that saved address to continue execution in `main`.

The text segment may also contain more than the programmer's direct function bodies. It can contain compiler-generated helper code, runtime startup code, procedure linkage stubs, exception-handling metadata-related code, and other low-level support routines. In a dynamically linked program, code from shared libraries is mapped into the same process address space, usually in separate text-like executable regions.

This is why a running process usually has more executable code mapped than the code written by the programmer. A small C program that calls `printf()` depends on C runtime and standard library code. A Python process contains the native code of the CPython interpreter, the C runtime, operating-system libraries, and possibly native extension modules.

The read-only nature of the text segment also allows sharing. If several processes run the same executable or use the same shared library, the operating system can map the same physical code pages into multiple process address spaces. Each process sees the code at its own virtual addresses, but the underlying physical memory containing read-only instructions may be shared. This is safe because ordinary processes are not supposed to modify those pages.

This is another reason code and data are separated. Writable data must usually be private to each process. If one process changes a global variable, that change should not automatically modify another process's data. But read-only code can be shared because it is supposed to remain identical.

The text segment therefore represents the executable core of a native process. It is the region where the compiled instruction stream lives. The processor advances through it using the instruction pointer. Branches, calls, and returns redirect that instruction pointer. The operating system protects the region as executable but normally non-writable. The result is a controlled separation between machine instructions and mutable runtime data.

This model is also useful for understanding Python. When a Python script runs, the Python source file does not become the native text segment of the process. The native text segment belongs to the CPython executable and its loaded native libraries. The Python script is compiled into Python bytecode, which is data interpreted by CPython. The physical processor executes machine instructions from CPython's text segment; CPython's interpreter loop then reads and executes Python bytecode as virtual instructions.

Thus, in C, the user's compiled functions become native instructions in the process text segment. In Python, the user's code normally becomes bytecode objects inside the interpreter runtime, while the text segment contains the interpreter's own native machine code. This distinction is one of the central memory-layout differences between bare-metal compiled execution and interpreted runtime execution.

The Data and BSS Segments: Initialized and Uninitialized Global/Static Variable Storage Boundaries

Besides the text segment, which stores machine instructions, a native compiled process also needs memory regions for values that exist for the whole lifetime of the program. In C, these are mainly global variables and static variables. They are not created and destroyed with ordinary function calls. They live for the duration of the process. Their storage is prepared when the program is loaded, before normal execution reaches `main()`.

Two traditional regions are used for this kind of storage:

- the **data segment**, for initialized global and static objects;

- the **BSS segment**, for uninitialized or zero-initialized global and static objects.

The distinction is based on whether the executable file must physically store an initial value. Consider the following C declarations:

```
int global_counter = 10;
static int module_flag = 1;
```

Both variables have explicit initial values. Their initial bytes must be stored somewhere in the executable file, because the loader needs to place those exact values into the process memory image. These objects belong conceptually to the initialized data region.

Now consider:

```
int global_total;
static int cached_value;
```

These variables have static storage duration, but no explicit initializer. In C, such objects are automatically initialized to zero before program execution begins. The executable does not need to store thousands of zero bytes for such objects. It only needs to record that a certain amount of zero-filled memory must be reserved. This region is traditionally called **BSS**.

The name BSS comes from older assembler terminology, but the modern practical meaning is simple: it is the region for static-lifetime objects that start as zero without requiring their zero bytes to be stored directly in the executable file. The loader creates the memory and ensures that it is zero-filled.

A simplified classification is:

```
int a = 5;           // initialized data
int b;               // BSS, zero-initialized

static int c = 7;    // initialized data
static int d;        // BSS, zero-initialized
```

This does not mean that **b** and **d** are somehow less real than **a** and **c**. Once the process is running, all of them occupy memory inside the process virtual address space. The difference is how their initial state is represented in the executable file and prepared by the loader.

The data and BSS regions are usually mapped as readable and writable. Unlike the text segment, they must be writable because the program is allowed to change global and static variables during execution. For example:

```
int counter = 0;

void increment(void) {
    counter++;
}
```

The variable **counter** has static storage duration. It is not recreated every time **increment()** is called. It exists once for the whole process, and every call to **increment()** modifies the same stored object.

This is different from an automatic local variable:

```
void f(void) {
    int x = 0;
    x++;
}
```

Here, **x** is normally created inside the function's stack frame each time the function is called. When the function returns, that storage is no longer part of the active function state. By contrast, a static local variable behaves differently:

```
void f(void) {
    static int x = 0;
    x++;
}
```

Although `x` is only visible by name inside `f()`, its storage is not on the ordinary call stack. It has static storage duration. It is initialized once and persists across calls. The name has local visibility, but the memory lifetime is global-like.

This distinction between **scope** and **storage duration** is important. Scope answers the question:

Where in the source code can this name be used?

Storage duration answers a different question:

How long does the object exist in memory?

A global variable has broad scope and static storage duration. A static local variable has local scope but still has static storage duration. An ordinary local variable usually has block scope and automatic storage duration.

The data and BSS segments therefore represent memory whose lifetime is tied to the process itself, not to one function call. They are prepared before `main()` runs and remain valid until the process exits.

The executable file also benefits from separating initialized and uninitialized static data. Suppose a program declares a large global array:

```
int buffer[1000000];
```

This array contains one million integers, all initially zero. If the executable had to store all those zero bytes explicitly, the binary file would become much larger. Instead, the executable records that the BSS region must reserve enough memory for the array and that this memory must begin as zero. The loader can create zero-filled memory without storing those zeros in the file.

By contrast:

```
int table[4] = {10, 20, 30, 40};
```

must store the initial values 10, 20, 30, and 40. These bytes must be present in the initialized data section of the executable, because the loader needs to copy or map them into memory.

A simplified view is:

Executable file:

text section	-> machine instructions
data section	-> initial bytes for initialized globals/statics
BSS description	-> size of zero-filled storage needed

Running process:

text segment	-> mapped executable instructions
data segment	-> writable initialized objects
BSS segment	-> writable zero-initialized objects

The data and BSS regions also interact with relocation and linking. Global variables may be referenced from different object files. A function in one translation unit may access a global variable defined in another translation unit, if it is declared with external linkage. The linker must resolve those symbols and ensure that instructions referring to them use the correct runtime addresses or relocatable access mechanisms.

For example:


```
/* globals.c */
int shared_counter = 0;
```

 Listing 2.1.1: The Data and BSS Segments: Initialized and Uninitialized Global/Static Variab...

```
1  /* main.c */
2  extern int shared_counter;
3
4  int main(void) {
5      shared_counter++;
6      return 0;
7  }
```

The object file produced from `main.c` contains a reference to `shared_counter`. The object file produced from `globals.c` defines it. The linker connects the reference to the definition and arranges the final memory location of the variable in the data or BSS region.

The distinction between data and BSS is mostly invisible at the level of ordinary C syntax, but it matters for executable layout, memory initialization, and binary size. It explains why initialized static objects occupy space in the executable file, while large zero-initialized objects can be represented compactly.

It also helps explain why global mutable state behaves differently from stack-local state. A global or static object is not attached to the lifetime of a function call. Its address remains stable during the process lifetime, unless special relocation or loading mechanisms change the base location before execution begins. Any function that can reach the object can observe or modify the same storage.

This makes global and static mutable state powerful but dangerous. It can be used for configuration, counters, caches, shared module state, and persistent function-local state. But because the state is shared across calls and possibly across many parts of the program, it can also create hidden dependencies. A function that reads or writes a global variable may behave differently depending on previous calls elsewhere in the program.

This memory model contrasts sharply with Python's model. In Python, module-level names may look similar to C global variables, but they do not normally correspond to fixed raw storage slots in a data or BSS segment. A Python module namespace is usually a dictionary mapping names to Python objects. The objects themselves live in the CPython-managed heap. When Python code writes:

```
counter = 10
```

it binds the name `counter` to an integer object. It does not reserve a fixed `int` slot in a native data segment in the same way C does.

However, CPython itself, as a native executable written largely in C, does have native data and BSS regions. The interpreter's own global C variables, runtime tables, static structures, and native library state are stored according to the native process memory model. The user's Python-level globals live above that, inside the runtime object system.

Therefore, in a bare-metal compiled C process, the data and BSS segments represent static-lifetime storage prepared by the loader before execution reaches the programmer's logic. Initialized global and static objects occupy the data segment because their initial bytes must be known. Uninitialized or zero-initialized global and static objects occupy the BSS region because only their size and zero-initialization requirement must be recorded. Together, these regions form the process-level storage boundary for values that persist independently of function call frames.

The Process Heap: Dynamic Manual Memory Allocation and Deallocation Schemes (`malloc()` and `free()`)

The **heap** is the region of a process address space used for dynamic memory allocation. Unlike the text segment, data segment, BSS segment, and ordinary stack frames, the heap is not mainly determined by the static structure of the executable file. It grows and shrinks according to allocation requests made while the program is running.

In C, the heap is the memory area used when the program explicitly asks for storage whose lifetime cannot be conveniently tied to a single function call. The most familiar interface is the standard library allocation family:

```
malloc()
calloc()
realloc()
free()
```

The central function is `malloc()`. It requests a block of memory of a given size and returns a pointer to the beginning of that block:

```
int *p = malloc(10 * sizeof(int));
```

This line asks for enough memory to store ten integers. If the allocation succeeds, `p` receives the address of the allocated block. The program can then use that memory through pointer arithmetic or array syntax:

```
p[0] = 42;
p[1] = 99;
```

This memory is not an automatic local variable. It does not disappear when the current function returns. It remains allocated until the program explicitly releases it with `free()`:

```
free(p);
```

This is the core distinction between stack storage and heap storage. Stack storage is tied to call frames. Heap storage is tied to explicit allocation and deallocation.

For example:

```
void f(void) {
    int x = 10;
}
```

The variable `x` is an automatic local variable. It normally lives in the stack frame of `f()`. When `f()` returns, that frame is no longer active, and `x` is no longer valid.

By contrast:

```
int *make_value(void) {
    int *p = malloc(sizeof(int));
    *p = 10;
    return p;
}
```

Here, `p` itself is a local variable inside the stack frame of `make_value()`, but the memory returned by `malloc()` is on the heap. When the function returns, the local variable `p` disappears, but the heap allocation remains. The returned pointer gives the caller a way to reach that allocated object.

This difference is essential. The pointer variable and the heap object are not the same thing. The pointer is a value that stores an address. The heap block is the memory region located at that address. The pointer may live on the stack, in global storage, inside another heap object, or in a register. The allocated block itself lives in the heap.

A simplified view is:

```

stack frame:
    p  ----+
        |
        v
heap:
+-----+
| allocated int object |
+-----+

```

The heap is useful because many programs need memory whose size or lifetime is not known at compile time. For example, a program may need to read a file of unknown size, build a list of unknown length, create tree nodes, allocate buffers based on user input, or create objects that outlive the function that created them.

A static array requires a known size at compile time or at least a size tied to a particular scope. A heap allocation can be decided dynamically:

```

int n;
scanf("%d", &n);

int *values = malloc(n * sizeof(int));

```

Here, the size of the allocation depends on a runtime value. This is one of the major reasons heap allocation exists.

The heap is managed by an allocator. The C standard library provides the familiar interface, but the allocator itself must obtain memory from the operating system and then divide it into blocks requested by the program. On Unix-like systems, allocators may request memory from the kernel using mechanisms such as `brk()` / `sbrk()` historically, or memory mappings such as `mmap()`. On Windows, equivalent behavior is provided through Windows heap and virtual memory APIs. The exact mechanism differs by platform and allocator implementation.

From the C programmer's point of view, `malloc()` returns a usable region of memory, or returns `NULL` if the allocation fails:

```

int *p = malloc(sizeof(int));

if (p == NULL) {
    /* allocation failed */
}

```

This check matters because heap memory is finite. Allocation may fail if the process cannot obtain enough memory, if address space is exhausted, or if system limits are reached.

The function `calloc()` is similar to `malloc()`, but it allocates memory for a number of elements and initializes the bytes to zero:

```

int *p = calloc(10, sizeof(int));

```

The function `realloc()` changes the size of an existing allocation:

```

p = realloc(p, 20 * sizeof(int));

```

This may enlarge the block in place, or it may allocate a new block, copy the old contents, free the old block, and return the address of the new one. Because the returned address may differ from the original address, `realloc()` must be used carefully.

The function `free()` releases a heap block back to the allocator:

```

free(p);

```

After `free(p)`, the pointer value `p` still contains an address, but that address no longer belongs to a valid allocation owned by the program. Using it after it has been freed is an error called **use after free**:

```
free(p);
p[0] = 13; /* invalid: use after free */
```

This is one of the classic dangers of manual memory management. The compiler cannot generally prevent it, because the invalidity depends on runtime allocation history.

Another common error is a **memory leak**. A memory leak happens when allocated memory is no longer reachable by the program but has not been freed. For example:

```
void leak(void) {
    int *p = malloc(sizeof(int));
    *p = 10;
}
```

When `leak()` returns, the local variable `p` disappears. If no other pointer to the allocated block exists, the program has lost the address of the heap memory. The allocator still considers the block allocated, but the program can no longer free it. Repeating this pattern can make the process consume more and more memory.

A **double free** is another serious error:

```
free(p);
free(p); /* invalid: same allocation released twice */
```

After the first call, the allocation has already been returned to the allocator. Freeing it again corrupts the allocator's understanding of heap state and may crash the program or create security vulnerabilities.

A **buffer overflow** can also occur on the heap. If the program writes beyond the allocated block, it may overwrite allocator metadata or neighboring heap objects:

```
int *p = malloc(10 * sizeof(int));
p[10] = 123; /* invalid: valid indexes are 0 through 9 */
```

The heap does not automatically know the programmer's intended array length at every access. The allocator knows the block size internally, but ordinary C pointer arithmetic does not perform automatic bounds checking. This is one of the reasons C is powerful but unsafe.

The heap allocator must track which blocks are allocated and which blocks are free. It must split large free regions into smaller blocks, merge adjacent free regions when possible, and handle fragmentation. **Fragmentation** means that free memory exists but is divided into pieces that may not satisfy a large allocation request efficiently.

There are two important forms:

- **internal fragmentation**: the allocator gives a block slightly larger than requested because of alignment or size-class rules;
- **external fragmentation**: free memory is scattered into many separate regions.

Allocation performance also depends on allocator strategy. A simple allocator might scan a free list for a block large enough. More advanced allocators use bins, arenas, thread-local caches, size classes, or other structures to make common allocation and deallocation patterns fast. These details are implementation-specific, but the general responsibility is the same: manage variable-lifetime memory inside the process.

The heap is part of the process virtual address space. It is not a separate physical device. When the allocator needs more memory than it currently has, it asks the operating system for additional virtual memory. The kernel updates the process mappings, and physical pages may be supplied lazily when touched. Thus, even heap growth is mediated by the operating system's virtual memory subsystem.

A simplified relationship is:

```

C program
  calls malloc()
    |
    v
C library allocator
  finds or requests heap memory
    |
    v
operating system kernel
  maps additional pages if needed
    |
    v
process virtual address space
  usable heap block returned to program

```

The heap also differs from the data and BSS segments. Global and static objects are allocated as part of the process image before normal execution begins. Heap objects are created during execution. Their number, size, and lifetime may depend on runtime input and control flow.

For example:

```

int global_array[100];          /* data or BSS, static lifetime */

int *dynamic_array = malloc(n * sizeof(int)); /* heap, dynamic lifetime */

```

The global array exists for the entire process and has a fixed size determined by the program. The heap allocation can depend on `n`, a runtime value. It must later be released manually.

This model explains why C programmers must think carefully about ownership. If a function allocates memory, who is responsible for freeing it? The same function? The caller? Another object or subsystem? C does not enforce an ownership discipline automatically. It is a design rule imposed by the programmer.

For example:

```

char *make_buffer(size_t n) {
    char *p = malloc(n);
    return p;
}

```

This function returns a heap pointer. The caller must understand that it has received ownership of an allocation that must eventually be freed:

```

char *buf = make_buffer(1024);

/* use buf */

free(buf);

```

If this convention is unclear, leaks, double frees, or invalid accesses become likely.

This is a major contrast with Python. Python code also creates objects dynamically, but the programmer normally does not call `malloc()` and `free()` directly. Python objects are allocated in the CPython-managed heap. The runtime tracks references to objects, reclaims most objects through reference counting, and uses a garbage collector for certain cyclic structures.

For example:

```

a = [1, 2, 3]

```

creates a Python list object and integer objects or references to integer objects inside the CPython runtime. The Python programmer does not manually free the list. When the list is no longer reachable, CPython eventually releases the memory according to its memory-management rules.

This does not mean Python has no heap. It means the heap is managed differently. In C, heap lifetime is explicit and manual. In Python, object lifetime is mediated by the interpreter's object model, reference counting, garbage collection, and allocator subsystems. The programmer still needs to understand references, aliasing, mutability, and resource lifetime, but not ordinary manual deallocation of each object.

The difference can be summarized as follows:

- in C, `malloc()` creates heap storage and `free()` releases it;
- in Python, object creation allocates runtime-managed heap objects, and the interpreter decides when their memory can be reclaimed;
- in C, losing the last pointer without freeing causes a memory leak;
- in Python, losing the last reference usually makes the object reclaimable.

Therefore, the process heap is the memory region that supports dynamic runtime allocation. In bare-metal compiled C programs, it is controlled through explicit allocation and deallocation functions. This gives the programmer precise control over memory lifetime and layout, but also exposes the program to leaks, dangling pointers, double frees, and buffer overflows. Python preserves the idea of dynamically allocated heap objects, but moves most allocation and reclamation responsibility into the runtime system.

The Process Stack: Automatic Variable Lifetime Tracking, Call Chains, and Push/Pop Stack Frames

The **process stack** is the memory region used to track active function calls and their local execution state. In a traditional compiled C program, the stack is where the runtime records the chain of function calls currently in progress. Each active function call receives a portion of the stack called a **stack frame**. When a function is called, a new frame is created. When the function returns, that frame is removed.

This gives stack memory a very different lifetime model from heap memory. Heap memory is controlled manually in C through functions such as `malloc()` and `free()`. Stack memory is controlled automatically by the call and return structure of the program. The programmer does not normally allocate and free ordinary stack frames directly. They are created and destroyed as functions are entered and exited.

Consider a simple C program:

</>C Listing 2.1.2: The Process Stack: Automatic Variable Lifetime Tracking, Call Chains, and Pus...

```
1  int square(int x) {
2      int result = x * x;
3      return result;
4  }
5
6  int main(void) {
7      int a = 5;
8      int b = square(a);
9      return 0;
10 }
```

When `main()` begins, a stack frame is created for `main`. That frame may contain local variables such as `a` and `b`, along with bookkeeping information required by the calling convention. When `main()` calls `square(a)`, a new stack frame is created for `square`. That frame contains the parameter `x`, the

local variable `result`, and information needed to return to `main`. When `square()` returns, its frame is removed, and execution continues in `main()`.

A simplified call chain may look like this:

```
main()
  calls square()
    square() returns
main() continues
```

The corresponding stack behavior is:

before call:

```
+-----+
| frame for main |
+-----+
```

during `square()`:

```
+-----+
| frame for square |
+-----+
| frame for main   |
+-----+
```

after `square` returns:

```
+-----+
| frame for main   |
+-----+
```

This is why the stack is often described as a *last-in, first-out* structure. The most recently called function is the first one that must return. A function cannot normally return before the functions it called have returned, unless abnormal control-flow mechanisms intervene. The active call chain is therefore naturally stack-shaped.

The word *push* means adding data to the top of the stack. The word *pop* means removing data from the top of the stack. Function calls push new call-frame information. Function returns pop that information. The exact machine instructions and layout depend on the processor architecture, compiler, optimization level, and calling convention, but the conceptual behavior remains the same.

A stack frame may contain several categories of information:

- the function's automatic local variables;
- copies of function arguments or references to argument locations;
- saved register values that must be restored after the call;
- the return address, meaning the instruction location where execution must continue after the function returns;
- bookkeeping data used for stack unwinding, debugging, or exception handling;
- temporary storage needed by the compiler.

The **return address** is especially important. When one function calls another, the processor must remember where to continue after the called function finishes. In many calling conventions, the return address is placed on the stack. When the called function executes a return instruction, that saved address is used to restore the instruction pointer.

For example:

```

main:
    call square
    ; execution continues here after square returns

square:
    ...
    ret

```

The `call` instruction transfers control to `square` and records where to return. The `ret` instruction later restores the instruction pointer to that saved location. This is how the processor moves through the text segment while the stack records the call chain.

Two processor registers are commonly involved in stack management:

- the **stack pointer**, which tracks the current top of the stack;
- the **frame pointer** or base pointer, which may provide a stable reference point for the current function frame.

On x86-64, these are commonly associated with registers such as `RSP` and `RBP`. Other architectures use different register names. The stack pointer changes as values are pushed or popped. The frame pointer, when used, gives the compiler a stable base from which it can address local variables and parameters using fixed offsets.

A simplified frame may look like this:

```

+-----+
| previous frame information |
+-----+
| return address            |
+-----+
| saved frame pointer       |
+-----+
| local variable 1          |
+-----+
| local variable 2          |
+-----+
| temporary compiler storage|
+-----+

```

The actual order and content are platform-dependent, but the idea is stable: a stack frame is a compact record of one active function call.

Automatic local variables in C are often stored in the function's stack frame. For example:

```

void f(void) {
    int x = 10;
    int y = 20;
}

```

The variables `x` and `y` are automatic variables. Their lifetime begins when execution enters the block where they are defined and ends when that block or function returns. In many ordinary cases, their storage is part of the stack frame. When the frame is removed, the variables no longer exist as valid objects.

This is why returning the address of a local variable is invalid:

```

int *bad_function(void) {
    int x = 10;
    return &x;    /* invalid: x dies when the function returns */
}

```


The pointer returned by this function points to memory that belonged to the stack frame of `bad_function`. Once the function returns, that frame is no longer active. The address may still contain the old numeric value, but the object `x` no longer exists. Later stack activity may overwrite the same memory. Using such a pointer produces undefined behavior.

This is different from returning a heap pointer:

```
int *make_value(void) {
    int *p = malloc(sizeof(int));
    *p = 10;
    return p;
}
```

Here, the local variable `p` disappears when the function returns, but the allocated heap object remains alive until `free()` is called. The lifetime of the pointer variable and the lifetime of the pointed-to object are different.

The stack is also where recursive calls accumulate. A recursive function calls itself, creating a new frame each time:

```
int factorial(int n) {
    if (n == 0) {
        return 1;
    }
    return n * factorial(n - 1);
}
```

Each recursive call has its own parameter `n` and its own frame. Calling `factorial(3)` creates a chain similar to:

```
factorial(3)
  factorial(2)
    factorial(1)
      factorial(0)
```

At the deepest point, several frames for the same function exist at the same time. They are separate frames, each with its own local state.

If recursion is too deep, or if functions allocate very large local objects, the stack can run out of space. This condition is usually called **stack overflow**. The stack has a limited size. Unlike the heap, it is not intended for arbitrary large dynamic storage. Large arrays or long-lived dynamically sized structures are usually better placed on the heap.

For example:

```
void dangerous(void) {
    int huge_array[1000000000];
}
```

This attempts to place a very large object in the function's stack frame. On many systems, it will exceed the available stack space and crash the program. A heap allocation would be more appropriate for such a large runtime object:

```
int *huge_array = malloc(1000000000 * sizeof(int));
```

The stack's automatic lifetime rules are useful because they make common function-local storage simple and fast. Allocating stack space is often just a matter of adjusting the stack pointer. Freeing it is often just reversing that adjustment when the function returns. This is much cheaper than general heap allocation, where the allocator must manage variable-sized blocks, free lists, fragmentation, and possible system calls.

However, the stack is less flexible than the heap. Stack objects have lifetimes tied to execution scope. They cannot safely outlive their function call unless their values are copied somewhere else. Heap objects can outlive the function that created them, but they require explicit lifetime management in C.

The process stack also plays a role in debugging and error reporting. A debugger can inspect the call stack to show which functions called which other functions. A stack trace or backtrace is a human-readable reconstruction of the active call chain. It may show, for example, that `main()` called `load_file()`, which called `parse_line()`, which crashed because of an invalid pointer.

A simplified backtrace might look like:

```
parse_line()
load_file()
main()
```

This is possible because call-frame information and return addresses preserve the chain of active calls.

In optimized builds, compilers may change the visible stack-frame structure. They may keep some local variables in registers, omit the frame pointer, inline functions, or remove unused variables. Therefore, the conceptual model of stack frames is essential, but the exact memory layout is not guaranteed to match a simple textbook diagram in every compiled binary.

This stack model is also important when moving from C to Python. Python has function calls and call chains, but Python-level frames are not the same as raw C stack frames. CPython maintains Python frame objects or internal frame structures to represent executing Python code. These frames contain references to Python objects, local names, bytecode execution state, and links to previous frames. At the same time, the CPython interpreter itself is a native C program, so it also uses the native process stack while executing its own C functions.

Therefore, when Python code calls a Python function, two levels exist:

- at the Python level, a Python execution frame is created or managed by the interpreter;
- at the native level, CPython's C code runs on the process stack like any other native program.

This distinction prevents a common confusion. A Python local variable is not usually a raw integer slot in the native stack like an automatic C variable. A Python local name is part of a Python execution frame and refers to a Python object stored in the managed heap. The native stack belongs to the interpreter's implementation; the Python frame belongs to the language runtime model.

Thus, the process stack is the native mechanism for tracking active function execution in compiled programs. It records call chains, return addresses, local automatic storage, saved registers, and temporary state. Its push/pop behavior matches the nested structure of function calls. It gives C efficient automatic variable lifetime management, but it also imposes lifetime limits: stack objects disappear when their frames disappear. Python preserves the idea of function frames and call chains, but it implements Python-level local variables and frames through the runtime system rather than exposing the raw native stack directly.

2.1.5 Operating System Resource Management

The Evolution of Multitasking Subsystems: Time-Slicing Hardware Resources

Early computers did not begin with the process model that modern programmers now assume. In the simplest execution model, one program occupied the machine, used the processor until it finished, and then another program could be loaded. The machine was expensive, the processor was scarce, and program execution was treated as a mostly exclusive activity. If the running program waited for input, output, tape, disk, printer access, or a human response, the processor could remain unused even though other work was available.

This was inefficient because processors are fast compared with many external resources. A program that waits for disk input or terminal input is not doing useful CPU computation during that waiting

period. If the operating system can suspend that waiting program and let another ready program use the processor, total machine utilization improves. This is the basic reason multitasking systems evolved.

Multitasking means that the operating system allows multiple execution activities to exist during the same time interval. This does not necessarily mean that all of them execute machine instructions at the exact same instant. On a single processor core, only one instruction stream can execute at a given exact moment. Multitasking means that the operating system switches among runnable activities quickly enough that several programs can make progress over time.

The central mechanism is **time-slicing**. The operating system divides CPU access into small intervals, often called time slices or scheduling quanta. A process or thread receives the processor for a short time. When that interval expires, or when the running activity blocks on input/output, the kernel may suspend it and choose another activity to run.

A simplified timeline on a single core may look like this:

```
time ----->

CPU:  Process A | Process B | Process C | Process A | Process B
      slice 1   slice 2   slice 3   slice 4   slice 5
```

At human time scales, this can look like simultaneous execution. A text editor remains responsive, a terminal command continues running, a network download progresses, and the graphical interface still reacts. Internally, the operating system is repeatedly choosing which execution context receives the processor.

This is not the same as physical parallelism. On one hardware execution context, the CPU executes one instruction stream at a time. The apparent simultaneity comes from rapid switching. True parallel execution requires multiple hardware execution contexts, such as multiple CPU cores or simultaneous multithreading. Multitasking was important even before multicore machines because it allowed the operating system to hide waiting time and share one processor among many active programs.

The need for multitasking becomes clear when considering input/output. Suppose a process reads data from a disk. The disk operation is slow compared with CPU instruction execution. If the operating system allowed the process to occupy the CPU while waiting, the CPU would waste time. Instead, the process can be moved into a waiting or blocked state. Another process that is ready to compute can run.

A simplified state model is:

```
ready  ----scheduled---->  running
running ----blocks----->  waiting
waiting ----event ready->  ready
running ----time slice-->  ready
running ----exits----->  terminated
```

A **ready** process can run but is not currently using the CPU. A **running** process is currently executing instructions. A **waiting** or **blocked** process cannot continue until some event occurs, such as disk completion, network input, a timer, or user input.

This state management is one of the main responsibilities of the operating system kernel. The kernel must know which execution contexts exist, which are runnable, which are blocked, and which have terminated. It must also decide how to divide CPU time fairly or according to priority.

The hardware timer is essential in many multitasking systems. The timer can periodically interrupt the currently running program and transfer control to the kernel. This prevents one program from keeping the CPU forever. When the timer interrupt occurs, the processor stops ordinary user-mode execution, enters kernel mode, and runs the kernel's interrupt-handling and scheduling code. The kernel may then decide to resume the same process or switch to another one.

This hardware-supported interruption is what makes forced CPU sharing possible. Without it, a badly written or malicious program could run an infinite loop and never voluntarily give control back

to the operating system. With timer interrupts, the kernel can regain control even if the program does not cooperate.

For example, a program may contain:

```
while (1) {  
    /* never voluntarily stops */  
}
```

In a system without preemptive control, this kind of loop could freeze the machine or the environment. In a modern operating system, the timer interrupt eventually transfers control back to the kernel, and the scheduler can allow other processes to run.

Time-slicing also requires the operating system to preserve execution state. If Process A is suspended and Process B is run, Process A must later continue from the exact point where it stopped. This means the kernel must save enough state to resume it: instruction pointer, stack pointer, CPU registers, scheduling state, and memory mapping identity. When Process A is selected again, the kernel restores this saved context, and the process continues as if execution had merely been delayed.

From the process's point of view, it usually does not know that it was suspended. It simply observes that time has passed. Its instruction pointer still identifies the next instruction to execute. Its stack still contains its call frames. Its heap and global data remain mapped. Its open files and other resources still exist unless another event changed them.

This preservation of state is what allows multiple processes to share one processor safely. The processor itself does not contain the complete permanent state of every process. It contains the state of the currently running execution context. The operating system saves and restores this state as it switches between contexts.

The evolution from single-program execution to multitasking also required memory isolation. If several programs exist in memory at the same time, one program must not freely overwrite another program's memory. Modern systems use virtual memory and the memory management unit to give each process its own address space. This allows the kernel to switch CPU time between processes while keeping their memory regions separated.

Therefore, multitasking is not only a scheduling idea. It depends on several architectural components working together:

- timer interrupts, so the kernel can regain control;
- privileged kernel mode, so user programs cannot control scheduling directly;
- saved execution contexts, so suspended processes can resume;
- virtual memory, so processes can coexist without overwriting each other;
- process tables and scheduler data structures, so the kernel can track runnable and blocked activities.

The operating system also time-shares other resources besides the processor. Disk access, network access, terminal access, memory pages, file descriptors, and device operations are all mediated by the kernel. However, CPU time-slicing is the central model because the CPU is the component that actually advances instruction execution. A process that does not receive CPU time cannot execute its next instruction.

This model explains why the process is not merely a loaded executable file. A process is a schedulable execution entity. It can be running, ready, blocked, stopped, or terminated. The same program file may produce several processes, and each process may receive CPU time independently. The operating system controls when each one advances.

For C programs, this means that the compiled native code does not own the machine. Even though C compiles to processor instructions, those instructions execute only when the scheduler gives the process CPU time. The process runs inside an operating-system-controlled environment. Its pointers

are process-virtual addresses. Its files are kernel-managed handles or descriptors. Its execution can be interrupted between instructions by hardware and kernel activity.

For Python programs, the same operating-system layer still exists. When a Python script runs, the schedulable process is the CPython interpreter process. The Python bytecode runs only because the interpreter process receives CPU time from the operating system. If the process is suspended by the scheduler, Python execution is suspended too. If the process blocks on file input, socket input, or sleep, the kernel may run another process during the wait.

This operating-system time-slicing idea later reappears inside high-level runtimes in a different form. Generators, coroutines, event loops, and asynchronous programming systems do not usually control the hardware scheduler. Instead, they reproduce a scheduling-like pattern inside one process: one task runs for a while, reaches a suspension point, and another task is allowed to progress. The mechanism is different, but the conceptual influence is clear.

Thus, the evolution of multitasking subsystems is the evolution from exclusive program execution to controlled sharing of hardware execution time. The operating system kernel divides CPU access into intervals, suspends and resumes execution contexts, hides waiting time, isolates processes, and creates the practical illusion that many programs are active at once. This resource-management model is one of the foundations on which both native compiled execution and high-level runtime execution are built.

Cooperative (Non-Preemptive) vs. Preemptive Multitasking Operating System Architectures

Multitasking systems need a rule for deciding when the currently running execution activity stops and another one receives the processor. Two major models exist: **cooperative multitasking** and **preemptive multitasking**. Both allow several programs or execution contexts to make progress over time, but they differ in who controls the moment of switching.

In **cooperative multitasking**, the running process or task keeps the processor until it voluntarily gives control back to the operating system or runtime scheduler. The program must cooperate. It may yield control explicitly, wait for input/output, call a system function that blocks, or reach another defined suspension point. If it never does so, other tasks may not get a chance to run.

A simplified cooperative timeline looks like this:

```

Process A runs
  |
  | voluntarily yields
  v
Process B runs
  |
  | voluntarily yields
  v
Process C runs
  |
  | voluntarily yields
  v
Process A resumes

```

In this model, the scheduler depends on the behavior of the running program. The operating system or runtime may keep a list of ready tasks, but it normally switches only when the active task gives up control. This can be simple and efficient because switching happens at known points. It can also reduce some synchronization problems, because a task is not interrupted at arbitrary machine-instruction boundaries.

However, cooperative multitasking has a serious weakness: one badly written task can block the whole system. If a task enters an infinite loop and never yields, it can prevent other tasks from running. For example:

```
while (1) {
    /* never yields */
}
```

In a purely cooperative system, this loop can monopolize the processor. The scheduler may never regain control unless the program performs a blocking operation, explicitly yields, or is interrupted by some external mechanism not part of the cooperative model.

This is why cooperative multitasking requires discipline from programs. Each task must be written so that it frequently reaches safe suspension points. This may be acceptable in controlled runtimes, embedded environments, event loops, or older graphical systems, but it is fragile as a general operating-system model.

In **preemptive multitasking**, the operating system kernel can interrupt the running process without asking its permission. The running process does not have to call `yield()` or voluntarily block. A hardware timer periodically transfers control to the kernel. The kernel scheduler then decides whether the current process should continue or whether another process or thread should run.

A simplified preemptive timeline looks like this:

```
Process A runs
|
| timer interrupt
v
kernel scheduler runs
|
v
Process B runs
|
| timer interrupt
v
kernel scheduler runs
|
v
Process C runs
```

The important difference is that the currently running process does not control the switching point. The kernel does. This gives the operating system stronger authority over CPU sharing. A process may contain an infinite loop, but the timer interrupt still allows the kernel to regain control and schedule other processes.

For example:

```
while (1) {
    /* CPU-bound loop */
}
```

In a preemptive system, this loop may consume a lot of CPU time, but it should not prevent the kernel from running other processes. The scheduler can interrupt it after a time slice and allow other ready processes to execute.

Preemption depends on hardware support. The processor must be able to receive interrupts. A timer device must generate periodic interrupts. The processor must be able to switch from user mode to kernel mode safely. The kernel must be able to save the current execution context, choose another one, and restore that other context.

The saved context includes the information needed to resume execution later:

- the instruction pointer;
- the stack pointer;

- general-purpose register values;
- processor status flags;
- memory mapping identity;
- scheduling metadata.

When the interrupted process is scheduled again, the kernel restores this state. The process continues from the instruction where it was interrupted, or from the next valid instruction according to the architecture's interrupt-return rules. From the process's point of view, it usually appears as if execution simply became slower for a while.

Preemptive multitasking is the dominant model in modern general-purpose operating systems because it provides stronger isolation and fairness. The kernel does not need to trust every user program to behave politely. It can enforce scheduling policy externally.

However, preemption introduces complexity. A process or thread can be interrupted almost anywhere. If several threads share memory, one thread may be preempted while it is halfway through modifying shared data. Another thread may then run and observe inconsistent state unless synchronization mechanisms are used. This is why preemptive multithreaded programming requires locks, mutexes, semaphores, atomic operations, or other coordination tools.

For example, suppose two threads update a shared counter:

```
counter = counter + 1;
```

At source level, this looks like one operation. At machine level, it may involve several steps:

```
load counter
add 1
store counter
```

If one thread is preempted between these steps and another thread modifies the same counter, the final result may be wrong. This is a race condition. Preemption makes such problems possible because execution can be interrupted between low-level operations.

Cooperative systems have different risks. Since tasks yield only at known points, there is less danger of being interrupted in the middle of a critical sequence, as long as the sequence does not explicitly yield. But cooperative systems can freeze if one task fails to yield. Therefore, the two models exchange one kind of difficulty for another:

- cooperative multitasking gives simpler switching points but depends on voluntary yielding;
- preemptive multitasking gives stronger scheduling control but requires careful synchronization.

The distinction can be summarized as follows:

Property	Cooperative multitasking	Preemptive multitasking
Switching control	Running task voluntarily yields	Kernel can interrupt running task
Main trigger	Yield, blocking call, explicit suspension point	Timer interrupt, priority decision, blocking event
Failure mode	One task can monopolize execution by not yielding	Shared data can be interrupted mid-update
Synchronization difficulty	Lower at non-yielding regions	Higher because interruption can happen almost anywhere
Typical use	Event loops, coroutines, older systems, controlled runtimes	Modern desktop/server operating systems

Modern operating systems usually use preemptive multitasking for native processes and threads. The kernel scheduler can interrupt a process after a time slice, handle interrupts, wake blocked processes, enforce priorities, and balance work across CPU cores. This gives the operating system central control over hardware resource sharing.

However, cooperative multitasking has not disappeared. It reappears inside high-level programming runtimes. Generators, coroutines, event loops, and asynchronous programming systems often use cooperative scheduling. A coroutine runs until it reaches an explicit suspension point, such as `yield` or `await`. Then another coroutine may run. The runtime scheduler does not usually interrupt it at arbitrary machine instructions.

For example, in Python asynchronous code, a coroutine gives up control when it awaits an awaitable operation:

```
await some_io_operation()
```

This is conceptually cooperative. The coroutine voluntarily reaches a suspension point. The event loop can then run other tasks while the awaited operation is incomplete. If a coroutine performs a long CPU-bound loop without awaiting, it can block the event loop in the same way that a non-yielding task blocks a cooperative multitasking system.

This is the important bridge from operating systems to language runtimes. At the operating-system level, modern systems use preemption to control native processes and threads. At the runtime level, languages often use cooperative scheduling to manage lightweight tasks inside one process.

The same Python program can therefore be subject to both models at different layers:

- the CPython process is scheduled preemptively by the operating system;
- Python coroutines inside that process are scheduled cooperatively by the event loop.

This layered view prevents confusion. When a Python coroutine uses `await`, it is not asking the operating system scheduler to preempt the process. It is giving control back to the Python event loop. The operating system may still preempt the entire CPython process independently at any time, because the process remains a normal native process.

Thus, cooperative and preemptive multitasking describe two different control policies for sharing execution time. Cooperative systems rely on voluntary suspension points. Preemptive systems allow the kernel to interrupt execution and enforce scheduling externally. Modern programming uses both: preemptive scheduling at the operating-system level and cooperative scheduling inside many high-level runtime systems.

The CPU Kernel Scheduler, Symmetrical Multiprocessing, and Context Switching Latency Overhead

The operating system scheduler is the kernel component that decides which execution context receives CPU time. A modern system may contain many runnable processes and threads, but the number of hardware execution units is limited. The scheduler must continuously choose which process or thread should run now, which ones must wait, which ones are blocked, and which ones should be resumed when an event completes.

A **scheduler** is therefore not a vague background mechanism. It is a concrete kernel subsystem that manages the queue of runnable execution contexts and assigns them to available CPU cores. Each process or thread has scheduling metadata: state, priority, recent CPU usage, waiting time, affinity rules, and other implementation-specific information. The scheduler uses this information to make execution decisions.

A simplified scheduling situation may look like this:

```
ready queue:
  Thread A
  Thread B
```

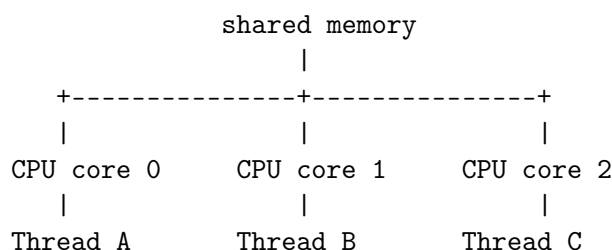

Thread C
Thread D

available CPU core:
runs one selected thread

If the machine has one CPU core, only one thread can execute at a given exact moment. The scheduler creates the appearance of concurrent execution by rapidly switching among runnable threads. If the machine has multiple cores, several threads can execute physically in parallel, but the scheduler still decides which runnable threads occupy which cores.

This brings us to **symmetric multiprocessing**, usually abbreviated as **SMP**. In an SMP system, multiple CPU cores or processors are available under one operating system image, and each core can run kernel code and user code. The cores share access to the same physical memory system, although modern hardware may also contain private caches, shared caches, memory-controller topology, and non-uniform access costs.

In a simplified SMP model:



Here, several threads can actually execute at the same time. This is real parallelism, not only time-sliced concurrency. If three CPU cores run three different threads simultaneously, three instruction streams are physically advancing at the same time.

However, SMP does not remove the need for scheduling. It makes scheduling more complex. The scheduler must choose not only *when* a thread runs, but also *where* it runs. It may need to decide whether a thread should stay on the same core where its data may still be warm in cache, or move to another core that is idle. It may need to balance the system so that one core is not overloaded while another is idle.

This creates a tension between **load balancing** and **cache locality**. Load balancing means spreading work across available cores. Cache locality means keeping a thread near the CPU cache state it has already built. Moving a thread from one core to another may make the system more balanced, but it can also hurt performance because the new core may not have the same data in its cache.

The scheduler must also handle threads with different states. A thread that is waiting for disk input, network input, a timer, a lock, or a child process is not runnable. It should not receive a CPU core until the event it is waiting for becomes ready. A thread that has all required data and is ready to compute can be placed in a run queue.

A simplified thread-state transition looks like:

```

ready  ----scheduler chooses----> running
running ----time slice expires--> ready
running ----waits for I/O-----> blocked
blocked ----I/O completes-----> ready
running ----finishes-----> terminated

```

The scheduler is involved every time a runnable execution context must be chosen. It may run because a timer interrupt occurred, because a thread blocked, because a blocked thread became ready, because a higher-priority thread appeared, or because a process voluntarily yielded.

When the scheduler stops one thread and starts another, the operating system performs a **context switch**. A context switch is the act of saving the execution state of the current thread and restoring

the execution state of another thread. The saved state must be sufficient for the old thread to continue later as if it had merely paused.

The context of a thread includes information such as:

- the instruction pointer;
- the stack pointer;
- general-purpose registers;
- processor flags;
- floating-point or vector register state when needed;
- kernel scheduling metadata;
- memory mapping identity, especially when switching between processes.

Switching between two threads in the same process may be cheaper than switching between two different processes because threads in the same process share the same virtual address space. Switching between processes may require changing the active address-space mapping, which can affect page-table-related processor state and memory translation caches.

This is where the memory management unit matters again. The processor uses translation structures to convert virtual addresses into physical addresses. To make this fast, processors use caches for address translations, commonly called translation lookaside buffers, or TLBs. A process switch may require changing the active page-table root or address-space identifier. Depending on the architecture and operating-system strategy, this can invalidate or reduce the usefulness of cached translations.

Therefore, a context switch is not free. It has **latency overhead**. During the switch itself, the CPU is executing kernel scheduling and bookkeeping code rather than advancing the user program's useful work. The kernel must save registers, update scheduler structures, choose the next thread, restore state, and return to user mode. Cache and TLB effects may also slow down the newly scheduled thread after the switch.

A simplified view is:

```
Thread A useful work
    |
    v
kernel saves Thread A context
kernel chooses Thread B
kernel restores Thread B context
    |
    v
Thread B useful work
```

The time spent in the middle is scheduling overhead. It is necessary overhead, because without it the operating system could not share the CPU safely. But it is still overhead. A system that switches too often may spend too much time managing execution and not enough time doing useful computation.

This is why scheduler design must balance responsiveness and throughput. Small time slices can make a system feel responsive because interactive programs get frequent chances to run. But very small time slices can increase context-switch overhead. Larger time slices reduce switching overhead but may make interactive tasks wait longer.

For example:

```
very small slices:
    responsive, but many switches

very large slices:
    fewer switches, but worse responsiveness
```

Schedulers also use priorities. A system may prefer interactive tasks, real-time tasks, I/O-bound tasks, or CPU-bound tasks depending on policy. A process waiting for keyboard input should often respond quickly when input arrives. A long numerical computation may be allowed to run in larger chunks. A real-time audio thread may need stricter scheduling guarantees to prevent audible glitches.

The scheduler must also interact with blocking operations. If a thread performs a blocking read from disk or network, it should not sit on a CPU core doing nothing. The kernel marks it as blocked and schedules another runnable thread. When the disk or network operation completes, an interrupt or kernel event marks the blocked thread as ready again. It can then compete for CPU time.

This is one of the major reasons multitasking improves system efficiency. The scheduler can run CPU-ready work while other tasks wait for slow external resources.

In SMP systems, context switching can occur independently on several cores. Each core may run its own current thread. The scheduler may maintain per-core run queues, global balancing mechanisms, or hybrid structures. The exact implementation is complex and operating-system-specific, but the conceptual goal is stable: keep available cores doing useful work while respecting priorities, fairness, locality, and synchronization constraints.

Parallel execution also introduces synchronization costs. If two threads on different cores access shared memory, they may need locks or atomic operations. These mechanisms can force coordination between cores. A lock can cause one thread to block while another holds the protected resource. Atomic operations can require cache coherence traffic. Therefore, using many cores does not automatically make a program faster. The program must have work that can truly proceed in parallel without excessive coordination.

This distinction is important for C and Python programmers alike. In C, native threads can often run on multiple cores at the same time, provided the program uses operating-system threading APIs or libraries and manages synchronization correctly. The operating system scheduler maps those threads to CPU cores.

In CPython, the situation has another layer. The CPython interpreter process is scheduled by the operating system like any other process. Its native threads are also visible to the operating system. However, in the traditional CPython execution model, the Global Interpreter Lock, or GIL, prevents multiple threads from executing Python bytecode at the same time inside one interpreter process. A thread may still run native code, wait for I/O, or release the GIL in certain extension modules, but ordinary CPU-bound Python bytecode execution is constrained by the interpreter's locking model.

This means two scheduling systems can be involved:

- the operating system scheduler assigns native threads to CPU cores;
- the CPython runtime controls which thread may execute Python bytecode through interpreter-level locking.

A Python program using threads is therefore not automatically parallel in the same way as a C program using native threads for CPU-bound work. The operating system may schedule several CPython threads, but the interpreter may allow only one of them to execute Python bytecode at a time. For I/O-bound work, Python threads can still be useful because one thread may wait while another progresses. For CPU-bound parallelism, Python often uses multiprocessing, native extensions, vectorized libraries, or implementations without the same locking behavior.

This layered model is the key point. The operating system scheduler manages hardware execution contexts. It knows processes, native threads, CPU cores, priorities, and blocking states. A language runtime may add additional scheduling or locking rules on top of that. The two layers are related, but they are not the same.

Thus, the CPU kernel scheduler is the authority that distributes processor time across runnable native execution contexts. Symmetric multiprocessing allows several of those contexts to execute physically in parallel on different cores. Context switching makes sharing possible, but it costs time through saved state, restored state, cache disruption, and memory-translation effects. Understanding this cost is essential before discussing higher-level runtime scheduling, because generators, coroutines, async tasks, and interpreter locks all exist above the lower operating-system layer that ultimately decides when the process itself receives CPU time.

Structural Differentiation: Heavyweight OS Processes (MMU Isolation) vs. Lightweight Native Threads (Shared Memory Spaces)

Operating systems do not schedule only whole programs. Modern systems usually schedule smaller execution units called **threads**. A process may contain one thread or many threads. This distinction is essential because processes and threads share some similarities, but they have very different memory and resource boundaries.

A **process** is a heavyweight operating-system execution container. It has its own virtual address space, its own memory mappings, its own process identifier, its own resource table, and its own protection boundary. The memory management unit, or MMU, helps enforce this boundary by translating virtual addresses differently for different processes. This is why one process cannot normally read or write another process's memory just by using a pointer.

A **thread**, by contrast, is a lighter execution path inside a process. Threads inside the same process share the process address space. They can execute different instruction streams, have separate stacks, and maintain separate register states, but they see the same heap, global variables, loaded libraries, and open resources belonging to the process.

The difference can be represented simply:

Process A

```
virtual address space A
heap A
globals A
open files A
thread A1 -> stack A1, registers A1
thread A2 -> stack A2, registers A2
```

Process B

```
virtual address space B
heap B
globals B
open files B
thread B1 -> stack B1, registers B1
```

Threads A1 and A2 share memory because they belong to the same process. Thread B1 does not share memory with them in the ordinary direct-addressing sense, because it belongs to a different process.

This means that processes provide stronger isolation. If Process A has a pointer value such as:

```
0x100000
```

and Process B has the same pointer value, those two values do not necessarily refer to the same physical memory. Each process has its own virtual address space, and the MMU translates those virtual addresses according to the active process page tables.

Inside one process, however, two threads share the same virtual address space. If Thread A1 and Thread A2 both use the address 0x100000, they are referring to the same virtual memory location inside Process A. If that address belongs to a heap object, both threads can access the same heap object. This is powerful, but dangerous.

A process contains at least one thread. That first thread begins executing when the process starts. Additional threads can be created later by the program or runtime. Each thread has its own execution state:

- its own instruction pointer;
- its own stack pointer;
- its own CPU register values;

- its own stack;
- its own scheduling state.

But threads in the same process share:

- the same virtual address space;
- the same heap;
- the same global and static storage;
- the same loaded executable and shared library code;
- the same open file descriptors or handles;
- many of the same process-level permissions and resources.

This shared-memory structure makes threads lighter than processes. Creating a new process requires creating or duplicating a process address space, setting up memory mappings, process tables, handles, and isolation structures. Creating a new thread inside an existing process requires less work because the address space and many resources already exist. The new thread mainly needs its own stack, register context, and scheduler metadata.

This is why processes are often called heavyweight and threads lightweight. The terms are relative. A thread is still an operating-system object and still has real scheduling cost. But compared with a full process, a thread usually has lower creation cost, lower memory overhead, and faster communication with other threads in the same process.

Communication between separate processes usually requires explicit inter-process communication mechanisms. Examples include:

- pipes;
- sockets;
- shared memory segments;
- files;
- message queues;
- signals;
- operating-system-specific IPC mechanisms.

The reason is simple: normal process memory is isolated. Process A cannot simply dereference a pointer to Process B's heap. If two processes need to exchange information, the kernel must mediate the communication or explicitly create a shared region.

Threads do not need such mechanisms to access shared data. If two threads belong to the same process, they can both access the same global variable or heap object directly:

```
int counter = 0;

void *worker(void *arg) {
    counter++;
    return NULL;
}
```

If two native threads execute `worker()`, both can modify the same `counter`. No serialization or message passing is required just to reach the variable. But this direct access introduces race conditions.

The source statement

```
counter++;
```

looks like one operation, but at machine level it may require several steps:

```
load counter
add 1
store counter
```

If two threads interleave these steps, one increment may be lost. For example:

```
Thread 1 reads counter = 0
Thread 2 reads counter = 0
Thread 1 stores counter = 1
Thread 2 stores counter = 1
```

The intended result after two increments was 2, but the actual result is 1. This is a race condition caused by shared mutable state.

Therefore, threads are lightweight partly because they share memory, but this same sharing makes them harder to reason about. Programs using threads must protect shared mutable data with synchronization mechanisms such as:

- mutexes;
- locks;
- semaphores;
- condition variables;
- atomic operations;
- barriers;
- thread-safe queues.

Processes avoid many of these accidental sharing problems by default because their memory is isolated. One process cannot corrupt another process's heap through an ordinary pointer bug. If a process crashes, the operating system can usually destroy that process and reclaim its resources without destroying unrelated processes. This makes process isolation valuable for reliability and security.

However, process isolation also makes communication more expensive. If two processes need to exchange large amounts of data, the data may need to be copied through kernel buffers, serialized, sent through a socket, written to a pipe, or placed in an explicitly shared memory region. Threads can share data cheaply because it is already in the same address space.

The contrast is therefore not simply “processes are better” or “threads are better.” They solve different problems:

- processes provide isolation, protection, and failure containment;
- threads provide cheaper concurrent execution paths inside one shared memory space.

A browser is a useful example. Modern browsers often use multiple processes for isolation between tabs, extensions, renderers, and privileged components. If one tab crashes, the whole browser does not necessarily die. At the same time, each process may use multiple threads internally for rendering, networking, JavaScript execution, disk cache work, and user-interface responsiveness.

In a native C or C++ program, threads are often used when multiple execution paths need fast shared access to the same in-memory data structures. For example, a server may create multiple worker threads that share a cache. A numerical program may divide a large computation across several worker

threads that operate on different parts of a shared array. In these cases, threads avoid the heavier boundary of process isolation.

But the programmer must then handle synchronization correctly. A pointer bug, buffer overflow, or data race in one thread can corrupt the entire process, because all threads share the same address space. There is no MMU boundary between threads of the same process. The MMU isolates processes from one another, not threads from one another inside the same process.

The scheduling model also differs slightly. The operating system scheduler may schedule threads rather than whole processes. If a process contains several runnable threads, those threads may run on different CPU cores at the same time. This means one process can have real parallel execution internally, if the operating system and hardware allow it.

A simplified SMP situation may look like this:

Process A

```
Thread A1 running on CPU core 0
Thread A2 running on CPU core 1
Thread A3 waiting for disk I/O
```

Here, Process A is not merely receiving one time slice. Two of its threads are physically executing in parallel. They share the same heap and globals, so synchronization is required if they touch the same mutable objects.

Context switching between threads of the same process may be cheaper than switching between processes because the virtual address space does not need to change. The page tables may remain the same. Translation caches may be less disturbed. However, the kernel still must save and restore register state, stack pointer, instruction pointer, and scheduler metadata. A thread switch is cheaper than a process switch in many cases, but it is not free.

The difference between processes and threads also clarifies Python's execution model. When a Python script is launched, the operating system creates a process for the CPython interpreter. Inside that process, CPython may create or manage native threads if the Python program uses the `threading` module or if libraries create threads internally.

Those Python-level threads are backed by native operating-system threads in standard CPython. They share the same process address space. They also share the same CPython heap, where Python objects live. Therefore, two Python threads can refer to the same list, dictionary, file object, or user-defined object.

For example:

```
shared_list = []

def worker():
    shared_list.append(1)
```

If multiple Python threads call `worker()`, they are all operating on the same Python list object. That list exists inside the CPython process heap. The operating system does not isolate it per thread.

However, CPython adds another rule above the operating-system thread model: the Global Interpreter Lock, or GIL, in the traditional CPython architecture. The GIL prevents multiple native threads from executing Python bytecode simultaneously inside one interpreter process. This means Python threads are native threads, but ordinary Python bytecode execution is still constrained by interpreter-level locking.

This produces a layered model:

- the operating system sees a CPython process and its native threads;
- those threads share the same process memory space;
- CPython uses internal locking to protect interpreter state;
- Python objects are shared by reference inside the interpreter heap;

- CPU-bound Python bytecode does not automatically scale across cores in traditional CPython.

This is different from using multiple Python processes through the `multiprocessing` module. Multiple processes do not share the same ordinary Python object heap. Each process has its own interpreter instance and its own address space. Communication must happen through pipes, queues, shared memory, sockets, files, or other mechanisms. The benefit is stronger isolation and the ability to use multiple CPU cores for Python execution without the same single-interpreter GIL bottleneck.

Thus, the distinction between heavyweight processes and lightweight threads is a structural memory distinction, not just a naming convention. A process is a protected address-space container enforced by the operating system and MMU. A thread is an execution path inside that container, with its own stack and registers but shared process memory. Processes are safer and more isolated. Threads are cheaper and more direct, but shared-memory concurrency introduces race conditions, synchronization costs, and failure coupling.

For students moving from C to Python, this distinction is essential. C exposes the raw power and danger of shared-memory threading directly. Python hides many low-level details behind runtime objects and libraries, but it does not remove the underlying operating-system model. Python threads still live inside a process. Python processes are still isolated by the operating system. Understanding the difference explains why threads, processes, coroutines, and async tasks are not interchangeable execution mechanisms.

From OS-Level Scheduling to Runtime-Level Scheduling

Operating-system scheduling and runtime-level scheduling solve similar coordination problems at different layers. The operating system scheduler decides which native process or native thread receives CPU time. A language runtime scheduler, when present, decides which language-level task, coroutine, fiber, generator, green thread, or event-loop callback should run inside an already existing process.

The first layer belongs to the kernel. The kernel controls hardware resources. It can interrupt a running process, save its CPU state, restore another execution context, and assign work to CPU cores. This is operating-system-level scheduling. It manages native execution contexts that the processor can actually run.

The second layer belongs to the runtime. A runtime such as CPython, a JavaScript host environment, the JVM, a green-thread system, or an async event loop may create smaller execution units inside one process. These units are not always known to the operating system as independently schedulable threads. They may exist only as runtime-managed objects: suspended frames, callbacks, coroutine objects, generator objects, promises, futures, or task descriptors.

This gives us two different scheduling layers:

```
hardware CPU core
  ^
  |
operating system scheduler
  schedules native processes and native threads
  ^
  |
language/runtime scheduler
  schedules coroutines, generators, callbacks, tasks
  ^
  |
user program logic
```

The operating system scheduler answers the question:

Which native execution context should the CPU run now?

The runtime scheduler answers a different question:

Inside this already running process, which runtime-level unit of work should continue now?

This distinction is essential. A Python coroutine, for example, is not automatically a separate operating-system thread. It is a Python runtime object representing suspended execution. The operating system does not normally inspect Python coroutine objects and choose between them. The operating system schedules the CPython process or one of its native threads. When CPython receives CPU time, Python's own runtime mechanisms decide which coroutine or callback should advance.

The reason runtime-level scheduling exists is similar to the reason operating-system scheduling exists: many programs spend time waiting. A program may wait for disk input, network data, user input, timers, database responses, or other slow external events. If one execution path waits, another execution path may be able to make progress. At the operating-system level, the kernel can run another process while one process is blocked. At the runtime level, an event loop can run another coroutine while one coroutine waits for I/O.

A simplified operating-system-level waiting pattern looks like this:

```
Process A starts disk read
Process A blocks
kernel schedules Process B
disk read completes
kernel marks Process A ready
Process A later resumes
```

A simplified runtime-level waiting pattern looks like this:

```
Coroutine A starts async read
Coroutine A awaits
event loop schedules Coroutine B
read operation completes
event loop marks Coroutine A ready
Coroutine A later resumes
```

The similarity is real, but the mechanism is not the same. In the first case, the kernel changes which native process or thread runs on the CPU. In the second case, the runtime changes which coroutine or task continues inside the same process.

This is why runtime-level scheduling is often called **cooperative**. A coroutine normally runs until it reaches an explicit suspension point, such as `await`, `yield`, or another runtime-defined pause operation. The runtime does not usually interrupt it at arbitrary bytecode instructions in the same way the operating system may preempt a native thread. The coroutine must reach a point where it gives control back.

For example, in Python:

```
async def fetch_data():
    data = await read_from_network()
    return data
```

The `await` expression is a suspension point. When execution reaches it, the coroutine can stop temporarily. The event loop can then run some other ready task. When the awaited operation completes, the coroutine can resume from the same logical point.

This resembles operating-system multitasking, but it is not hardware preemption. The operating system does not know that `fetch_data()` reached an `await`. The event loop knows. The Python runtime knows. The kernel only sees native threads and system calls.

Generators show an even smaller form of runtime-controlled suspension:

```
def numbers():
    yield 1
    yield 2
    yield 3
```

Each `yield` suspends the function's execution state. The generator does not finish and destroy all local state after the first value. Instead, it keeps enough state to resume later. This is a language-runtime behavior, not an operating-system process switch.

A generator object therefore stores a suspended execution position. It can be resumed by calling `next()`:

```
g = numbers()

next(g)  # resumes until first yield
next(g)  # resumes until second yield
next(g)  # resumes until third yield
```

At no point does the operating system create a separate process for each yielded value. The entire generator mechanism lives inside the Python runtime.

Runtime-level scheduling is useful because it avoids some costs of operating-system scheduling. Creating thousands of native threads can be expensive. Each native thread needs a stack, kernel scheduling metadata, and synchronization with the operating-system scheduler. Switching between native threads has context-switch overhead. If a program has many tasks that spend most of their time waiting for I/O, representing each one as a full native thread may be wasteful.

A runtime task can be much cheaper. A coroutine or generator may require only a runtime object containing its suspended state, local variables, instruction position, and links to awaited operations. The runtime can keep thousands or millions of such tasks more cheaply than the operating system can schedule the same number of native threads, depending on the implementation and workload.

However, runtime-level scheduling has a major limitation: it does not automatically create CPU parallelism. If all tasks run inside one native thread, only one task executes at a time. The runtime may interleave them, but it does not make them physically execute on multiple CPU cores. It improves waiting behavior, not raw CPU execution throughput.

This distinction can be summarized as:

- **operating-system scheduling:** assigns native execution contexts to hardware CPU time;
- **runtime scheduling:** assigns language-level tasks to execution opportunities inside a process;
- **preemptive kernel scheduling:** can interrupt native threads without their cooperation;
- **cooperative runtime scheduling:** usually requires explicit suspension points;
- **parallelism:** requires multiple hardware execution contexts actually running at the same time;
- **concurrency:** can exist through interleaving, even on one hardware thread.

This is why an `async` program can handle many network connections without using one native thread per connection, but it does not automatically make CPU-heavy computation faster. If a coroutine performs a long CPU-bound loop without reaching an `await`, it blocks the event loop:

```
async def bad_task():
    while True:
        pass
```

This coroutine does not give control back to the runtime scheduler. Even though the program is written using `async`, this task can prevent other coroutines from advancing. This is the cooperative multitasking failure mode reappearing inside a high-level runtime.

The operating system may still preempt the entire CPython process and run another process, but that does not help other coroutines inside the same blocked event loop. From the operating system's point of view, the CPython process is just executing CPU instructions. From the Python runtime's point of view, one coroutine is failing to cooperate.

This layered scheduling model is especially important in Python because several execution mechanisms coexist:

- **processes**, managed by the operating system and isolated by virtual memory;
- **native threads**, managed by the operating system but sharing one process address space;
- **Python threads**, backed by native threads in CPython, but constrained by interpreter-level rules such as the Global Interpreter Lock in traditional CPython;
- **generators**, resumed manually through iteration;
- **coroutines and async tasks**, scheduled cooperatively by an event loop;
- **callbacks**, invoked by runtime or framework dispatch rules when events occur.

These mechanisms should not be treated as interchangeable. They live at different layers and have different memory, scheduling, and parallelism properties.

For example, multiple processes provide operating-system isolation and can run on multiple cores, but they do not share ordinary memory directly. Multiple native threads share memory and may run on multiple cores, but they require synchronization. Multiple Python coroutines can be very lightweight and good for I/O interleaving, but they normally run cooperatively inside one event loop and do not by themselves provide CPU parallelism.

A useful comparison is:

Mechanism	Scheduled by	Memory relation	Typical use
Process	Operating system	Isolated address space	Isolation, CPU parallelism, independent programs
Native thread	Operating system	Shared process memory	Parallel or concurrent work inside one process
Coroutine / async task	Runtime event loop	Shared runtime memory	I/O concurrency with low overhead
Generator	Caller / iterator protocol	Suspended runtime frame	Lazy value production
Callback	Runtime / framework dispatcher	Shared runtime state	Event-driven execution hooks

The transition from OS-level scheduling to runtime-level scheduling is therefore not a replacement of one system by another. It is a layering. The runtime scheduler cannot run unless the operating system scheduler gives CPU time to the process. The event loop cannot advance coroutines unless the process is running. A generator cannot produce the next value unless the interpreter is executing. At the bottom, hardware execution is still controlled by the kernel scheduler.

At the same time, runtime-level scheduling gives the programming language more control over how logical tasks are structured. The language can represent suspended computations as objects. It can store local state on the heap. It can resume a computation from a previous bytecode position. It can dispatch callbacks when events occur. It can let a single native thread manage many logical tasks.

This is one of the reasons interpreted and runtime-managed languages can provide advanced control-flow abstractions more naturally than bare C. C can implement similar systems through libraries, state machines, callbacks, function pointers, operating-system threads, or manually managed stacks. But the language itself does not normally preserve and resume arbitrary function frames in the same way Python generators or coroutines do.

Python's runtime is able to provide these abstractions because execution is mediated by bytecode, frame objects or frame-like internal structures, object references, and interpreter-managed state. A Python generator can suspend because the runtime knows how to preserve the generator's execution state. A Python coroutine can await because the runtime and event loop know how to register the suspended task and resume it later.

Thus, operating-system scheduling explains how native execution contexts share hardware. Runtime-level scheduling explains how high-level language tasks share execution opportunities inside a process. The two mechanisms are connected but distinct. The operating system schedules the interpreter process and its native threads. The interpreter and its libraries may then schedule generators, coroutines, callbacks, and async tasks inside that process. Understanding this boundary is necessary before discussing Python’s advanced procedural extensions, because those extensions are runtime scheduling mechanisms built above the lower operating-system scheduler.

2.2 Process Virtual Machines and Interpreted Runtimes

The previous section followed the native execution path: a C source program is transformed into a binary executable, the operating system loads that executable into a process, and the processor executes machine instructions from the process text segment. That model is essential, but it is not the model normally used when a Python file is executed. Python introduces another execution layer between the user’s source code and the physical machine.

This section explains that additional layer. When we run a Python file, the operating system does not normally load the `.py` file as a native executable. Instead, it loads the CPython interpreter executable. The user’s Python source file becomes runtime input. CPython reads it, parses it, compiles it into bytecode, creates runtime execution structures, and executes that bytecode inside the interpreter process. The native process belongs to CPython; the Python program lives inside that process as bytecode, frames, objects, namespaces, and managed memory.

This architecture is an example of a **process virtual machine**. A process virtual machine is not a full computer emulator. It does not replace the operating system or boot a guest kernel. It is a software execution environment running inside an ordinary process. It defines its own virtual instruction set, its own frame model, its own object model, its own memory-management rules, and its own control-flow mechanisms. The physical CPU still executes native instructions, but those instructions belong mostly to the runtime engine.

The central representation in many such runtimes is **bytecode**: an instruction stream for a software machine rather than for the physical processor. CPython bytecode is executed by the CPython evaluation loop. JVM bytecode is loaded, verified, interpreted, and often JIT-compiled by the Java Virtual Machine. PHP source is compiled into Zend Engine opcodes. V8 uses internal bytecode-like representations and aggressively optimizes hot JavaScript paths into native code. Bash provides a contrasting model, where the shell interprets command syntax and often launches external programs as child processes.

The goal of this section is to make the boundary precise. Terms such as *compiled*, *interpreted*, *virtual machine*, and *runtime* are often used too loosely. Python is not simply “source code executed line by line.” CPython compiles source to bytecode and interprets that bytecode. Java is not directly executed from source; it usually runs from JVM bytecode. V8 is not merely an interpreter; it is an adaptive engine with JIT compilation. Bash is closer to command-by-command interpretation, but many commands become separate native processes.

By comparing these runtime systems, we can identify what is specific to Python and what belongs to a broader family of runtime-hosted execution models. This prepares the later chapters: Python variables are bindings to runtime objects, function calls create interpreter-managed frames, exceptions move through runtime-managed control paths, and advanced mechanisms such as generators, coroutines, and async execution exist because the runtime owns more execution state than a bare native process would expose directly.

2.2.1 From Native Executables to Runtime-Hosted Programs

What Actually Runs When We Run a Python File?

When a user writes and executes a command such as:

```
python script.py
```

it is tempting to say that the operating system runs the file `script.py`. This is not accurate. The operating system does not normally execute the Python source file directly. The file `script.py` is not a native executable binary in the same sense as a compiled C program. It does not contain machine instructions arranged in an ELF or PE/COFF executable format. It does not have a native text segment, a native executable entry point, or direct processor instructions produced from the user's source code.

What the operating system actually runs is the Python interpreter executable. On Linux or other Unix-like systems, this executable is often called `python`, `python3`, or something similar. On Windows, it is commonly `python.exe`. This interpreter executable is a native program. It has been compiled, assembled, linked, and stored in an executable format that the operating system loader understands.

Therefore, when the command is launched, the first native process created by the operating system is not a process whose executable image is `script.py`. It is a process whose executable image is the Python interpreter:

```
operating system
  loads native executable: python / python.exe
    |
    v
CPython interpreter process
  receives script.py as an argument
    |
    v
CPython opens, parses, compiles, and executes script.py
```

The Python file is input to the interpreter process. It is closer to data consumed by a runtime engine than to a native binary image loaded directly by the kernel. The interpreter process receives the file path `script.py` through the normal command-line argument mechanism. Once the CPython executable has started, it examines its arguments, finds the script name, opens the source file, reads the Python code, compiles it to an internal bytecode representation, and executes that bytecode inside the interpreter runtime.

This creates two distinct execution layers:

- the **native operating-system layer**, where the executable `python` or `python.exe` is loaded as a normal process;
- the **Python runtime layer**, where `script.py` is treated as source code to be compiled and executed by CPython.

At the native layer, the CPU executes machine instructions belonging to the interpreter executable and its native libraries. These instructions are stored in the text segment of the CPython process. The operating system scheduler gives CPU time to this process or to its native threads. The kernel sees it as an ordinary native process with virtual memory, mapped libraries, file descriptors or handles, stack, heap, and scheduling state.

At the Python layer, the user's program is represented by Python objects, code objects, bytecode instructions, module namespaces, function frames, and managed heap allocations. These do not exist as independent operating-system processes. They exist inside the CPython process.

This distinction is the main architectural difference between running a compiled C program and running a Python source file. In the ordinary C model, the user's source code is transformed into the native executable itself:

```
hello.c
  |
  | compile, assemble, link
  v
hello executable
```

```

    |
    | OS loader
    v
running process

```

In the ordinary Python model, the interpreter is already the native executable:

```

script.py
    |
    | input to CPython
    v
Python bytecode and runtime objects
    |
    | executed by CPython interpreter process
    v
behavior produced by the running interpreter

```

A more complete Python execution chain looks like this:

```

python executable
    |
    | OS loader maps native interpreter process
    v
CPython process starts
    |
    | reads script.py
    v
source text
    |
    | tokenizer and parser
    v
AST
    |
    | compiler
    v
code object / bytecode
    |
    | evaluation loop
    v
Python-level execution

```

The physical processor does not understand Python bytecode directly. Python bytecode is not x86-64, ARM64, or another hardware instruction set. It is an instruction stream for the CPython virtual machine. The processor executes the native machine instructions of CPython. CPython's interpreter loop then reads Python bytecode instructions and performs the corresponding runtime operations.

For example, a Python statement such as:

```
x = a + b
```

does not become a native instruction sequence in the usual CPython execution model. Instead, CPython compiles it into bytecode operations that load the objects bound to **a** and **b**, perform Python's addition protocol, and bind the result to the name **x**. The interpreter loop executes these bytecode instructions by running native C code inside the CPython process.

This is why saying that Python is simply “not compiled” is misleading. CPython does compile Python source code, but not usually into native machine code. It compiles source code into bytecode for its own virtual machine. That bytecode is then interpreted. Therefore, in CPython, Python execution contains both compilation and interpretation:

Python source code

- > compiled to Python bytecode
- > interpreted by the CPython virtual machine

The result is very different from traditional C execution:

C source code

- > compiled to native machine code
- > loaded and executed directly as a process

This also explains why Python can be interactive. In an interactive Python session, the interpreter process is already running. Each line or block entered by the user is read, parsed, compiled, and executed inside the existing process. The operating system is not creating a new native process for every expression typed at the prompt. The interpreter process remains alive and repeatedly processes new pieces of Python source code.

For example:

```
>>> x = 10
>>> x + 5
15
```

The name `x` persists because it is stored in the interpreter's current namespace, not because a native executable was rebuilt. The runtime environment remains active between inputs. This behavior would be unnatural in the traditional C compilation model, where source is normally compiled and linked before execution.

The same explanation applies to notebooks. In a Jupyter notebook, the visible cells are not separate native programs. A Python kernel process is running in the background. Each executed cell is sent to that kernel, compiled, and executed in the kernel's runtime state. Variables remain available across cells because the same interpreter process and namespace continue to exist.

This means that Python execution is process-hosted rather than directly source-file-hosted. The host process is CPython. The Python source file, interactive input, or notebook cell becomes material processed by that host process.

There are tools that can package Python applications into executable-looking files, and there are alternative implementations or compilers that change parts of this picture. However, the standard teaching model for CPython is clear: the operating system runs the interpreter executable; the interpreter runs the Python program.

This distinction will be used throughout the rest of the course. When we later discuss Python objects, variables, functions, modules, bytecode, frames, garbage collection, generators, coroutines, and exceptions, all of these are runtime structures inside the interpreter process. They are not separate operating-system processes, and they are not direct native-machine structures in the same way C stack variables or compiled function instructions are.

Thus, the answer to the question “what actually runs when we run a Python file?” is layered. At the operating-system level, the CPython interpreter executable runs. At the Python-language level, the user's source file is compiled to bytecode and executed by that interpreter. The user experiences this as running `script.py`, but the machine is executing the native interpreter that hosts the script.

The Native Interpreter Process: CPython as the Executable Loaded by the Operating System

When Python code is executed through the usual CPython implementation, the native executable loaded by the operating system is not the user's `.py` file. The native executable is the CPython interpreter itself. This distinction separates the operating-system execution layer from the Python language execution layer.

The file `script.py` is source text. It contains Python statements, expressions, function definitions, class definitions, imports, and other language constructs. But it is not a native binary program. It

does not contain an ELF header on Linux or a PE/COFF executable header on Windows. It does not contain processor instructions arranged for direct execution by x86-64, ARM64, or another physical instruction set. It is not mapped by the operating system as the process text segment.

By contrast, the CPython interpreter is a native executable. On Linux, this may be a file such as:

`python3`

On Windows, it may be:

`python.exe`

This executable has already gone through the native compilation pipeline. CPython is written mostly in C. Its source code is compiled, assembled, linked, and packaged as an executable binary for a specific operating system and processor architecture. Therefore, when the operating system creates a process for Python execution, it is creating a process from the CPython executable image.

A simplified view is:

```
python / python.exe
  native executable loaded by the OS
    |
    v
CPython process
  reads and executes script.py
```

The operating system loader performs the same kind of work for CPython that it performs for any other native executable. It validates the executable format, maps code and data segments into virtual memory, prepares the stack, processes dynamic library dependencies, applies required relocations, initializes the execution context, and transfers control to the interpreter's native entry point.

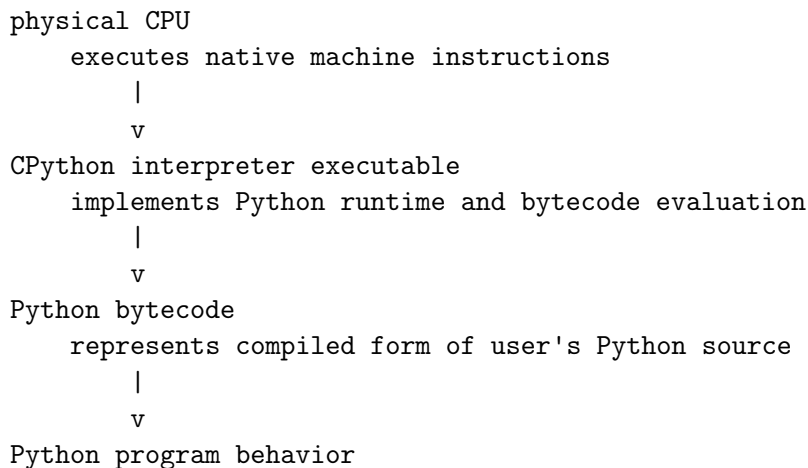
At this level, CPython is not special. It is a normal native process. It has:

- a text segment containing compiled machine instructions of the interpreter;
- data and BSS regions containing native global and static runtime state;
- a process heap used by CPython's allocator and the C runtime allocator;
- one or more native stacks used by interpreter and library calls;
- mapped shared libraries;
- file descriptors or handles;
- command-line arguments;
- environment variables;
- a scheduler-visible process identity.

Only after this native interpreter process exists can the user's Python source code begin to matter. The file name `script.py` is passed to the interpreter as a command-line argument. CPython then opens the file, reads its contents, tokenizes the source, parses it, builds internal structures, compiles it to bytecode, and evaluates that bytecode.

This means that the physical CPU is always executing native machine code. However, the native machine code belongs to the CPython interpreter, not directly to the user's Python source file. The Python program is executed because CPython's native code implements a virtual execution engine for Python bytecode.

A useful layered view is:



This also explains why the Python interpreter must match the operating system and architecture. A Windows `python.exe` is not the same binary as a Linux `python3` executable. A CPython binary compiled for x86-64 is not the same as one compiled for ARM64. The interpreter itself is native software and must be compatible with the platform on which it runs.

The Python source file, however, is mostly platform-independent at the source level, provided it does not depend on platform-specific libraries, paths, encodings, or system behavior. The same `script.py` file may be read by CPython on Linux, Windows, or macOS. This is possible because the source file is not directly executed by the processor. It is interpreted by a platform-specific CPython executable that provides a common Python language runtime.

This pattern is not unique to Python. Many language systems use a native host process that executes user code through a runtime layer. The exact internal strategy differs, but the architectural separation remains the same: the operating system runs the runtime executable; the runtime executes the user's program representation.

For example, PHP is also commonly executed through a native runtime. A PHP source file is not normally loaded by the operating system as a native executable. Instead, a PHP runtime executable or server module receives the PHP file, parses it, compiles it into internal opcodes, and executes those opcodes inside the PHP engine. In modern PHP, this compilation usually happens in memory, although opcode caches may preserve compiled forms to avoid repeated compilation. The user writes `.php` source code, but the native process is the PHP engine, the web server module, PHP-FPM worker, or command-line PHP executable that hosts and executes it.

JavaScript in V8 follows another variation. V8 is a JavaScript engine, but it is usually embedded inside another native application. In the browser, V8 is embedded inside a browser process. In Node.js, V8 is embedded inside the Node.js runtime. In Electron applications, V8 appears through the Chromium/Node-based application stack. Applications such as Visual Studio Code use this general model: the visible application is a native desktop program built around an embedded web/runtime engine, and JavaScript or TypeScript-derived code runs inside that hosted environment. Here again, the operating system does not run a `.js` file directly as a native executable. It runs the browser, Node.js, or Electron application, and the embedded engine executes the JavaScript program.

Bash shows a different and more direct interpretation model. A shell script is usually read by the shell process as text. Bash interprets the script command by command. For shell built-ins, the shell may execute the operation inside the shell process itself. For external commands such as:

```

ls
cat
grep

```

the shell typically creates a child process and asks the operating system to execute the corresponding native program. Control moves back and forth between the shell process and child processes. The script is interpreted by Bash, but many individual commands are separate native executables launched as child processes.

A simplified Bash execution model is:

```

bash script.sh
|
v
Bash process reads command text
|
+--> built-in command executed inside Bash
|
+--> external command:
    fork / create child process
    exec native program such as ls, cat, grep
    wait or continue depending on shell syntax

```

This is different from CPython bytecode execution. CPython usually compiles a Python source file into bytecode and then runs that bytecode inside the interpreter process. Bash more directly reads command syntax and dispatches commands. Some commands remain inside the shell; others cause process creation and native executable loading. Therefore, Bash is closer to real command-by-command interpretation, while Python is better described as source-to-bytecode compilation followed by bytecode interpretation inside a runtime.

These examples show that the phrase “interpreted language” is too vague if used alone. Different runtimes place the boundary in different places:

- CPython compiles Python source into bytecode and interprets that bytecode inside the CPython process;
- PHP commonly compiles source into engine opcodes in memory and executes those opcodes inside the PHP runtime;
- V8 is a JavaScript engine embedded in host programs such as browsers, Node.js, or Electron-style applications, and may use several execution tiers including interpretation and JIT compilation;
- Bash reads shell commands and often alternates between interpreting built-ins inside the shell and launching external native child processes.

The command:

```
python script.py
```

can therefore be understood as two different operations joined into one user-visible action:

1. the operating system starts the native CPython executable;
2. CPython reads and executes the Python script inside that process.

The first operation is native process creation. The second operation is runtime-hosted language execution.

This distinction is also visible when using an interactive Python shell. When the user starts:

```
python
```

without a script file, the operating system still loads and starts the same native interpreter executable. But instead of reading a complete source file from disk, CPython enters an interactive loop. It reads input from the user, compiles each complete statement or block, executes it, prints results when appropriate, and waits for more input. The interpreter process stays alive across many small pieces of source code.

The same principle applies to Jupyter notebooks. The visible notebook cells are not independent native programs. A Python kernel process is already running. Each executed cell is sent to the kernel,

compiled, and executed inside that process. Variables persist across cells because they live in the continuing interpreter runtime state, not because each cell becomes a separate executable.

The native interpreter process also owns the Python runtime environment. Python objects are allocated inside memory managed by CPython. Python function calls produce Python-level frames or internal frame structures. Python modules are represented by module objects and namespace dictionaries. Python exceptions are represented by runtime objects and propagated through interpreter-managed frame state. These structures exist inside the CPython process address space.

For example, when Python code creates a list:

```
values = [1, 2, 3]
```

the operating system does not create a new native process or a new executable segment for this list. CPython allocates Python objects inside its managed heap. The name `values` becomes a binding in a Python namespace to a list object. That list object stores references to other Python objects. All of this is interpreter-level runtime state inside the already existing native process.

This layered architecture also explains why native extension modules are possible. Some Python modules are not written purely in Python. They may be compiled shared libraries loaded into the CPython process. When CPython imports such a module, the operating system dynamic loader may map additional native code into the interpreter process. The Python program still appears to import a module, but the implementation crosses into native binary code.

A simplified picture is:

CPython process

```
native interpreter code
Python runtime heap
Python bytecode objects
loaded Python modules
loaded native extension modules
```

This is why Python sometimes behaves like a high-level interpreted language and sometimes exposes native dependency problems. The user's code may be Python source, but the interpreter and many libraries may depend on compiled C, C++, Fortran, or system libraries.

For students coming from C, the key point is that CPython is itself a C program running as a native process. The Python program is not outside this process. It is not directly run by the operating system. It is represented and executed inside CPython's memory, using CPython's runtime rules.

This gives the Python execution model a different shape from the C model:

C:

```
user source code
-> native executable
-> operating-system process
```

Python:

```
CPython executable
-> operating-system process
-> reads user source code
-> creates bytecode and runtime objects
-> executes them inside the interpreter
```

Therefore, the native interpreter process is the real operating-system object behind ordinary Python execution. The kernel schedules CPython. The loader maps CPython. The CPU executes CPython's native instructions. The Python file is handled later, by the interpreter, as source input to a managed runtime. Understanding this separation is the foundation for understanding Python bytecode, frames, objects, namespaces, memory management, and runtime introspection.

The User Script as Runtime Input: Source File, Module Body, and First Executable Statement

Once the interpreter process already exists, the user's Python file becomes runtime input. At this level we are no longer discussing how the operating system loads the interpreter executable. We are now discussing what CPython does with the source file it receives.

A file such as `script.py` is a text file containing Python source code. CPython reads that text and treats it as a **module body**: a top-level sequence of Python statements. The module body is not only a collection of declarations. It is executable code. Statements placed at the top level of the file are executed in order.

For example:

```
print("before")

x = 10

def f():
    print("inside f")

print("after")
```

Execution starts at the first top-level statement:

```
print("before")
```

Then the assignment statement is executed:

```
x = 10
```

Then the `def` statement is executed. This is important: executing a function definition does not execute the body of the function. It creates a function object and binds the name `f` to that object in the module namespace. The body of `f` runs only if the function is later called.

Therefore, the top-level execution order is:

1. execute `print("before")`
2. bind `x` to the integer object 10
3. create function object `f`
4. bind name `f` to that function object
5. execute `print("after")`

The function body:

```
print("inside f")
```

is not part of the immediate module execution path. It is compiled as part of the function's code object, but it is executed only when the function is called.

This is different from the C model. In C, the programmer's ordinary execution entry is conventionally the function `main()`. Top-level declarations in C do not execute as a sequence of runtime statements in the same way Python module statements do. In Python, the file itself has an executable body. A function named `main` has no automatic special status unless the programmer explicitly calls it.

For example:

```
def main():
    print("program logic")
```

This file creates a function object named `main`, but it does not run that function. To execute it, the file must contain a call:

```
def main():
    print("program logic")

main()
```

A common Python convention is:

```
def main():
    print("program logic")

if __name__ == "__main__":
    main()
```

This pattern separates two roles of the same file. If the file is executed directly, the special module variable `__name__` receives the value `"__main__"`, and `main()` is called. If the file is imported as a module, `__name__` receives the module's import name, and the guarded call is skipped.

This distinction is central to Python's module system. A Python file can be used as:

- a **script**, where the module body is executed as the main program;
- an **imported module**, where the module body is executed to create definitions and bindings for reuse.

Importing a module still executes its top-level body once. This often surprises students coming from C. Python's `import` is not equivalent to C's `#include`. C header inclusion is source-text insertion before compilation. Python `import` is runtime module loading. It creates or retrieves a module object, executes its body if needed, and then binds a name to that module or imports selected names from it.

For example, suppose `tools.py` contains:

```
print("loading tools")

def add(a, b):
    return a + b
```

If another file contains:

```
import tools
```

then the top-level statement:

```
print("loading tools")
```

runs during the import, and the function object `add` is created inside the module namespace. The function body of `add` still does not run until `tools.add(...)` is called.

At runtime, CPython represents the compiled module body as a code object. That code object contains bytecode for the top-level statements of the file. Function definitions, class definitions, imports, assignments, and ordinary expressions are all represented as executable operations inside that module-level code object.

A simplified view is:

```

script.py source text
  |
  v
module-level code object
  |
  v
module namespace dictionary
  |
  v
top-level statements execute in order

```

The module namespace is where top-level names are stored. If the script contains:

```

a = 10
b = 20

```

then execution creates bindings in the module namespace. The names `a` and `b` are not fixed native memory boxes like C local variables. They are entries in a Python namespace pointing to Python objects.

This idea also explains interactive execution. In an interactive Python prompt, the user does not provide a complete file. Instead, the interpreter repeatedly receives smaller source fragments. Each complete statement or block is compiled and executed in the current interactive namespace. The persistent namespace is why this works:

```

>>> x = 10
>>> x + 5
15

```

The second input can use `x` because the first input created a binding in the still-existing interpreter state.

Jupyter notebooks behave similarly. A notebook cell is source input sent to a running Python kernel. The cell is compiled and executed in the kernel's current namespace. Variables persist across cells because the kernel process and its runtime namespace remain alive.

Other runtime-hosted languages use related but not identical input models.

In PHP, a `.php` file is also a source file consumed by a runtime engine. The PHP engine parses the file, compiles it into internal opcodes, and executes the top-level script body. In command-line PHP, this resembles direct script execution. In web use, a PHP-FPM worker or server-integrated PHP runtime may execute the script body as part of a request. Modern PHP can cache opcodes, but the important model is that the source file is compiled into engine instructions and executed by the PHP runtime, not loaded as a native executable by itself.

In JavaScript systems using V8, the script or module source is input to an engine embedded inside a host. In a browser, the host is the browser. In Node.js, the host is the Node runtime. In Electron applications, the host is an application stack built around Chromium and Node-related components. Applications such as Visual Studio Code use this kind of embedded-runtime architecture. The JavaScript file is not the operating-system process image. It is source input compiled and executed inside the embedded engine and its host environment.

Java differs because the usual runtime input is not source text. A Java source file is normally compiled first:

```

Main.java
  |
  | javac
  v
Main.class

```

The JVM then receives bytecode class files or archives such as JAR files. Execution begins from a selected static method with the conventional signature:

```
public static void main(String[] args)
```

So Java has an explicit source-to-bytecode step before normal execution. Python and PHP usually hide this compilation step inside the runtime invocation. Java exposes it as a separate build step.

Bash is different again. A shell script is closer to command-by-command interpretation. Bash reads the script as command text. Some commands are shell built-ins and execute inside the Bash process. Other commands are external programs, so Bash creates child processes and asks the operating system to execute those native programs.

For example:

```
echo "files:"  
ls  
cat notes.txt
```

The `echo` command may be handled as a shell built-in. The commands `ls` and `cat` are commonly external executables. Bash interprets the script structure, expands variables and patterns, prepares redirections or pipes, and launches child processes for external commands. This is much closer to direct command dispatch than CPython's module-to-bytecode execution model.

The comparison can be summarized as follows:

- **Python:** source file becomes a module body, compiled to bytecode and executed in a module namespace;
- **PHP:** source file becomes a script body, compiled to engine opcodes and executed by the PHP runtime;
- **V8 JavaScript:** source or module input is compiled and executed inside an embedded engine hosted by a browser, Node.js, or Electron-like application;
- **Java:** source is normally compiled ahead of execution into JVM bytecode, and the JVM starts from a selected `main` method;
- **Bash:** script text is interpreted as shell commands, with built-ins executed in the shell and external commands launched as child processes.

For Python, the important conclusion is precise: the user's file is a module-level source input. Its top-level statements form the first executable Python path. Function and class bodies are compiled and stored for later execution, but they do not run merely because they are defined. Execution begins with the first top-level statement of the module body, inside the already initialized Python runtime.

Runtime Ownership of Execution State: Frames, Objects, Bytecode, and Managed Memory

After the user's source has been accepted as runtime input, the next important question is: who owns the execution state? In a native C program, much of the execution state is directly expressed through native stack frames, registers, machine-code instruction addresses, raw memory addresses, and manually managed heap allocations. In Python, the user's program state is not owned directly by the hardware in that form. It is owned by the CPython runtime.

This means that Python execution is represented through runtime structures created and managed by the interpreter. The processor still executes native machine instructions, but those native instructions belong to CPython. The user's program advances because CPython reads Python bytecode, maintains Python execution frames, allocates Python objects, updates namespaces, handles exceptions, and manages object lifetime.

A simplified CPython-level execution model is:

```

Python source
  |
  v
code object containing bytecode
  |
  v
Python execution frame
  |
  v
bytecode evaluation loop
  |
  v
Python objects, namespaces, exceptions, and return values

```

The central execution unit is not a native instruction from the user's source file. It is a Python bytecode instruction stored inside a code object. The code object represents compiled Python code. It contains bytecode plus metadata needed by the interpreter, such as constants, variable names, free-variable information, and other structural details.

For example, a Python function definition creates a function object that refers to a code object:

```

def add(a, b):
    return a + b

```

The body of `add` is compiled into a code object. When `add(2, 3)` is called, CPython creates a new execution frame for that call. The frame stores the runtime state needed to execute the function: local bindings, links to global and built-in namespaces, the current bytecode position, and the evaluation stack used by bytecode operations.

This frame is not the same thing as a C stack frame. CPython itself uses native C stack frames while running its interpreter code, but Python function calls are represented at the language-runtime level by Python frames or internal frame structures. A Python local variable is not simply a raw slot in the native process stack. It is a Python-level name or local slot referring to a Python object.

For example:

```

def f():
    x = 10
    y = x + 5
    return y

```

When `f()` runs, the integer objects and name bindings are managed by CPython. The name `x` refers to a Python integer object. The name `y` later refers to another Python integer object. The execution frame records these bindings and the current progress through the bytecode. The hardware does not know that a Python variable named `x` exists. CPython knows.

This is why Python execution state is inspectable in ways that native C execution state usually is not. Python can expose objects, types, frames, tracebacks, globals, locals, functions, closures, and bytecode-level information because these are runtime-managed structures. They are not merely temporary consequences of native machine execution; they are represented as objects or internal records inside the interpreter.

The object model is also runtime-owned. In C, an `int` local variable may be a fixed-width value stored directly in a stack slot or register. In Python, an integer value is an object. A list is an object. A function is an object. A class is an object. A module is an object. The runtime tracks these objects, stores their type information, manages references to them, and decides when their memory can be reclaimed.

A simplified Python binding looks like this:


```

name in namespace
  |
  v
reference to Python object
  |
  v
object header + type pointer + value-specific data

```

The name is not the object. The name is a binding. The object lives in memory managed by CPython. Multiple names can refer to the same object. Function arguments are also new local bindings to existing objects, not automatic deep copies of object contents.

This managed object model also controls memory lifetime. In C, the programmer often decides directly when heap memory is allocated and freed through `malloc()` and `free()`. In Python, ordinary user code does not call `free()` on every object. CPython tracks object references and reclaims memory when objects are no longer reachable according to its memory-management rules. CPython primarily uses reference counting, with additional garbage collection for certain cyclic object graphs.

Therefore, Python memory management exists above the native heap. CPython uses the process heap and its own allocator mechanisms internally, but the Python programmer usually sees object creation and object reachability, not raw allocation and deallocation calls.

The same principle applies to exceptions. In C, error handling is often represented through return codes, special values, flags, or manually checked pointers. In Python, an exception is a runtime object routed through interpreter-managed frames. When an exception is raised, CPython does not merely return a special integer. It creates or propagates an exception object, unwinds Python frames, searches for a matching handler, and constructs traceback information describing the route of failure.

This is runtime ownership of control state. The interpreter knows which Python frames are active, where execution is suspended or failing, which handlers exist, and where control should continue if an exception is caught.

Other runtime-hosted languages follow the same broad idea, although their internal structures differ.

In PHP, the PHP engine owns the script execution state. PHP source is compiled into internal opcodes. Those opcodes are executed by the Zend Engine. Variables live in PHP runtime symbol tables and value containers, not as direct native C variables belonging to the user's source. Function calls create runtime call frames managed by the engine. Object lifetime is handled by the PHP runtime, which uses reference counting and garbage collection mechanisms. As with Python, the user's script behavior is produced by a native engine maintaining language-level execution state.

A simplified PHP model is:

```

PHP source
  |
  v
PHP opcodes
  |
  v
Zend Engine call frames and symbol tables
  |
  v
PHP values and objects managed by the runtime

```

In Java, the JVM owns the execution state of Java bytecode. A Java source file is normally compiled ahead of time into `.class` files containing JVM bytecode. At runtime, the JVM loads classes, verifies bytecode, creates stack frames for method calls, manages the heap, performs garbage collection, and may compile hot bytecode paths into native machine code using a JIT compiler. The Java method frame is a JVM-level structure with local variables and an operand stack. Java objects live on the JVM-managed heap. The operating system runs the JVM process; the JVM runs the Java program.

A simplified Java model is:

```

.class bytecode
  |
  v
JVM method frames
  |
  v
operand stacks and local variable arrays
  |
  v
JVM heap objects and garbage collection

```

In V8, JavaScript execution state is owned by the JavaScript engine embedded inside a host program. The host may be a browser, Node.js, or an Electron-style application. V8 parses JavaScript, creates internal representations, executes bytecode, gathers profiling information, and may compile frequently executed paths into optimized native code. JavaScript objects, closures, lexical environments, promises, and call frames are engine-managed structures. The surrounding host supplies APIs such as timers, file operations in Node.js, DOM access in browsers, or application-level services in Electron programs such as Visual Studio Code.

A simplified V8 model is:

```

JavaScript source
  |
  v
V8 internal code representation / bytecode
  |
  v
V8 execution frames and lexical environments
  |
  v
JavaScript heap objects and garbage collection
  |
  v
host event loop and external APIs

```

Bash is different because its runtime state is closer to command interpretation than to a bytecode virtual machine. Bash owns shell variables, functions, aliases, expansion rules, redirections, pipelines, and job-control state. For shell built-ins, execution happens inside the Bash process. For external commands, Bash creates child processes and the operating system loads native executables such as `ls`, `cat`, or `grep`. Therefore, Bash execution state is split: the shell owns the script interpretation state, but many individual commands own their own separate process state.

A simplified Bash model is:

```

Bash script text
  |
  v
Bash parser and command dispatcher
  |
  +--> built-in command:
  |     shell state changes inside Bash
  |
  +--> external command:
  |     child process with separate native execution state

```

For example, a command such as:

```
cd /tmp
```

must be a shell built-in because it changes the current directory of the shell process itself. But a command such as:

```
ls
```

usually runs as a separate native program. Bash prepares the arguments, starts the child process, and waits or continues depending on the script structure.

These examples show that runtime ownership has different forms:

- CPython owns Python bytecode, Python frames, Python objects, namespaces, and managed memory;
- PHP owns opcodes, call frames, symbol tables, values, and request/runtime state;
- the JVM owns class metadata, method frames, operand stacks, heap objects, and garbage collection;
- V8 owns JavaScript execution frames, lexical environments, objects, optimized code tiers, and garbage-collected memory;
- Bash owns shell interpretation state, but external commands run in separate native child processes.

The common idea is that the user's high-level program is not directly identical to the native process machinery. A runtime stands between the source-level program and the hardware. That runtime decides how program state is represented, where values live, how calls are tracked, how memory is reclaimed, and how control flow is routed.

For Python, this point is fundamental. The operating system owns the native process boundary. CPython owns the Python execution boundary. The CPU executes CPython's native instructions, but CPython owns the Python-level state: bytecode, frames, objects, namespaces, exceptions, and managed memory. Understanding this separation makes later Python behavior much less mysterious. Variables are bindings, not raw boxes. Function calls create interpreter-managed frames. Exceptions unwind runtime frames. Objects live in managed memory. The Python program runs because the runtime continuously maps bytecode-level operations onto native work performed by the interpreter.

2.2.2 Abstracting the Hardware Layer: The Process VM Blueprint

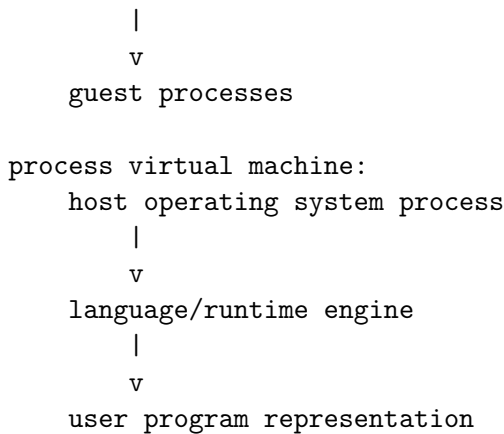
Definition, Scope, and Intent of a Software-Driven Execution Runtime Environment

A **process virtual machine** is a software execution environment that runs inside an ordinary operating-system process and presents the user program with a machine-like model that is not identical to the physical hardware. It is called a *process* virtual machine because it does not normally emulate an entire computer, boot an operating system, or replace the kernel. Instead, it lives inside one native process created by the real operating system.

This distinction matters. A system virtual machine, such as a full virtualized computer, tries to reproduce enough hardware for an operating system to run inside it. A process virtual machine has a narrower purpose. It exists to run programs written for a language-level or runtime-level execution model.

A simplified distinction is:

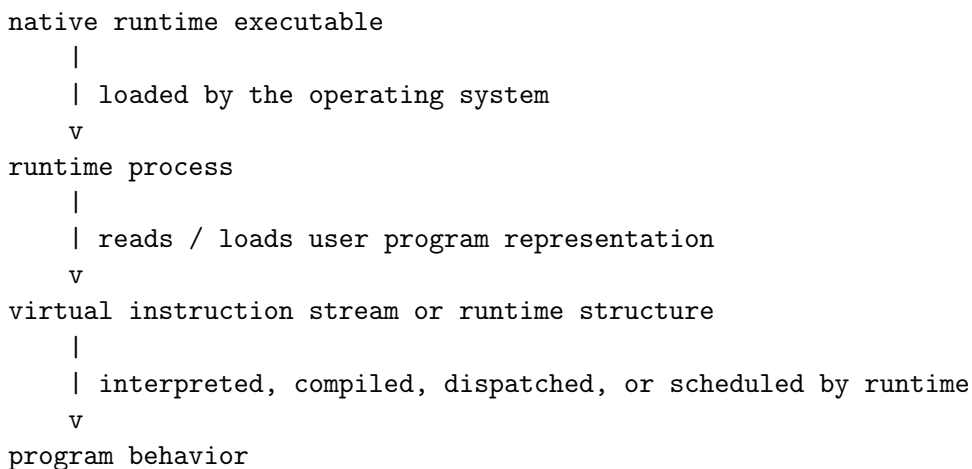
```
system virtual machine:
  virtual hardware
    |
    v
  guest operating system
```



The process VM is therefore not an operating system. It does not replace the kernel scheduler, the memory management unit, the file system, or the system-call interface. It depends on the real operating system for process creation, virtual memory, files, sockets, timers, native threads, and device access. However, inside that process, it creates another execution layer: a runtime environment in which the user's program can run according to rules defined by the language implementation.

In the C model, the compiled program is translated into native instructions for the physical processor. The operating system loads the executable, maps its text segment, prepares its stack and heap, and the CPU executes the program's instructions directly. In a process VM model, the operating system loads the runtime executable first. The user program is then represented inside that runtime as source code, bytecode, opcodes, class files, command structures, or another internal form.

A general process VM model looks like this:



The main intent is to separate the user program from direct dependence on the physical processor instruction set. The runtime can define its own execution model: its own bytecode, object representation, function-call mechanism, exception rules, memory-management strategy, type system, module system, and sometimes its own scheduling model. The user's program targets that runtime model rather than directly targeting x86-64, ARM64, or another hardware architecture.

This does not mean that the physical machine disappears. The processor still executes native instructions. The difference is that those native instructions mostly belong to the runtime engine. The runtime then performs the user's program operations indirectly. In CPython, the CPU executes CPython's native interpreter loop; that loop reads Python bytecode and performs the corresponding Python operations. In the JVM, the CPU executes JVM runtime code, interpreter code, or JIT-compiled native code derived from JVM bytecode. In PHP, the engine executes internal opcodes. In V8, the engine uses several internal execution tiers to run JavaScript code inside a host process.

A process VM usually provides more than instruction dispatch. It often owns a complete runtime environment. This may include:

- a representation for code, such as bytecode, opcodes, or internal compiled forms;

- a call-frame model for functions or methods;
- an object model and type system;
- a managed heap for runtime objects;
- memory reclamation, such as reference counting or garbage collection;
- exception propagation and stack unwinding rules;
- module, class, or package loading mechanisms;
- reflection or introspection facilities;
- foreign-function or native-extension interfaces;
- event loops, task queues, or coroutine schedulers in some environments.

This is why a process VM can support language features that are harder to express directly in the bare C execution model. If the runtime owns frames, objects, and control state, it can expose dynamic inspection, preserve suspended computations, route exceptions through managed frames, allocate objects uniformly on a managed heap, and decide at runtime how operations should be dispatched.

For example, Python can ask for the type of an object at runtime:

`type(x)`

It can inspect attributes, build functions dynamically, catch exceptions as objects, and keep generators suspended between calls. These features are possible because the CPython runtime keeps language-level state in runtime-managed structures. The hardware only sees native CPython instructions and memory accesses. The Python-level meaning is maintained by the interpreter.

The scope of a process VM is therefore language execution, not whole-machine execution. It abstracts selected parts of the hardware model and replaces them with runtime-defined structures. The physical processor has registers, instruction pointers, native stacks, memory addresses, and machine instructions. The process VM may instead expose or internally use virtual instructions, evaluation stacks, frame objects, operand stacks, local-variable arrays, object references, dynamic dispatch tables, and managed memory.

A useful comparison is:

Native process model	Process VM model
CPU executes compiled machine instructions from the user's program	CPU executes native runtime code that executes or compiles the user's program representation
Function calls use native calling conventions directly	Function or method calls may use runtime-managed frames or virtual call conventions
Values may be raw primitives in registers, stack slots, or memory	Values are often runtime objects or tagged values managed by the VM
Memory lifetime may be manual, as in C heap allocation	Memory lifetime is often managed by reference counting or garbage collection
Errors are often return codes, flags, signals, or hardware traps	Errors are often runtime exception objects propagated through managed frames

This abstraction has a cost. A runtime layer must do work that native compiled code may not need to do at runtime. It may need to decode bytecode, look up names dynamically, check object types, dispatch operations through type slots or method tables, manage reference counts, trace garbage, and maintain runtime metadata. These operations add overhead.

However, the runtime layer also provides important advantages. It can make programs more portable, because the same user program representation can run on different platforms if the runtime exists for those platforms. It can make the language more dynamic, because names, types, objects, and

modules can be handled at runtime. It can simplify memory management for the programmer. It can provide introspection, sandboxing mechanisms, debugging hooks, interactive execution, and advanced control-flow structures.

The balance differs between runtimes. CPython emphasizes a simple and predictable bytecode interpreter with a rich object model. The JVM emphasizes verified bytecode, managed memory, and strong runtime optimization through JIT compilation. V8 emphasizes aggressive adaptive optimization for JavaScript inside host environments such as browsers, Node.js, and Electron-style applications. PHP's Zend Engine compiles PHP source into opcodes and executes them inside request-oriented or command-line runtime contexts. Bash is less like a bytecode VM and more like a command interpreter, but it still acts as the runtime owner for shell variables, expansions, built-ins, redirections, and child-process dispatch.

The common pattern is that the user program targets the runtime environment first. The runtime then targets the real operating system and hardware. This creates a layered execution architecture:

```

user program semantics
    |
    v
runtime representation
    |
    v
process virtual machine
    |
    v
native process
    |
    v
operating system and hardware

```

For students moving from C to Python, this is the central shift. In C, the program is usually transformed into the native executable that becomes the process. In Python, the native process is CPython, and the Python program is represented inside CPython's runtime environment. The process VM is the software machine that gives Python code its execution world: bytecode, frames, objects, namespaces, exceptions, and managed memory.

The Bytecode Concept: Platform-Agnostic Virtual Instruction Set Architecture (ISA) Layouts

A process virtual machine needs a representation of the user's program that it can execute. That representation is often called **bytecode**. Bytecode is an instruction stream for a software machine, not for the physical processor. It is similar in purpose to machine code, but its target is a virtual instruction set implemented by a runtime engine.

In a native C program, the compiler eventually emits instructions for a real processor architecture. The final instruction stream may target x86-64, ARM64, RISC-V, or another hardware ISA. The physical CPU can fetch and execute those instructions directly. In a bytecode runtime, the source program is instead translated into instructions for a virtual machine. The physical CPU does not execute those bytecode instructions directly. The runtime reads them and performs the corresponding operations.

A simplified native compilation model is:

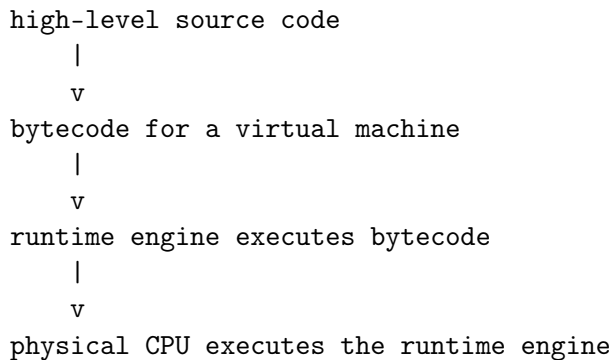
```

C source code
    |
    v
native machine code
    |
    v

```

physical CPU execution

A simplified bytecode runtime model is:



The term *bytecode* originally suggests instructions stored as compact byte-sized operation codes, but the exact layout varies across runtimes. Some virtual machines use one-byte opcodes. Others use wider instructions, wordcode, variable-length encodings, operands following the opcode, constant-pool indexes, register indexes, stack operations, or other implementation-specific forms. The important idea is not the literal byte size of every instruction. The important idea is that the instruction stream belongs to a virtual machine rather than to the hardware processor.

For example, a Python statement such as:

```
x = a + b
```

can be compiled by CPython into bytecode operations that conceptually mean:

```

load the value named a
load the value named b
perform Python addition
store the result under the name x

```

These are not CPU instructions. They are instructions for CPython's evaluation loop. The real CPU executes the native machine code of the interpreter. The interpreter reads each bytecode operation and executes the corresponding C-level runtime logic.

A virtual instruction set may include operations such as:

- loading constants;
- loading or storing local names;
- loading or storing global names;
- performing arithmetic operations;
- calling functions;
- returning values;
- creating objects;
- jumping to another bytecode position;
- handling exceptions;
- accessing attributes or indexes.

This resembles a hardware ISA because it defines the operations that the execution engine understands. However, it is not constrained in the same way as a physical ISA. A bytecode instruction can represent a high-level language action. For example, a Python addition operation does not simply mean “add two machine integers.” It may mean: inspect the runtime types of the two objects, dispatch through the appropriate numeric protocol, call special methods if needed, allocate a result object, update reference counts, and handle possible exceptions.

Therefore, a bytecode instruction can hide a large amount of runtime work. One virtual instruction may expand into many native instructions executed by the interpreter. This is one reason bytecode interpretation is usually slower than direct native execution, but also one reason it can support rich dynamic behavior.

The main advantage of bytecode is portability across hardware. If the same bytecode format can be understood by virtual machines on different platforms, the program representation becomes less tied to one processor architecture. The runtime must still be implemented separately for each platform, but the bytecode can remain mostly the same.

This is the principle behind the Java Virtual Machine. Java source code is compiled into JVM bytecode stored in `.class` files. The same class file can, in principle, be loaded by a JVM on different operating systems and processor architectures. The JVM implementation on each platform is responsible for interpreting or compiling that bytecode into behavior on the local machine.

A simplified Java model is:

```

Java source
  |
  | javac
  v
JVM bytecode (.class)
  |
  | JVM on Windows / Linux / macOS / different CPUs
  v
program execution

```

CPython also uses bytecode, but with a different portability intention. CPython bytecode is an internal implementation artifact, not a stable cross-version distribution format in the same strong sense as JVM bytecode. Python bytecode can be cached in `.pyc` files, but those files are tied to a compatible Python implementation and version. They are useful for avoiding repeated compilation, not as a universal long-term binary distribution format.

A simplified CPython model is:

```

Python source
  |
  | CPython compiler
  v
CPython bytecode
  |
  | CPython evaluation loop
  v
Python program execution

```

PHP uses a similar idea through engine opcodes. A PHP file is parsed and compiled into internal opcodes for the Zend Engine. These opcodes are usually created in memory, and an opcode cache may store them to reduce repeated compilation. They are virtual instructions for the PHP runtime, not direct hardware instructions.

V8, the JavaScript engine, also uses internal bytecode-like representations, but its execution model is more adaptive. JavaScript source may first be parsed and compiled into an internal bytecode form for an interpreter. As the program runs, V8 collects profiling information. Frequently executed paths

may then be compiled into optimized native code by a JIT compiler. Therefore, V8 demonstrates an important point: bytecode does not prevent later native-code generation. A runtime can start from bytecode and then compile selected parts into machine code at runtime.

Bash is different. It is not usually explained as a bytecode virtual machine in the same sense. A Bash script is interpreted as shell command syntax. Bash parses commands, expands variables and patterns, applies redirections and pipelines, executes built-ins inside the shell process, and launches external native programs as child processes. The runtime representation is more command-dispatch-oriented than bytecode-instruction-oriented. This makes Bash a useful contrast: not every interpreted runtime has the same bytecode architecture.

The bytecode concept also explains why virtual machines can define their own execution architecture. Some bytecode systems are stack-based. In a stack-based virtual machine, many operations take operands from an evaluation stack and push results back onto it. An addition may look conceptually like:

```
push a
push b
add
store x
```

The `add` instruction does not name its operands directly. It assumes they are already on the evaluation stack.

Other virtual machines are register-based. In a register-based virtual machine, instructions name virtual registers explicitly:

```
r1 = load a
r2 = load b
r3 = add r1, r2
store x, r3
```

Here, the instruction stream contains more explicit operand references. The registers are not necessarily physical CPU registers. They are virtual storage locations defined by the runtime's instruction set.

This distinction between stack-based and register-based bytecode is architectural. It affects bytecode density, interpreter design, instruction dispatch, optimization strategies, and how close the virtual machine feels to a real CPU model. It does not change the main idea: the instruction set belongs to a software machine.

Bytecode also provides a useful boundary for tooling. If a language implementation exposes bytecode inspection, students and developers can examine what the compiler produced from source code. In Python, the `dis` module can display CPython bytecode. This makes it possible to see that high-level source constructs are transformed into lower-level virtual instructions before execution.

For example, source code such as:

```
x = 1 + 2
```

does not remain only source text. CPython compiles it into a code object containing bytecode and constants. The exact bytecode varies across Python versions, but the principle remains: the interpreter executes a compiled internal instruction stream, not the raw source characters.

Bytecode sits between source code and native execution:

```
source code:
    human-readable language text

bytecode:
    virtual-machine instruction stream

native machine code:
    physical processor instruction stream
```

This middle layer is the main tool by which a process virtual machine abstracts the hardware. The user program is no longer expressed directly in terms of physical registers, raw addresses, and hardware instructions. It is expressed in terms of virtual operations chosen by the language runtime.

That abstraction has consequences. It can improve portability and make dynamic language behavior easier to implement. It can also add execution overhead, because a runtime must decode and execute virtual instructions. Some runtimes accept this overhead. Some reduce it through caching, specialization, adaptive interpretation, or JIT compilation. Others, such as CPython, historically prioritize a relatively simple interpreter model and a rich runtime object system over aggressive default native compilation.

For the transition from C to Python, the main conclusion is this: C normally produces machine instructions for the physical CPU, while CPython produces bytecode instructions for the CPython virtual machine. The physical CPU still does all real execution, but it executes the interpreter's native code. Python bytecode is the structured instruction stream that tells the interpreter what Python-level operation to perform next.

Structural Models of Execution Engines

Execution engines can be classified by the representation they execute and by the moment at which translation occurs. A C program follows the native ahead-of-time model: source code is transformed before execution into machine code for a physical processor. Runtime-hosted languages use different models. Some execute source-like structures, some execute bytecode or opcodes, some compile hot paths into native machine code while the program is already running, and some dispatch commands rather than virtual instructions.

A useful first classification is:

Model	What the engine executes	Typical examples
Native ahead-of-time execution	Physical CPU machine code produced before execution	C, C++ compiled binaries
Source or AST interpretation	Source-derived syntax tree or statement structure	Simple interpreters, teaching interpreters
Bytecode / opcode interpretation	Virtual instruction stream executed by a runtime loop	CPython, PHP Zend Engine, JVM interpreter tier
Hybrid bytecode and JIT execution	Bytecode first, selected paths compiled to native code at runtime	JVM, V8, modern PHP variants
Command interpretation and dispatch	Parsed shell commands, built-ins, and child-process launches	Bash and other shells

These categories are not language labels. They are implementation strategies. The same language can have several implementations with different execution engines. Python is usually taught through CPython, but other Python implementations may use different internal strategies. JavaScript in V8 is not executed exactly like JavaScript in every possible engine. PHP source is not executed in the same way as Bash command text, even though both may be casually called interpreted.

The simplest structural model is direct native execution. This is the C reference model:

```

C source
|
| ahead-of-time compiler
v
native machine code
|
| OS loader
v
process text segment
|

```

```

    | physical CPU fetch/decode/execute
    v
program behavior

```

Here, the user's program becomes the native instruction stream. The processor fetches the program's compiled instructions directly from the process text segment. The execution engine is essentially the physical CPU plus the operating-system process environment.

A source-level or tree-walking interpreter uses a different structure. It reads source code, parses it into a tree or similar representation, and then walks that structure to produce behavior:

```

source code
  |
  | parser
  v
syntax tree / AST
  |
  | interpreter walks tree nodes
  v
program behavior

```

For example, an assignment node may cause the interpreter to evaluate the right-hand expression and update an environment. An if node may cause the interpreter to evaluate a condition and choose one branch. This model is simple to understand and useful for teaching interpreter construction. However, repeatedly walking a high-level tree can be inefficient compared with executing a compact instruction stream.

Bytecode interpreters improve this by translating source-level structure into a sequence of simpler virtual instructions. CPython belongs primarily to this family. Python source is parsed and compiled into CPython bytecode. The interpreter loop reads bytecode instructions and performs the corresponding Python operations:

```

Python source
  |
  | CPython parser and compiler
  v
CPython code object / bytecode
  |
  | bytecode evaluation loop
  v
Python runtime behavior

```

This model separates source parsing from execution. The interpreter does not repeatedly analyze raw source text. It executes a lower-level internal instruction stream. Each bytecode instruction is still virtual, not physical. The hardware executes CPython's native interpreter code; CPython executes the Python bytecode.

PHP uses a closely related model. PHP source is parsed and compiled into internal opcodes for the Zend Engine. These opcodes are then executed by the PHP runtime. In many deployments, this compiled form is produced in memory, and an opcode cache may keep the compiled representation available for later requests:

```

PHP source
  |
  | PHP parser/compiler
  v
Zend opcodes
  |

```

```

    | Zend Engine executor
    v
PHP runtime behavior

```

The model resembles CPython structurally, but the surrounding execution environment is often request-oriented. A PHP script may be executed by a command-line PHP process, a PHP-FPM worker, or a web-server-integrated runtime. The user source is still not the native process image; it is compiled into engine instructions consumed by the PHP runtime.

Java uses a more explicit two-stage model. Java source is normally compiled before execution into class files containing JVM bytecode. The JVM then loads, verifies, interprets, and often JIT-compiles that bytecode:

```

Java source
  |
  | javac
  v
JVM bytecode (.class / .jar)
  |
  | JVM class loader and verifier
  v
JVM execution engine
  |
  +--> interpreter tier
  |
  +--> JIT-compiled native code for selected paths
  v
program behavior

```

This makes Java different from CPython in the usual workflow. CPython normally hides source-to-bytecode compilation inside interpreter startup or import. Java normally exposes source-to-bytecode compilation as a separate build step. At runtime, the JVM is the process virtual machine that owns method frames, operand stacks, class metadata, heap objects, and garbage collection.

V8, the JavaScript engine used in browsers, Node.js, and Electron-style applications, is a hybrid adaptive engine. It is embedded inside a larger host process. That host may be a browser, the Node.js executable, or a desktop application shell such as Electron. Applications such as Visual Studio Code belong to this broad family: the operating system runs a native application process, and JavaScript or TypeScript-derived code executes inside an embedded runtime stack.

A simplified V8 model is:

```

browser / node / electron host
  |
  v
embedded V8 engine
  |
  | parses JavaScript source
  v
internal bytecode / runtime representation
  |
  +--> interpreter or baseline execution
  |
  +--> optimized native code for hot paths
  v
JavaScript behavior inside host environment

```

V8 is not merely a simple source interpreter. It uses internal representations, bytecode-like execution, profiling, and runtime optimization. Frequently executed code paths may be compiled into native machine code while the program is running. This is called just-in-time compilation, or JIT compilation. The important structural point is that the JavaScript file is still not the operating-system process image. The host process and embedded engine own execution.

Bash provides a useful contrast because it is not normally described as a bytecode virtual machine. Bash reads shell command text, parses commands, performs expansions, applies redirections and pipelines, and then either executes built-ins inside the shell process or launches external programs as child processes.

Bash script

```

|
| Bash parser and expansion rules
v
command dispatch
|
+--> shell built-in:
|     executed inside Bash process
|
+--> external command:
|     fork / exec child process
|     OS loads native executable

```

For example, `cd` must affect the shell process itself, so it is a shell built-in. Commands such as `ls`, `cat`, and `grep` are usually separate native executables. Bash interprets the script structure, but many individual operations are delegated to child processes. This is different from CPython, where ordinary Python statements are compiled into bytecode and executed inside the interpreter process.

These models differ in where they place the execution burden. A native compiled program pushes most translation work before runtime. A bytecode interpreter keeps a virtual instruction stream and dispatches it at runtime. A JIT engine begins with a portable or internal representation and later compiles selected parts into native machine code. A shell mostly interprets command structure and delegates many operations to external native programs.

The structural difference can be sketched as follows:

A. Native ahead-of-time model

source

```

-> native compiler
-> native executable
-> CPU execution

```

B. Bytecode interpreter model

source

```

-> runtime compiler
-> bytecode / opcodes
-> interpreter loop
-> behavior

```

C. Hybrid VM + JIT model

source or bytecode

```
-> VM representation
-> interpreter / baseline tier
-> profiling
-> optimized native code for hot paths
-> behavior
```

D. Command interpreter model

```
script text
-> command parser
-> built-ins inside interpreter
-> external commands as child processes
-> behavior
```

The classification also helps explain why performance characteristics differ. In a bytecode interpreter, every virtual instruction normally requires runtime dispatch. The engine must fetch the virtual instruction, decode it, and execute the corresponding native implementation. In a JIT engine, the runtime tries to remove some of that repeated dispatch cost by compiling frequently used paths into native code. In a shell, much of the cost may come from launching child processes, performing I/O, and coordinating pipelines rather than from arithmetic instruction dispatch.

This does not mean that one model is always better. Each model serves different priorities:

- native ahead-of-time compilation prioritizes direct machine execution and low runtime translation overhead;
- bytecode interpretation prioritizes portability, simplicity of runtime execution, and dynamic language support;
- hybrid JIT execution prioritizes adaptive performance by observing runtime behavior;
- command interpretation prioritizes process orchestration, system composition, and interactive control.

For this course, CPython remains the central model. CPython is best understood as a native executable that hosts a bytecode-based execution engine. The user's Python code becomes bytecode and runtime objects. The CPython evaluation loop drives execution. This places Python between the C native model and more aggressive adaptive engines such as V8 or the JVM.

The next structural distinction is inside bytecode execution itself. A virtual machine can be organized around an evaluation stack or around virtual registers. Stack-based and register-based virtual machines both execute virtual instructions, but they encode operands differently. That difference affects bytecode layout, dispatch behavior, and how the runtime engine represents temporary computation.

Stack-Based Virtual Machines: Evaluation Stack Operations and Zero-Address Instruction Sets

A **stack-based virtual machine** is an execution engine in which most temporary computation is organized around an internal evaluation stack. Instead of naming explicit operand locations for every operation, many instructions take their input values from the top of the stack and place their result back on the stack.

This is different from the native process stack discussed earlier. The native process stack belongs to the operating system and hardware execution model. It stores native call frames, return addresses, saved registers, and automatic local storage for native code. A virtual-machine evaluation stack is a runtime data structure used by the interpreter to evaluate expressions inside a language-level frame.

The two must not be confused:

- the **native stack** belongs to the real process and is used by native machine-code calls;
- the **VM evaluation stack** belongs to the language runtime and is used to evaluate virtual instructions.

In CPython, when Python bytecode is executed, each active Python frame has access to evaluation-stack-like storage used by the bytecode evaluator. Bytecode instructions push Python object references onto this stack, pop them when operations need operands, and push results back. The values on the stack are not raw integers or raw C values from the user's program. They are references to Python objects managed by the runtime.

The basic pattern is:

```
push value
push value
operation consumes values
operation pushes result
```

For example, consider the expression:

```
a + b
```

A stack-based virtual machine may execute this expression conceptually as:

```
LOAD a      # push value of a
LOAD b      # push value of b
ADD         # pop two values, add them, push result
```

The evaluation stack changes like this:

initial stack:

```
[ empty ]
```

after LOAD a:

```
[ value_of_a ]
```

after LOAD b:

```
[ value_of_a ]
[ value_of_b ]
```

after ADD:

```
[ result_of_a_plus_b ]
```

The instruction **ADD** does not need to say explicitly where its operands are. Its operands are implicitly the top two stack entries. This is why stack-machine operations are often described as **zero-address instructions**. The arithmetic instruction itself does not name source registers or destination registers. The operand locations are implied by the stack discipline.

The phrase *zero-address* should be understood carefully. Not every bytecode instruction has no operand at all. An instruction such as **LOAD_CONST** may include an index into a constants table. An instruction such as **LOAD_FAST** may include an index into local variables. The point is narrower: many computational operations, especially arithmetic and logical operations, do not explicitly name both operands and a destination. They use the evaluation stack.

A more complete example is assignment:

`x = a + b`

A simplified stack VM execution may look like this:

```
LOAD a      # push object bound to a
LOAD b      # push object bound to b
ADD          # pop two objects, compute result, push result
STORE x     # pop result and bind/store it as x
```

The stack states are:

[empty]

LOAD a:

[a]

LOAD b:

[a]

[b]

ADD:

[a + b]

STORE x:

[empty]

The temporary result does not need a named virtual register. It exists briefly on the evaluation stack, then is consumed by the store operation.

This makes stack bytecode compact. Since many operations do not need to encode explicit operand locations, the instruction stream can be smaller. For example, a register-based instruction might need to say:

`r3 = add r1, r2`

A stack-based instruction can simply say:

ADD

provided that the correct operands are already on top of the stack.

The cost is that the compiler must arrange instructions so that the correct values appear on the stack at the correct time. Expression order becomes stack choreography. The compiler translates nested expressions into a sequence of pushes and operations.

For example:

`x = (a + b) * c`

can be represented as:

LOAD a

LOAD b

ADD

LOAD c

MULTIPLY

STORE x

The evaluation stack evolves as:


```

LOAD a:
[ a ]

LOAD b:
[ a ]
[ b ]

ADD:
[ a + b ]

LOAD c:
[ a + b ]
[ c ]

MULTIPLY:
[ (a + b) * c ]

STORE x:
[ empty ]

```

This is a natural fit for expression evaluation because expressions are often tree-shaped. A stack machine can evaluate the leaves, combine them, and leave the result on the stack.

Function calls can also be represented through stack activity. Conceptually, the callable object and arguments are prepared, then a call instruction consumes them and pushes the return value. A simplified model is:

```

LOAD function
LOAD argument1
LOAD argument2
CALL 2
STORE result

```

The call instruction knows how many arguments to consume. It invokes the runtime's function-call machinery, creates or enters a new language-level frame, executes the function, receives a return value, and pushes that value back to the caller's evaluation stack.

In CPython, the exact bytecode names and call protocol change across Python versions, but the structural idea remains stable: Python source is compiled into virtual instructions, and those instructions manipulate a frame-local evaluation stack of Python object references.

A simplified CPython-like frame can be sketched as:

```

Python frame
+-----+
| code object / bytecode          |
| current bytecode offset         |
| local variable slots / namespace |
| global namespace link           |
| builtins namespace link         |
| evaluation stack                |
+-----+

```

During execution, the interpreter loop repeatedly performs a cycle similar to:

```

fetch next bytecode instruction
decode instruction
perform runtime operation

```

```
update evaluation stack / frame state
move to next bytecode position
```

This resembles a hardware CPU fetch-decode-execute cycle, but at the software-machine level. The hardware CPU fetches native CPython instructions. CPython fetches Python bytecode instructions. The two layers are separate.

A stack VM also has a simple control-flow model. A jump instruction changes the current bytecode position. A conditional jump usually consumes or inspects a value from the evaluation stack. For example, an `if` statement may be compiled conceptually as:

```
LOAD condition
JUMP_IF_FALSE else_block

    ... then body ...
    JUMP end_if

else_block:
    ... else body ...

end_if:
```

The condition value is placed on the evaluation stack. The conditional jump uses its truth value to choose the next bytecode location. Again, the branch does not directly manipulate native instruction pointers. It manipulates the virtual instruction position inside the runtime frame.

The Java Virtual Machine is another major example of a stack-based virtual machine. JVM bytecode operates around operand stacks inside method frames. A Java method frame contains local variable slots and an operand stack. Instructions load values from local variables onto the operand stack, operate on stack values, and store results back into local variables or object fields.

A simplified JVM method model is:

```
JVM method frame
+-----+
| local variable array |
| operand stack        |
| constant pool links  |
+-----+
```

A Java expression such as:

```
x = a + b;
```

may conceptually involve loading local variables onto the operand stack, applying an addition instruction, and storing the result. The JVM's operand stack is not the native process stack. It is part of the JVM's method-frame execution model.

CPython and the JVM are therefore both useful examples of stack-oriented virtual machines, although their surrounding language rules are very different. Java bytecode is statically typed and verified before execution. CPython bytecode operates on dynamically typed Python objects and relies heavily on runtime object protocols. The stack structure is similar as an execution-engine idea, but the language semantics are different.

PHP's Zend Engine should be treated more cautiously in this classification. PHP does compile source into opcodes, but its opcode format is not a simple pure zero-address stack machine in the same sense as the JVM operand-stack model. Zend opcodes use explicit operands that refer to variables, temporaries, constants, and result locations inside the engine's execution structures. Therefore, PHP belongs clearly to the opcode-runtime family, but it is not the best example of a pure stack-based VM.

V8 is also not a clean stack-machine example. Its interpreter tier, Ignition, uses a bytecode model based around an accumulator and virtual registers. This places it closer to a register/accumulator-based engine than to a pure stack VM. V8 then adds profiling and optimized native-code generation on top of that. It is therefore better used as a contrast in the next subsection.

Bash is outside this bytecode-stack classification. Bash interprets command structures, manages shell state, executes built-ins, and launches external commands. It does not normally execute a compact stack bytecode instruction stream in the CPython or JVM sense.

The stack-based VM model can be summarized as follows:

```

source expression tree
    |
    v
linear bytecode sequence
    |
    v
evaluation stack
    |
    v
runtime operations on objects / values

```

A stack VM is attractive because it is simple to generate code for. The compiler can translate expression trees into stack operations using a direct traversal. It is also simple for an interpreter loop to understand: many operations consume the top stack entries and push one result.

The main disadvantage is that stack traffic can be high. Values may be pushed and popped frequently. The instruction stream may contain many load/store stack operations. A register-based virtual machine can sometimes express the same computation with fewer dispatches by naming virtual locations explicitly. This is one reason some engines prefer register-based or accumulator-based bytecode forms.

However, stack-based virtual machines remain important because they provide a clear, compact, and language-independent model of expression execution. They make the bridge from syntax trees to bytecode easy to understand: evaluate subexpressions, push their results, combine them, and leave the final result on the stack.

For Python students, the most important conclusion is this: CPython does not execute a Python expression by turning every source-level operation directly into native hardware instructions. It compiles the expression into bytecode. That bytecode operates inside Python frames and manipulates an evaluation stack of object references. The stack belongs to the runtime execution model, not to the physical CPU as a direct representation of the user's variables.

Register-Based Virtual Machines: Virtual CPU Register Mapping and Explicit-Address Instructions

A **register-based virtual machine** is an execution engine in which bytecode instructions refer explicitly to named virtual storage locations. These storage locations are usually called **virtual registers**. They are not necessarily physical CPU registers. They are runtime-defined slots used by the virtual machine to hold temporary values during execution.

This is the main contrast with a stack-based virtual machine. In a stack VM, many operations take their operands implicitly from the top of the evaluation stack. In a register VM, instructions usually name their input and output locations directly.

A stack-style operation may look conceptually like this:

```

LOAD a
LOAD b
ADD
STORE x

```

The ADD instruction implicitly consumes the two values on top of the stack.

A register-style operation may look conceptually like this:

```
r1 = LOAD a
r2 = LOAD b
r3 = ADD r1, r2
STORE x, r3
```

Here, the addition instruction explicitly says where its operands come from and where the result goes. The instruction has addresses or register references built into it. This is why register-machine instructions are often called **explicit-address instructions**. They name their data locations rather than relying only on stack position.

The word *register* must be interpreted carefully. A virtual register is not the same thing as a hardware register such as RAX, RBX, or X0. It is a logical slot inside the VM's execution frame. The runtime may store that slot in memory, in a native stack frame, in a heap-allocated frame object, or sometimes in a real CPU register after optimization. But at the bytecode level, it belongs to the virtual machine, not directly to the hardware.

A simplified register-based VM frame can be sketched as:

```
VM frame
+-----+
| bytecode / instruction pointer |
| virtual register r0             |
| virtual register r1             |
| virtual register r2             |
| virtual register r3             |
| ...                             |
| constants / metadata links     |
+-----+
```

An expression such as:

```
x = (a + b) * c
```

may be represented in a register-based bytecode style as:

```
r1 = LOAD a
r2 = LOAD b
r3 = ADD r1, r2
r4 = LOAD c
r5 = MUL r3, r4
STORE x, r5
```

The temporary values do not appear only as stack entries. They are assigned to explicit virtual registers. This can make the data flow easier to see: r3 holds $a + b$, and r5 holds the final result.

A diagram of the data flow would be:

```
a ----\
      ADD ----> r3 ----\
b ----/                  MUL ----> r5 ----> x
c -----/
```

This explicitness can reduce the number of virtual instructions needed for some programs. In a stack machine, intermediate values may need repeated pushes and pops. In a register machine, values can remain in named virtual locations and be reused directly.

For example, the expression:

$(a + b) + (a + b)$

could be represented inefficiently by recomputing the left and right sides. A register-based representation can make reuse obvious:

```
r1 = LOAD a
r2 = LOAD b
r3 = ADD r1, r2
r4 = ADD r3, r3
```

The intermediate result **r3** is explicitly available for reuse.

This does not mean register VMs are always faster. They have different trade-offs. Register instructions are often wider because they must encode operand locations. A stack instruction such as **ADD** may be very compact because it does not name operands. A register instruction such as **ADD r3, r1, r2** must encode at least three virtual locations. So register bytecode may have fewer instructions, but each instruction may be larger.

The trade-off can be summarized as:

Property	Stack VM	Register VM
Operand location	Mostly implicit, top of stack	Explicit virtual registers
Instruction size	Often compact	Often wider
Instruction count	May be higher because of stack traffic	May be lower because values can be reused directly
Expression evaluation	Stack choreography	Named temporary slots
Compiler output	Easy to generate from expression trees	Requires virtual register assignment

A register-based virtual machine also resembles a real CPU more closely than a stack VM in one specific sense: real CPUs usually operate on registers and memory locations, not on an abstract operand stack. However, virtual registers are still not hardware registers. They are a software abstraction that can later be mapped, optimized, interpreted, or compiled.

A hardware-like instruction may look like:

```
ADD RAX, RBX
```

A register VM instruction may look like:

```
ADD r3, r1, r2
```

The visual similarity is useful, but the meaning is different. **RAX** and **RBX** are physical CPU registers. **r1**, **r2**, and **r3** are virtual frame slots. The VM engine decides how they are represented during execution.

Register-based bytecode is used in several important runtime systems. Lua is a well-known example of a register-based virtual machine. Android's older Dalvik VM also used a register-based bytecode model, in contrast with the stack-based JVM bytecode model. These designs were chosen partly because register bytecode can reduce instruction dispatch count and make some data dependencies explicit.

V8's interpreter tier also provides an important modern contrast. V8's Ignition interpreter is not a pure stack machine like the JVM model. It uses an accumulator-based bytecode design together with virtual registers. The accumulator is a special implicit working location, while other values can live in virtual registers. This is a hybrid style: not every operation names all operands, but the engine is still much closer to register/accumulator bytecode than to a simple operand-stack VM.

A simplified accumulator/register model looks like this:

```
LOAD a          -> accumulator
ADD r1          -> accumulator = accumulator + r1
STORE r2        -> r2 = accumulator
```

The accumulator acts as an implicit current value. Virtual registers hold additional temporary values, local values, or intermediate results. This design can reduce instruction width compared with a fully explicit three-address register machine while still avoiding some of the push/pop traffic of a pure stack machine.

PHP's Zend Engine is also better understood as an explicit-operand opcode engine than as a pure stack VM. Zend opcodes can refer to variables, temporaries, constants, and result locations. For example, an operation may conceptually say: take operand 1, take operand 2, place the result in a temporary. This makes PHP's engine structurally closer to explicit-address opcode execution than to the JVM operand-stack model.

A simplified PHP-like opcode shape is:

```
T1 = ADD CV(a), CV(b)
ASSIGN CV(x), T1
```

Here, CV can be read as a compiled variable slot, and T1 as a temporary. The exact Zend internals are more specific, but the broad structural point is that operands and result locations are explicitly represented in the opcode execution model.

Java is the important opposite example. JVM bytecode is stack-based. A Java method frame has local variable slots and an operand stack. Instructions commonly load values from local variables onto the operand stack, operate on the stack, and store results back. Therefore, Java belongs mainly with the previous subsection, not this one, although JIT compilation inside the JVM eventually maps bytecode behavior onto real machine registers and native instructions.

CPython also belongs mainly with the stack-based family. CPython bytecode execution uses an evaluation-stack model inside Python frames. Python operations push and pop references to Python objects. CPython may contain optimizations and implementation changes across versions, but the conceptual teaching model remains stack-oriented bytecode execution.

Bash does not fit either the stack VM or register VM model very well. Bash does not normally compile a shell script into a compact virtual instruction stream with operand stacks or virtual registers. It parses shell syntax, performs expansions, executes built-ins, and launches external commands. Its execution structure is command-dispatch-oriented rather than register-bytecode-oriented.

The register-based model is useful because it makes explicit one possible answer to the question: where do temporary values live inside a virtual machine? In a stack VM, temporary values live at positions on an evaluation stack. In a register VM, temporary values live in named virtual slots. Both are software structures. Both are inside the runtime. Neither should be confused with the programmer's source-level variables or the hardware's physical registers.

The general picture is:

```
source code
  |
  v
compiler / runtime compiler
  |
  v
virtual instructions
  |
  +--> stack model:
  |       operands live on evaluation stack
  |
  +--> register model:
  |       operands live in virtual registers
```

For students moving from C to Python, the main value of this distinction is conceptual. A C compiler eventually maps values onto real hardware registers, stack slots, and memory addresses. A process virtual machine first maps values onto its own virtual execution structures. In CPython, that

structure is mainly stack-oriented. In engines such as V8 or PHP's Zend Engine, explicit virtual locations or accumulator/register patterns play a stronger role. In all cases, the runtime inserts a software machine between the user's program and the physical CPU.

2.2.3 Comparative Runtime Case Studies

The Java Virtual Machine (JVM): Compiled Bytecode, Type Verification, Managed Memory, and the WORA Paradigm

The Java Virtual Machine, usually abbreviated as JVM, is one of the clearest examples of a process virtual machine. It is a runtime environment that executes programs represented as JVM bytecode, not as direct native machine code produced from the original Java source file.

In the usual Java workflow, the source file is not executed directly by the JVM. First, Java source code is compiled by the Java compiler:

```
javac Main.java
```

This produces one or more `.class` files:

```
Main.java
  |
  | javac
  v
Main.class
```

The `.class` file contains JVM bytecode. This bytecode is not x86-64 machine code, ARM64 machine code, or any other physical processor instruction set. It is an instruction stream for the Java Virtual Machine.

At runtime, the operating system loads a native executable such as the Java launcher and JVM runtime. The JVM then loads the class file, verifies it, prepares runtime structures, and executes the bytecode:

```
Main.class
  |
  | loaded by JVM
  v
verified JVM bytecode
  |
  | interpreted or JIT-compiled by JVM
  v
program behavior
```

This produces a layered model:

```
physical CPU
  executes native JVM machine code
  |
  v
JVM runtime process
  loads and executes JVM bytecode
  |
  v
Java program behavior
```

The JVM is therefore similar to CPython in one broad sense: the operating system runs the runtime process, and the runtime executes the user's program representation. But the JVM differs from CPython in several important ways.

The first major difference is that Java normally exposes source-to-bytecode compilation as a separate build step. In CPython, source-to-bytecode compilation usually happens automatically when a file is run or imported. In Java, the programmer usually compiles source files with `javac` before running the program with `java`.

A simplified comparison is:

CPython:

```
script.py
  |
  | CPython compiles internally
  v
CPython bytecode
  |
  | CPython interpreter
  v
execution
```

Java:

```
Main.java
  |
  | javac
  v
Main.class
  |
  | JVM
  v
execution
```

The second major difference is bytecode verification. Before JVM bytecode is allowed to run, the JVM checks that it obeys structural safety rules. This process is called **bytecode verification**. The verifier checks properties such as:

- whether bytecode instructions have valid structure;
- whether stack operations are type-consistent;
- whether local variables are used correctly;
- whether method calls match declared method signatures;
- whether control flow jumps to valid instruction locations;
- whether object initialization rules are respected.

This verification step is important because JVM bytecode may come from files, libraries, downloaded archives, build tools, or other compilers targeting the JVM. The JVM cannot blindly trust every bytecode file. It must check that the bytecode does not violate the rules of the virtual machine before executing it.

The JVM is a stack-based virtual machine. Each method call creates a JVM frame. A JVM frame contains local variable slots and an operand stack. Instructions load values from local variables onto the operand stack, operate on those values, and store results back.

A simplified JVM method frame looks like this:


```
JVM method frame
+-----+
| local variable slots |
+-----+
| operand stack        |
+-----+
| runtime metadata     |
+-----+
```

For example, an expression such as:

```
x = a + b;
```

can be represented conceptually as:

```
load a onto operand stack
load b onto operand stack
add top two stack values
store result into x
```

The operand stack is not the native process stack. It is part of the JVM's method-frame model. The physical CPU does not directly execute these stack operations as Java operations. The JVM interpreter or JIT-compiled code implements them.

The JVM also owns Java memory management. Java objects are allocated on a managed heap. The programmer normally creates objects with `new`, but does not manually release them with something like C's `free()`.

For example:

```
Person p = new Person();
```

This creates a Java object managed by the JVM. When the object is no longer reachable, the JVM's garbage collector may reclaim its memory. The programmer does not explicitly destroy the object.

The managed-memory model can be sketched as:

```
Java reference variable
    |
    v
object on JVM-managed heap
    |
    v
reclaimed later by garbage collector when unreachable
```

This is one of the strongest contrasts with C. In C, the programmer may allocate with `malloc()` and release with `free()`. In Java, ordinary object memory is managed by the virtual machine. This reduces many manual memory errors, such as double free and use-after-free, but it does not eliminate all memory problems. A Java program can still keep references to objects it no longer needs, preventing them from being collected.

The JVM also provides a structured exception model. Exceptions are objects. When an exception is thrown, the JVM searches through active method frames for a matching handler. If no handler is found, the exception propagates upward and may terminate the thread. This is runtime-managed control flow, similar in broad structure to Python exceptions, but governed by Java's type system and JVM execution rules.

A simplified exception path is:

```

method C throws exception
    |
    v
JVM checks handler in method C
    |
    v
if not handled, unwind to method B
    |
    v
if not handled, unwind to method A

```

The JVM is also closely associated with the phrase **WORA**: *Write Once, Run Anywhere*. The idea is that Java source code can be compiled into JVM bytecode once, and that bytecode can run on any platform that has a compatible JVM.

The intended portability model is:

```

Java source
  |
  | javac
  v
JVM bytecode
  |
  +--> JVM on Linux x86-64
  |
  +--> JVM on Windows x86-64
  |
  +--> JVM on macOS ARM64
  |
  +--> JVM on other supported platforms

```

The same bytecode does not run directly on all processors. Instead, each platform provides its own native JVM implementation. The bytecode targets the JVM specification, and the JVM implementation targets the local operating system and processor.

This is different from C portability. C source code can often be recompiled for different platforms, but the compiled binary is normally platform-specific. A Linux x86-64 executable is not the same as a Windows x86-64 executable or a macOS ARM64 executable. Java moves the portability boundary from source code to bytecode.

A C-style model is:

```

C source
  |
  +--> compile for Linux x86-64  -> Linux binary
  |
  +--> compile for Windows x86-64 -> Windows binary
  |
  +--> compile for macOS ARM64   -> macOS binary

```

A Java-style model is:

```

Java source
  |
  | javac
  v
JVM bytecode
  |

```

```
+--> run on JVM for Linux
|
+--> run on JVM for Windows
|
+--> run on JVM for macOS
```

This does not mean that all Java programs are perfectly platform-independent. Programs can still depend on platform-specific file paths, native libraries, environment variables, GUI behavior, operating-system services, or external resources. But the bytecode execution model is designed to make the compiled program representation independent of one physical CPU instruction set.

The JVM also commonly uses just-in-time compilation. The JVM may begin by interpreting bytecode, but frequently executed methods or code paths can be compiled into native machine code while the program is running. This allows the JVM to combine portability with high performance.

A simplified JVM execution path is:

```
JVM bytecode
|
+--> interpreted execution
|
+--> profiling detects hot code
      |
      v
    JIT compiler emits native machine code
      |
      v
    CPU executes optimized native code
```

This makes the JVM more aggressive than the traditional CPython execution model. CPython normally interprets bytecode through its evaluation loop. The JVM often transforms selected bytecode into native machine code at runtime. The JVM can use runtime information to optimize code paths that are actually important during execution.

The JVM runtime therefore combines several responsibilities:

- loading classes;
- verifying bytecode;
- creating method frames;
- executing bytecode;
- managing the heap;
- collecting unreachable objects;
- routing exceptions;
- resolving symbolic references;
- compiling hot paths to native code through JIT compilation.

This makes the JVM a rich managed runtime, not just a simple bytecode interpreter.

For the purposes of this course, the JVM is useful mainly as a contrast point. It shows a runtime-hosted model where bytecode is explicit, verification is central, memory is managed, and JIT compilation is a normal performance strategy. CPython shares the idea of bytecode executed inside a native runtime process, but it has a different object model, different type timing, different bytecode stability guarantees, and a much less aggressive default JIT story.

The JVM therefore represents one major process-VM architecture: a portable bytecode target with verification, managed memory, runtime class loading, and adaptive native compilation. It demonstrates that a virtual machine is not merely an interpreter. It can be a complete execution environment that defines its own instruction set, memory model, safety checks, object lifecycle rules, and performance strategy.

The JavaScript V8 Engine: High-Performance Runtime Optimization, JIT Compilation, and Host-Provided Event Loops

The JavaScript V8 engine is a process-VM-style execution engine designed for running JavaScript with high runtime performance. It is best known as the engine used by Chromium-based browsers and by Node.js. It also appears indirectly inside Electron-style desktop applications, where a native application embeds a browser/runtime stack. Applications such as Visual Studio Code use this general architecture: the operating system runs the application process, while JavaScript or TypeScript-derived code executes inside an embedded JavaScript runtime environment.

V8 is not usually the entire application. It is an engine embedded inside a host. The host provides the surrounding environment: browser APIs, DOM access, timers, networking interfaces, file-system APIs in Node.js, event-loop integration, and application-specific services. V8 itself is responsible for parsing JavaScript, representing code internally, executing it, optimizing it, managing JavaScript objects, and collecting unused memory.

A simplified browser-side model is:

```

browser executable
  |
  v
browser process / renderer process
  |
  v
embedded V8 engine
  |
  v
JavaScript source from page or application
  |
  v
JavaScript execution inside browser-provided environment

```

A simplified Node.js model is:

```

node executable
  |
  v
Node.js runtime process
  |
  v
embedded V8 engine
  |
  v
JavaScript source / module graph
  |
  v
JavaScript execution with Node-provided APIs

```

In both cases, the operating system does not directly execute a `.js` file as a native binary. The operating system runs the host executable: a browser, the Node.js executable, or an Electron application. That host embeds V8. V8 then executes JavaScript code inside the host process.

This makes V8 different from CPython in an important way. CPython is normally used directly as the interpreter executable:

```
python script.py
```

Here, the Python interpreter is the visible runtime process. With V8, the engine is commonly hidden inside another program:

```
chrome / chromium / node / electron application
      |
      v
embedded V8 engine
      |
      v
JavaScript execution
```

The user may experience this as a web page, a server process, a command-line JavaScript program, or a desktop application. In all cases, V8 is the engine that gives JavaScript its execution machinery.

V8 does not simply interpret JavaScript source line by line. Modern JavaScript execution is too dynamic and too performance-sensitive for such a simple model. V8 parses source code, builds internal representations, executes code through interpreter tiers, observes runtime behavior, and compiles frequently executed paths into native machine code. This is called **just-in-time compilation**, or **JIT compilation**.

A simplified V8 execution pipeline can be sketched as:

```
JavaScript source
  |
  v
parser
  |
  v
internal representation / bytecode
  |
  v
interpreter or baseline execution
  |
  v
runtime profiling feedback
  |
  v
optimized native code for hot paths
  |
  v
deoptimization if assumptions fail
```

The important idea is that V8 does not need to know everything before execution begins. It can start running code, observe what actually happens, and then optimize based on that observed behavior. If a function is called many times with the same kinds of values, V8 may specialize the machine code for that pattern. If later calls violate the assumptions used by the optimizer, V8 can abandon the optimized code and fall back to a safer representation. This fallback is called **deoptimization**.

For example, consider JavaScript code such as:

```
function add(a, b) {
    return a + b;
}
```

At the source level, the operator `+` is not fixed to one machine operation. It may mean numeric addition, string concatenation, or another behavior depending on runtime values. If V8 observes that `add` is repeatedly called with numbers, it may optimize for numeric addition. If the function is later called with strings, the optimized assumptions may no longer be valid.

This is different from C. In C, if the parameters are declared as integers, the compiler already knows the operation category before execution:

```
int add(int a, int b) {  
    return a + b;  
}
```

The C compiler can generate native integer-addition code ahead of time. JavaScript's dynamic value model requires more runtime observation. V8 recovers performance by using feedback from real execution.

This is also different from traditional CPython. CPython usually executes bytecode through its evaluation loop and performs dynamic dispatch for object operations. V8, by contrast, aggressively tries to turn frequently used JavaScript paths into native machine code. This makes V8 a much more optimization-heavy runtime than the standard CPython execution model.

The reason this works is that many dynamic programs behave regularly in practice. A function may be legally able to receive many types of values, but in a real program it often receives the same shapes of objects and the same primitive categories repeatedly. V8 exploits that regularity.

A central optimization concept in JavaScript engines is the internal representation of object structure. JavaScript objects are dynamic: properties can be added, removed, or changed at runtime. A naive implementation would treat every property access as a dictionary-like lookup. That would be slow. V8 instead tracks object layout patterns internally. If many objects have the same property structure, the engine can optimize property access based on that stable shape.

For example:

```
function Point(x, y) {  
    this.x = x;  
    this.y = y;  
}  
  
let p = new Point(10, 20);
```

If many `Point` objects are created with the same property layout, V8 can treat property access such as:

```
p.x
```

as a predictable offset-like operation internally, rather than as a fully general dictionary lookup every time. The source language remains dynamic, but the engine tries to discover stable structure during execution.

This is the general pattern of high-performance JavaScript engines:

- accept a highly dynamic language at the source level;
- start execution with a general representation;
- collect runtime type and shape feedback;
- generate optimized native code when behavior becomes predictable;
- discard optimized code if later behavior breaks the assumptions.

V8 also owns JavaScript memory management. JavaScript objects live on a managed heap controlled by the engine. The programmer creates objects, arrays, functions, closures, promises, and other values, but does not manually free them. V8 uses garbage collection to reclaim objects that are no longer reachable.

A simplified JavaScript object-lifetime model is:

```
JavaScript variable / property / closure reference
      |
      v
object on V8-managed heap
      |
      v
garbage collector reclaims it when unreachable
```

This resembles Java and Python at the broad runtime level: objects are managed by the runtime, not by explicit user calls to `free()`. The details differ. CPython primarily uses reference counting plus a cyclic garbage collector. The JVM uses garbage collectors designed for Java heap management. V8 uses garbage collection strategies adapted to JavaScript's allocation patterns and object model.

One of the most important architectural points about V8 is that the event loop is not V8 alone. JavaScript programmers often associate JavaScript with asynchronous execution, timers, callbacks, promises, and event loops. However, the event loop is provided by the host environment around the engine.

In a browser, the browser host provides:

- timers such as `setTimeout`;
- event dispatch for user input;
- network operations;
- DOM events;
- rendering integration;
- task and microtask scheduling behavior.

In Node.js, the Node runtime provides:

- file-system operations;
- networking;
- timers;
- process APIs;
- event-loop integration;
- module loading;
- native bindings.

V8 executes JavaScript code. The host provides the surrounding world in which JavaScript code receives events and performs I/O.

This distinction matters. A JavaScript expression such as:

```
let x = 1 + 2;
```

is pure JavaScript execution handled by the engine. But a call such as:

```
setTimeout(callback, 1000);
```

depends on host-provided timer functionality. The callback is a JavaScript function object, but the timer mechanism itself belongs to the host environment.

A simplified event model is:

```
host environment
  receives external event or timer completion
    |
    v
queues callback / task
  |
  v
event loop selects runnable task
  |
  v
V8 executes JavaScript callback
```

This is why the same V8 engine can be used in different environments. In a browser, JavaScript code interacts with the DOM and web APIs. In Node.js, JavaScript code interacts with files, sockets, processes, and server-side APIs. In Electron, JavaScript code may interact with both browser-like and Node-like capabilities depending on configuration.

The host/engine split can be drawn as:

```
native host application
+-----+
| browser / node / electron |
|                             |
| host APIs: timers, I/O, DOM, filesystem, etc. |
|                             |
| +-----+ |
| | V8 engine | |
| | parser, bytecode, JIT, GC, JS heap | |
| +-----+ |
|                             |
+-----+
```

This architecture explains why JavaScript is not only a language but also an embedded runtime component. The same language engine can appear inside different native programs, each providing different external capabilities.

Compared with the JVM, V8 is usually more tightly tied to a dynamic source language and to host-driven evented execution. Java normally compiles source into class files before execution, verifies bytecode, and runs inside a JVM with a strongly specified bytecode model. V8 accepts JavaScript source from a host, compiles and optimizes dynamically, and interacts heavily with event loops supplied by that host.

Compared with CPython, V8 is much more JIT-centered. CPython traditionally prioritizes predictable bytecode interpretation and a rich object model. V8 prioritizes aggressive adaptive optimization because JavaScript performance in browsers and server environments is critical. V8 therefore spends significant runtime effort observing code, generating optimized native paths, and invalidating those paths when assumptions fail.

The trade-off is complexity. A simple bytecode interpreter is easier to reason about: fetch instruction, dispatch operation, update runtime state. A JIT engine has many more moving parts: profiling, inline caches, object-shape tracking, optimized code generation, deoptimization, garbage collection coordination, and host event integration. This complexity exists to make a highly dynamic language run fast in demanding environments.

For this course, V8 is useful mainly as a contrast to CPython. Both are native runtimes that execute high-level source programs inside a process. Both manage objects and memory. Both can expose dynamic language behavior. But V8 is architected around adaptive runtime optimization and host embedding, while CPython is best understood first as a bytecode interpreter with runtime-managed frames and objects.

A concise comparison is:

Runtime	Main execution style	Important surrounding structure
CPython	Bytecode interpretation	Python frames, object model, reference counting, import system
JVM	Bytecode execution plus JIT	Class loading, verification, managed heap, garbage collection
V8	Adaptive interpretation and JIT optimization	Host APIs, event loop, JavaScript heap, object-shape feedback

Thus, the V8 engine demonstrates a different process virtual machine philosophy. The user program is still not a native executable loaded directly by the operating system. It is source input to an engine embedded inside a native host. But unlike a simple interpreter, V8 aggressively transforms runtime observations into optimized machine code. Its design shows how a virtual machine can act not only as an interpreter, but as a dynamic optimizer that continuously adapts the program representation while execution is already in progress.

The CPython Virtual Machine: Bytecode Interpretation, Runtime Frames, Dynamic Inspection, and Managed Object Allocation

CPython is the reference implementation through which Python is most commonly understood. It is a native program, written mostly in C, that implements the Python language by compiling Python source code into bytecode and then executing that bytecode inside a runtime-managed environment.

The name *CPython virtual machine* refers to this internal execution environment. It is not a virtual machine in the sense of a full hardware emulator. It does not boot an operating system and it does not create a separate virtual computer. It is a process virtual machine: a software execution engine running inside the native CPython process.

A simplified CPython execution path is:

```

Python source file
|
v
tokenizer and parser
|
v
abstract syntax tree
|
v
compiler
|
v
code object containing bytecode
|
v
CPython bytecode evaluation loop
|

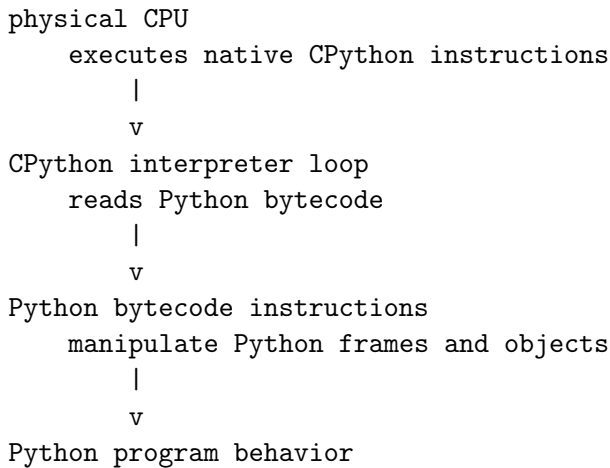
```

v

Python runtime behavior

The bytecode is the instruction stream of the CPython virtual machine. It is not hardware machine code. A physical CPU cannot directly execute Python bytecode. Instead, the CPU executes the native machine code of the CPython interpreter. That native interpreter code reads bytecode instructions and performs the corresponding Python-level operations.

This gives CPython a layered structure:



For example, a Python statement such as:

```
x = a + b
```

is not normally translated into a fixed native addition instruction in the way a C compiler might translate integer addition. CPython compiles the statement into bytecode operations that load the objects bound to **a** and **b**, perform Python's addition protocol, and bind the result to the name **x**. The addition operation is dynamic: the runtime must consider the actual objects involved.

If **a** and **b** are integers, the operation follows integer addition behavior. If they are strings, the same source-level **+** operator may mean concatenation. If they are user-defined objects, the operation may dispatch through special methods such as `__add__`. The bytecode instruction is therefore not equivalent to a single hardware instruction. It is an entry point into Python's object protocol.

This is one of the main differences between CPython and the C execution model. In C, much operation meaning is fixed by declared types before execution. In CPython, many operation meanings are resolved while the program runs, based on the objects that are actually present.

A simplified comparison is:

C:

```

declared types
  -> compiler chooses native operation
  -> CPU executes operation directly

```

CPython:

```

runtime objects
  -> interpreter checks object types/protocols
  -> runtime performs appropriate operation

```

CPython bytecode is organized around code objects. A code object stores compiled executable content and metadata. It may contain bytecode, constants, variable names, names used for global lookup, information about local variables, and other details needed by the interpreter.

For example, a function definition such as:

```
def add(a, b):
    return a + b
```

creates a function object. That function object refers to a code object representing the body of the function. The function object is callable. The code object contains the compiled instructions that run when the function is called.

A simplified structure is:

```
function object
  |
  v
code object
  |
  v
bytecode instructions and metadata
```

When a Python function is called, CPython creates or activates a runtime frame for that call. A frame represents an active execution of code. It contains the state needed to continue execution: local bindings or local slots, links to global and built-in namespaces, the current bytecode position, and the evaluation stack used by bytecode instructions.

A simplified Python frame can be represented as:

```
Python execution frame
+-----+
| code object                |
| current bytecode position  |
| local variable storage     |
| global namespace reference |
| builtins namespace reference |
| evaluation stack           |
| previous frame link        |
+-----+
```

This frame is not the same thing as a native C stack frame. CPython itself uses native C stack frames internally because it is a C program. But Python-level function calls are represented by interpreter-managed frame state. A Python local name is not simply a raw C stack variable. It is part of the Python runtime's frame and namespace machinery.

For example:

```
def f():
    x = 10
    y = x + 5
    return y
```

When `f()` runs, CPython manages the names `x` and `y` as Python-level local bindings. The value 10 is a Python integer object. The result of `x + 5` is also a Python object. The frame records the execution state that connects the bytecode, the local names, and the objects.

The evaluation stack is especially important. CPython is usually understood as a stack-based bytecode interpreter. Bytecode instructions push object references onto the evaluation stack, pop them as operands, and push results back.

A simplified bytecode-style evaluation of:

```
x = a + b
```

can be imagined as:

```

LOAD a      -> push object bound to a
LOAD b      -> push object bound to b
ADD         -> pop two objects, compute result, push result
STORE x     -> pop result and bind it to x

```

The actual bytecode instruction names vary between Python versions, but the structural idea is stable: bytecode manipulates Python object references inside a runtime frame.

CPython's object model is also runtime-owned. Python values are represented as objects. Integers, floats, strings, lists, dictionaries, functions, modules, classes, exceptions, and types are all runtime objects. These objects live in memory managed by CPython.

At the C implementation level, Python objects share a common object-header idea. A Python object carries information such as a reference count and a pointer to its type object. The exact structure depends on the object kind and CPython version, but the conceptual model is:

Python object

```

+-----+
| reference count          |
| pointer to type object   |
| object-specific data     |
+-----+

```

The type object tells CPython how that object behaves. It defines or points toward operations such as representation, arithmetic behavior, attribute lookup, calling behavior, item access, iteration, comparison, and memory deallocation.

This is the runtime basis of the phrase “everything is an object.” The statement does not mean that all objects have the same internal layout. It means that values participating in Python execution are represented through a uniform object model. Even primitive-looking values such as integers are Python objects.

For example:

```
x = 100
```

does not create a C-style local integer box named `x`. It binds the name `x` to a Python integer object. The name is part of a namespace or local frame. The integer object is managed by CPython.

This is why assignment in Python is binding, not copying into a typed variable slot:

```

a = [1, 2, 3]
b = a

```

After this code, `a` and `b` refer to the same list object. CPython has not duplicated the list merely because a second name was assigned. It has created another reference to the same runtime object.

A simplified binding diagram is:

```

a ----\
      v
    list object [1, 2, 3]
      ^
b ----/

```

This object-reference model is central to Python's memory behavior, argument passing, mutation semantics, and container behavior.

CPython also owns ordinary memory reclamation for Python objects. In C, a programmer may allocate memory with `malloc()` and release it with `free()`. In Python, normal user code does not manually free every object. CPython tracks references to objects and reclaims memory when objects are no longer reachable according to its memory-management rules.

The primary mechanism in CPython is reference counting. Each object tracks how many active references point to it. When the reference count drops to zero, the object can be destroyed immediately. CPython also includes garbage collection machinery for certain cyclic structures, because reference counting alone cannot reclaim objects that refer to each other in cycles.

A simple acyclic case is:

```
x = [1, 2, 3]
x = None
```

The first line binds `x` to a list object. The second line rebinds `x` to `None`. If no other references to the list exist, the list becomes reclaimable.

A cyclic case is more difficult:

```
a = []
a.append(a)
```

The list refers to itself. Even if the external name `a` is removed, the object participates in a reference cycle. CPython's cyclic garbage collector exists to handle such cases.

This managed allocation model is one of the major differences between CPython and C. The Python programmer still needs to understand object lifetimes, aliases, references, and resources, but not ordinary manual deallocation of every heap object.

CPython also exposes dynamic inspection facilities. Because code, functions, types, modules, frames, and objects are runtime structures, they can be inspected while the program is running.

For example:

```
type(x)
id(x)
dir(x)
```

These are not merely debugging conveniences. They reveal the fact that Python execution is represented in runtime metadata. The object has a type. The object has an identity. The runtime can enumerate available attributes. A function has a `__code__` object. A module has a namespace. A class has a dictionary-like attribute structure.

Python can also expose bytecode. The `dis` module can display the bytecode generated for a function. This allows the programmer to see that source code has been compiled into virtual instructions before execution.

For example:

```
import dis

def add(a, b):
    return a + b

dis.dis(add)
```

The exact output depends on the Python version, but the purpose is stable: the tool shows the virtual instruction stream used by CPython to execute the function.

Dynamic inspection is not free. It exists because CPython maintains rich runtime structures. Those structures make the language flexible, debuggable, introspective, and interactive, but they also add overhead compared with direct native execution.

This helps explain Python's performance profile. CPython is usually slower than optimized C for CPU-bound numerical loops because each Python-level operation involves runtime object handling, bytecode dispatch, dynamic type behavior, reference management, and possible exception checks. A C loop over raw integers can often become a compact sequence of native instructions operating on registers and memory. A Python loop over Python integers involves repeated interpreter-level work.

A simplified contrast is:

C integer addition:

```
load raw integer values
execute native add instruction
store raw result
```

Python integer addition in CPython:

```
load object references
check runtime types / dispatch operation
allocate or retrieve result object
update references
continue bytecode evaluation
```

This overhead is the cost of CPython's runtime flexibility. The benefit is that Python values are uniform objects, types can be inspected dynamically, names can be rebound, functions and classes can be created at runtime, modules can be imported dynamically, exceptions carry traceback objects, and interactive execution is natural.

Compared with the JVM, CPython is less centered on bytecode verification and aggressive JIT compilation. The JVM usually executes statically typed bytecode, verifies it, manages a heap, and often compiles hot code paths into native machine code. CPython bytecode is more closely tied to the CPython implementation and is usually interpreted by the evaluation loop.

Compared with V8, CPython is also less optimization-tier-driven in the traditional model. V8 aggressively profiles JavaScript execution and generates optimized native code for hot paths. CPython's first conceptual model is simpler: source becomes bytecode, bytecode is executed by a predictable interpreter loop, and Python operations are resolved through the runtime object system.

Compared with PHP's Zend Engine, CPython shares the idea of compiling source into runtime instructions and executing them inside an engine. PHP uses opcodes and request/runtime symbol tables. CPython uses code objects, bytecode, frames, namespaces, and Python objects. The broad architecture is similar, but the object model, lifecycle model, and execution environment differ.

Compared with Bash, CPython is much more bytecode-VM-like. Bash interprets command syntax and frequently dispatches external native programs as child processes. CPython executes Python statements mostly inside one interpreter process by evaluating bytecode and manipulating Python objects.

The CPython virtual machine can therefore be summarized as:

source code

```
-> AST
-> code object with bytecode
-> interpreter-managed frame
-> evaluation stack of object references
-> operations through Python object model
-> managed memory and exception routing
```

For students coming from C, the main transition is this: Python execution does not expose raw machine-level storage as the primary programming model. CPython owns the execution state. Variables are bindings to objects. Function calls create runtime frames. Operations dispatch through object types and protocols. Memory is managed by the interpreter. Bytecode, frames, namespaces, and objects are the real machinery behind ordinary Python source code.

The Pure Interpretation vs. JIT Compilation Boundary: Why CPython Prioritizes a Predictable, Linear Dispatch Loop over High-Overhead Runtime JIT Machine Code Emission

CPython is usually described as a bytecode interpreter. This means that Python source code is compiled into CPython bytecode, and that bytecode is executed by an interpreter loop inside the CPython process. The interpreter repeatedly reads the next bytecode instruction, dispatches to the native C implementation of that instruction, updates the current frame state, and continues.

A simplified view is:

```

Python source
  |
  v
CPython bytecode
  |
  v
interpreter dispatch loop
  |
  v
C implementation of each bytecode operation
  |
  v
Python runtime state changes

```

This is different from a just-in-time compiler. A JIT compiler observes or receives runtime code and emits native machine code while the program is already running. Instead of interpreting each virtual instruction one by one, the runtime may generate native instruction sequences for selected functions, loops, or hot paths. The CPU can then execute that generated machine code directly.

A simplified JIT model is:

```

source or bytecode
  |
  v
runtime profiling / analysis
  |
  v
JIT compiler emits native machine code
  |
  v
CPU executes generated native code

```

The difference is not that one model uses the CPU and the other does not. Both use the CPU. The difference is what the CPU is executing. In CPython's traditional model, the CPU executes the interpreter's native code, and the interpreter executes Python bytecode. In a JIT model, the runtime may generate new native code representing parts of the user's program, and the CPU can execute that generated code more directly.

This distinction can be summarized as:

Interpreter-centered execution	JIT-centered execution
Bytecode remains the main executed representation	Hot bytecode paths may become native machine code
Each virtual instruction is dispatched by the interpreter	Repeated dispatch can be reduced or removed for compiled paths
Runtime behavior stays close to the bytecode/frame/object model	Runtime may build optimized native traces or functions
Simpler to debug, inspect, port, and maintain	More complex optimizer, guards, invalidation, and deoptimization machinery
Predictable startup behavior	Extra compilation cost during execution

Historically, CPython has prioritized the interpreter-centered model. The central execution idea is a dispatch loop over bytecode instructions. This loop is sometimes introduced pedagogically as a large `switch`-style structure: inspect the current opcode, jump to the implementation of that opcode, execute it, then continue to the next instruction. Actual CPython implementations may use more

efficient dispatch techniques, generated interpreter code, computed gotos, inline caches, and specialized instructions depending on version and build configuration. The teaching point is not the literal C syntax of the loop. The teaching point is that CPython traditionally advances Python execution by dispatching bytecode operations through interpreter-controlled machinery.

A simplified interpreter loop can be imagined as:

```
while running:
    opcode = next_instruction()
    dispatch opcode:
        LOAD_FAST:
            ...
        STORE_FAST:
            ...
        CALL:
            ...
        RETURN_VALUE:
            ...
```

This model is linear and explicit. The runtime keeps a current frame, a current bytecode offset, an evaluation stack, local bindings, global and built-in namespace links, and references to Python objects. Each bytecode instruction modifies this interpreter-managed state.

For example, a Python expression such as:

```
x = a + b
```

does not normally become one native arithmetic instruction. It becomes bytecode-level work:

```
load object bound to a
load object bound to b
perform Python addition protocol
bind result to x
```

The addition operation is dynamic. The runtime must know what objects `a` and `b` refer to at that exact moment. They might be integers, floats, strings, lists, user-defined objects, or objects implementing special methods. The source operator `+` is therefore not enough to select one fixed hardware instruction ahead of time.

This dynamic object model makes aggressive JIT compilation harder. A JIT compiler must generate native code that is correct for the runtime objects actually observed. If it assumes that `a` and `b` are integers, it must include guards or checks to make sure that assumption remains true. If a later call uses different object types, the optimized machine code may no longer be valid and the runtime must fall back to a more general path.

A JIT path therefore often needs:

- profiling information about actual runtime values;
- guards that test whether optimization assumptions still hold;
- fallback paths for unexpected object types;
- deoptimization machinery to return from optimized native code to interpreter state;
- memory-management integration with the runtime heap;
- exception integration with the runtime frame model;
- debugging and tracing compatibility.

This is not impossible. Engines such as V8 and the JVM do it aggressively. But it is complex. The runtime must not only generate fast code; it must also preserve the exact language behavior when assumptions fail.

CPython's object model creates additional pressure against simple native compilation. Python code can inspect objects dynamically, modify classes, replace methods, monkey-patch modules, intercept attribute access, override operators, use tracing hooks, inspect frames, catch exceptions with tracebacks, and call into C extension modules. These features mean that the runtime must preserve a very rich execution model.

For example, the meaning of:

```
obj.x
```

may involve ordinary attribute lookup, descriptors, properties, `__getattr__`, `__getattribute__`, class dictionaries, instance dictionaries, slots, or extension-level behavior. A native optimizer can accelerate common cases, but it must still respect all these dynamic mechanisms.

The C extension ecosystem is another major constraint. A large part of Python's practical power comes from native extensions: numerical libraries, database bindings, image libraries, cryptography libraries, machine-learning frameworks, and many system-level modules. These extensions interact with CPython's object model and C API. A radically different execution strategy must still cooperate with the expectations of this ecosystem.

This is one reason CPython's default strategy has historically favored compatibility, debuggability, portability, and predictable semantics over aggressive runtime native-code generation. The interpreter loop is not the fastest possible execution architecture, but it is a stable center around which the rest of the runtime is built.

This does not mean CPython performs no optimization. Modern CPython has added adaptive specialization mechanisms that improve common bytecode paths while staying close to the interpreter model. Instead of replacing the whole execution model with a full aggressive JIT, CPython can specialize bytecode operations based on observed runtime behavior. For example, a generic operation may be specialized when the interpreter repeatedly sees the same common object types or access patterns.

Conceptually:

```
generic bytecode operation
|
| runtime observes common pattern
v
specialized interpreter operation
|
v
faster path inside interpreter architecture
```

This is different from full JIT compilation. The runtime may optimize dispatch and operation handling, but the program is still primarily executing through interpreter-managed bytecode machinery rather than through large generated native code regions.

The contrast with V8 is useful. JavaScript is also dynamic, but V8 was designed around aggressive adaptive optimization because browser and server-side JavaScript performance became critical. V8 invests heavily in profiling, hidden classes or shape-like structures, inline caches, optimized native code, and deoptimization. The result can be very fast execution for hot JavaScript paths, but at the cost of a much more complex engine.

A simplified V8-style model is:

```
JavaScript source
|
v
bytecode / baseline execution
```

```

    |
    v
profiling feedback
    |
    v
optimized native code for hot paths
    |
    v
deoptimization if assumptions fail

```

The JVM is another contrast. Java bytecode is usually more statically structured than Python bytecode. Java's type system, bytecode verifier, class metadata, and method signatures give the runtime more stable information before execution. The JVM can use this structure, plus runtime profiling, to compile hot methods into native code.

A simplified JVM model is:

```

JVM bytecode
    |
    v
verification and interpretation
    |
    v
profiling
    |
    v
JIT compilation of hot methods
    |
    v
native execution with managed-runtime support

```

CPython starts from a different set of priorities. Python's runtime behavior is highly dynamic, its C extension ecosystem is large, and its introspection features are central to how programmers use the language. A simple statement can require many runtime decisions. Therefore, the interpreter remains the natural baseline.

This baseline has practical advantages:

- startup can be simpler because there is no need to compile large native regions before running;
- debugging and tracing can remain close to the bytecode/frame model;
- bytecode inspection remains meaningful;
- portability is easier because the runtime does not need a production-grade native code generator for every target architecture;
- C extension compatibility remains central;
- memory management and exception routing stay concentrated in the interpreter runtime.

The cost is CPU-bound performance. A tight Python loop performing arithmetic on ordinary Python objects usually cannot compete with optimized C, JVM JIT-compiled code, or V8-optimized JavaScript in hot paths. Each Python-level operation involves bytecode dispatch, object handling, dynamic type behavior, reference management, and error checking.

For example:

```

total = 0
for i in range(1000000):
    total += i

```

At the Python level, this loop looks simple. At the CPython runtime level, it involves repeated iteration protocol operations, integer object operations, name or local-slot access, bytecode dispatch, and reference management. A C compiler can often reduce a similar loop over machine integers to a compact native instruction sequence. CPython preserves Python’s dynamic semantics, and that preservation has runtime cost.

This is why Python performance practice often moves CPU-heavy inner loops into native libraries. A Python program may call NumPy, PyTorch, OpenCV, database engines, compression libraries, or other native components. The Python layer coordinates work, while optimized native code performs heavy computation.

The architecture then becomes:

```

Python code
  |
  | CPython interpreter
  v
library call
  |
  v
native extension / optimized C, C++, Fortran, CUDA, etc.
  |
  v
fast computation outside ordinary Python bytecode loop

```

This is not an accident. It is one of the practical ways Python combines high-level flexibility with native performance where needed.

The phrase *pure interpretation* should therefore be used carefully. CPython is not merely reading source text line by line. It compiles source to bytecode. It uses specialized interpreter mechanisms. Recent and experimental builds may include JIT-related work. But the default conceptual model remains interpreter-centered: bytecode is executed through CPython’s runtime machinery, and Python operations are resolved through the object system.

A precise teaching statement is:

Traditional CPython is not a source-line interpreter and not primarily a production JIT compiler. It is a bytecode interpreter with a rich runtime object model, adaptive interpreter optimizations, and an implementation strategy that favors compatibility and predictable runtime semantics over aggressive default machine-code generation.

This boundary is important for students moving from C to Python. In C, the compiler tries to remove as much runtime uncertainty as possible before execution. In V8 or the JVM, the runtime may recover speed by compiling hot paths into native code while execution proceeds. In CPython, the main execution path remains close to bytecode, frames, objects, and interpreter dispatch. Understanding this explains both Python’s flexibility and its performance limits.

2.3 Architectural Roadmaps of Modern Procedural Extensions

2.4 Architectural Roadmaps of Modern Procedural Extensions

The previous sections established two execution models: the native compiled process, where control flow is mostly expressed through machine-level calls, stack frames, jumps, and returns; and the runtime-hosted model, where the language implementation owns bytecode, frames, objects, namespaces, managed memory, and sometimes task scheduling. This section studies the programming constructs that become possible once procedures are no longer treated only as fixed source-level blocks called by name.

We call these mechanisms **procedural extensions** because they extend the basic procedure/function model. The classical procedure starts, receives arguments, executes on the call stack, and returns

once. Callbacks, anonymous functions, closures, generators, coroutines, and asynchronous tasks all modify this simple picture. A function may be passed as a value, created inline, stored for later, combined with captured state, suspended before completion, resumed later, or scheduled by an event loop.

These mechanisms are not limited to procedural programming. They appear inside object-oriented systems, functional programming systems, scripting languages, web runtimes, GUI frameworks, and server frameworks. In object-oriented programming, callbacks often appear as listener objects, methods, lambdas, or route handlers. In functional programming, functions as values, anonymous functions, lexical closures, and higher-order transformations are central ideas. In runtime-managed languages such as Python and JavaScript, these ideas become natural because functions and execution contexts are already represented as runtime objects.

The reason these constructs belong together is that they all depend on separating executable behavior from a simple fixed call site. A callback lets another component call user code later. An anonymous function lets small executable logic be created at the point of use. A closure lets executable logic carry preserved lexical state. A generator preserves a suspended frame so values can be produced lazily. A coroutine generalizes suspension into cooperative task switching. Asynchronous programming uses these same ideas to resume work only when external I/O events become ready.

These ideas also returned to C and C++ in different forms. C has always had function pointers, which support callbacks, but it does not have standard closures or stack-preserving coroutines. C programmers emulate them with explicit context pointers, structs, state machines, callback tables, and sometimes control-flow tricks such as Duff's Device-style switch-based replay. C++ went further: function objects, lambdas, captures, `std::function`, futures, and modern coroutines brought many runtime-style control-flow ideas back into a compiled language, though with different implementation rules and stronger compile-time structure.

The central technical theme is stack management. A normal function call lives on the active stack and disappears when it returns. These procedural extensions challenge that rule. A closure keeps selected variables after the creating frame is gone. A generator suspends a function body without finishing it. A coroutine gives control back to a scheduler and later resumes. An async task records what should happen after an I/O result arrives. In each case, some part of the execution state must be stored outside the ordinary transient call frame: in heap objects, closure cells, state structs, runtime frames, promises, tasks, fibers, or explicit user-defined state machines.

This section therefore links programming style to execution architecture. These constructs are not merely syntactic conveniences. They are ways of reshaping control flow, lifetime, and ownership of execution state. Understanding them requires keeping the native stack, the runtime frame model, heap-preserved state, and scheduler/event-loop behavior separate.

2.4.1 Functions as Runtime Values and Saved Execution Context

The previous sections described runtime-hosted execution from the outside: the operating system runs a native interpreter or virtual machine, and the user's program is represented inside that runtime as bytecode, frames, objects, opcodes, or other managed structures. This subsection moves one level higher and examines what becomes possible when executable code itself can be treated as a runtime value.

In the traditional C model, a function is primarily a compiled region of machine instructions with a callable address. Even there, function pointers make it possible to pass executable behavior into another function. Higher-level runtimes extend this idea substantially. In Python, JavaScript, PHP, and Java, functions, lambdas, methods, or callable objects can be stored, passed, returned, registered, and invoked later. This changes the shape of control flow: code is no longer only called directly by name from a fixed location in the source text.

Callbacks introduce the first step in this shift. A callback lets one component pass executable behavior to another component, which later decides when to invoke it. This is the basis of sorting hooks, GUI event handlers, web route handlers, AJAX completion handlers, plugin systems, shell traps, and many framework-level extension points.

Anonymous subroutines introduce the second step. They allow small executable blocks to be written

inline, directly at the point where they are needed. Instead of defining a separate named function for every small transformation, comparison, event handler, or route body, the programmer can create a temporary function value locally. This makes sense only in languages or runtimes where executable code can exist as a value.

Closures introduce the deepest step. A closure is not just code passed around; it is code together with preserved surrounding state. The ordinary stack frame that created the function may disappear, but selected variables from that frame remain available through heap-preserved cells, lexical environments, closure objects, captured fields, or explicit environment pointers. This is where the runtime clearly exceeds the simple native stack model: execution context can survive beyond the call that created it.

The three topics therefore belong together because they describe the same progression:

```

executable address / function value
      |
      v
callback passed to another controller
      |
      v
anonymous function written inline
      |
      v
closure carrying preserved lexical state

```

This prepares the next control-flow abstractions. Once executable behavior can be passed as a value and can carry preserved state, the language can support more advanced patterns: decorators, lazy generators, coroutine suspension, event-loop dispatch, asynchronous callbacks, and runtime-managed execution graphs.

Callbacks: Inverting Program Flow Control via Code Execution References and Function Pointers

A **callback** is executable behavior handed to another component so that this component can invoke it later. The programmer does not directly write the complete control sequence. Instead, the programmer supplies a function, method, command name, closure, or callable object, and another piece of code decides when to execute it.

In a direct procedural call, the control path is simple:

```

caller
  calls function
    function runs
  caller continues

```

In a callback pattern, the control path is inverted:

```

caller
  gives callback to library / runtime / framework
    library / runtime / framework performs work
    library / runtime / framework calls callback when needed
  caller receives final result or continues later

```

This is why callbacks are often described as **inversion of control**. The user's code is no longer the only driver of execution. A sorting library, graphical interface framework, web server, asynchronous runtime, shell trap system, or browser event loop may call user code later, at a moment selected by that external system.

The same idea appears in many languages, but the representation differs:

- C uses function pointers and sometimes explicit context pointers;

- Python uses function objects and bound method objects;
- Java uses interfaces, anonymous classes, lambdas, and method references;
- JavaScript uses function objects, especially in event-driven and asynchronous code;
- PHP uses callable values, closures, hooks, and framework callbacks;
- Bash approximates callbacks through function names, traps, and command dispatch.

The stack behavior depends on whether the callback is synchronous or asynchronous. A synchronous callback runs before the caller returns. It creates an ordinary call frame above the code that invokes it. An asynchronous callback is registered first and called later, often after the original registering stack has disappeared.

C: function pointers, comparators, signal handlers, and C-style event hooks. C does not have runtime function objects in the Python or JavaScript sense. A C function compiled into a native program lives in the text segment as machine instructions. A function pointer is a value that stores the address of such a function.

A classical useful callback is the comparator passed to `qsort()`:

Listing 2.4.1: C: function pointers, comparators, signal handlers, and C-style event hooks.

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int compare_ints(const void *left, const void *right) {
5      int a = *(const int *)left;
6      int b = *(const int *)right;
7
8      if (a < b) return -1;
9      if (a > b) return 1;
10     return 0;
11 }
12
13 int main(void) {
14     int values[] = {4, 1, 3, 2};
15     size_t count = sizeof(values) / sizeof(values[0]);
16
17     qsort(values, count, sizeof(values[0]), compare_ints);
18
19     for (size_t i = 0; i < count; i++) {
20         printf("%d\n", values[i]);
21     }
22
23     return 0;
24 }
```

The useful application is sorting arbitrary data. The C standard library provides the sorting algorithm, but the programmer provides the comparison rule. The library does not know what a user-defined record means, so it calls the callback whenever two elements must be compared.

The control path is:

```

main()
  calls qsort()
    qsort() calls compare_ints()
    compare_ints() returns to qsort()
    qsort() calls compare_ints() again
    compare_ints() returns to qsort()
  qsort() returns to main()
```

During one comparator call, the native process stack is conceptually:

```

top
+-----+
| frame: compare_ints      |
+-----+
| frame: qsort             |
+-----+
| frame: main              |
+-----+
bottom

```

The callback is not a separate thread and not a saved continuation. It is an ordinary function call. The difference is only that `qsort()` calls through a function pointer supplied by the user.

At implementation level:

```

compare_ints
    -> compiled code in text segment

function pointer
    -> stores address of compare_ints

qsort
    -> performs indirect call through that address

```

A second useful C callback pattern appears in event libraries. A C API may let the user register a function and an opaque state pointer:

```

typedef void (*event_callback_t)(int event_code, void *user_data);

void dispatch_event(event_callback_t callback, void *user_data) {
    int event_code = 42;
    callback(event_code, user_data);
}

void print_event(int event_code, void *user_data) {
    const char *name = (const char *)user_data;
    printf("%s received event %d\n", name, event_code);
}

int main(void) {
    dispatch_event(print_event, "handler A");
    return 0;
}

```

The useful application is GUI programming, network libraries, embedded systems, plugin hooks, and low-level event loops. The function pointer supplies the code. The `void *user_data` pointer supplies state. This manually emulates what higher-level languages do automatically with closures.

The stack during callback execution is:

```

top
+-----+
| frame: print_event       |
+-----+
| frame: dispatch_event    |
+-----+

```

```

+-----+
| frame: main          |
+-----+
bottom

```

The state string `"handler A"` is not captured lexically. It is passed explicitly as a pointer. C makes the state channel visible and manual.

Python: sorting keys, GUI handlers, web callbacks, and data-processing hooks. In Python, functions are ordinary runtime objects. A function can be passed to another function exactly like a list, string, or integer object reference.

A simple useful callback is a key function for sorting:

```

names = ["Ana", "Constantin", "Ioana"]

names.sort(key=len)

print(names)

```

The useful application is custom sorting. The list object owns the sorting procedure. The programmer supplies `len` as the key extractor. The sorting machinery decides when to call it.

The Python-level control path is:

```

module body
  calls names.sort(...)
    sort machinery calls len(name)
    sort machinery calls len(name)
    sort machinery rearranges list
  sort returns
module body continues

```

During one callback call, the Python frame chain may be conceptually:

```

top Python frame
+-----+
| frame: len or key function |
+-----+
| frame: sort implementation |
+-----+
| frame: module body        |
+-----+
bottom

```

For built-in callbacks such as `len`, part of the call may enter C-level CPython implementation directly. For a Python-defined callback, CPython creates a Python frame for the callback function. Under both cases, CPython itself is still executing on the native process stack underneath.

A Python-defined callback example:

</> PYTHON Listing 2.4.2: Python: sorting keys, GUI handlers, web callbacks, and data-processing hooks.

```

1  def apply_to_values(values, callback):
2      results = []
3
4      for value in values:
5          results.append(callback(value))
6
7      return results
8
9
10 def square(x):
11     return x * x
12
13
14 print(apply_to_values([1, 2, 3], square))

```

Here, `square` is a function object. Passing it does not copy its bytecode. The parameter `callback` becomes another reference to the same function object.

A simplified CPython object picture is:

```

name square
  |
  v
function object
  |
  v
code object with bytecode

local name callback
  |
  v
same function object

```

Useful applications include:

- `sorted(..., key=callback)` for custom ordering;
- GUI event handlers, such as button-click callbacks;
- web route handlers in frameworks;
- data-processing hooks such as validators, filters, and transformation functions;
- asynchronous completion callbacks in older or lower-level APIs.

For example, a web framework route handler is callback-like:

```

@app.route("/hello")
def hello():
    return "Hello"

```

The programmer does not call `hello()` directly for each HTTP request. The framework stores the function object and calls it when a matching request arrives.

The stack during a request is conceptually:

```

top Python frame
+-----+
| frame: hello          |
+-----+

```

```

+-----+
| frame: framework dispatch |
+-----+
| frame: server request loop |
+-----+
bottom

```

The application is useful because control is inverted: the web server receives requests and invokes user functions only when routes match.

Java: listeners, GUI events, stream operations, and functional interfaces. Java traditionally represents callbacks through objects implementing interfaces. A very common useful application is GUI event handling.

For example, in Swing-style programming:

```

button.addActionListener(event -> {
    System.out.println("Button clicked");
});

```

The useful application is event-driven interface programming. The programmer registers behavior. The GUI framework later invokes that behavior when the user clicks the button.

A longer interface-based shape is:

```

interface Callback {
    void call(String message);
}

class Printer implements Callback {
    public void call(String message) {
        System.out.println(message);
    }
}

class Dispatcher {
    static void run(Callback callback) {
        callback.call("event happened");
    }
}

public class Main {
    public static void main(String[] args) {
        Dispatcher.run(new Printer());
    }
}

```

At source level, `callback.call(...)` looks like a method call. At JVM level, this creates ordinary method frames. During callback execution:

```

top JVM frame
+-----+
| frame: Printer.call      |
+-----+
| frame: Dispatcher.run    |
+-----+
| frame: Main.main         |

```

```
+-----+
bottom
```

Modern Java uses lambdas for this pattern:

</> JAVA Listing 2.4.3: Java: listeners, GUI events, stream operations, and functional interfaces.

```
1  import java.util.function.Consumer;
2
3  public class Main {
4      static void run(Consumer<String> callback) {
5          callback.accept("event happened");
6      }
7
8      public static void main(String[] args) {
9          run(message -> System.out.println(message));
10     }
11 }
```

The lambda is converted to an implementation of a functional interface. In common JVM implementations, this is connected through runtime linkage mechanisms such as `invokedynamic`. Conceptually, the result is still a callable object compatible with the expected interface.

Useful applications include:

- GUI listeners such as button clicks and window events;
- stream operations such as `map`, `filter`, and `forEach`;
- asynchronous callbacks using futures or completion handlers;
- dependency injection hooks and framework lifecycle methods;
- server handlers in web frameworks.

For a stream example:

```
List<Integer> values = List.of(1, 2, 3);

values.stream()
    .map(x -> x * x)
    .forEach(System.out::println);
```

The functions passed to `map` and `forEach` are callback-like. The stream machinery controls when each function is invoked.

The stack is ordinary JVM method-call stack while the stream pipeline is executing. However, the exact stack may include framework/library frames:

```
top JVM frame
+-----+
| frame: lambda body      |
+-----+
| frame: stream internals |
+-----+
| frame: Main.main        |
+-----+
bottom
```

If the stream is parallel, additional worker threads may be involved. Then each worker has its own native/JVM stack.

JavaScript and V8: DOM events, AJAX, timers, promises, and Node.js callbacks. JavaScript makes callbacks central because functions are objects and because JavaScript environments are heavily event-driven.

A classical browser example is a DOM event callback:

```
button.addEventListener("click", function (event) {
  console.log("button clicked");
});
```

The useful application is interactive web pages. The programmer registers a function. The browser calls it later when the user clicks the button.

The first stack turn registers the callback:

```
top-level script
  calls addEventListener()
  browser stores callback
top-level script continues
```

Later, when the event occurs:

```
browser event loop
  dispatches click event
    V8 calls callback function
  callback returns
event loop continues
```

The later JavaScript call stack is conceptually:

```
top JS frame
+-----+
| frame: click callback      |
+-----+
| frame: event dispatch     |
+-----+
bottom
```

The original registration stack is gone. The callback survives because the browser host stores a reference to the JavaScript function object.

A classic AJAX callback example uses XMLHttpRequest:

</>JAVASCRIPT Listing 2.4.4: JavaScript and V8: DOM events, AJAX, timers, promises, and Node.js callbacks.

```
1  const request = new XMLHttpRequest();
2
3  request.onload = function () {
4    console.log(request.responseText);
5  };
6
7  request.open("GET", "/data.json");
8  request.send();
```

The useful application is loading data from a server without reloading the whole page. The request is started, control returns to the browser, and the callback runs later when the response arrives.

The control path is:

```

script starts request
  registers onload callback
  returns to browser

network completes later
  browser queues event
  event loop invokes callback through V8

```

Modern JavaScript often uses promises and `fetch`, but callbacks are still underneath the event-driven model:

```

fetch("/data.json")
  .then(response => response.json())
  .then(data => console.log(data));

```

Each function passed to `then` is a callback scheduled when the previous promise is fulfilled. In Node.js, the traditional callback pattern appears in I/O:

</> JAVASCRIPT Listing 2.4.5: JavaScript and V8: DOM events, AJAX, timers, promises, and Node.js callbacks.

```

1  const fs = require("fs");
2
3  fs.readFile("notes.txt", "utf8", function (error, data) {
4    if (error) {
5      console.error(error);
6      return;
7    }
8
9    console.log(data);
10 });

```

The useful application is non-blocking file or network I/O. The Node runtime starts the operation, returns to the event loop, and later calls the callback when the operation completes.

In V8, JavaScript function objects live on the managed heap. A synchronous callback creates a normal JavaScript call frame above the caller. An asynchronous callback creates a frame later, when the host event loop dispatches it. The event loop belongs to the host environment: browser, Node.js, or Electron-style application. V8 executes the callback; the host decides when the callback becomes runnable.

PHP: array callbacks, sorting callbacks, request hooks, and framework handlers. PHP represents callbacks through callable values. A callable may be a function name, closure, object method, static method, or invokable object.

A useful callback appears in array processing:

</> PHP Listing 2.4.6: PHP: array callbacks, sorting callbacks, request hooks, and framework handlers.

```

1  <?php
2
3  $values = [1, 2, 3, 4];
4
5  $result = array_map(function ($x) {
6    return $x * $x;
7  }, $values);
8
9  print_r($result);

```

The useful application is transformation of collections. `array_map` owns the iteration and calls the supplied function for each element.

The Zend Engine call stack is conceptually:

```

top PHP frame
+-----+
| frame: anonymous callback |
+-----+
| frame: array_map internals |
+-----+
| frame: script body        |
+-----+
bottom

```

Another useful callback is custom sorting:

</>PHP Listing 2.4.7: PHP: array callbacks, sorting callbacks, request hooks, and framework handlers.

```

1  <?php
2
3  $users = [
4      ["name" => "Ana", "age" => 30],
5      ["name" => "Mihai", "age" => 20],
6  ];
7
8  usort($users, function ($a, $b) {
9      return $a["age"] <=> $b["age"];
10 });
11
12 print_r($users);

```

Here, `usort` owns the sorting algorithm. The programmer supplies comparison logic. This is directly analogous to C's `qsort`, but the callback is a PHP closure rather than a raw function pointer.

PHP callbacks are also common in web frameworks and plugin systems. For example, a route handler is callback-like:

```

$router->get('/hello', function () {
    return 'Hello';
});

```

The useful application is HTTP request routing. The programmer registers a callable. The framework invokes it when a request matches the route.

Conceptually:

```

request enters framework
  framework matches route
    framework calls registered PHP callable
      response is returned

```

During execution, the Zend Engine creates a PHP call frame for the route callable. If the callable uses variables from the surrounding scope, PHP stores those captured values according to closure rules.

PHP plugin systems also use callbacks. WordPress-style hooks are a common example conceptually:

```

add_action('init', function () {
    // initialization code
});

```

The useful application is extension points. The core system reaches a lifecycle event and calls all registered callbacks.

Bash: traps, shell hooks, command-name callbacks, and pipeline orchestration. Bash does not have first-class function objects like Python or JavaScript. Still, callback-like patterns exist through function names, traps, and command dispatch.

A useful Bash callback pattern is cleanup on exit:

```
cleanup() {
    rm -f "$temp_file"
    echo "cleaned up"
}

temp_file="$(mktemp)"

trap cleanup EXIT

echo "working with $temp_file"
```

The useful application is resource cleanup. The programmer registers `cleanup` as a handler for the shell's `EXIT` event. When the script exits, Bash invokes the registered function.

During normal execution, the stack may be only the script top level. During trap execution:

```
top shell frame
+-----+
| frame: cleanup          |
+-----+
| frame: shell trap dispatch |
+-----+
bottom
```

The trap is not a native hardware interrupt handler. It is shell-managed dispatch. Bash stores the trap action in its runtime state and evaluates it when the event occurs.

A command-name callback pattern is also possible:

```
run_for_each() {
    callback="$1"
    shift

    for item in "$@"; do
        "$callback" "$item"
    done
}

print_item() {
    echo "item: $1"
}

run_for_each print_item apple banana cherry
```

The useful application is small shell automation where one function controls iteration and another function defines the operation.

The shell stack during one callback-like invocation is:

```
top shell frame
+-----+
| frame: print_item       |
+-----+
```

```
| frame: run_for_each      |
+-----+
| frame: script top level  |
+-----+
bottom
```

If the callback name resolves to an external command rather than a shell function, Bash creates a child process. Then the external command has its own native process stack and address space. The shell waits or continues depending on whether the command is foreground, background, part of a pipeline, or part of another control structure.

For example:

```
run_for_each cat file1.txt file2.txt
```

Here, `cat` is external on most systems. Each invocation may involve a child process:

```
Bash run_for_each frame
  launches child process: cat file1.txt
  waits
  launches child process: cat file2.txt
  waits
```

So Bash callback-like behavior can cross the process boundary, unlike Python or JavaScript function-object callbacks, which normally execute inside the same runtime process.

Comparison and stack summary. Callbacks are useful because they separate *general control logic* from *custom behavior*. The library or runtime owns the general pattern: sorting, event dispatch, iteration, request routing, I/O completion, cleanup, or plugin notification. The user supplies the custom operation.

Language	Useful callback applications	Implementation and stack behavior
C	<code>qsort</code> comparators, event libraries, embedded callbacks, signal-like handlers	Function pointer stores code address; synchronous call creates ordinary native stack frame; state usually passed manually with <code>void *</code>
Python	Sorting keys, GUI handlers, web route handlers, validators, data-processing hooks	Function object reference; callback call creates Python frame; CPython native stack exists underneath
Java	GUI listeners, stream operations, completion handlers, framework lifecycle hooks	Interface/lambda object; JVM invokes method; callback creates JVM method frame
JavaScript / V8	DOM events, AJAX/fetch completion, timers, Node.js I/O callbacks	Function object stored by host/runtime; synchronous callback creates JS frame immediately; async callback creates frame later from event loop dispatch
PHP	<code>array_map</code> , <code>usort</code> , route handlers, hooks/plugin systems	Callable value resolved by Zend Engine; callback creates PHP call frame inside request/runtime execution
Bash	<code>trap</code> cleanup handlers, command-name dispatch, shell automation hooks	Function name or command text; shell functions create shell frames; external commands create child processes with separate native stacks

The key rule is simple: a callback is not special at the moment it runs. It uses the ordinary call machinery of its language or runtime. What is special is how the call target is chosen and when it is invoked. The callback is supplied earlier as a value, address, object, method implementation, closure, command name, or trap action. Later, another system calls it.

This prepares the next concepts. Anonymous functions make callback definitions shorter and more local. Closures allow callbacks to carry saved lexical state. Generators and coroutines go further: they do not merely pass executable behavior around; they preserve suspended execution state across multiple resumptions.

Anonymous Subroutines: Inline Block Definitions, Lambda Expressions, and Function Objects Without Stable Source-Level Names

A named function has a stable source-level identifier. It is declared or defined once, receives a name, and can later be called through that name. An **anonymous subroutine** is different: it is executable logic created without a permanent ordinary source-level function name. It may be written directly at the place where it is needed, passed immediately to another function, stored in a variable, returned from another function, or registered as a handler.

The basic contrast is:

`</> PYTHON` Listing 2.4.8: Anonymous Subroutines: Inline Block Definitions, Lambda Expressions, and Func...

```
1  named function:
2
3  def square(x):
4      return x * x
5
6  use(square)
7
8
9  anonymous function:
10
11 use(lambda x: x * x)
```

The second form creates executable behavior inline. The programmer does not introduce a separate stable top-level function name such as `square`. This is useful when the behavior is small, local, and meaningful only at the point where it is passed.

Anonymous subroutines are strongly connected to callbacks. A callback is the control-flow pattern: executable behavior is passed to another component. An anonymous function is one possible way to write that executable behavior without defining a separately named function first.

Python: lambda expressions and local function objects. Python supports anonymous functions through lambda expressions. A `lambda` creates a function object, but the function has no ordinary source-level name introduced by a `def` statement.

For example:

```
numbers = [1, 2, 3, 4]

squares = list(map(lambda x: x * x, numbers))

print(squares)
```

The useful application here is a small transformation function used immediately by `map`. The function object receives one argument and returns the square of that argument.

A sorting example is even more common:

```
users = [
    {"name": "Ana", "age": 30},
```

```

{"name": "Mihai", "age": 20},
{"name": "Ioana", "age": 25},
]

users.sort(key=lambda user: user["age"])

print(users)

```

The useful application is custom ordering. The lambda tells the sorting operation which value should be extracted from each item. The sorting algorithm owns the control flow; the lambda supplies the local rule.

At the CPython level, the `lambda` expression creates a function object with an associated code object, just like `def` does. The difference is mainly syntactic and structural: `lambda` is an expression and can appear inline, while `def` is a statement and binds a name.

Conceptually:

```

lambda user: user["age"]
    |
    v
function object
    |
    v
code object for expression body

```

When the lambda is called during sorting, CPython creates a Python frame for that call:

```

top Python frame
+-----+
| frame: lambda body      |
+-----+
| frame: sort machinery   |
+-----+
| frame: module body      |
+-----+
bottom

```

The lambda frame disappears when the lambda returns. If the lambda does not escape anywhere else, the function object itself may also become unreachable after the surrounding operation finishes.

Python lambdas are intentionally limited: they contain a single expression, not a full statement block. This is valid:

```
lambda x: x * x
```

This is not valid Python:

```

lambda x:
    y = x * x
    return y

```

For multi-statement behavior, Python uses a named local function:

```

def process(values):
    def normalize(x):
        y = x.strip()
        return y.lower()

    return list(map(normalize, values))

```

Here, `normalize` has a local name, but it is not a stable top-level function. It is created during execution of `process`. This is often more readable than forcing too much logic into a lambda.

JavaScript and V8: function expressions, arrow functions, AJAX, and DOM handlers.

JavaScript has several forms of anonymous function expressions. The older form is:

```
const square = function (x) {
    return x * x;
};
```

The function expression may be anonymous even though it is stored in a variable. The variable has a name, but the function object itself does not need a source-level declaration name.

Modern JavaScript often uses arrow functions:

```
const square = x => x * x;
```

Anonymous functions are central in browser programming. A DOM event handler can be written inline:

```
button.addEventListener("click", event => {
    console.log("button clicked");
});
```

The useful application is local event handling. The programmer does not need to define a separate named function if the logic is used only for that button event.

A classic AJAX-style example is:

```
fetch("/data.json")
    .then(response => response.json())
    .then(data => {
        console.log(data);
    });
```

The functions passed to `then` are anonymous callback functions. They are scheduled by the promise machinery when the previous asynchronous stage completes.

At the V8 level, each function expression or arrow function creates a JavaScript function object. If the function is called synchronously, V8 creates a normal JavaScript call frame:

```
top JS frame
+-----+
| frame: anonymous function |
+-----+
| frame: caller             |
+-----+
bottom
```

For asynchronous cases, the original registration stack is gone before the anonymous function runs. The host environment stores the function object and schedules it later:

registration turn:

```
top-level script
    creates anonymous function object
    passes it to event / promise / timer API
top-level script returns
```

later event-loop turn:

```
event loop dispatch
    calls anonymous function
anonymous function returns
```

The later stack is:

```
top JS frame
+-----+
| frame: anonymous callback |
+-----+
| frame: event/promise dispatch |
+-----+
bottom
```

V8 may optimize this call if the call site becomes hot and predictable. But semantically, the anonymous function is still a heap-managed function object invoked by the engine.

Java: anonymous classes, lambdas, GUI listeners, and stream functions. Older Java used anonymous classes for inline callback definitions. A common GUI-style example is:

```
button.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent event) {
        System.out.println("Button clicked");
    }
});
```

The useful application is event handling. The programmer defines the handler inline at the registration site. The anonymous class has no ordinary source-level class name written by the programmer, although the compiler still generates class-like runtime structures.

Modern Java usually uses lambdas when the target type is a functional interface:

```
button.addActionListener(event -> {
    System.out.println("Button clicked");
});
```

Another useful application is stream processing:

```
List<Integer> values = List.of(1, 2, 3, 4);

values.stream()
    .map(x -> x * x)
    .forEach(x -> System.out.println(x));
```

The lambdas are inline functions passed to the stream machinery. The stream controls traversal; the lambdas provide transformation and output behavior.

At the JVM level, a lambda is not a raw function pointer. It is linked to a functional interface. Common JVM implementations use runtime linkage mechanisms, such as `invokedynamic`, to create or obtain an object that can be called through the interface method.

Conceptually:

```
x -> x * x
  |
  v
functional-interface-compatible callable object
  |
  v
method invocation by stream / event framework
```

When the lambda body runs, the JVM uses ordinary method-frame execution:

```

top JVM frame
+-----+
| frame: lambda body      |
+-----+
| frame: stream/event internals |
+-----+
| frame: caller method    |
+-----+
bottom

```

The lambda itself does not create a new operating-system thread. It runs on whichever thread invokes it. In GUI code, that is often an event-dispatch thread. In stream code, it may be the current thread, or multiple worker threads if a parallel stream is used.

PHP: anonymous functions, closures, array processing, and route handlers. PHP supports anonymous functions directly:

 Listing 2.4.9: PHP: anonymous functions, closures, array processing, and route handlers.

```

1  <?php
2
3  $values = [1, 2, 3, 4];
4
5  $squares = array_map(function ($x) {
6      return $x * $x;
7  }, $values);
8
9  print_r($squares);

```

The useful application is collection transformation. `array_map` owns the traversal. The anonymous function supplies the operation.

Custom sorting is another common example:

 Listing 2.4.10: PHP: anonymous functions, closures, array processing, and route handlers.

```

1  <?php
2
3  $users = [
4      ["name" => "Ana", "age" => 30],
5      ["name" => "Mihai", "age" => 20],
6  ];
7
8  usort($users, function ($a, $b) {
9      return $a["age"] <=> $b["age"];
10 });
11
12 print_r($users);

```

The anonymous function supplies comparison logic directly at the call site.

PHP web frameworks also commonly use anonymous functions as route handlers:

```

$router->get('/hello', function () {
    return 'Hello';
});

```

The useful application is request routing. The route and the executable response logic are written together.

At the Zend Engine level, the anonymous function is represented as a closure object or callable structure. When it is invoked, the engine creates a call frame for it:

```

top PHP frame
+-----+
| frame: anonymous function |
+-----+
| frame: array_map / router |
+-----+
| frame: script body        |
+-----+
bottom

```

The function object can also be stored:

```

$double = function ($x) {
    return $x * 2;
};

echo $double(10);

```

Here, `$double` is a variable holding a callable object. The anonymous function has no ordinary declaration name, but it is still an executable runtime value.

C: no standard lambdas, but function pointers and nonstandard emulations. Standard C does not have anonymous functions or lambda expressions. A callback target must normally be a named function, and the program passes a function pointer.

For example:

```

int square(int x) {
    return x * x;
}

int apply(int value, int (*operation)(int)) {
    return operation(value);
}

```

The useful application is still callback-based customization, but the executable code must be named:

```

int result = apply(10, square);

```

At the native level, `square` is compiled into the text segment. The function pointer stores its address. Calling through the pointer creates an ordinary C stack frame:

```

top native frame
+-----+
| frame: square          |
+-----+
| frame: apply           |
+-----+
| frame: main            |
+-----+
bottom

```

C can emulate some anonymous-subroutine behavior, but not cleanly in portable standard C. One common manual pattern is to separate code and state:

</> C Listing 2.4.11: C: no standard lambdas, but function pointers and nonstandard emulations.

```

1  typedef int (*operation_t)(int value, void *context);
2
3  int apply(int value, operation_t operation, void *context) {
4      return operation(value, context);
5  }
6
7  struct multiplier {
8      int factor;
9  };
10
11 int multiply_by(int value, void *context) {
12     struct multiplier *m = context;
13     return value * m->factor;
14 }
15
16 int main(void) {
17     struct multiplier m = { .factor = 3 };
18     int result = apply(10, multiply_by, &m);
19     return 0;
20 }

```

This is useful in C libraries, embedded systems, GUI toolkits, and event loops. The function pointer supplies behavior. The `void *context` supplies state. Higher-level languages hide this pattern inside closures.

Some C compilers provide nonstandard extensions. GCC supports nested functions in some modes:

</> C Listing 2.4.12: C: no standard lambdas, but function pointers and nonstandard emulations.

```

1  int main(void) {
2      int factor = 3;
3
4      int multiply(int x) {
5          return x * factor;
6      }
7
8      return multiply(10);
9  }

```

This is not ISO C. Implementations may use mechanisms such as trampolines or hidden environment pointers, depending on compiler and target. The important point is that the compiler must somehow preserve access to `factor`. If the nested function escapes its defining stack frame, lifetime problems become serious unless the implementation prevents or handles them.

Clang/Apple ecosystems also have **blocks**, an extension used heavily in some Objective-C and C APIs:

```

int factor = 3;

int (^multiply)(int) = ^int(int x) {
    return x * factor;
};

int result = multiply(10);

```

Blocks behave much more like closures than ordinary C function pointers. They can capture surrounding values, and the runtime may copy them from stack-like storage to heap storage when needed. This is not standard C, but it shows how C-like environments can add anonymous callable objects through compiler/runtime support.

Duff's Device is sometimes mentioned as an example of unusual C control-flow construction, but it is not an anonymous function mechanism. It manually interleaves `switch` and loop structure to optimize or unroll copying. It shows that C can encode strange control-flow patterns with labels and jumps, but it does not give C true anonymous subroutines. For anonymous callable behavior, function pointers plus explicit context pointers are the standard portable tool.

Bash: no real anonymous functions, but inline commands and evaluated command bodies.

Bash does not have anonymous function objects. Shell functions are named:

```
say_hello() {
    echo "hello"
}
```

However, Bash can emulate local inline behavior through command strings, subshells, and `bash -c`. For example:

```
find . -name "*.txt" -exec bash -c '
    for file do
        echo "file: $file"
    done
' bash {} +
```

The useful application is inline command logic for tools such as `find`. The command body is not a function object in the Python sense. It is command text passed to a new Bash process.

The stack/process behavior is very different:

```
parent shell
    runs find
        find launches bash -c as child process
            child bash interprets inline command text
            child bash exits
        find continues / exits
parent shell continues
```

The inline command runs in a separate process. Its stack is the child Bash process's native/shell stack, not a frame inside the parent shell's current function stack.

Bash can also use command groups:

```
{
    echo "start"
    echo "work"
    echo "end"
}
```

or subshells:

```
(
    cd /tmp
    ls
)
```

These are anonymous blocks of command syntax, but they are not first-class function values. A command group runs in the current shell. A subshell runs in a child shell environment. They are useful for grouping redirections, temporary directory changes, and pipeline construction.

For example:


```
{
  echo "name,age"
  echo "Ana,30"
} > output.csv
```

The useful application is redirecting the output of several commands as one unit. The block has no name, but it is not a callable object stored for later.

Implementation comparison. Anonymous subroutines differ less in surface syntax than in runtime representation. The essential question is: when the inline code is written, what runtime object or control structure is created?

Language	Anonymous form	Implementation and stack behavior
Python	<code>lambda</code> ; local <code>def</code> for multi-statement logic	Creates function object with code object; call creates Python frame; CPython native stack underneath
JavaScript / V8	Function expressions and arrow functions	Creates JS function object; call creates JS frame; async cases are invoked later by host event loop
Java	Anonymous classes and lambdas	Produces functional-interface-compatible callable object; call creates JVM method frame
PHP	Anonymous functions / closures	Creates callable closure object; call creates Zend Engine call frame
C	No standard <code>lambda</code> ; named function pointer plus <code>void *</code> ; nonstandard nested functions/blocks	Function pointer is code address; call creates native frame; state must be explicit unless using extensions
Bash	No function objects; inline command text, command groups, subshells	Command group runs in shell; subshell or <code>bash -c</code> creates child process; no stable callable object model

Anonymous subroutines are useful when the code is small and local to the operation that consumes it:

- sort this list by this field;
- transform every element this way;
- handle this button click;
- respond to this AJAX request;
- run this route handler;
- clean up when this shell exits.

Their main advantage is locality. The behavior is written where it is used. Their main danger is overuse. If the inline function grows too large, contains complex branching, or needs to be reused, a named function becomes clearer.

The stack rule remains simple: when the anonymous subroutine is called, it creates the same kind of call frame that a named function would create in that language. Anonymous does not mean stackless. It only means that the executable unit was created without a stable ordinary source-level name. If the function is invoked later, especially through an event loop or asynchronous system, the original registration stack may already be gone; the runtime preserves the function object, not the old call stack. Closures extend this idea by preserving selected surrounding variables together with the executable code.

Closures: Heap-Preserved Execution Environments, Non-Local Scope State Retention, and Lexical Binding

A **closure** is a callable unit that keeps access to variables from an enclosing lexical scope even after that enclosing scope has finished executing. It is not merely an anonymous function, and it is not merely a callback. A callback answers the question: *who will call this code later?* An anonymous function answers the question: *can this code be written without a stable source-level name?* A closure answers a different question: *can this callable remember variables from the environment where it was created?*

The basic closure pattern is:

```
outer function starts
  local variable is created
  inner function is created and refers to that variable
outer function returns inner function
  outer stack frame disappears
inner function is called later
  inner function still has access to the old variable
```

This requires special runtime support. If the outer function's local variables lived only in an ordinary stack frame, they would disappear when the outer function returned. For a closure to work, the captured state must be preserved somewhere that outlives the stack frame. In managed runtimes, this usually means heap-allocated closure cells, environment objects, callable objects with captured fields, or similar structures.

The core idea is:

```
ordinary local variable:
  lives only while the function frame is active

closed-over variable:
  moved to or represented by heap-preserved environment storage
  so an inner callable can still access it later
```

Python / CPython: closure cells and non-local lexical bindings. Python supports closures directly. A nested function can refer to a variable from an enclosing function.

`</>PYTHON` Listing 2.4.13: Python / CPython: closure cells and non-local lexical bindings.

```
1  def make_multiplier(factor):
2      def multiply(value):
3          return value * factor
4
5      return multiply
6
7
8  times_three = make_multiplier(3)
9
10 print(times_three(10))
```

The useful application is function configuration. Instead of passing `factor` every time, the outer function builds a specialized function that remembers the value.

The execution sequence is:

```
make_multiplier(3)
  creates local binding factor = 3
  creates inner function multiply
  multiply refers to factor
```

```

    returns multiply

make_multiplier frame disappears

times_three(10)
    multiply runs
    multiply still sees factor = 3

```

At the Python frame level, while `make_multiplier` is running:

```

top Python frame
+-----+
| frame: make_multiplier |
| factor -> closure cell |
| multiply -> function object |
+-----+
| frame: module body |
+-----+
bottom

```

After `make_multiplier` returns, its normal frame is gone. However, the captured variable is not gone. CPython stores closed-over variables in **cell objects**. The returned function object keeps a reference to those cells through its closure.

Conceptually:

```

times_three
|
v
function object: multiply
|
v
__closure__
|
v
cell object containing factor = 3

```

When `times_three(10)` is later called, CPython creates a new frame for `multiply`. That frame can access the preserved cell:

```

top Python frame
+-----+
| frame: multiply |
| value = 10 |
| factor -> closure cell |
+-----+
| frame: module body |
+-----+
bottom

```

The old outer stack frame is not restored. Only the needed captured environment survives. A common practical use is creating validators or configured callbacks:

</>PYTHON Listing 2.4.14: Python / CPython: closure cells and non-local lexical bindings.

```

1  def min_length_validator(min_length):
2      def validate(text):
3          return len(text) >= min_length
4
5      return validate
6
7
8  password_validator = min_length_validator(8)
9
10 print(password_validator("abc"))
11 print(password_validator("long-enough"))

```

Here, `min_length` is remembered by the returned function. This is useful in web forms, data validation pipelines, GUI callbacks, decorators, and small configurable behaviors.

If the inner function must rebind a captured variable, Python requires `nonlocal`:

</>PYTHON Listing 2.4.15: Python / CPython: closure cells and non-local lexical bindings.

```

1  def make_counter():
2      count = 0
3
4      def next_value():
5          nonlocal count
6          count += 1
7          return count
8
9      return next_value
10
11
12 counter = make_counter()
13
14 print(counter())
15 print(counter())
16 print(counter())

```

The useful application is private state. The variable `count` is not global, but it persists between calls.

The preserved environment is:

```

counter
|
v
function object: next_value
|
v
closure cell: count

```

Each call to `counter()` creates a new frame for `next_value`, but all those frames access the same preserved `count` cell.

JavaScript / V8: lexical environments for callbacks, AJAX, timers, and factories. JavaScript closures are central to browser and Node.js programming. A function can capture variables from its lexical environment and use them later.

</> JAVASCRIPT Listing 2.4.16: JavaScript / V8: lexical environments for callbacks, AJAX, timers, and factor...

```

1  function makeMultiplier(factor) {
2      return function (value) {
3          return value * factor;
4      };
5  }
6
7  const timesThree = makeMultiplier(3);
8
9  console.log(timesThree(10));

```

The useful application is the same as in Python: creating specialized functions.

At the V8-level conceptual model, **factor** is stored in a heap-managed lexical environment because the returned inner function may outlive the call to **makeMultiplier**.

```

timesThree
  |
  v
JS function object
  |
  v
lexical environment
  |
  v
factor = 3

```

The original call stack is gone:

```

makeMultiplier frame:
  gone after return

```

But the captured environment remains reachable through the returned function object. Closures are especially useful in asynchronous JavaScript:

</> JAVASCRIPT Listing 2.4.17: JavaScript / V8: lexical environments for callbacks, AJAX, timers, and factor...

```

1  function loadUser(userId) {
2      fetch("/users/" + userId)
3          .then(response => response.json())
4          .then(user => {
5              console.log("Loaded user:", userId, user.name);
6          });
7  }

```

The useful application is AJAX/fetch completion handling. The callback passed to **then** runs later, after the network operation completes. The original **loadUser** stack frame is no longer active, but the callback still remembers **userId**.

The control and storage model is:

```

loadUser(42)
  creates callback that captures userId
  registers callback with promise machinery
  returns

```

```

later:
  network completes
  event loop schedules callback
  callback runs and still sees userId = 42

```

The later JavaScript stack is:

```
top JS frame
+-----+
| frame: promise callback |
| userId from lexical env |
+-----+
| frame: promise dispatch |
+-----+
bottom
```

The old `loadUser` frame is not on the stack. The captured lexical environment is heap-preserved by the engine.

Closures are also common for DOM event handlers:

`</>JAVASCRIPT` Listing 2.4.18: JavaScript / V8: lexical environments for callbacks, AJAX, timers, and factor...

```
1 function attachCounter(button) {
2     let count = 0;
3
4     button.addEventListener("click", () => {
5         count += 1;
6         console.log("clicked", count, "times");
7     });
8 }
```

The useful application is preserving per-button state without using a global variable. Each call to `attachCounter` creates a separate lexical environment with its own `count`. V8 keeps that environment alive as long as the event listener function remains reachable.

Java: captured variables in lambdas and generated callable objects. Java lambdas can capture local variables from the enclosing scope, but with an important restriction: captured local variables must be final or effectively final.

`</>JAVA` Listing 2.4.19: Java: captured variables in lambdas and generated callable objects.

```
1 import java.util.function.IntUnaryOperator;
2
3 public class Main {
4     static IntUnaryOperator makeMultiplier(int factor) {
5         return value -> value * factor;
6     }
7
8     public static void main(String[] args) {
9         IntUnaryOperator timesThree = makeMultiplier(3);
10
11         System.out.println(timesThree.applyAsInt(10));
12     }
13 }
```

The useful application is building configured functional objects for streams, callbacks, validators, listeners, and completion handlers.

The Java stack behavior is:

```
makeMultiplier(3)
    creates lambda object that captures factor
    returns lambda object
```

```
makeMultiplier frame disappears
```

```
timesThree.applyAsInt(10)
    lambda body runs
    lambda still has captured factor = 3
```

At JVM level, the captured value is not kept by preserving the old method stack frame. Instead, the lambda is represented as an object or runtime-generated callable structure with captured data stored as fields or equivalent internal state.

Conceptually:

```
timesThree
  |
  v
lambda / functional-interface object
  |
  v
captured field: factor = 3
```

When the lambda is invoked, the JVM creates a normal method frame for the call:

```
top JVM frame
+-----+
| frame: lambda body      |
| captured factor available |
+-----+
| frame: Main.main        |
+-----+
bottom
```

The lambda itself does not create a new operating-system thread. It runs on whichever thread invokes it. In GUI code, that is often an event-dispatch thread. In stream code, it may be the current thread, or multiple worker threads if a parallel stream is used.

Java does not allow this:

```
int count = 0;

Runnable r = () -> {
    count++; // invalid for local captured variable
};
```

The captured local variable cannot be mutated this way because Java captures local values under effectively-final rules. To preserve mutable state, the state must be placed inside a heap object:

 Listing 2.4.20: Java: captured variables in lambdas and generated callable objects.

```
1  import java.util.concurrent.atomic.AtomicInteger;
2
3  public class Main {
4      public static void main(String[] args) {
5          AtomicInteger count = new AtomicInteger(0);
6
7          Runnable r = () -> {
8              System.out.println(count.incrementAndGet());
9          };
10
11         r.run();
12         r.run();
13     }
14 }
```

Here, the lambda captures the reference to the `AtomicInteger` object. The object itself is mutable and lives on the heap.

The preserved state is:

```
lambda object
  |
  v
reference to AtomicInteger object
  |
  v
mutable count state
```

This pattern is useful in event listeners, stream pipelines, asynchronous callbacks, and small pieces of configured behavior.

PHP: closures with use, captured values, and captured references. PHP anonymous functions can capture variables from the surrounding scope using `use`.

`</> PHP` Listing 2.4.21: PHP: closures with use, captured values, and captured references.

```
1  <?php
2
3  function make_multiplier($factor) {
4      return function ($value) use ($factor) {
5          return $value * $factor;
6      };
7  }
8
9  $times_three = make_multiplier(3);
10
11 echo $times_three(10);
```

The useful application is configured callbacks, route handlers, array transformations, and plugin hooks.

The PHP execution sequence is:

```
make_multiplier(3)
  local variable $factor exists
  closure is created and captures $factor
  closure is returned
```

```
make_multiplier frame disappears
```

```
$times_three(10)
  closure runs
  closure still has captured $factor = 3
```

At the Zend Engine level, the closure is a runtime object containing the function body and captured variables. The old function call frame is not preserved. Captured values are stored in the closure structure.

Conceptually:

```
$times_three
  |
  v
Closure object
  |
```


v
captured variable: \$factor = 3

By default, PHP captures with `use ($factor)` by value. If the closure must share and mutate the same variable, it can capture by reference:

`</> PHP` Listing 2.4.22: PHP: closures with `use`, captured values, and captured references.

```

1  <?php
2
3  function make_counter() {
4      $count = 0;
5
6      return function () use (&$count) {
7          $count += 1;
8          return $count;
9      };
10 }
11
12 $counter = make_counter();
13
14 echo $counter(); // 1
15 echo $counter(); // 2
16 echo $counter(); // 3

```

The useful application is private persistent state inside a callable, similar to the Python counter closure.

The preserved state is:

```

$counter
|
v
Closure object
|
v
captured reference to $count storage

```

Each call creates a new PHP call frame for the closure body, but the captured `$count` storage is shared across calls.

Closures are also common in PHP routing:

```

$prefix = "/api";

$router->get($prefix . "/users", function () use ($prefix) {
    return "Route under " . $prefix;
});

```

The useful application is binding route-local or configuration-local state to a handler without using global variables.

C: no standard closures, explicit environment structs, and lifetime hazards. Standard C has function pointers, but not closures. A function pointer stores a code address. It does not automatically capture local variables.

This does not work in standard C:

```
/* Not standard C closure behavior */
```

`make_multiplier(3)` returns function that remembers 3

To emulate a closure in portable C, the programmer usually separates code from state manually:

</>C Listing 2.4.23: C: no standard closures, explicit environment structs, and lifetime hazards.

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  typedef int (*call_fn)(void *env, int value);
5
6  struct closure {
7      call_fn call;
8      void *env;
9  };
10
11 struct multiplier_env {
12     int factor;
13 };
14
15 int multiply_call(void *env, int value) {
16     struct multiplier_env *m = env;
17     return value * m->factor;
18 }
19
20 struct closure make_multiplier(int factor) {
21     struct multiplier_env *env = malloc(sizeof(*env));
22     env->factor = factor;
23
24     struct closure c;
25     c.call = multiply_call;
26     c.env = env;
27
28     return c;
29 }
30
31 void destroy_multiplier(struct closure c) {
32     free(c.env);
33 }
34
35 int main(void) {
36     struct closure times_three = make_multiplier(3);
37
38     printf("%d\n", times_three.call(times_three.env, 10));
39
40     destroy_multiplier(times_three);
41     return 0;
42 }

```

The useful application is callback APIs that need persistent state: event handlers, GUI callbacks, embedded systems, network protocol handlers, and plugin systems.

The implementation is explicit:

```

closure struct
+-----+
| function pointer: call |
| environment pointer: env |
+-----+
      |
      v
environment heap object
+-----+
| factor = 3 |

```

+-----+

The stack behavior is:

```
make_multiplier frame:
    creates heap environment
    returns struct containing code pointer + env pointer
    frame disappears
```

```
later call:
    multiply_call frame is created
    env pointer gives access to heap-preserved factor
```

The key rule is that the environment must outlive the callback. Therefore, allocating the environment on the stack would be wrong if the closure escapes:

```
struct closure bad_make_multiplier(int factor) {
    struct multiplier_env env;
    env.factor = factor;

    struct closure c;
    c.call = multiply_call;
    c.env = &env; /* invalid after return */

    return c;
}
```

After `bad_make_multiplier` returns, `env` no longer exists. The closure contains a dangling pointer.

Some C compilers provide nonstandard closure-like extensions. GCC nested functions can refer to enclosing variables in some modes, and Apple/Clang blocks provide a stronger closure-like extension:

```
int factor = 3;

int (^multiply)(int) = ^int(int value) {
    return value * factor;
};

printf("%d\n", multiply(10));
```

Blocks can capture surrounding variables and may be copied to the heap when they must outlive their creation scope. This is not ISO C, but it shows what extra compiler/runtime machinery is needed to support closures safely.

Duff's Device and similar C control-flow tricks are not closures. They can encode unusual jump and loop behavior, and they can be used in manual state-machine-style code, but they do not automatically preserve lexical environments. For closure behavior in C, the real portable pattern is still: function pointer plus explicit environment pointer.

Bash: no lexical closures, but state can be approximated with globals, subshells, and generated commands. Bash does not have lexical closures in the Python or JavaScript sense. Shell functions do not automatically capture local lexical environments as heap-preserved callable objects.

A Bash function can use global variables:

```
prefix="item:"

print_item() {
```

```

    echo "$prefix $1"
}

print_item "apple"

```

The useful application is small script configuration, but this is not a true closure. The function reads the variable `prefix` from the shell environment/scope rules when it runs. If `prefix` changes, the function sees the changed value.

```

prefix="first:"
print_item "apple"

prefix="second:"
print_item "apple"

```

This is not heap-preserved lexical state. It is lookup in the shell's current variable environment.

A callback-like function generator can be approximated with `eval`, but this is fragile and often unsafe:

```

make_printer() {
    local name="$1"
    local prefix="$2"

    eval "$name() { echo '$prefix' \"\$1\"; }"
}

make_printer print_error "ERROR:"
print_error "file missing"

```

This creates a named function whose body contains the chosen prefix text. The useful application is limited script metaprogramming, but it is not the same as a runtime closure object. It is command-text generation.

The shell stack behavior is ordinary shell function execution:

```

top shell frame
+-----+
| frame: print_error      |
+-----+
| frame: script top level |
+-----+
bottom

```

If Bash runs a subshell:

```

prefix="inside"

(
    echo "$prefix"
)

```

The subshell receives a copy of the shell state, but this is process/environment behavior, not a lexical closure object. External commands receive environment variables, arguments, and file descriptors, not live references to the parent shell's local variables.

Therefore, Bash can emulate some closure-like uses through globals, command strings, exported environment, or generated functions, but it does not provide true lexical closures as first-class callable objects.

Closures and stack preservation: the crucial distinction. A closure does not usually preserve the entire old stack frame. It preserves only the variables that the inner function needs, and it stores them in runtime-managed environment storage.

The wrong mental model is:

```
outer function returns
  whole old stack frame remains alive
```

The better model is:

```
outer function returns
  ordinary stack frame disappears
  captured variables survive in heap/environment objects
  returned function points to that environment
```

A general closure diagram is:

```
callable object
+-----+
| pointer to code          |
| pointer to environment   |
+-----+
      |
      v
environment object / cells
+-----+
| captured variable 1      |
| captured variable 2      |
+-----+
```

Different runtimes implement the environment differently:

Language	Closure representation	Stack and lifetime behavior
Python / CPython	Function object plus <code>__closure__</code> cells	Outer Python frame disappears; captured variables survive in cell objects referenced by function
JavaScript / V8	Function object plus lexical environment	Old JS call frame disappears; lexical environment survives while reachable from function/event structures
Java / JVM	Lambda or anonymous object with captured values as fields or equivalent state	Method frame disappears; captured values live inside heap object or referenced heap objects
PHP / Zend	Closure object with variables captured by <code>use</code> , by value or by reference	Function frame disappears; captured variables live in closure storage
C	No standard closure; manual struct with function pointer and environment pointer	Caller must place environment on heap or otherwise ensure lifetime; stack environment causes dangling pointer if it escapes
Bash	No true lexical closures	Uses current shell variables, generated functions, subshell copies, or command strings; no first-class heap-preserved lexical environment

Closures are useful because they combine executable behavior with remembered context. They allow small pieces of code to carry private configuration or state without using global variables. They appear in validators, decorators, route handlers, GUI event handlers, AJAX callbacks, timer callbacks, stream operations, plugin hooks, and factory functions.

The architectural point is deeper than syntax. A closure proves that the runtime can separate function lifetime from stack-frame lifetime. The function may outlive the call that created it. The stack frame may disappear. The selected environment variables remain reachable because the runtime moved or represented them in heap-preserved storage. This is one of the clearest examples of a high-level runtime owning execution state beyond the simple native call stack model.

2.4.2 Suspended Execution and Runtime-Managed Control Flow

The previous subsection showed that functions can become runtime values: they can be passed as callbacks, created anonymously, and preserved together with captured lexical state through closures. This subsection moves one step further. It examines what happens when execution itself can be paused, stored, and resumed later. At that point, the runtime is no longer only calling functions; it is managing unfinished computations.

In the traditional native C model, an ordinary function call creates a stack frame, runs, and eventually returns. Once the function returns, its stack frame is gone. Generators and coroutines break this simple lifecycle. A generator can execute part of its body, produce a value, suspend, and later continue from the same point. A coroutine can suspend not only to produce a value, but also to cooperate with other in-flight tasks. This requires the runtime to preserve execution state: local variables, instruction position, pending values, and sometimes a larger logical call context.

These mechanisms belong together because they all depend on the same architectural shift: control flow is no longer only a linear call-and-return chain on the native stack. Some execution state is moved into runtime-managed objects. In Python this may be a generator object, coroutine object, task, frame, or future. In JavaScript it may be a generator object, promise continuation, or async function state. In PHP it may be a generator or fiber. In C, where the language does not provide this directly, the programmer must emulate it with explicit state machines, structs, callbacks, or non-portable stack-switching mechanisms.

This is also where the distinction between concurrency and parallelism becomes necessary. A single thread can manage many suspended tasks by interleaving them: one task runs until it waits, then another task runs, then the first resumes later. This is concurrency without parallelism. It is especially useful for I/O-bound programs, where much time is spent waiting for files, sockets, timers, HTTP responses, user input, or database operations.

The purpose of this subsection is to connect generators, coroutines, and asynchronous event loops to the memory and execution model already established earlier. These are not magical language features. They are runtime control-flow mechanisms built on saved execution state. Understanding where the stack stops, where the runtime object begins, and who resumes the computation is the key to understanding lazy sequences, cooperative multitasking, and single-threaded asynchronous systems.

Generators: Lazy Sequence Evaluation, In-Flight Stream Interfaces, and State-Preserving Suspended Subroutines

A **generator** is a callable execution unit that can produce a sequence of values over time without computing and storing the whole sequence at once. The central idea is suspension. A normal function starts, runs until it returns or raises an exception, and then its execution frame disappears. A generator starts, runs until it reaches a suspension point, returns one value to the caller, and keeps enough internal state to continue later from the same point.

The basic model is:

</> PYTHON Listing 2.4.24: Generators: Lazy Sequence Evaluation, In-Flight Stream Interfaces, and State...

```
1  caller asks for next value
2      generator frame resumes
3      runs until yield
4      produces value
5      suspends
6  caller receives value
7
8  caller asks again
9      same generator frame resumes after previous yield
10     runs until next yield
11     produces value
12     suspends again
```

This is different from returning a list. A list is materialized: all elements are stored in memory before the caller consumes them. A generator is lazy: values are produced only when requested.

For example, this creates all values immediately:

```
values = [1, 2, 3, 4, 5]
```

A generator-like object can produce them one at a time:

```
next -> 1
next -> 2
next -> 3
next -> 4
next -> 5
finished
```

The main useful applications are:

- processing large files line by line;
- producing infinite or very large sequences;
- building data-processing pipelines;
- avoiding unnecessary list materialization;
- representing streams of events, rows, records, tokens, or packets;
- separating value production from value consumption.

Python / CPython: generator functions and suspended Python frames. In Python, a function containing `yield` becomes a generator function. Calling it does not run the body immediately. It creates a generator object.

</>PYTHON Listing 2.4.25: Python / CPython: generator functions and suspended Python frames.

```

1  def count_up_to(limit):
2      current = 1
3
4      while current <= limit:
5          yield current
6          current += 1
7
8
9  g = count_up_to(3)
10
11 print(next(g)) # 1
12 print(next(g)) # 2
13 print(next(g)) # 3

```

The execution sequence is:

</>PYTHON Listing 2.4.26: Python / CPython: generator functions and suspended Python frames.

```

1  count_up_to(3)
2      creates generator object
3      body has not run yet
4
5  next(g)
6      creates/resumes generator frame
7      current = 1
8      reaches yield current
9      returns 1 and suspends
10
11 next(g)
12     resumes after yield
13     current += 1
14     reaches yield current
15     returns 2 and suspends

```

At the CPython level, the generator object owns a suspended execution state. The old Python frame is not simply destroyed after the first `yield`. Its local variables and bytecode position remain associated with the generator.

Conceptually:

```

g
|
v
generator object
+-----+
| code object          |
| suspended bytecode position |
| local variable: current |
| local variable: limit  |
| evaluation stack state |
+-----+

```

When the generator is suspended, the native C stack used by CPython is not preserved as a raw stack segment for that generator. The generator's Python-level state is stored in interpreter-managed structures. Later, when `next(g)` is called again, CPython re-enters that saved Python execution state.

During active execution, the Python frame chain may look like:

top Python frame


```

+-----+
| frame: count_up_to      |
+-----+
| frame: caller/module body |
+-----+
bottom

```

After `yield`, the caller continues, and the generator frame is suspended inside the generator object:

```

active caller frame
+-----+
| frame: caller/module body |
+-----+

suspended generator object
+-----+
| saved frame: count_up_to   |
+-----+

```

This distinction is essential. A generator does not keep running in the background. It is paused. It advances only when the caller requests another value.

A practical Python example is reading a large file lazily:

</> PYTHON Listing 2.4.27: Python / CPython: generator functions and suspended Python frames.

```

1  def non_empty_lines(path):
2      with open(path, "r", encoding="utf-8") as file:
3          for line in file:
4              line = line.strip()
5
6              if line:
7                  yield line
8
9
10 for line in non_empty_lines("log.txt"):
11     print(line)

```

The useful application is memory-safe file processing. The file is not loaded entirely into a list. Each useful line is produced when the loop asks for it.

The `for` loop drives the generator through the iterator protocol:

```

iter(generator)
next(generator)
next(generator)
...
StopIteration

```

When the generator finishes normally, Python raises `StopIteration` internally to signal exhaustion to the loop machinery.

JavaScript / V8: generator functions and resumable function state. JavaScript supports generator functions using `function*` and `yield`.

</>JAVASCRIPT Listing 2.4.28: JavaScript / V8: generator functions and resumable function state.

```

1  function* countUpTo(limit) {
2      let current = 1;
3
4      while (current <= limit) {
5          yield current;
6          current += 1;
7      }
8  }
9
10 const g = countUpTo(3);
11
12 console.log(g.next()); // { value: 1, done: false }
13 console.log(g.next()); // { value: 2, done: false }
14 console.log(g.next()); // { value: 3, done: false }
15 console.log(g.next()); // { value: undefined, done: true }

```

The useful application is lazy traversal, custom iterable objects, streaming transformations, and controlled production of values.

Calling the generator function does not execute its body immediately:

```

countUpTo(3)
  returns generator object
  body not executed yet

```

Each `next()` resumes the generator until the next `yield`.

The V8-level conceptual structure is:

```

generator object
+-----+
| generator function code |
| suspended instruction position |
| local binding: current |
| local binding: limit |
| generator state |
+-----+

```

When the generator yields, its JavaScript call frame is not kept as an ordinary active stack frame. The resumable state is stored by the engine in heap-managed generator structures. Later, `next()` reconstructs or resumes the required execution state.

A practical JavaScript example is lazy ID generation:

</>JAVASCRIPT Listing 2.4.29: JavaScript / V8: generator functions and resumable function state.

```

1  function* idGenerator(prefix) {
2      let id = 1;
3
4      while (true) {
5          yield `${prefix}-${id}`;
6          id += 1;
7      }
8  }
9
10 const ids = idGenerator("user");
11
12 console.log(ids.next().value); // user-1
13 console.log(ids.next().value); // user-2
14 console.log(ids.next().value); // user-3

```

This is useful because the sequence is infinite. A list of all possible IDs cannot be materialized. The generator stores only the current state.

The active stack during one `next()` call is:

```
top JS frame
+-----+
| frame: idGenerator |
+-----+
| frame: caller      |
+-----+
bottom
```

After `yield`, the generator frame is suspended and removed from the ordinary active call stack:

caller continues

generator object remembers:

```
    prefix
    id
    suspended position
```

PHP / Zend Engine: generators with `yield`. PHP also supports generators with `yield`. A function containing `yield` returns a generator object instead of executing to completion immediately.

`</>PHP` Listing 2.4.30: PHP / Zend Engine: generators with `yield`.

```
1  <?php
2
3  function count_up_to($limit) {
4      $current = 1;
5
6      while ($current <= $limit) {
7          yield $current;
8          $current += 1;
9      }
10 }
11
12 foreach (count_up_to(3) as $value) {
13     echo $value . PHP_EOL;
14 }
```

The useful application is streaming data without building large arrays. For example, reading a file lazily:

</> PHP Listing 2.4.31: PHP / Zend Engine: generators with yield.

```

1  <?php
2
3  function non_empty_lines($path) {
4      $file = fopen($path, "r");
5
6      while (($line = fgets($file)) !== false) {
7          $line = trim($line);
8
9          if ($line !== "") {
10             yield $line;
11         }
12     }
13
14     fclose($file);
15 }
16
17 foreach (non_empty_lines("log.txt") as $line) {
18     echo $line . PHP_EOL;
19 }

```

Without a generator, one might build an array of all lines and return it. With a generator, each line is produced only when the `foreach` loop requests it.

At the Zend Engine level, the generator object preserves the function's execution state: local variables, current opcode position, and yielded value state. The ordinary active call frame does not remain on the native C stack while the generator is suspended. The engine stores the resumable state in runtime-managed generator structures.

Conceptually:

Generator object

```

+-----+
| compiled opcodes          |
| suspended opcode position |
| local variable: $current  |
| local variable: $limit    |
| current yielded value     |
+-----+

```

During one resume, the PHP frame is active:

top PHP frame

```

+-----+
| frame: count_up_to       |
+-----+
| frame: foreach/script body |
+-----+
bottom

```

After `yield`, the script body continues and the generator frame is suspended inside the generator object.

Java: no native generator syntax, but iterator objects emulate lazy production. Java does not have Python-style generator functions in the core language. The usual Java approach is to represent lazy production with an `Iterator`, `Iterable`, `Stream`, or a custom object that stores iteration state explicitly.

A manual iterator example:

</> JAVA Listing 2.4.32: Java: no native generator syntax, but iterator objects emulate lazy production.

```

1  import java.util.Iterator;
2  import java.util.NoSuchElementException;
3
4  class CountUpTo implements Iterator<Integer> {
5      private int current = 1;
6      private final int limit;
7
8      CountUpTo(int limit) {
9          this.limit = limit;
10     }
11
12     public boolean hasNext() {
13         return current <= limit;
14     }
15
16     public Integer next() {
17         if (!hasNext()) {
18             throw new NoSuchElementException();
19         }
20
21         int value = current;
22         current += 1;
23         return value;
24     }
25 }
26
27 public class Main {
28     public static void main(String[] args) {
29         Iterator<Integer> it = new CountUpTo(3);
30
31         while (it.hasNext()) {
32             System.out.println(it.next());
33         }
34     }
35 }

```

The useful application is still lazy traversal. The values are not stored in a list. The object stores the state needed to compute the next value.

However, the implementation is different from Python generators. Java does not suspend a method frame at a `yield` statement here. The state is manually stored in object fields:

CountUpTo object

```

+-----+
| current          |
| limit           |
+-----+

```

Each call to `next()` creates an ordinary JVM method frame:

top JVM frame

```

+-----+
| frame: CountUpTo.next      |
+-----+
| frame: Main.main          |
+-----+

```

bottom

When `next()` returns, that frame disappears. The persistent state is not the old stack frame. It is the object field `current` stored on the heap.

Java streams can also express lazy pipelines:

`</> JAVA` Listing 2.4.33: Java: no native generator syntax, but iterator objects emulate lazy production.

```

1  import java.util.stream.IntStream;
2
3  public class Main {
4      public static void main(String[] args) {
5          IntStream.iterate(1, x -> x + 1)
6                      .limit(3)
7                      .forEach(System.out::println);
8      }
9  }
```

The useful application is declarative lazy data processing. The stream pipeline stores transformation logic and pulls values as needed. Internally, Java stream implementations use objects, spliterators, lambdas, and library machinery, not preserved Java method stack frames in the Python generator sense.

C: emulating generators with explicit state machines. Standard C has no generator syntax and no automatic suspension of function frames. A C function call normally runs until it returns. Once it returns, its stack frame is gone. To emulate a generator, the programmer must store state explicitly.

A simple C generator-like object:

`</> C` Listing 2.4.34: C: emulating generators with explicit state machines.

```

1  #include <stdio.h>
2
3  struct counter {
4      int current;
5      int limit;
6  };
7
8  int counter_next(struct counter *c, int *out) {
9      if (c->current > c->limit) {
10         return 0;
11     }
12
13     *out = c->current;
14     c->current += 1;
15     return 1;
16 }
17
18 int main(void) {
19     struct counter c = { .current = 1, .limit = 3 };
20
21     int value;
22     while (counter_next(&c, &value)) {
23         printf("%d\n", value);
24     }
25
26     return 0;
27 }
```

The useful application is lazy production in embedded systems, parsers, stream processors, or low-level libraries where allocating a full list would be wasteful.

The state is explicit:

```

struct counter
+-----+
```

```
| current          |
| limit           |
+-----+

```

The stack behavior is ordinary C behavior:

```
main()
    calls counter_next()
        counter_next frame is created
        computes one value
        returns
    counter_next frame disappears

main()
    calls counter_next() again
        new counter_next frame is created
        uses saved state from struct
        returns

```

There is no suspended C stack frame. The persistent state lives in the `struct counter`, not in the old call frame.

For more complex control flow, C programmers sometimes build manual state machines with `switch`:

 Listing 2.4.35: C: emulating generators with explicit state machines.

```
1  #include <stdio.h>
2
3  struct generator {
4      int state;
5      int i;
6  };
7
8  int gen_next(struct generator *g, int *out) {
9      switch (g->state) {
10         case 0:
11             g->i = 1;
12
13         case 1:
14             if (g->i <= 3) {
15                 *out = g->i;
16                 g->i += 1;
17                 g->state = 1;
18                 return 1;
19             }
20
21             g->state = 2;
22
23         case 2:
24             return 0;
25     }
26
27     return 0;
28 }

```

This uses a stored program counter-like state variable. It is a crude manual version of what a generator runtime does automatically: remember where execution should continue.

Duff's Device is related only as a demonstration that C `switch` and loop control can be arranged in unusual ways to resume at different points in a block. It is not a true generator feature. However, the same idea of storing a state label and jumping into the right part of a function appears in coroutine/generator emulation macros.

A famous style of C coroutine macro uses `switch` and `__LINE__` to remember the resume point. Conceptually:

```
state says where to resume
switch(state):
    case previous_yield_location:
        continue execution there
```

This is powerful but fragile. It depends on source layout tricks and does not preserve ordinary C local variables automatically unless they are stored outside the function frame.

Bash: stream-like generation through output and pipelines. Bash has no generator objects or suspended shell function frames in the Python sense. However, shell programming often uses stream-like lazy processing through pipes. A command produces output gradually, and another command consumes it gradually.

Example:

```
generate_numbers() {
    i=1

    while [ "$i" -le 3 ]; do
        echo "$i"
        i=$((i + 1))
    done
}

generate_numbers | while read -r value; do
    echo "value: $value"
done
```

The useful application is pipeline processing. The producer does not need to store all output in an array. It writes lines. The consumer reads lines.

The implementation is not a generator frame. In many shells, the two sides of a pipeline run in separate processes or subshells:

```
process 1:
    generate_numbers writes to pipe

process 2:
    while read consumes from pipe
```

The operating system pipe provides buffering. The producer and consumer are coordinated by I/O blocking, not by a language-level `yield` instruction. If the pipe buffer fills, the producer blocks. If the pipe is empty, the consumer blocks.

A Bash function can also be called repeatedly, but it does not preserve a suspended frame automatically. Persistent state usually lives in variables, files, or external processes, not in a generator object.

Implementation comparison. Generators and generator-like mechanisms all solve the same practical problem: produce values gradually while remembering progress. The implementation differs sharply by language.

Language	Generator form	State and stack behavior
Python / CPython	Function with <code>yield</code>	Generator object preserves Python frame state, locals, and bytecode position; native C stack is not preserved while suspended
JavaScript / V8	<code>function*</code> with <code>yield</code>	Generator object preserves engine-managed continuation state and lexical environment; JS stack frame is suspended into heap-managed state
PHP / Zend	Function with <code>yield</code>	Generator object preserves opcode position and local variables; ordinary active call frame is not kept on native stack
Java	Iterator, Iterable, Stream	No built-in suspended method frame; state stored manually or by library objects on heap
C	Manual state machine, struct, function pointer patterns	No suspended C frame; programmer stores state explicitly in structs, often heap-allocated if it must escape
Bash	Pipes, command output streams, repeated reads	No generator object; OS pipes and child processes provide streaming behavior

The most important stack rule is this:

A real generator does not usually keep the old native call stack alive. It stores the resumable language-level state in a heap-managed generator object or equivalent runtime structure.

That is why a generator can survive after the function call that created it appears to have returned. The call did not return a final result. It returned a resumable object.

The difference between a generator and a list is therefore not only memory size. It is execution architecture. A list is stored data. A generator is suspended computation.

list:

all values already materialized

generator:

code + saved state + ability to produce next value

This prepares the transition to coroutines. A generator produces values outward to a caller. A coroutine generalizes the idea: execution can suspend and resume as part of a larger cooperative control-flow system, sometimes receiving values, waiting for events, or interleaving multiple in-flight tasks.

Coroutines: Non-Preemptive Cooperative Tasking, Symmetric Yield Transfers, and Context Interleaving

A **coroutine** is a resumable execution unit that can suspend itself and later continue from the same logical point. A generator is already a restricted coroutine: it yields values outward to its caller. A fuller coroutine can also receive values, cooperate with other execution units, and participate in a scheduler-controlled flow where several in-progress computations take turns.

The key word is **cooperative**. A coroutine is not usually interrupted by the operating system in the middle of an arbitrary instruction. It runs until it reaches an explicit suspension point such as `yield`, `await`, `send`, or a runtime-specific operation. At that point, it gives control back to a caller or scheduler.

```

coroutine A runs
    reaches suspension point
scheduler resumes coroutine B
    coroutine B reaches suspension point
scheduler resumes coroutine A
    coroutine A continues from where it stopped

```

This is non-preemptive tasking. The coroutine must cooperate. If it enters a long CPU loop and never suspends, other coroutines in the same runtime scheduler do not progress.

Python: generator-based coroutines with send. Before modern `async/await`, Python generators could already behave like simple coroutines because values can be sent back into them.

`</>PYTHON` Listing 2.4.36: Python: generator-based coroutines with send.

```

1  def accumulator():
2      total = 0
3
4      while True:
5          value = yield total
6          total += value
7
8
9  acc = accumulator()
10
11 print(next(acc))      # starts coroutine, yields 0
12 print(acc.send(5))    # sends 5, yields 5
13 print(acc.send(7))    # sends 7, yields 12

```

The useful application is stateful stream processing. The coroutine keeps `total` between resumptions without using a global variable.

At the CPython level, `accumulator()` creates a generator/coroutine-like object. The local variable `total`, the bytecode position, and the suspended execution state are stored inside interpreter-managed structures.

```

acc
|
v
generator/coroutine object
+-----+
| code object                |
| suspended bytecode position |
| local variable: total      |
| slot receiving sent value   |
+-----+

```

The native C stack is not kept alive while the coroutine is suspended. When `yield` executes, control returns to the caller. The Python frame state is preserved by the runtime object. When `send()` is called, CPython resumes that saved frame and injects the sent value as the result of the suspended `yield` expression.

During active execution:

```

top Python frame
+-----+
| frame: accumulator         |
+-----+
| frame: caller/module body  |

```

```
+-----+
bottom
```

After suspension:

active caller frame remains

acc object stores:

```
total
bytecode position
suspended yield point
```

A small cooperative scheduler can be sketched with generators:

</>PYTHON Listing 2.4.37: Python: generator-based coroutines with send.

```
1  def task(name, count):
2      for i in range(count):
3          print(name, i)
4          yield
5
6
7  tasks = [task("A", 3), task("B", 3)]
8
9  while tasks:
10     current = tasks.pop(0)
11
12     try:
13         next(current)
14         tasks.append(current)
15     except StopIteration:
16         pass
```

The output interleaves the tasks:

```
A 0
B 0
A 1
B 1
A 2
B 2
```

This is concurrency by interleaving, not parallelism. One Python task runs at a time. The scheduler advances each task until its next `yield`.

JavaScript: generator coroutines and explicit resumption. JavaScript generators can also receive values through `next(value)`.

</> JAVASCRIPT Listing 2.4.38: JavaScript: generator coroutines and explicit resumption.

```

1  function* accumulator() {
2      let total = 0;
3
4      while (true) {
5          const value = yield total;
6          total += value;
7      }
8  }
9
10 const acc = accumulator();
11
12 console.log(acc.next().value);    // 0
13 console.log(acc.next(5).value);  // 5
14 console.log(acc.next(7).value);  // 12

```

The useful application is resumable state machines, parser pipelines, controlled data producers, and cooperative simulations.

V8 stores the generator's resumable state in a heap-managed generator object:

generator object

```

+-----+
| generator function code      |
| suspended instruction position |
| local binding: total        |
| generator state              |
+-----+

```

When the generator is suspended, the ordinary JavaScript call stack is not still active. A later `next(...)` call creates/resumes the engine frame needed to continue from the saved point.

JavaScript's common asynchronous programming model later developed around promises and `async/await`, but generator coroutines are the simpler mechanical model: resume explicitly, run until `yield`, store state, return control.

PHP: generator coroutines with send. PHP generators also support sending values back into a suspended generator.

</> PHP Listing 2.4.39: PHP: generator coroutines with send.

```

1  <?php
2
3  function accumulator() {
4      $total = 0;
5
6      while (true) {
7          $value = yield $total;
8          $total += $value;
9      }
10 }
11
12 $acc = accumulator();
13
14 echo $acc->current() . PHP_EOL; // starts, yields 0
15
16 $acc->send(5);
17 echo $acc->current() . PHP_EOL; // 5
18
19 $acc->send(7);
20 echo $acc->current() . PHP_EOL; // 12

```

The useful application is cooperative pipelines and older generator-based async frameworks. Before native async syntax was common in many ecosystems, generator suspension was often used as a building block for user-space schedulers.

At the Zend Engine level, the generator object stores the suspended opcode position and local variables. The old PHP call frame is not kept as an active native stack frame.

Generator object

```
+-----+
| compiled opcodes          |
| suspended opcode position |
| local variable: $total    |
| current yielded value     |
+-----+
```

Each resume re-enters the generator's saved execution state until it yields again or finishes.

Lua: classic coroutine-style cooperative transfer. Lua is worth mentioning because it has direct coroutine support and is often used as the clean textbook example of cooperative coroutines.

 Listing 2.4.40: Lua: classic coroutine-style cooperative transfer.

```
1 co = coroutine.create(function ()
2     for i = 1, 3 do
3         print("coroutine", i)
4         coroutine.yield()
5     end
6 end)
7
8 coroutine.resume(co)
9 print("main")
10 coroutine.resume(co)
11 print("main again")
12 coroutine.resume(co)
```

The useful application is game scripting, embedded scripting, cooperative task systems, and lightweight state machines.

Lua coroutines are stackful at the Lua-runtime level: each coroutine has its own Lua call stack managed by the Lua VM. When it yields, that Lua stack is preserved by the runtime. This is stronger than Python-style generator suspension, which is usually a single suspended generator frame rather than a whole independent call stack of arbitrary depth.

Conceptually:

Lua coroutine object

```
+-----+
| Lua call stack            |
| current instruction position |
| local variables          |
| coroutine status         |
+-----+
```

The operating system still sees one native process and usually one native thread unless the host program creates more. Lua coroutine switching is runtime-level switching, not OS preemption.

Java: no direct language coroutine, but runtime-managed alternatives. Java does not have a direct yield-based coroutine syntax equivalent to Python generators or Lua coroutines. Similar behavior is usually expressed through objects, iterators, streams, callbacks, futures, or more recently virtual threads.

A manual cooperative task can be represented as an object with explicit state:

```

interface StepTask {
    boolean step();
}

class CounterTask implements StepTask {
    private final String name;
    private int current = 0;
    private final int limit;

    CounterTask(String name, int limit) {
        this.name = name;
        this.limit = limit;
    }

    public boolean step() {
        if (current >= limit) {
            return false;
        }

        System.out.println(name + " " + current);
        current++;
        return true;
    }
}

```

A scheduler can interleave tasks:

`</> JAVA` Listing 2.4.41: Java: no direct language coroutine, but runtime-managed alternatives.

```

1  import java.util.*;
2
3  public class Main {
4      public static void main(String[] args) {
5          Queue<StepTask> tasks = new ArrayDeque<>();
6
7          tasks.add(new CounterTask("A", 3));
8          tasks.add(new CounterTask("B", 3));
9
10         while (!tasks.isEmpty()) {
11             StepTask task = tasks.remove();
12
13             if (task.step()) {
14                 tasks.add(task);
15             }
16         }
17     }
18 }

```

The useful application is simulations, event loops, and manually scheduled state machines.

There is no suspended Java method frame here. Each call to `step()` creates an ordinary JVM frame, then returns. Persistent state lives in object fields:

CounterTask object

```

+-----+
| name   |
| current|
| limit  |
+-----+

```

Java virtual threads are different. They are JVM-managed lightweight threads that can be parked and resumed by the runtime. They are useful for writing blocking-looking server code with much lower thread overhead than traditional platform threads. However, they are not the same as lexical generator coroutines. They are scheduled by the JVM and integrate with blocking operations, while source code still looks like ordinary method calls.

C: coroutine emulation with explicit state machines or separate stacks. Standard C has no coroutine feature. A C function call uses the native call stack and normally runs until it returns. To emulate coroutines portably, the programmer stores the resume state manually.

A cooperative task as an explicit state machine:

`</> C` Listing 2.4.42: C: coroutine emulation with explicit state machines or separate stacks.

```

1  #include <stdio.h>
2
3  struct task {
4      const char *name;
5      int state;
6      int current;
7      int limit;
8  };
9
10 int task_step(struct task *t) {
11     switch (t->state) {
12         case 0:
13             t->current = 0;
14             t->state = 1;
15
16         case 1:
17             if (t->current < t->limit) {
18                 printf("%s %d\n", t->name, t->current);
19                 t->current++;
20                 return 1;
21             }
22
23             t->state = 2;
24
25         case 2:
26             return 0;
27     }
28
29     return 0;
30 }
31
32 int main(void) {
33     struct task a = {"A", 0, 0, 3};
34     struct task b = {"B", 0, 0, 3};
35
36     while (task_step(&a) | task_step(&b)) {
37         /* keep stepping while either task can run */
38     }
39
40     return 0;
41 }

```

The useful application is embedded systems, protocol parsers, cooperative schedulers, and low-level event loops.

The stack behavior is ordinary C:

main()

```

    calls task_step(&a)
        task_step frame created
        one step executes
        task_step returns
    frame disappears

main()
    calls task_step(&b)
        new task_step frame created
        one step executes
        task_step returns
    frame disappears

```

No C stack frame is suspended. The resume point is stored in `state`; local state is stored in the `struct task`.

Duff's Device is relevant as a C control-flow trick because it shows how `switch` labels can be interleaved with loop structure. Coroutine macros in C often use a similar idea: store a resume label, then enter a `switch` at the right case on the next call. But this is not true native coroutine support. It is manual transformation of a coroutine into a state machine.

Non-portable C coroutine systems can use separate stacks, assembly, POSIX `ucontext` APIs, Windows fibers, or library-level context switching. In that case, each coroutine may have its own native stack region:

```

coroutine A
+-----+
| native stack for A          |
+-----+

coroutine B
+-----+
| native stack for B          |
+-----+

```

Switching saves CPU registers and stack pointers, then restores another coroutine's saved context. This is much closer to stackful coroutines, but it is outside portable standard C.

Bash: cooperative interleaving through processes, pipes, and jobs. Bash has no coroutine object or suspended shell frame. It can still interleave work through loops, background jobs, pipes, and process coordination.

A simple cooperative-looking loop:

```

task_a_step() {
    echo "A $1"
}

task_b_step() {
    echo "B $1"
}

for i in 0 1 2; do
    task_a_step "$i"
    task_b_step "$i"
done

```


This is manual interleaving, not coroutine suspension. Each shell function call runs and returns normally. Its frame disappears.

Bash can also run background jobs:

```
long_task_a &
pid_a=$!

long_task_b &
pid_b=$!

wait "$pid_a"
wait "$pid_b"
```

This is not runtime-level coroutine scheduling. Bash asks the operating system to run child processes. Each child has its own process state and native stack. The OS scheduler, not Bash coroutine logic, interleaves them.

Pipelines also interleave processes:

```
producer | consumer
```

The producer and consumer may run concurrently as separate processes connected by an OS pipe. This is useful for streaming, but it is process-level coordination, not a Bash coroutine frame.

Symmetric and asymmetric transfer. Many common generators are **asymmetric**: they yield back to their immediate caller or scheduler. The caller resumes them later.

```
caller -> generator
generator -> caller
caller -> generator
generator -> caller
```

A more symmetric coroutine model allows coroutine A to transfer control directly to coroutine B, or to a scheduler that chooses the next coroutine.

```
coroutine A -> coroutine B
coroutine B -> coroutine C
coroutine C -> coroutine A
```

Many practical systems implement this through a scheduler:

```
coroutine A -> scheduler
scheduler -> coroutine B
coroutine B -> scheduler
scheduler -> coroutine C
```

The scheduler keeps a queue of resumable tasks. Each task runs until it suspends. No task is preempted in the middle of ordinary computation.

Comparison.

Language	Coroutine-like mechanism	Stack and state behavior
Python / CPython	Generators with <code>send</code> ; later <code>async/await</code>	Runtime stores suspended Python frame state; native C stack is not preserved while suspended
JavaScript / V8	Generators with <code>next(value)</code> ; <code>async</code> functions later	V8 stores generator/coroutine state in heap-managed engine structures
PHP / Zend	Generators with <code>send</code>	Zend generator object stores opcode position and locals
Lua	Built-in coroutines	Lua VM preserves coroutine call stack and state inside coroutine object
Java	Manual state objects, futures, virtual threads	Ordinary methods do not suspend; state is stored in objects, or JVM manages virtual-thread continuation state
C	Manual state machines; non-portable fibers/context switching	Portable C stores state explicitly; stackful libraries use separate stacks and saved register contexts
Bash	Manual interleaving, jobs, pipelines	No coroutine frames; shell functions return normally, or OS processes are scheduled separately

The important architectural point is that a coroutine separates *returning control* from *finishing execution*. A normal function returns and is done. A coroutine yields, pauses, and can later continue. This is possible only if the runtime or programmer stores enough state: current instruction position, local variables, pending values, and sometimes an entire call stack.

This prepares the next distinction: coroutine interleaving can create concurrency without parallelism. Several tasks may be in progress at once from the program's point of view, even though only one task is executing on the CPU at any exact moment.

Concurrency Without Parallelism: Interleaving Waiting Tasks on One Thread

Concurrency means that several tasks are in progress during the same time interval. **Parallelism** means that several tasks are executing at the exact same time on different CPU cores or hardware threads. These are not the same idea.

A single-threaded event loop can be concurrent without being parallel. It can manage many tasks that are waiting for timers, network data, file descriptors, user input, or other external events. At any exact moment, only one piece of user code runs on the thread. However, while one task is waiting, the event loop can let another task advance.

The general shape is:

```

task A starts
task A reaches waiting point
task A is suspended

task B starts
task B reaches waiting point
task B is suspended

external event for task A becomes ready
task A resumes

external event for task B becomes ready
task B resumes

```

The key is that waiting must be non-blocking. If a task blocks the only thread with a normal blocking operation, no other task can run.

Python / CPython: asyncio tasks on one event-loop thread. Python's `asyncio` model is a direct example of concurrency without parallelism. Multiple coroutines can be active, but only one coroutine executes Python bytecode at a time on the event-loop thread.

`</>PYTHON` Listing 2.4.43: Python / CPython: asyncio tasks on one event-loop thread.

```

1  import asyncio
2
3
4  async def worker(name, delay):
5      print(name, "start")
6
7      await asyncio.sleep(delay)
8
9      print(name, "done")
10
11
12  async def main():
13      await asyncio.gather(
14          worker("A", 1),
15          worker("B", 1),
16      )
17
18
19  asyncio.run(main())

```

The useful application is I/O-heavy programming: network clients, web servers, websocket handlers, database calls, crawlers, API clients, and other tasks where much time is spent waiting.

The output is approximately:

```

A start
B start
A done
B done

```

Both tasks finish after about one second, not because they ran in parallel on two cores, but because `asyncio.sleep()` does not block the thread. It registers a timer and gives control back to the event loop.

The stack behavior is:

```

event loop resumes worker A
    worker A runs until await
    worker A frame is suspended in a Task object

event loop resumes worker B
    worker B runs until await
    worker B frame is suspended in a Task object

timer for A becomes ready
    event loop resumes worker A

timer for B becomes ready
    event loop resumes worker B

```

At suspension time, the coroutine's Python frame is not active on the call stack. Its execution state is preserved in runtime-managed coroutine/task structures:

Task A

```
+-----+
| coroutine object      |
| suspended frame       |
| bytecode position at await |
| local variables       |
| awaited Future / timer |
+-----+
```

The wrong version is:

```
import time
import asyncio
```

```
async def bad_worker():
    time.sleep(5)      # blocks the whole event-loop thread
    print("done")
```

Here, `time.sleep()` blocks the OS thread. No other coroutine can advance during that time. The correct async version is:

```
await asyncio.sleep(5)
```

That suspends only the coroutine, not the whole event-loop thread.

JavaScript / V8: browser and Node.js event loops. JavaScript in browsers and Node.js is strongly associated with single-threaded concurrency. The engine runs one JavaScript call stack at a time, while the host environment manages timers, network events, DOM events, file events, and promise continuations.

Example with timers:

`</> JAVASCRIPT` Listing 2.4.44: JavaScript / V8: browser and Node.js event loops.

```
1  console.log("start");
2
3  setTimeout(() => {
4      console.log("timer A");
5  }, 1000);
6
7  setTimeout(() => {
8      console.log("timer B");
9  }, 1000);
10
11 console.log("after scheduling");
```

The useful application is browser UI programming, AJAX/fetch completion handling, timers, Node.js servers, websocket servers, and event-driven desktop applications built on Electron-style run-times.

The execution order is:

```
start
after scheduling
timer A
timer B
```

The first stack turn registers the callbacks:

```

top-level script
  calls setTimeout for A
  host stores callback A
  calls setTimeout for B
  host stores callback B
top-level script finishes

```

Later, when timers expire, the host event loop calls the callbacks:

```

event loop turn:
+-----+
| frame: callback A      |
+-----+
| frame: host dispatch   |
+-----+

next event loop turn:
+-----+
| frame: callback B      |
+-----+
| frame: host dispatch   |
+-----+

```

The old registration stack is gone. The callbacks survive because the host/runtime keeps references to function objects.

AJAX/fetch has the same architecture:

```

fetch("/data.json")
  .then(response => response.json())
  .then(data => {
    console.log(data);
  });

```

The network operation is not performed by continuously running JavaScript code. The host starts the operation, JavaScript returns to the event loop, and the callback resumes later when the result is ready.

PHP / Zend Engine: Fibers and user-space async frameworks. Modern PHP has Fiber, which allows cooperative suspension and resumption. A fiber is not an OS thread. It is a runtime-managed execution context.

</> PHP Listing 2.4.45: PHP / Zend Engine: Fibers and user-space async frameworks.

```

1  <?php
2
3  $fiberA = new Fiber(function () {
4      echo "A start\n";
5      Fiber::suspend();
6      echo "A done\n";
7  });
8
9  $fiberB = new Fiber(function () {
10     echo "B start\n";
11     Fiber::suspend();
12     echo "B done\n";
13 });
14
15 $fiberA->start();
16 $fiberB->start();
17
18 $fiberA->resume();
19 $fiberB->resume();

```

Expected Output

```

A start
B start
A done
B done

```

The useful application is user-space async frameworks, cooperative I/O libraries, request pipelines, and systems that want to write code in a sequential-looking style while still yielding control at waiting points.

At the Zend Engine level, the fiber preserves PHP execution state. When `Fiber::suspend()` is called, the currently running fiber stops and returns control to the caller/scheduler. Its PHP-level call stack and local variables are preserved by the runtime.

Conceptually:

Fiber A

```

+-----+
| suspended PHP execution context |
| local variables                  |
| current opcode position          |
+-----+

```

Fiber B

```

+-----+
| suspended PHP execution context |
| local variables                  |
| current opcode position          |
+-----+

```

Only one fiber runs at a time on the current thread. This is concurrency by cooperative interleaving, not CPU parallelism.

Java: single-thread event loops and callback dispatch. Java does not need coroutines to demonstrate concurrency without parallelism. A single-threaded event loop can run callbacks one at

a time. GUI systems and server frameworks often use this idea. For example, Swing has an Event Dispatch Thread; frameworks such as Netty and Vert.x use event-loop-style execution.

A small simplified Java event loop:

`</> JAVA` Listing 2.4.46: Java: single-thread event loops and callback dispatch.

```

1  import java.util.ArrayDeque;
2  import java.util.Queue;
3
4  public class EventLoop {
5      private final Queue<Runnable> queue = new ArrayDeque<>();
6
7      public void callSoon(Runnable task) {
8          queue.add(task);
9      }
10
11     public void run() {
12         while (!queue.isEmpty()) {
13             Runnable task = queue.remove();
14             task.run();
15         }
16     }
17
18     public static void main(String[] args) {
19         EventLoop loop = new EventLoop();
20
21         loop.callSoon(() -> System.out.println("A"));
22         loop.callSoon(() -> System.out.println("B"));
23         loop.callSoon(() -> System.out.println("C"));
24
25         loop.run();
26     }
27 }

```

The useful application is event dispatch: GUI actions, network events, timers, and framework callbacks.

The stack during one callback is:

```

top JVM frame
+-----+
| frame: lambda body          |
+-----+
| frame: Runnable.run         |
+-----+
| frame: EventLoop.run        |
+-----+
| frame: main                  |
+-----+
bottom

```

When the callback returns, its JVM method frame disappears. The event loop then invokes the next callback. There is no parallel execution here. The state of pending work lives in heap objects inside the queue.

In a real Java event-loop framework, the queue is fed by socket readiness, timers, completed operations, or other system events. The principle remains the same: one event-loop thread executes one callback at a time.

C: select/poll/epoll-style event loops. C can implement concurrency without parallelism through non-blocking I/O and an event loop. The program stores state for each connection and uses the oper-

ating system to wait until one or more file descriptors become ready.

A simplified `select()`-style sketch:

</>C Listing 2.4.47: C: select/poll/epoll-style event loops.

```

1  #include <sys/select.h>
2  #include <unistd.h>
3  #include <stdio.h>
4
5  void handle_stdin(void) {
6      char buffer[256];
7      ssize_t n = read(STDIN_FILENO, buffer, sizeof(buffer) - 1);
8
9      if (n > 0) {
10         buffer[n] = '\0';
11         printf("input: %s", buffer);
12     }
13 }
14
15 int main(void) {
16     while (1) {
17         fd_set read_set;
18
19         FD_ZERO(&read_set);
20         FD_SET(STDIN_FILENO, &read_set);
21
22         int max_fd = STDIN_FILENO;
23
24         select(max_fd + 1, &read_set, NULL, NULL, NULL);
25
26         if (FD_ISSET(STDIN_FILENO, &read_set)) {
27             handle_stdin();
28         }
29     }
30
31     return 0;
32 }

```

The useful application is single-threaded servers, terminal programs, network daemons, embedded event loops, and systems that must manage many waiting I/O sources without one thread per source.

The stack behavior is ordinary C:

```

main()
    calls select()
        kernel blocks process until fd is ready
    select returns

```

```

main()
    calls handle_stdin()
        handle_stdin frame is created
        reads available data
        returns
    handle_stdin frame disappears

```

```

main()
    loops again

```

There is no suspended C stack frame for each waiting file descriptor. Waiting state is stored in kernel data structures and in user-defined C structs.

A more realistic C event loop uses callbacks and explicit state:


```
struct watcher {
    int fd;
    void (*on_readable)(int fd, void *data);
    void *data;
};
```

The callback supplies behavior; the `data` pointer supplies state. This is the C equivalent of a runtime task object:

```
watcher
+-----+
| fd                |
| function pointer   |
| user data pointer  |
+-----+
```

The event loop does not preserve function frames. It repeatedly waits for readiness, then calls the appropriate handler.

Duff's Device is not an event loop and not a concurrency mechanism by itself. It is useful only as a reminder that C can manually encode unusual control-flow/state-machine behavior. In C event loops, the usual serious technique is explicit state structs plus callbacks, not preserved stack frames.

Bash: limited shell-level interleaving and process-based contrast. Bash does not provide coroutine objects or a general single-threaded event loop like `asyncio`. It can still perform simple event-like polling in the shell itself.

Example:

```
while true; do
    if read -t 1 -r line; then
        echo "input: $line"
    else
        echo "tick"
    fi
done
```

The useful application is simple supervisors, terminal scripts, polling loops, and small automation tasks.

The stack behavior is simple:

```
shell loop
    read with timeout
    if input exists, handle it
    otherwise run timeout branch
    loop again
```

There is no saved coroutine frame. State lives in shell variables.

Bash pipelines and background jobs are different:

```
producer | consumer
```

or:

```
long_task &
other_task &
wait
```

These use operating-system processes. They may run in parallel if the OS schedules them on different cores. That is not single-threaded concurrency inside Bash. It is process-level concurrency managed by the operating system.

The core implementation idea. Concurrency without parallelism works by separating **waiting** from **executing**. A task that waits must leave behind a resumable description of what should happen later.

That description may be:

- a Python coroutine frame stored in an `asyncio.Task`;
- a JavaScript callback or promise continuation stored by the host/runtime;
- a PHP fiber object;
- a Java `Runnable` object in an event-loop queue;
- a C struct containing file descriptor, callback, and user data;
- a shell variable/loop state in Bash.

The single-threaded event-loop pattern is:

```

ready queue / event source
    |
    v
event loop chooses one ready task
    |
    v
task runs on the only thread
    |
    +--> finishes
    |
    +--> registers waiting operation and suspends/returns
    |
    v
event loop chooses next ready task

```

Only one task has an active call stack at a time. Suspended or waiting tasks are represented by heap/runtime/kernel state, not by several simultaneously executing stacks on the same thread.

The comparison is:

Model	What is active?	Where waiting state lives
Python <code>asyncio</code>	One coroutine executes Python bytecode at a time	Task/coroutine objects, Futures, event loop timers/I/O registrations
JavaScript / V8	One JS call stack executes at a time	Host queues, promise continuations, callback references
PHP Fibers	One fiber runs at a time on the thread	Fiber objects preserve PHP execution context
Java event loop	One callback runs at a time	Heap queue of <code>Runnable</code> objects and framework event state
C event loop	One handler runs at a time	Kernel readiness state plus user structs and callback pointers
Bash polling loop	One shell command sequence runs at a time	Shell variables; external jobs are OS processes, not shell coroutines

The major limitation is CPU-bound work. If a task performs a long computation without yielding or returning to the event loop, it blocks the whole single-threaded system.

For example, in Python:

```

async def bad():
    while True:
        pass

```

In JavaScript:

```

while (true) {
    // blocks the event loop
}

```

In both cases, no other task can run on that thread. The operating system may still preempt the whole process and run other processes, but the runtime's internal tasks are stuck because the event loop has not regained control.

Therefore, concurrency without parallelism is excellent for waiting-heavy programs, not for CPU-heavy programs. It lets one thread manage many in-flight operations by storing suspended state and resuming tasks only when they can make progress.

Asynchronicity: Non-Blocking Event-Driven I/O Execution Profiles, Engine Event Loops, and Single-Threaded Concurrency Subsystems

Asynchronicity means that an operation is started now, but its result is handled later. The calling code does not wait by occupying the execution thread until the operation finishes. Instead, the program registers interest in the future result, returns control to a scheduler or event loop, and resumes the appropriate continuation when the result becomes available.

This is most useful for I/O. Disk, network, timers, user input, database requests, and subprocess output are slow compared with CPU execution. If a program blocks the only thread while waiting for such operations, no other task inside that thread can advance. Asynchronous execution avoids that by separating *starting* an operation from *handling* its completion.

The basic pattern is:

```

start operation
    operation is not ready yet
    register callback / future / task / continuation
    return to event loop

event loop waits for readiness or completion

operation becomes ready
    event loop resumes the waiting task
    result is processed

```

Asynchronicity is therefore not the same as parallelism. A single-threaded asynchronous runtime may manage many in-flight operations, but it still executes only one callback or coroutine at a time on that thread.

Python / CPython: `asyncio`, `await`, `tasks`, and `futures`. Python's `asyncio` library provides a standard event-loop model. An `async def` function creates a coroutine. The coroutine runs until it reaches an `await`. At that point, it suspends and gives control back to the event loop.

</>PYTHON Listing 2.4.48: Python / CPython: asyncio, await, tasks, and futures.

```

1  import asyncio
2
3
4  async def fetch_like_task(name, delay):
5      print(name, "started")
6
7      await asyncio.sleep(delay)
8
9      print(name, "finished")
10
11
12  async def main():
13      await asyncio.gather(
14          fetch_like_task("A", 1),
15          fetch_like_task("B", 1),
16          fetch_like_task("C", 1),
17      )
18
19
20  asyncio.run(main())

```

The useful application is waiting-heavy work: HTTP clients, websocket servers, database clients, queue consumers, API crawlers, and many open network connections.

The important line is:

```
await asyncio.sleep(delay)
```

This does not block the whole thread. It registers a timer with the event loop and suspends the current coroutine. The event loop can then run another coroutine.

The runtime state is roughly:

event loop

```

+-----+
| ready task queue           |
| timer registrations        |
| I/O readiness registrations |
+-----+

```

Task A

```

+-----+
| coroutine object           |
| suspended frame at await   |
| local variables            |
| awaited Future / timer     |
+-----+

```

The stack behavior is:

event loop resumes Task A

```

Python frame for fetch_like_task runs
reaches await
frame is suspended into coroutine/task state
control returns to event loop

```

event loop resumes Task B

```
same pattern
```

```
later timer for Task A is ready
    event loop resumes Task A after await
```

The suspended coroutine is not an active native stack frame while waiting. CPython preserves the Python-level frame state in runtime-managed coroutine/task structures. The native thread is free to return to the event loop.

A wrong version is:

```
import time

async def bad():
    time.sleep(5)
```

This blocks the event-loop thread. Other coroutines cannot run during the sleep. In asynchronous code, waiting operations must be expressed through awaitable non-blocking operations.

JavaScript / V8: host event loops, AJAX, promises, and async/await. JavaScript asynchronous execution is normally provided by a host environment around the engine. V8 executes JavaScript, but the browser, Node.js, or Electron-style host provides timers, network operations, file-system APIs, event queues, and the event loop.

A browser example using `fetch`:

`</> JAVASCRIPT` Listing 2.4.49: JavaScript / V8: host event loops, AJAX, promises, and async/await.

```
1  async function loadData() {
2      console.log("before request");
3
4      const response = await fetch("/data.json");
5      const data = await response.json();
6
7      console.log(data);
8  }
9
10 loadData();
11
12 console.log("after scheduling");
```

The useful application is AJAX-style loading: retrieving data from the server without freezing the page or reloading the document.

The control behavior is:

```
loadData starts
    logs "before request"
    starts fetch request
    reaches await
    loadData is suspended

top-level script continues
    logs "after scheduling"

network response arrives later
    host queues continuation
    event loop resumes loadData
```

During the wait, the JavaScript call stack is empty or running other work. The old stack is not preserved as an active stack. The continuation is represented by promise and async-function runtime state.

Conceptually:

async function state

```
+-----+
| suspended instruction position |
| local bindings                |
| pending Promise               |
+-----+
```

A Node.js example:

`</> JAVASCRIPT` Listing 2.4.50: JavaScript / V8: host event loops, AJAX, promises, and async/await.

```
1  const fs = require("fs/promises");
2
3  async function readFile() {
4      console.log("before read");
5
6      const text = await fs.readFile("notes.txt", "utf8");
7
8      console.log(text);
9  }
10
11  readFile();
12
13  console.log("after call");
```

The file operation is started through Node's host APIs. JavaScript execution returns to the event loop. When the operation completes, the promise is resolved and the async function continues.

The event-loop side is:

`</> JAVASCRIPT` Listing 2.4.51: JavaScript / V8: host event loops, AJAX, promises, and async/await.

```
1  host starts I/O operation
2      stores promise continuation
3
4  I/O completes
5      host queues microtask / continuation
6
7  V8 executes continuation
8      async function resumes after await
```

Again, V8 executes the JavaScript frame only when resumed. The host provides the asynchronous I/O machinery.

PHP: request-oriented execution, Fibers, and async frameworks. Traditional PHP web execution is often synchronous: a PHP-FPM worker receives a request, runs the PHP script, blocks during ordinary database or file operations, returns a response, and then handles another request. However, modern PHP also supports coroutine-like async systems through Fibers and async frameworks.

A simple Fiber sketch:

</> PHP Listing 2.4.52: PHP: request-oriented execution, Fibers, and async frameworks.

```

1  <?php
2
3  $fiber = new Fiber(function () {
4      echo "before wait\n";
5
6      Fiber::suspend("waiting");
7
8      echo "after wait\n";
9  });
10
11 $value = $fiber->start();
12 echo $value . PHP_EOL;
13
14 $fiber->resume();

```

This is not yet real I/O, but it shows the mechanism: the PHP execution context can suspend and later resume.

For actual async frameworks, the pattern is:

```

start non-blocking socket / timer / HTTP request
    suspend current fiber or promise continuation
    return to event loop

```

```

I/O ready or complete
    event loop resumes fiber / continuation

```

At the Zend Engine level, a Fiber preserves a PHP execution context:

Fiber object

```

+-----+
| suspended PHP call stack |
| local variables          |
| current opcode position  |
| status                   |
+-----+

```

The useful applications are high-concurrency servers, websocket systems, async HTTP clients, message consumers, and event-driven workers. The key difference from ordinary PHP request execution is that one worker process can keep many operations in flight and resume them when ready.

Java: NIO selectors, event loops, and completion callbacks. Java has several asynchronous styles. One low-level model is Java NIO, where a selector waits for readiness on multiple channels. A single thread can watch many sockets and process whichever ones become ready.

A simplified shape is:

```

Selector selector = Selector.open();

while (true) {
    selector.select();

    for (SelectionKey key : selector.selectedKeys()) {
        if (key.isReadable()) {
            // read available data
        }
    }
}

```

```

        if (key.isWritable()) {
            // write pending data
        }
    }

    selector.selectedKeys().clear();
}

```

The useful application is scalable network servers. Instead of one blocking thread per connection, the program keeps connection state in heap objects and lets the selector report which channels can make progress.

The stack behavior is:

```

event-loop thread
  calls selector.select()
    blocks in kernel until at least one channel is ready
  returns to Java code
  invokes handler logic for ready channels
  handler returns
  loop repeats

```

There is no preserved Java method frame per socket. The per-connection state lives in objects:

```

ConnectionState object
+-----+
| socket/channel reference |
| input buffer             |
| output buffer            |
| parser state             |
+-----+

```

Java also supports future/completion-style async code:

```

CompletableFuture
  .supplyAsync(() -> loadData())
  .thenApply(data -> transform(data))
  .thenAccept(result -> System.out.println(result));

```

This is useful for composing operations that complete later. However, this example may use worker threads from a pool, so it may involve parallelism depending on the executor. It is asynchronous, but not necessarily single-threaded. The key architectural idea is still delayed continuation: the next function is invoked when the previous result becomes available.

C: non-blocking file descriptors and explicit event loops. C has no built-in `async/await` syntax. Asynchronous I/O is usually built with operating-system APIs and explicit state machines.

A simplified non-blocking event loop uses readiness APIs such as `select`, `poll`, `epoll`, `kqueue`, or platform-specific completion mechanisms.

Conceptual C structure:

</> C Listing 2.4.53: C: non-blocking file descriptors and explicit event loops.

```

1  struct connection {
2      int fd;
3      char input_buffer[4096];
4      size_t input_used;
5      void (*on_readable)(struct connection *);
6  };
7
8  while (1) {
9      wait_for_ready_file_descriptors();
10
11     for each ready connection:
12         connection->on_readable(connection);
13 }

```

The useful application is high-performance servers, proxies, terminal multiplexers, embedded event systems, and network daemons.

The stack behavior is ordinary C:

```

main event loop
  waits in kernel readiness function
  returns with ready fd list
  calls on_readable(connection)
    handler frame is created
    reads available bytes
    updates connection struct
    returns
  handler frame disappears
  loop continues

```

There is no suspended C function frame for each connection. The state must be stored explicitly:

```

connection struct
+-----+
| file descriptor          |
| buffers                  |
| parser state             |
| callback pointer         |
| user data                |
+-----+

```

Duff's Device and similar tricks can be used to hand-write state-machine-like code, but serious C asynchronous systems usually rely on explicit state structs, callbacks, readiness APIs, or non-portable coroutine/fiber libraries.

A key distinction in C systems is readiness versus completion:

- **readiness:** the OS says a descriptor can be read or written without blocking;
- **completion:** the OS says an operation that was submitted earlier has completed.

APIs such as `select`, `poll`, and `epoll` are readiness-oriented. Some Windows I/O models and newer systems can be completion-oriented. The event-loop design changes, but the stack rule remains similar: waiting state is stored outside ordinary call frames.

Bash: asynchronous-looking process orchestration, not runtime async. Bash does not provide an `async/await` runtime. It can run background processes and wait for them later:

```
long_task_one &
pid_one=$!

long_task_two &
pid_two=$!

wait "$pid_one"
wait "$pid_two"
```

The useful application is shell-level orchestration: start several external commands, let the operating system run them, and collect their exit statuses.

This is not single-threaded runtime asynchronicity. Bash creates child processes. Each child has its own process state, native stack, and scheduler identity. The operating system may run them in parallel on multiple cores.

Bash can also use polling:

```
while true; do
    if read -t 1 -r line; then
        echo "input: $line"
    else
        echo "tick"
    fi
done
```

This is useful for small interactive scripts, but it is not a general asynchronous runtime. The shell loop remains explicit. State lives in shell variables and child process status, not in coroutine objects.

Event-loop architecture. A single-threaded asynchronous event loop has four main responsibilities:

- track operations that are waiting;
- ask the operating system or host which operations are ready or complete;
- place ready continuations into a queue;
- execute one continuation at a time.

The structure is:

```
event loop
+-----+
| ready queue          |
| timers               |
| I/O registrations   |
| pending futures/promises/tasks |
+-----+
      |
      v
choose one ready continuation
      |
      v
run it until it returns or suspends
      |
      v
repeat
```

Only one callback or coroutine owns the thread at a time. If it returns quickly or awaits properly, the system remains responsive. If it runs CPU-heavy code for too long, the event loop is blocked.

Comparison.

Language/runtime	Async mechanism	Stack and state model
Python / <code>asyncio</code>	Coroutines, Tasks, Futures, selector/proactor event loop	Coroutine frames suspend into Task/Future state; one event-loop thread executes one coroutine at a time
JavaScript / V8	Promises, <code>async/await</code> , host event loop, callbacks	Async function state and promise continuations are stored by engine/host; callback stack is created later
PHP / Zend	Fibers and async frameworks	Fiber preserves PHP execution context; event loop resumes fibers when I/O is ready
Java	NIO selectors, event-loop frameworks, futures/completion stages	Event-loop handlers use normal JVM frames; connection state lives in heap objects; futures may use thread pools
C	Non-blocking descriptors, callbacks, explicit state machines, OS readiness/completion APIs	No suspended C frame; state stored in structs; handlers create ordinary native frames when called
Bash	Background jobs, <code>wait</code> , polling loops	Mostly process orchestration; child processes have separate stacks; shell has no coroutine runtime

The central architectural rule is:

Asynchronous systems do not make waiting disappear. They move waiting out of the active call stack and into runtime, host, kernel, or heap-managed state.

That is the real mechanism. A blocking program waits by keeping the current call path occupied. An asynchronous program records what should happen later, returns to the event loop, and lets other ready work run. When the external event completes, the runtime reconstructs or resumes the appropriate continuation.

This is why async programming belongs in the same family as generators and coroutines. All of them depend on runtime-managed continuation state. The difference is purpose: generators suspend to produce values lazily; coroutines suspend to cooperate with other execution units; asynchronous tasks suspend because the next step depends on an external event that is not ready yet.

Chapter 3

Introduction to the Python Language Architecture

3.1 Historical Context and Design Evolution

3.1.1 Chronological Timeline: The Genesis, Origins, and Formal Release of Python (Guido van Rossum, 1989–1991)

Python did not appear as an isolated act of language invention. It appeared at a very specific historical crossing point: workstations were becoming powerful enough to host richer interactive environments, Unix culture had normalized small tools and scripting, distributed systems research needed glue code around large native components, and programmers were increasingly tired of writing every utility in C. Python belongs to this moment. It is not a language born from the dream of replacing low-level systems programming. It is a language born from the practical need to stand above it.

The central figure in this origin is Guido van Rossum, a Dutch programmer trained in mathematics and computer science and working at Centrum Wiskunde & Informatica (CWI) in Amsterdam. Python's early character is easier to understand when one remembers that it was not born in the commercial software culture of Silicon Valley, nor in the pure systems culture of Bell Labs, nor in the academic formalism of a language theory department alone. It came from a European research institute, from a programmer with mathematical training, working near operating systems, distributed systems, and experimental programming environments. The result was a language with a strange mixture of traits: mathematically tidy in some areas, pragmatic in others, strongly opinionated about readability, but also full of decisions that were not inevitable laws of programming language design.

The most important ancestor of Python was ABC. ABC was an educational and high-level language developed at CWI before Python. It had several properties that later reappeared in Python: readable syntax, high-level data structures, interactive use, and an attempt to make programming less ceremonious than in languages such as C or Pascal. But ABC was also too closed, too abstract, and insufficiently connected to the surrounding operating-system world. It was elegant, but it was not enough of a working programmer's tool. It did not fit well when one needed to talk to files, processes, servers, and real system interfaces.

This tension explains Python's first historical necessity. Python was not created because the world lacked another pretty syntax. It was created because there was a gap between two unsatisfactory poles. On one side stood C: powerful, direct, close to the operating system, but verbose and dangerous for everyday utility work. On the other side stood ABC: pleasant, high-level, and structured, but not sufficiently useful as a system-facing tool. Python began as an attempt to keep the usability lessons of ABC while restoring contact with the operating system.

The usual origin date is late December 1989. During a holiday period, van Rossum began implementing a new language that would take inspiration from ABC but would be more extensible and more practical for writing utilities around the Amoeba distributed operating system. The detail is important: the first Python was not primarily imagined as a teaching toy, nor as a scientific-computing platform, nor as a web language. Those identities came later. At birth, Python was a tool-building

language for a programmer surrounded by C, Unix-like workflows, and experimental operating-system research.

The name itself also tells part of the story. Python was named after *Monty Python's Flying Circus*, not after the snake. This is culturally revealing. The name was informal, ironic, memorable, and slightly unserious. That tone became part of Python culture: technical seriousness wrapped in anti-ceremonial presentation. This matters because Python later developed a culture that often avoided the heavy institutional tone of languages designed by committees. It preferred examples, jokes, mailing-list decisions, practical aphorisms, and community convention. That culture was not always rigorous, and sometimes it became smug, but it helped Python feel like a language used by people rather than issued by a ministry.

By the first months of 1990, a working version existed. One of the earliest implementation pieces was a parser generator called `pgen`. This fact matters for the present chapter because it already places Python in the architecture discussed previously: Python was not merely a loose interpreter reading text line by line. From the beginning, it required a front-end structure: lexical rules, grammar, parsing, internal representation, and later execution inside a runtime. Python's later path toward source files, abstract syntax trees, code objects, bytecode, frames, and a virtual-machine loop was not an accidental decoration added to a scripting language. It followed naturally from the decision to build a real language implementation, not only a command macro processor.

During 1990, Python was used internally at CWI by a small group of programmers. This internal use is another important historical filter. Languages designed only on paper tend to optimize for conceptual purity. Languages used by their own authors for daily tasks are forced to become tolerable. Python's early shape was therefore influenced by immediate use: interactive experimentation, small utilities, rapid feedback, readable source files, and enough access to the host system to be useful outside a sealed educational environment.

On 20 February 1991, Python was released publicly to the `alt.sources` Usenet newsgroup as version 0.9.0. This was not a polished corporate product launch. It was a source distribution posted to a programmer community. That matters because Python entered the world through the culture of shared source code, mailing lists, patches, and peer feedback. The language was effectively open-source before the modern term had stabilized. Its early growth depended not on advertising but on people trying it, complaining about it, extending it, and sending feedback.

The first public Python already contained many of the seeds of the later language. It had functions, modules, exceptions, core container types, and a class mechanism. This is why Python should not be introduced as if it began as a trivial shell-script replacement and only later became a serious language. It was small, but it was structurally ambitious. Its early implementation already treated programs as organized runtime objects: modules, functions, exceptions, classes, and data structures. The later CPython machinery discussed in this chapter grows from that beginning.

The 1989–1991 origin can therefore be read as a sequence of practical pressures rather than as a heroic myth. ABC provided the high-level taste. C and Unix-like system work provided the discomfort. Amoeba provided the concrete need for system utilities. CWI provided the research environment. Van Rossum's mathematical and programming background provided the habit of structural clarity. Usenet provided the first public distribution channel. The growing availability of capable workstations made such a managed language increasingly acceptable in practice.

This also prevents two opposite mistakes. The first mistake is to treat Python as a miracle of simplicity, as if its later success proves that all of its design decisions were obviously correct. They were not. Some were elegant, some were pragmatic, some were cultural, and some were accidental. The second mistake is to dismiss Python as merely a slow scripting language with pretty formatting. That is also wrong. Python survived because it occupied a real architectural layer: above native compiled code, below application logic, close enough to the operating system to automate real work, and high-level enough to let programmers express structures quickly.

For a student coming from C, this historical origin explains why Python must be studied in two layers. At the surface, it presents a readable language with indentation, dynamic names, built-in containers, functions, modules, and exceptions. Underneath, it is a C program implementing a managed runtime, object system, parser, compiler, bytecode evaluator, allocator, and import mechanism. Python's history and Python's architecture are therefore connected. The language appeared because

programmers needed more abstraction than C gave them for many tasks; it succeeded because the abstraction remained attached to the real machine through the CPython runtime and the extension ecosystem that grew around it.

3.1.2 Design Philosophy and Tensions: Reading “The Zen of Python” Critically

Python culture is often summarized through a short text known as *The Zen of Python*. It can be displayed inside the interpreter by writing:

```
import this
```

This is already a very Pythonic joke: a philosophical statement hidden behind an import statement, presented as if wisdom itself were a module. The text was written by Tim Peters and later formalized as PEP 20. It contains aphorisms such as “Beautiful is better than ugly”, “Explicit is better than implicit”, “Simple is better than complex”, and “There should be one— and preferably only one —obvious way to do it.” These lines are often quoted as if they were neutral engineering principles. They are not neutral. They are cultural positions.

To read *The Zen of Python* correctly, one must first understand what it was reacting against. Python did not evolve in a vacuum. It appeared in a world where Perl was one of the dominant scripting languages. Perl was powerful, flexible, compact, and extremely useful, but it also became famous for allowing dense, cryptic, highly individualized code. Perl culture often celebrated the idea that “there is more than one way to do it.” Python culture pushed in the opposite direction. The Zen is therefore not only a description of Python. It is also an ugly, partly comic jab at Perl.

This matters because Python’s design philosophy is not merely technical. It is polemical. It says: source code should not be a private puzzle. The programmer should not show cleverness by compressing meaning into unreadable symbolic density. The language should push the writer toward code that another human can follow. In that sense, Python’s readability culture is not decorative. It is a social rule imposed through syntax, naming convention, library style, and community expectation.

The most famous expression of this culture is indentation-based block structure. Python does not use curly braces to mark blocks. It uses indentation. Technically, this means that whitespace is not merely visual layout; it becomes part of the lexical and syntactic structure of the program. The tokenizer emits indentation and dedentation information, and the parser uses that information to recognize nested blocks. A Python block is therefore not only displayed by indentation. It is defined by indentation.

This design has a rational side. In C-like languages, indentation and block structure can disagree:

```
if (x > 0)
    y = 1;
    z = 2;
```

A human reader may visually group both assignments under the condition, while the compiler only attaches the first statement to the `if`. Python prevents this exact mismatch by making the visual structure and syntactic structure the same thing. What the indentation says is what the parser sees.

But the decision also has a theatrical side. It is a form of syntactic terribilism: a deliberate refusal of the braced-language convention, not because braces are technically impossible or fundamentally broken, but because the language wants to force a style. Python does not merely recommend indentation. It weaponizes indentation. That choice made Python visually clean, but it also made layout errors semantic errors, made copy-paste from badly formatted sources more dangerous, and created a language where invisible whitespace can decide program structure.

This is the first major tension in Python design: readability is improved by reducing visual noise, but the reduction is achieved by giving layout grammatical authority. The language becomes cleaner by becoming stricter in a place where many earlier languages were permissive.

Another core Python principle is that explicitness is preferred over hidden machinery. At the surface level, this explains many Python habits: named imports, visible method calls, readable standard-library functions, and a preference for direct expression over symbolic compression. For example, Python generally prefers:

```
for item in collection:
    process(item)
```

over a more cryptic style based on pointer arithmetic, macro expansion, or punctuation-heavy operators. This is one of the places where Python clearly separates itself from C. C exposes memory and machine-level structure directly; Python exposes program intention more directly, while moving memory structure into the runtime.

However, Python is not actually free of implicit behavior. It only moves implicit behavior into controlled runtime protocols. Operators call special methods. Attribute access may trigger descriptors. Iteration calls `__iter__()` and `__next__()`. Context managers call `__enter__()` and `__exit__()`. Object construction passes through `__new__()` and `__init__()`. Imports execute module code. Function default values are created once, at function definition time. Names are resolved through namespace lookup rules. In other words, Python is not a language without hidden machinery. It is a language where the hidden machinery is usually regularized through object protocols.

This produces a second tension: Python claims explicitness, but its power depends on implicit runtime dispatch. The difference is that Python tries to make the implicit mechanisms inspectable and conventional. The programmer does not see every call to `__iter__()` during a `for` loop, but the mechanism is not mystical. It is part of the object model and can be explained.

The aphorism “Simple is better than complex” also requires careful reading. Python often makes simple programs simple, but it does not eliminate complexity. It relocates complexity. A list append looks simple because the list object, allocator, capacity management, reference storage, and error handling are managed inside CPython. A function call looks simple because frame creation, argument binding, local namespace management, bytecode execution, and return handling are hidden behind the call syntax. An import looks simple because module search paths, caches, bytecode artifacts, package initialization, and execution of module bodies are handled by the import system.

Thus, Python simplicity is not the absence of machinery. It is the concealment of machinery behind stable abstractions. This is useful, but it can mislead beginners. A student may think Python variables are boxes, that assignment copies values, that lists contain values directly, or that importing a module merely “makes it available.” All of these explanations are too shallow. Python is simple at the notation level because the runtime performs a large amount of work.

The line “There should be one obvious way to do it” is also more cultural than factual. Real Python often has many ways to do the same thing. Strings can be formatted with concatenation, `%` formatting, `str.format()`, f-strings, templates, and manual joins. Files can be handled directly, through context managers, through `pathlib`, through binary streams, or through higher-level libraries. Iteration can be written with loops, comprehensions, generator expressions, `map()`, `filter()`, or library pipelines.

The useful interpretation is not that Python literally provides one method. The useful interpretation is that Python culture prefers one locally readable method in ordinary code. It discourages gratuitous cleverness. It does not eliminate alternatives; it tries to make the ordinary path recognizable.

This is a third tension: Python values uniform readability, but its ecosystem is large enough that uniformity is never complete. The language core may prefer clarity, while the ecosystem contains web frameworks, numerical libraries, machine-learning libraries, asynchronous frameworks, packaging tools, and metaprogramming systems that each develop their own idioms. Modern Python culture is therefore not one culture. It is a federation of subcultures held together by a common runtime and syntax.

Python’s design philosophy also carries a strong preference for programmer convenience over early mechanical certainty. Unlike C, Python does not require most type decisions to be fixed before execution. Names are bound to objects at runtime. The same name can later be rebound to another object of a different type. Operations are checked when they execute, not when the source file is compiled in the C sense. This makes exploratory programming, scripting, and interactive work easier. It also means that some errors appear only when a particular execution path is reached.

For example:

```
x = 10
x = "ten"
```

This is not a type error in Python. The name `x` first refers to an integer object, then to a string object. The name does not have a fixed C-like storage type. The object has the type. This is one of the deepest architectural differences between Python and C, and it follows naturally from Python's runtime-centered design.

The gain is flexibility. The cost is that the runtime must carry more responsibility. Every object must know its type. Operations must inspect objects and dispatch behavior dynamically. Function calls, attribute access, operator behavior, exception routing, and iteration all depend on runtime objects rather than compile-time fixed layouts. Python culture often celebrates the surface freedom, but the technical reality is that this freedom is purchased with runtime machinery.

This gives Python one of its central identities: it is not anti-structure, but it is anti-ceremony. It wants structure to exist in objects, protocols, modules, and conventions, not in heavy declarations written before every operation. Compared with C, Python removes much of the visible scaffolding. Compared with Perl, it tries to reduce symbolic chaos. Compared with Java, it avoids forcing every small piece of behavior into class-heavy architecture. Compared with shell scripting, it provides a real object model and a large structured standard library.

The problem is that anti-ceremony can become its own ideology. Python code can be beautifully readable, but Python culture can also become overconfident in readability as a substitute for understanding. A line of Python may be easy to read and still hide a large amount of execution behavior. A beginner may understand what a line appears to do without understanding what the interpreter must allocate, look up, call, compare, catch, or release in order to do it.

That is why this chapter does not treat *The Zen of Python* as scripture. It treats it as a cultural artifact. It tells us how Python wants to see itself: readable, explicit, simple, practical, and resistant to unnecessary cleverness. But it also reveals the tensions that define the language: visual cleanliness versus syntactic whitespace, explicit source code versus implicit runtime protocols, simplicity of notation versus complexity of implementation, dynamic flexibility versus delayed error detection, and cultural uniformity versus ecosystem diversity.

For a C programmer, the correct conclusion is not that Python is “easier” in a childish sense. The correct conclusion is that Python moves effort from the programmer's source code into the runtime. C asks the programmer to describe memory, types, compilation units, headers, allocation, and deallocation more directly. Python asks the programmer to trust a managed object system, a bytecode interpreter, a namespace model, an allocator, an import system, and a large set of protocols. The surface becomes lighter because the runtime becomes heavier.

This is the philosophical bridge into the technical part of the chapter. Python's culture cannot be separated from Python's implementation. The readable syntax is supported by tokenization and parsing rules. Dynamic names require namespace dictionaries and frame objects. “Everything is an object” requires a common object header and type-pointer architecture. Flexible operations require runtime dispatch. Automatic memory behavior requires reference counting, garbage collection, and allocator subsystems. The philosophy is visible at the surface, but it is paid for under the bonnet.

Therefore, *The Zen of Python* should be read neither as propaganda nor as a joke alone. It is a compact record of the language's self-image and historical argument. It explains why Python rejected some older forms of programming-language cleverness, why it became attractive to working programmers, and why it also developed its own dogmas. A serious reader should keep both sides in view: Python's design philosophy made the language unusually readable and productive, but every readability decision has a runtime, syntactic, cultural, or pedagogical cost.

3.1.3 Why Python Spread: Scripting, Web Frameworks, Scientific Computing, Research Workflows, and Package Ecosystem Expansion

Python spread because it occupied a useful middle layer. It was not the fastest language, the most formally pure language, or the most revolutionary language. It spread because many programmers had daily problems that were too annoying to solve in C, too large for shell scripts, too system-facing for sealed educational languages, and too dependent on glue logic for rigid compiled workflows. Python became attractive because it sat between these worlds: high-level enough to write quickly, but close enough to the operating system and native libraries to do real work.

Its expansion was therefore not caused by syntax alone. Python became a control language: it opened files, moved data, called programs, connected services, wrapped native libraries, automated workflows, launched experiments, and coordinated systems written in other languages. In many domains, Python was not the engine itself. It was the dashboard, wiring, and control surface above the engine.

Scripting: The First Practical Expansion Layer

The first major field of expansion was scripting. Programmers needed to rename files, transform text, inspect logs, generate reports, configure systems, call external tools, copy data between formats, test network services, and automate repetitive actions. Shell scripts were useful, but fragile for complex data structures. Perl was powerful, but often culturally tolerant of dense symbolic code. C was too heavy for everyday automation.

Python offered a practical compromise: readable scripts with real functions, modules, exceptions, files, sockets, regular expressions, dictionaries, lists, and operating-system interfaces. A programmer could begin with one small file and later split it into functions, modules, or packages. This gradual growth mattered. Python did not force a script to begin as a large software project, but it also gave the script enough structure to survive if it became larger.

The standard library amplified this effect. File handling, path manipulation, text processing, serialization, dates, compression, networking, subprocess management, and many other tools were immediately available. The phrase “batteries included” should not be read as mythology. It simply means that a fresh Python installation already contained enough machinery to solve many practical tasks before installing anything else.

Web Frameworks: Python as Application Glue

The web created another natural opening. Web applications often do not spend most of their time performing raw numerical computation. They receive requests, validate input, query databases, build responses, handle sessions, render templates, and coordinate external services. This is coordination work, and Python is good at coordination work.

Python web development passed through many forms: CGI scripts, Zope, CherryPy, Twisted, Pylons, TurboGears, Django, Flask, Pyramid, FastAPI, and others. The frameworks differed, but the reason Python fit the domain remained stable: web programming rewards fast iteration, readable control flow, and strong libraries for text, networking, database access, and structured data.

Django was especially important because it showed that Python could support large, coherent web applications, not only loose scripts. It provided routing, models, templates, forms, administrative tools, and deployment conventions. Flask later showed the opposite style: a small core around which the programmer could assemble chosen components. Together, these frameworks helped Python cover both integrated and minimalist web development cultures.

Scientific Computing: Python Above Native Kernels

At first glance, Python’s success in scientific computing seems strange. Ordinary Python loops are slow compared with C or Fortran loops. Each operation may involve bytecode execution, dynamic type handling, object dispatch, reference counting, and runtime checks. For raw numerical loops, this overhead is real.

The explanation is architectural. Scientific Python did not win by making every numerical operation pure Python. It won by placing Python above optimized native kernels. NumPy arrays, for example, are not ordinary Python lists. They expose structured native memory through Python objects. Large array operations can run in compiled C or Fortran-level code while the user writes compact Python expressions.

This created a powerful division of labor. Python handled experiment logic, data loading, orchestration, visualization, and interactive control. Native libraries handled heavy computation. SciPy, Matplotlib, pandas, scikit-learn, PyTorch, TensorFlow, and related tools expanded this pattern. Python became the workbench, not necessarily the innermost computation loop.

For a C programmer, this is the important point: Python's scientific success is not proof that Python has no runtime cost. It is proof that a high-level runtime can be extremely effective when it controls lower-level optimized code.

Research Workflows and Notebooks

Research programming is often exploratory. The researcher does not always know the final structure of the program before starting. They inspect data, try transformations, plot intermediate results, adjust parameters, test hypotheses, and gradually convert experiments into scripts or reusable code.

Python fit this style well. The interactive interpreter, IPython, and later notebook environments made Python attractive for teaching, experimentation, and research communication. A notebook could combine code, output, plots, notes, and explanations in one document. This made the computational path visible, not only the final result.

However, notebooks also introduced a serious weakness. Their visible order is not always their execution order. Hidden kernel state can affect results. Cells may be run, changed, skipped, and rerun in ways that the final document does not clearly show. Notebooks helped Python spread through research culture, but they also require discipline if the result is meant to become reproducible software.

Package Ecosystem Expansion

A language spreads faster when useful code is easy to share. Python's package ecosystem became one of its strongest forces. PyPI, `pip`, virtual environments, wheels, source distributions, and later packaging standards made it possible to install and reuse community code across many domains.

This changed what it meant to choose Python. Choosing Python no longer meant choosing only the core language. It meant choosing access to web frameworks, HTTP clients, parsers, testing tools, plotting libraries, numerical libraries, database connectors, image-processing tools, machine-learning systems, command-line builders, and infrastructure utilities.

The ecosystem also helped Python spread outside professional programming. Scientists, teachers, journalists, economists, biologists, linguists, and data analysts could use packages written by specialists without writing every component themselves. Python became a shared interface to other people's work.

But packaging also became one of Python's persistent pain points. Pure Python packages are relatively easy to distribute. Packages with native dependencies are harder. Scientific packages may depend on C, C++, Fortran, BLAS, LAPACK, GPU libraries, or platform-specific binaries. This is why tools such as `conda` became important: they manage not only Python packages, but also binary dependencies and complete environments.

The Cost of Easy Beginnings

Python often makes beginnings easy. A script can be written quickly. A notebook can be started without project structure. A web service can be prototyped in a few files. A scientific experiment can be assembled interactively.

The cost appears later. A quick script can become a fragile production tool. A notebook can become an undocumented pipeline. A dynamic web application can become difficult to refactor. A package environment can become hard to reproduce. Python lowers the entry cost, but it does not automatically impose long-term discipline.

This explains several later developments in Python culture: type hints, testing frameworks, virtual environments, packaging standards, linters, formatters, asynchronous frameworks, and deployment tools. These were not signs that the original language philosophy was complete. They were responses to Python being used in larger, more serious, more durable systems than its early scripting identity suggested.

Why the Spread Was Durable

Python's spread was durable because it was porous. It could call native code. Native code could expose Python interfaces. C extensions, Cython, foreign-function interfaces, and binding systems

allowed Python to sit above efficient implementations. This meant Python did not need to win every performance contest directly. It only needed to be a good coordination layer over code that was already fast.

This also explains why Python entered so many communities. System administrators used it for automation. Web developers used it for applications. Scientists used it for computation and data. Researchers used it for exploration. Teachers used it because the syntax was readable. Machine-learning engineers used it because the libraries gathered there.

The result is that modern Python is not one culture. It is a federation of cultures sharing one language, one dominant runtime, and one enormous ecosystem.

For this course, the main conclusion is architectural. Python spread because it became a managed runtime language above native execution, but not isolated from native execution. The operating system still runs a native CPython process. CPython still allocates objects, manages reference counts, executes bytecode, calls native libraries, loads extension modules, and exposes system services. Python looks light at the surface because the runtime and ecosystem carry much of the weight underneath.

This prepares the transition into the next sections. Python's spread was not only a story of readable syntax. It was a story of runtime architecture, ecosystem architecture, and cultural architecture working together. To understand Python seriously, one must understand all three.

3.2 Python Implementations and the Role of CPython

Python is often spoken about as if it were one concrete program, but this is only partly true. The language defines the visible rules: syntax, objects, assignment, functions, classes, exceptions, modules, and the expected behavior of built-in types. An implementation is the machinery that makes those rules execute on a real machine or on another host runtime.

This distinction matters because Python can be implemented in more than one way. CPython executes Python through a C-based runtime and bytecode interpreter. PyPy uses a different runtime strategy with just-in-time compilation. Jython and IronPython connect Python semantics to the Java and .NET worlds. Other tools compile, translate, specialize, or package Python code for narrower purposes.

However, this plurality should not obscure the practical fact: CPython is the center of ordinary Python. It is the reference implementation, the default runtime for most users, and the target assumed by much of the package ecosystem. Therefore, before we study Python objects, frames, bytecode, memory management, and execution, we must first separate Python as a language from CPython as the concrete runtime that most programmers actually encounter.

3.2.1 Python as a Language vs. CPython as the Reference Implementation

Before studying Python's runtime architecture, one distinction must be made very clearly: Python and CPython are not the same thing. Python is the programming language. CPython is the dominant implementation of that language.

The difference is similar to the difference between the C language and a particular C compiler. C is defined by a language standard and a set of rules: syntax, types, expressions, declarations, undefined behaviors, translation phases, and so on. GCC or Clang are concrete implementations that accept C source code and turn it into executable behavior. In the same way, Python is the language described by its syntax, semantics, data model, standard behavior, and library expectations. CPython is the concrete runtime system, written mostly in C, that parses Python source code, compiles it to bytecode, creates Python objects, manages memory, and executes the program.

This distinction is not academic decoration. It prevents a major confusion. When a programmer says "Python does reference counting," that statement is not fully precise. CPython uses reference counting as its primary object lifetime mechanism. The Python language itself does not require every implementation to use reference counting. Another implementation may use a different garbage collector and still be a valid Python implementation if it preserves the observable language behavior expected by Python programs.

The Language: Rules Visible to the Programmer

The Python language defines what source code means. It defines that indentation creates block structure, that assignment binds names to objects, that functions are objects, that classes create type objects, that exceptions propagate through call frames, that iteration follows the iterator protocol, and that operators are resolved through the object data model.

For example:

```
a = [1, 2, 3]
b = a
b.append(4)
```

The language-level explanation is that `a` is bound to a list object, `b` is then bound to the same list object, and `append` mutates that object. Therefore, observing `a` later shows the mutation. This behavior belongs to Python as a language. Any serious Python implementation must preserve it.

But the language does not require that this list object be represented internally by one exact C structure, allocated by one exact allocator, or reclaimed through one exact reference-counting path. Those are implementation details.

The language therefore answers questions such as:

- What is valid Python syntax?
- What does assignment mean?
- How are names resolved?
- How do functions, classes, modules, and exceptions behave?
- What operations must built-in objects support?
- What behavior should a program observe when it runs?

These are the rules that a programmer normally experiences.

The Implementation: The Machine That Realizes the Rules

An implementation answers a different question: how are those rules made to happen on a real machine?

CPython answers that question in a very concrete way. It is a native program, compiled from C source code into an executable such as `python` or `python.exe`. When the operating system runs Python code, it does not run the `.py` file directly as machine code. It runs the CPython executable. CPython then reads the Python source file, tokenizes it, parses it, builds an abstract syntax tree, compiles that tree into bytecode, creates code objects and frame objects, and executes bytecode instructions inside its interpreter loop.

So when the user writes:

```
python script.py
```

the operating system starts CPython. CPython starts the Python program.

This distinction repeats the architecture established in the previous chapter. At the operating-system level, CPython is the native process. At the Python-language level, `script.py` is the program being executed inside that process.

Why CPython Became the Reference Implementation

CPython is called the reference implementation because it is the implementation against which Python behavior is normally understood, tested, and developed. It is not merely one interpreter among many with equal cultural weight. It is the implementation maintained together with the main evolution of the language. New language features are normally developed, tested, and released first in CPython.

This historical dominance has practical consequences. When documentation, tutorials, packages, debugging tools, and extension modules describe “Python behavior,” they often silently mean CPython behavior unless they say otherwise. This is convenient, but it can also mislead students.

For example, in CPython:

- most objects are destroyed immediately when their reference count reaches zero;
- `id(obj)` often corresponds to the object’s memory address;
- the Global Interpreter Lock affects native threading behavior;
- Python bytecode is stored in `.pyc` files;
- many packages rely on C extension modules built for CPython’s C API.

These facts are extremely important in real Python programming, but they are not all pure language rules. Some are CPython implementation facts. Other Python implementations may behave differently while still preserving the intended high-level semantics of Python code.

The Boundary Between Guaranteed Behavior and CPython Behavior

A serious Python programmer must learn to separate two layers.

The first layer is guaranteed or intended Python behavior. For example, integers behave as arbitrary-precision integer objects. Lists are mutable sequences. Dictionaries map keys to values. Function calls create local scope. Exceptions propagate until caught. These are language-level facts.

The second layer is CPython-specific realization. For example, a Python object in CPython begins with a C-level object header containing a reference count and a pointer to its type object. A list is implemented internally as a dynamic array of pointers to Python objects. A function call creates a frame object or frame-like execution structure. The interpreter loop dispatches bytecode instructions. Memory is managed through CPython’s private allocator and reference-counting machinery.

The first layer tells us what Python code means. The second layer tells us how CPython makes that meaning happen.

This distinction is especially important for students coming from C. A C programmer naturally asks where things are stored, what the pointer is, who owns the memory, when allocation happens, and when the object is released. Python does not remove those questions. It moves them into the implementation. CPython still has memory layouts, pointers, allocators, stacks, heaps, and C structures. The difference is that ordinary Python code does not manipulate most of them directly.

Why This Course Focuses on CPython

This chapter focuses mainly on CPython because CPython is the concrete runtime most Python programmers actually use. It is also the implementation that best explains the everyday behavior encountered in normal Python work: reference counting, object identity, bytecode, `.pyc` files, stack frames, namespace dictionaries, the import system, and the Python/C extension ecosystem.

Studying only the abstract language would leave the same problem that a syntax-first course creates. The student would know what Python code looks like, but not what kind of machine executes it. CPython gives us that machine. It turns Python from an abstract language into a real process with memory, objects, frames, bytecode, native calls, and runtime-managed execution.

However, the focus on CPython must not become a conceptual prison. Python is broader than CPython. Other implementations exist because the language can be realized through different runtime strategies. PyPy uses a different implementation strategy with just-in-time compilation. Jython

connects Python semantics to the Java Virtual Machine. IronPython connects Python to the .NET runtime. These implementations show that Python is a language specification and culture, not only one C program.

Still, CPython remains the practical center. It is the implementation used by most packages, the implementation targeted by the most native extensions, and the implementation whose behavior most programmers implicitly learn first.

The Practical Rule

The practical rule is this:

Python is the language contract; CPython is the dominant machine that realizes that contract.

When explaining syntax, name binding, mutability, exceptions, or object behavior, we are usually talking about Python. When explaining `PyObject`, reference counts, bytecode dispatch, `.pyc` files, the Global Interpreter Lock, memory arenas, or C extension modules, we are talking about CPython.

This distinction prepares the rest of the chapter. We first separate the abstract language from its implementation, then we study CPython as the concrete runtime that makes the language executable. Only after that can Python's object model, memory behavior, stack frames, bytecode execution, and namespace machinery be explained without myth.

3.2.2 CPython, PyPy, Jython, IronPython, and Alternative Runtime Strategies

Once Python is separated from CPython, another fact becomes visible: the same language can be realized through different runtime strategies. Python source code is not tied by nature to one exact execution engine. It can be interpreted by CPython, optimized by PyPy, hosted on the Java Virtual Machine through Jython, hosted on the .NET runtime through IronPython, or adapted to smaller and more specialized environments through other implementations.

This does not mean that all implementations are interchangeable in practice. They are not. The language may be shared, but the execution machinery, package compatibility, memory behavior, native-extension support, performance profile, and host-platform integration can be very different. The phrase “Python implementation” therefore means more than “another program that runs Python files.” It means a complete answer to the question: how is Python source code made executable on a real machine or host runtime?

CPython: The Dominant Reference Runtime

CPython is the standard practical baseline. It is written mostly in C, compiled as a native executable, and normally executes Python code by compiling source files into bytecode and running that bytecode through an interpreter loop. It provides the object model, reference-counting machinery, memory allocators, import system, bytecode format, frame mechanics, and C extension interface most Python programmers implicitly depend on.

Its great advantage is compatibility. Most packages are developed first for CPython. Most tutorials assume CPython. Most debugging tools, profilers, native extensions, and packaging workflows are designed around CPython. When a programmer installs Python from the main Python distribution, they are almost always installing CPython.

Its disadvantage is also architectural. CPython preserves a very dynamic object model and executes most ordinary Python operations through runtime dispatch. This makes the language flexible and inspectable, but it also creates overhead. A Python addition is not simply one CPU addition instruction. CPython must load Python objects, inspect types, dispatch the operation, manage references, and handle possible exceptions. The runtime performs a large amount of work for operations that look small in source code.

CPython is therefore not “the fastest possible Python.” It is the most compatible and culturally central Python. It is the machine that defines the ordinary Python experience.

PyPy: Python with a JIT-Oriented Runtime Strategy

PyPy takes a different route. Its main idea is not to preserve CPython's internal machinery exactly, but to execute Python through a runtime designed around just-in-time compilation. Instead of only interpreting bytecode instruction by instruction, PyPy observes running code and can compile frequently executed paths into faster machine-level code.

This matters because many Python programs spend time repeatedly executing the same loops, function calls, and object operations. A JIT compiler can notice stable patterns during execution. If a variable repeatedly refers to integers, or a loop repeatedly performs the same operations over similar objects, the runtime may generate optimized code for that observed behavior.

The important point is that PyPy's optimization is runtime-based. It does not require the programmer to write C or to manually compile the Python file ahead of time. The program begins as Python, but the runtime may optimize parts of it while it runs.

This gives PyPy a different performance profile from CPython. Pure Python code, especially long-running code with stable hot paths, may run significantly faster. But programs that depend heavily on CPython-specific C extension modules may not benefit in the same way, because PyPy must preserve compatibility through additional layers. In such cases, the boundary between Python and native CPython extensions can become a problem.

So PyPy is not simply "CPython but faster." It is a different implementation strategy. It trades some CPython-internal compatibility for a runtime that can optimize Python execution more aggressively.

Jython: Python on the Java Virtual Machine

Jython implements Python on top of the Java Virtual Machine. Instead of running inside CPython's C-based runtime, Python code runs in a JVM-based environment and can interact with Java classes and libraries.

This changes the meaning of Python's host environment. With CPython, the natural native boundary is the C ABI and the Python/C API. With Jython, the natural boundary is the Java ecosystem. Python code can use Java libraries, Java objects, Java threading models, Java deployment systems, and JVM infrastructure.

The attraction is integration. If an organization already has a large Java system, Jython can make Python behave as a scripting or extension language inside that world. Python becomes a more flexible language surface over JVM-hosted components.

The cost is version and ecosystem distance. Jython historically lagged behind modern Python 3. Its current stable line has remained tied to Python 2.7 semantics. That makes it unsuitable as a normal modern replacement for CPython in most Python 3 projects. Its main interest is architectural: it proves that Python can be implemented over a host virtual machine that is not CPython.

IronPython: Python on the .NET Runtime

IronPython plays a similar role in the .NET world. It implements Python for the Common Language Runtime, allowing Python code to interact with .NET libraries and languages such as C#.

Again, the main advantage is not that IronPython is the universal best way to run Python. Its advantage is host integration. Python code can use .NET objects, call .NET libraries, and participate in .NET application environments. This can be valuable when the surrounding system is already built on that runtime.

The tradeoff is compatibility with the wider CPython-centered ecosystem. Many Python packages assume CPython internals, CPython bytecode behavior, or CPython C extensions. Those assumptions do not automatically transfer to IronPython. Pure Python packages are more portable; packages depending on native CPython extensions are harder.

IronPython therefore shows another implementation strategy: keep Python's language surface, but attach it to a different managed runtime and library universe.

Alternative Strategies and Specialized Pythons

Other Python implementations exist because not every environment wants the same kind of runtime. MicroPython, for example, targets microcontrollers and constrained devices. It cannot simply carry the full CPython runtime onto small embedded systems. It must reduce, adapt, and specialize the language environment for limited memory and hardware.

GraalPy takes another direction by placing Python in the GraalVM ecosystem. Some tools compile restricted Python-like code ahead of time. Other systems translate Python-adjacent code into C, C++, or machine code for specific performance or deployment goals.

These systems should not all be placed in one simple category. Some are general Python implementations. Some are restricted subsets. Some are acceleration tools. Some are embedding strategies. Some are compatibility layers. The common idea is that Python source or Python-like semantics can be connected to different execution backends.

Compatibility Is Not Only Syntax

A program may be syntactically valid Python and still fail on a different implementation. This happens because real compatibility is larger than grammar.

A Python implementation must deal with many layers:

- language syntax;
- built-in object behavior;
- standard-library behavior;
- exception semantics;
- import behavior;
- memory-management expectations;
- bytecode and frame inspection behavior;
- C extension compatibility;
- package ecosystem assumptions;
- host-platform integration.

For simple pure Python code, many implementations may behave similarly. For programs that inspect frames, depend on object finalization timing, use CPython-specific bytecode, rely on native extensions, or assume CPython memory behavior, portability becomes much weaker.

For example, in CPython an object may be destroyed immediately when its reference count reaches zero. A program should not depend on this as a general Python-language guarantee, because another implementation may use a different garbage-collection strategy. If the program depends on immediate destruction to close a file, that program is really depending on CPython behavior. The portable Python solution is to use explicit resource management, such as a context manager.

Different Runtime Strategies, Different Tradeoffs

The important comparison is not “which implementation is best.” The important comparison is what each implementation optimizes for.

CPython optimizes for compatibility, simplicity of implementation model, reference behavior, and the central Python ecosystem. PyPy optimizes for faster execution of many pure Python workloads through JIT compilation. Jython optimizes for Java integration. IronPython optimizes for .NET integration. MicroPython optimizes for constrained devices. Other systems optimize for deployment, ahead-of-time compilation, static analysis, or specialized acceleration.

Each strategy pays a price. A JIT runtime may be faster after warm-up but more complex. A JVM-hosted implementation gains Java access but loses CPython extension compatibility. A .NET-hosted implementation gains .NET access but moves away from the CPython package universe. An embedded implementation gains small-device suitability but cannot carry all ordinary Python behavior unchanged.

Thus, alternative implementations are not curiosities. They reveal what Python is made of. They show that Python has a language layer, a runtime layer, a host-platform layer, and an ecosystem layer. CPython combines these in one dominant form, but the combination is not logically forced.

The Practical Consequence

For ordinary programming, CPython should be treated as the default. It is what most students, tools, packages, and documentation assume. For understanding architecture, however, alternative implementations are extremely useful because they separate Python-the-language from Python-the-runtime.

The practical rule is:

Python code describes language-level behavior; the implementation decides how that behavior is executed.

In CPython, execution means C structures, reference counts, bytecode frames, interpreter dispatch, and native extension modules. In PyPy, it means a JIT-oriented runtime with different internal assumptions. In Jython, it means Python semantics hosted by the JVM. In IronPython, it means Python semantics hosted by .NET.

This prepares the next subsection. Once Python can have different execution backends, we must also distinguish full implementations from compilers, accelerators, translators, and hybrid tools. Not every tool that makes Python faster is a Python implementation. Some tools compile selected code. Some generate C extensions. Some accelerate numerical kernels. Some type-check without executing. The implementation question and the acceleration question are related, but they are not the same.

3.2.3 Python Compilers, Accelerators, and Translators: Cython, Nuitka, Numba, MyPyC, and Related Tools

After separating Python from CPython and after comparing several Python implementations, another category must be separated: compilers, accelerators, translators, and packaging compilers. These tools are often discussed as if they all answer the same question: “How do I make Python faster?” In reality, they answer different questions.

Some tools translate Python-like code into C extension modules. Some compile selected functions at runtime. Some turn Python programs into native executables that still carry a Python runtime. Some use type annotations to generate more efficient C-level code. Some are mainly distribution tools, not performance tools. If these categories are mixed together, the student receives a false picture of Python compilation.

The central fact is this: ordinary Python is hard to compile like C because ordinary Python deliberately leaves many decisions until runtime. A name can be rebound to a different kind of object. An operator can call user-defined methods. Attribute access can trigger descriptors. Imports can execute code. Classes can be modified. Functions are objects. The program can inspect and sometimes modify its own execution environment. This flexibility is useful, but it prevents a simple ahead-of-time compiler from treating Python source as if it were statically typed C source.

Therefore, most successful Python acceleration tools do not compile all possible Python behavior equally. They restrict part of the language, specialize part of the program, use type information, delegate work to native libraries, or keep CPython nearby.

Why Python Compilation Is Not C Compilation

In C, many facts are known before the program runs. The compiler knows that an `int` variable stores an integer of a certain representation. It knows the layout of structures. It knows function signatures.

It can generate direct machine instructions for arithmetic, memory access, and function calls.

In ordinary Python, this is not generally true. Consider:

```
x = a + b
```

At compile time, CPython may know that the source contains a name load, another name load, an addition operation, and a store. But it does not necessarily know what objects `a` and `b` will refer to when the line executes. They may be integers, floats, strings, lists, NumPy arrays, user-defined objects, or objects whose `__add__()` method has unusual behavior.

So the compiler cannot simply emit one CPU addition instruction in the general case. It must preserve Python's runtime semantics. This means object lookup, type dispatch, method resolution, exception handling, reference management, and fallback behavior.

This is why Python acceleration usually depends on one of four strategies:

- add type information;
- restrict the accepted Python subset;
- compile only selected hot functions;
- keep the Python runtime but reduce overhead around it.

Cython: Python Near the C Boundary

Cython is best understood as a bridge between Python and C. It can compile Python-like code into C or C++ code that becomes a CPython extension module. It also allows the programmer to add C-level type declarations, call C functions directly, and define data structures closer to native representation.

A simple Cython file may look almost like Python. But the performance gain becomes stronger when the programmer gives Cython more information:

```
def add_python(a, b):  
    return a + b  
  
cdef int add_c(int a, int b):  
    return a + b
```

The first function still behaves like ordinary Python. The second function gives Cython C-level information. Now the operation can be compiled much closer to ordinary C integer addition.

Cython is therefore not magic. It becomes powerful when the programmer deliberately moves selected parts of the program toward C. This is especially useful for tight loops, numeric kernels, wrappers around existing C libraries, and extension modules that must expose a Python interface while doing heavy work in native code.

Its cost is that the code is no longer purely ordinary Python in spirit. The programmer begins to think about C types, C arrays, memoryviews, extension-module building, compilation, and native binary compatibility. Cython gives speed by partially abandoning the fully dynamic surface of Python where necessary.

Nuitka: Whole-Program Compilation with Runtime Preservation

Nuitka has a different goal. It compiles Python programs into C/C++ code and then into executables or extension modules. But this should not be misunderstood as “Python becomes C in the same sense that C source becomes C machine code.” Nuitka must still preserve Python semantics.

This means that a compiled Nuitka program may still carry or depend on the Python runtime machinery needed to execute Python behavior correctly. The output can be useful for distribution, startup control, packaging, and sometimes performance, but the dynamic nature of Python does not disappear simply because the program was compiled.

Nuitka is useful when one wants to ship a Python application as a compiled artifact, reduce some interpreter overhead, or produce a more self-contained program. It is not the same kind of tool as Cython. Cython asks the programmer to expose C-level facts in selected code. Nuitka tries to accept normal Python programs and compile them while preserving Python behavior.

The distinction is important:

- Cython is often used for extension modules and native-speed inner loops;
- Nuitka is often used for compiling larger Python programs or packaging them as executables.

Both involve compilation, but they sit at different architectural points.

Numba: Runtime Compilation of Numerical Kernels

Numba is mostly useful for numerical functions. Its common pattern is simple: the programmer writes a Python function, decorates it, and Numba compiles suitable versions of that function at runtime.

A typical example is:

```
from numba import njit

@njit
def sum_values(values):
    total = 0.0
    for x in values:
        total += x
    return total
```

The important detail is that Numba does not try to compile every possible Python program equally. It works best when the function uses numeric types, arrays, loops, and operations that can be lowered into efficient machine code. In such cases, Numba can avoid much of the ordinary CPython bytecode dispatch overhead.

Numba's strength is that it allows a programmer to keep a Python-like development style while accelerating selected hot functions. Its weakness is that it is not a universal Python compiler. Code that depends heavily on arbitrary Python objects, dynamic attribute tricks, complex class behavior, or unsupported library calls may fall outside the efficient compilation path.

Architecturally, Numba is very different from Cython. Cython is normally an ahead-of-time translation path through C/C++ extension modules. Numba is normally a just-in-time specialization path for selected functions, especially numerical functions.

MyPyC: Type-Annotated Python Compiled to C Extensions

MyPyC uses Python type annotations to compile modules into CPython C extension modules. Its central idea is that type hints can be used not only for checking, but also for producing faster code.

For example, ordinary Python type hints such as:

```
def add(a: int, b: int) -> int:
    return a + b
```

give the compiler more information than an unannotated function. If the tool can rely on those types under its rules, it can generate code with less dynamic overhead.

MyPyC is especially interesting because it grows from modern Python's gradual typing culture. Type hints were not originally added as a C-like compilation system. They were mainly introduced for readability, tooling, checking, and large-codebase maintainability. But once type information exists, compilation becomes more plausible for selected code.

The tradeoff is that MyPyC does not preserve the full freedom of arbitrary Python in every corner. It works best with code that is already disciplined, typed, and written in a style compatible with its restrictions. This makes it attractive for accelerating large typed Python codebases, but not for every dynamic script.

Packaging Tools Are Not Necessarily Compilers

Some tools are often confused with compilers because they produce executable-looking artifacts. Examples include tools that bundle a Python program together with an interpreter, libraries, and resources. Such tools may create an `.exe` or an application bundle, but that does not necessarily mean the Python code has been translated into optimized native machine code.

This distinction matters. A packaged Python program may still execute Python bytecode through an embedded interpreter. The user sees a native-looking file, but internally the architecture may still be:

```
application executable
  contains or launches Python runtime
    loads Python bytecode or source
      executes through interpreter
```

This is useful for deployment. It is not automatically a performance transformation.

The Common Pattern: Move the Hot Path Downward

The practical reason these tools exist is that most programs are not uniformly expensive. A large application may spend most of its time in a few small regions: a numerical loop, a parser, an image operation, a geometry transformation, a serialization step, or a repeated calculation.

Python acceleration often means identifying those hot paths and moving them downward:

- from Python objects to typed native values;
- from interpreter loops to compiled loops;
- from Python lists to contiguous arrays;
- from dynamic dispatch to known operations;
- from pure Python functions to native extension modules.

This preserves Python's role as the orchestration layer while moving the expensive inner work closer to the machine.

This is exactly the same architectural reason Python became dominant in scientific computing. Python remains the outer language of expression, experiment, and coordination. The inner loops move to C, C++, Fortran, LLVM-generated code, GPU kernels, or specialized native libraries.

A Practical Comparison

The following simplified comparison is useful:

- **Cython** is best when the programmer is willing to add C-level information and build extension modules.
- **Nuitka** is best when the goal is compiling or packaging larger Python programs while preserving Python semantics.
- **Numba** is best for accelerating selected numerical functions at runtime.
- **MyPyC** is best for compiling disciplined, type-annotated Python modules into CPython extensions.
- **Bundling tools** are best for distribution, not necessarily for speed.

These tools do not replace one another. They solve different problems.

The Conceptual Lesson

The existence of these tools proves that Python is not a single execution story. Python code can be interpreted, JIT-compiled, translated to C, compiled into extension modules, bundled with an interpreter, or used as a control layer above native libraries.

But the same warning must be repeated: a tool that accelerates Python does not necessarily turn Python into C. Python’s full language semantics are too dynamic for that simple statement. Speed usually appears when the tool can reduce uncertainty. Type declarations, type hints, numerical arrays, restricted subsets, and stable hot loops all give the compiler more facts. More facts allow less runtime dispatch. Less runtime dispatch allows faster execution.

For this course, the architectural conclusion is direct. Python’s surface simplicity is supported by heavy runtime machinery. Acceleration tools work by bypassing, specializing, or reducing parts of that machinery in selected regions. They do not abolish the CPython model; they exploit boundaries inside it.

Thus, Python performance should not be understood as one number. There is pure Python interpreter speed. There is CPython extension speed. There is vectorized native-library speed. There is JIT-specialized speed. There is packaged-application behavior. Serious Python programming requires knowing which layer is currently doing the work.

3.2.4 The Practical Consequence: Python Code Can Have Different Execution Backends but One Dominant Reference Runtime

The practical consequence of the previous sections is simple: Python code is not naturally tied to one single execution backend, but real-world Python programming is still strongly centered on CPython. This creates a useful but slightly confusing situation. At the language level, Python is broader than CPython. At the practical level, most Python code, tools, packages, documentation, and programmer expectations are shaped by CPython.

A Python file is therefore not like a C executable. It is source code written in a language that can, in principle, be consumed by different execution systems. CPython may compile it to CPython bytecode and execute it through an interpreter loop. PyPy may execute it through a JIT-oriented runtime. Jython may host Python semantics on the Java Virtual Machine. IronPython may host them on .NET. Cython, Numba, Nuitka, and MyPyC may compile or specialize selected parts of the program under particular assumptions.

This means that Python has several possible execution paths, but one overwhelmingly important reference path.

The Same Source Surface Can Hide Different Machines

Consider a simple Python statement:

```
x = a + b
```

At the language level, this means that the names `a` and `b` are resolved, the addition operation is performed according to the objects involved, and the name `x` is bound to the result.

But different runtimes may realize this behavior differently. CPython may dispatch a bytecode instruction through its interpreter loop. PyPy may eventually compile a hot path after observing repeated behavior. Cython may generate C-level addition if the types are declared. Numba may compile a numerical version of the function if the operation fits its supported subset. A packaging tool may simply bundle the same CPython execution model inside an executable-looking file.

The visible Python line is the same. The machinery underneath is not.

This is why one must not treat “Python execution” as one mechanism. Python source code describes behavior. The backend decides how that behavior is produced.

Why CPython Still Dominates Practice

Despite this plurality, CPython remains the practical center. There are several reasons.

First, CPython is the reference implementation. New language features, documentation, tests, and ordinary programmer expectations are usually developed around it first.

Second, CPython is the default installation for most users. When someone downloads Python from the main Python website, installs it through many operating-system package managers, or follows most introductory tutorials, they are usually using CPython.

Third, the package ecosystem is deeply CPython-shaped. Many packages are pure Python and may run elsewhere, but many important packages depend on native extension modules compiled for CPython's C API. This matters especially in scientific computing, machine learning, image processing, cryptography, databases, and systems programming.

Fourth, CPython's behavior has become part of Python culture. Programmers often expect immediate object destruction through reference counting, `id()` values that resemble memory addresses, `.pyc` bytecode caches, and CPython-style stack traces and frames. Some of these are language-level behaviors; others are CPython implementation habits that programmers have learned as if they were universal Python facts.

Portability Is Real, but Not Automatic

A simple pure Python program may run without modification on several Python implementations. For example, code that uses ordinary functions, lists, dictionaries, loops, exceptions, and pure standard-library modules is often portable.

But portability becomes weaker when the program depends on implementation details. A program becomes less portable when it relies on:

- CPython C extension modules;
- exact object destruction timing;
- CPython bytecode details;
- frame inspection behavior;
- memory-address-like interpretations of `id()`;
- CPython-specific performance assumptions;
- packages distributed only as CPython binary wheels.

This distinction is important. The Python language may allow different backends, but the ecosystem may quietly pull the program back toward CPython. In practice, many Python projects are not only Python projects. They are CPython projects.

The False Simplicity of “Compiled vs. Interpreted”

This backend diversity also shows why the old question “Is Python compiled or interpreted?” is too crude.

In CPython, Python source is compiled to bytecode, and that bytecode is interpreted by the CPython virtual machine. In PyPy, execution may involve interpretation plus JIT compilation. In Cython, Python-like code may be translated into C and compiled into a native extension. In Numba, selected functions may be compiled at runtime. In Nuitka, a larger program may be compiled while still preserving Python runtime behavior.

So Python is not defined by one compilation story. The correct question is:

Which implementation or toolchain is executing this Python code, and what runtime machinery does it use?

That question is much more useful than asking whether Python is simply compiled or interpreted.

The Teaching Consequence

For this course, the consequence is methodological. We will not pretend that CPython is the whole Python language. But we will study CPython because it is the concrete runtime through which most Python programmers encounter the language.

This gives us two layers of explanation.

At the first layer, we explain Python as a language: names, objects, assignment, functions, classes, exceptions, modules, mutability, iteration, and type behavior.

At the second layer, we explain CPython as the dominant implementation: `PyObject`, reference counts, type pointers, heap allocation, frames, bytecode, namespace dictionaries, the interpreter loop, `.pyc` files, and the Python/C extension boundary.

The first layer explains what Python code means. The second layer explains how the dominant runtime makes that meaning happen.

The Practical Rule for Programmers

The practical rule is:

Write Python according to the language model, but understand performance, memory, packaging, and debugging through the implementation model.

If the question is whether assigning `b = a` copies a list, the answer belongs mostly to the Python language model: it binds another name to the same object. If the question is when that object is released from memory, the answer depends strongly on CPython's reference counting and garbage-collection behavior. If the question is why a numerical loop is slow, the answer involves CPython bytecode dispatch and dynamic object operations. If the question is why NumPy is fast, the answer involves native code underneath Python objects.

3.3 The CPython Reference Implementation as Python's Concrete Runtime

After separating Python as a language from CPython as its dominant implementation, we can now examine the runtime machinery that makes ordinary Python execution concrete. The goal is not to study CPython as an implementation project in its own right. The goal is to expose the internal structures that a serious Python programmer must understand in order to reason correctly about names, objects, functions, frames, bytecode, memory behavior, and execution cost.

CPython is the bridge between the high-level language and the native machine. The operating system runs a compiled C program; that C program reads Python source files, parses them, compiles them into code objects and bytecode, allocates Python objects, creates execution frames, resolves names through namespaces, and dispatches bytecode instructions through an interpreter loop. Python code therefore runs inside a managed runtime world built inside a native process.

This section opens that world only as far as needed. We will not catalogue CPython source files or C API functions. Instead, we will follow the execution path that matters: from the general runtime model established earlier, to CPython's internal C object structures, to source compilation, to code objects and frames, and finally to the interpreter loop where Python bytecode is actually executed.

3.3.1 From Chapter 1's Runtime Model to CPython's Concrete Implementation

3.3.2 From Chapter 1's Runtime Model to CPython's Concrete Implementation

The previous chapter established the general execution model needed for understanding Python seriously. A program does not run as an abstract text. Something concrete must be loaded, memory must be allocated, instructions must be executed, function calls must create execution state, and values must live somewhere. In C, these facts are visible through native compilation, process memory layout, stack

frames, heap allocation, pointers, and machine instructions. In Python, the same general questions remain, but the answers move one level upward into the CPython runtime.

When a command such as

```
python script.py
```

is executed, the operating system does not load `script.py` as a native executable. The operating system loads the native executable `python` or `python.exe`. That executable is CPython. It is a compiled C program with its own text segment, data segment, heap, stack, native libraries, startup code, and operating-system resources. The Python file is input to this process, not the process itself.

This is the central bridge from the previous chapter:

At the operating-system level, CPython is the running native program. At the language level, the Python source file is the program executed inside that native runtime.

This distinction prevents many false explanations. Python is not executed by the processor in the same direct way as compiled C code. The processor executes CPython's native machine instructions. CPython then reads Python source, compiles it into an internal executable representation, creates runtime objects and frames, and executes Python bytecode through its interpreter machinery.

The Native Layer: CPython as an Ordinary Process

CPython begins life like any other native executable. The operating system loader maps its executable regions into memory, prepares the process stack, loads required dynamic libraries, resolves startup requirements, and transfers control to CPython's native entry point. Before any user Python code runs, the interpreter process already exists.

This means that CPython is not outside the native execution model. It is built on top of it. It has C functions, C structures, native memory allocations, operating-system calls, file descriptors or handles, dynamic libraries, and platform-specific behavior. When Python code opens a file, creates a socket, starts a subprocess, or imports a native extension module, CPython eventually crosses into the operating system or native library world.

Therefore, Python does not abolish the earlier model. It hosts another execution model inside it.

The Runtime Layer: Python Code as Managed Execution

Once CPython is running, it treats the user's `.py` file as source input. The source text is not executed directly. It passes through a pipeline:

```
.py source text
-> tokens
-> parse tree / AST
-> code object
-> bytecode instructions
-> frame execution inside CPython
```

This pipeline is the Python equivalent of moving from source text toward executable behavior. But the destination is different from C. In C, the pipeline normally produces native machine code before the program runs. In CPython, the source is compiled into bytecode for the Python virtual machine. That bytecode is then executed by CPython's interpreter loop.

The result is that Python source code becomes executable only after CPython has transformed it into runtime objects. Code itself becomes an object. Functions are objects. Classes are objects. Modules are objects. Integers, strings, lists, dictionaries, exceptions, and types are objects. The runtime does not merely store raw values; it stores structured objects with identity, type, value, and implementation-level metadata.

Replacing C Variables with Python Bindings

This runtime model explains why Python variables are not C variables. In C, a local variable often corresponds to a typed storage slot in a stack frame. If the programmer writes

```
int x = 10;
```

then `x` names a storage location capable of holding an `int` value.

In Python, when the programmer writes

```
x = 10
```

the name `x` is bound to an integer object. The name is not a typed box containing the integer bytes directly. It is an entry in a namespace, pointing to a Python object managed by the runtime. If the programmer later writes

```
x = "hello"
```

the name `x` is rebound to a different object. The name changed its binding; the original name slot did not transform from an integer box into a string box.

This is one of the most important consequences of CPython's concrete implementation. Python names are references to objects. They are not C-style typed storage containers.

Replacing Native Stack Frames with Python Frame Execution

The previous chapter explained native function calls through stack frames. CPython also needs execution frames, but Python frames are not simply the same as native C stack frames.

A Python function call creates Python-level execution state: local names, references to global and built-in namespaces, a code object, an instruction position, and evaluation state needed to execute bytecode. CPython must represent this state inside its own runtime structures.

So there are several layers that must not be confused:

- the native C stack used by the CPython executable itself;
- CPython's internal machinery for evaluating Python calls;
- Python frame objects or frame-like execution records representing Python-level calls;
- the bytecode evaluation stack used while executing expressions.

When a Python function calls another Python function, the source-level idea resembles an ordinary function call, but the mechanism is runtime-managed. CPython must create or prepare a frame, bind arguments to local names, execute the callee's bytecode, handle returns or exceptions, and then resume the caller.

This is why Python can support introspection, tracebacks, debuggers, exceptions, generators, and coroutines in ways that differ from plain C function execution. The runtime keeps structured information about Python execution.

Replacing Raw Memory Ownership with Runtime Object Management

In C, heap allocation is explicit. The programmer may call `malloc()` and later `free()`. In Python, ordinary code does not manually free objects. CPython allocates Python objects on its managed heap and tracks their lifetimes mostly through reference counting, with additional garbage-collection support for cyclic object graphs.

This does not mean memory management disappears. It means that memory management becomes a runtime responsibility. Every object must be allocated, referenced, passed, stored, possibly shared, and eventually released. Assignment, function calls, container insertion, exception handling, and return values all affect object reachability and reference ownership internally.

For the Python programmer, the practical effect is simpler source code. For the implementation, the work is substantial. CPython must constantly maintain the object graph that makes Python's high-level behavior possible.

The Correct Mental Model

The correct mental model is therefore layered:

Operating system

runs a native CPython process

CPython runtime

parses, compiles, allocates, dispatches, and manages Python execution

Python program

exists as source, code objects, bytecode, frames, namespaces, and objects

This layered view explains why Python feels high-level but still has concrete execution costs. A simple line of Python may require name lookup, object access, type dispatch, method invocation, reference-count updates, exception checks, and bytecode interpretation. The syntax is small because CPython performs the machinery underneath.

The purpose of this section is not to turn the reader into a CPython core developer. We do not need every source file, every C API function, or every internal optimization. But we do need enough of CPython's structure to avoid childish explanations. Python assignment is not copying values into boxes. Python functions are not merely labeled text blocks. Python objects are not vague abstract values. Python execution is not source text being read line by line.

Python code runs because CPython, as a native process, builds a managed runtime world inside itself. The rest of this section opens that world just enough to explain how Python source becomes bytecode, how bytecode is attached to code objects, how function calls create frame state, how namespaces store name bindings, and how the interpreter loop becomes the concrete execution site of Python programs.

3.3.3 The Concrete Internal C Structure of the CPython Runtime

CPython should not be imagined as a mysterious interpreter floating above the machine. It is a large native C program. It contains C source files, C structures, C functions, native memory allocation, operating-system calls, and a large amount of bookkeeping code whose purpose is to make Python's high-level behavior possible.

For this course, we do not need to read CPython as a core developer would read it. We do not need every source file, every macro, every optimization, or every C API function. But we do need the basic structural picture, because many Python behaviors become clear only when we understand what CPython is storing and moving underneath the surface.

The most important idea is this:

In CPython, Python-level concepts are represented by C-level structures.

A Python object is represented by a C object structure. A Python type is represented by a C type object. A Python function has a runtime object that refers to executable code. A module has a namespace dictionary. A frame stores execution state for a running Python code block. Namespaces are dictionaries or dictionary-like mappings. The interpreter itself keeps runtime state, interpreter state, and thread state.

Thus, Python's apparent simplicity is built from a concrete internal object world.

CPython Is Not One Object but a Runtime System

There is no single C structure called "the Python program." CPython is a system of cooperating structures. At a simplified level, it contains:

- process-level runtime state;

- interpreter-level state;
- thread-level execution state;
- Python object structures;
- type objects;
- code objects;
- frame execution structures;
- namespace dictionaries;
- allocator and garbage-collection structures;
- import-system state;
- native extension interfaces.

These layers correspond to different questions. Runtime state answers: what does the CPython process know globally? Interpreter state answers: what belongs to this Python interpreter instance? Thread state answers: what is this executing thread currently doing? Object structures answer: what values exist? Frame structures answer: what code is currently executing? Namespace dictionaries answer: which names are bound to which objects?

This is why CPython cannot be reduced to “an interpreter loop.” The interpreter loop is central, but it is only the execution site. The loop needs objects to operate on, code objects to read from, frames to store execution state, dictionaries to resolve names, and type objects to decide how operations behave.

The Object Header: Why Every Object Can Be Treated as an Object

At the lowest practical level of Python values, CPython uses a common object header. Conceptually, an ordinary Python object begins with information like this:

```
PyObject
  reference count
  pointer to type object
```

This is not meant as exact source-code replacement for all builds and all versions. It is the conceptual skeleton. The reference count helps CPython track object lifetime. The type pointer tells CPython what kind of object this is and which operations the object supports.

This is the concrete meaning behind the phrase “everything is an object” in CPython. It does not mean that objects are magical. It means that runtime values are represented as structured objects that CPython can access through a common initial layout.

For example, when CPython has a pointer to some runtime value, it can ask two fundamental questions:

- How many strong references are currently keeping this object alive?
- Which type object describes this object's behavior?

The second question is especially important. CPython does not need the C compiler to know that a value is an integer, string, list, dictionary, function, or user-defined instance at every source location. The object itself carries a pointer to its runtime type.

Variable-Sized Objects: The Role of PyVarObject

Some Python objects have a natural size that varies from instance to instance. A tuple may have three elements or ten elements. A bytes object may contain five bytes or five thousand bytes. A string may contain different amounts of character data.

For such objects, CPython uses an extended base form commonly represented by `PyVarObject`. Conceptually:

```
PyVarObject
    PyObject base
    size field
```

The important idea is simple. A variable-sized object still begins like an object, so CPython can treat it as a Python object. But it also stores size-related information needed by that particular family of objects.

This explains why Python containers are not C arrays in the naive sense. A Python list, tuple, string, or bytes object is not just raw adjacent user values. It is an object with a header, type information, size information, and implementation-specific storage.

Type Objects: Behavior Stored as Runtime Metadata

The type pointer inside each object points to a type object. In CPython, a type object is itself a large C-level structure. It stores information about the type's name, size, construction behavior, destruction behavior, attribute access, numeric operations, sequence operations, mapping operations, method tables, inheritance relationships, and other behavior.

A simplified view is:

```
object
    ob_type ---> type object
                        name
                        size information
                        deallocation function
                        attribute access functions
                        call behavior
                        numeric operation slots
                        sequence operation slots
                        mapping operation slots
                        method definitions
```

This is the implementation-level reason operators and built-in functions work dynamically. When Python evaluates an operation such as

```
a + b
```

CPython does not treat `+` as one fixed machine instruction. It must look at the objects involved and use their type information to decide what addition means. Integer addition, string concatenation, list concatenation, NumPy array addition, and user-defined `__add__` behavior are not the same operation internally.

The type object is therefore the behavior descriptor of Python objects. It is what turns a memory object into a value with operations.

Reference Counts: Object Lifetime as Runtime Bookkeeping

CPython primarily tracks object lifetime through reference counting. When a strong reference to an object is created, the object's reference count is increased. When a strong reference is released, the count is decreased. When the last strong reference is released, CPython can call the object's

deallocation function. The official C API exposes this through operations such as `Py_INCREF()` and `Py_DECREF()`, with the latter invoking the type's deallocation function when the last strong reference is gone. :contentReference[oaicite:1]index=1

This gives CPython a very direct memory-management style compared with tracing-only garbage collectors. Many objects can be destroyed as soon as they become unreachable through ordinary reference paths.

However, the student should not oversimplify this into “Python immediately frees everything.” There are complications:

- some objects may be cached or reused;
- some objects may be immortal in modern CPython builds;
- reference cycles need garbage-collector support;
- finalizers can execute Python code during destruction;
- alternative Python implementations may not use the same lifetime strategy.

The useful mental model is not that memory disappears magically. The useful model is that CPython constantly maintains object-lifetime bookkeeping while Python code runs.

Namespaces as Dictionaries of Object References

Python names are not C variables holding raw values. Names live in namespaces. In CPython, the most important namespaces are represented by dictionaries or dictionary-like internal structures.

A module namespace is essentially a mapping from names to objects:

```
module namespace
    "x"          ---> integer object
    "message"    ---> string object
    "func"       ---> function object
    "MyClass"    ---> class object
```

This is why assignment in Python is binding, not typed storage mutation. When Python executes

```
x = 10
```

CPython creates or reuses an integer object and binds the name `x` to that object in the relevant namespace.

When Python later executes

```
x = "hello"
```

the namespace entry for `x` is changed to point to a different object. There was no fixed `int` box that became a `str` box. There was a name-to-object binding that changed.

This one idea removes many beginner misconceptions about Python variables.

Code Objects and Function Objects

Python code is also represented by objects. When CPython compiles source code, it produces code objects. A code object is not a running function call. It is an executable description: bytecode, constants, names, local-variable information, and other metadata needed to execute a block of Python code.

A function object wraps a code object together with additional runtime context, especially the global namespace and default arguments.

A simplified view is:

```

function object
  code object
    bytecode
    constants
    names
    local variable metadata
  globals dictionary
  defaults
  closure cells, if needed

```

This distinction matters. A function definition does not merely mark a region of source text. Executing a `def` statement creates a function object. That function object can be assigned, passed as an argument, returned from another function, stored in a list, or attached to a class.

This is the implementation basis for Python's first-class functions.

Frame State: Running Code Needs a Runtime Record

A code object is executable, but it is not by itself an active execution. To run code, CPython needs execution state. This state is represented by frame machinery.

A Python frame contains the information needed to execute a code object at a particular moment. Conceptually, it includes:

- the code object being executed;
- the current instruction position;
- local variables or local reference slots;
- access to global and built-in namespaces;
- an evaluation stack for intermediate expression values;
- exception-handling state and control-flow metadata.

This is why tracebacks can show a chain of Python calls. CPython has structured runtime records for Python-level execution. When an exception propagates, CPython can report which frames were active and which lines or instructions were involved.

Again, this should not be confused with the native C stack alone. CPython itself uses the native C stack because it is a C program. But Python-level execution also has its own frame model. That frame model is what makes Python-level introspection, debugging, tracebacks, generators, and coroutines possible.

Interpreter State, Thread State, and the GIL

Above individual objects and frames, CPython also maintains larger execution state. The official C API documentation separates topics such as interpreter initialization, thread states, the global interpreter lock, and multiple interpreters in one process. [:contentReference\[oaicite:2\]index=2](#)

For our purposes, the simplified model is enough:

```

CPython process
  runtime state
    interpreter state
      modules
      builtins
      import machinery
      memory-management state
      thread states

```

```

current frame / execution state
exception state
thread-specific runtime information

```

The thread state connects a native thread to the Python interpreter machinery. CPython needs to know which thread is currently executing Python code, what frame it is executing, and what exception state belongs to it.

The Global Interpreter Lock, usually called the GIL, is part of this execution-control story in standard CPython builds. Its practical consequence is that CPython historically allows only one thread at a time to execute Python bytecode in one interpreter, even though threads may still be useful for I/O-bound work or for native code that releases the lock. The detailed history of the GIL belongs elsewhere, but its existence is important because it shows that Python execution is controlled by runtime state, not only by operating-system threads.

Native Extensions Use the Same Object World

One reason CPython became so important is that native extension modules can participate in this object system. A C extension can create Python objects, receive Python objects, inspect types, call functions, raise exceptions, define new types, and expose C or C++ functionality through Python interfaces.

This is how much of the high-performance Python ecosystem works. NumPy, parts of pandas, image-processing libraries, cryptographic libraries, database drivers, and many other packages cross the boundary between Python and native code.

The bridge works because CPython has a concrete C-level object protocol. A native extension does not receive vague Python values. It receives pointers to CPython object structures and obeys CPython's reference-counting and type rules.

This is powerful, but it also explains why many packages are CPython-specific. If a package depends on the CPython C API, then it is not merely depending on Python syntax. It is depending on CPython's concrete runtime representation.

The Practical Mental Model

The practical mental model is this:

```

Python source code
    becomes code objects

code objects
    execute inside frames

frames
    use namespaces to resolve names

namespaces
    map names to objects

objects
    have headers, reference counts, and type pointers

type objects
    define behavior through runtime metadata and C function slots

CPython
    coordinates all of this inside a native process

```

This is enough internal structure for an effective Python programmer. The goal is not to memorize CPython's source tree. The goal is to understand why Python behaves differently from C.

Python assignment is name binding because namespaces map names to object references. Python operations are dynamic because objects carry type pointers. Python memory management is automatic at the source level because CPython maintains reference counts and garbage-collection support. Python function calls are introspectable because execution uses frame state. Python extension libraries can be fast because they cross into native code while still speaking CPython's object protocol.

The phrase "Python is high-level" therefore has a concrete meaning. It does not mean there is no machinery. It means the machinery has been moved into CPython.

3.3.4 Parsing and Compiling Python Source: From .py Files to AST, Code Objects, Bytecode, and .pyc Marshaled Disk Artifacts

A Python source file is not executed as raw text. When CPython runs a file, it first transforms that file into internal structures that the runtime can execute. This is one of the most important corrections to the beginner's mental model. CPython does not simply read a line, understand it as English-like text, and perform it immediately. It performs a compilation pipeline.

The pipeline can be simplified as:

```
.py source file
-> token stream
-> parser output
-> abstract syntax tree
-> code object
-> bytecode instructions
-> frame execution by CPython
```

This pipeline resembles the compilation ideas discussed in the previous chapter, but the destination is different. A C compiler normally moves toward native machine code. CPython moves toward bytecode for the CPython virtual machine. The physical CPU still executes CPython's native machine instructions; the Python program is represented as bytecode and runtime objects inside that process.

The Source File Is Input, Not the Native Program

When the programmer writes

```
python example.py
```

the operating system starts the CPython executable. The file `example.py` is opened and processed by CPython. It is not loaded by the operating system as an ELF or PE native executable.

This distinction matters because it explains why Python source can be portable while CPython remains platform-specific. The same `.py` file may run on Linux, Windows, or macOS, but the `python` executable that reads it is different on each platform. The source file is portable input. CPython is the native program doing the work.

Tokenization: Characters Become Language Units

The first stage is lexical. CPython reads characters and groups them into tokens. A token is a classified piece of source code: a name, keyword, operator, literal, newline marker, indentation marker, punctuation symbol, or similar unit.

For example:

```
x = 10 + y
```

is not handled as isolated characters. It becomes a sequence of meaningful units:


```
NAME(x)
OPERATOR(=)
NUMBER(10)
OPERATOR(+)
NAME(y)
```

Python's tokenization differs from C in one crucial way: indentation is syntactic. Whitespace at the beginning of lines can produce `INDENT` and `DEDENT` structure. This is how Python turns visual block layout into grammatical structure.

So this source:

```
if x > 0:
    print(x)
print("done")
```

contains an indented block after the condition. CPython's tokenizer and parser must preserve that block structure. In C, braces provide this information. In Python, indentation does.

Parsing: Tokens Become Program Structure

After tokenization, CPython parses the token stream according to Python's grammar. Parsing answers the question:

Do these tokens form valid Python structures?

The parser recognizes constructs such as assignments, expressions, function definitions, class definitions, loops, conditionals, imports, and exception handlers. If the tokens do not fit Python's grammar, CPython raises a syntax error before the program executes.

For example:

```
if x > 0
    print(x)
```

is not valid Python because the colon is missing after the condition. The error is not a runtime error in the ordinary sense. The code cannot be compiled into executable Python bytecode because it is not syntactically complete.

The Abstract Syntax Tree: Source Shape Becomes Meaning Shape

After parsing, CPython builds an abstract syntax tree, usually called an AST. The AST represents the meaningful structure of the program without preserving every surface detail of the original text.

For example:

```
x = a + b * c
```

is structurally not a flat sequence. The multiplication belongs inside the addition because multiplication has higher precedence. A simplified AST-like view is:

```
Assignment
  target: Name(x)
  value:
    Add
      left: Name(a)
      right:
        Multiply
          left: Name(b)
          right: Name(c)
```

The exact internal AST is more detailed, but this simplified shape is enough. The AST records what the program means structurally. It is no longer plain text and no longer just tokens.

This stage is important because some errors and transformations can happen before bytecode exists. CPython can analyze scopes, decide which names are local or free, record constants, and prepare the program for bytecode generation.

Symbol Analysis: Names Are Classified Before Execution

Python is dynamic, but not everything is postponed until runtime. CPython performs compile-time analysis of name roles.

Consider:

```
x = 10
```

```
def f():  
    print(x)
```

Inside `f`, `x` is read but not assigned. CPython can classify it as a global name lookup.

Now compare:

```
x = 10
```

```
def f():  
    print(x)  
    x = 20
```

Because `x` is assigned inside `f`, CPython classifies `x` as a local name in that function. The earlier `print(x)` then tries to read the local `x` before it has received a value, producing an error when executed.

This surprises beginners because they think Python decides everything line by line while running. It does not. CPython has already classified name roles during compilation.

So Python is dynamic, but not structureless. The compiler still analyzes scopes before execution.

Code Objects: Executable Descriptions

After AST and symbol analysis, CPython creates code objects. A code object is a low-level executable description of a block of Python code. It is not the same thing as a function object, and it is not an active call. It is a stored representation of executable Python instructions and metadata.

A code object contains things such as:

- bytecode instructions;
- constants used by the code;
- names referenced by the code;
- local-variable information;
- free-variable and cell-variable information for closures;
- line-number and debugging metadata;
- flags describing execution properties.

For example, the module body has a code object. Each function body has a code object. A nested function has its own code object stored among the constants of the enclosing code object until a function object is created at runtime.

This is the key point:

A code object is executable material, but it is not yet a running execution.

To run it, CPython must create a frame and execute the code object's bytecode inside that frame.

Bytecode: Instructions for the CPython Virtual Machine

The bytecode inside a code object is an instruction stream for CPython's virtual machine. It is not native CPU machine code. It is not x86-64, ARM64, or RISC-V. It is a CPython-specific instruction format consumed by CPython's interpreter.

For example, Python source like

```
x = a + b
```

may conceptually compile into operations like:

```
load name a
load name b
perform binary addition
store result in name x
```

The exact bytecode instructions differ across Python versions, so the important lesson is not the exact opcode list. The important lesson is that Python source becomes a lower-level instruction stream before execution.

This is why the question “is Python interpreted or compiled?” is badly formed. In CPython, Python source is compiled to bytecode, and then that bytecode is interpreted by CPython.

.pyc Files: Cached Bytecode Artifacts

CPython may store compiled bytecode artifacts on disk as .pyc files. These files usually live inside a `__pycache__` directory. The filename encodes information such as the Python implementation and version tag, because bytecode is not a stable cross-version format.

A typical cache path may look like:

```
__pycache__/module.cpython-314.pyc
```

The purpose of a .pyc file is not to make Python code execute like C machine code. It is a cache. It lets CPython skip some front-end compilation work on later imports when the cache is valid.

The rough import-time path is:

```
import module
-> find module.py
-> check for valid cached .pyc
-> if valid, load code object from .pyc
-> if invalid or absent, compile source
-> execute module code object
```

This is especially important for imported modules. When a module is imported, CPython commonly creates or uses a .pyc cache file. A top-level script run directly with `python script.py` is different: CPython normally does not create a .pyc cache for that top-level script merely because it was run directly.

Marshaling: Storing Code Objects, Not Source Text

A .pyc file stores a serialized form of a code object plus header metadata. The serialization mechanism is associated with Python's internal `marshal` format. This should not be confused with `pickle`.

The distinction matters:

- `marshal` is an internal serialization format used by Python for objects such as code objects;
- `pickle` is a general Python object serialization system intended for broader program use;
- .pyc files are implementation artifacts, not portable source-code replacements.

A .pyc file is therefore not a protected binary executable. It is not a secure hiding place for source logic. It is a cached compiled artifact for the interpreter.

Executing a Module: Compilation Is Not the Same as Running

A module can be compiled without its top-level code being executed. But when a module is imported normally, CPython executes the module's code object to create the module's runtime contents.

For example, if a file contains:

```
print("loading module")

x = 10

def f():
    return x + 1
```

then importing the module executes the top-level `print`, binds `x`, and creates the function object `f`. The body of `f` is not executed at import time. Instead, executing the `def` statement creates a function object whose code object will run later when `f()` is called.

This distinction is essential:

- compiling creates executable descriptions;
- executing a module creates module-level bindings;
- executing a function call creates a frame for that function's code object.

The Complete Mental Model

The complete simplified path is:

```
source file: example.py
    characters written by programmer

tokenizer
    groups characters into Python tokens

parser
    checks grammar and builds structure

AST
    represents meaningful program form

compiler
    analyzes scopes and generates code objects

code object
    stores bytecode and metadata

.pyc cache, sometimes
    stores marshaled code object for future imports

frame
    active runtime environment for executing code

interpreter loop
    reads bytecode and performs operations on objects
```

This model gives the right intuition for an effective Python programmer. Python source is not native machine code. It is also not executed as raw text. CPython compiles it into bytecode-bearing code objects, optionally caches those objects in `.pyc` files, and executes them inside frames through the interpreter loop.

That is the bridge to the next subsection. Once source has become code objects, the next question is: what happens when those code objects run? The answer requires code objects, frame objects, and namespace dictionaries.

3.3.5 Code Objects, Frame Objects, and Namespace Dictionaries

After CPython has parsed and compiled source code, the important runtime question becomes: what exactly is executed? The answer is not “the source text.” CPython executes code objects inside frame objects, using namespace dictionaries to resolve names and store bindings.

These three structures form one of the most important runtime triangles in Python:

```
code object
    describes what can be executed

frame object
    represents one active execution of that code

namespace dictionaries
    map names to objects during that execution
```

This distinction is essential because Python code and Python execution are not the same thing. A code object is executable material. A frame is a running activation of that material. A namespace is the name-to-object environment used while that frame executes.

Code Objects: Stored Executable Descriptions

A code object is CPython's internal representation of a compiled block of Python code. Module bodies, function bodies, lambdas, comprehensions, class bodies, and nested functions all have code objects.

A code object contains the information needed to execute a block, including:

- bytecode instructions;
- constants used by the code;
- names referenced by the code;
- local variable names;
- free variables and cell variables for closures;
- required evaluation stack size;
- filename, function name, and line-number metadata.

A simplified view is:

```
code object
    bytecode
    constants
    referenced names
    local variable names
    closure variable metadata
    source-location metadata
```

A code object is not a function call. It does not by itself have current local variable values, an instruction position, or a live execution stack. It is closer to a compiled recipe: it says what instructions exist and what names and constants those instructions refer to.

For example:

```
def add(a, b):  
    return a + b
```

When this definition is compiled, the body of `add` has a code object. That code object knows that there are local names such as `a` and `b`, and that the function body contains bytecode for loading them, adding them, and returning the result.

But the code object does not yet know the runtime values of `a` and `b`. Those values exist only when the function is called.

Function Objects Wrap Code Objects

A function object is not the same thing as a code object. A function object wraps a code object together with runtime context.

Conceptually:

```
function object  
  code object  
  global namespace  
  default argument values  
  keyword-only defaults  
  annotations  
  closure cells, if needed
```

This explains why Python functions are first-class runtime objects. When CPython executes a `def` statement, it creates a function object and binds the function name to that object.

So this:

```
def add(a, b):  
    return a + b
```

does not merely “declare” a function in the C sense. It executes a statement that creates a function object and stores it in the current namespace under the name `add`.

This is why the following works:

```
f = add  
result = f(2, 3)
```

Both `add` and `f` refer to the same function object. The function object contains the code object that will be executed when the function is called.

Frame Objects: One Active Execution

A frame object represents an active execution of a code object. If a code object is the executable recipe, a frame is one running use of that recipe.

When a function is called, CPython prepares a frame for that call. The frame connects:

- the code object being executed;
- the current instruction position;
- the local names or local slots;

- the global namespace;
- the built-in namespace;
- temporary expression values;
- exception and control-flow state.

A simplified view is:

```
frame
  code object being executed
  current bytecode position
  local namespace / local slots
  global namespace
  builtins namespace
  evaluation stack
  link to caller frame
```

This is the Python-level equivalent of an activation record, but it should not be confused with a raw C stack frame. CPython is a C program and it uses the native C stack internally, but Python execution also has its own frame model.

This is why Python tracebacks can show a chain of calls. When an exception propagates, CPython can inspect the active or recorded frames and report which code object, line, and call level were involved.

The Same Code Object Can Have Many Frames

A code object can be executed many times. Each execution needs its own frame.

For example:

```
def add(a, b):
    return a + b

x = add(1, 2)
y = add(10, 20)
```

The function `add` has one code object. But the two calls create two separate executions. In the first call, `a` is bound to 1 and `b` to 2. In the second call, `a` is bound to 10 and `b` to 20.

The code object is shared executable structure. The frame is per-call execution state.

This is the same distinction as:

```
recipe          -> code object
cooking once    -> frame
```

The recipe can be reused. Each actual cooking event has its own ingredients, current step, and temporary state.

Namespace Dictionaries: Names Point to Objects

Python names live in namespaces. A namespace is a mapping from names to objects. At module level, this mapping is normally a dictionary.

For example:

```
x = 10
message = "hello"
```

creates bindings conceptually like:

```
module namespace
    "x"          -> int object 10
    "message" -> str object "hello"
```

This is why Python variables are not C boxes. A name does not own a fixed typed storage cell. A name is an entry in a namespace, and that entry points to an object.

When Python executes:

```
x = 10
x = "hello"
```

the binding changes:

```
before:
    "x" -> int object 10

after:
    "x" -> str object "hello"
```

The name `x` has been rebound. It did not mutate from an integer storage box into a string storage box.

Local, Global, and Built-in Name Resolution

When code executes, CPython must resolve names. The basic lookup chain for ordinary function code is:

```
local namespace
    then global namespace
        then builtins namespace
```

For example:

```
x = 10

def f():
    print(x)
```

Inside `f`, the name `print` is usually found in the built-in namespace. The name `x` is found in the global namespace, because `x` is not local to `f`.

The frame provides access to these namespaces. Conceptually:

```
frame for f
    locals    -> local names of f
    globals   -> module dictionary
    builtins  -> built-in names dictionary
```

This explains why Python can resolve a name that was not declared inside the function. It is not reading raw storage from the C stack. It is performing namespace lookup according to Python's execution rules.

Fast Locals: The Practical Optimization

Although it is useful to explain namespaces as dictionaries, CPython does not always store ordinary function locals as a normal dictionary during execution. For speed, local variables in optimized function scopes are usually stored internally in a faster array-like layout associated with the frame.

The important conceptual point remains the same: local names are bindings to objects. But the implementation may avoid using a normal dictionary for every local variable access because dictionary lookup would be expensive inside every function call.

So the teaching model is:

At the language level, locals behave like a local namespace. At the CPython implementation level, optimized function locals may be stored in faster internal slots and exposed as a mapping when introspection asks for them.

This explains why `locals()` inside a function should not be treated as an ordinary mutable dictionary that directly controls execution in all cases. It is a view or mapping of local state, not the same simple thing as a module dictionary.

Module Namespaces Are Real Dictionaries

Module namespaces are easier. A module object has a dictionary that stores its top-level names.

For example, a file:

```
x = 10
```

```
def f():  
    return x + 1
```

creates a module namespace roughly like:

```
module.__dict__  
  "__name__" -> module name  
  "x"         -> int object 10  
  "f"         -> function object
```

The function object `f` also remembers the global namespace in which it was defined. This is why `f` can later find `x`, even if `f` is passed somewhere else.

For example:

```
g = f
```

does not copy the code or copy the global variables. It binds another name, `g`, to the same function object. That function object still carries a reference to its original global namespace.

Class Body Execution Uses a Namespace Too

Class definitions also use namespaces, but they behave differently from ordinary functions.

When Python executes:

```
class Point:  
    x = 0  
    y = 0  
  
    def move(self, dx, dy):  
        self.x += dx  
        self.y += dy
```

the class body is executed in a temporary namespace. Names assigned in the class body become entries in that namespace. After the body finishes, Python creates a class object using that namespace. Conceptually:

```
temporary class namespace
    "x"      -> int object 0
    "y"      -> int object 0
    "move"   -> function object

type machinery creates class object Point
Point.__dict__ contains these entries
```

This is why a class definition is executable code, not only a static declaration. The body runs, creates objects, binds names, and then those names become the class object's attributes.

Closures: When Frames Need Preserved Cells

Nested functions introduce one more mechanism: closure cells.

Example:

```
def outer():
    x = 10

    def inner():
        return x

    return inner
```

Here, `inner` uses `x` from `outer`. But `outer` returns, so its ordinary call frame cannot simply remain active as a normal stack frame forever. CPython solves this by storing captured variables in cell objects. The returned function keeps references to those cells.

Conceptually:

```
inner function object
    code object
    globals
    closure cells
        cell for x -> int object 10
```

This is the concrete runtime basis of closures. A closure is not a mystical memory of the past. It is a function object carrying references to cells that preserve needed bindings from an enclosing scope.

Why This Matters for Python Programming

This machinery explains several everyday Python facts.

First, function definitions create objects:

```
def f():
    pass
```

After this executes, `f` is a name bound to a function object.

Second, assignment binds names:

```
a = []
b = a
```

Both names point to the same list object.

Third, function calls create frames:

```
f()
```

Calling `f` does not copy the source text. It creates execution state for `f`'s code object.

Fourth, globals are namespace entries:

```
x = 5
```

At module level, this stores `x` in the module dictionary.

Fifth, closures preserve selected bindings:

```
def outer():
    x = 5
    def inner():
        return x
    return inner
```

The returned `inner` function can still access `x` because the needed binding is preserved in closure machinery.

The Mental Model

The complete model is:

```
source code
    is compiled into code objects

function definitions
    create function objects that wrap code objects

function calls
    create frames that execute code objects

frames
    contain or access local, global, and built-in namespaces

namespaces
    bind names to objects

objects
    carry identity, type, value, and implementation metadata
```

This is the correct level of detail for learning Python effectively. We do not need to memorize CPython's source files, but we must understand the architecture.

Python execution is not text being read casually. It is compiled code objects running inside frame state, resolving names through namespace mappings, and manipulating objects through the CPython runtime.

3.3.6 The CPython Interpreter Loop as the Concrete Execution Site of Python Bytecode

After source code has been compiled into code objects, and after execution has been placed inside a frame, the remaining question is direct: where does the program actually run?

In CPython, the immediate answer is the interpreter loop. This loop is the part of the runtime that repeatedly reads bytecode instructions from the current frame and performs the corresponding operations on Python objects.

A simplified view is:

```
current frame
  contains code object
    contains bytecode instruction stream

interpreter loop
  fetch next bytecode instruction
  decode what operation it means
  perform that operation on Python objects
  update frame state
  move to the next instruction
```

This is the concrete execution site of Python code in CPython. The CPU is still executing native machine instructions from the CPython executable. But the user's Python program is being executed as CPython bytecode interpreted by this runtime loop.

The Interpreter Loop Is a Virtual Instruction Engine

A physical CPU executes machine instructions such as loads, stores, jumps, additions, calls, and returns. CPython's interpreter loop executes virtual instructions: bytecode operations designed for the Python runtime.

A Python expression such as

```
x = a + b
```

does not become one ordinary CPU addition instruction in the general case. Conceptually, CPython must do something closer to this:

```
load object bound to name a
load object bound to name b
perform Python-level addition
bind result to name x
```

Each step is runtime-managed. Loading a name may require namespace lookup. Addition may require type inspection and dynamic dispatch. Binding a name may update a local slot or namespace dictionary. References must be maintained. Exceptions must be possible at almost every stage.

The bytecode instruction stream is therefore not a machine-code replacement. It is an instruction stream for the Python object machine.

The Evaluation Stack: Temporary Objects During Expression Execution

CPython bytecode execution uses an evaluation stack. This stack is not the same thing as the native C call stack. It is a Python bytecode evaluation structure used to hold temporary object references while an expression is being evaluated.

For example, the expression

```
a + b * c
```

requires intermediate values. CPython must load `a`, load `b`, load `c`, compute `b * c`, then compute `a + result`. During this process, temporary object references are pushed onto and popped from the frame's evaluation stack.

A simplified stack-machine view is:

```

load a      -> push object bound to a
load b      -> push object bound to b
load c      -> push object bound to c
multiply    -> pop b and c, push result
add         -> pop a and previous result, push result
store x     -> pop result and bind it to x

```

This explains why Python bytecode is often described as stack-oriented. Operations commonly take their operands from the evaluation stack and place their result back onto that stack.

The important lesson is not the exact opcode names. Those can change between Python versions. The important lesson is the execution model: bytecode instructions manipulate references to Python objects through a runtime evaluation stack.

Name Lookup Is Runtime Work

When bytecode asks for a name, CPython must find the object currently bound to that name. The lookup path depends on how the compiler classified the name.

A local variable in an optimized function may be read from a fast local slot. A global name may be searched in the module's global dictionary and then in the built-in namespace. An attribute access may call descriptor logic or custom attribute access methods. A closure variable may be read from a cell.

For example:

```

def f():
    return len(items)

```

Inside this function, `len` is usually not a local variable. CPython must resolve it through global and built-in lookup rules. The name `items` may be local, global, or closed over from another scope depending on the surrounding program.

This is a major difference from C. In C, many variable addresses and storage layouts are resolved by the compiler before execution. In Python, name lookup is often runtime work. CPython can optimize some cases, but the language model remains name-to-object binding, not fixed typed storage.

Operations Dispatch Through Object Types

When CPython executes an operation such as addition, comparison, indexing, calling, iteration, or attribute access, it must respect Python's object model.

For example:

```
a + b
```

may mean integer addition, floating-point addition, string concatenation, list concatenation, NumPy array addition, or a user-defined operation. CPython cannot assume one fixed operation from the symbol `+` alone.

The interpreter must inspect the objects and their types, then dispatch to the behavior defined by those types. At the C level, this often means using function pointers and slots stored in type objects.

Conceptually:

```
bytecode says: perform addition
```

```

runtime asks:
    what is the type of a?
    what is the type of b?
    which addition behavior applies?
    did the operation succeed?
    did it raise an exception?

```

This is why Python operators are powerful but expensive compared with fixed native arithmetic. One visible symbol may activate a large amount of runtime machinery.

Function Calls Create New Execution State

A Python function call is also executed through bytecode and runtime frame machinery. When CPython calls a Python function, it must prepare a new execution context for that function's code object.

Conceptually, a call requires:

- evaluating the callable object;
- evaluating argument expressions;
- matching arguments to parameters;
- applying default values if needed;
- preparing local bindings;
- creating or activating a frame for the callee;
- executing the callee's bytecode;
- returning a result or propagating an exception.

So this source code:

```
result = f(10, 20)
```

does not simply jump to a fixed native address in the C sense. CPython must first determine what object `f` currently refers to. It may be a Python function, a built-in function, a bound method, a class object, or any object implementing call behavior. Only after resolving that callable can CPython perform the call according to Python's rules.

Again, the source is short because the runtime is doing the work.

Control Flow Is Bytecode Movement

Python control flow also becomes bytecode-level movement. An `if` statement, loop, exception handler, return statement, `break`, or `continue` eventually changes which bytecode instruction executes next.

For example:

```
if x > 0:
    y = 1
else:
    y = -1
```

compiles into bytecode that evaluates the condition and jumps to the correct block. The source-level branch becomes instruction-position control inside the current frame.

A loop works similarly:

```
for item in items:
    process(item)
```

The interpreter must obtain an iterator, request the next value, bind it to `item`, execute the loop body, then return to the iteration point until the iterator is exhausted.

The source-level `for` loop therefore hides a protocol:

```

get iterator from items
repeat:
    ask iterator for next object
    if exhausted, exit loop
    bind object to item
    execute loop body

```

This is not a C-style index loop unless the object involved happens to implement iteration that way internally.

Exceptions Are Also Part of the Loop

Almost every bytecode operation can fail. Name lookup can fail. Attribute access can fail. Addition can fail. Function calls can fail. Iterator access can signal exhaustion. File operations can raise exceptions. User-defined code can raise exceptions directly.

Therefore, the interpreter loop must constantly be prepared for exceptional control flow.

If an operation raises an exception, CPython does not merely print an error immediately. It checks whether the current frame has a matching exception-handling region. If not, the frame is unwound and the exception propagates to the caller. This continues until a handler is found or the exception reaches the top level.

Conceptually:

```

bytecode operation raises exception
current frame checks for handler
    if handler exists:
        jump to handler bytecode
    else:
        leave frame
        propagate exception to caller frame

```

This is why Python exceptions can cross many function calls. They are not local error messages. They are structured non-local control flow managed by the runtime.

Return Means Leaving the Current Frame

When a function executes

```
return value
```

CPython evaluates `value`, places the result where the caller can receive it, and ends the current frame's execution. The caller then continues after the call site.

A return is therefore not only a source-level statement. It is a bytecode-level frame transition:

```

callee frame produces return object
callee frame finishes
caller frame receives object
caller frame continues

```

The returned object is not copied as a raw memory block. The caller receives a reference to a Python object.

This explains common behavior:

```
def make_list():
    return []
```

```
a = make_list()
```

The function creates a list object and returns a reference to it. The list survives after the function frame ends because the caller now has a reference to the object.

Why Python Has Runtime Overhead

The interpreter loop explains Python's ordinary runtime overhead. A small Python operation may require several layers of work:

- fetch a bytecode instruction;
- decode or dispatch that instruction;
- read or write frame state;
- look up names;
- manipulate the evaluation stack;
- inspect object types;
- call type-specific behavior;
- update reference counts;
- check for exceptions;
- move to the next bytecode instruction.

In C, an integer addition may compile into a direct CPU instruction when the types and storage are known. In Python, the expression may represent many possible behaviors, so CPython must preserve that flexibility at runtime.

This is the cost of Python's dynamic object model. The programmer writes less machinery. CPython executes more machinery.

Modern CPython Optimizes, but the Model Remains

Modern CPython versions include many optimizations. Some bytecode instructions are specialized after CPython observes common runtime patterns. Some call paths are faster than older versions. Some interpreter internals have been reorganized to reduce overhead.

But these optimizations do not change the basic teaching model. CPython is still executing Python through runtime-managed bytecode, frames, objects, namespaces, and type-based behavior. The exact internal opcodes may change, but the conceptual structure remains useful.

Therefore, one should not memorize bytecode listings as if they were stable language law. Bytecode is a CPython implementation detail. It is useful for understanding execution, debugging performance, and seeing how source is lowered, but it is not the Python language itself.

The Final Mental Model

The interpreter loop connects all previous machinery:

```
code object
  provides bytecode

frame
  provides current execution state

namespace mappings / local slots
  provide name bindings

evaluation stack
  stores temporary object references
```



```
type objects
    define operation behavior

interpreter loop
    repeatedly executes bytecode operations
```

This is the concrete execution site of Python code in CPython.
The programmer sees:

```
x = a + b
```

CPython performs:

```
load names
retrieve objects
dispatch operation through object types
manage references
handle possible exceptions
store resulting binding
continue bytecode execution
```

This is enough CPython detail for the purpose of this course. We do not need to become CPython implementers. But we do need to know that Python code runs inside a real interpreter loop, manipulating real objects, through real frames and namespaces, inside a native process. That is the level where Python’s behavior becomes explainable instead of magical.

3.4 CPython Memory Management Dynamics and Object Topologies

3.4.1 The Base Object Framework

Before discussing heaps, stacks, reference counts, or allocation arenas, we must first clarify what CPython is allocating and managing. CPython does not mainly manage raw integers, raw strings, raw functions, or raw classes as separate unrelated categories. It manages Python objects. These objects may represent numbers, text, containers, functions, modules, classes, exceptions, frames, iterators, files, or user-defined instances, but they all enter the runtime through the same general object model.

This is the concrete foundation beneath the phrase “everything is an object.” The phrase should not be read poetically. It means that values visible in Python source code are represented at runtime as objects with identity, type, value, and implementation metadata. The programmer sees simple notation; CPython sees structured runtime entities.

This base object framework is the reason Python can treat values uniformly. A number can be stored in a list. A function can be passed as an argument. A class can be assigned to a variable. A module can be inspected. An exception can be caught as an object. A type can itself have a type. These behaviors are not separate tricks. They follow from the decision to represent program material through a common object system.

The Realization of the “Everything is an Object” Paradigm: Unifying Functions, Types, and Primitives as First-Class Runtime Objects

In C, there is a strong difference between categories such as integers, functions, structs, pointers, and types. An `int` value is not a function. A function is not an ordinary value in the same sense as a heap-allocated object. A type declaration is not a runtime object that can normally be passed around, inspected, modified, or stored in a list. C has runtime values, but much of its structure is resolved before execution and disappears into compiled machine code, addresses, layouts, and calling conventions.

Python makes a different decision. It keeps much more program structure alive at runtime. Integers are objects. Strings are objects. Lists are objects. Functions are objects. Classes are objects. Modules are objects. Exceptions are objects. Even compiled code blocks are represented by code objects. This does not mean that all these objects have the same internal layout or the same behavior. It means that they participate in the same runtime object universe.

For example:

```
x = 10
name = "Python"

def f():
    return x

class C:
    pass
```

After this code executes, the module namespace contains bindings similar to:

```
"x"    -> integer object
"name" -> string object
"f"    -> function object
"C"    -> class object
```

The names do not hold raw values directly. They point to runtime objects. The integer 10, the string "Python", the function `f`, and the class `C` are different kinds of objects, but they are all objects.

This is why the following operations are natural in Python:

```
items = [10, "Python", f, C]
```

The list does not need to be declared as a list of integers, strings, functions, or classes. It stores references to Python objects. Each referenced object carries its own runtime type information.

This is a decisive difference from a C array such as:

```
int values[4];
```

That C array is a contiguous region of storage for integer values of one fixed type. A Python list is not that. A Python list is a container object that stores references to other objects. The elements may point to objects of different types because the object itself carries the type, not the list slot.

Primitives Are Not Bare Machine Primitives

Python integers look like primitive values at the source level:

```
a = 5
b = 7
c = a + b
```

But in CPython, these are not bare CPU integers stored directly inside variable boxes. They are Python integer objects. The addition operation must operate according to Python object rules. CPython must load the objects referenced by `a` and `b`, determine that integer addition applies, produce or reuse an integer result object, and bind `c` to that object.

This is why Python integers can behave differently from fixed-width C integers. Python integers are arbitrary-precision integer objects. They can grow beyond the size of a C `int` or `long`, limited by available memory rather than by one fixed machine word size.

For example:

```
x = 10 ** 100
```

This is a normal Python integer. In C, a value of that size would require a special library or a custom representation. In Python, the object model hides that representation behind the ordinary `int` interface.

The cost is that a Python integer is heavier than a raw C integer. It has object metadata, type information, reference-management overhead, and a variable-sized internal representation. Python gains uniformity and power by giving up the direct simplicity of raw machine values.

Functions Are Runtime Objects

A function definition in Python is executable code that creates a function object. It is not merely a compile-time declaration.

When CPython executes:

```
def add(a, b):  
    return a + b
```

it creates a function object and binds the name `add` to that object. This is why the function can be assigned to another name:

```
operation = add
```

Now `add` and `operation` refer to the same function object. No source code was copied. No new function body was created. A second name was bound to the same runtime object.

The function object can also be passed as an argument:

```
def apply(func, x, y):  
    return func(x, y)
```

```
result = apply(add, 2, 3)
```

This works because `add` is an object. The parameter `func` becomes another reference to that same object. The call `func(x, y)` then invokes the function object currently bound to `func`.

This is the runtime basis of first-class functions. A function is not only something that can be called. It is a value that can be stored, passed, returned, placed in containers, attached to objects, and inspected.

Classes and Types Are Runtime Objects

Classes are also objects. When Python executes a class statement, it does not merely record a static type declaration. It executes the class body, collects the resulting namespace, and creates a class object.

For example:

```
class Point:  
    x = 0  
    y = 0
```

After this executes, `Point` is a name bound to a class object. That class object can be passed around like any other object:

```
T = Point  
p = T()
```

Here, `T` and `Point` refer to the same class object. Calling `T()` creates an instance of that class.

This is why Python can use classes dynamically. A class is not only a compile-time type descriptor. It is a runtime object that can be stored in variables, returned from functions, inspected, subclassed, and used to create instances.

Even types themselves fit into the same model. The expression:

```
type(10)
```

returns the type object representing `int`. The expression:

```
type(Point)
```

usually returns `type`, because user-defined classes are themselves objects created by the metaclass `type`. This recursive structure is unusual for programmers coming from C, but it is central to Python's runtime design.

Containers Store References to Objects

Because everything visible at the Python level is an object, Python containers store object references.

For example:

```
a = []
b = [a, a]
```

The list `b` contains two references to the same list object `a`. It does not contain two independent copies of `a`. Therefore:

```
a.append(1)
```

makes both elements of `b` appear changed, because both positions refer to the same object.

Conceptually:

```
a -----+
          |
          v
        list object []

b -> list object [ reference to same list,
                  reference to same list ]
```

This is not a special list trick. It is the normal consequence of Python's object-reference model. Names point to objects. Containers hold references to objects. Assignment copies references, not the entire object graph.

Uniformity Does Not Mean Identical Behavior

Saying that everything is an object does not mean that all objects are mutable, all objects are stored the same way, or all objects allow the same operations.

An integer object is immutable. A list object is mutable. A function object is callable. A module object has a namespace. A class object creates instances. A file object represents an external resource. A code object stores executable bytecode. These objects are different, but CPython can still manage them through the same general object framework.

The type of the object determines its behavior. This is why `+` has different meanings depending on the objects involved:

```
1 + 2           # integer addition
"ab" + "cd"     # string concatenation
[1, 2] + [3, 4] # list concatenation
```

The syntax is the same, but the operation is selected through the runtime types of the objects. This is not possible if values are treated only as raw memory without type metadata.

The Practical Consequence

The practical consequence is that a Python program is an object graph. Names, containers, functions, classes, modules, and frames are connected by references to objects.

A simplified module after execution may look like this:

```
module namespace
    "x"          -> int object
    "text"       -> str object
    "items"      -> list object
                  -> references to other objects
    "f"          -> function object
                  -> code object
                  -> global namespace
    "C"          -> class object
                  -> class namespace
```

This object graph is what CPython must allocate, connect, track, and eventually reclaim. Memory management in CPython is therefore not only about bytes. It is about object topology: which objects exist, which objects refer to which other objects, which names or containers keep objects alive, and when an object becomes unreachable.

This prepares the next subsection. If everything is represented as an object, then CPython needs a common base representation for objects. That representation begins with the `PyObject` structure, whose core fields explain both object identity at runtime and CPython's reference-counting memory discipline.

Underlying C Representation: Unpacking the `PyObject` Base Struct (Reference Count Tracker `ob_refcnt` and Type Object Pointer `ob_type`)

The Python object model has a concrete C-level foundation in CPython. At the Python level, we say that every object has identity, type, and value. At the CPython level, every object begins with enough internal information for the runtime to treat a pointer as a Python object. In normal release builds, the common base object representation contains two central pieces of information: a reference count and a pointer to the object's type. `:contentReference[oaicite:0]index=0`

A simplified conceptual form is:

```
PyObject
    ob_refcnt    -> reference count
    ob_type      -> pointer to type object
```

This is not meant as code that students should use directly. The official C API warns that these fields should not be accessed directly; code should use access macros and functions such as `Py_REFCNT()`, `Py_TYPE()`, `Py_INCREF()`, and `Py_DECREF()`. `:contentReference[oaicite:1]index=1` The purpose here is conceptual: the base object header explains why CPython can manage very different Python values through one common runtime discipline.

Every Python Object Begins with Runtime Metadata

Consider the Python statement:

```
x = 10
```

At the source level, this looks like a simple value assignment. At the CPython level, 10 is represented by a Python integer object. That object is not only the numeric value. It also has object metadata.

Conceptually:

```

name x
  |
  v
integer object
  ob_refcnt -> number of active strong references
  ob_type   -> pointer to int type object
  value     -> integer-specific stored number data

```

The same general pattern applies to other objects:

```

string object
  object header
  string-specific data

list object
  object header
  list-specific storage of element references

function object
  object header
  function-specific references to code, globals, defaults

class object
  object header
  type/class-specific data

```

The object header is what lets CPython treat all of these as Python objects while still allowing each object kind to have its own specific internal layout.

ob_refcnt: The Reference Count Tracker

The field usually explained as `ob_refcnt` stores the object's reference count. Its purpose is to help CPython decide whether the object is still in use.

When a new strong reference to an object is created, CPython increases the count. When a strong reference is released, CPython decreases the count. When the last strong reference is released, the object's type-specific deallocation function can be invoked. `:contentReference[oaicite:2]index=2`

Conceptually:

```

a = []
  list object refcount: 1

b = a
  list object refcount: 2

a = None
  list object refcount: 1

b = None
  list object refcount: 0
  object may be deallocated

```

This explains why assignment and rebinding have memory consequences. Rebinding a name does not merely change a symbolic label. It releases one reference and creates another reference.

For example:

```
x = []
x = 5
```

The second assignment removes the reference from `x` to the list object and binds `x` to an integer object. If the list had no other references, it becomes eligible for immediate deallocation in CPython.

The reference count is therefore not an optional statistic. It is part of the ordinary object-lifetime machinery of CPython.

Reference Count Does Not Always Mean Exact Human-Visible Reachability

The reference count is useful, but it must not be overinterpreted. Modern CPython has details such as immortal objects, and the official documentation warns that the value returned by `Py_REFCNT()` may not always reflect the exact number of actual references in the simple human sense. Some objects may have artificially high reference counts or may not be modified by increment/decrement operations.

For learning Python, the correct rule is:

Reference counting is CPython's main object-lifetime mechanism, but individual reference-count numbers are implementation details, not a normal programming interface.

So the student should understand reference counting as a model of persistence, not as something ordinary Python code should depend on numerically.

`ob_type`: The Pointer to the Type Object

The second central field is `ob_type`. This field points to the type object that describes the object's behavior. The official C API says this is the object's type and recommends using `Py_TYPE()` rather than direct field access.

Conceptually:

```
object
  ob_type ----> type object
                    name
                    size information
                    method table
                    numeric operation slots
                    sequence operation slots
                    mapping operation slots
                    attribute access behavior
                    deallocation behavior
```

This is the concrete basis of dynamic typing in Python. A Python name does not carry a fixed declared type. The object carries its runtime type.

For example:

```
x = 10
x = "hello"
```

The name `x` first points to an integer object whose `ob_type` points to the integer type object. Later, `x` points to a string object whose `ob_type` points to the string type object. The name is dynamically rebound; the objects themselves carry their types.

This is why `type(x)` can work at runtime. CPython follows the object's type pointer and exposes the type object at the Python level.

The Type Pointer Explains Dynamic Operations

When CPython evaluates:

```
a + b
```

it cannot assume that `+` means one fixed CPU instruction. It must look at the objects involved and use their type information.

For example:

```
1 + 2           # integer addition
"ab" + "cd"     # string concatenation
[1, 2] + [3, 4] # list concatenation
```

The same source operator leads to different behavior because the operands are different objects with different type objects.

Conceptually:

bytecode says: add two objects

CPython checks:

```
left object's type
right object's type
applicable operation slot
result object or exception
```

This is one reason Python is flexible and one reason Python operations are heavier than fixed C operations. In C, `int + int` can compile into direct arithmetic because the operand types are known statically. In Python, `a + b` must preserve runtime dispatch.

The Base Header Is Followed by Type-Specific Data

`PyObject` is the common beginning, not the whole object. Real object types extend this base with fields appropriate to their own representation.

A simplified integer object might be imagined as:

```
integer object
  PyObject header
    reference count
    type pointer
  integer-specific storage
```

A simplified list object might be imagined as:

```
list object
  PyObject header
    reference count
    type pointer
  list size
  allocated capacity
  pointer to array of element references
```

A simplified function object might be imagined as:


```
function object
    PyObject header
        reference count
        type pointer
    code object reference
    globals dictionary reference
    defaults reference
    closure reference
```

The common header allows uniform runtime handling. The type-specific tail allows each kind of object to store what it needs.

PyVarObject: Objects with a Size Field

Some objects need an additional size field because their logical length varies between instances. CPython uses `PyVarObject` for variable-sized objects. It extends `PyObject` with an `ob_size` field, intended for objects that have some notion of length. :contentReference[oaicite:5]index=5

Conceptually:

```
PyVarObject
    PyObject base
        ob_refcnt
        ob_type
    ob_size
```

This kind of structure is useful for objects such as tuples, bytes, and other variable-sized built-in representations. The important idea is that variable-sized objects still begin with the object header. They remain ordinary Python objects from the runtime's point of view.

The size field should not be confused blindly with `len(obj)` in every possible case. The official documentation treats `ob_size` as internal implementation detail and says that public length should be obtained through the proper object API. :contentReference[oaicite:6]index=6

Why Direct Field Access Is Discouraged

Although the fields `ob_refcnt` and `ob_type` are useful for explanation, extension code should not treat them as ordinary public struct fields. CPython documentation recommends using macros and functions instead of direct access, because implementation details may vary by build configuration, platform, version, debug mode, or free-threading work. :contentReference[oaicite:7]index=7

For this course, the reason is also pedagogical. We are not teaching students to write CPython internals. We are using CPython internals to correct their Python mental model.

The mental model should be:

```
Python object
    has lifetime tracking
    has runtime type information
    has type-specific storage
    is manipulated through references
```

not:

```
Python variable
    is a C-style typed box containing a raw value
```

The Practical Consequence

The `PyObject` base structure explains several everyday Python facts:

- assignment binds names to objects rather than copying raw values into typed boxes;
- objects know their type at runtime;
- `type(obj)` is possible because the object carries a type pointer;
- object lifetime can be tracked because CPython maintains reference counts;
- different object kinds can still participate in one common runtime model;
- containers store references to objects, not raw heterogeneous values directly;
- operations dispatch through object type information rather than fixed source syntax alone.

A compact final picture is:

```

name
  |
  v
PyObject-based runtime object
  ob_refcnt  -> lifetime bookkeeping
  ob_type    -> behavior descriptor / type object
  ...
  type-specific data

```

This is the structural core of CPython's object model. Python's high-level uniformity is possible because CPython gives every object a common runtime beginning. The next memory-management layers build on this: objects are allocated on the managed heap, referenced from frames and containers, and reclaimed when the reference topology no longer keeps them alive.

3.4.2 The Heap Subsystem and Object Allocation

The Managed Runtime Allocation Arena: PyMalloc and Private Heap Segmentation Subsystems

When Python code creates an object, it does not normally ask the operating system directly for memory. The programmer writes:

```
x = [1, 2, 3]
```

but CPython must allocate several runtime objects behind that line: at least a list object and references to the element objects. This allocation happens inside CPython's managed memory system, not through a direct user-visible call to `malloc()`.

The official CPython model describes Python memory management as involving a private heap containing Python objects and internal data structures. This private heap is managed by the Python memory manager, not by ordinary Python code. The user manipulates objects, but CPython manages the memory blocks behind those objects. :contentReference[oaicite:0]index=0

This distinction is essential:

Python objects live in a runtime-managed heap, not in programmer-controlled C-style storage.

In C, the programmer may write:

```
int *p = malloc(10 * sizeof(int));
free(p);
```

In Python, ordinary code does not allocate and release object memory this way. CPython decides where an object is placed, how much memory it needs, how the allocation is grouped, and when the memory can be reused.

Why CPython Uses Its Own Allocator

CPython could theoretically ask the system allocator for every small object. But Python programs create many small objects: integers, tuples, strings, dictionaries, frames, iterators, temporary values, function objects, method objects, and countless short-lived internal structures.

Calling the platform allocator for every small object would be expensive. It would also produce fragmentation and poor locality. CPython therefore uses specialized allocation machinery for Python objects and internal buffers.

The Python memory manager has several layers. At the lowest level, a raw memory allocator obtains memory from the operating system. Above that, CPython has object-specific and size-specific allocators that manage smaller blocks more efficiently. The official documentation explicitly notes that different components handle sharing, segmentation, preallocation, caching, and object-specific allocation policies. [:contentReference\[oaicite:1\]index=1](https://docs.python.org/3/c-api/memory.html)

The important teaching idea is simple: CPython does not treat every object allocation as an isolated operating-system allocation. It reserves larger regions and subdivides them for Python's object workload.

Arenas, Pools, and Blocks

The small-object allocator traditionally known as `pymalloc` organizes memory into layers. The exact constants may vary across versions and platforms, but the conceptual structure is stable enough for understanding:

arena

large region obtained from the system allocator

pool

subdivision inside an arena, usually dedicated to one size class

block

small allocation unit returned for one object or internal structure

A simplified diagram is:

CPython private heap

arena

pool for 32-byte blocks

block

block

block

pool for 64-byte blocks

block

block

block

pool for 128-byte blocks

block

```

    block
    block

```

The goal is to make small allocations fast. If CPython needs a small block, it can often take one from an existing pool instead of asking the operating system for new memory.

This is why Python object allocation is not the same as a direct call to `C malloc()` for every object. There may be a system allocation underneath at some point, but CPython often reuses already reserved memory managed inside its own allocator.

Private Heap Does Not Mean Separate Physical RAM

The phrase “private heap” can mislead if read too literally. It does not mean Python owns a physically separate memory device. CPython is still a native process. Its memory ultimately comes from the operating system through normal virtual memory mechanisms.

The phrase means that CPython manages a region of process memory according to Python’s object-allocation rules. The operating system sees process memory. CPython sees arenas, pools, blocks, object headers, type-specific layouts, free lists, and garbage-collection metadata.

So the layers are:

```

operating system virtual memory
    gives memory to CPython process

```

```

CPython memory manager
    organizes memory into allocator structures

```

```

Python object system
    places objects into allocated blocks

```

```

Python code
    manipulates references to those objects

```

The programmer does not normally see these layers, but their effects appear in performance, memory usage, object lifetime, and extension-module rules.

Small Objects and Large Objects

`pymalloc` is mainly useful for small objects and small internal allocations. Large allocations may be handled differently and may go closer to the platform allocator. This is practical: a tiny tuple, a small integer object, and a large byte buffer do not have the same allocation needs.

Small objects benefit from size-class pooling. Large objects are less suitable for tiny fixed-size pools and may need larger contiguous regions. CPython’s memory manager therefore does not use one identical strategy for everything.

This matters for intuition. Python code creates objects uniformly at the source level, but CPython may allocate them through different paths internally. A small tuple, a dictionary table, a huge string, and a large list backing array are not all handled identically.

Object-Specific Allocation

Above the general allocator, some object types have specialized allocation behavior. The official documentation notes that object-specific allocators operate on the same private heap and may use policies adapted to each object type. For example, integers, strings, tuples, and dictionaries have different storage requirements and speed-space tradeoffs. [:contentReference\[oaicite:2\]index=2](#)

This is why “object allocation” in Python is not one single action. Creating a dictionary is not the same internal operation as creating an integer. Creating a list involves a list object plus an internal array of object references. Creating a string involves character storage and string metadata. Creating a tuple involves fixed-size reference storage.

At the Python level, all of these are objects. At the CPython level, each type has its own layout and allocation needs.

Why Extension Modules Must Respect the Python Memory Manager

This allocator structure is also why C extension modules must be careful. A native extension that creates Python objects must use the Python memory and object APIs, not randomly mix allocation and deallocation rules.

The official documentation warns that extension writers should not operate on Python objects with ordinary C library `malloc()`, `calloc()`, `realloc()`, and `free()` because mixing the C allocator and the Python memory manager can corrupt memory.

The reason is straightforward: the allocator that gives memory must be compatible with the allocator that releases it. If CPython allocates a Python object through its own allocator, freeing it through the wrong mechanism breaks the allocator's internal bookkeeping.

This is not a Python syntax issue. It is a concrete runtime ownership issue.

The Practical Mental Model

The practical model is:

Python code asks for objects indirectly

```
x = SomeObject()
```

CPython type machinery decides object layout

```
how large?
```

```
which type?
```

```
which allocator path?
```

CPython memory manager provides memory

```
often from private heap structures
```

```
often through pymalloc for small blocks
```

object is initialized

```
header
```

```
type pointer
```

```
value-specific fields
```

```
possible GC metadata
```

Python name receives reference

```
x -> object
```

The object is not created in a vague abstract space. It is allocated inside the CPython process, usually through CPython's managed memory machinery. The name in Python receives a reference to that object. The object then remains alive as long as references or runtime structures keep it reachable.

This prepares the next allocation topic. Once the allocator can provide memory, CPython must actually instantiate objects: choose a type, allocate the right layout, initialize fields, connect references, and return a usable Python object to the executing frame.

Explicit Dynamic Instantiation: Analyzing Instance Generation on the Heap

Object creation in Python looks simple at the source level:

```
p = Point(2, 3)
```

but this line hides several runtime steps. CPython must find the object bound to the name `Point`, determine that it is callable, execute the type's construction protocol, allocate memory for a new instance, initialize that instance, and finally bind the name `p` to the resulting object.

The important point is that Python objects are created dynamically. The program does not reserve a fixed compile-time storage slot for every future object. Instead, objects are produced while the program runs, placed in CPython-managed heap memory, and connected to the executing program through references.

A simplified model is:

source code

```
p = Point(2, 3)
```

runtime behavior

```
resolve name Point
call class object Point
allocate new instance object on heap
initialize instance state
bind name p to the new object
```

This differs sharply from a C local variable such as:

```
struct Point p;
```

In C, this may reserve storage directly in the current stack frame, depending on where the declaration appears. In Python, a statement such as `p = Point(2, 3)` does not create an object inside the variable `p`. It creates a heap object and binds the name `p` to it.

Calling a Class Means Running the Construction Protocol

A class in Python is itself an object. When the programmer writes:

```
Point(2, 3)
```

the class object `Point` is called. This call is not just a jump to one fixed constructor function. It follows Python's object-construction protocol.

At a simplified level, instance creation involves two conceptual phases:

- allocation: create a new object of the requested type;
- initialization: set up the object's initial state.

In Python terms, these phases correspond roughly to:

```
__new__    -> creates and returns the new object
__init__   -> initializes the returned object
```

For example:

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y
```

When the program executes:

```
p = Point(2, 3)
```

CPython creates a new `Point` instance, then calls `__init__` with that instance as `self`. The assignments inside `__init__` attach state to the new object.

Conceptually:

```
new Point instance
    initially allocated as an object
```

```
__init__(self, 2, 3)
    self.x = 2
    self.y = 3
```

```
final object
    x -> int object 2
    y -> int object 3
```

The object is created before `__init__` finishes. `__init__` does not return the object. It configures an object that has already been allocated.

The Instance Lives on the Managed Heap

The new instance is placed in CPython-managed heap memory. The Python name receives a reference to it:

```
p ---> Point instance object
        x -> int object 2
        y -> int object 3
```

This is the central memory distinction:

The name is not the object. The name points to the object.

If another name is assigned to the same object:

```
q = p
```

then no new `Point` instance is created. The binding becomes:

```
p ----+
      |
      v
    Point instance object

q ----+
```

Both names refer to the same heap object. If the object is mutable and one reference modifies it, the change is visible through the other reference.

For example:

```
q.x = 100
print(p.x)
```

prints the modified value because `p` and `q` are not two independent objects. They are two references to one object.

Instance State Is Usually Stored in an Attribute Namespace

Most ordinary Python objects store instance attributes in an instance namespace, commonly exposed as `__dict__`.

For example:

```
p = Point(2, 3)
print(p.__dict__)
```

may show a mapping similar to:

```
{'x': 2, 'y': 3}
```

Conceptually:

```
Point instance
  type -> Point
  instance namespace
    "x" -> int object 2
    "y" -> int object 3
```

So the attributes `p.x` and `p.y` are not C struct fields in the simple fixed-layout sense. For ordinary dynamic Python objects, they are name-to-object bindings stored in an attribute namespace.

This is why Python objects can often receive new attributes after creation:

```
p.z = 9
```

The instance namespace becomes:

```
"x" -> int object 2
"y" -> int object 3
"z" -> int object 9
```

This flexibility has a cost. A dynamic attribute dictionary is heavier than a fixed C struct layout. Python gains dynamic object shape; CPython pays with extra memory and lookup machinery.

Slots Reduce Dynamic Attribute Storage

Python also allows more fixed instance layouts through `__slots__`:

```
class Point:
    __slots__ = ("x", "y")

    def __init__(self, x, y):
        self.x = x
        self.y = y
```

With slots, instances do not need the same ordinary per-object attribute dictionary for those attributes. CPython can store references in a more fixed internal layout.

The conceptual difference is:

```
ordinary instance
  attribute dictionary:
    "x" -> object
    "y" -> object
```

```
slotted instance
  fixed slots:
    slot x -> object
    slot y -> object
```


This does not turn Python into C, but it shows that Python object layout can vary depending on class definition. The runtime still stores references to Python objects, but it may store them through a more compact structure.

Built-in Objects Use Type-Specific Layouts

User-defined instances are only one case. Built-in objects use specialized internal layouts.

A list object is not a dictionary of attributes holding its elements. It is a list object with internal storage for references to element objects. A tuple has fixed reference storage. A dictionary has a hash-table structure. A string has a Unicode representation. An integer has a variable-sized numeric representation.

At source level:

```
items = [1, 2, 3]
```

Conceptually:

```
items -> list object
        element slot 0 -> int object 1
        element slot 1 -> int object 2
        element slot 2 -> int object 3
```

The list object itself lives on the managed heap. Its internal element storage contains references to other objects. The integers are also Python objects, though CPython may cache or reuse some common integer objects.

This means that container creation is object-graph creation. The runtime is not merely placing raw values in a flat memory region. It is building a network of objects and references.

Construction May Reuse Existing Objects

Dynamic instantiation does not always mean a fresh object is created in the intuitive sense. CPython and Python types may reuse objects in some cases.

For example, small integers and some strings may be cached or interned. Immutable objects may be reused because their visible value cannot be changed. A constructor may also return an existing object from `__new__` instead of allocating a fresh one.

This means the programmer should not reason as follows:

Every expression that looks like construction necessarily creates a brand-new object.

The safer model is:

An expression produces a reference to an object. Whether that object is newly allocated, cached, reused, or internally shared depends on the type and implementation.

For mutable user-defined instances, a class call usually creates a fresh object. For immutable or optimized built-in objects, reuse is possible.

Instantiation Creates References, Not Deep Copies

Object creation often connects existing objects into a new object. It does not automatically copy everything deeply.

Example:

```
data = [1, 2, 3]
```

```
class Box:
    def __init__(self, value):
        self.value = value
```

```
b = Box(data)
```

The new `Box` instance does not contain an independent deep copy of `data`. It stores a reference to the same list object:

```
b -> Box instance
      value ----+
                |
                v
data -----> list object [1, 2, 3]
```

If the list is mutated:

```
data.append(4)
```

then `b.value` observes the same changed list.

This is the same reference model as assignment, now appearing inside object construction. Initializing an instance attribute usually stores a reference to an object, not a deep structural clone.

The Practical Mental Model

The practical model of instance generation is:

```
class call
    resolves class object

type machinery
    invokes allocation protocol

allocator
    reserves memory for a new object layout

object initialization
    fills object header and type-specific fields

__init__
    binds initial attribute references

assignment
    binds a Python name to the resulting object
```

At the source level, this may be one line:

```
p = Point(2, 3)
```

At the runtime level, it is a sequence of object operations:

```
resolve Point
call Point
allocate Point instance
initialize attributes
return object reference
bind p to that reference
```

This explains why Python object creation is flexible but not free. Every object must have a type, a lifetime, reference relationships, and memory storage. Every dynamic attribute adds namespace structure. Every container connects references to other objects. CPython hides this machinery from the source code, but the machinery exists.

For an effective Python programmer, the essential conclusion is this:

Instantiation creates heap objects and reference relationships. Names and attributes do not contain objects directly; they point to objects managed by CPython.

Object Persistence Invariants: Reference Counting Foundations and Automated Slot Reclamation Realities

Once a Python object has been allocated, CPython must decide how long that object remains alive. This is the problem of object persistence. An object should remain valid while the program can still reach it, and it should become reclaimable when the program can no longer use it. CPython solves most of this problem through reference counting, supplemented by cyclic garbage collection.

The central invariant is simple:

An object remains alive while there are active strong references to it.

A reference may be held by a name, a container, an attribute, a frame, a closure cell, an exception object, an internal runtime structure, or a temporary value on the bytecode evaluation stack. The object does not care whether the reference came from a variable, a list, a dictionary, or a function call. From the memory-management point of view, what matters is that something still points to it.

References Keep Objects Alive

Consider:

```
a = []
b = a
```

Only one list object has been created. Both names refer to it:

```
a ----+
      |
      v
    list object
      ^
      |
b ----+
```

The object persists because references to it exist. If one name is rebound:

```
a = None
```

the list object is not automatically destroyed, because `b` still refers to it:

```
a -> None object
b -> list object
```

Only when the last strong reference disappears can the object become reclaimable.

Reference Count Updates

At the CPython level, an object carries a reference count. When a new strong reference is created, the count is increased. When a strong reference is released, the count is decreased.

Conceptually:

```
a = object()
    reference count becomes 1

b = a
    reference count becomes 2

a = None
    reference count becomes 1

b = None
    reference count becomes 0
    object can be deallocated
```

This is not exact visible Python code behavior in every optimized internal case, but it is the correct conceptual model for CPython object lifetime.

Reference count changes happen constantly. Assignment, function calls, returns, container insertion, container removal, attribute assignment, frame creation, frame destruction, and exception handling can all create or release references.

For example:

```
items = []
items.append(object())
```

The newly created object remains alive because the list now holds a reference to it. If the list is later cleared:

```
items.clear()
```

the list releases its reference to the object. If no other reference exists, the object can be destroyed.

Slot Reclamation Means Releasing Stored References

A container does not merely hold abstract values. It holds references to objects. When a container slot is overwritten or removed, CPython must release the old reference stored in that slot.

Example:

```
items = [object()]
items[0] = "new"
```

Before replacement:

```
items -> list object
      slot 0 -> old object
```

After replacement:

```
items -> list object
      slot 0 -> string object "new"
```

The list slot no longer refers to the old object. CPython must decrease the old object's reference count. If that slot was the last strong reference, the old object can be deallocated.

This is what automated slot reclamation means in practice. When a name, attribute, list position, dictionary entry, tuple during destruction, frame local slot, or internal temporary slot stops holding a reference, CPython releases that reference. The memory discipline is attached to reference ownership, not to source-level variable declarations.

Name Rebinding Also Releases References

The same idea applies to ordinary names:

```
x = []
x = 10
```

The second assignment does not mutate the list into an integer. It rebinds the name `x`. The old reference from `x` to the list is released. The new reference from `x` points to the integer object.

Conceptually:

before:

```
x -> list object
```

after:

```
x -> int object 10
```

\subsection{The Stack Subsystem and Reference Storage}

\subsubsection{Native Stack vs. Python Frame Stack vs. Bytecode Evaluation Stack}

The word `\emph{stack}` appears in several different places when Python execution is discussed,

The native stack belongs to CPython as a compiled C program. When the operating system starts

However, a Python function call is not simply the same thing as a C function call visible to

```
\begin{verbatim}
```

```
def f():
    return 10
```

```
def g():
    return f()
```

```
g()
```

there is a Python-level call from `g` to `f`. CPython must represent that call in its own execution machinery. The runtime needs to know which Python code object is currently executing, where execution is inside that bytecode, what local names exist, what global namespace is available, what built-ins can be searched, and where to continue after the call returns. This information is represented by Python frame machinery.

A Python frame is therefore the runtime record of one active Python code execution. It is similar in purpose to a native stack frame, but it is not the same thing as a raw C stack frame. A native stack frame exists because the CPU and C calling convention need a place to continue native execution. A Python frame exists because CPython needs a structured record for executing Python bytecode. It stores Python-level information: local bindings, the code object, namespace access, instruction position, temporary evaluation data, and exception/control-flow information.

This gives us two different call chains. One call chain is native and internal to the interpreter:

C function calls C function calls C function ...

The other call chain is Python-level and visible to the Python program:

```
module code -> g() -> f()
```

The second chain is the one shown in tracebacks. When Python prints an exception traceback, it is not merely dumping the raw native C stack. It is showing the Python frame chain: which Python code blocks were active, which function called which function, and where the exception propagated. This is possible because CPython keeps Python execution state as structured runtime information, not only as machine return addresses.

There is also a third stack: the bytecode evaluation stack. This one is even more specific. It exists inside the execution of a Python frame and is used for temporary values while bytecode instructions are evaluated. Python bytecode is stack-oriented: many instructions push object references onto an evaluation stack, pop object references from it, perform operations, and push results back.

For example, consider:

```
x = a + b * c
```

At source level, this is one assignment statement. At bytecode-execution level, CPython must load the object bound to `a`, load the object bound to `b`, load the object bound to `c`, multiply the last two, add the result to the first, and then bind the final result to `x`. During this process, intermediate object references must be stored somewhere. That temporary storage is the bytecode evaluation stack.

A simplified evaluation sequence looks like this:

```
load a      -> push object referenced by a
load b      -> push object referenced by b
load c      -> push object referenced by c
multiply    -> pop b and c, push multiplication result
add         -> pop a and multiplication result, push addition result
store x     -> pop final result and bind x to it
```

The important word here is *reference*. The evaluation stack does not normally contain raw C-style integer values or raw object contents. It contains references to Python objects. If the result of `b * c` is an integer object, the evaluation stack receives a reference to that integer object. If it is a NumPy array object, the stack receives a reference to that array object. If the operation raises an exception, the normal sequence is interrupted and exception machinery takes over.

So the three stacks can be separated like this. The native stack is for CPython's own C execution. The Python frame chain is for Python-level function/module/class/comprehension execution. The bytecode evaluation stack is for temporary operands and results inside one executing Python frame.

The distinction is easier to see with a function call:

```
def square(x):
    return x * x

def compute():
    y = square(5)
    return y + 1
```

When `compute()` runs, CPython has a Python frame for `compute`. Inside that frame, bytecode loads the function object `square`, loads the argument object `5`, and performs a call operation. That call creates or activates another Python frame for `square`. Inside the `square` frame, the bytecode evaluation stack temporarily holds references to the two uses of `x`, performs multiplication, and returns the resulting object reference to the caller frame. Then the `compute` frame continues and uses its own evaluation stack to add 1.

Conceptually:

```
Python frame chain:
  compute frame
    calls square frame
```

```

inside square frame:
    evaluation stack handles x * x

inside compute frame:
    evaluation stack handles returned_value + 1

```

Meanwhile, underneath all of this, CPython's C code is running on the native stack. The CPU is not executing Python bytecode directly. It is executing CPython's native interpreter machinery, and that machinery is manipulating Python frames and evaluation stacks.

This separation also explains why Python execution can be introspected. A C program does not normally expose ordinary call frames as first-class runtime objects in the same way. In Python, frame information can appear in tracebacks, debuggers, profilers, inspection tools, and exception reports. That is possible because Python-level execution is represented by managed runtime structures.

It also explains generators and coroutines later. A normal function frame runs and then disappears when the function returns. But a generator must pause and later resume. This means its execution state cannot be understood as a simple native stack frame that vanished after return. CPython must preserve enough Python frame state: the code object, the instruction position, local references, and evaluation state needed to continue later. The fact that Python execution is frame-managed makes this possible.

The practical model is therefore:

```

native C stack
    CPython's own machine-level execution

Python frame chain
    Python-level active calls and resumable execution records

bytecode evaluation stack
    temporary object references during expression execution

```

This is the correct basis for understanding local variables in Python. A local name is not a C stack variable holding a raw value. In an executing Python frame, local names correspond to reference slots or namespace entries that point to heap-allocated Python objects. The next subsection follows that idea directly: local names are frame-level reference storage, not boxes containing objects directly.

Local Names as Reference Slots: Frame-Level Storage Pointing to Heap Objects

Once the difference between the native stack, Python frames, and the bytecode evaluation stack is clear, Python local variables can be described more accurately. A local name inside a Python function is not a C-style variable box that contains the object directly. It is better understood as a reference slot inside the function's active frame. That slot points to a Python object, and that object normally lives in CPython-managed heap memory.

Consider the simple function:

```

def f():
    x = [1, 2, 3]
    y = x
    return y

```

A beginner may imagine that `x` contains the list and that `y` receives a copy of the list. That is not the Python model. When the list literal is evaluated, CPython creates a list object on the managed heap. The local name `x` is then bound to that object. When `y = x` executes, CPython does not copy the list. It binds another local slot, `y`, to the same heap object.

Conceptually, during the function call, the frame contains local reference slots:

```

frame for f
  local slot x ----+
                  |
                  v
                list object [1, 2, 3]
                  ^
                  |
  local slot y ----+

```

The list object is not inside `x`. The object exists separately. The local names are ways to reach it. This is the central idea:

A Python local variable is a name-bound reference to an object, not an object-containing storage box.

In C, a local declaration such as

```
int x = 10;
```

usually creates a typed storage location in the current stack frame. The compiler knows that `x` is an integer slot. Operations on `x` can often be compiled as direct reads and writes to that storage location or even optimized into registers.

In Python, a local assignment such as

```
x = 10
```

binds the local name `x` to an integer object. The integer object carries its own type. The local slot does not become an integer slot in the C sense. It is a place where CPython stores a reference to a Python object.

This is why the following is legal:

```

x = 10
x = "hello"
x = [1, 2, 3]

```

The local name `x` is first bound to an integer object, then rebound to a string object, then rebound to a list object. The slot did not change its declared storage type, because it did not have one in the C sense. It remained a reference slot. What changed was the object reference stored in that slot.

A simplified sequence is:

```

x = 10

local slot x -> int object 10

x = "hello"

local slot x -> str object "hello"

x = [1, 2, 3]

local slot x -> list object [1, 2, 3]

```

Each rebinding also has reference-count consequences. When `x` stops pointing to the previous object, CPython releases that reference. If no other reference keeps the old object alive, that object can be reclaimed. If another name, container, frame, or runtime structure still refers to it, the object remains alive.

For example:


```
def f():
    x = []
    y = x
    x = 10
    return y
```

After `y = x`, both local slots point to the same list. When `x = 10` executes, only the `x` slot is rebound. The list object does not disappear, because `y` still points to it.

Conceptually:

after `x = []` and `y = x`:

```

x ----+
      |
      v
    list object []
      ^
      |
y ----+
```

after `x = 10`:

```

x -> int object 10

y -> list object []
```

The list survives and is returned by the function. Its lifetime is controlled by references, not by the specific name that first received it.

This model also explains mutation. If the object is mutable, changing it through one name changes the object seen through all other references to the same object.

```
def f():
    x = []
    y = x
    y.append(1)
    return x
```

The call to `append` does not rebound `y`. It mutates the list object reached through `y`. Since `x` reaches the same object, `x` observes the change.

The frame still looks like this:

```

frame for f
  x ----+
      |
      v
    list object [1]
      ^
      |
  y ----+
```

There are two local names, but only one list object.

This distinction between rebinding and mutation is one of the most important consequences of Python's reference-slot model. Rebinding changes which object a name points to. Mutation changes the object itself.

Compare:

```

x = []
y = x

x = [1]

    with:

x = []
y = x

x.append(1)

```

In the first case, `x` is rebound to a new list object. `y` still points to the original empty list:

```

x -> list object [1]

y -> list object []

```

In the second case, the original list object is mutated:

```

x ----+
      |
      v
    list object [1]
      ^
      |
y ----+

```

The source code difference is small, but the object topology is different.

Inside CPython, ordinary function locals are usually stored more efficiently than a visible dictionary lookup on every access. The frame may use fast local slots indexed by the compiler's local-variable analysis. This is why local variable access is faster than repeated global dictionary lookup. But this optimization does not change the conceptual model. The slots still store references to Python objects. They are not raw C value cells.

A useful simplified picture is:

```

function code object
  knows local names:
    x, y, z

active frame
  local slot 0 for x -> object reference
  local slot 1 for y -> object reference
  local slot 2 for z -> object reference

```

The compiler determines that these names are local to the function. At runtime, the frame provides storage for their object references. The objects themselves live independently on the heap.

This also explains why local variables exist only while their frame exists, unless their objects are returned, stored elsewhere, or captured by closures.

Consider:

```

def make_list():
    data = [1, 2, 3]
    return data

result = make_list()

```

During the call:

```
frame for make_list
  data -> list object [1, 2, 3]
```

When the function returns, the frame's local slot `data` is destroyed or cleared. But the returned object is now referenced by `result` in the caller's namespace:

```
result -> list object [1, 2, 3]
```

The local name disappeared. The object survived. This would be impossible if the object were physically stored inside the local variable slot. The object survives because it is heap-allocated and reference-managed.

Now compare a function that does not return the object:

```
def make_list():
    data = [1, 2, 3]

make_list()
```

Here, when the frame ends, the local slot `data` releases its reference. If no other reference exists, the list object becomes reclaimable. The object lifetime follows the reference graph, not merely the textual scope.

Closures add one important variation. If a nested function needs a local variable from an enclosing function, CPython cannot store that variable only as an ordinary disappearing local slot. It must preserve the needed binding in a cell object.

Example:

```
def outer():
    x = 10

    def inner():
        return x

    return inner
```

When `outer` returns, its ordinary frame is gone. Yet `inner` can still read `x`. This is because the binding for `x` has been placed into closure storage, not lost with the ordinary frame.

Conceptually:

```
inner function object
  closure cell for x -> int object 10
```

Again, the object survives because a reference survives. In this case, the reference is held by a closure cell attached to the returned function object.

This frame-local reference-slot model also explains why function arguments behave as they do. When a function is called, argument passing binds local parameter names to the objects supplied by the caller.

Example:

```
def add_item(items):
    items.append("new")

data = []
add_item(data)
```

During the call:

```
caller namespace
  data -> list object []

callee frame
  items -> same list object []
```

The parameter `items` is a local reference slot in the callee frame. It points to the same list object as `data`. Therefore, mutating `items` mutates the object seen through `data`.

But if the function rebinds the parameter:

```
def replace(items):
    items = ["new"]

data = []
replace(data)
```

then only the local slot `items` is changed. The caller's `data` still points to the original list. Rebinding a local parameter does not rebind the caller's name.

During the call:

before rebinding:

```
data  ----+
      |
      v
      list object []

items ----+
```

after `items = ["new"]`:

```
data  -> original list object []

items -> new list object ["new"]
```

This is why the common phrase “Python passes by reference” is not precise enough. Python passes object references by assignment. The callee receives local names bound to the same objects. It can mutate shared mutable objects, but it cannot rebind the caller's names.

The practical rule is:

Function parameters are local reference slots initialized from the argument objects supplied by the caller.

This is also why immutable and mutable objects seem to behave differently in functions. The passing mechanism is the same. The difference is whether the object itself can be changed.

For an immutable object:

```
def f(x):
    x = x + 1

n = 10
f(n)
```

The expression `x + 1` creates or produces another integer object, and the local slot `x` is rebound to it. The caller's `n` is unchanged.

For a mutable object:

```
def f(x):
    x.append(1)

values = []
f(values)
```

The local slot `x` points to the same list object as `values`. The method call mutates that shared object. The caller observes the mutation.

The local-slot model explains both cases without inventing two different parameter-passing mechanisms.

The final picture is:

```
Python frame
  local slots / local namespace
    name -> object reference
    name -> object reference
    name -> object reference
```

```
CPython heap
  object
  object
  object
```

The frame stores references. The heap stores objects. Assignment changes references. Mutation changes objects. Function calls create new frames with new local reference slots. Function returns destroy or clear those slots. Objects persist only if some reference survives elsewhere.

This is the level at which Python variables become understandable. A Python local name is not a direct memory box. It is a reference-bearing entry in a frame. Once that is understood, assignment, aliasing, mutation, argument passing, closures, and object lifetime all become one coherent mechanism rather than a collection of special rules.

Lifecycle Transitions: Frame Destruction, Stack Unwinding, and Reference Count Updates

A Python frame is not permanent. It exists because some block of Python code is currently executing or must be resumable later. When that execution finishes, the frame's local storage is no longer needed in the ordinary way. This transition is one of the main places where Python's reference model becomes visible: ending a frame means releasing the references held by that frame.

Consider a simple function:

```
def f():
    data = [1, 2, 3]
    total = len(data)
    return total

result = f()
```

While `f` is running, CPython has a frame for `f`. That frame has local reference slots. One slot named `data` points to the list object. Another slot named `total` points to the integer object returned by `len(data)`.

Conceptually:

```

frame for f
    data -> list object [1, 2, 3]
    total -> int object 3

```

When the function returns, the frame is finished. Its local slots are cleared or released. Clearing a slot means that the frame stops holding a reference to the object stored there. If that object has no other references, its reference count can fall to zero and CPython can reclaim it.

In the example above, the list object `[1, 2, 3]` is not returned and is not stored anywhere else. When the frame releases `data`, the list may become unreachable and can be deallocated. The integer object returned as `total`, however, is transferred to the caller and then bound to `result`. That object survives because a new reference keeps it alive.

The important rule is:

A frame ending does not destroy every object created inside it. It releases the frame's references. Objects survive if some other reference still reaches them.

This distinction is essential. The end of a function destroys the local name environment, not necessarily the objects that were mentioned by those names.

For example:

```

def make_list():
    data = [1, 2, 3]
    return data

```

```

x = make_list()

```

During execution:

```

frame for make_list
    data -> list object [1, 2, 3]

```

After return:

```

caller namespace
    x -> same list object [1, 2, 3]

```

The local name `data` is gone, but the list object remains alive. The reference has moved from the callee's local frame to the caller's namespace. The object was not copied out of the function; a reference to the same heap object was returned.

Now compare:

```

def make_list():
    data = [1, 2, 3]

```

```

make_list()

```

Here, no reference escapes the function. When the frame ends, `data` is released. If nothing else refers to the list, the list becomes reclaimable. The object dies not because it was textually local, but because no surviving reference points to it.

This is the ordinary lifecycle path:

```

function call
    creates frame

assignment inside function
    local slots receive object references

```

```
function return
    return value reference is passed to caller
```

```
frame cleanup
    local slots release their references
```

```
reference counts update
    unreachable objects may be deallocated
```

The same idea applies when a function exits because of an exception. If an exception is raised and not handled inside the current function, CPython must leave that frame and propagate the exception to the caller. This process is called stack unwinding. In Python terms, stack unwinding means that active Python frames are exited one by one until a matching exception handler is found or the program reaches the top level.

Consider:

```
def inner():
    data = [1, 2, 3]
    raise ValueError("bad value")

def outer():
    inner()

outer()
```

The call chain is:

```
module frame
    outer frame
        inner frame
```

When `inner` raises the exception, CPython checks whether `inner` has a handler. If not, the `inner` frame is exited. Its local references, including `data`, are released. Then the exception is propagated to `outer`. If `outer` also has no handler, the `outer` frame is exited and its local references are released. The exception continues upward.

Conceptually:

```
exception raised in inner
    no handler in inner
        release inner frame references
        propagate to outer

    no handler in outer
        release outer frame references
        propagate to module/top level
```

This is not just control flow. It is also object-lifetime flow. Every unwound frame stops holding its local references. Those reference-count decreases may trigger object destruction.

If a handler exists, unwinding stops there:

```
def outer():
    try:
        inner()
    except ValueError:
        print("handled")
```

Now **inner** still exists, so its frame is cleaned up. But **outer** continues execution inside the exception handler. The **outer** frame remains alive because the exception was caught there.

The frame lifecycle is therefore tied to control flow. A frame may end normally through **return**, abnormally through an exception, or indirectly through generator/coroutine finalization. In each case, the runtime must deal with the references held by that frame.

A normal return is the simplest case:

```
def f():
    x = object()
    return 10
```

When **return 10** executes, the return value is prepared for the caller. Then the frame's local references are released. If **x** was the only reference to its object, that object can be destroyed.

An exception exit is similar, except the outgoing object is the exception state rather than an ordinary return value:

```
def f():
    x = object()
    raise RuntimeError()
```

The frame exits because control is leaving through an exception. The local **x** is released. The exception object is propagated.

There is also an important special case: frames may survive longer than a normal call if they are part of generators or coroutines. A generator does not finish when it executes **yield**. It suspends.

Example:

```
def gen():
    data = [1, 2, 3]
    yield data
    data.append(4)
    yield data
```

After the first **yield**, the generator frame is not destroyed. It is paused. Its local slot **data** still points to the list object because the generator may resume later.

Conceptually:

```
generator object
  suspended frame
    data -> list object [1, 2, 3]
    current instruction position -> after first yield
```

This is why generators can preserve local state between iterations. The local names are not recreated from nothing each time. The frame remains suspended, carrying references to the objects it needs.

Only when the generator finishes, is closed, or becomes unreachable can its frame release those references. This is a controlled delay of frame destruction.

A similar principle applies to coroutines. A coroutine may suspend while waiting. Its execution frame must remain alive so the coroutine can resume with its local variables and instruction position intact. Again, the runtime preserves references because execution has not truly ended.

So there are three broad frame states:

```
active frame
  currently executing

suspended frame
  paused, but may resume later

finished frame
  execution ended; references can be released
```


Only the finished frame follows the ordinary destruction path immediately.

Reference count updates are the memory-management side of all these transitions. When a local slot receives an object reference, the object must be kept alive. When the local slot is cleared, that reference is released. When a return value is passed to the caller, the caller receives a surviving reference. When an exception object is propagated, the exception path keeps the needed exception references alive. When a frame is suspended, its references remain alive.

This can be seen with a return example:

```
def f():
    a = []
    b = a
    return a
```

```
x = f()
```

Inside the frame:

```
a ----+
      |
      v
    list object []
      ^
      |
b ----+
```

At return, the return value is the object referenced by **a**. The frame then releases **a** and **b**. But the caller binds **x** to the returned list. The list survives.

After return:

```
x -> list object []
```

The local names are gone; the object remains.

Now consider:

```
def f():
    a = []
    b = a
```

```
f()
```

Inside the frame, the list has references from **a** and **b**. When the frame ends, both local slots are released. If there are no other references, the list becomes reclaimable.

The programmer should not imagine frame destruction as deleting boxes with values inside them. It is better to imagine the frame as a small table of outgoing arrows. When the frame ends, those arrows are removed. Objects with no remaining arrows pointing to them can be reclaimed.

Conceptually:

before frame cleanup:

```
frame
  a -> object A
  b -> object B
  c -> object C
```

after frame cleanup:

```
frame removed
  references a, b, c released
```

```
object A survives only if referenced elsewhere
object B survives only if referenced elsewhere
object C survives only if referenced elsewhere
```

This also explains why tracebacks can affect object lifetime. An exception traceback may keep references to frames, and frames may keep references to their local variables. Therefore, exception objects and traceback chains can sometimes keep objects alive longer than expected. This is not a contradiction of reference counting. It is reference counting doing exactly what it should: if something still references a frame, and the frame still references local objects, those objects are still reachable.

A simplified picture is:

```
exception object
  -> traceback
    -> frame
      -> local variables
        -> objects
```

This is one reason long-lived stored exceptions can accidentally retain memory.

The final lifecycle model is:

```
call creates frame
  frame owns local reference slots

execution fills slots
  assignments, arguments, temporaries bind objects

normal return
  result reference moves to caller
  frame releases remaining references

exception
  frames unwind until handler
  each exited frame releases its references
  traceback may preserve some frame references

generator/coroutine suspension
  frame is not destroyed
  local references remain alive

final frame destruction
  slots are cleared
  reference counts decrease
  objects with no remaining references are reclaimed
```

This gives a precise meaning to automatic memory management in Python. CPython does not search source code and decide that a variable is “out of scope” in a literary sense. It manages runtime structures. Frames hold references. Ending or unwinding a frame releases those references. Reference counts update. Objects survive if still reachable and disappear when the reference topology no longer keeps them alive.

Thus, stack unwinding and memory management are not separate stories. In CPython, control-flow transitions reshape the object graph. Function returns, exceptions, and suspended execution all decide which references remain alive, and those references decide which heap objects persist.

3.4.3 Introspection of Core Object Properties

Identity Verification: Object Identity Observation via the `id()` Function

Every Python object has identity, type, and value. The identity is the object's permanent runtime identity during its lifetime. The type determines what operations the object supports. The value is the object's current content or state, if the object has mutable state. Python exposes object identity through the built-in `id()` function and compares identity through the `is` operator. The language documentation states that `id()` returns an integer representing an object's identity, and that this identity is unique and constant for the object during its lifetime. In CPython specifically, this integer is the object's memory address. [3] [1]

The `id()` function is therefore not mainly a normal programming tool for application logic. It is an observation tool. It lets us see whether two names, container positions, attributes, or temporary expressions refer to the same object or to different objects.

Consider:

```
a = [1, 2, 3]
b = a
```

The names `a` and `b` now refer to the same list object. This can be observed with `id()`:

```
print(id(a))
print(id(b))
```

Both calls return the same integer, because both names point to the same object.

Conceptually:

```
a ----+
      |
      v
    list object [1, 2, 3]
      ^
      |
b ----+
```

The same identity tells us that there are two references but only one object.

Now compare:

```
a = [1, 2, 3]
b = [1, 2, 3]
```

```
print(id(a))
print(id(b))
```

The two lists have equal contents, but they are distinct objects. Their identities are different.

Conceptually:

```
a -> list object [1, 2, 3]
```

```
b -> list object [1, 2, 3]
```

This difference is the difference between equality and identity. Equality asks whether two objects should be considered equal in value. Identity asks whether two references point to the exact same object.

In Python syntax:

```
a == b      # value equality
a is b      # object identity
```

For lists:

```
a = [1, 2, 3]
b = [1, 2, 3]

print(a == b)  # True
print(a is b)  # False
```

The lists are equal because their elements compare equal. They are not identical because they are two different list objects.

The `id()` function makes this visible:

```
print(id(a) == id(b))  # False
```

However, this should normally be written as:

```
print(a is b)
```

The `is` operator is the proper direct identity comparison operator. The `id()` function is mostly useful for explanation, debugging, and introspection.

This distinction becomes especially important with mutable objects. If two names refer to the same mutable object, mutating through one name changes what the other name observes.

```
a = []
b = a

b.append(10)

print(a)
print(id(a) == id(b))
```

The output shows that `a` sees the appended value because `a` and `b` refer to the same list object. Conceptually:

before `append`:

```
a ----+
      |
      v
  list object []
      ^
      |
b ----+
```

after `append`:

```
a ----+
      |
      v
  list object [10]
      ^
      |
b ----+
```

No assignment to `a` was needed. The object itself changed. Both names still point to the same object identity.

For immutable objects, identity can be more confusing because the visible value cannot be changed. For example:

```
x = 10
y = 10
```

In CPython, `x` and `y` may point to the same cached integer object. But the programmer should not build general logic around that implementation detail. Small integers, some strings, and other immutable values may be reused or interned by CPython. Other equal immutable values may or may not share identity depending on implementation details, compiler optimizations, runtime state, and Python version.

Therefore, this is bad reasoning:

```
x = 1000
y = 1000

if x is y:
    print("same number")
```

For numbers and strings, value comparison should normally use `==`, not `is`. Identity comparison should be used when identity is the actual question.

The most common correct everyday use of `is` is with singleton objects such as `None`:

```
if value is None:
    print("no value")
```

This checks whether `value` refers to the specific singleton object `None`. It does not ask whether `value` is merely equal to `None`. In this case identity is exactly the intended test.

The `id()` function also helps clarify rebinding. Consider:

```
x = []
print(id(x))

x.append(1)
print(id(x))

x = [1]
print(id(x))
```

After `x.append(1)`, the identity of `x` remains the same because the same list object was mutated. After `x = [1]`, the identity changes because the name `x` has been rebound to a new list object.

The difference is:

```
mutation:
    same name -> same object, changed value

rebinding:
    same name -> different object
```

This is one of the most useful lessons of `id()`.

The CPython-specific detail that `id(obj)` is the object's memory address must be handled carefully. It is useful for understanding CPython's object model, but it is not portable Python reasoning. The language only promises that the returned integer is unique and constant for that object during its lifetime. It also permits two objects with non-overlapping lifetimes to have the same `id()` value. [1]

This means that the following can happen conceptually:

```
object A exists
    id(A) -> 12345
```

```
object A is destroyed
```

```
later, object B is created
    id(B) -> 12345
```

There is no contradiction. The uniqueness guarantee applies while objects are alive at the same time. Once an object is gone, its identity value may be reused.

For that reason, storing `id()` values as long-term object handles is a mistake. An `id()` value is not a safe persistent pointer, not a database identifier, not a stable address across program runs, and not a portable representation of the object.

The correct role of `id()` is narrower and more useful:

`id()` lets us observe whether a live object reference points to the same runtime object as another reference.

It is especially useful for explaining aliasing:

```
a = []
b = a
c = []

print(id(a))
print(id(b))
print(id(c))
```

The identities of `a` and `b` match. The identity of `c` differs. Therefore, `a` and `b` are aliases for the same object, while `c` is a separate object with equal-looking contents.

In compact form:

```
same id      -> same object
different id -> different live objects
==           -> value equality
is           -> identity equality
```

The practical conclusion is that `id()` is a window into Python's object-reference model. It proves that names do not contain objects directly. Names point to objects. Several names may point to the same object. One name may later be rebound to another object. Mutable objects may change while preserving identity. Equal objects may still be distinct objects.

Object lifetime, aliasing, mutation, reference counts, and frame-local slots all depend on object identity. CPython manages objects, not variable boxes. The `id()` function gives the programmer a limited but useful way to observe that object identity from Python code.

Dynamic Metadata Analysis: Runtime Type Descriptor Extraction via the `type()` Framework

If `id()` lets us observe object identity, `type()` lets us observe the runtime descriptor that tells Python what kind of object we are dealing with. The built-in `type()` function, when called with one argument, returns the type object of that argument. With three arguments, it can create a new type object dynamically, essentially acting as a dynamic form of the `class` statement. [2]

For the present discussion, the first form is the most important:

```
type(object)
```

This call does not return a string label. It returns an actual type object.

For example:

```
x = 10
print(type(x))
```

prints something like:

```
<class 'int'>
```

This means that `x` currently points to an object whose runtime type is `int`. The type belongs to the object, not to the name `x`. If `x` is rebound, `type(x)` changes because `x` now points to a different object.

```
x = 10
print(type(x))      # <class 'int'>

x = "hello"
print(type(x))      # <class 'str'>

x = [1, 2, 3]
print(type(x))      # <class 'list'>
```

Nothing mysterious happened to `x`. It did not change from an integer variable into a string variable and then into a list variable. The name `x` was rebound to different objects. Each object carried its own type descriptor.

This is the correct interpretation:

```
x -> int object
    type: int

x -> str object
    type: str

x -> list object
    type: list
```

The type is runtime metadata attached to the object. In CPython terms, this corresponds to the object's type pointer in its object header. At the Python level, `type()` exposes that information as a type object.

This is why Python is dynamically typed in a precise sense. It does not mean that Python values have no types. Python values absolutely have types. It means that names do not have fixed declared storage types. Names can be rebound to objects of different types because the type travels with the object, not with the variable name.

Compare this with C:

```
int x = 10;
x = "hello";  /* invalid */
```

The C variable `x` has a declared storage type. It is an `int` variable. A string pointer cannot be assigned to it as if the slot had changed its nature.

In Python:

```
x = 10
x = "hello"
```

the name `x` is a reference-bearing binding. It can point first to an integer object and later to a string object. The objects are typed; the name is dynamically rebound.

The `type()` function therefore helps expose one of Python’s deepest architectural rules:

Python names do not carry fixed type declarations; Python objects carry runtime type descriptors.

This runtime type descriptor is not only used for printing. It controls behavior. When Python evaluates an operation, the interpreter uses the involved objects and their types to decide what should happen.

For example:

```
a = 1
b = 2
print(a + b)
```

Here, `type(a)` and `type(b)` are both `int`, so integer addition is performed.

Now compare:

```
a = "py"
b = "thon"
print(a + b)
```

Here, `type(a)` and `type(b)` are both `str`, so the `+` operator performs string concatenation.

And:

```
a = [1, 2]
b = [3, 4]
print(a + b)
```

Here, the operands are lists, so `+` performs list concatenation.

The source operator is the same. The runtime behavior changes because the object types are different.

This is the actual mechanism behind many things that are often explained vaguely as “Python figures it out.” Python does not guess. CPython follows object metadata and type-defined behavior. The type object contains or points to the operation machinery for that kind of object.

The return value of `type()` is itself an object. This is where Python’s object model becomes recursive in a useful way.

```
print(type(10))          # <class 'int'>
print(type(int))         # <class 'type'>
print(type(type))        # <class 'type'>
```

The object `10` has type `int`. The object `int` is itself an object, and its type is usually `type`. The object `type` is also an object, and it is its own type in the ordinary built-in metaclass structure.

This is not a decorative curiosity. It explains why classes are runtime objects. When the programmer writes:

```
class Point:
    pass
```

the result is not merely a static type declaration. The name `Point` is bound to a class object. Calling:

```
type(Point)
```


usually returns:

```
<class 'type'>
```

That means `Point` is an object whose type is `type`. In ordinary terms, `Point` is a class, and classes are instances of the metaclass `type`.

This gives the class creation model:

```
Point object
  is a class object
  type(Point) -> type
```

And an instance model:

```
p = Point()

p object
  is an instance object
  type(p) -> Point
```

So `type()` lets us see both levels:

```
p = Point()

print(type(p))      # <class '__main__.Point'>
print(type(Point))  # <class 'type'>
```

This helps connect object instances and class objects. An instance points to its class as its type. The class is itself an object, usually created by `type`.

The three-argument form of `type()` shows this even more directly:

```
Point = type("Point", (), {"x": 0, "y": 0})
```

This dynamically creates a new class object named `Point`, with no explicit base classes beyond the default object base behavior, and with a namespace containing `x` and `y`. The official documentation describes this as essentially a dynamic form of the `class` statement. [2]

It is conceptually close to:

```
class Point:
    x = 0
    y = 0
```

This does not mean beginners should normally create classes with `type()` directly. The ordinary `class` statement is clearer. But the three-argument form proves an important architectural point: a class is an object constructed at runtime. The `class` statement is not only a compile-time declaration syntax.

The one-argument `type()` call should also be distinguished from `isinstance()`.

```
type(obj) is SomeClass
```

asks whether the object's immediate runtime type is exactly `SomeClass`.

By contrast:

```
isinstance(obj, SomeClass)
```

asks whether the object is an instance of `SomeClass` or of a subclass of it. This distinction matters in object-oriented code.

Example:

```
class Animal:
    pass

class Dog(Animal):
    pass

d = Dog()

print(type(d) is Animal)      # False
print(isinstance(d, Animal))  # True
```

The immediate type of `d` is `Dog`, not `Animal`. But `d` is still an instance of `Animal` in the inheritance sense. For ordinary program decisions, `isinstance()` is usually the better test. For introspection and teaching the object model, `type()` shows the direct runtime type descriptor.

This also matters for duck-typed Python style. In many Python programs, asking for the exact type is less useful than asking whether the object supports the needed operation. For example, instead of requiring exactly a list, a function may accept any iterable. In such cases, `type()` is too rigid.

For example:

```
def total(values):
    result = 0
    for value in values:
        result += value
    return result
```

This function does not need `values` to be exactly a list. It only needs an object that can be iterated. A tuple, generator, range, or custom iterable may work. Therefore, `type()` should not be overused as a gatekeeper.

The role of `type()` in this chapter is not to encourage type-check-heavy programming. Its role is to expose runtime metadata:

`type(obj)` reveals the type object that currently governs `obj`'s behavior.

It is a diagnostic and conceptual tool.

Using `type()` together with `id()` gives a compact way to observe Python's object model:

```
a = []
b = a
c = []

print(id(a), type(a))
print(id(b), type(b))
print(id(c), type(c))
```

The result shows that `a` and `b` have the same identity and the same type because they are two names for the same list object. `c` has the same type but a different identity because it is a different list object.

Conceptually:

```
a ----+
      |
```

```

      v
    list object
      type -> list
      ^
      |
b -----+

c -> another list object
    type -> list

```

This separates three different questions:

- identity: is it the same object?
- type: what runtime descriptor controls the object?
- value: what contents or state does the object currently have?

For example:

```

a = [1, 2]
b = [1, 2]

print(a is b)           # identity: False
print(type(a) is type(b)) # type: True
print(a == b)           # value equality: True

```

The two objects are different identities, have the same type, and compare equal by value. This gives the correct mental model:

```

object
  identity -> observed with id() / is
  type     -> observed with type()
  value    -> compared with ==, inspected through object behavior

```

The practical conclusion is that `type()` is a runtime metadata extractor. It exposes the type object attached to the object currently reached by a name or expression. It does not tell us the fixed type of a variable, because Python variables do not have fixed C-style types. It tells us what kind of object is currently there.

This is why `type()` belongs in the memory-management and object-topology discussion. CPython manages a graph of objects. Each object has identity, type, references, and type-specific storage. The `type()` function lets Python code observe one part of that graph: the runtime descriptor that determines how the object behaves.

The Variable-as-Label Paradigm Shift: Binding Multiple Names to an Identical Runtime Object (`a = b`)

One of the hardest transitions for a C programmer learning Python is abandoning the idea that a variable is a box containing a value. In Python, a variable name is better understood as a label attached to an object. Assignment does not usually copy an object. It binds a name to an object. A second assignment can bind another name to the same object.

Consider:

```

a = [1, 2, 3]
b = a

```

After the first line, CPython has a list object on the managed heap, and the name `a` refers to it. After the second line, no new list is created. The name `b` is bound to the same list object.

Conceptually:

```

a ----+
    |
    v
  list object [1, 2, 3]
    ^
    |
b ----+

```

This is the variable-as-label paradigm. The names `a` and `b` are two labels attached to one runtime object. They are not two boxes containing two independent lists.

This distinction becomes visible as soon as the object is mutated:

```

a = [1, 2, 3]
b = a

```

```

b.append(4)

```

```

print(a)

```

The output is:

```

[1, 2, 3, 4]

```

The mutation was performed through the name `b`, but the object reached through `a` is the same object. Therefore, `a` observes the changed list. Nothing was copied from `b` back into `a`. There was only one list object from the beginning.

The object topology before mutation is:

```

a ----+
    |
    v
  list object [1, 2, 3]
    ^
    |
b ----+

```

After mutation:

```

a ----+
    |
    v
  list object [1, 2, 3, 4]
    ^
    |
b ----+

```

The arrows did not change. The object changed.

This is different from rebinding:

```

a = [1, 2, 3]
b = a

```

```

b = [9, 9, 9]

```

Now **b** is not mutating the original list. It is being rebound to a new list object.
After rebinding:

```
a -> list object [1, 2, 3]
```

```
b -> list object [9, 9, 9]
```

The name **a** still points to the original list. The name **b** now points to another list. Rebinding changes the arrow. Mutation changes the object at the end of the arrow.

This gives the key distinction:

Assignment changes name-to-object bindings. Mutation changes the object itself.

The statement **a = b** should therefore be read as:

Bind the name **a** to the same object currently referenced by **b**.

It should not be read as:

Copy the full contents of **b** into a separate variable named **a**.

This is why identity testing reveals the truth:

```
a = [1, 2, 3]
b = a

print(a is b)
print(id(a) == id(b))
```

Both checks show that **a** and **b** refer to the same object. The names are different. The object is identical.

Now compare:

```
a = [1, 2, 3]
b = [1, 2, 3]

print(a == b)
print(a is b)
```

The first comparison is true because the lists have equal values. The second comparison is false because they are different objects.

Conceptually:

```
a -> list object [1, 2, 3]

b -> list object [1, 2, 3]
```

They look the same by value, but they are not the same by identity.

This distinction is also why copying must be explicit. If a separate list is needed, assignment is not enough:

```
a = [1, 2, 3]
b = a.copy()
```

Now **b** refers to a different list object with the same initial contents:

```
a -> list object [1, 2, 3]
```

```
b -> list object [1, 2, 3]
```

If `b.append(4)` is executed, `a` is unchanged because `a` and `b` no longer point to the same list.

However, even copying has levels. A shallow copy creates a new outer container, but it does not necessarily copy the objects inside it.

Example:

```
inner = [1, 2]
```

```
a = [inner]
```

```
b = a.copy()
```

```
b[0].append(3)
```

```
print(a)
```

```
print(b)
```

Both lists show the modified inner list, because the outer lists are different but their first element points to the same inner object.

Conceptually:

```
a -> outer list
      slot 0 ----+
                |
                v
            inner list [1, 2, 3]
                ^
                |
b -> outer list ----+
      slot 0
```

The shallow copy copied the references inside the outer list. It did not recursively duplicate the entire object graph.

This is why Python programmers must reason in terms of object topology. The question is not only “what value does this variable have?” The better question is:

Which object does this name currently refer to, and who else refers to the same object?

The same principle explains function arguments. When an object is passed to a function, the parameter name inside the function is bound to the same object supplied by the caller.

Example:

```
def add_item(items):
    items.append("x")
```

```
data = []
```

```
add_item(data)
```

```
print(data)
```

During the call:

```
caller namespace
  data -> list object []
```

```
callee frame
  items -> same list object []
```

The function mutates the object through the local name `items`. The caller sees the change through the name `data` because both names refer to the same object.

But if the function rebinds the parameter:

```
def replace(items):
    items = ["x"]

data = []
replace(data)

print(data)
```

the caller's list is unchanged. The assignment inside the function changes only the local binding `items`. It does not rebind the caller's name `data`.

During the call, before rebinding:

```
data  ----+
      |
      v
      list object []
      ^
      |
items ----+
```

After `items = ["x"]`:

```
data  -> original list object []

items -> new list object ["x"]
```

The parameter was local. Rebinding it changed the local frame, not the caller's namespace.

This also explains why mutable and immutable objects do not require two different argument-passing theories. Python always binds parameter names to objects. The visible difference comes from what operations are possible on the object.

With an immutable integer:

```
def f(x):
    x = x + 1

n = 10
f(n)
print(n)
```

The expression `x + 1` produces another integer object, and the local name `x` is rebound to it. The caller's name `n` still points to the original integer object.

With a mutable list:

```
def f(x):
    x.append(1)

values = []
f(values)
print(values)
```

The local name `x` points to the same list object as `values`. The method call mutates that shared object. The caller observes the mutation.

The passing mechanism is the same. The object behavior differs.

The variable-as-label model is also visible in chained assignment:

```
a = b = []
```

This creates one list object and binds both names to it:

```
a ----+
      |
      v
    list object []
      ^
      |
b ----+
```

Therefore:

```
a.append(1)
print(b)
```

prints:

```
[1]
```

This surprises only if one imagines assignment as copying into boxes. It is natural if assignment is understood as name binding.

The same danger appears in nested structures:

```
rows = [[0] * 3] * 4
```

This does not create four independent inner lists. It creates one inner list and repeats references to it:

```
rows -> outer list
      slot 0 -> inner list [0, 0, 0]
      slot 1 -> same inner list
      slot 2 -> same inner list
      slot 3 -> same inner list
```

So:

```
rows[0][0] = 1
```

appears to modify all rows, because all rows are the same inner list object.

The correct independent construction is:

```
rows = [[0] * 3 for _ in range(4)]
```

This evaluates the inner list expression separately for each row, producing separate inner list objects.

This example is important because it shows that aliasing is not an exotic topic. It is a direct consequence of Python's ordinary assignment and container rules.

The model also explains why object identity survives renaming:


```
original = {"a": 1}
alias = original

del original
```

The object is not destroyed merely because one name was deleted. The name `alias` still refers to the dictionary. Deleting a name removes one binding. It does not necessarily destroy the object.

Conceptually:

before del:

```
original ----+
              |
              v
          dict object {"a": 1}
              ^
              |
alias -----+
```

after del original:

```
alias -> dict object {"a": 1}
```

The object persists while at least one reference keeps it alive.

This connects directly to reference counting. Binding another name to the same object increases the number of active references. Rebinding or deleting a name releases a reference. The object is reclaimed only when no remaining references keep it alive, except for cyclic cases handled by the garbage collector.

A compact lifecycle example is:

```
a = []
b = a
del a
b.append(1)
print(b)
```

The list still exists after `del a` because `b` still points to it. The name `a` was removed. The object was not.

The practical rules are:

- `a = b` makes `a` refer to the same object as `b`;
- assignment does not deep-copy objects;
- rebinding changes a name-to-object relationship;
- mutation changes the object reached by one or more names;
- containers store references to objects;
- function parameters are local names bound to argument objects;
- objects persist while some reference path reaches them.

The final mental picture is:

names are labels
labels point to objects
objects live independently of any one label
several labels may point to the same object
assignment moves or creates labels
mutation changes objects
copying must be requested explicitly

This paradigm shift is necessary before Python memory behavior makes sense. Without it, assignment, function calls, list mutation, default argument traps, nested container aliasing, closures, and object lifetime all seem like separate special cases. With it, they become one mechanism: Python programs manipulate references to heap-managed runtime objects.

3.5 Comparative Syntax and Language Paradigm Divergence

After establishing Python's object model, frame mechanics, heap allocation, reference behavior, and CPython execution machinery, we can now return to the visible language surface. Syntax should not be treated as mere decoration. The way a language looks is often a compressed expression of how that language expects programs to be structured, executed, and understood.

This section compares Python's source-code shape with the C-style braced tradition. The purpose is not to rank the two syntaxes aesthetically, nor to repeat the shallow claim that Python is simply "cleaner." The difference is deeper. C exposes more of the compiled, typed, storage-oriented model through declarations, braces, semicolons, pointers, and explicit memory-oriented operations. Python removes much of that visible machinery, but only because CPython carries more responsibility at runtime through objects, namespaces, frames, bytecode, dynamic type descriptors, and managed memory.

The comparison therefore begins at the surface, with code shape, but moves quickly underneath it. Braces versus indentation is not only a formatting preference. It reflects a different treatment of block structure by the lexer and parser. Python's lighter syntax is tied to runtime object behavior, while C's heavier syntax is tied to native compilation and fixed storage expectations. Understanding this divergence prevents the reader from translating Python mechanically into C-shaped ideas and helps explain why Python code behaves the way it does.

3.5.1 Visual Comparison: Python Code Shape vs. C-Style Braced Code

Before comparing Python and C at the level of memory management, objects, frames, and runtime execution, it is useful to look at the visible shape of the two languages. Syntax is not only decoration. Syntax is the surface through which a language reveals its assumptions about structure, control, readability, and programmer discipline.

C-style languages usually mark program structure with explicit punctuation. Blocks are enclosed by braces, statements usually end with semicolons, and indentation is a visual convention chosen by the programmer. Python makes a different choice. It removes most of that structural punctuation and turns indentation into part of the grammar. A block is not merely shown by indentation; it is defined by indentation.

Consider a simple conditional in C:

```
if (x > 0) {  
    printf("positive\n");  
} else {  
    printf("not positive\n");  
}
```

The same structure in Python is:

```

if x > 0:
    print("positive")
else:
    print("not positive")

```

The visible difference is obvious. C uses parentheses around the condition, braces around the blocks, and semicolons after statements. Python uses a colon to introduce the block and indentation to define the block body. The Python version has less punctuation, but the indentation is no longer optional style. It is syntax.

In C, this is valid even if it is ugly:

```

if (x > 0) { printf("positive\n"); } else { printf("not positive\n"); }

```

The braces still define the structure. The compiler does not care much about the visual layout. In Python, the layout cannot be separated from the structure in the same way:

```

if x > 0:
print("positive")

```

This is not merely badly formatted Python. It is invalid Python, because the body of the `if` statement must be indented.

The same contrast appears in loops. In C:

```

for (int i = 0; i < 10; i++) {
    printf("%d\n", i);
}

```

In Python:

```

for i in range(10):
    print(i)

```

This example shows two differences at once. The first difference is visual: C uses braces and semicolons, Python uses indentation. The second difference is conceptual: the C loop describes initialization, condition checking, and incrementing directly. The Python loop asks an iterable object, here `range(10)`, to produce values one by one. The simpler surface syntax is connected to a deeper runtime protocol.

A C programmer may be tempted to read Python as merely “C without braces.” That is a mistake. Python does not only replace braces with indentation. It also moves many structural responsibilities into runtime objects and protocols. A Python `for` loop is not primarily a counter-loop. It is an iteration over an iterable object. A Python function is not only a source block. It becomes a runtime function object. A Python class is not only a static type declaration. It is executable code that creates a class object.

Still, the visual comparison is a good starting point because it shows the first layer of divergence. C makes structure explicit through punctuation. Python makes structure visible through layout.

The difference is especially clear in nested code. C:

```

if (user_active) {
    for (int i = 0; i < count; i++) {
        if (items[i].enabled) {
            process(items[i]);
        }
    }
}

```

Python:

```
if user_active:
    for item in items:
        if item.enabled:
            process(item)
```

The Python version is visually lighter. There are fewer punctuation marks competing for attention. The nesting is shown almost entirely by indentation. This can make code easier to scan, especially for ordinary application logic. The eye follows the left margin and sees the control structure.

But this same design has a cost. In C, braces provide visible block delimiters. A block has an explicit opening and closing marker:

```
{
    ...
}
```

In Python, the block ends when indentation returns to a previous level:

```
if condition:
    statement_1
    statement_2
statement_3
```

Here, `statement_3` is outside the `if` block because it is aligned with the `if`, not with the indented body. The end of the block is not marked by a closing token. It is marked by dedentation.

This means that Python code shape is more visually honest but also more whitespace-sensitive. If indentation is wrong, the program structure is wrong. In C, bad indentation can mislead the human reader while the braces still control the compiler. In Python, indentation controls both the reader and the parser.

A classic C danger is this:

```
if (x > 0)
    printf("positive\n");
    printf("done\n");
```

Visually, a beginner may think both `printf` statements belong to the `if`. They do not. Without braces, only the first statement is controlled by the `if`. The second statement always runs.

Python avoids that exact mismatch:

```
if x > 0:
    print("positive")
    print("done")
```

Both statements are inside the block because both are indented. If the second statement should always run, it must be dedented:

```
if x > 0:
    print("positive")
print("done")
```

This is Python's strongest argument for indentation-based syntax: the visual structure and grammatical structure are forced to coincide. The program is shaped the way it is read.

However, this does not make Python syntax automatically superior. It only means Python chooses a different enforcement point. C says: use explicit block tokens, and format the code responsibly. Python says: the formatting is the block structure. That is cleaner when code is well written, but stricter and sometimes irritating when code is copied, pasted, generated, or mixed with inconsistent tabs and spaces.

Function definitions show the same contrast. C:

```
int add(int a, int b) {
    return a + b;
}
```

Python:

```
def add(a, b):
    return a + b
```

The C version specifies the return type and parameter types directly in the function signature. The Python version does not. Its parameters are names that will be bound to objects when the function is called. The absence of type declarations is not only a visual simplification. It reflects Python's dynamic binding model. The function does not promise that `a` and `b` are machine integers. It promises only that, when the function runs, the operation `a + b` will be attempted on the objects supplied.

This is why the same Python function may work for several object types:

```
add(1, 2)
add("py", "thon")
add([1], [2])
```

The visible syntax is short because the runtime does the type-dependent work later.

In C, the visible code carries more compile-time information. In Python, the visible code carries less compile-time information and relies more on runtime object behavior. Therefore, the visual simplicity of Python is not free. It is paid for by dynamic dispatch, object metadata, runtime checks, and later error discovery.

A class definition also looks much lighter in Python. C does not have classes in the same sense, but a C-style data structure might look like this:

```
struct Point {
    int x;
    int y;
};
```

Python:

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y
```

The Python class body is executable code. The method definition creates a function object. The class statement creates a class object. Instance attributes such as `self.x` and `self.y` are usually stored as object references in an instance namespace. The C `struct`, by contrast, defines a fixed memory layout for fields of specified types.

Thus, the visual comparison again points toward a deeper architectural difference. Python looks less rigid because its runtime carries more structure dynamically. C looks more explicit because much of its structure must be known before execution.

A compact comparison is useful:

C-style code shape:

```
braces delimit blocks
semicolons terminate statements
declarations often expose storage/type expectations
indentation is human convention
compiler relies mainly on punctuation and declarations
```

Python code shape:

```
indentation delimits blocks
newlines usually terminate statements
names bind to runtime objects
indentation is grammar
interpreter/runtime relies on object metadata and namespaces
```

The student should therefore avoid two shallow conclusions.

The first shallow conclusion is that Python is just easier because it has less punctuation. That is incomplete. Python has less syntactic punctuation because it moves more responsibility into indentation, object protocols, dynamic lookup, and runtime checks.

The second shallow conclusion is that C is more serious because it has more visible machinery. That is also incomplete. C exposes more low-level structure because it is closer to native compilation and fixed memory layout. Python hides more low-level structure because CPython manages it inside the runtime.

The correct conclusion is that syntax reflects execution philosophy. C source code is shaped around explicit declarations, statement delimiters, braced blocks, and compiled native structure. Python source code is shaped around readable layout, name binding, runtime objects, and interpreter-managed execution.

This visual comparison prepares the next subsection. Once the surface shape is clear, we can compare the architectural reality underneath: Python's high-level managed code model versus C's bare-metal procedural coding model.

3.5.2 Architectural Case Study: High-Level Python Managed Code Realities vs. Bare-Metal Procedural C Coding

The visual difference between Python and C is only the surface. The deeper difference is architectural. C code is written closer to the native process model: typed storage, explicit memory layout, direct control flow, stack-local variables, heap allocation through explicit calls, and compilation toward machine instructions. Python code is written inside a managed runtime model: names bind to objects, objects live on a managed heap, function calls create Python frames, operations dispatch through runtime types, and source code is compiled into bytecode executed by CPython.

A small example is enough to expose the difference. Suppose we want to represent a point and move it.

In C, one natural version is:

```
#include <stdio.h>

struct Point {
    int x;
    int y;
};

void move(struct Point *p, int dx, int dy) {
    p->x += dx;
    p->y += dy;
}

int main(void) {
    struct Point p = {2, 3};

    move(&p, 10, 20);
```

```

    printf("%d %d\n", p.x, p.y);
    return 0;
}

```

The structure is explicit. `struct Point` defines a fixed memory layout containing two integer fields. The local variable `p` is a concrete object of that structure type, commonly stored in the stack frame of `main`. The call `move(&p, 10, 20)` passes the address of `p`, so `move` can modify the original storage. The function receives a pointer, dereferences it, and writes into the two fields.

The programmer can reason in storage terms:

```

main stack frame
  p:
    x: int
    y: int

move receives:
  p -> address of that struct

```

This is a bare-metal procedural style. Data layout is visible. Address passing is explicit. Mutation happens through a pointer. The compiler knows the field offsets, the integer types, and the function signature before execution.

A comparable Python version is:

```

class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

def move(p, dx, dy):
    p.x += dx
    p.y += dy

p = Point(2, 3)
move(p, 10, 20)

print(p.x, p.y)

```

The visible code is shorter and less mechanical. There are no explicit pointers, no address operator, no declared integer fields, no header include, no format string, and no manual entry-point function. But this does not mean the machine work disappeared. It moved into the runtime.

When Python executes:

```

p = Point(2, 3)

```

CPython resolves the name `Point`, calls the class object, allocates a new instance on the managed heap, calls `__init__`, binds `self.x` and `self.y` to integer objects, and then binds the name `p` to the new instance object.

Conceptually:

```

module namespace
  p -> Point instance object
    x -> int object 2
    y -> int object 3

```

The Python variable `p` does not contain the point. It refers to a heap object. The attributes `x` and `y` are not raw C `int` fields in the ordinary Python object case. They are attribute bindings pointing to Python integer objects.

When Python executes:

```
move(p, 10, 20)
```

the object referenced by the caller's name `p` is passed into the function call by binding the parameter name `p` inside the new function frame to the same object.

During the call:

```
module namespace
  p -> Point instance object

move frame
  p -> same Point instance object
  dx -> int object 10
  dy -> int object 20
```

This is not C pointer syntax, but it still creates shared access to the same object. The parameter `p` in `move` and the global name `p` both refer to the same heap object. Therefore, modifying attributes through the parameter changes the object observed later through the module-level name.

The line:

```
p.x += dx
```

also hides more work than the C line:

```
p->x += dx;
```

In C, the compiler knows the address of the struct, the offset of field `x`, and the integer operation required. In Python, CPython must resolve `p`, look up attribute `x`, resolve `dx`, perform Python-level addition according to the runtime types of the objects, and then assign the resulting object reference back to attribute `x`. Since Python integers are immutable, the operation does not modify the old integer object 2. It produces another integer object representing 12, and the attribute `x` is rebound to that result.

So after the first attribute update:

```
Point instance object
  x -> int object 12
  y -> int object 3
```

The instance object was mutated because its attribute binding changed. The integer object was not mutated.

This difference is central. In C:

```
p->x += dx;
```

usually means: find a memory slot inside a struct and modify the integer stored there.

In Python:

```
p.x += dx
```

means: read an attribute reference, compute a new object through Python addition, and store a new reference into the attribute.

The source expressions look similar, but the execution realities are different.

The same contrast appears with collections. In C, an array of integers is usually a contiguous region of raw integer storage:


```
int values[3] = {1, 2, 3};
```

Conceptually:

```
values:
    [ int ][ int ][ int ]
```

In Python:

```
values = [1, 2, 3]
```

the list is a heap object whose element storage contains references to Python objects:

```
values -> list object
        slot 0 -> int object 1
        slot 1 -> int object 2
        slot 2 -> int object 3
```

This is why Python lists can hold mixed objects:

```
values = [1, "two", print]
```

The list does not need one fixed element type because each element slot holds an object reference, and each object carries its own type. The cost is indirection and object overhead. The gain is flexibility.

This case study also explains error timing. In C, many type errors are rejected before execution:

```
int x = 10;
x = "hello";
```

This is invalid because `x` is an integer storage slot. The compiler can reject the assignment.

In Python:

```
x = 10
x = "hello"
```

there is no type error. The name `x` is simply rebound from one object to another. But an operation may fail later if the current object does not support the requested behavior:

```
x = "hello"
print(x + 5)
```

The name binding is valid. The addition fails at runtime because a string object and an integer object do not define that operation together.

Thus, C places more responsibility in the compile-time type and storage model. Python places more responsibility in runtime object behavior.

Function calls show the same divergence. In C:

```
int add(int a, int b) {
    return a + b;
}
```

the compiler knows that `a` and `b` are integers. The generated code can perform integer addition.

In Python:

```
def add(a, b):
    return a + b
```

the function accepts object references. The operation `a + b` is resolved when the function runs. The same function can work for several object types:

```
add(1, 2)
add("a", "b")
add([1], [2])
```

This is useful, but it also means that the runtime must inspect and dispatch operations dynamically. The Python function is more general at the source level because less is fixed before execution.

The architectural contrast can be summarized as follows:

C:

```
variables often correspond to typed storage
structs define fixed memory layouts
arrays store raw elements contiguously
pointers expose addresses explicitly
many operations compile to direct machine instructions
memory lifetime may be manual or scope-bound
```

Python:

```
names bind to objects
objects live on a managed heap
containers store references to objects
object types are runtime descriptors
operations dispatch through type behavior
memory lifetime follows reference topology
```

This does not mean Python is merely easier and C is merely harder. They expose different layers of the machine. C exposes storage and native execution more directly. Python exposes program structure, object relationships, and high-level operations more directly. C asks the programmer to manage more low-level detail. Python asks the runtime to manage it, and the programmer must understand the consequences.

The danger for a Python beginner coming from C is to keep using C-shaped explanations. A Python name is not a C variable. A Python list is not a C array. A Python object attribute is not necessarily a fixed struct field. A Python function call is not merely a native jump with typed parameters. A Python class is not merely a compile-time layout declaration.

The danger in the opposite direction is to believe that Python has no memory reality because it hides pointers and allocation. That is also false. Python objects still occupy memory. References still connect objects. Containers still store references. Frames still hold local slots. The runtime still allocates, releases, dispatches, and checks. The machinery is real; it is simply managed by CPython rather than written manually in the source program.

The correct conclusion is that Python is high-level because it relocates low-level responsibilities into the runtime. C is bare-metal in comparison because it leaves more of those responsibilities visible and controllable in the program text. A serious Python programmer does not need to write Python as if it were C, but must understand which parts of C's visible machinery have been moved underneath Python's surface.

3.5.3 Code Blocks: From Explicit Braces ({}) to Indentation Tokens and Lexer-Level Whitespace Rules

A code block is a group of statements treated as one structural unit. In C-style languages, this unit is usually marked by explicit tokens, especially braces. In Python, the same structural role is handled by indentation. This is not a formatting recommendation. It is a lexical and syntactic rule. Python's tokenizer observes changes in indentation and produces structural tokens that the parser uses to build the program.

In C, a block is visibly enclosed:

```
if (x > 0) {
    y = 1;
    z = 2;
}
```

The opening brace starts the block. The closing brace ends it. The indentation helps humans read the code, but the compiler follows the braces. This means that the following is still structurally the same program, even though it is badly formatted:

```
if (x > 0) { y = 1; z = 2; }
```

Python removes those block delimiters and gives indentation the structural job:

```
if x > 0:
    y = 1
    z = 2
```

Here, the colon announces that a block is expected. The indented lines form the block. The block ends when indentation returns to the previous level.

Conceptually:

```
if x > 0:
    y = 1      inside block
    z = 2      inside block
w = 3         outside block
```

The line `w = 3` is outside the `if` block because it is dedented. There is no closing brace. The return to the previous indentation level is the closing signal.

This is why Python whitespace is not merely whitespace. At the beginning of logical lines, indentation can become syntax.

A simplified tokenizer view is:

```
if x > 0:
    y = 1
    z = 2
w = 3
```

becomes conceptually:

```
IF NAME OP NUMBER COLON NEWLINE
INDENT
    NAME ASSIGN NUMBER NEWLINE
    NAME ASSIGN NUMBER NEWLINE
DEDENT
NAME ASSIGN NUMBER NEWLINE
```

The exact internal tokens are more detailed, but the important point is that `INDENT` and `DEDENT` are real structural markers for the parser. Python therefore still has block delimiters, but they are generated from indentation rather than written as braces.

This design has one clear advantage: the program's visual shape and grammatical shape are forced to match. In C, it is possible to write misleading indentation:

```
if (x > 0)
    y = 1;
    z = 2;
```

A beginner may visually read both assignments as controlled by the `if`. The compiler does not. Without braces, only `y = 1;` belongs to the `if`. The statement `z = 2;` always executes.

Python prevents this exact mismatch. If both statements are intended to be inside the block, both must be indented:

```
if x > 0:
    y = 1
    z = 2
```

If only the first statement belongs to the block, the second must be dedented:

```
if x > 0:
    y = 1
z = 2
```

The parser and the reader see the same structure.

But this advantage comes with a cost. Python makes layout semantically binding. A misplaced indentation level is not just ugly. It can change the meaning of the program or make it invalid.

For example:

```
if x > 0:
    y = 1
    z = 2
```

This indentation is invalid because `z = 2` is indented more deeply without a construct that opens a new nested block. Python does not allow arbitrary indentation for visual decoration. Indentation must correspond to syntactic structure.

Nested blocks work by repeated indentation:

```
if user_active:
    for item in items:
        if item.enabled:
            process(item)
```

The indentation levels show the nesting:

```
level 0: if user_active:
level 1:     for item in items:
level 2:         if item.enabled:
level 3:             process(item)
```

Each increase creates a deeper block. Each decrease closes one or more blocks. This is the indentation equivalent of nested braces:

```
if (user_active) {
    for (...) {
        if (...) {
            process(...);
        }
    }
}
```

The difference is that C writes the block stack explicitly with `{` and `}`, while Python derives it from leading whitespace.

This also explains why Python needs a placeholder statement such as `pass`. Since a block must contain at least one statement, an intentionally empty block cannot be represented by empty braces as in C.

In C:

```
if (x > 0) {  
}
```

In Python:

```
if x > 0:  
    pass
```

The `pass` statement means: this block intentionally does nothing. It exists because Python's block structure is statement-based, not brace-pair-based. A block cannot be only a visual indentation area with no statement inside it.

The same rule appears in class and function definitions:

```
def unfinished_function():  
    pass  
  
class Empty:  
    pass
```

The `pass` statement is not decoration. It is a syntactic placeholder that satisfies the grammar.

Python's indentation rules also explain why tabs and spaces matter. Since indentation is syntax, inconsistent indentation can break the program. Visually similar indentation may not be equivalent if tabs and spaces are mixed. Modern Python rejects inconsistent tab/space indentation in many cases because the parser must be able to determine exact block levels.

The practical rule is simple:

Use spaces consistently for indentation, normally four spaces per level.

This is partly style, but it is style aligned with syntax. In Python, indentation convention and parseable structure are joined.

Line endings also matter more than in C. In C, semicolons usually terminate statements:

```
x = 1; y = 2; z = 3;
```

Python normally uses the logical newline as the statement boundary:

```
x = 1  
y = 2  
z = 3
```

This again reduces punctuation, but it makes the physical shape of the source file more important. Python does allow multi-line expressions inside parentheses, brackets, and braces:

```
values = [  
    1,  
    2,  
    3,  
]
```

Inside such open delimiters, line breaks can continue the same logical statement. This is why long function calls and data structures are usually written with parentheses or brackets rather than backslash continuation.

For example:

```
result = compute_value(
    first_argument,
    second_argument,
    third_argument,
)
```

The indentation inside the parenthesized call is not a block indentation in the same sense as an `if` or `for` body. It is continuation layout inside an expression. This distinction matters: not every indentation in Python creates an `INDENT` block. Block indentation is produced after a logical line that opens a suite, such as an `if`, `for`, `while`, `def`, `class`, `try`, or similar construct.

So this:

```
if x > 0:
    print(x)
```

uses indentation to define a block.

But this:

```
print(
    x
)
```

uses indentation only as visual continuation inside parentheses.

This is one reason Python syntax cannot be understood merely as “indentation means nesting.” More precisely:

Indentation at the beginning of logical lines creates block structure only where the grammar expects a suite.

The colon is the visible cue for many such suites:

```
if condition:
    block

while condition:
    block

for item in collection:
    block

def function():
    block

class Name:
    block

try:
    block
except Error:
    block
```

The colon does not itself create the block. It marks the end of the header line and signals that a suite follows. The indentation of the following logical lines then defines the suite.

This design creates a different reading discipline from C. In C, one can often recover block structure by matching braces. In Python, one reads the left margin. The shape of the program is the nesting structure.

That is why Python code tends to have a distinctive vertical form:

```

header:
    body
    nested_header:
        nested_body
after_block

```

The block is not closed by a token. It is closed by returning left.

This has consequences for generated code, copied code, and teaching. Generated Python code must manage indentation precisely. Copying from web pages, PDFs, or formatted documents can introduce broken whitespace. Beginners must learn that moving a line left or right changes the program, not only its appearance.

At the same time, this rule can make Python code easier to audit. There is no hidden disagreement between braces and indentation because there are no braces for ordinary blocks. The structure the eye sees is the structure the parser receives.

The comparison can be summarized as follows:

```

C-style block model:
    block starts with {
    block ends with }
    indentation is conventional
    semicolons terminate statements
    layout can lie to the reader

```

```

Python block model:
    block header usually ends with :
    block starts with increased indentation
    block ends with dedentation
    indentation is lexical structure
    layout cannot be separated from meaning

```

The philosophical consequence is that Python treats code layout as part of the language contract. This is one of its most controversial design choices. It creates clean visual structure, but it also makes whitespace carry semantic weight. It removes braces, but it does not remove block delimiters; it converts them into indentation tokens produced by the lexer.

For a programmer coming from C, the correct adjustment is not to think “Python has no block markers.” Python has block markers, but they are implicit in indentation. The lexer sees changes in leading whitespace and turns them into `INDENT` and `DEDENT`. The parser then uses those tokens just as seriously as a C parser uses braces.

Thus, Python’s block syntax is not informal. It is strict. It only looks lighter because the structural tokens are generated from layout rather than typed as punctuation.

3.6 Data Type Architecture and Runtime Type Systems

After examining CPython’s execution model, memory behavior, object identity, reference topology, and syntax-level divergence from C, we can now describe Python’s data types more systematically. Data types in Python are not merely labels attached to values, nor are they equivalent to C declarations. They are runtime object categories: each object has identity, type, value, storage behavior, mutability rules, and a defined interface through which operations are accepted or rejected.

This section therefore treats Python’s type system as part of the runtime architecture, not as a list of built-in names to memorize. The central distinction is that Python names are dynamic, while Python objects are strongly typed at runtime. A name can be rebound from an integer to a string to a list, but every operation is checked against the object that is actually present when execution reaches that operation.

The following subsections build this model step by step: first by separating static and dynamic typing from strong and weak enforcement, then by explaining Python's runtime type boundary, then by returning to names and object references, storage categories, mutability, identity, equality, and finally the object interfaces that make operators, construction, representation, indexing, calling, and iteration work. The goal is to replace the C-shaped idea of typed boxes with the Python-shaped idea of typed runtime objects connected by references.

3.6.1 Classifying Type Systems: Static vs. Dynamic Variable Typing, Strong vs. Weak Enforcement Boundaries

Before Python's concrete data types can be studied, the word *type* must be separated into several different ideas. Programmers often say that a language is “typed”, “untyped”, “strongly typed”, “weakly typed”, “statically typed”, or “dynamically typed” as if these labels were simple and self-explanatory. They are not. They describe different questions.

The first question is:

When is type information checked?

This separates static typing from dynamic typing.

The second question is:

How strictly are operations prevented when the involved values do not support them?

This separates stronger enforcement from weaker enforcement.

These two axes are independent. A language can be statically typed but still allow unsafe conversions. A language can be dynamically typed but still reject invalid operations strictly at runtime. Python belongs to the second category: names are dynamically bound to objects, but operations are strongly checked when they execute.

Static Typing: Type Restrictions Before Execution

In a statically typed language, many type constraints are checked before the program runs. C is the reference example for this course. When the programmer writes:

```
int x = 10;
```

the name `x` is associated with an `int` storage type. The compiler uses that declaration to decide what kind of values may be stored there, what operations are valid, how much memory is needed, and what machine instructions can be generated.

If the programmer writes:

```
int x = 10;  
x = "hello";
```

a C compiler can reject the program because `x` is an integer variable and `"hello"` is not an integer value. The failure is detected before normal execution. This is static type checking.

Static typing therefore ties type information to program structure before runtime. The compiler can use this information to detect errors early and to generate efficient code. If the compiler knows that two operands are machine integers, it can often generate direct integer arithmetic instructions. It does not need to inspect runtime object descriptors before every addition.

The advantage is early certainty. The disadvantage is reduced flexibility. A storage location declared as an `int` cannot later become a string, list, function, or dictionary. The program must respect the declared type structure.

Dynamic Typing: Type Information at Runtime Objects

In a dynamically typed language, type decisions are attached mainly to runtime values, not to fixed variable declarations. Python is dynamically typed in this sense.

When Python executes:

```
x = 10
```

the name `x` is bound to an integer object. Later:

```
x = "hello"
```

the same name `x` is rebound to a string object. This is legal because `x` is not a fixed typed storage box. It is a name that can be bound to different objects at different moments.

The type belongs to the object:

```
x -> int object
```

```
x -> str object
```

```
x -> list object
```

not to the name as a permanent compile-time declaration.

This is why `type(x)` may return different results at different points in the same program:

```
x = 10
print(type(x))      # int
```

```
x = "hello"
print(type(x))      # str
```

The name `x` did not change its declared type. It has no declared type in the C sense. It simply points to different typed objects over time.

Dynamic typing therefore gives the programmer more flexibility. The same function may accept different kinds of objects as long as those objects support the operations used inside the function.

For example:

```
def combine(a, b):
    return a + b
```

This may work for integers:

```
combine(1, 2)
```

for strings:

```
combine("py", "thon")
```

and for lists:

```
combine([1], [2])
```

The function does not declare a fixed parameter type. The operation is checked when the function runs with actual objects.

Static vs. Dynamic Does Not Mean Safe vs. Unsafe

A common mistake is to treat static typing as safe and dynamic typing as unsafe. This is too simple. Static and dynamic typing describe when type constraints are checked, not whether all errors are impossible.

C is statically typed, but it still allows many dangerous operations: pointer misuse, invalid casts, buffer overflows, use-after-free errors, and undefined behavior. The compiler checks many type rules, but it does not make arbitrary memory manipulation safe.

Python is dynamically typed, but it still has strict runtime type enforcement. It does not silently reinterpret most incompatible objects just to continue execution.

For example:

```
"hello" + 5
```

raises an error. Python does not silently convert 5 to "5" and concatenate it. The operation is rejected because the involved object types do not define that operation together.

So dynamic typing means:

The check happens when the operation executes.

It does not mean:

There is no check.

Strong Enforcement: Operations Must Make Sense for the Objects

The phrase *strong typing* is used inconsistently in programming discussions, but the useful idea for this course is enforcement strength. A strongly enforced type system does not freely treat values of unrelated types as interchangeable. If an operation does not make sense for the objects involved, it is rejected.

Python is strongly enforced at runtime. Examples:

```
1 + 2
```

is valid integer addition.

```
"1" + "2"
```

is valid string concatenation.

But:

```
"1" + 2
```

is rejected.

The string object and the integer object both exist. Python knows their types. It refuses to invent a hidden conversion rule for this operation.

This is different from languages that perform many implicit conversions. In some languages, adding a string and a number may silently convert the number into text or convert the string into a number. That kind of behavior is weaker enforcement because the language crosses type boundaries automatically.

Python usually requires such conversions to be explicit:

```
"1" + str(2)
```

or:

```
int("1") + 2
```

The programmer must state which domain is intended: text concatenation or numeric addition.

Weak Enforcement: Automatic Boundary Crossing

Weakly enforced systems allow more automatic conversion between types. This can make small programs shorter, but it can also hide mistakes.

For example, in a weakly converting language, an expression similar to:

```
"10" + 5
```

might become "105" or 15, depending on the language's rules and context. The program continues, but the programmer may not have intended the conversion.

Python rejects this ambiguity. It forces the programmer to choose:

```
"10" + str(5)      # text result: "105"
```

```
int("10") + 5      # numeric result: 15
```

This is why Python can be dynamically typed and still strict. It permits names to refer to different kinds of objects over time, but it does not usually permit incompatible objects to be combined without a defined operation or explicit conversion.

The Four-Part Classification

The two axes can be combined:

static typing:

many type rules checked before execution

dynamic typing:

type rules checked when objects are used at runtime

strong enforcement:

invalid type combinations are rejected

weak enforcement:

the language performs many implicit conversions

Python's position is:

dynamic names

strong runtime enforcement

C's position is more complicated. It is statically typed, but it also permits casts, pointer conversions, integer promotions, and low-level memory operations that can weaken safety. Therefore, C should not be described simplistically as "stronger" than Python. It is more statically declared, but it also gives the programmer escape routes that can bypass safety.

A better comparison is:

C:

variable declarations carry compile-time type information

compiler checks many operations before execution

raw memory and casts can bypass or weaken safety

Python:

names do not carry fixed declared types

objects carry runtime type descriptors

operations are checked when executed

incompatible operations usually raise exceptions

Why Python Feels Flexible

Python feels flexible because a function can be written around behavior rather than around declared storage types.

Example:

```
def first_item(collection):  
    return collection[0]
```

This function may work for a list, tuple, string, or another object that supports indexing. The function does not need to declare all possible accepted types. It simply uses an operation, and the runtime checks whether the object supports it.

This is often called duck typing: if an object supports the needed behavior, the function can use it. The name is informal, but the mechanism is concrete. The runtime attempts the operation through the object's type-defined interface. If the object supports it, execution continues. If not, an exception is raised.

For example:

```
first_item([10, 20])  
first_item((10, 20))  
first_item("abc")
```

all work.

But:

```
first_item(123)
```

fails because an integer object is not indexable.

Thus, Python flexibility is not the absence of type rules. It is operation-based runtime checking.

Why Python Errors Often Appear Later

The cost of dynamic typing is that some errors are discovered only when execution reaches the invalid operation.

For example:

```
def add_one(x):  
    return x + 1
```

This function can be defined without knowing what `x` will be. The definition itself is valid. The error appears only if the function is called with an object that cannot be added to 1:

```
add_one("hello")
```

The failure occurs at runtime. The compiler could not reject the function definition, because there may be valid calls:

```
add_one(10)
```

This is one of the tradeoffs of Python. The same flexibility that allows generic behavior also delays certain errors until actual execution paths are tested.

This is why testing, clear interfaces, type hints, assertions, and defensive validation become important in larger Python programs. Dynamic typing gives freedom, but freedom must be disciplined.

Type Hints Do Not Turn Python into C

Modern Python supports type hints:

```
def add_one(x: int) -> int:
    return x + 1
```

These annotations help readers, editors, linters, type checkers, and documentation tools. But ordinary CPython does not use them as C-style enforced declarations during normal execution.

For example, the annotation says that `x` is intended to be an integer, but CPython still permits a call at runtime unless additional checking is performed by external tools or explicit code. If an invalid object is passed, the error usually appears when the operation fails, not because CPython rejected the call based only on the annotation.

So type hints are not the same as C variable declarations. They are metadata for tools and humans unless a specific runtime validation system uses them.

This distinction will matter later. Python's type-hint system improves maintainability, but it does not erase Python's dynamic runtime object model.

The Practical Classification for Python

The correct classification for Python is therefore:

Python is dynamically typed at the level of names and strongly checked at the level of runtime operations.

Names are dynamic because they can be rebound to objects of different types. Operations are strongly checked because Python refuses invalid type combinations unless an explicit conversion or type-defined behavior exists.

This prepares the next subsection. We can now describe Python's type boundary more precisely: Python allows dynamic name binding, but once an operation is attempted, the runtime verifies whether the involved objects actually support it. That boundary is where Python's flexibility meets Python's strictness.

3.6.2 Python's Type Boundary: Dynamic Names with Strong Runtime Type Constraint Verification

Python's type system is often misunderstood because two different facts are mixed together. The first fact is that Python names are dynamic: the same name can be bound to different kinds of objects at different moments. The second fact is that Python operations are not loose: when an operation is executed, the runtime verifies whether the actual objects involved support that operation. Python is flexible at the binding boundary, but strict at the operation boundary.

This is the practical meaning of Python's type boundary:

Names may point to any object, but operations must be valid for the objects that are actually there.

Consider:

```
x = 10
x = "hello"
x = [1, 2, 3]
```

All three assignments are valid. The name `x` is first bound to an integer object, then to a string object, then to a list object. Python does not require a declaration saying that `x` must always be an integer, string, or list.

But this freedom does not mean that every operation is allowed:

```
x = "hello"
result = x + 5
```

This fails at runtime. The assignment is legal. The operation is not. Python reaches the expression `x + 5`, observes that `x` refers to a string object and `5` is an integer object, and refuses the operation because that combination does not define a valid addition.

So the type boundary is not at assignment. It is at use.

Assignment Does Not Prove Future Operation Validity

In C, a declaration restricts a variable early:

```
int x = 10;
```

The compiler can use the declared type of `x` to reject incompatible assignments and generate integer-specific operations. In Python, assignment only creates or changes a binding:

```
x = 10
```

This means:

```
name x -> int object 10
```

It does not mean:

```
x is now permanently an integer storage box
```

Therefore, a later operation must be checked against the object that `x` currently references.

For example:

```
def add_one(x):
    return x + 1
```

This function can be defined successfully. At definition time, Python does not know which objects will later be passed as `x`. The function is not invalid by itself. It becomes invalid only for particular calls:

```
add_one(10)          # valid
add_one(3.5)         # valid
add_one("hello")     # invalid
```

The boundary is reached when `x + 1` is executed. At that moment, the runtime checks the actual object bound to `x`.

Strong Runtime Verification

Python's runtime verification is strict. It does not usually guess the programmer's intention by performing broad implicit conversions.

For example:

```
"10" + 5
```

could theoretically mean text concatenation:

```
"10" + "5"          # "105"
```

or numeric addition:

```
10 + 5          # 15
```

Python refuses to choose silently. The programmer must state the intended conversion:

```
"10" + str(5)
```

or:

```
int("10") + 5
```

This is why Python is dynamically typed but not weakly permissive. The runtime lets names be flexible, but it requires operations to make sense according to object behavior.

The same principle applies to many operations:

```
len([1, 2, 3])    # valid
len("abc")        # valid
len(123)          # invalid
```

The name passed into `len()` may refer to any object. But the object must support the length protocol. If it does not, Python raises an error.

The Object Determines the Operation

Python operations are object-driven. The source syntax provides the requested operation; the runtime objects decide whether and how that operation can be performed.

For example:

```
a + b
```

does not specify one fixed operation. It requests addition-like behavior from the involved objects.

If `a` and `b` are integers, the result is numeric addition:

```
1 + 2
```

If they are strings, the result is concatenation:

```
"py" + "thon"
```

If they are lists, the result is list concatenation:

```
[1, 2] + [3, 4]
```

If the objects do not define a compatible operation, execution fails:

```
[1, 2] + 3
```

This is not arbitrary. CPython follows the type-defined operation machinery of the objects involved. The operator is resolved through the runtime object model.

Thus, Python's type boundary is not a declaration boundary:

```
name x must always be int
```

It is an operation boundary:

```
the object currently reached by x must support this operation
```

Dynamic Does Not Mean Unstructured

Because Python does not require many declarations, beginners sometimes think Python is structurally loose. That is wrong. Python has strong runtime structure. Every object has a type. Every type defines behavior. Every operation is checked against those behaviors.

The following code is valid:

```
x = 10
x = "hello"
```

because rebinding a name is allowed.

The following code is invalid:

```
x = "hello"
x.append("!")
```

because the object currently bound to `x` is a string, and string objects do not have an `append` method.

If `x` were a list, the method call would be valid:

```
x = []
x.append("!")
```

So Python does not ask, “What was `x` declared to be?” It asks, “What object does `x` currently refer to, and does that object support the requested operation?”

Behavior-Based Programming

This runtime boundary supports behavior-based programming. A function can be written for objects that provide the needed behavior, rather than for one exact declared type.

Example:

```
def show_first(items):
    print(items[0])
```

This function works for several object types:

```
show_first([10, 20, 30])
show_first((10, 20, 30))
show_first("abc")
```

The function requires index access at position 0. It does not require the object to be exactly a list. But the boundary remains strict:

```
show_first(123)
```

This fails because an integer object does not support indexing. Python permits the call to be attempted, but rejects the invalid operation when it is reached.

This is Python’s flexibility in its most useful form. A function can accept any object that behaves correctly for the operations it uses. But if the behavior is missing, the runtime does not pretend otherwise.

Runtime Type Errors Are Execution-Path Dependent

Because type verification often happens at use time, some errors appear only on execution paths that reach the invalid operation.

Example:

```
def process(value, use_math):
    if use_math:
        return value + 1
    return str(value)

process("hello", False)    # valid
process("hello", True)     # invalid
```

The same function and the same string object can be involved in one valid call and one invalid call. The invalid operation is executed only when `use_math` is true.

This is why testing matters in Python. Merely importing or defining a function does not prove that every possible operation inside it is valid for every possible input. Runtime type constraints are verified along the paths that actually run.

This also explains why Python programs can fail late. A program may run for a long time before it reaches a branch where an object is used incorrectly. That is a cost of dynamic typing.

Type Hints Move Some Checking Earlier, but Not Into the Core Runtime

Type hints can document intended types and allow external tools to detect many problems before execution:

```
def add_one(x: int) -> int:
    return x + 1
```

A static checker may warn if the function is called with a string:

```
add_one("hello")
```

But ordinary CPython execution does not automatically enforce the annotation as a C-like parameter declaration. The function still receives object references. If a bad object is passed, the ordinary runtime error appears when the operation fails, unless additional runtime validation code or a validation framework is used.

So annotations are useful, but they do not change the basic runtime boundary:

```
normal Python execution:
    bind names dynamically
    check operations when executed
```

Explicit Conversion as Boundary Crossing

Python allows type boundary crossing, but it usually asks the programmer to say so.

Examples:

```
int("123")
float("3.14")
str(42)
list("abc")
tuple([1, 2, 3])
```

These are explicit conversions or constructions. The programmer states the target domain. This is different from silently converting values inside arbitrary operations.

For example:

```
age = 20
message = "Age: " + str(age)
```

The call to `str(age)` makes the conversion visible. Python does not silently treat every integer as a string whenever text is nearby.

This explicitness is important for reliability. It makes the boundary crossing readable in the source code.

The Practical Boundary

The practical rule is:

Python accepts dynamic bindings freely, but every operation is checked against the runtime object actually used.

This can be summarized as:

```
x = any_object
    allowed

x.some_operation()
    allowed only if current object supports it

x + y
    allowed only if the current object types define compatible addition

x[0]
    allowed only if current object supports indexing

len(x)
    allowed only if current object supports length
```

The programmer should therefore reason in two stages.

First:

What object does this name currently refer to?

Second:

Does that object support the operation being requested here?

This is the correct mental model for Python's type boundary. Names are dynamic. Objects are typed. Operations are verified at runtime. Invalid combinations fail with exceptions instead of being silently forced through unrelated conversions.

That boundary is one of Python's defining compromises: it gives the freedom of dynamic naming while preserving strict operational meaning.

3.6.3 Names, Variables, and Object References: Why Python Variables Are Not C Boxes

The word *variable* is dangerous when moving from C to Python, because it encourages the wrong image. In C, a variable is often naturally imagined as a box: a named storage location with a declared type, placed somewhere in memory, capable of holding a value of that type. This image is not perfect for all C cases, because compilers optimize aggressively, but it is close enough to the ordinary source-level model. An `int x` behaves like a storage place intended to contain an integer value.

Python does not work this way. A Python name is not a typed box containing an object. A Python name is a binding to an object. The object exists independently in the runtime, usually on CPython's managed heap, and the name is one way to reach it.

This difference is small in syntax but large in meaning.

In C:

```
int x = 10;
```

the declaration says that `x` is an integer variable. The storage associated with `x` is meant to hold an integer value. The compiler uses that information before the program runs.

In Python:

```
x = 10
```

there is no declaration of an integer box. CPython has an integer object representing 10, and the name `x` is bound to that object.

Conceptually:

```
x -> int object 10
```

The arrow is the important part. The name points to an object. The name does not contain the object.

This explains why the following is legal:

```
x = 10
x = "hello"
x = [1, 2, 3]
```

The name `x` is not changing its storage type. It is being rebound. First it points to an integer object, then to a string object, then to a list object.

Conceptually:

```
after x = 10:
```

```
    x -> int object 10
```

```
after x = "hello":
```

```
    x -> str object "hello"
```

```
after x = [1, 2, 3]:
```

```
    x -> list object [1, 2, 3]
```

The objects carry their types. The name does not have a permanent C-style type.

This is why saying “`x` is an integer” is acceptable in casual speech but imprecise. More accurately:

The name `x` currently refers to an integer object.

That wording matters because a later assignment may make `x` refer to a different object.

The box image also fails for assignment between names. Consider:

```
a = [1, 2, 3]
b = a
```

If variables were boxes containing values, one might imagine that the list inside **a** is copied into **b**. That is not what happens. The name **b** is bound to the same object currently referenced by **a**.

Conceptually:

```

a ----+
    |
    v
  list object [1, 2, 3]
    ^
    |
b ----+

```

There is one list object and two names pointing to it.

This is why mutation through one name is visible through the other:

```
b.append(4)
```

```
print(a)
```

The result is:

```
[1, 2, 3, 4]
```

The list object was mutated. Both names still point to that same object.

This is not the same as rebinding:

```
a = [1, 2, 3]
```

```
b = a
```

```
b = [9, 9, 9]
```

After the last line, **b** no longer points to the original list. It points to a new list object.

Conceptually:

```
a -> list object [1, 2, 3]
```

```
b -> list object [9, 9, 9]
```

The original list was not changed. Only the binding of **b** changed.

This gives one of the core rules of Python:

Assignment rebinds names. It does not mutate objects unless the assignment target is an object component such as an attribute, item, or slice.

For example:

```
x = [1, 2, 3]
```

```
x = [4, 5, 6]
```

rebounds the name **x**.

But:

```
x = [1, 2, 3]
```

```
x[0] = 99
```

mutates the list object that **x** refers to.

Conceptually:

rebinding:

```
x -> old list
x -> new list
```

item assignment:

```
x -> same list object, but its internal slot changes
```

The same model explains function arguments. A function call does not copy objects into parameter boxes. It creates a new frame with local names bound to the argument objects.

Example:

```
def change(items):
    items.append(10)
```

```
values = []
change(values)
```

During the function call:

```
caller namespace
  values -> list object []
```

```
function frame for change
  items -> same list object []
```

The function mutates the list object. The caller sees the change because both names point to the same object.

But if the function rebinds the parameter:

```
def change(items):
    items = [10]
```

```
values = []
change(values)
```

then only the local name `items` changes.

During the call, before rebinding:

```
values ----+
          |
          v
        list object []
          ^
          |
items -----+
```

After rebinding:

```
values -> original list object []

items -> new list object [10]
```

The caller's name `values` is not affected. The function cannot rebind the caller's local name by assigning to its own parameter. It can only rebind its own local name.

This is why the common phrase “Python passes by reference” is not precise enough. Python passes object references by assignment. The callee receives new local names bound to the same objects that the caller supplied. Mutation of a shared mutable object is visible. Rebinding a local parameter is not visible to the caller.

The same rule also explains why immutable objects appear to behave differently:

```
def increment(x):
    x = x + 1

n = 5
increment(n)

print(n)
```

The value of `n` remains 5. Inside the function, `x` is initially bound to the same integer object as `n`. But `x + 1` produces another integer object, and the local name `x` is rebound to that new object. The caller's name `n` still points to the original integer object.

Conceptually:

before `x = x + 1`:

```
n ----+
      |
      v
    int object 5
      ^
      |
x ----+
```

after `x = x + 1`:

`n -> int object 5`

`x -> int object 6`

The mechanism is the same as with lists. The difference is that integers are immutable, so there is no operation like `append` that changes the integer object in place.

The box model also fails for containers. A Python list does not contain raw values directly in the simple C-array sense. It contains references to objects.

For example:

```
items = [10, "hello", []]
```

Conceptually:

```
items -> list object
        slot 0 -> int object 10
        slot 1 -> str object "hello"
        slot 2 -> list object []
```

The list can hold mixed types because each slot stores an object reference, and each object carries its own type.

This also explains nested aliasing:

```
inner = []
a = [inner]
b = [inner]
```

The two outer lists are different objects, but both contain a reference to the same inner list.

Conceptually:

```

a -> outer list
      slot 0 ----+
                |
                v
            inner list []
                ^
                |
b -> outer list ----+
      slot 0

```

If the inner list is mutated:

```
inner.append(1)
```

both `a[0]` and `b[0]` show the change, because both reach the same inner object.

This is why Python programmers must think in object graphs. Names, containers, attributes, frames, and closures all create reference paths between objects. The question is not only what a variable “contains.” The better question is:

Which object does this name or slot refer to, and what other references reach the same object?

The `del` statement also becomes clear with this model. It does not necessarily destroy an object. It removes a binding.

Example:

```
a = [1, 2, 3]
b = a
```

```
del a
```

After `del a`, the list object still exists because `b` still refers to it. Conceptually:

before `del`:

```

a ----+
      |
      v
    list object [1, 2, 3]
      ^
      |
b ----+

```

after `del a`:

```
b -> list object [1, 2, 3]
```

Only when no references remain can the object become reclaimable.

This connects directly to CPython reference counting. Binding a name to an object creates a reference. Binding another name to the same object creates another reference. Rebinding or deleting a name releases a reference. If the reference count reaches zero, CPython can deallocate the object, except for cyclic structures handled by the cyclic garbage collector.

A compact reference-lifetime example is:

```
a = []  
b = a  
del a  
b = None
```

The list survives after `del a`, because `b` still points to it. It becomes reclaimable after `b = None`, assuming no other references exist.

Conceptually:

```
a = []      creates list and binds a  
b = a      binds b to same list  
del a      removes one reference  
b = None   removes last visible reference
```

This is the real meaning of Python variable behavior. A variable name is not the owner of an object in a simple absolute sense. It is one reference path among possibly many.

The difference between C boxes and Python bindings can be summarized as follows:

C-style mental model:

```
variable has declared type  
variable corresponds to storage  
assignment writes into storage  
copying depends on value/pointer rules  
object lifetime may be scope-bound or manual
```

Python mental model:

```
name has no fixed declared type  
name is a binding in a namespace or frame  
assignment binds name to object reference  
multiple names may refer to one object  
object lifetime follows reference topology
```

This does not mean that Python has no memory structure. It means the memory structure is not the one suggested by C variables. Python's structure is an object graph managed by the runtime.

The final practical rules are:

- a Python name points to an object;
- assignment changes bindings;
- assignment between names does not copy the object;
- mutation changes an object reached through a reference;
- rebinding changes which object a name reaches;
- function parameters are local names bound to argument objects;
- containers store references to objects;
- objects survive while some reference path reaches them.

Once these rules are understood, many Python behaviors stop being surprising. List aliasing, function argument effects, object mutation, `del`, shallow copies, closures, and reference-counted lifetime all follow from the same principle: Python variables are not C boxes. They are names bound to runtime objects.

3.6.4 Memory Storage Classifications: Built-in Single-Value Objects vs. Composite Reference Collections

Python data types should not be understood only by their surface syntax. They should also be understood by the kind of object topology they create in memory. Some built-in objects behave like single conceptual values: an integer, a floating-point number, a Boolean, `None`, or a string. Other objects behave as composite containers: lists, tuples, dictionaries, sets, user-defined instances, and other structures that hold references to other objects.

This distinction is not the same as “simple” versus “advanced.” It is a distinction about object structure. A single-value object is usually treated as one conceptual value. A composite object is a container or structured object whose internal state connects to other objects.

At the Python level:

```
x = 10
name = "Python"
items = [10, "Python"]
```

these assignments look similar. Each binds a name to an object. But the resulting object topologies differ.

Conceptually:

```
x -> int object 10

name -> str object "Python"

items -> list object
        slot 0 -> int object 10
        slot 1 -> str object "Python"
```

The integer and string are objects, but they are not containers of arbitrary Python object references in the way a list is. The list is a composite object: it stores references to other objects.

Single-Value Objects

Single-value objects represent one conceptual value. Common examples include:

```
int
float
complex
bool
None
str
bytes
```

These objects still have identity, type, value, and CPython object metadata. They are not raw C primitives at the Python level. The important point is that they are normally used as individual values rather than as containers of other independently referenced Python objects.

For example:

```
a = 42
b = 3.14
c = True
d = None
text = "hello"
```

Conceptually:

```

a    -> int object 42
b    -> float object 3.14
c    -> bool object True
d    -> None object
text -> str object "hello"

```

A Python integer object is not a raw machine integer stored inside the name `a`. It is a heap-managed object. But from the programmer's structural point of view, it behaves as one value.

Most of these single-value objects are immutable. If the apparent value changes, the name is usually rebound to another object.

For example:

```

x = 10
x = x + 1

```

This should not be imagined as modifying the integer object 10. Integers are immutable. The expression `x + 1` produces an integer object representing 11, and the name `x` is rebound to that object.

Conceptually:

```

before:
  x -> int object 10

after:
  x -> int object 11

```

The old object may be reused, cached, or eventually released depending on CPython's internal rules, but the language-level idea is clear: the integer object was not mutated in place.

The same applies to strings:

```

s = "py"
s = s + "thon"

```

The string `"py"` is not extended in place. A new string object representing `"python"` is produced, and `s` is rebound.

Composite Reference Collections

Composite objects hold references to other objects. Lists, tuples, dictionaries, and sets are the most important built-in examples.

A list:

```
items = [1, "two", []]
```

should be understood as:

```

items -> list object
      slot 0 -> int object 1
      slot 1 -> str object "two"
      slot 2 -> list object []

```

The list does not contain a raw integer, raw string bytes, and another list physically embedded as complete values in the C-array sense. It contains references to Python objects.

This is why lists can contain heterogeneous elements:

```
items = [10, "hello", print, int, [1, 2]]
```

Every element position stores a reference to an object. Since each object carries its own type, the list itself does not need one fixed element type.

A tuple is also a composite reference collection, but its structure is immutable:

```
t = (1, "two", [])
```

Conceptually:

```
t -> tuple object
    slot 0 -> int object 1
    slot 1 -> str object "two"
    slot 2 -> list object []
```

The tuple cannot be resized and its slots cannot be rebound after construction. But the objects referenced by the tuple may themselves be mutable. Therefore:

```
t = (1, "two", [])
t[2].append(3)
```

is valid, because the tuple slot still points to the same list object. The tuple was not changed; the list object inside it was mutated.

After mutation:

```
t -> tuple object
    slot 0 -> int object 1
    slot 1 -> str object "two"
    slot 2 -> list object [3]
```

This distinction is important. Immutability of a container means its own reference slots cannot be changed. It does not automatically mean every object reachable from it is immutable.

Dictionaries as Reference Mappings

A dictionary is also a composite object, but it is not positional like a list or tuple. It maps keys to values.

Example:

```
record = {
    "name": "Ada",
    "age": 36,
    "skills": ["math", "programming"],
}
```

Conceptually:

```
record -> dict object
    key "name"    -> value str object "Ada"
    key "age"     -> value int object 36
    key "skills"  -> value list object
                        slot 0 -> str object "math"
                        slot 1 -> str object "programming"
```

Both keys and values are object references. Dictionary keys must satisfy hash and equality requirements, which is why mutable lists cannot be used as dictionary keys. A key must remain stable enough for the hash table to find it again.

The dictionary itself is mutable. Adding, removing, or replacing a key-value entry changes the dictionary object:

The instance object does not contain the full string and full list directly as raw embedded fields. Its attribute namespace maps attribute names to object references.

This is why aliasing works through attributes too:

```
skills = ["math"]
p.skills = skills

skills.append("programming")
print(p.skills)
```

The attribute `p.skills` and the name `skills` refer to the same list object.

Composite Objects Create Object Graphs

Once composite objects are introduced, a Python program becomes an object graph. Objects point to other objects. Containers point to elements. Dictionaries point to keys and values. Instances point to attribute values. Functions point to code objects, globals, defaults, and closure cells. Frames point to local objects.

A small example:

```
data = {
    "numbers": [1, 2, 3],
    "label": "sample",
}
```

creates a structure like:

```
data -> dict object
      "numbers" -> list object
                      slot 0 -> int object 1
                      slot 1 -> int object 2
                      slot 2 -> int object 3
      "label"    -> str object "sample"
```

This graph shape matters for memory management. The dictionary keeps the list and string alive. The list keeps the integers alive. If the name `data` is removed, the dictionary may become unreachable. If no other references point into the graph, the whole structure may become reclaimable.

But if another name points to the inner list:

```
numbers = data["numbers"]
del data
```

then the list survives:

```
numbers -> list object [1, 2, 3]
```

The outer dictionary may disappear, but the inner list remains because another reference path reaches it.

Single-Value Objects Can Still Be Shared

Single-value objects may also be shared. This is especially common for immutable objects.

Example:

```
a = 10
b = 10
```

In CPython, `a` and `b` may refer to the same integer object for small cached integers. But program logic should not depend on this identity-sharing behavior for numbers or strings. The correct value comparison is:

```
a == b
```

```
not:
```

```
a is b
```

The important point is that immutable sharing is safe because the object cannot be changed in place. If two names refer to the same immutable integer or string object, neither name can mutate it and surprise the other.

Composite mutable sharing is different. If two names refer to the same list, dictionary, set, or instance, mutation through one reference is visible through the other.

Why This Distinction Matters

The distinction between single-value objects and composite reference collections explains many practical Python behaviors.

For a single immutable value:

```
x = 5
y = x
y = y + 1
```

After this:

```
x -> int object 5
y -> int object 6
```

The operation on `y` produced a new object and rebound `y`. The object reached by `x` was not changed.

For a mutable composite object:

```
x = []
y = x
y.append(1)
```

After this:

```
x ----+
      |
      v
    list object [1]
      ^
      |
y ----+
```

The list object was mutated. Both names observe the change.

This is why Python beginners are often surprised by lists but not by integers. The assignment mechanism is the same. The object category is different. Integers are immutable single-value objects. Lists are mutable composite reference collections.

The Practical Classification

A useful practical classification is:

single-value style objects:

```
int
float
complex
bool
NoneType
str
bytes
```

composite reference collections:

```
list
tuple
dict
set
frozenset
user-defined instances
function objects
module objects
frame objects
```

This classification is not a formal complete taxonomy of Python's entire data model. It is a memory-intuition taxonomy. It helps answer the question:

Is this object mainly one conceptual value, or is it a structure that holds references to other objects?

That question matters for copying, aliasing, mutation, function arguments, object lifetime, and memory use.

The final model is:

single-value object

```
name -> object
```

composite object

```
name -> container object
           internal slots / entries / attributes -> other objects
```

Python programs are built by connecting these objects. Simple names point to objects. Composite objects point onward to other objects. Memory management follows this topology. Mutation changes objects or their internal references. Rebinding changes which object a name reaches. Understanding this distinction prepares the next subsection, where mutability and identity must be separated from value equality.

3.6.5 Mutability and Identity: Immutable Objects, Mutable Objects, Shared References, and `is` vs. `==`

Once Python variables are understood as names bound to objects, the next distinction is mutability. Mutability answers a simple question:

Can this object be changed in place while keeping the same identity?

Identity and value must be kept separate. Identity tells us whether two references point to the same object. Value tells us what that object represents or contains. Mutability tells us whether the object's value or internal state can change without replacing the object.

Python exposes identity with `is` and `id()`. It exposes value comparison through `==`. These two operations answer different questions.

`a is b`

asks:

Do `a` and `b` refer to the exact same object?

while:

`a == b`

asks:

Do the objects currently referenced by `a` and `b` compare equal by value?

For example:

```
a = [1, 2, 3]
b = [1, 2, 3]

print(a == b)    # True
print(a is b)    # False
```

The two lists have the same contents, so they compare equal. But they are two different list objects, so they are not identical.

Conceptually:

`a -> list object [1, 2, 3]`

`b -> list object [1, 2, 3]`

There are two objects with equal values.

Now compare:

```
a = [1, 2, 3]
b = a

print(a == b)    # True
print(a is b)    # True
```

Here, both names refer to the same list object.

```
a ----+
      |
      v
  list object [1, 2, 3]
      ^
      |
b ----+
```

The value comparison is true because the object equals itself. The identity comparison is true because there is only one object.

Mutable Objects

A mutable object can be changed in place. Lists, dictionaries, sets, most user-defined instances, byte arrays, and many other composite objects are mutable.

Example:

```
items = [1, 2, 3]
print(id(items))

items.append(4)
print(id(items))
```

The identity remains the same. The list object was not replaced. Its internal state changed. Conceptually:

before append:

```
items -> list object [1, 2, 3]
```

after append:

```
items -> same list object [1, 2, 3, 4]
```

This is why shared references to mutable objects matter. If two names point to the same mutable object, mutation through one name is visible through the other.

```
a = []
b = a

b.append("x")

print(a)
```

The result is:

```
['x']
```

The name **b** was used to mutate the list, but **a** observes the same object.

This is not because Python copied the change from **b** to **a**. There was only one list object.

Immutable Objects

An immutable object cannot be changed in place after it is created. Integers, floats, Booleans, strings, bytes, tuples, and **None** are common examples. Some of these are simple single-value objects; tuples are composite but immutable in their own slots.

Example:

```
x = 10
print(id(x))

x = x + 1
print(id(x))
```

This does not mutate the integer object 10. The expression **x + 1** produces an integer object representing 11, and the name **x** is rebound to that object.

Conceptually:

before:

```
x -> int object 10
```

after:

```
x -> int object 11
```

The arrow changed. The old integer object did not become 11.

The same applies to strings:

```
s = "py"
s = s + "thon"
```

The string object "py" is not extended in place. A new string object representing "python" is produced, and `s` is rebound.

This is why repeated string concatenation in loops can be inefficient: many intermediate string objects may be created. A list accumulation followed by `"".join(...)` is often better when building large strings.

Immutability of a Container Is Not Deep Immutability

A tuple is immutable because its slots cannot be rebound after creation:

```
t = (1, 2, 3)
t[0] = 99          # invalid
```

But if a tuple contains a mutable object, that contained object can still be mutated:

```
t = ([1, 2], "hello")
t[0].append(3)
```

```
print(t)
```

The result is:

```
([1, 2, 3], 'hello')
```

The tuple itself was not changed by replacing slot 0. Slot 0 still points to the same list object. The list object was mutated.

Conceptually:

```
t -> tuple object
    slot 0 -> list object [1, 2]
    slot 1 -> str object "hello"
```

after append:

```
t -> same tuple object
    slot 0 -> same list object [1, 2, 3]
    slot 1 -> same str object "hello"
```

So immutability may be shallow. The tuple's own reference slots are fixed, but the objects reachable through those slots may still be mutable.

Shared References and Aliasing

Aliasing means that two or more names or container positions refer to the same object. Aliasing is harmless with immutable objects in most ordinary cases because the object cannot be changed in place. It becomes important with mutable objects because a mutation through one reference affects all other references to that object.

Example:

```
row = [0, 0, 0]
matrix = [row, row, row]

matrix[0][0] = 1

print(matrix)
```

The result is:

```
[[1, 0, 0], [1, 0, 0], [1, 0, 0]]
```

This happens because all three positions in `matrix` point to the same inner list.

Conceptually:

```
matrix -> outer list
          slot 0 ----+
          slot 1 ----+--> inner list [1, 0, 0]
          slot 2 ----+
```

The correct way to create independent rows is:

```
matrix = [[0, 0, 0] for _ in range(3)]
```

Now each row expression is evaluated separately, creating separate list objects.

```
matrix -> outer list
          slot 0 -> list object [0, 0, 0]
          slot 1 -> list object [0, 0, 0]
          slot 2 -> list object [0, 0, 0]
```

A mutation to one row no longer changes the others.

is: Identity Comparison

The `is` operator checks identity. It should be used when the question is truly whether two references point to the same object.

The most common correct use is testing for `None`:

```
if value is None:
    print("missing value")
```

This is correct because `None` is a singleton object. The question is identity: does `value` point to the specific object `None`?

The same applies to `True` and `False` in some cases, although ordinary Boolean conditions usually do not require explicit comparison:

```
if flag:
    ...
```

is normally better than:

```
if flag is True:
    ...
```

Identity comparison is also useful for proving aliasing:

```
a = []
b = a
c = []

print(a is b)    # True
print(a is c)    # False
```

This tells us that `a` and `b` refer to the same list, while `c` refers to another list.

==: Value Equality

The `==` operator checks equality according to the object's equality behavior. For built-in containers, this usually means comparing contents.

Example:

```
a = [1, 2]
b = [1, 2]

print(a == b)    # True
print(a is b)    # False
```

The objects are distinct but equal.

For user-defined classes, equality can be customized through special methods such as `__eq__`. If no meaningful equality is defined, objects may fall back to identity-based comparison behavior.

Example:

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

p1 = Point(1, 2)
p2 = Point(1, 2)

print(p1 == p2)
```

Without a custom equality method, this may be false because the two instances are distinct objects. To make equality value-based, the class must define how equality should be tested.

Conceptually:

```
identity:
    are these the same object?
```

```
equality:
    should these objects be considered equal?
```

Those are not the same question.

Do Not Use `is` for Numbers and Strings

Because CPython may reuse some immutable objects, especially small integers and interned strings, identity tests may appear to work accidentally:

```
a = 10
b = 10

print(a is b)    # may be True in CPython
```

But this is not the correct way to compare numbers. The correct comparison is:

```
a == b
```

The same warning applies to strings:

```
a = "hello"
b = "hello"

print(a is b)    # implementation-dependent behavior may appear
print(a == b)    # correct value comparison
```

The fact that two equal immutable values may share identity is an implementation optimization or caching behavior. It is not the right semantic test for ordinary program logic.

The rule is:

Use `==` when asking whether values are equal. Use `is` when asking whether references point to the same object.

Mutation vs. Rebinding

Many Python bugs come from confusing mutation and rebinding.

Mutation:

```
a = []
b = a

a.append(1)
```

After mutation:

```
a ----+
      |
      v
    list object [1]
      ^
      |
b ----+
```

Rebinding:

```
a = []
b = a

a = [1]
```

After rebinding:

```
a -> new list object [1]

b -> original list object []
```

The first program changes the shared object. The second program changes only which object the name `a` refers to.

This distinction is also visible with augmented assignment. For mutable objects, an operation such as:

```
items += [3]
```

may mutate the existing list object in place. For immutable objects, an operation such as:

```
number += 1
```

must produce another integer object and rebind the name.

So `+=` does not always mean the same memory behavior. It depends on the object's type and its operation implementation.

Example:

```
a = []
b = a
a += [1]
```

```
print(b)
```

The list object is mutated, so `b` sees the change.

But:

```
a = 10
b = a
a += 1
```

```
print(b)
```

The integer object is not mutated. The name `a` is rebound to another integer object. The name `b` still refers to the original integer object.

Function Arguments Revisited

Mutability and identity also explain function argument behavior.

Example with mutation:

```
def add_item(items):
    items.append("x")
```

```
data = []
add_item(data)
```

```
print(data)
```

The function receives a local name `items` bound to the same list object as `data`. The method call mutates the object. The caller observes the mutation.

Example with rebinding:

```
def replace(items):  
    items = ["x"]  
  
data = []  
replace(data)  
  
print(data)
```

The local name `items` is rebound to a new list. The caller's name `data` still points to the original list.

The rule is not that Python treats mutable and immutable arguments through different passing mechanisms. The passing mechanism is the same. The difference is whether the function mutates a shared object or rebinds a local name.

The Practical Model

The full model is:

```
identity:  
    which exact object is this?  
  
value equality:  
    do these objects compare equal?  
  
mutability:  
    can this object change in place?  
  
shared reference:  
    do multiple names or slots point to the same object?
```

These questions combine.

For immutable objects:

```
a = 10  
b = a  
a = a + 1
```

The operation produces a new object and rebinds `a`. The object reached by `b` is unchanged.

For mutable objects:

```
a = []  
b = a  
a.append(1)
```

The operation changes the shared object. Both names observe the change.

The final rules are:

- `is` checks whether two references point to the same object;
- `==` checks whether two objects compare equal by value;
- mutable objects can change while keeping identity;
- immutable objects cannot change in place;
- assignment between names shares object references;

- mutation through one reference is visible through all aliases;
- rebinding a name does not change other names that pointed to the old object.

This distinction is one of the foundations of Python programming. Without it, list behavior, function arguments, default mutable parameters, shallow copies, tuples containing lists, and identity comparisons appear inconsistent. With it, they all follow from the same runtime model: Python names and containers hold references to objects, and objects may or may not be mutable.

3.6.6 Object Interfaces: Dunder Methods, Internal Slots, Operators, Construction, and Representation

Python objects are not used only by calling ordinary named methods directly. They also participate in a larger interface system defined by the Python data model. This system explains why objects can be added with `+`, called with parentheses, iterated with `for`, indexed with square brackets, printed, formatted, compared, constructed, entered with `with`, and inspected at runtime.

The visible syntax is often short:

```
a + b
items[0]
len(items)
for x in items:
    ...
str(obj)
repr(obj)
obj()
```

but these operations are not primitive in the C sense. They are routed through object interfaces. In Python, much of the language syntax is connected to special methods, often called *dunder methods* because their names begin and end with double underscores: `__add__`, `__len__`, `__iter__`, `__getitem__`, `__call__`, `__repr__`, and so on.

These methods should not be treated as mysterious magic. They are ordinary-looking names with special meaning to the interpreter. They define how an object responds to language-level operations.

For example:

```
a + b
```

is connected to addition behavior, commonly through methods such as:

```
a.__add__(b)
```

This does not mean that Python literally rewrites every expression in this exact source form. The implementation uses type-level lookup and internal operation slots. But conceptually, the point is correct: the operator `+` is not one fixed machine operation. It asks the involved objects how addition should work.

This is why the same operator can mean different things:

```
1 + 2           # integer addition
"py" + "thon"   # string concatenation
[1, 2] + [3, 4] # list concatenation
```

The syntax is the same, but the involved object types provide different behavior.

The same principle appears in length:

```
len(items)
```


The built-in `len()` function does not inspect every possible object by hand. It asks the object for its length through the object's length protocol, usually associated with `__len__`.

For example:

```
class Bag:
    def __init__(self, values):
        self.values = values

    def __len__(self):
        return len(self.values)

bag = Bag([1, 2, 3])
print(len(bag))
```

The object `bag` becomes acceptable to `len()` because it provides the required interface.

This is the general pattern:

```
visible operation
    asks object interface for behavior
```

Some common mappings are:

<code>x + y</code>	-> addition protocol, e.g. <code>__add__</code>
<code>x == y</code>	-> equality protocol, e.g. <code>__eq__</code>
<code>x < y</code>	-> ordering protocol, e.g. <code>__lt__</code>
<code>len(x)</code>	-> length protocol, e.g. <code>__len__</code>
<code>x[i]</code>	-> item access protocol, e.g. <code>__getitem__</code>
<code>x[i] = value</code>	-> item assignment protocol, e.g. <code>__setitem__</code>
<code>for item in x</code>	-> iteration protocol, e.g. <code>__iter__</code> / <code>__next__</code>
<code>x()</code>	-> call protocol, e.g. <code>__call__</code>
<code>str(x)</code>	-> string conversion, e.g. <code>__str__</code>
<code>repr(x)</code>	-> programmer representation, e.g. <code>__repr__</code>

This interface system is one reason Python supports behavior-based programming. A function does not always need to demand one exact type. It may need only an object that supports a certain protocol.

For example:

```
def print_all(values):
    for value in values:
        print(value)
```

This function does not require `values` to be a list. It requires `values` to be iterable. A list, tuple, string, generator, file object, dictionary, set, or user-defined iterable may work. The common requirement is not shared inheritance from one visible class, but support for the iteration interface.

A user-defined iterable can expose that behavior:

```
class Countdown:
    def __init__(self, start):
        self.current = start

    def __iter__(self):
        return self

    def __next__(self):
        if self.current <= 0:
```

```

        raise StopIteration
    value = self.current
    self.current -= 1
    return value

for number in Countdown(3):
    print(number)

```

The `for` loop works because the object supports the iteration protocol. The loop asks for an iterator, repeatedly requests the next value, and stops when `StopIteration` is raised.

This is not syntactic sugar in the trivial sense. It is a runtime protocol. The syntax is short because the object interface supplies the behavior.

Indexing works the same way:

```

class Box:
    def __init__(self, values):
        self.values = values

    def __getitem__(self, index):
        return self.values[index]

box = Box(["a", "b", "c"])

print(box[1])

```

The expression:

```
box[1]
```

is accepted because `box` provides item access behavior.

This also means that Python syntax is extensible through object interfaces. The programmer does not add new operators to the language, but can define how existing language operations behave for new object types.

Construction is also part of this interface system. When a class is called:

```
p = Point(2, 3)
```

the class object is invoked. Instance creation normally passes through two main special methods:

```
__new__
__init__
```

The method `__new__` is responsible for creating and returning the new object. The method `__init__` initializes that object after it exists.

A simplified example is:

```

class Point:
    def __new__(cls, x, y):
        obj = super().__new__(cls)
        return obj

    def __init__(self, x, y):
        self.x = x
        self.y = y

```

Most ordinary classes do not need to define `__new__`; they only define `__init__`. But the separation is important. Initialization is not allocation. The object must exist before `__init__` can set attributes on it.

Conceptually:

```
Point(2, 3)
    call class object
    allocate instance through __new__
    initialize instance through __init__
    return reference to initialized object
```

This also explains why `__init__` should not return the new object. Its role is to configure an already created instance.

Representation is another object interface. Python distinguishes between a programmer-oriented representation and a user-oriented string form.

The method `__repr__` should produce a representation useful for debugging and inspection. The method `__str__` should produce a more readable human-facing string when such a distinction is useful.

Example:

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __repr__(self):
        return f"Point({self.x!r}, {self.y!r})"

    def __str__(self):
        return f"({self.x}, {self.y})"

p = Point(2, 3)

print(repr(p))
print(str(p))
print(p)
```

The output is conceptually:

```
Point(2, 3)
(2, 3)
(2, 3)
```

The call `print(p)` uses the string form. Interactive inspection often uses the representation form. If `__str__` is not defined, Python may fall back to `__repr__`.

This interface matters because objects should be inspectable. A badly represented object makes debugging harder. A useful `__repr__` tells the programmer what object exists and what important state it carries.

Comparison is also controlled by object methods. The expression:

```
a == b
```

uses equality behavior, commonly `__eq__`. Without a custom equality definition, user-defined instances often compare by identity-like default behavior.

Example:

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y
```

```
p1 = Point(2, 3)
p2 = Point(2, 3)
```

```
print(p1 == p2)
```

The two objects may not compare equal because they are distinct instances. To define value equality:

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __eq__(self, other):
        if not isinstance(other, Point):
            return NotImplemented
        return self.x == other.x and self.y == other.y
```

Now two points with the same coordinates can compare equal.

The value `NotImplemented` is important in binary operations and comparisons. It tells Python that this method does not know how to handle the other operand, allowing Python to try reflected or alternative behavior. It is not the same as raising `NotImplementedError`.

There is also a deeper implementation layer behind these special methods: internal slots. CPython's type objects contain slots for categories of behavior: numeric operations, sequence operations, mapping operations, call behavior, attribute access, deallocation, representation, and so on. Special methods at the Python level are connected to these type-level operation slots.

This matters because special method lookup is not always the same as ordinary instance attribute lookup. For many language operations, Python looks for special methods on the type, not merely as arbitrary attributes stored on one instance.

For example:

```
class Number:
    def __init__(self, value):
        self.value = value
```

```
n = Number(10)
```

```
n.__add__ = lambda other: 999
```

```
print(n + 5)
```

One should not expect this instance-level assignment to make `n + 5` work. Operator behavior is normally resolved through the object's type, not through a random instance attribute named `__add__`. The correct place is the class:

```
class Number:
    def __init__(self, value):
        self.value = value

    def __add__(self, other):
        return Number(self.value + other)
```

This is a critical distinction. Dunder methods are not just decorative ordinary methods with funny names. They are hooks recognized by the interpreter and connected to type-level behavior.

The word *slot* can also refer to `__slots__`, which is a different but related object-layout feature. By default, many user-defined instances store attributes in a per-instance dictionary:

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y
```

Conceptually:

```
Point instance
  __dict__
    "x" -> object
    "y" -> object
```

With `__slots__`:

```
class Point:
    __slots__ = ("x", "y")

    def __init__(self, x, y):
        self.x = x
        self.y = y
```

the instance can store those attributes in fixed slots instead of an ordinary per-instance attribute dictionary.

Conceptually:

```
Point instance
  slot x -> object
  slot y -> object
```

This can reduce memory use for many instances and can prevent arbitrary new attributes unless a dictionary slot is explicitly allowed. But `__slots__` should not be confused with operator slots inside CPython’s type object. They are connected by the general idea of fixed places for behavior or data, but they are not the same mechanism.

So there are at least three meanings of “slot” that must be kept separate:

```
frame local slots:
    fast storage for local references inside a frame

instance __slots__:
    fixed attribute storage for instances

type operation slots:
    internal behavior entries used for operators and protocols
```

All three are reference-oriented runtime storage or dispatch mechanisms, but they operate at different levels.

Attribute access is another important object interface. The expression:

```
obj.name
```

may look like direct field access, but in Python it is a protocol-driven operation. Attribute lookup may involve the instance dictionary, the class dictionary, descriptors, properties, inherited attributes, and special methods such as `__getattr__` and `__getattribute__`.

For example:

```
class Person:
    def __init__(self, name):
        self.name = name

p = Person("Ada")
print(p.name)
```

The simple case finds `name` in the instance attribute storage. But Python also allows controlled computed attributes:

```
class Person:
    def __init__(self, first, last):
        self.first = first
        self.last = last

    @property
    def full_name(self):
        return self.first + " " + self.last
```

Now:

```
p.full_name
```

looks like attribute access, but it calls descriptor logic behind the property. This is another example of Python syntax being mediated by object interfaces.

The call interface is equally important. Any object can be made callable by defining `__call__`.

Example:

```
class Multiplier:
    def __init__(self, factor):
        self.factor = factor

    def __call__(self, value):
        return self.factor * value
```

```
double = Multiplier(2)
```

```
print(double(10))
```

The object `double` is not a function, but it is callable. Parentheses trigger its call behavior. Conceptually:

```
double(10)
    asks object double for call behavior
    invokes __call__
```

This explains why Python call syntax is broader than function calls. Classes are callable. Functions are callable. Methods are callable. Some instances are callable. The syntax asks for the call protocol; the object decides whether it supports it.

Context managers show the same interface design:

```
with open("data.txt") as f:
    text = f.read()
```

The `with` statement uses methods associated with entering and exiting a context, commonly `__enter__` and `__exit__`. This allows objects to define setup and cleanup behavior around a block.

A simplified custom context manager:

```
class Announce:
    def __enter__(self):
        print("enter")
        return self

    def __exit__(self, exc_type, exc, tb):
        print("exit")

with Announce():
    print("inside")
```

The object interface determines what happens before and after the block. This is not merely syntactic decoration; it is structured resource control.

The practical lesson is that Python objects are defined not only by stored data, but by supported interfaces. An object may be:

```
iterable
callable
indexable
comparable
hashable
context-manageable
representable
numeric-like
sequence-like
mapping-like
```

These categories are behavior categories. They are not always tied to one exact class. Python code often works best when it asks for behavior rather than one rigid type.

For example, a function that consumes lines does not need a list:

```
def count_lines(lines):
    count = 0
    for line in lines:
        count += 1
    return count
```

It needs an iterable. A list, tuple, file object, generator, or custom iterable may work. The object interface is the contract.

This is the correct way to understand dunder methods: they are the named hooks through which objects participate in Python syntax and built-in operations. They are not meant to be called constantly by hand in ordinary code. Usually, the programmer writes:

```
len(x)
x + y
x[i]
for item in x:
    ...
str(x)
repr(x)
```

rather than directly writing:

```
x.__len__()  
x.__add__(y)  
x.__getitem__(i)  
x.__iter__()  
x.__str__()  
x.__repr__()
```

The direct calls are useful for explanation, but the idiomatic code uses the language operation. The operation then dispatches through the object's interface.

The final model is:

```
object  
    identity  
    type  
    value/state  
    references to other objects  
    interface methods / type slots  
  
language syntax  
    requests behavior from object interfaces  
  
runtime  
    dispatches operation through type-defined machinery
```

This completes the data-type architecture picture. Python data types are not only memory containers and not only names printed by `type()`. They are runtime objects with behavior interfaces. Operators, construction, representation, iteration, indexing, calling, comparison, and context management all flow through those interfaces.

This is why Python can be both dynamic and structured. Names are flexible, but objects are not shapeless. Each object has a type, and each type defines what the object can do.

Chapter 4

Variables and Singular Data Types

A Python program does not begin with objects floating in isolation. It begins with names that allow the programmer to reach objects, inspect them, combine them, and replace them with other objects during execution. This chapter introduces that layer: how Python names are written, how assignment binds names to objects, and how the simplest built-in value domains behave once those bindings exist.

For a C programmer, this is one of the first places where Python must be read differently. A Python variable is not a declared memory box with a fixed storage type. It is a name in a namespace, and that name can be bound to an integer object, a floating-point object, a string object, a boolean object, `None`, or another runtime object. The object carries the type; the name provides access to it.

The chapter therefore starts with identifiers and syntax rules, because a name must first be valid source text before it can become a runtime binding. It then moves to assignment, reassignment, object sharing, unpacking, augmented assignment, and deletion. These operations establish the practical binding model used everywhere else in Python.

After that, the chapter studies singular data types: `int`, `float`, `complex`, `bool`, and `None`. These are the direct, non-container value domains that appear constantly in ordinary programs. The goal is not only to know their syntax, but to understand their runtime behavior: exact integer growth, floating-point approximation, complex-number boundaries, boolean singleton values, and `None` as the explicit object of absence.

The chapter ends by connecting dynamic assignment with strong runtime operation checking. Python allows a name to move freely from one object to another, but it does not allow arbitrary operations between incompatible objects. This gives the central rule for the whole chapter:

Names are flexible bindings. Objects keep their runtime type. Operations are checked against the objects involved.

4.1 Identifiers and Syntax Rules for Variables

Before a Python program can bind a name to an object, the name must first be accepted as valid source-code text. This section therefore begins before runtime execution. It looks at the lexical rules that decide whether a written name is a possible Python identifier at all.

This distinction is important. Python variables are flexible runtime bindings, but their written form is not arbitrary. A programmer may rebind the same name to an integer, a string, a floating-point value, or another object, but the name itself must still obey the syntactic rules of the language.

A variable name must be readable both by Python and by humans. The interpreter needs a form that can be separated cleanly from numbers, keywords, operators, and other tokens. The programmer needs a form that explains the role of the value without creating unnecessary visual noise.

This section therefore separates two questions:

What names does Python legally accept, and what names should a programmer actually choose?

The first question belongs to syntax: digits, letters, underscores, Unicode characters, reserved keywords, and case sensitivity. The second question belongs to style: readable `snake_case`, careful use of underscores, and names that make program state easier to inspect.

Only after these rules are clear does assignment make full sense. A name must first be valid text before it can become a binding to a runtime object.

4.1.1 Legal and Structural Lexer Constraints

Before a Python name can become a runtime binding, it must first survive the source-code reading phase. Python does not begin by asking what value a variable should contain. It first reads the program as text and divides that text into meaningful pieces: names, keywords, literals, operators, delimiters, indentation markers, and so on.

A variable name is therefore not only a programmer's label. It is also a piece of source syntax that must be recognizable by Python's lexical rules. If the text cannot be tokenized as a valid identifier, execution never begins. No object is created, no name is bound, and no runtime type question exists yet.

This matters because the previous chapter established that Python names are flexible bindings, not C-style typed boxes. That flexibility starts only after the source code has been accepted. The syntax of the name is still fixed by the language.

The central rule is simple:

A Python identifier must look like a valid name before it can be used as a runtime name.

A valid ordinary identifier may contain letters, digits, and underscores, but it cannot start with a digit. It also cannot be one of Python's reserved keywords.

The following program uses strings to test possible names before trying to use them as real source-code identifiers. This is useful because invalid identifiers cannot be written directly as assignment targets without stopping the whole file with a syntax error.

```
import keyword

candidate = "chuck_fact_42"

print(f"candidate text: {candidate}")
print(f"type(candidate): {type(candidate)}")

print("selected string methods:")
print("isidentifier" in dir(candidate))
print("upper" in dir(candidate))
print("replace" in dir(candidate))

print(f"is valid identifier shape: {candidate.isidentifier()}")
print(f"is reserved keyword: {keyword.iskeyword(candidate)}")
print(f"usable ordinary variable name: "
      f"{candidate.isidentifier() and not keyword.iskeyword(candidate)}")
```

Possible output:

```
candidate text: chuck_fact_42
type(candidate): <class 'str'>
selected string methods:
True
True
True
is valid identifier shape: True
```

```
is reserved keyword: False
usable ordinary variable name: True
```

The object being tested here is a string object. The method `isidentifier()` checks whether the string has the structural form of a Python identifier. The `keyword.iskeyword()` function checks the second part of the rule: whether the word is reserved by the language.

This distinction is important. A word can have the shape of an identifier and still be forbidden as an ordinary variable name.

Digit Restrictions: Why Identifiers Cannot Start with a Numerical Character

A Python identifier cannot begin with a digit:

```
42answer = "Nobody expects the Spanish Inquisition!"
```

This is not rejected because the value is wrong. It is rejected because the text cannot be interpreted as a valid assignment target.

The reason is lexical clarity. When Python reads source code, a leading digit begins a numeric literal. If names were allowed to start with digits, the lexer would face confusing cases such as:

```
123abc
42answer
3value
```

Should `42answer` begin as the integer literal `42`, followed by a name? Should it be one name? Should it be an invalid number? Python avoids this ambiguity by using a clean rule: digits may appear after the first character, but not as the first character.

Valid examples:

```
answer42 = "The answer is 42."
fact_7 = "Chuck Norris can divide by zero."
room101 = "Seinfeld parking garage energy."
```

Invalid examples:

```
42answer = "The answer is 42."
7fact = "Chuck Norris counted to infinity. Twice."
101room = "No soup for you."
```

A practical test:

```
names = ["answer42", "fact_7", "room101",
         "42answer", "7fact", "101room"]

for name in names:
    print(f"{name:<10} identifier shape: {name.isidentifier()}")
```

Possible output:

```
answer42    identifier shape: True
fact_7      identifier shape: True
room101     identifier shape: True
42answer    identifier shape: False
7fact       identifier shape: False
101room     identifier shape: False
```

The method `isidentifier()` checks the form of the text. It does not create a variable. It only answers the question: “Could this string be shaped like a Python identifier?”

To see what happens when invalid source text is compiled, we can ask Python to compile small snippets:

```
source_code = '42answer = "The answer is 42."'

try:
    compile(source_code, "<student-example>", "exec")
    print("compiled successfully")
except SyntaxError as err:
    print("syntax error")
    print(f"message: {err.msg}")
```

Possible output:

```
syntax error
message: invalid decimal literal
```

The exact error message may vary between Python versions, but the structural rule is stable: an ordinary identifier cannot start with a digit.

Valid Characters: Letters, Digits, Underscore (`_`), and Unicode Identifier Rules

The simplest safe rule for beginners is this:

Use lowercase English letters, digits after the first character, and underscores.

That gives names such as:

```
user_name = "George Costanza"
line_1 = "It is merely a flesh wound."
answer_42 = "The answer is still 42."
```

The underscore is an ordinary character inside an identifier. It is not subtraction, spacing, or decoration. It is part of the name.

```
favorite_line = "No soup for you."
favorite_line_2 = "These pretzels are making me thirsty."

print(favorite_line)
print(favorite_line_2)
```

Possible output:

```
No soup for you.
These pretzels are making me thirsty.
```

A space is not allowed inside a name:

```
favorite line = "No soup for you."
```

Python reads the space as a separator between tokens. It does not treat `favorite line` as one identifier. If the programmer wants a multi-word name, the normal Python form is `snake_case`:

```
favorite_line = "No soup for you."
```

Python also supports Unicode identifiers. This means that names are not limited to ASCII letters. For example, the following names are structurally valid:

```
café = "coffee"
π_value = 3.14
răspuns_42 = "patruzeci și doi"

print(café)
print(π_value)
print(răspuns_42)
```

Possible output:

```
coffee
3.14
patruzeci și doi
```

However, valid does not always mean wise. Unicode identifiers can be useful in mathematical, linguistic, or local teaching examples, but they can also make code harder to type, search, review, or share across keyboards. For ordinary programming, especially in mixed teams, simple ASCII `snake_case` is usually the clearest choice.

A small inspection program:

```
names = [
    "snake_case",
    "answer_42",
    "_temporary",
    "café",
    "π_value",
    "bad-name",
    "has space",
    "42_first"
]

for name in names:
    print(f"{name:<12} {name.isidentifier()}")
```

Possible output:

```
snake_case    True
answer_42     True
_temporary    True
café          True
π_value       True
bad-name      False
has space     False
42_first      False
```

The invalid cases fail for different reasons:

- `bad-name` contains a hyphen. Python reads the hyphen as the subtraction operator.
- `has space` contains a space. Python reads it as two separate words.
- `42_first` starts with a digit. Python begins reading a number.

The string object gives us the `isidentifier()` method, but the final answer also depends on keywords. For example:

```
import keyword

name = "class"

print(f"name: {name}")
print(f"type(name): {type(name)}")
print(f"isidentifier: {name.isidentifier()}")
print(f"is keyword: {keyword.iskeyword(name)}")
print(f"safe ordinary name: "
      f"{name.isidentifier() and not keyword.iskeyword(name)}")
```

Possible output:

```
name: class
type(name): <class 'str'>
isidentifier: True
is keyword: True
safe ordinary name: False
```

So `class` has the shape of an identifier, but it is not available as an ordinary variable name.

Reserved Keywords: Why `def`, `class`, `import`, and Similar Tokens Cannot Be Used as Names

Some words are reserved because Python uses them to express program structure. Examples include `def`, `class`, `import`, `if`, `else`, `for`, `while`, `return`, `try`, and `with`.

These words are not forbidden because they are bad names in English. They are forbidden because they already have grammatical jobs.

For example, `def` starts a function definition:

```
def say_line():
    print("Nobody expects the Spanish Inquisition!")
```

If Python also allowed `def` to be an ordinary variable name, then this line would become structurally unclear:

```
def = 42
```

The parser must be able to rely on `def` meaning function-definition syntax, not a user-chosen name. The same applies to `class`:

```
class = "Springfield Elementary"
```

This is invalid because `class` is reserved for class definitions.

The same applies to `import`:

```
import = "Bring me the module."
```

This is invalid because `import` is reserved for import statements.

The correct approach is to choose ordinary names that do not collide with Python grammar:

```

definition_text = "Nobody expects the Spanish Inquisition!"
class_name = "Springfield Elementary"
import_note = "The module is loaded later."

print(definition_text)
print(class_name)
print(import_note)

```

Python provides the standard `keyword` module so that the current interpreter can report its own reserved words:

```

import keyword

print(f"type(keyword.kwlist): {type(keyword.kwlist)}")
print(f"number of keywords: {len(keyword.kwlist)}")

print("selected list methods:")
print("append" in dir(keyword.kwlist))
print("count" in dir(keyword.kwlist))
print("index" in dir(keyword.kwlist))

print()
print("Some keyword checks:")
print(f"{'def':<10} {keyword.iskeyword('def')}")
print(f"{'class':<10} {keyword.iskeyword('class')}")
print(f"{'import':<10} {keyword.iskeyword('import')}")
print(f"{'answer_42':<10} {keyword.iskeyword('answer_42')}")

```

Possible output:

```

type(keyword.kwlist): <class 'list'>
number of keywords: 35
selected list methods:
True
True
True

```

Some keyword checks:

```

def      True
class    True
import   True
answer_42 False

```

The exact keyword count may change between Python versions. That is why asking the interpreter through `keyword.kwlist` is better than memorizing a number.

There is also a distinction between hard keywords and soft keywords in modern Python. Hard keywords are always reserved. Soft keywords are treated specially only in specific grammatical contexts. For this introductory section, the practical rule is enough:

Before using a word as a normal variable name, it must be identifier-shaped and not a hard keyword.

The following program checks several candidates:

```

import keyword

candidates = ["def", "class", "import",
              "match", "case", "answer_42",
              "chuck_norris"]

for name in candidates:
    identifier_shape = name.isidentifier()
    hard_keyword = keyword.iskeyword(name)

    print(f"{name:<13} "
          f"identifier={identifier_shape!s:<5} "
          f"keyword={hard_keyword!s:<5} "
          f"usable={identifier_shape and not hard_keyword}")

```

Possible output:

def	identifier=True	keyword=True	usable=False
class	identifier=True	keyword=True	usable=False
import	identifier=True	keyword=True	usable=False
match	identifier=True	keyword=False	usable=True
case	identifier=True	keyword=False	usable=True
answer_42	identifier=True	keyword=False	usable=True
chuck_norris	identifier=True	keyword=False	usable=True

This output demonstrates the important difference. `def`, `class`, and `import` are structurally shaped like identifiers, but they are reserved. A name must pass both tests.

Case Sensitivity: name, Name, and NAME as Distinct Bindings

Python identifiers are case-sensitive. This means that uppercase and lowercase letters are not interchangeable.

The following three names are different:

```

name = "Jerry"
Name = "George"
NAME = "Elaine"

print(name)
print(Name)
print(NAME)

```

Possible output:

```

Jerry
George
Elaine

```

Python does not see these as three spellings of the same name. It sees three distinct identifiers. Therefore, it creates three distinct bindings.

Conceptually:

```

name -> "Jerry"
Name -> "George"
NAME -> "Elaine"

```


This follows directly from the earlier binding model. The source spelling determines the name. If the spelling changes by case, the name changes.

A small runtime inspection:

```
name = "Jerry"
Name = "George"
NAME = "Elaine"

print(f"{'source name':<12} {'value':<10} {'type'}")
print(f"{'name':<12} {name:<10} {type(name)}")
print(f"{'Name':<12} {Name:<10} {type(Name)}")
print(f"{'NAME':<12} {NAME:<10} {type(NAME)}")
```

Possible output:

source name	value	type
name	Jerry	<class 'str'>
Name	George	<class 'str'>
NAME	Elaine	<class 'str'>

The three objects happen to be strings here, but the names are independent. They can even point to objects of different types:

```
answer = 42
Answer = "forty-two"
ANSWER = True

print(f"{'answer':<8} value={answer!r:<12} type={type(answer)}")
print(f"{'Answer':<8} value={Answer!r:<12} type={type(Answer)}")
print(f"{'ANSWER':<8} value={ANSWER!r:<12} type={type(ANSWER)}")
```

Possible output:

	value	type
answer	value=42	type=<class 'int'>
Answer	value='forty-two'	type=<class 'str'>
ANSWER	value=True	type=<class 'bool'>

This is legal, but it is usually a bad style choice. It makes code harder to read because the reader must notice a small visual difference that carries a real runtime consequence.

For ordinary variables, the practical convention is:

Use lowercase *snake_case* names, and do not create different variables whose only difference is letter case.

So instead of this:

```
score = 42
Score = 99
SCORE = 100
```

prefer names that state the difference explicitly:

```
current_score = 42
bonus_score = 99
max_score = 100

print(current_score)
print(bonus_score)
print(max_score)
```

This is not only prettier. It prevents a common reading error. Python will obey case-sensitive spelling exactly, but human readers often miss case-only distinctions when scanning code quickly.

At this level, the complete rule for ordinary variable names can be summarized as follows:

A Python variable name must be a valid identifier, must not be a reserved keyword, and is matched by exact spelling, including letter case.

4.1.2 Stylistic Coding Conventions

The previous subsection described what Python accepts as a legal identifier. That is only the first boundary. A name can be legal and still be a bad programming name.

Python will accept this:

```
xqz_42 = "Nobody expects the Spanish Inquisition!"
```

The name is legal, but it does not explain the role of the value. The interpreter is satisfied. The reader is not.

A programming language has two readers:

- the interpreter, which needs syntactically valid source code;
- the human programmer, who needs understandable source code.

The syntax rules protect the interpreter. Style rules protect the human reader. They do not usually change what the program does, but they strongly affect whether the program can be read, debugged, maintained, and taught.

```
line = "No soup for you."
quote_from_seinfeld = "No soup for you."

print(line)
print(quote_from_seinfeld)
```

Both names are legal. The second name carries more information. It tells the reader what kind of value the name is expected to reference.

This is especially important in Python because names do not declare fixed types. In C, a declaration such as `int count` already gives the reader one piece of type information. In Python, the name itself often carries more of the explanatory burden.

Alignment with the PEP 8 Style Guide

PEP 8 is Python's standard style guide. It is not the grammar of the language. Python does not reject a program merely because it violates PEP 8. Instead, PEP 8 describes common conventions that make Python code easier to read in a shared community.

For ordinary variable names, the most important rule is simple:

Use lowercase words separated by underscores.

This style is commonly called `snake_case`.

Poor style:

```
StudentFinalScore = 42
studentFinalScore = 42
sfs = 42
```

Preferred ordinary-variable style:

```
student_final_score = 42
```

A short comparison:

```
student_name = "Bart Simpson"
quiz_score = 42
final_score = 87.5
passed_course = True

print(f"{'student':<12} {student_name}")
print(f"{'quiz':<12} {quiz_score}")
print(f"{'final':<12} {final_score:.1f}")
print(f"{'passed':<12} {passed_course}")

print()
print(f"type(student_name): {type(student_name)}")
print(f"type(quiz_score): {type(quiz_score)}")
print(f"type(final_score): {type(final_score)}")
print(f"type(passed_course): {type(passed_course)}")
```

Possible output:

```
student      Bart Simpson
quiz         42
final        87.5
passed       True

type(student_name): <class 'str'>
type(quiz_score):   <class 'int'>
type(final_score):  <class 'float'>
type(passed_course): <class 'bool'>
```

The names do not force the types. The object types still belong to the objects. However, readable names help the programmer understand the intended role of each binding.

PEP 8 is not meant to replace judgment. A short name is acceptable when its meaning is obvious from a very small local context:

```
x = 42
y = 7

print(f"x + y = {x + y}")
```

But in ordinary program text, names should usually say what they represent:

```
current_score = 42
bonus_score = 7

print(f"total score: {current_score + bonus_score}")
```

The second version gives more information without requiring comments.

Readable Naming Conventions: Lowercase snake_case for Ordinary Variables

The standard form for ordinary Python variables is lowercase **snake_case**:

```

user_name = "Cosmo Kramer"
episode_count = 180
has_finished = False
favorite_number = 42

```

Each word is lowercase. Spaces are replaced with underscores. Digits may appear after the first character when they are part of the name.

The goal is not decoration. The goal is fast reading.

Compare:

```

n = "Homer"
a = 39
s = "Springfield"

print(f"{n} is {a} and lives in {s}.")

with:

person_name = "Homer"
person_age = 39
home_city = "Springfield"

print(f"{person_name} is {person_age} and lives in {home_city}.")

```

The second version is longer, but it is not harder. It removes guessing.

A good variable name usually answers one of these questions:

- What kind of thing is this?
- What role does it play here?
- What does this value mean in this calculation?

Example:

```

quote = "It is merely a flesh wound."
quote_length = len(quote)
quote_uppercase = quote.upper()

print(f"quote:           {quote}")
print(f"length:           {quote_length}")
print(f"uppercase:          {quote_uppercase}")

print()
print(f"type(quote):         {type(quote)}")
print(f"type(quote_length):   {type(quote_length)}")
print(f"type(quote_uppercase): {type(quote_uppercase)}")

```

Possible output:

```

quote:           It is merely a flesh wound.
length:          29
uppercase:       IT IS MERELY A FLESH WOUND.

type(quote):      <class 'str'>
type(quote_length): <class 'int'>
type(quote_uppercase): <class 'str'>

```

The names help us see the flow:

```
quote          -> original string object
quote_length   -> integer result returned by len()
quote_uppercase -> new uppercase string object
```

The name `quote_uppercase` does not modify `quote`. It names the result of calling `upper()` on the string object.

We can inspect a few relevant string capabilities:

```
quote = "No soup for you."

print(f"type(quote): {type(quote)}")

print("selected string methods:")
print(f"{'upper':<15} {'upper' in dir(quote)}")
print(f"{'lower':<15} {'lower' in dir(quote)}")
print(f"{'replace':<15} {'replace' in dir(quote)}")
print(f"{'split':<15} {'split' in dir(quote)}")

print()
print(quote.upper())
print(quote.replace("soup", "debugger"))
```

Possible output:

```
type(quote): <class 'str'>
selected string methods:
upper          True
lower          True
replace        True
split          True
```

```
NO SOUP FOR YOU.
No debugger for you.
```

The naming convention does not belong to the string object. It belongs to the source-code name that refers to the object. The object has methods such as `upper()`, `lower()`, and `replace()`. The variable name is only the programmer's chosen binding label.

Some names are too vague:

```
data = "Chuck Norris can hear sign language."
thing = len(data)
result = data.upper()
```

Better:

```
fact_text = "Chuck Norris can hear sign language."
fact_length = len(fact_text)
fact_uppercase = fact_text.upper()
```

The improved version does not change the runtime behavior. It changes the clarity of the program.

Underscore Conventions: Internal Names, Throwaway Names, and Special Method Boundaries

The underscore is legal inside Python identifiers, but it is also used by convention in several specific ways. These conventions are not all enforced by the interpreter. Some are agreements between Python programmers.

The most common use has already appeared: separating words in ordinary names.

```
favorite_quote = "These pretzels are making me thirsty."
current_value = 42
final_answer = current_value + 1

print(favorite_quote)
print(final_answer)
```

A single leading underscore usually means: this name is meant for internal use.

```
_public_title = "This name is legal, but the style says internal."

print(_public_title)
```

Python will still execute this. The leading underscore does not make the name truly private. It tells human readers: do not treat this as part of the normal public interface of a module, object, or component.

For now, at the level of simple scripts, the practical advice is:

Do not start ordinary beginner variable names with an underscore unless you are intentionally marking them as internal or temporary.

A single underscore by itself is often used as a throwaway name. It means that a value is required syntactically, but the program does not intend to use it.

Example:

```
name, _, city = ("Jerry", "ignored middle field", "New York")

print(f"name: {name}")
print(f"city: {city}")
```

Possible output:

```
name: Jerry
city: New York
```

The second value exists in the tuple, but the program deliberately ignores it. The underscore is still a real name, but by convention it says: this binding is not important.

Another common use appears in loops:

```
for _ in range(3):
    print("And now for something completely different.")
```

Possible output:

```
And now for something completely different.
And now for something completely different.
And now for something completely different.
```

The loop variable is not used. Only the repetition count matters.

A trailing underscore is often used when a good name would otherwise collide with a Python keyword.

The word `class` is a keyword, so this is invalid:

```
class = "Python 101"
```

A common replacement is:

```
class_ = "Python 101"

print(class_)
```

The trailing underscore keeps the name readable while avoiding the reserved keyword.

Double-leading and double-trailing underscores mark a different category. Names such as `__init__`, `__str__`, `__repr__`, and `__add__` are often called dunder names, meaning double-underscore names.

These names are not ordinary decoration. They are used by Python's object model for special behavior.

Even simple objects already have many of them:

```
number = 42

print(f"type(number): {type(number)}")

print("selected integer special methods:")
print(f"{'__add__':<12} {'__add__' in dir(number)}")
print(f"{'__str__':<12} {'__str__' in dir(number)}")
print(f"{'__repr__':<12} {'__repr__' in dir(number)}")
print(f"{'__eq__':<12} {'__eq__' in dir(number)}")

print()
print(number.__add__(8))
print(number.__str__())
```

Possible output:

```
type(number): <class 'int'>
selected integer special methods:
__add__      True
__str__      True
__repr__     True
__eq__       True

50
42
```

The expression:

```
number + 8
```

is connected to integer addition behavior exposed through special methods such as `__add__`. This does not mean that beginners should call these methods directly in ordinary code. The point here is only to show that double-underscore names are part of Python's deeper object protocol.

Therefore, do not invent ordinary variable names like this:

```
__score__ = 42
__student_name__ = "Bart"
```

Python may accept them as identifiers, but they visually imitate names reserved for special object behavior. For ordinary variables, use plain `snake_case`:

```
score = 42
student_name = "Bart"

print(score)
print(student_name)
```

At this point, the practical naming rules are:

- use lowercase `snake_case` for ordinary variables;
- use names that explain the value's role;
- avoid case-only distinctions;
- avoid leading underscores unless intentionally marking internal or throwaway names;
- use a trailing underscore when avoiding a keyword collision;
- do not invent double-underscore names for ordinary variables.

These rules do not make Python more powerful. They make Python programs easier to inspect. In a language where names are flexible bindings, readable naming is one of the main tools that keeps the source code understandable.

4.2 Assignment and Name Binding

Assignment is the operation that connects source-code names to runtime objects. In Python, this connection is not the same as declaring a typed storage box, as a C programmer might expect. A name is not an integer slot, a floating-point slot, or a character buffer. A name is a binding that allows the program to reach an object.

This section develops that idea through ordinary assignment, reassignment, object sharing, unpacking, augmented assignment, and deletion. The goal is not only to learn the syntax of `=`, `+=`, or `del`, but to understand what happens to names and objects when those statements execute.

The central rule is simple:

Python assignment does not copy values into variables. It binds names to objects.

This rule explains why the same name can later refer to an object of another type, why two names can refer to the same object, why rebinding one name does not automatically affect another name, and why deleting a name is not the same thing as directly destroying an object.

The examples in this section are deliberately small. Assignment is not a decorative part of Python syntax. It is one of the main places where Python's runtime model becomes visible in ordinary code.

4.2.1 Simple Assignment: Creating or Rebinding a Name

Assignment is the first operation that makes Python variables feel different from C variables. In C, a variable often suggests a typed storage location. In Python, assignment creates or changes a binding between a name and an object.

This continues the model already established earlier: Python variables are not C boxes; they are names that refer to runtime objects.

The basic assignment form is:


```
name = object_expression
```

Python first evaluates the expression on the right side. Then it binds the name on the left side to the resulting object.

```
answer = 42

print(answer)
print(type(answer))

print("selected int methods:")
print(f"{'bit_length':<15} {'bit_length' in dir(answer)}")
print(f"{'to_bytes':<15} {'to_bytes' in dir(answer)}")
print(f"{'__add__':<15} {'__add__' in dir(answer)}")

print()
print(f"bit length: {answer.bit_length()}")
print(f"answer + 8: {answer + 8}")
```

Possible output:

```
42
<class 'int'>
selected int methods:
bit_length      True
to_bytes        True
__add__         True

bit length: 6
answer + 8: 50
```

The name `answer` does not contain the integer in the C sense. It refers to an integer object. That object has a runtime type, and the type supplies operations such as addition, conversion to string form, and integer-specific methods such as `bit_length()`.

Assignment as Binding, Not Memory Copying

The assignment:

```
answer = 42
```

does not mean: copy the raw binary value 42 into a named integer box.

A better conceptual picture is:

```
answer ---> int object 42
```

The source name `answer` is a binding label. The object is the runtime value. The object has a type. The name does not have a declared type.

A simple inspection:

```
answer = 42

print(f"name:  answer")
print(f"value: {answer}")
print(f"type:  {type(answer)}")
print(f"id:    {id(answer)}")
```

Possible output:

```
name:  answer
value: 42
type:  <class 'int'>
id:    140734432946520
```

The exact `id()` value is not important. It is an implementation-level identity number for the object during that run of the program. It lets us observe whether two names refer to the same object.

Assignment can also bind a name to a string object:

```
quote = "No soup for you."

print(f"quote: {quote}")
print(f"type:  {type(quote)}")
print(f"id:    {id(quote)}")

print()
print("selected str methods:")
print(f"{'upper':<12} {'upper' in dir(quote)}")
print(f"{'replace':<12} {'replace' in dir(quote)}")
print(f"{'split':<12} {'split' in dir(quote)}")

print()
print(quote.upper())
print(quote.replace("soup", "debugger"))
```

Possible output:

```
quote: No soup for you.
type:  <class 'str'>
id:    2113631854576
```

```
selected str methods:
upper      True
replace    True
split      True
```

```
NO SOUP FOR YOU.
No debugger for you.
```

The string object supplies string operations. The name `quote` only gives us access to that object. This distinction is essential:

- the name is not the object;
- the name is not the type;
- the name is a binding to an object;
- the object carries its runtime type and behavior.

Reassignment: Moving a Name from One Object to Another

A Python name can be rebound. Rebinding means that the name stops referring to one object and starts referring to another object.

```
item = 42

print("first binding")
print(f"value: {item!r}")
print(f"type: {type(item)}")
print(f"id: {id(item)}")

print()

item = "It is merely a flesh wound."

print("second binding")
print(f"value: {item!r}")
print(f"type: {type(item)}")
print(f"id: {id(item)}")
```

Possible output:

```
first binding
value: 42
type: <class 'int'>
id: 140734432946520

second binding
value: 'It is merely a flesh wound.'
type: <class 'str'>
id: 2113631854928
```

The name `item` did not transform from an integer box into a string box. The binding changed. Conceptually:

```
first:
    item ---> int object 42

second:
    item ---> str object "It is merely a flesh wound."
```

This is why Python allows this:

```
value = 42
value = 3.14
value = "D'oh!"
value = True

print(value)
print(type(value))
```

Possible output:

```
True
<class 'bool'>
```

Only the final binding remains accessible through the name `value`. The earlier objects may still exist for a while because of interpreter internals, caching, constants, or other references, but the name `value` no longer points to them.

A slightly clearer trace:

```
thing = 42
print(f"{'step 1':<8} {thing!r:<30} {type(thing)}")

thing = "Chuck Norris can hear sign language."
print(f"{'step 2':<8} {thing!r:<30} {type(thing)}")

thing = 99.5
print(f"{'step 3':<8} {thing!r:<30} {type(thing)}")
```

Possible output:

```
step 1    42                                <class 'int'>
step 2    'Chuck Norris can hear sign language.' <class 'str'>
step 3    99.5                             <class 'float'>
```

The name stayed the same. The referenced object changed.

This is dynamic binding at the level of assignment. Python does not require the name to keep one declared type for the whole program.

Object Sharing: Binding Multiple Names to the Same Runtime Object (`a = b`)

Assignment can also bind a second name to an object that already has a name.

```
a = 42
b = a

print(f"a value: {a}")
print(f"b value: {b}")

print()
print(f"a type: {type(a)}")
print(f"b type: {type(b)}")

print()
print(f"a id: {id(a)}")
print(f"b id: {id(b)}")
print(f"a is b: {a is b}")
```

Possible output:

```
a value: 42
b value: 42

a type: <class 'int'>
b type: <class 'int'>

a id: 140734432946520
b id: 140734432946520
a is b: True
```

The assignment:

```
b = a
```

does not copy the integer object into a new independent object. It binds `b` to the same object that `a` already references.

Conceptually:

```
a ----\
      ---> int object 42
b ----/
```

The same behavior can be shown with a string:

```
line1 = "Nobody expects the Spanish Inquisition!"
line2 = line1

print(f"line1: {line1}")
print(f"line2: {line2}")

print()
print(f"type(line1): {type(line1)}")
print(f"type(line2): {type(line2)}")

print()
print(f"id(line1): {id(line1)}")
print(f"id(line2): {id(line2)}")
print(f"line1 is line2: {line1 is line2}")
```

Possible output:

```
line1: Nobody expects the Spanish Inquisition!
line2: Nobody expects the Spanish Inquisition!

type(line1): <class 'str'>
type(line2): <class 'str'>

id(line1): 2113631855760
id(line2): 2113631855760
line1 is line2: True
```

Again, there are two names but one object.

If one name is later rebound, the other name is not dragged along with it:

```
a = 42
b = a

print("before rebinding")
print(f"a = {a}, id(a) = {id(a)}")
print(f"b = {b}, id(b) = {id(b)}")
print(f"a is b: {a is b}")

print()

a = 100

print("after rebinding a")
```

```
print(f"a = {a}, id(a) = {id(a)}")
print(f"b = {b}, id(b) = {id(b)}")
print(f"a is b: {a is b}")
```

Possible output:

```
before rebinding
a = 42, id(a) = 140734432946520
b = 42, id(b) = 140734432946520
a is b: True

after rebinding a
a = 100, id(a) = 140734432948376
b = 42, id(b) = 140734432946520
a is b: False
```

The line:

```
a = 100
```

does not edit the old integer object 42. It rebinds `a` to a different integer object. The name `b` still refers to the original object.

Conceptually:

```
before:
a ----\
      ---> int object 42
b ----/

after:
a ---> int object 100

b ---> int object 42
```

This example uses integers, which are immutable objects. Immutable means that the object itself cannot be changed in place. Later, when mutable containers are introduced, the distinction between rebinding a name and mutating a shared object will become even more important.

For now, the rule is:

Assignment binds a name to an object. Assignment from another name copies the reference, not the object. Reassignment moves one name to another object; it does not move all names that previously shared the old object.

4.2.2 Multiple Assignment and Unpacking

Python assignment does not have to bind only one name at a time. A single assignment statement can bind several names from several produced values.

The simplest form is:

```
a, b = 10, 20
```

This should be read as:

Evaluate the right side first, then bind the left-side names to the resulting values.

A first inspection:

```

a, b = 10, 20

print(f"a = {a}")
print(f"b = {b}")

print()
print(f"type(a): {type(a)}")
print(f"type(b): {type(b)}")

```

Possible output:

```

a = 10
b = 20

type(a): <class 'int'>
type(b): <class 'int'>

```

The statement binds `a` to the integer object 10 and `b` to the integer object 20. It does not create an array of variables. It is still name binding, only with several names in one statement.

A more explicit version uses parentheses on the right side:

```

a, b = (10, 20)

print(f"a = {a}")
print(f"b = {b}")

```

The right side is a tuple object. Python unpacks that tuple into the names on the left.

```

pair = (10, 20)

print(f"pair: {pair}")
print(f"type(pair): {type(pair)}")

print("selected tuple methods:")
print(f"{'count':<10} {'count' in dir(pair)}")
print(f"{'index':<10} {'index' in dir(pair)}")
print(f"{'__len__':<10} {'__len__' in dir(pair)}")

print()
print(f"length: {len(pair)}")
print(f"count of 10: {pair.count(10)}")
print(f"index of 20: {pair.index(20)}")

```

Possible output:

```

pair: (10, 20)
type(pair): <class 'tuple'>
selected tuple methods:
count      True
index      True
__len__    True

length: 2
count of 10: 1
index of 20: 1

```

A tuple is an ordered collection object. For now, we only need this much: it can hold several values in order, and Python can unpack those values into several names.

Parallel Assignment: Swapping Values with `a, b = b, a`

A classic use of multiple assignment is swapping two names.

In many languages, swapping requires a temporary variable:

```
a = "Jerry"
b = "George"

temp = a
a = b
b = temp

print(f"a = {a}")
print(f"b = {b}")
```

Possible output:

```
a = George
b = Jerry
```

Python can express the same operation directly:

```
a = "Jerry"
b = "George"

print("before swap")
print(f"a = {a}")
print(f"b = {b}")

a, b = b, a

print()
print("after swap")
print(f"a = {a}")
print(f"b = {b}")
```

Possible output:

```
before swap
a = Jerry
b = George

after swap
a = George
b = Jerry
```

The important rule is that the right side is evaluated before the left side is rebound.

So this:

```
a, b = b, a
```

means:

1. read the current object referenced by `b`;
2. read the current object referenced by `a`;

3. create the pair of right-side results;
4. bind `a` to the first result;
5. bind `b` to the second result.

Conceptually:

before:

```
a ---> "Jerry"
b ---> "George"
```

right side produces:

```
("George", "Jerry")
```

after:

```
a ---> "George"
b ---> "Jerry"
```

A version with object identity:

```
a = "Nobody expects the Spanish Inquisition!"
```

```
b = "No soup for you."
```

```
print("before swap")
print(f"a id: {id(a)}")
print(f"b id: {id(b)}")
```

```
old_a_id = id(a)
old_b_id = id(b)
```

```
a, b = b, a
```

```
print()
print("after swap")
print(f"a id: {id(a)}")
print(f"b id: {id(b)}")
```

```
print()
print(f"a now has old b object: {id(a) == old_b_id}")
print(f"b now has old a object: {id(b) == old_a_id}")
```

Possible output:

```
before swap
a id: 2144421957392
b id: 2144421957776
```

```
after swap
a id: 2144421957776
b id: 2144421957392
```

```
a now has old b object: True
b now has old a object: True
```

The objects were not edited. The names were rebound.
This remains the central assignment model:

Assignment changes bindings between names and objects. It does not pour values into named storage boxes.

Sequence Unpacking and Arity Requirements

Unpacking means taking values from an ordered object and binding them to names.

A tuple can be unpacked:

```
person = ("Bart", 10, "Springfield")

name, age, city = person

print(f"name: {name}")
print(f"age: {age}")
print(f"city: {city}")

print()
print(f"type(person): {type(person)}")
print(f"type(name): {type(name)}")
print(f"type(age): {type(age)}")
print(f"type(city): {type(city)}")
```

Possible output:

```
name: Bart
age: 10
city: Springfield

type(person): <class 'tuple'>
type(name): <class 'str'>
type(age): <class 'int'>
type(city): <class 'str'>
```

A list can also be unpacked:

```
scores = [42, 87, 99]

first_score, second_score, third_score = scores

print(f"first: {first_score}")
print(f"second: {second_score}")
print(f"third: {third_score}")

print()
print(f"type(scores): {type(scores)}")
print(f"type(first_score): {type(first_score)}")
```

Possible output:

```
first: 42
second: 87
third: 99

type(scores): <class 'list'>
type(first_score): <class 'int'>
```

A list is also an ordered collection object. Unlike a tuple, a list is mutable. That distinction becomes important later. For this subsection, the point is simpler: Python can unpack ordered objects into names.

A short list inspection:

```
scores = [42, 87, 99]

print(f"scores: {scores}")
print(f"type(scores): {type(scores)}")

print("selected list methods:")
print(f"{'append':<10} {'append' in dir(scores)}")
print(f"{'count':<10} {'count' in dir(scores)}")
print(f"{'index':<10} {'index' in dir(scores)}")
print(f"{'pop':<10} {'pop' in dir(scores)}")

print()
print(f"length: {len(scores)}")
print(f"index of 87: {scores.index(87)}")
```

Possible output:

```
scores: [42, 87, 99]
type(scores): <class 'list'>
selected list methods:
append      True
count       True
index       True
pop         True

length: 3
index of 87: 1
```

Unpacking has an arity requirement. Arity means the number of values must match the number of receiving names.

This works:

```
x, y, z = [10, 20, 30]

print(x)
print(y)
print(z)
```

This fails because there are too many right-side values:

```
x, y = [10, 20, 30]
```

A safe demonstration:

```
try:
    x, y = [10, 20, 30]
except ValueError as err:
    print("unpacking failed")
    print(f"error type: {type(err)}")
    print(f"message: {err}")
```

Possible output:

```
unpacking failed
error type: <class 'ValueError'>
message: too many values to unpack (expected 2)
```

This also fails because there are too few right-side values:

```
x, y, z = [10, 20]
```

Safe demonstration:

```
try:
    x, y, z = [10, 20]
except ValueError as err:
    print("unpacking failed")
    print(f"error type: {type(err)}")
    print(f"message: {err}")
```

Possible output:

```
unpacking failed
error type: <class 'ValueError'>
message: not enough values to unpack (expected 3, got 2)
```

The rule is exact unless extended unpacking is used:

Without a starred target, the number of receiving names must equal the number of produced values.

Strings are also ordered objects, so they can be unpacked character by character:

```
a, b, c = "ABC"

print(a)
print(b)
print(c)

print()
print(f"type(a): {type(a)}")
```

Possible output:

```
A
B
C

type(a): <class 'str'>
```

This is legal, but it should be used carefully. Unpacking a string means unpacking its characters, not its words.

```
try:
    word1, word2 = "No soup"
except ValueError as err:
    print("unpacking failed")
    print(f"message: {err}")
```

Possible output:

```
unpacking failed
message: too many values to unpack (expected 2)
```

The string "No soup" has seven characters, including the space. It does not have two word-values from Python's point of view.

Extended Unpacking with the Star Target (*rest)

Extended unpacking allows one left-side target to collect several values.

The starred name receives a list.

```
first, *rest = [10, 20, 30, 40]

print(f"first: {first}")
print(f"rest: {rest}")

print()
print(f"type(first): {type(first)}")
print(f"type(rest): {type(rest)}")
```

Possible output:

```
first: 10
rest: [20, 30, 40]

type(first): <class 'int'>
type(rest): <class 'list'>
```

This is useful when the first value has a special role and the remaining values should be kept together.

Example:

```
main_quote, *other_quotes = [
    "Nobody expects the Spanish Inquisition!",
    "No soup for you.",
    "D'oh!",
    "It is merely a flesh wound."
]

print(f"main quote: {main_quote}")
print(f"other quotes: {other_quotes}")

print()
print(f"type(main_quote): {type(main_quote)}")
print(f"type(other_quotes): {type(other_quotes)}")
```

Possible output:

```
main quote: Nobody expects the Spanish Inquisition!
other quotes: ['No soup for you.', 'D'oh!', 'It is merely a flesh wound.']

type(main_quote): <class 'str'>
type(other_quotes): <class 'list'>
```

The starred target can also appear in the middle:

```
first, *middle, last = [10, 20, 30, 40, 50]

print(f"first: {first}")
print(f"middle: {middle}")
print(f"last: {last}")

print()
print(f"type(middle): {type(middle)}")
```

Possible output:

```
first: 10
middle: [20, 30, 40]
last: 50

type(middle): <class 'list'>
```

Or at the beginning:

```
*start, last = [10, 20, 30, 40]

print(f"start: {start}")
print(f"last: {last}")

print()
print(f"type(start): {type(start)}")
print(f"type(last): {type(last)}")
```

Possible output:

```
start: [10, 20, 30]
last: 40

type(start): <class 'list'>
type(last): <class 'int'>
```

Only one starred target is allowed in a single unpacking assignment. This is invalid:

```
*left, *right = [10, 20, 30, 40]
```

Python rejects it because two starred targets would make the split ambiguous. There would be no single obvious rule for deciding how many values go into `left` and how many go into `right`.

A safe syntax demonstration:

```
source_code = "*left, *right = [10, 20, 30, 40]"

try:
    compile(source_code, "<student-example>", "exec")
    print("compiled successfully")
except SyntaxError as err:
    print("syntax error")
    print(f"message: {err.msg}")
```

Possible output:

```
syntax error
message: multiple starred expressions in assignment
```

The starred target may receive an empty list:

```
first, *rest = [42]

print(f"first: {first}")
print(f"rest: {rest}")

print()
print(f"type(rest): {type(rest)}")
print(f"length of rest: {len(rest)}")
```

Possible output:

```
first: 42
rest: []

type(rest): <class 'list'>
length of rest: 0
```

This means the star target does not require at least one value. It collects whatever is left after the non-starred targets have received their required values.

However, the non-starred targets still have to be satisfied:

```
try:
    first, *middle, last = [42]
except ValueError as err:
    print("unpacking failed")
    print(f"message: {err}")
```

Possible output:

```
unpacking failed
message: not enough values to unpack (expected at least 2, got 1)
```

The rule is:

The starred target absorbs the flexible middle part, but the ordinary targets still require their own values.

For assignment reasoning, multiple assignment and unpacking do not change the basic model. They only make the binding operation wider.

```
first, *middle, last = [10, 20, 30, 40]

print(f"{'name':<10} {'value':<15} {'type':<15}")
print(f"{'first':<10} {str(first):<15} {type(first):<15}")
print(f"{'middle':<10} {str(middle):<15} {type(middle):<15}")
print(f"{'last':<10} {str(last):<15} {type(last):<15}")
```

Possible output:

name	value	type
first	10	<class 'int'>
middle	[20, 30]	<class 'list'>
last	40	<class 'int'>

The practical summary is:

Multiple assignment evaluates the right side first, then binds several names. Unpacking distributes ordered values into names. Without a star target, the number of values must match exactly. With one star target, that name receives a list containing the flexible remainder.

4.2.3 Augmented Assignment

Augmented assignment is assignment combined with an operation.

Instead of writing:

```
score = score + 10
```

Python allows:

```
score += 10
```

The second form is shorter, but it should not be misunderstood. For immutable singular values such as integers, floats, booleans, strings, and complex numbers, augmented assignment does not edit the object in place. It reads the old object, computes a new result object, and rebinds the name.

The basic forms are:

```
x += y
x -= y
x *= y
x /= y
x //= y
x %= y
x **= y
```

For now, the important conceptual form is:

```
name = name operator value
```

So:

```
count += 1
```

means approximately:

```
count = count + 1
```

The approximation is good enough for immutable singular values. Later, mutable containers will make the story more delicate.

Numeric Augmented Assignment (+, -=, *=, /=) as Read-Operate-Rebind for Immutable Singular Values

Start with a plain integer.

```
score = 42
```

```
print("before")
print(f"score: {score}")
print(f"type: {type(score)}")
print(f"id: {id(score)}")
```



```

old_id = id(score)

score += 8

print()
print("after score += 8")
print(f"score: {score}")
print(f"type: {type(score)}")
print(f"id: {id(score)}")
print(f"same object as before: {id(score) == old_id}")

```

Possible output:

```

before
score: 42
type: <class 'int'>
id: 140734432946520

after score += 8
score: 50
type: <class 'int'>
id: 140734432946776
same object as before: False

```

The exact `id()` values are not important. The important point is this: the integer object 42 was not edited into 50. Integers are immutable. Python computed the result of `42 + 8`, then rebound the name `score` to the resulting integer object.

Conceptually:

```

before:
    score ---> int object 42

operation:
    42 + 8 produces int object 50

after:
    score ---> int object 50

```

The integer object has methods and special methods that support this behavior.

```

value = 42

print(f"type(value): {type(value)}")

print("selected int methods:")
print(f"{'bit_length':<15} {'bit_length' in dir(value)}")
print(f"{'__add__':<15} {'__add__' in dir(value)}")
print(f"{'__sub__':<15} {'__sub__' in dir(value)}")
print(f"{'__mul__':<15} {'__mul__' in dir(value)}")
print(f"{'__truediv__':<15} {'__truediv__' in dir(value)}")

print()
print(f"value.bit_length(): {value.bit_length()}")
print(f"value + 8: {value + 8}")
print(f"value * 2: {value * 2}")

```

Possible output:

```
type(value): <class 'int'>
selected int methods:
bit_length      True
__add__         True
__sub__         True
__mul__         True
__truediv__     True

value.bit_length(): 6
value + 8:       50
value * 2:       84
```

The method `bit_length()` is an integer-specific method. The double-underscore methods such as `__add__`, `__sub__`, and `__mul__` are part of the object protocol that connects operators to object behavior. In normal code, we write `value + 8`, not `value.__add__(8)`.

Several numeric augmented assignments:

```
number = 42

print(f"start:      {number}")

number += 10
print(f"after +=:   {number}")

number -= 2
print(f"after -=:   {number}")

number *= 3
print(f"after *=:    {number}")

number /= 5
print(f"after /=:    {number}")

print()
print(f"final type: {type(number)}")
```

Possible output:

```
start:      42
after +=:   52
after -=:   50
after *=:   150
after /=:   30.0

final type: <class 'float'>
```

The last result is important. The operator `/` performs true division. Even when the mathematical result is whole, Python returns a floating-point object.

So this:

```
number /= 5
```

does not mean: keep `number` as an integer and divide its storage by five.

It means:

```
number = number / 5
```

If `number / 5` produces a `float`, then the name `number` is rebound to a `float` object.

A direct demonstration:

```
value = 10
```

```
print("before")
print(f"value: {value!r}")
print(f"type: {type(value)}")
```

```
value /= 2
```

```
print()
print("after value /= 2")
print(f"value: {value!r}")
print(f"type: {type(value)}")
```

Possible output:

```
before
value: 10
type: <class 'int'>

after value /= 2
value: 5.0
type: <class 'float'>
```

The name stayed the same. The referenced object changed from an integer object to a floating-point object.

For integer-preserving division, floor division uses `//`:

```
value = 10
```

```
value //= 2
```

```
print(f"value: {value!r}")
print(f"type: {type(value)}")
```

Possible output:

```
value: 5
type: <class 'int'>
```

But `//` means floor division, not exact mathematical division. With values that do not divide evenly, the result is rounded down toward negative infinity.

```
value = 11
```

```
value //= 2
```

```
print(f"value: {value}")
print(f"type: {type(value)}")
```

Possible output:

```
value: 5
type:  <class 'int'>
```

Floating-point values behave similarly in the assignment model. The old object is not edited. A new numeric result is produced, and the name is rebound.

```
price = 19.99

print("before")
print(f"price: {price}")
print(f"type:  {type(price)}")
print(f"id:    {id(price)}")

old_id = id(price)

price *= 2

print()
print("after price *= 2")
print(f"price: {price}")
print(f"type:  {type(price)}")
print(f"id:    {id(price)}")
print(f"same object as before: {id(price) == old_id}")
```

Possible output:

```
before
price: 19.99
type:  <class 'float'>
id:    2144421961872

after price *= 2
price: 39.98
type:  <class 'float'>
id:    2144421962416
same object as before: False
```

A float object also exposes operator-related methods.

```
price = 19.99

print(f"type(price): {type(price)}")

print("selected float methods:")
print(f"{'as_integer_ratio':<20} {'as_integer_ratio' in dir(price)}")
print(f"{'is_integer':<20} {'is_integer' in dir(price)}")
print(f"{'__add__':<20} {'__add__' in dir(price)}")
print(f"{'__mul__':<20} {'__mul__' in dir(price)}")

print()
print(f"price.is_integer(): {price.is_integer()}")
print(f"price.as_integer_ratio(): {price.as_integer_ratio()}")
```

Possible output:

```

type(price): <class 'float'>
selected float methods:
as_integer_ratio      True
is_integer            True
__add__              True
__mul__              True

price.is_integer(): False
price.as_integer_ratio(): (5626684784442573, 281474976710656)

```

The strange-looking ratio is a reminder that many decimal-looking floating-point values are stored as binary approximations. This will matter more in the later floating-point subsection.

Augmented assignment also works with strings, but strings are immutable too.

```

line = "D'oh"

print("before")
print(f"line: {line!r}")
print(f"type: {type(line)}")
print(f"id: {id(line)}")

old_id = id(line)

line += "!"

print()
print("after line += '!')
print(f"line: {line!r}")
print(f"type: {type(line)}")
print(f"id: {id(line)}")
print(f"same object as before: {id(line) == old_id}")

```

Possible output:

```

before
line: "D'oh"
type: <class 'str'>
id: 2144421959120

after line += '!'
line: "D'oh!"
type: <class 'str'>
id: 2144421959280
same object as before: False

```

The original string object was not edited. A new string object was produced.
For immutable singular values, the safe rule is:

Augmented assignment reads the current object, performs the operation, and rebinds the name to the result.

Mutation vs. Rebinding Preview for Later Mutable Containers

The previous rule is reliable for immutable singular values. However, Python also has mutable objects. A mutable object can be changed in place.

Lists are mutable containers. They will be studied properly later, but they are useful here as a preview because they show why augmented assignment must be understood carefully.

First, inspect a list object:

```
scores = [10, 20, 30]

print(f"scores: {scores}")
print(f"type(scores): {type(scores)}")

print("selected list methods:")
print(f"{'append':<12} {'append' in dir(scores)}")
print(f"{'extend':<12} {'extend' in dir(scores)}")
print(f"{'pop':<12} {'pop' in dir(scores)}")
print(f"{'__iadd__':<12} {'__iadd__' in dir(scores)}")
```

Possible output:

```
scores: [10, 20, 30]
type(scores): <class 'list'>
selected list methods:
append      True
extend      True
pop          True
__iadd__     True
```

The method `append()` adds one element to the existing list. The method `extend()` adds several elements. The special method `__iadd__` is connected to in-place addition behavior for lists.

Now compare rebinding with mutation.

```
numbers = [10, 20]
alias = numbers

print("before")
print(f"numbers: {numbers}")
print(f"alias: {alias}")
print(f"id(numbers): {id(numbers)}")
print(f"id(alias): {id(alias)}")
print(f"numbers is alias: {numbers is alias}")

numbers += [30]

print()
print("after numbers += [30]")
print(f"numbers: {numbers}")
print(f"alias: {alias}")
print(f"id(numbers): {id(numbers)}")
print(f"id(alias): {id(alias)}")
print(f"numbers is alias: {numbers is alias}")
```

Possible output:

```
before
numbers: [10, 20]
alias: [10, 20]
id(numbers): 2144421972096
```

```

id(alias): 2144421972096
numbers is alias: True

after numbers += [30]
numbers: [10, 20, 30]
alias: [10, 20, 30]
id(numbers): 2144421972096
id(alias): 2144421972096
numbers is alias: True

```

Here, the list object itself changed. Both names still refer to the same list object, and both names see the changed content.

Conceptually:

```

before:
  numbers ----\
                  ---> list object [10, 20]
  alias  ----/

after numbers += [30]:
  numbers ----\
                  ---> same list object [10, 20, 30]
  alias  ----/

```

This is mutation, not simple rebinding.

Now compare with ordinary + followed by assignment:

```

numbers = [10, 20]
alias = numbers

print("before")
print(f"numbers: {numbers}")
print(f"alias: {alias}")
print(f"id(numbers): {id(numbers)}")
print(f"id(alias): {id(alias)}")

numbers = numbers + [30]

print()
print("after numbers = numbers + [30]")
print(f"numbers: {numbers}")
print(f"alias: {alias}")
print(f"id(numbers): {id(numbers)}")
print(f"id(alias): {id(alias)}")
print(f"numbers is alias: {numbers is alias}")

```

Possible output:

```

before
numbers: [10, 20]
alias: [10, 20]
id(numbers): 2144421972096
id(alias): 2144421972096

after numbers = numbers + [30]

```

```

numbers: [10, 20, 30]
alias:   [10, 20]
id(numbers): 2144421972288
id(alias):   2144421972096
numbers is alias: False

```

This time, the expression:

```
numbers + [30]
```

created a new list object. Then the name `numbers` was rebound to that new object. The name `alias` still points to the original list.

Conceptually:

before:

```

numbers ----\
              ---> list object [10, 20]
alias  ----/

```

after `numbers = numbers + [30]`:

```

numbers ---> new list object [10, 20, 30]

alias  ---> old list object [10, 20]

```

So for mutable containers, these two forms can differ:

```

numbers += [30]
numbers = numbers + [30]

```

For immutable singular values, they usually behave conceptually the same: compute a result and rebound the name.

For mutable containers, augmented assignment may mutate the existing object in place.

This is only a preview. The deeper rule will be developed later when lists, dictionaries, sets, and shared references are studied properly.

For now, keep this practical distinction:

- `x = x + y` normally computes a result object and binds `x` to that result.
- `x += y` asks the object whether it can update itself in place.
- immutable objects cannot be updated in place, so the name is rebound to a result object;
- mutable objects may update themselves in place, so other names sharing the same object may see the change.

The beginner-safe rule is:

For numbers and strings, read augmented assignment as read-operate-rebind. For mutable containers, do not assume this; check whether the operation mutates the object in place.

4.2.4 Deleting Names

Assignment creates or changes a binding between a name and an object. The statement `del` can remove such a binding.

The simplest form is:

```
del name
```


This does not mean: destroy the object immediately. It means: remove this name from the current namespace.

Start with an ordinary binding:

```
quote = "No soup for you."

print(f"quote: {quote}")
print(f"type(quote): {type(quote)}")
print(f"id(quote): {id(quote)}")
```

Possible output:

```
quote: No soup for you.
type(quote): <class 'str'>
id(quote): 2168389812912
```

The name `quote` refers to a string object. After `del quote`, that name no longer exists in the current namespace.

Removing a Binding with `del`

The statement `del` removes a name binding.

```
line = "Nobody expects the Spanish Inquisition!"

print("before del")
print(f"line: {line}")
print(f"type(line): {type(line)}")

del line

print("after del")
print(line)
```

This program does not finish normally. After `del line`, the name `line` is no longer bound. Trying to read it raises a `NameError`.

A safer version catches the error:

```
line = "Nobody expects the Spanish Inquisition!"

print("before del")
print(f"line: {line}")
print(f"type(line): {type(line)}")

del line

print()
print("after del")

try:
    print(line)
except NameError as err:
    print("name lookup failed")
    print(f"error object: {err}")
    print(f"type(err): {type(err)}")
```

```

print()
print("selected NameError methods/attributes:")
print(f"{'args':<12} {'args' in dir(err)}")
print(f"{'add_note':<12} {'add_note' in dir(err)}")
print(f"{'with_traceback':<12} {'with_traceback' in dir(err)}")

```

Possible output:

```

before del
line: Nobody expects the Spanish Inquisition!
type(line): <class 'str'>

```

```

after del
name lookup failed
error object: name 'line' is not defined
type(err):    <class 'NameError'>

```

```

selected NameError methods/attributes:
args          True
add_note      True
with_traceback True

```

The important point is not the exception syntax. The important point is the binding change:

```

before:
line ---> str object "Nobody expects the Spanish Inquisition!"

```

```

after:
line no longer exists as a usable name here

```

The object and the name are different things. `del line` deletes the binding named `line`. It does not mean that the programmer has directly reached into memory and destroyed a string object.

A name can also be rebound after deletion:

```
fact = "Chuck Norris counted to infinity. Twice."
```

```

print(f"fact: {fact}")
print(f"type(fact): {type(fact)}")

```

```
del fact
```

```
fact = 42
```

```

print()
print("after deletion and new assignment")
print(f"fact: {fact}")
print(f"type(fact): {type(fact)}")

```

Possible output:

```

fact: Chuck Norris counted to infinity. Twice.
type(fact): <class 'str'>

```

```

after deletion and new assignment
fact: 42
type(fact): <class 'int'>

```

This is a new binding using the same source-code name. The old binding was removed. Then assignment created a new binding.

Conceptually:

```
step 1:
    fact ---> str object "Chuck Norris counted to infinity. Twice."

step 2:
    del fact
    fact is not bound

step 3:
    fact ---> int object 42
```

The word `del` is therefore about removing a reference from a namespace. It is not the same operation as setting a name to `None`.

Compare:

```
value = "D'oh!"
value = None

print(f"value: {value}")
print(f"type(value): {type(value)}")
```

Possible output:

```
value: None
type(value): <class 'NoneType'>
```

Here the name still exists. It is bound to the singleton object `None`.
With `del`, the name itself is removed:

```
value = "D'oh!"
del value

try:
    print(value)
except NameError as err:
    print(f"cannot read value: {err}")
```

Possible output:

```
cannot read value: name 'value' is not defined
```

So the difference is:

```
value = None    -> name exists and refers to None
del value       -> name no longer exists here
```

For ordinary beginner code, `del` is not used often with simple variables. It is more common later with container elements, module cleanup, or cases where the programmer deliberately wants to remove a name.

Name Deletion, Object Reachability, and Possible Reference Count Changes

Deleting one name does not necessarily make the object unreachable. If another name still refers to the same object, the object is still usable.

```
a = "It is merely a flesh wound."
b = a
```

```
print("before del a")
print(f"a: {a}")
print(f"b: {b}")
print(f"id(a): {id(a)}")
print(f"id(b): {id(b)}")
print(f"a is b: {a is b}")
```

```
del a
```

```
print()
print("after del a")
print(f"b: {b}")
print(f"id(b): {id(b)}")
```

Possible output:

```
before del a
a: It is merely a flesh wound.
b: It is merely a flesh wound.
id(a): 2168389813296
id(b): 2168389813296
a is b: True
```

```
after del a
b: It is merely a flesh wound.
id(b): 2168389813296
```

Before deletion:

```
a ----\
      ---> str object "It is merely a flesh wound."
b ----/
```

After del a:

```
b ---> str object "It is merely a flesh wound."
```

The object is still reachable through **b**. Therefore, it cannot be treated as gone. This is the key rule:

An object can remain alive as long as it is reachable from at least one active reference.

In CPython, the main Python implementation, object lifetime is strongly connected to reference counting. A reference count is the number of active references that CPython is currently tracking for an object. When a binding is removed, the object's reference count may decrease.

This can be observed with the standard `sys` module:

```
import sys

quote = "These pretzels are making me thirsty."

print(f"quote: {quote}")
print(f"type(quote): {type(quote)}")

print()
print(f"refcount through quote: {sys.getrefcount(quote)}")

alias = quote

print()
print("after alias = quote")
print(f"quote is alias: {quote is alias}")
print(f"refcount through quote: {sys.getrefcount(quote)}")

del alias

print()
print("after del alias")
print(f"refcount through quote: {sys.getrefcount(quote)}")
```

Possible output:

```
quote: These pretzels are making me thirsty.
type(quote): <class 'str'>

refcount through quote: 4

after alias = quote
quote is alias: True
refcount through quote: 5

after del alias
refcount through quote: 4
```

The exact numbers should not be memorized. They depend on interpreter details, temporary references, constants, and how the code is executed. Also, `sys.getrefcount()` itself temporarily receives the object as an argument, so the displayed count includes that temporary reference.

The useful observation is the direction:

- creating another binding can increase the reference count;
- deleting that binding can decrease the reference count;
- the object remains reachable while at least one usable reference remains.

A clearer example uses a list because lists are normally not reused as constants in the same way small integers or some strings may be.

```
import sys

items = ["spam", "eggs", 42]
```

```

print(f"items: {items}")
print(f"type(items): {type(items)}")

print("selected list methods:")
print(f"{'append':<12} {'append' in dir(items)}")
print(f"{'extend':<12} {'extend' in dir(items)}")
print(f"{'pop':<12} {'pop' in dir(items)}")

print()
print(f"initial refcount: {sys.getrefcount(items)}")

same_items = items

print()
print("after same_items = items")
print(f"items is same_items: {items is same_items}")
print(f"refcount: {sys.getrefcount(items)}")

del same_items

print()
print("after del same_items")
print(f"refcount: {sys.getrefcount(items)}")

```

Possible output:

```

items: ['spam', 'eggs', 42]
type(items): <class 'list'>
selected list methods:
append      True
extend      True
pop         True

initial refcount: 2

after same_items = items
items is same_items: True
refcount: 3

after del same_items
refcount: 2

```

Again, the exact count is not the lesson. The lesson is that bindings participate in reachability.

If the last reachable name is deleted, the program can no longer access the object through that name:

```

items = ["spam", "eggs", 42]

print(f"items before del: {items}")

del items

try:
    print(items)
except NameError as err:

```

```
print("items is no longer a usable name")
print(f"message: {err}")
```

Possible output:

```
items before del: ['spam', 'eggs', 42]
items is no longer a usable name
message: name 'items' is not defined
```

At that point, if no other references exist, CPython can usually reclaim the object immediately by reference counting. Other Python implementations may use different memory-management strategies. Even in CPython, object lifetime can be affected by reference cycles, temporary references, exception traces, interned constants, and implementation details.

Therefore the careful rule is:

`del` removes a binding. Removing a binding may reduce an object's reference count. If the object becomes unreachable, the interpreter may reclaim it. The programmer should not read `del name` as a direct memory-free instruction.

For this chapter, the practical mental model is enough:

```
assignment  -> create or change a name-to-object binding
reassignment -> move a name to another object
a = b        -> bind another name to the same object
del name     -> remove one name-to-object binding
```

4.3 Singular Data Domains and Operator Precedence

After names and assignment, the next question is what kind of objects those names can reach. This section introduces Python's singular built-in data domains: integers, floats, complex numbers, booleans, and `None`. These are not containers. Each one represents one direct value-domain at a time: a whole number, an approximate real number, a complex number, a truth value, or absence.

The section also connects these objects to operators. In Python, arithmetic expressions are not performed on raw C-style storage cells. Operators are dispatched to object behavior, and the result is another object with its own runtime type. This is why `10 / 2` produces a `float`, why `2 ** 100` remains an exact `int`, and why `3 + 4j` belongs to the built-in `complex` domain.

Finally, the section explains operator precedence. Python must decide how an expression such as `-2 ** 2` or `2 + 3 * 4` is grouped before it can execute the operations. Parentheses are therefore not just visual decoration; they are the programmer's explicit way of controlling the expression tree.

The goal is to make numeric and singular-value behavior predictable: what object is created, what type it has, what operation is dispatched, and how Python decides the order of evaluation.

4.3.1 The Integer Representation Architecture (`int`)

Python's `int` type represents whole numbers: negative integers, zero, and positive integers. Unlike a typical C `int`, Python's `int` is not a fixed-width storage cell such as 32 bits or 64 bits. It is a runtime object that can grow as needed.

A first inspection:

```
answer = 42

print(f"value: {answer}")
print(f"type: {type(answer)}")
print(f"id: {id(answer)}")
```

```

print()
print("selected int methods:")
print(f"{'bit_length':<18} {'bit_length' in dir(answer)}")
print(f"{'bit_count':<18} {'bit_count' in dir(answer)}")
print(f"{'to_bytes':<18} {'to_bytes' in dir(answer)}")
print(f"{'__add__':<18} {'__add__' in dir(answer)}")
print(f"{'__mul__':<18} {'__mul__' in dir(answer)}")

print()
print(f"bit_length(): {answer.bit_length()}")
print(f"bit_count(): {answer.bit_count()}")

```

Possible output:

```

value: 42
type:  <class 'int'>
id:    140734432946520

```

```

selected int methods:
bit_length      True
bit_count       True
to_bytes        True
__add__         True
__mul__         True

```

```

bit_length(): 6
bit_count():  3

```

The method `bit_length()` reports how many binary digits are needed to represent the magnitude of the integer. The method `bit_count()` reports how many binary 1 bits appear in the integer's absolute value.

For example, 42 is 101010 in base two. That takes six bits and contains three 1 bits.

```

number = 42

print(f"decimal: {number}")
print(f"binary:   {bin(number)}")
print(f"bits needed: {number.bit_length()}")
print(f"one bits:     {number.bit_count()}")

```

Possible output:

```

decimal: 42
binary:  0b101010
bits needed: 6
one bits:   3

```

This is already different from C thinking. In C, asking how many bits are needed for the value is not the same as asking how many bits the variable has. A C `int` may occupy 32 bits even when it stores the small value 42. In Python, the integer object has a representation that scales with the value.

Arbitrary-Precision Math Engine: Eliminating Fixed-Width Integer Overflow Boundaries

In many C programs, integer arithmetic is constrained by the width of the selected integer type. A signed 32-bit integer has a maximum positive value of 2147483647. Adding past that boundary does not produce a larger mathematical integer inside the same type.

Python's `int` does not behave like that. It grows to represent larger whole numbers.


```

small_limit = 2_147_483_647
bigger_value = small_limit + 1

print(f"small_limit: {small_limit}")
print(f"bigger_value: {bigger_value}")

print()
print(f"type(small_limit): {type(small_limit)}")
print(f"type(bigger_value): {type(bigger_value)}")

print()
print(f"small_limit bits: {small_limit.bit_length()}")
print(f"bigger_value bits: {bigger_value.bit_length()}")

```

Possible output:

```

small_limit: 2147483647
bigger_value: 2147483648

type(small_limit): <class 'int'>
type(bigger_value): <class 'int'>

small_limit bits: 31
bigger_value bits: 32

```

The value passes the usual signed 32-bit positive boundary, but the type remains `int`. Python does not silently wrap the value back into a negative number.

A much larger example:

```

huge_number = 2 ** 200

print(f"huge_number: {huge_number}")
print(f"type: {type(huge_number)}")
print(f"bit length: {huge_number.bit_length()}")
print(f"decimal digits: {len(str(huge_number))}")

```

Possible output:

```

huge_number: 1606938044258990275541962092341162602522202993782792835301376
type: <class 'int'>
bit length: 201
decimal digits: 61

```

The expression `2 ** 200` produces a normal Python integer object. It is not a separate “big integer” type at the Python level. In Python 3, small integers and enormous integers both have type `int`.

```

small = 42
large = 10 ** 100

print(f"small type: {type(small)}")
print(f"large type: {type(large)}")

print()
print(f"small bit length: {small.bit_length()}")
print(f"large bit length: {large.bit_length()}")

```


Dynamic Bit Scaling: CPython's Digit-Array Structural Allocation for High-Magnitude Values

At the Python-language level, the rule is simple: `int` supports arbitrary precision. At the CPython implementation level, a Python integer is represented internally using a variable-size structure.

CPython does not store a huge integer as one impossible machine word. It stores the integer's magnitude across several internal chunks. These chunks are often called digits in the CPython source code, but they are not decimal digits. They are base-2ⁿ chunks used by the implementation.

Python exposes some of this implementation information through `sys.int_info`.

```
import sys

print(sys.int_info)
print()
print(f"type(sys.int_info): {type(sys.int_info)}")

print()
print(f"bits per internal digit: {sys.int_info.bits_per_digit}")
print(f"size of internal digit: {sys.int_info.sizeof_digit} bytes")
```

Possible output on a common 64-bit CPython build:

```
sys.int_info(bits_per_digit=30, sizeof_digit=4, default_max_str_digits=4300, str_digits_check=
type(sys.int_info): <class 'sys.int_info'>

bits per internal digit: 30
size of internal digit: 4 bytes
```

The exact values are implementation details, but the idea is stable for CPython: larger integers require more internal integer digits.

A useful analogy is not a C scalar variable but an expandable sequence of number chunks:

```
small integer:
    one or very few internal chunks

large integer:
    many internal chunks
```

We can observe the growth indirectly with `bit_length()` and `sys.getsizeof()`.

```
import sys

numbers = [
    0,
    42,
    2 ** 30 - 1,
    2 ** 30,
    2 ** 60,
    2 ** 120,
    2 ** 240,
]

print(f"{'value expression':<16} {'bits':>6} {'bytes':>6}")
print("-" * 32)
```

```

for number in numbers:
    if number == 0:
        label = "0"
    else:
        label = f"2**{number.bit_length() - 1}"

    print(f"{label:<16} {number.bit_length():>6} {sys.getsizeof(number):>6}")

```

Possible output on one CPython build:

value	expression	bits	bytes
0		0	28
2**5		6	28
2**29		30	28
2**30		31	32
2**60		61	36
2**120		121	44
2**240		241	60

The exact byte counts may differ between builds and Python versions. The pattern is the important part: as the integer needs more bits, the object can require more memory.

This is not the C model:

C-like fixed-width intuition:

```

int x;
x always has the same storage width

```

Python int intuition:

```

x is a name
x refers to an int object
the int object representation can scale with the value

```

The name itself is still only a binding. When arithmetic creates a larger result, Python binds the name to a result object capable of representing that value.

```

import sys

value = 42

print("start")
print(f"value: {value}")
print(f"bits: {value.bit_length()}")
print(f"bytes: {sys.getsizeof(value)}")
print(f"id: {id(value)}")

old_id = id(value)

value = value ** 20

print()
print("after value = value ** 20")
print(f"value: {value}")
print(f"bits: {value.bit_length()}")

```

```
print(f"bytes: {sys.getsizeof(value)}")
print(f"id:      {id(value)}")
print(f"same object as before: {id(value) == old_id}")
```

Possible output:

```
start
value: 42
bits: 6
bytes: 28
id:      140734432946520

after value = value ** 20
value: 278218429446951548637196401
bits: 88
bytes: 36
id:      2168389813520
same object as before: False
```

The integer object 42 was not expanded in place. The expression produced a new integer object. The name `value` was rebound to that object.

This connects the integer architecture back to assignment:

Python's integer arithmetic creates exact integer result objects. Assignment then binds names to those objects.

For teaching purposes, the most useful CPython facts are:

- a Python `int` is an object, not a raw C `int`;
- its magnitude can require one or more internal digits;
- the number of required internal digits grows with the number's bit length;
- arithmetic results are exact whole-number objects unless another numeric type is introduced;
- the precise internal layout is an implementation detail, not source-level Python syntax.

Integer Literals: Decimal, Binary, Octal, and Hexadecimal Notation

An integer literal is a way of writing an integer value directly in source code.

The normal form is decimal:

```
count = 42
year = 2026
temperature = -5

print(count)
print(year)
print(temperature)
```

Possible output:

```
42
2026
-5
```

The minus sign in `-5` is not part of the integer literal in the deepest lexical sense. It is a unary operator applied to the integer literal `5`. This will matter more when operator precedence is discussed later.

Python also supports base-specific integer literal notation:

- decimal: ordinary digits, such as `42`;
- binary: prefix `0b`, such as `0b101010`;
- octal: prefix `0o`, such as `0o52`;
- hexadecimal: prefix `0x`, such as `0x2a`.

All four examples below create the same integer value:

```
decimal_value = 42
binary_value = 0b101010
octal_value = 0o52
hex_value = 0x2a

print(f"decimal:      {decimal_value}")
print(f"binary src:   {binary_value}")
print(f"octal src:     {octal_value}")
print(f"hex src:       {hex_value}")

print()
print(f"all equal: {decimal_value == binary_value == octal_value == hex_value}")

print()
print(f"type(decimal_value): {type(decimal_value)}")
print(f"type(binary_value): {type(binary_value)}")
print(f"type(octal_value):   {type(octal_value)}")
print(f"type(hex_value):     {type(hex_value)}")
```

Possible output:

```
decimal:      42
binary src:   42
octal src:    42
hex src:      42

all equal: True

type(decimal_value): <class 'int'>
type(binary_value):  <class 'int'>
type(octal_value):   <class 'int'>
type(hex_value):     <class 'int'>

42          -> int object with value 42
0b101010    -> int object with value 42
0o52        -> int object with value 42
0x2a        -> int object with value 42
```

The base notation changes how the value is written in source code. It does not create different integer types.

Conceptually:

Python can also convert an existing integer into string representations in these bases:

```
number = 42

print(f"decimal string:      {str(number)}")
print(f"binary string:      {bin(number)}")
print(f"octal string:       {oct(number)}")
print(f"hexadecimal string: {hex(number)}")
```

Possible output:

```
decimal string:      42
binary string:       0b101010
octal string:        0o52
hexadecimal string: 0x2a
```

The functions `bin()`, `oct()`, and `hex()` return strings, not integers.

```
number = 42

binary_text = bin(number)
octal_text = oct(number)
hex_text = hex(number)

print(f"binary_text: {binary_text}")
print(f"octal_text:   {octal_text}")
print(f"hex_text:     {hex_text}")

print()
print(f"type(binary_text): {type(binary_text)}")
print(f"type(octal_text):  {type(octal_text)}")
print(f"type(hex_text):    {type(hex_text)}")
```

Possible output:

```
binary_text: 0b101010
octal_text:  0o52
hex_text:    0x2a

type(binary_text): <class 'str'>
type(octal_text):  <class 'str'>
type(hex_text):    <class 'str'>
```

To parse text as an integer, use `int()` with a base argument:

```
binary_text = "101010"
octal_text = "52"
hex_text = "2a"

from_binary = int(binary_text, 2)
from_octal = int(octal_text, 8)
from_hex = int(hex_text, 16)

print(f"from binary: {from_binary}")
print(f"from octal:   {from_octal}")
```

```

print(f"from hex:      {from_hex}")

print()
print(f"type(from_binary): {type(from_binary)}")
print(f"type(from_octal):  {type(from_octal)}")
print(f"type(from_hex):    {type(from_hex)}")

```

Possible output:

```

from binary: 42
from octal:  42
from hex:    42

type(from_binary): <class 'int'>
type(from_octal):  <class 'int'>
type(from_hex):    <class 'int'>

```

The prefix may also be included if base 0 is used. In that mode, Python detects the base from the prefix:

```

texts = ["42", "0b101010", "0o52", "0x2a"]

for text in texts:
    number = int(text, 0)
    print(f"{text:<10} -> {number:<3} type={type(number)}")

```

Possible output:

```

42          -> 42  type=<class 'int'>
0b101010    -> 42  type=<class 'int'>
0o52        -> 42  type=<class 'int'>
0x2a        -> 42  type=<class 'int'>

```

Integer literals may also use underscores for readability:

```

population = 1_000_000
mask = 0b1111_0000
hex_color = 0xff_aa_00

print(f"population: {population}")
print(f"mask:        {mask}")
print(f"hex_color:    {hex_color}")

print()
print(f"type(population): {type(population)}")
print(f"type(mask):       {type(mask)}")
print(f"type(hex_color):  {type(hex_color)}")

```

Possible output:

```

population: 1000000
mask:       240
hex_color:  16755200

type(population): <class 'int'>
type(mask):       <class 'int'>
type(hex_color):  <class 'int'>

```


The underscores are not part of the value. They only make the source code easier to read.

A final compact table:

```
number = 42
```

```
print(f"{'form':<14} {'text':<12} {'value':<8} {'type'}")
print("-" * 50)

print(f"{'decimal':<14} {'42':<12} {42:<8} {type(42)}")
print(f"{'binary':<14} {'0b101010':<12} {0b101010:<8} {type(0b101010)}")
print(f"{'octal':<14} {'0o52':<12} {0o52:<8} {type(0o52)}")
print(f"{'hexadecimal':<14} {'0x2a':<12} {0x2a:<8} {type(0x2a)}")
```

Possible output:

form	text	value	type
decimal	42	42	<class 'int'>
binary	0b101010	42	<class 'int'>
octal	0o52	42	<class 'int'>
hexadecimal	0x2a	42	<class 'int'>

The practical summary is:

Python's `int` is an exact whole-number object with arbitrary precision. The source code may write that value in decimal, binary, octal, or hexadecimal notation, but the resulting runtime object is still an `int`.

4.3.2 The Floating-Point Representation Architecture (`float`)

Python's `float` type represents real-number approximations, not exact real numbers. This distinction is essential. An `int` can represent arbitrarily large whole numbers exactly. A `float` represents numbers using a fixed-size binary format, so many decimal fractions can only be approximated.

A first inspection:

```
price = 19.99

print(f"value: {price}")
print(f"type: {type(price)}")
print(f"id: {id(price)}")

print()
print("selected float methods:")
print(f"{'as_integer_ratio':<20} {'as_integer_ratio' in dir(price)}")
print(f"{'is_integer':<20} {'is_integer' in dir(price)}")
print(f"{'hex':<20} {'hex' in dir(price)}")
print(f"{'__add__':<20} {'__add__' in dir(price)}")
print(f"{'__truediv__':<20} {'__truediv__' in dir(price)}")

print()
print(f"is_integer(): {price.is_integer()}")
print(f"as_integer_ratio(): {price.as_integer_ratio()}")
print(f"hex(): {price.hex()}")
```

Possible output:

```

value: 19.99
type:  <class 'float'>
id:    2214135699408

selected float methods:
as_integer_ratio      True
is_integer            True
hex                   True
__add__               True
__truediv__           True

is_integer():         False
as_integer_ratio():   (5626684784442573, 281474976710656)
hex():                0x1.3fd70a3d70a3dp+4

```

The method `as_integer_ratio()` exposes the exact fraction represented by the stored floating-point value. The method `hex()` exposes a hexadecimal form of the binary floating-point value. These are useful because they show that 19.99 is not stored as the exact decimal fraction that the programmer wrote.

Architectural Realities: Mapping Python Floats to C Doubles via the IEEE 754 Double-Precision Binary Standard

In CPython, the ordinary Python `float` is implemented using the platform C `double`. On modern machines, that usually means IEEE 754 double-precision binary floating point.

This format uses a fixed amount of storage. It does not grow like Python's `int`. A float has limited precision and limited exponent range.

Python exposes the main limits through `sys.float_info`:

```

import sys

print(sys.float_info)

print()
print(f"max:      {sys.float_info.max}")
print(f"min:      {sys.float_info.min}")
print(f"epsilon:    {sys.float_info.epsilon}")
print(f"mantissa digits in base 2: {sys.float_info.mant_dig}")

```

Possible output:

```

sys.float_info(max=1.7976931348623157e+308, min=2.2250738585072014e-308,
dig=15, mant_dig=53, epsilon=2.220446049250313e-16, radix=2, rounds=1)

max:      1.7976931348623157e+308
min:      2.2250738585072014e-308
epsilon:  2.220446049250313e-16
mantissa digits in base 2: 53

```

The exact formatting may vary, but the important values are stable on normal CPython builds:

- `radix = 2`: the representation is binary;
- `mant_dig = 53`: the precision is about 53 binary digits;
- `dig = 15`: about 15 decimal digits can be represented reliably;

- **epsilon**: the distance between 1.0 and the next representable float above it.

This means that `float` is not an arbitrary-precision decimal type. It is a fixed-precision binary approximation type.

```
small = 42.0
large = 10.0 ** 100

print(f"small: {small}")
print(f"large: {large}")

print()
print(f"type(small): {type(small)}")
print(f"type(large): {type(large)}")

print()
print(f"small.hex(): {small.hex()}")
print(f"large.hex(): {large.hex()}")
```

Possible output:

```
small: 42.0
large: 1e+100

type(small): <class 'float'>
type(large): <class 'float'>

small.hex(): 0x1.5000000000000p+5
large.hex(): 0x1.249ad2594c37dp+332
```

Both values have type `float`. The second value is much larger, but the representation is still constrained by the same fixed floating-point architecture.

This also means that float arithmetic can overflow:

```
import sys

big = sys.float_info.max

print(f"big: {big}")

too_big = big * 2

print(f"too_big: {too_big}")
print(f"type(too_big): {type(too_big)}")
```

Possible output:

```
big: 1.7976931348623157e+308
too_big: inf
type(too_big): <class 'float'>
```

Unlike `int`, a `float` cannot keep allocating more storage to represent a larger exact value. Once the representable range is exceeded, the result may become infinity.

Machine Epsilon and Precision Loss: The Pitfalls of Representing Base-10 Fractions in Base-2 Memory Buffers

The most famous floating-point surprise is this:

```
total = 0.1 + 0.2

print(f"total: {total}")
print(f"total == 0.3: {total == 0.3}")
```

Possible output:

```
total: 0.30000000000000004
total == 0.3: False
```

This is not a Python bug. It is a consequence of representing decimal fractions in binary floating point.

Fractions such as 0.5, 0.25, and 0.125 are easy in binary because they are powers of two in the denominator:

```
values = [0.5, 0.25, 0.125]

for value in values:
    print(f"{value:<8} ratio={value.as_integer_ratio()}")
```

Possible output:

```
0.5      ratio=(1, 2)
0.25     ratio=(1, 4)
0.125    ratio=(1, 8)
```

But 0.1 does not have a finite binary expansion, just as $1/3$ does not have a finite decimal expansion.

```
value = 0.1

print(f"value: {value}")
print(f"ratio: {value.as_integer_ratio()}")
print(f"hex:    {value.hex()}")
```

Possible output:

```
value: 0.1
ratio: (3602879701896397, 36028797018963968)
hex:    0x1.999999999999ap-4
```

The source text 0.1 is converted to the closest representable binary floating-point value. That stored value is very close to one tenth, but it is not exactly one tenth.

Machine epsilon gives a scale for this limitation around 1.0.

```
import sys

eps = sys.float_info.epsilon

print(f"epsilon: {eps}")
print(f"1.0 + epsilon:      {1.0 + eps}")
print(f"1.0 + epsilon / 2: {1.0 + eps / 2}")

print()
print(f"(1.0 + epsilon) == 1.0:      {(1.0 + eps) == 1.0}")
print(f"(1.0 + epsilon / 2) == 1.0: {(1.0 + eps / 2) == 1.0}")
```

Possible output:

```
epsilon: 2.220446049250313e-16
1.0 + epsilon:      1.0000000000000002
1.0 + epsilon / 2: 1.0

(1.0 + epsilon) == 1.0:      False
(1.0 + epsilon / 2) == 1.0: True
```

The second addition is too small to change the stored value at that scale. This is the concrete meaning of limited precision: not every tiny mathematical difference can be represented.

For equality checks with floats, direct equality is often wrong for computed values:

```
total = 0.1 + 0.2
expected = 0.3

print(f"total:      {total}")
print(f"expected: {expected}")
print(f"direct equality: {total == expected}")

print()
print(f"difference: {abs(total - expected)}")
```

Possible output:

```
total:      0.30000000000000004
expected: 0.3
direct equality: False

difference: 5.551115123125783e-17
```

A better test is to check whether the values are close enough for the problem being solved:

```
import math

total = 0.1 + 0.2
expected = 0.3

print(f"math.isclose(total, expected): {math.isclose(total, expected)}")
```

Possible output:

```
math.isclose(total, expected): True
```

The practical rule is:

Use `float` for approximate real-number computation, not for exact decimal accounting.

For money, exact decimal rules, or base-10 financial calculations, Python's standard `decimal` module is often more appropriate, but that belongs to a later discussion.

Special Floating-Point Values: Infinity, Negative Infinity, and NaN

Floating-point arithmetic also includes special values. The most important are:

- positive infinity: `inf`;
- negative infinity: `-inf`;
- not-a-number: `nan`.

They can be created with `float()` or accessed through the `math` module.

```
import math

pos_inf = float("inf")
neg_inf = float("-inf")
not_number = float("nan")

print(f"pos_inf:    {pos_inf}")
print(f"neg_inf:    {neg_inf}")
print(f"not_number: {not_number}")

print()
print(f"type(pos_inf):    {type(pos_inf)}")
print(f"type(neg_inf):    {type(neg_inf)}")
print(f"type(not_number): {type(not_number)}")
```

Possible output:

```
pos_inf:    inf
neg_inf:    -inf
not_number: nan

type(pos_inf):    <class 'float'>
type(neg_inf):    <class 'float'>
type(not_number): <class 'float'>
```

These are still `float` objects. They are not separate Python types.

Infinity compares larger than ordinary finite numbers:

```
pos_inf = float("inf")
neg_inf = float("-inf")

print(f"pos_inf > 10 ** 300: {pos_inf > 10 ** 300}")
print(f"neg_inf < -10 ** 300: {neg_inf < -10 ** 300}")

print()
print(f"pos_inf + 42: {pos_inf + 42}")
print(f"neg_inf * 2:  {neg_inf * 2}")
```

Possible output:

```
pos_inf > 10 ** 300: True
neg_inf < -10 ** 300: True

pos_inf + 42: inf
neg_inf * 2:  -inf
```

The value `nan` is different. It represents an undefined or unrepresentable floating-point result.

```
not_number = float("nan")

print(f"not_number: {not_number}")
print(f"not_number == not_number: {not_number == not_number}")
print(f"not_number != not_number: {not_number != not_number}")
```

Possible output:

```
not_number: nan
not_number == not_number: False
not_number != not_number: True
```

This is strange but intentional. A NaN is not considered equal even to itself. Therefore, do not test for NaN using equality.

Use `math.isnan()`:

```
import math

value1 = 42.0
value2 = float("nan")

print(f"value1 is NaN: {math.isnan(value1)}")
print(f"value2 is NaN: {math.isnan(value2)}")
```

Possible output:

```
value1 is NaN: False
value2 is NaN: True
```

Similarly, use `math.isinf()` to test for infinity:

```
import math

values = [42.0, float("inf"), float("-inf"), float("nan")]

for value in values:
    print(f"{str(value):<8} "
          f"isinf={math.isinf(value)!s:<5} "
          f"isnan={math.isnan(value)!s:<5} "
          f"isfinite={math.isfinite(value)!s:<5}")
```

Possible output:

```
42.0      isinf=False isnan=False isfinite=True
inf       isinf=True  isnan=False isfinite=False
-inf      isinf=True  isnan=False isfinite=False
nan       isinf=False isnan=True  isfinite=False
```

The practical summary is:

Python's `float` is a fixed-precision binary floating-point object. It is fast and useful for approximate numerical computation, but it cannot represent all decimal fractions exactly, can overflow to infinity, and includes special values such as `inf`, `-inf`, and `nan`.

4.3.3 The Complex Representation Architecture (complex)

Python has a built-in `complex` type. A complex number contains two floating-point parts:

- a real part;
- an imaginary part.

In Python source code, the imaginary component is written with `j`.

```
signal = 3 + 4j

print(f"signal: {signal}")
print(f"type:    {type(signal)}")

print()
print("selected complex methods/attributes:")
print(f"{'real':<15} {'real' in dir(signal)}")
print(f"{'imag':<15} {'imag' in dir(signal)}")
print(f"{'conjugate':<15} {'conjugate' in dir(signal)}")
print(f"{'__add__':<15} {'__add__' in dir(signal)}")
print(f"{'__mul__':<15} {'__mul__' in dir(signal)}")

print()
print(f"real part:      {signal.real}")
print(f"imaginary part: {signal.imag}")
print(f"conjugate:      {signal.conjugate()}")
```

Possible output:

```
signal: (3+4j)
type:   <class 'complex'>

selected complex methods/attributes:
real          True
imag          True
conjugate     True
__add__       True
__mul__       True

real part:    3.0
imaginary part: 4.0
conjugate:    (3-4j)
```

The value `3 + 4j` is one object of type `complex`. It is not a pair of separate Python variables. The real and imaginary components are accessible through attributes.

Language-Native Integration via the Mathematical Imaginary Component `j` Literal

Python uses `j` for the imaginary unit. This follows the convention used in engineering, where `i` is often already used for electric current.

The imaginary literal must have a numeric coefficient:

```
z1 = 4j
z2 = 3 + 4j
z3 = -2j
```



```

z4 = 0.5j

print(f"z1: {z1}")
print(f"z2: {z2}")
print(f"z3: {z3}")
print(f"z4: {z4}")

print()
print(f"type(z1): {type(z1)}")
print(f"type(z2): {type(z2)}")

```

Possible output:

```

z1: 4j
z2: (3+4j)
z3: (-0-2j)
z4: 0.5j

type(z1): <class 'complex'>
type(z2): <class 'complex'>

```

The text `4j` is a complex literal. The text `j` alone is not an imaginary literal. It is treated as an ordinary name.

```

try:
    print(j)
except NameError as err:
    print("name lookup failed")
    print(f"message: {err}")

```

Possible output:

```

name lookup failed
message: name 'j' is not defined

```

This works only after binding `j` as a normal variable:

```

j = "Jerry"

print(j)
print(type(j))

```

Possible output:

```

Jerry
<class 'str'>

```

So the rule is:

`4j` is an imaginary numeric literal. Plain `j` is just a name.

Uppercase `J` also works:

```

z1 = 4j
z2 = 4J

print(f"z1: {z1}")
print(f"z2: {z2}")
print(f"z1 == z2: {z1 == z2}")

```

Possible output:

```
z1: 4j
z2: 4j
z1 == z2: True
```

Complex arithmetic is built into the language:

```
a = 3 + 4j
b = 2 - 1j

print(f"a: {a}")
print(f"b: {b}")

print()
print(f"a + b: {a + b}")
print(f"a - b: {a - b}")
print(f"a * b: {a * b}")
print(f"a / b: {a / b}")
```

Possible output:

```
a: (3+4j)
b: (2-1j)

a + b: (5+3j)
a - b: (1+5j)
a * b: (10+5j)
a / b: (0.4+2.2j)
```

The operators are dispatched to complex-number behavior. The name `a` is still only a binding to a complex object.

Component Access Mechanisms: Extracting `.real` and `.imag` Floating-Point Parts

A complex object exposes its two components through `.real` and `.imag`.

```
position = 12.5 - 7.25j

real_part = position.real
imag_part = position.imag

print(f"position: {position}")
print(f"real_part: {real_part}")
print(f"imag_part: {imag_part}")

print()
print(f"type(position): {type(position)}")
print(f"type(real_part): {type(real_part)}")
print(f"type(imag_part): {type(imag_part)}")
```

Possible output:

```
position: (12.5-7.25j)
real_part: 12.5
imag_part: -7.25
```

```
type(position): <class 'complex'>
type(real_part): <class 'float'>
type(imag_part): <class 'float'>
```

The components are floating-point values. Even when the source text uses whole numbers, the extracted parts are floats:

```
z = 3 + 4j

print(f"z.real: {z.real}")
print(f"z.imag: {z.imag}")

print()
print(f"type(z.real): {type(z.real)}")
print(f"type(z.imag): {type(z.imag)}")
```

Possible output:

```
z.real: 3.0
z.imag: 4.0

type(z.real): <class 'float'>
type(z.imag): <class 'float'>
```

A complex number with no explicit real part has real part 0.0:

```
z = 42j

print(f"z:      {z}")
print(f"real:    {z.real}")
print(f"imag:    {z.imag}")
print(f"type:    {type(z)}")
```

Possible output:

```
z:      42j
real:    0.0
imag:    42.0
type:    <class 'complex'>
```

A complex number with no imaginary contribution can still be complex:

```
z = complex(42, 0)

print(f"z:      {z}")
print(f"real:    {z.real}")
print(f"imag:    {z.imag}")
print(f"type:    {type(z)}")
```

Possible output:

```
z:      (42+0j)
real:    42.0
imag:    0.0
type:    <class 'complex'>
```

The constructor `complex()` can build a complex object from two numeric parts:

```
real_part = 3
imag_part = 4

z = complex(real_part, imag_part)

print(f"z: {z}")
print(f"type(z): {type(z)}")
print(f"z.real: {z.real}")
print(f"z.imag: {z.imag}")
```

Possible output:

```
z: (3+4j)
type(z): <class 'complex'>
z.real: 3.0
z.imag: 4.0
```

The method `conjugate()` returns a related complex number where the sign of the imaginary part is reversed:

```
z = 3 + 4j
z_conj = z.conjugate()

print(f"z:          {z}")
print(f"conjugate:   {z_conj}")

print()
print(f"z real:       {z.real}")
print(f"z imag:        {z.imag}")
print(f"conj real:      {z_conj.real}")
print(f"conj imag:      {z_conj.imag}")
```

Possible output:

```
z:          (3+4j)
conjugate:   (3-4j)

z real:      3.0
z imag:      4.0
conj real:   3.0
conj imag:  -4.0
```

This method does not mutate the original complex object. Complex numbers are immutable. It returns a new complex object.

```
z = 3 + 4j

old_id = id(z)
z_conj = z.conjugate()

print(f"z:          {z}")
print(f"z_conj:        {z_conj}")
```

```
print()
print(f"id(z):      {id(z)}")
print(f"id(z_conj): {id(z_conj)}")
print(f"same object: {id(z) == id(z_conj)}")
```

Possible output:

```
z:      (3+4j)
z_conj: (3-4j)

id(z):      2214135700944
id(z_conj): 2214135700976
same object: False
```

Complex Arithmetic Boundaries: Why Ordering Comparisons Are Not Defined for Complex Numbers

Complex numbers support equality comparison:

```
a = 3 + 4j
b = 3 + 4j
c = 3 - 4j

print(f"a == b: {a == b}")
print(f"a == c: {a == c}")
print(f"a != c: {a != c}")
```

Possible output:

```
a == b: True
a == c: False
a != c: True
```

However, Python does not define ordinary ordering for complex numbers:

```
a = 3 + 4j
b = 5 + 0j

print(a < b)
```

This raises a `TypeError`.
Safe demonstration:

```
a = 3 + 4j
b = 5 + 0j

try:
    print(a < b)
except TypeError as err:
    print("ordering failed")
    print(f"message: {err}")
    print(f"type(err): {type(err)}")
```

Possible output:

```
ordering failed
message: '<' not supported between instances of 'complex' and 'complex'
type(err): <class 'TypeError'>
```

The reason is conceptual. Real numbers have a natural left-to-right order on a single number line. Complex numbers have two components. There is no single built-in rule that says whether $3 + 4j$ should be less than $5 + 0j$.

A programmer can choose a comparison rule for a specific problem, but Python does not pretend that one universal rule exists.

For example, one possible rule is to compare by magnitude. Python's `abs()` returns the distance of the complex number from zero:

```
a = 3 + 4j
b = 5 + 0j
c = 1 + 1j

print(f"abs(a): {abs(a)}")
print(f"abs(b): {abs(b)}")
print(f"abs(c): {abs(c)}")

print()
print(f"abs(a) == abs(b): {abs(a) == abs(b)}")
print(f"abs(c) < abs(a): {abs(c) < abs(a)}")
```

Possible output:

```
abs(a): 5.0
abs(b): 5.0
abs(c): 1.4142135623730951

abs(a) == abs(b): True
abs(c) < abs(a): True
```

This comparison is not complex ordering. It is float ordering after reducing each complex number to a magnitude.

A list of complex values cannot be sorted directly:

```
values = [3 + 4j, 1 + 1j, 5 + 0j]
```

```
try:
    print(sorted(values))
except TypeError as err:
    print("sorting failed")
    print(f"message: {err}")
```

Possible output:

```
sorting failed
message: '<' not supported between instances of 'complex' and 'complex'
```

But it can be sorted by a chosen key, such as magnitude:

```
values = [3 + 4j, 1 + 1j, 5 + 0j]

sorted_by_magnitude = sorted(values, key=abs)

print(f"original: {values}")
print(f"sorted by magnitude: {sorted_by_magnitude}")
```

Possible output:

```
original:          [(3+4j), (1+1j), (5+0j)]
sorted by magnitude: [(1+1j), (3+4j), (5+0j)]
```

The practical summary is:

Python's `complex` type is a built-in numeric object with floating-point real and imaginary parts. It supports complex arithmetic and equality checks, but it does not support ordinary ordering comparisons such as `<` or `>`. If ordering is needed, the programmer must explicitly choose what property is being ordered.

4.3.4 The Boolean Representation Architecture (`bool`)

Python's `bool` type represents truth values. It has only two ordinary values:

```
True
False
```

These are not strings. They are not numbers written with special spelling. They are boolean objects.

```
is_ready = True
is_finished = False

print(f"is_ready:    {is_ready}")
print(f"is_finished: {is_finished}")

print()
print(f"type(is_ready):    {type(is_ready)}")
print(f"type(is_finished): {type(is_finished)}")

print()
print("selected bool methods:")
print(f"{'__and__':<12} {'__and__' in dir(is_ready)}")
print(f"{'__or__':<12} {'__or__' in dir(is_ready)}")
print(f"{'__invert__':<12} {'__invert__' in dir(is_ready)}")
print(f"{'bit_length':<12} {'bit_length' in dir(is_ready)}")
print(f"{'to_bytes':<12} {'to_bytes' in dir(is_ready)}")
```

Possible output:

```
is_ready:    True
is_finished: False

type(is_ready):    <class 'bool'>
type(is_finished): <class 'bool'>

selected bool methods:
__and__        True
__or__         True
__invert__     True
bit_length     True
to_bytes       True
```

The presence of methods such as `bit_length()` and `to_bytes()` is not accidental. In Python, `bool` is closely connected to `int`. That connection is useful, but it also creates traps.

Boolean Values as Singleton Objects: True and False

There is only one normal `True` object and one normal `False` object in a Python process. They are singleton objects.

This means that every ordinary use of `True` refers to the same object, and every ordinary use of `False` refers to the same object.

```
a = True
b = True

c = False
d = False

print(f"a == b: {a == b}")
print(f"a is b: {a is b}")

print()
print(f"c == d: {c == d}")
print(f"c is d: {c is d}")

print()
print(f"id(a): {id(a)}")
print(f"id(b): {id(b)}")
print(f"id(c): {id(c)}")
print(f"id(d): {id(d)}")
```

Possible output:

```
a == b: True
a is b: True

c == d: True
c is d: True

id(a): 140734431495200
id(b): 140734431495200
id(c): 140734431495232
id(d): 140734431495232
```

The exact `id()` values do not matter. The important part is that `a is b` and `c is d` are both true. Conceptually:

```
a ----\
      ---> True singleton object
b ----/

c ----\
      ---> False singleton object
d ----/
```

The constructor `bool()` also returns one of these two singleton objects:

```
x = bool(42)
y = bool("No soup for you.")
z = bool("")
```



```

print(f"x: {x}, x is True: {x is True}")
print(f"y: {y}, y is True: {y is True}")
print(f"z: {z}, z is False: {z is False}")

print()
print(f"type(x): {type(x)}")
print(f"type(y): {type(y)}")
print(f"type(z): {type(z)}")

```

Possible output:

```

x: True, x is True: True
y: True, y is True: True
z: False, z is False: True

type(x): <class 'bool'>
type(y): <class 'bool'>
type(z): <class 'bool'>

```

The function `bool()` does not create a new private truth object. It decides whether the input is truthy or falsy and returns either `True` or `False`.

Some common truth tests:

```

values = [0, 1, 42, "", "D'oh!", [], [1, 2, 3], None]

for value in values:
    truth_value = bool(value)
    print(f"{repr(value):<12} -> {truth_value:<5} type={type(truth_value)}")

```

Possible output:

```

0          -> False type=<class 'bool'>
1          -> True  type=<class 'bool'>
42         -> True  type=<class 'bool'>
''         -> False type=<class 'bool'>
"D'oh!"    -> True  type=<class 'bool'>
[]         -> False type=<class 'bool'>
[1, 2, 3]  -> True  type=<class 'bool'>
None       -> False type=<class 'bool'>

```

This introduces truth-value testing. It does not mean that all these objects become booleans. The object `42` remains an integer. The object `"D'oh!"` remains a string. The call `bool(...)` produces a boolean result.

Boolean as a Subclass of Integer: Numeric Compatibility and Practical Warnings

In Python, `bool` is a subclass of `int`. This means that `True` and `False` can participate in many integer operations.

```

print(f"isinstance(True, bool): {isinstance(True, bool)}")
print(f"isinstance(True, int):  {isinstance(True, int)}")

print()
print(f"True == 1: {True == 1}")

```

```
print(f"False == 0: {False == 0}")

print()
print(f"int(True): {int(True)}")
print(f"int(False): {int(False)}")
```

Possible output:

```
isinstance(True, bool): True
isinstance(True, int): True
```

```
True == 1: True
False == 0: True
```

```
int(True): 1
int(False): 0
```

This explains why boolean values have some integer-like methods:

```
value = True

print(f"value: {value}")
print(f"type: {type(value)}")

print()
print(f"value.bit_length(): {value.bit_length()}")
print(f"value.bit_count(): {value.bit_count()}")

print()
print(f"True + True: {True + True}")
print(f"True + False: {True + False}")
print(f"False + False: {False + False}")
```

Possible output:

```
value: True
type: <class 'bool'>
```

```
value.bit_length(): 1
value.bit_count(): 1
```

```
True + True: 2
True + False: 1
False + False: 0
```

This can be useful when counting true conditions:

```
has_ticket = True
is_late = False
knows_python = True

true_count = has_ticket + is_late + knows_python

print(f"true_count: {true_count}")
print(f"type(true_count): {type(true_count)}")
```

Possible output:

```
true_count: 2
type(true_count): <class 'int'>
```

The result is an integer because arithmetic with booleans uses their integer compatibility.

However, this compatibility should not be abused. Booleans are meant to express truth, not ordinary quantities.

Bad style:

```
is_valid = True
total = is_valid + 41

print(total)
```

Possible output:

```
42
```

This works, but it is unclear. A reader must remember that `True` behaves like 1. If the program means a number, use a number. If the program means a truth value, use a boolean.

Better:

```
is_valid = True

if is_valid:
    total = 42
else:
    total = 0

print(total)
```

The practical rule is:

`True` is numerically compatible with 1, and `False` is numerically compatible with 0, but boolean names should normally describe conditions, not quantities.

Use names like:

```
is_ready = True
has_access = False
can_continue = True
found_match = False
```

These names tell the reader that the value is meant to answer a yes/no question.

Logical Operators vs. Bitwise Operators: `and` / `or` / `not` Compared with `&` / `|` / `~`

Python has logical operators and bitwise operators. They are not the same thing.

The logical operators are:

```
and
or
not
```

They are used for truth-value logic.

```

has_ticket = True
is_late = False

print(f"has_ticket and not is_late: {has_ticket and not is_late}")
print(f"has_ticket or is_late:      {has_ticket or is_late}")
print(f"not has_ticket:             {not has_ticket}")

```

Possible output:

```

has_ticket and not is_late: True
has_ticket or is_late:      True
not has_ticket:             False

```

The bitwise operators are:

```

&
|
~

```

For integers, they operate on binary bits:

```

a = 0b1010
b = 0b1100

print(f"a:      {a:04b}")
print(f"b:      {b:04b}")

print()
print(f"a & b: {a & b:04b}")
print(f"a | b: {a | b:04b}")
print(f"~a:    {~a}")

```

Possible output:

```

a:      1010
b:      1100

a & b: 1000
a | b: 1110
~a:    -11

```

The result of `~a` may look surprising because Python integers are not fixed-width unsigned buffers. Conceptually, `~x` is equal to `-x - 1`.

```

x = 42

print(f"x:      {x}")
print(f"~x:      {~x}")
print(f"-x - 1: {-x - 1}")
print(f"same:    {~x == -x - 1}")

```

Possible output:

```

x:      42
~x:     -43
-x - 1: -43
same:   True

```

Because booleans are integer-compatible, bitwise operators also work on booleans:

```
print(f"True & True:    {True & True}")
print(f"True & False:   {True & False}")
print(f"True | False:   {True | False}")
print(f"False | False:  {False | False}")
```

Possible output:

```
True & True:    True
True & False:   False
True | False:   True
False | False:  False
```

This may look the same as logical behavior for simple boolean values:

```
print(f"True and False: {True and False}")
print(f"True & False:    {True & False}")

print()
print(f"True or False:   {True or False}")
print(f"True | False:    {True | False}")
```

Possible output:

```
True and False: False
True & False:    False

True or False:   True
True | False:    True
```

But the operators are still different.

First difference: `and` and `or` do not necessarily return a boolean. They return one of the original operands.

```
first = "No soup for you."
second = "D'oh!"

print(f"first and second: {first and second!r}")
print(f"first or second:  {first or second!r}")

print()
print(f"type(first and second): {type(first and second)}")
print(f"type(first or second):  {type(first or second)}")
```

Possible output:

```
first and second: "D'oh!"
first or second:  'No soup for you.'

type(first and second): <class 'str'>
type(first or second):  <class 'str'>
```

Second difference: `and` and `or` short-circuit. They may skip the right side when the result is already decided.

```

print("case 1")
result1 = False and (1 / 0)
print(f"result1: {result1}")

print()
print("case 2")
result2 = True or (1 / 0)
print(f"result2: {result2}")

```

Possible output:

```

case 1
result1: False

case 2
result2: True

```

No division error occurs because the right side is not evaluated.
Bitwise operators do not short-circuit in the same way:

```

try:
    result = False & (1 / 0)
except ZeroDivisionError as err:
    print("operation failed")
    print(f"message: {err}")
    print(f"type(err): {type(err)}")

```

Possible output:

```

operation failed
message: division by zero
type(err): <class 'ZeroDivisionError'>

```

The expression on the right side was evaluated before the bitwise operation could be applied.
Third difference: `not` and `~` are completely different.

```

print(f"not True: {not True}")
print(f"not False: {not False}")

print()
print(f"~True: {~True}")
print(f"~False: {~False}")

```

Possible output:

```

not True:  False
not False: True

~True:  -2
~False: -1

```

Why?

Because `True` is numerically compatible with 1, and `False` is numerically compatible with 0.

```

print(f"~1: {~1}")
print(f"~0: {~0}")

```

Possible output:

```
~1: -2
```

```
~0: -1
```

So `~` is not a logical not operator. It is bitwise inversion.

The practical rule is:

Use `and`, `or`, and `not` for logical conditions. Use `&`, `|`, and `~` only when you intentionally want bit-level operations or when a specific library type defines those operators for its own purpose.

A final comparison:

```
is_python = True
is_clear = True

print("logical condition")
print(is_python and is_clear)

flags_a = 0b1010
flags_b = 0b1100

print()
print("bitwise flags")
print(f"{flags_a & flags_b:04b}")
```

Possible output:

```
logical condition
True
```

```
bitwise flags
1000
```

The practical summary is:

Python booleans are singleton truth-value objects. They are also integer-compatible, with `True` behaving numerically like `1` and `False` like `0`. This compatibility explains some behavior, but ordinary logical code should use `and`, `or`, and `not`, not the bitwise operators.

4.3.5 The Null-Sentinel Object (None)

Python uses `None` to represent absence, missing result, or “no useful value here”. It is not the same thing as zero, an empty string, an empty list, or `False`.

A first inspection:

```
value = None

print(f"value: {value}")
print(f"type: {type(value)}")
print(f"id: {id(value)}")

print()
print("selected None methods:")
print(f"{'__bool__':<15} {'__bool__' in dir(value)}")
```

```

print(f"{'__repr__':<15} {'__repr__' in dir(value)}")
print(f"{'__eq__':<15} {'__eq__' in dir(value)}")
print(f"{'__class__':<15} {'__class__' in dir(value)}")

print()
print(f"bool(None): {bool(None)}")

```

Possible output:

```

value: None
type:  <class 'NoneType'>
id:    140734431489696

```

selected None methods:

```

__bool__      True
__repr__      True
__eq__        True
__class__     True

```

```
bool(None): False
```

The type of `None` is `NoneType`. The value `None` is falsy, but it is not the same object as `False`.

```

print(f"None == False: {None == False}")
print(f"None is False: {None is False}")

print()
print(f"type(None): {type(None)}")
print(f"type(False): {type(False)}")

```

Possible output:

```

None == False: False
None is False: False

```

```

type(None):  <class 'NoneType'>
type(False): <class 'bool'>

```

None as a Singleton Object, Not a Null Pointer

In C, a null pointer is a pointer value that points to no valid object. Python's `None` should not be understood that way.

`None` is a real Python object. Names can be bound to it, functions can return it, it has a type, and it has identity.

```

a = None
b = None

```

```

print(f"a: {a}")
print(f"b: {b}")

```

```

print()
print(f"a == b: {a == b}")
print(f"a is b: {a is b}")

```



```
print()
print(f"id(a): {id(a)}")
print(f"id(b): {id(b)}")
```

Possible output:

```
a: None
b: None
```

```
a == b: True
a is b: True
```

```
id(a): 140734431489696
id(b): 140734431489696
```

There is only one normal `None` object in a Python process. It is a singleton.
Conceptually:

```
a ----\
      ---> None singleton object
b ----/
```

This is different from an unbound name.

```
value = None
```

```
print(value)
print(type(value))
```

This works because the name `value` exists and points to `None`.
But this fails:

```
try:
    print(missing_value)
except NameError as err:
    print("name lookup failed")
    print(f"message: {err}")
```

Possible output:

```
name lookup failed
message: name 'missing_value' is not defined
```

The difference is:

```
value = None      -> the name exists and refers to None
missing_value     -> the name does not exist here
```

So `None` is not “no variable”. It is an object used to represent no useful value.

Absence, Default Return Values, and Missing-Value Signaling

`None` is commonly used when a value is absent.

```
middle_name = None

print(f"middle_name: {middle_name}")
print(f"type:          {type(middle_name)}")

if middle_name is None:
    print("No middle name was provided.")
else:
    print(f"Middle name: {middle_name}")
```

Possible output:

```
middle_name: None
type:         <class 'NoneType'>
No middle name was provided.
```

This is different from an empty string:

```
name1 = None
name2 = ""

print(f"name1: {name1!r}, type={type(name1)}")
print(f"name2: {name2!r}, type={type(name2)}")

print()
print(f"name1 == name2: {name1 == name2}")
print(f"bool(name1): {bool(name1)}")
print(f"bool(name2): {bool(name2)}")
```

Possible output:

```
name1: None, type=<class 'NoneType'>
name2: '', type=<class 'str'>

name1 == name2: False
bool(name1): False
bool(name2): False
```

Both `None` and the empty string are falsy, but they do not mean the same thing.

- `None` usually means: no value was supplied, found, or produced.
- `""` means: a string exists, but it contains no characters.

A function that does not explicitly return a value returns `None` automatically.

```
def say_quote():
    print("And now for something completely different.")

result = say_quote()

print()
print(f"result: {result}")
print(f"type(result): {type(result)}")
print(f"result is None: {result is None}")
```

Possible output:

And now for something completely different.

```
result: None
type(result): <class 'NoneType'>
result is None: True
```

The function printed text, but it did not return a useful value. Therefore, Python returned `None`. Compare that with a function that returns a value:

```
def get_answer():
    return 42

result = get_answer()

print(f"result: {result}")
print(f"type(result): {type(result)}")
print(f"result is None: {result is None}")
```

Possible output:

```
result: 42
type(result): <class 'int'>
result is None: False
```

`None` is also useful as a default marker:

```
user_name = None

if user_name is None:
    display_name = "Anonymous Pythonista"
else:
    display_name = user_name

print(f"display_name: {display_name}")
```

Possible output:

```
display_name: Anonymous Pythonista
```

This pattern is common because `None` clearly separates “missing value” from valid values that merely happen to be falsy.

For example, zero may be a valid value:

```
score = 0

if score is None:
    print("No score was recorded.")
else:
    print(f"Recorded score: {score}")
```

Possible output:

```
Recorded score: 0
```

If the program used only a truth test, it would confuse 0 with absence:

```
score = 0

if score:
    print(f"Recorded score: {score}")
else:
    print("This branch also runs for zero.")
```

Possible output:

This branch also runs for zero.

So `None` is the better sentinel when absence must be distinguished from falsy but valid values.

Identity Testing with `is None` and `is not None`

The standard way to test for `None` is identity testing:

```
value is None
value is not None
```

Because `None` is a singleton, identity is the precise question. We are asking whether the name refers to the one `None` object.

```
values = [None, 0, "", False, 42, "D'oh!"]

for value in values:
    print(f"{repr(value):<8} "
          f"is None: {value is None!s:<5} "
          f"is not None: {value is not None!s:<5}")
```

Possible output:

```
None      is None: True  is not None: False
0         is None: False is not None: True
''        is None: False is not None: True
False     is None: False is not None: True
42        is None: False is not None: True
"D'oh!"   is None: False is not None: True
```

This keeps the distinction clean:

```
if value is None:
    print("missing")
else:
    print("present")
```

Do not write this for ordinary `None` checks:

```
if value == None:
    print("missing")
```

It often works, but it asks the wrong question. The operator `==` asks whether two objects compare as equal. The operator `is` asks whether two names refer to the same object.

For `None`, identity is the intended test.

A compact comparison:

```

value = None

print(f"value == None: {value == None}")
print(f"value is None: {value is None}")

value = 0

print()
print(f"value == None: {value == None}")
print(f"value is None: {value is None}")

```

Possible output:

```

value == None: True
value is None: True

value == None: False
value is None: False

```

The first case looks the same, but the meaning differs. `is None` is clearer and more exact. The opposite test is `is not None`:

```

quote = "No soup for you."

if quote is not None:
    print(f"quote exists: {quote}")
else:
    print("quote is missing")

```

Possible output:

```

quote exists: No soup for you.

```

This does not test whether the string is empty. It tests whether the value is missing.

```

quote = ""

if quote is not None:
    print("quote was supplied")
else:
    print("quote is missing")

if quote:
    print("quote has visible content")
else:
    print("quote is empty or otherwise falsy")

```

Possible output:

```

quote was supplied
quote is empty or otherwise falsy

```

The two tests answer different questions:

- `quote is not None`: was a value supplied?

- `bool(quote)` or `if quote:` is the supplied value truthy?

The practical summary is:

`None` is Python's singleton object for absence or no useful result. It is not a null pointer and not the same thing as `False`, zero, or an empty string. Test it with `is None` and `is not None`.

4.3.6 Numerical Operator Architecture and Precedence Graphs

Numerical operators are not magic syntax pasted on top of raw memory. In Python, an operator is an expression-level request sent to runtime objects. The objects involved decide whether the operation is supported and what kind of result object should be produced.

For example:

```
result = 40 + 2

print(f"result: {result}")
print(f"type:    {type(result)}")
```

Possible output:

```
result: 42
type:    <class 'int'>
```

The expression `40 + 2` produces a new integer object. The name `result` is then bound to that object.

Standard Arithmetic: Operator Dispatch and Result Object Creation

The usual arithmetic operators are:

```
+    addition
-    subtraction
*    multiplication
/    true division
//   floor division
%    remainder
**   exponentiation
```

A first example:

```
a = 42
b = 8

print(f"a + b:  {a + b}")
print(f"a - b:  {a - b}")
print(f"a * b:  {a * b}")
print(f"a / b:  {a / b}")
print(f"a // b: {a // b}")
print(f"a % b:  {a % b}")
print(f"a ** 2: {a ** 2}")

print()
print(f"type(a + b):  {type(a + b)}")
print(f"type(a / b):  {type(a / b)}")
print(f"type(a // b): {type(a // b)}")
```

Possible output:

```
a + b: 50
a - b: 34
a * b: 336
a / b: 5.25
a // b: 5
a % b: 2
a ** 2: 1764

type(a + b): <class 'int'>
type(a / b): <class 'float'>
type(a // b): <class 'int'>
```

Operators are connected to special methods on objects.

```
number = 42
```

```
print(f"type(number): {type(number)}")

print()
print("selected operator methods:")
print(f"{'__add__':<15} {'__add__' in dir(number)}")
print(f"{'__sub__':<15} {'__sub__' in dir(number)}")
print(f"{'__mul__':<15} {'__mul__' in dir(number)}")
print(f"{'__truediv__':<15} {'__truediv__' in dir(number)}")
print(f"{'__floordiv__':<15} {'__floordiv__' in dir(number)}")
print(f"{'__mod__':<15} {'__mod__' in dir(number)}")
print(f"{'__pow__':<15} {'__pow__' in dir(number)}")

print()
print(f"number.__add__(8): {number.__add__(8)}")
print(f"number + 8: {number + 8}")
```

Possible output:

```
type(number): <class 'int'>

selected operator methods:
__add__      True
__sub__      True
__mul__      True
__truediv__  True
__floordiv__ True
__mod__      True
__pow__      True

number.__add__(8): 50
number + 8: 50
```

In normal code, we do not call `__add__()` directly. We write `number + 8`. The method inspection only shows the object-protocol machinery behind the operator.

Mixed numeric types may produce a wider result type:

```
int_value = 42
float_value = 0.5
complex_value = 2j

result1 = int_value + float_value
result2 = int_value + complex_value
result3 = float_value + complex_value

print(f"result1: {result1}, type={type(result1)}")
print(f"result2: {result2}, type={type(result2)}")
print(f"result3: {result3}, type={type(result3)}")
```

Possible output:

```
result1: 42.5, type=<class 'float'>
result2: (42+2j), type=<class 'complex'>
result3: (0.5+2j), type=<class 'complex'>
```

The name on the left is not typed. It simply receives the result object produced by the expression.

Division Divergence: Floating-Point Division (/) vs. Floor Division (//)

Python has two different division operators.

/ performs true division and produces a floating-point result for ordinary integer operands:

```
a = 10
b = 2

result = a / b

print(f"result: {result}")
print(f"type: {type(result)}")
```

Possible output:

```
result: 5.0
type: <class 'float'>
```

Even though the mathematical result is whole, the result object is a `float`.

// performs floor division:

```
a = 11
b = 2

result = a // b

print(f"result: {result}")
print(f"type: {type(result)}")
```

Possible output:

```
result: 5
type: <class 'int'>
```

Floor division means the result is rounded down toward negative infinity, not merely chopped toward zero.


```
print(f"11 // 2:   {11 // 2}")
print(f"-11 // 2:  {-11 // 2}")
print(f"11 // -2:  {11 // -2}")
print(f"-11 // -2: {-11 // -2}")
```

Possible output:

```
11 // 2:   5
-11 // 2:  -6
11 // -2:  -6
-11 // -2: 5
```

This is a common trap for C programmers. In many C contexts, integer division truncates toward zero. Python's `//` floors.

With floating-point operands, floor division still floors, but the result is a float:

```
a = 11.0
b = 2.0

result = a // b

print(f"result: {result}")
print(f"type:   {type(result)}")
```

Possible output:

```
result: 5.0
type:   <class 'float'>
```

The practical rule is:

Use `/` when you want true division. Use `//` when you want floor division. Do not treat `//` as C-style integer truncation.

Modular Math (%) and Exponentiation (**) Mechanics

The modulo operator `%` returns the remainder connected to floor division.

```
a = 42
b = 8

quotient = a // b
remainder = a % b

print(f"a:      {a}")
print(f"b:      {b}")
print(f"quotient:  {quotient}")
print(f"remainder: {remainder}")

print()
print(f"check: {quotient} * {b} + {remainder} = {quotient * b + remainder}")
```

Possible output:

```
a:      42
b:      8
quotient: 5
remainder: 2
```

```
check: 5 * 8 + 2 = 42
```

Python keeps this relationship:

```
a == (a // b) * b + (a % b)
```

This also applies with negative values:

```
pairs = [
    (11, 2),
    (-11, 2),
    (11, -2),
    (-11, -2),
]

for a, b in pairs:
    q = a // b
    r = a % b

    print(f"a={a:>4}, b={b:>3} | "
          f"a//b={q:>4}, a%b={r:>3} | "
          f"check={q * b + r:>4}")
```

Possible output:

```
a=  11, b=  2 | a//b=  5, a%b=  1 | check=  11
a= -11, b=  2 | a//b= -6, a%b=  1 | check= -11
a=  11, b= -2 | a//b= -6, a%b= -1 | check=  11
a= -11, b= -2 | a//b=  5, a%b= -1 | check= -11
```

The function `divmod()` gives both results at once:

```
a = 42
b = 8

q, r = divmod(a, b)

print(f"quotient: {q}")
print(f"remainder: {r}")
print(f"type(q): {type(q)}")
print(f"type(r): {type(r)}")
```

Possible output:

```
quotient: 5
remainder: 2
type(q): <class 'int'>
type(r): <class 'int'>
```

Exponentiation uses `**`:

```
base = 2
power = 10

result = base ** power

print(f"result: {result}")
print(f"type: {type(result)}")
```

Possible output:

```
result: 1024
type: <class 'int'>
```

With integers, large exponentiation can produce arbitrarily large exact integers:

```
big = 2 ** 100

print(f"big: {big}")
print(f"type: {type(big)}")
print(f"bit length: {big.bit_length()}")
```

Possible output:

```
big: 1267650600228229401496703205376
type: <class 'int'>
bit length: 101
```

A negative exponent produces a floating-point result:

```
result = 2 ** -3

print(f"result: {result}")
print(f"type: {type(result)}")
```

Possible output:

```
result: 0.125
type: <class 'float'>
```

So `**` is still operator dispatch. The result type depends on the operation and operands.

Unary Operators and Precedence Traps: `-x`, `+x`, and `-2 ** 2`

Python has unary numeric operators:

```
+x
-x
```

Unary plus usually returns the numeric value unchanged:

```
x = 42

print(f"+x: {+x}")
print(f"type(+x): {type(+x)}")
```

Possible output:

```
+x: 42
type(+x): <class 'int'>
```

Unary minus negates the value:

```
x = 42

print(f"-x: {-x}")
print(f"type(-x): {type(-x)}")
```

Possible output:

```
-x: -42
type(-x): <class 'int'>
```

These operators are also connected to special methods:

```
x = 42

print(f"{'__pos__':<12} {'__pos__' in dir(x)}")
print(f"{'__neg__':<12} {'__neg__' in dir(x)}")

print()
print(f"x.__pos__(): {x.__pos__()}")
print(f"x.__neg__(): {x.__neg__()}")
```

Possible output:

```
__pos__      True
__neg__      True

x.__pos__(): 42
x.__neg__(): -42
```

The famous precedence trap is:

```
result = -2 ** 2

print(result)
```

Possible output:

```
-4
```

This surprises many students because they read it as:

```
(-2) ** 2
```

But Python reads it as:

```
-(2 ** 2)
```

A direct comparison:

```

a = -2 ** 2
b = (-2) ** 2
c = -(2 ** 2)

print(f"-2 ** 2:    {a}")
print(f"(-2) ** 2:  {b}")
print(f"-(2 ** 2):  {c}")

```

Possible output:

```

-2 ** 2:    -4
(-2) ** 2:   4
-(2 ** 2):  -4

```

Exponentiation binds more tightly than unary minus on its left side.

There is a second detail: exponentiation binds less tightly than unary operators on its right side.

```

result = 2 ** -2

print(f"2 ** -2: {result}")
print(f"type:    {type(result)}")

```

Possible output:

```

2 ** -2: 0.25
type:    <class 'float'>

```

This is read as:

```
2 ** (-2)
```

For this chapter, the practical warning is enough:

When exponentiation and unary signs appear together, use parentheses unless the intended grouping is visually obvious.

Parentheses as Explicit Precedence Control

Operator precedence decides how an expression is grouped when parentheses are absent.

A simplified numerical precedence ladder is:

highest	
**	exponentiation
+x, -x	unary plus, unary minus
*, /, //, %	multiplication and division family
+, -	addition and subtraction
lowest	

This ladder is simplified for the arithmetic operators used here. It is enough for the present section.

Examples:

```

result1 = 2 + 3 * 4
result2 = (2 + 3) * 4

print(f"2 + 3 * 4:    {result1}")
print(f"(2 + 3) * 4:   {result2}")

```

Possible output:

```
2 + 3 * 4:      14
(2 + 3) * 4:    20
```

Without parentheses, multiplication happens before addition.

Another example:

```
result1 = 20 - 6 // 2
result2 = (20 - 6) // 2

print(f"20 - 6 // 2:      {result1}")
print(f"(20 - 6) // 2:    {result2}")
```

Possible output:

```
20 - 6 // 2:      17
(20 - 6) // 2:    7
```

Parentheses do not create a new numeric type. They only control expression grouping.

```
a = 2 + 3 * 4
b = (2 + 3) * 4

print(f"a: {a}, type={type(a)}")
print(f"b: {b}, type={type(b)}")
```

Possible output:

```
a: 14, type=<class 'int'>
b: 20, type=<class 'int'>
```

For teaching and production code, parentheses are often better than relying on the reader to remember every precedence rule.

Poor readability:

```
score = base + bonus * level - penalty // 2 ** round_count
```

Clearer:

```
score = base + (bonus * level) - (penalty // (2 ** round_count))
```

A complete small script:

```
base = 42
bonus = 5
level = 3
penalty = 16
round_count = 2

score = base + (bonus * level) - (penalty // (2 ** round_count))

print(f"base:      {base}")
print(f"bonus:      {bonus}")
print(f"level:       {level}")
print(f"penalty:     {penalty}")
print(f"round_count: {round_count}")

print()
print(f"score:        {score}")
print(f"type(score): {type(score)}")
```

Possible output:

```
base:      42
bonus:     5
level:     3
penalty:   16
round_count: 2

score:     53
type(score): <class 'int'>
```

The practical summary is:

Numerical operators create result objects through type-defined behavior. Operator precedence controls how expressions are grouped before those operations execute. Parentheses are the clearest way to state the grouping that the programmer intends.

4.4 Runtime Typing Consequences in Singular Operations

At this point, the basic pieces are in place: names are bindings, singular values are objects, and operators are dispatched according to object behavior. This section brings those ideas together and states their direct consequence for Python's type system.

Python is dynamic at the level of names. A name can first refer to an integer, later to a string, later to `None`, and Python will allow that rebinding. But Python is not loose at the level of operations. Once an expression executes, the actual objects involved must support that operation.

This is why the same name can legally move from 42 to "42", but the expression "42" + 1 still fails. Python does not silently guess whether the programmer wanted numeric addition or string concatenation. The programmer must make the conversion explicit.

The purpose of this section is to make that boundary precise:

Names are flexible. Objects are typed. Operations are checked.

This distinction explains much of Python's everyday behavior: why reassignment is easy, why `type()` and `isinstance()` inspect runtime objects, why mixed numerical operations can promote toward `float` or `complex`, and why unrelated combinations such as text plus number raise errors instead of being silently coerced.

4.4.1 Dynamic Typing in Assignment

Python is dynamically typed in the sense that names do not have fixed declared types. The object has a type. The name is only a binding to an object.

This follows directly from the assignment model developed earlier:

```
name ---> object
```

The name can later be rebound to another object. That other object may have a different type.

Names Do Not Have Fixed Declared Types

In C, a declaration such as:

```
int score;
```

creates a variable whose type is fixed by the declaration. The storage location is meant to hold an integer value compatible with that declared type.

Python does not work like that.

```
score = 42

print(f"score: {score}")
print(f"type(score): {type(score)}")
```

Possible output:

```
score: 42
type(score): <class 'int'>
```

The name `score` is not an integer variable in the C sense. It is a name currently bound to an integer object.

The integer object has integer behavior:

```
score = 42

print("selected int methods:")
print(f"{'bit_length':<15} {'bit_length' in dir(score)}")
print(f"{'bit_count':<15} {'bit_count' in dir(score)}")
print(f"{'__add__':<15} {'__add__' in dir(score)}")

print()
print(f"score.bit_length(): {score.bit_length()}")
print(f"score + 8:          {score + 8}")
```

Possible output:

```
selected int methods:
bit_length      True
bit_count       True
__add__         True

score.bit_length(): 6
score + 8:         50
```

Those methods belong to the object reached through `score`. They do not belong to the spelling of the name.

If the same name is rebound to a string, the available behavior changes because the object changed:

```
score = "No soup for you."

print(f"score: {score}")
print(f"type(score): {type(score)}")

print()
print("selected str methods:")
print(f"{'upper':<15} {'upper' in dir(score)}")
print(f"{'replace':<15} {'replace' in dir(score)}")
print(f"{'split':<15} {'split' in dir(score)}")

print()
print(score.upper())
print(score.replace("soup", "debugger"))
```

Possible output:


```
score: No soup for you.
type(score): <class 'str'>
```

```
selected str methods:
upper          True
replace        True
split          True
```

```
NO SOUP FOR YOU.
No debugger for you.
```

The name is the same. The object is different. Therefore the type and behavior are different. The practical rule is:

Do not ask what type a Python name has permanently. Ask what object the name currently references.

Rebinding the Same Name to Objects of Different Types

The same Python name can be rebound to objects of different types during execution.

```
value = 42
print(f"{'step 1':<8} value={value!r:<30} type={type(value)}")

value = 3.14
print(f"{'step 2':<8} value={value!r:<30} type={type(value)}")

value = "D'oh!"
print(f"{'step 3':<8} value={value!r:<30} type={type(value)}")

value = True
print(f"{'step 4':<8} value={value!r:<30} type={type(value)}")

value = None
print(f"{'step 5':<8} value={value!r:<30} type={type(value)}")
```

Possible output:

step 1	value=42	type=<class 'int'>
step 2	value=3.14	type=<class 'float'>
step 3	value="D'oh!"	type=<class 'str'>
step 4	value=True	type=<class 'bool'>
step 5	value=None	type=<class 'NoneType'>

This is legal because the name `value` is not declared as one fixed type. Each assignment changes the binding.

Conceptually:

```
step 1:
    value ---> int object 42

step 2:
    value ---> float object 3.14

step 3:
```

```
value ---> str object 'D'oh!'
```

step 4:

```
value ---> bool object True
```

step 5:

```
value ---> None singleton object
```

This flexibility is useful in small scripts and interactive work, but it can also make code unclear if the same name changes meaning too often.

Poor style:

```
data = 42
data = "forty-two"
data = True
```

Better:

```
answer_number = 42
answer_text = "forty-two"
answer_is_known = True
```

The second version uses separate names for separate roles. Python does not require this, but human readers benefit from it.

A more realistic example:

```
raw_score = "42"
score = int(raw_score)
passed = score >= 40

print(f"raw_score: {raw_score!r}, type={type(raw_score)}")
print(f"score:      {score!r}, type={type(score)}")
print(f"passed:     {passed!r}, type={type(passed)}")
```

Possible output:

```
raw_score: '42', type=<class 'str'>
score:      42, type=<class 'int'>
passed:     True, type=<class 'bool'>
```

Here the names explain the transformation:

```
raw_score  -> original string text
score      -> converted integer
passed     -> boolean result of comparison
```

This is clearer than repeatedly reusing one name for all stages.

Runtime Type Inspection with `type()` and `isinstance()`

Python allows type inspection at runtime.

The function `type()` returns the actual type object of a value:

```
items = [42, 3.14, "No soup for you.", True, None]

for item in items:
    print(f"{repr(item):<20} type={type(item)}")
```

Possible output:

```
42                type=<class 'int'>
3.14              type=<class 'float'>
'No soup for you.' type=<class 'str'>
True              type=<class 'bool'>
None              type=<class 'NoneType'>
```

The function `isinstance()` asks whether an object belongs to a type or to a family of types.

```
value = 42
```

```
print(f"is int:      {isinstance(value, int)}")
print(f"is float:    {isinstance(value, float)}")
print(f"is str:      {isinstance(value, str)}")
print(f"is bool:     {isinstance(value, bool)}")
```

Possible output:

```
is int:      True
is float:    False
is str:      False
is bool:     False
```

A string example:

```
quote = "And now for something completely different."

print(f"type(quote): {type(quote)}")
print(f"is str:      {isinstance(quote, str)}")
print(f"is int:      {isinstance(quote, int)}")

print()
print("selected str methods:")
print(f"{'upper':<12} {'upper' in dir(quote)}")
print(f"{'lower':<12} {'lower' in dir(quote)}")
print(f"{'replace':<12} {'replace' in dir(quote)}")
```

Possible output:

```
type(quote): <class 'str'>
is str:      True
is int:      False

selected str methods:
upper       True
lower       True
replace     True
```

For simple exact checks, `type()` is direct:

```
value = 42
```

```
print(type(value) is int)
```

Possible output:

True

But `isinstance()` is usually better when inheritance matters.
The classic example is `bool`:

```
flag = True

print(f"type(flag): {type(flag)}")

print()
print(f"type(flag) is bool: {type(flag) is bool}")
print(f"type(flag) is int: {type(flag) is int}")

print()
print(f"isinstance(flag, bool): {isinstance(flag, bool)}")
print(f"isinstance(flag, int): {isinstance(flag, int)}")
```

Possible output:

```
type(flag): <class 'bool'>

type(flag) is bool: True
type(flag) is int:  False

isinstance(flag, bool): True
isinstance(flag, int):  True
```

This happens because `bool` is a subclass of `int`. The exact type is `bool`, but the object also passes an `int` family check.

`isinstance()` can also check against several possible types at once:

```
values = [42, 3.14, 2 + 5j, "D'oh!", True]

for value in values:
    is_number = isinstance(value, (int, float, complex))
    print(f"{repr(value):<10} numeric-like: {is_number}")
```

Possible output:

```
42          numeric-like: True
3.14        numeric-like: True
(2+5j)      numeric-like: True
"D'oh!"     numeric-like: False
True        numeric-like: True
```

The final line is true because `bool` is integer-compatible. Sometimes that is desirable. Sometimes it is not.

If booleans should be excluded, say so explicitly:

```
values = [42, 3.14, True, False]

for value in values:
    is_plain_number = (
        isinstance(value, (int, float)) and
        not isinstance(value, bool)
    )

    print(f"{repr(value):<8} plain number: {is_plain_number}")
```

Possible output:

```
42      plain number: True
3.14    plain number: True
True    plain number: False
False   plain number: False
```

The practical rule is:

Use `type()` when you need to inspect the exact runtime type. Use `isinstance()` when you need to ask whether an object belongs to an acceptable category of types.

The structural summary for dynamic assignment is:

```
name = 42      -> name references an int object
name = "text"  -> same name now references a str object
name = None    -> same name now references the None singleton

type(name)     -> inspect the current object's exact type
isinstance(...) -> test whether the current object fits a type category
```

Python assignment is dynamic because the name can move. Python objects remain typed because each object still carries its runtime type and behavior.

4.4.2 Strong Typing in Operations

Python names are dynamically bound, but Python operations are strongly checked at runtime. This means that a name may be rebound to different kinds of objects, but an operation still requires objects that know how to participate in that operation.

Dynamic typing does not mean “anything combines with anything”.

```
value = 42
value = "42"

print(f"value: {value!r}")
print(f"type: {type(value)}")
```

Possible output:

```
value: '42'
type: <class 'str'>
```

The name `value` was allowed to move from an integer object to a string object. But once it refers to a string object, string rules apply.

Why Heterogeneous Operations Such as `"3" + 4` Fail Instead of Silently Converting

The expression:

```
"3" + 4
```

looks tempting, but Python rejects it.

A safe demonstration:

```

text_value = "3"
number_value = 4

try:
    result = text_value + number_value
    print(result)
except TypeError as err:
    print("operation failed")
    print(f"message: {err}")
    print(f"type(err): {type(err)}")

```

Possible output:

```

operation failed
message: can only concatenate str (not "int") to str
type(err): <class 'TypeError'>

```

The reason is that `+` does not have one universal meaning. For integers, `+` means numeric addition. For strings, `+` means concatenation.

```

print(3 + 4)
print("3" + "4")

print()
print(f"type(3 + 4):      {type(3 + 4)}")
print(f"type('3' + '4'): {type('3' + '4')}")

```

Possible output:

```

7
34

type(3 + 4):      <class 'int'>
type('3' + '4'): <class 'str'>

```

So the mixed expression is ambiguous:

```
"3" + 4
```

Should Python produce the integer 7? Should it produce the string "34"? Should it convert the integer to text, or the text to an integer?

Python refuses to guess.

This is strong typing in practice:

Python does not silently convert unrelated object types just to make an operation succeed.

Inspect the two objects:

```

text_value = "3"
number_value = 4

print(f"text_value:    {text_value!r}")
print(f"number_value: {number_value!r}")

print()
print(f"type(text_value): {type(text_value)}")

```

```

print(f"type(number_value): {type(number_value)}")

print()
print("selected str methods:")
print(f"{'upper':<12} {'upper' in dir(text_value)}")
print(f"{'replace':<12} {'replace' in dir(text_value)}")
print(f"{'__add__':<12} {'__add__' in dir(text_value)}")

print()
print("selected int methods:")
print(f"{'bit_length':<12} {'bit_length' in dir(number_value)}")
print(f"{'__add__':<12} {'__add__' in dir(number_value)}")
print(f"{'__mul__':<12} {'__mul__' in dir(number_value)}")

```

Possible output:

```

text_value:      '3'
number_value:    4

type(text_value): <class 'str'>
type(number_value): <class 'int'>

selected str methods:
upper           True
replace         True
__add__         True

selected int methods:
bit_length      True
__add__         True
__mul__         True

```

Both objects have `__add__` behavior, but not for the same kind of partner. The string can concatenate with another string. The integer can add with another number. Python will not invent a hidden conversion between them.

More examples:

```

examples = [
    ("3" + 4, "3" + 4),
    ("D" * 3, "D" * 3),
    ("D" - 1, "D" - 1),
]

for label, source_code in examples:
    print(f"trying: {label}")

    try:
        result = eval(source_code)
        print(f"result: {result!r}")
        print(f"type: {type(result)}")
    except TypeError as err:
        print("TypeError")
        print(f"message: {err}")

print()

```

Possible output:

```
trying: "3" + 4
TypeError
message: can only concatenate str (not "int") to str

trying: "D" * 3
result: 'DDD'
type:   <class 'str'>

trying: "D" - 1
TypeError
message: unsupported operand type(s) for -: 'str' and 'int'
```

The expression `"D" * 3` works because string repetition by an integer is a defined operation. The expression `"D" - 1` fails because subtracting an integer from a string has no defined meaning.

Strong typing does not mean that mixed-type operations are always forbidden. It means that mixed-type operations must be explicitly supported by the involved types.

Explicit Conversion with `int()`, `float()`, `complex()`, `bool()`, and `str()`

When the programmer wants conversion, the programmer must say so.

To add numerically:

```
text_value = "3"
number_value = 4

result = int(text_value) + number_value

print(f"result: {result}")
print(f"type:   {type(result)}")
```

Possible output:

```
result: 7
type:   <class 'int'>
```

To concatenate as text:

```
text_value = "3"
number_value = 4

result = text_value + str(number_value)

print(f"result: {result!r}")
print(f"type:   {type(result)}")
```

Possible output:

```
result: '34'
type:   <class 'str'>
```

The two programs give different results because they express different intentions. Python provides built-in conversion functions:


```
int()
float()
complex()
bool()
str()
```

A compact demonstration:

```
text = "42"

as_int = int(text)
as_float = float(text)
as_complex = complex(text)
as_bool = bool(text)
as_str = str(as_int)

print(f"original:    {text!r:<10} type={type(text)}")
print(f"int:         {as_int!r:<10} type={type(as_int)}")
print(f"float:       {as_float!r:<10} type={type(as_float)}")
print(f"complex:     {as_complex!r:<10} type={type(as_complex)}")
print(f"bool:        {as_bool!r:<10} type={type(as_bool)}")
print(f"str:         {as_str!r:<10} type={type(as_str)}")
```

Possible output:

```
original:    '42'          type=<class 'str'>
int:         42           type=<class 'int'>
float:       42.0         type=<class 'float'>
complex:     (42+0j)      type=<class 'complex'>
bool:        True         type=<class 'bool'>
str:         '42'         type=<class 'str'>
```

Conversions are not magic. They obey the rules of the target type.

This works:

```
print(int("42"))
print(float("3.14"))
print(complex("2+5j"))
```

Possible output:

```
42
3.14
(2+5j)
```

This fails:

```
try:
    number = int("3.14")
    print(number)
except ValueError as err:
    print("conversion failed")
    print(f"message: {err}")
    print(f"type(err): {type(err)}")
```

Possible output:

```

conversion failed
message: invalid literal for int() with base 10: '3.14'
type(err): <class 'ValueError'>

```

The string "3.14" is valid floating-point text, but it is not valid integer text.

A two-step conversion is possible if the programmer accepts the meaning:

```

text = "3.14"

as_float = float(text)
as_int = int(as_float)

print(f"as_float: {as_float}, type={type(as_float)}")
print(f"as_int:    {as_int}, type={type(as_int)}")

```

Possible output:

```

as_float: 3.14, type=<class 'float'>
as_int:    3, type=<class 'int'>

```

This does not round. It truncates toward zero.

```

values = [3.99, -3.99, 42.0]

for value in values:
    converted = int(value)
    print(f"int({value}) = {converted}")

```

Possible output:

```

int(3.99) = 3
int(-3.99) = -3
int(42.0) = 42

```

The function `bool()` converts by truth-value testing:

```

values = [0, 1, 42, "", "0", "False", None]

for value in values:
    converted = bool(value)
    print(f"{repr(value):<8} -> {converted:<5} type={type(converted)}")

```

Possible output:

```

0          -> False type=<class 'bool'>
1          -> True  type=<class 'bool'>
42         -> True  type=<class 'bool'>
''         -> False type=<class 'bool'>
'0'        -> True  type=<class 'bool'>
'False'    -> True  type=<class 'bool'>
None       -> False type=<class 'bool'>

```

The strings "0" and "False" are true because they are non-empty strings. `bool()` does not parse English meaning. It checks truthiness.

The function `str()` creates a text representation:

```

values = [42, 3.14, True, None, 2 + 5j]

for value in values:
    text = str(value)
    print(f"{repr(value):<10} -> {text!r:<10} type={type(text)}")

```

Possible output:

```

42          -> '42'          type=<class 'str'>
3.14        -> '3.14'        type=<class 'str'>
True        -> 'True'        type=<class 'str'>
None        -> 'None'        type=<class 'str'>
(2+5j)      -> '(2+5j)'      type=<class 'str'>

```

The practical rule is:

If the program needs a conversion, write the conversion explicitly. Do not depend on Python to guess whether text should become a number or a number should become text.

Numeric Promotion Boundaries: Integer, Float, Complex, and Boolean Interactions

Python does allow many mixed numeric operations. These are not silent conversions between unrelated domains. They are defined interactions inside Python's numeric tower.

A simple table:

```

values = [
    ("int + int",      40 + 2),
    ("int + float",    40 + 2.5),
    ("int + complex",  40 + 2j),
    ("float + complex", 3.5 + 2j),
    ("bool + int",     True + 41),
    ("bool + float",   True + 0.5),
]

for label, result in values:
    print(f"{label:<18} -> {result!r:<10} type={type(result)}")

```

Possible output:

```

int + int      -> 42          type=<class 'int'>
int + float    -> 42.5        type=<class 'float'>
int + complex  -> (40+2j)     type=<class 'complex'>
float + complex -> (3.5+2j)    type=<class 'complex'>
bool + int     -> 42          type=<class 'int'>
bool + float   -> 1.5         type=<class 'float'>

```

The broad pattern is:

```

int + int      -> int
int + float    -> float
int + complex  -> complex
float + complex -> complex
bool + int     -> int
bool + float   -> float

```

This happens because Python defines these numeric combinations. The result type is chosen so that it can represent the result of the operation.

Inspect the operands:

```
a = 42
b = 0.5
c = 2j
d = True

print(f"a: {a!r:<8} type={type(a)}")
print(f"b: {b!r:<8} type={type(b)}")
print(f"c: {c!r:<8} type={type(c)}")
print(f"d: {d!r:<8} type={type(d)}")
```

Possible output:

```
a: 42          type=<class 'int'>
b: 0.5         type=<class 'float'>
c: 2j          type=<class 'complex'>
d: True        type=<class 'bool'>
```

Now combine them:

```
print(f"a + b: {a + b!r:<8} type={type(a + b)}")
print(f"a + c: {a + c!r:<8} type={type(a + c)}")
print(f"b + c: {b + c!r:<8} type={type(b + c)}")
print(f"d + a: {d + a!r:<8} type={type(d + a)}")
```

Possible output:

```
a + b: 42.5     type=<class 'float'>
a + c: (42+2j)  type=<class 'complex'>
b + c: (0.5+2j) type=<class 'complex'>
d + a: 43       type=<class 'int'>
```

This is still strongly typed. Python is not guessing that arbitrary objects should convert. It is applying defined numeric rules.

For example, `int` and `str` are not part of one shared numeric operation:

```
try:
    result = 42 + "1"
except TypeError as err:
    print("operation failed")
    print(f"message: {err}")
```

Possible output:

```
operation failed
message: unsupported operand type(s) for +: 'int' and 'str'
```

Similarly, `complex` supports arithmetic but not ordering:

```
try:
    print((3 + 4j) < 10)
except TypeError as err:
    print("comparison failed")
    print(f"message: {err}")
```

Possible output:

```
comparison failed
message: '<' not supported between instances of 'complex' and 'int'
```

Equality is allowed:

```
print(f"3 + 0j == 3:    {3 + 0j == 3}")
print(f"3.0 == 3:      {3.0 == 3}")
print(f"True == 1:     {True == 1}")
print(f"False == 0:    {False == 0}")
```

Possible output:

```
3 + 0j == 3:    True
3.0 == 3:       True
True == 1:      True
False == 0:     True
```

This is numeric equality, not type identity.

```
print(f"type(3 + 0j): {type(3 + 0j)}")
print(f"type(3):      {type(3)}")

print()
print(f"(3 + 0j) == 3:      {(3 + 0j) == 3}")
print(f"type(3 + 0j) is int: {type(3 + 0j) is int}")
```

Possible output:

```
type(3 + 0j): <class 'complex'>
type(3):      <class 'int'>

(3 + 0j) == 3:      True
type(3 + 0j) is int: False
```

Booleans require special care because `bool` is a subclass of `int`.

```
flag = True
```

```
print(f"flag: {flag}")
print(f"type(flag): {type(flag)}")

print()
print(f"isinstance(flag, bool): {isinstance(flag, bool)}")
print(f"isinstance(flag, int):  {isinstance(flag, int)}")

print()
print(f"flag + 41: {flag + 41}")
print(f"type(flag + 41): {type(flag + 41)}")
```

Possible output:

```
flag: True
type(flag): <class 'bool'>

isinstance(flag, bool): True
isinstance(flag, int): True

flag + 41: 42
type(flag + 41): <class 'int'>
```

This works, but it is often bad style unless the programmer is deliberately counting truth values.
Acceptable:

```
passed_test_1 = True
passed_test_2 = False
passed_test_3 = True

passed_count = passed_test_1 + passed_test_2 + passed_test_3

print(f"passed_count: {passed_count}")
```

Possible output:

```
passed_count: 2
```

Less clear:

```
total = True + 41
print(total)
```

Possible output:

```
42
```

The practical numeric promotion rule is:

Python defines mixed operations among its numeric types, commonly moving from `bool/int` toward `float` and then toward `complex` when needed. This is not the same as arbitrary silent conversion between unrelated types such as `str` and `int`.

The operational summary is:

```
"3" + 4          -> TypeError
int("3") + 4     -> 7
"3" + str(4)     -> "34"

42 + 0.5         -> 42.5
42 + 2j          -> (42+2j)
True + 41        -> 42
```

Names are flexible. Operations are checked. Objects keep their runtime type rules.

4.4.3 Structural Summary

The preceding sections separate two ideas that are often confused:

- Python names are dynamically bound.
- Python objects are strongly typed at runtime.

This means that Python is flexible at the level of assignment, but strict at the level of operation. A name may point to different objects at different moments, but each object still brings its own type rules with it.

```
value = 42
print(f"value={value!r:<10} type={type(value)}")

value = "42"
print(f"value={value!r:<10} type={type(value)}")

value = None
print(f"value={value!r:<10} type={type(value)}")
```

Possible output:

```
value=42          type=<class 'int'>
value='42'        type=<class 'str'>
value=None        type=<class 'NoneType'>
```

The name `value` moved. The objects did not lose their type identity.

Categorizing Python as a Dynamically Bound, Strongly Verified Runtime Typing Environment

Python is dynamically typed because names do not have fixed declared types.

There is no declaration such as:

```
int value;
```

Instead, assignment binds a name to an object:

```
value = 42
```

Later, the same name can be rebound:

```
value = "No soup for you."
```

A compact trace:

```
value = 42

print("first binding")
print(f"value: {value!r}")
print(f"type: {type(value)}")
print(f"id: {id(value)}")

value = "No soup for you."

print()
print("second binding")
print(f"value: {value!r}")
print(f"type: {type(value)}")
print(f"id: {id(value)}")
```

Possible output:

```
first binding
value: 42
type:  <class 'int'>
id:    140734432946520

second binding
value: 'No soup for you.'
type:  <class 'str'>
id:    2209266113904
```

This is dynamic binding.

However, Python is also strongly checked at runtime. The interpreter does not ignore object types when operations execute.

```
left = "3"
right = 4

try:
    result = left + right
    print(result)
except TypeError as err:
    print("operation failed")
    print(f"message: {err}")
    print(f"type(err): {type(err)}")
```

Possible output:

```
operation failed
message: can only concatenate str (not "int") to str
type(err): <class 'TypeError'>
```

The failure is not caused by the name `left`. It is caused by the object reached through `left`. That object is a string, and string addition does not accept an integer as the right operand.

The correct operation must be stated explicitly:

```
text_number = "3"
real_number = 4

numeric_result = int(text_number) + real_number
text_result = text_number + str(real_number)

print(f"numeric_result: {numeric_result!r}, type={type(numeric_result)}")
print(f"text_result:    {text_result!r}, type={type(text_result)}")
```

Possible output:

```
numeric_result: 7, type=<class 'int'>
text_result:    '34', type=<class 'str'>
```

So the proper classification is not simply “dynamic”.

A more precise description is:

Python uses dynamically bound names and strongly verified runtime operations.

That means:

- names can be rebound freely;
- objects carry runtime types;
- operators consult object behavior;
- unsupported combinations fail instead of being silently converted;
- explicit conversion is the programmer's responsibility.

The Practical Rule: Names Are Flexible, Objects Keep Their Runtime Type

The practical rule for reading Python code is:

A name may change what it references, but the object it currently references keeps its type and behavior.

Example:

```
thing = 42

print("integer phase")
print(f"thing: {thing!r}")
print(f"type: {type(thing)}")
print(f"bit_length available: {'bit_length' in dir(thing)}")
print(f"upper available:      {'upper' in dir(thing)}")

thing = "D'oh!"

print()
print("string phase")
print(f"thing: {thing!r}")
print(f"type: {type(thing)}")
print(f"bit_length available: {'bit_length' in dir(thing)}")
print(f"upper available:      {'upper' in dir(thing)}")
```

Possible output:

```
integer phase
thing: 42
type: <class 'int'>
bit_length available: True
upper available:      False

string phase
thing: "D'oh!"
type: <class 'str'>
bit_length available: False
upper available:      True
```

The name `thing` did not carry the methods. The current object did.
The same rule explains operation behavior:

```

thing = 42
print(thing + 8)

thing = "D'oh!"
print(thing + "!")

```

Possible output:

```

50
D'oh!

```

Both uses of `+` work, but they do not mean exactly the same operation:

- for integers, `+` performs numeric addition;
- for strings, `+` performs text concatenation.

The object type decides which behavior is available.

If the operation is not defined, Python rejects it:

```

thing = "42"

try:
    print(thing + 8)
except TypeError as err:
    print("operation failed")
    print(f"message: {err}")

```

Possible output:

```

operation failed
message: can only concatenate str (not "int") to str

```

A final compact script:

```

values = [42, 3.14, 2 + 5j, True, None, "No soup for you."]

for value in values:
    print(f"{repr(value):<20} type={type(value)}")

```

Possible output:

```

42                type=<class 'int'>
3.14              type=<class 'float'>
(2+5j)            type=<class 'complex'>
True              type=<class 'bool'>
None              type=<class 'NoneType'>
'No soup for you.' type=<class 'str'>

```

Every value is an object with a runtime type. A name can point to any of them, but it does not erase their differences.

The structural summary for this whole section is:

assignment:
flexible name-to-object binding

reassignment:
same name, possibly different object type

type():
exact current runtime type

isinstance():
category or family test

operation:
checked against the current objects involved

conversion:
explicit when crossing type boundaries

The final rule is:

Python is permissive about rebinding names, but strict about what objects can do together.

Chapter 5

Sequential Component Data Architectures (Strings, Lists, and Tuples)

A single value is useful, but most real programs do not work with isolated values for very long. They work with ordered groups: characters in a text, rows of values, coordinates, records, command-line arguments, tokens, names, paths, and many other structures whose components must be stored and accessed in sequence.

This chapter studies Python’s core sequential data architectures: `str`, `list`, and `tuple`. All three support positional access, slicing, length testing, membership testing, and iteration. They share the general sequence model, but they do not share the same internal meaning. A string is immutable Unicode text. A list is a mutable dynamic array of object references. A tuple is a fixed-size immutable sequence of object references.

The goal is practical and structural at the same time. We will write code that indexes, slices, searches, formats, mutates, copies, sorts, and unpacks these objects. But we will also keep the runtime model visible: Python variables refer to objects, lists and tuples store references, strings store text rather than raw bytes, and every operation must be understood through the object that receives it.

This is the point where the course becomes more directly programmatic. The earlier chapters established the execution and object model. Here that model is used repeatedly in ordinary code, because sequence handling is one of the basic daily activities of Python programming.

5.1 The General Sequence Model in Python

Python uses the word *sequence* for objects whose components are arranged in a definite order. A sequence is not just “several values together”. It is several values placed in positions, so that each component can be reached by its location.

This section introduces the shared access model used by strings, lists, and tuples. Before studying the internal memory behavior of each object separately, we first need the common surface rules: indexing, slicing, length checking, membership testing, traversal, concatenation, repetition, comparison, and the basic split between mutable and immutable sequences.

The goal is practical. A programmer must be able to look at a sequence and know how Python reads it, cuts it, walks through it, compares it, and combines it with another sequence. Later sections will explain why strings, lists, and tuples behave differently internally, but their common positional behavior starts here.

5.1.1 Ordered Component Storage and Positional Access

A sequence is an object that stores components in a fixed left-to-right order. The word “fixed” does not mean that the object itself is always immutable. A list can still be changed. It means that, at any given moment, the components have positions.

Python has several built-in sequence types. In this chapter, the main ones are:

- `str`: a sequence of text characters;

- **list**: a mutable sequence of object references;
- **tuple**: an immutable sequence of object references.

The same basic access rules apply to all of them: indexing, slicing, length testing, membership testing, and iteration.

```
text = "D'oh!"
items = ["spam", "eggs", 42, "larch"]
point = (10, 20, 30)

print(f"text: {text!r}")
print(f"items: {items!r}")
print(f"point: {point!r}")

print()
print(f"type(text): {type(text)}")
print(f"type(items): {type(items)}")
print(f"type(point): {type(point)}")
```

Possible output:

```
text: "D'oh!"
items: ['spam', 'eggs', 42, 'larch']
point: (10, 20, 30)

type(text): <class 'str'>
type(items): <class 'list'>
type(point): <class 'tuple'>
```

At the object level, these are different types. At the sequence level, they share a common positional model.

A first inspection with `dir()` shows that these objects expose many operations. The full output of `dir()` is long, so the following program checks only a few names that matter for basic sequence behavior.

```
text = "D'oh!"
items = ["spam", "eggs", 42, "larch"]
point = (10, 20, 30)

names = ["__len__", "__getitem__", "__contains__", "index", "count"]

print("selected sequence-related attributes")
print()

for obj_name, obj in [("text", text), ("items", items), ("point", point)]:
    print(f"{obj_name}: {type(obj)}")
    for name in names:
        print(f"  {name:<14} {name in dir(obj)}")
    print()
```

Possible output:

```
selected sequence-related attributes
```

```
text: <class 'str'>
  __len__      True
  __getitem__  True
  __contains__ True
  index        True
  count        True

items: <class 'list'>
  __len__      True
  __getitem__  True
  __contains__ True
  index        True
  count        True

point: <class 'tuple'>
  __len__      True
  __getitem__  True
  __contains__ True
  index        True
  count        True
```

The names beginning and ending with double underscores are special method names. They are not usually called directly in beginner-level code. Instead, Python calls them when normal syntax is used:

- `len(obj)` uses the object's length behavior;
- `obj[i]` uses the object's item-access behavior;
- `x in obj` uses the object's membership behavior.

So the syntax is simple, but the action still belongs to the object.

Indexing: Zero-Based Component Addressing with Positive and Negative Offsets

Indexing extracts one component from a sequence. Python indexes start at zero, not one. The first component is at position 0, the second at position 1, and so on.

```
quote = "Ni!"

print(f"quote: {quote!r}")
print(f"quote[0]: {quote[0]!r}")
print(f"quote[1]: {quote[1]!r}")
print(f"quote[2]: {quote[2]!r}")
```

Possible output:

```
quote: 'Ni!'
quote[0]: 'N'
quote[1]: 'i'
quote[2]: '!'
```

For a string, indexing returns a one-character string object. Python does not have a separate character type comparable to C's `char`.

```
quote = "Ni!"
ch = quote[0]

print(f"ch: {ch!r}")
print(f"type(ch): {type(ch)}")
```

Possible output:

```
ch: 'N'
type(ch): <class 'str'>
```

The same positional rule applies to lists and tuples.

```
items = ["Kramer", "Jerry", "Elaine", "George"]
point = (42, 100, 200)

print(f"items[0]: {items[0]}")
print(f"items[1]: {items[1]}")
print(f"items[2]: {items[2]}")
print(f"items[3]: {items[3]}")

print()
print(f"point[0]: {point[0]}")
print(f"point[1]: {point[1]}")
print(f"point[2]: {point[2]}")
```

Possible output:

```
items[0]: Kramer
items[1]: Jerry
items[2]: Elaine
items[3]: George

point[0]: 42
point[1]: 100
point[2]: 200
```

A positive index counts from the left. A negative index counts from the right.

```
line = "Nobody expects the Spanish Inquisition"

print(f"line[0]: {line[0]!r}")
print(f"line[1]: {line[1]!r}")
print(f"line[-1]: {line[-1]!r}")
print(f"line[-2]: {line[-2]!r}")
```

Possible output:

```
line[0]: 'N'
line[1]: 'o'
line[-1]: 'n'
line[-2]: 'o'
```

Negative indexing is not a different storage direction. It is a convenience rule. Python translates the negative offset into a real position from the left.

For example, in a four-element list:

```
names = ["Homer", "Marge", "Bart", "Lisa"]

print(f"names[0]: {names[0]}")
print(f"names[1]: {names[1]}")
print(f"names[2]: {names[2]}")
print(f"names[3]: {names[3]}")

print()
print(f"names[-1]: {names[-1]}")
print(f"names[-2]: {names[-2]}")
print(f"names[-3]: {names[-3]}")
print(f"names[-4]: {names[-4]}")
```

Possible output:

```
names[0]: Homer
names[1]: Marge
names[2]: Bart
names[3]: Lisa

names[-1]: Lisa
names[-2]: Bart
names[-3]: Marge
names[-4]: Homer
```

The valid index range depends on the sequence length. For a sequence of length 4, the valid positive indexes are 0, 1, 2, and 3. The valid negative indexes are -1, -2, -3, and -4.

An index outside the valid range raises an error.

```
names = ["Homer", "Marge", "Bart", "Lisa"]

print(names[4])
```

Possible output:

```
IndexError: list index out of range
```

The error is useful. Python does not return random memory content. It refuses the invalid access.

Slicing: Extracting Sub-Sequences with [start:stop:step]

Slicing extracts a new sequence from part of an existing sequence. The general form is:

```
sequence[start:stop:step]
```

The important rule is that **start** is included, but **stop** is excluded.

```
text = "abcdef"

print(f"text: {text!r}")
print(f"text[0:3]: {text[0:3]!r}")
print(f"text[2:5]: {text[2:5]!r}")
```

Possible output:


```
text:      'abcdef'
text[0:3]: 'abc'
text[2:5]: 'cde'
```

The slice `text[0:3]` means: start at index 0, move up to index 3, but do not include index 3.

```
text = "abcdef"

print(f"index 0: {text[0]!r}")
print(f"index 1: {text[1]!r}")
print(f"index 2: {text[2]!r}")
print(f"index 3: {text[3]!r}")

print()
print(f"text[0:3]: {text[0:3]!r}")
```

Possible output:

```
index 0: 'a'
index 1: 'b'
index 2: 'c'
index 3: 'd'

text[0:3]: 'abc'
```

If **start** is omitted, Python starts at the beginning. If **stop** is omitted, Python continues to the end.

```
quote = "These pretzels are making me thirsty"

print(f"quote[:5]: {quote[:5]!r}")
print(f"quote[6:14]: {quote[6:14]!r}")
print(f"quote[15:]: {quote[15:]!r}")
```

Possible output:

```
quote[:5]: 'These'
quote[6:14]: 'pretzels'
quote[15:]: 'are making me thirsty'
```

The same slicing model applies to lists.

```
items = ["spam", "eggs", "bacon", "beans", 42]

print(f"items: {items}")
print(f"items[:2]: {items[:2]}")
print(f"items[2:4]: {items[2:4]}")
print(f"items[3:]: {items[3:]}")
```

Possible output:

```
items:      ['spam', 'eggs', 'bacon', 'beans', 42]
items[:2]:  ['spam', 'eggs']
items[2:4]: ['bacon', 'beans']
items[3:]:  ['beans', 42]
```

The result of slicing has the same sequence type as the original sequence.

```
text = "abcdef"
items = ["a", "b", "c", "d"]
point = (10, 20, 30, 40)

text_part = text[1:3]
items_part = items[1:3]
point_part = point[1:3]

print(f"text_part: {text_part!r:<20} type: {type(text_part)}")
print(f"items_part: {items_part!r:<20} type: {type(items_part)}")
print(f"point_part: {point_part!r:<20} type: {type(point_part)}")
```

Possible output:

```
text_part:  'bc'                type: <class 'str'>
items_part: ['b', 'c']          type: <class 'list'>
point_part: (20, 30)            type: <class 'tuple'>
```

The optional `step` controls how many positions Python jumps after taking a component.

```
text = "abcdefghij"

print(f"text[0:10:1]: {text[0:10:1]!r}")
print(f"text[0:10:2]: {text[0:10:2]!r}")
print(f"text[1:10:2]: {text[1:10:2]!r}")
```

Possible output:

```
text[0:10:1]: 'abcdefghij'
text[0:10:2]: 'acegi'
text[1:10:2]: 'bdfhj'
```

A negative step walks backward.

```
text = "abcdef"

print(f"text[::-1]: {text[::-1]!r}")
print(f"text[4:1:-1]: {text[4:1:-1]!r}")
```

Possible output:

```
text[::-1]:  'fedcba'
text[4:1:-1]: 'edc'
```

The compact form `text[::-1]` is often used to create a reversed copy of a sequence.

Slicing is safer than indexing when boundaries are outside the sequence. Invalid indexing raises an error, but slicing trims the requested range to the available components.

```
text = "abc"

print(f"text[0:50]: {text[0:50]!r}")
print(f"text[20:30]: {text[20:30]!r}")
```

Possible output:

```
text[0:50]: 'abc'
text[20:30]: ''
```

This does not mean slicing ignores all mistakes. A step of zero is impossible because Python would not know how to move through the sequence.

```
text = "abc"

print(text[0:3:0])
```

Possible output:

```
ValueError: slice step cannot be zero
```

Length, Membership, and Iteration: `len()`, `in`, and Sequential Traversal

The function `len()` returns the number of components in a sequence.

```
text = "D'oh!"
items = ["spam", "eggs", 42]
point = (10, 20, 30, 40)

print(f"len(text): {len(text)}")
print(f"len(items): {len(items)}")
print(f"len(point): {len(point)}")
```

Possible output:

```
len(text): 5
len(items): 3
len(point): 4
```

For a string, `len()` counts text components, not printed screen width and not storage bytes. The byte representation issue belongs to the later text-encoding section.

The operator `in` tests membership.

```
text = "Nobody expects the Spanish Inquisition"
items = ["spam", "eggs", 42, "larch"]
point = (10, 20, 30)

print(f"{'Spanish' in text} = {True}")
print(f"{'French' in text} = {False}")

print()
print(f"{42 in items} = {True}")
print(f"{'eggs' in items} = {True}")
print(f"{'beans' in items} = {False}")

print()
print(f"{20 in point} = {True}")
print(f"{99 in point} = {False}")
```

Possible output:

```
'Spanish' in text = True
'French' in text = False

42 in items = True
'eggs' in items = True
'beans' in items = False

20 in point = True
99 in point = False
```

For strings, `in` can test whether a smaller string occurs inside a larger string. For lists and tuples, `in` tests whether an element equal to the searched value exists as one component.

```
text = "abc"
items = ["abc", "def"]

print(f"{'b' in text = }")
print(f"{'b' in items = }")
print(f"{'abc' in items = }")
```

Possible output:

```
'b' in text = True
'b' in items = False
'abc' in items = True
```

This distinction matters. The string `"abc"` contains the smaller string `"b"`. The list `["abc", "def"]` does not contain `"b"` as a separate list element.

Iteration means visiting the components of a sequence one at a time.

```
names = ["Homer", "Marge", "Bart", "Lisa"]

for name in names:
    print(f"current name: {name}")
```

Possible output:

```
current name: Homer
current name: Marge
current name: Bart
current name: Lisa
```

The same idea works for strings. Iterating over a string visits one-character string objects.

```
word = "Ni!"

for ch in word:
    print(f"ch={ch!r:<4} type={type(ch)}")
```

Possible output:

```
ch='N'   type=<class 'str'>
ch='i'   type=<class 'str'>
ch='!'   type=<class 'str'>
```

When both the position and the component are needed, `enumerate()` is clearer than manually maintaining a counter.

```
names = ["Jerry", "Elaine", "George", "Kramer"]

for index, name in enumerate(names):
    print(f"{index:>2}: {name}")
```

Possible output:

```
0: Jerry
1: Elaine
2: George
3: Kramer
```

A loop can also accumulate a result. The following program walks through a list of numbers and computes their sum. This is intentionally simple: the point is traversal, not a complicated mathematical formula.

```
numbers = [10, 20, 12]
total = 0

for nr in numbers:
    print(f"before: total={total:<3} nr={nr}")
    total = total + nr
    print(f"after:  total={total:<3}")
    print()

print(f"final total: {total}")
```

Possible output:

```
before: total=0   nr=10
after:  total=10

before: total=10  nr=20
after:  total=30

before: total=30  nr=12
after:  total=42

final total: 42
```

The loop does not need to know whether the sequence is backed by a string object, a list object, or a tuple object. It asks the object for its next component according to Python's iteration protocol.

The practical rule for this section is:

A sequence is an ordered object. Indexing selects one component. Slicing creates a smaller sequence. `len()` counts components. `in` tests containment. A `for` loop traverses the components from left to right.

5.1.2 Sequence Operators and Shared Behaviors

Python sequences share several operators. The same symbols can work on strings, lists, and tuples, but they do not always mean numeric arithmetic.

For sequences, the most important shared operators are:

- `+`: concatenation;

- *: repetition;
- ==: equality comparison;
- <, >, <=, >=: lexicographic ordering, when the compared components support ordering.

These operators are still type-checked at runtime. Python does not abandon the strong operation-checking rule introduced earlier. A sequence operation succeeds only when the involved objects define that operation for that kind of partner.

```
text = "spam"
items = ["spam", "eggs"]
point = (10, 20)

names = ["__add__", "__mul__", "__eq__", "__lt__"]

for obj_name, obj in [("text", text), ("items", items), ("point", point)]:
    print(f"{obj_name}: {type(obj)}")
    for name in names:
        print(f"  {name:<10} {name in dir(obj)}")
    print()
```

Possible output:

```
text: <class 'str'>
  __add__      True
  __mul__      True
  __eq__       True
  __lt__       True

items: <class 'list'>
  __add__      True
  __mul__      True
  __eq__       True
  __lt__       True

point: <class 'tuple'>
  __add__      True
  __mul__      True
  __eq__       True
  __lt__       True
```

The presence of these special methods does not mean that every possible operand combination is accepted. It means that the object has some defined behavior for these operators.

Concatenation and Repetition: + and * Across Compatible Sequence Types

For compatible sequences, + creates a new sequence containing the left sequence followed by the right sequence.

```
text1 = "Hello, "
text2 = "Newman!"

items1 = ["spam", "eggs"]
items2 = ["bacon", 42]
```

```

point1 = (10, 20)
point2 = (30, 40)

print(f"text1 + text2: {text1 + text2!r}")
print(f"items1 + items2: {items1 + items2!r}")
print(f"point1 + point2: {point1 + point2!r}")

print()
print(f"type(text1 + text2): {type(text1 + text2)}")
print(f"type(items1 + items2): {type(items1 + items2)}")
print(f"type(point1 + point2): {type(point1 + point2)}")

```

Possible output:

```

text1 + text2: 'Hello, Newman!'
items1 + items2: ['spam', 'eggs', 'bacon', 42]
point1 + point2: (10, 20, 30, 40)

type(text1 + text2): <class 'str'>
type(items1 + items2): <class 'list'>
type(point1 + point2): <class 'tuple'>

```

The result has the same type as the sequences being concatenated.

Concatenation does not mean that lists and tuples can be freely mixed.

```

items = ["Homer", "Marge"]
point = ("Bart", "Lisa")

try:
    result = items + point
    print(result)
except TypeError as err:
    print("operation failed")
    print(f"message: {err}")

```

Possible output:

```

operation failed
message: can only concatenate list (not "tuple") to list

```

The components may be heterogeneous inside a list, but the two outer sequence objects must still be compatible for concatenation.

```

items1 = ["Homer", 42]
items2 = [3.14, True]

result = items1 + items2

print(f"result: {result!r}")
print(f"type: {type(result)}")

```

Possible output:

```

result: ['Homer', 42, 3.14, True]
type: <class 'list'>

```

The heterogeneous part is inside the list. The operation itself is still list concatenation.
The operator `*` repeats a sequence a specified number of times.

```
text = "Ni! "  
items = ["spam", 42]  
point = ("D'oh!",)  
  
print(f"text * 3: {text * 3!r}")  
print(f"items * 3: {items * 3!r}")  
print(f"point * 3: {point * 3!r}")
```

Possible output:

```
text * 3:  'Ni! Ni! Ni! '  
items * 3:  ['spam', 42, 'spam', 42, 'spam', 42]  
point * 3:  ("D'oh!", "D'oh!", "D'oh!")
```

The number on the right must be an integer.

```
text = "yada "  
  
print(f"text * 2: {text * 2!r}")  
  
try:  
    result = text * 2.5  
    print(result)  
except TypeError as err:  
    print("operation failed")  
    print(f"message: {err}")
```

Possible output:

```
text * 2:  'yada yada '  
operation failed  
message: can't multiply sequence by non-int of type 'float'
```

The integer can also appear on the left.

```
text = "ha"  
items = ["larch"]  
  
print(f"text * 4: {text * 4!r}")  
print(f"4 * text: {4 * text!r}")  
  
print()  
print(f"items * 3: {items * 3!r}")  
print(f"3 * items: {3 * items!r}")
```

Possible output:

```
text * 4:  'hahahaha'  
4 * text:  'hahahaha'  
  
items * 3:  ['larch', 'larch', 'larch']  
3 * items:  ['larch', 'larch', 'larch']
```


A repetition count of zero creates an empty sequence of the same type. A negative repetition count also produces an empty sequence.

```
text = "abc"
items = ["a", "b"]
point = (1, 2)

print(f"text * 0: {text * 0!r}")
print(f"items * 0: {items * 0!r}")
print(f"point * 0: {point * 0!r}")

print()
print(f"text * -1: {text * -1!r}")
print(f"items * -1: {items * -1!r}")
print(f"point * -1: {point * -1!r}")
```

Possible output:

```
text * 0: ''
items * 0: []
point * 0: ()

text * -1: ''
items * -1: []
point * -1: ()
```

For immutable elements such as numbers and strings, repetition is usually harmless. With mutable elements inside a list, repetition deserves caution because repeated positions may refer to the same inner object. That issue will be handled more directly in the later section on shallow and deep copying.

Equality and Lexicographic Comparison Rules

Equality checks whether two sequences have equal contents according to the equality rules of their components.

```
text1 = "spam"
text2 = "spam"
text3 = "eggs"

items1 = [10, 20, 42]
items2 = [10, 20, 42]
items3 = [10, 20, 99]

point1 = (10, 20, 42)
point2 = (10, 20, 42)
point3 = (10, 20, 99)

print(f"text1 == text2: {text1 == text2}")
print(f"text1 == text3: {text1 == text3}")

print()
print(f"items1 == items2: {items1 == items2}")
print(f"items1 == items3: {items1 == items3}")

print()
```

```
print(f"point1 == point2: {point1 == point2}")
print(f"point1 == point3: {point1 == point3}")
```

Possible output:

```
text1 == text2:    True
text1 == text3:    False

items1 == items2: True
items1 == items3: False

point1 == point2: True
point1 == point3: False
```

For lists and tuples, equality is positional. The first component is compared with the first component, the second with the second, and so on.

```
a = ["Jerry", "Elaine"]
b = ["Jerry", "Elaine"]
c = ["Elaine", "Jerry"]

print(f"a == b: {a == b}")
print(f"a == c: {a == c}")
```

Possible output:

```
a == b: True
a == c: False
```

The same elements in a different order do not make the same sequence.

A list and a tuple may contain equal components, but the outer sequence types are different.

```
items = [10, 20]
point = (10, 20)

print(f"items: {items!r}")
print(f"point: {point!r}")

print()
print(f"items == point: {items == point}")
print(f"type(items):    {type(items)}")
print(f"type(point):    {type(point)}")
```

Possible output:

```
items: [10, 20]
point: (10, 20)

items == point: False
type(items):    <class 'list'>
type(point):    <class 'tuple'>
```

Ordering comparisons are lexicographic. Python compares from left to right. The first unequal component decides the result.

```

a = [10, 20, 30]
b = [10, 20, 40]

print(f"a: {a}")
print(f"b: {b}")

print()
print(f"a < b: {a < b}")
print(f"a > b: {a > b}")

```

Possible output:

```

a: [10, 20, 30]
b: [10, 20, 40]

a < b: True
a > b: False

```

The first two components are equal: 10 equals 10, and 20 equals 20. The third comparison decides the result: 30 is smaller than 40.

If all compared components are equal, but one sequence ends earlier, the shorter sequence is smaller.

```

a = [10, 20]
b = [10, 20, 30]

print(f"a < b: {a < b}")
print(f"b < a: {b < a}")

```

Possible output:

```

a < b: True
b < a: False

```

This is similar to dictionary ordering of words, but it is not exactly human dictionary ordering. It is component-by-component comparison.

Strings are also compared lexicographically.

```

word1 = "spam"
word2 = "span"
word3 = "spammy"

print(f"word1 < word2: {word1 < word2}")
print(f"word1 < word3: {word1 < word3}")

```

Possible output:

```

word1 < word2: True
word1 < word3: True

```

In the first comparison, the decisive position is the fourth character:

```

spam
span
^

```

The character 'm' is smaller than 'n' in Python's text ordering. Case also matters.

```
print(f'{'A' < 'a' = }')
print(f'{'Z' < 'a' = }')
print(f'{'abc' < 'Abc' = }')
```

Possible output:

```
'A' < 'a' = True
'Z' < 'a' = True
'abc' < 'Abc' = False
```

This is not because uppercase letters are morally earlier or later. Python compares text according to character codepoint ordering. The details of character encoding and Unicode are handled in the next section.

Ordering comparison requires comparable components. Python will not silently invent an ordering between unrelated object types.

```
a = [10, "spam"]
b = [10, 42]

try:
    print(a < b)
except TypeError as err:
    print("comparison failed")
    print(f"message: {err}")
```

Possible output:

```
comparison failed
message: '<' not supported between instances of 'str' and 'int'
```

The first components are equal: 10 and 10. Then Python must compare "spam" and 42. That comparison is not defined, so the whole sequence comparison fails.

Immutability vs. Mutability as the Major Structural Split

The shared sequence model does not mean that all sequences can be changed in the same way.

The major structural split is:

- **str**: immutable;
- **tuple**: immutable;
- **list**: mutable.

Immutable means that the object cannot be changed in place after it is created. Mutable means that the object can be modified in place.

The following program tries index assignment on all three sequence types.

```
text = "D'oh!"
items = ["D", "", "o", "h", "!"]
point = (10, 20, 30)

print("before")
print(f"text: {text!r}")
print(f"items: {items!r}")
print(f"point: {point!r}")
```

```

print()
print("changing the list")
items[0] = "W"
print(f"items: {items!r}")

print()
print("trying to change the string")
try:
    text[0] = "W"
except TypeError as err:
    print("string change failed")
    print(f"message: {err}")

print()
print("trying to change the tuple")
try:
    point[0] = 99
except TypeError as err:
    print("tuple change failed")
    print(f"message: {err}")

```

Possible output:

```

before
text: "D'oh!"
items: ['D', "'", 'o', 'h', '!']
point: (10, 20, 30)

changing the list
items: ['W', "'", 'o', 'h', '!']

trying to change the string
string change failed
message: 'str' object does not support item assignment

trying to change the tuple
tuple change failed
message: 'tuple' object does not support item assignment

```

The list is changed in place. The string and tuple reject the assignment.

This difference can be seen with `id()`, which shows the identity of the object during the current program run.

```

text = "spam"
items = ["spam", "eggs"]

print("initial objects")
print(f"text: {text!r:<20} id={id(text)}")
print(f"items: {items!r:<20} id={id(items)}")

print()
text = text + " and eggs"
items.append(42)

print("after operations")

```

```
print(f"text: {text!r:<20} id={id(text)}")
print(f"items: {items!r:<20} id={id(items)}")
```

Possible output:

```
initial objects
text: 'spam'          id=139935850269424
items: ['spam', 'eggs'] id=139935850316032

after operations
text: 'spam and eggs' id=139935850318384
items: ['spam', 'eggs', 42] id=139935850316032
```

The exact `id()` numbers will differ from one run to another. The important observation is the pattern:

- the string operation creates a different string object;
- the list operation modifies the same list object.

Concatenation does not mutate the original sequence, even for lists.

```
a = ["Homer"]
b = ["Marge"]

old_id = id(a)
c = a + b

print(f"a:      {a!r}")
print(f"b:      {b!r}")
print(f"c:      {c!r}")

print()
print(f"id(a) before saved: {old_id}")
print(f"id(a) after:      {id(a)}")
print(f"id(c):            {id(c)}")
```

Possible output:

```
a:      ['Homer']
b:      ['Marge']
c:      ['Homer', 'Marge']

id(a) before saved: 139935850315776
id(a) after:      139935850315776
id(c):            139935850317184
```

The expression `a + b` creates a new list. It does not extend `a`. To mutate `a`, use a list mutation operation such as `append()` or `extend()`.

```
a = ["Homer"]
b = ["Marge"]

old_id = id(a)
a.extend(b)

print(f"a: {a!r}")
print(f"id(a) before saved: {old_id}")
print(f"id(a) after:      {id(a)}")
```

Possible output:

```
a: ['Homer', 'Marge']
id(a) before saved: 139935850315776
id(a) after:       139935850315776
```

The practical distinction is simple:

Shared sequence operations describe how objects can be read and combined. Mutability describes whether the existing object can be changed in place.

This split is the reason strings and tuples feel stable, while lists behave like editable containers.

5.2 Text Encoding Systems, Lexical Escape Maps, and Character Representation

A Python string looks simple when it is printed on the screen, but text is one of the places where several representation layers meet. A visible character is not the same thing as a numeric codepoint, and a numeric codepoint is not the same thing as the bytes written to a file or sent through a network connection.

This section separates those layers. We first look at ASCII, Unicode, and byte encodings such as UTF-8, UTF-16, and UTF-32. Then we examine Python's string-literal escape forms, where source-code spellings such as `\x41`, `\u03C0`, or `\N{...}` are converted by the lexer into actual string characters.

The practical goal is to prevent one common mistake: treating text, codepoints, escape sequences, and bytes as if they were the same object. They are related, but they live at different boundaries of the Python execution model.

5.2.1 Character Interoperability Standards

Text is not stored by magic. A visible character must be represented by some numeric convention before software can store it, transmit it, compare it, or write it into a file. Python hides much of this complexity during ordinary string use, but it does not remove the boundary. It separates *text* from *bytes*.

This section introduces the external standards that make text exchange possible. We are not yet studying the full CPython string memory layout. That comes later. Here the goal is simpler: understand the difference between characters, character numbers, and byte encodings.

```
text = "Ni!"
raw = b"Ni!"

print(f"text: {text!r}")
print(f"raw: {raw!r}")

print()
print(f"type(text): {type(text)}")
print(f"type(raw): {type(raw)}")
```

Possible output:

```
text: 'Ni!'
raw: b'Ni!'

type(text): <class 'str'>
type(raw):  <class 'bytes'>
```

A `str` object stores text. A `bytes` object stores integer byte values from 0 to 255. These two objects may look similar when they contain plain English letters, but they do not represent the same conceptual layer.

```
text = "Ni!"
raw = b"Ni!"

names = ["encode", "decode", "__len__", "__getitem__", "__contains__"]

print("selected attributes")
print()

for obj_name, obj in [("text", text), ("raw", raw)]:
    print(f"{obj_name}: {type(obj)}")
    for name in names:
        print(f"  {name:<12} {name in dir(obj)}")
    print()
```

Possible output:

```
selected attributes

text: <class 'str'>
  encode      True
  decode      False
  __len__     True
  __getitem__ True
  __contains__ True

raw: <class 'bytes'>
  encode      False
  decode      True
  __len__     True
  __getitem__ True
  __contains__ True
```

The direction is important:

- `str.encode()` converts text into bytes;
- `bytes.decode()` converts bytes back into text.

The Classical Domain: 7-Bit ASCII Codepoints (0-127) and the Later Confusion with 8-Bit Extended Encodings

ASCII is the old small map that assigns numbers to basic English letters, digits, punctuation marks, and control characters. Classical ASCII uses only seven bits, so it contains values from 0 to 127.

The function `ord()` returns the numeric codepoint of a one-character string. The function `chr()` performs the reverse operation.

```
chars = ["A", "B", "a", "0", "!", " "]

for ch in chars:
    nr = ord(ch)
    back = chr(nr)
    print(f"ch={ch!r:<4} ord={nr:<3} chr(ord)={back!r}")
```


Possible output:

```
ch='A'   ord=65   chr(ord)='A'
ch='B'   ord=66   chr(ord)='B'
ch='a'   ord=97   chr(ord)='a'
ch='0'   ord=48   chr(ord)='0'
ch='!'   ord=33   chr(ord)='!'
ch=' '   ord=32   chr(ord)=' '
```

These values are ASCII values. They are also Unicode codepoints, because Unicode deliberately keeps the first 128 codepoints compatible with ASCII.

```
text = "Chuck Norris counted to 42 twice."
```

```
for ch in text[:15]:
    print(f"{ch!r:<4} {ord(ch):>3}")
```

Possible output:

```
'C'   67
'h'  104
'u'  117
'c'   99
'k'  107
' '   32
'N'   78
'o'  111
'r'  114
'r'  114
'i'  105
's'  115
' '   32
'c'   99
'o'  111
```

ASCII is small. It cannot directly represent Romanian letters, Greek letters, mathematical symbols, Chinese characters, Arabic script, emoji, and many other writing systems.

```
chars = ["A", "z", "ă", "ș", "π"]
```

```
for ch in chars:
    nr = ord(ch)
    print(f"{ch!r:<4} codepoint={nr:<6} inside_ascii={nr <= 127}")
```

Possible output:

```
'A'   codepoint=65      inside_ascii=True
'z'   codepoint=122     inside_ascii=True
'ă'   codepoint=259     inside_ascii=False
'ș'   codepoint=537     inside_ascii=False
'π'   codepoint=960     inside_ascii=False
```

The later confusion came from the unused eighth bit. A byte has eight bits, so one byte can store values from 0 to 255. Classical ASCII used only 0 to 127. Many systems used the remaining range, 128 to 255, for different local extensions.

That produced many incompatible “extended ASCII” encodings. The phrase is dangerous because it sounds like one standard, but historically it often referred to different byte-to-character maps.

The same byte value can mean different characters in different 8-bit encodings.

```

raw = bytes([0xE9])

print(f"raw: {raw!r}")
print(f"raw[0]: {raw[0]}")

print()
print(f"latin-1: {raw.decode('latin-1')!r}")
print(f"cp1252: {raw.decode('cp1252')!r}")
print(f"cp437: {raw.decode('cp437')!r}")

print()
try:
    print(raw.decode("ascii"))
except UnicodeDecodeError as err:
    print("ascii decoding failed")
    print(f"message: {err}")

```

Possible output:

```

raw: b'\xe9'
raw[0]: 233

latin-1: 'é'
cp1252: 'é'
cp437: 'Θ'

ascii decoding failed
message: 'ascii' codec can't decode byte 0xe9 in position 0:
ordinal not in range(128)

```

The byte `0xE9` is not classical ASCII. Different encodings may interpret it differently. Therefore, a byte value alone is not enough to identify text once we leave the ASCII range.

The practical rule is:

ASCII is the stable 0-127 core. The range 128-255 is not one universal text system. It depends on the chosen encoding.

The Universal Map: Unicode Codepoints as Abstract Character Numbers, Separate from Byte Encodings

Unicode solves the local-map problem by defining a large universal character map. In that map, each character has an abstract number called a codepoint.

A Unicode codepoint is usually written in the form `U+XXXX`. For example:

- A has codepoint `U+0041`;
- ä has codepoint `U+0103`;
- π has codepoint `U+03C0`.

Python's `str` works at this text/codepoint level, not at the raw byte level.

```

chars = ["A", "ä", "ş", "π", ""]

for ch in chars:
    nr = ord(ch)
    print(f"char={ch!r:<4} decimal={nr:<6} unicode=U+{nr:04X}")

```

Possible output:

```
char='A'   decimal=65      unicode=U+0041
char='ă'   decimal=259     unicode=U+0103
char='ș'   decimal=537     unicode=U+0219
char='π'   decimal=960     unicode=U+03C0
char=''    decimal=28450   unicode=U+6F22
```

The important point is that these are character numbers, not byte sequences. A codepoint says *which character* is meant. It does not by itself say *which bytes* will be written to a file or sent over a network.

```
text = "π"

print(f"text:      {text!r}")
print(f"ord(text): {ord(text)}")
print(f"unicode:   U+{ord(text):04X}")

print()
print(f"utf-8 bytes: {text.encode('utf-8')}")
print(f"utf-16 bytes: {text.encode('utf-16')}")
print(f"utf-32 bytes: {text.encode('utf-32')}")
```

Possible output:

```
text:      'π'
ord(text): 960
unicode:   U+03C0

utf-8 bytes:  b'\xc3\xbc'
utf-16 bytes:  b'\xff\xfe\x00\x03'
utf-32 bytes:  b'\xff\xfe\x00\x00\x00\x03\x00\x00'
```

The character is the same: π . The Unicode codepoint is the same: U+03C0. The byte representation changes depending on the encoding.

This distinction is one of the most important text-processing boundaries in Python:

A Unicode codepoint is an abstract character number. An encoding is a rule for turning that character number into bytes.

The following program makes the boundary visible by storing several characters in one Python string, then printing their codepoints.

```
text = "Aăπ"

print(f"text: {text!r}")
print(f"len(text): {len(text)}")

print()
for ch in text:
    print(f"{ch!r:<4} U+{ord(ch):04X}")
```

Possible output:

```
text: 'Aăπ'
len(text): 3

'A'   U+0041
'ă'   U+0103
'π'   U+03C0
```

The string has three text components. It does not have three bytes in every encoding.

```
text = "Aăπ"

utf8 = text.encode("utf-8")
utf16 = text.encode("utf-16")
utf32 = text.encode("utf-32")

print(f"text:      {text!r}")
print(f"len(text):  {len(text)}")

print()
print(f"utf8:      {utf8!r}")
print(f"len(utf8):   {len(utf8)}")

print()
print(f"utf16:     {utf16!r}")
print(f"len(utf16):  {len(utf16)}")

print()
print(f"utf32:     {utf32!r}")
print(f"len(utf32):  {len(utf32)}")
```

Possible output:

```
text:      'Aăπ'
len(text): 3

utf8:      b'A\xc4\x83\xcf\x80'
len(utf8): 5

utf16:     b'\xff\xfeA\x00\x03\x01\xc0\x03'
len(utf16): 8

utf32:     b'\xff\xfe\x00\x00A\x00\x00\x00\x03\x01\x00\x00\xc0\x03\x00\x00'
len(utf32): 16
```

The same text gives different byte lengths. This is why text length and byte length must not be confused.

Encoding Boundaries: UTF-8, UTF-16, and UTF-32 as Byte Representations of Unicode Text

UTF-8, UTF-16, and UTF-32 are encodings. They are not three different character sets. They are three different ways of representing Unicode text as bytes.

Python exposes this boundary directly:

- `text.encode("utf-8")` turns a `str` into bytes;

- `raw.decode("utf-8")` turns `bytes` back into `str`.

```
text = "No soup for you!"

raw = text.encode("utf-8")
again = raw.decode("utf-8")

print(f"text: {text!r}")
print(f"raw: {raw!r}")
print(f"again: {again!r}")

print()
print(f"type(text): {type(text)}")
print(f"type(raw): {type(raw)}")
print(f"type(again): {type(again)}")
```

Possible output:

```
text: 'No soup for you!'
raw:  b'No soup for you!'
again: 'No soup for you!'

type(text): <class 'str'>
type(raw):  <class 'bytes'>
type(again): <class 'str'>
```

For pure ASCII-range text, UTF-8 uses the same byte values as ASCII.

```
text = "ABC"

ascii_raw = text.encode("ascii")
utf8_raw = text.encode("utf-8")

print(f"ascii_raw: {ascii_raw!r}")
print(f"utf8_raw: {utf8_raw!r}")
print(f"same: {ascii_raw == utf8_raw}")
```

Possible output:

```
ascii_raw: b'ABC'
utf8_raw:  b'ABC'
same:      True
```

For non-ASCII text, UTF-8 uses more than one byte for some characters.

```
text = "Aăπ"

raw = text.encode("utf-8")

print(f"text: {text!r}")
print(f"raw: {raw!r}")

print()
for byte in raw:
    print(f"byte decimal={byte:<3} hex=0x{byte:02X}")
```

Possible output:

```
text: 'Aăπ'
raw:  b'A\xc4\x83\xcf\x80'

byte decimal=65  hex=0x41
byte decimal=196 hex=0xC4
byte decimal=131 hex=0x83
byte decimal=207 hex=0xCF
byte decimal=128 hex=0x80
```

The string has three characters. The UTF-8 byte sequence has five bytes.

```
text = "Aăπ"
raw = text.encode("utf-8")

print(f"len(text): {len(text)}")
print(f"len(raw): {len(raw)}")
```

Possible output:

```
len(text): 3
len(raw): 5
```

UTF-16 and UTF-32 use different storage rules. Python can show their produced bytes directly.

```
text = "Aăπ"

for enc in ["utf-8", "utf-16", "utf-32"]:
    raw = text.encode(enc)
    print(f"{enc:<7} bytes={raw!r}")
    print(f"{'':<7} len={len(raw)}")
    print()
```

Possible output:

```
utf-8  bytes=b'A\xc4\x83\xcf\x80'
      len=5

utf-16 bytes=b'\xff\xfeA\x00\x03\x01\xc0\x03'
      len=8

utf-32 bytes=b'\xff\xfe\x00\x00A\x00\x00\x00\x03\x01\x00\x00\xc0\x03\x00\x00'
      len=16
```

The leading bytes in the UTF-16 and UTF-32 examples are a byte-order mark. They help identify byte order. To inspect the raw little-endian forms without that marker, Python also provides explicit encoding names.

```
text = "Aăπ"

for enc in ["utf-16-le", "utf-32-le"]:
    raw = text.encode(enc)
    print(f"{enc:<10} {raw!r}")
```

Possible output:

```
utf-16-le  b'A\x00\x03\x01\xc0\x03'
utf-32-le  b'A\x00\x00\x00\x03\x01\x00\x00\xc0\x03\x00\x00'
```

Decoding must use the correct encoding. If bytes were produced as UTF-8, they must be decoded as UTF-8.

```
text = "ă"
raw = text.encode("utf-8")

print(f"text: {text!r}")
print(f"raw: {raw!r}")

print()
print(f"correct decode: {raw.decode('utf-8')!r}")

try:
    wrong = raw.decode("ascii")
    print(wrong)
except UnicodeDecodeError as err:
    print("wrong decode failed")
    print(f"message: {err}")
```

Possible output:

```
text: 'ă'
raw:  b'\xc4\x83'

correct decode: 'ă'
wrong decode failed
message: 'ascii' codec can't decode byte 0xc4 in position 0:
ordinal not in range(128)
```

A different wrong decoding may not fail. It may produce incorrect text.

```
text = "ă"
raw = text.encode("utf-8")

print(f"raw: {raw!r}")
print(f"utf-8: {raw.decode('utf-8')!r}")
print(f"latin-1: {raw.decode('latin-1')!r}")
```

Possible output:

```
raw: b'\xc4\x83'
utf-8:  'ă'
latin-1: 'Ă\x83'
```

This is worse than an exception because the program continues with corrupted text. The bytes were valid according to Latin-1, but they were not meant to be read as Latin-1.

The practical boundary is therefore:

Inside Python, ordinary text is **str**. At file, network, and binary protocol boundaries, text must become **bytes**. Encoding moves from **str** to **bytes**; decoding moves from **bytes** back to **str**.

5.2.2 Native Conversion and Boundary Escape Sequences

Python source code is text. When the interpreter reads a string literal, some character sequences beginning with a backslash are treated specially. These are called escape sequences.

An escape sequence is not processed later by the string object. It is processed while the literal is being parsed.

```
text1 = "A"
text2 = "\101"
text3 = "\x41"

print(f"text1: {text1!r}")
print(f"text2: {text2!r}")
print(f"text3: {text3!r}")

print()
print(f"text1 == text2: {text1 == text2}")
print(f"text1 == text3: {text1 == text3}")
```

Possible output:

```
text1: 'A'
text2: 'A'
text3: 'A'

text1 == text2: True
text1 == text3: True
```

All three literals create the same final string object value: the character "A". The source spelling differs, but the parsed text component is the same.

The functions `ord()` and `chr()` make this boundary visible.

```
ch = "A"
nr = ord(ch)
back = chr(nr)

print(f"ch:    {ch!r}")
print(f"nr:    {nr}")
print(f"back: {back!r}")
```

Possible output:

```
ch:    'A'
nr:    65
back:  'A'
```

`ord()` moves from a one-character string to its integer codepoint. `chr()` moves from an integer codepoint back to a one-character string.

Octal Literal Parsing Boundaries (`\000` Values through Base-8 Conversions)

An octal escape uses base 8 digits. The allowed digits are:

0 1 2 3 4 5 6 7

In a Python string literal, an octal escape has this shape:


```
"\ooo"
```

where o means an octal digit.

For example, octal 101 is decimal 65, and Unicode codepoint 65 is "A".

```
octal_text = "101"

nr = int(octal_text, 8)
ch = chr(nr)

print(f"octal_text: {octal_text!r}")
print(f"decimal:    {nr}")
print(f"character:  {ch!r}")
```

Possible output:

```
octal_text: '101'
decimal:    65
character:  'A'
```

The literal escape `\101` performs this kind of conversion during literal parsing.

```
text = "\101"

print(f"text:      {text!r}")
print(f"ord(text): {ord(text)}")
```

Possible output:

```
text:      'A'
ord(text): 65
```

Several octal escapes can appear in the same string literal.

```
text = "\110\145\154\154\157"

print(f"text: {text!r}")
print(text)
```

Possible output:

```
text: 'Hello'
Hello
```

The octal values are:

```
\110 -> H
\145 -> e
\154 -> l
\154 -> l
\157 -> o
```

A null character can also be written with an octal escape.

```

text = "A\000B"

print(f"text:      {text!r}")
print(f"len(text): {len(text)}")

print()
for index, ch in enumerate(text):
    print(f"index={index} ch={ch!r:<4} ord={ord(ch)}")

```

Possible output:

```

text:      'A\x00B'
len(text): 3

index=0 ch='A'   ord=65
index=1 ch='\x00' ord=0
index=2 ch='B'   ord=66

```

The null character is inside the Python string. It does not terminate the string. This is different from the way C programmers often think about null-terminated character arrays.

Python consumes at most three octal digits for this escape form.

```

text1 = "\101"
text2 = "\1012"

print(f"text1: {text1!r}")
print(f"text2: {text2!r}")

print()
for index, ch in enumerate(text2):
    print(f"index={index} ch={ch!r} ord={ord(ch)}")

```

Possible output:

```

text1: 'A'
text2: 'A2'

index=0 ch='A'   ord=65
index=1 ch='2'   ord=50

```

The literal "\1012" is not read as one four-digit octal escape. Python reads \101, creates "A", and then leaves "2" as an ordinary character.

The practical rule is:

An octal escape is a source-code spelling that inserts a character by base-8 number. It is parsed before the final string object exists.

Hexadecimal Literal Parsing Boundaries (\xhh Values through Base-16 Conversions)

A hexadecimal escape uses base 16 digits. The allowed digits are:

```
0 1 2 3 4 5 6 7 8 9 A B C D E F
```

Lowercase letters are also accepted:

```
a b c d e f
```

In a Python string literal, the short hexadecimal escape has this shape:

```
"\xhh"
```

It requires exactly two hexadecimal digits after `\x`.

Hexadecimal 41 is decimal 65, and decimal 65 is "A".

```
hex_text = "41"

nr = int(hex_text, 16)
ch = chr(nr)

print(f"hex_text: {hex_text!r}")
print(f"decimal:  {nr}")
print(f"char:      {ch!r}")
```

Possible output:

```
hex_text: '41'
decimal:  65
char:     'A'
```

The escape `\x41` performs this conversion during string literal parsing.

```
text = "\x41"

print(f"text:      {text!r}")
print(f"ord(text): {ord(text)}")
```

Possible output:

```
text:      'A'
ord(text): 65
```

Several hexadecimal escapes can be placed next to each other.

```
text = "\x4e\x69\x21"

print(f"text: {text!r}")
print(text)
```

Possible output:

```
text: 'Ni!'
Ni!
```

The values are:

```
\x4e -> N
\x69 -> i
\x21 -> !
```

The `\xhh` form consumes exactly two hexadecimal digits.

```

text = "\x411"

print(f"text: {text!r}")

for index, ch in enumerate(text):
    print(f"index={index} ch={ch!r} ord={ord(ch)}")

```

Possible output:

```

text: 'A1'
index=0 ch='A' ord=65
index=1 ch='1' ord=49

```

Python reads `\x41` as "A", then reads the remaining "1" as an ordinary character.

A short hex escape is invalid if fewer than two hexadecimal digits follow `\x`. Because this is a source-code parsing error, it cannot be caught by putting the bad literal directly in normal code. The program would fail before it starts running. To demonstrate it safely, we can compile source text from another string.

```

source = 'bad = "\x4"'

try:
    compile(source, "<demo>", "exec")
    print("compiled")
except SyntaxError as err:
    print("syntax error")
    print(f"message: {err.msg}")

```

Possible output:

```

syntax error
message: (unicode error) 'unicodeescape' codec can't decode bytes
in position 0-2: truncated \xXX escape

```

The exact wording may vary slightly between Python versions, but the cause is the same: `\x` needs exactly two hexadecimal digits.

A common confusion is to treat `\xhh` as if it were the same thing as UTF-8 decoding. It is not.

```

wrong_text = "\xC4\x83"
right_text = b"\xC4\x83".decode("utf-8")

print(f"wrong_text: {wrong_text!r}")
print(f"right_text: {right_text!r}")

print()
print(f"len(wrong_text): {len(wrong_text)}")
print(f"len(right_text): {len(right_text)}")

print()
for ch in wrong_text:
    print(f"wrong component: {ch!r:<4} U+{ord(ch):04X}")

print()
for ch in right_text:
    print(f"right component: {ch!r:<4} U+{ord(ch):04X}")

```

Possible output:

```
wrong_text: 'Ä\x83'
right_text: 'ä'

len(wrong_text): 2
len(right_text): 1

wrong component: 'Ä' U+00C4
wrong component: '\x83' U+0083

right component: 'ä' U+0103
```

The source literal `"\xC4\x83"` creates two text characters. The bytes literal `b"\xC4\x83"` creates two raw bytes, and UTF-8 decoding interprets those bytes as one text character.

So the layer matters:

- `"\xC4\x83"` is a `str` literal with two character escapes;
- `b"\xC4\x83"` is a `bytes` literal with two byte escapes;
- `decode("utf-8")` interprets the bytes according to UTF-8.

The practical rule is:

The escape `\xhh` inserts a character with a two-digit hexadecimal value in a string literal. It does not automatically mean “UTF-8 text”.

Raw String Literals: Suppressing Most Escape Processing with the `r"..."` Prefix

A raw string literal is written with an `r` or `R` prefix before the quote.

```
normal = "C:\new\test"
raw = r"C:\new\test"

print(f"normal: {normal!r}")
print(f"raw: {raw!r}")
```

Possible output:

```
normal: 'C:\new\test'
raw: 'C:\\new\\test'
```

The displayed `repr()` result shows what happened. In the normal string, `\n` became a newline and `\t` became a tab. In the raw string, the backslashes were kept as ordinary backslash characters.

Printing the values makes the difference more visible.

```
normal = "C:\new\test"
raw = r"C:\new\test"

print("normal printed:")
print(normal)

print()
print("raw printed:")
print(raw)
```

Possible output:

```
normal printed:
C:
ew est
```

```
raw printed:
C:\new\test
```

Raw string literals are useful when the text itself contains many backslashes, especially file paths and regular expression patterns.

The raw prefix affects literal parsing. It does not create a special runtime string type.

```
normal = "abc"
raw = r"abc"

print(f"normal: {normal!r}")
print(f"raw:     {raw!r}")

print()
print(f"type(normal): {type(normal)}")
print(f"type(raw):    {type(raw)}")
print(f"same value:   {normal == raw}")
```

Possible output:

```
normal: 'abc'
raw:    'abc'

type(normal): <class 'str'>
type(raw):    <class 'str'>
same value:   True
```

There is no `raw string object` type. There is only a `str` object created from a raw string literal spelling.

Raw strings suppress most escape processing, including octal, hex, and Unicode-looking escapes.

```
normal1 = "\101"
raw1 = r"\101"

normal2 = "\x41"
raw2 = r"\x41"

print(f"normal1: {normal1!r}")
print(f"raw1:     {raw1!r}")

print()
print(f"normal2: {normal2!r}")
print(f"raw2:     {raw2!r}")
```

Possible output:

```
normal1: 'A'
raw1:    '\\101'

normal2: 'A'
raw2:    '\\x41'
```

The raw literal keeps the backslash and the following characters as part of the resulting string. The length confirms the difference.

```
normal = "\x41"
raw = r"\x41"

print(f"normal:      {normal!r}")
print(f"len(normal): {len(normal)}")

print()
print(f"raw:         {raw!r}")
print(f"len(raw):     {len(raw)}")

print()
for index, ch in enumerate(raw):
    print(f"index={index} ch={ch!r} ord={ord(ch)}")
```

Possible output:

```
normal:      'A'
len(normal): 1

raw:         '\\x41'
len(raw):    4

index=0 ch='\\' ord=92
index=1 ch='x' ord=120
index=2 ch='4' ord=52
index=3 ch='1' ord=49
```

Raw strings have one important syntactic boundary: they cannot end with a single unpaired backslash, because that backslash would escape the closing quote in the source code.

This is not valid Python source:

```
path = r"C:\temp\"
```

A simple workaround is to concatenate a final backslash.

```
path = r"C:\temp" + "\\"

print(f"path: {path!r}")
print(path)
```

Possible output:

```
path: 'C:\\temp\\'
C:\temp\
```

The practical rule is:

A raw string literal changes how Python reads backslashes in the source code. It does not change the runtime type. The result is still an ordinary `str`.

5.2.3 Extended Universal Codepoint Escape Access

The earlier escape forms, such as octal escapes and `\xhh`, are useful for small numeric values. Unicode needs a larger address space. Python therefore supports additional string-literal escape forms for inserting Unicode characters by name or by larger hexadecimal codepoint values.

These are still lexical forms. They are processed while Python reads the source code literal.

```
text1 = "π"
text2 = "\N{GREEK SMALL LETTER PI}"
text3 = "\u03C0"
text4 = "\U000003C0"

print(f"text1: {text1!r}")
print(f"text2: {text2!r}")
print(f"text3: {text3!r}")
print(f"text4: {text4!r}")

print()
print(f"text1 == text2: {text1 == text2}")
print(f"text1 == text3: {text1 == text3}")
print(f"text1 == text4: {text1 == text4}")
```

Possible output:

```
text1: 'π'
text2: 'π'
text3: 'π'
text4: 'π'

text1 == text2: True
text1 == text3: True
text1 == text4: True
```

All four spellings create the same final one-character string.

Formal Lexical Extraction: Querying Literals via System Naming Maps (`\N{...}`)

The escape `\N{...}` inserts a Unicode character by its official Unicode name.

```
pi = "\N{GREEK SMALL LETTER PI}"
snowman = "\N{SNOWMAN}"
face = "\N{GRINNING FACE}"

print(f"pi:      {pi!r}      U+{ord(pi):04X}")
print(f"snowman: {snowman!r}  U+{ord(snowman):04X}")
print(f"face:     {face!r}     U+{ord(face):04X}")
```

Possible output:

```
pi:      'π'      U+03C0
snowman: '⛄'      U+2603
face:    '😄'      U+1F600
```

The name must be known by Python's Unicode database. The standard library module `unicodedata` lets us query the same naming system at runtime.


```
import unicodedata

chars = ["A", "π", "⛄", "😄"]

for ch in chars:
    name = unicodedata.name(ch)
    print(f"{ch!r:<4} U+{ord(ch):06X} {name}")
```

Possible output:

```
'A' U+000041 LATIN CAPITAL LETTER A
'π' U+0003C0 GREEK SMALL LETTER PI
'⛄' U+002603 SNOWMAN
'😄' U+01F600 GRINNING FACE
```

The lookup can also go in the opposite direction.

```
import unicodedata

names = [
    "LATIN CAPITAL LETTER A",
    "GREEK SMALL LETTER PI",
    "SNOWMAN",
    "GRINNING FACE",
]

for name in names:
    ch = unicodedata.lookup(name)
    print(f"{name:<30} -> {ch!r} U+{ord(ch):06X}")
```

Possible output:

```
LATIN CAPITAL LETTER A      -> 'A' U+000041
GREEK SMALL LETTER PI      -> 'π' U+0003C0
SNOWMAN                     -> '⛄' U+002603
GRINNING FACE               -> '😄' U+01F600
```

A bad Unicode name is not a runtime string error. It is a source parsing error. To demonstrate it safely, we compile source text from another string.

```
source = 'bad = "\\N{THIS NAME DOES NOT EXIST}"'

try:
    compile(source, "<demo>", "exec")
    print("compiled")
except SyntaxError as err:
    print("syntax error")
    print(f"message: {err.msg}")
```

Possible output:

```
syntax error
message: (unicode error) 'unicodeescape' codec can't decode bytes
in position 0-29: unknown Unicode character name
```

The exact position numbers may differ if the source string changes. The important rule remains the same: `\N{...}` requires a valid Unicode character name.

Planar Transformations: 16-Bit Base-16 Codepoint Escapes via \u

The escape \u inserts one Unicode character by a four-digit hexadecimal codepoint.

The shape is:

```
"\uhhhh"
```

where each `h` is a hexadecimal digit.

```
text = "\u0041\u03C0\u2603"

print(f"text: {text!r}")
print(text)

print()
for ch in text:
    print(f"{ch!r:<4} U+{ord(ch):04X}")
```

Possible output:

```
text: 'Aπ³'
Aπ³

'A'  U+0041
'π'  U+03C0
'³'  U+2603
```

The escape \u0041 creates the same character as ordinary "A".

```
normal = "A"
escaped = "\u0041"

print(f"normal: {normal!r}")
print(f"escaped: {escaped!r}")
print(f"same: {normal == escaped}")
```

Possible output:

```
normal: 'A'
escaped: 'A'
same: True
```

The four hex digits are required. Fewer digits produce a syntax error while the source code is parsed.

```
source = 'bad = "\\u03C0"'

try:
    compile(source, "<demo>", "exec")
    print("compiled")
except SyntaxError as err:
    print("syntax error")
    print(f"message: {err.msg}")
```

Possible output:

```
syntax error
message: (unicode error) 'unicodeescape' codec can't decode bytes
in position 0-4: truncated \uXXXX escape
```

The `\u` form reaches codepoints from U+0000 through U+FFFF. For characters above that range, use the eight-digit `\U` form.

```
pi = "\u03C0"
snowman = "\u2603"

print(f"pi:      {pi!r} U+{ord(pi):04X}")
print(f"snowman: {snowman!r} U+{ord(snowman):04X}")
```

Possible output:

```
pi:      'π' U+03C0
snowman: '☺' U+2603
```

The practical rule is:

`\uhhhh` inserts one Unicode character using exactly four hexadecimal digits.

Absolute Space Mapping: 32-Bit Base-16 Extended Codepoint Escapes via `\U`

The escape `\U` inserts one Unicode character by an eight-digit hexadecimal codepoint.

The shape is:

```
"\Uhhhhhhhhh"
```

where each `h` is a hexadecimal digit.

```
face = "\U0001F600"
g_clef = "\U0001D11E"
pi = "\U000003C0"

print(f"face:   {face!r}   U+{ord(face):06X}")
print(f"g_clef: {g_clef!r}   U+{ord(g_clef):06X}")
print(f"pi:      {pi!r}     U+{ord(pi):06X}")
```

Possible output:

```
face:   '😄'   U+01F600
g_clef: '🎸'   U+01D11E
pi:      'π'   U+0003C0
```

The eight-digit form can represent codepoints that the four-digit `\u` form cannot write directly.

The number of characters is still counted at the text level.

```
text = "\U0001F600\U0001D11E\U000003C0"

print(f"text: {text!r}")
print(f"len(text): {len(text)}")

print()
for ch in text:
    print(f"{ch!r:<4} U+{ord(ch):06X}")
```

Possible output:

```
text: '😊 π'
len(text): 3

'😊' U+01F600
'π' U+01D11E
'π' U+0003C0
```

The `\U` escape also requires exactly eight hexadecimal digits.

```
source = 'bad = "\\U0001F60"'

try:
    compile(source, "<demo>", "exec")
    print("compiled")
except SyntaxError as err:
    print("syntax error")
    print(f"message: {err.msg}")
```

Possible output:

```
syntax error
message: (unicode error) 'unicodeescape' codec can't decode bytes
in position 0-8: truncated \UXXXXXXX escape
```

Unicode also has a maximum valid codepoint: `U+10FFFF`. A larger value is not accepted.

```
source = 'bad = "\\U00110000"'

try:
    compile(source, "<demo>", "exec")
    print("compiled")
except SyntaxError as err:
    print("syntax error")
    print(f"message: {err.msg}")
```

Possible output:

```
syntax error
message: (unicode error) 'unicodeescape' codec can't decode bytes
in position 0-9: illegal Unicode character
```

The three universal escape forms can be compared directly.

```
name_escape = "\N{GREEK SMALL LETTER PI}"
u_escape = "\u03C0"
U_escape = "\U000003C0"

print(f"name_escape: {name_escape!r} U+{ord(name_escape):04X}")
print(f"u_escape:      {u_escape!r} U+{ord(u_escape):04X}")
print(f"U_escape:      {U_escape!r} U+{ord(U_escape):04X}")

print()
print(f"all same: {name_escape == u_escape == U_escape}")
```

Possible output:

```
name_escape: 'π' U+03C0
u_escape:    'π' U+03C0
U_escape:    'π' U+03C0
```

```
all same: True
```

The source spelling differs. The resulting string component is the same.

`\N{...}` uses a Unicode character name. `\uhhhh` uses exactly four hexadecimal digits. `\Uhhhhhhhh` uses exactly eight hexadecimal digits and reaches the full valid Unicode code-point range.

5.3 CPython String Memory Realities (`str`)

A Python string is used as a text sequence, but CPython must still store that sequence inside real memory. The language view is simple: a `str` contains ordered text components and cannot be changed in place. The implementation view is more concrete: CPython must choose a memory layout, preserve Unicode meaning, support fast indexing and hashing, and avoid wasting space for common ASCII text.

This section connects those two views. We will keep the programmer-facing rule clear: `str` stores text, not raw bytes. Then we will look under that rule and see how CPython represents Unicode strings efficiently, why immutability matters, why repeated concatenation can be expensive, and why some string objects may be reused through interning.

The goal is not to replace ordinary string programming with implementation trivia. The goal is to make string behavior unsurprising: indexing gives one-character strings, slicing creates new strings, formatting builds new strings, and every apparent modification produces another object rather than editing the old one.

5.3.1 Structural Abstraction of the Unicode Standard

Python’s `str` object represents text. More precisely, it represents a sequence of Unicode codepoints. This is already a higher-level structure than a C-style byte array.

A Python string does not expose “the bytes of the text” during normal string operations. It exposes text components. When bytes are needed, the string must be encoded.

```
text = "D'oh!"
raw = text.encode("utf-8")

print(f"text: {text!r}")
print(f"raw:  {raw!r}")

print()
print(f"type(text): {type(text)}")
print(f"type(raw):  {type(raw)}")
```

Possible output:

```
text: "D'oh!"
raw:  b"D'oh!"

type(text): <class 'str'>
type(raw):  <class 'bytes'>
```

The visible result is simple, but the layers are different:

- **str**: text-level object;
- Unicode codepoint: abstract character number;
- glyph: visible drawing on the screen;
- **bytes**: transmitted or stored byte values.

The following inspection shows that **str** and **bytes** are different object types with different available operations.

```
text = "Ni!"
raw = b"Ni!"

selected_names = [
    "encode",
    "decode",
    "__len__",
    "__getitem__",
    "split",
    "join",
    "lower",
    "hex",
]

for obj_name, obj in [("text", text), ("raw", raw)]:
    print(f"{obj_name}: {type(obj)}")
    for name in selected_names:
        print(f"  {name:<12} {name in dir(obj)}")
    print()
```

Possible output:

```
text: <class 'str'>
  encode      True
  decode      False
  __len__     True
  __getitem__ True
  split       True
  join        True
  lower       True
  hex         False

raw: <class 'bytes'>
  encode      False
  decode      True
  __len__     True
  __getitem__ True
  split       True
  join        True
  lower       True
  hex         True
```

The important boundary is directional:

- `str.encode(...)` produces **bytes**;
- `bytes.decode(...)` produces **str**.

Distinction Between Abstract Codepoints, Visible Glyphs, and Transmitted Byte Serializations

A Unicode codepoint is an abstract number assigned to a text character. Python exposes that number with `ord()`.

```
chars = ["A", "ă", "π", "⌘", "😄 "]

for ch in chars:
    print(f"ch={ch!r:<4} ord={ord(ch):<7} unicode=U+{ord(ch):06X}")
```

Possible output:

```
ch='A'   ord=65      unicode=U+000041
ch='ă'   ord=259     unicode=U+000103
ch='π'   ord=960     unicode=U+0003C0
ch='⌘'   ord=9731    unicode=U+002603
ch='😄'   ord=128512  unicode=U+01F600
```

This is not byte inspection. It is text inspection. The codepoint tells us which abstract Unicode character is present.

The byte serialization is obtained only after encoding.

```
chars = ["A", "ă", "π", "⌘", "😄 "]

for ch in chars:
    raw = ch.encode("utf-8")
    print(f"ch={ch!r:<4} codepoint=U+{ord(ch):06X} utf8={raw!r}")
```

Possible output:

```
ch='A'   codepoint=U+000041 utf8=b'A'
ch='ă'   codepoint=U+000103 utf8=b'\xc4\x83'
ch='π'   codepoint=U+0003C0 utf8=b'\xcf\x80'
ch='⌘'   codepoint=U+002603 utf8=b'\xe2\x98\x83'
ch='😄'   codepoint=U+01F600 utf8=b'\xf0\x9f\x98\x80'
```

The same one-character string can need one, two, three, or four bytes in UTF-8.

```
chars = ["A", "ă", "π", "⌘", "😄 "]

print(f"{'char':<6} {'len(str)':<10} {'len(utf8)':<10} {'utf8 bytes'}")

for ch in chars:
    raw = ch.encode("utf-8")
    print(f"{'ch!r:<6} {'len(ch):<10} {'len(raw):<10} {'raw!r}")
```

Possible output:

char	len(str)	len(utf8)	utf8 bytes
'A'	1	1	b'A'
'ă'	1	2	b'\xc4\x83'
'π'	1	2	b'\xcf\x80'
'⌘'	1	3	b'\xe2\x98\x83'
'😄'	1	4	b'\xf0\x9f\x98\x80'

So `len(ch)` counts Python string components, not bytes.

There is another layer: the visible glyph. A glyph is what appears on the screen. A visible glyph can sometimes be built from more than one Unicode codepoint.

```
text1 = "é"
text2 = "e\u0301"

print(f"text1: {text1!r}")
print(f"text2: {text2!r}")

print()
print(f"printed text1: {text1}")
print(f"printed text2: {text2}")

print()
print(f"len(text1): {len(text1)}")
print(f"len(text2): {len(text2)}")
```

Possible output:

```
text1: 'é'
text2: 'e´'

printed text1: é
printed text2: e´

len(text1): 1
len(text2): 2
```

The two strings may look nearly identical when printed. Internally, they are not the same sequence of codepoints.

```
text1 = "é"
text2 = "e\u0301"

print("text1 components")
for ch in text1:
    print(f"{ch!r:<4} U+{ord(ch):04X}")

print()
print("text2 components")
for ch in text2:
    print(f"{ch!r:<4} U+{ord(ch):04X}")

print()
print(f"text1 == text2: {text1 == text2}")
```

Possible output:

```
text1 components
'é'  U+00E9

text2 components
'e'  U+0065
'´'  U+0301

text1 == text2: False
```


The second string contains:

- U+0065: the letter e;
- U+0301: a combining acute accent.

A combining mark modifies the visual rendering of the preceding character, but it is still a separate codepoint inside the Python string.

The standard library module `unicodedata` can show the names of those components.

```
import unicodedata

text = "e\u0301"

for ch in text:
    name = unicodedata.name(ch)
    print(f"{ch!r:<4} U+{ord(ch):04X} {name}")
```

Possible output:

```
'e'  U+0065 LATIN SMALL LETTER E
'´'  U+0301 COMBINING ACUTE ACCENT
```

Normalization can convert between some equivalent Unicode spellings.

```
import unicodedata

text1 = "é"
text2 = "e\u0301"

n1 = unicodedata.normalize("NFC", text1)
n2 = unicodedata.normalize("NFC", text2)

print(f"text1 == text2: {text1 == text2}")
print(f"n1 == n2:      {n1 == n2}")

print()
print(f"n1: {n1!r} len={len(n1)}")
print(f"n2: {n2!r} len={len(n2)}")
```

Possible output:

```
text1 == text2: False
n1 == n2:      True

n1: 'é' len=1
n2: 'é' len=1
```

This example separates three notions:

- codepoint sequence: what Python's `str` stores and indexes;
- glyph rendering: what the terminal, editor, browser, or font draws;
- byte serialization: what encoding produces for storage or transmission.

A Python string is therefore not a drawing and not a byte buffer. It is a text sequence.

Text vs. Bytes: Why str Stores Text and bytes Stores Raw Byte Values

The `str` type stores text-level components. The `bytes` type stores raw byte values. A byte value is an integer from 0 to 255.

The difference appears immediately during indexing.

```
text = "Aă"
raw = text.encode("utf-8")

print(f"text: {text!r}")
print(f"raw: {raw!r}")

print()
print(f"text[0]: {text[0]!r} type={type(text[0])}")
print(f"raw[0]: {raw[0]!r} type={type(raw[0])}")
```

Possible output:

```
text: 'Aă'
raw: b'A\xc4\x83'

text[0]: 'A' type=<class 'str'>
raw[0]: 65 type=<class 'int'>
```

Indexing a string returns a one-character string. Indexing a bytes object returns an integer byte value.

Iteration shows the same split.

```
text = "Aă"
raw = text.encode("utf-8")

print("string iteration")
for ch in text:
    print(f"{ch!r:<4} U+{ord(ch):04X}")

print()
print("bytes iteration")
for byte in raw:
    print(f"{byte:<3} 0x{byte:02X}")
```

Possible output:

```
string iteration
'A'  U+0041
'ă'  U+0103

bytes iteration
65  0x41
196 0xC4
131 0x83
```

The string has two text components. The UTF-8 byte serialization has three byte values.

```
text = "Aă"
raw = text.encode("utf-8")

print(f"len(text): {len(text)}")
print(f"len(raw): {len(raw)}")
```

Possible output:

```
len(text): 2
len(raw): 3
```

This difference explains why Python does not automatically mix `str` and `bytes` in ordinary operations.

```
text = "Ni!"
raw = b"Ni!"

print(f"text == raw: {text == raw}")

try:
    result = text + raw
    print(result)
except TypeError as err:
    print("concatenation failed")
    print(f"message: {err}")
```

Possible output:

```
text == raw: False
concatenation failed
message: can only concatenate str (not "bytes") to str
```

The values look similar to a human reader, but they are not the same layer. Python refuses to guess which encoding should be used.

The correct conversion must be explicit.

```
text = "No soup for you!"

raw = text.encode("utf-8")
again = raw.decode("utf-8")

print(f"text: {text!r}")
print(f"raw: {raw!r}")
print(f"again: {again!r}")

print()
print(f"text == again: {text == again}")
```

Possible output:

```
text: 'No soup for you!'
raw:  b'No soup for you!'
again: 'No soup for you!'

text == again: True
```

When data crosses a file, socket, database driver, HTTP body, or binary protocol boundary, bytes usually appear. When the program wants to reason about human text, it should decode those bytes into `str`.

```
incoming = b"Chuck Norris counted to 42."

text = incoming.decode("utf-8")

print(f"incoming: {incoming!r}")
print(f"text:      {text!r}")

print()
print(f"type(incoming): {type(incoming)}")
print(f"type(text):    {type(text)}")
```

Possible output:

```
incoming: b'Chuck Norris counted to 42.'
text:      'Chuck Norris counted to 42.'

type(incoming): <class 'bytes'>
type(text):     <class 'str'>
```

The practical rule is:

Use `str` for text meaning. Use `bytes` for raw byte values. Move between them only with an explicit encoding or decoding operation.

5.3.2 CPython Memory Optimization Architecture (PEP 393)

Python's `str` object is a language-level text sequence. CPython, the standard Python implementation, must still store that text in real memory. Since Python strings can contain anything from plain ASCII to emoji, using the same fixed-width storage for every string would waste memory in common cases.

PEP 393 introduced CPython's flexible string representation. The basic idea is simple: CPython chooses an internal character storage width according to the largest codepoint contained in the string.

```
texts = [
    "ABC",
    "ééé",
    "ăπʘ ",
    "😄 😊 😊 ",
]

for text in texts:
    max_codepoint = max(ord(ch) for ch in text)
    print(f"text={text!r:<10} max=U+{max_codepoint:06X}")
```

Possible output:

```
text='ABC'          max=U+000043
text='ééé'          max=U+0000E9
text='ăπʘ '         max=U+002603
text='😄 😊 😊 '      max=U+01F600
```

The important value is not the first character, nor the average character. The important value is the largest codepoint. That value determines the smallest internal storage width that can hold every component of the string.

The Flexible String Representation Framework: Dynamic Character Data Width Selection Based on Maximum Codepoint Value

CPython's flexible representation can be understood with three ordinary ranges:

- codepoints up to U+00FF can fit in one byte per codepoint;
- codepoints up to U+FFFF can fit in two bytes per codepoint;
- larger valid Unicode codepoints need four bytes per codepoint.

This is not UTF-8. It is an internal CPython storage strategy for the canonical string data.

```
texts = [
    "ABC",
    "ééé",
    "ăπ𐀀 ",
    "😄 😊 😊 ",
]

for text in texts:
    max_nr = max(ord(ch) for ch in text)

    if max_nr <= 0xFF:
        width = 1
    elif max_nr <= 0xFFFF:
        width = 2
    else:
        width = 4

    print(f"text={text!r:<10} max=U+{max_nr:06X} likely_width={width}")
```

Possible output:

```
text='ABC'          max=U+000043 likely_width=1
text='ééé'          max=U+0000E9 likely_width=1
text='ăπ𐀀 '         max=U+002603 likely_width=2
text='😄 😊 😊 '      max=U+01F600 likely_width=4
```

This program does not directly read CPython's private C fields. Normal Python code does not expose the internal string kind as a public attribute. The program only computes the expected storage category from the visible Unicode codepoints.

The memory effect can be observed indirectly with `sys.getsizeof()`.

```
import sys
import platform

texts = [
    ("ascii", "ABC"),
    ("latin1", "ééé"),
    ("bmp", "ăπ𐀀 "),
    ("non_bmp", "😄 😊 😊 "),
]

print(f"implementation: {platform.python_implementation()}")
print()
```

```

for label, text in texts:
    max_nr = max(ord(ch) for ch in text)
    size = sys.getsizeof(text)

    print(f"{label:<8} text={text!r:<10} len={len(text):<2} "
          f"max=U+{max_nr:06X} size={size}")

```

Possible output on one 64-bit CPython build:

implementation: CPython

```

ascii    text='ABC'          len=3  max=U+000043 size=44
latin1   text='ééé'          len=3  max=U+0000E9 size=60
bmp      text='ăπ'           len=3  max=U+002603 size=64
non_bmp  text='😊😊😊'         len=3  max=U+01F600 size=72

```

The exact numbers are version-dependent and platform-dependent. The important pattern is that the internal representation is not simply “one Python character equals one byte”.

Also, `sys.getsizeof()` reports the memory directly attributed to the object. It is useful for demonstrations, not a complete model of all memory activity in a running process.

The flexible representation also keeps `len()` simple. Length is still the number of Python string components, not the number of UTF-8 bytes.

```

texts = [
    "ABC",
    "ééé",
    "ăπ",
    "😊😊😊",
]

for text in texts:
    raw = text.encode("utf-8")
    print(f"text={text!r:<10} len(str)={len(text):<2} len(utf8)={len(raw):<2}")

```

Possible output:

```

text='ABC'          len(str)=3  len(utf8)=3
text='ééé'          len(str)=3  len(utf8)=6
text='ăπ'           len(str)=3  len(utf8)=7
text='😊😊😊'         len(str)=3  len(utf8)=12

```

The runtime string and its external UTF-8 byte serialization are different objects with different storage rules.

Core Immutability: Fixed Heap Allocations and Read-Only Character Storage

A CPython string object is immutable. Once the object exists, its character sequence cannot be edited in place from Python code.

```

text = "D'oh!"

print(f"text before: {text!r}")
print(f"id before:   {id(text)}")

```

```

try:
    text[0] = "W"
except TypeError as err:
    print("mutation failed")
    print(f"message: {err}")

print()
print(f"text after:  {text!r}")
print(f"id after:    {id(text)}")

```

Possible output:

```

text before: "D'oh!"
id before:   140225874655600
mutation failed
message: 'str' object does not support item assignment

text after:  "D'oh!"
id after:    140225874655600

```

The exact `id()` value is not important. The useful fact is that the existing string object was not edited.

String operations that appear to change text actually create another string object.

```

text = "spam"

old_id = id(text)
text = text.upper()

print(f"text:      {text!r}")
print(f"old id:     {old_id}")
print(f"new id:      {id(text)}")
print(f"same obj:    {old_id == id(text)}")

```

Possible output:

```

text:      'SPAM'
old id:     140225874655920
new id:     140225874656176
same obj:   False

```

The name `text` now refers to a different string object. The old object was not modified. The same rule applies to concatenation.

```

text = "spam"

old_id = id(text)
text = text + " and eggs"

print(f"text:      {text!r}")
print(f"old id:     {old_id}")
print(f"new id:      {id(text)}")
print(f"same obj:    {old_id == id(text)}")

```

Possible output:

```

text:      'spam and eggs'
old id:    140225874655920
new id:    140225874644720
same obj: False

```

This connects directly to the earlier distinction between rebinding and mutation. The assignment sign does not edit the old string. It binds the name to the result object.

Because strings are immutable, CPython can safely reuse, cache, hash, and intern some string objects. If a string could be changed in place, those optimizations would be dangerous.

Computational and Allocation Overhead of Iterative String Concatenation

Since strings are immutable, repeated concatenation can create many temporary string objects. Each new result must contain the previous text plus the newly added text.

The following program shows the identity changes during repeated concatenation.

```

text = ""

for part in ["Nobody", " expects", " the Spanish", " Inquisition"]:
    old_id = id(text)
    text = text + part

    print(f"added={part!r:<15} len={len(text):<3} "
          f"old_id={old_id} new_id={id(text)}")

print()
print(f"final text: {text!r}")

```

Possible output:

```

added='Nobody'      len=6   old_id=140225874320944 new_id=140225874656112
added=' expects'    len=14   old_id=140225874656112 new_id=140225874644720
added=' the Spanish' len=26   old_id=140225874644720 new_id=140225874646320
added=' Inquisition' len=38   old_id=140225874646320 new_id=140225874646512

```

```
final text: 'Nobody expects the Spanish Inquisition'
```

The repeated + pattern is easy to read for a small number of pieces. It is not the right construction pattern for many pieces.

The preferred pattern is:

1. collect the pieces in a list;
2. join them once at the end.

```

parts = []

parts.append("Nobody")
parts.append(" expects")
parts.append(" the Spanish")
parts.append(" Inquisition")

text = "".join(parts)

print(f"parts: {parts!r}")
print(f"text: {text!r}")

```


Possible output:

```
parts: ['Nobody', ' expects', ' the Spanish', ' Inquisition']
text: 'Nobody expects the Spanish Inquisition'
```

The list is mutable, so `append()` changes the list structure in place. The final `join()` operation computes the required final string once.

A larger demonstration can compare the two styles.

```
import time

count = 50000
piece = "ha"

start = time.perf_counter()

text = ""
for _ in range(count):
    text = text + piece

end = time.perf_counter()
concat_time = end - start

start = time.perf_counter()

parts = []
for _ in range(count):
    parts.append(piece)

text2 = "".join(parts)

end = time.perf_counter()
join_time = end - start

print(f"same result: {text == text2}")
print(f"len result:  {len(text)}")
print(f"+ loop:      {concat_time:.6f} seconds")
print(f"join style:  {join_time:.6f} seconds")
```

Possible output:

```
same result: True
len result:  100000
+ loop:      0.090000 seconds
join style:  0.003000 seconds
```

The exact timings depend on the machine, Python version, and current system load. The structural point is more important than the exact numbers: repeated immutable concatenation can allocate and copy repeatedly, while list accumulation followed by `join()` is the standard construction pattern.

For short direct expressions, `+` is fine.

```
name = "Kramer"
line = "Hello, " + name + "!"

print(line)
```

Possible output:

```
Hello, Kramer!
```

The warning is not “never use + with strings”. The warning is: do not build long strings by repeated concatenation inside large loops when a list plus `join()` expresses the construction more directly.

The Interning Subsystem: Immutable String Optimization and Singly Allocated Literals inside CPython

Interning is an optimization where CPython keeps one shared string object for selected equal string values. If two names refer to the same interned string object, identity comparison with `is` will be true.

This is not the same as ordinary equality.

```
a = "python"
b = "python"

print(f"a == b: {a == b}")
print(f"a is b: {a is b}")
print(f"id(a): {id(a)}")
print(f"id(b): {id(b)}")
```

Possible output on CPython:

```
a == b: True
a is b: True
id(a): 140225874658096
id(b): 140225874658096
```

The result above is common for simple string literals, but program logic must not depend on it. Equality asks whether the text values are the same. Identity asks whether the two names refer to the same object.

A dynamically built string may be equal without being the same object.

```
a = "python"
b = "".join(["py", "thon"])

print(f"a: {a!r}")
print(f"b: {b!r}")

print()
print(f"a == b: {a == b}")
print(f"a is b: {a is b}")

print()
print(f"id(a): {id(a)}")
print(f"id(b): {id(b)}")
```

Possible output:

```
a: 'python'
b: 'python'

a == b: True
a is b: False

id(a): 140225874658096
id(b): 140225874657840
```

The text is equal. The objects may be different.

The `sys.intern()` function explicitly asks CPython to intern a string.

```
import sys

a = "python"
b = "".join(["py", "thon"])

print("before interning")
print(f"a == b: {a == b}")
print(f"a is b: {a is b}")

print()
a = sys.intern(a)
b = sys.intern(b)

print("after interning")
print(f"a == b: {a == b}")
print(f"a is b: {a is b}")

print()
print(f"id(a): {id(a)}")
print(f"id(b): {id(b)}")
```

Possible output:

```
before interning
a == b: True
a is b: False

after interning
a == b: True
a is b: True
```

```
id(a): 140225874658096
id(b): 140225874658096
```

Interning is useful for repeated string keys, especially identifier-like strings used in dictionaries. If both the stored key and the lookup key are interned, CPython can sometimes use pointer comparison after hashing instead of comparing the full character sequence.

For ordinary beginner-level code, the practical rule is still simple:

Use `==` to compare string text. Treat `is` results on strings as CPython optimization evidence, not as string equality logic.

The following example shows why.

```
word1 = "springfield"
word2 = "spring" + "field"
word3 = "".join(["spring", "field"])

print(f"word1 == word2: {word1 == word2}")
print(f"word1 == word3: {word1 == word3}")

print()
print(f"word1 is word2: {word1 is word2}")
print(f"word1 is word3: {word1 is word3}")
```

Possible output on CPython:

```
word1 == word2: True
word1 == word3: True

word1 is word2: True
word1 is word3: False
```

The first two expressions may be folded or interned by CPython because their values are visible early. The third one is constructed at runtime. These details can vary. The semantic test remains `==`.

Interning is possible because strings are immutable. If the shared interned object could be edited in place, every name pointing to it would see the change, and dictionary key behavior would become unsafe. Immutable storage makes sharing safe.

5.3.3 Structural Extraction and Evaluation

A Python string is a sequence. That means it supports the same basic positional operations introduced earlier: indexing, slicing, length checking, membership testing, and iteration. The difference is that every component extracted from a string is itself a string.

Python does not expose a separate character type. A single character is represented as a `str` object of length 1.

```
text = "Ni!"

print(f"text: {text!r}")
print(f"type(text): {type(text)}")
print(f"len(text): {len(text)}")

print()
for ch in text:
    print(f"ch={ch!r:<4} type={type(ch)} len={len(ch)}")
```

Possible output:

```
text: 'Ni!'
type(text): <class 'str'>
len(text): 3

ch='N'  type=<class 'str'> len=1
ch='i'  type=<class 'str'> len=1
ch='!'  type=<class 'str'> len=1
```

Some string operations used in this section are visible through `dir()`.

```
text = "Nobody expects the Spanish Inquisition"

selected_names = [
    "__getitem__",
    "__len__",
    "__contains__",
    "format",
    "format_map",
    "center",
    "ljust",
    "rjust",
```

```
]

for name in selected_names:
    print(f"{name:<14} {name in dir(text)}")
```

Possible output:

```
__getitem__    True
__len__        True
__contains__   True
format         True
format_map     True
center         True
ljust          True
rjust          True
```

The syntax `text[index]` and `text[start:stop:step]` is ordinary Python syntax, but internally it is still item access on the string object.

Indexing and Slicing: Characters as One-Character String Objects

Indexing extracts one component from a string.

```
text = "D'oh!"

print(f"text: {text!r}")
print(f"text[0]: {text[0]!r}")
print(f"text[1]: {text[1]!r}")
print(f"text[2]: {text[2]!r}")
print(f"text[3]: {text[3]!r}")
print(f"text[4]: {text[4]!r}")
```

Possible output:

```
text: "D'oh!"
text[0]: 'D'
text[1]: "'"
text[2]: 'o'
text[3]: 'h'
text[4]: '!'
```

Each extracted component is a string object.

```
text = "D'oh!"

ch = text[0]

print(f"ch: {ch!r}")
print(f"type(ch): {type(ch)}")
print(f"len(ch): {len(ch)}")
```

Possible output:

```
ch: 'D'
type(ch): <class 'str'>
len(ch): 1
```

Negative indexing counts from the right.

```
text = "D'oh!"

print(f"text[-1]: {text[-1]!r}")
print(f"text[-2]: {text[-2]!r}")
print(f"text[-3]: {text[-3]!r}")
print(f"text[-4]: {text[-4]!r}")
print(f"text[-5]: {text[-5]!r}")
```

Possible output:

```
text[-1]: '!'
text[-2]: 'h'
text[-3]: 'o'
text[-4]: "'"
text[-5]: 'D'
```

An invalid index raises `IndexError`.

```
text = "D'oh!"

try:
    print(text[42])
except IndexError as err:
    print("indexing failed")
    print(f"message: {err}")
```

Possible output:

```
indexing failed
message: string index out of range
```

Slicing extracts a substring.

```
text = "Nobody expects the Spanish Inquisition"

print(f"text[0:6]: {text[0:6]!r}")
print(f"text[7:14]: {text[7:14]!r}")
print(f"text[19:26]: {text[19:26]!r}")
print(f"text[27:]: {text[27:]!r}")
```

Possible output:

```
text[0:6]: 'Nobody'
text[7:14]: 'expects'
text[19:26]: 'Spanish'
text[27:]: 'Inquisition'
```

The result of a string slice is also a `str`.

```
text = "No soup for you!"

part = text[3:7]

print(f"part: {part!r}")
print(f"type(part): {type(part)}")
print(f"len(part): {len(part)}")
```

Possible output:

```
part: 'soup'
type(part): <class 'str'>
len(part): 4
```

An empty slice is still a string.

```
text = "abc"

part = text[10:20]

print(f"part: {part!r}")
print(f"type(part): {type(part)}")
print(f"len(part): {len(part)}")
```

Possible output:

```
part: ''
type(part): <class 'str'>
len(part): 0
```

So the structural rule is consistent:

Indexing a string returns a one-character `str`. Slicing a string returns a possibly empty `str`.

Substring Evaluation Mechanisms: Step-Based Slice Offsets (`[start:stop:step]`) vs. Manual Pointer Offsets

A slice uses three values:

- **start**: where extraction begins;
- **stop**: where extraction stops, not included;
- **step**: how far Python moves after taking a component.

```
text = "abcdefghij"

print(f"text[0:10:1]: {text[0:10:1]!r}")
print(f"text[0:10:2]: {text[0:10:2]!r}")
print(f"text[1:10:2]: {text[1:10:2]!r}")
print(f"text[2:9:3]: {text[2:9:3]!r}")
```

Possible output:

```
text[0:10:1]: 'abcdefghij'
text[0:10:2]: 'acegi'
text[1:10:2]: 'bdfhj'
text[2:9:3]: 'cfi'
```

The slice `text[2:9:3]` means:

1. start at index 2;
2. stop before index 9;
3. take every third component.

This can be shown manually.

```
text = "abcdefghij"

print("manual positions for text[2:9:3]")
print()

for index in range(2, 9, 3):
    ch = text[index]
    print(f"index={index} ch={ch!r}")

print()
print(f"slice result: {text[2:9:3]!r}")
```

Possible output:

```
manual positions for text[2:9:3]

index=2 ch='c'
index=5 ch='f'
index=8 ch='i'

slice result: 'cfi'
```

This is not C pointer arithmetic. In C, a programmer may think in terms of a starting address plus offsets into memory. In Python, the programmer describes positions in a sequence. Python evaluates the requested positions and constructs the resulting string value.

A manual reconstruction makes the difference visible.

```
text = "abcdefghij"

manual = ""

for index in range(2, 9, 3):
    manual = manual + text[index]

sliced = text[2:9:3]

print(f"manual: {manual!r}")
print(f"sliced: {sliced!r}")
print(f"same:    {manual == sliced}")
```

Possible output:

```
manual: 'cfi'
sliced: 'cfi'
same:    True
```

The two results are equal, but the slice is the direct Python expression for the operation. The manual loop is useful for understanding, not for ordinary substring extraction.

A negative step walks backward.

```
text = "abcdef"

print(f"text[::-1]:    {text[::-1]!r}")
print(f"text[4:1:-1]:   {text[4:1:-1]!r}")
print(f"text[5:0:-2]:    {text[5:0:-2]!r}")
```


Possible output:

```
text[::-1]:    'fedcba'
text[4:1:-1]:  'edc'
text[5:0:-2]:  'fdb'
```

The same operation can again be expanded into indexes.

```
text = "abcdef"

print("manual positions for text[5:0:-2]")
print()

for index in range(5, 0, -2):
    print(f"index={index} ch={text[index]!r}")

print()
print(f"slice result: {text[5:0:-2]!r}")
```

Possible output:

```
manual positions for text[5:0:-2]

index=5 ch='f'
index=3 ch='d'
index=1 ch='b'

slice result: 'fdb'
```

The `step` cannot be zero.

```
text = "abc"

try:
    part = text[0:3:0]
    print(part)
except ValueError as err:
    print("slicing failed")
    print(f"message: {err}")
```

Possible output:

```
slicing failed
message: slice step cannot be zero
```

A zero step would mean “move by no positions”, so Python refuses the slice.

The practical rule is:

A string slice describes a positional extraction pattern. It is not manual memory traversal. Python chooses the indexes, reads the corresponding string components, and returns a string result.

String Formatting Paradigms: Variable Injection via Modern f-Strings

String formatting builds a string by combining literal text with values computed by the program. Modern Python code commonly uses f-strings.

An f-string begins with `f` before the opening quote. Expressions inside braces are evaluated and inserted into the resulting string.

```
name = "Kramer"
score = 42

line = f"{name} has score {score}."

print(line)
```

Possible output:

```
Kramer has score 42.
```

The expression inside braces can be a variable, an arithmetic expression, or an indexing operation.

```
name = "Homer"
numbers = [10, 20, 12]

line1 = f"name: {name}"
line2 = f"first letter: {name[0]}"
line3 = f"total: {numbers[0] + numbers[1] + numbers[2]}"

print(line1)
print(line2)
print(line3)
```

Possible output:

```
name: Homer
first letter: H
total: 42
```

The inserted value is converted to text. For debugging, `!r` asks for the representation form, which is often more explicit than the printed form.

```
text = "C:\new\test"
raw_text = r"C:\new\test"

print(f"text printed:    {text}")
print(f"text repr:      {text!r}")

print()
print(f"raw printed:      {raw_text}")
print(f"raw repr:         {raw_text!r}")
```

Possible output:

```
text printed:    C:
ew est
text repr:      'C:\new\test'

raw printed:    C:\new\test
raw repr:      'C:\\new\\test'
```

The `!r` form is useful when teaching strings because it exposes quotes, backslashes, newline escapes, and tabs more clearly.

F-strings also support alignment.

```
names = ["Jerry", "Elaine", "George", "Kramer"]

for index, name in enumerate(names):
    print(f"{index:>2} | {name:<10} | len={len(name):>2}")
```

Possible output:

```
0 | Jerry      | len= 5
1 | Elaine     | len= 6
2 | George     | len= 6
3 | Kramer     | len= 6
```

The formatting fields above mean:

- `>2`: align to the right in a width of 2;
- `<10`: align to the left in a width of 10;
- `>2`: align to the right in a width of 2.

Floating-point values can be formatted with explicit precision.

```
value = 10 / 3

print(f"default: {value}")
print(f"two decimals: {value:.2f}")
print(f"four decimals: {value:.4f}")
```

Possible output:

```
default: 3.3333333333333335
two decimals: 3.33
four decimals: 3.3333
```

The older `str.format()` method is also still common.

```
name = "Elaine"
score = 42

line = "{} has score {}".format(name, score)

print(line)
```

Possible output:

```
Elaine has score 42.
```

It also supports named fields.

```
line = "{name:<10} | score={score:>3}".format(
    name="George",
    score=42,
)

print(line)
```

Possible output:

```
George      | score= 42
```

For most ordinary code, f-strings are clearer because the expression appears directly where its value will be inserted.

```
name = "Bart"
age = 10

print(f"{name} is {age} years old.")
print("{} is {} years old.".format(name, age))
```

Possible output:

```
Bart is 10 years old.
Bart is 10 years old.
```

The practical rule is:

Use f-strings when building readable output from current variables and expressions. Use explicit alignment, precision, and `!r` when the exact textual representation matters.

5.3.4 String Sequence Operations

A string is a sequence, but it is also a text object with many specialized methods. These methods do not change the original string. Since `str` is immutable, operations such as searching, splitting, replacing, and case conversion return results while leaving the original object unchanged.

```
text = "These pretzels are making me thirsty"
```

```
selected_names = [
    "find",
    "index",
    "startswith",
    "endswith",
    "split",
    "join",
    "replace",
    "lower",
    "upper",
    "casefold",
]

print(f"text: {text!r}")
print(f"type(text): {type(text)}")
print()

for name in selected_names:
    print(f"{name:<12} {name in dir(text)}")
```

Possible output:

```
text: 'These pretzels are making me thirsty'
type(text): <class 'str'>
```

find	True
index	True
startswith	True
endswith	True
split	True
join	True
replace	True
lower	True
upper	True
casefold	True

The methods in this section are ordinary string methods. They are called on a string object using dot notation.

Searching and Membership: `in`, `.find()`, `.index()`, `.startswith()`, and `.endswith()`

The simplest search operation is `in`. It returns a Boolean result.

```
text = "Nobody expects the Spanish Inquisition"
```

```
print(f"{'Spanish' in text = }")
print(f"{'French' in text = }")
print(f"{'Nobody' in text = }")
print(f"{'nobody' in text = }")
```

Possible output:

```
'Spanish' in text = True
'French' in text = False
'Nobody' in text = True
'nobody' in text = False
```

String search is case-sensitive. "Nobody" and "nobody" are different strings.

The method `.find()` searches for a substring and returns the index where it first appears. If the substring is not found, it returns `-1`.

```
text = "Nobody expects the Spanish Inquisition"
```

```
pos1 = text.find("Nobody")
pos2 = text.find("expects")
pos3 = text.find("Spanish")
pos4 = text.find("French")
```

```
print(f"Nobody: {pos1}")
print(f"expects: {pos2}")
print(f"Spanish: {pos3}")
print(f"French: {pos4}")
```

Possible output:

```
Nobody: 0
expects: 7
Spanish: 19
French: -1
```

The return value is a position. It can be used for slicing.

```
text = "Nobody expects the Spanish Inquisition"

pos = text.find("Spanish")

print(f"pos: {pos}")
print(f"text[pos:]: {text[pos:]!r}")
```

Possible output:

```
pos: 19
text[pos:]: 'Spanish Inquisition'
```

The method `.index()` is similar to `.find()`, but it raises an error when the substring is absent.

```
text = "No soup for you!"

print(f"text.index('soup'): {text.index('soup')}")

try:
    print(text.index("coffee"))
except ValueError as err:
    print("index search failed")
    print(f"message: {err}")
```

Possible output:

```
text.index('soup'): 3
index search failed
message: substring not found
```

The choice is practical:

- use `.find()` when absence is an expected result;
- use `.index()` when absence should be treated as an error.

The methods `.startswith()` and `.endswith()` test boundaries.

```
text = "Hello, Newman!"

print(f"{text.startswith('Hello')} = {True}")
print(f"{text.startswith('Newman')} = {False}")
print(f"{text.endswith('Newman!')} = {True}")
print(f"{text.endswith('Jerry')} = {False}")
```

Possible output:

```
text.startswith('Hello') = True
text.startswith('Newman') = False
text.endswith('Newman!') = True
text.endswith('Jerry') = False
```

These methods are clearer than manual slicing when the intention is boundary testing.

```
filename = "report_final.txt"

print(f"manual suffix test: {filename[-4:] == '.txt'}")
print(f"endswith test: {filename.endswith('.txt')}")
```

Possible output:

```
manual suffix test: True
endswith test:      True
```

The second line communicates the intention directly: check whether the string ends with ".txt".

Splitting and Joining: `.split()` and `.join()` as Core Text-Sequence Transformations

The method `.split()` breaks a string into a list of smaller strings.

```
line = "spam eggs bacon"

parts = line.split()

print(f"line: {line!r}")
print(f"parts: {parts!r}")
print(f"type(parts): {type(parts)}")
```

Possible output:

```
line: 'spam eggs bacon'
parts: ['spam', 'eggs', 'bacon']
type(parts): <class 'list'>
```

With no argument, `.split()` splits on runs of whitespace.

```
line = "spam  eggs\tbacon\nbeans"

parts = line.split()

print(f"line: {line!r}")
print(f"parts: {parts!r}")
```

Possible output:

```
line: 'spam  eggs\tbacon\nbeans'
parts: ['spam', 'eggs', 'bacon', 'beans']
```

A separator can be supplied explicitly.

```
csv_line = "Homer,Marge,Bart,Lisa,Maggie"

parts = csv_line.split(",")

print(f"csv_line: {csv_line!r}")
print(f"parts: {parts!r}")
```

Possible output:

```
csv_line: 'Homer,Marge,Bart,Lisa,Maggie'
parts:    ['Homer', 'Marge', 'Bart', 'Lisa', 'Maggie']
```

The separator is removed from the result.

The method `.join()` performs the reverse kind of transformation. It takes an iterable of strings and joins them with a separator string.

```
parts = ["Homer", "Marge", "Bart", "Lisa", "Maggie"]

line1 = ",".join(parts)
line2 = " | ".join(parts)
line3 = " ".join(parts)

print(f"line1: {line1!r}")
print(f"line2: {line2!r}")
print(f"line3: {line3!r}")
```

Possible output:

```
line1: 'Homer,Marge,Bart,Lisa,Maggie'
line2: 'Homer | Marge | Bart | Lisa | Maggie'
line3: 'Homer Marge Bart Lisa Maggie'
```

The separator is the string before `.join()`.

```
parts = ["Ni", "Ni", "Ni"]

text1 = " ".join(parts)
text2 = "-".join(parts)
text3 = "".join(parts)

print(f"text1: {text1!r}")
print(f"text2: {text2!r}")
print(f"text3: {text3!r}")
```

Possible output:

```
text1: 'Ni Ni Ni'
text2: 'Ni-Ni-Ni'
text3: 'NiNiNi'
```

A common mistake is to try to join non-string elements directly.

```
numbers = [10, 20, 12]

try:
    text = ", ".join(numbers)
    print(text)
except TypeError as err:
    print("join failed")
    print(f"message: {err}")
```

Possible output:

```
join failed
message: sequence item 0: expected str instance, int found
```

The elements must be converted to strings first.

```
numbers = [10, 20, 12]

text_parts = []
```



```

for nr in numbers:
    text_parts.append(str(nr))

text = " + ".join(text_parts)

print(f"text_parts: {text_parts!r}")
print(f"text:      {text}")

```

Possible output:

```

text_parts: ['10', '20', '12']
text:      10 + 20 + 12

```

Splitting and joining are central because they move between:

- one text sequence;
- a list of text components;
- another text sequence built from those components.

```

line = "jerry;elaine;george;kramer"

parts = line.split(";")
pretty = " / ".join(parts)

print(f"line:   {line!r}")
print(f"parts:  {parts!r}")
print(f"pretty: {pretty!r}")

```

Possible output:

```

line:   'jerry;elaine;george;kramer'
parts:  ['jerry', 'elaine', 'george', 'kramer']
pretty: 'jerry / elaine / george / kramer'

```

Replacement and Case Transformation: `.replace()`, `.lower()`, `.upper()`, and `.casefold()`

The method `.replace()` creates a new string where occurrences of one substring are replaced with another substring.

```

text = "yada yada yada"

new_text = text.replace("yada", "blah")

print(f"text:      {text!r}")
print(f"new_text: {new_text!r}")

print()
print(f"text id:      {id(text)}")
print(f"new_text id: {id(new_text)}")

```

Possible output:

```

text:      'yada yada yada'
new_text:  'blah blah blah'

text id:    140225874656944
new_text id: 140225874657392

```

The original string is unchanged.

```
text = "D'oh! D'oh! D'oh!"

once = text.replace("D'oh", "Woohoo", 1)
twice = text.replace("D'oh", "Woohoo", 2)
all_items = text.replace("D'oh", "Woohoo")

print(f"once:      {once!r}")
print(f"twice:     {twice!r}")
print(f"all_items: {all_items!r}")
```

Possible output:

```
once:      "Woohoo! D'oh! D'oh!"
twice:     "Woohoo! Woohoo! D'oh!"
all_items: "Woohoo! Woohoo! Woohoo!"
```

The optional third argument limits the number of replacements.

Case transformation methods also return new strings.

```
text = "No Soup For You!"

lower_text = text.lower()
upper_text = text.upper()

print(f"text:      {text!r}")
print(f"lower_text: {lower_text!r}")
print(f"upper_text: {upper_text!r}")
```

Possible output:

```
text:      'No Soup For You!'
lower_text: 'no soup for you!'
upper_text: 'NO SOUP FOR YOU!'
```

These are useful for normalization before comparison.

```
answer = "YES"

print(f"answer == 'yes':      {answer == 'yes'}")
print(f"answer.lower() == 'yes': {answer.lower() == 'yes'}")
```

Possible output:

```
answer == 'yes':      False
answer.lower() == 'yes': True
```

The method `.casefold()` is a stronger case-normalization method intended for caseless text comparison.

```
word1 = "straße"
word2 = "STRASSE"

print(f"word1.lower():      {word1.lower()!r}")
print(f"word2.lower():      {word2.lower()!r}")
```

```

print(f"lower same:      {word1.lower() == word2.lower()}")

print()
print(f"word1.casefold(): {word1.casefold()!r}")
print(f"word2.casefold(): {word2.casefold()!r}")
print(f"casefold same:   {word1.casefold() == word2.casefold()}")

```

Possible output:

```

word1.lower():      'straße'
word2.lower():      'strasse'
lower same:         False

word1.casefold():   'strasse'
word2.casefold():   'strasse'
casefold same:      True

```

For simple English text, `.lower()` is often enough. For broader Unicode text comparison, `.casefold()` is usually the better tool.

A final example combines several operations in a normal cleaning pipeline.

```

line = "  These Pretzels Are Making Me Thirsty!  "

clean = line.strip()
clean = clean.lower()
clean = clean.replace("pretzels", "bagels")

print(f"line:  {line!r}")
print(f"clean: {clean!r}")

```

Possible output:

```

line:  '  These Pretzels Are Making Me Thirsty!  '
clean: 'these bagels are making me thirsty!'

```

The method `.strip()` removes leading and trailing whitespace. It is included here only because it commonly appears next to replacement and case normalization in real text-processing code.

The practical rule is:

String methods return new string values. Searching reads the string, splitting converts text into pieces, joining builds text from pieces, replacement creates modified text, and case conversion creates normalized text.

5.4 Dynamic Pointer Arrays: The Python List Architecture (list)

A Python list is an ordered and mutable sequence. It can grow, shrink, replace elements, reorder elements, and hold objects of different types in the same structure. This makes it feel like a flexible array, but its internal model is different from a C array of direct primitive values.

In CPython, a list is best understood as a dynamic array of references. The list owns a contiguous sequence of pointer slots, and each slot points to a Python object stored elsewhere. This explains why lists can contain mixed types, why appending at the end is usually efficient, why insertion near the front is expensive, and why copying lists requires careful attention.

This section studies lists from that concrete runtime perspective: first the memory layout contrast with C arrays, then CPython's resizing behavior, then the ordinary mutation operations, and finally the difference between assignment, shallow copying, and deep copying.

5.4.1 The Memory Layout Paradigm Contrast

A Python list looks like a flexible array, but it should not be understood as a C array of raw values. The similarity is positional access: both C arrays and Python lists allow access by numeric index. The difference is what is stored in each position.

A C array usually stores raw values directly. A CPython list stores references to Python objects. This means that the list's internal slot storage is contiguous, but the actual objects referenced by those slots may live in different heap locations.

```
items = ["spam", 42, 3.14, True]

print(f"items: {items!r}")
print(f"type(items): {type(items)}")
print(f"len(items): {len(items)}")

print()
print("selected list attributes")
for name in ["append", "extend", "insert", "pop", "remove", "sort", "copy"]:
    print(f"{name:<8} {name in dir(items)}")

print()
for index, item in enumerate(items):
    print(f"index={index} value={item!r:<8} type={type(item)} id={id(item)}")
```

Possible output:

```
items: ['spam', 42, 3.14, True]
type(items): <class 'list'>
len(items): 4

selected list attributes
append    True
extend    True
insert    True
pop        True
remove    True
sort       True
copy       True

index=0 value='spam'    type=<class 'str'> id=140619786560368
index=1 value=42        type=<class 'int'> id=140619796012304
index=2 value=3.14      type=<class 'float'> id=140619786981552
index=3 value=True      type=<class 'bool'> id=140619795594560
```

The list has four positions. Each position refers to one Python object. The objects are allowed to have different types because the list does not store their raw physical representation directly inside its own slot array.

The C Array Blueprint: Fixed, Contiguous Structures Storing Homogeneous Raw Primitive Values Directly

In C, an array such as the following stores integer values directly in adjacent memory cells:

```
int nums[4] = {10, 20, 30, 42};
```

Conceptually, the storage looks like this:

```

+----+----+----+----+
| 10 | 20 | 30 | 42 |
+----+----+----+----+
  0    1    2    3

```

The array positions contain the actual integer values. If an `int` occupies four bytes on that platform, then the array body contains four consecutive integer-sized storage cells.

Python's built-in `array` module can be used to demonstrate this homogeneous-storage idea from Python code. It is not a normal Python list. It is a compact array constrained to one primitive storage code.

```

from array import array
import sys

nums = array("i", [10, 20, 30, 42])

print(f"nums: {nums}")
print(f"type(nums): {type(nums)}")
print(f"len(nums): {len(nums)}")
print(f"item size: {nums.itemsize} bytes")
print(f"data bytes: {len(nums) * nums.itemsize}")
print(f"object size: {sys.getsizeof(nums)} bytes")

print()
for index, nr in enumerate(nums):
    print(f"index={index} value={nr}")

```

Possible output:

```

nums: array('i', [10, 20, 30, 42])
type(nums): <class 'array.array'>
len(nums): 4
item size: 4 bytes
data bytes: 16
object size: 96 bytes

index=0 value=10
index=1 value=20
index=2 value=30
index=3 value=42

```

The important restriction is visible in the type code `"i"`. This array is for integer storage. It is not a general container for arbitrary Python objects.

A C-style fixed numeric array has three main properties:

- the elements have one declared storage format;
- the element values are stored directly in the array body;
- indexing can compute the address of an element by using the base address and the element size.

For example, in a simple C mental model:

```
address of nums[i] = base address + i * sizeof(int)
```

That model works because every element has the same physical size and the same direct storage format.

A normal Python list is different.

```
items = [10, "twenty", 30.0, True]

print(f"items: {items!r}")
print(f"type(items): {type(items)}")

print()
for index, item in enumerate(items):
    print(f"index={index} value={item!r:<10} type={type(item)}")
```

Possible output:

```
items: [10, 'twenty', 30.0, True]
type(items): <class 'list'>

index=0 value=10           type=<class 'int'>
index=1 value='twenty'     type=<class 'str'>
index=2 value=30.0         type=<class 'float'>
index=3 value=True         type=<class 'bool'>
```

This cannot be a direct array of raw `int`, raw `str`, raw `float`, and raw `bool` data stored in one uniform primitive layout. The list can be heterogeneous because its slots store references.

The Python List Blueprint: A Contiguous Array Storing Heterogeneous PyObject* References on the Heap

In CPython, a list stores a contiguous array of object references. At the C level, each slot stores a `PyObject*`. This is a pointer to a Python object.

A simplified picture is:

```
Python list object
+-----+
| current length                |
| allocated slot capacity      |
| pointer to internal slot array |
+-----+

internal slot array
+-----+-----+-----+-----+
| PyObject* | PyObject* | PyObject* | PyObject* |
+-----+-----+-----+-----+
|           |           |           |           |
|  v        |  v        |  v        |  v        |
| "spam"    | 42        | 3.14     | True     |
```

The list's slot array is contiguous. The Python objects reached through those slots are separate objects.

This explains why the same object can appear in several positions.

```
inner = ["larch"]

items = [inner, inner, inner]
```

```

print(f"items: {items!r}")
print(f"id(inner): {id(inner)}")

print()
for index, item in enumerate(items):
    print(f"index={index} id(item)={id(item)} item={item!r}")

```

Possible output:

```

items: [['larch'], ['larch'], ['larch']]
id(inner): 140619786871040

index=0 id(item)=140619786871040 item=['larch']
index=1 id(item)=140619786871040 item=['larch']
index=2 id(item)=140619786871040 item=['larch']

```

The outer list does not contain three independent inner lists. It contains three references to the same inner list object.

If that inner object is mutated, all three positions appear to change because all three positions lead to the same object.

```

inner = ["larch"]
items = [inner, inner, inner]

items[0].append(42)

print(f"items: {items!r}")

print()
for index, item in enumerate(items):
    print(f"index={index} id(item)={id(item)} item={item!r}")

```

Possible output:

```

items: [['larch', 42], ['larch', 42], ['larch', 42]]

index=0 id(item)=140619786871040 item=['larch', 42]
index=1 id(item)=140619786871040 item=['larch', 42]
index=2 id(item)=140619786871040 item=['larch', 42]

```

This is not three mutations. It is one mutation observed through three references.

Replacing a list element is a different operation. It changes one slot in the outer list so that the slot points to another object.

```

items = ["spam", "eggs", "bacon"]

old_obj = items[1]

print("before replacement")
print(f"items: {items!r}")
print(f"old_obj: {old_obj!r:<8} id={id(old_obj)}")
print(f"items[1]: {items[1]!r:<8} id={id(items[1])}")

items[1] = "beans"

```

```

print()
print("after replacement")
print(f"items: {items!r}")
print(f"old_obj: {old_obj!r:<8} id={id(old_obj)}")
print(f"items[1]: {items[1]!r:<8} id={id(items[1])}")

```

Possible output:

```

before replacement
items: ['spam', 'eggs', 'bacon']
old_obj: 'eggs' id=140619786560432
items[1]: 'eggs' id=140619786560432

```

```

after replacement
items: ['spam', 'beans', 'bacon']
old_obj: 'eggs' id=140619786560432
items[1]: 'beans' id=140619786560560

```

The old string object was not edited. The list slot at index 1 was redirected to another object. The same distinction matters with mutable objects.

```

a = ["Jerry"]
b = ["Elaine"]

items = [a, b]

print("before")
print(f"items: {items!r}")
print(f"id(a): {id(a)}")
print(f"id(items[0]): {id(items[0])}")

a.append("Kramer")

print()
print("after mutating a")
print(f"items: {items!r}")
print(f"a: {a!r}")
print(f"id(a): {id(a)}")
print(f"id(items[0]): {id(items[0])}")

```

Possible output:

```

before
items: [['Jerry'], ['Elaine']]
id(a): 140619786871232
id(items[0]): 140619786871232

after mutating a
items: [['Jerry', 'Kramer'], ['Elaine']]
a: ['Jerry', 'Kramer']
id(a): 140619786871232
id(items[0]): 140619786871232

```

The outer list did not change its first slot. The object referenced by that slot changed internally. The practical rule is:

A Python list owns a sequence of reference slots. It does not contain independent physical copies of all objects assigned into those slots.

This is the foundation for understanding assignment, shallow copies, repeated inner lists, slice assignment, and the difference between replacing a list element and mutating the object found at that element.

5.4.2 CPython Dynamic Scaling Mechanics

A Python list can grow and shrink. This does not mean that CPython asks the memory allocator for a new block every time one element is appended. That would make repeated appending unnecessarily expensive.

Instead, CPython keeps two related quantities:

- the current logical length of the list;
- the allocated internal slot capacity.

The logical length is what `len()` returns. The allocated capacity is an implementation detail: it is the number of pointer slots currently reserved internally.

```
items = []

print(f"items: {items!r}")
print(f"len(items): {len(items)}")

items.append("spam")
items.append("eggs")
items.append(42)

print()
print(f"items: {items!r}")
print(f"len(items): {len(items)}")
```

Possible output:

```
items: []
len(items): 0

items: ['spam', 'eggs', 42]
len(items): 3
```

The program sees length. CPython also manages capacity behind that visible length.

Dynamic Over-Allocation Invariants: How CPython Pre-Allocates Extra Slots During List Resizing to Support Amortized $O(1)$ Appends

When a list grows beyond its current internal capacity, CPython resizes the internal pointer array. During that resize, it usually allocates more slots than are immediately needed. Those extra slots are empty capacity for future appends.

The effect can be observed indirectly with `sys.getsizeof()`.

```
import sys

items = []
last_size = sys.getsizeof(items)
```

```

print(f"len={len(items):>2} size={last_size}")

for nr in range(1, 35):
    items.append(nr)
    size = sys.getsizeof(items)

    if size != last_size:
        print(f"len={len(items):>2} size={size} growth={size - last_size}")
        last_size = size

```

Possible output on one 64-bit CPython build:

```

len= 0 size=56
len= 1 size=88 growth=32
len= 5 size=120 growth=32
len= 9 size=184 growth=64
len=17 size=248 growth=64
len=25 size=312 growth=64
len=33 size=376 growth=64

```

The exact byte values depend on Python version, build, and platform. The important pattern is that the object size does not increase after every single append. It increases in jumps.

This means that several appends can reuse already allocated pointer slots.

```

import sys

items = []

for nr in range(12):
    before_size = sys.getsizeof(items)
    items.append(nr)
    after_size = sys.getsizeof(items)

    changed = before_size != after_size

    print(f"append {nr:>2}: len={len(items):>2} "
          f"before={before_size:<4} after={after_size:<4} "
          f"resized={changed}")

```

Possible output:

```

append 0: len= 1 before=56   after=88   resized=True
append 1: len= 2 before=88   after=88   resized=False
append 2: len= 3 before=88   after=88   resized=False
append 3: len= 4 before=88   after=88   resized=False
append 4: len= 5 before=88   after=120  resized=True
append 5: len= 6 before=120  after=120  resized=False
append 6: len= 7 before=120  after=120  resized=False
append 7: len= 8 before=120  after=120  resized=False
append 8: len= 9 before=120  after=184  resized=True
append 9: len=10 before=184  after=184  resized=False
append 10: len=11 before=184 after=184  resized=False
append 11: len=12 before=184 after=184  resized=False

```

The visible operation is always **append**. Internally, some appends only place a pointer into an already available slot. Other appends force a resize.

This is why list appending is described as amortized $O(1)$. One particular append may trigger allocation and copying of pointer slots. But across a long sequence of appends, the extra allocated space prevents resizing on every operation.

A simplified mental model is:

```
visible list:
+---+---+---+
| 10 | 20 | 42 |
+---+---+---+
length = 3

internal slot capacity:
+-----+-----+-----+-----+-----+
| used | used | used | free | free |
+-----+-----+-----+-----+-----+
capacity = 5
```

The free slots are not visible as list elements. They are only reserved internal space.

Appending into free capacity is cheap.

```
items = [10, 20, 30]

items.append(42)

print(items)
```

Possible output:

```
[10, 20, 30, 42]
```

But if there is no free internal slot, CPython must create or resize the internal slot array, copy existing references as needed, and then store the new reference.

The practical programmer rule is:

Appending to the end of a list is normally efficient because CPython grows the internal pointer array in spare-capacity jumps, not one exact slot at a time.

This does not mean that a list has infinite free space. It means that CPython uses extra reserved slots to make repeated appending efficient over time.

Memory Shifting Costs: The $O(N)$ Penalty of Arbitrary Index Insertions and Deletions (`.insert()`, `.pop()`)

Appending at the end is special because no existing element has to move to another logical position. Inserting near the front is different. To insert a new element at index 0, every existing pointer slot must shift one position to the right.

```
items = ["B", "C", "D"]

print("before insert")
print(items)

items.insert(0, "A")
```

```
print()
print("after insert")
print(items)
```

Possible output:

```
before insert
['B', 'C', 'D']

after insert
['A', 'B', 'C', 'D']
```

The result looks simple, but the internal slot movement is conceptually:

```
before:
index:  0    1    2
        B    C    D

make room at index 0:
index:  0    1    2    3
        ?    B    C    D

place new reference:
index:  0    1    2    3
        A    B    C    D
```

The list does not copy the full Python objects. It shifts references. But shifting many references still costs time proportional to the number of shifted slots.

Inserting in the middle shifts only the elements after the insertion point.

```
items = ["Jerry", "Elaine", "Kramer"]

items.insert(2, "George")

print(items)
```

Possible output:

```
['Jerry', 'Elaine', 'George', 'Kramer']
```

Conceptually:

```
before:
0: Jerry
1: Elaine
2: Kramer

after inserting at index 2:
0: Jerry
1: Elaine
2: George
3: Kramer
```

Only the old element at index 2 and everything after it must move.

The worst case is insertion near the beginning of a large list.

```

import time

count = 50000

items = []

start = time.perf_counter()

for nr in range(count):
    items.append(nr)

end = time.perf_counter()
append_time = end - start

items = []

start = time.perf_counter()

for nr in range(count):
    items.insert(0, nr)

end = time.perf_counter()
insert_front_time = end - start

print(f"append loop:      {append_time:.6f} seconds")
print(f"insert front loop: {insert_front_time:.6f} seconds")

```

Possible output:

```

append loop:      0.002800 seconds
insert front loop: 0.410000 seconds

```

The exact times depend on the machine and Python build. The important point is the growth pattern: repeated front insertion is much more expensive than repeated appending because existing slots are shifted again and again.

Deletion has the same structural issue. Removing from the end is cheap because no later elements exist.

```

items = ["A", "B", "C", "D"]

last = items.pop()

print(f"last:  {last!r}")
print(f"items: {items!r}")

```

Possible output:

```

last:  'D'
items: ['A', 'B', 'C']

```

Popping from the middle or beginning requires shifting the later elements left.

```

items = ["A", "B", "C", "D"]

removed = items.pop(0)

```

```
print(f"removed: {removed!r}")
print(f"items:    {items!r}")
```

Possible output:

```
removed: 'A'
items:   ['B', 'C', 'D']
```

Conceptually:

```
before:
index:  0    1    2    3
        A    B    C    D
```

```
remove index 0:
index:  0    1    2
        B    C    D
```

The references to B, C, and D must move left by one slot.
The timing difference can be observed.

```
import time

count = 50000

items = list(range(count))

start = time.perf_counter()

while items:
    items.pop()

end = time.perf_counter()
pop_end_time = end - start

items = list(range(count))

start = time.perf_counter()

while items:
    items.pop(0)

end = time.perf_counter()
pop_front_time = end - start

print(f"pop end loop:    {pop_end_time:.6f} seconds")
print(f"pop front loop: {pop_front_time:.6f} seconds")
```

Possible output:

```
pop end loop:    0.002500 seconds
pop front loop: 0.180000 seconds
```

Again, the exact numbers are not the lesson. The structural cost is the lesson.

- `append(x)` adds at the end and is amortized $O(1)$;
- `pop()` removes from the end and is $O(1)$;
- `insert(i, x)` may shift many elements and is $O(N)$;
- `pop(i)` away from the end may shift many elements and is $O(N)$.

If a program repeatedly adds or removes values at the front, a list is usually the wrong structure. The standard library provides `collections.deque` for efficient append and pop operations at both ends.

```
from collections import deque

items = deque()

items.append("right")
items.appendleft("left")

print(items)

left = items.popleft()
right = items.pop()

print(f"left: {left!r}")
print(f"right: {right!r}")
print(items)
```

Possible output:

```
deque(['left', 'right'])
left: 'left'
right: 'right'
deque([])
```

The practical rule is:

Use lists when the main growth pattern is appending at the end and indexing by position. Be cautious with repeated insertion or deletion near the beginning, because the internal pointer slots must be shifted.

5.4.3 List Mutation Operations

A list is mutable. This means that the list object can be changed in place after it has been created. New references can be added, existing slots can be replaced, ranges can be overwritten, elements can be removed, and the current order can be changed.

```
items = ["spam", "eggs"]

print(f"before: {items!r}")
print(f"id:      {id(items)}")

items.append(42)

print()
print(f"after:   {items!r}")
print(f"id:     {id(items)}")
```

Possible output:

```
before: ['spam', 'eggs']
id:     140619786870656

after:  ['spam', 'eggs', 42]
id:     140619786870656
```

The identity of the list is the same. The object was mutated.

This is different from string operations, where a new string object is created because strings are immutable.

```
text = "spam"
items = ["spam"]

text_id = id(text)
items_id = id(items)

text = text + " eggs"
items.append("eggs")

print(f"text: {text!r}")
print(f"items: {items!r}")

print()
print(f"text same object: {id(text) == text_id}")
print(f"items same object: {id(items) == items_id}")
```

Possible output:

```
text:  'spam eggs'
items: ['spam', 'eggs']

text same object:  False
items same object:  True
```

The methods in this section are the ordinary mutation tools used with lists.

```
items = ["Homer", "Marge", "Bart"]

selected_names = [
    "append",
    "extend",
    "insert",
    "remove",
    "pop",
    "sort",
    "reverse",
    "copy",
]

print(f"type(items): {type(items)}")
print()

for name in selected_names:
    print(f"{name:<10} {name in dir(items)}")
```


Possible output:

```
type(items): <class 'list'>

append      True
extend      True
insert      True
remove      True
pop         True
sort        True
reverse     True
copy        True
```

Appending and Extending: `.append()` vs. `.extend()`

The method `.append()` adds one object to the end of the list.

```
items = ["spam", "eggs"]

items.append("bacon")
items.append(42)

print(items)
```

Possible output:

```
['spam', 'eggs', 'bacon', 42]
```

The argument passed to `append()` is inserted as one element. If the argument is itself a list, that whole list becomes one element.

```
items = ["spam", "eggs"]
more = ["bacon", "beans"]

items.append(more)

print(f"items: {items!r}")
print(f"len(items): {len(items)}")

print()
for index, item in enumerate(items):
    print(f"index={index} value={item!r} type={type(item)}")
```

Possible output:

```
items: ['spam', 'eggs', ['bacon', 'beans']]
len(items): 3

index=0 value='spam' type=<class 'str'>
index=1 value='eggs' type=<class 'str'>
index=2 value=['bacon', 'beans'] type=<class 'list'>
```

The list `more` was not unpacked. It was appended as one object.

The method `.extend()` takes an iterable and appends its elements one by one.

```

items = ["spam", "eggs"]
more = ["bacon", "beans"]

items.extend(more)

print(f"items: {items!r}")
print(f"len(items): {len(items)}")

print()
for index, item in enumerate(items):
    print(f"index={index} value={item!r} type={type(item)}")

```

Possible output:

```

items: ['spam', 'eggs', 'bacon', 'beans']
len(items): 4

index=0 value='spam' type=<class 'str'>
index=1 value='eggs' type=<class 'str'>
index=2 value='bacon' type=<class 'str'>
index=3 value='beans' type=<class 'str'>

```

So the practical distinction is:

- `append(x)` adds `x` as one new element;
- `extend(xs)` walks through `xs` and adds each component.

This difference is especially visible with strings.

```

items1 = ["Ni"]
items2 = ["Ni"]

items1.append("abc")
items2.extend("abc")

print(f"items1: {items1!r}")
print(f"items2: {items2!r}")

```

Possible output:

```

items1: ['Ni', 'abc']
items2: ['Ni', 'a', 'b', 'c']

```

A string is iterable, so `extend("abc")` appends its characters one by one. Since Python has no separate character type, each appended character is a one-character `str` object.

Both methods mutate the same list object.

```

items = ["Jerry"]

old_id = id(items)

items.append("Elaine")
items.extend(["George", "Kramer"])

print(f"items: {items!r}")
print(f"same object: {id(items) == old_id}")

```

Possible output:

```
items: ['Jerry', 'Elaine', 'George', 'Kramer']
same object: True
```

These methods return `None`. The result is stored in the mutated list, not in the method return value.

```
items = ["Homer"]

result = items.append("Marge")

print(f"items: {items!r}")
print(f"result: {result!r}")
```

Possible output:

```
items: ['Homer', 'Marge']
result: None
```

A common mistake is therefore:

```
items = ["Homer"]
items = items.append("Marge")

print(items)
```

Possible output:

```
None
```

The name `items` was rebound to the return value of `append()`, which is `None`. The correct form is to call `append()` without assigning its return value.

Index Assignment and Slice Assignment

Index assignment replaces the object reference stored at one list position.

```
items = ["spam", "eggs", "bacon"]

print(f"before: {items!r}")

items[1] = "beans"

print(f"after: {items!r}")
```

Possible output:

```
before: ['spam', 'eggs', 'bacon']
after:  ['spam', 'beans', 'bacon']
```

The list object stays the same. One slot now points to another object.

```

items = ["spam", "eggs", "bacon"]

old_id = id(items)
old_element = items[1]

items[1] = "beans"

print(f"items: {items!r}")
print(f"same list object: {id(items) == old_id}")

print()
print(f"old_element: {old_element!r}")
print(f"new element: {items[1]!r}")

```

Possible output:

```

items: ['spam', 'beans', 'bacon']
same list object: True

```

```

old_element: 'eggs'
new element: 'beans'

```

An invalid index assignment raises `IndexError`.

```

items = ["spam", "eggs"]

try:
    items[5] = "bacon"
except IndexError as err:
    print("assignment failed")
    print(f"message: {err}")

```

Possible output:

```

assignment failed
message: list assignment index out of range

```

Index assignment replaces an existing position. It does not create missing positions.

Slice assignment replaces a range of positions.

```

items = ["a", "b", "c", "d", "e"]

print(f"before: {items!r}")

items[1:4] = ["X", "Y"]

print(f"after: {items!r}")

```

Possible output:

```

before: ['a', 'b', 'c', 'd', 'e']
after:  ['a', 'X', 'Y', 'e']

```

The slice `items[1:4]` selected "b", "c", and "d". Those three elements were replaced by two new elements.

Slice assignment can therefore change the length of the list.

```

items = ["a", "b", "c"]

print(f"before: {items!r}")
print(f"len before: {len(items)}")

items[1:2] = ["X", "Y", "Z"]

print()
print(f"after: {items!r}")
print(f"len after: {len(items)}")

```

Possible output:

```

before: ['a', 'b', 'c']
len before: 3

after: ['a', 'X', 'Y', 'Z', 'c']
len after: 5

```

An empty slice can be used as an insertion point.

```

items = ["a", "d"]

items[1:1] = ["b", "c"]

print(items)

```

Possible output:

```
['a', 'b', 'c', 'd']
```

The slice `items[1:1]` selects no existing elements, but it identifies the boundary before index 1. Slice assignment requires an iterable on the right side.

```

items = ["a", "b", "c"]

try:
    items[1:2] = 42
except TypeError as err:
    print("slice assignment failed")
    print(f"message: {err}")

```

Possible output:

```

slice assignment failed
message: can only assign an iterable

```

If a string is assigned to a slice, the string is iterated character by character.

```

items = ["a", "b", "c"]

items[1:2] = "XY"

print(items)

```

Possible output:

```
['a', 'X', 'Y', 'c']
```

This is the same iterable rule seen with `extend()`.

The practical rule is:

Index assignment replaces one existing slot. Slice assignment replaces a range of slots and may also change the list length.

Removing Elements: `.remove()`, `.pop()`, and `del`

Lists provide several removal mechanisms. They differ by how the target is chosen.

- `remove(value)` removes the first equal value;
- `pop(index)` removes by position and returns the removed object;
- `del items[index]` removes by position without returning the object;
- `del items[start:stop]` removes a slice.

The method `.remove()` searches by value.

```
items = ["spam", "eggs", "spam", "bacon"]
```

```
items.remove("spam")
```

```
print(items)
```

Possible output:

```
['eggs', 'spam', 'bacon']
```

Only the first matching value was removed.

If the value is absent, `remove()` raises `ValueError`.

```
items = ["spam", "eggs"]
```

```
try:
    items.remove("bacon")
except ValueError as err:
    print("remove failed")
    print(f"message: {err}")
```

Possible output:

```
remove failed
message: list.remove(x): x not in list
```

The method `.pop()` removes by index and returns the removed object. With no argument, it removes the last element.

```
items = ["Jerry", "Elaine", "George", "Kramer"]
```

```
last = items.pop()
```

```
print(f"last: {last!r}")
print(f"items: {items!r}")
```

Possible output:

```
last: 'Kramer'
items: ['Jerry', 'Elaine', 'George']
```

With an explicit index, `pop()` removes that position.

```
items = ["Jerry", "Elaine", "George", "Kramer"]

removed = items.pop(1)

print(f"removed: {removed!r}")
print(f"items: {items!r}")
```

Possible output:

```
removed: 'Elaine'
items: ['Jerry', 'George', 'Kramer']
```

An invalid `pop` index raises `IndexError`.

```
items = ["Jerry"]

try:
    items.pop(42)
except IndexError as err:
    print("pop failed")
    print(f"message: {err}")
```

Possible output:

```
pop failed
message: pop index out of range
```

The statement `del` removes a list element by position.

```
items = ["Homer", "Marge", "Bart", "Lisa"]

del items[2]

print(items)
```

Possible output:

```
['Homer', 'Marge', 'Lisa']
```

Unlike `pop()`, `del` does not return the removed object. It is a statement, not a method call.

```
items = ["Homer", "Marge", "Bart", "Lisa", "Maggie"]

del items[1:4]

print(items)
```

Possible output:

```
['Homer', 'Maggie']
```

The slice `items[1:4]` selected "Marge", "Bart", and "Lisa", and `del` removed that range. The three removal styles are often chosen like this:

```
items = ["a", "b", "c", "d"]

# remove by known value
items.remove("b")

# remove by position and keep removed object
removed = items.pop(1)

# remove by position and ignore removed object
del items[0]

print(f"removed: {removed!r}")
print(f"items:    {items!r}")
```

Possible output:

```
removed: 'c'
items:   ['d']
```

The practical rule is:

Use `remove()` when the value identifies the target. Use `pop()` when the position identifies the target and the removed object is needed. Use `del` when the position or slice identifies the target and no returned object is needed.

Sorting and Reversing In Place: `.sort()` and `.reverse()`

The method `.sort()` sorts a list in place.

```
items = [30, 10, 42, 20]

old_id = id(items)

result = items.sort()

print(f"items:    {items!r}")
print(f"result:    {result!r}")
print(f"same object: {id(items) == old_id}")
```

Possible output:

```
items:  [10, 20, 30, 42]
result: None
same object: True
```

The method changes the list and returns `None`. This follows the same mutation convention as `append()` and `extend()`.

The method `.reverse()` also mutates the list in place.

```
items = ["Jerry", "Elaine", "George", "Kramer"]

old_id = id(items)
```



```
result = items.reverse()

print(f"items: {items!r}")
print(f"result: {result!r}")
print(f"same object: {id(items) == old_id}")
```

Possible output:

```
items: ['Kramer', 'George', 'Elaine', 'Jerry']
result: None
same object: True
```

Sorting requires the elements to be comparable.

```
items = [10, "twenty", 30]

try:
    items.sort()
except TypeError as err:
    print("sort failed")
    print(f"message: {err}")
```

Possible output:

```
sort failed
message: '<' not supported between instances of 'str' and 'int'
```

The list may hold heterogeneous objects, but sorting still needs a meaningful ordering between the compared elements.

A key function can tell Python what value to sort by. For simple cases, built-in functions are enough.

```
names = ["Kramer", "Ed", "George", "Elaine", "Bob"]

names.sort(key=len)

print(names)
```

Possible output:

```
['Ed', 'Bob', 'Kramer', 'George', 'Elaine']
```

The list was sorted by string length.

Sorting can also be reversed.

```
numbers = [10, 30, 20, 42]

numbers.sort(reverse=True)

print(numbers)
```

Possible output:

```
[42, 30, 20, 10]
```

For strings, the default order is lexicographic and case-sensitive.

```
words = ["spam", "Eggs", "bacon", "Beans"]

words.sort()

print(words)
```

Possible output:

```
['Beans', 'Eggs', 'bacon', 'spam']
```

A case-normalizing key can make the ordering less surprising for ordinary text.

```
words = ["spam", "Eggs", "bacon", "Beans"]

words.sort(key=str.casefold)

print(words)
```

Possible output:

```
['bacon', 'Beans', 'Eggs', 'spam']
```

The function `sorted()` is different from `.sort()`. It creates a new sorted list and leaves the original object unchanged.

```
items = [30, 10, 42, 20]

new_items = sorted(items)

print(f"items:      {items!r}")
print(f"new_items: {new_items!r}")

print()
print(f"same object: {items is new_items}")
```

Possible output:

```
items:      [30, 10, 42, 20]
new_items: [10, 20, 30, 42]
```

```
same object: False
```

The same distinction exists between `.reverse()` and `reversed()`.

```
items = ["a", "b", "c"]

new_items = list(reversed(items))

print(f"items:      {items!r}")
print(f"new_items: {new_items!r}")
```

Possible output:

```
items:      ['a', 'b', 'c']
new_items: ['c', 'b', 'a']
```

The practical rule is:

`.sort()` and `.reverse()` mutate the existing list and return `None`. Use `sorted()` or `reversed()` when a separate result is wanted.

5.4.4 Deep vs. Shallow Structural Cloning

Copying a list is not the same operation as assigning another name to the same list. This distinction is essential because a list is a mutable object. If two names point to the same list, a mutation through one name is visible through the other.

The issue becomes even more important with nested lists. A copy may duplicate only the outer list structure while still sharing the inner list objects.

```
a = ["spam", "eggs"]

b = a

print(f"a: {a!r}")
print(f"b: {b!r}")

print()
print(f"id(a): {id(a)}")
print(f"id(b): {id(b)}")
print(f"a is b: {a is b}")
```

Possible output:

```
a: ['spam', 'eggs']
b: ['spam', 'eggs']

id(a): 140619786870656
id(b): 140619786870656
a is b: True
```

There are two names, but only one list object.

Assignment Is Not Copying: Shared List References with `b = a`

The assignment `b = a` does not create a new list. It binds the name `b` to the same object already referenced by `a`.

```
a = ["Homer", "Marge"]

b = a

b.append("Bart")

print(f"a: {a!r}")
print(f"b: {b!r}")

print()
print(f"a is b: {a is b}")
```

Possible output:

```
a: ['Homer', 'Marge', 'Bart']
b: ['Homer', 'Marge', 'Bart']

a is b: True
```

The mutation was performed through `b`, but `a` sees it because both names refer to the same list object.

This is not special to `append()`. Any in-place mutation has the same visibility.

```
a = ["Jerry", "Elaine", "George"]

b = a

b[2] = "Kramer"

print(f"a: {a!r}")
print(f"b: {b!r}")
print(f"a is b: {a is b}")
```

Possible output:

```
a: ['Jerry', 'Elaine', 'Kramer']
b: ['Jerry', 'Elaine', 'Kramer']
a is b: True
```

The list slot at index 2 was replaced in the one shared list object.

The same identity relation can be drawn as:

```
a  ----+
    |
    v
  list object: ["Jerry", "Elaine", "Kramer"]
    ^
    |
b  ----+
```

The practical rule is:

Assignment creates another reference to the same object. It does not clone the object.

Shallow Copies: `a[:]`, `list(a)`, and `.copy()`

A shallow copy creates a new outer list. The new list has its own slot array, but the slots refer to the same element objects as the original list.

There are several common ways to create a shallow list copy:

- `a[:]`;
- `list(a)`;
- `a.copy()`.

```
a = ["spam", "eggs", 42]

b = a[:]
c = list(a)
d = a.copy()

print(f"a: {a!r}")
print(f"b: {b!r}")
print(f"c: {c!r}")
```

```
print(f"d: {d!r}")

print()
print(f"a is b: {a is b}")
print(f"a is c: {a is c}")
print(f"a is d: {a is d}")
```

Possible output:

```
a: ['spam', 'eggs', 42]
b: ['spam', 'eggs', 42]
c: ['spam', 'eggs', 42]
d: ['spam', 'eggs', 42]
```

```
a is b: False
a is c: False
a is d: False
```

The outer lists are different objects.

A mutation of the outer structure of one list does not change the outer structure of the other list.

```
a = ["spam", "eggs", 42]

b = a.copy()

b.append("bacon")
b[0] = "beans"

print(f"a: {a!r}")
print(f"b: {b!r}")

print()
print(f"a is b: {a is b}")
```

Possible output:

```
a: ['spam', 'eggs', 42]
b: ['beans', 'eggs', 42, 'bacon']
```

```
a is b: False
```

The shallow copy protected the outer list structure. The name `b` refers to a separate list.

However, shallow copying does not copy the element objects themselves. It copies the references.

```
a = ["spam", "eggs", 42]
b = a.copy()

print("outer list identities")
print(f"id(a): {id(a)}")
print(f"id(b): {id(b)}")

print()
print("element identities")
for index in range(len(a)):
    print(f"index={index} id(a[{index}])={id(a[{index}]}) "
          f"id(b[{index}])={id(b[{index}]}) same={a[{index}] is b[{index}]}")
```

Possible output:

```
outer list identities
id(a): 140619786870656
id(b): 140619786871040

element identities
index=0 id(a[index])=140619786560368 id(b[index])=140619786560368 same=True
index=1 id(a[index])=140619786560432 id(b[index])=140619786560432 same=True
index=2 id(a[index])=140619796012304 id(b[index])=140619796012304 same=True
```

The outer list is new. The element references are shared.

For immutable elements such as strings, integers, floats, and tuples of immutable objects, this sharing is usually harmless because the shared element objects cannot be changed in place.

```
a = ["Ni", 42]
b = a.copy()

b[0] = "Ekki-ekki"
b[1] = 100

print(f"a: {a!r}")
print(f"b: {b!r}")
```

Possible output:

```
a: ['Ni', 42]
b: ['Ekki-ekki', 100]
```

The list slots were replaced in `b`. The original list `a` kept its own slots.

Nested Structures: Why Shallow Copying Fails for Lists Inside Lists

A shallow copy becomes dangerous when the list contains mutable objects, especially other lists.

```
a = [
    ["Homer", "Marge"],
    ["Bart", "Lisa"],
]

b = a.copy()

print("before mutation")
print(f"a: {a!r}")
print(f"b: {b!r}")

print()
print(f"a is b: {a is b}")
print(f"a[0] is b[0]: {a[0] is b[0]}")
print(f"a[1] is b[1]: {a[1] is b[1]}")
```

Possible output:

```
before mutation
a: [['Homer', 'Marge'], ['Bart', 'Lisa']]
b: [['Homer', 'Marge'], ['Bart', 'Lisa']]
```

```
a is b: False
a[0] is b[0]: True
a[1] is b[1]: True
```

The outer list was copied. The inner lists were not copied.

A mutation inside an inner list is therefore visible through both outer lists.

```
a = [
    ["Homer", "Marge"],
    ["Bart", "Lisa"],
]

b = a.copy()

b[0].append("Maggie")

print(f"a: {a!r}")
print(f"b: {b!r}")

print()
print(f"a[0] is b[0]: {a[0] is b[0]}")
```

Possible output:

```
a: [['Homer', 'Marge', 'Maggie'], ['Bart', 'Lisa']]
b: [['Homer', 'Marge', 'Maggie'], ['Bart', 'Lisa']]

a[0] is b[0]: True
```

This is the common shallow-copy surprise. The outer container is separate, but the nested mutable objects are still shared.

The structure is:

```
a ---> outer list A
      slot 0 ----+
                |
                v
            inner list ["Homer", "Marge", "Maggie"]
                ^
                |
b ---> outer list B |
      slot 0 ----+
```

Replacing an inner list is different from mutating the shared inner list.

```
a = [
    ["Homer", "Marge"],
    ["Bart", "Lisa"],
]

b = a.copy()

b[0] = ["Ned", "Maude"]
```

```

print(f"a: {a!r}")
print(f"b: {b!r}")

print()
print(f"a[0] is b[0]: {a[0] is b[0]}")
print(f"a[1] is b[1]: {a[1] is b[1]}")

```

Possible output:

```

a: [['Homer', 'Marge'], ['Bart', 'Lisa']]
b: [['Ned', 'Maude'], ['Bart', 'Lisa']]

a[0] is b[0]: False
a[1] is b[1]: True

```

The first inner-list slot in `b` was redirected to a new list. The second inner list is still shared. This is the key distinction:

- replacing a slot changes one outer list;
- mutating a shared inner object is visible through all references to that object.

A repeated-list construction can create the same problem even without copying.

```

row = [0, 0, 0]

grid = [row] * 3

print("before")
print(grid)

grid[0][1] = 42

print()
print("after")
print(grid)

print()
for index, item in enumerate(grid):
    print(f"row {index}: id={id(item)} value={item}")

```

Possible output:

```

before
[[0, 0, 0], [0, 0, 0], [0, 0, 0]]

after
[[0, 42, 0], [0, 42, 0], [0, 42, 0]]

row 0: id=140619786871232 value=[0, 42, 0]
row 1: id=140619786871232 value=[0, 42, 0]
row 2: id=140619786871232 value=[0, 42, 0]

```

The outer list contains three references to the same row object.

A safer construction creates a new row each time.


```

grid = []

for _ in range(3):
    grid.append([0, 0, 0])

grid[0][1] = 42

print(grid)

print()
for index, item in enumerate(grid):
    print(f"row {index}: id={id(item)} value={item}")

```

Possible output:

```

[[0, 42, 0], [0, 0, 0], [0, 0, 0]]

row 0: id=140619786871232 value=[0, 42, 0]
row 1: id=140619786871296 value=[0, 0, 0]
row 2: id=140619786871360 value=[0, 0, 0]

```

Each row is now a separate list object.
The practical rule is:

A shallow copy is enough for a flat list whose elements will not be mutated in place. It is not enough when the list contains mutable nested structures that must become independent.

Deep Copying with `copy.deepcopy()`

The standard library module `copy` provides `deepcopy()`. A deep copy recursively copies the outer object and the objects contained inside it.

```

import copy

a = [
    ["Homer", "Marge"],
    ["Bart", "Lisa"],
]

b = copy.deepcopy(a)

print("before mutation")
print(f"a: {a!r}")
print(f"b: {b!r}")

print()
print(f"a is b: {a is b}")
print(f"a[0] is b[0]: {a[0] is b[0]}")
print(f"a[1] is b[1]: {a[1] is b[1]}")

```

Possible output:

```

before mutation
a: [['Homer', 'Marge'], ['Bart', 'Lisa']]
b: [['Homer', 'Marge'], ['Bart', 'Lisa']]

```

```
a is b: False
a[0] is b[0]: False
a[1] is b[1]: False
```

The outer list is different, and the inner lists are also different.

Now an inner mutation remains local to the copied structure.

```
import copy

a = [
    ["Homer", "Marge"],
    ["Bart", "Lisa"],
]

b = copy.deepcopy(a)

b[0].append("Maggie")

print(f"a: {a!r}")
print(f"b: {b!r}")

print()
print(f"a[0] is b[0]: {a[0] is b[0]}")
```

Possible output:

```
a: [['Homer', 'Marge'], ['Bart', 'Lisa']]
b: [['Homer', 'Marge', 'Maggie'], ['Bart', 'Lisa']]

a[0] is b[0]: False
```

This is the behavior wanted when nested mutable structures must be independent.

The difference between assignment, shallow copy, and deep copy can be shown together.

```
import copy

a = ["spam", ["eggs"]]

b = a
c = a.copy()
d = copy.deepcopy(a)

a[0].append(42)

print(f"a: {a!r}")
print(f"b: {b!r}")
print(f"c: {c!r}")
print(f"d: {d!r}")

print()
print(f"b is a:      {b is a}")
print(f"c is a:      {c is a}")
print(f"d is a:      {d is a}")
```

```
print()
print(f"c[0] is a[0]: {c[0] is a[0]}")
print(f"d[0] is a[0]: {d[0] is a[0]}")
```

Possible output:

```
a: [['spam', 42], ['eggs']]
b: [['spam', 42], ['eggs']]
c: [['spam', 42], ['eggs']]
d: [['spam'], ['eggs']]
```

```
b is a:      True
c is a:      False
d is a:      False
```

```
c[0] is a[0]: True
d[0] is a[0]: False
```

The result is:

- `b = a`: same outer list;
- `c = a.copy()`: different outer list, shared inner lists;
- `d = copy.deepcopy(a)`: different outer list, different inner lists.

Deep copying should not be used blindly. It may be unnecessary for flat structures, and it can be expensive for large nested object graphs. But when independent nested mutable structures are required, it is the standard tool.

The practical rule is:

Use assignment when another name should refer to the same list. Use a shallow copy when only the outer list must be independent. Use `copy.deepcopy()` when nested mutable objects must also be independent.

5.5 Fixed Structural Contiguity: The Tuple Architecture (tuple)

A tuple is Python's fixed-size sequential structure. It keeps the positional model of a list, but removes the ability to change the sequence after construction. Once a tuple exists, its length and element positions are fixed.

This makes tuples useful for small, stable groups of values: coordinates, records, pairs, return values, and other structures where the number of components is part of the meaning. A tuple should not be read as a weaker list. It has a different role: it represents a sequence whose structure is not meant to be edited.

This section explains tuple syntax, packing and unpacking, single-element tuple construction, structural immutability, the difference between tuple immutability and mutability of referenced objects, and the CPython memory consequences of using a fixed-size reference sequence.

5.5.1 Tuple Syntax and Structural Role

A tuple is an ordered sequence, like a list, but its structure is fixed after creation. The tuple itself cannot grow, shrink, or have its positions reassigned. For that reason, tuples are commonly used for small fixed groups of related values.

A tuple is not defined mainly by parentheses. It is defined mainly by commas.

```
point = (10, 20)
person = "Kramer", "Cosmo", 42
empty = ()

print(f"point: {point!r}")
print(f"person: {person!r}")
print(f"empty: {empty!r}")

print()
print(f"type(point): {type(point)}")
print(f"type(person): {type(person)}")
print(f"type(empty): {type(empty)}")
```

Possible output:

```
point: (10, 20)
person: ('Kramer', 'Cosmo', 42)
empty: ()

type(point): <class 'tuple'>
type(person): <class 'tuple'>
type(empty): <class 'tuple'>
```

Tuples support the ordinary sequence operations already introduced: indexing, slicing, length testing, membership testing, and iteration.

```
data = ("spam", "eggs", 42, "larch")

print(f"data: {data!r}")
print(f"len(data): {len(data)}")

print()
print(f"data[0]: {data[0]!r}")
print(f"data[-1]: {data[-1]!r}")
print(f"data[1:3]: {data[1:3]!r}")

print()
print(f"{42 in data = }")
print(f"{'beans' in data = }")
```

Possible output:

```
data: ('spam', 'eggs', 42, 'larch')
len(data): 4

data[0]: 'spam'
data[-1]: 'larch'
data[1:3]: ('eggs', 42)

42 in data = True
'beans' in data = False
```

A quick `dir()` inspection shows that tuples have sequence behavior, but not list mutation methods.

```

data = ("Jerry", "Elaine", "George")

selected_names = [
    "__len__",
    "__getitem__",
    "__contains__",
    "count",
    "index",
    "append",
    "extend",
    "remove",
    "sort",
]

print(f"type(data): {type(data)}")
print()

for name in selected_names:
    print(f"{name:<14} {name in dir(data)}")

```

Possible output:

```

type(data): <class 'tuple'>

__len__      True
__getitem__  True
__contains__ True
count        True
index        True
append       False
extend       False
remove       False
sort         False

```

The methods `count()` and `index()` are available because they only inspect the sequence. Methods such as `append()`, `remove()`, and `sort()` are absent because they would require changing the tuple structure.

Tuple Packing: Comma-Based Construction with or Without Parentheses

Tuple packing means collecting several expressions into one tuple object.

```

data = "Homer", "Marge", "Bart"

print(f"data: {data!r}")
print(f"type(data): {type(data)}")

```

Possible output:

```

data: ('Homer', 'Marge', 'Bart')
type(data): <class 'tuple'>

```

The commas created the tuple. Parentheses are often written because they make the grouping clearer, but they are not always required.

```

a = "spam", "eggs", 42
b = ("spam", "eggs", 42)

print(f"a: {a!r}")
print(f"b: {b!r}")

print()
print(f"type(a): {type(a)}")
print(f"type(b): {type(b)}")
print(f"a == b: {a == b}")

```

Possible output:

```

a: ('spam', 'eggs', 42)
b: ('spam', 'eggs', 42)

type(a): <class 'tuple'>
type(b): <class 'tuple'>
a == b: True

```

Parentheses are still required or strongly useful in some syntactic positions, especially when the tuple must appear as one grouped expression.

```

items = [("Homer", 39), ("Marge", 36), ("Bart", 10)]

print(items)

print()
for item in items:
    print(f"item={item!r} type={type(item)}")

```

Possible output:

```

[('Homer', 39), ('Marge', 36), ('Bart', 10)]

item=('Homer', 39) type=<class 'tuple'>
item=('Marge', 36) type=<class 'tuple'>
item=('Bart', 10) type=<class 'tuple'>

```

Without parentheses, the list literal would not have the same structure.

```

items = ["Homer", 39, "Marge", 36]

print(items)

```

Possible output:

```

['Homer', 39, 'Marge', 36]

```

That is a list with four elements, not a list containing two fixed two-element tuples.

Tuple packing is also commonly used to return or pass small records of values.

```

name = "Elaine"
score = 42
active = True

```

```
record = name, score, active

print(f"record: {record!r}")
print(f"type(record): {type(record)}")
```

Possible output:

```
record: ('Elaine', 42, True)
type(record): <class 'tuple'>
```

The practical rule is:

Commas create tuples. Parentheses usually clarify grouping, but the comma is the essential tuple-building signal.

Tuple Unpacking: Decomposing Fixed-Length Structures into Multiple Names

Tuple unpacking takes the components of a tuple and assigns them to separate names.

```
record = ("Jerry", "comedian", 42)

name, job, score = record

print(f"name: {name!r}")
print(f"job: {job!r}")
print(f"score: {score!r}")
```

Possible output:

```
name: 'Jerry'
job: 'comedian'
score: 42
```

The number of names on the left must match the number of elements being unpacked, unless extended unpacking is used.

```
record = ("Homer", "Marge", "Bart")

try:
    first, second = record
except ValueError as err:
    print("unpacking failed")
    print(f"message: {err}")
```

Possible output:

```
unpacking failed
message: too many values to unpack (expected 2)
```

Too many target names also fails.

```
record = ("Homer", "Marge")

try:
    first, second, third = record
except ValueError as err:
    print("unpacking failed")
    print(f"message: {err}")
```

Possible output:

```
unpacking failed
message: not enough values to unpack (expected 3, got 2)
```

Unpacking is not limited to tuple literals. It works with other iterable objects too, but tuples are the clearest fixed-structure case.

```
point = (10, 20)

x, y = point

print(f"x: {x}")
print(f"y: {y}")
```

Possible output:

```
x: 10
y: 20
```

The same idea is useful when iterating through a list of tuples.

```
people = [
    ("Jerry", 38),
    ("Elaine", 36),
    ("George", 37),
    ("Kramer", 42),
]

for name, age in people:
    print(f"{name:<8} age={age}")
```

Possible output:

```
Jerry    age=38
Elaine   age=36
George   age=37
Kramer   age=42
```

Each loop step receives one tuple. The tuple is then unpacked into `name` and `age`.

Tuple unpacking also makes value swapping clear.

```
a = "spam"
b = "eggs"

print("before")
print(f"a={a!r} b={b!r}")

a, b = b, a

print()
print("after")
print(f"a={a!r} b={b!r}")
```

Possible output:


```
before
a='spam' b='eggs'
```

```
after
a='eggs' b='spam'
```

The right side creates a packed group of values. The left side unpacks that group into names. Extended unpacking uses `*` to collect remaining values into a list.

```
data = ("Nobody", "expects", "the", "Spanish", "Inquisition")

first, second, *rest = data

print(f"first: {first!r}")
print(f"second: {second!r}")
print(f"rest: {rest!r}")
print(f"type(rest): {type(rest)}")
```

Possible output:

```
first: 'Nobody'
second: 'expects'
rest: ['the', 'Spanish', 'Inquisition']
type(rest): <class 'list'>
```

The starred target receives a list, not a tuple.

The practical rule is:

Tuple packing combines values into one fixed ordered structure. Tuple unpacking distributes the components of such a structure into separate names.

Single-Element Tuple Syntax: Why `(x,)` Is a Tuple but `(x)` Is Not

A one-element tuple needs a comma.

```
a = (42)
b = (42,)

print(f"a: {a!r}")
print(f"b: {b!r}")

print()
print(f"type(a): {type(a)}")
print(f"type(b): {type(b)}")
```

Possible output:

```
a: 42
b: (42,)

type(a): <class 'int'>
type(b): <class 'tuple'>
```

The expression `(42)` is just the integer `42` grouped by parentheses. The expression `(42,)` is a tuple containing one element.

The same rule applies to strings.

```

a = ("Ni")
b = ("Ni",)

print(f"a: {a!r}")
print(f"b: {b!r}")

print()
print(f"type(a): {type(a)}")
print(f"type(b): {type(b)}")

```

Possible output:

```

a: 'Ni'
b: ('Ni',)

type(a): <class 'str'>
type(b): <class 'tuple'>

```

The comma is still the tuple marker.

A single-element tuple can also be written without parentheses, but this is usually less readable.

```

a = 42,
b = (42,)

print(f"a: {a!r}")
print(f"b: {b!r}")
print(f"a == b: {a == b}")

```

Possible output:

```

a: (42,)
b: (42,)
a == b: True

```

The trailing comma created the tuple.

This matters when a function or operation expects a tuple. The difference between a value and a one-element tuple is structural.

```

value = "spam"
one_item_tuple = ("spam",)

print(f"value:           {value!r}")
print(f"one_item_tuple: {one_item_tuple!r}")

print()
print(f"len(value):           {len(value)}")
print(f"len(one_item_tuple): {len(one_item_tuple)}")

```

Possible output:

```

value:           'spam'
one_item_tuple: ('spam',)

len(value):           4
len(one_item_tuple): 1

```

The string `"spam"` is a sequence of four one-character strings. The tuple `("spam",)` is a sequence with one element, and that one element is the whole string `"spam"`.

The practical rule is:

Parentheses group expressions. Commas build tuples. Therefore `(x)` is just `x`, while `(x,)` is a one-element tuple.

5.5.2 Immutable Sequence Invariants

A tuple is an immutable sequence. This means that the tuple's structure cannot be changed after the tuple object has been created. Its length is fixed, its positions are fixed, and its slots cannot be reassigned.

This does not mean that every object referenced by the tuple is immutable. The tuple is structurally immutable. The referenced objects keep their own type and their own mutability rules.

```
data = ("spam", "eggs", 42)

print(f"data: {data!r}")
print(f"type(data): {type(data)}")
print(f"len(data): {len(data)}")

print()
for index, item in enumerate(data):
    print(f"index={index} value={item!r:<8} type={type(item)} id={id(item)}")
```

Possible output:

```
data: ('spam', 'eggs', 42)
type(data): <class 'tuple'>
len(data): 3

index=0 value='spam'    type=<class 'str'> id=140619786560368
index=1 value='eggs'    type=<class 'str'> id=140619786560432
index=2 value=42        type=<class 'int'> id=140619796012304
```

A tuple is therefore similar to a list in one internal sense: it stores references to Python objects. The crucial difference is that the tuple's reference slots are fixed after construction.

```
data = ("Jerry", "Elaine", "George")

selected_names = [
    "__len__",
    "__getitem__",
    "__contains__",
    "count",
    "index",
    "append",
    "extend",
    "remove",
    "pop",
    "sort",
]

for name in selected_names:
    print(f"{name:<14} {name in dir(data)}")
```

Possible output:

```
__len__      True
__getitem__  True
__contains__ True
count        True
index        True
append       False
extend       False
remove       False
pop          False
sort         False
```

The tuple supports inspection operations. It does not expose list-style mutation operations.

Structural Definition: Fixed-Size, Read-Only Sequential Storage of `PyObject*` Addresses

At the CPython implementation level, a tuple can be understood as a fixed-size sequence of object-reference slots. Each slot stores a `PyObject*`, meaning a pointer to a Python object.

A simplified picture is:

```
tuple object
+-----+
| length                |
| fixed slot array      |
+-----+

fixed slot array
+-----+-----+-----+
| PyObject* | PyObject* | PyObject* |
+-----+-----+-----+
|           |           |           |
|   v       |   v       |   v       |
| "spam"    | "eggs"    | 42        |
```

The tuple owns the sequence of slots. It does not store raw primitive values directly in those slots. The slots contain references.

This is why a tuple can contain heterogeneous objects.

```
data = ("Homer", 39, 3.14, True)

print(f"data: {data!r}")

print()
for index, item in enumerate(data):
    print(f"index={index} value={item!r:<8} type={type(item)}")
```

Possible output:

```
data: ('Homer', 39, 3.14, True)

index=0 value='Homer'  type=<class 'str'>
index=1 value=39       type=<class 'int'>
index=2 value=3.14     type=<class 'float'>
index=3 value=True     type=<class 'bool'>
```

The tuple's slots all store references. The referenced objects can have different types.
The fixed-size rule appears when trying to replace an element.

```
data = ("Homer", "Marge", "Bart")
```

```
try:
    data[0] = "Lisa"
except TypeError as err:
    print("assignment failed")
    print(f"message: {err}")
```

Possible output:

```
assignment failed
message: 'tuple' object does not support item assignment
```

The tuple refuses slot reassignment.
A tuple also cannot grow or shrink.

```
data = ("Homer", "Marge")
```

```
try:
    data.append("Bart")
except AttributeError as err:
    print("append failed")
    print(f"message: {err}")
```

Possible output:

```
append failed
message: 'tuple' object has no attribute 'append'
```

The operation is not merely forbidden at runtime. The tuple type does not even provide an `append()` method.

Concatenation creates a new tuple. It does not extend the old tuple.

```
a = ("Homer", "Marge")
old_id = id(a)

b = a + ("Bart",)

print(f"a: {a!r}")
print(f"b: {b!r}")

print()
print(f"id(a): {id(a)}")
print(f"old_id == id(a): {old_id == id(a)}")
print(f"id(b): {id(b)}")
```

Possible output:

```
a: ('Homer', 'Marge')
b: ('Homer', 'Marge', 'Bart')

id(a): 140619786871232
old_id == id(a): True
id(b): 140619786871488
```

The original tuple `a` did not change. The expression `a + ("Bart",)` created another tuple object. The same applies to repetition.

```
data = ("Ni!",)

result = data * 3

print(f"data:    {data!r}")
print(f"result: {result!r}")

print()
print(f"data is result: {data is result}")
```

Possible output:

```
data:    ('Ni!',)
result: ('Ni!', 'Ni!', 'Ni!')

data is result: False
```

The repeated tuple is a new tuple. The original tuple remains fixed. The practical rule is:

A tuple is a fixed-size sequence of reference slots. The slots can be read, but they cannot be reassigned, inserted, deleted, appended to, or sorted in place.

The Concept of Transitive Mutability: Why a Tuple Is Structurally Unalterable, Yet May Reference Internally Mutable Objects

Tuple immutability applies to the tuple's own structure. It does not automatically freeze every object referenced by the tuple.

A tuple can contain a list.

```
data = ("Homer", ["Bart", "Lisa"])

print(f"data: {data!r}")
print(f"type(data): {type(data)}")
print(f"type(data[1]): {type(data[1])}")
```

Possible output:

```
data: ('Homer', ['Bart', 'Lisa'])
type(data): <class 'tuple'>
type(data[1]): <class 'list'>
```

The tuple cannot replace its second slot.

```
data = ("Homer", ["Bart", "Lisa"])

try:
    data[1] = ["Maggie"]
except TypeError as err:
    print("tuple slot replacement failed")
    print(f"message: {err}")
```

Possible output:

```
tuple slot replacement failed
message: 'tuple' object does not support item assignment
```

But the list referenced by the second slot can still mutate.

```
data = ("Homer", ["Bart", "Lisa"])

print("before")
print(f"data: {data!r}")
print(f"id(data):    {id(data)}")
print(f"id(data[1]): {id(data[1])}")

data[1].append("Maggie")

print()
print("after")
print(f"data: {data!r}")
print(f"id(data):    {id(data)}")
print(f"id(data[1]): {id(data[1])}")
```

Possible output:

```
before
data: ('Homer', ['Bart', 'Lisa'])
id(data):    140619786871360
id(data[1]): 140619786870656

after
data: ('Homer', ['Bart', 'Lisa', 'Maggie'])
id(data):    140619786871360
id(data[1]): 140619786870656
```

The tuple object did not change its slots. The second slot still points to the same list object. That list object changed internally.

The structure is:

```
tuple
+-----+-----+
| PyObject* | PyObject* |
+-----+-----+
      |           |
      v           v
"Homer"    list object
           ["Bart", "Lisa", "Maggie"]
```

The tuple slot still points to the same list. The list's own contents changed.

This is called transitive mutability in practice: an immutable container can reference mutable objects. The container is fixed, but mutation may still happen inside objects reached through it.

A more direct example:

```
inner = ["spam"]

data = (inner, inner)

print("before")
```

```

print(f"data: {data!r}")
print(f"data[0] is data[1]: {data[0] is data[1]}")

inner.append(42)

print()
print("after")
print(f"data: {data!r}")
print(f"data[0] is data[1]: {data[0] is data[1]}")

```

Possible output:

```

before
data: (['spam'], ['spam'])
data[0] is data[1]: True

after
data: (['spam', 42], ['spam', 42])
data[0] is data[1]: True

```

The tuple contains two references to the same list object. Mutating that one list is visible through both tuple positions.

This also affects hashability. A tuple can be used as a dictionary key only if all its components are hashable.

```

good_key = ("Homer", 42)
bad_key = ("Homer", [42])

print(f"hash(good_key): {hash(good_key)}")

try:
    print(hash(bad_key))
except TypeError as err:
    print("hash failed")
    print(f"message: {err}")

```

Possible output:

```

hash(good_key): -4189771609282103040
hash failed
message: unhashable type: 'list'

```

The exact hash value can differ between runs. The important point is that a tuple containing a list is not hashable, because the list is mutable and unhashable.

The practical rule is:

Tuple immutability is structural, not magical. The tuple cannot change which objects its slots reference, but mutable objects referenced by those slots can still change themselves.

5.5.3 CPython Allocation Optimizations

A tuple is structurally simpler than a list. It has a fixed length, so CPython does not need to maintain spare capacity for future appends. Once the tuple exists, its slot count is fixed.

This gives tuples two practical implementation advantages:

- the tuple object can be allocated with exactly the needed number of reference slots;

- CPython can apply allocation shortcuts for some tuple cases.

These are CPython implementation details, not general Python-language promises. Other Python implementations may use different internal layouts.

```
import platform
import sys

data = ("spam", "eggs", 42)

print(f"implementation: {platform.python_implementation()}")
print(f"python version: {sys.version.split()[0]}")

print()
print(f"data: {data!r}")
print(f"type(data): {type(data)}")
print(f"len(data): {len(data)}")
print(f"size: {sys.getsizeof(data)} bytes")
```

Possible output on one 64-bit CPython build:

```
implementation: CPython
python version: 3.12.3

data: ('spam', 'eggs', 42)
type(data): <class 'tuple'>
len(data): 3
size: 64 bytes
```

The exact byte count can differ between Python versions, builds, and platforms. The important structural point is that a tuple does not carry list-style spare capacity for later growth.

CPython Allocation Caching: Version-Dependent Recycling Optimizations for Small Tuple Objects

CPython has historically used special allocation behavior for tuples. One simple case is the empty tuple. CPython reuses the empty tuple object.

```
a = ()
b = ()
c = tuple()

print(f"a: {a!r}")
print(f"b: {b!r}")
print(f"c: {c!r}")

print()
print(f"a is b: {a is b}")
print(f"a is c: {a is c}")

print()
print(f"id(a): {id(a)}")
print(f"id(b): {id(b)}")
print(f"id(c): {id(c)}")
```

Possible output on CPython:

```
a: ()
b: ()
c: ()

a is b: True
a is c: True

id(a): 140619796135544
id(b): 140619796135544
id(c): 140619796135544
```

This is safe because the empty tuple cannot be changed. There is no internal slot that could later be reassigned.

CPython also performs recycling optimizations for some non-empty tuple allocations. The exact rules are implementation details and may change between versions. Therefore, tuple identity reuse must never be used as program logic.

The following demonstration may show object-id reuse on CPython, but it is intentionally presented as an observation experiment, not as a guarantee.

```
import gc

ids = []

for nr in range(10):
    data = (nr, nr + 1)
    ids.append(id(data))
    del data
    gc.collect()

print("observed ids")
for value in ids:
    print(value)

print()
print(f"unique id count: {len(set(ids))}")
print(f"total creations: {len(ids)}")
```

Possible output on one CPython run:

```
observed ids
140619786871232
140619786871232
140619786871232
140619786871232
140619786871232
140619786871232
140619786871232
140619786871232
140619786871232
140619786871232

unique id count: 1
total creations: 10
```

Another run, another Python version, or another implementation may show different results. The only valid lesson is that CPython may recycle tuple memory after a tuple is destroyed.

The same warning applies to identity comparisons.

```
a = (10, 20)
b = (10, 20)

print(f"a == b: {a == b}")
print(f"a is b: {a is b}")

print()
print(f"id(a): {id(a)}")
print(f"id(b): {id(b)}")
```

Possible output:

```
a == b: True
a is b: False

id(a): 140619786871232
id(b): 140619786871488
```

The tuples have equal contents. They should be compared with `==`, not `is`.

Some constant tuples may appear to be shared because the compiler can store constants and reuse them inside code objects. This is another implementation detail.

```
a = (1, 2, 3)
b = (1, 2, 3)

print(f"a == b: {a == b}")
print(f"a is b: {a is b}")

print()
print(f"id(a): {id(a)}")
print(f"id(b): {id(b)}")
```

Possible output on one CPython run:

```
a == b: True
a is b: True

id(a): 140619786873024
id(b): 140619786873024
```

This result is not the tuple equality rule. It is a storage optimization effect. The semantic rule is still:

Use `==` to compare tuple contents. Do not use `is` to test whether two tuples contain the same values.

Tuple construction from an existing exact tuple may also return the same object, because there is no reason to copy an immutable tuple.

```

a = ("Homer", "Marge")
b = tuple(a)

print(f"a: {a!r}")
print(f"b: {b!r}")

print()
print(f"a == b: {a == b}")
print(f"a is b: {a is b}")

```

Possible output:

```

a: ('Homer', 'Marge')
b: ('Homer', 'Marge')

a == b: True
a is b: True

```

This is safe because neither `a` nor `b` can mutate the tuple structure.
The practical rule is:

Tuple allocation optimizations are allowed because tuples are structurally immutable. They are useful for CPython performance, but they must not be treated as part of normal program meaning.

Memory Footprint Metrics: Comparing Fixed tuple Layouts Against the Dynamic Tracking Buffers of list

A list must support growth. For that reason, CPython lists maintain dynamic internal storage and may reserve extra pointer slots beyond the visible length.

A tuple does not support growth. Its fixed slot count can be allocated directly.

The difference can be measured with `sys.getsizeof()`.

```

import sys

values = [10, 20, 30, 42]

items_list = [10, 20, 30, 42]
items_tuple = (10, 20, 30, 42)

print(f"items_list: {items_list!r}")
print(f"items_tuple: {items_tuple!r}")

print()
print(f"len(items_list): {len(items_list)}")
print(f"len(items_tuple): {len(items_tuple)}")

print()
print(f"sys.getsizeof(items_list): {sys.getsizeof(items_list)}")
print(f"sys.getsizeof(items_tuple): {sys.getsizeof(items_tuple)}")

```

Possible output on one 64-bit CPython build:

```

items_list: [10, 20, 30, 42]
items_tuple: (10, 20, 30, 42)

```

```
len(items_list): 4
len(items_tuple): 4

sys.getsizeof(items_list): 88
sys.getsizeof(items_tuple): 72
```

The exact numbers are not universal. The usual pattern is that a tuple with the same number of elements is smaller than a list because the tuple does not need dynamic spare capacity.

A size table makes the pattern clearer.

```
import sys

print(f"{'n':>2} {'list':>8} {'tuple':>8}")

for n in range(0, 11):
    items_list = list(range(n))
    items_tuple = tuple(range(n))

    list_size = sys.getsizeof(items_list)
    tuple_size = sys.getsizeof(items_tuple)

    print(f"{'n':>2} {'list_size':>8} {'tuple_size':>8}")
```

Possible output on one 64-bit CPython build:

n	list	tuple
0	56	40
1	72	48
2	72	56
3	88	64
4	88	72
5	104	80
6	104	88
7	120	96
8	120	104
9	136	112
10	136	120

The tuple size usually grows in a direct slot-count pattern. The list size often grows in jumps because of allocation rounding and spare capacity.

This difference becomes more visible when a list is grown gradually with `append()`.

```
import sys

items = []

print(f"{'len':>3} {'list size':>10}")

last_size = sys.getsizeof(items)
print(f"{'len(items)':>3} {'last_size':>10}")

for nr in range(1, 13):
    items.append(nr)
    size = sys.getsizeof(items)
```

```

if size != last_size:
    print(f"{len(items):>3} {size:>10}")
    last_size = size

```

Possible output on one 64-bit CPython build:

```

len  list  size
0      56
1      88
5     120
9     184

```

The list size does not increase at every length. It increases when the internal storage is resized. A tuple of the same visible lengths does not need this spare-capacity behavior.

```

import sys

print(f"{'len':>3} {'tuple size':>10}")

for n in [0, 1, 5, 9]:
    data = tuple(range(n))
    print(f"{n:>3} {sys.getsizeof(data):>10}")

```

Possible output on one 64-bit CPython build:

```

len tuple size
0      40
1      48
5      80
9     112

```

This does not mean tuples are always the correct replacement for lists. The choice depends on the required behavior.

- Use a list when the sequence must be edited in place.
- Use a tuple when the sequence represents a fixed structure.

The memory comparison is only one consequence of the deeper structural difference.

The following example shows the behavioral reason first.

```

items_list = ["Jerry", "Elaine"]
items_tuple = ("Jerry", "Elaine")

items_list.append("George")

print(f"items_list: {items_list!r}")

try:
    items_tuple.append("George")
except AttributeError as err:
    print("tuple append failed")
    print(f"message: {err}")

```

Possible output:

```
items_list: ['Jerry', 'Elaine', 'George']  
tuple.append failed  
message: 'tuple' object has no attribute 'append'
```

The list pays for editability with dynamic storage machinery. The tuple avoids that machinery because its structure is fixed.

The practical rule is:

A tuple is usually smaller because it stores a fixed number of reference slots. A list is more flexible because it owns a dynamically resized slot buffer. Choose between them by meaning first: editable sequence or fixed record-like sequence.

Chapter 6

Control Flow Graphs, Lazy Traversal, and Exception Routing

A Python program is written as linear source text, but it does not execute as a simple uninterrupted list of lines. Conditions create alternative routes, loops create repeated routes, iterator-based traversal creates value-producing routes, and exceptions create non-local routes that can jump out of the current block or function. To understand Python control flow properly, the program must be read as an execution graph, not merely as text.

This chapter builds that graph-based view step by step. It begins with the structural rules that define blocks and indentation, then moves into conditional decision trees, loop mechanics, lazy traversal objects, and the iterator protocol. The goal is to show that Python's high-level syntax is supported by precise runtime mechanisms: truth-value testing, iterator production, iterator exhaustion, short-circuit evaluation, and exception propagation.

For a programmer coming from C, this chapter is especially important because Python moves several familiar responsibilities into different mechanisms. Counting loops are replaced by iterable traversal. Manual error-code routing is often replaced by exceptions. Explicit block delimiters are replaced by syntactic indentation. These are not cosmetic differences. They change how execution paths are formed, interrupted, resumed, and completed.

6.1 From Linear Source Text to Control Flow Graphs

6.1.1 Statements, Basic Blocks, and Execution Edges

A Python source file is written as a vertical sequence of statements. At first sight, this makes a program look like a simple list: first line, second line, third line, and so on. That view is useful only for the simplest programs. As soon as the program contains a condition, a loop, or an exception, the execution path no longer follows the written text in a single uninterrupted direction.

A more accurate model is a control flow graph. In this model, the program is divided into pieces of code that can execute straight through without an internal jump. Such a piece is usually called a basic block. Between these blocks there are execution edges. An edge means: after this block finishes, execution may continue in that other block.

This does not mean that the beginner must draw a graph for every program. The point is mental, not decorative. A program is not merely text. It is a set of possible routes through text.

Source code is linear as written. Execution is graph-shaped as soon as the program contains alternatives, repetition, or failure paths.

This matters especially for programmers coming from C, because the same deep idea exists there too. In C, `if`, `while`, `for`, `return`, `break`, `continue`, and `goto` all alter the possible route of execution. Python has different surface syntax, but the control problem is the same: the runtime must know which statement may execute next.

The following very small program is written as a sequence:


```
name = "George"
score = 42

print(f"name: {name}")
print(f"score: {score}")
print("These pretzels are making me thirsty.")
```

Possible output:

```
name: George
score: 42
These pretzels are making me thirsty.
```

Here, the control flow is still simple. Each statement falls into the next one. There is only one ordinary route:

```
statement 1 -> statement 2 -> statement 3 -> statement 4 -> end
```

Once branching, looping, and exceptions appear, that simple route becomes only one special case of the larger execution model.

Sequential Flow: The Default Fall-Through Path from One Statement to the Next

Sequential flow is the default case. If nothing interrupts execution, Python executes one statement and then continues with the next statement in the same block.

This ordinary movement from one statement to the next is often called fall-through. The name is useful because it describes the simplest edge in the control flow graph: execution falls from the current statement into the next statement below it.

```
quote = "Nobody expects the Spanish Inquisition!"
count = 3
result = count + 39

print(f"quote: {quote}")
print(f"count: {count}")
print(f"result: {result}")
```

Possible output:

```
quote: Nobody expects the Spanish Inquisition!
count: 3
result: 42
```

The route is direct:

```
quote assignment
-> count assignment
-> result assignment
-> first print
-> second print
-> third print
```

There is no condition to test, no loop target to revisit, and no handler to enter. Every statement is reached exactly once, in written order.

This is the simplest kind of basic block: a straight-line segment of code.

A basic block is a piece of code whose internal statements execute in sequence, without an internal branch target or jump out of the middle.

In real Python programs, a basic block may be very short. Sometimes it is only one statement. This is normal. The important property is not size. The important property is that once execution enters the block, ordinary execution proceeds through it in order until the block ends or an interrupting control transfer occurs.

Consider this block inside a future `if` statement:

```
print("checking soup status")
soup_available = False
print(f"soup_available: {soup_available}")
```

Those three statements form a straight route. If execution enters the block, the ordinary path runs through them from top to bottom.

Sequential flow is therefore the foundation. Branches, loops, and exceptions do not replace sequential flow. They connect sequential blocks in more complex ways.

Conditional Edges: Branching Execution Paths Created by `if`, `elif`, and `else`

A conditional statement creates multiple possible outgoing edges from a decision point. Python evaluates a condition, then chooses one continuation path.

In ordinary written form, the code is still vertical:

```
score = 42

if score > 90:
    print("excellent")
elif score >= 50:
    print("acceptable")
else:
    print("needs work")

print("grading finished")
```

Possible output:

```
needs work
grading finished
```

The written order is not the same thing as the execution route. Python does not execute every branch body. It tests the first condition. If that condition is false, it tests the next condition. If all previous tested conditions fail, it enters the `else` body when one exists.

The control shape is closer to this:

```
start
-> test score > 90
    true  -> print "excellent"   -> after if
    false -> test score >= 50
                true  -> print "acceptable" -> after if
                false -> print "needs work" -> after if
-> print "grading finished"
```

The `if`, `elif`, and `else` keywords therefore do not merely group text. They define possible execution edges.

A useful way to read the statement is:

At this point, execution must choose one of several possible routes. After the chosen route finishes, execution joins again after the conditional structure.

Only one branch body is selected in a normal `if/elif/else` chain. This is different from writing separate `if` statements.

```
value = 42

if value > 0:
    print("positive")

if value % 2 == 0:
    print("even")

if value == 42:
    print("the answer")
```

Possible output:

```
positive
even
the answer
```

Here, there are three separate decision points. All three tests are evaluated, because each `if` statement starts a new conditional structure.

By contrast:

```
value = 42

if value < 0:
    print("negative")
elif value % 2 == 0:
    print("even")
elif value == 42:
    print("the answer")
else:
    print("something else")
```

Possible output:

```
even
```

The third condition is not reached, even though `value == 42` is true. The earlier `elif` branch already captured the route.

This distinction is easier to understand with the graph model. Separate `if` statements create separate decision nodes. An `if/elif/else` chain creates one connected decision structure where the first successful branch consumes the path.

Loop Back-Edges: Repeated Execution Paths Created by `while` and `for`

A loop creates a route that can return to an earlier decision point. This returning route is the essential difference between a loop and a simple conditional branch.

A conditional statement chooses a path and then usually moves forward. A loop may choose a path, execute a block, and then jump back to test or retrieve the next value again.

A `while` loop repeats as long as its condition remains true:

```

n = 1

while n <= 3:
    print(f"n: {n}")
    n = n + 1

print("loop finished")

```

Possible output:

```

n: 1
n: 2
n: 3
loop finished

```

The graph shape is not a straight line:

```

start
-> set n = 1
-> test n <= 3
    true  -> print n
           -> increment n
           -> go back to test n <= 3
    false -> print "loop finished"

```

The edge from the end of the loop body back to the condition is the loop back-edge. Without this edge, there is no repetition.

A for loop has a similar graph shape, but the repeated decision is not written as an explicit boolean condition. Instead, Python asks an iterable object for successive values. Later sections will examine that mechanism through `iter()`, `next()`, and `StopIteration`. For now, the control-flow idea is enough.

```

names = ["Jerry", "Elaine", "Kramer"]

for name in names:
    print(f"current name: {name}")

print("all names processed")

```

Possible output:

```

current name: Jerry
current name: Elaine
current name: Kramer
all names processed

```

Its control shape can be read like this:

```

start
-> get traversal source
-> ask for next value
    value exists -> bind name
                  -> execute loop body
                  -> go back and ask for next value
    no value     -> continue after loop

```

This is why loops must be treated as graph structures, not merely repeated text. The loop body is written once, but the execution route may pass through that body many times.

The difference between `while` and `for` is therefore not that one loops and the other does not. Both loop. The difference is where the continuation decision comes from.

- A `while` loop repeats by rechecking a condition written by the programmer.
- A `for` loop repeats by asking a traversal object for the next value.

Both forms create a back-edge in the control flow graph.

Exceptional Edges: Non-Local Control Transfers Created by Exceptions

An exception creates a different kind of execution edge. It does not merely choose between ordinary nearby branches, and it does not simply loop back to an earlier point. It can leave the current expression, the current statement, the current block, and sometimes the current function call, searching for a handler.

This is why exceptions are called non-local control transfers. The next executed statement may not be the next statement in the same block.

Consider this simple program:

```
text = "forty-two"

print("before conversion")

number = int(text)

print(f"number: {number}")
print("after conversion")
```

Possible output:

```
before conversion
Traceback (most recent call last):
...
ValueError: invalid literal for int() with base 10: 'forty-two'
```

The assignment to `number` does not complete. The following two `print()` calls are not executed. The ordinary fall-through edge is abandoned because `int(text)` raises a `ValueError`.

With a handler, the exceptional route is caught:

```
text = "forty-two"

print("before conversion")

try:
    number = int(text)
    print(f"number: {number}")
except ValueError as err:
    print("conversion failed")
    print(f"error type: {type(err)}")
    print(f"error message: {err}")

print("after try statement")
```

Possible output:

```

before conversion
conversion failed
error type: <class 'ValueError'>
error message: invalid literal for int() with base 10: 'forty-two'
after try statement

```

The important fact is not only that an error occurred. The important control-flow fact is that execution jumped from inside the `try` block to the matching `except` block. The rest of the ordinary `try` block was skipped.

The graph shape is approximately:

```

start
-> print "before conversion"
-> enter try block
-> attempt int(text)
    success -> bind number
              -> print number
              -> after try statement
    ValueError -> matching except block
                  -> print failure information
                  -> after try statement

```

This is very different from the C habit of checking returned error codes after each operation. In C, a common pattern is to keep execution local and manually inspect the result:

```

/* C-style idea, not Python code */
result = parse_number(text);
if (result.failed) {
    handle_error(result);
}

```

Python can also use explicit checks, but its exception mechanism gives operations the ability to redirect execution when normal completion is impossible.

The object captured by `except ValueError as err` is itself a runtime object. It has a type, a textual representation, and methods inherited from the exception hierarchy.

```

text = "forty-two"

try:
    number = int(text)
except ValueError as err:
    print(f"type(err): {type(err)}")
    print(f"str(err): {str(err)}")
    print(f"args: {err.args}")
    print(f"has __traceback__: {'__traceback__' in dir(err)}")

```

Possible output:

```

type(err): <class 'ValueError'>
str(err): invalid literal for int() with base 10: 'forty-two'
args: ("invalid literal for int() with base 10: 'forty-two'",)
has __traceback__: True

```

The `args` attribute contains the construction arguments stored by the exception object. The `__traceback__` attribute connects the exception object to traceback information, which records where the failure passed through the active execution frames. Later sections will study this mechanism more carefully.

For now, the essential rule is:

An exception adds a hidden possible edge from the place where failure occurs to the nearest matching handler, or farther upward if no local handler exists.

This completes the first control-flow map:

- Sequential statements create fall-through edges.
- Conditional statements create branch edges.
- Loops create back-edges.
- Exceptions create non-local exceptional edges.

Once these four edge types are understood, Python source code can be read as more than text. It becomes a set of possible execution routes through runtime state.

6.2 Code Block Syntax and Indentation Semantics

Before Python can execute branches, loops, exception handlers, functions, or classes, it must first know which statements belong together. This is the purpose of block syntax. A block is not merely a group of lines that look related to the human reader. It is a structural unit that the tokenizer and parser must recognize before the program can become executable.

Python makes an unusually direct design choice here: indentation defines block membership. The left margin is therefore not decoration. It is part of the language's structural machinery. A line moved one indentation level to the left or right may no longer belong to the same control-flow region. In Python, visual alignment is executable information.

For programmers coming from C, C++, or Java, this requires a deliberate mental adjustment. Curly-brace languages separate formal block delimiters from human formatting. Python removes that separation. The visible shape of the program and the grammatical structure of the program are forced to match.

This section explains that mechanism from both the programmer's view and the tokenizer's view: why Python uses syntactic whitespace, how indentation replaces explicit block tokens, how invalid indentation becomes a syntax error, and why mixed tabs and spaces can make the block structure ambiguous.

6.2.1 The Philosophy of Syntactic Whitespace

In many programming languages, whitespace is mostly decorative. A C program may be written with careful indentation, ugly indentation, or almost no indentation, and the compiler still recognizes the structure from explicit tokens such as `{` and `}`. Python makes a different choice. In Python, indentation is not merely a visual aid. It is part of the syntax.

This design often feels strange to programmers trained in C, because C separates visual layout from formal block structure. Python deliberately refuses that separation. The visual structure and the executable structure are forced to agree.

In Python, indentation is not a comment made by the programmer to the reader. It is structural information consumed by the interpreter.

This choice connects directly to the previous control-flow discussion. An `if` statement, a `while` loop, a `for` loop, a `try` block, an `except` block, and a function body all need a way to mark which statements belong inside them. Python uses indentation for that boundary.

Python's Core Design Choice: Whitespace Semantics for Structural Block Definitions

A block is a group of statements controlled by another statement. In Python, the block starts after a header line ending in a colon. The indented statements below that header belong to the block.

```
mood = "confident"

if mood == "confident":
    print("Nobody expects the Spanish Inquisition!")
    print("This line belongs to the if block.")

print("This line is outside the if block.")
```

Possible output:

```
Nobody expects the Spanish Inquisition!
This line belongs to the if block.
This line is outside the if block.
```

The indentation is not optional. The two indented `print()` statements are controlled by the `if`. The final `print()` statement is not controlled by the `if`, because it returns to the previous indentation level.

The control-flow structure is therefore visible directly in the left margin:

```
if condition:
    inside block
    inside block
outside block
```

The colon announces that a nested block is expected. The increased indentation defines the body of that block. Returning to the earlier indentation level ends the block.

This means that Python code cannot lie about its structure in the same way badly formatted C code can. In C, this is legal but misleading:

```
/* C example, not Python */
if (ready)
    printf("ready\n");
    printf("still printed even when ready is false\n");
```

The second `printf()` visually looks connected to the `if`, but it is not controlled by the `if`. In Python, that ambiguity is removed because indentation itself defines the block.

```
ready = False

if ready:
    print("ready")

print("still printed even when ready is false")
```

Possible output:

```
still printed even when ready is false
```

If the second `print()` should belong to the `if`, it must be indented:


```
ready = False

if ready:
    print("ready")
    print("also controlled by the if")
```

No output appears, because both `print()` statements are inside the false branch.
The rule is direct:

Same indentation level means same structural layer. Greater indentation means a nested block. Returning to an earlier indentation level closes one or more nested blocks.

Structural Isolation: Eliminating Explicit Block Tokens (Curly Braces {}, begin/end)

Python does not use curly braces to define ordinary code blocks. It also does not use paired words such as `begin` and `end`. Instead, the block boundary is carried by indentation.

A C-style structure uses explicit delimiters:

```
/* C example, not Python */
if (score == 42) {
    printf("the answer\n");
    printf("do not panic\n");
}
printf("done\n");
```

A Python structure uses colon plus indentation:

```
score = 42

if score == 42:
    print("the answer")
    print("do not panic")

print("done")
```

Possible output:

```
the answer
do not panic
done
```

The missing curly braces are not an omission. They are a design decision. Python removes one class of structural tokens and transfers their job to the layout of the code.

This has several consequences.

First, block structure becomes visually unavoidable. If two statements are aligned, Python treats them as belonging to the same block level.

```
name = "Kramer"

if name == "Kramer":
    print("Giddy up!")
    print("This is still inside the if.")

print("This is outside the if.")
```

Second, indentation mistakes become syntax mistakes, not merely style mistakes. If Python expects an indented block and does not receive one, the program is invalid.

```
name = "Bart"

if name == "Bart":
    print("Eat my shorts.")
```

This raises an indentation error before the program can run normally:

```
IndentationError: expected an indented block after 'if' statement
```

Third, Python makes the physical shape of the code part of the language contract. This does not mean that any whitespace matters. Spaces between words and operators are usually flexible. The special whitespace is the indentation at the beginning of logical lines.

For example, these assignments are equivalent:

```
x=42
x = 42
x      =      42
```

But these block structures are not equivalent:

```
if x == 42:
    print("inside")
```

```
print("outside")
```

and:

```
if x == 42:
    print("inside")
    print("also inside")
```

The left margin changes the control-flow graph.

Lexer Parsing Rules: How the Tokenizer Tracks Indentation via Stacked `INDENT` / `DEDENT` States and Generates `IndentationError`

Python does not wait until execution to discover block structure. The source text is first processed by the tokenizer. The tokenizer reads logical lines and emits tokens. Among these tokens are special structural markers named `INDENT` and `DEDENT`.

A useful mental model is an indentation stack.

At the start of a file, Python is at indentation level zero. When a new block begins and the next logical line is more indented than the previous block level, Python pushes the new indentation level and emits an `INDENT` token. When a later line returns to an earlier indentation level, Python pops indentation levels and emits one or more `DEDENT` tokens.

The following script shows these tokens using the standard `tokenize` module:

```
import io
import tokenize

src = """\
score = 42

if score == 42:
    print("the answer")
    print("do not panic")
```

```

print("done")
"""

tokens = tokenize.generate_tokens(io.StringIO(src).readline)

for tok in tokens:
    tok_name = tokenize.tok_name[tok.type]
    if tok_name in ("INDENT", "DEDENT", "NAME", "OP", "STRING", "NUMBER"):
        print(f"{tok_name:<8} {tok.string!r}")

```

Possible output:

```

NAME      'score'
OP         '='
NUMBER    '42'
NAME      'if'
NAME      'score'
OP         '=='
NUMBER    '42'
OP         ':'
INDENT    '    '
NAME      'print'
OP         '('
STRING    '"the answer"'
OP         ')'
NAME      'print'
OP         '('
STRING    '"do not panic"'
OP         ')'
DEDENT    ''
NAME      'print'
OP         '('
STRING    '"done"'
OP         ')'

```

The important lines are:

```

INDENT    '    '
DEDENT    ''

```

They show that indentation is turned into formal structure before the program executes. The interpreter is not guessing from appearance. The tokenizer has already converted indentation changes into tokens.

The token objects produced by `tokenize` are runtime objects too:

```

import io
import tokenize

src = "x = 42\n"
tokens = tokenize.generate_tokens(io.StringIO(src).readline)

first_tok = next(tokens)

print(f"type(first_tok): {type(first_tok)}")
print(f"first_tok: {first_tok}")

```

```
print(f"first_tok.type: {first_tok.type}")
print(f"first_tok.string: {first_tok.string!r}")
print(f"has start: {'start' in dir(first_tok)}")
print(f"has end: {'end' in dir(first_tok)}")
print(f"has line: {'line' in dir(first_tok)}")
```

Possible output:

```
type(first_tok): <class 'tokenize.TokenInfo'>
first_tok: TokenInfo(type=1 (NAME), string='x', start=(1, 0), end=(1, 1), line='x = 42\n')
first_tok.type: 1
first_tok.string: 'x'
has start: True
has end: True
has line: True
```

The `type` field identifies the token category. The `string` field stores the exact source text for that token. The `start` and `end` fields store source positions. The `line` field stores the physical line from which the token came.

If Python expects indentation and the indentation does not form a valid block, it raises `IndentationError`.

```
bad_src = """\
if True:
print("Ni!")
"""

try:
    compile(bad_src, "<bad_indent>", "exec")
except IndentationError as err:
    print(f"type(err): {type(err)}")
    print(f"message: {err}")
    print(f"args: {err.args}")
    print(f"has lineno: {'lineno' in dir(err)}")
    print(f"line number: {err.lineno}")
```

Possible output:

```
type(err): <class 'IndentationError'>
message: expected an indented block after 'if' statement on line 1 (<bad_indent>, line 2)
args: ("expected an indented block after 'if' statement on line 1", (<bad_indent>, 2, 1, 'print(
has lineno: True
line number: 2
```

This example uses `compile()` so the error can be caught and inspected instead of stopping the whole demonstration script. The bad source is never executed. It fails during compilation, while Python is building the executable structure.

Mixed Tabs and Spaces: How `TabError` Emerges from Ambiguous Indentation

Python allows indentation with spaces or tabs in principle, but mixing them in the same block structure is dangerous. A tab is not visually stable across editors. One editor may display it as four columns, another as eight columns, and another may use a configurable width.

Python must decide block structure mechanically, not visually. When tabs and spaces create ambiguous indentation, Python raises `TabError`. `TabError` is a specialized form of `IndentationError`.

The following script builds problematic source text as a string. The `\t` sequence represents a tab character.

```
bad_src = "if True:\n        print('spaces')\n\tprint('tab')\n"

try:
    compile(bad_src, "<mixed_tabs_spaces>", "exec")
except TabError as err:
    print(f"type(err): {type(err)}")
    print(f"is IndentationError: {isinstance(err, IndentationError)}")
    print(f"message: {err}")
    print(f"args: {err.args}")
```

Possible output:

```
type(err): <class 'TabError'>
is IndentationError: True
message: inconsistent use of tabs and spaces in indentation (<mixed_tabs_spaces>, line 3)
args: ('inconsistent use of tabs and spaces in indentation', (<mixed_tabs_spaces>, 3, 1, "\
```

The line with spaces and the line with a tab may look similarly indented in some editors, but Python rejects the structure because the indentation is not reliably defined.

The inheritance relationship is visible directly:

```
print(TabError)
print(TabError.__mro__)
```

Possible output:

```
<class 'TabError'>
(<class 'TabError'>, <class 'IndentationError'>, <class 'SyntaxError'>, <class 'Exception'>,
```

This means a handler for `IndentationError` can also catch `TabError`, because `TabError` is lower in the same exception class tree.

```
bad_src = "if True:\n        print('spaces')\n\tprint('tab')\n"

try:
    compile(bad_src, "<mixed_tabs_spaces>", "exec")
except IndentationError as err:
    print(f"caught as indentation problem: {type(err)}")
```

Possible output:

```
caught as indentation problem: <class 'TabError'>
```

The practical rule is simple: use spaces for indentation, and use the same number of spaces for the same block depth. Four spaces per indentation level is the common Python convention.

Tabs and spaces are not morally different characters. The problem is that mixed indentation can make the visual block and the tokenizer's block disagree.

For this course, the deeper point is not merely style. It is structural determinism. Python must transform source text into a definite execution graph. Indentation is one of the inputs to that transformation. If the indentation cannot be converted into a clear nesting structure, Python refuses to compile the program.

6.2.2 Comparative Paradigm Analysis

Python's indentation system is not just a cosmetic alternative to curly braces. It changes the relationship between source layout and program structure. In C, C++, and Java, the compiler identifies blocks primarily through explicit delimiter tokens. In Python, the tokenizer identifies blocks through indentation changes. This means that Python moves part of the structural burden from punctuation into visual alignment.

This is not a minor syntactic taste difference. It changes how a programmer reads code. In curly-brace languages, the eye may look at indentation first, but the compiler follows the braces. In Python, the eye and the tokenizer are forced to follow the same visible structure.

In C-like languages, indentation describes structure to humans. In Python, indentation defines structure for both humans and the interpreter.

The comparison should not be reduced to the simplistic claim that one style is better in every context. Curly braces have advantages: they make block boundaries explicit, they survive bad formatting, and they allow automated formatters to reconstruct layout from token structure. Python's design makes a different trade: it forbids a gap between visual nesting and syntactic nesting.

Deviations and Readability Metrics Against Curly-Brace Compiled Languages (C, C++, Java)

A C programmer is used to seeing block structure marked by braces:

```
/* C example, not Python */
if (score == 42) {
    printf("the answer\n");
    printf("do not panic\n");
}
printf("done\n");
```

The braces define the block. The indentation merely helps the human reader. This means the following ugly version is still structurally equivalent:

```
/* C example, not Python */
if (score == 42) {
printf("the answer\n");
printf("do not panic\n");
}
printf("done\n");
```

The compiler can still parse it because the braces remain intact. The human reader, however, pays a cost. The source text no longer shows its control-flow shape clearly.

Python refuses that separation:

```
score = 42

if score == 42:
    print("the answer")
    print("do not panic")

print("done")
```

Possible output:

```
the answer
do not panic
done
```

In Python, the layout is not a second layer placed on top of syntax. The layout is part of the syntax. This creates a useful readability invariant:

A correctly parsed Python block has a visible left-margin shape that directly corresponds to its nesting structure.

Curly-brace languages can express structure even when indentation is misleading. For example:

```
/* C example, not Python */
if (ready)
    printf("ready\n");
    printf("finished\n");
```

A beginner may read both `printf()` calls as controlled by the `if`, because they are visually close. But only the first statement is controlled by the `if`. The second statement is always executed. The compiler does not care about the visual suggestion.

The equivalent Python structure cannot hide this mistake in the same way:

```
ready = False

if ready:
    print("ready")
    print("finished")
```

Here both `print()` statements are inside the `if`. If `ready` is false, neither executes.

If the second statement should always execute, it must move back to the outer indentation level:

```
ready = False

if ready:
    print("ready")

print("finished")
```

Possible output:

```
finished
```

This is why Python's whitespace system can be treated as a readability metric. The code's visible shape carries executable meaning. The depth of indentation measures the depth of nesting. A sudden return to the left margin is not merely visual relief; it is a structural exit from the current block.

A rough comparison is:

- In C, C++, and Java, block structure is token-delimited and indentation is conventional.
- In Python, block structure is indentation-delimited and indentation is grammatical.
- In curly-brace languages, bad indentation can mislead the reader while remaining legal.
- In Python, bad indentation often changes the program or prevents compilation.

This also affects how code reviews happen. In C-like languages, reviewers must sometimes mentally reconcile indentation with braces. In Python, alignment itself is part of the program. Misalignment is not merely ugly; it may be a control-flow defect.

Visual Layout as Functional Logic: Enforcing Uniform Alignment to Prevent Logical Scope Bleed

Logical scope bleed occurs when a statement appears to belong to one control structure but actually belongs to another. In C-like languages, this usually happens when the indentation suggests one structure while the braces or missing braces define another.

For example:

```
/* C example, not Python */
if (user_is_admin)
    printf("access granted\n");
    printf("audit log updated\n");
```

The second `printf()` line looks visually associated with the `if`, but it is not. It is outside the conditional body. The logical scope has bled away from the visual scope.

Python prevents this exact form of mismatch by making the visual indentation the actual block boundary:

```
user_is_admin = False

if user_is_admin:
    print("access granted")
    print("audit log updated")
```

No output appears. Both statements are inside the same conditional block.

If the audit log should always update, the source must show that explicitly by returning to the outer indentation level:

```
user_is_admin = False

if user_is_admin:
    print("access granted")

print("audit log updated")
```

Possible output:

```
audit log updated
```

The left margin now shows the real control relation. The `if` controls only the indented line. The final `print()` is outside the conditional block.

This rule becomes more important in nested structures:

```
name = "Bart"
age = 10

if name == "Bart":
    print("Eat my shorts.")
    if age < 18:
        print("minor character")
    print("still inside the name check")

print("outside all checks")
```

Possible output:


```
Eat my shorts.
minor character
still inside the name check
outside all checks
```

There are three visible structural levels:

```
level 0: outside all blocks
level 1: inside the name check
level 2: inside the age check
```

The indentation itself shows the containment relation. The line:

```
print("still inside the name check")
```

is not inside the age check, because it has returned from level 2 to level 1. But it is still inside the name check, because it has not returned to level 0.

This is functional logic, not decoration. Moving a line left or right changes the program.

```
name = "Bart"
age = 10

if name == "Bart":
    print("Eat my shorts.")
    if age < 18:
        print("minor character")
        print("now this is also inside the age check")

print("outside all checks")
```

The second message is now controlled by the nested `if`. Its indentation changed its execution condition.

This is the practical rule:

In Python, horizontal movement at the beginning of a line is a control-flow operation.

Uniform alignment therefore prevents accidental scope bleed. Statements that belong together must start at the same indentation level. Statements that belong to different structural layers must not be aligned as if they were peers.

A short diagnostic example makes the point:

```
items = ["soup", "pretzels", "muffin"]

for item in items:
    print(f"checking item: {item}")
    if item == "pretzels":
        print("These pretzels are making me thirsty.")
    print("finished one item")

print("finished all items")
```

Possible output:

```
checking item: soup
finished one item
checking item: pretzels
These pretzels are making me thirsty.
finished one item
checking item: muffin
finished one item
finished all items
```

The line:

```
print("finished one item")
```

belongs to the `for` loop, not to the nested `if`. This is visible because it is aligned with the `if` statement, not with the body of the `if` statement.

The line:

```
print("finished all items")
```

belongs to neither the `if` nor the `for`. It has returned to the outermost level.

This is the central contrast with curly-brace syntax. In C, C++, or Java, indentation helps the reader reconstruct the token-defined block structure. In Python, indentation is the block structure. The source layout does not merely represent the logic. It participates in creating the logic.

6.2.3 Empty Blocks and Structural Placeholders

Python requires every block header to have a syntactically valid body. A header line such as `if condition:`, `while condition:`, `for name in iterable:`, `try:`, `except:`, `def name():`, or `class Name:` announces that an indented block must follow.

This is not optional. The block body is part of the grammar. If the programmer writes a block header and then leaves the body empty, Python cannot construct a valid control-flow structure.

For example, this is invalid:

```
ready = True

if ready:

print("done")
```

Python raises an indentation-related syntax error, because the `if` statement promises a body but no body is provided.

```
IndentationError: expected an indented block after 'if' statement
```

This matters because, during program construction, programmers often know that a block must exist structurally before they know what the block should do. Python therefore provides a special placeholder statement: `pass`.

The `pass` Statement: Syntactically Valid Empty Control-Flow Bodies

The `pass` statement means: this block intentionally contains no operational statement yet. It satisfies the syntax requirement without producing an action at runtime.

```
ready = True

if ready:
    pass

print("done")
```

Possible output:

```
done
```

The `if` block is not missing anymore. It contains one valid statement: `pass`. That statement does nothing, but doing nothing is different from being absent. The first is valid syntax. The second is incomplete syntax.

A simple way to read `pass` is:

`pass` is an explicit empty statement used where Python requires a statement but the programmer wants no runtime action.

It is common in temporary control-flow scaffolding:

```
score = 42

if score > 90:
    print("excellent")
elif score > 50:
    pass
else:
    print("needs work")

print("grading finished")
```

Possible output:

```
grading finished
```

The `elif` branch is selected because `42 > 50` is false? No. In this exact program, the `elif` branch is not selected. The program reaches the `else` branch and would print `needs work`. If we want to demonstrate the selected empty branch, the value must satisfy the middle condition:

```
score = 72

if score > 90:
    print("excellent")
elif score > 50:
    pass
else:
    print("needs work")

print("grading finished")
```

Possible output:

```
grading finished
```

The `elif` branch was entered, but it had no visible effect because its only statement was `pass`. After that empty branch completes, execution continues after the whole conditional structure.

The control-flow graph still has a real branch:

```
test score > 90
  true  -> print "excellent" -> after if
  false -> test score > 50
            true  -> pass -> after if
            false -> print "needs work" -> after if
```

So `pass` does not remove the branch. It merely gives that branch an empty body.

The same placeholder is useful in loops:

```
names = ["Jerry", "Elaine", "Kramer"]
```

```
for name in names:
    pass
```

```
print("all names traversed")
```

Possible output:

```
all names traversed
```

The loop still runs. The name `name` still receives each value in sequence. The loop body simply performs no visible operation.

We can prove that the loop still executes by printing after the loop:

```
names = ["Jerry", "Elaine", "Kramer"]
```

```
for name in names:
    pass
```

```
print(f"last bound name: {name}")
```

Possible output:

```
last bound name: Kramer
```

The final value remains bound after the loop because the `for` statement really did traverse the list. The body was empty in behavior, not absent in structure.

The `pass` statement is also useful when defining a function or class before implementing it:

```
def convert_score(score):
    pass
```

```
class EmptyBox:
    pass
```

```
print(f"type(convert_score): {type(convert_score)}")
print(f"type(EmptyBox): {type(EmptyBox)}")
print(f"type(EmptyBox()): {type(EmptyBox())}")
```

Possible output:

```
type(convert_score): <class 'function'>
type(EmptyBox): <class 'type'>
type(EmptyBox()): <class '__main__.EmptyBox'>
```

The function exists even though its body does no useful work. If it is called, it returns `None`, because a Python function without an explicit `return` statement returns `None` automatically.

```
def convert_score(score):
    pass
```

```
result = convert_score(42)
```

```
print(f"result: {result}")
print(f"type(result): {type(result)}")
print(f"dir(result) sample: {dir(result)[:5]}")
```

Possible output:

```
result: None
type(result): <class 'NoneType'>
dir(result) sample: ['__bool__', '__class__', '__delattr__', '__dir__', '__doc__']
```

The object `None` is Python's single built-in object for representing absence of a returned value. Its type is `NoneType`. The method `__bool__` is important because `None` is false in truth-value testing:

```
result = None

print(f"bool(result): {bool(result)}")
print(f"result is None: {result is None}")
```

Possible output:

```
bool(result): False
result is None: True
```

This distinction is important:

- `pass` is a statement.
- `None` is an object.
- A function body containing only `pass` still returns `None` when called.

The statement `pass` itself does not create a value that can be assigned. This is invalid:

```
x = pass
```

It fails because `pass` is not an expression. It is a statement. It can occupy a required statement position, but it cannot produce a value.

For control-flow analysis, `pass` is best understood as a zero-operation node. It occupies a place in the graph, but it does not transform program state in any meaningful way.

```
ready = True

if ready:
    pass

print("No soup for you!")
```

Possible output:

```
No soup for you!
```

The branch exists. The body exists. The runtime action is intentionally empty.

This makes `pass` useful during staged construction of a program. The programmer can first build the control-flow skeleton, verify that the source has valid structure, and later replace the placeholder with real operations.

An empty block is illegal because Python needs a statement. `pass` is the explicit statement that says: this block is intentionally empty.

6.3 Conditional Statements and Decision Trees

A program that only runs from top to bottom can describe a fixed procedure, but it cannot react to different runtime states. Conditional statements introduce decision points into the execution graph. At each decision point, Python evaluates a condition and selects one route while skipping the others.

This section studies how those branching routes are written, nested, and evaluated. The visible syntax of `if`, `elif`, and `else` defines the shape of the decision tree, while Python's truth-value rules decide which edge is followed at runtime.

The central idea is that a condition is not merely a decorative test placed before some code. It is a routing mechanism. It decides whether a block becomes reachable, whether a nested decision is even inspected, and whether execution continues through one path or another.

6.3.1 Syntax of Branching Control Graphs

A conditional statement is the first major structure where the written order of source code visibly separates from the possible execution routes. The source still appears as text written from top to bottom, but the runtime does not necessarily execute every line. It tests a condition, selects a branch, executes the selected block, and skips the others.

The basic syntax is:

```
if condition:
    statements_for_true_condition
elif another_condition:
    statements_for_second_condition
else:
    statements_for_all_previous_conditions_false
```

The colon announces that a block must follow. The indentation defines the block body. The `if`, `elif`, and `else` headers are decision nodes. Their indented bodies are the possible route bodies attached to those nodes.

A conditional statement is not only a textual structure. It is a control-flow structure that creates several possible forward edges and then rejoins after the selected branch.

This subsection focuses on the visible syntax of branching. The next subsection will examine the deeper evaluation rules: truth-value testing, short-circuit operators, and chained comparisons.

Sequential Evaluation: The Structure of `if`, `elif`, and `else` Nodes

An `if/elif/else` chain is evaluated from top to bottom. Python tests the `if` condition first. If it is true, Python executes that block and skips the remaining `elif` and `else` parts. If it is false, Python moves to the first `elif`. This continues until one condition succeeds or until the optional `else` block is reached.

```
score = 42

if score >= 90:
    print("excellent")
elif score >= 70:
    print("good")
elif score >= 50:
    print("acceptable")
else:
    print("needs work")

print("decision complete")
```

Possible output:

```
needs work
decision complete
```

The route is:

```
test score >= 90
  false -> test score >= 70
    false -> test score >= 50
      false -> execute else block
        -> continue after whole conditional
```

For a different value, a different route is chosen:

```
score = 72

if score >= 90:
    print("excellent")
elif score >= 70:
    print("good")
elif score >= 50:
    print("acceptable")
else:
    print("needs work")

print("decision complete")
```

Possible output:

```
good
decision complete
```

Even though `score >= 50` is also true, that later branch is not tested after `score >= 70` succeeds. The chain selects the first successful branch.

This is the main structural difference between an `if/elif/else` chain and several independent `if` statements.

```
score = 72

if score >= 90:
    print("excellent")

if score >= 70:
    print("good")

if score >= 50:
    print("acceptable")

print("decision complete")
```

Possible output:

```
good
acceptable
decision complete
```

Here Python encounters three separate `if` statements. Each one is its own decision node. The success of the second `if` does not prevent the third `if` from being evaluated.

The shape is different:

```
if / elif / else chain:
    one connected decision structure
    first successful branch wins

separate if statements:
    several independent decision structures
    each condition is tested separately
```

The condition expression itself usually produces or behaves like a boolean decision. The direct result of a comparison is a `bool` object:

```
score = 72
check = score >= 70

print(f"check: {check}")
print(f"type(check): {type(check)}")
print(f"dir(check) sample: {dir(check)[:4]}")
print(f"check.__class__: {check.__class__}")
print(f"check.__int__(): {check.__int__()}")
```

Possible output:

```
check: True
type(check): <class 'bool'>
dir(check) sample: ['__abs__', '__add__', '__and__', '__bool__']
check.__class__: <class 'bool'>
check.__int__(): 1
```

The `bool` type has two built-in instances: `True` and `False`. Its `__bool__()` method returns its own truth value. Its `__int__()` method exposes the historical numeric relationship where `True` behaves as 1 and `False` behaves as 0 in numeric contexts. This does not mean that conditions should be mentally reduced to arithmetic. In ordinary branching, the important fact is simply that the condition selects the next execution edge.

The `else` block has no condition. It is the fallback route.

```
name = "Kramer"

if name == "Jerry":
    print("Hello, Newman.")
elif name == "Elaine":
    print("Get out!")
else:
    print("Giddy up!")
```

Possible output:

```
Giddy up!
```

The `else` route is selected only after all earlier tests in the same chain fail.

Nested Code Blocks: Creating Complex Hierarchical Decision Trees

A conditional body may contain another conditional statement. This creates a nested decision tree. The outer decision must be entered before the inner decision can even be reached.

```
name = "Bart"
age = 10

if name == "Bart":
    print("character found")

    if age < 18:
        print("minor character")
    else:
        print("adult character")

else:
    print("different character")

print("analysis complete")
```

Possible output:

```
character found
minor character
analysis complete
```

The inner `if age < 18:` statement exists inside the body of the outer `if name == "Bart":` statement. Python reaches the age test only if the name test succeeds.

The route shape is:

```
test name == "Bart"
  true:
    print "character found"
    test age < 18
      true:
        print "minor character"
      false:
        print "adult character"
  false:
    print "different character"
```

```
after whole nested structure:
  print "analysis complete"
```

The indentation levels describe the containment relation:

```
level 0: outside all conditionals
level 1: inside the outer name decision
level 2: inside the inner age decision
```

A nested decision tree is useful when one question depends on the answer to an earlier question.

```
has_ticket = True
has_secret_word = False
```

```
if has_ticket:
    print("ticket accepted")

    if has_secret_word:
        print("Nobody expects the Spanish Inquisition!")
    else:
        print("ordinary entrance")

else:
    print("no ticket, no entrance")

print("gate check finished")
```

Possible output:

```
ticket accepted
ordinary entrance
gate check finished
```

The secret-word test is not a peer of the ticket test. It is subordinate to it. If `has_ticket` is false, Python never evaluates the inner `if has_secret_word:` statement.

```
has_ticket = False
has_secret_word = True

if has_ticket:
    print("ticket accepted")

    if has_secret_word:
        print("Nobody expects the Spanish Inquisition!")
    else:
        print("ordinary entrance")

else:
    print("no ticket, no entrance")

print("gate check finished")
```

Possible output:

```
no ticket, no entrance
gate check finished
```

Even though `has_secret_word` is true, that fact is irrelevant because the program never enters the outer true branch.

Nested conditionals should be read from the outside inward:

The outer condition decides whether the inner decision point is reachable. The inner condition decides only after execution has already entered the outer branch.

This is where indentation becomes more than neat formatting. Moving one line changes the tree.

```
has_ticket = True
has_secret_word = False
```

```
if has_ticket:
    print("ticket accepted")

if has_secret_word:
    print("secret entrance")
else:
    print("not a secret entrance")
```

Possible output:

```
ticket accepted
not a secret entrance
```

Here the second `if` is not nested. It is aligned with the first `if`, so it is a separate decision node. The secret-word decision happens regardless of the ticket decision.

By contrast:

```
has_ticket = True
has_secret_word = False

if has_ticket:
    print("ticket accepted")

    if has_secret_word:
        print("secret entrance")
    else:
        print("not a secret entrance")
```

Possible output:

```
ticket accepted
not a secret entrance
```

The output happens to be the same for this input, but the graph is not the same. If `has_ticket` becomes false, the difference appears.

```
has_ticket = False
has_secret_word = False

if has_ticket:
    print("ticket accepted")

    if has_secret_word:
        print("secret entrance")
    else:
        print("not a secret entrance")

print("finished")
```

Possible output:

```
finished
```

The whole inner decision is unreachable because it is inside the false outer branch.

This is the reason nested conditional code must be read structurally, not only line by line. Each indentation level modifies reachability.

Conditional Expressions: Inline Branch Selection with `x if condition else y`

Python also has an expression form for selecting between two values:

```
value_if_true if condition else value_if_false
```

This is called a conditional expression. It is sometimes informally called a ternary expression because it has three parts: the true-side value, the condition, and the false-side value.

The order is important. Python does not write it as `condition ? x : y`, as in C-like languages. Python writes the selected true value first, then the condition, then the false value.

```
score = 42

status = "passing" if score >= 50 else "failing"

print(f"score: {score}")
print(f"status: {status}")
print(f"type(status): {type(status)}")
print(f"dir(status) sample: {dir(status)[:8]}")
```

Possible output:

```
score: 42
status: failing
type(status): <class 'str'>
dir(status) sample: ['__add__', '__class__', '__contains__', '__delattr__', '__dir__', '__doc__',
```

The selected result is an ordinary object. There is no special “conditional expression object” left behind. In this example, the selected object is a string. The string method `__contains__()` supports membership tests such as `"fail" in status`. The method `__add__()` supports string concatenation with `+`. The method `__format__()` is used by formatting operations such as f-strings.

```
status = "failing"

print(f"'fail' in status: {'fail' in status}")
print(f"status + '!': {status + '!'}")
print(f"formatted status: {status:>10}")
```

Possible output:

```
'fail' in status: True
status + '!': failing!
formatted status:      failing
```

A conditional expression is useful when the branch selects a value, not when the branch contains several actions.

Good use:

```
name = "Chuck"
power = 42

label = "legend" if name == "Chuck" and power >= 40 else "ordinary"

print(f"name: {name}")
print(f"label: {label}")
```

Possible output:

```
name: Chuck
label: legend
```

Less readable use:

```
name = "Chuck"
power = 42
```

```
message = "Chuck Norris counted to infinity twice." if name == "Chuck" and power >= 40 else "
print(message)
```

Possible output:

```
Chuck Norris counted to infinity twice.
```

This still works, but long expressions can become hard to read. If the logic needs several steps, ordinary `if/else` blocks are clearer.

The block form:

```
score = 72

if score >= 50:
    status = "passing"
else:
    status = "failing"

print(f"status: {status}")
```

Possible output:

```
status: passing
```

The expression form:

```
score = 72

status = "passing" if score >= 50 else "failing"

print(f"status: {status}")
```

Possible output:

```
status: passing
```

Both versions select between two values. The difference is structural.

- The `if/else` statement selects a block of statements to execute.
- The conditional expression selects one value inside a larger expression.

Because a conditional expression is an expression, it can appear where a value is needed.

```
temperature = 18

print(f"weather: {'warm' if temperature >= 20 else 'cold'}")
```

Possible output:

```
weather: cold
```

It can also be used inside a list:

```
scores = [42, 88, 15]

labels = [
    "pass" if score >= 50 else "fail"
    for score in scores
]

print(f"scores: {scores}")
print(f"labels: {labels}")
```

Possible output:

```
scores: [42, 88, 15]
labels: ['fail', 'pass', 'fail']
```

This example also contains a list comprehension, which will be studied elsewhere. At this point, the important observation is simpler: the conditional expression contributes one selected value for each score.

The conditional expression must include an `else` part. This is invalid:

```
status = "passing" if score >= 50
```

Python rejects it because an expression must produce a value in every possible route. Without the `else` part, the false route has no value to return.

The practical rule is:

Use a conditional statement when you need to choose between execution blocks. Use a conditional expression when you need to choose between two values.

The control-flow graph still exists in the expression form, but it is compressed into a value-producing expression. One route produces the true-side value. The other route produces the false-side value. Execution then continues with that selected object.

6.3.2 Logic Evaluation Architecture

Branching syntax defines where the possible routes are. Logic evaluation decides which route is selected. In Python, the expression written after `if`, `elif`, or `while` does not always need to be a literal `True` or `False`. Python can ask many kinds of objects whether they should count as true or false in a control-flow decision.

This is why Python conditionals are not limited to direct comparisons such as `x > 0`. They may contain names, containers, strings, numbers, function results, logical operators, and chained comparisons.

A conditional expression does not merely calculate a value. In a branch or loop header, Python converts that value into a routing decision.

This subsection studies three parts of that routing machinery: truth-value testing, short-circuit evaluation, and comparison chaining.

Truth Value Testing: Evaluating Implicit Truthiness and Falsiness via `__bool__` and `__len__`

Python accepts any object in a conditional position. It then asks whether that object should be treated as true or false. This process is called truth-value testing.

The direct conversion can be seen with `bool()`:

```
values = [
    True,
    False,
    42,
    0,
    "spam",
    "",
    [1, 2, 3],
    [],
    None,
]

for value in values:
    print(f"{repr(value):>10} -> {bool(value)}")
```

Possible output:

```
True -> True
False -> False
42 -> True
0 -> False
'spam' -> True
'' -> False
[1, 2, 3] -> True
[] -> False
None -> False
```

This does not mean that all these objects are booleans. It means they can be tested as booleans.

```
items = []

print(f"items: {items}")
print(f"type(items): {type(items)}")
print(f"bool(items): {bool(items)}")
print(f"len(items): {len(items)}")
print(f"has __len__: {'__len__' in dir(items)}")
print(f"has __bool__: {'__bool__' in dir(items)}")
```

Possible output:

```
items: []
type(items): <class 'list'>
bool(items): False
len(items): 0
has __len__: True
has __bool__: False
```

A list does not normally define its own `__bool__()` method. Instead, Python can use its length. An empty list is false. A non-empty list is true.

```
items = ["soup"]

print(f"items: {items}")
print(f"type(items): {type(items)}")
print(f"bool(items): {bool(items)}")
print(f"len(items): {len(items)}")
print(f"items.__len__(): {items.__len__()}")
```

Possible output:

```
items: ['soup']
type(items): <class 'list'>
bool(items): True
len(items): 1
items.__len__(): 1
```

The practical rule is:

If an object has `__bool__()`, Python uses it for truth-value testing. If it does not, but it has `__len__()`, zero length means false and non-zero length means true.

Numbers usually have direct boolean behavior:

```
num1 = 42
num2 = 0

print(f"type(num1): {type(num1)}")
print(f"bool(num1): {bool(num1)}")
print(f"num1.__bool__(): {num1.__bool__()}")
print(f"has __bool__: {'__bool__' in dir(num1)}")
print(f"has __len__: {'__len__' in dir(num1)}")

print(f"bool(num2): {bool(num2)}")
print(f"num2.__bool__(): {num2.__bool__()}")
```

Possible output:

```
type(num1): <class 'int'>
bool(num1): True
num1.__bool__(): True
has __bool__: True
has __len__: False
bool(num2): False
num2.__bool__(): False
```

Strings are length-based:

```
quote = "Ni!"

print(f"quote: {quote!r}")
print(f"type(quote): {type(quote)}")
print(f"bool(quote): {bool(quote)}")
print(f"len(quote): {len(quote)}")
print(f"quote.__len__(): {quote.__len__()}")
print(f"has __len__: {'__len__' in dir(quote)}")
```


Possible output:

```
quote: 'Ni!'
type(quote): <class 'str'>
bool(quote): True
len(quote): 3
quote.__len__(): 3
has __len__: True
```

An empty string is false because its length is zero:

```
quote = ""

if quote:
    print("quote exists")
else:
    print("quote is empty")
```

Possible output:

```
quote is empty
```

This is common Python style. Instead of writing:

```
if len(quote) > 0:
    print("quote exists")
```

Python code often writes:

```
if quote:
    print("quote exists")
```

The shorter version does not mean that `quote` is a boolean. It means the string object is being used in a truth-value test.

The object `None` is always false:

```
result = None

print(f"result: {result}")
print(f"type(result): {type(result)}")
print(f"bool(result): {bool(result)}")
print(f"dir(result) sample: {dir(result)[:8]}")
print(f"result is None: {result is None}")
```

Possible output:

```
result: None
type(result): <class 'NoneType'>
bool(result): False
dir(result) sample: ['__bool__', '__class__', '__delattr__', '__dir__', '__doc__', '__eq__',
result is None: True
```

The `is None` test checks identity with the single `None` object. It is not the same as a general false test.

```

value = 0

if value:
    print("truthy")
else:
    print("false by truth-value testing")

if value is None:
    print("missing value")
else:
    print("actual object present")

```

Possible output:

```

false by truth-value testing
actual object present

```

The integer 0 is false in a truth-value test, but it is not `None`. This distinction matters in real programs. A missing value and a present-but-zero value are not the same fact.

Short-Circuit Evaluation: How Logical Operators (and, or) Halt Redundant Condition Execution

The logical operators `and` and `or` do not always evaluate both sides. They stop as soon as the final truth result is already determined. This is called short-circuit evaluation.

For `and`, if the left side is false, the whole expression must be false. Python does not need to evaluate the right side.

```

items = []

if items and items[0] == "soup":
    print("first item is soup")
else:
    print("no soup check completed safely")

```

Possible output:

```

no soup check completed safely

```

The expression `items[0]` would raise an error if evaluated on an empty list. But it is not evaluated. Since `items` is false, Python already knows that the whole `and` expression cannot be true.

This is the route:

```

evaluate items
    false -> stop immediately
            whole and-expression is false

```

With a non-empty list, the second part can be reached:

```

items = ["soup", "muffin"]

if items and items[0] == "soup":
    print("first item is soup")
else:
    print("first item is not soup")

```

Possible output:

```
first item is soup
```

For `or`, if the left side is true, the whole expression must be true. Python does not need to evaluate the right side.

```
name = "Chuck"
```

```
if name == "Chuck" or name[100] == "x":
    print("safe because the right side was not evaluated")
```

Possible output:

```
safe because the right side was not evaluated
```

The expression `name[100]` would normally fail because the string is not that long. But the left side is already true, so the right side is not evaluated.

The route is:

```
evaluate name == "Chuck"
    true -> stop immediately
           whole or-expression is true
```

This behavior is not only an optimization. It is a control-flow feature. Programmers use it to guard operations that are only safe after earlier checks.

A common shape is:

```
if object_exists and object_is_usable:
    use_object
```

A concrete version:

```
text = ""

if text and text[0].isupper():
    print("starts with uppercase")
else:
    print("empty text or not uppercase")
```

Possible output:

```
empty text or not uppercase
```

The second test is reached only if the string is non-empty.

The logical operators also return one of their operands, not necessarily a separate `True` or `False` object.

```
value1 = "" or "fallback"
value2 = "ready" and "go"

print(f"value1: {value1!r}")
print(f"type(value1): {type(value1)}")
print(f"value2: {value2!r}")
print(f"type(value2): {type(value2)}")
```

Possible output:

```
value1: 'fallback'
type(value1): <class 'str'>
value2: 'go'
type(value2): <class 'str'>
```

The expression `" "` or `"fallback"` returns the first truthy operand it finds. The expression `"ready"` and `"go"` returns the last operand because all operands tested so far are truthy.

More examples:

```
print(f"{0 or 42 = }")
print(f"{42 or 100 = }")
print(f"{0 and 42 = }")
print(f"{42 and 100 = }")
```

Possible output:

```
0 or 42 = 42
42 or 100 = 42
0 and 42 = 0
42 and 100 = 100
```

This is why it is slightly inaccurate to say that `and` and `or` always return booleans. In a conditional header, the returned object is then truth-tested. But as expressions, `and` and `or` return operand objects.

The object type can therefore depend on which route was selected:

```
text = ""
result = text or 42

print(f"result: {result!r}")
print(f"type(result): {type(result)}")
print(f"dir(result) sample: {dir(result)[:8]}")
```

Possible output:

```
result: 42
type(result): <class 'int'>
dir(result) sample: ['__abs__', '__add__', '__and__', '__bool__', '__ceil__', '__class__', '__delattr__', '__delat
```

If `text` changes, the result type can change too:

```
text = "No soup for you!"
result = text or 42

print(f"result: {result!r}")
print(f"type(result): {type(result)}")
```

Possible output:

```
result: 'No soup for you!'
type(result): <class 'str'>
```

The control-flow rule remains simple:

- `and` stops at the first false operand; otherwise it returns the last operand.
- `or` stops at the first true operand; otherwise it returns the last operand.

Comparison Chaining: Interpreting Expressions Such as $a < b < c$

Python allows comparison expressions to be chained. The expression:

```
a < b < c
```

means:

```
a < b and b < c
```

but with one important structural detail: the middle expression is evaluated only once.

A simple example:

```
a = 10
b = 20
c = 30

check = a < b < c

print(f"a: {a}")
print(f"b: {b}")
print(f"c: {c}")
print(f"check: {check}")
print(f"type(check): {type(check)}")
print(f"dir(check) sample: {dir(check)[:4]}")
```

Possible output:

```
a: 10
b: 20
c: 30
check: True
type(check): <class 'bool'>
dir(check) sample: ['__abs__', '__add__', '__and__', '__bool__']
```

The result of the whole comparison chain is a `bool` object. The comparison itself still follows short-circuit logic. If the first comparison fails, Python does not need the later comparisons.

```
a = 30
b = 20
c = 10

check = a < b < c

print(f"check: {check}")
```

Possible output:

```
check: False
```

The first comparison `a < b` is false, so the whole chain is false.

This syntax is especially useful for interval tests:

```
value = 42

if 0 <= value <= 100:
    print("value is inside the accepted interval")
else:
    print("value is outside the accepted interval")
```

Possible output:

```
value is inside the accepted interval
```

Written without chaining, the same idea is:

```
value = 42
```

```
if 0 <= value and value <= 100:
    print("value is inside the accepted interval")
```

The chained version is closer to the intended reading: the value is between the lower and upper limits.

For C programmers, this is a major semantic difference. In C, an expression that visually resembles this:

```
/* C idea, not Python */
0 <= value <= 100
```

does not mean the same thing. C evaluates it roughly as:

```
/* C idea, not Python */
(0 <= value) <= 100
```

The first comparison produces an integer-like truth result, usually 0 or 1, and then that result is compared with 100. That is not an interval test. Python's comparison chaining is designed to mean the interval test directly.

Python also supports mixed comparison operators in one chain:

```
a = 5
b = 5
c = 10

check = a == b < c

print(f"check: {check}")
```

Possible output:

```
check: True
```

This means:

```
a == b and b < c
```

It does not mean:

```
(a == b) < c
```

The shared middle operand is part of both neighboring comparisons.

Membership and identity comparisons can also be chained, although readability should control whether this is a good idea.

```
x = 42
items = [10, 42, 90]

check = 10 < x in items

print(f"check: {check}")
```

Possible output:

```
check: True
```

This means:

```
10 < x and x in items
```

The chain is legal, but it may surprise readers. In most teaching and production code, simple numeric chains are clearer than clever mixed chains.

A more readable form is:

```
x = 42
items = [10, 42, 90]

check = 10 < x and x in items

print(f"check: {check}")
```

Possible output:

```
check: True
```

The practical rule is:

Use comparison chaining for natural mathematical or interval-like relations. Avoid chains that force the reader to stop and decode the grammar.

The control-flow connection is direct. A chained comparison is one condition expression, but internally it contains several comparison edges. Python evaluates them from left to right and stops as soon as the chain cannot succeed. The final result is a `bool`, and that boolean selects the branch route.

6.4 Arithmetic Sequence Descriptors: The `range()` Object

Loops often need a controlled sequence of integer values: positions, counters, repeated step numbers, or numeric boundaries. In Python, this role is usually handled by `range()`. However, `range()` should not be understood as a function that builds a list of numbers. It builds a compact object that describes how those numbers can be produced.

This distinction is central to Python's traversal model. A `range` object stores an arithmetic rule: where to start, where to stop, and how far to move at each step. The actual values are computed only when they are requested by indexing, iteration, membership testing, or explicit materialization.

This section studies `range()` as a lazy arithmetic sequence descriptor. It explains why it uses fixed memory, how it behaves like a sequence without storing all elements, why membership testing can be arithmetic rather than linear, and why the same range object can be reused safely across multiple loops.

6.4.1 The Lazy Evaluation Invariant

The object returned by `range()` is often introduced as if it were a list of numbers. That description is convenient, but technically wrong. A `range` object is not a stored list. It is a compact descriptor of an arithmetic sequence.

The difference is fundamental. A list contains references to its elements. A `range` object contains the information needed to compute its elements when they are requested.

A `range` object stores the rule for producing numbers, not the produced numbers themselves.

This is why `range()` belongs naturally in a chapter about control flow and lazy traversal. It is one of the first Python objects that clearly separates the idea of a sequence from the idea of materialized storage.

Abstracting Arithmetic Progressions: The Memory Profile of Fixed-Space Object Storage

An arithmetic progression is a sequence where each value is obtained by repeatedly adding the same step. For example:

0, 1, 2, 3, 4

can be described as:

start at 0
stop before 5
move by 1

The `range()` object stores this kind of description.

```
nums = range(0, 5, 1)

print(f"nums: {nums}")
print(f"type(nums): {type(nums)}")
print(f"dir(nums) sample: {dir(nums)[:12]}")

print(f"nums.start: {nums.start}")
print(f"nums.stop: {nums.stop}")
print(f"nums.step: {nums.step}")
```

Possible output:

```
nums: range(0, 5)
type(nums): <class 'range'>
dir(nums) sample: ['__bool__', '__class__', '__contains__', '__delattr__', '__dir__',
'__doc__', '__eq__', '__format__', '__ge__', '__getattr__', '__getitem__',
'__gt__']
nums.start: 0
nums.stop: 5
nums.step: 1
```

The printed representation `range(0, 5)` is already a clue. Python does not print:

```
[0, 1, 2, 3, 4]
```

because the object is not a list. It is a range descriptor.

The important exposed attributes are:

- **start**: the first value of the arithmetic description;
- **stop**: the boundary value that is not crossed;
- **step**: the amount added between successive values.

The object also provides sequence-like methods through special methods. For example, `__len__()` gives the number of values described by the range, and `__getitem__()` computes an item at a requested index.

```
nums = range(10, 20, 2)

print(f"nums: {nums}")
print(f"len(nums): {len(nums)}")
```



```
print(f"nums.__len__(): {nums.__len__()}")

print(f"nums[0]: {nums[0]}")
print(f"nums[1]: {nums[1]}")
print(f"nums[2]: {nums[2]}")
print(f"nums.__getitem__(3): {nums.__getitem__(3)}")
```

Possible output:

```
nums: range(10, 20, 2)
len(nums): 5
nums.__len__(): 5
nums[0]: 10
nums[1]: 12
nums[2]: 14
nums.__getitem__(3): 16
```

The values are obtained from the arithmetic rule:

```
index 0 -> 10 + 0 * 2 -> 10
index 1 -> 10 + 1 * 2 -> 12
index 2 -> 10 + 2 * 2 -> 14
index 3 -> 10 + 3 * 2 -> 16
```

No separate stored array of these integers is required for the `range` object itself.

This can be compared with a list created from the same range:

```
nums_range = range(10, 20, 2)
nums_list = list(nums_range)

print(f"nums_range: {nums_range}")
print(f"nums_list: {nums_list}")

print(f"type(nums_range): {type(nums_range)}")
print(f"type(nums_list): {type(nums_list)}")
```

Possible output:

```
nums_range: range(10, 20, 2)
nums_list: [10, 12, 14, 16, 18]
type(nums_range): <class 'range'>
type(nums_list): <class 'list'>
```

The list is materialized. It contains element slots. The range is descriptive. It contains the arithmetic boundaries.

This distinction matters because a loop does not need the whole list at once:

```
for n in range(1, 4):
    print(f"n: {n}")
```

Possible output:

```
n: 1
n: 2
n: 3
```

The loop asks for successive values. The `range` object can provide them without storing the entire sequence beforehand.

The $O(1)$ Memory Footprint: Why `range(1000000)` Stores Only Start, Stop, and Step Parameters

The memory cost of a `range` object does not grow with the number of values it describes. Whether the range describes ten values, one million values, or one billion values, the object stores a fixed amount of descriptor information.

At the conceptual Python level, that information is:

```
start
stop
step
```

In CPython implementation terms, the object may also keep derived internal information such as the computed length, but it still does not store every produced integer. The important invariant remains the same: the memory used by the `range` descriptor is constant with respect to the number of described values.

This is called $O(1)$ memory use. Here, $O(1)$ means that the storage size stays fixed as the logical sequence length grows.

```
import sys

small_range = range(10)
large_range = range(1_000_000)
huge_range = range(1_000_000_000)

print(f"small_range: {small_range}")
print(f"large_range: {large_range}")
print(f"huge_range: {huge_range}")

print(f"size of small_range: {sys.getsizeof(small_range)} bytes")
print(f"size of large_range: {sys.getsizeof(large_range)} bytes")
print(f"size of huge_range: {sys.getsizeof(huge_range)} bytes")
```

Possible output on one CPython build:

```
small_range: range(0, 10)
large_range: range(0, 1000000)
huge_range: range(0, 1000000000)
size of small_range: 48 bytes
size of large_range: 48 bytes
size of huge_range: 48 bytes
```

The exact byte count is implementation-dependent. Another Python build may report a different number. The relevant observation is that all three range objects have the same reported size on the same build.

The contrast with a list is immediate:

```
import sys

nums_range = range(1_000_000)
nums_list = list(nums_range)

print(f"type(nums_range): {type(nums_range)}")
print(f"type(nums_list): {type(nums_list)}")

print(f"size of nums_range: {sys.getsizeof(nums_range)} bytes")
print(f"size of nums_list: {sys.getsizeof(nums_list)} bytes")
```

Possible output on one CPython build:

```
type(nums_range): <class 'range'>
type(nums_list): <class 'list'>
size of nums_range: 48 bytes
size of nums_list: 8000056 bytes
```

The list size shown here is only the size of the list container’s internal reference array. It does not even fully count the memory of every integer object that may be referenced. The range object avoids that problem because it does not contain one reference slot per value.

This is the concrete storage difference:

```
range(1_000_000):
    stores a compact arithmetic descriptor

list(range(1_000_000)):
    stores a list object with one million element positions
```

The word “lazy” should be read literally here. A `range` delays value production until a value is demanded by indexing, iteration, membership testing, or materialization.

```
nums = range(0, 10)

print("range object created")
print(f"nums: {nums}")

print("now asking for one value")
print(f"nums[4]: {nums[4]}")

print("now materializing all values")
print(f"list(nums): {list(nums)}")
```

Possible output:

```
range object created
nums: range(0, 10)
now asking for one value
nums[4]: 4
now materializing all values
list(nums): [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

The conversion with `list(nums)` is the materialization boundary. Before that line, the program has a range descriptor. After that line, the program has an actual list containing the produced values.

The same rule applies to very large ranges:

```
nums = range(0, 1_000_000)

print(f"nums: {nums}")
print(f"first value: {nums[0]}")
print(f"middle value: {nums[500_000]}")
print(f"last value: {nums[-1]}")
```

Possible output:

```

nums: range(0, 1000000)
first value: 0
middle value: 500000
last value: 999999

```

The program can inspect selected positions without building a million-element list. Each requested value is computed from the descriptor.

This makes `range()` suitable for loop control:

```

for i in range(3):
    print(f"step {i}: And now for something completely different.")

```

Possible output:

```

step 0: And now for something completely different.
step 1: And now for something completely different.
step 2: And now for something completely different.

```

The loop receives the values one by one. The range remains a compact reusable object.

The practical rule is:

Use `range()` when you need an arithmetic sequence for traversal. Convert it to `list` only when you truly need all values stored at once.

For control-flow analysis, `range()` is important because it shows that a loop source does not have to be a fully materialized container. It can be a compact object that describes how traversal values should be generated.

6.4.2 Sequence Behavior Metrics

Although a `range` object does not store all its values, it still behaves like a sequence in several important ways. It has a length, supports indexing, supports slicing, can be traversed in a `for` loop, and can be tested for membership with `in`.

The important difference is that these behaviors are computed from the arithmetic descriptor. The `range` object does not need a stored array of produced integers in order to act like an integer sequence.

A `range` is sequence-like in behavior, but descriptor-like in storage.

This makes `range` a good bridge object. It looks simple at the surface, but it already introduces lazy computation, virtual indexing, efficient membership testing, immutability, and reusable traversal.

Virtual Indexing Lookups: How the range Object Computes Values Arithmetically on Demand

Indexing a list retrieves a reference from a stored element slot. Indexing a `range` computes the requested value from the stored `start`, `stop`, and `step` information.

```

nums = range(10, 20, 2)

print(f"nums: {nums}")
print(f"type(nums): {type(nums)}")
print(f"dir(nums) sample: {dir(nums)[:12]}")

print(f"nums.start: {nums.start}")
print(f"nums.stop: {nums.stop}")
print(f"nums.step: {nums.step}")

```

```
print(f"nums[0]: {nums[0]}")
print(f"nums[1]: {nums[1]}")
print(f"nums[2]: {nums[2]}")
print(f"nums[3]: {nums[3]}")
```

Possible output:

```
nums: range(10, 20, 2)
type(nums): <class 'range'>
dir(nums) sample: ['__bool__', '__class__', '__contains__', '__delattr__',
 '__dir__', '__doc__', '__eq__', '__format__', '__ge__',
 '__getattribute__', '__getitem__', '__gt__']
nums.start: 10
nums.stop: 20
nums.step: 2
nums[0]: 10
nums[1]: 12
nums[2]: 14
nums[3]: 16
```

The lookup rule is:

$$\text{value} = \text{start} + \text{index} * \text{step}$$

So for `range(10, 20, 2)`:

```
index 0 -> 10 + 0 * 2 -> 10
index 1 -> 10 + 1 * 2 -> 12
index 2 -> 10 + 2 * 2 -> 14
index 3 -> 10 + 3 * 2 -> 16
```

The `__getitem__()` method is the special method behind indexing:

```
nums = range(10, 20, 2)

print(f"nums[4]: {nums[4]}")
print(f"nums.__getitem__(4): {nums.__getitem__(4)}")
```

Possible output:

```
nums[4]: 18
nums.__getitem__(4): 18
```

Negative indexing also works, because the range has a computable length:

```
nums = range(10, 20, 2)

print(f"len(nums): {len(nums)}")
print(f"nums[-1]: {nums[-1]}")
print(f"nums[-2]: {nums[-2]}")
```

Possible output:

```
len(nums): 5
nums[-1]: 18
nums[-2]: 16
```

The negative index is translated into a positive position relative to the sequence length:

```
nums[-1] -> nums[len(nums) - 1] -> nums[4]
nums[-2] -> nums[len(nums) - 2] -> nums[3]
```

Out-of-range indexing still fails:

```
nums = range(10, 20, 2)

try:
    print(nums[99])
except IndexError as err:
    print(f"type(err): {type(err)}")
    print(f"message: {err}")
```

Possible output:

```
type(err): <class 'IndexError'>
message: range object index out of range
```

Slicing a `range` does not materialize a list. It produces another `range` object.

```
nums = range(0, 20, 2)
part = nums[2:7]

print(f"nums: {nums}")
print(f"part: {part}")
print(f"type(part): {type(part)}")
print(f"list(part): {list(part)}")
```

Possible output:

```
nums: range(0, 20, 2)
part: range(4, 14, 2)
type(part): <class 'range'>
list(part): [4, 6, 8, 10, 12]
```

This confirms the same invariant again: the result of slicing is still descriptive until it is explicitly materialized with `list()`.

Arithmetic Membership Testing: Why Integer Containment in `range` Can Be Checked Without Linear Scanning

The expression:

```
x in range_obj
```

does not necessarily mean that Python must walk through every value one by one. For integer membership in a `range`, Python can use arithmetic.

For a value to belong to a positive-step range, three facts must hold:

- the value must be greater than or equal to `start`;
- the value must be before `stop`;
- the distance from `start` must be exactly divisible by `step`.

Example:

```
nums = range(10, 20, 2)

print(f"nums: {nums}")
print(f"12 in nums: {12 in nums}")
print(f"13 in nums: {13 in nums}")
print(f"20 in nums: {20 in nums}")
```

Possible output:

```
nums: range(10, 20, 2)
12 in nums: True
13 in nums: False
20 in nums: False
```

The value 12 is present because:

```
12 is inside the interval 10 <= value < 20
12 - 10 = 2
2 is divisible by step 2
```

The value 13 is not present because:

```
13 is inside the interval 10 <= value < 20
13 - 10 = 3
3 is not divisible by step 2
```

The value 20 is not present because `range()` stops before the stop boundary.

The `__contains__()` method is the special method behind membership testing:

```
nums = range(10, 20, 2)

print(f"nums.__contains__(12): {nums.__contains__(12)}")
print(f"nums.__contains__(13): {nums.__contains__(13)}")
```

Possible output:

```
nums.__contains__(12): True
nums.__contains__(13): False
```

This matters for large ranges:

```
nums = range(0, 1_000_000_000, 3)

print(f"nums: {nums}")
print(f"999_999_999 in nums: {999_999_999 in nums}")
print(f"999_999_998 in nums: {999_999_998 in nums}")
```

Possible output:

```
nums: range(0, 1000000000, 3)
999_999_999 in nums: True
999_999_998 in nums: False
```

Python does not need to generate one billion possible values to answer these questions. For integer containment, it can decide from the arithmetic descriptor.

Negative steps follow the same principle, with the interval direction reversed:

```
nums = range(10, 0, -2)

print(f"nums: {nums}")
print(f"list(nums): {list(nums)}")
print(f"8 in nums: {8 in nums}")
print(f"7 in nums: {7 in nums}")
print(f"0 in nums: {0 in nums}")
```

Possible output:

```
nums: range(10, 0, -2)
list(nums): [10, 8, 6, 4, 2]
8 in nums: True
7 in nums: False
0 in nums: False
```

Again, the stop boundary is excluded. The value 0 is the boundary, not a produced element.

The important control-flow implication is that `range` is not only memory-efficient. It also supports efficient decision checks. A condition such as this:

```
n = 42

if n in range(0, 100, 2):
    print("n is an accepted even value")
else:
    print("n is outside the accepted sequence")
```

Possible output:

```
n is an accepted even value
```

is not equivalent to manually scanning a list of values. For integer membership, the range can answer the containment question arithmetically.

Immutability and Reusability: Using a Single Range Descriptor Across Multiple Traversal Pipelines

A `range` object is immutable. Its `start`, `stop`, and `step` attributes cannot be changed after construction, and its virtual elements cannot be assigned.

```
nums = range(0, 5)

print(f"nums: {nums}")
print(f"type(nums): {type(nums)}")
print(f"nums.start: {nums.start}")
print(f"nums.stop: {nums.stop}")
print(f"nums.step: {nums.step}")
```

Possible output:


```
nums: range(0, 5)
type(nums): <class 'range'>
nums.start: 0
nums.stop: 5
nums.step: 1
```

Trying to change an element fails:

```
nums = range(0, 5)

try:
    nums[0] = 99
except TypeError as err:
    print(f"type(err): {type(err)}")
    print(f"message: {err}")
```

Possible output:

```
type(err): <class 'TypeError'>
message: 'range' object does not support item assignment
```

Trying to change an attribute also fails:

```
nums = range(0, 5)

try:
    nums.start = 100
except AttributeError as err:
    print(f"type(err): {type(err)}")
    print(f"message: {err}")
```

Possible output:

```
type(err): <class 'AttributeError'>
message: readonly attribute
```

The range object has no list-like mutation methods such as `append()`:

```
nums = range(0, 5)

print(f"has append: {'append' in dir(nums)}")
print(f"has __getitem__: {'__getitem__' in dir(nums)}")
print(f"has __contains__: {'__contains__' in dir(nums)}")
print(f"has __iter__: {'__iter__' in dir(nums)}")
```

Possible output:

```
has append: False
has __getitem__: True
has __contains__: True
has __iter__: True
```

So a `range` is not a mutable container. It is an immutable arithmetic descriptor with sequence behavior.

This immutability supports safe reuse. The same `range` object can be traversed several times:

```
nums = range(1, 4)

print("first traversal")
for n in nums:
    print(f"n: {n}")

print("second traversal")
for n in nums:
    print(f"n: {n}")
```

Possible output:

```
first traversal
n: 1
n: 2
n: 3
second traversal
n: 1
n: 2
n: 3
```

This works because the `range` object is iterable, not an already-consumed iterator. Each new `for` loop asks the `range` object for a fresh iterator.

This can be shown explicitly:

```
nums = range(1, 4)

it1 = iter(nums)
it2 = iter(nums)

print(f"type(it1): {type(it1)}")
print(f"type(it2): {type(it2)}")
print(f"it1 is it2: {it1 is it2}")

print(f"next(it1): {next(it1)}")
print(f"next(it1): {next(it1)}")

print(f"next(it2): {next(it2)}")
```

Possible output:

```
type(it1): <class 'range_iterator'>
type(it2): <class 'range_iterator'>
it1 is it2: False
next(it1): 1
next(it1): 2
next(it2): 1
```

The two iterators are separate traversal state machines over the same immutable descriptor. Advancing `it1` does not advance `it2`.

This distinction will matter later when iterator exhaustion is discussed. For now, the key difference is:

- `range` is reusable because it can produce new iterators.
- a specific `range_iterator` is consumed as it advances.

A range can therefore be shared across several traversal pipelines:

```
steps = range(1, 6)

print("plain traversal")
for n in steps:
    print(f"n: {n}")

print("squared values")
for n in steps:
    print(f"{n} squared is {n * n}")

print("formatted table")
for n in steps:
    print(f"{n:>2} | {n * 42:>3}")
```

Possible output:

```
plain traversal
n: 1
n: 2
n: 3
n: 4
n: 5
squared values
1 squared is 1
2 squared is 4
3 squared is 9
4 squared is 16
5 squared is 25
formatted table
 1 |  42
 2 |  84
 3 | 126
 4 | 168
 5 | 210
```

No traversal changes the `steps` object. The descriptor remains the same.
The practical rule is:

A **range** object is a stable, immutable description of an arithmetic sequence. Traversing it creates iterator state, but the range object itself is not consumed.

This is why **range** is so useful in control-flow code. It supplies repeatable arithmetic traversal without materializing a list and without changing itself during traversal.

6.5 Iterative Loops and the Iterator Protocol

Loops are the control-flow structures that make repeated execution possible. A conditional statement chooses one route once. A loop creates a route that can return to an earlier point and execute a block again. This repeated route may be controlled by a condition, as in **while**, or by a traversal source, as in **for**.

Python's loop model is closely connected to its object model. A **for** loop is not merely a counter mechanism. It is a protocol-driven traversal system: Python asks an iterable object for an iterator, repeatedly asks that iterator for the next value, and stops when the iterator reports completion.

This section separates the two major loop families. The **while** loop is studied as condition-controlled indefinite iteration. The **for** loop is studied as object-driven definite traversal. From there, the section explains the difference between iterable objects and iterator objects, why iterators can become exhausted, how **break** and **continue** alter loop routes, and why Python's loop-else clause means normal completion rather than ordinary conditional failure.

6.5.1 Indefinite Iteration: The while Loop

A **while** loop is Python's direct mechanism for indefinite iteration. The word indefinite does not mean uncontrolled. It means that the number of repetitions is not fixed by the loop syntax itself. The loop continues as long as its condition keeps evaluating as true.

This is different from a **for** loop over a known traversal source such as a list, tuple, string, or **range**. A **while** loop does not ask an iterable for the next value. It repeatedly asks a condition whether the loop should continue.

A **while** loop is a repeated conditional branch with a back-edge to the condition test.

The programmer is responsible for making sure that something inside or around the loop eventually changes the condition, unless the loop is intentionally meant to run forever.

Syntax Mechanics and Condition Evaluation Pipelines

The syntax of a **while** loop has three visible parts:

```
while condition:
    loop_body
```

The **while** keyword introduces the repeated decision point. The condition is evaluated before each possible execution of the body. The colon announces the block. The indented statements form the body that executes when the condition is true.

A small example:

```
count = 1

while count <= 3:
    print(f"count: {count}")
    count = count + 1

print("loop finished")
```

Possible output:

```
count: 1
count: 2
count: 3
loop finished
```

The execution route is:

```
set count = 1
test count <= 3
true:
    print count
    increase count
    go back to test count <= 3
false:
    continue after the loop
```

The condition is not checked only once. It is checked before every iteration.

The condition expression produces a routing decision. In this example, the comparison `count <= 3` produces a `bool` object:

```
count = 1
check = count <= 3

print(f"check: {check}")
print(f"type(check): {type(check)}")
print(f"dir(check) sample: {dir(check)[:4]}")
print(f"check.__bool__(): {check.__bool__()}")
```

Possible output:

```
check: True
type(check): <class 'bool'>
dir(check) sample: ['__abs__', '__add__', '__and__', '__bool__']
check.__bool__(): True
```

The object `True` tells the loop to enter the body. When the comparison later produces `False`, the loop body is skipped and execution continues after the loop.

A `while` condition does not have to be a comparison. Any object can be truth-tested:

```
lines = ["Ni!", "Spam!", "Giddy up!"]

while lines:
    line = lines.pop(0)
    print(f"line: {line}")

print("no lines left")
```

Possible output:

```
line: Ni!
line: Spam!
line: Giddy up!
no lines left
```

Here the condition is the list object `lines`. A non-empty list is true. An empty list is false.

```
lines = ["Ni!", "Spam!"]

print(f"type(lines): {type(lines)}")
print(f"dir(lines) sample: {dir(lines)[:10]}")
print(f"len(lines): {len(lines)}")
print(f"bool(lines): {bool(lines)}")
print(f"has __len__: {'__len__' in dir(lines)}")
print(f"has pop: {'pop' in dir(lines)}")
```

Possible output:

```
type(lines): <class 'list'>
dir(lines) sample: ['__add__', '__class__', '__class_getitem__', '__contains__', '__delattr__',
len(lines): 2
bool(lines): True
has __len__: True
has pop: True
```

The `__len__()` method supports length-based truth-value testing. The `pop()` method removes and returns an element. In this loop, every call to `pop(0)` makes the list shorter. Eventually the list becomes empty, and the condition becomes false.

The condition pipeline is therefore:

```
evaluate condition expression
-> truth-test resulting object
   -> true: execute loop body
   -> false: skip loop body and exit loop
```

A `while` loop may also begin false and never execute its body:

```
count = 5

while count < 3:
    print(f"count: {count}")
    count = count + 1

print("body was not entered")
```

Possible output:

```
body was not entered
```

This is because `while` is a pre-test loop. The condition is checked before the first iteration.

Guarding Against Resource Exhaustion: Engineering Manual Exit Conditions and Loop Invariants

A `while` loop can run forever if its condition never becomes false. This is not a special Python problem. It is a direct consequence of the control-flow graph. A loop has a back-edge. If nothing breaks the back-edge, execution keeps returning to the same test.

Example of a dangerous loop:

```
count = 1

while count <= 3:
    print(f"count: {count}")
```

The variable `count` never changes. The condition `count <= 3` remains true forever. The program keeps printing until it is externally interrupted or until some resource limit is reached.

A safe `while` loop usually needs a loop invariant and a progress operation.

A loop invariant is a fact that should remain controlled and meaningful while the loop runs. A progress operation is the update that moves the loop toward termination.

```
count = 1
limit = 3

while count <= limit:
    print(f"count: {count}, limit: {limit}")
    count = count + 1

print("safe exit")
```

Possible output:

```
count: 1, limit: 3
count: 2, limit: 3
count: 3, limit: 3
safe exit
```

In this example:

- `limit` stays fixed;
- `count` starts below or equal to the limit;
- each iteration increases `count`;
- eventually `count <= limit` becomes false.

The invariant is not a magic formal object. It is the programmer's controlled statement about what should remain true while the loop operates.

For this loop, a useful plain-language invariant is:

At the start of each iteration, `count` is the next number to process, and all smaller numbers from the starting point have already been processed.

The progress operation is:

```
count = count + 1
```

Without that line, the loop has no internal route toward termination.

A second common safety pattern is a manual attempt counter:

```
attempt = 1
max_attempts = 3
password_ok = False

while not password_ok and attempt <= max_attempts:
    print(f"attempt {attempt}: checking password")
    attempt = attempt + 1

print("login loop stopped")
```

Possible output:

```
attempt 1: checking password
attempt 2: checking password
attempt 3: checking password
login loop stopped
```

The condition has two parts:

```
not password_ok
attempt <= max_attempts
```

The loop continues only while the password is not accepted and the attempt limit has not been exceeded.

The state objects are ordinary runtime objects:

```

attempt = 1
max_attempts = 3
password_ok = False

print(f"type(attempt): {type(attempt)}")
print(f"dir(attempt) sample: {dir(attempt)[:8]}")

print(f"type(password_ok): {type(password_ok)}")
print(f"dir(password_ok) sample: {dir(password_ok)[:8]}")
print(f"password_ok.__bool__(): {password_ok.__bool__()}")

```

Possible output:

```

type(attempt): <class 'int'>
dir(attempt) sample: ['__abs__', '__add__', '__and__', '__bool__', '__ceil__', '__class__', '__del
type(password_ok): <class 'bool'>
dir(password_ok) sample: ['__abs__', '__add__', '__and__', '__bool__', '__ceil__', '__class__', '
password_ok.__bool__(): False

```

The integer object supports arithmetic methods such as `__add__()`, which is used by `attempt +`

1. The boolean object supports `__bool__()`, which participates in the condition.

A third useful safety technique is to place a clear stop condition near the top of the loop body:

```

energy = 5

while True:
    print(f"energy: {energy}")

    if energy <= 0:
        print("No soup for the loop.")
        break

    energy = energy - 2

print("loop ended")

```

Possible output:

```

energy: 5
energy: 3
energy: 1
energy: -1
No soup for the loop.
loop ended

```

This form uses `while True:` to create an intentionally open loop, then uses `break` to leave it. The detailed mechanics of `break` will be studied later. Here, the important point is that even an apparently infinite loop must have a deliberate exit route if it is not meant to run forever.

A safe `while` loop should usually answer these questions:

- What state does the condition inspect?
- What state changes during each iteration?
- Why must the condition eventually become false?

- What prevents the loop from consuming unbounded time, memory, input, or output?

For example, this loop has bounded output:

```
n = 1

while n <= 4:
    print(f"{n:>2} | {n * 42:>3}")
    n = n + 1

print("table complete")
```

Possible output:

```
1 | 42
2 | 84
3 | 126
4 | 168
table complete
```

The formatting is not part of the loop logic, but it helps expose the loop state. The condition controls how many rows can be produced. The update moves the loop to the next row. The fixed limit prevents unbounded printing.

The practical rule is:

A **while** loop is safe when its continuation condition is tied to state that moves predictably toward an exit.

This is the programmer's responsibility. Python supplies the syntax, the truth-value test, and the back-edge. It does not automatically prove that the back-edge will eventually stop.

6.5.2 Definite Iteration: The for Loop

A **for** loop is Python's main syntax for definite iteration. Definite does not mean that the number of iterations must be written as a literal number. It means that the loop receives its values from a traversal source: a string, list, tuple, range, or any other iterable object.

A **while** loop repeats while a condition remains true. A **for** loop repeats by requesting successive values from an object that knows how to provide traversal.

A **for** loop does not count by itself. It asks an iterable object for values until that iterable's iterator reports that no more values exist.

The ordinary syntax is:

```
for name in iterable:
    loop_body
```

The name on the left is rebound for each produced value. The object on the right supplies the traversal stream.

Abstracting Sequential Traversal: Structural Syntax over Strings, Lists, Tuples, and Range Objects

The strength of the `for` loop is that the same syntax works over many different object types. The loop body does not need to know whether the values came from a string, a list, a tuple, or a `range`. It only receives one value at a time.

A string traversal produces characters:

```
text = "Ni!"

print(f"type(text): {type(text)}")
print(f"dir(text) sample: {dir(text)[:10]}")
print(f"len(text): {len(text)}")

for ch in text:
    print(f"character: {ch!r}")
```

Possible output:

```
type(text): <class 'str'>
dir(text) sample: ['__add__', '__class__', '__contains__', '__delattr__',
 '__dir__', '__doc__', '__eq__', '__format__', '__ge__', '__getattr__']
len(text): 3
character: 'N'
character: 'i'
character: '!'
```

The string object supports traversal because it is iterable. Its `__iter__()` method can produce an iterator over its characters. Its `__len__()` method gives its length, and `__getitem__()` supports indexed access.

A list traversal produces stored elements:

```
quotes = [
    "No soup for you!",
    "Giddy up!",
    "D'oh!",
]

print(f"type(quotes): {type(quotes)}")
print(f"dir(quotes) sample: {dir(quotes)[:10]}")
print(f"len(quotes): {len(quotes)}")
print(f"has append: {'append' in dir(quotes)}")

for quote in quotes:
    print(f"quote: {quote}")
```

Possible output:

```
type(quotes): <class 'list'>
dir(quotes) sample: ['__add__', '__class__', '__class_getitem__',
 '__contains__', '__delattr__', '__delitem__', '__dir__', '__doc__',
 '__eq__', '__format__']
len(quotes): 3
has append: True
quote: No soup for you!
quote: Giddy up!
quote: D'oh!
```

A list is a mutable container. Its `append()` method can add elements, its `__getitem__()` method supports indexing, and its `__iter__()` method can produce a traversal object.

A tuple traversal looks similar, but the tuple itself is immutable:

```
facts = (
    "Chuck Norris counted to infinity.",
    "Twice.",
    42,
)

print(f"type(facts): {type(facts)}")
print(f"dir(facts) sample: {dir(facts)[:10]}")
print(f"len(facts): {len(facts)}")
print(f"has append: {'append' in dir(facts)}")

for fact in facts:
    print(f"fact: {fact!r}")
```

Possible output:

```
type(facts): <class 'tuple'>
dir(facts) sample: ['__add__', '__class__', '__class_getitem__',
 '__contains__', '__delattr__', '__dir__', '__doc__', '__eq__',
 '__format__', '__ge__']
len(facts): 3
has append: False
fact: 'Chuck Norris counted to infinity.'
fact: 'Twice.'
fact: 42
```

The tuple can be traversed, but it cannot be structurally modified like a list. Traversal and mutability are separate properties. An object may be iterable without being mutable.

A `range` traversal produces computed integers:

```
steps = range(1, 4)

print(f"type(steps): {type(steps)}")
print(f"dir(steps) sample: {dir(steps)[:10]}")
print(f"steps.start: {steps.start}")
print(f"steps.stop: {steps.stop}")
print(f"steps.step: {steps.step}")

for step in steps:
    print(f"step {step}: And now for something completely different.")
```

Possible output:

```
type(steps): <class 'range'>
dir(steps) sample: ['__bool__', '__class__', '__contains__',
 '__delattr__', '__dir__', '__doc__', '__eq__', '__format__',
 '__ge__', '__getattribute__']
steps.start: 1
steps.stop: 4
steps.step: 1
step 1: And now for something completely different.
step 2: And now for something completely different.
step 3: And now for something completely different.
```

The `range` object does not store the produced integers as a list. It stores the arithmetic rule. The `for` loop does not care. It only receives the next produced value.

The same loop shape therefore covers several storage models:

```
for x in "abc":
    print(x)

for x in ["a", "b", "c"]:
    print(x)

for x in ("a", "b", "c"):
    print(x)

for x in range(3):
    print(x)
```

The surface syntax is the same. The traversal source changes.

The loop variable is rebound on each iteration:

```
items = ["soup", "pretzels", "muffin"]

for item in items:
    print(f"inside loop item: {item}")

print(f"after loop item: {item}")
```

Possible output:

```
inside loop item: soup
inside loop item: pretzels
inside loop item: muffin
after loop item: muffin
```

The name `item` is not a new private slot created only for the loop body. It is a normal Python name in the surrounding scope. After the loop finishes, it remains bound to the last value produced by the traversal.

This is important for programmers coming from C-style loop headers. Python's `for` loop is not primarily:

```
initialize counter
test counter
increment counter
```

It is primarily:

```
obtain iterator from iterable
ask iterator for next value
bind value to loop name
execute body
repeat until no more values exist
```

When numeric positions are needed, `range()` supplies them:

```
names = ["Jerry", "Elaine", "Kramer"]

for i in range(len(names)):
    print(f"{i:>2} | {names[i]}")
```

Possible output:

```
0 | Jerry
1 | Elaine
2 | Kramer
```

This works, but if both index and value are needed, Python commonly uses `enumerate()`, which will be examined later.

For now, the basic rule is:

A `for` loop abstracts traversal. It hides the repeated calls needed to fetch successive values from the source object.

Under the Hood: How CPython Implicitly Calls `iter()` and `next()` Until `StopIteration`

The `for` loop is built on the iterator protocol. At the language level, this means two operations matter:

- `iter(obj)` asks an iterable object for an iterator.
- `next(iterator)` asks that iterator for the next value.

When the iterator has no more values, it raises `StopIteration`. The `for` loop catches that signal internally and exits normally.

A normal `for` loop:

```
items = ["soup", "pretzels", "muffin"]

for item in items:
    print(f"item: {item}")

print("loop finished")
```

Possible output:

```
item: soup
item: pretzels
item: muffin
loop finished
```

A manual version using `iter()` and `next()`:

```
items = ["soup", "pretzels", "muffin"]

it = iter(items)

while True:
    try:
        item = next(it)
    except StopIteration:
        break

    print(f"item: {item}")

print("loop finished")
```

Possible output:

```

item: soup
item: pretzels
item: muffin
loop finished

```

The two versions have the same traversal result. The `for` loop is the clean syntax. The manual version exposes the mechanism.

The iterator object is separate from the list:

```

items = ["soup", "pretzels", "muffin"]
it = iter(items)

print(f"type(items): {type(items)}")
print(f"type(it): {type(it)}")
print(f"items is it: {items is it}")

print(f"has __iter__ on items: {'__iter__' in dir(items)}")
print(f"has __next__ on items: {'__next__' in dir(items)}")

print(f"has __iter__ on it: {'__iter__' in dir(it)}")
print(f"has __next__ on it: {'__next__' in dir(it)}")

```

Possible output:

```

type(items): <class 'list'>
type(it): <class 'list_iterator'>
items is it: False
has __iter__ on items: True
has __next__ on items: False
has __iter__ on it: True
has __next__ on it: True

```

The list is iterable because it has `__iter__()`. The list iterator is an iterator because it has `__next__()`. The iterator stores traversal position. The list stores the elements.

Calling `next()` advances the iterator:

```

items = ["soup", "pretzels", "muffin"]
it = iter(items)

print(f"next(it): {next(it)}")
print(f"next(it): {next(it)}")
print(f"next(it): {next(it)}")

```

Possible output:

```

next(it): soup
next(it): pretzels
next(it): muffin

```

One more call raises `StopIteration`:

```

items = ["soup"]
it = iter(items)

print(f"next(it): {next(it)}")

```

```

try:
    print(next(it))
except StopIteration as err:
    print(f"type(err): {type(err)}")
    print(f"message: {err}")
    print(f"dir(err) sample: {dir(err)[:8]}")

```

Possible output:

```

next(it): soup
type(err): <class 'StopIteration'>
message:
dir(err) sample: ['__cause__', '__class__', '__context__', '__delattr__',
'__dict__', '__dir__', '__doc__', '__eq__']

```

The `StopIteration` object is an exception object. Its role here is not to report a program failure to the user. It is the internal signal that traversal is complete. A `for` loop handles that signal automatically.

The rough expansion of a `for` loop is therefore:

```

_iterator = iter(iterable)

while True:
    try:
        target_name = next(_iterator)
    except StopIteration:
        break

    loop_body

```

CPython implements this through bytecode instructions rather than literally rewriting the source into this code. Conceptually, however, the protocol is the same: get an iterator, request values, stop when `StopIteration` appears.

A `range` object follows the same mechanism:

```

nums = range(1, 4)
it = iter(nums)

print(f"type(nums): {type(nums)}")
print(f"type(it): {type(it)}")

print(f"next(it): {next(it)}")
print(f"next(it): {next(it)}")
print(f"next(it): {next(it)}")

```

Possible output:

```

type(nums): <class 'range'>
type(it): <class 'range_iterator'>
next(it): 1
next(it): 2
next(it): 3

```

The source object is a `range`. The traversal object is a `range_iterator`. The range remains reusable; the iterator advances.

A tuple also produces a separate iterator:

```
data = ("red", "green", "blue")
it = iter(data)

print(f"type(data): {type(data)}")
print(f"type(it): {type(it)}")

print(f"next(it): {next(it)}")
print(f"next(it): {next(it)}")
```

Possible output:

```
type(data): <class 'tuple'>
type(it): <class 'tuple_iterator'>
next(it): red
next(it): green
```

For strings, the exact iterator type name may differ between Python versions and builds, but the protocol remains the same:

```
text = "abc"
it = iter(text)

print(f"type(text): {type(text)}")
print(f"type(it): {type(it)}")

print(f"has __next__: {'__next__' in dir(it)}")

print(f"next(it): {next(it)}")
print(f"next(it): {next(it)}")
print(f"next(it): {next(it)}")
```

Possible output:

```
type(text): <class 'str'>
type(it): <class 'str_ascii_iterator'>
has __next__: True
next(it): a
next(it): b
next(it): c
```

On some versions, the displayed iterator class name may be different, such as `str_iterator`. The important property is not the exact printed class name. The important property is that the object returned by `iter(text)` has `__next__()` and advances one character at a time.

The `for` loop hides this repeated protocol call:

```
text = "abc"

for ch in text:
    print(f"ch: {ch}")
```

Possible output:

```
ch: a
ch: b
ch: c
```


The loop performs the repeated `next()` calls and exits cleanly when the iterator raises `StopIteration`. This gives the exact mental model needed for later sections:

An iterable can produce an iterator. An iterator produces values one at a time. A `for` loop connects these two operations and treats `StopIteration` as normal loop completion.

6.5.3 Iterable vs. Iterator

The words *iterable* and *iterator* are closely related, but they do not name the same thing. Confusing them makes Python loops look more mysterious than they really are.

An iterable is an object that can provide a traversal object. An iterator is the traversal object itself. The iterable is the source. The iterator is the moving cursor over that source.

An iterable can be asked to begin traversal. An iterator is the object that performs the traversal step by step.

This distinction explains why a list, tuple, string, and `range` can be traversed many times, while a specific iterator is consumed as it advances.

Iterable Objects: Objects That Can Produce an Iterator via `iter()`

An iterable object is an object that can be passed to `iter()`. If the call succeeds, Python receives an iterator that can produce values one at a time.

Examples of common iterable objects include strings, lists, tuples, dictionaries, sets, and `range` objects. In this chapter, the most important examples are strings, lists, tuples, and ranges.

```
text = "Ni!"
items = ["soup", "pretzels", "muffin"]
coords = (10, 20, 30)
steps = range(1, 4)

print(f"type(text): {type(text)}")
print(f"type(items): {type(items)}")
print(f"type(coords): {type(coords)}")
print(f"type(steps): {type(steps)}")

print(f"has __iter__ text: {'__iter__' in dir(text)}")
print(f"has __iter__ items: {'__iter__' in dir(items)}")
print(f"has __iter__ coords: {'__iter__' in dir(coords)}")
print(f"has __iter__ steps: {'__iter__' in dir(steps)}")
```

Possible output:

```
type(text): <class 'str'>
type(items): <class 'list'>
type(coords): <class 'tuple'>
type(steps): <class 'range'>
has __iter__ text: True
has __iter__ items: True
has __iter__ coords: True
has __iter__ steps: True
```

The presence of `__iter__` is the important signal here. It means the object can produce an iterator.

```

items = ["soup", "pretzels", "muffin"]

it = iter(items)

print(f"items: {items}")
print(f"type(items): {type(items)}")
print(f"type(it): {type(it)}")
print(f"items is it: {items is it}")

```

Possible output:

```

items: ['soup', 'pretzels', 'muffin']
type(items): <class 'list'>
type(it): <class 'list_iterator'>
items is it: False

```

The list and the iterator are different objects. The list stores the elements. The iterator stores the traversal position.

The same iterable can produce more than one iterator:

```

items = ["soup", "pretzels", "muffin"]

it1 = iter(items)
it2 = iter(items)

print(f"type(it1): {type(it1)}")
print(f"type(it2): {type(it2)}")
print(f"it1 is it2: {it1 is it2}")

print(f"next(it1): {next(it1)}")
print(f"next(it1): {next(it1)}")
print(f"next(it2): {next(it2)}")

```

Possible output:

```

type(it1): <class 'list_iterator'>
type(it2): <class 'list_iterator'>
it1 is it2: False
next(it1): soup
next(it1): pretzels
next(it2): soup

```

Advancing `it1` does not advance `it2`. They are separate traversal objects over the same list.

A `range` behaves the same way:

```

steps = range(1, 4)

it1 = iter(steps)
it2 = iter(steps)

print(f"type(steps): {type(steps)}")
print(f"type(it1): {type(it1)}")
print(f"steps is it1: {steps is it1}")

print(f"next(it1): {next(it1)}")
print(f"next(it2): {next(it2)}")

```

Possible output:

```
type(steps): <class 'range'>
type(it1): <class 'range_iterator'>
steps is it1: False
next(it1): 1
next(it2): 1
```

The `range` object remains a compact arithmetic descriptor. Each call to `iter(steps)` creates a fresh iterator over that descriptor.

For an iterable, some important methods are:

- `__iter__()`: produces an iterator;
- `__len__()`: often gives the number of elements or described values;
- `__getitem__()`: often supports indexed access, depending on the object type;
- `__contains__()`: often supports membership tests with `in`.

Example with a tuple:

```
coords = (10, 20, 30)

print(f"type(coords): {type(coords)}")
print(f"dir(coords) sample: {dir(coords)[:12]}")

print(f"len(coords): {len(coords)}")
print(f"coords[1]: {coords[1]}")
print(f"20 in coords: {20 in coords}")

print(f"coords.__iter__(): {coords.__iter__()}")
print(f"coords.__len__(): {coords.__len__()}")
print(f"coords.__getitem__(1): {coords.__getitem__(1)}")
print(f"coords.__contains__(20): {coords.__contains__(20)}")
```

Possible output:

```
type(coords): <class 'tuple'>
dir(coords) sample: ['__add__', '__class__', '__class_getitem__',
 '__contains__', '__delattr__', '__dir__', '__doc__', '__eq__',
 '__format__', '__ge__', '__getattr__', '__getitem__']
len(coords): 3
coords[1]: 20
20 in coords: True
coords.__iter__(): <tuple_iterator object at 0x...>
coords.__len__(): 3
coords.__getitem__(1): 20
coords.__contains__(20): True
```

The exact memory address printed for the iterator will differ between executions. The important fact is that `coords.__iter__()` returns a `tuple_iterator` object.

Iterator Objects: Objects That Return Successive Values via `next()`

An iterator is an object that can return the next value in a traversal. It does this through `__next__()`, which is called by the built-in `next()` function.

```
items = ["soup", "pretzels", "muffin"]
it = iter(items)

print(f"type(it): {type(it)}")
print(f"dir(it) sample: {dir(it)[:12]}")
print(f"has __iter__: {'__iter__' in dir(it)}")
print(f"has __next__: {'__next__' in dir(it)}")

print(f"next(it): {next(it)}")
print(f"next(it): {next(it)}")
print(f"next(it): {next(it)}")
```

Possible output:

```
type(it): <class 'list_iterator'>
dir(it) sample: ['__class__', '__delattr__', '__dir__', '__doc__',
 '__eq__', '__format__', '__ge__', '__getattribute__', '__getstate__',
 '__gt__', '__hash__', '__init__']
has __iter__: True
has __next__: True
next(it): soup
next(it): pretzels
next(it): muffin
```

Each call to `next(it)` advances the iterator. The iterator remembers where it is. That position is internal state.

The special method call is visible directly:

```
items = ["soup", "pretzels"]
it = iter(items)

print(f"it.__next__(): {it.__next__()}")
print(f"next(it): {next(it)}")
```

Possible output:

```
it.__next__(): soup
next(it): pretzels
```

The built-in `next(it)` and the direct call `it.__next__()` advance the same iterator. They are not separate traversal channels.

An iterator also has `__iter__()`, but for an iterator this method usually returns the iterator itself:

```
items = ["soup", "pretzels"]
it = iter(items)

same_it = iter(it)

print(f"type(it): {type(it)}")
print(f"type(same_it): {type(same_it)}")
print(f"it is same_it: {it is same_it}")
```

```
print(f"next(it): {next(it)}")
print(f"next(same_it): {next(same_it)}")
```

Possible output:

```
type(it): <class 'list_iterator'>
type(same_it): <class 'list_iterator'>
it is same_it: True
next(it): soup
next(same_it): pretzels
```

This is a crucial distinction:

- Calling `iter()` on a list creates a new iterator.
- Calling `iter()` on a list iterator returns the same iterator.

That is why iterators are not automatically reusable.

The same idea appears with `range` iterators:

```
steps = range(1, 4)
it = iter(steps)

print(f"type(steps): {type(steps)}")
print(f"type(it): {type(it)}")

print(f"iter(steps) is iter(steps): {iter(steps) is iter(steps)}")
print(f"iter(it) is it: {iter(it) is it}")
```

Possible output:

```
type(steps): <class 'range'>
type(it): <class 'range_iterator'>
iter(steps) is iter(steps): False
iter(it) is it: True
```

The `range` object can produce fresh iterators. The `range_iterator` is already the moving traversal object.

This explains the exact mechanism behind a `for` loop:

```
for value in iterable:
    body
```

is conceptually:

```
_iterator = iter(iterable)

while True:
    try:
        value = next(_iterator)
    except StopIteration:
        break

    body
```

The `for` loop does not need a length. It needs an iterator that either produces a next value or raises `StopIteration`.

Iterator Exhaustion: Why Some Traversal Objects Cannot Be Reused After Completion

Iterator exhaustion means that an iterator has reached the end of its traversal stream. Once exhausted, further calls to `next()` keep raising `StopIteration`.

```
items = ["soup", "pretzels"]
it = iter(items)

print(f"next(it): {next(it)}")
print(f"next(it): {next(it)}")

try:
    print(next(it))
except StopIteration as err:
    print(f"type(err): {type(err)}")
    print("iterator is exhausted")
```

Possible output:

```
next(it): soup
next(it): pretzels
type(err): <class 'StopIteration'>
iterator is exhausted
```

The iterator does not automatically restart. It remains at the exhausted position.

```
items = ["soup", "pretzels"]
it = iter(items)

for item in it:
    print(f"first traversal: {item}")

for item in it:
    print(f"second traversal: {item}")

print("done")
```

Possible output:

```
first traversal: soup
first traversal: pretzels
done
```

The second loop prints nothing because it receives the same already-exhausted iterator.

By contrast, looping over the original list twice works:

```
items = ["soup", "pretzels"]

for item in items:
    print(f"first traversal: {item}")

for item in items:
    print(f"second traversal: {item}")
```

Possible output:

```
first traversal: soup
first traversal: pretzels
second traversal: soup
second traversal: pretzels
```

The list is reusable because each `for` loop calls `iter(items)` and receives a fresh iterator. The same difference can be shown with `range`:

```
steps = range(1, 4)

for n in steps:
    print(f"first range traversal: {n}")

for n in steps:
    print(f"second range traversal: {n}")
```

Possible output:

```
first range traversal: 1
first range traversal: 2
first range traversal: 3
second range traversal: 1
second range traversal: 2
second range traversal: 3
```

But if the iterator itself is reused:

```
steps = range(1, 4)
it = iter(steps)

for n in it:
    print(f"first iterator traversal: {n}")

for n in it:
    print(f"second iterator traversal: {n}")

print("finished")
```

Possible output:

```
first iterator traversal: 1
first iterator traversal: 2
first iterator traversal: 3
finished
```

Again, the second traversal prints nothing because the same iterator has already been consumed. This matters when assigning iterators to names:

```
items = ["Jerry", "Elaine", "Kramer"]

it = iter(items)

first = list(it)
second = list(it)

print(f"first: {first}")
print(f"second: {second}")
```

Possible output:

```
first: ['Jerry', 'Elaine', 'Kramer']
second: []
```

The first `list(it)` consumes the iterator. The second `list(it)` receives no remaining values.

The type of the materialized result is separate from the iterator:

```
items = ["Jerry", "Elaine", "Kramer"]
it = iter(items)
first = list(it)

print(f"type(items): {type(items)}")
print(f"type(it): {type(it)}")
print(f"type(first): {type(first)}")

print(f"first: {first}")
print(f"has append on first: {'append' in dir(first)}")
print(f"has __next__ on first: {'__next__' in dir(first)}")
```

Possible output:

```
type(items): <class 'list'>
type(it): <class 'list_iterator'>
type(first): <class 'list'>
first: ['Jerry', 'Elaine', 'Kramer']
has append on first: True
has __next__ on first: False
```

The materialized list is a normal list. It is not an iterator, and it does not continue the original traversal state.

The practical distinction is:

- Store an iterable when you need a reusable traversal source.
- Store an iterator when you intentionally need one moving traversal state.
- Convert an iterator to a list or tuple when you need to preserve all remaining values.

A final example shows the difference clearly:

```
source = range(1, 4)
moving = iter(source)

print("using the source twice")
print(list(source))
print(list(source))

print("using the iterator twice")
print(list(moving))
print(list(moving))
```

Possible output:


```
using the source twice
[1, 2, 3]
[1, 2, 3]
using the iterator twice
[1, 2, 3]
[]
```

The `range` object is reusable. The iterator is consumable.

An iterable is a source of traversal. An iterator is a traversal in progress. Once that traversal reaches the end, it is exhausted.

6.5.4 Interrupting Execution Flow Subsystems

Loops normally follow a stable repetition pattern. A `while` loop returns to its condition. A `for` loop asks its iterator for the next value. This ordinary loop route can be interrupted from inside the loop body.

Python provides two local loop-control statements for this purpose:

- **break**: leave the loop immediately;
- **continue**: skip the rest of the current iteration and move to the next loop decision.

These statements are not expressions. They do not produce values. They alter the control-flow route.

`break` exits the loop. `continue` exits only the current iteration.

Terminating the Iteration Invariant: The Immediate Exit Properties of `break`

The `break` statement terminates the nearest enclosing loop immediately. When Python reaches `break`, it does not finish the remaining statements in the current loop body. It jumps to the first statement after the loop.

```
items = ["soup", "pretzels", "muffin", "coffee"]

for item in items:
    print(f"checking: {item}")

    if item == "muffin":
        print("muffin found, stopping loop")
        break

    print("not the muffin yet")

print("after loop")
```

Possible output:

```
checking: soup
not the muffin yet
checking: pretzels
not the muffin yet
checking: muffin
muffin found, stopping loop
after loop
```

The line:

```
print("not the muffin yet")
```

does not execute for "muffin", because `break` exits the loop before Python reaches that statement. The route is:

```
get next item
-> print checking
-> test item == "muffin"
    true -> print found
        -> break
    -> jump after loop
    false -> print not found
        -> ask for next item
```

The `break` statement exits only the nearest loop:

```
groups = [
    ["Jerry", "Elaine"],
    ["Bart", "Lisa"],
]

for group in groups:
    print(f"group: {group}")

    for name in group:
        print(f"checking name: {name}")

        if name == "Bart":
            print("found Bart, leaving inner loop")
            break

    print("after inner loop")

print("after outer loop")
```

Possible output:

```
group: ['Jerry', 'Elaine']
checking name: Jerry
checking name: Elaine
after inner loop
group: ['Bart', 'Lisa']
checking name: Bart
found Bart, leaving inner loop
after inner loop
after outer loop
```

The `break` exits the inner `for name in group:` loop. It does not exit the outer `for group in groups:` loop.

The same rule applies to `while`:

```
n = 1
```

```
while True:
    print(f"n: {n}")

    if n == 3:
        print("stop at three")
        break

    n = n + 1

print("loop stopped")
```

Possible output:

```
n: 1
n: 2
n: 3
stop at three
loop stopped
```

Here `while True:` creates an intentionally open loop. The actual exit route is inside the body. Without the `break`, the loop would not stop by itself.

This pattern should be used carefully. A `while True:` loop is safe only if the programmer can clearly identify the internal exit path.

A loop containing `break` has an additional exit edge from inside the body to the statement after the loop.

The `break` statement is often used when the loop is searching for something:

```
names = ["Jerry", "Elaine", "Kramer", "Newman"]
target = "Kramer"
found = False

for name in names:
    print(f"checking: {name}")

    if name == target:
        found = True
        print(f"found: {name}")
        break

print(f"found status: {found}")
```

Possible output:

```
checking: Jerry
checking: Elaine
checking: Kramer
found: Kramer
found status: True
```

The list object is the reusable traversal source:

```
names = ["Jerry", "Elaine", "Kramer", "Newman"]
```

```
print(f"type(names): {type(names)}")
print(f"dir(names) sample: {dir(names)[:10]}")
print(f"len(names): {len(names)}")
print(f"has __iter__: {'__iter__' in dir(names)}")
print(f"has __getitem__: {'__getitem__' in dir(names)}")
```

Possible output:

```
type(names): <class 'list'>
dir(names) sample: ['__add__', '__class__', '__class_getitem__',
 '__contains__', '__delattr__', '__delitem__', '__dir__', '__doc__',
 '__eq__', '__format__']
len(names): 4
has __iter__: True
has __getitem__: True
```

The `__iter__()` method lets the list produce an iterator for the loop. The `__getitem__()` method supports indexed access. The `break` statement does not modify the list. It modifies only the route of execution through the loop.

A loop stopped by `break` may leave some values unvisited:

```
nums = range(1, 10)

for n in nums:
    print(f"n: {n}")

    if n == 4:
        print("breaking at 4")
        break

print("done")
```

Possible output:

```
n: 1
n: 2
n: 3
n: 4
breaking at 4
done
```

The values 5, 6, 7, 8, and 9 are not requested from the traversal route. The loop ended early.

Short-Circuiting Current Iterations: The Jump Mechanics of `continue`

The `continue` statement does not exit the whole loop. It exits the current iteration only. When Python reaches `continue`, it skips the rest of the current loop body and returns to the loop's next decision point.

In a `for` loop, that means Python asks the iterator for the next value. In a `while` loop, that means Python re-evaluates the condition.

```
items = ["soup", "", "pretzels", "", "muffin"]

for item in items:
    if item == "":
```

```

        print("empty item skipped")
        continue

    print(f"processing: {item}")

print("loop finished")

```

Possible output:

```

processing: soup
empty item skipped
processing: pretzels
empty item skipped
processing: muffin
loop finished

```

When `item == ""`, the loop prints the skip message and reaches `continue`. The later line:

```
print(f"processing: {item}")
```

is skipped for that iteration only. The loop then moves on to the next item.
The route is:

```

get next item
-> test item == ""
    true  -> print skipped
           -> continue
           -> ask for next item
    false -> print processing
           -> ask for next item

```

The `continue` statement is useful for guard-style filtering. It lets the loop reject cases early and keep the main processing path less nested.

Without `continue`:

```

items = ["soup", "", "pretzels"]

for item in items:
    if item != "":
        print(f"processing: {item}")
    else:
        print("empty item skipped")

```

With `continue`:

```

items = ["soup", "", "pretzels"]

for item in items:
    if item == "":
        print("empty item skipped")
        continue

    print(f"processing: {item}")

```

Both versions can produce the same result. The second version says: reject the unwanted case immediately, then let the rest of the body describe the normal case.

In a `while` loop, `continue` jumps back to the condition test. This means the progress update must still happen before the `continue`, or the loop may become infinite.

Safe version:

```
n = 0

while n < 5:
    n = n + 1

    if n == 3:
        print("skipping three")
        continue

    print(f"n: {n}")

print("done")
```

Possible output:

```
n: 1
n: 2
skipping three
n: 4
n: 5
done
```

The update:

```
n = n + 1
```

happens before the possible `continue`. Therefore the loop still moves toward termination.

Dangerous version:

```
n = 0

while n < 5:
    if n == 0:
        print("stuck before progress")
        continue

    n = n + 1
```

Here `n` starts as 0. The condition `n == 0` is true. Python prints the message and reaches `continue`. The update is skipped. Then Python checks `n < 5` again, finds it still true, and repeats forever.

So the practical rule for `while` loops is:

If a `continue` can run before the loop's progress update, check carefully that the loop still has a route toward termination.

The difference between `break` and `continue` is therefore exact:

```
break:
    leave the nearest loop completely

continue:
    leave the current iteration only
    continue with the next loop decision
```

A final example shows both statements in the same loop:

```
items = ["soup", "", "pretzels", "stop", "muffin"]

for item in items:
    print(f"seen: {item!r}")

    if item == "":
        print("skip empty item")
        continue

    if item == "stop":
        print("stop marker found")
        break

    print(f"processed: {item}")

print("after loop")
```

Possible output:

```
seen: 'soup'
processed: soup
seen: ''
skip empty item
seen: 'pretzels'
processed: pretzels
seen: 'stop'
stop marker found
after loop
```

The empty string causes `continue`, so only that iteration is skipped. The string `"stop"` causes `break`, so the entire loop terminates. The later value `"muffin"` is never reached.

`continue` preserves the loop but shortens the current iteration. `break` destroys the current loop route entirely and resumes after the loop.

6.5.5 Loop-else Completion Semantics

Python allows an `else` clause not only after `if`, but also after `for` and `while`. This surprises many programmers because the word `else` is usually associated with conditional branching. In loop syntax, however, `else` has a different meaning.

A loop-`else` block does not mean “if the loop condition was false at the beginning.” It means: run this block only if the loop finished normally.

For a `for` loop, normal completion means that the iterator was exhausted. For a `while` loop, normal completion means that the loop condition became false. If the loop is terminated by `break`, the `else` block is skipped.

A loop-`else` block runs after normal loop completion, not after early loop interruption.

This makes `loop-else` useful for search patterns: if the loop finds the desired item, it uses `break`; if no item was found and the loop completed naturally, the `else` block reports the unsuccessful search.

The Unique `else` Clause Semantics Applied to `while` and `for` Blocks

The syntax is simple:

```
for item in items:
    loop_body
else:
    completion_body
```

or:

```
while condition:
    loop_body
else:
    completion_body
```

The `else` block is aligned with the loop header, not with an inner `if`. This indentation is essential. The `else` belongs to the loop.

A for example without `break`:

```
items = ["soup", "pretzels", "muffin"]

for item in items:
    print(f"checking: {item}")
else:
    print("loop completed normally")

print("after loop")
```

Possible output:

```
checking: soup
checking: pretzels
checking: muffin
loop completed normally
after loop
```

The iterator produced all values. No `break` interrupted the loop. Therefore the `else` block executed. Now add a `break`:

```
items = ["soup", "pretzels", "muffin"]

for item in items:
    print(f"checking: {item}")

    if item == "pretzels":
        print("pretzels found, leaving loop")
        break
else:
    print("loop completed normally")

print("after loop")
```


Possible output:

```
checking: soup
checking: pretzels
pretzels found, leaving loop
after loop
```

The `else` block did not execute. The loop did not complete by exhausting the iterable. It was interrupted by `break`.

The control-flow shape is:

```
for loop starts
-> get next item
-> execute body
    if break occurs:
        jump after loop and skip else
-> if iterator is exhausted:
    execute else
    continue after loop
```

The same rule applies to `while`:

```
n = 1

while n <= 3:
    print(f'n: {n}')
    n = n + 1
else:
    print("while condition became false")

print("after loop")
```

Possible output:

```
n: 1
n: 2
n: 3
while condition became false
after loop
```

The loop stopped because `n <= 3` became false. This is normal completion, so the `else` block executed.

With `break`:

```
n = 1

while n <= 5:
    print(f'n: {n}')

    if n == 3:
        print("breaking at three")
        break

    n = n + 1
else:
    print("while condition became false")

print("after loop")
```

Possible output:

```
n: 1
n: 2
n: 3
breaking at three
after loop
```

The `else` block was skipped because the loop ended through `break`, not through the condition becoming false.

The loop-`else` rule is therefore not symmetric with `if-else`. In an `if` statement, `else` means “the condition was false.” In a loop, `else` means “the loop was not interrupted by `break`.”

```
if condition:
    runs when condition is true
else:
    runs when condition is false

for item in items:
    runs for each produced item
else:
    runs when traversal finishes without break
```

This is why the name `else` can feel misleading in loops. It is better to read it mentally as:

after normal loop completion

Conditional Execution: Triggering Logic Blocks Only After Non-Interrupted Loop Completion

The most useful application of loop-`else` is search logic.

Without loop-`else`, programmers often use a flag:

```
names = ["Jerry", "Elaine", "Kramer"]
target = "Newman"
found = False

for name in names:
    print(f"checking: {name}")

    if name == target:
        found = True
        print(f"found: {name}")
        break

if not found:
    print("target was not found")
```

Possible output:

```
checking: Jerry
checking: Elaine
checking: Kramer
target was not found
```

The flag `found` records whether the loop ended because the target was found.

The loop-else version removes the flag:

```
names = ["Jerry", "Elaine", "Kramer"]
target = "Newman"

for name in names:
    print(f"checking: {name}")

    if name == target:
        print(f"found: {name}")
        break
else:
    print("target was not found")
```

Possible output:

```
checking: Jerry
checking: Elaine
checking: Kramer
target was not found
```

The meaning is direct: if the loop reaches a match, it breaks; if it never reaches a match, it completes normally and the `else` block runs.

When the target exists:

```
names = ["Jerry", "Elaine", "Kramer"]
target = "Elaine"

for name in names:
    print(f"checking: {name}")

    if name == target:
        print(f"found: {name}")
        break
else:
    print("target was not found")
```

Possible output:

```
checking: Jerry
checking: Elaine
found: Elaine
```

The `else` block is skipped because `break` interrupted the loop.

The list being searched is still an ordinary list object:

```
names = ["Jerry", "Elaine", "Kramer"]

print(f"type(names): {type(names)}")
print(f"dir(names) sample: {dir(names)[:10]}")
print(f"len(names): {len(names)}")
print(f"has __iter__: {'__iter__' in dir(names)}")
print(f"has __contains__: {'__contains__' in dir(names)}")
```

Possible output:

```

type(names): <class 'list'>
dir(names) sample: ['__add__', '__class__', '__class_getitem__',
 '__contains__', '__delattr__', '__delitem__', '__dir__', '__doc__',
 '__eq__', '__format__']
len(names): 3
has __iter__: True
has __contains__: True

```

The `__iter__()` method allows traversal. The `__contains__()` method supports membership testing with `in`. The `loop-else` structure does not change the list. It changes only how the program reacts to the traversal outcome.

The same pattern works with numeric search over a `range`:

```

target = 42

for n in range(1, 50):
    if n == target:
        print(f"found target: {n}")
        break
else:
    print("target was not reached")

```

Possible output:

```
found target: 42
```

If the target is outside the described range:

```

target = 100

for n in range(1, 50):
    if n == target:
        print(f"found target: {n}")
        break
else:
    print("target was not reached")

```

Possible output:

```
target was not reached
```

The `range` object completed traversal normally. No `break` occurred. Therefore the `else` block ran.

A `continue` statement does not suppress the `loop-else`. It interrupts only the current iteration, not the whole loop.

```

items = ["soup", "", "pretzels", ""]

for item in items:
    if item == "":
        print("empty item skipped")
        continue

    print(f"processed: {item}")
else:
    print("loop completed without break")

```

Possible output:

```
processed: soup
empty item skipped
processed: pretzels
empty item skipped
loop completed without break
```

The loop used `continue`, but it did not use `break`. Therefore the `else` block executed. This gives an exact distinction:

- `break` prevents loop-else.
- `continue` does not prevent loop-else.
- normal exhaustion of a `for` loop triggers loop-else.
- normal falsification of a `while` condition triggers loop-else.

An exception that escapes the loop also prevents normal loop completion:

```
items = ["10", "20", "bad", "30"]

try:
    for item in items:
        print(f"converting: {item}")
        number = int(item)
        print(f"number: {number}")
    else:
        print("all items converted")
except ValueError as err:
    print("conversion failed")
    print(f"type(err): {type(err)}")
    print(f"message: {err}")
```

Possible output:

```
converting: 10
number: 10
converting: 20
number: 20
converting: bad
conversion failed
type(err): <class 'ValueError'>
message: invalid literal for int() with base 10: 'bad'
```

The loop-else block did not run because the loop did not complete normally. The exception transferred control to the `except` block.

The practical reading is:

Use loop-else when the loop body contains a `break` that represents success, discovery, or early termination, and the `else` block represents the no-break case.

A final compact pattern:

```
passwords = ["abc", "guest", "letmein"]
wanted = "swordfish"

for password in passwords:
    if password == wanted:
        print("access word found")
        break
else:
    print("Nobody expects the missing password.")
```

Possible output:

```
Nobody expects the missing password.
```

The loop inspected every candidate and never broke. The `else` block is therefore the completion route.

For control-flow graph analysis, `loop-else` adds one more outgoing edge from the loop structure: the normal-completion edge. A `break` follows the interrupted-exit edge and skips that completion block. This is the entire rule.

6.6 Lazy Traversal Objects and Iteration Helpers

Python provides several built-in helpers that modify traversal without immediately building new containers. Functions such as `enumerate()`, `zip()`, `map()`, and `filter()` return lazy traversal objects. They produce values only when a loop, `next()`, or a materializing function asks for them.

This section studies those helpers as parts of the iterator protocol, not as list-building shortcuts. `enumerate()` attaches counters to values. `zip()` synchronizes several iterables. `map()` transforms each value. `filter()` allows only selected values through.

The central distinction is between traversal and storage. These helper objects describe or manage a stream of values in progress. They become stored data only when explicitly materialized with tools such as `list()` or `tuple()`.

6.6.1 Enumerated Traversal

A normal `for` loop gives the next value from an iterable. Sometimes that is enough. Other times, the program also needs the position of that value: the first item, the second item, the third item, and so on.

A beginner coming from C may try to solve this by manually combining `range()` and indexing:

```
names = ["Jerry", "Elaine", "Kramer"]

for i in range(len(names)):
    print(f"{i}: {names[i]}")
```

Possible output:

```
0: Jerry
1: Elaine
2: Kramer
```

This works, but it is not the usual Python shape. The loop is no longer directly traversing the values. It is traversing numeric positions and then using those positions to index the list.

Python provides `enumerate()` for the more direct case: traverse the values and receive a counter beside each value.

`enumerate()` turns a value stream into an index-value stream, without building a separate list of pairs in advance.

enumerate() as a Lazy Pair Generator for Index-Value Iteration

The `enumerate()` function receives an iterable and returns an `enumerate` object. That object is itself an iterator. Each call to `next()` produces a pair containing the current index and the current value.

```
names = ["Jerry", "Elaine", "Kramer"]

pairs = enumerate(names)

print(f"type(names): {type(names)}")
print(f"type(pairs): {type(pairs)}")
print(f"dir(pairs) sample: {dir(pairs)[:12]}")

print(f"has __iter__: {'__iter__' in dir(pairs)}")
print(f"has __next__: {'__next__' in dir(pairs)}")
```

Possible output:

```
type(names): <class 'list'>
type(pairs): <class 'enumerate'>
dir(pairs) sample: ['__class__', '__class_getitem__', '__delattr__',
'__dir__', '__doc__', '__eq__', '__format__', '__ge__',
'__getattribute__', '__getstate__', '__gt__', '__hash__']
has __iter__: True
has __next__: True
```

Because the object has `__iter__()` and `__next__()`, it is an iterator. It is consumed as it advances.

Manual inspection with `next()` shows what it produces:

```
names = ["Jerry", "Elaine", "Kramer"]
pairs = enumerate(names)

pair1 = next(pairs)
pair2 = next(pairs)
pair3 = next(pairs)

print(f"pair1: {pair1}")
print(f"pair2: {pair2}")
print(f"pair3: {pair3}")

print(f"type(pair1): {type(pair1)}")
print(f"dir(pair1) sample: {dir(pair1)[:10]}")
```

Possible output:

```
pair1: (0, 'Jerry')
pair2: (1, 'Elaine')
pair3: (2, 'Kramer')
type(pair1): <class 'tuple'>
dir(pair1) sample: ['__add__', '__class__', '__class_getitem__',
'__contains__', '__delattr__', '__dir__', '__doc__', '__eq__',
'__format__', '__ge__']
```

Each produced object is a tuple. The first component is the index. The second component is the original value.

The tuple can be unpacked directly in the `for` loop header:

```
names = ["Jerry", "Elaine", "Kramer"]
```

```
for i, name in enumerate(names):  
    print(f"{i:>2} | {name}")
```

Possible output:

```
0 | Jerry  
1 | Elaine  
2 | Kramer
```

The loop header:

```
for i, name in enumerate(names):
```

means that each produced pair is decomposed into two names. The first tuple element is assigned to `i`. The second tuple element is assigned to `name`.

This is equivalent to writing the unpacking manually inside the loop:

```
names = ["Jerry", "Elaine", "Kramer"]
```

```
for pair in enumerate(names):  
    i = pair[0]  
    name = pair[1]  
    print(f"{i:>2} | {name}")
```

Possible output:

```
0 | Jerry  
1 | Elaine  
2 | Kramer
```

The unpacked version is usually clearer because the structure of the pair is visible in the loop header.

The starting index can be changed with the `start` argument:

```
names = ["Jerry", "Elaine", "Kramer"]
```

```
for i, name in enumerate(names, start=1):  
    print(f"{i:>2}. {name}")
```

Possible output:

```
1. Jerry  
2. Elaine  
3. Kramer
```

This is useful when displaying human-facing numbering. Python list indexes normally start at 0, but printed lists often start at 1.

The `enumerate()` object is lazy. It does not immediately create all pairs.

```
names = ["Jerry", "Elaine", "Kramer"]
```

```
pairs = enumerate(names, start=1)
```

```
print(f"pairs: {pairs}")  
print("nothing has been printed from the names yet")
```

```
print(f"next(pairs): {next(pairs)}")  
print(f"next(pairs): {next(pairs)}")
```


Possible output:

```
pairs: <enumerate object at 0x...>
nothing has been printed from the names yet
next(pairs): (1, 'Jerry')
next(pairs): (2, 'Elaine')
```

The exact memory address will differ. The important fact is that `pairs` is a traversal object. It yields pairs only when asked.

If all pairs are needed as stored data, the object can be materialized:

```
names = ["Jerry", "Elaine", "Kramer"]

pairs = enumerate(names, start=1)
stored_pairs = list(pairs)

print(f"stored_pairs: {stored_pairs}")
print(f"type(stored_pairs): {type(stored_pairs)}")
```

Possible output:

```
stored_pairs: [(1, 'Jerry'), (2, 'Elaine'), (3, 'Kramer')]
type(stored_pairs): <class 'list'>
```

The call to `list(pairs)` consumes the `enumerate` iterator and stores the remaining produced tuples in a list.

After consumption, the same `enumerate` object has no remaining values:

```
names = ["Jerry", "Elaine", "Kramer"]

pairs = enumerate(names)

first = list(pairs)
second = list(pairs)

print(f"first: {first}")
print(f"second: {second}")
```

Possible output:

```
first: [(0, 'Jerry'), (1, 'Elaine'), (2, 'Kramer')]
second: []
```

This follows the iterator exhaustion rule from the previous section. The `enumerate` object is not the same kind of reusable source as the original list. It is a traversal in progress.

To traverse again, create a new `enumerate` object from the original iterable:

```
names = ["Jerry", "Elaine", "Kramer"]

first = list(enumerate(names))
second = list(enumerate(names))

print(f"first: {first}")
print(f"second: {second}")
```

Possible output:

```
first: [(0, 'Jerry'), (1, 'Elaine'), (2, 'Kramer')]
second: [(0, 'Jerry'), (1, 'Elaine'), (2, 'Kramer')]
```

The original list is reusable. Each call to `enumerate(names)` creates a new iterator over it.

The values produced by `enumerate()` reflect the current traversal of the underlying iterable. For simple teaching code, avoid changing the list while enumerating it, because that mixes traversal with structural mutation.

Clear version:

```
names = ["Jerry", "Elaine", "Kramer"]

for i, name in enumerate(names):
    print(f"{i}: {name}")
```

Avoid this shape while learning:

```
names = ["Jerry", "Elaine", "Kramer"]

for i, name in enumerate(names):
    names.append("Newman")
    print(f"{i}: {name}")
```

The second program changes the traversal source while it is being traversed. That is a separate mutation problem, not a useful demonstration of `enumerate()`.

A compact practical example:

```
quotes = [
    "No soup for you!",
    "Nobody expects the Spanish Inquisition!",
    "D'oh!",
]

for line_no, quote in enumerate(quotes, start=1):
    print(f"{line_no:>2} | {quote}")
```

Possible output:

```
1 | No soup for you!
2 | Nobody expects the Spanish Inquisition!
3 | D'oh!
```

The `enumerate()` call supplies the line number. The list supplies the quote. The loop receives both together.

The practical rule is:

Use `enumerate()` when you need both the traversal value and a counter. It is clearer than manually looping over `range(len(obj))` when the value itself is the main object being processed.

6.6.2 Parallel Traversal

Sometimes a program must traverse more than one iterable at the same time. One list may contain names, another may contain scores, and another may contain labels. The task is not to finish one list and then start the next. The task is to take the first value from each source, then the second value from each source, then the third value from each source, and so on.

Python provides `zip()` for this pattern.

`zip()` synchronizes several traversal sources and produces one tuple per parallel step.

This is not the same thing as merging containers permanently. Like `enumerate()`, `zip()` creates a lazy iterator. It produces combined values only when the loop or `next()` asks for them.

zip() as a Lazy Synchronization Mechanism Across Multiple Iterables

The `zip()` function receives two or more iterables and returns a `zip` object. Each produced item is a tuple containing one value from each input iterable.

```
names = ["Jerry", "Elaine", "Kramer"]
scores = [72, 91, 42]

pairs = zip(names, scores)

print(f"type(names): {type(names)}")
print(f"type(scores): {type(scores)}")
print(f"type(pairs): {type(pairs)}")
print(f"dir(pairs) sample: {dir(pairs)[:12]}")

print(f"has __iter__: {'__iter__' in dir(pairs)}")
print(f"has __next__: {'__next__' in dir(pairs)}")
```

Possible output:

```
type(names): <class 'list'>
type(scores): <class 'list'>
type(pairs): <class 'zip'>
dir(pairs) sample: ['__class__', '__delattr__', '__dir__', '__doc__',
'__eq__', '__format__', '__ge__', '__getattr__', '__getstate__',
'__gt__', '__hash__', '__init__']
has __iter__: True
has __next__: True
```

The `zip` object is an iterator. It has `__iter__()` and `__next__()`. Therefore it is consumed as it advances.

Manual inspection with `next()` shows the produced tuples:

```
names = ["Jerry", "Elaine", "Kramer"]
scores = [72, 91, 42]

pairs = zip(names, scores)

pair1 = next(pairs)
pair2 = next(pairs)
pair3 = next(pairs)

print(f"pair1: {pair1}")
print(f"pair2: {pair2}")
print(f"pair3: {pair3}")

print(f"type(pair1): {type(pair1)}")
print(f"dir(pair1) sample: {dir(pair1)[:10]}")
```

Possible output:

```
pair1: ('Jerry', 72)
pair2: ('Elaine', 91)
pair3: ('Kramer', 42)
type(pair1): <class 'tuple'>
```

```
dir(pair1) sample: ['__add__', '__class__', '__class_getitem__',
'__contains__', '__delattr__', '__dir__', '__doc__', '__eq__',
'__format__', '__ge__']
```

Each produced tuple contains one value from each input iterable. The first tuple contains the first name and the first score. The second tuple contains the second name and the second score.

The tuple can be unpacked directly in the loop header:

```
names = ["Jerry", "Elaine", "Kramer"]
scores = [72, 91, 42]

for name, score in zip(names, scores):
    print(f"{name:<8} | {score:>3}")
```

Possible output:

```
Jerry    | 72
Elaine   | 91
Kramer   | 42
```

The loop header:

```
for name, score in zip(names, scores):
```

means that each tuple produced by `zip()` is decomposed into two names. The first tuple component is assigned to `name`. The second tuple component is assigned to `score`.

This is equivalent to unpacking manually:

```
names = ["Jerry", "Elaine", "Kramer"]
scores = [72, 91, 42]

for pair in zip(names, scores):
    name = pair[0]
    score = pair[1]
    print(f"{name:<8} | {score:>3}")
```

Possible output:

```
Jerry    | 72
Elaine   | 91
Kramer   | 42
```

The unpacked version is usually clearer because the structure of each produced tuple is visible in the loop header.

`zip()` can synchronize more than two iterables:

```
names = ["Jerry", "Elaine", "Kramer"]
scores = [72, 91, 42]
labels = ["ok", "excellent", "answer"]

for name, score, label in zip(names, scores, labels):
    print(f"{name:<8} | {score:>3} | {label}")
```

Possible output:

```

Jerry    | 72 | ok
Elaine   | 91 | excellent
Kramer   | 42 | answer

```

Now each produced tuple has three components.

The important point is that `zip()` advances all input iterators together. One output tuple is produced from one synchronized step.

```

names:  Jerry      Elaine      Kramer
scores: 72         91         42

```

```

zip output:
    ('Jerry', 72)
    ('Elaine', 91)
    ('Kramer', 42)

```

By default, `zip()` stops when the shortest input iterable is exhausted.

```

names = ["Jerry", "Elaine", "Kramer", "Newman"]
scores = [72, 91]

for name, score in zip(names, scores):
    print(f"{name:<8} | {score:>3}")

print("loop finished")

```

Possible output:

```

Jerry    | 72
Elaine   | 91
loop finished

```

The values "Kramer" and "Newman" are not produced because the `scores` list ends first. The default `zip()` rule is shortest-input completion.

This behavior is useful when the shortest stream naturally defines the valid synchronized region. But it can hide mistakes when the lists were expected to have equal length.

A direct length check is simple:

```

names = ["Jerry", "Elaine", "Kramer", "Newman"]
scores = [72, 91]

print(f"len(names): {len(names)}")
print(f"len(scores): {len(scores)}")

if len(names) != len(scores):
    print("length mismatch")
else:
    for name, score in zip(names, scores):
        print(f"{name:<8} | {score:>3}")

```

Possible output:

```

len(names): 4
len(scores): 2
length mismatch

```

In Python versions that support it, `zip()` also has a `strict=True` option. With this option, unequal input lengths raise a `ValueError` instead of silently stopping at the shortest iterable.

```
names = ["Jerry", "Elaine", "Kramer", "Newman"]
scores = [72, 91]

try:
    for name, score in zip(names, scores, strict=True):
        print(f"{name:<8} | {score:>3}")
except ValueError as err:
    print(f"type(err): {type(err)}")
    print(f"message: {err}")
```

Possible output:

```
type(err): <class 'ValueError'>
message: zip() argument 2 is shorter than argument 1
```

The exact message may vary slightly by Python version, but the meaning is stable: strict zipping requires all input iterables to finish together.

The `zip()` object is lazy. It does not build all tuples immediately.

```
names = ["Jerry", "Elaine", "Kramer"]
scores = [72, 91, 42]

pairs = zip(names, scores)

print(f"pairs: {pairs}")
print("no pairs have been printed yet")

print(f"next(pairs): {next(pairs)}")
print(f"next(pairs): {next(pairs)}")
```

Possible output:

```
pairs: <zip object at 0x...>
no pairs have been printed yet
next(pairs): ('Jerry', 72)
next(pairs): ('Elaine', 91)
```

The memory address will differ between executions. The important point is that `pairs` is not a list. It is a traversal object.

When stored results are needed, the `zip` object can be materialized:

```
names = ["Jerry", "Elaine", "Kramer"]
scores = [72, 91, 42]

stored_pairs = list(zip(names, scores))

print(f"stored_pairs: {stored_pairs}")
print(f"type(stored_pairs): {type(stored_pairs)}")
```

Possible output:

```
stored_pairs: [('Jerry', 72), ('Elaine', 91), ('Kramer', 42)]
type(stored_pairs): <class 'list'>
```

The materialized result is a list of tuples. The list stores the produced pairs. Before materialization, the `zip` object produces pairs only on demand.

Like other iterators, a `zip` object is consumed after traversal:

```
names = ["Jerry", "Elaine", "Kramer"]
scores = [72, 91, 42]

pairs = zip(names, scores)

first = list(pairs)
second = list(pairs)

print(f"first: {first}")
print(f"second: {second}")
```

Possible output:

```
first: [('Jerry', 72), ('Elaine', 91), ('Kramer', 42)]
second: []
```

The first `list(pairs)` consumes the `zip` iterator. The second one receives no remaining values.

To repeat the traversal, create a new `zip` object from the original iterables:

```
names = ["Jerry", "Elaine", "Kramer"]
scores = [72, 91, 42]

first = list(zip(names, scores))
second = list(zip(names, scores))

print(f"first: {first}")
print(f"second: {second}")
```

Possible output:

```
first: [('Jerry', 72), ('Elaine', 91), ('Kramer', 42)]
second: [('Jerry', 72), ('Elaine', 91), ('Kramer', 42)]
```

The original lists are reusable. Each call to `zip(names, scores)` creates a new iterator.

`zip()` can also synchronize different iterable types:

```
letters = "abc"
nums = range(1, 4)
words = ("soup", "pretzels", "muffin")

for letter, num, word in zip(letters, nums, words):
    print(f"{letter} | {num:>2} | {word}")
```

Possible output:

```
a |  1 | soup
b |  2 | pretzels
c |  3 | muffin
```

The string supplies characters. The `range` supplies computed integers. The tuple supplies stored references. The `zip` object only coordinates the parallel traversal.

The practical rule is:

Use `zip()` when several iterables must be consumed in synchronized steps. Remember that the result is a lazy iterator and, by default, traversal stops at the shortest input.

6.6.3 Transforming and Filtering Iteration Streams

Traversal does not always mean merely receiving values exactly as they appear in the original source. Sometimes each value must be transformed before use. Sometimes only some values should continue through the pipeline. Python provides `map()` and `filter()` for these two common cases.

The important point is that both functions are lazy in Python 3. They do not immediately build a list of results. They return iterator objects that produce values only when the program asks for them.

`map()` transforms values lazily. `filter()` keeps or rejects values lazily.

This makes them part of the same traversal architecture as `enumerate()` and `zip()`. They are not containers. They are traversal objects.

`map()` and `filter()` as Lazy Iterator-Producing Transformations

The `map()` function applies a callable object to each value produced by an iterable. Its basic shape is:

```
map(callable_object, iterable)
```

A simple example uses the built-in `int` constructor to transform strings into integers:

```
texts = ["10", "20", "42"]

nums = map(int, texts)

print(f"type(texts): {type(texts)}")
print(f"type(nums): {type(nums)}")
print(f"dir(nums) sample: {dir(nums)[:12]}")

print(f"has __iter__: {'__iter__' in dir(nums)}")
print(f"has __next__: {'__next__' in dir(nums)}")
```

Possible output:

```
type(texts): <class 'list'>
type(nums): <class 'map'>
dir(nums) sample: ['__class__', '__delattr__', '__dir__', '__doc__',
'__eq__', '__format__', '__ge__', '__getattribute__', '__getstate__',
'__gt__', '__hash__', '__init__']
has __iter__: True
has __next__: True
```

The `map` object is an iterator. It has `__iter__()` and `__next__()`. Therefore it is consumed as it advances.

Manual traversal shows the transformation:

```
texts = ["10", "20", "42"]

nums = map(int, texts)

print(f"next(nums): {next(nums)}")
print(f"next(nums): {next(nums)}")
print(f"next(nums): {next(nums)}")
```

Possible output:


```
next(nums): 10
next(nums): 20
next(nums): 42
```

Each original string is passed to `int`. The produced value is an integer object.

```
text = "42"
num = int(text)

print(f"text: {text!r}")
print(f"type(text): {type(text)}")
print(f"num: {num}")
print(f"type(num): {type(num)}")
print(f"dir(num) sample: {dir(num)[:10]}")
print(f"num.__bool__(): {num.__bool__()}")
print(f"num.__add__(8): {num.__add__(8)}")
```

Possible output:

```
text: '42'
type(text): <class 'str'>
num: 42
type(num): <class 'int'>
dir(num) sample: ['__abs__', '__add__', '__and__', '__bool__',
'__ceil__', '__class__', '__delattr__', '__dir__', '__divmod__',
'__doc__']
num.__bool__(): True
num.__add__(8): 50
```

The `__add__()` method supports integer addition. The `__bool__()` method supports truth-value testing.

A `map` object can be used directly in a `for` loop:

```
texts = ["10", "20", "42"]

for num in map(int, texts):
    print(f"{num:>3} | {num * 2:>3}")
```

Possible output:

```
10 | 20
20 | 40
42 | 84
```

The loop receives one transformed value at a time. No list of transformed values is required unless the programmer explicitly asks for one.

Materialization creates stored results:

```
texts = ["10", "20", "42"]

nums = list(map(int, texts))

print(f"nums: {nums}")
print(f"type(nums): {type(nums)}")
```

Possible output:

```
nums: [10, 20, 42]
type(nums): <class 'list'>
```

The call to `list()` is the materialization boundary. Before that point, the result is a lazy `map` iterator. After that point, the result is a list containing the transformed values.

A `map` object is consumed after traversal:

```
texts = ["10", "20", "42"]
```

```
nums = map(int, texts)
```

```
first = list(nums)
second = list(nums)
```

```
print(f"first: {first}")
print(f"second: {second}")
```

Possible output:

```
first: [10, 20, 42]
second: []
```

The first `list(nums)` consumes the iterator. The second call receives no remaining values.

To repeat the transformation, create a new `map` object:

```
texts = ["10", "20", "42"]
```

```
first = list(map(int, texts))
second = list(map(int, texts))
```

```
print(f"first: {first}")
print(f"second: {second}")
```

Possible output:

```
first: [10, 20, 42]
second: [10, 20, 42]
```

The source list is reusable. The `map` iterator is not reusable after exhaustion.

The transformation callable does not have to be `int`. It can be another callable object. A simple named function makes the lazy behavior visible:

```
def loud_text(text):
    print(f"transforming: {text}")
    return text.upper()
```

```
quotes = ["ni", "spam", "giddy up"]
```

```
loud_quotes = map(loud_text, quotes)
```

```
print("map object created")
print(f"type(loud_quotes): {type(loud_quotes)}")
```

```
print("asking for first value")
print(next(loud_quotes))
```

```
print("asking for second value")
print(next(loud_quotes))
```

Possible output:

```
map object created
type(loud_quotes): <class 'map'>
asking for first value
transforming: ni
NI
asking for second value
transforming: spam
SPAM
```

The function is not called when the `map` object is created. It is called when the iterator is advanced. The function object itself is a runtime object:

```
def loud_text(text):
    return text.upper()

print(f"type(loud_text): {type(loud_text)}")
print(f"dir(loud_text) sample: {dir(loud_text)[:12]}")
print(f"loud_text.__name__: {loud_text.__name__}")
```

Possible output:

```
type(loud_text): <class 'function'>
dir(loud_text) sample: ['__annotations__', '__builtins__', '__call__',
'__class__', '__closure__', '__code__', '__defaults__',
'__delattr__', '__dict__', '__dir__', '__doc__', '__eq__']
loud_text.__name__: loud_text
```

The important method here is `__call__()`. A callable object can be called with parentheses. `map()` stores that callable and applies it to incoming values one at a time.

The `filter()` function has a related but different role. It keeps only the values for which a predicate succeeds.

Its basic shape is:

```
filter(predicate, iterable)
```

The predicate is called for each value. If the predicate result is true, the original value is yielded. If the predicate result is false, the value is skipped.

```
def is_even(num):
    return num % 2 == 0

nums = [10, 15, 20, 42, 99]

even_nums = filter(is_even, nums)

print(f"type(nums): {type(nums)}")
print(f"type(even_nums): {type(even_nums)}")
print(f"dir(even_nums) sample: {dir(even_nums)[:12]}")

print(f"has __iter__: {'__iter__' in dir(even_nums)}")
print(f"has __next__: {'__next__' in dir(even_nums)}")
```

Possible output:

```

type(nums): <class 'list'>
type(even_nums): <class 'filter'>
dir(even_nums) sample: ['__class__', '__delattr__', '__dir__', '__doc__',
'__eq__', '__format__', '__ge__', '__getattr__', '__getstate__',
'__gt__', '__hash__', '__init__']
has __iter__: True
has __next__: True

```

The `filter` object is also an iterator. It is lazy and consumable.

Using it in a loop:

```

def is_even(num):
    return num % 2 == 0

nums = [10, 15, 20, 42, 99]

for num in filter(is_even, nums):
    print(f"accepted: {num}")

```

Possible output:

```

accepted: 10
accepted: 20
accepted: 42

```

The values 15 and 99 were tested but not yielded, because the predicate returned false for them.

A manual version makes the logic explicit:

```

def is_even(num):
    return num % 2 == 0

nums = [10, 15, 20, 42, 99]

for num in nums:
    if is_even(num):
        print(f"accepted: {num}")

```

Possible output:

```

accepted: 10
accepted: 20
accepted: 42

```

The `filter()` version packages this pattern into a lazy iterator.

The predicate result is truth-tested, not required to be exactly `True` or `False`. This means `filter(None, iterable)` keeps truthy values and rejects falsy values.

```

values = ["spam", "", "Ni!", 0, 42, [], [1, 2]]

kept = filter(None, values)

print(f"type(kept): {type(kept)}")
print(f"list(kept): {list(kept)}")

```

Possible output:

```
type(kept): <class 'filter'>
list(kept): ['spam', 'Ni!', 42, [1, 2]]
```

The empty string, zero, and empty list were rejected because they are false in truth-value testing. This connects `filter()` directly to the earlier truth-value rules:

```
values = ["spam", "", 0, 42, []]

for value in values:
    print(f"{repr(value):>8} -> bool: {bool(value)}")
```

Possible output:

```
'spam' -> bool: True
'' -> bool: False
0 -> bool: False
42 -> bool: True
[] -> bool: False
```

The `filter(None, values)` form keeps only those values whose `bool()` result is true. Like `map`, a `filter` object is consumed after traversal:

```
def is_even(num):
    return num % 2 == 0

nums = [10, 20, 42]

even_nums = filter(is_even, nums)

first = list(even_nums)
second = list(even_nums)

print(f"first: {first}")
print(f"second: {second}")
```

Possible output:

```
first: [10, 20, 42]
second: []
```

The first materialization consumes the iterator.

The two functions can be combined into a traversal pipeline:

```
texts = ["10", "15", "20", "42"]

nums = map(int, texts)
even_nums = filter(lambda num: num % 2 == 0, nums)

print(list(even_nums))
```

Possible output:

```
[10, 20, 42]
```

This works, but the `lambda` expression introduces another compact function syntax. For early teaching code, a named function is often clearer:

```
def is_even(num):
    return num % 2 == 0

texts = ["10", "15", "20", "42"]

nums = map(int, texts)
even_nums = filter(is_even, nums)

print(list(even_nums))
```

Possible output:

```
[10, 20, 42]
```

The pipeline is:

```
texts
  -> map int over each text
  -> filter only even integers
  -> materialize as list
```

The important detail is that `nums` is itself a `map` iterator. The `filter` object consumes it as needed. Values pass through the chain one at a time.

A final example makes the stream-like behavior visible:

```
def show_int(text):
    print(f"mapping: {text}")
    return int(text)

def is_large(num):
    print(f"filtering: {num}")
    return num > 20

texts = ["10", "20", "42", "100"]

nums = map(show_int, texts)
large_nums = filter(is_large, nums)

print("pipeline created")
print("asking for first accepted value")
print(next(large_nums))
```

Possible output:

```
pipeline created
asking for first accepted value
mapping: 10
filtering: 10
mapping: 20
filtering: 20
mapping: 42
filtering: 42
42
```

The pipeline stops as soon as `next(large_nums)` receives one accepted value. The value "100" has not yet been mapped or filtered, because no one has asked for another accepted value.

The practical rule is:

Use `map()` when every traversed value should be transformed. Use `filter()` when some traversed values should be allowed through and others rejected. Remember that both return lazy iterators, not stored result lists.

6.6.4 Materialization Boundaries

Lazy traversal objects are useful because they delay work and avoid storing all produced values at once. Objects returned by `enumerate()`, `zip()`, `map()`, and `filter()` are not result containers. They are iterators. They produce values as the program asks for them.

Sometimes this is exactly what the program needs. A `for` loop can consume values one at a time without ever storing the entire result stream. Other times, the program really does need stored results: to inspect them repeatedly, index them, count them after production, print them as a complete collection, or pass them to code that expects a concrete sequence.

That crossing point is a materialization boundary.

Materialization means converting a lazy traversal stream into a stored container such as a `list` or `tuple`.

Before materialization, the program has a traversal process. After materialization, the program has a container holding the produced objects.

Converting Lazy Iterables into Lists or Tuples When Stored Results Are Needed

The most common materialization functions are `list()` and `tuple()`.

```
names = ["Jerry", "Elaine", "Kramer"]

pairs = enumerate(names)

stored_list = list(pairs)

print(f"stored_list: {stored_list}")
print(f"type(stored_list): {type(stored_list)}")
print(f"dir(stored_list) sample: {dir(stored_list)[:10]}")
```

Possible output:

```
stored_list: [(0, 'Jerry'), (1, 'Elaine'), (2, 'Kramer')]
type(stored_list): <class 'list'>
dir(stored_list) sample: ['__add__', '__class__', '__class_getitem__',
'__contains__', '__delattr__', '__delitem__', '__dir__', '__doc__',
'__eq__', '__format__']
```

The `enumerate` object has been consumed. The produced tuples are now stored inside a list.

The resulting list is reusable:

```
names = ["Jerry", "Elaine", "Kramer"]

stored = list(enumerate(names))

print("first traversal")
for pair in stored:
    print(pair)

print("second traversal")
for pair in stored:
    print(pair)
```

Possible output:

```
first traversal
(0, 'Jerry')
(1, 'Elaine')
(2, 'Kramer')
second traversal
(0, 'Jerry')
(1, 'Elaine')
(2, 'Kramer')
```

The list can be traversed repeatedly because it is a reusable iterable container. Each `for` loop creates a new iterator over the stored list.

This is different from reusing the original lazy iterator:

```
names = ["Jerry", "Elaine", "Kramer"]

pairs = enumerate(names)

print("first materialization")
print(list(pairs))

print("second materialization")
print(list(pairs))
```

Possible output:

```
first materialization
[(0, 'Jerry'), (1, 'Elaine'), (2, 'Kramer')]
second materialization
[]
```

The second list is empty because the `enumerate` iterator was already exhausted by the first `list()` call.

The same rule applies to `zip()`:

```
names = ["Jerry", "Elaine", "Kramer"]
scores = [72, 91, 42]

pairs = zip(names, scores)

stored = list(pairs)

print(f"stored: {stored}")
print(f"type(stored): {type(stored)}")
```

Possible output:

```
stored: [('Jerry', 72), ('Elaine', 91), ('Kramer', 42)]
type(stored): <class 'list'>
```

Before the `list()` call, `pairs` is a lazy `zip` iterator. After the call, `stored` is a list of tuple objects. Materializing as a tuple works similarly:


```

names = ["Jerry", "Elaine", "Kramer"]
scores = [72, 91, 42]

pairs = zip(names, scores)

stored = tuple(pairs)

print(f"stored: {stored}")
print(f"type(stored): {type(stored)}")
print(f"dir(stored) sample: {dir(stored)[:10]}")

```

Possible output:

```

stored: (('Jerry', 72), ('Elaine', 91), ('Kramer', 42))
type(stored): <class 'tuple'>
dir(stored) sample: ['__add__', '__class__', '__class_getitem__',
'__contains__', '__delattr__', '__dir__', '__doc__', '__eq__',
'__format__', '__ge__']

```

The outer container is now a tuple. The produced pairs are also tuples. This gives a fixed stored structure.

The choice between `list()` and `tuple()` should follow the intended use:

- Use `list()` when the stored result may need later structural modification.
- Use `tuple()` when the stored result should be treated as a fixed structure.

For example, a materialized list can be appended to:

```

names = ["Jerry", "Elaine"]

stored = list(enumerate(names))
stored.append((2, "Kramer"))

print(f"stored: {stored}")

```

Possible output:

```

stored: [(0, 'Jerry'), (1, 'Elaine'), (2, 'Kramer')]

```

A materialized tuple cannot be appended to:

```

names = ["Jerry", "Elaine"]

stored = tuple(enumerate(names))

try:
    stored.append((2, "Kramer"))
except AttributeError as err:
    print(f"type(err): {type(err)}")
    print(f"message: {err}")

```

Possible output:

```

type(err): <class 'AttributeError'>
message: 'tuple' object has no attribute 'append'

```

The tuple stores the produced values, but its structure cannot be expanded. Materialization is also common after `map()`:

```
texts = ["10", "20", "42"]

nums_iter = map(int, texts)
nums = list(nums_iter)

print(f"nums: {nums}")
print(f"type(nums): {type(nums)}")
print(f"nums[0]: {nums[0]}")
print(f"sum(nums): {sum(nums)}")
```

Possible output:

```
nums: [10, 20, 42]
type(nums): <class 'list'>
nums[0]: 10
sum(nums): 72
```

The stored list can now be indexed, traversed repeatedly, and passed to operations that need all values available.

Without materialization, the iterator is consumed once:

```
texts = ["10", "20", "42"]

nums_iter = map(int, texts)

print(f"first sum: {sum(nums_iter)}")
print(f"second sum: {sum(nums_iter)}")
```

Possible output:

```
first sum: 72
second sum: 0
```

The second result is 0 because the `map` iterator has no remaining values. This is not because the original strings disappeared. It is because the traversal object was exhausted.

If the values are needed more than once, materialize them first:

```
texts = ["10", "20", "42"]

nums = list(map(int, texts))

print(f"first sum: {sum(nums)}")
print(f"second sum: {sum(nums)}")
```

Possible output:

```
first sum: 72
second sum: 72
```

The list is reusable because the integer objects are now stored in the list.

The same applies to `filter()`:

```
def is_even(num):
    return num % 2 == 0

nums = [10, 15, 20, 42, 99]

even_iter = filter(is_even, nums)
even_nums = list(even_iter)

print(f"even_nums: {even_nums}")
print(f"type(even_nums): {type(even_nums)}")
```

Possible output:

```
even_nums: [10, 20, 42]
type(even_nums): <class 'list'>
```

The filtered values are now stored.

Materialization also makes length and indexing available when the target container supports them:

```
def is_even(num):
    return num % 2 == 0

nums = [10, 15, 20, 42, 99]

even_nums = list(filter(is_even, nums))

print(f"len(even_nums): {len(even_nums)}")
print(f"even_nums[0]: {even_nums[0]}")
print(f"even_nums[-1]: {even_nums[-1]}")
```

Possible output:

```
len(even_nums): 3
even_nums[0]: 10
even_nums[-1]: 42
```

A lazy filter object does not support indexing:

```
def is_even(num):
    return num % 2 == 0

nums = [10, 15, 20, 42, 99]

even_iter = filter(is_even, nums)

try:
    print(even_iter[0])
except TypeError as err:
    print(f"type(err): {type(err)}")
    print(f"message: {err}")
```

Possible output:

```
type(err): <class 'TypeError'>
message: 'filter' object is not subscriptable
```

The iterator can produce values, but it is not an indexable sequence. If indexed access is needed, materialization is the correct boundary.

The same distinction can be seen with `map`:

```
texts = ["10", "20", "42"]

nums_iter = map(int, texts)

print(f"type(nums_iter): {type(nums_iter)}")
print(f"has __next__: {'__next__' in dir(nums_iter)}")
print(f"has __getitem__: {'__getitem__' in dir(nums_iter)}")
```

Possible output:

```
type(nums_iter): <class 'map'>
has __next__: True
has __getitem__: False
```

A `map` object is an iterator, not a sequence. It can be advanced with `next()`, but it cannot be indexed.

Materialization has a cost. A lazy iterator may hold only traversal state and references to its input sources. A list or tuple must store all produced values. For small results, this cost is usually harmless. For very large or infinite streams, materialization may be impossible or wasteful.

A simple comparison:

```
import sys

nums_range = range(1_000_000)
nums_map = map(lambda n: n * 2, nums_range)

print(f"type(nums_range): {type(nums_range)}")
print(f"type(nums_map): {type(nums_map)}")

print(f"size of range: {sys.getsizeof(nums_range)} bytes")
print(f"size of map: {sys.getsizeof(nums_map)} bytes")
```

Possible output on one CPython build:

```
type(nums_range): <class 'range'>
type(nums_map): <class 'map'>
size of range: 48 bytes
size of map: 48 bytes
```

The exact byte counts are implementation-dependent. The important point is that neither object stores one million transformed integers.

Materializing the transformed values changes the storage situation:

```
import sys

nums = list(map(lambda n: n * 2, range(1_000_000)))

print(f"type(nums): {type(nums)}")
print(f"len(nums): {len(nums)}")
print(f"size of nums list object: {sys.getsizeof(nums)} bytes")
```

Possible output on one CPython build:

```

type(nums): <class 'list'>
len(nums): 1000000
size of nums list object: 8448728 bytes

```

The reported list size is only the list container's storage for references. It does not necessarily include the full memory cost of every integer object. The exact number also depends on the CPython build. The structural point is enough: materialization creates a stored collection.

A useful practical pattern is to leave the pipeline lazy until the point where storage is actually needed:

```

def is_even(num):
    return num % 2 == 0

texts = ["10", "15", "20", "42", "99"]

nums = map(int, texts)
even_nums = filter(is_even, nums)

print("pipeline is still lazy")

stored = list(even_nums)

print(f"stored: {stored}")

```

Possible output:

```

pipeline is still lazy
stored: [10, 20, 42]

```

The transformation and filtering occur when `list(even_nums)` consumes the pipeline.

A pipeline can also be consumed directly by another function without explicit materialization:

```

def is_even(num):
    return num % 2 == 0

texts = ["10", "15", "20", "42", "99"]

total = sum(filter(is_even, map(int, texts)))

print(f"total: {total}")

```

Possible output:

```

total: 72

```

Here `sum()` consumes the lazy pipeline. No list is created. This is useful when the program only needs the final aggregate result.

But if the intermediate values must be printed, inspected, reused, or indexed, materialize them:

```

def is_even(num):
    return num % 2 == 0

texts = ["10", "15", "20", "42", "99"]

even_nums = list(filter(is_even, map(int, texts)))

```

```
print(f"even_nums: {even_nums}")
print(f"first even number: {even_nums[0]}")
print(f"total: {sum(even_nums)}")
print(f"again total: {sum(even_nums)}")
```

Possible output:

```
even_nums: [10, 20, 42]
first even number: 10
total: 72
again total: 72
```

The practical rule is:

Keep traversal lazy while values are only passing through a pipeline. Materialize with `list()` or `tuple()` when the produced values must become stored, reusable data.

This boundary is one of the most important differences between Python's traversal model and a simple container-based mental model. Not every object that can be looped over is a stored collection. Some objects are active traversal states. Materialization is the explicit step that turns the remaining traversal output into a concrete container.

6.7 Exceptional Control Flow and Error Routing

Not every execution route fails locally. Some operations fail in a way that cannot be handled by simply moving to the next statement or checking a returned value. Python uses exceptions for this kind of route change. An exception interrupts the ordinary path and sends execution toward a matching handler, possibly outside the current block or even outside the current function call.

This section studies exceptions as control-flow structures, not merely as error messages. A raised exception is a non-local jump through the active execution graph. The exception object carries a runtime type, the handler selection mechanism matches that type against exception classes, and the traceback records the route through active frames.

The goal is to make exception handling precise rather than magical: **try** marks a protected region, **except** catches selected failure routes, **else** isolates success-only logic, **finally** guarantees cleanup, and **raise** deliberately creates or continues an exceptional edge.

6.7.1 The Architecture of Non-Local Jumps

Ordinary control flow moves locally. A sequential statement falls into the next statement. A condition chooses one nearby branch. A loop returns to its own condition or iterator. An exception is different. When an exception is raised, execution may leave the current expression, the current statement, the current block, and even the current function call.

This is why exceptions are a form of non-local control transfer. The next relevant statement may not be physically near the failing operation. Python begins searching for a matching exception handler. If none exists in the current block, the exception can continue outward through active function calls until it is caught or until it reaches the interpreter boundary.

An exception is not merely an error value. It is a runtime event that changes the execution route.

This section introduces exceptions as routing structures. Later subsections will examine exception classes, handler matching, **try/except/else/finally**, traceback construction, explicit **raise**, re-raising, and exception chaining.

The Paradigm Shift: Contrast with C's Manual Error Return Codes and Pointer Check Patterns

In C, failure is often represented through returned values or modified output parameters. A function may return -1, NULL, 0, or an error code. The caller must inspect that result manually before continuing.

A simplified C-style pattern looks like this:

```
/* C-style idea, not Python */
FILE *fp = fopen("data.txt", "r");

if (fp == NULL) {
    printf("could not open file\n");
    return 1;
}

read_from_file(fp);
fclose(fp);
```

The failure information stays local unless the programmer explicitly forwards it. Every caller must remember to check the returned value. If the programmer forgets a check, the program may continue with invalid state.

Another common C-style pattern is pointer checking:

```
/* C-style idea, not Python */
char *name = get_name();

if (name != NULL) {
    printf("%s\n", name);
}
```

The programmer manually asks whether the pointer is usable before dereferencing it. The safety of the next operation depends on the explicit check.

Python can also use ordinary conditional checks, but its exception mechanism changes the default failure route. If an operation cannot complete normally, it can raise an exception instead of returning a special error value.

```
text = "forty-two"

print("before conversion")

try:
    number = int(text)
    print(f"number: {number}")
except ValueError as err:
    print("conversion failed")
    print(f"type(err): {type(err)}")
    print(f"message: {err}")

print("after conversion attempt")
```

Possible output:

```
before conversion
conversion failed
type(err): <class 'ValueError'>
message: invalid literal for int() with base 10: 'forty-two'
after conversion attempt
```

The expression `int(text)` does not return a special error number. It raises a `ValueError`. The ordinary route inside the `try` block is abandoned, and execution jumps to the matching `except` block.

The exception object is a runtime object:

```
text = "forty-two"

try:
    number = int(text)
except ValueError as err:
    print(f"type(err): {type(err)}")
    print(f"dir(err) sample: {dir(err)[:10]}")
    print(f"err.args: {err.args}")
    print(f"has __traceback__: {'__traceback__' in dir(err)}")
```

Possible output:

```
type(err): <class 'ValueError'>
dir(err) sample: ['__cause__', '__class__', '__context__', '__delattr__',
 '__dict__', '__dir__', '__doc__', '__eq__', '__format__', '__ge__',
 '__gt__', '__hash__', '__init__', '__init_subclass__', '__le__', '__lt__',
 '__module__', '__ne__', '__new__', '__reduce__', '__reduce_ex__', '__repr__',
 '__setattr__', '__sizeof__', '__str__', '__subclasshook__']
err.args: ("invalid literal for int() with base 10: 'forty-two'",)
has __traceback__: True
```

The `args` attribute stores the construction arguments carried by the exception. The `__traceback__` attribute connects the exception to traceback information. Later sections will examine how tracebacks preserve the route of failure through active frames.

The important control-flow difference is this:

C-style error code:

```
operation returns an error marker
caller manually checks marker
caller manually chooses next route
```

Python exception:

```
operation raises an exception
ordinary route is interrupted
Python searches for a matching handler
```

This does not mean that Python eliminates responsibility. The programmer still decides which exceptions to catch, where to catch them, and what recovery means. The difference is that failure travels through a separate control-flow channel rather than through an ordinary returned value.

A simple dictionary example shows the same idea:

```
scores = {
    "Jerry": 72,
    "Elaine": 91,
    "Kramer": 42,
}

name = "Newman"

try:
    score = scores[name]
    print(f"{name}: {score}")
except KeyError as err:
    print(f"missing key: {err}")
```



```
print(f"type(err): {type(err)}")

print("lookup attempt finished")
```

Possible output:

```
missing key: 'Newman'
type(err): <class 'KeyError'>
lookup attempt finished
```

The dictionary object is an ordinary runtime object:

```
scores = {
    "Jerry": 72,
    "Elaine": 91,
    "Kramer": 42,
}

print(f"type(scores): {type(scores)}")
print(f"dir(scores) sample: {dir(scores)[:12]}")
print(f"has __getitem__: {'__getitem__' in dir(scores)}")
print(f"has get: {'get' in dir(scores)}")
print(f"has keys: {'keys' in dir(scores)}")
```

Possible output:

```
type(scores): <class 'dict'>
dir(scores) sample: ['__class__', '__class_getitem__', '__contains__',
'__delattr__', '__delitem__', '__dir__', '__doc__', '__eq__',
'__format__', '__ge__', '__getattribute__', '__getitem__']
has __getitem__: True
has get: True
has keys: True
```

The `__getitem__()` method is involved in bracket lookup such as `scores[name]`. If the key is absent, that lookup raises `KeyError`. The `get()` method offers a different design: return a fallback value instead of raising.

```
scores = {
    "Jerry": 72,
    "Elaine": 91,
    "Kramer": 42,
}

name = "Newman"
score = scores.get(name, "not found")

print(f"name: {name}")
print(f"score: {score}")
```

Possible output:

```
name: Newman
score: not found
```

Both approaches are valid in different situations. The bracket form treats a missing key as an exceptional route. The `get()` form treats a missing key as an ordinary value-selection route.

Look Before You Leap (LBYL) vs. Easier to Ask Forgiveness than Permission (EAFP) Execution Philosophies

Two common styles appear in Python error-handling discussions: LBYL and EAFP.

LBYL means “look before you leap.” In this style, the program checks whether an operation appears safe before performing it.

```
scores = {
    "Jerry": 72,
    "Elaine": 91,
    "Kramer": 42,
}

name = "Newman"

if name in scores:
    score = scores[name]
    print(f"{name}: {score}")
else:
    print(f"{name} was not found")
```

Possible output:

Newman was not found

The condition checks membership before the lookup. The lookup happens only if the key is present.

EAFP means “easier to ask forgiveness than permission.” In this style, the program attempts the operation directly and handles the exception if the operation fails.

```
scores = {
    "Jerry": 72,
    "Elaine": 91,
    "Kramer": 42,
}

name = "Newman"

try:
    score = scores[name]
    print(f"{name}: {score}")
except KeyError:
    print(f"{name} was not found")
```

Possible output:

Newman was not found

Both programs produce the same visible result. Their control-flow shape is different.

The LBYL route is:

```
test whether key exists
true  -> perform lookup
false -> run missing-key branch
```

The EAFP route is:

```

attempt lookup
    success -> continue normally
    KeyError -> jump to handler

```

The EAFP version avoids duplicating the key lookup logic. This can be useful when the safest test is the operation itself.

A list example makes the difference clear:

```

items = ["soup", "pretzels"]

index = 2

if index < len(items):
    print(f"item: {items[index]}")
else:
    print("index is outside the list")

```

Possible output:

```
index is outside the list
```

That is the LBYL version. It checks the index before using it.
The EAFP version is:

```

items = ["soup", "pretzels"]

index = 2

try:
    print(f"item: {items[index]}")
except IndexError as err:
    print("index is outside the list")
    print(f"type(err): {type(err)}")

```

Possible output:

```
index is outside the list
type(err): <class 'IndexError'>
```

The list object supports indexed access through `__getitem__()`:

```

items = ["soup", "pretzels"]

print(f"type(items): {type(items)}")
print(f"dir(items) sample: {dir(items)[:10]}")
print(f"len(items): {len(items)}")
print(f"items.__getitem__(0): {items.__getitem__(0)}")

```

Possible output:

```

type(items): <class 'list'>
dir(items) sample: ['__add__', '__class__', '__class_getitem__',
 '__contains__', '__delattr__', '__delitem__', '__dir__', '__doc__',
 '__eq__', '__format__']
len(items): 2
items.__getitem__(0): soup

```

If the requested index is invalid, the indexed operation raises `IndexError`. The exception route is part of the object's interface behavior.

Neither LBYL nor EAFP is universally correct. They are different ways to arrange control flow.

LBYL is often clearer when the condition is cheap, direct, and part of the intended ordinary logic:

```
age_text = ""

if age_text:
    age = int(age_text)
    print(f"age: {age}")
else:
    print("no age was entered")
```

Possible output:

```
no age was entered
```

Here the empty-string check expresses an ordinary input condition. There is no need to force that case through an exception.

EAFP is often clearer when the operation itself is the authoritative test:

```
age_text = "forty-two"

try:
    age = int(age_text)
    print(f"age: {age}")
except ValueError:
    print("age text was not a valid integer")
```

Possible output:

```
age text was not a valid integer
```

Checking every possible invalid integer string manually would be awkward and incomplete. The conversion operation already knows the valid format. Letting `int()` decide and catching `ValueError` gives a precise route.

A useful combined version is:

```
age_text = "forty-two"

if age_text:
    try:
        age = int(age_text)
        print(f"age: {age}")
    except ValueError:
        print("age text was not a valid integer")
else:
    print("no age was entered")
```

Possible output:

```
age text was not a valid integer
```

The empty-input case is ordinary branching. The invalid-format case is exceptional routing.

The practical rule is:

Use `LBYL` when the precondition is simple and genuinely part of normal decision logic. Use `EAFP` when the attempted operation is the most accurate way to discover whether it can succeed.

For control-flow graph analysis, the distinction is structural. `LBYL` adds explicit conditional edges before the operation. `EAFP` adds exceptional edges after the operation begins. Both are routing tools. The correct choice depends on which route makes the program's intended facts clearer.

6.7.2 Exception Class Hierarchies and Handler Selection

Python exceptions are not untyped failure messages. They are objects created from exception classes. This means that exception handling is tied directly to Python's runtime type system. An `except` clause does not merely catch a string or a numeric error code. It matches an exception object by its class and by that class's position in the exception inheritance hierarchy.

This is why handler order matters. A specific exception class should usually be handled before a more general parent class. If the general handler appears first, it can catch the exception before the more specific handler ever has a chance to run.

Exception handling is runtime type selection applied to non-local control flow.

Exception Classes as Runtime Types

An exception object has a type, just like an integer, string, list, tuple, or dictionary has a type.

```
try:
    number = int("forty-two")
except ValueError as err:
    print(f"err: {err}")
    print(f"type(err): {type(err)}")
    print(f"is ValueError: {isinstance(err, ValueError)}")
    print(f"is Exception: {isinstance(err, Exception)}")
    print(f"is BaseException: {isinstance(err, BaseException)}")
```

Possible output:

```
err: invalid literal for int() with base 10: 'forty-two'
type(err): <class 'ValueError'>
is ValueError: True
is Exception: True
is BaseException: True
```

The object bound to `err` is a `ValueError` object. Because `ValueError` inherits from `Exception`, and `Exception` inherits from `BaseException`, the same object also counts as an `Exception` and as a `BaseException`.

The class route can be inspected with `__mro__`. MRO means method resolution order: the ordered chain of classes Python searches through for inherited behavior.

```
print(ValueError.__mro__)
print(KeyError.__mro__)
print(IndexError.__mro__)
```

Possible output:

```
(<class 'ValueError'>, <class 'Exception'>, <class 'BaseException'>, <class 'object'>)
(<class 'KeyError'>, <class 'LookupError'>, <class 'Exception'>, <class 'BaseException'>, <cl
(<class 'IndexError'>, <class 'LookupError'>, <class 'Exception'>, <class 'BaseException'>, <
```

This tells us that `KeyError` and `IndexError` are both more specific forms of `LookupError`. Therefore, a handler for `LookupError` can catch both dictionary missing-key failures and sequence invalid-index failures.

```
data = ["soup", "pretzels"]

try:
    print(data[42])
except LookupError as err:
    print("lookup failed")
    print(f"type(err): {type(err)}")
    print(f"message: {err}")
```

Possible output:

```
lookup failed
type(err): <class 'IndexError'>
message: list index out of range
```

The actual exception object is an `IndexError`, but it is caught by `LookupError` because `IndexError` is a child class of `LookupError`.

The exception object also has attributes and inherited methods:

```
try:
    score = int("D'oh!")
except ValueError as err:
    print(f"type(err): {type(err)}")
    print(f"dir(err) sample: {dir(err)[:12]}")
    print(f"err.args: {err.args}")
    print(f"str(err): {str(err)}")
    print(f"repr(err): {repr(err)}")
```

Possible output:

```
type(err): <class 'ValueError'>
dir(err) sample: ['__cause__', '__class__', '__context__', '__delattr__',
 '__dict__', '__dir__', '__doc__', '__eq__', '__format__', '__ge__',
 '__getattr__', '__getstate__']
err.args: ("invalid literal for int() with base 10: \"D'oh!\",")
str(err): invalid literal for int() with base 10: "D'oh!"
repr(err): ValueError('invalid literal for int() with base 10: "D\'oh!"')
```

The `args` attribute stores the construction arguments of the exception object. The `str()` conversion gives the human-readable message. The `repr()` conversion shows a more explicit object representation.

Matching Specific Exceptions Before General Exceptions

Python checks `except` clauses from top to bottom. The first matching handler is selected. After one handler is selected, later handlers in the same `try` statement are skipped.

This means specific handlers must come before general handlers.

Correct order:

```
text = "forty-two"

try:
    number = int(text)
```

```

    print(f"number: {number}")
except ValueError:
    print("specific handler: invalid integer text")
except Exception:
    print("general handler: some other ordinary exception")

```

Possible output:

```
specific handler: invalid integer text
```

The `ValueError` handler runs because it appears first and matches the raised exception exactly. Bad order:

```

text = "forty-two"

try:
    number = int(text)
    print(f"number: {number}")
except Exception:
    print("general handler: catches the ValueError first")
except ValueError:
    print("specific handler: unreachable here")

```

Possible output:

```
general handler: catches the ValueError first
```

The `ValueError` handler is structurally useless here. The broader `Exception` handler already caught the object.

The same problem appears with `LookupError`, `KeyError`, and `IndexError`.

```

scores = {
    "Jerry": 72,
    "Elaine": 91,
}

name = "Kramer"

try:
    print(scores[name])
except KeyError:
    print("specific handler: missing dictionary key")
except LookupError:
    print("general lookup handler")

```

Possible output:

```
specific handler: missing dictionary key
```

Here the specific `KeyError` handler comes first. That is correct if missing dictionary keys need special treatment.

If the broader handler comes first, it catches the same exception:

```
scores = {
    "Jerry": 72,
    "Elaine": 91,
}

name = "Kramer"

try:
    print(scores[name])
except LookupError:
    print("general lookup handler")
except KeyError:
    print("specific handler: unreachable here")
```

Possible output:

```
general lookup handler
```

The practical rule is:

Place narrow exception classes first and broad exception classes later.

This is not a style preference. It follows directly from the handler-selection algorithm.

Capturing Exception Objects with `except SomeError as e`

An `except` clause can bind the caught exception object to a name. The usual syntax is:

```
except SomeError as e:
```

The name `e` is not special. It is just a local name bound to the exception object. More descriptive names such as `err` are often clearer in teaching code.

```
values = ["10", "twenty", "42"]

for text in values:
    try:
        number = int(text)
        print(f"{text!r:>8} -> {number:>3}")
    except ValueError as err:
        print(f"{text!r:>8} -> conversion failed")
        print(f"error type: {type(err)}")
        print(f"error args: {err.args}")
```

Possible output:

```
'10' -> 10
'twenty' -> conversion failed
error type: <class 'ValueError'>
error args: ("invalid literal for int() with base 10: 'twenty'",)
'42' -> 42
```

Capturing the object is useful when the program needs the error message, the exact type, the construction arguments, or traceback-related information.

Another example with dictionary lookup:


```

scores = {
    "Jerry": 72,
    "Elaine": 91,
    "Kramer": 42,
}

names = ["Jerry", "Newman", "Kramer"]

for name in names:
    try:
        score = scores[name]
        print(f"{name:<8} | {score:>3}")
    except KeyError as err:
        print(f"{name:<8} | missing key object: {err}")
        print(f"err.args: {err.args}")

```

Possible output:

```

Jerry      | 72
Newman     | missing key object: 'Newman'
err.args:  ('Newman',)
Kramer     | 42

```

For a `KeyError`, the missing key is commonly stored in `err.args`. In this case, the missing key is the string "Newman".

The captured exception object exists only because the exceptional route was taken. If no exception occurs, the `except` block is not entered, and the exception name is not created by that block.

```

text = "42"

try:
    number = int(text)
    print(f"number: {number}")
except ValueError as err:
    print(f"conversion failed: {err}")

print("after try")

```

Possible output:

```

number: 42
after try

```

No `ValueError` object was created here, because the conversion succeeded.

Capturing is also useful when several exception types share one handler:

```

items = ["soup", "pretzels"]

cases = [0, 42, "bad"]

for index in cases:
    try:
        item = items[index]
        print(f"index {index!r}: {item}")
    except (IndexError, TypeError) as err:

```

```

print(f"index {index!r}: lookup failed")
print(f"error type: {type(err)}")
print(f"message: {err}")

```

Possible output:

```

index 0: soup
index 42: lookup failed
error type: <class 'IndexError'>
message: list index out of range
index 'bad': lookup failed
error type: <class 'TypeError'>
message: list indices must be integers or slices, not str

```

The tuple in:

```
except (IndexError, TypeError) as err:
```

means that either exception class can be handled by the same block. The captured object still keeps its exact runtime type.

The Risk of Bare `except` and Over-Broad Exception Traps

A bare `except` clause has no exception class:

```

try:
    risky_operation()
except:
    print("something went wrong")

```

This catches too much. It can catch exceptions that usually should not be swallowed, including interruption and interpreter-exit signals such as `KeyboardInterrupt` and `SystemExit`. For normal application errors, catching `Exception` is already broad; bare `except` is broader still.

A safer teaching rule is:

Catch the exception class that represents the failure you actually know how to handle.

Bad pattern:

```

texts = ["10", "20", "bad", "42"]

for text in texts:
    try:
        number = int(text)
        print(f"{text!r} -> {number}")
    except:
        print("ignored an error")

```

Possible output:

```

'10' -> 10
'20' -> 20
ignored an error
'42' -> 42

```

The program hides what kind of error occurred. It also makes future bugs harder to diagnose, because unrelated mistakes would be swallowed by the same handler.

Better pattern:

```
texts = ["10", "20", "bad", "42"]

for text in texts:
    try:
        number = int(text)
        print(f"{text!r} -> {number}")
    except ValueError as err:
        print(f"{text!r} -> invalid integer text")
        print(f"message: {err}")
```

Possible output:

```
'10' -> 10
'20' -> 20
'bad' -> invalid integer text
message: invalid literal for int() with base 10: 'bad'
'42' -> 42
```

Now the handler says exactly what it expects: `int()` may fail because the text is not a valid integer. Over-broad Exception handlers can also hide bugs:

```
items = ["10", "20", "42"]

try:
    for item in items:
        number = int(item)
        total = total + number
except Exception as err:
    print("some error happened")
    print(f"type(err): {type(err)}")
    print(f"message: {err}")
```

Possible output:

```
some error happened
type(err): <class 'NameError'>
message: name 'total' is not defined
```

The real problem is not bad input. The real problem is a programmer mistake: `total` was used before being assigned. A broad handler catches it, but catching it does not make the program correct.

A better version initializes the state and catches only conversion failures:

```
items = ["10", "20", "42"]
total = 0

for item in items:
    try:
        number = int(item)
    except ValueError as err:
        print(f"invalid integer text: {item!r}")
        print(f"message: {err}")
```

```

    else:
        total = total + number

print(f"total: {total}")

```

Possible output:

```
total: 72
```

The handler is now narrow. It catches the failure the program intends to recover from. Other programming mistakes are allowed to surface.

There are cases where a broad handler is useful, such as logging an unexpected failure before re-raising it. But swallowing a broad exception silently is usually dangerous.

Acceptable diagnostic pattern:

```

try:
    number = int("D'oh!")
except Exception as err:
    print("unexpected failure during conversion block")
    print(f"type(err): {type(err)}")
    print(f"message: {err}")
    raise

```

Possible output:

```

unexpected failure during conversion block
type(err): <class 'ValueError'>
message: invalid literal for int() with base 10: "D'oh!"
Traceback (most recent call last):
...
ValueError: invalid literal for int() with base 10: "D'oh!"

```

The handler records information, then `raise` sends the same exception onward. Re-raising will be discussed later in the propagation section.

The practical rules are:

- Avoid bare `except` in ordinary application code.
- Avoid broad `except Exception` unless there is a clear reason.
- Catch specific exception classes when recovery is specific.
- Do not use exception handling to hide programming mistakes.
- If a broad handler is used for logging, usually re-raise the exception afterward.

For control-flow graph analysis, a bare or over-broad handler creates a very large exceptional edge. Many unrelated failures are routed into the same block. That may keep the program running, but it can also destroy the information needed to understand why the program failed.

6.7.3 Exception Handling Infrastructure

Python exception handling is built around a structured trap. The `try` block marks a region where exceptional control flow may begin. The `except` blocks describe which exception classes can be intercepted. The optional `else` block describes code that should run only when the protected region succeeds. The optional `finally` block describes code that must run whether the protected region succeeds, fails, or transfers control outward.

The full structure is:

```

try:
    protected_statements
except SomeError:
    recovery_statements
else:
    success_only_statements
finally:
    always_run_statements

```

Not every part is required in every program. The most common form is **try/except**. The other clauses exist because exception handling is not only about catching failures. It is also about separating normal success logic from recovery logic and ensuring cleanup logic runs unconditionally.

try marks the danger region. **except** handles selected failures. **else** runs only after success. **finally** runs no matter how the region is left.

The try / except Trap: Intercepting and Deflecting Specific Exception Class Trees

A **try/except** statement creates a controlled region where specific exceptional routes can be intercepted. If an exception is raised inside the **try** block, Python stops the ordinary route through that block and searches the attached **except** clauses for a matching exception class.

```

text = "forty-two"

try:
    number = int(text)
    print(f"number: {number}")
except ValueError:
    print("conversion failed")

print("after try/except")

```

Possible output:

```

conversion failed
after try/except

```

The line:

```
print(f"number: {number}")
```

does not run, because `int(text)` raises `ValueError`. Execution is deflected into the matching `except ValueError:` block.

The control route is:

```

enter try
-> attempt int(text)
    success:
        bind number
        print number
        continue after try/except
    ValueError:
        jump to except ValueError
        print conversion failed
        continue after try/except

```

The `try` block should normally contain only the statements that belong to the risky operation and its immediate success-dependent continuation. If too much unrelated code is placed inside the same `try` block, the handler may catch exceptions from a larger region than intended.

Too broad:

```
text = "42"

try:
    number = int(text)
    print(f"number: {number}")
    missing_name = unknown_variable
except ValueError:
    print("conversion failed")
```

Possible output:

```
number: 42
Traceback (most recent call last):
...
NameError: name 'unknown_variable' is not defined
```

The `NameError` is not caught because the handler catches only `ValueError`. That is good: the handler should not pretend that a programmer mistake is an input conversion problem.

If the handler were too broad, it could hide the real issue:

```
text = "42"

try:
    number = int(text)
    print(f"number: {number}")
    missing_name = unknown_variable
except Exception as err:
    print("some ordinary exception was caught")
    print(f"type(err): {type(err)}")
    print(f"message: {err}")
```

Possible output:

```
number: 42
some ordinary exception was caught
type(err): <class 'NameError'>
message: name 'unknown_variable' is not defined
```

The program did not have a conversion problem. It had a name-resolution problem. Catching `Exception` trapped both possibilities.

A better structure keeps the risky conversion narrow:

```
text = "42"

try:
    number = int(text)
except ValueError as err:
    print("conversion failed")
    print(f"message: {err}")
else:
    print(f"number: {number}")
    print("conversion succeeded")
```

Possible output:

```
number: 42
conversion succeeded
```

The handler now corresponds exactly to the operation that can raise the expected `ValueError`. Multiple handlers can be attached to one `try` block:

```
items = ["10", "20", "D'oh!"]
index = 2

try:
    text = items[index]
    number = int(text)
    print(f"number: {number}")
except IndexError as err:
    print("bad index")
    print(f"type(err): {type(err)}")
except ValueError as err:
    print("bad integer text")
    print(f"type(err): {type(err)}")
    print(f"message: {err}")
```

Possible output:

```
bad integer text
type(err): <class 'ValueError'>
message: invalid literal for int() with base 10: "D'oh!"
```

The first risky operation is indexing. It may raise `IndexError`. The second risky operation is conversion. It may raise `ValueError`. The handlers separate the two exceptional routes.

The exception object is an ordinary runtime object:

```
try:
    number = int("No soup for you!")
except ValueError as err:
    print(f"type(err): {type(err)}")
    print(f"dir(err) sample: {dir(err)[:12]}")
    print(f"err.args: {err.args}")
    print(f"str(err): {str(err)}")
    print(f"is Exception: {isinstance(err, Exception)}")
```

Possible output:

```
type(err): <class 'ValueError'>
dir(err) sample: ['__cause__', '__class__', '__context__', '__delattr__',
 '__dict__', '__dir__', '__doc__', '__eq__', '__format__', '__ge__',
 '__getattr__', '__getstate__']
err.args: ('invalid literal for int() with base 10: 'No soup for you! ',)
str(err): invalid literal for int() with base 10: 'No soup for you!'
is Exception: True
```

The `args` attribute stores the arguments carried by the exception object. The `str()` conversion gives the message. The `isinstance()` test confirms that the object belongs to the exception class tree.

The `try/except` structure is therefore a class-based routing trap:

If the protected route raises an exception whose class matches a handler, Python deflects execution into that handler instead of continuing through the ordinary route.

The Exception `else` Clause: Isolating Code Paths That Run Only When No Exceptions Occur

The `else` clause of a `try` statement runs only if the `try` block finishes without raising an exception. It is not the same as an `if-else` branch. It is a success-only continuation attached to exception handling.

```
text = "42"
```

```
try:
    number = int(text)
except ValueError as err:
    print("conversion failed")
    print(f"message: {err}")
else:
    print("conversion succeeded")
    print(f"number: {number}")
```

Possible output:

```
conversion succeeded
number: 42
```

If the conversion fails, the `else` block is skipped:

```
text = "forty-two"
```

```
try:
    number = int(text)
except ValueError as err:
    print("conversion failed")
    print(f"message: {err}")
else:
    print("conversion succeeded")
    print(f"number: {number}")
```

Possible output:

```
conversion failed
message: invalid literal for int() with base 10: 'forty-two'
```

The route is:

```
try block succeeds:
    -> skip except
    -> run else
```

```
try block raises handled exception:
    -> run matching except
    -> skip else
```

The main reason to use `else` is to keep the protected region narrow. Code in the `else` block depends on success, but it is not part of the risky operation being guarded.

Less precise:


```

text = "42"

try:
    number = int(text)
    print(f"number: {number}")
    print("Nobody expects the Spanish Inquisition!")
except ValueError:
    print("conversion failed")

```

More precise:

```

text = "42"

try:
    number = int(text)
except ValueError:
    print("conversion failed")
else:
    print(f"number: {number}")
    print("Nobody expects the Spanish Inquisition!")

```

Possible output:

```

number: 42
Nobody expects the Spanish Inquisition!

```

The second version says exactly what is being protected: the conversion. The printing after successful conversion is success-path logic, not conversion-risk logic.

The same pattern is useful with dictionary lookup:

```

scores = {
    "Jerry": 72,
    "Elaine": 91,
    "Kramer": 42,
}

name = "Kramer"

try:
    score = scores[name]
except KeyError as err:
    print(f"missing score for: {err}")
else:
    print(f"{name:<8} | {score:>3}")
    print("score lookup succeeded")

```

Possible output:

```

Kramer   | 42
score lookup succeeded

```

If the key is absent:

```

scores = {
    "Jerry": 72,

```

```

    "Elaine": 91,
    "Kramer": 42,
}

name = "Newman"

try:
    score = scores[name]
except KeyError as err:
    print(f"missing score for: {err}")
else:
    print(f"{name:<8} | {score:>3}")
    print("score lookup succeeded")

```

Possible output:

```
missing score for: 'Newman'
```

The dictionary object has its own lookup behavior:

```

scores = {
    "Jerry": 72,
    "Elaine": 91,
    "Kramer": 42,
}

print(f"type(scores): {type(scores)}")
print(f"dir(scores) sample: {dir(scores)[:12]}")
print(f"has __getitem__: {'__getitem__' in dir(scores)}")
print(f"has get: {'get' in dir(scores)}")
print(f"has __contains__: {'__contains__' in dir(scores)}")

```

Possible output:

```

type(scores): <class 'dict'>
dir(scores) sample: ['__class__', '__class_getitem__', '__contains__',
'__delattr__', '__delitem__', '__dir__', '__doc__', '__eq__',
'__format__', '__ge__', '__getattr__', '__getitem__']
has __getitem__: True
has get: True
has __contains__: True

```

The bracket operation `scores[name]` uses lookup behavior associated with `__getitem__()`. A missing key raises `KeyError`. The `else` block runs only if that lookup succeeds.

The practical rule is:

Put only the risky operation in `try`. Put the code that should run after successful completion in `else`.

This keeps exception routes readable. The `except` block describes failure recovery. The `else` block describes the ordinary success continuation.

Cleansing and Post-Processing: The Unconditional Execution Invariants of the `finally` Block

The `finally` block runs after the `try` structure is left, regardless of whether the protected code succeeded, failed, or was interrupted by a control-flow transfer such as `break`, `return`, or an uncaught exception.

At this stage of the chapter, the simplest rule is:

`finally` means: run this cleanup code no matter how the protected region was exited.

Success case:

```
text = "42"

try:
    number = int(text)
    print(f"number: {number}")
except ValueError:
    print("conversion failed")
finally:
    print("finally block ran")

print("after try statement")
```

Possible output:

```
number: 42
finally block ran
after try statement
```

Handled failure case:

```
text = "forty-two"

try:
    number = int(text)
    print(f"number: {number}")
except ValueError:
    print("conversion failed")
finally:
    print("finally block ran")

print("after try statement")
```

Possible output:

```
conversion failed
finally block ran
after try statement
```

In both programs, the `finally` block runs.

The `finally` block also runs when the exception is not handled locally. To demonstrate that without stopping the entire script, an outer `try` can catch the propagated exception:

```

try:
    try:
        print("inner try begins")
        number = int("D'oh!")
        print(f"number: {number}")
    finally:
        print("inner finally ran before propagation")
except ValueError as err:
    print("outer handler caught the propagated exception")
    print(f"type(err): {type(err)}")
    print(f"message: {err}")

```

Possible output:

```

inner try begins
inner finally ran before propagation
outer handler caught the propagated exception
type(err): <class 'ValueError'>
message: invalid literal for int() with base 10: "D'oh!"

```

The inner `try` has no `except` handler. The `ValueError` must propagate outward. But before that propagation reaches the outer handler, the inner `finally` block runs.

This means `finally` is not a success block and not a failure block. It is an exit block.

A typical use is cleanup state:

```

resource_open = False

try:
    print("opening resource")
    resource_open = True

    print("using resource")
    number = int("42")
    print(f"number: {number}")
finally:
    print("cleanup section")
    resource_open = False
    print(f"resource_open: {resource_open}")

```

Possible output:

```

opening resource
using resource
number: 42
cleanup section
resource_open: False

```

If an error happens, cleanup still runs:

```

resource_open = False

try:
    print("opening resource")
    resource_open = True

```

```

    print("using resource")
    number = int("forty-two")
    print(f"number: {number}")
except ValueError as err:
    print("conversion failed")
    print(f"message: {err}")
finally:
    print("cleanup section")
    resource_open = False
    print(f"resource_open: {resource_open}")

```

Possible output:

```

opening resource
using resource
conversion failed
message: invalid literal for int() with base 10: 'forty-two'
cleanup section
resource_open: False

```

The variable `resource_open` is only a simple teaching flag here. In real programs, the cleanup section commonly closes files, releases locks, closes network connections, ends temporary states, or restores invariants.

A `finally` block can also run when a loop is interrupted with `break`:

```

items = ["soup", "pretzels", "muffin"]

for item in items:
    try:
        print(f"checking: {item}")

        if item == "pretzels":
            print("pretzels found")
            break
    finally:
        print(f"finished attempt for: {item}")

print("after loop")

```

Possible output:

```

checking: soup
finished attempt for: soup
checking: pretzels
pretzels found
finished attempt for: pretzels
after loop

```

The `break` leaves the loop, but the `finally` block for the current iteration still runs before execution reaches the statement after the loop.

The full clause order is fixed:

```

try:
    ...
except SomeError:

```

```

    ...
else:
    ...
finally:
    ...

```

The `else` clause, when present, comes after all `except` clauses and before `finally`. The `finally` block is last because it runs after the success or failure route has been determined.

All three routes together:

```

texts = ["42", "forty-two"]

for text in texts:
    print(f"input: {text!r}")

    try:
        number = int(text)
    except ValueError as err:
        print("except route")
        print(f"message: {err}")
    else:
        print("else route")
        print(f"number: {number}")
    finally:
        print("finally route")
        print("-" * 20)

```

Possible output:

```

input: '42'
else route
number: 42
finally route
-----
input: 'forty-two'
except route
message: invalid literal for int() with base 10: 'forty-two'
finally route
-----

```

For the first input, the `try` block succeeds, so `else` runs, then `finally`. For the second input, the `try` block raises `ValueError`, so `except` runs, then `finally`.

The practical reading is:

- `try`: attempt the protected operation;
- `except`: recover from selected exception classes;
- `else`: run success-only code when no exception occurred;
- `finally`: run cleanup code regardless of success or failure.

For control-flow graph analysis, `finally` adds a guaranteed exit checkpoint. Ordinary success routes, handled exception routes, unhandled propagation routes, and local loop-interruption routes must pass through it before continuing outward.

6.7.4 Exception Handling Infrastructure

Python exception handling is built around a structured trap. The `try` block marks a region where exceptional control flow may begin. The `except` blocks describe which exception classes can be intercepted. The optional `else` block describes code that should run only when the protected region succeeds. The optional `finally` block describes code that must run whether the protected region succeeds, fails, or transfers control outward.

The full structure is:

```
try:
    protected_statements
except SomeError:
    recovery_statements
else:
    success_only_statements
finally:
    always_run_statements
```

Not every part is required in every program. The most common form is `try/except`. The other clauses exist because exception handling is not only about catching failures. It is also about separating normal success logic from recovery logic and ensuring cleanup logic runs unconditionally.

`try` marks the danger region. `except` handles selected failures. `else` runs only after success. `finally` runs no matter how the region is left.

The `try / except` Trap: Intercepting and Deflecting Specific Exception Class Trees

A `try/except` statement creates a controlled region where specific exceptional routes can be intercepted. If an exception is raised inside the `try` block, Python stops the ordinary route through that block and searches the attached `except` clauses for a matching exception class.

```
text = "forty-two"

try:
    number = int(text)
    print(f"number: {number}")
except ValueError:
    print("conversion failed")

print("after try/except")
```

Possible output:

```
conversion failed
after try/except
```

The line:

```
print(f"number: {number}")
```

does not run, because `int(text)` raises `ValueError`. Execution is deflected into the matching `except ValueError:` block.

The control route is:

```

enter try
  -> attempt int(text)
      success:
          bind number
          print number
          continue after try/except
      ValueError:
          jump to except ValueError
          print conversion failed
          continue after try/except

```

The `try` block should normally contain only the statements that belong to the risky operation and its immediate success-dependent continuation. If too much unrelated code is placed inside the same `try` block, the handler may catch exceptions from a larger region than intended.

Too broad:

```

text = "42"

try:
    number = int(text)
    print(f"number: {number}")
    missing_name = unknown_variable
except ValueError:
    print("conversion failed")

```

Possible output:

```

number: 42
Traceback (most recent call last):
...
NameError: name 'unknown_variable' is not defined

```

The `NameError` is not caught because the handler catches only `ValueError`. That is good: the handler should not pretend that a programmer mistake is an input conversion problem.

If the handler were too broad, it could hide the real issue:

```

text = "42"

try:
    number = int(text)
    print(f"number: {number}")
    missing_name = unknown_variable
except Exception as err:
    print("some ordinary exception was caught")
    print(f"type(err): {type(err)}")
    print(f"message: {err}")

```

Possible output:

```

number: 42
some ordinary exception was caught
type(err): <class 'NameError'>
message: name 'unknown_variable' is not defined

```

The program did not have a conversion problem. It had a name-resolution problem. Catching `Exception` trapped both possibilities.

A better structure keeps the risky conversion narrow:


```

text = "42"

try:
    number = int(text)
except ValueError as err:
    print("conversion failed")
    print(f"message: {err}")
else:
    print(f"number: {number}")
    print("conversion succeeded")

```

Possible output:

```

number: 42
conversion succeeded

```

The handler now corresponds exactly to the operation that can raise the expected `ValueError`. Multiple handlers can be attached to one `try` block:

```

items = ["10", "20", "D'oh!"]
index = 2

try:
    text = items[index]
    number = int(text)
    print(f"number: {number}")
except IndexError as err:
    print("bad index")
    print(f"type(err): {type(err)}")
except ValueError as err:
    print("bad integer text")
    print(f"type(err): {type(err)}")
    print(f"message: {err}")

```

Possible output:

```

bad integer text
type(err): <class 'ValueError'>
message: invalid literal for int() with base 10: "D'oh!"

```

The first risky operation is indexing. It may raise `IndexError`. The second risky operation is conversion. It may raise `ValueError`. The handlers separate the two exceptional routes.

The exception object is an ordinary runtime object:

```

try:
    number = int("No soup for you!")
except ValueError as err:
    print(f"type(err): {type(err)}")
    print(f"dir(err) sample: {dir(err)[:12]}")
    print(f"err.args: {err.args}")
    print(f"str(err): {str(err)}")
    print(f"is Exception: {isinstance(err, Exception)}")

```

Possible output:

```

type(err): <class 'ValueError'>
dir(err) sample: ['__cause__', '__class__', '__context__', '__delattr__',
 '__dict__', '__dir__', '__doc__', '__eq__', '__format__', '__ge__',
 '__getattr__', '__getstate__']
err.args: ('invalid literal for int() with base 10: 'No soup for you!',)
str(err): invalid literal for int() with base 10: 'No soup for you!'
is Exception: True

```

The `args` attribute stores the arguments carried by the exception object. The `str()` conversion gives the message. The `isinstance()` test confirms that the object belongs to the exception class tree.

The `try/except` structure is therefore a class-based routing trap:

If the protected route raises an exception whose class matches a handler, Python deflects execution into that handler instead of continuing through the ordinary route.

The Exception `else` Clause: Isolating Code Paths That Run Only When No Exceptions Occur

The `else` clause of a `try` statement runs only if the `try` block finishes without raising an exception. It is not the same as an `if-else` branch. It is a success-only continuation attached to exception handling.

```

text = "42"

try:
    number = int(text)
except ValueError as err:
    print("conversion failed")
    print(f"message: {err}")
else:
    print("conversion succeeded")
    print(f"number: {number}")

```

Possible output:

```

conversion succeeded
number: 42

```

If the conversion fails, the `else` block is skipped:

```

text = "forty-two"

try:
    number = int(text)
except ValueError as err:
    print("conversion failed")
    print(f"message: {err}")
else:
    print("conversion succeeded")
    print(f"number: {number}")

```

Possible output:

```

conversion failed
message: invalid literal for int() with base 10: 'forty-two'

```

The route is:

```
try block succeeds:
    -> skip except
    -> run else
```

```
try block raises handled exception:
    -> run matching except
    -> skip else
```

The main reason to use `else` is to keep the protected region narrow. Code in the `else` block depends on success, but it is not part of the risky operation being guarded.

Less precise:

```
text = "42"
```

```
try:
    number = int(text)
    print(f"number: {number}")
    print("Nobody expects the Spanish Inquisition!")
except ValueError:
    print("conversion failed")
```

More precise:

```
text = "42"
```

```
try:
    number = int(text)
except ValueError:
    print("conversion failed")
else:
    print(f"number: {number}")
    print("Nobody expects the Spanish Inquisition!")
```

Possible output:

```
number: 42
Nobody expects the Spanish Inquisition!
```

The second version says exactly what is being protected: the conversion. The printing after successful conversion is success-path logic, not conversion-risk logic.

The same pattern is useful with dictionary lookup:

```
scores = {
    "Jerry": 72,
    "Elaine": 91,
    "Kramer": 42,
}

name = "Kramer"

try:
    score = scores[name]
except KeyError as err:
    print(f"missing score for: {err}")
else:
    print(f"{name:<8} | {score:>3}")
    print("score lookup succeeded")
```

Possible output:

```
Kramer    | 42
score lookup succeeded
```

If the key is absent:

```
scores = {
    "Jerry": 72,
    "Elaine": 91,
    "Kramer": 42,
}

name = "Newman"

try:
    score = scores[name]
except KeyError as err:
    print(f"missing score for: {err}")
else:
    print(f"{name:<8} | {score:>3}")
    print("score lookup succeeded")
```

Possible output:

```
missing score for: 'Newman'
```

The dictionary object has its own lookup behavior:

```
scores = {
    "Jerry": 72,
    "Elaine": 91,
    "Kramer": 42,
}

print(f"type(scores): {type(scores)}")
print(f"dir(scores) sample: {dir(scores)[:12]}")
print(f"has __getitem__: {'__getitem__' in dir(scores)}")
print(f"has get: {'get' in dir(scores)}")
print(f"has __contains__: {'__contains__' in dir(scores)}")
```

Possible output:

```
type(scores): <class 'dict'>
dir(scores) sample: ['__class__', '__class_getitem__', '__contains__',
'__delattr__', '__delitem__', '__dir__', '__doc__', '__eq__',
'__format__', '__ge__', '__getattribute__', '__getitem__']
has __getitem__: True
has get: True
has __contains__: True
```

The bracket operation `scores[name]` uses lookup behavior associated with `__getitem__()`. A missing key raises `KeyError`. The `else` block runs only if that lookup succeeds.

The practical rule is:

Put only the risky operation in `try`. Put the code that should run after successful completion in `else`.

This keeps exception routes readable. The `except` block describes failure recovery. The `else` block describes the ordinary success continuation.

Cleansing and Post-Processing: The Unconditional Execution Invariants of the `finally` Block

The `finally` block runs after the `try` structure is left, regardless of whether the protected code succeeded, failed, or was interrupted by a control-flow transfer such as `break`, `return`, or an uncaught exception.

At this stage of the chapter, the simplest rule is:

`finally` means: run this cleanup code no matter how the protected region was exited.

Success case:

```
text = "42"

try:
    number = int(text)
    print(f"number: {number}")
except ValueError:
    print("conversion failed")
finally:
    print("finally block ran")

print("after try statement")
```

Possible output:

```
number: 42
finally block ran
after try statement
```

Handled failure case:

```
text = "forty-two"

try:
    number = int(text)
    print(f"number: {number}")
except ValueError:
    print("conversion failed")
finally:
    print("finally block ran")

print("after try statement")
```

Possible output:

```
conversion failed
finally block ran
after try statement
```

In both programs, the `finally` block runs.

The `finally` block also runs when the exception is not handled locally. To demonstrate that without stopping the entire script, an outer `try` can catch the propagated exception:

```

try:
    try:
        print("inner try begins")
        number = int("D'oh!")
        print(f"number: {number}")
    finally:
        print("inner finally ran before propagation")
except ValueError as err:
    print("outer handler caught the propagated exception")
    print(f"type(err): {type(err)}")
    print(f"message: {err}")

```

Possible output:

```

inner try begins
inner finally ran before propagation
outer handler caught the propagated exception
type(err): <class 'ValueError'>
message: invalid literal for int() with base 10: "D'oh!"

```

The inner `try` has no `except` handler. The `ValueError` must propagate outward. But before that propagation reaches the outer handler, the inner `finally` block runs.

This means `finally` is not a success block and not a failure block. It is an exit block.

A typical use is cleanup state:

```

resource_open = False

try:
    print("opening resource")
    resource_open = True

    print("using resource")
    number = int("42")
    print(f"number: {number}")
finally:
    print("cleanup section")
    resource_open = False
    print(f"resource_open: {resource_open}")

```

Possible output:

```

opening resource
using resource
number: 42
cleanup section
resource_open: False

```

If an error happens, cleanup still runs:

```

resource_open = False

try:
    print("opening resource")
    resource_open = True

```

```

    print("using resource")
    number = int("forty-two")
    print(f"number: {number}")
except ValueError as err:
    print("conversion failed")
    print(f"message: {err}")
finally:
    print("cleanup section")
    resource_open = False
    print(f"resource_open: {resource_open}")

```

Possible output:

```

opening resource
using resource
conversion failed
message: invalid literal for int() with base 10: 'forty-two'
cleanup section
resource_open: False

```

The variable `resource_open` is only a simple teaching flag here. In real programs, the cleanup section commonly closes files, releases locks, closes network connections, ends temporary states, or restores invariants.

A `finally` block can also run when a loop is interrupted with `break`:

```

items = ["soup", "pretzels", "muffin"]

for item in items:
    try:
        print(f"checking: {item}")

        if item == "pretzels":
            print("pretzels found")
            break
    finally:
        print(f"finished attempt for: {item}")

print("after loop")

```

Possible output:

```

checking: soup
finished attempt for: soup
checking: pretzels
pretzels found
finished attempt for: pretzels
after loop

```

The `break` leaves the loop, but the `finally` block for the current iteration still runs before execution reaches the statement after the loop.

The full clause order is fixed:

```

try:
    ...
except SomeError:

```

```

    ...
else:
    ...
finally:
    ...

```

The `else` clause, when present, comes after all `except` clauses and before `finally`. The `finally` block is last because it runs after the success or failure route has been determined.

All three routes together:

```

texts = ["42", "forty-two"]

for text in texts:
    print(f"input: {text!r}")

    try:
        number = int(text)
    except ValueError as err:
        print("except route")
        print(f"message: {err}")
    else:
        print("else route")
        print(f"number: {number}")
    finally:
        print("finally route")
        print("-" * 20)

```

Possible output:

```

input: '42'
else route
number: 42
finally route
-----
input: 'forty-two'
except route
message: invalid literal for int() with base 10: 'forty-two'
finally route
-----

```

For the first input, the `try` block succeeds, so `else` runs, then `finally`. For the second input, the `try` block raises `ValueError`, so `except` runs, then `finally`.

The practical reading is:

- `try`: attempt the protected operation;
- `except`: recover from selected exception classes;
- `else`: run success-only code when no exception occurred;
- `finally`: run cleanup code regardless of success or failure.

For control-flow graph analysis, `finally` adds a guaranteed exit checkpoint. Ordinary success routes, handled exception routes, unhandled propagation routes, and local loop-interruption routes must pass through it before continuing outward.

Chapter 7

Hash-Based Collections and Associative Mappings

Sequential containers organize components by position. Hash-based containers organize access around membership, identity rules, and key-based lookup. This chapter moves from the ordered world of strings, lists, and tuples into the non-sequential storage architecture used by `set`, `frozenset`, and `dict`.

The central mechanism is hashing. Python uses hash values, together with equality checks, to place and retrieve elements in internal tables. This makes sets efficient for uniqueness and membership testing, and dictionaries efficient for associating stable keys with stored values.

The chapter first examines sets as unordered domains of unique hashable elements. It then studies mathematical set operations, in-place set mutation, and structural relationship tests. The final part turns to dictionaries, where the same hash-table principles support key-value mapping, dynamic views, insertion-order behavior, safe lookup, mutation, merging, and deletion.

7.1 Hash Tables as Non-Sequential Component Access Structures

Sequential containers teach one model of storage: components arranged in a visible order and accessed by integer position. A string, list, or tuple can answer questions such as “what is at index 0?” or “what slice lies between these two boundaries?” That model is useful, but it is not the only way to organize components.

Sets and dictionaries use a different strategy. They are not primarily built around position. They are built around hash-based access. A set asks whether a value is present. A dictionary asks which value is associated with a key. In both cases, Python uses a hash value to move quickly toward the internal table area where the element or key may be stored.

This section introduces that change of perspective. The focus is not yet on all set and dictionary operations, but on the access architecture underneath them: how Python moves from sequential indexing to hash-derived placement, why equality is still needed after hashing, and why only hash-stable objects may safely participate as set elements or dictionary keys.

7.1.1 From Positional Access to Hash-Based Access

The previous chapter studied Python objects whose components are arranged as sequences: `str`, `list`, and `tuple`. Their access model is positional. If an object is a sequence, the question “where is the component?” can be answered with an integer index.

Hash-based collections use a different access model. A `set` does not ask for the third element. A `dict` does not normally ask for the value at position three. Instead, these objects use hash-derived table placement. The stored object, or the stored key, is processed into an integer hash value, and that value guides Python toward a table slot where the relevant entry may be found.

This is the main transition of the chapter:

Sequential collections locate components by position. Hash-based collections locate components by value identity rules: hash first, equality confirmation after that.

The difference is not cosmetic syntax. It changes what the structure is good at. A list is natural when order and position matter. A set is natural when uniqueness and membership matter. A dictionary is natural when a key must lead directly to an associated value.

Sequential Collections: Why Lists, Tuples, and Strings Locate Components by Index

A sequence is built around component order. The object has a first component, a second component, a third component, and so on. Python exposes that model through indexing, slicing, iteration, and length calculation.

```
text = "D'oh!"
items = ["spam", "eggs", 42, "larch"]
point = ("x", "y", "z")

print(f"text: {text!r}")
print(f"items: {items!r}")
print(f"point: {point!r}")

print()
print(f"type(text): {type(text)}")
print(f"type(items): {type(items)}")
print(f"type(point): {type(point)}")
```

Possible output:

```
text: "D'oh!"
items: ['spam', 'eggs', 42, 'larch']
point: ('x', 'y', 'z')

type(text): <class 'str'>
type(items): <class 'list'>
type(point): <class 'tuple'>
```

The common mechanism is visible if we inspect a few selected names from `dir()`.

```
text = "D'oh!"
items = ["spam", "eggs", 42, "larch"]
point = ("x", "y", "z")

names = ["__len__", "__getitem__", "__contains__", "index", "count"]

for obj_name, obj in [("text", text), ("items", items), ("point", point)]:
    print(f"{obj_name}: {type(obj)}")
    for name in names:
        print(f" {name:<14} {name in dir(obj)}")
    print()
```

Possible output:

```
text: <class 'str'>
__len__      True
__getitem__  True
```

```

__contains__    True
index           True
count           True

items: <class 'list'>
__len__         True
__getitem__     True
__contains__    True
index           True
count           True

point: <class 'tuple'>
__len__         True
__getitem__     True
__contains__    True
index           True
count           True

```

The important name here is `__getitem__`. It is the operation behind square-bracket access.

```

items = ["spam", "eggs", 42, "larch"]

print(f"items[0]: {items[0]!r}")
print(f"items[1]: {items[1]!r}")
print(f"items[-1]: {items[-1]!r}")
print(f"items[1:3]: {items[1:3]!r}")

```

Possible output:

```

items[0]: 'spam'
items[1]: 'eggs'
items[-1]: 'larch'
items[1:3]: ['eggs', 42]

```

For a sequence, the integer index is meaningful because the container has a positional structure. Index 0 means the first component. Index 1 means the second component. A slice such as `items[1:3]` means a new sequence containing the components found from one positional boundary to another.

Membership testing is available too, but it is not the same kind of access.

```

items = ["spam", "eggs", 42, "larch"]

print(f"{42 in items = }")
print(f"{'beans' in items = }")
print(f"{items.index(42) = }")

```

Possible output:

```

42 in items = True
'beans' in items = False
items.index(42) = 2

```

In a list, membership testing checks whether a value is present. The method `index()` then reports where that value was found. This still belongs to the positional world: after the value is discovered, Python can report a numeric location.

At the conceptual level, a sequence resembles a row of slots:

```

index:    0        1        2        3
value:   "spam"   "eggs"   42      "larch"

```

The position is part of the structure. If two elements are swapped, the sequence has changed even if it still contains the same values.

```

a = ["spam", "eggs", 42]
b = [42, "eggs", "spam"]

print(f"a: {a!r}")
print(f"b: {b!r}")
print(f"a == b: {a == b}")

```

Possible output:

```

a: ['spam', 'eggs', 42]
b: [42, 'eggs', 'spam']
a == b: False

```

The two lists contain the same three values, but not in the same positions. For a sequence, that is a different structure.

Hash-Based Collections: Why Sets and Dictionaries Locate Components by Hash-Derived Table Slots

A hash-based collection is not organized around integer positions. It is organized around hashable values.

Python's main built-in hash-based collections are:

- **set**: stores unique elements;
- **dict**: stores key-value associations.

A first inspection shows that these objects do not expose the ordinary sequence indexing model.

```

names = {"Arthur", "Brian", "Lancelot"}
quotes = {
    "Homer": "D'oh!",
    "Jerry": "What's the deal with airline food?",
    "Arthur": "It is only a model.",
}

print(f"names: {names!r}")
print(f"quotes: {quotes!r}")

print()
print(f"type(names): {type(names)}")
print(f"type(quotes): {type(quotes)}")

```

Possible output:

```

names: {'Brian', 'Arthur', 'Lancelot'}
quotes: {'Homer': 'D'oh!', 'Jerry': 'What's the deal with airline food?', 'Arthur': 'It is only a

type(names): <class 'set'>
type(quotes): <class 'dict'>

```

The displayed order of the set must not be read as the structural order of the set. A set is not a row of positions. It is a uniqueness domain: each accepted element is present once.

The selected `dir()` inspection also shows the change in available behavior.

```
names = {"Arthur", "Brian", "Lancelot"}
quotes = {
    "Homer": "D'oh!",
    "Jerry": "What's the deal with airline food?",
    "Arthur": "It is only a model.",
}

set_names = ["__len__", "__contains__", "__getitem__", "add", "remove", "union"]
dict_names = ["__len__", "__contains__", "__getitem__", "get", "keys", "items"]

print(f"names: {type(names)}")
for name in set_names:
    print(f"  {name:<14} {name in dir(names)}")

print()
print(f"quotes: {type(quotes)}")
for name in dict_names:
    print(f"  {name:<14} {name in dir(quotes)}")
```

Possible output:

```
names: <class 'set'>
__len__      True
__contains__ True
__getitem__  False
add          True
remove       True
union        True

quotes: <class 'dict'>
__len__      True
__contains__ True
__getitem__  True
get          True
keys         True
items        True
```

A set has no `__getitem__` operation because there is no meaningful `names[0]` access. The set can answer whether an element is present, but it is not designed to answer which element occupies the first, second, or third position.

```
names = {"Arthur", "Brian", "Lancelot"}

print(f"{'Arthur' in names = }")
print(f"{'Galahad' in names = }")

print()
try:
    print(names[0])
except TypeError as err:
    print(f"set indexing failed: {err}")
```

Possible output:

```
'Arthur' in names = True
'Galahad' in names = False
```

```
set indexing failed: 'set' object is not subscriptable
```

A dictionary does have `__getitem__`, but the square brackets do not mean positional access. In a dictionary, the bracket expression supplies a key, not an index.

```
quotes = {
    "Homer": "D'oh!",
    "Jerry": "What's the deal with airline food?",
    "Arthur": "It is only a model.",
}
```

```
print(f"quotes['Homer']: {quotes['Homer']!r}")
print(f"quotes['Arthur']: {quotes['Arthur']!r}")
```

```
print()
print(f"{'Homer' in quotes = }")
print(f"{'D\\'oh!' in quotes = }")
```

Possible output:

```
quotes['Homer']: "D'oh!"
quotes['Arthur']: 'It is only a model.'
```

```
'Homer' in quotes = True
"D'oh!" in quotes = False
```

The expression `'Homer' in quotes` checks whether `'Homer'` is a key. It does not check whether `'Homer'` appears somewhere among the values. This is one of the most important dictionary boundaries: membership belongs to the key domain.

The bridge from value to table slot is the hash value. Python exposes this through the built-in `hash()` function.

```
values = ["Arthur", "Brian", "Lancelot", 42]
```

```
for value in values:
    print(f"value={value!r:<12} type={type(value)} hash={hash(value)}")
```

Possible output, with hash integers shortened here because some hash values may differ between Python runs:

```
value='Arthur'      type=<class 'str'> hash=...
value='Brian'       type=<class 'str'> hash=...
value='Lancelot'    type=<class 'str'> hash=...
value=42            type=<class 'int'> hash=42
```

The hash value is not the stored value itself. It is an integer descriptor used by the hash table machinery to narrow the search area. For a set, the element is hashed. For a dictionary, the key is hashed.

Conceptually:

```

set lookup:
    element -> hash(element) -> candidate table slot -> equality check

dict lookup:
    key -> hash(key) -> candidate table slot -> equality check -> associated value

```

This explains why a set can check membership without scanning from the first element to the last in the ordinary list sense. It also explains why a dictionary can retrieve a value by key without asking where that pair appears visually when printed.

```

names = {"Arthur", "Brian", "Lancelot"}
quotes = {
    "Homer": "D'oh!",
    "Jerry": "What's the deal with airline food?",
    "Arthur": "It is only a model.",
}

print("set membership")
print(f'{"Brian" in names = }')

print()
print("dictionary key lookup")
key = "Jerry"
print(f"key: {key!r}")
print(f"value: {quotes[key]!r}")

```

Possible output:

```

set membership
'Brian' in names = True

dictionary key lookup
key: 'Jerry'
value: "What's the deal with airline food?"

```

The practical distinction is therefore simple:

- use a sequence when position is part of the meaning;
- use a set when uniqueness and membership are the main operations;
- use a dictionary when a key must lead to a value.

A list answers: “what object is stored at this numeric position?”

A set answers: “is this hashable object present?”

A dictionary answers: “which value is associated with this hashable key?”

7.1.2 The Hash and Equality Contract

Hash-based collections do not place objects by walking through a visible sequence of positions. They first ask an object for a hash value. That hash value guides the table lookup. After Python finds a plausible table location, it still needs equality testing to confirm whether the discovered object is truly the object being searched for.

The two operations involved are:

- `__hash__()`: produces an integer used by the hash table machinery;

- `__eq__()`: decides whether two objects should be treated as equal.

The relationship between them is strict. If two objects compare equal, they must produce the same hash value. Without that rule, a set or dictionary could place equal objects in incompatible table regions and later fail to find them correctly.

Hashing narrows the search. Equality confirms the match.

`__hash__()` as the Stable Integer Descriptor Used for Table Placement

The built-in `hash()` function calls an object's hash operation. The result is an integer. This integer is not the object's memory address in the ordinary Python-language sense, and it is not the object itself. It is a descriptor used by the hash table implementation.

```
value1 = 42
value2 = "spam"
value3 = ("eggs", 42)

print(f"value1: {value1!r}")
print(f"type(value1): {type(value1)}")
print(f"hash(value1): {hash(value1)}")
print()

print(f"value2: {value2!r}")
print(f"type(value2): {type(value2)}")
print(f"hash(value2): {hash(value2)}")
print()

print(f"value3: {value3!r}")
print(f"type(value3): {type(value3)}")
print(f"hash(value3): {hash(value3)}")
```

Possible output:

```
value1: 42
type(value1): <class 'int'>
hash(value1): 42

value2: 'spam'
type(value2): <class 'str'>
hash(value2): -7236028022913602481

value3: ('eggs', 42)
type(value3): <class 'tuple'>
hash(value3): 5120461032027266771
```

The exact hash value for strings may differ between Python runs. CPython deliberately randomizes string hashing between interpreter processes. The important fact is not the exact number printed. The important fact is that during one execution, the object has a hash value that the table machinery can use.

A few relevant methods can be seen with `dir()`.

```
value1 = 42
value2 = "spam"
value3 = ("eggs", 42)
```



```

for name, obj in [("value1", value1), ("value2", value2), ("value3", value3)]:
    print(f"{name}: {type(obj)}")
    print(f"    __hash__ in dir: {'__hash__' in dir(obj)}")
    print(f"    __eq__    in dir: {'__eq__' in dir(obj)}")
    print(f"    __repr__ in dir: {'__repr__' in dir(obj)}")
    print()

```

Possible output:

```

value1: <class 'int'>
    __hash__ in dir: True
    __eq__    in dir: True
    __repr__ in dir: True

value2: <class 'str'>
    __hash__ in dir: True
    __eq__    in dir: True
    __repr__ in dir: True

value3: <class 'tuple'>
    __hash__ in dir: True
    __eq__    in dir: True
    __repr__ in dir: True

```

The relevant methods are:

- `__hash__()`: returns the integer descriptor used for hash-table placement;
- `__eq__()`: checks value equality;
- `__repr__()`: produces the developer-facing representation shown by `repr()` and often by containers.

The hash value becomes visible in set behavior. A set does not store repeated equal elements.

```

numbers = {42, 42, 42}
words = {"spam", "spam", "eggs"}

print(f"numbers: {numbers!r}")
print(f"type(numbers): {type(numbers)}")
print(f"len(numbers): {len(numbers)}")
print()

print(f"words: {words!r}")
print(f"type(words): {type(words)}")
print(f"len(words): {len(words)}")

```

Possible output:

```

numbers: {42}
type(numbers): <class 'set'>
len(numbers): 1

words: {'eggs', 'spam'}
type(words): <class 'set'>
len(words): 2

```

Inspecting the set object:

```
words = {"spam", "spam", "eggs"}

print(f"type(words): {type(words)}")
print(f"__contains__ in dir: {'__contains__' in dir(words)}")
print(f"add          in dir: {'add' in dir(words)}")
print(f"remove       in dir: {'remove' in dir(words)}")
print(f"union        in dir: {'union' in dir(words)}")
```

Possible output:

```
type(words): <class 'set'>
__contains__ in dir: True
add          in dir: True
remove       in dir: True
union        in dir: True
```

For now, the important set operation is `__contains__`, which is behind the `in` operator.

```
words = {"spam", "eggs"}

print(f"{'spam' in words = }")
print(f"{'larch' in words = }")
```

Possible output:

```
'spam' in words = True
'larch' in words = False
```

When Python evaluates `'spam' in words`, it does not need the question “at which index is `'spam'`?” A set has no such index. The question is instead: compute the hash for `'spam'`, use it to find a candidate table location, then use equality testing if a possible match is found.

`__eq__()` as the Equality Check Used After Candidate Slot Discovery

A hash value is not enough to prove equality. Two different objects can have the same hash value. This is called a collision. A collision is not an error. Hash tables are designed to handle it.

Because collisions are possible, Python must still use equality testing after hash-based candidate discovery.

```
a = "spam"
b = "spam"
c = "eggs"

print(f"a: {a!r}")
print(f"b: {b!r}")
print(f"c: {c!r}")
print()

print(f"a == b: {a == b}")
print(f"a == c: {a == c}")
print()

print(f"a.__eq__(b): {a.__eq__(b)}")
print(f"a.__eq__(c): {a.__eq__(c)}")
```

Possible output:

```
a: 'spam'
b: 'spam'
c: 'eggs'

a == b: True
a == c: False

a.__eq__(b): True
a.__eq__(c): False
```

The operator `==` is ordinary syntax for calling equality behavior. For built-in strings, equality means same character sequence. For integers, equality means same numeric value. For tuples, equality means the same length and equal corresponding components.

```
t1 = ("Arthur", 42)
t2 = ("Arthur", 42)
t3 = ("Arthur", 43)

print(f"t1: {t1!r}")
print(f"t2: {t2!r}")
print(f"t3: {t3!r}")
print()

print(f"t1 == t2: {t1 == t2}")
print(f"t1 == t3: {t1 == t3}")
print()

print(f"hash(t1): {hash(t1)}")
print(f"hash(t2): {hash(t2)}")
print(f"hash(t3): {hash(t3)}")
```

Possible output:

```
t1: ('Arthur', 42)
t2: ('Arthur', 42)
t3: ('Arthur', 43)

t1 == t2: True
t1 == t3: False

hash(t1): -5207665904908724718
hash(t2): -5207665904908724718
hash(t3): 7785522842538555942
```

The first two tuples are distinct objects, but they compare equal. Because they compare equal, they also have the same hash value.

That is exactly what a hash table needs.

```
seen = set()

t1 = ("Arthur", 42)
t2 = ("Arthur", 42)
t3 = ("Arthur", 43)
```

```

seen.add(t1)
seen.add(t2)
seen.add(t3)

print(f"seen: {seen!r}")
print(f"len(seen): {len(seen)}")

```

Possible output:

```

seen: {('Arthur', 42), ('Arthur', 43)}
len(seen): 2

```

The object `t2` was not added as a third distinct element because the set found an existing equal element. The hash guided Python toward the relevant table region. Equality confirmed that `t1` and `t2` represent the same set element.

A dictionary uses the same idea for keys.

```

quote_by_key = {}

key1 = ("Homer", 42)
key2 = ("Homer", 42)

quote_by_key[key1] = "D'oh!"
quote_by_key[key2] = "Mmm... donuts."

print(f"quote_by_key: {quote_by_key!r}")
print(f"len(quote_by_key): {len(quote_by_key)}")
print(f"quote_by_key[key1]: {quote_by_key[key1]!r}")
print(f"quote_by_key[key2]: {quote_by_key[key2]!r}")

```

Possible output:

```

quote_by_key: {('Homer', 42): 'Mmm... donuts.'}
len(quote_by_key): 1
quote_by_key[key1]: 'Mmm... donuts.'
quote_by_key[key2]: 'Mmm... donuts.'

```

The second assignment did not create a second key. It replaced the value associated with the existing equal key.

This is not list behavior. It is associative mapping behavior. The dictionary key is not a position. It is a hashable lookup object.

The Required Invariant: Equal Objects Must Produce Equal Hash Values

The central rule is:

If `a == b` is true, then `hash(a) == hash(b)` must also be true.

The reverse is not required. If two objects have the same hash value, they do not have to be equal. Equal objects must share a hash value, but shared hash values do not prove equality.

The following examples show the required direction.

```

pairs = [
    (42, 42),
    ("spam", "spam"),

```

```

    (("eggs", 42), ("eggs", 42)),
]

for a, b in pairs:
    print(f"a: {a!r}")
    print(f"b: {b!r}")
    print(f"a == b: {a == b}")
    print(f"hash(a): {hash(a)}")
    print(f"hash(b): {hash(b)}")
    print(f"hash(a) == hash(b): {hash(a) == hash(b)}")
    print()

```

Possible output:

```

a: 42
b: 42
a == b: True
hash(a): 42
hash(b): 42
hash(a) == hash(b): True

a: 'spam'
b: 'spam'
a == b: True
hash(a): -7236028022913602481
hash(b): -7236028022913602481
hash(a) == hash(b): True

a: ('eggs', 42)
b: ('eggs', 42)
a == b: True
hash(a): 5120461032027266771
hash(b): 5120461032027266771
hash(a) == hash(b): True

```

The rule is not merely decorative. It is required because a hash table first uses the hash value to decide where to search. If two equal objects produced unrelated hash values, the table might store one object in one region and search for the other object in a different region.

A simplified lookup can be described like this:

```

candidate_hash = hash(search_key)
go to the table region suggested by candidate_hash
for each plausible stored key in that region:
    if stored_key == search_key:
        return the associated value
raise KeyError

```

That process assumes that an equal stored key and an equal search key lead into the same hash-based search path. The equal-hash invariant gives Python that guarantee.

This also explains why Python refuses to hash some mutable built-in containers.

```

items = ["spam", "eggs", 42]

print(f"items: {items!r}")
print(f"type(items): {type(items)}")

```

```

print(f"__hash__ in dir(items): {'__hash__' in dir(items)}")
print(f"items.__hash__: {items.__hash__}")

try:
    print(hash(items))
except TypeError as err:
    print(f"hash failed: {err}")

```

Possible output:

```

items: ['spam', 'eggs', 42]
type(items): <class 'list'>
__hash__ in dir(items): True
items.__hash__: None
hash failed: unhashable type: 'list'

```

The name `__hash__` appears, but its value is `None`. That is Python's way of saying that list objects are intentionally unhashable.

A list can change its contents.

```

items = ["spam", "eggs", 42]

print(f"before: {items!r}")
items.append("larch")
print(f"after:  {items!r}")

```

Possible output:

```

before: ['spam', 'eggs', 42]
after:  ['spam', 'eggs', 42, 'larch']

```

If a list were hashable by its contents, its hash value would have to change when its contents changed. That would be dangerous inside a set or dictionary, because the object might already be stored in a table slot chosen from its old hash value.

For the same reason, a tuple is hashable only when all its components are hashable.

```

safe_key = ("Arthur", 42)
bad_key = ("Arthur", [42])

print(f"safe_key: {safe_key!r}")
print(f"type(safe_key): {type(safe_key)}")
print(f"hash(safe_key): {hash(safe_key)}")
print()

print(f"bad_key: {bad_key!r}")
print(f"type(bad_key): {type(bad_key)}")

try:
    print(hash(bad_key))
except TypeError as err:
    print(f"hash failed: {err}")

```

Possible output:

```

safe_key: ('Arthur', 42)
type(safe_key): <class 'tuple'>
hash(safe_key): -5207665904908724718

bad_key: ('Arthur', [42])
type(bad_key): <class 'tuple'>
hash failed: unhashable type: 'list'

```

The tuple itself is immutable, but it contains a list. Since the list is mutable and unhashable, the tuple cannot safely produce a content-based hash value.

Hash Stability: Why Keys and Set Elements Must Not Change Their Hash-Relevant State While Stored

Hash stability means that an object used as a set element or dictionary key must keep the same hash-relevant state while it is stored in the table.

For immutable built-in objects such as integers, strings, and tuples of hashable elements, this is natural.

```

key = ("Jerry", 42)

quotes = {
    key: "No soup for you!"
}

print(f"key: {key!r}")
print(f"type(key): {type(key)}")
print(f"hash(key): {hash(key)}")
print(f"quotes[key]: {quotes[key]!r}")

```

Possible output:

```

key: ('Jerry', 42)
type(key): <class 'tuple'>
hash(key): -8427085969355501069
quotes[key]: 'No soup for you!'

```

The tuple cannot be changed in place. Its hash-relevant contents are stable. Therefore, it is safe as a dictionary key.

A list is not safe as a key.

```

key = ["Jerry", 42]

print(f"key: {key!r}")
print(f"type(key): {type(key)}")

try:
    quotes = {key: "No soup for you!"}
except TypeError as err:
    print(f"dictionary construction failed: {err}")

```

Possible output:

```

key: ['Jerry', 42]
type(key): <class 'list'>
dictionary construction failed: unhashable type: 'list'

```

Python blocks the operation before the list can corrupt the table.

The same rule applies to set elements.

```
safe_item = ("Homer", "D'oh!")
bad_item = ["Homer", "D'oh!"]

items = set()

items.add(safe_item)
print(f"items after safe add: {items!r}")

try:
    items.add(bad_item)
except TypeError as err:
    print(f"set add failed: {err}")
```

Possible output:

```
items after safe add: {('Homer', 'D'oh!')}
set add failed: unhashable type: 'list'
```

The table needs the hash value to remain valid. If an object's hash-relevant state changed after insertion, the table could lose the ability to find the object by normal lookup.

A simplified failure would look like this:

```
object inserted with old hash
table slot chosen from old hash

object later changes
new hash is different

lookup uses new hash
lookup searches a different table region
stored object is not found correctly
```

Built-in mutable containers avoid this failure by being unhashable. User-defined objects can still create this kind of bug if they define equality and hashing carelessly. The rule remains the same:

Do not let a stored key or set element change the data that its hash and equality depend on.

The practical summary is:

- `hash(x)` gives the table a placement/search descriptor;
- `x == y` confirms whether a candidate object is actually equal;
- equal objects must have equal hash values;
- dictionary keys and set elements must be hash-stable while stored;
- mutable built-in containers such as `list`, `dict`, and `set` are unhashable because their contents can change.

7.2 Unordered Unique Domains: The Set Architecture (`set`)

A `set` is Python’s built-in structure for representing a collection of unique elements. Unlike a list or tuple, it does not preserve components as a positional sequence. The central question for a set is not “what is stored at index three?”, but “does this value belong to the collection?”

This section studies sets as hash-based membership domains. We will examine their syntax, their automatic duplicate removal, their requirement that elements be hashable, their mutation operations, and the immutable `frozenset` variant. The goal is to understand `set` not as a strange list, but as a different storage architecture built for uniqueness and efficient membership testing.

7.2.1 Mathematical Foundations and Structural Syntax

A `set` represents a domain of unique elements. The word “domain” is useful here because a set is not mainly about position, repetition, or sequence traversal. It is about whether a value belongs to a collection.

A list may contain the same value many times. A set does not. If the same value is inserted repeatedly, the set keeps only one logical occurrence.

A list remembers positions and repetitions. A set remembers membership.

This makes sets useful for questions such as:

- has this value already appeared?
- which unique values exist in this input?
- which values belong to both groups?
- which values belong to one group but not another?

The mathematical idea is old, but Python implements it with a runtime hash table. That means a `set` is not merely a mathematical notation. It is also a concrete memory structure with hash-based placement rules.

Defining Unique Unordered Domains Using the Curly-Brace `{}` Construct

A non-empty set can be written with curly braces. The elements are separated by commas.

```
facts = {"spam", "eggs", "spam", 42}

print(f"facts: {facts!r}")
print(f"type(facts): {type(facts)}")
print(f"len(facts): {len(facts)}")
```

Possible output:

```
facts: {'eggs', 42, 'spam'}
type(facts): <class 'set'>
len(facts): 3
```

The input contained `"spam"` twice, but the resulting set contains it only once. The duplicate was not stored as a second independent component because sets enforce uniqueness.

The printed order should not be interpreted as a stable structural order. A set is not a list with unusual brackets. It is not a tuple with mutable syntax. It is a hash-based uniqueness domain.

We can inspect the object with `dir()`.

```
facts = {"spam", "eggs", 42}

print(f"type(facts): {type(facts)}")
print()

print("__contains__" in dir(facts))
print("__iter__" in dir(facts))
print("__len__" in dir(facts))
print("add" in dir(facts))
print("remove" in dir(facts))
print("union" in dir(facts))
print("intersection" in dir(facts))
```

Possible output:

```
type(facts): <class 'set'>

True
True
True
True
True
True
True
True
```

The most important names at this stage are:

- `__contains__`: supports membership testing with `in`;
- `__iter__`: allows traversal over the current set elements;
- `__len__`: supports `len()`;
- `add` and `remove`: later used for direct mutation;
- `union` and `intersection`: later used for mathematical set calculations.

Membership is the natural first operation for a set.

```
facts = {"spam", "eggs", 42}

print(f"{'spam' in facts = }")
print(f"{'larch' in facts = }")
print(f"{42 in facts = }")
```

Possible output:

```
'spam' in facts = True
'larch' in facts = False
42 in facts = True
```

This code does not ask where "spam" is located. It asks whether "spam" belongs to the set. That is the correct mental model.

A set may contain different hashable object types.

```

mixed = {"D'oh!", 42, ("Arthur", "Brian")}

print(f"mixed: {mixed!r}")
print(f"type(mixed): {type(mixed)}")

for item in mixed:
    print(f"item={item!r:<22} type={type(item)}")

```

Possible output:

```

mixed: {42, "D'oh!", ('Arthur', 'Brian')}
type(mixed): <class 'set'>
item=42                                type=<class 'int'>
item="D'oh!"                          type=<class 'str'>
item=('Arthur', 'Brian')               type=<class 'tuple'>

```

The tuple is accepted because it is immutable and its contents are hashable. A list would not be accepted.

```

try:
    bad_set = {"spam", ["eggs", 42]}
except TypeError as err:
    print(f"set construction failed: {err}")

```

Possible output:

```
set construction failed: unhashable type: 'list'
```

The list is rejected because a list can mutate. A mutable object cannot safely act as a hash-table element when its hash-relevant state may change after insertion.

Instantiation Boundaries: Differentiating Empty Set Initialization `set()` from Empty Dictionary Literal Declaration `{}`

The syntax for a non-empty set uses curly braces, but the syntax for an empty set does not.

This is a deliberate boundary in Python syntax:

- `{}` creates an empty dictionary;
- `set()` creates an empty set.

The distinction exists because curly braces were already used for dictionaries before set literals were added to Python. Therefore, the empty curly-brace form kept its dictionary meaning.

```

empty_a = {}
empty_b = set()

print(f"empty_a: {empty_a!r}")
print(f"type(empty_a): {type(empty_a)}")
print()

print(f"empty_b: {empty_b!r}")
print(f"type(empty_b): {type(empty_b)}")

```

Possible output:

```
empty_a: {}
type(empty_a): <class 'dict'>

empty_b: set()
type(empty_b): <class 'set'>
```

This is one of the common beginner traps. The expression {} does not mean “empty curly-brace collection in general.” It specifically means empty dictionary.

Because this subsection introduces both objects syntactically, we can inspect both types briefly.

```
empty_dict = {}
empty_set = set()

print("empty_dict")
print(f"  type: {type(empty_dict)}")
print(f"  keys in dir: {'keys' in dir(empty_dict)}")
print(f"  items in dir: {'items' in dir(empty_dict)}")
print(f"  add in dir: {'add' in dir(empty_dict)}")
print()

print("empty_set")
print(f"  type: {type(empty_set)}")
print(f"  keys in dir: {'keys' in dir(empty_set)}")
print(f"  items in dir: {'items' in dir(empty_set)}")
print(f"  add in dir: {'add' in dir(empty_set)}")
```

Possible output:

```
empty_dict
  type: <class 'dict'>
  keys in dir: True
  items in dir: True
  add in dir: False

empty_set
  type: <class 'set'>
  keys in dir: False
  items in dir: False
  add in dir: True
```

The dictionary has `keys()` and `items()` because it stores key-value associations. The set has `add()` because it stores individual unique elements.

A single value inside curly braces creates a set, not a dictionary.

```
one_item = {"Serenity now!"}

print(f"one_item: {one_item!r}")
print(f"type(one_item): {type(one_item)}")
```

Possible output:

```
one_item: {'Serenity now!'}
type(one_item): <class 'set'>
```

A colon creates dictionary syntax.

```
one_pair = {"Jerry": "Serenity now!"}

print(f"one_pair: {one_pair!r}")
print(f"type(one_pair): {type(one_pair)}")
```

Possible output:

```
one_pair: {'Jerry': 'Serenity now!'}
type(one_pair): <class 'dict'>
```

The syntax boundary is therefore:

```
{"spam"}           -> set with one element
{"spam", "eggs"}   -> set with two elements
{}                 -> empty dict
set()              -> empty set
{"name": "spam"}   -> dict with one key-value pair
```

This boundary matters because set code and dictionary code support different operations. Confusing `{}` with an empty set often leads to errors later, when the program tries to call set-specific methods on a dictionary.

```
items = {}

print(f"type(items): {type(items)}")

try:
    items.add("spam")
except AttributeError as err:
    print(f"method call failed: {err}")
```

Possible output:

```
type(items): <class 'dict'>
method call failed: 'dict' object has no attribute 'add'
```

The fix is to use `set()` when the intended object is an empty set.

```
items = set()

items.add("spam")

print(f"items: {items!r}")
print(f"type(items): {type(items)}")
```

Possible output:

```
items: {'spam'}
type(items): <class 'set'>
```

The mutation operation `add()` will be studied later in more detail. Here it is used only to expose the syntactic difference between an empty dictionary and an empty set.

Set Comprehensions as Hash-Based Filtering Structures

A set comprehension builds a set by evaluating an expression across an iterable source. Its syntax resembles a list comprehension, but it uses curly braces and produces a set.

```
words = ["spam", "eggs", "spam", "larch", "eggs"]

unique_words = {word for word in words}

print(f"words: {words!r}")
print(f"type(words): {type(words)}")
print()

print(f"unique_words: {unique_words!r}")
print(f"type(unique_words): {type(unique_words)}")
print(f"len(unique_words): {len(unique_words)}")
```

Possible output:

```
words: ['spam', 'eggs', 'spam', 'larch', 'eggs']
type(words): <class 'list'>

unique_words: {'eggs', 'spam', 'larch'}
type(unique_words): <class 'set'>
len(unique_words): 3
```

The source list has five components. The resulting set has three elements because repeated values collapse into one set member.

A set comprehension can also include a filtering condition.

```
nums = [1, 2, 3, 4, 5, 42, 42]

large_nums = {num for num in nums if num > 3}

print(f"nums: {nums!r}")
print(f"large_nums: {large_nums!r}")
print(f"type(large_nums): {type(large_nums)}")
```

Possible output:

```
nums: [1, 2, 3, 4, 5, 42, 42]
large_nums: {42, 4, 5}
type(large_nums): <class 'set'>
```

This does two things at once:

- it filters the input values with `if num > 3`;
- it removes duplicates because the result is a set.

The same idea works with transformed values.

```
quotes = [
    "D'oh!",
    "d'oh!",
    "SPAM",
    "spam",
```

```

    "Nobody expects the Spanish Inquisition!",
]

normalized = {text.lower() for text in quotes}

print("original values")
for text in quotes:
    print(f" {text!r}")

print()
print(f"normalized: {normalized!r}")
print(f"type(normalized): {type(normalized)}")

```

Possible output:

```

original values
'D'oh!"
'd'oh!"
'SPAM'
'spam'
'Nobody expects the Spanish Inquisition!'

normalized: {"d'oh!", 'spam', 'nobody expects the spanish inquisition!'}
type(normalized): <class 'set'>

```

The transformation `text.lower()` runs before set insertion. After that, the hash-based uniqueness rule applies. Therefore, "SPAM" and "spam" become the same resulting value.

Set comprehensions are useful when the program needs a unique domain derived from another structure.

```

names = ["Homer", "Marge", "Lisa", "Homer", "Bart", "Lisa"]

name_lengths = {len(name) for name in names}

print(f"names: {names!r}")
print(f"name_lengths: {name_lengths!r}")
print(f"type(name_lengths): {type(name_lengths)}")

```

Possible output:

```

names: ['Homer', 'Marge', 'Lisa', 'Homer', 'Bart', 'Lisa']
name_lengths: {4, 5}
type(name_lengths): <class 'set'>

```

The result does not say which name had which length. It only says which distinct lengths occurred. That is the correct set-level meaning.

A set comprehension should be read as:

```

build a set
for each source element
    compute the result expression
    optionally reject it with an if condition
    insert the result into the hash-based uniqueness domain

```

So the comprehension:

```
{word.lower() for word in words if len(word) > 4}
```

means:

- traverse `words`;
- keep only words whose length is greater than 4;
- convert each kept word to lowercase;
- insert the lowercase result into a set;
- collapse duplicates automatically.

A final example shows this complete path.

```
words = [
    "Spam",
    "eggs",
    "SPAM",
    "larch",
    "LARCH",
    "Ni!",
]

result = {word.lower() for word in words if len(word) > 3}

print(f"words: {words!r}")
print(f"result: {result!r}")
print(f"type(result): {type(result)}")
print(f"len(result): {len(result)}")
```

Possible output:

```
words: ['Spam', 'eggs', 'SPAM', 'larch', 'LARCH', 'Ni!']
result: {'eggs', 'spam', 'larch'}
type(result): <class 'set'>
len(result): 3
```

The result is not a filtered list. It is a filtered, transformed, hash-based uniqueness domain.

7.2.2 Constraints of Element Ingestion and Runtime Engines

A set accepts elements only under specific rules. These rules are not optional decorations around the syntax. They are required by the internal table engine that makes set membership efficient.

When a value is inserted into a set, Python must decide whether that value is already present. It does this through the hash and equality contract introduced earlier. The hash value guides Python toward a candidate region of the internal table. Equality then confirms whether an existing element should be treated as the same logical element.

Therefore, set ingestion has two central constraints:

- the element must be hashable;
- equal elements must collapse into one stored set member.

This is why a set is not just a list without duplicates. A list can accept almost any object reference and preserve repeated occurrences. A set accepts only hashable elements and organizes them through a hash-table engine.

The Uniqueness Invariant: Automated De-Duplication Mechanics at Runtime

A set stores each equal value only once. If the same value is supplied several times, Python does not store several independent copies of that value as separate set elements.

```
words = {"spam", "eggs", "spam", "spam", "larch"}

print(f"words: {words!r}")
print(f"type(words): {type(words)}")
print(f"len(words): {len(words)}")
```

Possible output:

```
words: {'eggs', 'spam', 'larch'}
type(words): <class 'set'>
len(words): 3
```

The source expression contains five textual entries. The set contains only three elements because "spam" is equal to the other "spam" values.

This behavior is not a later cleanup pass. It happens during insertion. Each candidate element is checked against the set's existing contents according to hash-table rules.

A list preserves duplicates because repetition is part of list structure.

```
word_list = ["spam", "eggs", "spam", "spam", "larch"]
word_set = {"spam", "eggs", "spam", "spam", "larch"}

print(f"word_list: {word_list!r}")
print(f"len(word_list): {len(word_list)}")
print()

print(f"word_set: {word_set!r}")
print(f"len(word_set): {len(word_set)}")
```

Possible output:

```
word_list: ['spam', 'eggs', 'spam', 'spam', 'larch']
len(word_list): 5

word_set: {'eggs', 'spam', 'larch'}
len(word_set): 3
```

The list answers: “which values appear at which positions?”

The set answers: “which distinct values are members of this domain?”

The same rule applies to numeric values.

```
nums = {42, 42, 42, 7, 7, 1}

print(f"nums: {nums!r}")
print(f"type(nums): {type(nums)}")
print(f"len(nums): {len(nums)}")
```

Possible output:

```
nums: {1, 42, 7}
type(nums): <class 'set'>
len(nums): 3
```

The repeated integer 42 appears only once. Repetition in the source syntax does not create repetition inside the set.

We can inspect the set object again with `dir()`.

```
nums = {42, 7, 1}

print(f"type(nums): {type(nums)}")
print(f"__contains__ in dir(nums): {'__contains__' in dir(nums)}")
print(f"__iter__      in dir(nums): {'__iter__' in dir(nums)}")
print(f"add          in dir(nums): {'add' in dir(nums)}")
print(f"discard      in dir(nums): {'discard' in dir(nums)}")
print(f"copy         in dir(nums): {'copy' in dir(nums)}")
```

Possible output:

```
type(nums): <class 'set'>
__contains__ in dir(nums): True
__iter__     in dir(nums): True
add          in dir(nums): True
discard      in dir(nums): True
copy         in dir(nums): True
```

The relevant methods here are:

- `__contains__`: checks membership with `in`;
- `__iter__`: permits traversal over the current elements;
- `add`: inserts one element while preserving uniqueness;
- `discard`: removes an element if present;
- `copy`: creates a shallow copy of the set.

The `add()` method also respects the uniqueness invariant.

```
names = set()

names.add("Homer")
names.add("Marge")
names.add("Homer")
names.add("Lisa")
names.add("Homer")

print(f"names: {names!r}")
print(f"len(names): {len(names)}")
```

Possible output:

```
names: {'Lisa', 'Homer', 'Marge'}
len(names): 3
```

Calling `add("Homer")` multiple times does not create multiple stored set elements. Once an equal element is already present, the set remains structurally unchanged.

The Hashability Criterion: Value Equality, Hash Stability, and Table Placement Requirements

A set element must be hashable. In practical terms, Python must be able to call `hash()` on the object, and the object's hash-relevant state must remain stable while the object is stored.

Immutable built-in values such as integers, strings, and tuples of hashable elements are accepted.

```
item1 = 42
item2 = "D'oh!"
item3 = ("Arthur", "Brian")

print(f"item1: {item1!r}, hash: {hash(item1)}")
print(f"item2: {item2!r}, hash: {hash(item2)}")
print(f"item3: {item3!r}, hash: {hash(item3)}")
```

Possible output:

```
item1: 42, hash: 42
item2: "D'oh!", hash: -5190362190904680204
item3: ('Arthur', 'Brian'), hash: 7652311734482103245
```

The exact string and tuple hash values may differ between Python executions. The important point is that these objects can produce hash values during the running process.

These values can be inserted into a set.

```
items = {42, "D'oh!", ("Arthur", "Brian")}

print(f"items: {items!r}")
print(f"type(items): {type(items)}")
```

Possible output:

```
items: {42, ('Arthur', 'Brian'), "D'oh!"}
type(items): <class 'set'>
```

Mutable built-in containers such as `list`, `dict`, and `set` are not accepted as set elements.

```
bad_item = ["spam", "eggs"]

print(f"bad_item: {bad_item!r}")
print(f"type(bad_item): {type(bad_item)}")
print(f"bad_item.__hash__: {bad_item.__hash__}")

try:
    values = {bad_item}
except TypeError as err:
    print(f"set construction failed: {err}")
```

Possible output:

```
bad_item: ['spam', 'eggs']
type(bad_item): <class 'list'>
bad_item.__hash__: None
set construction failed: unhashable type: 'list'
```

The method name `__hash__` exists on the object, but its value is `None`. This means Python deliberately marks the object as unhashable.

The reason is not that Python is unable to invent an integer for a list. The reason is that a list can change its equality-relevant contents after insertion.

```
items = ["spam", "eggs"]

print(f"before: {items!r}")
items.append(42)
print(f"after:  {items!r}")
```

Possible output:

```
before: ['spam', 'eggs']
after:  ['spam', 'eggs', 42]
```

If the list were used as a set element and its contents changed, the set could lose the ability to locate it correctly. The table slot would have been chosen from one state, while later lookup would describe another state.

A tuple is accepted only if all elements inside it are hashable.

```
safe_tuple = ("spam", "eggs", 42)
bad_tuple = ("spam", ["eggs", 42])

print(f"safe_tuple: {safe_tuple!r}")
print(f"hash(safe_tuple): {hash(safe_tuple)}")
print()

print(f"bad_tuple: {bad_tuple!r}")

try:
    print(hash(bad_tuple))
except TypeError as err:
    print(f"hash failed: {err}")
```

Possible output:

```
safe_tuple: ('spam', 'eggs', 42)
hash(safe_tuple): 6766429815130234107

bad_tuple: ('spam', ['eggs', 42])
hash failed: unhashable type: 'list'
```

The outer tuple is immutable, but it contains a mutable list. Since tuple equality depends on its contained elements, and the contained list is not hashable, the tuple cannot be safely used as a set element.

The ingestion rule can be stated directly:

A set element must be usable as a stable hash-table descriptor. It must not change the data that determines its hash and equality while it is stored.

CPython Open-Addressing Architecture: Hash Table Slots, Collision Probing, Empty Slots, and Deleted-Entry Markers

At the implementation level, CPython sets are based on a hash table. The table contains slots. A slot may be empty, may contain an active element, or may hold an internal marker left behind by a deletion.

The exact implementation is CPython-specific and may change between versions, but the useful model is stable enough for understanding ordinary behavior.

A simplified table can be imagined like this:

```
slot 0: empty
slot 1: active element
slot 2: active element
slot 3: deleted marker
slot 4: empty
slot 5: active element
```

When an element is inserted, CPython computes the element's hash and uses part of that hash to choose a starting slot. If that slot is empty, the element can be placed there. If that slot is occupied, CPython probes other slots according to its collision-resolution procedure.

A collision happens when two different elements compete for the same initial table area.

```
hash(element_a) -> starting slot 5
hash(element_b) -> starting slot 5

slot 5 already occupied
probe another slot
place element_b in a later acceptable slot
```

A collision does not mean equality. It only means that the hash-table placement path overlaps. Equality is still checked to decide whether the candidate element is already present.

This is the reason for the two-stage logic:

```
first:  use hash to find where to search
second: use equality to confirm the actual match
```

A simplified membership lookup can be described as follows:

```
search_hash = hash(search_value)
start at a table slot derived from search_hash

while the table path continues:
    if the current slot is empty:
        report "not present"
    if the current slot contains an equal element:
        report "present"
    otherwise:
        probe another slot
```

The empty slot is important. It tells the lookup that the searched element is not present along that probe path. If the search reaches a genuinely empty slot, there is no later element to find for that lookup path.

Deletion complicates this slightly. Suppose an element was removed from the middle of a probe path. CPython cannot always turn that slot into a normal empty slot immediately, because doing so could break future lookups for other elements placed later in the same collision path.

So implementations use a deleted-entry marker, often described as a dummy marker.

```

before deletion:
slot 5: element_a
slot 6: element_b
slot 7: empty

after deleting element_a:
slot 5: deleted marker
slot 6: element_b
slot 7: empty

```

During lookup, the deleted marker does not count as a successful match, but it also does not stop the search in the way a truly empty slot would.

```

lookup path:
  active slot      -> compare equality
  deleted marker   -> keep probing
  empty slot       -> stop, not present

```

This is why removal in a hash table is not the same operation as removing an item from a list. A list deletion shifts later components left to close the positional gap. A set deletion must preserve the correctness of hash-table lookup paths.

The following program does not expose internal slots directly, because Python deliberately hides those implementation details. It demonstrates the external behavior that the internal engine supports.

```

names = {"Homer", "Marge", "Lisa", "Bart"}

print(f"start: {names!r}")
print(f"{'Lisa' in names = }")

names.remove("Lisa")

print(f"after remove: {names!r}")
print(f"{'Lisa' in names = }")
print(f"{'Bart' in names = }")

```

Possible output:

```

start: {'Lisa', 'Homer', 'Bart', 'Marge'}
'Lisa' in names = True
after remove: {'Homer', 'Bart', 'Marge'}
'Lisa' in names = False
'Bart' in names = True

```

After removing "Lisa", other elements must remain findable. Internally, this is why deletion must respect the table's probing structure.

The important architectural picture is:

- hash values guide the starting position;
- collisions are resolved by probing;
- equality confirms actual element identity by value;
- empty slots can terminate failed searches;
- deleted-entry markers preserve lookup paths after removal.

Algorithmic Efficiency Matrix: Amortized $O(1)$ Membership Lookups vs. $O(N)$ Sequential Traversal

The reason sets exist as a separate structure is not only syntax. Their hash-table architecture changes the expected cost of common operations.

The clearest comparison is membership testing.

In a list, membership testing is sequential. Python may need to compare the searched value against many elements.

```
items = ["Homer", "Marge", "Lisa", "Bart", "Maggie"]

print(f{'Maggie' in items = })
print(f{'Burns' in items = })
```

Conceptually, a failed list membership test looks like this:

```
compare with "Homer"
compare with "Marge"
compare with "Lisa"
compare with "Bart"
compare with "Maggie"
not found
```

If the list has N elements, a failed membership check may require N comparisons. This is called $O(N)$ traversal.

A set uses hash-derived lookup instead.

```
items = {"Homer", "Marge", "Lisa", "Bart", "Maggie"}

print(f{'Maggie' in items = })
print(f{'Burns' in items = })
```

Conceptually, set membership looks more like this:

```
compute hash("Burns")
go to the table region indicated by the hash
probe if needed
stop when the value is found or the path reaches an empty slot
```

On average, this is treated as amortized $O(1)$ membership testing. That does not mean literally one CPU instruction. It means the expected amount of work does not grow linearly with the number of stored elements under normal table conditions.

The comparison can be summarized as follows:

Operation	List	Set
Membership test with <code>in</code>	Sequential scan	Hash-table lookup
Typical membership cost	$O(N)$	Amortized $O(1)$
Duplicate storage	Preserved	Removed automatically
Positional indexing	Supported	Not supported
Order as structure	Yes	No
Element requirement	Any object reference	Hashable object

A small script can show the semantic difference, without pretending to be a precise benchmark.

```

values_list = ["name_0", "name_1", "name_2", "name_3", "name_4"]
values_set = set(values_list)

target = "name_4"

print(f"values_list: {values_list!r}")
print(f"values_set: {values_set!r}")
print()

print(f"target: {target!r}")
print(f"target in values_list: {target in values_list}")
print(f"target in values_set: {target in values_set}")

```

Possible output:

```

values_list: ['name_0', 'name_1', 'name_2', 'name_3', 'name_4']
values_set: {'name_0', 'name_4', 'name_3', 'name_1', 'name_2'}

target: 'name_4'
target in values_list: True
target in values_set: True

```

Both containers answer the membership question correctly. The difference is the access route. The list reaches the answer through sequence traversal. The set reaches the answer through hash-table lookup.

For tiny containers, the practical timing difference may be irrelevant. For large membership-heavy workloads, the structural difference becomes important.

```

names = [
    "Arthur",
    "Brian",
    "Lancelot",
    "Galahad",
    "Bedevere",
    "Robin",
]

allowed = set(names)

candidate = "Galahad"

print(f"candidate: {candidate!r}")
print(f"candidate in names: {candidate in names}")
print(f"candidate in allowed: {candidate in allowed}")

```

Possible output:

```

candidate: 'Galahad'
candidate in names: True
candidate in allowed: True

```

The list `names` is appropriate if the original order matters. The set `allowed` is appropriate if the main question is membership.

The algorithmic lesson is:

Use a list when positional sequence is part of the data. Use a set when the program mainly needs a unique membership domain.

This is not a matter of taste. It follows from the internal access architecture.

7.2.3 Core Set Mutation Operations

A **set** is a mutable hash-based collection. Its elements are unique, unordered, and hashable. Mutation means that the set object itself remains the same object, but its internal membership domain changes.

This is different from creating a new set. When `.add()`, `.remove()`, `.discard()`, `.pop()`, or `.clear()` is used, Python modifies the existing set object in place.

```
names = {"Homer", "Marge"}

print(f"before: {names!r}")
print(f"type(names): {type(names)}")
print(f"id(names): {id(names)}")

names.add("Lisa")

print()
print(f"after: {names!r}")
print(f"type(names): {type(names)}")
print(f"id(names): {id(names)}")
```

Possible output:

```
before: {'Homer', 'Marge'}
type(names): <class 'set'>
id(names): 2199772358016

after: {'Lisa', 'Homer', 'Marge'}
type(names): <class 'set'>
id(names): 2199772358016
```

The memory identity printed by `id()` remains the same. The set object was not replaced. Its contents were changed.

Adding Elements with `.add()`

The method `.add()` inserts one element into a set. If an equal element is already present, the set remains unchanged.

```
quotes = set()

print(f"start: {quotes!r}")
print(f"type(quotes): {type(quotes)}")

quotes.add("D'oh!")
quotes.add("No soup for you!")
quotes.add("D'oh!")
quotes.add(42)

print()
print(f"after additions: {quotes!r}")
print(f"len(quotes): {len(quotes)}")
```

Possible output:

```
start: set()
type(quotes): <class 'set'>
```

```
after additions: {"D'oh!", 42, 'No soup for you!'}
len(quotes): 3
```

The value "D'oh!" was added twice, but it appears only once. The uniqueness invariant is enforced during insertion.

The method itself returns `None`. Its purpose is to mutate the set, not to produce a new set.

```
names = {"Arthur", "Brian"}

result = names.add("Lancelot")

print(f"names: {names!r}")
print(f"result: {result!r}")
print(f"type(result): {type(result)}")
```

Possible output:

```
names: {'Lancelot', 'Arthur', 'Brian'}
result: None
type(result): <class 'NoneType'>
```

This is common for in-place mutation methods in Python. The changed object is `names`; the method call itself does not return the changed set.

The inserted element must be hashable.

```
items = set()

items.add("spam")
items.add(("eggs", 42))

print(f"items: {items!r}")

try:
    items.add(["larch", 42])
except TypeError as err:
    print(f"add failed: {err}")
```

Possible output:

```
items: {('eggs', 42), 'spam'}
add failed: unhashable type: 'list'
```

The tuple is accepted because its elements are hashable. The list is rejected because lists are mutable and unhashable.

We can inspect the set methods relevant to this point.

```
items = {"spam"}

print(f"type(items): {type(items)}")
print(f"add in dir(items): {'add' in dir(items)}")
print(f"__contains__ in dir(items): {'__contains__' in dir(items)}")
print(f"__iter__ in dir(items): {'__iter__' in dir(items)}")
print(f"__len__ in dir(items): {'__len__' in dir(items)}")
```

Possible output:

```

type(items): <class 'set'>
add in dir(items): True
__contains__ in dir(items): True
__iter__ in dir(items): True
__len__ in dir(items): True

```

The important methods are:

- `add`: inserts one hashable element;
- `__contains__`: supports membership testing with `in`;
- `__iter__`: allows iteration over the set's current elements;
- `__len__`: supports `len()`.

Removing Elements with `.remove()`, `.discard()`, `.pop()`, and `.clear()`

A set has several removal operations because different situations need different failure behavior.

The method `.remove()` removes a specific element. If the element is not present, Python raises `KeyError`.

```

names = {"Homer", "Marge", "Lisa"}

print(f"before: {names!r}")

names.remove("Marge")

print(f"after: {names!r}")

try:
    names.remove("Bart")
except KeyError as err:
    print(f"remove failed: {err}")

```

Possible output:

```

before: {'Lisa', 'Homer', 'Marge'}
after: {'Lisa', 'Homer'}
remove failed: 'Bart'

```

Use `.remove()` when absence should be treated as an error. The method says: remove this element, and fail loudly if it is not there.

The method `.discard()` also removes a specific element, but it does not raise an error if the element is absent.

```

names = {"Homer", "Marge", "Lisa"}

print(f"before: {names!r}")

names.discard("Marge")
names.discard("Bart")

print(f"after: {names!r}")

```

Possible output:

```
before: {'Lisa', 'Homer', 'Marge'}
after:  {'Lisa', 'Homer'}
```

Use `.discard()` when absence is acceptable. The method says: remove this element if it exists; otherwise do nothing.

Both `.remove()` and `.discard()` return `None`.

```
items = {"spam", "eggs", "larch"}

result1 = items.remove("spam")
result2 = items.discard("beans")

print(f"items: {items!r}")
print(f"result1: {result1!r}")
print(f"result2: {result2!r}")
```

Possible output:

```
items: {'eggs', 'larch'}
result1: None
result2: None
```

The method `.pop()` removes and returns one arbitrary element from the set.

```
items = {"spam", "eggs", "larch"}

print(f"before: {items!r}")

removed = items.pop()

print(f"removed: {removed!r}")
print(f"type(removed): {type(removed)}")
print(f"after: {items!r}")
```

Possible output:

```
before: {'eggs', 'spam', 'larch'}
removed: 'eggs'
type(removed): <class 'str'>
after: {'spam', 'larch'}
```

The word “arbitrary” matters. Since a set is not a positional sequence, `.pop()` does not mean “remove the first element” or “remove the last element.” Those ideas belong to ordered containers, not to sets.

Calling `.pop()` on an empty set raises `KeyError`.

```
items = set()

try:
    removed = items.pop()
except KeyError as err:
    print(f"pop failed: {err}")
```

Possible output:

```
pop failed: 'pop from an empty set'
```

The method `.clear()` removes all elements from the set.

```
items = {"Arthur", "Brian", "Lancelot", 42}

print(f"before: {items!r}")
print(f"len before: {len(items)}")

items.clear()

print()
print(f"after: {items!r}")
print(f"len after: {len(items)}")
print(f"type(items): {type(items)}")
```

Possible output:

```
before: {'Lancelot', 42, 'Arthur', 'Brian'}
len before: 4

after: set()
len after: 0
type(items): <class 'set'>
```

The set still exists after `.clear()`. It is simply empty.

The removal methods can be summarized as follows:

Method	Meaning	Absent Element Behavior
<code>.remove(x)</code>	Remove <code>x</code>	Raises <code>KeyError</code>
<code>.discard(x)</code>	Remove <code>x</code> if present	Does nothing
<code>.pop()</code>	Remove and return one arbitrary element	Raises <code>KeyError</code> if empty
<code>.clear()</code>	Remove all elements	Always leaves an empty set

A complete small script shows the operations together.

```
words = {"spam", "eggs", "larch", "ni"}

print(f"start: {words!r}")

words.remove("eggs")
print(f"after remove('eggs'): {words!r}")

words.discard("beans")
print(f"after discard('beans'): {words!r}")

one_word = words.pop()
print(f"popped: {one_word!r}")
print(f"after pop(): {words!r}")

words.clear()
print(f"after clear(): {words!r}")
```

Possible output:

```
start: {'eggs', 'ni', 'spam', 'larch'}
after remove('eggs'): {'ni', 'spam', 'larch'}
after discard('beans'): {'ni', 'spam', 'larch'}
popped: 'ni'
after pop(): {'spam', 'larch'}
after clear(): set()
```

Membership Testing with in

Membership testing is the central read operation for sets. The expression `x in some_set` asks whether `x` is currently a member of the set.

```
allowed = {"Homer", "Marge", "Lisa", "Bart"}

candidate1 = "Lisa"
candidate2 = "Burns"

print(f"allowed: {allowed!r}")
print()

print(f"{candidate1!r} in allowed: {candidate1 in allowed}")
print(f"{candidate2!r} in allowed: {candidate2 in allowed}")
```

Possible output:

```
allowed: {'Lisa', 'Homer', 'Bart', 'Marge'}

'Lisa' in allowed: True
'Burns' in allowed: False
```

The operator `in` calls the set's membership machinery. Internally, this depends on hashing and equality, not positional scanning.

The searched value must also be hashable.

```
items = {"spam", "eggs", 42}

print(f"'spam' in items = ")
print(f"{42 in items = }")

try:
    print(["spam"] in items)
except TypeError as err:
    print(f"membership test failed: {err}")
```

Possible output:

```
'spam' in items = True
42 in items = True
membership test failed: unhashable type: 'list'
```

The list cannot be searched as a set member because checking membership would require computing its hash. Since the list is unhashable, the membership operation fails.

The opposite operator, `not in`, checks absence.

```
blocked = {"Newman", "Burns", "Sideshow Bob"}

name = "Jerry"

print(f"blocked: {blocked!r}")
print(f"name: {name!r}")
print(f"name not in blocked: {name not in blocked}")
```

Possible output:

```
blocked: {'Sideshow Bob', 'Burns', 'Newman'}
name: 'Jerry'
name not in blocked: True
```

Membership testing is often used before mutation when the program needs explicit control over what happens next.

```
names = {"Arthur", "Brian"}

new_name = "Arthur"

if new_name in names:
    print(f"{new_name!r} is already present")
else:
    names.add(new_name)
    print(f"{new_name!r} was added")

print(f"names: {names!r}")
```

Possible output:

```
'Arthur' is already present
names: {'Arthur', 'Brian'}
```

However, this check is not needed merely to preserve uniqueness. The set already does that.

```
names = {"Arthur", "Brian"}

names.add("Arthur")
names.add("Lancelot")

print(f"names: {names!r}")
print(f"len(names): {len(names)}")
```

Possible output:

```
names: {'Lancelot', 'Arthur', 'Brian'}
len(names): 3
```

The explicit membership check is useful when the program wants a different action depending on presence or absence. It is not required for safe insertion.

A final example combines membership and mutation.

```
seen = set()

line1 = "Nobody expects the Spanish Inquisition!"
line2 = "D'oh!"
line3 = "Nobody expects the Spanish Inquisition!"

for line in [line1, line2, line3]:
    if line in seen:
        print(f"duplicate ignored: {line!r}")
    else:
        seen.add(line)
        print(f"new line stored: {line!r}")
```

```
print()
print(f"seen: {seen!r}")
print(f"len(seen): {len(seen)}")
```

Possible output:

```
new line stored:  'Nobody expects the Spanish Inquisition!'
new line stored:  'D'oh!'
duplicate ignored: 'Nobody expects the Spanish Inquisition!'

seen: {'Nobody expects the Spanish Inquisition!', 'D'oh!'}
len(seen): 2
```

The set acts as a memory of already-seen values. It does not remember where they appeared, and it does not remember how many times they appeared. It remembers membership.

7.2.4 Immutable Set Variants

Python also provides an immutable set-like object: `frozenset`. A normal `set` is mutable. Elements can be added and removed after construction. A `frozenset` has the same basic membership-domain meaning, but its contents cannot be changed after creation.

This matters because hash-based containers require stable hash behavior. A mutable `set` cannot be used as a dictionary key or as an element inside another set. A `frozenset`, however, can be hashable if all of its elements are hashable.

A `set` is a mutable uniqueness domain. A `frozenset` is a fixed uniqueness domain.

`frozenset` as an Immutable Hashable Set-Like Object

A `frozenset` can be created by passing an iterable to the `frozenset()` constructor.

```
names = frozenset(["Arthur", "Brian", "Arthur", "Lancelot"])

print(f"names: {names!r}")
print(f"type(names): {type(names)}")
print(f"len(names): {len(names)}")
```

Possible output:

```
names: frozenset({'Arthur', 'Brian', 'Lancelot'})
type(names): <class 'frozenset'>
len(names): 3
```

The duplicate value "Arthur" appears only once. A `frozenset` still obeys the uniqueness rule of sets.

The main difference is mutation. A normal `set` has methods such as `add()`, `remove()`, `discard()`, `pop()`, and `clear()`. A `frozenset` does not have these mutating methods.

```
normal_set = {"spam", "eggs"}
fixed_set = frozenset(["spam", "eggs"])

print("normal_set")
print(f"  type: {type(normal_set)}")
print(f"  add in dir: {'add' in dir(normal_set)}")
print(f"  remove in dir: {'remove' in dir(normal_set)}")
```



```

print(f" union in dir: {'union' in dir(normal_set)}")
print()

print("fixed_set")
print(f" type: {type(fixed_set)}")
print(f" add in dir: {'add' in dir(fixed_set)}")
print(f" remove in dir: {'remove' in dir(fixed_set)}")
print(f" union in dir: {'union' in dir(fixed_set)}")

```

Possible output:

```

normal_set
  type: <class 'set'>
  add in dir: True
  remove in dir: True
  union in dir: True

fixed_set
  type: <class 'frozenset'>
  add in dir: False
  remove in dir: False
  union in dir: True

```

The **frozenset** keeps non-mutating set operations such as membership testing, union, intersection, difference, and symmetric difference. It rejects operations that would change its contents in place.

```

fixed_set = frozenset(["spam", "eggs", 42])

print(f"fixed_set: {fixed_set!r}")
print(f"{'spam' in fixed_set} = {True}")
print(f"{'larch' in fixed_set} = {False}")
print(f"{42 in fixed_set} = {True}")

```

Possible output:

```

fixed_set: frozenset({'eggs', 42, 'spam'})
'spam' in fixed_set = True
'larch' in fixed_set = False
42 in fixed_set = True

```

Trying to mutate a **frozenset** fails because the method does not exist.

```

fixed_set = frozenset(["spam", "eggs"])

try:
    fixed_set.add("larch")
except AttributeError as err:
    print(f"mutation failed: {err}")

```

Possible output:

```

mutation failed: 'frozenset' object has no attribute 'add'

```

This is not a temporary restriction. It is the defining property of the object. Once created, the membership domain is fixed.

A **frozenset** is also hashable when all its elements are hashable.

```
fixed_set = frozenset(["spam", "eggs", 42])

print(f"fixed_set: {fixed_set!r}")
print(f"type(fixed_set): {type(fixed_set)}")
print(f"hash(fixed_set): {hash(fixed_set)}")
```

Possible output:

```
fixed_set: frozenset({'eggs', 42, 'spam'})
type(fixed_set): <class 'frozenset'>
hash(fixed_set): -7694472856633011256
```

The exact hash value may differ between Python runs. The important point is that the object can produce a hash value because its membership cannot change after construction.

A mutable `set` cannot do this.

```
normal_set = {"spam", "eggs"}

print(f"normal_set: {normal_set!r}")
print(f"type(normal_set): {type(normal_set)}")
print(f"normal_set.__hash__: {normal_set.__hash__}")

try:
    print(hash(normal_set))
except TypeError as err:
    print(f"hash failed: {err}")
```

Possible output:

```
normal_set: {'eggs', 'spam'}
type(normal_set): <class 'set'>
normal_set.__hash__: None
hash failed: unhashable type: 'set'
```

The normal set is unhashable because its contents can change. If its hash depended on its current elements, adding or removing an element would change the hash-relevant state. That would be unsafe inside another hash table.

A `frozenset` can still be iterated.

```
fixed_set = frozenset(["Homer", "Marge", "Lisa"])

print(f"fixed_set: {fixed_set!r}")

for name in fixed_set:
    print(f"name: {name!r}")
```

Possible output:

```
fixed_set: frozenset({'Lisa', 'Homer', 'Marge'})
name: 'Lisa'
name: 'Homer'
name: 'Marge'
```

The iteration order must not be treated as a meaningful stored order. A `frozenset`, like a `set`, is still unordered.

Using frozenset as a Dictionary Key or Set Element

The practical reason `frozenset` exists is that sometimes the program needs a set-like value to behave as one stable hashable object.

A dictionary key must be hashable. A normal set cannot be used as a key.

```
bad_key = {"spam", "eggs"}
```

```
try:
```

```
    data = {bad_key: "breakfast"}
```

```
except TypeError as err:
```

```
    print(f"dictionary construction failed: {err}")
```

Possible output:

```
dictionary construction failed: unhashable type: 'set'
```

A `frozenset` can be used instead.

```
good_key = frozenset(["spam", "eggs"])
```

```
data = {
    good_key: "breakfast"
}
```

```
print(f"good_key: {good_key!r}")
print(f"type(good_key): {type(good_key)}")
print(f"data: {data!r}")
print(f"data[good_key]: {data[good_key]!r}")
```

Possible output:

```
good_key: frozenset({'eggs', 'spam'})
type(good_key): <class 'frozenset'>
data: {frozenset({'eggs', 'spam'}): 'breakfast'}
data[good_key]: 'breakfast'
```

This is useful when the key represents an unordered group rather than a sequence. For example, a pair of people can be treated as the same group regardless of order.

```
pair1 = frozenset(["Jerry", "George"])
pair2 = frozenset(["George", "Jerry"])
```

```
print(f"pair1: {pair1!r}")
print(f"pair2: {pair2!r}")
print(f"pair1 == pair2: {pair1 == pair2}")
print(f"hash(pair1) == hash(pair2): {hash(pair1) == hash(pair2)}")
```

Possible output:

```
pair1: frozenset({'George', 'Jerry'})
pair2: frozenset({'George', 'Jerry'})
pair1 == pair2: True
hash(pair1) == hash(pair2): True
```

Because the two `frozenset` objects contain the same members, they compare equal. Since they compare equal, they must also have equal hash values.

This makes them safe dictionary keys.

```

dialogue = {}

pair1 = frozenset(["Jerry", "George"])
pair2 = frozenset(["George", "Jerry"])

dialogue[pair1] = "coffee shop argument"
dialogue[pair2] = "updated coffee shop argument"

print(f"dialogue: {dialogue!r}")
print(f"len(dialogue): {len(dialogue)}")
print(f"dialogue[pair1]: {dialogue[pair1]!r}")
print(f"dialogue[pair2]: {dialogue[pair2]!r}")

```

Possible output:

```

dialogue: {frozenset({'George', 'Jerry'}): 'updated coffee shop argument'}
len(dialogue): 1
dialogue[pair1]: 'updated coffee shop argument'
dialogue[pair2]: 'updated coffee shop argument'

```

The second assignment did not create a second dictionary entry. It used an equal key and replaced the associated value.

A `frozenset` can also be stored as an element inside a normal set.

```

groups = set()

group1 = frozenset(["Arthur", "Brian"])
group2 = frozenset(["Brian", "Arthur"])
group3 = frozenset(["Homer", "Marge"])

groups.add(group1)
groups.add(group2)
groups.add(group3)

print(f"groups: {groups!r}")
print(f"type(groups): {type(groups)}")
print(f"len(groups): {len(groups)}")

```

Possible output:

```

groups: {frozenset({'Arthur', 'Brian'}), frozenset({'Homer', 'Marge'})}
type(groups): <class 'set'>
len(groups): 2

```

The first two groups collapse into one set element because they are equal `frozenset` objects.

A normal set cannot be placed inside another set.

```

outer = set()
inner = {"Arthur", "Brian"}

try:
    outer.add(inner)
except TypeError as err:
    print(f"nested set failed: {err}")

```

Possible output:

```
nested set failed: unhashable type: 'set'
```

The fix is to freeze the inner set-like group.

```
outer = set()
inner = frozenset(["Arthur", "Brian"])

outer.add(inner)

print(f"outer: {outer!r}")
print(f"type(outer): {type(outer)}")
```

Possible output:

```
outer: {frozenset({'Arthur', 'Brian'})}
type(outer): <class 'set'>
```

The practical distinction is:

Object	Mutable?	Hashable?
<code>set</code>	Yes	No
<code>frozenset</code>	No	Yes, if all elements are hashable

A `frozenset` should be used when the program needs an unordered unique group that must itself be stored inside a hash-based structure.

In ordinary words:

- use `set` for a membership domain that must change;
- use `frozenset` for a membership domain that must remain fixed;
- use `frozenset` when a set-like group must become a dictionary key or another set's element.

7.3 Set Mathematical Operators and Mutator Methods

Sets become especially useful when they are compared or combined as membership domains. Python supports this directly through mathematical set operators such as union, intersection, difference, and symmetric difference. These operations answer questions about how two domains overlap, separate, or combine.

This section separates two related ideas. First, it studies non-mutating calculations, where Python creates a new set result and leaves the original sets unchanged. Then, it studies in-place mutator methods, where the existing set object is rewritten with the result. The final part covers relationship tests such as subset, superset, and disjointness, where Python does not build a new domain but simply answers a structural question with `True` or `False`.

7.3.1 Fundamental Set Calculations

Set calculations combine or compare membership domains. These operations do not care about positions, because a set has no positional structure. They care about membership: which elements are present in the left set, which elements are present in the right set, and which elements should appear in the result.

The four fundamental calculations are:

- union: everything from both sets;
- intersection: only what appears in both sets;

- difference: what appears in the first set but not the second;
- symmetric difference: what appears in exactly one of the two sets.

Each operation has an operator form and a method form. The operator form is compact. The method form is more explicit and can accept general iterables in more cases.

The Union Operator `|` and Method Equivalent `.union()`

Union builds a new set containing every distinct element from both input sets.

```
python_lines = {"spam", "eggs", "larch"}
tv_lines = {"D'oh!", "spam", "Serenity now!"}

result1 = python_lines | tv_lines
result2 = python_lines.union(tv_lines)

print(f"python_lines: {python_lines!r}")
print(f"tv_lines:      {tv_lines!r}")
print()
print(f"result1: {result1!r}")
print(f"result2: {result2!r}")
print(f"same result: {result1 == result2}")
print(f"type(result1): {type(result1)}")
```

Possible output:

```
python_lines: {'eggs', 'larch', 'spam'}
tv_lines:      {'D'oh!', 'Serenity now!', 'spam'}

result1: {'Serenity now!', 'eggs', 'D'oh!', 'larch', 'spam'}
result2: {'Serenity now!', 'eggs', 'D'oh!', 'larch', 'spam'}
same result: True
type(result1): <class 'set'>
```

The value "spam" appears in both input sets, but it appears only once in the result. Union preserves the set uniqueness rule.

The original sets are not changed.

```
left = {"Arthur", "Brian"}
right = {"Brian", "Lancelot"}

result = left | right

print(f"left:      {left!r}")
print(f"right:     {right!r}")
print(f"result: {result!r}")
```

Possible output:

```
left:   {'Arthur', 'Brian'}
right:  {'Lancelot', 'Brian'}
result: {'Arthur', 'Lancelot', 'Brian'}
```

The operator `|` expects set-like operands. The method `.union()` can accept other iterable objects.

```
base = {"Homer", "Marge"}
more_names = ["Lisa", "Bart", "Homer"]

result = base.union(more_names)

print(f"base: {base!r}")
print(f"more_names: {more_names!r}")
print(f"result: {result!r}")
```

Possible output:

```
base: {'Homer', 'Marge'}
more_names: ['Lisa', 'Bart', 'Homer']
result: {'Lisa', 'Homer', 'Bart', 'Marge'}
```

Here `more_names` is a list, not a set. The method reads its elements and inserts them into the resulting set.

The Intersection Operator `&` and Method Equivalent `.intersection()`

Intersection builds a new set containing only elements that appear in both input sets.

```
python_lines = {"spam", "eggs", "larch", "ni"}
famous_words = {"spam", "D'oh!", "ni", "yada yada"}

result1 = python_lines & famous_words
result2 = python_lines.intersection(famous_words)

print(f"python_lines: {python_lines!r}")
print(f"famous_words: {famous_words!r}")
print()
print(f"result1: {result1!r}")
print(f"result2: {result2!r}")
print(f"same result: {result1 == result2}")
```

Possible output:

```
python_lines: {'eggs', 'larch', 'spam', 'ni'}
famous_words: {'yada yada', 'D'oh!', 'spam', 'ni'}

result1: {'spam', 'ni'}
result2: {'spam', 'ni'}
same result: True
```

The result contains "spam" and "ni" because those values appear in both sets.

Intersection is useful when two independent domains must both approve the same value.

```
allowed_users = {"Jerry", "Elaine", "George", "Kramer"}
present_users = {"Elaine", "Newman", "Kramer"}

active_allowed = allowed_users & present_users

print(f"allowed_users: {allowed_users!r}")
print(f"present_users: {present_users!r}")
print(f"active_allowed: {active_allowed!r}")
```

Possible output:

```
allowed_users: {'George', 'Elaine', 'Jerry', 'Kramer'}
present_users: {'Elaine', 'Newman', 'Kramer'}
active_allowed: {'Elaine', 'Kramer'}
```

The method form can also accept a general iterable.

```
known = {"Homer", "Marge", "Lisa", "Bart"}
incoming = ["Lisa", "Maggie", "Bart", "Burns"]

result = known.intersection(incoming)

print(f"known: {known!r}")
print(f"incoming: {incoming!r}")
print(f"result: {result!r}")
```

Possible output:

```
known: {'Lisa', 'Homer', 'Bart', 'Marge'}
incoming: ['Lisa', 'Maggie', 'Bart', 'Burns']
result: {'Lisa', 'Bart'}
```

Again, the result is a set. The list is only used as a source of candidate values.

The Difference Operator - and Method Equivalent `.difference()`

Difference builds a new set containing elements from the left set that do not appear in the right set.

```
all_items = {"spam", "eggs", "larch", "beans"}
blocked_items = {"eggs", "beans"}

result1 = all_items - blocked_items
result2 = all_items.difference(blocked_items)

print(f"all_items:      {all_items!r}")
print(f"blocked_items: {blocked_items!r}")
print()
print(f"result1: {result1!r}")
print(f"result2: {result2!r}")
print(f"same result: {result1 == result2}")
```

Possible output:

```
all_items:      {'eggs', 'beans', 'larch', 'spam'}
blocked_items: {'eggs', 'beans'}

result1: {'larch', 'spam'}
result2: {'larch', 'spam'}
same result: True
```

The result keeps only the elements that belong to `all_items` and do not belong to `blocked_items`. Difference is directional. Reversing the operands changes the question.


```
a = {"Arthur", "Brian", "Lancelot"}
b = {"Brian", "Galahad"}
```

```
print(f"a - b: {a - b}")
print(f"b - a: {b - a}")
```

Possible output:

```
a - b: {'Arthur', 'Lancelot'}
b - a: {'Galahad'}
```

The first expression asks: what is in a but not in b? The second expression asks: what is in b but not in a? These are not the same question.

The method form can accept an iterable.

```
seen = {"Homer", "Marge", "Lisa", "Bart"}
already_processed = ["Homer", "Lisa"]

remaining = seen.difference(already_processed)

print(f"seen: {seen!r}")
print(f"already_processed: {already_processed!r}")
print(f"remaining: {remaining!r}")
```

Possible output:

```
seen: {'Lisa', 'Homer', 'Bart', 'Marge'}
already_processed: ['Homer', 'Lisa']
remaining: {'Bart', 'Marge'}
```

The original set is not changed. A new set is produced.

The Symmetric Difference Operator $\sim$ and Method Equivalent `.symmetric_difference()`

Symmetric difference builds a new set containing elements that appear in exactly one of the two sets. It excludes elements that appear in both.

```
left = {"spam", "eggs", "larch"}
right = {"eggs", "beans", "larch", 42}

result1 = left ^ right
result2 = left.symmetric_difference(right)

print(f"left: {left!r}")
print(f"right: {right!r}")
print()
print(f"result1: {result1!r}")
print(f"result2: {result2!r}")
print(f"same result: {result1 == result2}")
```

Possible output:

```
left: {'eggs', 'larch', 'spam'}
right: {42, 'eggs', 'beans', 'larch'}

result1: {42, 'beans', 'spam'}
result2: {42, 'beans', 'spam'}
same result: True
```

The values "eggs" and "larch" are excluded because they appear in both sets. The values "spam", "beans", and 42 remain because each appears on only one side.

Symmetric difference is useful when the program needs disagreement between two domains.

```
expected = {"Jerry", "Elaine", "George"}
actual = {"Jerry", "Elaine", "Kramer"}

mismatch = expected ^ actual

print(f"expected: {expected!r}")
print(f"actual: {actual!r}")
print(f"mismatch: {mismatch!r}")
```

Possible output:

```
expected: {'Elaine', 'George', 'Jerry'}
actual:   {'Elaine', 'Jerry', 'Kramer'}
mismatch: {'George', 'Kramer'}
```

The mismatch contains "George" because he was expected but not actual, and "Kramer" because he was actual but not expected.

The method form can accept an iterable.

```
base = {"Arthur", "Brian", "Lancelot"}
other = ["Brian", "Galahad", "Arthur"]

result = base.symmetric_difference(other)

print(f"base: {base!r}")
print(f"other: {other!r}")
print(f"result: {result!r}")
```

Possible output:

```
base: {'Arthur', 'Brian', 'Lancelot'}
other: ['Brian', 'Galahad', 'Arthur']
result: {'Galahad', 'Lancelot'}
```

The values "Arthur" and "Brian" are removed from the result because they appear in both sources. The values "Lancelot" and "Galahad" remain because each appears only on one side.

The four calculations can be summarized compactly:

Calculation	Operator	Method	Meaning
Union		.union()	In either set
Intersection	&	.intersection()	In both sets
Difference	-	.difference()	In left, not in right
Symmetric difference	^	.symmetric_difference()	In exactly one set

All four operations produce new sets. They do not mutate the left operand.

```
a = {"spam", "eggs"}
b = {"eggs", "larch"}

union_result = a | b
intersection_result = a & b
difference_result = a - b
```

```

symmetric_result = a ^ b

print(f"a after calculations: {a!r}")
print(f"b after calculations: {b!r}")
print()
print(f"union_result:          {union_result!r}")
print(f"intersection_result: {intersection_result!r}")
print(f"difference_result:    {difference_result!r}")
print(f"symmetric_result:     {symmetric_result!r}")

```

Possible output:

```

a after calculations: {'eggs', 'spam'}
b after calculations: {'eggs', 'larch'}

union_result:          {'eggs', 'larch', 'spam'}
intersection_result: {'eggs'}
difference_result:     {'spam'}
symmetric_result:      {'larch', 'spam'}

```

7.3.2 In-Place Memory Mutation

The previous subsection described set calculations that create new sets. For example, `a | b` produces a result without changing `a` or `b`. Python also provides mutating versions of these operations. These methods change the left-hand set object in place.

The word “destructive” does not mean that the object becomes invalid. It means that the previous membership state is overwritten by a new membership state.

A non-mutating set calculation produces a new set. An in-place set update changes the existing set object.

This distinction can be seen with `id()`.

```

names = {"Homer", "Marge"}
extra = {"Lisa", "Bart"}

print(f"before: {names!r}")
print(f"id before: {id(names)}")

names.update(extra)

print()
print(f"after: {names!r}")
print(f"id after: {id(names)}")

```

Possible output:

```

before: {'Homer', 'Marge'}
id before: 2199812636800

after: {'Bart', 'Lisa', 'Homer', 'Marge'}
id after: 2199812636800

```

The set object has the same identity before and after the call. Its internal membership domain changed.

Destructive Union Updates via `.update()`

The method `.update()` performs an in-place union. It adds all elements from another iterable into the existing set.

```
left = {"spam", "eggs"}
right = {"eggs", "larch", 42}

print(f"left before: {left!r}")

result = left.update(right)

print(f"left after:  {left!r}")
print(f"right:       {right!r}")
print(f"result:      {result!r}")
```

Possible output:

```
left before: {'eggs', 'spam'}
left after:  {42, 'eggs', 'larch', 'spam'}
right:       {42, 'eggs', 'larch'}
result:      None
```

The method returns `None` because the useful result is the changed state of `left`. This is the same convention seen in other in-place mutation methods.

The non-mutating version and mutating version differ like this:

```
a = {"Arthur", "Brian"}
b = {"Brian", "Lancelot"}

new_set = a | b

print("after a | b")
print(f"a:      {a!r}")
print(f"b:      {b!r}")
print(f"new_set: {new_set!r}")

a.update(b)
```

```
print()
print("after a.update(b)")
print(f"a:      {a!r}")
print(f"b:      {b!r}")
```

Possible output:

```
after a | b
a:      {'Arthur', 'Brian'}
b:      {'Lancelot', 'Brian'}
new_set: {'Arthur', 'Lancelot', 'Brian'}

after a.update(b)
a:      {'Arthur', 'Lancelot', 'Brian'}
b:      {'Lancelot', 'Brian'}
```

The expression `a | b` leaves `a` unchanged. The call `a.update(b)` changes `a`. The method can accept more than one iterable.

```

items = {"spam"}

items.update(["eggs", "spam"], ("larch", 42), {"ni"})

print(f"items: {items!r}")
print(f"type(items): {type(items)}")

```

Possible output:

```

items: {'eggs', 42, 'spam', 'ni', 'larch'}
type(items): <class 'set'>

```

Each source is traversed, and each hashable value is inserted into the target set. Duplicate values collapse automatically.

The inserted values must still be hashable.

```

items = {"spam"}

try:
    items.update(["eggs", ["bad", "list"]])
except TypeError as err:
    print(f"update failed: {err}")

print(f"items after failure: {items!r}")

```

Possible output:

```

update failed: unhashable type: 'list'
items after failure: {'eggs', 'spam'}

```

The set may already have been partly changed before the error occurred. In-place mutation is immediate; it is not a guaranteed all-or-nothing transaction.

Destructive Intersection Updates via `.intersection_update()`

The method `.intersection_update()` keeps only elements that also appear in the supplied iterable or iterables. It changes the target set in place.

```

allowed = {"Homer", "Marge", "Lisa", "Bart"}
present = {"Lisa", "Bart", "Burns"}

print(f"allowed before: {allowed!r}")

result = allowed.intersection_update(present)

print(f"allowed after:  {allowed!r}")
print(f"present:        {present!r}")
print(f"result:         {result!r}")

```

Possible output:

```

allowed before: {'Lisa', 'Homer', 'Bart', 'Marge'}
allowed after:  {'Lisa', 'Bart'}
present:        {'Lisa', 'Burns', 'Bart'}
result:         None

```

The set `allowed` was reduced. Only values also found in `present` survived.

The non-mutating and mutating forms differ like this:

```
a = {"spam", "eggs", "larch"}
b = {"eggs", "beans", "larch"}

new_set = a & b

print("after a & b")
print(f"a:      {a!r}")
print(f"b:      {b!r}")
print(f"new_set: {new_set!r}")

a.intersection_update(b)

print()
print("after a.intersection_update(b)")
print(f"a:      {a!r}")
print(f"b:      {b!r}")
```

Possible output:

```
after a & b
a:      {'eggs', 'spam', 'larch'}
b:      {'eggs', 'beans', 'larch'}
new_set: {'eggs', 'larch'}

after a.intersection_update(b)
a:      {'eggs', 'larch'}
b:      {'eggs', 'beans', 'larch'}
```

The operator `&` creates a new result. The method `.intersection_update()` overwrites the membership state of `a`.

The method can also accept several iterables. The target keeps only values found in all of them.

```
names = {"Jerry", "Elaine", "George", "Kramer"}
group1 = ["Jerry", "Elaine", "Newman"]
group2 = {"Elaine", "Jerry", "Puddy"}

names.intersection_update(group1, group2)

print(f"names: {names!r}")
```

Possible output:

```
names: {'Elaine', 'Jerry'}
```

The final set contains only names present in the original `names`, in `group1`, and in `group2`.

Destructive Difference Updates via `.difference_update()`

The method `.difference_update()` removes from the target set all elements found in the supplied iterable or iterables.

```

items = {"spam", "eggs", "larch", "beans"}
blocked = {"eggs", "beans"}

print(f"items before: {items!r}")

result = items.difference_update(blocked)

print(f"items after:  {items!r}")
print(f"blocked:      {blocked!r}")
print(f"result:       {result!r}")

```

Possible output:

```

items before: {'eggs', 'beans', 'larch', 'spam'}
items after:  {'larch', 'spam'}
blocked:      {'eggs', 'beans'}
result:       None

```

The target set lost the values that were present in `blocked`. The `blocked` set itself was not changed.

The operation is directional. It removes from the target. It does not compute a balanced disagreement between both sets.

```

a = {"Arthur", "Brian", "Lancelot"}
b = {"Brian", "Galahad"}

a.difference_update(b)

print(f"a after difference_update: {a!r}")
print(f"b:                          {b!r}")

```

Possible output:

```

a after difference_update: {'Arthur', 'Lancelot'}
b:                          {'Brian', 'Galahad'}

```

Only "Brian" was removed from `a`. The value "Galahad" did not matter because it was not present in `a`.

The method can receive more than one iterable.

```

names = {"Homer", "Marge", "Lisa", "Bart", "Maggie"}

names.difference_update(["Homer", "Bart"], {"Maggie"})

print(f"names: {names!r}")

```

Possible output:

```

names: {'Lisa', 'Marge'}

```

The target set removed all values found in either removal source.

The non-mutating version is `.difference()` or the `-` operator.

```

original = {"spam", "eggs", "larch"}
blocked = {"eggs"}

new_set = original - blocked

```

```

print(f"original after '-': {original!r}")
print(f"new_set:           {new_set!r}")

original.difference_update(blocked)

print()
print(f"original after difference_update: {original!r}")

```

Possible output:

```

original after '-': {'eggs', 'spam', 'larch'}
new_set:           {'spam', 'larch'}

original after difference_update: {'spam', 'larch'}

```

The first calculation creates a new set. The update method changes the existing set.

Destructive Symmetric Difference Updates via `.symmetric_difference_update()`

The method `.symmetric_difference_update()` changes the target set so that it keeps only values that appear in exactly one of the two sources: the original target or the supplied iterable.

```

left = {"spam", "eggs", "larch"}
right = {"eggs", "beans", "larch", 42}

print(f"left before: {left!r}")

result = left.symmetric_difference_update(right)

print(f"left after:  {left!r}")
print(f"right:       {right!r}")
print(f"result:      {result!r}")

```

Possible output:

```

left before: {'eggs', 'larch', 'spam'}
left after:  {42, 'beans', 'spam'}
right:       {42, 'eggs', 'beans', 'larch'}
result:      None

```

The values "eggs" and "larch" were removed because they appeared in both. The values "beans" and 42 were added because they appeared only in `right`. The value "spam" remained because it appeared only in the original `left`.

This operation can be understood as two simultaneous actions:

- remove elements shared by both sides;
- add elements that appear only in the supplied iterable.

The non-mutating and mutating forms differ like this:

```

a = {"Jerry", "Elaine", "George"}
b = {"Jerry", "Elaine", "Kramer"}

new_set = a ^ b

```



```

print("after a ^ b")
print(f"a:      {a!r}")
print(f"b:      {b!r}")
print(f"new_set: {new_set!r}")

a.symmetric_difference_update(b)

print()
print("after a.symmetric_difference_update(b)")
print(f"a:      {a!r}")
print(f"b:      {b!r}")

```

Possible output:

```

after a ^ b
a:      {'Elaine', 'George', 'Jerry'}
b:      {'Elaine', 'Kramer', 'Jerry'}
new_set: {'George', 'Kramer'}

after a.symmetric_difference_update(b)
a:      {'George', 'Kramer'}
b:      {'Elaine', 'Kramer', 'Jerry'}

```

The operator `^` produces a new set. The method `.symmetric_difference_update()` rewrites the target set.

Unlike `.update()`, `.intersection_update()`, and `.difference_update()`, this method takes one main iterable argument.

```

base = {"Arthur", "Brian", "Lancelot"}
other = ["Brian", "Galahad", "Arthur"]

base.symmetric_difference_update(other)

print(f"base: {base!r}")

```

Possible output:

```
base: {'Galahad', 'Lancelot'}
```

The list was accepted as an iterable source. The result remains a set.

The in-place set update methods can be summarized as follows:

Method	Meaning	Target Set After Call
<code>.update()</code>	Destructive union	Everything from target and source
<code>.intersection_update()</code>	Destructive intersection	Only values common to all sources
<code>.difference_update()</code>	Destructive difference	Target values not found in sources
<code>.symmetric_difference_update()</code>	Destructive symmetric difference	Values found on exactly one side

All these methods return `None`. Their purpose is to mutate the existing set object.

```

items = {"spam", "eggs"}
other = {"eggs", "larch"}

print(f"start id: {id(items)}")

```

```
print(f"start items: {items!r}")

items.update(other)

print()
print(f"after update id: {id(items)}")
print(f"after update items: {items!r}")
```

Possible output:

```
start id: 2199812636800
start items: {'eggs', 'spam'}

after update id: 2199812636800
after update items: {'eggs', 'larch', 'spam'}
```

The object identity is stable. The membership domain changed in place.

7.3.3 Structural Relationship Evaluation

The previous operations produce or mutate sets. Structural relationship methods answer a different kind of question. They do not ask for a new membership domain. They ask whether one set has a specific relationship to another set.

The main relationship questions are:

- are all elements of this set also inside another set?
- does this set contain all elements of another set?
- do these two sets share no elements at all?

These methods return Boolean values. They answer with `True` or `False`.

Containment and Scope Testing: Identifying Subsets `.issubset()` and Supersets `.issuperset()`

A set `a` is a subset of set `b` when every element of `a` also appears in `b`.

```
small = {"spam", "eggs"}
large = {"spam", "eggs", "larch", 42}

print(f"small: {small!r}")
print(f"large: {large!r}")
print()

print(f"small.issubset(large): {small.issubset(large)}")
print(f"large.issubset(small): {large.issubset(small)}")
```

Possible output:

```
small: {'eggs', 'spam'}
large: {42, 'eggs', 'larch', 'spam'}

small.issubset(large): True
large.issubset(small): False
```

The first result is `True` because every element in `small` exists inside `large`. The second result is `False` because `large` has elements that `small` does not have.

A superset is the same relationship viewed from the opposite direction. A set `a` is a superset of set `b` when `a` contains every element of `b`.

```
small = {"spam", "eggs"}
large = {"spam", "eggs", "larch", 42}

print(f"large.issuperset(small): {large.issuperset(small)}")
print(f"small.issuperset(large): {small.issuperset(large)}")
```

Possible output:

```
large.issuperset(small): True
small.issuperset(large): False
```

The two method calls express the same relationship from opposite sides:

```
small.issubset(large)
large.issuperset(small)
```

Both mean that `large` covers all elements from `small`.

The operators `<=` and `>=` can also be used.

```
small = {"Arthur", "Brian"}
large = {"Arthur", "Brian", "Lancelot"}

print(f"small <= large: {small <= large}")
print(f"large >= small: {large >= small}")
```

Possible output:

```
small <= large: True
large >= small: True
```

The method names are usually clearer in teaching code because they say the relationship directly. Subset does not mean smaller by length only. It means that all members are included.

```
a = {"Homer", "Marge"}
b = {"Lisa", "Bart"}

print(f"len(a): {len(a)}")
print(f"len(b): {len(b)}")
print(f"a.issubset(b): {a.issubset(b)}")
```

Possible output:

```
len(a): 2
len(b): 2
a.issubset(b): False
```

The two sets have the same length, but neither contains the other's elements.

An empty set is a subset of every set, because there is no element inside the empty set that violates the containment rule.

```

empty = set()
names = {"Jerry", "Elaine", "George"}

print(f"empty: {empty!r}")
print(f"names: {names!r}")
print()

print(f"empty.issubset(names): {empty.issubset(names)}")
print(f"names.issuperset(empty): {names.issuperset(empty)}")

```

Possible output:

```

empty: set()
names: {'Elaine', 'George', 'Jerry'}

empty.issubset(names): True
names.issuperset(empty): True

```

This can feel strange at first, but the rule is literal: every element of `empty` is also in `names`. Since `empty` has no elements, there is no counterexample.

The methods can also accept other iterable objects.

```

allowed = {"Homer", "Marge", "Lisa", "Bart"}
requested = ["Lisa", "Bart"]

print(f"allowed: {allowed!r}")
print(f"requested: {requested!r}")
print()

print(f"allowed.issuperset(requested): {allowed.issuperset(requested)}")

```

Possible output:

```

allowed: {'Lisa', 'Homer', 'Bart', 'Marge'}
requested: ['Lisa', 'Bart']

allowed.issuperset(requested): True

```

The list `requested` is traversed as a source of values. The relationship question is still set-like: does `allowed` contain all requested values?

A practical use is permission checking.

```

user_permissions = {"read", "write", "export"}
required_permissions = {"read", "export"}

if user_permissions.issuperset(required_permissions):
    print("operation allowed")
else:
    print("operation blocked")

```

Possible output:

```
operation allowed
```

The code asks whether the user's permission domain covers the required permission domain.

Intersection Disjoint Verification via `.isdisjoint()`

Two sets are disjoint when they share no elements. The method `.isdisjoint()` returns `True` if the intersection would be empty.

```
python_words = {"spam", "eggs", "larch"}
simpsons_words = {"D'oh!", "Mmm... donuts", "excellent"}

print(f"python_words: {python_words!r}")
print(f"simpsons_words: {simpsons_words!r}")
print()

print(f"python_words.isdisjoint(simpsons_words): "
      f"{python_words.isdisjoint(simpsons_words)}")
```

Possible output:

```
python_words: {'eggs', 'larch', 'spam'}
simpsons_words: {'excellent', 'D'oh!', 'Mmm... donuts'}
```

```
python_words.isdisjoint(simpsons_words): True
```

The result is `True` because the two sets do not share any value.

If even one value is shared, the sets are not disjoint.

```
left = {"spam", "eggs", "larch"}
right = {"D'oh!", "spam", "Serenity now!"}

print(f"left: {left!r}")
print(f"right: {right!r}")
print()

print(f"left.isdisjoint(right): {left.isdisjoint(right)}")
print(f"left & right: {left & right}")
```

Possible output:

```
left: {'eggs', 'larch', 'spam'}
right: {'D'oh!', 'Serenity now!', 'spam'}
```

```
left.isdisjoint(right): False
left & right: {'spam'}
```

The intersection contains `"spam"`, so the sets are not disjoint.

The method `.isdisjoint()` is often clearer and more direct than computing the intersection manually.

```
blocked_users = {"Newman", "Burns", "Sideshow Bob"}
current_users = {"Jerry", "Elaine", "George"}

print(f"blocked_users: {blocked_users!r}")
print(f"current_users: {current_users!r}")
print()

if blocked_users.isdisjoint(current_users):
    print("no blocked users are active")
else:
    print("at least one blocked user is active")
```

Possible output:

```
blocked_users: {'Sideshow Bob', 'Burns', 'Newman'}
current_users: {'Elaine', 'George', 'Jerry'}
```

no blocked users are active

This method also accepts iterable objects.

```
blocked = {"Newman", "Burns", "Sideshow Bob"}
incoming = ["Jerry", "Elaine", "Newman"]

print(f"blocked: {blocked!r}")
print(f"incoming: {incoming!r}")
print()

print(f"blocked.isdisjoint(incoming): {blocked.isdisjoint(incoming)}")
```

Possible output:

```
blocked: {'Sideshow Bob', 'Burns', 'Newman'}
incoming: ['Jerry', 'Elaine', 'Newman']
```

```
blocked.isdisjoint(incoming): False
```

The list contains "Newman", which is also in the blocked set. Therefore, the domains are not disjoint. The relationship methods can be summarized as follows:

Method	Question	Returns
<code>a.issubset(b)</code>	Are all elements of <code>a</code> in <code>b</code> ?	bool
<code>a.issuperset(b)</code>	Does <code>a</code> contain all elements of <code>b</code> ?	bool
<code>a.isdisjoint(b)</code>	Do <code>a</code> and <code>b</code> share no elements?	bool

These methods do not change either set.

```
a = {"spam", "eggs"}
b = {"spam", "eggs", "larch"}

print(f"a before: {a!r}")
print(f"b before: {b!r}")

result = a.issubset(b)

print()
print(f"result: {result}")
print(f"a after: {a!r}")
print(f"b after: {b!r}")
```

Possible output:

```
a before: {'eggs', 'spam'}
b before: {'eggs', 'spam', 'larch'}

result: True
a after: {'eggs', 'spam'}
b after: {'eggs', 'spam', 'larch'}
```

They are inspection operations, not mutation operations. They read the membership domains and answer a structural question about them.

7.4 Dictionaries (dict) — Key-Value Associative Mappings

A `dict` is Python’s main structure for associating one object with another. Instead of asking for a component by numeric position, a dictionary asks for a value by key. The key acts as the lookup handle, and the value is the stored object attached to that key.

This section studies dictionaries as hash-based associative mappings. We will examine their key-value syntax, the requirement that keys be hashable and hash-stable, the freedom of dictionary values, construction patterns, insertion-order behavior, dynamic view objects, safe and unsafe mutation patterns, merging, lookup, and deletion.

The central idea is simple: a dictionary is not a list of pairs with nicer syntax. It is a runtime mapping engine where stable keys provide direct access to associated values.

7.4.1 Foundations of Associative Mapping

A dictionary is Python’s built-in associative mapping object. It stores relationships between keys and values. The key is the lookup handle. The value is the object associated with that key.

A sequence answers a positional question:

What object is stored at index 2?

A dictionary answers an associative question:

What value is associated with this key?

This changes the shape of access. A dictionary is not mainly a row of components. It is a mapping from a hashable key domain to an arbitrary value domain.

```
quotes = {
    "Homer": "D'oh!",
    "Jerry": "What's the deal with airline food?",
    "Arthur": "It is only a model.",
}

print(f"quotes: {quotes!r}")
print(f"type(quotes): {type(quotes)}")
print(f"quotes['Homer']: {quotes['Homer']!r}")
```

Possible output:

```
quotes: {'Homer': 'D'oh!', 'Jerry': "What's the deal with airline food?", 'Arthur': 'It is on
type(quotes): <class 'dict'>
quotes['Homer']: 'D'oh!'
```

The expression `quotes['Homer']` does not mean “the item at position ‘Homer’.” It means: use the key ‘Homer’ to retrieve the value associated with that key.

Structural Syntax: The Key-Value Paradigm and Curly-Brace {} Literals

A dictionary literal uses curly braces with key-value pairs inside. Each pair has the form:

`key: value`

Several pairs are separated by commas.

```

person = {
    "name": "Homer",
    "job": "safety inspector",
    "age": 42,
}

print(f"person: {person!r}")
print(f"type(person): {type(person)}")

```

Possible output:

```

person: {'name': 'Homer', 'job': 'safety inspector', 'age': 42}
type(person): <class 'dict'>

```

The keys in this dictionary are "name", "job", and "age". The values are "Homer", "safety inspector", and 42.

Access is performed through the key.

```

person = {
    "name": "Homer",
    "job": "safety inspector",
    "age": 42,
}

print(f"name: {person['name']!r}")
print(f"job: {person['job']!r}")
print(f"age: {person['age']!r}")

```

Possible output:

```

name: 'Homer'
job: 'safety inspector'
age: 42

```

The dictionary object exposes methods that reflect mapping behavior.

```

person = {
    "name": "Homer",
    "job": "safety inspector",
    "age": 42,
}

print(f"type(person): {type(person)}")
print(f"__getitem__ in dir: {'__getitem__' in dir(person)}")
print(f"__setitem__ in dir: {'__setitem__' in dir(person)}")
print(f"__contains__ in dir: {'__contains__' in dir(person)}")
print(f"keys in dir: {'keys' in dir(person)}")
print(f"values in dir: {'values' in dir(person)}")
print(f"items in dir: {'items' in dir(person)}")
print(f"get in dir: {'get' in dir(person)}")

```

Possible output:

```

type(person): <class 'dict'>
__getitem__ in dir: True

```



```
__setitem__ in dir: True
__contains__ in dir: True
keys in dir: True
values in dir: True
items in dir: True
get in dir: True
```

The crucial dictionary operations are:

- `__getitem__`: retrieves a value by key, as in `d[key]`;
- `__setitem__`: assigns a value to a key, as in `d[key] = value`;
- `__contains__`: checks key membership with `in`;
- `keys`: returns a dynamic view of the dictionary keys;
- `values`: returns a dynamic view of the dictionary values;
- `items`: returns a dynamic view of key-value pairs;
- `get`: retrieves a value safely when the key may be absent.

Assigning to an existing key replaces the associated value. It does not create a second copy of the key.

```
person = {
    "name": "Homer",
    "job": "safety inspector",
}

print(f"before: {person!r}")

person["job"] = "donut inspector"

print(f"after: {person!r}")
print(f"len(person): {len(person)}")
```

Possible output:

```
before: {'name': 'Homer', 'job': 'safety inspector'}
after:  {'name': 'Homer', 'job': 'donut inspector'}
len(person): 2
```

The dictionary still has two keys. The value associated with `"job"` changed. Assigning to a new key inserts a new key-value pair.

```
person = {
    "name": "Homer",
    "job": "safety inspector",
}

person["quote"] = "D'oh!"

print(f"person: {person!r}")
print(f"len(person): {len(person)}")
```

Possible output:

```
person: {'name': 'Homer', 'job': 'safety inspector', 'quote': "D'oh!"}
len(person): 3
```

The syntax boundary between sets and dictionaries is important. A colon creates dictionary entries.

```
one_set = {"spam"}
one_dict = {"food": "spam"}
empty_dict = {}

print(f"one_set: {one_set!r}")
print(f"type(one_set): {type(one_set)!r}")
print()

print(f"one_dict: {one_dict!r}")
print(f"type(one_dict): {type(one_dict)!r}")
print()

print(f"empty_dict: {empty_dict!r}")
print(f"type(empty_dict): {type(empty_dict)!r}")
```

Possible output:

```
one_set: {'spam'}
type(one_set): <class 'set'>

one_dict: {'food': 'spam'}
type(one_dict): <class 'dict'>

empty_dict: {}
type(empty_dict): <class 'dict'>
```

The empty curly-brace literal {} creates an empty dictionary, not an empty set.

Integrity Constraints: Why Dictionary Keys Must Be Hashable and Hash-Stable

Dictionary lookup is hash-based. The key must be hashable because Python uses the key's hash value to locate the internal table area where the key-value pair belongs.

Immutable built-in objects such as strings, integers, and tuples of hashable elements can be used as dictionary keys.

```
data = {
    "name": "Arthur",
    42: "answer",
    ("x", "y"): "coordinate labels",
}

print(f"data: {data!r}")
print(f"type(data): {type(data)!r}")
print()

print(f"data['name']: {data['name']!r}")
print(f"data[42]: {data[42]!r}")
print(f"data[('x', 'y')]: {data[('x', 'y')]!r}")
```

Possible output:

```
data: {'name': 'Arthur', 42: 'answer', ('x', 'y'): 'coordinate labels'}
type(data): <class 'dict'>
```

```
data['name']: 'Arthur'
data[42]: 'answer'
data[('x', 'y')]: 'coordinate labels'
```

The keys can be inspected with `hash()`.

```
keys = [
    "name",
    42,
    ("x", "y"),
]

for key in keys:
    print(f"key={key!r:<12} type={type(key)} hash={hash(key)}")
```

Possible output:

```
key='name'          type=<class 'str'> hash=5524778262942691467
key=42              type=<class 'int'> hash=42
key=('x', 'y')      type=<class 'tuple'> hash=7801337796950902376
```

The exact hash values for strings and tuples may differ between Python runs. The important point is that each key can produce a hash value during the current execution.

Mutable built-in containers such as lists cannot be used as dictionary keys.

```
bad_key = ["name"]

print(f"bad_key: {bad_key!r}")
print(f"type(bad_key): {type(bad_key)}")
print(f"bad_key.__hash__: {bad_key.__hash__}")

try:
    data = {bad_key: "Homer"}
except TypeError as err:
    print(f"dictionary construction failed: {err}")
```

Possible output:

```
bad_key: ['name']
type(bad_key): <class 'list'>
bad_key.__hash__: None
dictionary construction failed: unhashable type: 'list'
```

A list is unhashable because its contents can change. If a list could be used as a key, the dictionary could place it according to one content state and later fail to find it after the list changed.

```
items = ["spam", "eggs"]

print(f"before: {items!r}")
items.append(42)
print(f"after: {items!r}")
```

Possible output:

```
before: ['spam', 'eggs']
after:  ['spam', 'eggs', 42]
```

Dictionary keys must be hash-stable while stored. This means the data used by their hash and equality behavior must not change while the key is inside the dictionary.

A tuple can be used as a key only if all its elements are hashable.

```
safe_key = ("Homer", 42)
bad_key = ("Homer", [42])

print(f"safe_key: {safe_key!r}")
print(f"hash(safe_key): {hash(safe_key)}")
print()

print(f"bad_key: {bad_key!r}")

try:
    print(hash(bad_key))
except TypeError as err:
    print(f"hash failed: {err}")
```

Possible output:

```
safe_key: ('Homer', 42)
hash(safe_key): -9029866530700280856

bad_key: ('Homer', [42])
hash failed: unhashable type: 'list'
```

The outer tuple is immutable, but the inner list is not hashable. Since tuple equality and hashing depend on the contained elements, the tuple cannot be used as a safe key.

Two equal keys identify the same dictionary entry.

```
key1 = ("Jerry", "George")
key2 = ("Jerry", "George")

data = {}

data[key1] = "first version"
data[key2] = "second version"

print(f"key1 == key2: {key1 == key2}")
print(f"hash(key1) == hash(key2): {hash(key1) == hash(key2)}")
print(f"data: {data!r}")
print(f"len(data): {len(data)}")
```

Possible output:

```
key1 == key2: True
hash(key1) == hash(key2): True
data: {('Jerry', 'George'): 'second version'}
len(data): 1
```

The second assignment replaced the value associated with the existing equal key. It did not create a second independent mapping entry.

The key rule is the same rule used by sets:

Dictionary keys must be hashable, and their hash-relevant state must remain stable while they are stored.

Value Flexibility: Storing Arbitrary, Nested Data Types and Mutable Heap Objects

Dictionary values are much less restricted than dictionary keys. A value does not need to be hashable because Python does not use the value to place the entry in the hash table. The key controls lookup. The value is the payload associated with that key.

Values can be strings, numbers, lists, dictionaries, sets, tuples, or other objects.

```
profile = {
    "name": "Lisa",
    "age": 8,
    "quotes": ["If anyone wants me, I'll be in my room."],
    "scores": {"math": 10, "music": 10},
    "tags": {"student", "saxophone"},
}

print(f"profile: {profile!r}")
print(f"type(profile): {type(profile)}")
print()

print(f"type(profile['name']): {type(profile['name'])}")
print(f"type(profile['age']): {type(profile['age'])}")
print(f"type(profile['quotes']): {type(profile['quotes'])}")
print(f"type(profile['scores']): {type(profile['scores'])}")
print(f"type(profile['tags']): {type(profile['tags'])}")
```

Possible output:

```
profile: {'name': 'Lisa', 'age': 8, 'quotes': ["If anyone wants me, I'll be in my room."], 's
type(profile): <class 'dict'>

type(profile['name']): <class 'str'>
type(profile['age']): <class 'int'>
type(profile['quotes']): <class 'list'>
type(profile['scores']): <class 'dict'>
type(profile['tags']): <class 'set'>
```

This is legal because only the keys must be hashable. The values may be mutable.

A dictionary can store a reference to a mutable object. If that object is mutated later, the dictionary still refers to the same object.

```
quotes = ["D'oh!", "Mmm... donuts"]

profile = {
    "name": "Homer",
    "quotes": quotes,
}

print(f"profile before: {profile!r}")
print(f"id(quotes): {id(quotes)}")
print(f"id(profile['quotes']): {id(profile['quotes'])}")

quotes.append("Why you little!")

print()
print(f"profile after: {profile!r}")
```

```
print(f"id(quotes): {id(quotes)}")
print(f"id(profile['quotes']): {id(profile['quotes'])}")
```

Possible output:

```
profile before: {'name': 'Homer', 'quotes': ["D'oh!", 'Mmm... donuts']}
id(quotes): 2199812427072
id(profile['quotes']): 2199812427072

profile after: {'name': 'Homer', 'quotes': ["D'oh!", 'Mmm... donuts', 'Why you little!']}
id(quotes): 2199812427072
id(profile['quotes']): 2199812427072
```

The dictionary did not copy the list automatically. It stored a reference to the same list object. The list object can be inspected separately.

```
quotes = ["D'oh!", "Mmm... donuts"]

print(f"type(quotes): {type(quotes)}")
print(f"append in dir: {'append' in dir(quotes)}")
print(f"__getitem__ in dir: {'__getitem__' in dir(quotes)}")
print(f"__len__ in dir: {'__len__' in dir(quotes)}")
```

Possible output:

```
type(quotes): <class 'list'>
append in dir: True
__getitem__ in dir: True
__len__ in dir: True
```

The dictionary value is a normal list object. It keeps its own list behavior even though it is stored inside a dictionary.

Nested dictionaries are common because they allow named access at several levels.

```
characters = {
    "Homer": {
        "show": "The Simpsons",
        "quote": "D'oh!",
    },
    "Arthur": {
        "show": "Monty Python",
        "quote": "It is only a model.",
    },
}

print(f"type(characters): {type(characters)}")
print(f"type(characters['Homer']): {type(characters['Homer'])}")
print()

print(f"Homer quote: {characters['Homer']['quote']!r}")
print(f"Arthur show: {characters['Arthur']['show']!r}")
```

Possible output:

```
type(characters): <class 'dict'>
type(characters['Homer']): <class 'dict'>
```

```
Homer quote: "D'oh!"
Arthur show: 'Monty Python'
```

The expression `characters['Homer']['quote']` performs two dictionary lookups:

```
characters['Homer']          -> inner dictionary
inner_dictionary['quote']    -> final string value
```

This is one of the main reasons dictionaries are central in Python. They allow a program to represent named structure without relying on numeric positions.

Membership Testing: Why `x in d` Checks Keys, Not Values

For dictionaries, the expression `x in d` checks whether `x` is a key in the dictionary. It does not check whether `x` is one of the values.

```
quotes = {
    "Homer": "D'oh!",
    "Jerry": "No soup for you!",
    "Arthur": "It is only a model.",
}

print(f"quotes: {quotes!r}")
print()

print(f"{'Homer' in quotes = }")
print(f"{'D\\'oh!' in quotes = }")
```

Possible output:

```
quotes: {'Homer': 'D'oh!', 'Jerry': 'No soup for you!', 'Arthur': 'It is only a model.'}

'Homer' in quotes = True
'D'oh!' in quotes = False
```

The value `"D'oh!"` exists inside the dictionary, but it is not a key. Therefore, `"D'oh!" in quotes` is `False`.

This behavior follows from the dictionary's architecture. A dictionary is optimized to find values by key. The key is hashed. The value is not used for table placement.

The membership operation is equivalent to asking whether the key exists.

```
quotes = {
    "Homer": "D'oh!",
    "Jerry": "No soup for you!",
    "Arthur": "It is only a model.",
}

print(f"'Homer' in quotes:          {'Homer' in quotes}")
print(f"'Homer' in quotes.keys(): {'Homer' in quotes.keys()}")
```

Possible output:

```
'Homer' in quotes:          True
'Homer' in quotes.keys(): True
```

The first form is preferred because it is direct and idiomatic. Checking values requires using `.values()` explicitly.

```
quotes = {
    "Homer": "D'oh!",
    "Jerry": "No soup for you!",
    "Arthur": "It is only a model.",
}

print(f"'D\\'oh!' in quotes.values() = {}")
print(f"'Homer' in quotes.values() = {")
```

Possible output:

```
"D'oh!" in quotes.values() = True
'Homer' in quotes.values() = False
```

This is a different operation. Key lookup uses the dictionary hash table. Value membership must inspect values, because values are not the hash-table lookup domain.

The view objects can be inspected.

```
quotes = {
    "Homer": "D'oh!",
    "Jerry": "No soup for you!",
    "Arthur": "It is only a model.",
}

keys_view = quotes.keys()
values_view = quotes.values()
items_view = quotes.items()

print(f"type(keys_view): {type(keys_view)}")
print(f"type(values_view): {type(values_view)}")
print(f"type(items_view): {type(items_view)}")
print()

print(f"keys_view: {keys_view}")
print(f"values_view: {values_view}")
print(f"items_view: {items_view}")
```

Possible output:

```
type(keys_view): <class 'dict_keys'>
type(values_view): <class 'dict_values'>
type(items_view): <class 'dict_items'>

keys_view: dict_keys(['Homer', 'Jerry', 'Arthur'])
values_view: dict_values(["D'oh!", 'No soup for you!', 'It is only a model.'])
items_view: dict_items([('Homer', "D'oh!"), ('Jerry', 'No soup for you!'), ('Arthur', 'It is only
```

The details of dictionary view objects will be developed later. For now, the important distinction is simple:

- `x in d`: checks keys;
- `x in d.keys()`: also checks keys, but is usually unnecessary;

- `x in d.values()`: checks values;
- `x in d.items()`: checks key-value pair tuples.

A final example shows all three domains.

```
quote_map = {
    "Homer": "D'oh!",
    "Jerry": "No soup for you!",
}

print(f{'Homer' in quote_map = })
print(f{'D\\'oh!' in quote_map = })
print(f{'D\\'oh!' in quote_map.values() = })
print(f({'Homer', 'D\\'oh!') in quote_map.items() = })
```

Possible output:

```
'Homer' in quote_map = True
'D'oh!' in quote_map = False
'D'oh!' in quote_map.values() = True
('Homer', 'D'oh!') in quote_map.items() = True
```

The dictionary has three visible logical domains: keys, values, and key-value pairs. Bare membership testing belongs to the key domain.

7.4.2 Dictionary Construction Patterns

A dictionary can be constructed in several ways, but all construction patterns produce the same kind of object: a `dict`. The differences are mostly about where the key-value pairs come from.

The common construction patterns are:

- writing key-value pairs directly with a dictionary literal;
- calling the `dict()` constructor;
- generating pairs with a dictionary comprehension;
- converting a sequence of two-item pairs into a dictionary.

All of these patterns must still obey the same dictionary rules. Keys must be hashable. Equal keys collapse into one mapping entry. Values may be arbitrary Python objects.

Literal Construction with {key: value} Pairs

The most direct way to build a dictionary is the literal form. Each entry is written as a key, a colon, and a value.

```
quote_by_name = {
    "Homer": "D'oh!",
    "Jerry": "No soup for you!",
    "Arthur": "It is only a model.",
}

print(f"quote_by_name: {quote_by_name!r}")
print(f"type(quote_by_name): {type(quote_by_name)}")
print(f"len(quote_by_name): {len(quote_by_name)}")
```

Possible output:

```
quote_by_name: {'Homer': "D'oh!", 'Jerry': 'No soup for you!', 'Arthur': 'It is only a model.'}
type(quote_by_name): <class 'dict'>
len(quote_by_name): 3
```

The dictionary has three keys and three associated values.

A few important dictionary methods can be inspected with `dir()`.

```
quote_by_name = {
    "Homer": "D'oh!",
    "Jerry": "No soup for you!",
    "Arthur": "It is only a model.",
}

print(f"type(quote_by_name): {type(quote_by_name)}")
print(f"__getitem__ in dir: {'__getitem__' in dir(quote_by_name)}")
print(f"__setitem__ in dir: {'__setitem__' in dir(quote_by_name)}")
print(f"keys in dir: {'keys' in dir(quote_by_name)}")
print(f"values in dir: {'values' in dir(quote_by_name)}")
print(f"items in dir: {'items' in dir(quote_by_name)}")
print(f"update in dir: {'update' in dir(quote_by_name)}")
```

Possible output:

```
type(quote_by_name): <class 'dict'>
__getitem__ in dir: True
__setitem__ in dir: True
keys in dir: True
values in dir: True
items in dir: True
update in dir: True
```

The central methods here are:

- `__getitem__`: value retrieval by key;
- `__setitem__`: key-value insertion or replacement;
- `keys`: view over the key domain;
- `values`: view over the value domain;
- `items`: view over key-value pairs;
- `update`: later used to merge or overwrite mappings.

Literal construction can use different hashable key types.

```
mixed_keys = {
    "answer": 42,
    42: "the number itself",
    ("x", "y"): "coordinate names",
}

print(f"mixed_keys: {mixed_keys!r}")
print(f"mixed_keys['answer']: {mixed_keys['answer']!r}")
print(f"mixed_keys[42]: {mixed_keys[42]!r}")
print(f"mixed_keys[('x', 'y')]: {mixed_keys[('x', 'y')]!r}")
```

Possible output:

```
mixed_keys: {'answer': 42, 42: 'the number itself', ('x', 'y'): 'coordinate names'}
mixed_keys['answer']: 42
mixed_keys[42]: 'the number itself'
mixed_keys[('x', 'y')]: 'coordinate names'
```

The keys are not required to be strings. They are required to be hashable.

If the same key appears more than once in a literal, the later value wins.

```
data = {
    "name": "Homer",
    "job": "safety inspector",
    "job": "donut inspector",
}

print(f"data: {data!r}")
print(f"len(data): {len(data)}")
print(f"data['job']: {data['job']!r}")
```

Possible output:

```
data: {'name': 'Homer', 'job': 'donut inspector'}
len(data): 2
data['job']: 'donut inspector'
```

The dictionary does not store two separate "job" entries. The second "job" assignment replaces the earlier value for the same key.

Constructor-Based Construction with dict()

The `dict()` constructor creates dictionaries from several kinds of input.

With no arguments, it creates an empty dictionary.

```
empty_data = dict()

print(f"empty_data: {empty_data!r}")
print(f"type(empty_data): {type(empty_data)}")
print(f"len(empty_data): {len(empty_data)}")
```

Possible output:

```
empty_data: {}
type(empty_data): <class 'dict'>
len(empty_data): 0
```

This is equivalent in result to `{}`, although `{}` is more common for an empty dictionary.

The constructor can also use keyword arguments.

```
person = dict(name="Homer", job="safety inspector", age=42)

print(f"person: {person!r}")
print(f"type(person): {type(person)}")
print(f"person['name']: {person['name']!r}")
```

Possible output:

```

person: {'name': 'Homer', 'job': 'safety inspector', 'age': 42}
type(person): <class 'dict'>
person['name']: 'Homer'

```

In this form, the keyword names become string keys. The expression above creates keys "name", "job", and "age".

This form is limited to keys that are valid Python identifier names. It cannot directly create keys such as "first name", 42, or ("x", "y").

```

literal_data = {
    "first name": "Homer",
    42: "answer",
    ("x", "y"): "coordinate names",
}

print(f"literal_data: {literal_data!r}")

```

Possible output:

```
literal_data: {'first name': 'Homer', 42: 'answer', ('x', 'y'): 'coordinate names'}
```

For those keys, the literal form is clearer and more general.

The constructor can also copy an existing dictionary.

```

original = {
    "name": "Marge",
    "quote": "Mmm-hmm.",
}

copy_data = dict(original)

print(f"original: {original!r}")
print(f"copy_data: {copy_data!r}")
print(f"same object: {original is copy_data}")
print(f"equal value: {original == copy_data}")

```

Possible output:

```

original: {'name': 'Marge', 'quote': 'Mmm-hmm.'}
copy_data: {'name': 'Marge', 'quote': 'Mmm-hmm.'}
same object: False
equal value: True

```

The new dictionary has the same key-value pairs, but it is a different dictionary object.

This is a shallow copy. If a value is a mutable object, the value object itself is still shared.

```

original = {
    "name": "Homer",
    "quotes": ["D'oh!"],
}

copy_data = dict(original)

copy_data["quotes"].append("Mmm... donuts")

print(f"original: {original!r}")
print(f"copy_data: {copy_data!r}")
print(f"same list object: {original['quotes'] is copy_data['quotes']}")

```

Possible output:

```
original: {'name': 'Homer', 'quotes': ['D'oh!', 'Mmm... donuts']}
copy_data: {'name': 'Homer', 'quotes': ['D'oh!', 'Mmm... donuts']}
same list object: True
```

The dictionary object was copied, but the nested list was not duplicated.

Dictionary Comprehensions as Key-Value Generation Pipelines

A dictionary comprehension builds a dictionary by generating key-value pairs from an iterable source.

The general shape is:

```
{key_expression: value_expression for item in source}
```

A simple example maps names to their lengths.

```
names = ["Homer", "Marge", "Lisa", "Bart"]

name_lengths = {name: len(name) for name in names}

print(f"names: {names!r}")
print(f"name_lengths: {name_lengths!r}")
print(f"type(name_lengths): {type(name_lengths)}")
```

Possible output:

```
names: ['Homer', 'Marge', 'Lisa', 'Bart']
name_lengths: {'Homer': 5, 'Marge': 5, 'Lisa': 4, 'Bart': 4}
type(name_lengths): <class 'dict'>
```

The expression before the colon generates the key. The expression after the colon generates the value.

The same construction can be written more mechanically as a loop.

```
names = ["Homer", "Marge", "Lisa", "Bart"]

name_lengths = {}

for name in names:
    name_lengths[name] = len(name)

print(f"name_lengths: {name_lengths!r}")
```

Possible output:

```
name_lengths: {'Homer': 5, 'Marge': 5, 'Lisa': 4, 'Bart': 4}
```

The comprehension is the compact form of this construction pattern.

A dictionary comprehension can include a filter.

```
names = ["Homer", "Marge", "Lisa", "Bart", "Maggie"]

long_names = {name: len(name) for name in names if len(name) > 4}

print(f"names: {names!r}")
print(f"long_names: {long_names!r}")
```

Possible output:

```
names: ['Homer', 'Marge', 'Lisa', 'Bart', 'Maggie']
long_names: {'Homer': 5, 'Marge': 5, 'Maggie': 6}
```

Only names with length greater than 4 become dictionary entries.

The generated key must still be hashable.

```
words = ["spam", "eggs"]
```

```
try:
```

```
    bad_map = {[word]: len(word) for word in words}
```

```
except TypeError as err:
```

```
    print(f"comprehension failed: {err}")
```

Possible output:

```
comprehension failed: unhashable type: 'list'
```

The generated key `[word]` is a list. Lists are unhashable, so the dictionary cannot use them as keys.

If the comprehension generates the same key more than once, the later value replaces the earlier one.

```
words = ["SPAM", "spam", "Spam"]
```

```
normalized = {word.lower(): word for word in words}
```

```
print(f"words: {words!r}")
```

```
print(f"normalized: {normalized!r}")
```

```
print(f"len(normalized): {len(normalized)}")
```

Possible output:

```
words: ['SPAM', 'spam', 'Spam']
```

```
normalized: {'spam': 'Spam'}
```

```
len(normalized): 1
```

All three source words generate the same key, `"spam"`. The last generated value survives.

A dictionary comprehension is useful when the program naturally computes a mapping.

```
quotes = [
    "D'oh!",
    "No soup for you!",
    "It is only a model.",
]
```

```
quote_lengths = {quote: len(quote) for quote in quotes}
```

```
for quote, size in quote_lengths.items():
```

```
    print(f"{quote!r:<28} -> {size}")
```

Possible output:

```
"D'oh!"                -> 5
'No soup for you!'     -> 16
'It is only a model.'  -> 19
```

The result maps each quote to a computed property of that quote.

Building Dictionaries from Pair Sequences

The `dict()` constructor can also build a dictionary from a sequence of pairs. Each pair must contain exactly two elements: the key and the value.

```
pairs = [
    ("Homer", "D'oh!"),
    ("Jerry", "No soup for you!"),
    ("Arthur", "It is only a model."),
]

quote_by_name = dict(pairs)

print(f"pairs: {pairs!r}")
print(f"type(pairs): {type(pairs)}")
print()
print(f"quote_by_name: {quote_by_name!r}")
print(f"type(quote_by_name): {type(quote_by_name)}")
```

Possible output:

```
pairs: [('Homer', 'D\'oh!'), ('Jerry', 'No soup for you!'), ('Arthur', 'It is only a model.')]
type(pairs): <class 'list'>

quote_by_name: {'Homer': 'D\'oh!', 'Jerry': 'No soup for you!', 'Arthur': 'It is only a model.'}
type(quote_by_name): <class 'dict'>
```

The outer object is a list. Each inner object is a two-item tuple. The constructor reads each pair as:

```
(first item)  -> key
(second item) -> value
```

The inner pair type can be another iterable with two elements.

```
pairs = [
    ["Homer", "D'oh!"],
    ["Jerry", "No soup for you!"],
]

quote_by_name = dict(pairs)

print(f"quote_by_name: {quote_by_name!r}")
```

Possible output:

```
quote_by_name: {'Homer': 'D\'oh!', 'Jerry': 'No soup for you!'}
```

The inner lists are accepted as pair containers. The keys themselves are strings, so they are hashable. However, each pair must have exactly two elements.

```
bad_pairs = [
    ("Homer", "D'oh!"),
    ("Jerry", "No soup for you!", "extra"),
]
```

```
try:
    data = dict(bad_pairs)
except ValueError as err:
    print(f"construction failed: {err}")
```

Possible output:

```
construction failed: dictionary update sequence element #1 has length 3; 2 is required
```

The second pair has three elements, so Python cannot interpret it as one key and one value. Duplicate keys collapse during construction.

```
pairs = [
    ("Homer", "D'oh!"),
    ("Homer", "Mmm... donuts"),
    ("Lisa", "If anyone wants me, I'll be in my room."),
]

data = dict(pairs)

print(f"data: {data!r}")
print(f"len(data): {len(data)}")
print(f"data['Homer']: {data['Homer']!r}")
```

Possible output:

```
data: {'Homer': 'Mmm... donuts', 'Lisa': "If anyone wants me, I'll be in my room."}
len(data): 2
data['Homer']: 'Mmm... donuts'
```

The later "Homer" pair overwrote the earlier "Homer" pair.

A common pair-producing function is `zip()`. It combines parallel iterables into pairs.

```
names = ["Homer", "Jerry", "Arthur"]
quotes = ["D'oh!", "No soup for you!", "It is only a model."]

pairs = zip(names, quotes)
quote_by_name = dict(pairs)

print(f"names: {names!r}")
print(f"quotes: {quotes!r}")
print(f"quote_by_name: {quote_by_name!r}")
```

Possible output:

```
names: ['Homer', 'Jerry', 'Arthur']
quotes: ['D'oh!', 'No soup for you!', 'It is only a model.']
quote_by_name: {'Homer': 'D'oh!', 'Jerry': 'No soup for you!', 'Arthur': 'It is only a model.'}
```

The conceptual transformation is:

```
"Homer" + "D'oh!"           -> ("Homer", "D'oh!")
"Jerry" + "No soup for you!" -> ("Jerry", "No soup for you!")
"Arthur" + "It is only a model." -> ("Arthur", "It is only a model.")
```

Then `dict()` turns those pairs into key-value entries.

The construction patterns can be summarized as follows:

Construction Form	Typical Use
<code>{"a": 1, "b": 2}</code>	Directly known key-value pairs
<code>dict()</code>	Empty dictionary or constructor-based creation
<code>dict(name="Homer")</code>	Simple string keys that are valid identifiers
<code>{k: v for ...}</code>	Generated mappings
<code>dict([("a", 1), ("b", 2)])</code>	Pair sequence conversion
<code>dict(zip(keys, values))</code>	Parallel key and value sources

All forms create the same object type: `dict`. The difference is the source and shape of the input data.

7.4.3 CPython Architectural Evolution

Python dictionaries have kept the same visible programming role for a long time: they map keys to values. The internal storage strategy, however, has changed substantially. This matters because a dictionary is not only a convenient syntax form. It is also one of the most important runtime structures in CPython.

Dictionaries are used directly by programmers, but they are also used internally for many name-to-object mappings. Module namespaces, object attribute storage, class bodies, keyword arguments, and many ordinary program structures depend on dictionary-like behavior.

The programmer-facing rule is simple:

A dictionary retrieves a value by hashing a key, finding a candidate table location, and confirming the key with equality.

The implementation question is more specific:

How are those keys, values, hashes, and table slots physically organized in memory?

This subsection gives a simplified architectural view. It is not a replacement for reading CPython source code. It is the operational model needed to understand why dictionaries are fast, why their order behavior changed historically, and why insertion order is now preserved without turning dictionaries into lists.

The Classic Dictionary Model: Sparse Hash Table Storage and Historically Unspecified Iteration Order

The classic dictionary model can be imagined as one sparse hash table. The table had many slots, and each active slot stored the information needed for one dictionary entry.

A simplified slot looked conceptually like this:

```
slot:
    stored hash
    key reference
    value reference
```

The table could not be completely full. Hash tables need empty slots so lookup can stop when a key is not present. Therefore, the table contained active entries and unused space.

A simplified table might look like this:

```
slot 0: empty
slot 1: hash/key/value
slot 2: empty
slot 3: hash/key/value
slot 4: deleted marker
slot 5: empty
slot 6: hash/key/value
slot 7: empty
```

The entries are not stored because the programmer wrote them first, second, and third. They are stored where the hash-table placement machinery put them.

Consider this ordinary dictionary:

```
quotes = {
    "Homer": "D'oh!",
    "Jerry": "No soup for you!",
    "Arthur": "It is only a model.",
}

print(f"quotes: {quotes!r}")
print(f"type(quotes): {type(quotes)}")
print(f"len(quotes): {len(quotes)}")
```

Possible output in modern Python:

```
quotes: {'Homer': 'D'oh!', 'Jerry': 'No soup for you!', 'Arthur': 'It is only a model.'}
type(quotes): <class 'dict'>
len(quotes): 3
```

The visible output in modern Python follows insertion order. Historically, that was not guaranteed for normal dictionaries. The table was a hash table first, and iteration order was not part of the language-level promise.

The important distinction is:

- lookup order is controlled by hash-table search;
- historical iteration order was an implementation detail;
- a dictionary was not meant to be read as a positional sequence.

The dictionary object still exposes mapping methods, not sequence-index methods.

```
quotes = {
    "Homer": "D'oh!",
    "Jerry": "No soup for you!",
    "Arthur": "It is only a model.",
}

print(f"type(quotes): {type(quotes)}")
print(f"__getitem__ in dir: {'__getitem__' in dir(quotes)}")
print(f"__setitem__ in dir: {'__setitem__' in dir(quotes)}")
print(f"__contains__ in dir: {'__contains__' in dir(quotes)}")
print(f"keys in dir: {'keys' in dir(quotes)}")
print(f"items in dir: {'items' in dir(quotes)}")
print(f"append in dir: {'append' in dir(quotes)}")
print(f"index in dir: {'index' in dir(quotes)}")
```

Possible output:

```
type(quotes): <class 'dict'>
__getitem__ in dir: True
__setitem__ in dir: True
__contains__ in dir: True
keys in dir: True
items in dir: True
append in dir: False
index in dir: False
```

The crucial methods here are:

- `__getitem__`: retrieve a value by key;
- `__setitem__`: insert or replace a key-value association;
- `__contains__`: test key membership;
- `keys`: expose the key domain;
- `items`: expose key-value pair views.

There is no `append()` because a dictionary is not a growing positional row. There is no list-style `index()` because a dictionary does not locate entries by scanning for their numeric position.

Historically, this meant programmers were not supposed to depend on ordinary dictionary iteration order.

```
data = {}

data["first"] = "spam"
data["second"] = "eggs"
data["third"] = "larch"

for key in data:
    print(f"{key!r} -> {data[key]!r}")
```

In modern Python, the output follows insertion order:

```
'first' -> 'spam'
'second' -> 'eggs'
'third' -> 'larch'
```

In older Python versions, the language did not promise that behavior for normal dictionaries. The dictionary still worked correctly as a mapping, but the order of traversal was not a semantic feature of the object.

The classic mental model is therefore:

```
dictionary = sparse hash table

key -> hash(key)
hash -> candidate slot
slot/probe path -> equality check
equality success -> return associated value
```

That model explains fast key lookup. It does not explain insertion-order preservation, because insertion order was not a required property of the classic model.

The Modern Compact Dictionary: Dense Entry Storage, Sparse Indexing, Memory Reduction, and Insertion-Order Preservation

Modern CPython dictionaries use a more compact organization. A useful simplified model separates two ideas:

- a sparse index table used for hash lookup;
- a dense entry area that stores the actual entries in insertion order.

The sparse index table helps find the entry. The dense entry storage keeps the entries packed more tightly.

A simplified picture looks like this:

sparse index table:

```
slot 0 -> empty
slot 1 -> entry index 2
slot 2 -> empty
slot 3 -> entry index 0
slot 4 -> entry index 1
```

dense entries:

```
entry 0 -> hash, key="Homer", value="D'oh!"
entry 1 -> hash, key="Jerry", value="No soup for you!"
entry 2 -> hash, key="Arthur", value="It is only a model."
```

The sparse table is still needed for hash-table lookup. The dense entries avoid storing full entry data in every sparse slot. This reduces wasted space and also makes insertion-order traversal natural: iterate the dense entries in the order in which they were inserted.

The external behavior can be seen directly.

```
quotes = {}

quotes["Homer"] = "D'oh!"
quotes["Jerry"] = "No soup for you!"
quotes["Arthur"] = "It is only a model."

print(f"quotes: {quotes!r}")
print()

print("iteration:")
for key in quotes:
    print(f"{key!r} -> {quotes[key]!r}")
```

Possible output:

```
quotes: {'Homer': 'D'oh!', 'Jerry': 'No soup for you!', 'Arthur': 'It is only a model.'}

iteration:
'Homer' -> "D'oh!"
'Jerry' -> 'No soup for you!'
'Arthur' -> 'It is only a model.'
```

The dictionary preserves insertion order. This does not mean that the dictionary is a list. It means that traversal follows the order in which keys were first inserted.

Updating an existing key does not move that key to the end.

```
quotes = {}

quotes["Homer"] = "D'oh!"
quotes["Jerry"] = "No soup for you!"
quotes["Arthur"] = "It is only a model."

quotes["Jerry"] = "These pretzels are making me thirsty."
```

```
print(f"quotes: {quotes!r}")
print()

for key, value in quotes.items():
    print(f"{key!r:<8} -> {value!r}")
```

Possible output:

```
quotes: {'Homer': "D'oh!", 'Jerry': 'These pretzels are making me thirsty.', 'Arthur': 'It is

'Homer'  -> "D'oh!"
'Jerry'   -> 'These pretzels are making me thirsty.'
'Arthur'  -> 'It is only a model.'
```

The key "Jerry" stayed in its original insertion position. The value changed, but the key was not newly inserted.

If a key is deleted and then inserted again, it appears at the end.

```
quotes = {
    "Homer": "D'oh!",
    "Jerry": "No soup for you!",
    "Arthur": "It is only a model.",
}

del quotes["Jerry"]
quotes["Jerry"] = "Serenity now!"

print(f"quotes: {quotes!r}")
print()

for key, value in quotes.items():
    print(f"{key!r:<8} -> {value!r}")
```

Possible output:

```
quotes: {'Homer': "D'oh!", 'Arthur': 'It is only a model.', 'Jerry': 'Serenity now!'}

'Homer'  -> "D'oh!"
'Arthur'  -> 'It is only a model.'
'Jerry'   -> 'Serenity now!'
```

The second insertion of "Jerry" is a new insertion event, so it appears after the keys that remained. This is not sorted order.

```
data = {}

data["zebra"] = 1
data["apple"] = 2
data["mango"] = 3

print(f"data: {data!r}")
print(f"list(data): {list(data)}")
print(f"sorted(data): {sorted(data)}")
```

Possible output:

```
data: {'zebra': 1, 'apple': 2, 'mango': 3}
list(data): ['zebra', 'apple', 'mango']
sorted(data): ['apple', 'mango', 'zebra']
```

The dictionary remembers insertion order. The function `sorted()` computes sorted order separately. These are different ideas.

The compact dictionary also reduced memory usage compared with the older representation. The exact size of a dictionary depends on Python version, platform, number of entries, current table capacity, and implementation details. Still, `sys.getsizeof()` can show that a dictionary is a real heap object with storage overhead, not only the sum of its visible key and value objects.

```
import platform
import sys

empty = {}
small = {
    "Homer": "D'oh!",
    "Jerry": "No soup for you!",
    "Arthur": "It is only a model.",
}

print(f"implementation: {platform.python_implementation()}")
print(f"python version: {platform.python_version()}")
print()
print(f"type(empty): {type(empty)}")
print(f"sys.getsizeof(empty): {sys.getsizeof(empty)}")
print()
print(f"type(small): {type(small)}")
print(f"sys.getsizeof(small): {sys.getsizeof(small)}")
print(f"len(small): {len(small)}")
```

Possible output:

```
implementation: CPython
python version: 3.13.0

type(empty): <class 'dict'>
sys.getsizeof(empty): 64

type(small): <class 'dict'>
sys.getsizeof(small): 184
len(small): 3
```

The exact numbers may differ. The point is that a dictionary has internal table machinery. It is not only three visible pairs printed between braces.

The architectural summary is:

- classic dictionaries can be modeled as sparse hash tables containing entry data in table slots;
- modern compact dictionaries separate sparse lookup indexes from dense entries;
- dense entries make insertion-order traversal natural;
- since Python 3.7, insertion-order preservation is a language guarantee for normal dictionaries;
- this guarantee still does not make dictionaries positional sequences.

Algorithmic Efficiency Matrix: Amortized $O(1)$ Complexity for Key Insertion, Value Retrieval, and Structural Deletion

Dictionary performance comes from hash-table access. The usual operations are efficient on average because Python does not normally scan all keys from first to last.

The three central operations are:

- insert or replace a key-value pair;
- retrieve a value by key;
- delete a key-value pair.

All three are treated as amortized $O(1)$ average-case operations under ordinary hash behavior. Retrieval by key is the most visible case.

```
quotes = {
    "Homer": "D'oh!",
    "Jerry": "No soup for you!",
    "Arthur": "It is only a model.",
}

key = "Jerry"

print(f"key: {key!r}")
print(f"value: {quotes[key]!r}")
```

Possible output:

```
key: 'Jerry'
value: 'No soup for you!'
```

Conceptually, the lookup is:

```
hash("Jerry")
use the hash to search the sparse index table
find a candidate entry
confirm the stored key with equality
return the associated value
```

Insertion or replacement uses the same key-routing idea.

```
quotes = {}

quotes["Homer"] = "D'oh!"
quotes["Jerry"] = "No soup for you!"
quotes["Homer"] = "Mmm... donuts"

print(f"quotes: {quotes!r}")
print(f"len(quotes): {len(quotes)}")
```

Possible output:

```
quotes: {'Homer': 'Mmm... donuts', 'Jerry': 'No soup for you!'}
len(quotes): 2
```

The first assignment to "Homer" inserts a new key. The second assignment to "Homer" finds the existing equal key and replaces the value.

Deletion removes a key-value association.

```
quotes = {
    "Homer": "D'oh!",
    "Jerry": "No soup for you!",
    "Arthur": "It is only a model.",
}

print(f"before: {quotes!r}")

del quotes["Jerry"]

print(f"after: {quotes!r}")
```

Possible output:

```
before: {'Homer': 'D'oh!', 'Jerry': 'No soup for you!', 'Arthur': 'It is only a model.'}
after:  {'Homer': 'D'oh!', 'Arthur': 'It is only a model.'}
```

The deletion removes the mapping for the key. Internally, hash tables must still preserve correct probe behavior after deletion, just as sets do.

A dictionary should be compared with a list carefully. A list can store pairs too, but lookup by key would require sequential traversal unless the programmer builds another indexing structure.

```
pairs = [
    ("Homer", "D'oh!"),
    ("Jerry", "No soup for you!"),
    ("Arthur", "It is only a model."),
]

target = "Arthur"

for key, value in pairs:
    print(f"checking {key!r}")
    if key == target:
        print(f"found value: {value!r}")
        break
```

Possible output:

```
checking 'Homer'
checking 'Jerry'
checking 'Arthur'
found value: 'It is only a model.'
```

This list-based representation works, but lookup is positional traversal over pairs. If there are many pairs, the number of checks may grow with the number of entries.

The dictionary representation asks the runtime hash table to do key lookup.

```
quote_by_name = {
    "Homer": "D'oh!",
    "Jerry": "No soup for you!",
    "Arthur": "It is only a model.",
}
```



```

}

target = "Arthur"

print(f"found value: {quote_by_name[target]!r}")

```

Possible output:

```
found value: 'It is only a model.'
```

The source code is shorter, but the deeper point is the access architecture. The dictionary is built for key lookup.

The complexity comparison can be summarized as follows:

Operation	List of Pairs	Dictionary
Find value by key	$O(N)$ sequential scan	Amortized $O(1)$ key lookup
Insert new association	$O(1)$ append, if duplicates ignored	Amortized $O(1)$ key insertion
Replace existing key	$O(N)$ search, then replace	Amortized $O(1)$ key update
Delete by key	$O(N)$ search, then delete	Amortized $O(1)$ key deletion
Preserve duplicate equal keys	Possible unless manually prevented	No; equal keys collapse
Lookup requirement	Equality scan	Hashability and equality contract

The phrase amortized $O(1)$ should be read carefully. It does not mean that every operation takes exactly the same number of CPU instructions. It means that, under normal conditions, the expected cost does not grow linearly with the number of dictionary entries.

Some events are more expensive. A dictionary may resize its internal table when it grows. That resizing operation costs more than a simple insertion, but it happens occasionally. Spread over many insertions, the average insertion cost remains constant-time in the usual analysis.

A small script can show when the object size changes as entries are added.

```

import sys

data = {}

for num in range(12):
    data[f"key_{num}"] = num
    print(
        f"len={len(data):>2} "
        f"size={sys.getsizeof(data):>4} "
        f"data={data!r}"
    )

```

Possible output:

```

len= 1 size= 184 data={'key_0': 0}
len= 2 size= 184 data={'key_0': 0, 'key_1': 1}
len= 3 size= 184 data={'key_0': 0, 'key_1': 1, 'key_2': 2}
len= 4 size= 184 data={'key_0': 0, 'key_1': 1, 'key_2': 2, 'key_3': 3}
len= 5 size= 184 data={'key_0': 0, 'key_1': 1, 'key_2': 2, 'key_3': 3, 'key_4': 4}
len= 6 size= 272 data={'key_0': 0, 'key_1': 1, 'key_2': 2, 'key_3': 3, 'key_4': 4, 'key_5': 5}
...

```

The exact thresholds and sizes vary by CPython version and platform. The important observation is that dictionary storage grows in table-sized steps, not one tiny independent memory block per visible pair.

The practical rule is:

Use a dictionary when the program needs direct access from a stable key to an associated value.

A dictionary is not just syntactic sugar for a list of pairs. It is a hash-based associative mapping with its own storage engine, ordering rules, and average-case performance profile.

7.4.4 Ordering Guarantees and Misconceptions

Dictionaries and sets are both hash-based collections, but their visible ordering behavior is not the same. Modern Python dictionaries preserve insertion order. Sets do not provide an insertion-order guarantee.

This creates two common misconceptions:

- a dictionary that preserves insertion order is sometimes mistaken for a sorted mapping;
- a set that prints in a stable-looking order is sometimes mistaken for an ordered collection.

Both interpretations are wrong. A dictionary is still a hash-based key-value mapping. A set is still an unordered uniqueness domain.

Dictionary Insertion Order: Preserved Order Is Not Sorted Order

A modern dictionary preserves the order in which keys are first inserted.

```
data = {}

data["zebra"] = 1
data["apple"] = 2
data["mango"] = 3

print(f"data: {data!r}")
print(f"type(data): {type(data)}")
print(f"list(data): {list(data)}")
```

Possible output:

```
data: {'zebra': 1, 'apple': 2, 'mango': 3}
type(data): <class 'dict'>
list(data): ['zebra', 'apple', 'mango']
```

The keys appear as "zebra", "apple", and "mango" because that is the insertion order. This is not alphabetical order.

```
data = {}

data["zebra"] = 1
data["apple"] = 2
data["mango"] = 3

print(f"list(data): {list(data)}")
print(f"sorted(data): {sorted(data)}")
```

Possible output:

```
list(data): ['zebra', 'apple', 'mango']
sorted(data): ['apple', 'mango', 'zebra']
```

The expression `list(data)` follows dictionary iteration order. The expression `sorted(data)` constructs a new sorted list of keys. Sorting is a separate operation.

The dictionary methods expose the same insertion order.

```
data = {}

data["zebra"] = 1
data["apple"] = 2
data["mango"] = 3

print(f"keys: {list(data.keys())}")
print(f"values: {list(data.values())}")
print(f"items: {list(data.items())}")
```

Possible output:

```
keys: ['zebra', 'apple', 'mango']
values: [1, 2, 3]
items: [('zebra', 1), ('apple', 2), ('mango', 3)]
```

The dictionary is not sorted by key and not sorted by value. It is preserving the order of first insertion.

Updating an existing key changes the value, not the key position.

```
quotes = {}

quotes["Homer"] = "D'oh!"
quotes["Jerry"] = "No soup for you!"
quotes["Arthur"] = "It is only a model."

quotes["Jerry"] = "Serenity now!"

print(f"quotes: {quotes!r}")
print(f"list(quotes): {list(quotes)}")
```

Possible output:

```
quotes: {'Homer': 'D'oh!', 'Jerry': 'Serenity now!', 'Arthur': 'It is only a model.'}
list(quotes): ['Homer', 'Jerry', 'Arthur']
```

The key "Jerry" remains in its original position because the key was not newly inserted. Only its associated value changed.

If a key is removed and inserted again, it receives a new insertion position.

```
quotes = {
    "Homer": "D'oh!",
    "Jerry": "No soup for you!",
    "Arthur": "It is only a model.",
}

del quotes["Jerry"]
quotes["Jerry"] = "Serenity now!"

print(f"quotes: {quotes!r}")
print(f"list(quotes): {list(quotes)}")
```

Possible output:

```
quotes: {'Homer': "D'oh!", 'Arthur': 'It is only a model.', 'Jerry': 'Serenity now!'}
list(quotes): ['Homer', 'Arthur', 'Jerry']
```

The second insertion of "Jerry" occurs after the remaining keys, so it appears at the end.

The relevant dictionary methods can be inspected directly.

```
data = {"zebra": 1, "apple": 2, "mango": 3}

print(f"type(data): {type(data)}")
print(f"keys in dir:  {'keys' in dir(data)}")
print(f"values in dir: {'values' in dir(data)}")
print(f"items in dir:  {'items' in dir(data)}")
print(f"sort in dir:   {'sort' in dir(data)}")
```

Possible output:

```
type(data): <class 'dict'>
keys in dir:  True
values in dir: True
items in dir:  True
sort in dir:   False
```

There is no `sort()` method on a dictionary. Sorting is not part of the dictionary's internal mapping behavior. If sorted output is needed, it must be requested explicitly.

```
data = {"zebra": 1, "apple": 2, "mango": 3}

for key in sorted(data):
    print(f"{key!r}:<8> -> {data[key]}")
```

Possible output:

```
'apple' -> 2
'mango' -> 3
'zebra' -> 1
```

This loop uses a sorted list of keys to control traversal. The dictionary itself is not rearranged.

```
data = {"zebra": 1, "apple": 2, "mango": 3}

sorted_keys = sorted(data)

print(f"sorted_keys: {sorted_keys}")
print(f"data:         {data}")
```

Possible output:

```
sorted_keys: ['apple', 'mango', 'zebra']
data:        {'zebra': 1, 'apple': 2, 'mango': 3}
```

The practical rule is:

Dictionary order means insertion order, not sorted order.

Set Iteration Order: Why Sets Remain Unordered Even When They Appear Stable in Small Examples

A set is an unordered collection of unique hashable elements. It may print in some order, and that order may appear stable during one run, but the program must not treat that order as meaningful.

```
items = {"zebra", "apple", "mango"}

print(f"items: {items!r}")
print(f"type(items): {type(items)}")
print(f"list(items): {list(items)}")
```

Possible output:

```
items: {'apple', 'zebra', 'mango'}
type(items): <class 'set'>
list(items): ['apple', 'zebra', 'mango']
```

The printed order is not insertion order. It is not sorted order either. It is the current traversal order produced by the set's internal hash-table state.

This is easy to misunderstand because small examples may appear stable.

```
items = set()

items.add("zebra")
items.add("apple")
items.add("mango")

print(f"items: {items!r}")
print(f"list(items): {list(items)}")
print(f"sorted(items): {sorted(items)}")
```

Possible output:

```
items: {'apple', 'zebra', 'mango'}
list(items): ['apple', 'zebra', 'mango']
sorted(items): ['apple', 'mango', 'zebra']
```

The set was filled in the order "zebra", "apple", "mango", but the displayed set does not preserve that insertion order.

The set object exposes membership and mathematical operations, not sequence positioning operations.

```
items = {"zebra", "apple", "mango"}

print(f"type(items): {type(items)}")
print(f"__contains__ in dir: {'__contains__' in dir(items)}")
print(f"__iter__ in dir: {'__iter__' in dir(items)}")
print(f"add in dir: {'add' in dir(items)}")
print(f"union in dir: {'union' in dir(items)}")
print(f"sort in dir: {'sort' in dir(items)}")
print(f"index in dir: {'index' in dir(items)}")
```

Possible output:

```

type(items): <class 'set'>
__contains__ in dir: True
__iter__ in dir:      True
add in dir:           True
union in dir:         True
sort in dir:          False
index in dir:         False

```

The absence of `sort()` and `index()` is not accidental. A set is not a sequence. It has no promised first element, second element, or last element.

Trying to index a set fails.

```

items = {"zebra", "apple", "mango"}

try:
    print(items[0])
except TypeError as err:
    print(f"indexing failed: {err}")

```

Possible output:

```
indexing failed: 'set' object is not subscriptable
```

If sorted traversal is needed, explicitly sort the set into a list.

```

items = {"zebra", "apple", "mango"}

for item in sorted(items):
    print(f"item: {item!r}")

```

Possible output:

```

item: 'apple'
item: 'mango'
item: 'zebra'

```

The sorted list is ordered. The original set remains unordered.

```

items = {"zebra", "apple", "mango"}
ordered_items = sorted(items)

print(f"ordered_items: {ordered_items!r}")
print(f"type(ordered_items): {type(ordered_items)}")
print(f"items: {items!r}")
print(f"type(items): {type(items)}")

```

Possible output:

```

ordered_items: ['apple', 'mango', 'zebra']
type(ordered_items): <class 'list'>
items: {'apple', 'zebra', 'mango'}
type(items): <class 'set'>

```

The call to `sorted()` did not transform the set into an ordered set. It created a separate list.

The difference between dictionary and set ordering can be summarized as follows:

Object	Iteration Behavior	Meaning of Order
<code>dict</code>	Preserves key insertion order	Order of first key insertion
<code>set</code>	No insertion-order guarantee	No semantic order

A final comparison makes the boundary explicit.

```
keys = ["zebra", "apple", "mango"]

data = {}
items = set()

for key in keys:
    data[key] = len(key)
    items.add(key)

print(f"keys source: {keys!r}")
print(f"dict keys:    {list(data)!r}")
print(f"set values:   {list(items)!r}")
print(f"sorted set:   {sorted(items)!r}")
```

Possible output:

```
keys source: ['zebra', 'apple', 'mango']
dict keys:   ['zebra', 'apple', 'mango']
set values:  ['apple', 'zebra', 'mango']
sorted set:  ['apple', 'mango', 'zebra']
```

The dictionary preserves the order in which keys were inserted. The set stores the same unique values, but it does not preserve their insertion order. Sorting must be requested separately.

The practical rule is:

Trust dictionary insertion order. Do not trust set iteration order as program meaning.

7.4.5 Querying, Manipulating, and Mutating Dictionary States

A dictionary is not only created and then read passively. Most real dictionary use involves querying, inserting, replacing, merging, removing, and inspecting changing state. These operations all revolve around the same central rule: the dictionary is organized by keys.

Some operations only read dictionary state. Some operations mutate the existing dictionary object. Some operations create a new dictionary result. This subsection separates those cases carefully.

Explicit Lookups and the `KeyError` Boundary vs. Safe Access via `.get()`

The most direct dictionary lookup uses square brackets.

```
quotes = {
    "Homer": "D'oh!",
    "Jerry": "No soup for you!",
    "Arthur": "It is only a model.",
}

print(f"quotes['Homer']: {quotes['Homer']!r}")
print(f"quotes['Jerry']: {quotes['Jerry']!r}")
```

Possible output:

```
quotes['Homer']: "D'oh!"
quotes['Jerry']: 'No soup for you!'
```

This lookup is explicit and strict. If the key is absent, Python raises `KeyError`.

```
quotes = {
    "Homer": "D'oh!",
    "Jerry": "No soup for you!",
}

try:
    print(quotes["Elaine"])
except KeyError as err:
    print(f"lookup failed: {err}")
```

Possible output:

```
lookup failed: 'Elaine'
```

The exception is not accidental. Square-bracket lookup means: this key is expected to exist. If it does not exist, the program has crossed a boundary that Python reports immediately.

The safer lookup method is `.get()`.

```
quotes = {
    "Homer": "D'oh!",
    "Jerry": "No soup for you!",
}

print(f"quotes.get('Homer'): {quotes.get('Homer')!r}")
print(f"quotes.get('Elaine'): {quotes.get('Elaine')!r}")
```

Possible output:

```
quotes.get('Homer'): "D'oh!"
quotes.get('Elaine'): None
```

When the key is missing, `.get()` returns `None` by default instead of raising `KeyError`.

A custom fallback value can be supplied.

```
quotes = {
    "Homer": "D'oh!",
    "Jerry": "No soup for you!",
}

value1 = quotes.get("Homer", "unknown quote")
value2 = quotes.get("Elaine", "unknown quote")

print(f"value1: {value1!r}")
print(f"value2: {value2!r}")
```

Possible output:

```
value1: "D'oh!"
value2: 'unknown quote'
```

The method `.get()` does not insert the fallback value into the dictionary. It only returns it.


```

quotes = {
    "Homer": "D'oh!",
}

value = quotes.get("Bart", "Ay caramba!")

print(f"value: {value!r}")
print(f"quotes: {quotes!r}")
print(f"'Bart' in quotes: {'Bart' in quotes}")

```

Possible output:

```

value: 'Ay caramba!'
quotes: {'Homer': "D'oh!"}
'Bart' in quotes: False

```

The fallback was returned, but the key "Bart" was not added.

The relevant dictionary methods can be inspected directly.

```

quotes = {"Homer": "D'oh!"}

print(f"type(quotes): {type(quotes)}")
print(f"__getitem__ in dir: {'__getitem__' in dir(quotes)}")
print(f"get in dir: {'get' in dir(quotes)}")
print(f"setdefault in dir: {'setdefault' in dir(quotes)}")
print(f"pop in dir: {'pop' in dir(quotes)}")

```

Possible output:

```

type(quotes): <class 'dict'>
__getitem__ in dir: True
get in dir: True
setdefault in dir: True
pop in dir: True

```

The practical distinction is:

- use `d[key]` when absence is an error;
- use `d.get(key)` when absence is an expected possibility;
- use `d.get(key, fallback)` when a specific fallback value should be returned.

Default Insertion Patterns with `.setdefault()`

The method `.setdefault()` is different from `.get()`. It does not merely return a fallback. If the key is missing, it inserts the key with the supplied default value and then returns that value.

```

scores = {
    "Lisa": 10,
    "Bart": 6,
}

value1 = scores.setdefault("Lisa", 0)
value2 = scores.setdefault("Maggie", 0)

print(f"value1: {value1}")
print(f"value2: {value2}")
print(f"scores: {scores!r}")

```

Possible output:

```
value1: 10
value2: 0
scores: {'Lisa': 10, 'Bart': 6, 'Maggie': 0}
```

The key "Lisa" already existed, so its existing value was returned. The key "Maggie" did not exist, so it was inserted with value 0.

This pattern is often used when building grouped data.

```
pairs = [
    ("Homer", "D'oh!"),
    ("Homer", "Mmm... donuts"),
    ("Jerry", "No soup for you!"),
    ("Jerry", "Serenity now!"),
]

quote_map = {}

for name, quote in pairs:
    quote_map.setdefault(name, []).append(quote)

print(f"quote_map: {quote_map!r}")
```

Possible output:

```
quote_map: {'Homer': ["D'oh!", 'Mmm... donuts'], 'Jerry': ['No soup for you!', 'Serenity now!']}
```

The expression:

```
quote_map.setdefault(name, [])
```

means:

- if `name` already exists, return its current value;
- if `name` does not exist, insert `name` with a new empty list and return that list.

Then `.append(quote)` mutates the returned list.

The value returned by `.setdefault()` is the actual object stored in the dictionary.

```
data = {}

items = data.setdefault("numbers", [])

print(f"before append: {data!r}")
print(f"same object: {items is data['numbers']}")

items.append(42)

print(f"after append: {data!r}")
```

Possible output:

```
before append: {'numbers': []}
same object: True
after append: {'numbers': [42]}
```

This is useful, but it must be understood literally. The dictionary stores a reference to the list object. Mutating that list changes the value visible through the dictionary.

A careful warning: do not reuse one external mutable default object for unrelated keys unless sharing is intended.

```
data = {}
shared_list = []

data.setdefault("a", shared_list)
data.setdefault("b", shared_list)

data["a"].append("spam")

print(f"data: {data!r}")
print(f"data['a'] is data['b']: {data['a'] is data['b']}")
```

Possible output:

```
data: {'a': ['spam'], 'b': ['spam']}
data['a'] is data['b']: True
```

Both keys refer to the same list. That is correct if sharing is intended, and wrong if each key should have an independent list.

View Object Subsystems: Interrogating `.keys()`, `.values()`, and `.items()` Dynamic Proxies

Dictionaries expose three main view objects:

- `d.keys()`: a view of the key domain;
- `d.values()`: a view of the value domain;
- `d.items()`: a view of key-value pair tuples.

These are not lists. They are dynamic view objects connected to the dictionary.

```
quotes = {
    "Homer": "D'oh!",
    "Jerry": "No soup for you!",
    "Arthur": "It is only a model.",
}

keys_view = quotes.keys()
values_view = quotes.values()
items_view = quotes.items()

print(f"keys_view: {keys_view}")
print(f"values_view: {values_view}")
print(f"items_view: {items_view}")
print()

print(f"type(keys_view): {type(keys_view)}")
print(f"type(values_view): {type(values_view)}")
print(f"type(items_view): {type(items_view)}")
```

Possible output:

```

keys_view:    dict_keys(['Homer', 'Jerry', 'Arthur'])
values_view:  dict_values(['D'oh!', 'No soup for you!', 'It is only a model.'])
items_view:   dict_items([('Homer', 'D'oh!'), ('Jerry', 'No soup for you!'), ('Arthur', 'It is only

type(keys_view):    <class 'dict_keys'>
type(values_view):  <class 'dict_values'>
type(items_view):   <class 'dict_items'>

```

The view objects can be iterated.

```

quotes = {
    "Homer": "D'oh!",
    "Jerry": "No soup for you!",
    "Arthur": "It is only a model.",
}

print("keys")
for key in quotes.keys():
    print(f" {key!r}")

print()
print("values")
for value in quotes.values():
    print(f" {value!r}")

print()
print("items")
for key, value in quotes.items():
    print(f" {key!r:<8} -> {value!r}")

```

Possible output:

```

keys
'Homer'
'Jerry'
'Arthur'

values
'D'oh!'
'No soup for you!'
'It is only a model.'

items
'Homer' -> "D'oh!"
'Jerry' -> 'No soup for you!'
'Arthur' -> 'It is only a model.'

```

The `.items()` view is especially common because it gives both the key and the value in one traversal. The view objects have their own methods.

```

quotes = {
    "Homer": "D'oh!",
    "Jerry": "No soup for you!",
}

```

```

keys_view = quotes.keys()
values_view = quotes.values()
items_view = quotes.items()

print("dict_keys")
print(f" __iter__ in dir: {'__iter__' in dir(keys_view)}")
print(f" __contains__ in dir: {'__contains__' in dir(keys_view)}")
print(f" mapping in dir: {'mapping' in dir(keys_view)}")
print()

print("dict_values")
print(f" __iter__ in dir: {'__iter__' in dir(values_view)}")
print(f" __contains__ in dir: {'__contains__' in dir(values_view)}")
print(f" mapping in dir: {'mapping' in dir(values_view)}")
print()

print("dict_items")
print(f" __iter__ in dir: {'__iter__' in dir(items_view)}")
print(f" __contains__ in dir: {'__contains__' in dir(items_view)}")
print(f" mapping in dir: {'mapping' in dir(items_view)}")

```

Possible output:

```

dict_keys
  __iter__ in dir: True
  __contains__ in dir: True
  mapping in dir: True

dict_values
  __iter__ in dir: True
  __contains__ in dir: True
  mapping in dir: True

dict_items
  __iter__ in dir: True
  __contains__ in dir: True
  mapping in dir: True

```

Membership testing on these views checks different logical domains.

```

quotes = {
    "Homer": "D'oh!",
    "Jerry": "No soup for you!",
}

print(f"'Homer' in quotes.keys(): {'Homer' in quotes.keys()}")
print(f"D\\'oh!' in quotes.values(): {'D'oh!' in quotes.values()}")
print(f"('Homer', \\\"D'oh!\") in quotes.items(): "
      f"{'('Homer', 'D'oh!') in quotes.items()}")

```

The previous example contains nested quoting that is visually noisy. A clearer version stores the tested values first.

```

quotes = {
    "Homer": "D'oh!",

```

```

    "Jerry": "No soup for you!",
}

key = "Homer"
value = "D'oh!"
pair = ("Homer", "D'oh!")

print(f"key in quotes.keys():    {key in quotes.keys()}")
print(f"value in quotes.values(): {value in quotes.values()}")
print(f"pair in quotes.items():   {pair in quotes.items()}")

```

Possible output:

```

key in quotes.keys():    True
value in quotes.values(): True
pair in quotes.items():  True

```

Usually, checking keys should be written directly as `key in d`, not `key in d.keys()`.

```

quotes = {
    "Homer": "D'oh!",
}

print(f"'Homer' in quotes:          {'Homer' in quotes}")
print(f"'Homer' in quotes.keys(): {'Homer' in quotes.keys()}")

```

Possible output:

```

'Homer' in quotes:          True
'Homer' in quotes.keys():  True

```

The first form is shorter and directly expresses dictionary key membership.

Dynamic Views: Why Dictionary Views Reflect Later Dictionary Mutations

Dictionary views are dynamic. They do not freeze the dictionary state at the moment they are created. If the dictionary changes later, the views reflect the new state.

```

quotes = {
    "Homer": "D'oh!",
    "Jerry": "No soup for you!",
}

keys_view = quotes.keys()
values_view = quotes.values()
items_view = quotes.items()

print("before mutation")
print(f"keys_view:    {keys_view}")
print(f"values_view: {values_view}")
print(f"items_view:   {items_view}")

quotes["Arthur"] = "It is only a model."

print()

```

```
print("after mutation")
print(f"keys_view:    {keys_view}")
print(f"values_view: {values_view}")
print(f"items_view:   {items_view}")
```

Possible output:

```
before mutation
keys_view:    dict_keys(['Homer', 'Jerry'])
values_view:  dict_values(['D'oh!', 'No soup for you!'])
items_view:   dict_items([('Homer', 'D'oh!'), ('Jerry', 'No soup for you!')])

after mutation
keys_view:    dict_keys(['Homer', 'Jerry', 'Arthur'])
values_view:  dict_values(['D'oh!', 'No soup for you!', 'It is only a model.'])
items_view:   dict_items([('Homer', 'D'oh!'), ('Jerry', 'No soup for you!'), ('Arthur', 'It is only a model.')])
```

The view objects are not independent snapshots. They are live windows over the dictionary.

If a snapshot is needed, explicitly build a list or tuple.

```
quotes = {
    "Homer": "D'oh!",
    "Jerry": "No soup for you!",
}

keys_view = quotes.keys()
keys_snapshot = list(quotes.keys())

quotes["Arthur"] = "It is only a model."

print(f"keys_view:    {keys_view}")
print(f"keys_snapshot: {keys_snapshot}")
```

Possible output:

```
keys_view:    dict_keys(['Homer', 'Jerry', 'Arthur'])
keys_snapshot: ['Homer', 'Jerry']
```

The view changed because the dictionary changed. The list did not change because it is a separate object created from the old key sequence.

The same distinction applies to items.

```
quotes = {
    "Homer": "D'oh!",
    "Jerry": "No soup for you!",
}

items_view = quotes.items()
items_snapshot = list(quotes.items())

quotes["Jerry"] = "Serenity now!"

print(f"items_view:    {items_view}")
print(f"items_snapshot: {items_snapshot}")
```

Possible output:

```
items_view:      dict_items([('Homer', 'D'oh!'), ('Jerry', 'Serenity now!')])
items_snapshot:  [('Homer', 'D'oh!'), ('Jerry', 'No soup for you!')]
```

The view reflects the updated value. The list snapshot preserves the old key-value pairs.

This dynamic behavior is useful because a view stays attached to the dictionary without copying all entries. It is dangerous only if the programmer assumes that the view is a frozen record.

The Mutation Trap: Why Mutating Dictionary Geometry During Iteration Triggers Runtime Exceptions

A dictionary may be mutated, but not every mutation is safe during active iteration. The main danger is changing the dictionary's geometry while Python is walking through it.

Here, dictionary geometry means the set of keys: adding keys or deleting keys changes the shape of the mapping.

```
scores = {
    "Lisa": 10,
    "Bart": 6,
    "Milhouse": 7,
}

try:
    for name in scores:
        print(f"checking {name!r}")
        if scores[name] < 7:
            del scores[name]
except RuntimeError as err:
    print(f"iteration failed: {err}")

print(f"scores: {scores!r}")
```

Possible output:

```
checking 'Lisa'
checking 'Bart'
iteration failed: dictionary changed size during iteration
scores: {'Lisa': 10, 'Milhouse': 7}
```

The deletion changed the number of keys while the dictionary iterator was active. Python stops the operation because continuing would make traversal inconsistent.

Adding a key during iteration has the same kind of problem.

```
data = {
    "a": 1,
    "b": 2,
}

try:
    for key in data:
        print(f"key: {key!r}")
        data["c"] = 3
except RuntimeError as err:
    print(f"iteration failed: {err}")
```

Possible output:


```
key: 'a'
iteration failed: dictionary changed size during iteration
```

The safe pattern is to iterate over a snapshot of the keys.

```
scores = {
    "Lisa": 10,
    "Bart": 6,
    "Milhouse": 7,
}

for name in list(scores):
    if scores[name] < 7:
        del scores[name]

print(f"scores: {scores!r}")
```

Possible output:

```
scores: {'Lisa': 10, 'Milhouse': 7}
```

The expression `list(scores)` creates a separate list of keys before the loop begins. The loop walks over that list, not over the live dictionary geometry.

Another safe pattern is to build a new dictionary.

```
scores = {
    "Lisa": 10,
    "Bart": 6,
    "Milhouse": 7,
}

kept_scores = {
    name: score
    for name, score in scores.items()
    if score >= 7
}

print(f"scores: {scores!r}")
print(f"kept_scores: {kept_scores!r}")
```

Possible output:

```
scores: {'Lisa': 10, 'Bart': 6, 'Milhouse': 7}
kept_scores: {'Lisa': 10, 'Milhouse': 7}
```

The original dictionary is not mutated during the iteration. A new dictionary is constructed instead.

Changing the value for an existing key is different from adding or deleting keys. It does not change the dictionary size.

```
scores = {
    "Lisa": 10,
    "Bart": 6,
    "Milhouse": 7,
}
```

```
for name in scores:
    scores[name] = scores[name] + 1

print(f"scores: {scores!r}")
```

Possible output:

```
scores: {'Lisa': 11, 'Bart': 7, 'Milhouse': 8}
```

This works because the key domain did not change. The values changed, but the dictionary geometry remained stable.

The practical rule is:

Do not add or delete dictionary keys while iterating over the live dictionary. Iterate over a snapshot, or build a new dictionary.

Dynamic Modifications: In-Place Mutation, Dictionary Merging Operators |, |=, and Key-Value Eviction .pop(), del

A dictionary can be modified in place by assigning to keys.

```
quotes = {
    "Homer": "D'oh!",
}

print(f"before: {quotes!r}")
print(f"id before: {id(quotes)}")

quotes["Jerry"] = "No soup for you!"
quotes["Homer"] = "Mmm... donuts"

print()
print(f"after: {quotes!r}")
print(f"id after: {id(quotes)}")
```

Possible output:

```
before: {'Homer': "D'oh!"}
id before: 2199812435584

after: {'Homer': 'Mmm... donuts', 'Jerry': 'No soup for you!'}
id after: 2199812435584
```

The dictionary object has the same identity. Its internal key-value state changed.

Python also supports dictionary merging operators.

The operator | creates a new dictionary.

```
base = {
    "Homer": "D'oh!",
    "Jerry": "No soup for you!",
}

extra = {
    "Arthur": "It is only a model.",
    "Jerry": "Serenity now!",
}
```

```
merged = base | extra

print(f"base: {base!r}")
print(f"extra: {extra!r}")
print(f"merged: {merged!r}")
```

Possible output:

```
base: {'Homer': 'D'oh!', 'Jerry': 'No soup for you!'}
extra: {'Arthur': 'It is only a model.', 'Jerry': 'Serenity now!'}
merged: {'Homer': 'D'oh!', 'Jerry': 'Serenity now!', 'Arthur': 'It is only a model.'}
```

When both dictionaries contain the same key, the right-hand value wins. Here, "Jerry" receives the value from `extra`.

The operator `|=` mutates the left dictionary in place.

```
base = {
    "Homer": "D'oh!",
    "Jerry": "No soup for you!",
}

extra = {
    "Arthur": "It is only a model.",
    "Jerry": "Serenity now!",
}

print(f"base before: {base!r}")
print(f"id before: {id(base)}")

base |= extra

print()
print(f"base after: {base!r}")
print(f"id after: {id(base)}")
```

Possible output:

```
base before: {'Homer': 'D'oh!', 'Jerry': 'No soup for you!'}
id before: 2199812435584

base after: {'Homer': 'D'oh!', 'Jerry': 'Serenity now!', 'Arthur': 'It is only a model.'}
id after: 2199812435584
```

The dictionary object remains the same object. The mapping state is changed.

The method `.update()` is the older method form of in-place merging.

```
base = {
    "Homer": "D'oh!",
}

result = base.update({
    "Jerry": "No soup for you!",
    "Arthur": "It is only a model.",
})
```

```
print(f"base: {base!r}")
print(f"result: {result!r}")
```

Possible output:

```
base: {'Homer': "D'oh!", 'Jerry': 'No soup for you!', 'Arthur': 'It is only a model.'}
result: None
```

Like many in-place mutation methods, `.update()` returns `None`.

Key-value eviction can be performed with `.pop()` or `del`.

The method `.pop(key)` removes a key and returns its associated value.

```
quotes = {
    "Homer": "D'oh!",
    "Jerry": "No soup for you!",
}
```

```
removed = quotes.pop("Jerry")
```

```
print(f"removed: {removed!r}")
print(f"quotes: {quotes!r}")
```

Possible output:

```
removed: 'No soup for you!'
quotes: {'Homer': "D'oh!"}
```

If the key is absent, `.pop()` raises `KeyError` unless a fallback is supplied.

```
quotes = {
    "Homer": "D'oh!",
}
```

```
try:
    value = quotes.pop("Jerry")
except KeyError as err:
    print(f"pop failed: {err}")
```

```
value = quotes.pop("Jerry", "missing quote")
```

```
print(f"value with fallback: {value!r}")
print(f"quotes: {quotes!r}")
```

Possible output:

```
pop failed: 'Jerry'
value with fallback: 'missing quote'
quotes: {'Homer': "D'oh!"}
```

The `del` statement removes a key-value pair but does not return the removed value.

```
quotes = {
    "Homer": "D'oh!",
    "Jerry": "No soup for you!",
}
```

```
del quotes["Homer"]

print(f"quotes: {quotes!r}")
```

Possible output:

```
quotes: {'Jerry': 'No soup for you!'}
```

If the key is absent, `del` raises `KeyError`.

```
quotes = {
    "Homer": "D'oh!",
}

try:
    del quotes["Jerry"]
except KeyError as err:
    print(f"delete failed: {err}")
```

Possible output:

```
delete failed: 'Jerry'
```

The main dynamic modification operations can be summarized as follows:

Operation	Effect	Mutates Existing Dict?
<code>d[key] = value</code>	Insert or replace one key-value pair	Yes
<code>d1 d2</code>	Create merged dictionary	No
<code>d1 = d2</code>	Merge into <code>d1</code>	Yes
<code>d.update(other)</code>	Merge into <code>d</code>	Yes
<code>d.pop(key)</code>	Remove key and return value	Yes
<code>del d[key]</code>	Remove key-value pair	Yes

A final script shows insertion, replacement, merging, and deletion in one state trace.

```
data = {}

print(f"start: {data!r}")

data["Homer"] = "D'oh!"
print(f"after insert Homer: {data!r}")

data["Homer"] = "Mmm... donuts"
print(f"after replace Homer: {data!r}")

data |= {
    "Jerry": "No soup for you!",
    "Arthur": "It is only a model.",
}
print(f"after merge: {data!r}")

removed = data.pop("Jerry")
print(f"removed: {removed!r}")
print(f"after pop: {data!r}")

del data["Arthur"]
print(f"after del: {data!r}")
```

Possible output:

```
start: {}  
after insert Homer: {'Homer': "D'oh!"}  
after replace Homer: {'Homer': 'Mmm... donuts'}  
after merge: {'Homer': 'Mmm... donuts', 'Jerry': 'No soup for you!', 'Arthur': 'It is only a model'}  
removed: 'No soup for you!'  
after pop: {'Homer': 'Mmm... donuts', 'Arthur': 'It is only a model.'}  
after del: {'Homer': 'Mmm... donuts'}
```

The dictionary changes over time, but the access model remains constant: stable hashable keys control the mapping, and values are the associated payloads.

Chapter 8

Function Execution Mechanics, Lexical Scopes, and Advanced Control Architecture

Functions are the point where Python programs stop being only linear sequences of statements and become structured networks of calls, frames, bindings, returns, suspensions, and transformations. A function definition creates a runtime object. A function call creates a new frame. Arguments are bound to local names. Returned objects cross back to the caller. These mechanics form the base layer for almost every advanced control structure in Python.

This chapter follows that machinery from the ordinary to the advanced. It begins with function objects and call frames, then moves into parameter binding, flexible signatures, namespaces, scope resolution, closures, decorators, generators, and native coroutines. Each topic is treated as a continuation of the same runtime model rather than as an isolated syntax feature.

The central purpose is to make Python's control architecture explicit. Closures are not mysterious memory leaks, decorators are not special magic above functions, and generators are not merely unusual loops. They are all consequences of the same core system: objects referenced by names, frames that hold local execution state, compiled code objects that guide execution, and controlled transitions between active, suspended, and terminated computations.

8.1 Anatomy and Definition of Functions

Functions are the first major point where Python execution stops being a simple top-to-bottom sequence of statements and becomes a controlled movement between execution frames. A function definition creates an executable object. A function call creates a temporary frame. A **return** statement terminates that frame and transfers one object reference back to the caller.

This section therefore does not treat functions as a convenient syntax shortcut for repeated code. That view is incomplete. In Python, a function is a runtime object with a name, metadata, defaults, annotations, and a compiled code object behind it. When the program reaches a **def** statement, Python builds such an object and binds it to a name, just as it binds names to strings, lists, dictionaries, or class instances.

The goal of this section is to make that machinery visible before moving into parameter binding, scope resolution, closures, decorators, generators, and coroutines. Those later mechanisms all depend on the same foundation: function objects, call frames, local names, returned object references, and frame termination. Without that model, advanced function behavior looks like special syntax. With it, most of Python's control architecture becomes a direct consequence of ordinary runtime object handling.

8.1.1 Structural Syntax and Function Object Creation

A Python function is not only a named block of source code. At runtime, it is an object. The `def` statement creates that object, connects it to a code object, attaches metadata such as its name and documentation string, and then binds a name to it in the current namespace.

This follows the same object-reference model already used throughout the previous chapters: a name is not a C-style storage box. A name points to an object. In this section, the object happens to be executable.

The important distinction is this:

- executing a `def` statement creates a function object;
- calling the function later creates a new execution frame;
- the body of the function does not run merely because the function was defined.

The next subsection will study return behavior and frame termination. Here we focus only on function creation, function objects, code objects, and lambda expressions.

The `def` Keyword: Execution-Time Function Object Creation and Name Binding

The `def` statement is executed by the interpreter like other statements. When execution reaches it, Python creates a function object and binds the chosen name to that object.

The body is not executed at definition time.

```
print("before def")

def say_fact():
    """Print one short fact."""
    print("Chuck Norris counted to infinity. Twice.")

print("after def")
print()

print(f"say_fact object: {say_fact}")
print(f"type(say_fact): {type(say_fact)}")

print()
print("now calling say_fact()")
result = say_fact()

print()
print(f"returned value: {result!r}")
print(f"type(result): {type(result)}")
```

Possible output:

```
before def
after def

say_fact object: <function say_fact at 0x...>
type(say_fact): <class 'function'>

now calling say_fact()
Chuck Norris counted to infinity. Twice.

returned value: None
type(result): <class 'NoneType'>
```


The line `def say_fact():` creates the function object. The line `say_fact()` calls it.

This means that a function definition is already a runtime event. Python does not merely remember a textual block for later. It constructs an executable object and places a reference to that object under a name.

A function object can be inspected like other objects.

```
def say_fact():
    """Print one short fact."""
    print("Chuck Norris counted to infinity. Twice.")

selected_names = [
    "__call__",
    "__name__",
    "__doc__",
    "__defaults__",
    "__kwdefaults__",
    "__annotations__",
    "__code__",
]

print(f"type(say_fact): {type(say_fact)}")
print()

for name in selected_names:
    print(f"{name:<18} {name in dir(say_fact)}")
```

Possible output:

```
type(say_fact): <class 'function'>

__call__      True
__name__      True
__doc__       True
__defaults__  True
__kwdefaults__ True
__annotations__ True
__code__      True
```

The presence of `__call__` is what makes the object callable. The programmer writes `say_fact()`, but that syntax is backed by the object's call behavior.

The name created by `def` is still only a name. It can be rebound. The old function object does not become invalid just because another name now points somewhere else.

```
def line():
    return "Nobody expects the Spanish Inquisition!"

old_line = line

def line():
    return "It is only a model."

print(f"old_line(): {old_line()}")
print(f"line():     {line()}")

print()
```

```
print(f"old_line object: {old_line}")
print(f"line object:      {line}")
print(f"same object:      {old_line is line}")
```

Possible output:

```
old_line(): Nobody expects the Spanish Inquisition!
line():      It is only a model.
```

```
old_line object: <function line at 0x...>
line object:      <function line at 0x...>
same object:      False
```

The second `def line()`: creates a new function object and binds the name `line` to that new object. The name `old_line` still points to the first function object.

So the real structure is:

```
first def line():
    creates function object A
    binds name line -> A

old_line = line:
    binds name old_line -> A

second def line():
    creates function object B
    rebinds name line -> B
```

This is the same naming model already seen for integers, strings, lists, tuples, sets, and dictionaries. The difference is that this object can be executed.

Function Objects as Runtime Values: `__name__`, `__doc__`, `__defaults__`, `__kwdefaults__`, and `__annotations__`

A function object carries metadata. Some of that metadata comes directly from the source code.

```
def build_line(name: str, score: int = 42, *, mood: str = "calm") -> str:
    """Build one formatted display line."""
    return f"{name:<10} score={score:04d} mood={mood}"
```

```
print(f"function object: {build_line}")
print(f"type:             {type(build_line)}")
print()

print(f"__name__:         {build_line.__name__!r}")
print(f"__doc__:           {build_line.__doc__!r}")
print(f"__defaults__:        {build_line.__defaults__!r}")
print(f"__kwdefaults__:      {build_line.__kwdefaults__!r}")
print(f"__annotations__:     {build_line.__annotations__!r}")

print()
print(build_line("Homer"))
print(build_line("Arthur", 7, mood="confused"))
```

Possible output:

```

function object: <function build_line at 0x...>
type:           <class 'function'>

__name__:       'build_line'
__doc__:        'Build one formatted display line.'
__defaults__:   (42,)
__kwdefaults__: {'mood': 'calm'}
__annotations__: {'name': <class 'str'>, 'score': <class 'int'>, 'mood': <class 'str'>, 'return': <class 'str'>}

Homer          score=0042 mood=calm
Arthur         score=0007 mood=confused

```

The metadata can be read directly:

- `__name__` stores the function's internal name;
- `__doc__` stores the docstring, or `None` if there is no docstring;
- `__defaults__` stores positional default values as a tuple;
- `__kwdefaults__` stores keyword-only default values as a dictionary, or `None`;
- `__annotations__` stores annotations as a dictionary.

The default value 42 is not stored as text. It is stored as an object inside the function object's metadata. This matters later when we discuss default argument evaluation and the mutable default argument trap.

The annotations are also metadata. They do not automatically force runtime type checking.

```

def show_value(x: int) -> str:
    return f"value={x!r}"

print(f"annotations: {show_value.__annotations__}")
print(show_value(42))
print(show_value("No soup for you!"))

```

Possible output:

```

annotations: {'x': <class 'int'>, 'return': <class 'str'>}}
value=42
value='No soup for you!'

```

The annotation says that `x` is intended to be an `int`, but Python does not enforce that automatically. Tools may read the annotation. The runtime call itself still proceeds unless the function body performs an operation that fails.

Function objects also have a writable attribute dictionary. This should not be overused, but it proves again that a function is a normal runtime object.

```

def quote():
    """Return a small quote."""
    return "D'oh!"

quote.source = "The Simpsons"
quote.level = 42

print(f"quote():      {quote()}")
print(f"quote.__dict__: {quote.__dict__}")

```

```
print()

print(f"type(quote.__dict__): {type(quote.__dict__)}")
print(f"'keys' in dir:      {'keys' in dir(quote.__dict__)}")
print(f"'items' in dir:     {'items' in dir(quote.__dict__)}")
```

Possible output:

```
quote():      D'oh!
quote.__dict__: {'source': 'The Simpsons', 'level': 42}

type(quote.__dict__): <class 'dict'>
'keys' in dir:      True
'items' in dir:     True
```

This does not mean ordinary code should use functions as arbitrary storage bags. It only exposes the object model clearly: a function is an object with metadata, behavior, and references to other objects.

The Anatomy of Function Code Objects: Inspecting Bytecode Attributes (`__code__`, `co_code`, `co_varnames`, `co_consts`, `co_names`)

A function object contains a reference to a code object. The function object is the callable runtime wrapper. The code object contains compiled execution information: local variable names, constants, bytecode, and several other low-level attributes.

```
def make_message(name, number=42):
    text = f"{name} says {number}"
    return text.upper()

code_obj = make_message.__code__

print(f"function type: {type(make_message)}")
print(f"code type:     {type(code_obj)}")
print()

selected_names = [
    "co_code",
    "co_varnames",
    "co_consts",
    "co_names",
    "co_argcount",
    "co_filename",
    "co_firstlineno",
]

for name in selected_names:
    print(f"{name:<16} {name in dir(code_obj)}")
```

Possible output:

```
function type: <class 'function'>
code type:     <class 'code'>

co_code        True
```

```

co_varnames      True
co_consts        True
co_names         True
co_argcount      True
co_filename      True
co_firstlineno   True

```

The function object and the code object are different objects. The function object is callable. The code object is not normally called directly by beginner-level Python code.

The most useful code-object fields for inspection are the following:

```

def make_message(name, number=42):
    text = f"{name} says {number}"
    return text.upper()

code_obj = make_message.__code__

print(f"co_argcount:      {code_obj.co_argcount}")
print(f"co_varnames:      {code_obj.co_varnames}")
print(f"co_consts:         {code_obj.co_consts}")
print(f"co_names:          {code_obj.co_names}")
print(f"co_firstlineno:    {code_obj.co_firstlineno}")
print()

print(f"co_code type:      {type(code_obj.co_code)}")
print(f"co_code length:     {len(code_obj.co_code)}")
print(f"co_code hex:          {code_obj.co_code.hex(' ')}")

```

Possible output:

```

co_argcount:      2
co_varnames:      ('name', 'number', 'text')
co_consts:        (None, ' says ')
co_names:         ('upper',)
co_firstlineno:   1

co_code type:      <class 'bytes'>
co_code length:    58
co_code hex:       97 00 7c 00 9b 00 64 01 7c 01 9b 00 9d 03 7d 02 ...

```

The exact byte sequence is version-dependent. CPython bytecode changes across Python releases. The stable idea is not the exact numbers, but the separation:

```

function object:
    callable object with metadata and defaults

code object:
    compiled execution description

co_code:
    raw bytecode bytes

co_varnames:
    local variable names known by the compiled function

```

```
co_consts:
    constants embedded in the compiled function

co_names:
    external names referenced by the compiled function
```

Because `co_code` is a `bytes` object, it can also be inspected as an object.

```
def make_message(name, number=42):
    text = f"{name} says {number}"
    return text.upper()

code_bytes = make_message.__code__.co_code

print(f"type(code_bytes): {type(code_bytes)}")
print()

selected_names = ["hex", "__len__", "__getitem__", "count", "index"]

for name in selected_names:
    print(f"{name:<14} {name in dir(code_bytes)}")

print()
print(f"first 8 byte values: {list(code_bytes[:8])}")
print(f"hex view:           {code_bytes[:8].hex(' ')}")
```

Possible output:

```
type(code_bytes): <class 'bytes'>

hex          True
__len__      True
__getitem__  True
count        True
index        True

first 8 byte values: [151, 0, 124, 0, 155, 0, 100, 1]
hex view:          97 00 7c 00 9b 00 64 01
```

The `bytes` object is not the source code. It is the low-level instruction stream used by CPython's virtual machine. Reading it directly is not pleasant. The standard `dis` module displays it in a more readable form.

```
import dis

def make_message(name, number=42):
    text = f"{name} says {number}"
    return text.upper()

dis.dis(make_message)
```

Possible output on a modern CPython version:

```
3          0 RESUME          0
```

4	2 LOAD_FAST	0 (name)
	4 FORMAT_VALUE	0
	6 LOAD_CONST	1 (' says ')
	8 LOAD_FAST	1 (number)
	10 FORMAT_VALUE	0
	12 BUILD_STRING	3
	14 STORE_FAST	2 (text)
5	16 LOAD_FAST	2 (text)
	18 LOAD_METHOD	0 (upper)
	40 CALL	0
	50 RETURN_VALUE	

The exact instruction list may differ by Python version. The important structure remains the same: the source-level function body has been compiled into instructions. When the function is called, a frame executes those instructions.

A compact way to see the object chain is:

```
def make_message(name, number=42):
    text = f"{name} says {number}"
    return text.upper()

print("name -> function object -> code object -> bytecode bytes")
print()

print(f"name value:      {make_message}")
print(f"function type:    {type(make_message)}")
print(f"code object:      {make_message.__code__}")
print(f"code object type:  {type(make_message.__code__)}")
print(f"bytecode type:    {type(make_message.__code__.co_code)}")
```

Possible output:

```
name -> function object -> code object -> bytecode bytes

name value:      <function make_message at 0x...>
function type:    <class 'function'>
code object:      <code object make_message at 0x..., file "...", line 1>
code object type: <class 'code'>
bytecode type:    <class 'bytes'>
```

This is why a Python function should not be imagined as only a label placed on top of source text. It is a runtime object connected to compiled execution data.

Lambda Expressions: Expression-Level Construction of Anonymous Function Objects

A `lambda` expression also creates a function object. The difference is syntactic. `def` is a statement. `lambda` is an expression.

Because `lambda` is an expression, it can appear where a value is expected.

```
add_42 = lambda x: x + 42

print(f"add_42 object: {add_42}")
print(f"type(add_42):  {type(add_42)}")
print(f"add_42(8):     {add_42(8)}")
```

```
print()

print(f"__name__:      {add_42.__name__!r}")
print(f"__doc__:      {add_42.__doc__!r}")
print(f"__defaults__: {add_42.__defaults__!r}")
print(f"__code__:      {add_42.__code__}")
```

Possible output:

```
add_42 object: <function <lambda> at 0x...>
type(add_42):  <class 'function'>
add_42(8):      50

__name__:      '<lambda>'
__doc__:      None
__defaults__:  None
__code__:      <code object <lambda> at 0x..., file "...", line 1>
```

The name `add_42` is not the `lambda`'s internal function name. It is only a variable name bound to the function object. The function object's internal name is `'<lambda>'`.

A `lambda` still has a code object.

```
add_42 = lambda x: x + 42

code_obj = add_42.__code__

print(f"type(add_42):   {type(add_42)}")
print(f"type(code_obj): {type(code_obj)}")
print()

print(f"co_varnames:    {code_obj.co_varnames}")
print(f"co_consts:       {code_obj.co_consts}")
print(f"co_names:         {code_obj.co_names}")
print(f"co_code type:      {type(code_obj.co_code)}")
print(f"co_code hex:      {code_obj.co_code.hex(' ')}")
```

Possible output:

```
type(add_42):  <class 'function'>
type(code_obj): <class 'code'>

co_varnames:   ('x',)
co_consts:     (None, 42)
co_names:      ()
co_code type:  <class 'bytes'>
co_code hex:   97 00 7c 00 64 01 7a 00 00 00 53 00
```

A `lambda` is therefore not a special lightweight mathematical object. It is still a Python function object. It has a type, a code object, bytecode, local variable names, constants, and call behavior.

The difference between `def` and `lambda` can be shown directly.

```
def add_def(x):
    return x + 42

add_lambda = lambda x: x + 42
```



```

items = [
    ("add_def", add_def),
    ("add_lambda", add_lambda),
]

for label, fn in items:
    print(f"{label:<12} type={type(fn)}")
    print(f"{'':<12} __name__={fn.__name__!r}")
    print(f"{'':<12} result={fn(8)}")
    print(f"{'':<12} vars={fn.__code__.co_varnames}")
    print(f"{'':<12} consts={fn.__code__.co_consts}")
    print()

```

Possible output:

```

add_def      type=<class 'function'>
              __name__='add_def'
              result=50
              vars=('x',)
              consts=(None, 42)

add_lambda   type=<class 'function'>
              __name__='<lambda>'
              result=50
              vars=('x',)
              consts=(None, 42)

```

Both objects are functions. Both can be called. Both have code objects. The lambda form is only more restricted: it contains one expression, not a full statement block.

This is valid:

```

small = lambda x: x.strip().upper()
print(small("  ni!  "))

```

Possible output:

NI!

This is not valid Python:

```

bad = lambda x:
    y = x.strip()
    return y.upper()

```

For multi-step behavior, use `def`.

```

def clean_text(x):
    y = x.strip()
    return y.upper()

print(clean_text("  ni!  "))

```

Possible output:

NI!

The practical rule is simple:

Use `def` when the behavior deserves a named, readable block. Use `lambda` only when a small expression-level function is clearer than a separate definition.

At the runtime level, both forms create function objects.

8.1.2 Return Semantics and Frame Termination

A function call creates a temporary execution frame. That frame contains the local names of the function call and the current execution position inside the function body. When the function finishes, the frame is terminated. The caller receives one object reference as the result of the call.

That result may be an integer, a string, a list, a tuple, `None`, or any other Python object. The caller does not receive the whole local namespace of the callee. It receives only the object explicitly or implicitly returned by the function.

This subsection studies four related mechanisms:

- explicit frame termination through `return`;
- implicit return of `None`;
- multiple return values as ordinary tuple packing;
- recursive calls as repeated frame creation.

The return Statement: Explicit Value Transfer from Callee Frame to Caller Frame

The `return` statement ends the current function call and transfers one object reference back to the caller.

```
def get_answer():
    print("inside get_answer()")
    return 42

print("before call")

value = get_answer()

print("after call")
print(f"value:      {value}")
print(f"type(value): {type(value)}")
```

Possible output:

```
before call
inside get_answer()
after call
value:      42
type(value): <class 'int'>
```

The call expression `get_answer()` is replaced by the returned object. In this example, the caller receives a reference to the integer object 42, then binds the name `value` to that object.

The execution shape is:

```
caller frame:
    reaches: value = get_answer()
```

```
callee frame:
    runs get_answer()
    reaches: return 42
    terminates
```

```
caller frame:
    receives returned object
    binds: value -> 42
```

A `return` statement does not merely send a value. It also stops the active function call immediately. Any normal statement after that `return` in the same path is not executed.

```
def get_quote():
    print("line 1: entering function")
    return "Nobody expects the Spanish Inquisition!"
    print("line 3: this line is unreachable")
```

```
result = get_quote()
```

```
print(f"result: {result}")
```

Possible output:

```
line 1: entering function
result: Nobody expects the Spanish Inquisition!
```

The last `print()` inside the function is not executed because the frame has already terminated. The returned object can be inspected like any other object.

```
def get_quote():
    return "No soup for you!"
```

```
text = get_quote()
```

```
print(f"text:      {text!r}")
print(f"type(text): {type(text)}")
print()
```

```
selected_names = [
    "upper",
    "lower",
    "replace",
    "split",
    "__len__",
    "__getitem__",
]
```

```
for name in selected_names:
    print(f"{name:<14} {name in dir(text)}")
```

```
print()
print(f"text.upper():      {text.upper()}")
```

```
print(f"text.replace(...): {text.replace('soup', 'cake')}")
print(f"len(text): {len(text)}")
print(f"text[0]: {text[0]!r}")
```

Possible output:

```
text:      'No soup for you!'
type(text): <class 'str'>

upper      True
lower      True
replace    True
split      True
__len__    True
__getitem__ True

text.upper():      NO SOUP FOR YOU!
text.replace(...): No cake for you!
len(text):         16
text[0]:           'N'
```

The function call did not return a special “function result” container. It returned a normal `str` object.

A function may contain several `return` statements. Only one return path is taken during one call.

```
def describe_number(n):
    print(f"checking n={n}")

    if n == 42:
        return "the answer"

    if n < 0:
        return "negative"

    return "ordinary"

for value in [42, -3, 7]:
    result = describe_number(value)
    print(f"{value:>4} -> {result}")
```

Possible output:

```
checking n=42
 42 -> the answer
checking n=-3
-3 -> negative
checking n=7
 7 -> ordinary
```

Each call has its own frame. Each frame reaches one return path, gives one object back to the caller, and terminates.

The returned object may also be a mutable object. In that case, the caller receives a reference to that mutable object.

```
def make_lines():
    lines = []
    lines.append("D'oh!")
    lines.append("It is only a model.")
    return lines

result = make_lines()

print(f"result:      {result}")
print(f"type(result): {type(result)}")
print()

selected_names = ["append", "pop", "clear", "__len__", "__getitem__"]

for name in selected_names:
    print(f"{name:<14} {name in dir(result)}")

print()
result.append("Chuck Norris can divide by zero.")

print(f"after append: {result}")
```

Possible output:

```
result:      ["D'oh!", 'It is only a model.']
type(result): <class 'list'>
```

```
append      True
pop          True
clear        True
__len__      True
__getitem__  True
```

```
after append: ["D'oh!", 'It is only a model.', 'Chuck Norris can divide by zero.']
```

The local name `lines` existed inside the function frame. After the frame terminates, that local name is gone. But the list object itself remains alive because the caller now has a reference to it through `result`.

So this is wrong:

The function returns its local variable.

The more precise description is:

The function returns the object currently referenced by that local name.

Implicit Return Defaults: Why Functions Without `return` Produce `None`

A function that reaches the end of its body without executing a `return` statement returns `None` automatically.

```
def print_fact():
    print("Chuck Norris does not sleep. He waits.")

result = print_fact()
```

```
print()
print(f"result:      {result!r}")
print(f"type(result): {type(result)}")
```

Possible output:

```
Chuck Norris does not sleep. He waits.
```

```
result:      None
type(result): <class 'NoneType'>
```

The function produced visible output, but it did not explicitly return a useful object to the caller. Printing and returning are different actions.

- `print()` sends text to the standard output stream.
- `return` sends an object back to the caller.

The object `None` is Python's standard object for "no useful value was returned here." It can be inspected.

```
x = None

print(f"x:      {x!r}")
print(f"type(x): {type(x)}")
print()

selected_names = [
    "__bool__",
    "__class__",
    "__eq__",
    "__repr__",
]

for name in selected_names:
    print(f"{name:<12} {name in dir(x)}")

print()
print(f"bool(None): {bool(None)}")
print(f"repr(None): {repr(None)}")
print(f"x is None:  {x is None}")
```

Possible output:

```
x:      None
type(x): <class 'NoneType'>

__bool__    True
__class__   True
__eq__      True
__repr__    True

bool(None): False
repr(None): None
x is None:  True
```

There is normally only one `None` object in a running Python process. For this reason, code usually checks it with `is None`, not `== None`.

The distinction between printing and returning is one of the most important early function mistakes.

```
def noisy_add(a, b):
    print(a + b)

def real_add(a, b):
    return a + b

x = noisy_add(20, 22)
y = real_add(20, 22)

print()
print(f"x: {x!r}")
print(f"y: {y!r}")
```

Possible output:

```
42
```

```
x: None
y: 42
```

The function `noisy_add()` prints 42, but the caller receives `None`. The function `real_add()` returns 42, so the caller can store it, pass it to another function, or use it in a larger expression.

This difference becomes visible when composing function calls.

```
def noisy_add(a, b):
    print(a + b)

def real_add(a, b):
    return a + b

print("using real_add inside another expression:")
total = real_add(10, 32) * 2
print(f"total: {total}")

print()
print("using noisy_add inside another expression:")
bad_total = noisy_add(10, 32)
print(f"bad_total: {bad_total!r}")
```

Possible output:

```
using real_add inside another expression:
total: 84
```

```
using noisy_add inside another expression:
42
bad_total: None
```

If we tried to multiply the result of `noisy_add(10, 32)` directly, Python would fail because the returned object is `None`, not an integer.

```
def noisy_add(a, b):
    print(a + b)

try:
    value = noisy_add(10, 32) * 2
except TypeError as err:
    print(f"error type: {type(err)}")
    print(f"message: {err}")
```

Possible output:

```
42
error type: <class 'TypeError'>
message: unsupported operand type(s) for *: 'NoneType' and 'int'
```

The function call printed 42, but the expression tried to multiply `None` by 2. These are separate events.

An explicit empty `return` also returns `None`.

```
def stop_early(flag):
    print("entered function")

    if flag:
        print("stopping early")
        return

    print("finished normally")

a = stop_early(True)
b = stop_early(False)

print()
print(f"a: {a!r}")
print(f"b: {b!r}")
```

Possible output:

```
entered function
stopping early
entered function
finished normally
```

```
a: None
b: None
```

Both calls return `None`. The difference is only where the frame terminated.

Multiple Return Values as Tuple Packing: The Real Structure Behind `return a, b`

Python functions do not truly return several independent values. A function always returns one object reference. When the source code says `return a, b`, Python creates a tuple and returns that tuple.

```
def get_pair():
    a = "Homer"
    b = 42
```



```

    return a, b

result = get_pair()

print(f"result:      {result}")
print(f"type(result): {type(result)}")
print()
print(f"first item:   {result[0]!r}")
print(f"second item:  {result[1]!r}")

```

Possible output:

```

result:      ('Homer', 42)
type(result): <class 'tuple'>

first item:   'Homer'
second item:  42

```

The source line:

```
return a, b
```

really means:

```

make tuple object containing references to a and b
return that one tuple object

```

The tuple object can be inspected.

```

def get_pair():
    return "Jerry", "These pretzels are making me thirsty."

pair = get_pair()

print(f"pair:      {pair}")
print(f"type(pair): {type(pair)}")
print()

selected_names = [
    "count",
    "index",
    "__len__",
    "__getitem__",
    "__contains__",
]

for name in selected_names:
    print(f"{name:<14} {name in dir(pair)}")

print()
print(f"len(pair):      {len(pair)}")
print(f"pair[0]:         {pair[0]!r}")
print(f"pair.count('Jerry'): {pair.count('Jerry')}")
print(f"'Jerry' in pair:   {'Jerry' in pair}")

```

Possible output:

```

pair:      ('Jerry', 'These pretzels are making me thirsty.')
type(pair): <class 'tuple'>

count      True
index      True
__len__    True
__getitem__ True
__contains__ True

len(pair):      2
pair[0]:        'Jerry'
pair.count('Jerry'): 1
'Jerry' in pair: True

```

The caller may keep the tuple as one object.

```

def get_status():
    return "ok", 200, "D'oh!"

status = get_status()

print(f"status:      {status}")
print(f"type(status): {type(status)}")

```

Possible output:

```

status:      ('ok', 200, "D'oh!")
type(status): <class 'tuple'>

```

Or the caller may immediately unpack the tuple into several names.

```

def get_status():
    return "ok", 200, "D'oh!"

label, code, message = get_status()

print(f"label:      {label!r}")
print(f"code:       {code!r}")
print(f"message:    {message!r}")

```

Possible output:

```

label:      'ok'
code:       200
message:    "D'oh!"

```

The function did not return into three different caller variables. The function returned one tuple. Then the caller unpacked that tuple.

This can be made explicit:

```

def get_status():
    return "ok", 200, "D'oh!"

temp = get_status()

```

```

label = temp[0]
code = temp[1]
message = temp[2]

print(f"temp:      {temp}")
print(f"label:    {label!r}")
print(f"code:     {code!r}")
print(f"message: {message!r}")

```

Possible output:

```

temp:      ('ok', 200, "D'oh!")
label:     'ok'
code:      200
message:   "D'oh!"

```

Tuple unpacking requires the number of target names to match the number of elements, unless starred unpacking is used.

```

def get_three():
    return "Arthur", "Patsy", "Bridgekeeper"

try:
    a, b = get_three()
except ValueError as err:
    print(f"error type: {type(err)}")
    print(f"message:    {err}")

```

Possible output:

```

error type: <class 'ValueError'>
message:    too many values to unpack (expected 2)

```

The error belongs to the unpacking operation in the caller, not to the `return` statement itself. Starred unpacking can collect the overflow into a list.

```

def get_names():
    return "Arthur", "Patsy", "Robin", "Galahad"

first, second, *others = get_names()

print(f"first:      {first!r}")
print(f"second:     {second!r}")
print(f"others:      {others}")
print(f"type(others): {type(others)}")

```

Possible output:

```

first:      'Arthur'
second:     'Patsy'
others:     ['Robin', 'Galahad']
type(others): <class 'list'>

```

Notice the detail: the returned object is a tuple, but the starred target receives a list. The list can also be inspected.

```

def get_names():
    return "Arthur", "Patsy", "Robin", "Galahad"

first, second, *others = get_names()

print(f"type(others): {type(others)}")
print()

selected_names = [
    "append",
    "extend",
    "pop",
    "__len__",
    "__getitem__",
]

for name in selected_names:
    print(f"{name:<14} {name in dir(others)}")

print()
others.append("Lancelot")
print(f"others after append: {others}")

```

Possible output:

```

type(others): <class 'list'>

append      True
extend      True
pop          True
__len__     True
__getitem__ True

others after append: ['Robin', 'Galahad', 'Lancelot']

```

So the complete structure is:

```

return a, b, c:
    creates tuple (a, b, c)
    returns that tuple object

x, y, z = function_call():
    receives tuple
    unpacks tuple into names x, y, z

x, *rest = function_call():
    receives tuple
    binds first element to x
    collects remaining elements into a new list

```

Recursive Calls: Repeated Frame Allocation, Base Cases, and Recursion Depth Boundaries

A recursive function is a function that calls itself. This does not reuse the same active frame. Each call creates a new frame with its own local names.

The simplest useful recursive example is a sum over a decreasing integer range.

```
def sum_down(n):
    print(f"enter sum_down({n})")

    if n == 0:
        print("base case reached")
        return 0

    result = n + sum_down(n - 1)

    print(f"leave sum_down({n}) -> {result}")
    return result

total = sum_down(4)

print()
print(f"total: {total}")
```

Possible output:

```
enter sum_down(4)
enter sum_down(3)
enter sum_down(2)
enter sum_down(1)
enter sum_down(0)
base case reached
leave sum_down(1) -> 1
leave sum_down(2) -> 3
leave sum_down(3) -> 6
leave sum_down(4) -> 10

total: 10
```

The call sequence first goes downward:

```
sum_down(4)
  calls sum_down(3)
    calls sum_down(2)
      calls sum_down(1)
        calls sum_down(0)
```

Then the returns come back upward:

```
sum_down(0) returns 0
sum_down(1) receives 0 and returns 1 + 0
sum_down(2) receives 1 and returns 2 + 1
sum_down(3) receives 3 and returns 3 + 3
sum_down(4) receives 6 and returns 4 + 6
```

The base case is the condition that stops the repeated calls. Without a base case, or without progress toward the base case, recursion does not terminate normally.

```
def bad_count(n):
    print(f"bad_count({n})")
    return bad_count(n)
```

```
try:
    bad_count(42)
except RecursionError as err:
    print()
    print(f"error type: {type(err)}")
    print(f"message: {err}")
```

Possible output:

```
bad_count(42)
bad_count(42)
bad_count(42)
...
error type: <class 'RecursionError'>
message: maximum recursion depth exceeded
```

Python prevents infinite recursive frame creation by enforcing a recursion depth limit.

The current limit can be inspected with the standard `sys` module.

```
import sys

limit = sys.getrecursionlimit()

print(f"recursion limit: {limit}")
print(f"type(limit): {type(limit)}")
```

Possible output:

```
recursion limit: 1000
type(limit): <class 'int'>
```

The exact value may differ if the process changed it earlier. The important point is that recursive calls consume stack depth because every active call needs its own frame.

We can inspect active frames with the standard `inspect` module. This is not ordinary application code, but it is useful for seeing the mechanics.

```
import inspect

def show_frame(n):
    frame = inspect.currentframe()

    print(f"n: {n}")
    print(f"type(frame): {type(frame)}")
    print(f"function name: {frame.f_code.co_name}")
    print(f"local names: {list(frame.f_locals.keys())}")

show_frame(42)
```

Possible output:

```
n: 42
type(frame): <class 'frame'>
function name: show_frame
local names: ['n', 'frame']
```

The frame object also has inspectable attributes.

```

import inspect

def show_frame():
    frame = inspect.currentframe()

    selected_names = [
        "f_code",
        "f_locals",
        "f_globals",
        "f_back",
        "f_lineno",
    ]

    print(f"type(frame): {type(frame)}")
    print()

    for name in selected_names:
        print(f"{name:<12} {name in dir(frame)}")

show_frame()

```

Possible output:

```

type(frame): <class 'frame'>

f_code      True
f_locals    True
f_globals   True
f_back      True
f_lineno    True

```

A recursive call chain has several frames alive at the same time. The deepest active call can look backward through previous caller frames using `f_back`.

```

import inspect

def show_chain(n):
    if n > 0:
        return show_chain(n - 1)

    frame = inspect.currentframe()
    depth = 0

    while frame is not None:
        name = frame.f_code.co_name
        print(f"{depth:>2}: {name}")
        frame = frame.f_back
        depth += 1

show_chain(3)

```

Possible output:

```

0: show_chain
1: show_chain

```

```

2: show_chain
3: show_chain
4: <module>

```

The repeated name `show_chain` does not mean there is one frame being reused. It means there are several active frames, all running the same function code object with different local states.

A clearer recursive sum can print local state at each level.

```

def sum_down(n, depth=0):
    pad = "  " * depth
    print(f"{pad}enter n={n}")

    if n == 0:
        print(f"{pad}return 0")
        return 0

    sub_total = sum_down(n - 1, depth + 1)
    result = n + sub_total

    print(f"{pad}return {result}")
    return result

answer = sum_down(4)

print()
print(f"answer: {answer}")

```

Possible output:

```

enter n=4
  enter n=3
    enter n=2
      enter n=1
        enter n=0
          return 0
        return 1
      return 3
    return 6
  return 10

answer: 10

```

This indentation is only printed text, but it mirrors the nested frame structure. Each call waits for the smaller call to finish. When the smaller call returns, the waiting call continues.

The base case is not an optional decoration. It is the exit door.

```

def sum_down(n):
    if n == 0:
        return 0

    return n + sum_down(n - 1)

print(sum_down(4))

```

This version works for 4, but it is incomplete as a general function because negative input moves away from the base case.


```
def sum_down(n):
    if n == 0:
        return 0

    return n + sum_down(n - 1)

try:
    print(sum_down(-1))
except RecursionError as err:
    print(f"error type: {type(err)}")
    print(f"message: {err}")
```

The problem is not that recursion is mysterious. The problem is that the function never reaches `n == 0` when it starts at `-1` and keeps subtracting.

A safer version checks the input boundary.

```
def sum_down(n):
    if n < 0:
        raise ValueError("n must be zero or positive")

    if n == 0:
        return 0

    return n + sum_down(n - 1)

for value in [4, 0, -1]:
    try:
        result = sum_down(value)
        print(f"sum_down({value:>2}) -> {result}")
    except ValueError as err:
        print(f"sum_down({value:>2}) -> {type(err).__name__}: {err}")
```

Possible output:

```
sum_down( 4) -> 10
sum_down( 0) -> 0
sum_down(-1) -> ValueError: n must be zero or positive
```

Recursive programming requires three concrete questions:

- What is the smallest case that can be answered immediately?
- How does each recursive call move toward that smallest case?
- What does the caller do with the returned object when the smaller call finishes?

For `sum_down(n)`, the answers are simple:

- `n == 0` returns 0;
- each call uses `n - 1`;
- each waiting caller adds its own `n` to the returned subtotal.

That is the whole machine. Recursion is repeated frame allocation plus delayed continuation. The function does not magically loop inside one frame. It creates a chain of calls, reaches a base case, and then unwinds by returning one object at a time.

8.2 Function Call Mechanics and Parameter Binding

A function definition creates a callable object, but the real execution machinery begins when that object is called. A call is not only a jump to another block of code. It is a structured operation: argument expressions are evaluated, object references are collected, a new frame is created, and parameter names inside that frame are bound according to the function signature.

This section studies that entrance mechanism in detail. First, it explains what is actually passed across the call boundary: not C-style variable boxes, not copied Python objects, but references to already existing objects bound to new local names. This is why reassignment inside a function stays local, while mutation of a shared mutable object can be visible to the caller.

After that, the section turns to signatures: positional arguments, keyword arguments, default values, positional-only parameters, keyword-only parameters, and annotations. These features are not decorative syntax. They are the rules by which Python constructs the initial local namespace of a function call.

8.2.1 Memory Semantics of Parameter Passing

A function call does not move variables from the caller into the callee. It evaluates argument expressions, obtains object references, creates a new function frame, and binds the function's parameter names to those same objects.

This is the central point: the callee receives access to objects, not ownership of the caller's variable names.

In C, it is natural to ask whether something is passed by value or by pointer. In Python, that question must be translated. Python names are already references to objects. Function parameters are new local names created inside the callee frame. Those local names are bound to the objects produced by the caller's argument expressions.

So the call boundary does not copy the full object. It also does not give the callee direct control over the caller's name slot. It shares the object reference.

The Call-by-Object / Call-by-Sharing Evaluation Model: Passing Object References into New Local Bindings

Python's parameter passing model is often called *call by object*, *call by object reference*, or *call by sharing*. The names differ, but the mechanism is simple:

1. the caller evaluates the argument expression;
2. the result is an object reference;
3. the callee frame is created;
4. the parameter name inside the callee is bound to that same object;
5. reassignment of the parameter name affects only the callee's local binding;
6. in-place mutation of the shared object is visible through all names that still reference that object.

Start with an immutable object, such as an integer.

```
def inspect_number(n):
    print("inside inspect_number()")
    print(f"n:         {n}")
    print(f"type(n):    {type(n)}")
    print(f"id(n):      {id(n)}")
```

```

print("before call")
print(f"x:      {x}")
print(f"type(x): {type(x)}")
print(f"id(x):   {id(x)}")
print()

```

```
inspect_number(x)
```

Possible output:

```

before call
x:      42
type(x): <class 'int'>
id(x):   140731031890376

inside inspect_number()
n:      42
type(n): <class 'int'>
id(n):   140731031890376

```

The exact identity number is not important. The useful observation is that `x` in the caller and `n` in the callee refer to the same integer object during the call.

The object can be inspected.

```

x = 42

print(f"type(x): {type(x)}")
print()

selected_names = [
    "__add__",
    "__sub__",
    "__mul__",
    "__eq__",
    "__repr__",
    "bit_length",
]

for name in selected_names:
    print(f"{name:<14} {name in dir(x)}")

print()
print(f"x + 8:      {x + 8}")
print(f"x.bit_length(): {x.bit_length()}")
print(f"x.__add__(8): {x.__add__(8)}")

```

Possible output:

```

type(x): <class 'int'>

__add__      True
__sub__      True
__mul__      True
__eq__       True
__repr__     True

```

```
bit_length      True
```

```
x + 8:          50
x.bit_length(): 6
x.__add__(8):   50
```

The integer object is immutable. There is no operation that changes the integer object 42 into another integer object. Arithmetic creates another integer object and binds a name to it.

```
def change_number(n):
    print("inside change_number(), before reassignment")
    print(f"n:      {n}")
    print(f"id(n): {id(n)}")
    print()

    n = n + 1

    print("inside change_number(), after reassignment")
    print(f"n:      {n}")
    print(f"id(n): {id(n)}")

x = 42

print("caller, before call")
print(f"x:      {x}")
print(f"id(x): {id(x)}")
print()

change_number(x)

print()
print("caller, after call")
print(f"x:      {x}")
print(f"id(x): {id(x)}")
```

Possible output:

```
caller, before call
x:      42
id(x): 140731031890376

inside change_number(), before reassignment
n:      42
id(n): 140731031890376

inside change_number(), after reassignment
n:      43
id(n): 140731031890408

caller, after call
x:      42
id(x): 140731031890376
```

The assignment:

```
n = n + 1
```

does not modify the caller's `x`. It creates or obtains another integer object and rebinds the local name `n`. The caller's name `x` still points to the original object.

The same call boundary behaves differently when the object is mutable.

```
def add_line(lines):
    print("inside add_line(), before mutation")
    print(f"lines:      {lines}")
    print(f"id(lines): {id(lines)}")
    print()

    lines.append("Nobody expects the Spanish Inquisition!")

    print("inside add_line(), after mutation")
    print(f"lines:      {lines}")
    print(f"id(lines): {id(lines)}")

quotes = ["D'oh!", "No soup for you!"]

print("caller, before call")
print(f"quotes:      {quotes}")
print(f"id(quotes): {id(quotes)}")
print()

add_line(quotes)

print()
print("caller, after call")
print(f"quotes:      {quotes}")
print(f"id(quotes): {id(quotes)}")
```

Possible output:

```
caller, before call
quotes:      ["D'oh!", 'No soup for you!']
id(quotes): 2283758137600

inside add_line(), before mutation
lines:      ["D'oh!", 'No soup for you!']
id(lines): 2283758137600

inside add_line(), after mutation
lines:      ["D'oh!", 'No soup for you!', 'Nobody expects the Spanish Inquisition!']
id(lines): 2283758137600

caller, after call
quotes:      ["D'oh!", 'No soup for you!', 'Nobody expects the Spanish Inquisition!']
id(quotes): 2283758137600
```

The local parameter name `lines` and the caller name `quotes` refer to the same list object. The function does not modify the caller's name. It modifies the shared list object.

The list object has mutating methods.

```

quotes = ["D'oh!", "No soup for you!"]

print(f"type(quotes): {type(quotes)}")
print()

selected_names = [
    "append",
    "extend",
    "insert",
    "pop",
    "clear",
    "__len__",
    "__getitem__",
    "__setitem__",
]

for name in selected_names:
    print(f"{name:<14} {name in dir(quotes)}")

print()
quotes.append("These pretzels are making me thirsty.")
print(f"after append: {quotes}")

quotes[0] = "Mmm... donuts."
print(f"after item assignment: {quotes}")

```

Possible output:

```
type(quotes): <class 'list'>
```

```

append      True
extend      True
insert      True
pop         True
clear       True
__len__     True
__getitem__ True
__setitem__ True

```

```
after append: ["D'oh!", 'No soup for you!', 'These pretzels are making me thirsty.']
```

```
after item assignment: ['Mmm... donuts.', 'No soup for you!', 'These pretzels are making me thirsty.']
```

The important distinction is not whether the object was passed differently. The call mechanism is the same. The object type determines what can happen after the reference is shared.

- An immutable object cannot be changed in place.
- A mutable object can be changed in place.
- Reassigning a parameter name never reassigns the caller's name.

C vs. Python Call Frames: Copied Primitive Values and Pointers vs. Python Names Bound to Shared Heap Objects

A C programmer often carries two mental models into function calls:

- passing a primitive value copies the value into the callee;
- passing a pointer copies an address into the callee, so the callee can access external storage through that pointer.

Python should not be forced into that exact pair of categories. Python source code does not expose variable boxes in the same way.

In C, a local variable such as `int x` names a storage location that directly contains an integer value. In Python, a name such as `x` is a binding to an object. The integer object lives independently from the name.

A Python call creates a new local name in the callee frame. That local name is bound to the same object as the argument expression.

```
def show_binding(n):
    print("callee frame")
    print(f"n:      {n}")
    print(f"id(n): {id(n)}")
```

```
x = 42
```

```
print("caller frame")
print(f"x:      {x}")
print(f"id(x): {id(x)}")
print()
```

```
show_binding(x)
```

Possible output:

```
caller frame
x:      42
id(x): 140731031890376

callee frame
n:      42
id(n): 140731031890376
```

A C-like but wrong description would be:

The Python variable `x` is passed into `n`.

A better description is:

The object referenced by `x` is also referenced by the new local name `n`.

Now compare reassignment.

```
def reassign_number(n):
    print("inside reassign_number()")
    print(f"before: n={n}, id(n)={id(n)}")
```

```
n = 100
```

```
print(f"after:  n={n}, id(n)={id(n)}")
```

```
x = 42
```

```
print("caller before")
print(f"x={x}, id(x)={id(x)}")
print()
```

```
reassign_number(x)
```

```
print()
print("caller after")
print(f"x={x}, id(x)={id(x)}")
```

Possible output:

```
caller before
x=42, id(x)=140731031890376
```

```
inside reassign_number()
before: n=42, id(n)=140731031890376
after:  n=100, id(n)=140731031892232
```

```
caller after
x=42, id(x)=140731031890376
```

The assignment `n = 100` changes the local binding inside the callee. It does not reach backward into the caller's frame and change `x`.

The same is true for lists if the parameter name itself is rebound.

```
def reassign_list(lines):
    print("inside reassign_list()")
    print(f"before: lines={lines}")
    print(f"before: id(lines)={id(lines)}")
    print()
```

```
    lines = ["It is only a model.", "Ni!"]
```

```
    print(f"after:  lines={lines}")
    print(f"after:  id(lines)={id(lines)}")
```

```
quotes = ["D'oh!", "No soup for you!"]
```

```
print("caller before")
print(f"quotes:      {quotes}")
print(f"id(quotes): {id(quotes)}")
print()
```

```
reassign_list(quotes)
```

```
print()
print("caller after")
print(f"quotes:      {quotes}")
print(f"id(quotes): {id(quotes)}")
```

Possible output:


```

caller before
quotes:      ["D'oh!", 'No soup for you!']
id(quotes): 2283758137600

inside reassign_list()
before: lines=["D'oh!", 'No soup for you!']
before: id(lines)=2283758137600

after: lines=['It is only a model.', 'Ni!']
after: id(lines)=2283758110976

caller after
quotes:      ["D'oh!", 'No soup for you!']
id(quotes): 2283758137600

```

Even though the original object was mutable, the function did not mutate it. It only made the local parameter name point to a different list.

This is the crucial difference:

```
lines.append("Ni!")
```

mutates the object referenced by `lines`.

```
lines = ["Ni!"]
```

rebinds the local name `lines`.

The first operation can be visible to the caller if the caller also references the same object. The second operation is local to the callee frame.

A useful C comparison is this:

- Python does not copy the full object into the callee.
- Python also does not pass the caller's variable slot itself.
- Python creates a new local binding to the same object.

So Python parameter passing resembles pointer sharing only in the sense that the callee can reach the same heap object. It does not resemble C pointer parameters in the sense of allowing direct reassignment of the caller's variable slot.

This is why a Python function cannot swap two caller names by ordinary assignment.

```

def bad_swap(a, b):
    print("inside bad_swap(), before")
    print(f"a={a!r}, b={b!r}")

    a, b = b, a

    print("inside bad_swap(), after")
    print(f"a={a!r}, b={b!r}")

x = "Arthur"
y = "Patsy"

print("caller before")
print(f"x={x!r}, y={y!r}")
print()

```

```
bad_swap(x, y)

print()
print("caller after")
print(f"x={x!r}, y={y!r}")
```

Possible output:

```
caller before
x='Arthur', y='Patsy'

inside bad_swap(), before
a='Arthur', b='Patsy'
inside bad_swap(), after
a='Patsy', b='Arthur'

caller after
x='Arthur', y='Patsy'
```

The swap happened only inside the callee frame. The caller's names were not changed.

The correct Python way is to return the new values and let the caller rebind its own names.

```
def make_swapped(a, b):
    return b, a

x = "Arthur"
y = "Patsy"

print("before")
print(f"x={x!r}, y={y!r}")

x, y = make_swapped(x, y)

print()
print("after")
print(f"x={x!r}, y={y!r}")
```

Possible output:

```
before
x='Arthur', y='Patsy'

after
x='Patsy', y='Arthur'
```

The callee returns one tuple object. The caller unpacks that tuple and explicitly rebinds its own names.

Side Effects Matrix: Mutating Mutable Objects In Place vs. Reassigning Local Names

The main source of confusion is that mutation and reassignment look superficially similar in prose. They are not the same operation.

Operation inside function	Object kind	What changes?	Caller sees it?
<code>n = n + 1</code>	immutable int	local name <code>n</code> is rebound	no
<code>text = text.upper()</code>	immutable str	local name <code>text</code> is rebound	no
<code>items.append(x)</code>	mutable list	shared list object is mutated	yes
<code>items[0] = x</code>	mutable list	shared list object is mutated	yes
<code>items = []</code>	mutable list	local name <code>items</code> is rebound	no

The table can be tested directly.

```
def change_int(n):
    print("change_int()")
    print(f"before: n={n}, id(n)={id(n)}")
    n = n + 1
    print(f"after:  n={n}, id(n)={id(n)}")

x = 42

print("caller before")
print(f"x={x}, id(x)={id(x)}")
print()

change_int(x)

print()
print("caller after")
print(f"x={x}, id(x)={id(x)}")
```

Possible output:

```
caller before
x=42, id(x)=140731031890376

change_int()
before: n=42, id(n)=140731031890376
after:  n=43, id(n)=140731031890408

caller after
x=42, id(x)=140731031890376
```

The caller sees no change because no shared object was mutated. The local name `n` was rebound. Strings behave the same way because strings are immutable.

```
def change_text(text):
    print("change_text()")
    print(f"before: text={text!r}, id(text)={id(text)}")
    text = text.upper()
    print(f"after:  text={text!r}, id(text)={id(text)}")

msg = "d'oh!"
```

```

print("caller before")
print(f"msg={msg!r}, id(msg)={id(msg)}")
print()

change_text(msg)

print()
print("caller after")
print(f"msg={msg!r}, id(msg)={id(msg)}")

```

Possible output:

```

caller before
msg="d'oh!", id(msg)=2283758078832

change_text()
before: text="d'oh!", id(text)=2283758078832
after:  text="D'OH!", id(text)=2283758068400

caller after
msg="d'oh!", id(msg)=2283758078832

```

The method `upper()` does not modify the original string. It returns a new string. The local name `text` is then rebound to that new string.

The string object can be inspected.

```

msg = "d'oh!"

print(f"type(msg): {type(msg)}")
print()

selected_names = [
    "upper",
    "lower",
    "replace",
    "split",
    "__len__",
    "__getitem__",
]

for name in selected_names:
    print(f"{name:<14} {name in dir(msg)}")

print()
print(f"msg.upper():      {msg.upper()}")
print(f"msg.replace():     {msg.replace('d', 'D')}")
print(f"original msg:       {msg}")

```

Possible output:

```

type(msg): <class 'str'>

upper      True
lower      True

```

```

replace      True
split        True
__len__      True
__getitem__  True

msg.upper():   D'OH!
msg.replace(): D'oh!
original msg:  d'oh!

```

Now compare a list mutation.

```

def mutate_list(items):
    print("mutate_list()")
    print(f"before: items={items}")
    print(f"before: id(items)={id(items)}")
    print()

    items.append("Chuck Norris can hear sign language.")

    print(f"after:  items={items}")
    print(f"after:  id(items)={id(items)}")

facts = ["The answer is 42."]

print("caller before")
print(f"facts:      {facts}")
print(f"id(facts): {id(facts)}")
print()

mutate_list(facts)

print()
print("caller after")
print(f"facts:      {facts}")
print(f"id(facts): {id(facts)}")

```

Possible output:

```

caller before
facts:      ['The answer is 42.']
id(facts):  2283758137600

mutate_list()
before: items=['The answer is 42.']
before: id(items)=2283758137600

after:  items=['The answer is 42.', 'Chuck Norris can hear sign language.']
after:  id(items)=2283758137600

caller after
facts:      ['The answer is 42.', 'Chuck Norris can hear sign language.']
id(facts):  2283758137600

```

The caller sees the change because the list object itself was changed.
Now compare local rebinding of a list parameter.

```
def replace_list(items):
    print("replace_list()")
    print(f"before: items={items}")
    print(f"before: id(items)={id(items)}")
    print()

    items = ["This is an ex-list."]

    print(f"after:  items={items}")
    print(f"after:  id(items)={id(items)}")

facts = ["The answer is 42."]

print("caller before")
print(f"facts:      {facts}")
print(f"id(facts): {id(facts)}")
print()

replace_list(facts)

print()
print("caller after")
print(f"facts:      {facts}")
print(f"id(facts): {id(facts)}")
```

Possible output:

```
caller before
facts:      ['The answer is 42.']
id(facts):  2283758137600

replace_list()
before: items=['The answer is 42.']
before: id(items)=2283758137600

after:  items=['This is an ex-list.']
after:  id(items)=2283758110976

caller after
facts:      ['The answer is 42.']
id(facts):  2283758137600
```

The caller sees no change because the function did not mutate the original list. It created a new list and rebound the local name `items`.

The same distinction applies to dictionaries.

```
def mutate_dict(data):
    data["quote"] = "These pretzels are making me thirsty."

def reassign_dict(data):
    data = {"quote": "It is only a model."}
    print(f"inside reassign_dict(): {data}")

info = {"answer": 42}
```

```

print("before")
print(f"info:      {info}")
print(f"id(info): {id(info)}")
print()

mutate_dict(info)

print("after mutate_dict()")
print(f"info:      {info}")
print(f"id(info): {id(info)}")
print()

reassign_dict(info)

print()
print("after reassign_dict()")
print(f"info:      {info}")
print(f"id(info): {id(info)}")

```

Possible output:

```

before
info:      {'answer': 42}
id(info): 2283758217728

after mutate_dict()
info:      {'answer': 42, 'quote': 'These pretzels are making me thirsty.'}
id(info): 2283758217728

inside reassign_dict(): {'quote': 'It is only a model.'}

after reassign_dict()
info:      {'answer': 42, 'quote': 'These pretzels are making me thirsty.'}
id(info): 2283758217728

```

The dictionary object can be inspected too.

```

info = {"answer": 42}

print(f"type(info): {type(info)}")
print()

selected_names = [
    "keys",
    "values",
    "items",
    "get",
    "update",
    "__getitem__",
    "__setitem__",
]

for name in selected_names:
    print(f"{name:<14} {name in dir(info)}")

```

```

print()
print(f"info.keys():          {list(info.keys())}")
print(f"info.get('answer'): {info.get('answer')}")
info.update({"quote": "Ni!"})
print(f"after update:       {info}")

```

Possible output:

```

type(info): <class 'dict'>

keys          True
values        True
items         True
get           True
update        True
__getitem__   True
__setitem__   True

info.keys():   ['answer']
info.get('answer'): 42
after update:  {'answer': 42, 'quote': 'Ni!'}
```

The final rule is compact:

Function calls share objects, not caller variable slots. Mutation changes a shared object.
Reassignment changes only the local name.

This rule explains most apparent contradictions in Python parameter passing.

8.2.2 Function Signature Architecture

A function signature is the visible call contract of a function. It tells the caller how arguments may be supplied and how those argument objects will be bound to local parameter names when the call frame is created.

The signature does not describe the whole function body. It describes the entrance gate into the function frame.

A Python signature can specify:

- positional parameters;
- keyword parameters;
- default values;
- positional-only parameters;
- keyword-only parameters;
- annotations.

The previous subsection explained what is passed across the call boundary: object references are bound to new local parameter names. This subsection explains how Python decides which argument object is bound to which parameter name.

Positional Parameters and Positional Argument Binding

The simplest function signature lists parameters in order.

```
def show_line(name, score):  
    print(f"name={name!r}")  
    print(f"score={score!r}")
```

```
show_line("Homer", 42)
```

Possible output:

```
name='Homer'  
score=42
```

The call:

```
show_line("Homer", 42)
```

binds arguments by position:

```
first argument  -> name  
second argument -> score
```

So the function call creates a frame whose local parameter bindings begin like this:

```
name  -> "Homer"  
score -> 42
```

The order matters.

```
def show_line(name, score):  
    print(f"name={name!r}")  
    print(f"score={score!r}")
```

```
show_line(42, "Homer")
```

Possible output:

```
name=42  
score='Homer'
```

Python does not know that the programmer probably meant the opposite order. It follows the signature and binds positionally.

The objects received by the function are ordinary objects.

```
def show_line(name, score):  
    print(f"name:          {name!r}")  
    print(f"type(name):    {type(name)}")  
    print(f"score:         {score!r}")  
    print(f"type(score):   {type(score)}")
```

```
show_line("Homer", 42)
```

Possible output:

```

name:      'Homer'
type(name): <class 'str'>
score:     42
type(score): <class 'int'>

```

The string object bound to `name` has string behavior.

```

name = "Homer"

print(f"type(name): {type(name)}")
print()

selected_names = [
    "upper",
    "lower",
    "replace",
    "__len__",
    "__getitem__",
]

for attr in selected_names:
    print(f"{attr:<14} {attr in dir(name)}")

print()
print(f"name.upper():      {name.upper()}")
print(f"name.replace(...): {name.replace('H', 'B')}")
print(f"len(name):           {len(name)}")

```

Possible output:

```

type(name): <class 'str'>

upper      True
lower      True
replace    True
__len__    True
__getitem__ True

name.upper():      HOMER
name.replace(...): Bomer
len(name):         5

```

The integer object bound to `score` has integer behavior.

```

score = 42

print(f"type(score): {type(score)}")
print()

selected_names = [
    "__add__",
    "__sub__",
    "__mul__",
    "__eq__",
    "bit_length",
]

```

```

]

for attr in selected_names:
    print(f"{attr:<14} {attr in dir(score)}")

print()
print(f"score + 8:           {score + 8}")
print(f"score.bit_length(): {score.bit_length()}")

```

Possible output:

```
type(score): <class 'int'>
```

```

__add__      True
__sub__      True
__mul__      True
__eq__       True
bit_length   True

```

```

score + 8:           50
score.bit_length():  6

```

The binding mechanism is not affected by these object types. The first argument is bound to the first parameter. The second argument is bound to the second parameter.

If the caller supplies too few positional arguments, Python raises `TypeError` before the function body runs.

```

def show_line(name, score):
    print("this line will not run if binding fails")
    print(f"{name}: {score}")

try:
    show_line("Homer")
except TypeError as err:
    print(f"error type: {type(err)}")
    print(f"message:     {err}")

```

Possible output:

```

error type: <class 'TypeError'>
message:     show_line() missing 1 required positional argument: 'score'

```

If the caller supplies too many positional arguments, Python also raises `TypeError` before entering the function body.

```

def show_line(name, score):
    print("this line will not run if binding fails")
    print(f"{name}: {score}")

try:
    show_line("Homer", 42, "D'oh!")
except TypeError as err:
    print(f"error type: {type(err)}")
    print(f"message:     {err}")

```

Possible output:

```
error type: <class 'TypeError'>
message:    show_line() takes 2 positional arguments but 3 were given
```

The important detail is that these failures happen during argument binding. The callee frame is not allowed to begin normal body execution with an incomplete or inconsistent parameter setup.

The standard `inspect` module can show the function signature as a runtime object.

```
import inspect

def show_line(name, score):
    print(f"{name}: {score}")

sig = inspect.signature(show_line)

print(f"signature: {sig}")
print(f"type(sig): {type(sig)}")
print()

selected_names = [
    "parameters",
    "return_annotation",
    "bind",
    "bind_partial",
    "replace",
]

for attr in selected_names:
    print(f"{attr:<20} {attr in dir(sig)}")
```

Possible output:

```
signature: (name, score)
type(sig): <class 'inspect.Signature'>

parameters          True
return_annotation    True
bind                 True
bind_partial         True
replace              True
```

The `Signature` object is not needed for ordinary calls. It is useful for inspection, teaching, decorators, frameworks, and tools that need to reason about how a function can be called.

Its `parameters` attribute is an ordered mapping-like object that describes the parameters.

```
import inspect

def show_line(name, score):
    print(f"{name}: {score}")

sig = inspect.signature(show_line)

print(f"sig.parameters:      {sig.parameters}")
print(f"type(sig.parameters): {type(sig.parameters)}")
print()
```

```

for param_name, param_obj in sig.parameters.items():
    print(f"name:           {param_name}")
    print(f"parameter obj:   {param_obj}")
    print(f"type:           {type(param_obj)}")
    print(f"kind:           {param_obj.kind}")
    print(f"default:         {param_obj.default}")
    print()

```

Possible output:

```

sig.parameters:      OrderedDict([('name', <Parameter "name">), ('score', <Parameter "score">)])
type(sig.parameters): <class 'mappingproxy'>

```

```

name:                name
parameter obj:       name
type:                <class 'inspect.Parameter'>
kind:                POSITIONAL_OR_KEYWORD
default:             <class 'inspect._empty'>

```

```

name:                score
parameter obj:       score
type:                <class 'inspect.Parameter'>
kind:                POSITIONAL_OR_KEYWORD
default:             <class 'inspect._empty'>

```

The exact display of the mapping type may differ by Python version, but the useful idea is stable: the signature can be inspected as structured runtime data.

A parameter object also has inspectable attributes.

```

import inspect

def show_line(name, score):
    print(f"{name}: {score}")

sig = inspect.signature(show_line)
param_obj = sig.parameters["name"]

print(f"type(param_obj): {type(param_obj)}")
print()

selected_names = [
    "name",
    "default",
    "annotation",
    "kind",
    "replace",
]

for attr in selected_names:
    print(f"{attr:<14} {attr in dir(param_obj)}")

print()
print(f"param_obj.name:      {param_obj.name}")
print(f"param_obj.kind:      {param_obj.kind}")
print(f"param_obj.default:    {param_obj.default}")

```

```
print(f"param_obj.annotation: {param_obj.annotation}")
```

Possible output:

```
type(param_obj): <class 'inspect.Parameter'>

name          True
default        True
annotation     True
kind           True
replace        True

param_obj.name:      name
param_obj.kind:      POSITIONAL_OR_KEYWORD
param_obj.default:   <class 'inspect._empty'>
param_obj.annotation: <class 'inspect._empty'>
```

This inspection object makes visible what the source already implies: each parameter has a name, a binding kind, possibly a default value, and possibly an annotation.

Keyword Arguments and Explicit Name-Based Binding

Keyword arguments bind by parameter name instead of by position.

```
def show_line(name, score):
    print(f"name={name!r}")
    print(f"score={score!r}")

show_line(name="Homer", score=42)
```

Possible output:

```
name='Homer'
score=42
```

Because the arguments are named, they can be supplied in a different order.

```
def show_line(name, score):
    print(f"name={name!r}")
    print(f"score={score!r}")

show_line(score=42, name="Homer")
```

Possible output:

```
name='Homer'
score=42
```

The call:

```
show_line(score=42, name="Homer")
```

binds like this:

```
parameter named name -> "Homer"
parameter named score -> 42
```

The order of the keyword arguments in the call is not what determines the binding. The explicit names determine the binding.

Positional and keyword arguments can be mixed, but positional arguments must come first.

```
def show_line(name, score, mood):
    print(f"name={name!r}")
    print(f"score={score!r}")
    print(f"mood={mood!r}")

show_line("Homer", score=42, mood="hungry")
```

Possible output:

```
name='Homer'
score=42
mood='hungry'
```

The first positional argument binds to the first available positional parameter:

```
"Homer" -> name
```

The remaining keyword arguments bind by name:

```
score="42"      is not used here; the actual object is integer 42
score -> 42
mood  -> "hungry"
```

A parameter cannot receive two values.

```
def show_line(name, score):
    print(f"{name}: {score}")

try:
    show_line("Homer", name="Bart")
except TypeError as err:
    print(f"error type: {type(err)}")
    print(f"message:    {err}")
```

Possible output:

```
error type: <class 'TypeError'>
message:    show_line() got multiple values for argument 'name'
```

The positional argument already supplied a value for `name`. The keyword argument tries to supply another value for the same parameter. Python rejects the call before the function body runs.

An unknown keyword is also rejected unless the function explicitly accepts arbitrary keyword arguments, which will be discussed in the later section on flexible parametrization systems.

```
def show_line(name, score):
    print(f"{name}: {score}")

try:
    show_line(name="Homer", score=42, mood="hungry")
except TypeError as err:
    print(f"error type: {type(err)}")
    print(f"message:    {err}")
```

Possible output:

```
error type: <class 'TypeError'>
message:    show_line() got an unexpected keyword argument 'mood'
```

Keyword binding is especially useful when several arguments have the same primitive type and their order would otherwise be easy to confuse.

```
def make_box(left, top, width, height):
    print(f"left={left:>3} top={top:>3} width={width:>3} height={height:>3}")

make_box(left=10, top=20, width=300, height=80)
make_box(width=300, height=80, left=10, top=20)
```

Possible output:

```
left= 10 top= 20 width=300 height= 80
left= 10 top= 20 width=300 height= 80
```

Both calls bind to the same local parameter names.

Keyword arguments are not a different kind of object being passed. They are a different binding route. The same object-reference passing model still applies.

```
def add_fact(lines, fact):
    lines.append(fact)

facts = ["The answer is 42."]

add_fact(lines=facts, fact="Chuck Norris can slam a revolving door.")

print(facts)
```

Possible output:

```
['The answer is 42.', 'Chuck Norris can slam a revolving door.']
```

The keyword `lines=` chooses the parameter. The object referenced by `facts` is still the same list object shared with the function frame.

Default Parameter Values and Definition-Time Evaluation

A parameter may have a default value. If the caller does not provide an argument for that parameter, Python uses the default object stored in the function object.

```
def show_line(name, score=42):
    print(f"{name:<10} score={score:04d}")

show_line("Homer")
show_line("Bart", 7)
show_line("Lisa", score=100)
```

Possible output:

```
Homer      score=0042
Bart       score=0007
Lisa       score=0100
```


The default belongs to the function object. It is visible through `__defaults__`.

```
def show_line(name, score=42):
    print(f"{name:<10} score={score:04d}")

print(f"show_line.__defaults__: {show_line.__defaults__}")
print(f"type(__defaults__):      {type(show_line.__defaults__)}")
```

Possible output:

```
show_line.__defaults__: (42,)
type(__defaults__):      <class 'tuple'>
```

The defaults for positional-or-keyword parameters are stored as a tuple. That tuple can be inspected.

```
def show_line(name, score=42):
    print(f"{name:<10} score={score:04d}")

defaults = show_line.__defaults__

print(f"type(defaults): {type(defaults)}")
print()

selected_names = [
    "count",
    "index",
    "__len__",
    "__getitem__",
    "__contains__",
]

for attr in selected_names:
    print(f"{attr:<14} {attr in dir(defaults)}")

print()
print(f"len(defaults):      {len(defaults)}")
print(f"defaults[0]:        {defaults[0]}")
print(f"42 in defaults:      {42 in defaults}")
```

Possible output:

```
type(defaults): <class 'tuple'>

count          True
index          True
__len__        True
__getitem__    True
__contains__   True

len(defaults):      1
defaults[0]:        42
42 in defaults:     True
```

Default values are evaluated when the `def` statement executes, not each time the function is called.

```

default_score = 42

def show_line(name, score=default_score):
    print(f"{name:<10} score={score:04d}")

default_score = 999

show_line("Homer")
show_line("Bart", 7)

print()
print(f"default_score now:      {default_score}")
print(f"show_line.__defaults__: {show_line.__defaults__}")

```

Possible output:

```

Homer      score=0042
Bart       score=0007

default_score now:      999
show_line.__defaults__: (42,)

```

The name `default_score` was used when the function object was created. At that time, it referred to the integer object 42. The function stored that object as its default. Later rebinding `default_score` to 999 does not rewrite the function object's stored default.

This can be made even more visible with a function call used as a default expression.

```

def make_default_tag():
    print("make_default_tag() is running")
    return "D'oh!"

def show_tag(text=make_default_tag()):
    print(f"text={text!r}")

print("after function definitions")
print()

show_tag()
show_tag()
show_tag("No soup for you!")

```

Possible output:

```

make_default_tag() is running
after function definitions

text="D'oh!"
text="D'oh!"
text='No soup for you!'

```

The function `make_default_tag()` ran once, while Python was executing the `def show_tag(...)` statement. It did not run again for every call to `show_tag()`.

So the signature:

```

def show_tag(text=make_default_tag()):

```

does not mean:

call `make_default_tag()` every time `text` is omitted

It means:

execute `make_default_tag()` now
store its returned object as the default value for `text`

This fact becomes dangerous when the default object is mutable. That specific trap will be studied later in the section on architectural pitfalls of argument evaluation. For now, the rule is enough:

Default values are objects stored in the function object at definition time.

Keyword-only defaults are stored separately in `__kwdefaults__`.

```
def show_line(name, *, score=42, mood="calm"):
    print(f"{name:<10} score={score:04d} mood={mood}")

print(f"__defaults__: {show_line.__defaults__}")
print(f"__kwdefaults__: {show_line.__kwdefaults__}")
print(f"type: {type(show_line.__kwdefaults__)}")
```

Possible output:

```
__defaults__: None
__kwdefaults__: {'score': 42, 'mood': 'calm'}
type: <class 'dict'>
```

The dictionary stores default values for parameters that must be supplied by keyword.
The dictionary can be inspected.

```
def show_line(name, *, score=42, mood="calm"):
    print(f"{name:<10} score={score:04d} mood={mood}")

kw_defaults = show_line.__kwdefaults__

print(f"type(kw_defaults): {type(kw_defaults)}")
print()

selected_names = [
    "keys",
    "values",
    "items",
    "get",
    "__getitem__",
    "__setitem__",
]

for attr in selected_names:
    print(f"{attr:<14} {attr in dir(kw_defaults)}")

print()
print(f"keys: {list(kw_defaults.keys())}")
print(f"score default: {kw_defaults.get('score')}")
print(f"items: {list(kw_defaults.items())}")
```

Possible output:

```
type(kw_defaults): <class 'dict'>

keys          True
values        True
items         True
get           True
__getitem__   True
__setitem__   True

keys:          ['score', 'mood']
score default: 42
items:         [('score', 42), ('mood', 'calm')]
```

This again shows the same object model: the function object stores references to default objects.

Positional-Only Parameters Using /

A slash in the function signature marks all parameters before it as positional-only.

```
def divide(a, b, /):
    return a / b

print(divide(84, 2))
```

Possible output:

```
42.0
```

The parameters `a` and `b` can be supplied positionally, but not by keyword.

```
def divide(a, b, /):
    return a / b

try:
    print(divide(a=84, b=2))
except TypeError as err:
    print(f"error type: {type(err)}")
    print(f"message:    {err}")
```

Possible output:

```
error type: <class 'TypeError'>
message:    divide() got some positional-only arguments passed as keyword arguments: 'a, b'
```

The exact wording of the error message may differ across Python versions. The rule is stable: parameters before `/` cannot be bound by keyword.

This is visible in the inspected signature.

```
import inspect

def divide(a, b, /):
    return a / b

sig = inspect.signature(divide)
```

```

print(f"signature: {sig}")
print()

for name, param in sig.parameters.items():
    print(f"{name:<6} kind={param.kind}")

```

Possible output:

```

signature: (a, b, /)

a      kind=POSITIONAL_ONLY
b      kind=POSITIONAL_ONLY

```

Positional-only parameters are useful when the parameter names are implementation details or when the function is trying to match built-in function behavior.

For example, many built-in functions do not accept certain arguments by keyword.

```

print(len("D'oh!"))

try:
    print(len(obj="D'oh!"))
except TypeError as err:
    print(f"error type: {type(err)}")
    print(f"message:    {err}")

```

Possible output:

```

5
error type: <class 'TypeError'>
message:    len() takes no keyword arguments

```

A user-defined function can express similar call restrictions with `/`.

Parameters after the slash are not positional-only unless another marker restricts them.

```

def scale(value, /, factor=2):
    return value * factor

print(scale(21))
print(scale(21, 2))
print(scale(21, factor=2))

```

Possible output:

```

42
42
42

```

Here `value` is positional-only. `factor` may be supplied positionally or by keyword.

```

def scale(value, /, factor=2):
    return value * factor

try:
    print(scale(value=21, factor=2))
except TypeError as err:
    print(f"error type: {type(err)}")
    print(f"message:    {err}")

```

Possible output:

```
error type: <class 'TypeError'>
message:    scale() got some positional-only arguments passed as keyword arguments: 'value'
```

The slash affects binding rules only. It does not change what object is passed. It only says how the object may be connected to the parameter name.

Keyword-Only Parameters Using *

A bare star in the function signature marks all following parameters as keyword-only.

```
def make_card(name, *, score=42, mood="calm"):
    print(f"{name:<10} score={score:04d} mood={mood}")

make_card("Homer")
make_card("Lisa", score=100)
make_card("Bart", mood="restless", score=7)
```

Possible output:

```
Homer      score=0042 mood=calm
Lisa       score=0100 mood=calm
Bart       score=0007 mood=restless
```

The parameter `name` can be positional. The parameters after `*` must be supplied by keyword if the caller supplies them at all.

This call fails:

```
def make_card(name, *, score=42, mood="calm"):
    print(f"{name:<10} score={score:04d} mood={mood}")

try:
    make_card("Homer", 100, "hungry")
except TypeError as err:
    print(f"error type: {type(err)}")
    print(f"message:    {err}")
```

Possible output:

```
error type: <class 'TypeError'>
message:    make_card() takes 1 positional argument but 3 were given
```

Python sees only one parameter before the bare star that can accept positional arguments. The later parameters are keyword-only.

Keyword-only parameters are useful when positional order would be unclear.

```
def save_report(title, *, overwrite=False, verbose=False):
    print(f"title={title!r}")
    print(f"overwrite={overwrite}")
    print(f"verbose={verbose}")

save_report("annual report", overwrite=True, verbose=True)
```

Possible output:

```
title='annual report'
overwrite=True
verbose=True
```

The call is clearer than this would be:

```
save_report("annual report", True, True)
```

With booleans especially, positional arguments can become unreadable. The keyword-only marker forces the call site to say what the flags mean.

Keyword-only parameters can be required by omitting a default value.

```
def connect(host, *, port):
    print(f"host={host!r}")
    print(f"port={port!r}")

connect("localhost", port=8080)
```

Possible output:

```
host='localhost'
port=8080
```

The caller must use the keyword.

```
def connect(host, *, port):
    print(f"host={host!r}")
    print(f"port={port!r}")

try:
    connect("localhost", 8080)
except TypeError as err:
    print(f"error type: {type(err)}")
    print(f"message:    {err}")
```

Possible output:

```
error type: <class 'TypeError'>
message:    connect() takes 1 positional argument but 2 were given
```

If the required keyword-only argument is missing, that is also a binding error.

```
def connect(host, *, port):
    print(f"host={host!r}")
    print(f"port={port!r}")

try:
    connect("localhost")
except TypeError as err:
    print(f"error type: {type(err)}")
    print(f"message:    {err}")
```

Possible output:

```
error type: <class 'TypeError'>
message:    connect() missing 1 required keyword-only argument: 'port'
```

The inspected signature shows the keyword-only kind.

```
import inspect

def connect(host, *, port):
    print(f"host={host!r}")
    print(f"port={port!r}")

sig = inspect.signature(connect)

print(f"signature: {sig}")
print()

for name, param in sig.parameters.items():
    print(f"{name:<8} kind={param.kind} default={param.default}")
```

Possible output:

```
signature: (host, *, port)

host      kind=POSITIONAL_OR_KEYWORD default=<class 'inspect._empty'>
port      kind=KEYWORD_ONLY default=<class 'inspect._empty'>
```

A full signature may combine positional-only, normal positional-or-keyword, and keyword-only parameters.

```
def build_line(name, /, score=42, *, mood="calm"):
    print(f"{name:<10} score={score:04d} mood={mood}")

build_line("Homer")
build_line("Lisa", 100)
build_line("Bart", score=7, mood="restless")
```

Possible output:

```
Homer      score=0042 mood=calm
Lisa       score=0100 mood=calm
Bart       score=0007 mood=restless
```

The structure is:

```
name:
    positional-only

score:
    positional or keyword

mood:
    keyword-only
```

Inspection makes that structure explicit.

```
import inspect

def build_line(name, /, score=42, *, mood="calm"):
    print(f"{name:<10} score={score:04d} mood={mood}")
```



```

sig = inspect.signature(build_line)

print(f"signature: {sig}")
print()

for name, param in sig.parameters.items():
    print(f"{name:<8} kind={param.kind} default={param.default}")

```

Possible output:

```

signature: (name, /, score=42, *, mood='calm')

name      kind=POSITIONAL_ONLY default=<class 'inspect._empty'>
score     kind=POSITIONAL_OR_KEYWORD default=42
mood      kind=KEYWORD_ONLY default=calm

```

The markers / and * are not parameters. They are separators that change the binding rules for the real parameters around them.

Function Annotations: Metadata for Tools, Not Automatic Runtime Type Enforcement

Function annotations attach metadata to parameters and to the return value.

```

def build_line(name: str, score: int = 42) -> str:
    return f"{name:<10} score={score:04d}"

text = build_line("Homer")

print(text)
print()
print(f"annotations: {build_line.__annotations__}")
print(f"type:         {type(build_line.__annotations__)}")

```

Possible output:

```

Homer      score=0042

annotations: {'name': <class 'str'>, 'score': <class 'int'>, 'return': <class 'str'>}
type:       <class 'dict'>

```

The annotations are stored in the function object's `__annotations__` dictionary. That dictionary is ordinary dictionary-like data.

```

def build_line(name: str, score: int = 42) -> str:
    return f"{name:<10} score={score:04d}"

ann = build_line.__annotations__

print(f"type(ann): {type(ann)}")
print()

selected_names = [
    "keys",
    "values",

```

```

    "items",
    "get",
    "__getitem__",
    "__contains__",
]

for attr in selected_names:
    print(f"{attr:<14} {attr in dir(ann)}")

print()
print(f"keys:           {list(ann.keys())}")
print(f"ann['name']:      {ann['name']}")
print(f"ann.get('return'): {ann.get('return')}")

```

Possible output:

```

type(ann): <class 'dict'>

keys           True
values         True
items          True
get            True
__getitem__    True
__contains__   True

keys:          ['name', 'score', 'return']
ann['name']:    <class 'str'>
ann.get('return'): <class 'str'>

```

Annotations do not automatically enforce types at runtime.

```

def echo_count(value: int) -> str:
    return f"value={value!r}"

print(echo_count(42))
print(echo_count("No soup for you!"))

```

Possible output:

```

value=42
value='No soup for you!'

```

The function annotation says that `value` is intended to be an `int`. Python still allows the string argument because no automatic runtime type check is performed.

The function body may later fail if it performs an operation that the actual object does not support.

```

def double_count(value: int) -> int:
    return value + value

print(double_count(21))
print(double_count("Ni!"))

```

Possible output:

```

42
Ni!Ni!

```

Even here, no type error occurs. String addition with another string is valid concatenation. The annotation did not force integer addition.

A clearer failure appears when the function uses an integer-specific operation.

```
def show_bits(value: int) -> int:
    return value.bit_length()

print(show_bits(42))

try:
    print(show_bits("D'oh!"))
except AttributeError as err:
    print(f"error type: {type(err)}")
    print(f"message:    {err}")
```

Possible output:

```
6
error type: <class 'AttributeError'>
message:    'str' object has no attribute 'bit_length'
```

The failure occurs because the string object does not have the method used by the function body. The failure does not occur because Python enforced the annotation.

The inspected signature also stores annotation data.

```
import inspect

def build_line(name: str, score: int = 42) -> str:
    return f"{name:<10} score={score:04d}"

sig = inspect.signature(build_line)

print(f"signature: {sig}")
print(f"return annotation: {sig.return_annotation}")
print()

for name, param in sig.parameters.items():
    print(f"{name:<8} annotation={param.annotation} default={param.default}")
```

Possible output:

```
signature: (name: str, score: int = 42) -> str
return annotation: <class 'str'>

name      annotation=<class 'str'> default=<class 'inspect._empty'>
score     annotation=<class 'int'> default=42
```

Annotations are therefore part of the function's inspectable interface. They are useful for documentation, static analysis, editors, linters, type checkers, validation libraries, and framework code. They are not automatic guards placed at the function entrance.

The runtime rule is:

A function signature controls argument binding. Annotations describe intended meaning for tools and readers. They do not, by themselves, change the runtime object-passing model.

8.3 Flexible Parametrization Systems

Not every function call has a fixed number of inputs. Some functions must accept an open-ended sequence of positional values, an open-ended set of named options, or a mixture of both. Python handles this through variadic parameters and argument unpacking: `*args`, `**kwargs`, `*iterable`, and `**mapping`.

These mechanisms do not change the basic call model. Arguments are still evaluated into object references, a new frame is still created, and parameter names are still bound inside that frame. What changes is the shape of the binding operation. Positional overflow can be packed into a tuple. Keyword overflow can be packed into a dictionary. Existing sequences and mappings can also be expanded across the call boundary.

This section also covers one of the most important traps in Python function design: default argument expressions are evaluated when the function object is created, not each time the function is called. That single rule explains both the power and the danger of flexible signatures, especially when mutable default objects are involved.

8.3.1 Variadic Positional Parameters

A normal positional parameter receives one argument object. A variadic positional parameter receives any remaining positional argument objects and packs them into one tuple.

The syntax uses a single star in the function definition:

```
def func(*args):
    ...
```

The name `args` is only a convention. The star is the important syntax. Python programmers usually use `args` because it clearly means “positional arguments collected here.”

This mechanism does not create a special argument object. It creates a normal tuple.

Argument Packing Mechanics: Collecting Positional Overflow into `*args`

A function can collect an arbitrary number of positional arguments using `*args`.

```
def show_args(*args):
    print(f"args:          {args}")
    print(f"type(args): {type(args)}")
```

```
show_args("Homer", "Bart", 42)
```

Possible output:

```
args:          ('Homer', 'Bart', 42)
type(args): <class 'tuple'>
```

The call supplied three positional arguments. The function definition had one variadic positional parameter. Python packed the three argument objects into one tuple and bound the local name `args` to that tuple.

The binding shape is:

```
show_args("Homer", "Bart", 42)
```

```
inside function frame:
    args -> ("Homer", "Bart", 42)
```

The tuple is a normal tuple object.

```
def show_args(*args):
    print(f"type(args): {type(args)}")
    print()

    selected_names = [
        "count",
        "index",
        "__len__",
        "__getitem__",
        "__contains__",
    ]

    for name in selected_names:
        print(f"{name:<14} {name in dir(args)}")

    print()
    print(f"len(args):          {len(args)}")
    print(f"args[0]:                {args[0]!r}")
    print(f"args.count(42):          {args.count(42)}")
    print(f"42 in args:              {42 in args}")

show_args("Homer", "Bart", 42)
```

Possible output:

```
type(args): <class 'tuple'>
```

```
count          True
index          True
__len__        True
__getitem__    True
__contains__   True
```

```
len(args):      3
args[0]:        'Homer'
args.count(42): 1
42 in args:     True
```

The tuple is immutable. The function cannot append directly to `args` because tuples do not have `append()`.

```
def show_args(*args):
    print(f"args: {args}")
    print(f"'append' in dir(args): {'append' in dir(args)}")

    try:
        args.append("D'oh!")
    except AttributeError as err:
        print(f"error type: {type(err)}")
        print(f"message:      {err}")

show_args("Homer", 42)
```

Possible output:

```
args: ('Homer', 42)
'append' in dir(args): False
error type: <class 'AttributeError'>
message:    'tuple' object has no attribute 'append'
```

The tuple itself cannot be changed in place, but the objects inside the tuple may still be mutable. The tuple stores references to the argument objects.

```
def add_quote(*args):
    print(f"args:      {args}")
    print(f"type(args): {type(args)}")

    first_item = args[0]
    print(f"type(first_item): {type(first_item)}")

    first_item.append("Nobody expects the Spanish Inquisition!")

quotes = ["D'oh!", "No soup for you!"]

print("before call")
print(f"quotes:      {quotes}")
print(f"id(quotes): {id(quotes)}")
print()

add_quote(quotes)

print()
print("after call")
print(f"quotes:      {quotes}")
print(f"id(quotes): {id(quotes)}")
```

Possible output:

```
before call
quotes:      ["D'oh!", 'No soup for you!']
id(quotes): 2276834282880

args:        (["D'oh!", 'No soup for you!'],)
type(args): <class 'tuple'>
type(first_item): <class 'list'>

after call
quotes:      ["D'oh!", 'No soup for you!', 'Nobody expects the Spanish Inquisition!']
id(quotes): 2276834282880
```

The tuple `args` was not mutated. The list object inside the tuple was mutated. This follows the same rule from the previous section: function calls share object references. The `*args` tuple is only the container used to collect the remaining positional references.

A variadic positional parameter can appear after ordinary positional parameters.

```
def show_report(title, *lines):
    print(f"title: {title}")
    print(f"lines: {lines}")
    print(f"type(lines): {type(lines)}")
```

```
show_report(
    "Important facts",
    "The answer is 42.",
    "Chuck Norris can divide by zero.",
    "It is only a model.",
)
```

Possible output:

```
title: Important facts
lines: ('The answer is 42.', 'Chuck Norris can divide by zero.', 'It is only a model.')
type(lines): <class 'tuple'>
```

The first positional argument is bound normally:

```
title -> "Important facts"
```

The remaining positional arguments are packed into the variadic tuple:

```
lines -> (
    "The answer is 42.",
    "Chuck Norris can divide by zero.",
    "It is only a model."
)
```

If there are no remaining positional arguments, Python creates an empty tuple.

```
def show_report(title, *lines):
    print(f"title:      {title!r}")
    print(f"lines:      {lines}")
    print(f"type(lines): {type(lines)}")
    print(f"len(lines):  {len(lines)}")
```

```
show_report("Empty report")
```

Possible output:

```
title:      'Empty report'
lines:      ()
type(lines): <class 'tuple'>
len(lines): 0
```

An empty `*args` value is not `None`. It is an empty tuple.
The parameter name does not have to be `args`.

```
def show_people(*people):
    print(f"people:      {people}")
    print(f"type(people): {type(people)}")
```

```
show_people("Jerry", "George", "Elaine", "Kramer")
```

Possible output:

```
people:      ('Jerry', 'George', 'Elaine', 'Kramer')
type(people): <class 'tuple'>
```

The star makes the parameter variadic. The name only chooses the local name inside the function frame.

The standard `inspect` module shows this parameter kind explicitly.

```
import inspect

def show_report(title, *lines):
    print(title, lines)

sig = inspect.signature(show_report)

print(f"signature: {sig}")
print()

for name, param in sig.parameters.items():
    print(f"{name:<8} kind={param.kind}")
```

Possible output:

```
signature: (title, *lines)

title    kind=POSITIONAL_OR_KEYWORD
lines    kind=VAR_POSITIONAL
```

The `VAR_POSITIONAL` kind means that the parameter collects positional overflow.

A common use is to process all remaining values in a loop.

```
def print_total(label, *values):
    total = 0

    print(f"label: {label}")

    for value in values:
        print(f"adding {value}")
        total = total + value

    print(f"total: {total}")

print_total("scores", 10, 20, 12)
```

Possible output:

```
label: scores
adding 10
adding 20
adding 12
total: 42
```

Here, `values` is the tuple `(10, 20, 12)`. The loop iterates over that tuple.

This is still ordinary tuple iteration. There is no hidden magic after the packing step.

Argument Unpacking Operations: Expanding Sequences Across Function Call Boundaries with `*`

The single star has a second use at the call site. In a function definition, `*args` packs positional overflow into a tuple. In a function call, `*some_iterable` unpacks an iterable object into separate positional arguments.

These are opposite directions:


```

definition side:
    def func(*args):
        pack many positional arguments into one tuple

call side:
    func(*items)
        unpack one iterable into many positional arguments

```

Start with a normal function.

```

def show_line(name, score, mood):
    print(f"name={name!r}")
    print(f"score={score!r}")
    print(f"mood={mood!r}")

```

```
data = ["Homer", 42, "hungry"]
```

```
show_line(*data)
```

Possible output:

```

name='Homer'
score=42
mood='hungry'

```

The call:

```
show_line(*data)
```

means:

```

take elements from data
send them as separate positional arguments

```

So this:

```
show_line(*data)
```

is equivalent to:

```
show_line(data[0], data[1], data[2])
```

for this specific three-element list.
The list object can be inspected.

```
data = ["Homer", 42, "hungry"]
```

```

print(f"type(data): {type(data)}")
print()

```

```

selected_names = [
    "append",
    "extend",
    "__len__",
    "__getitem__",
    "__iter__",

```

```

]

for name in selected_names:
    print(f"{name:<14} {name in dir(data)}")

print()
print(f"len(data):          {len(data)}")
print(f"data[0]:           {data[0]!r}")
print(f"list iteration:    {[item for item in data]}")

```

Possible output:

```

type(data): <class 'list'>

append      True
extend      True
__len__     True
__getitem__ True
__iter__    True

len(data):   3
data[0]:     'Homer'
list iteration: ['Homer', 42, 'hungry']

```

The unpacking operation uses iteration. The object must be iterable.
A tuple can also be unpacked.

```

def show_line(name, score, mood):
    print(f"{name:<10} score={score:04d} mood={mood}")

data = ("Lisa", 100, "focused")

print(f"type(data): {type(data)}")
show_line(*data)

```

Possible output:

```

type(data): <class 'tuple'>
Lisa       score=0100 mood=focused

```

A range can also be unpacked because it is iterable.

```

def show_three(a, b, c):
    print(f"a={a}, b={b}, c={c}")

nums = range(1, 4)

print(f"nums:          {nums}")
print(f"type(nums):    {type(nums)}")
print()

selected_names = [
    "__len__",
    "__getitem__",
    "__iter__",

```

```

    "count",
    "index",
]

for name in selected_names:
    print(f"{name:<14} {name in dir(nums)}")

print()
show_three(*nums)

```

Possible output:

```

nums:          range(1, 4)
type(nums):    <class 'range'>

__len__        True
__getitem__    True
__iter__       True
count          True
index          True

a=1, b=2, c=3

```

The `range` object is not a list, but it can produce values one by one. The star operator can expand those produced values into positional arguments.

A string can also be unpacked. This is sometimes useful, but often surprising.

```

def show_three(a, b, c):
    print(f"a={a!r}")
    print(f"b={b!r}")
    print(f"c={c!r}")

text = "Ni!"

print(f"type(text): {type(text)}")
show_three(*text)

```

Possible output:

```

type(text): <class 'str'>
a='N'
b='i'
c='!'

```

The string is iterable over characters, so `*text` expands it into separate character arguments.

The number of unpacked items must match the receiving signature unless the receiving function has a variadic positional parameter.

```

def show_three(a, b, c):
    print(f"a={a}, b={b}, c={c}")

short_data = [1, 2]
long_data = [1, 2, 3, 4]

try:

```

```

    show_three(*short_data)
except TypeError as err:
    print("short_data failed")
    print(f"error type: {type(err)}")
    print(f"message:    {err}")
    print()

try:
    show_three(*long_data)
except TypeError as err:
    print("long_data failed")
    print(f"error type: {type(err)}")
    print(f"message:    {err}")

```

Possible output:

```

short_data failed
error type: <class 'TypeError'>
message:    show_three() missing 1 required positional argument: 'c'

long_data failed
error type: <class 'TypeError'>
message:    show_three() takes 3 positional arguments but 4 were given

```

The unpacking operation expands the iterable first. Then normal argument binding rules apply. A variadic receiving function can accept any number of unpacked positional arguments.

```

def show_all(*items):
    print(f"items:        {items}")
    print(f"type(items): {type(items)}")
    print(f"len(items):  {len(items)}")

data = ["Arthur", "Patsy", "Robin", "Galahad"]

show_all(*data)

```

Possible output:

```

items:        ('Arthur', 'Patsy', 'Robin', 'Galahad')
type(items):  <class 'tuple'>
len(items):   4

```

This call performs two opposite operations in sequence:

```

show_all(*data)

call side:
    unpack data into separate positional arguments

definition side:
    pack those positional arguments into the local tuple items

```

For this example, the overall effect looks similar to converting the list into a tuple. But conceptually, two separate call-boundary operations happened.

Unpacking does not copy the contained objects. It passes references to the same objects.

```
def mutate_first(*items):
    first = items[0]
    first.append("It is only a model.")

quotes = ["D'oh!"]
data = [quotes]

print("before")
print(f"quotes:      {quotes}")
print(f"id(quotes): {id(quotes)}")
print()

mutate_first(*data)

print("after")
print(f"quotes:      {quotes}")
print(f"id(quotes): {id(quotes)}")
```

Possible output:

```
before
quotes:      ["D'oh!"]
id(quotes):  2276834282880

after
quotes:      ["D'oh!", 'It is only a model.']
id(quotes):  2276834282880
```

The list `data` was unpacked. Its first element was the list object also referenced by `quotes`. The function received that same inner list object and mutated it.

The star can also be mixed with normal positional arguments.

```
def show_line(name, score, mood):
    print(f"name={name!r}")
    print(f"score={score!r}")
    print(f"mood={mood!r}")
```

```
tail = [42, "hungry"]

show_line("Homer", *tail)
```

Possible output:

```
name='Homer'
score=42
mood='hungry'
```

The call expands to:

```
show_line("Homer", 42, "hungry")
```

Multiple starred iterables can appear in one call.

```
def show_all(*items):
    for index, item in enumerate(items, start=1):
```

```

        print(f"{index:>2}. {item}")

first = ["Homer", "Bart"]
second = ["Lisa", "Marge"]
third = ["Maggie", 42]

show_all(*first, *second, *third)

```

Possible output:

```

1. Homer
2. Bart
3. Lisa
4. Marge
5. Maggie
6. 42

```

Python expands each starred iterable into positional arguments from left to right.
The argument stream seen by `show_all()` is:

```
"Homer", "Bart", "Lisa", "Marge", "Maggie", 42
```

The receiving `*items` parameter then packs that stream into one tuple.
A non-iterable object cannot be unpacked with `*`.

```

def show_all(*items):
    print(items)

value = 42

try:
    show_all(*value)
except TypeError as err:
    print(f"error type: {type(err)}")
    print(f"message:      {err}")

```

Possible output:

```

error type: <class 'TypeError'>
message:    __main__.show_all() argument after * must be an iterable, not int

```

The exact module name in the error message may differ. The rule is stable: call-side `*` requires an iterable object.

The compact model is:

```

def f(*args):
    star in definition packs positional overflow into a tuple

f(*items):
    star in call unpacks an iterable into positional arguments

```

The two uses are mirror images. One collects many arguments into one tuple. The other expands one iterable into many arguments.

8.3.2 Variadic Keyword Parameters

A variadic keyword parameter collects keyword arguments that were not bound to ordinary named parameters. The syntax uses two stars in the function definition:

```
def func(**kwargs):
    ...
```

The name `kwargs` is only a convention. The two stars are the important syntax. Python programmers usually use `kwargs` because it means “keyword arguments collected here.”

The collected object is a normal dictionary.

Argument Packing Mechanics: Collecting Keyword Overflow into `**kwargs`

A function can collect arbitrary keyword arguments using `**kwargs`.

```
def show_kwargs(**kwargs):
    print(f"kwargs:      {kwargs}")
    print(f"type(kwargs): {type(kwargs)}")

show_kwargs(name="Homer", score=42, mood="hungry")
```

Possible output:

```
kwargs:      {'name': 'Homer', 'score': 42, 'mood': 'hungry'}
type(kwargs): <class 'dict'>
```

The call supplied three keyword arguments. The function definition had one variadic keyword parameter. Python packed the keyword names and their argument objects into one dictionary.

The binding shape is:

```
show_kwargs(name="Homer", score=42, mood="hungry")

inside function frame:
    kwargs -> {
        "name": "Homer",
        "score": 42,
        "mood": "hungry"
    }
```

The dictionary is a normal dictionary object.

```
def show_kwargs(**kwargs):
    print(f"type(kwargs): {type(kwargs)}")
    print()

    selected_names = [
        "keys",
        "values",
        "items",
        "get",
        "update",
        "__getitem__",
        "__setitem__",
    ]
```

```

for name in selected_names:
    print(f"{name:<14} {name in dir(kwargs)}")

print()
print(f"keys:           {list(kwargs.keys())}")
print(f"items:          {list(kwargs.items())}")
print(f"kwargs.get('score'): {kwargs.get('score')}")

show_kwargs(name="Homer", score=42, mood="hungry")

```

Possible output:

```

type(kwargs): <class 'dict'>

keys           True
values         True
items          True
get            True
update         True
__getitem__    True
__setitem__    True

keys:          ['name', 'score', 'mood']
items:          [('name', 'Homer'), ('score', 42), ('mood', 'hungry')]
kwargs.get('score'): 42

```

Unlike `*args`, which creates a tuple, `**kwargs` creates a dictionary. The keys are the keyword names. The values are the objects supplied by the caller.

A function can combine ordinary parameters with `**kwargs`.

```

def show_report(title, **options):
    print(f"title: {title!r}")
    print(f"options: {options}")
    print(f"type(options): {type(options)}")

show_report(
    "Springfield report",
    score=42,
    mood="calm",
    quote="D'oh!",
)

```

Possible output:

```

title: 'Springfield report'
options: {'score': 42, 'mood': 'calm', 'quote': "D'oh!"}
type(options): <class 'dict'>

```

The keyword `title` is bound to the ordinary parameter `title`. The other keyword arguments are collected into `options`.

The binding shape is:

```

title -> "Springfield report"
options -> {
    "score": 42,

```



```

    "mood": "calm",
    "quote": "D'oh!"
}

```

If there is no keyword overflow, Python creates an empty dictionary.

```

def show_report(title, **options):
    print(f"title:          {title!r}")
    print(f"options:         {options}")
    print(f"type(options): {type(options)}")
    print(f"len(options):  {len(options)}")

show_report("Empty report")

```

Possible output:

```

title:          'Empty report'
options:        {}
type(options):  <class 'dict'>
len(options):   0

```

An empty `**kwargs` value is not `None`. It is an empty dictionary.
The parameter name does not have to be `kwargs`.

```

def show_settings(**settings):
    print(f"settings:         {settings}")
    print(f"type(settings): {type(settings)}")

show_settings(theme="light", font_size=14, answer=42)

```

Possible output:

```

settings:        {'theme': 'light', 'font_size': 14, 'answer': 42}
type(settings):  <class 'dict'>

```

The two stars make the parameter variadic. The name only chooses the local name inside the function frame.

The standard `inspect` module shows this parameter kind explicitly.

```

import inspect

def show_report(title, **options):
    print(title, options)

sig = inspect.signature(show_report)

print(f"signature: {sig}")
print()

for name, param in sig.parameters.items():
    print(f"{name:<8} kind={param.kind}")

```

Possible output:

```

signature: (title, **options)

title      kind=POSITIONAL_OR_KEYWORD
options    kind=VAR_KEYWORD

```

The `VAR_KEYWORD` kind means that the parameter collects keyword overflow.
A function can have both `*args` and `**kwargs`.

```
def show_call(*args, **kwargs):
    print(f"args:          {args}")
    print(f"type(args): {type(args)}")
    print()
    print(f"kwargs:          {kwargs}")
    print(f"type(kwargs): {type(kwargs)}")

show_call("Homer", "Bart", score=42, mood="restless")
```

Possible output:

```
args:          ('Homer', 'Bart')
type(args): <class 'tuple'>

kwargs:        {'score': 42, 'mood': 'restless'}
type(kwargs): <class 'dict'>
```

The positional overflow goes into the tuple. The keyword overflow goes into the dictionary.
The binding shape is:

```
args    -> ("Homer", "Bart")
kwargs -> {
    "score": 42,
    "mood": "restless"
}
```

The order in the function definition matters. A typical flexible function has this shape:

```
def func(required, *args, named=default, **kwargs):
    ...
```

The meaning is:

- ordinary parameters receive explicitly declared arguments;
- `*args` receives extra positional arguments;
- keyword-only parameters receive explicitly named arguments after `*args`;
- `**kwargs` receives extra keyword arguments.

Here is a small inspection example.

```
import inspect

def demo(a, b=42, *args, mode="calm", **kwargs):
    print(a, b, args, mode, kwargs)

sig = inspect.signature(demo)

print(f"signature: {sig}")
print()

for name, param in sig.parameters.items():
    print(f"{name:<8} kind={param.kind} default={param.default}")
```

Possible output:

```
signature: (a, b=42, *args, mode='calm', **kwargs)

a      kind=POSITIONAL_OR_KEYWORD default=<class 'inspect._empty'>
b      kind=POSITIONAL_OR_KEYWORD default=42
args   kind=VAR_POSITIONAL default=<class 'inspect._empty'>
mode   kind=KEYWORD_ONLY default=calm
kwargs kind=VAR_KEYWORD default=<class 'inspect._empty'>
```

This signature says exactly how the entrance into the function frame is constructed.

Argument Unpacking Operations: Expanding Mappings Across Function Call Boundaries with **

The double star has a second use at the call site. In a function definition, ****kwargs** packs keyword overflow into a dictionary. In a function call, ****some_mapping** unpacks a mapping into separate keyword arguments.

These are opposite directions:

```
definition side:
    def func(**kwargs):
        pack keyword overflow into one dictionary

call side:
    func(**mapping)
        unpack one mapping into many keyword arguments
```

Start with a normal function.

```
def show_line(name, score, mood):
    print(f"name={name!r}")
    print(f"score={score!r}")
    print(f"mood={mood!r}")
```

```
data = {
    "name": "Homer",
    "score": 42,
    "mood": "hungry",
}
```

```
show_line(**data)
```

Possible output:

```
name='Homer'
score=42
mood='hungry'
```

The call:

```
show_line(**data)
```

means:

```
take key-value pairs from data
send them as separate keyword arguments
```

So this:

```
show_line(**data)
```

is equivalent to:

```
show_line(name=data["name"], score=data["score"], mood=data["mood"])
```

for this specific dictionary.

The dictionary object can be inspected before unpacking.

```
data = {
    "name": "Homer",
    "score": 42,
    "mood": "hungry",
}

print(f"type(data): {type(data)}")
print()

selected_names = [
    "keys",
    "values",
    "items",
    "get",
    "__iter__",
    "__getitem__",
]

for name in selected_names:
    print(f"{name:<14} {name in dir(data)}")

print()
print(f"keys:      {list(data.keys())}")
print(f"items:     {list(data.items())}")
print(f"name:      {data['name']!r}")
```

Possible output:

```
type(data): <class 'dict'>

keys          True
values        True
items         True
get           True
__iter__      True
__getitem__   True

keys:         ['name', 'score', 'mood']
items:        [('name', 'Homer'), ('score', 42), ('mood', 'hungry')]
name:         'Homer'
```

The unpacking operation uses the mapping keys as keyword names.

If a required key is missing, normal argument binding fails.

```
def show_line(name, score, mood):
    print(f"name={name!r}")
    print(f"score={score!r}")
    print(f"mood={mood!r}")

data = {
    "name": "Homer",
    "score": 42,
}

try:
    show_line(**data)
except TypeError as err:
    print(f"error type: {type(err)}")
    print(f"message: {err}")
```

Possible output:

```
error type: <class 'TypeError'>
message: show_line() missing 1 required positional argument: 'mood'
```

The mapping did not provide a value for mood.

If the mapping contains an unknown keyword and the function has no ****kwargs** receiver, binding also fails.

```
def show_line(name, score):
    print(f"name={name!r}")
    print(f"score={score!r}")

data = {
    "name": "Homer",
    "score": 42,
    "mood": "hungry",
}

try:
    show_line(**data)
except TypeError as err:
    print(f"error type: {type(err)}")
    print(f"message: {err}")
```

Possible output:

```
error type: <class 'TypeError'>
message: show_line() got an unexpected keyword argument 'mood'
```

If the function has a ****kwargs** receiver, the unknown keyword can be collected.

```
def show_line(name, score, **extra):
    print(f"name={name!r}")
    print(f"score={score!r}")
    print(f"extra={extra}")

data = {
    "name": "Homer",
```

```

    "score": 42,
    "mood": "hungry",
}

```

```
show_line(**data)
```

Possible output:

```

name='Homer'
score=42
extra={'mood': 'hungry'}

```

The keyword names from the mapping first try to bind to declared parameters. The leftover keyword names are packed into the variadic keyword dictionary.

The keys used by `**` must be strings.

```

def show_all(**kwargs):
    print(kwargs)

data = {
    "name": "Homer",
    42: "answer",
}

try:
    show_all(**data)
except TypeError as err:
    print(f"error type: {type(err)}")
    print(f"message:    {err}")

```

Possible output:

```

error type: <class 'TypeError'>
message:    keywords must be strings

```

The double-star call syntax creates keyword arguments, and keyword names must be strings.

Several mappings can be unpacked in one call, but duplicate keyword names are not allowed in a function call.

```

def show_all(**kwargs):
    print(kwargs)

first = {"name": "Homer", "score": 42}
second = {"mood": "hungry"}

show_all(**first, **second)

```

Possible output:

```
{'name': 'Homer', 'score': 42, 'mood': 'hungry'}
```

This works because the keys are distinct.

Duplicate keys fail at call binding time.

```
def show_all(**kwargs):
    print(kwargs)

first = {"name": "Homer", "score": 42}
second = {"score": 100, "mood": "focused"}

try:
    show_all(**first, **second)
except TypeError as err:
    print(f"error type: {type(err)}")
    print(f"message: {err}")
```

Possible output:

```
error type: <class 'TypeError'>
message: __main__.show_all() got multiple values for keyword argument 'score'
```

The exact module name may differ. The rule is stable: a function call cannot receive the same keyword name twice.

This is different from dictionary merging, where later values can overwrite earlier values.

```
first = {"name": "Homer", "score": 42}
second = {"score": 100, "mood": "focused"}

merged = {**first, **second}

print(f"merged: {merged}")
```

Possible output:

```
merged: {'name': 'Homer', 'score': 100, 'mood': 'focused'}
```

Inside a dictionary display, duplicate keys are resolved by the resulting dictionary construction. In a function call, duplicate keyword arguments are rejected because a parameter cannot receive two values.

The double-star unpacking operation does not copy the contained objects. It passes references to the same objects.

```
def mutate_option(**kwargs):
    items = kwargs["items"]
    items.append("No soup for you!")

quotes = ["D'oh!"]

data = {
    "items": quotes,
}

print("before")
print(f"quotes: {quotes}")
print(f"id(quotes): {id(quotes)}")
print()

mutate_option(**data)
```

```
print("after")
print(f"quotes:      {quotes}")
print(f"id(quotes): {id(quotes)}")
```

Possible output:

```
before
quotes:      ["D'oh!"]
id(quotes):  2292527068032

after
quotes:      ["D'oh!", 'No soup for you!']
id(quotes):  2292527068032
```

The dictionary `kwargs` inside the function is newly created for the call, but the value stored under `"items"` is a reference to the same list object. Mutating that list is visible outside the function.

Key-Value Parameter Extraction, Mapping Lookups, and Safe Override Patterns

A `**kwargs` dictionary is often used for optional named settings. The function can look up keys, provide defaults, reject unknown options, or forward options to another function.

The simplest lookup uses square brackets.

```
def show_required(**kwargs):
    name = kwargs["name"]
    score = kwargs["score"]

    print(f"name={name!r}")
    print(f"score={score!r}")

show_required(name="Homer", score=42)
```

Possible output:

```
name='Homer'
score=42
```

Square-bracket lookup requires the key to exist. If it does not exist, Python raises `KeyError`.

```
def show_required(**kwargs):
    name = kwargs["name"]
    score = kwargs["score"]

    print(f"name={name!r}")
    print(f"score={score!r}")

try:
    show_required(name="Homer")
except KeyError as err:
    print(f"error type: {type(err)}")
    print(f"missing key: {err}")
```

Possible output:

```
error type: <class 'KeyError'>
missing key: 'score'
```


The safer lookup method for optional keys is `get()`.

```
def show_optional(**kwargs):
    name = kwargs.get("name", "unknown")
    score = kwargs.get("score", 42)
    mood = kwargs.get("mood", "calm")

    print(f"{name:<10} score={score:04d} mood={mood}")

show_optional(name="Homer")
show_optional(name="Lisa", score=100, mood="focused")
```

Possible output:

```
Homer      score=0042 mood=calm
Lisa       score=0100 mood=focused
```

The method call:

```
kwargs.get("score", 42)
```

means:

```
if "score" exists:
    return kwargs["score"]
else:
    return 42
```

For required-but-manual validation, check membership explicitly.

```
def show_checked(**kwargs):
    if "name" not in kwargs:
        raise ValueError("missing required option: name")

    name = kwargs["name"]
    score = kwargs.get("score", 42)

    print(f"{name:<10} score={score:04d}")

show_checked(name="Homer")

try:
    show_checked(score=100)
except ValueError as err:
    print(f"error type: {type(err)}")
    print(f"message:    {err}")
```

Possible output:

```
Homer      score=0042
error type: <class 'ValueError'>
message:    missing required option: name
```

This kind of check is clearer than allowing a low-level `KeyError` when the missing key is part of the function's public call contract.

A function can also reject unknown keys.

```
def show_limited(**kwargs):
    allowed = {"name", "score", "mood"}

    for key in kwargs:
        if key not in allowed:
            raise ValueError(f"unknown option: {key}")

    name = kwargs.get("name", "unknown")
    score = kwargs.get("score", 42)
    mood = kwargs.get("mood", "calm")

    print(f"{name:<10} score={score:04d} mood={mood}")

show_limited(name="Homer", score=42)

try:
    show_limited(name="Bart", snack="donut")
except ValueError as err:
    print(f"error type: {type(err)}")
    print(f"message: {err}")
```

Possible output:

```
Homer      score=0042 mood=calm
error type: <class 'ValueError'>
message:    unknown option: snack
```

The set object used for allowed keys is also an ordinary object.

```
allowed = {"name", "score", "mood"}

print(f"type(allowed): {type(allowed)}")
print()

selected_names = [
    "add",
    "remove",
    "discard",
    "union",
    "__contains__",
    "__iter__",
]

for name in selected_names:
    print(f"{name:<14} {name in dir(allowed)}")

print()
print(f"'score' in allowed: { 'score' in allowed }")
print(f"'snack' in allowed: { 'snack' in allowed }")
```

Possible output:

```
type(allowed): <class 'set'>

add          True
```

```

remove          True
discard         True
union           True
__contains__    True
__iter__        True

'score' in allowed: True
'snack' in allowed: False

```

A set is useful here because membership checks are direct and readable.

When extracting options from `kwargs`, `pop()` is useful if the function wants to consume recognized options and then check whether anything unknown remains.

```

def show_consumed(**kwargs):
    name = kwargs.pop("name", "unknown")
    score = kwargs.pop("score", 42)
    mood = kwargs.pop("mood", "calm")

    if kwargs:
        raise ValueError(f"unknown options: {list(kwargs.keys())}")

    print(f"{name:<10} score={score:04d} mood={mood}")

show_consumed(name="Homer", score=42)

try:
    show_consumed(name="Bart", snack="donut", town="Springfield")
except ValueError as err:
    print(f"error type: {type(err)}")
    print(f"message:    {err}")

```

Possible output:

```

Homer      score=0042 mood=calm
error type: <class 'ValueError'>
message:    unknown options: ['snack', 'town']

```

This mutates the local `kwargs` dictionary. That is usually safe because `kwargs` is a fresh dictionary created for the current function call.

However, if the function receives an ordinary dictionary as a normal parameter, mutating it directly would mutate the caller's dictionary. In that case, make a shallow copy first if the caller's mapping should be preserved.

```

def consume_mapping(options):
    local_options = dict(options)

    name = local_options.pop("name", "unknown")
    score = local_options.pop("score", 42)

    print(f"name={name!r}, score={score}")
    print(f"remaining local_options: {local_options}")

settings = {
    "name": "Homer",
    "score": 42,
}

```

```

        "mood": "hungry",
    }

    print("before")
    print(f"settings: {settings}")
    print()

    consume_mapping(settings)

    print()
    print("after")
    print(f"settings: {settings}")

```

Possible output:

```

before
settings: {'name': 'Homer', 'score': 42, 'mood': 'hungry'}

name='Homer', score=42
remaining local_options: {'mood': 'hungry'}

after
settings: {'name': 'Homer', 'score': 42, 'mood': 'hungry'}

```

The expression:

```
local_options = dict(options)
```

creates a new dictionary containing the same key-value references. The dictionary container is new. The contained objects are not deeply copied.

A common safe override pattern is to build a new options dictionary before calling another function.

```

def show_line(name, score, mood):
    print(f"{name:<10} score={score:04d} mood={mood}")

defaults = {
    "name": "unknown",
    "score": 42,
    "mood": "calm",
}

user_options = {
    "name": "Lisa",
    "score": 100,
}

final_options = {
    **defaults,
    **user_options,
}

print(f"final_options: {final_options}")
show_line(**final_options)

```

Possible output:

```
final_options: {'name': 'Lisa', 'score': 100, 'mood': 'calm'}
Lisa          score=0100 mood=calm
```

The dictionary construction:

```
final_options = {
    **defaults,
    **user_options,
}
```

means:

```
start with defaults
then apply user_options on top
if the same key appears twice, the later value wins
```

This avoids duplicate keyword errors in the function call because the merging happened before the call. The function receives each keyword only once.

This is unsafe:

```
def show_line(name, score, mood):
    print(f"{name:<10} score={score:04d} mood={mood}")

defaults = {
    "name": "unknown",
    "score": 42,
    "mood": "calm",
}

user_options = {
    "name": "Lisa",
    "score": 100,
}

try:
    show_line(**defaults, **user_options)
except TypeError as err:
    print(f"error type: {type(err)}")
    print(f"message:    {err}")
```

Possible output:

```
error type: <class 'TypeError'>
message:    __main__.show_line() got multiple values for keyword argument 'name'
```

The call attempts to pass `name` twice and `score` twice. Python rejects that.

This is safe:

```
final_options = {**defaults, **user_options}
show_line(**final_options)
```

Now the duplicate keys have already been resolved by dictionary construction.

For forwarding options, a function can separate what it consumes from what it forwards.

```
def render_text(text, *, uppercase=False, **style):
    if uppercase:
        text = text.upper()

    print(f"text: {text}")
    print(f"style: {style}")

render_text(
    "No soup for you!",
    uppercase=True,
    font="serif",
    size=14,
    answer=42,
)
```

Possible output:

```
text:  NO SOUP FOR YOU!
style: {'font': 'serif', 'size': 14, 'answer': 42}
```

The keyword-only parameter `uppercase` is consumed by the function directly. The remaining keyword arguments are collected into `style`.

A more explicit forwarding example:

```
def final_display(text, **style):
    print(f"text: {text}")
    print(f"style: {style}")

def display_quote(text, **options):
    uppercase = options.pop("uppercase", False)

    if uppercase:
        text = text.upper()

    final_display(text, **options)

display_quote(
    "It is only a model.",
    uppercase=True,
    font="mono",
    size=12,
)
```

Possible output:

```
text:  IT IS ONLY A MODEL.
style: {'font': 'mono', 'size': 12}
```

Here, `display_quote()` consumes `uppercase` and forwards the remaining options to `final_display()`. The safe structure is:

1. receive `**options`
2. extract options that belong to the current function
3. validate or reject unknown options if necessary
4. forward only the remaining dictionary with `**options`

The important rule is:

****kwargs** packs keyword overflow into a dictionary. ****mapping** at the call site unpacks a mapping into keyword arguments. Dictionary merging can resolve overrides before a call; function-call binding itself rejects duplicate keyword names.

8.3.3 Architectural Pitfalls of Argument Evaluation

Default arguments look like call-time conveniences, but they are not evaluated at call time. They are evaluated when the `def` statement executes and the function object is created.

This matters because a default value is not a piece of source text kept for later. It is an object stored inside the function object. If that object is immutable, this usually causes no visible problem. If that object is mutable, repeated calls may accidentally share state through the same stored object.

The danger is not caused by `list` specifically. The danger comes from storing one persistent mutable object as a default value.

The Mutable Default Arguments Trap: Why `def func(x=[])` Reuses One Persistent Mutable Object

Consider this function:

```
def add_quote(text, lines=[]):
    lines.append(text)
    return lines

a = add_quote("D'oh!")
b = add_quote("No soup for you!")
c = add_quote("The answer is 42.")

print(f"a: {a}")
print(f"b: {b}")
print(f"c: {c}")
print()
print(f"a is b: {a is b}")
print(f"b is c: {b is c}")
```

Possible output:

```
a: ["D'oh!", 'No soup for you!', 'The answer is 42.']
b: ["D'oh!", 'No soup for you!', 'The answer is 42.']
c: ["D'oh!", 'No soup for you!', 'The answer is 42.']

a is b: True
b is c: True
```

This is usually not what the programmer intended.

The source line:

```
def add_quote(text, lines=[]):
```

does not mean:

create a new empty list every time `lines` is omitted

It means:

create one empty list now, while defining the function
 store that list inside the function object as the default for lines
 reuse that same list whenever the caller omits lines

The stored default object can be inspected through `__defaults__`.

```
def add_quote(text, lines=[]):
    lines.append(text)
    return lines

print(f"defaults:      {add_quote.__defaults__}")
print(f"type(defaults): {type(add_quote.__defaults__)}")
print()

default_lines = add_quote.__defaults__[0]

print(f"default_lines:      {default_lines}")
print(f"type(default_lines): {type(default_lines)}")
print(f"id(default_lines):   {id(default_lines)}")
```

Possible output:

```
defaults:      ([,])
type(defaults): <class 'tuple'>

default_lines:      []
type(default_lines): <class 'list'>
id(default_lines):   2293103759744
```

The function's `__defaults__` value is a tuple. Inside that tuple is the default list object.
 After calls, that same stored list has changed.

```
def add_quote(text, lines=[]):
    lines.append(text)
    return lines

print("before calls")
print(f"defaults: {add_quote.__defaults__}")
print()

add_quote("D'oh!")
add_quote("No soup for you!")
add_quote("Chuck Norris can divide by zero.")

print("after calls")
print(f"defaults: {add_quote.__defaults__}")
```

Possible output:

```
before calls
defaults: ([,])

after calls
defaults: (['D'oh!', 'No soup for you!', 'Chuck Norris can divide by zero.'],)
```


The default list is not recreated. It is reused and mutated.

The list object has mutating methods, which is why the problem appears.

```
def add_quote(text, lines=[]):
    lines.append(text)
    return lines

default_lines = add_quote.__defaults__[0]

print(f"type(default_lines): {type(default_lines)}")
print()

selected_names = [
    "append",
    "extend",
    "insert",
    "pop",
    "clear",
    "__len__",
    "__getitem__",
    "__setitem__",
]

for name in selected_names:
    print(f"{name:<14} {name in dir(default_lines)}")

print()
default_lines.append("It is only a model.")
print(f"default_lines: {default_lines}")
print(f"defaults:      {add_quote.__defaults__}")
```

Possible output:

```
type(default_lines): <class 'list'>

append      True
extend      True
insert      True
pop          True
clear       True
__len__     True
__getitem__ True
__setitem__ True

default_lines: ['It is only a model.']
defaults:      (['It is only a model.'],)
```

The function default and the external name `default_lines` both refer to the same list object.

The same trap appears with dictionaries.

```
def remember_score(name, score, table={}):
    table[name] = score
    return table

first = remember_score("Homer", 42)
```

```
second = remember_score("Lisa", 100)

print(f"first:  {first}")
print(f"second: {second}")
print()
print(f"first is second: {first is second}")
print(f"defaults:      {remember_score.__defaults__}")
```

Possible output:

```
first:  {'Homer': 42, 'Lisa': 100}
second: {'Homer': 42, 'Lisa': 100}

first is second: True
defaults:      ({'Homer': 42, 'Lisa': 100},)
```

Again, one dictionary was created when the function was defined. Repeated calls reuse it. The dictionary object has mutating methods and item assignment behavior.

```
def remember_score(name, score, table={}):
    table[name] = score
    return table

default_table = remember_score.__defaults__[0]

print(f"type(default_table): {type(default_table)}")
print()

selected_names = [
    "keys",
    "values",
    "items",
    "get",
    "update",
    "clear",
    "__getitem__",
    "__setitem__",
]

for name in selected_names:
    print(f"{name:<14} {name in dir(default_table)}")

print()
default_table["Bart"] = 7
print(f"default_table: {default_table}")
print(f"defaults:      {remember_score.__defaults__}")
```

Possible output:

```
type(default_table): <class 'dict'>

keys      True
values    True
items     True
get       True
```

```

update      True
clear       True
__getitem__ True
__setitem__ True

default_table: {'Bart': 7}
defaults:      ({'Bart': 7},)

```

The trap is not that lists and dictionaries are bad default values in every possible context. The trap is that their mutation persists across calls when the caller omits that argument.

A caller can avoid the stored default by providing an explicit object.

```

def add_quote(text, lines=[]):
    lines.append(text)
    return lines

own_lines = []

a = add_quote("D'oh!", own_lines)
b = add_quote("No soup for you!", own_lines)
c = add_quote("The answer is 42.")

print(f"own_lines: {own_lines}")
print(f"a:         {a}")
print(f"b:         {b}")
print(f"c:         {c}")
print()
print(f"defaults:  {add_quote.__defaults__}")

```

Possible output:

```

own_lines: ["D'oh!", 'No soup for you!']
a:         ["D'oh!", 'No soup for you!']
b:         ["D'oh!", 'No soup for you!']
c:         ['The answer is 42.']

defaults:  (['The answer is 42.'],)

```

The calls using `own_lines` mutate the caller-provided list. The call that omits `lines` mutates the stored default list.

So the real rule is:

A default argument is used only when the caller omits that argument. If the stored default is mutable and the function mutates it, the mutation persists.

Static Expression Evaluation: Why Default Values Are Created at Function Definition Time

Default values are evaluated while Python is executing the `def` statement. This is true for immutable defaults, mutable defaults, function calls, arithmetic expressions, and object construction.

A simple example uses a printed side effect.

```

def make_default():
    print("make_default() is running")
    return "D'oh!"

```

```
def show_line(text=make_default()):
    print(f"text={text!r}")

print("after def")
print()

show_line()
show_line()
show_line("No soup for you!")
```

Possible output:

```
make_default() is running
after def

text="D'oh!"
text="D'oh!"
text='No soup for you!'
```

The function `make_default()` ran once, while `show_line` was being defined. It did not run again for each omitted argument.

This means the function object stores the returned object.

```
def make_default():
    print("make_default() is running")
    return "D'oh!"

def show_line(text=make_default()):
    print(f"text={text!r}")

print()
print(f"show_line.__defaults__: {show_line.__defaults__}")
print(f"type(__defaults__):      {type(show_line.__defaults__)}")
```

Possible output:

```
make_default() is running

show_line.__defaults__: ("D'oh!",)
type(__defaults__):      <class 'tuple'>
```

The tuple stores the default object.

The tuple itself is an ordinary tuple.

```
def show_line(text="D'oh!"):
    print(f"text={text!r}")

defaults = show_line.__defaults__

print(f"type(defaults): {type(defaults)}")
print()

selected_names = [
    "count",
```

```

    "index",
    "__len__",
    "__getitem__",
    "__contains__",
]

for name in selected_names:
    print(f"{name:<14} {name in dir(defaults)}")

print()
print(f"len(defaults):      {len(defaults)}")
print(f"default[0]:         {default[0]!r}")
print(f"'D\\'oh!' in defaults: {'D\\'oh!' in defaults}")

```

Possible output:

```
type(defaults): <class 'tuple'>
```

```

count          True
index           True
__len__        True
__getitem__    True
__contains__   True

```

```

len(defaults):      1
default[0]:         "D'oh!"
'D\\'oh!' in defaults: True

```

Default expressions may refer to names that exist at definition time.

```

base_score = 42

def show_score(name, score=base_score):
    print(f"{name:<10} score={score:04d}")

base_score = 999

show_score("Homer")
show_score("Lisa", 100)

print()
print(f"base_score now:      {base_score}")
print(f"show_score defaults:  {show_score.__defaults__}")

```

Possible output:

```

Homer      score=0042
Lisa       score=0100

base_score now:      999
show_score defaults: (42,)
```

The default object was selected while the function was being created. Later rebinding of `base_score` does not rewrite the function object's stored default.

This behavior is not limited to names. Any default expression is evaluated once.

```
def show_total(x=40 + 2):
    print(f"x={x}")

show_total()
show_total()

print(f"defaults: {show_total.__defaults__}")
```

Possible output:

```
x=42
x=42
defaults: (42,)
```

The expression `40 + 2` is not kept as a pending expression for every call. Its result is stored as the default.

The same applies to object construction.

```
def make_list(items=list()):
    print(f"items:      {items}")
    print(f"id(items): {id(items)}")
    items.append("Ni!")

make_list()
make_list()
make_list()

print()
print(f"defaults: {make_list.__defaults__}")
```

Possible output:

```
items:      []
id(items):  2293103759744
items:      ['Ni!']
id(items):  2293103759744
items:      ['Ni!', 'Ni!']
id(items):  2293103759744

defaults:  (['Ni!', 'Ni!', 'Ni!'],)
```

Using `list()` instead of `[]` does not fix the problem. The problem is not the spelling of the list. The problem is that the expression is evaluated once at function definition time.

The same is true for a dictionary constructor.

```
def remember(name, data=dict()):
    data[name] = len(data) + 1
    return data

print(remember("Arthur"))
print(remember("Patsy"))
print(remember("Galahad"))
print()
print(f"defaults: {remember.__defaults__}")
```

Possible output:

```
{'Arthur': 1}
{'Arthur': 1, 'Patsy': 2}
{'Arthur': 1, 'Patsy': 2, 'Galahad': 3}

defaults: ({'Arthur': 1, 'Patsy': 2, 'Galahad': 3},)
```

The expression `dict()` created one dictionary while the function object was created. That same dictionary became the persistent default.

The compact rule is:

Default argument expressions are definition-time expressions. They are not call-time expressions.

Defending Against State Contamination Using the Immutable None Idiom and Sentinel Guard Clauses

The usual defense is to use `None` as the default value and create a new mutable object inside the function body when needed.

```
def add_quote(text, lines=None):
    if lines is None:
        lines = []

    lines.append(text)
    return lines

a = add_quote("D'oh!")
b = add_quote("No soup for you!")
c = add_quote("The answer is 42.")

print(f"a: {a}")
print(f"b: {b}")
print(f"c: {c}")
print()
print(f"a is b: {a is b}")
print(f"b is c: {b is c}")
```

Possible output:

```
a: ["D'oh!"]
b: ['No soup for you!']
c: ['The answer is 42.']

a is b: False
b is c: False
```

Now each omitted argument creates a fresh list during the call.

The function default is no longer a list. It is `None`.

```
def add_quote(text, lines=None):
    if lines is None:
        lines = []

    lines.append(text)
    return lines
```

```
print(f"defaults:      {add_quote.__defaults__}")
print(f"type(default): {type(add_quote.__defaults__[0])}")
print(f"default is None: {add_quote.__defaults__[0] is None}")
```

Possible output:

```
defaults:      (None,)
type(default): <class 'NoneType'>
default is None: True
```

The `None` object is immutable and is not a list. The function does not mutate `None`. It only uses `None` as a signal meaning “the caller did not provide a list.”

The `None` object can be inspected.

```
x = None
```

```
print(f"x:      {x!r}")
print(f"type(x): {type(x)}")
print()
```

```
selected_names = [
    "__bool__",
    "__class__",
    "__eq__",
    "__repr__",
]
```

```
for name in selected_names:
    print(f"{name:<12} {name in dir(x)}")
```

```
print()
print(f"bool(None): {bool(None)}")
print(f"repr(None): {repr(None)}")
print(f"x is None:  {x is None}")
```

Possible output:

```
x:      None
type(x): <class 'NoneType'>
```

```
__bool__    True
__class__    True
__eq__       True
__repr__     True
```

```
bool(None): False
repr(None): None
x is None:  True
```

The check should normally use `is None`, not `== None`. This checks object identity, which is the correct test for this sentinel value.

The same pattern works for dictionaries.


```
def remember_score(name, score, table=None):
    if table is None:
        table = {}

    table[name] = score
    return table

first = remember_score("Homer", 42)
second = remember_score("Lisa", 100)

print(f"first: {first}")
print(f"second: {second}")
print()
print(f"first is second: {first is second}")
```

Possible output:

```
first: {'Homer': 42}
second: {'Lisa': 100}
```

```
first is second: False
```

If the caller does provide a dictionary, the function still mutates that caller-provided dictionary.

```
def remember_score(name, score, table=None):
    if table is None:
        table = {}

    table[name] = score
    return table

scores = {}

remember_score("Homer", 42, scores)
remember_score("Lisa", 100, scores)

print(f"scores: {scores}")
```

Possible output:

```
scores: {'Homer': 42, 'Lisa': 100}
```

This is correct if the function is supposed to update a provided dictionary. The `None` idiom only prevents accidental reuse of a hidden default dictionary.

Sometimes `None` is a legitimate caller value. In that case, using `None` as the sentinel is ambiguous. For example:

```
def show_value(value=None):
    if value is None:
        print("no value supplied")
    else:
        print(f"value supplied: {value!r}")

show_value()
show_value(None)
show_value(42)
```

Possible output:

```
no value supplied
no value supplied
value supplied: 42
```

This function cannot distinguish between:

```
caller omitted value
```

and:

```
caller explicitly passed None
```

When that distinction matters, use a private sentinel object.

```
missing = object()

def show_value(value=missing):
    if value is missing:
        print("no value supplied")
    else:
        print(f"value supplied: {value!r}")

show_value()
show_value(None)
show_value(42)
```

Possible output:

```
no value supplied
value supplied: None
value supplied: 42
```

The sentinel object is a unique object used only for identity comparison. It can be inspected like any other object.

```
missing = object()

print(f"missing:      {missing}")
print(f"type(missing): {type(missing)}")
print(f"id(missing):   {id(missing)}")
print()

selected_names = [
    "__class__",
    "__eq__",
    "__hash__",
    "__repr__",
]

for name in selected_names:
    print(f"{name:<12} {name in dir(missing)}")

print()
other = object()
```

```
print(f"other:          {other}")
print(f"missing is other: {missing is other}")
print(f"missing == other: {missing == other}")
```

Possible output:

```
missing:          <object object at 0x...>
type(missing):    <class 'object'>
id(missing):      2293100615536

__class__      True
__eq__         True
__hash__       True
__repr__       True

other:          <object object at 0x...>
missing is other: False
missing == other: False
```

Each call to `object()` creates a distinct object. That makes it useful as a sentinel.

A sentinel guard clause can protect the function entrance.

```
missing = object()

def build_lines(first=missing, lines=missing):
    if first is missing:
        first = "D'oh!"

    if lines is missing:
        lines = []

    lines.append(first)
    return lines

a = build_lines()
b = build_lines(None)
c = build_lines("No soup for you!")

print(f"a: {a}")
print(f"b: {b}")
print(f"c: {c}")
```

Possible output:

```
a: ["D'oh!"]
b: [None]
c: ['No soup for you!']
```

Here, `None` is a valid explicit value. The private sentinel object means “not supplied.”

The important structure is:

```
missing = object()

def func(x=missing):
    if x is missing:
        create or choose default now
```

This creates the real default value during the call, not during function definition.

The safe patterns are:

- use immutable defaults such as numbers, strings, booleans, tuples of immutable values, or `None`;
- use `None` when `None` is not a meaningful caller value;
- use a private sentinel object when `None` must remain a valid explicit value;
- create new mutable objects inside the function body when they are needed per call.

The final rule is:

Never use a mutable object as a default value unless persistent shared state is deliberately intended and clearly documented.

8.4 Namespaces and Variable Scope Resolution

Names do not exist in one flat container. During execution, Python keeps different namespaces for different levels of the program: the active function frame, enclosing function frames, the current module, and the built-in namespace. When a name is read, Python searches these namespaces in a fixed order: Local, Enclosing, Global, Built-in.

This section explains that lookup chain and then separates lookup from assignment. Reading a name from an outer namespace is usually automatic. Rebinding a name in an outer namespace is not automatic and requires explicit syntax: `global` for module-level bindings and `nonlocal` for enclosing function bindings.

The central distinction is the same one used throughout the chapter: names are bindings, objects are separate runtime values. Scope rules decide where a name is searched or rebound. They do not change the identity of the objects themselves.

8.4.1 The LEGB Rule Invariant

When Python sees an unqualified name such as `x`, `print`, `len`, or `answer`, it must decide which object that name refers to. The search follows a stable order known as LEGB:

L = Local
E = Enclosing
G = Global
B = Built-in

This rule applies to ordinary name lookup. It does not mean Python searches every object in memory. It searches a chain of namespaces connected to the current execution point.

The four levels are:

- **Local**: names inside the currently executing function frame;
- **Enclosing**: names in outer function frames for nested functions;
- **Global**: names in the current module namespace;
- **Built-in**: names from the built-in namespace, such as `len`, `print`, `dict`, and `TypeError`.

The rule is not decorative terminology. It is the actual direction of lookup for unqualified names.

Local (L): Active Function-Frame Bindings and CPython Fast-Local Storage

The local namespace is the first place Python checks when a function body refers to a name.

```
def show_local():
    quote = "D'oh!"
    answer = 42

    print(f"quote: {quote!r}")
    print(f"answer: {answer!r}")

show_local()
```

Possible output:

```
quote:  "D'oh!"
answer: 42
```

Inside `show_local()`, the names `quote` and `answer` are local names. They belong to the active function call frame.

They are not global names.

```
def show_local():
    quote = "D'oh!"
    answer = 42

    print("inside function")
    print(f"'quote' in locals(): {'quote' in locals()}")
    print(f"'answer' in locals(): {'answer' in locals()}")

show_local()

print()
print("outside function")

try:
    print(quote)
except NameError as err:
    print(f"error type: {type(err)}")
    print(f"message: {err}")
```

Possible output:

```
inside function
'quote' in locals(): True
'answer' in locals(): True

outside function
error type: <class 'NameError'>
message: name 'quote' is not defined
```

The local names existed while the function frame was active. After the call ended, that frame was gone.

The `locals()` function displays the current local namespace.

```
def show_local_namespace():
    name = "Homer"
    score = 42

    local_map = locals()

    print(f"local_map:          {local_map}")
    print(f"type(local_map): {type(local_map)}")

show_local_namespace()
```

Possible output:

```
local_map:          {'name': 'Homer', 'score': 42}
type(local_map): <class 'dict'>
```

The displayed local namespace is dictionary-like.

```
def inspect_local_namespace():
    name = "Homer"
    score = 42

    local_map = locals()

    print(f"type(local_map): {type(local_map)}")
    print()

    selected_names = [
        "keys",
        "values",
        "items",
        "get",
        "__contains__",
        "__getitem__",
    ]

    for attr in selected_names:
        print(f"{attr:<14} {attr in dir(local_map)}")

    print()
    print(f"keys:          {list(local_map.keys())}")
    print(f"local_map['name']: {local_map['name']!r}")
    print(f"local_map.get('score'): {local_map.get('score')}")

inspect_local_namespace()
```

Possible output:

```
type(local_map): <class 'dict'>
```

```
keys          True
values        True
items         True
get           True
__contains__  True
```

```
__getitem__      True

keys:            ['name', 'score']
local_map['name']: 'Homer'
local_map.get('score'): 42
```

This display is useful for inspection, but it should not be confused with the exact internal storage strategy used by CPython for function locals.

In CPython, ordinary function local variables are stored in an optimized internal layout often called *fast locals*. The function's compiled code object already knows the local variable names.

```
def sample_func(name, score):
    message = f"{name}: {score}"
    return message

code_obj = sample_func.__code__

print(f"co_varnames:  {code_obj.co_varnames}")
print(f"co_argcount:  {code_obj.co_argcount}")
print(f"co_nlocals:   {code_obj.co_nlocals}")
```

Possible output:

```
co_varnames:  ('name', 'score', 'message')
co_argcount:  2
co_nlocals:   3
```

The code object says that the function frame needs local slots for:

```
name
score
message
```

That is why local variable access inside a function is not just a normal dictionary lookup every time. CPython can use the compiled local-variable layout.

We can inspect the code object itself.

```
def sample_func(name, score):
    message = f"{name}: {score}"
    return message

code_obj = sample_func.__code__

print(f"type(code_obj): {type(code_obj)}")
print()

selected_names = [
    "co_varnames",
    "co_argcount",
    "co_nlocals",
    "co_consts",
    "co_names",
    "co_code",
]

for attr in selected_names:
    print(f"{attr:<14} {attr in dir(code_obj)}")
```

Possible output:

```
type(code_obj): <class 'code'>
```

```
co_varnames    True
co_argcount    True
co_nlocals    True
co_consts     True
co_names      True
co_code       True
```

The local namespace therefore has two useful views:

- the programmer-facing view: local names visible through `locals()`;
- the CPython execution view: optimized local slots described by the code object.

For lookup behavior, the important rule is simpler:

If a name exists in the active local function frame, Python uses that local binding before looking anywhere else.

This can shadow a global name.

```
answer = 100

def show_answer():
    answer = 42
    print(f"inside function: answer={answer}")

show_answer()

print(f"outside function: answer={answer}")
```

Possible output:

```
inside function: answer=42
outside function: answer=100
```

Inside the function, `answer` is found locally first. Python does not continue outward to the global `answer` because the local one already satisfies the lookup.

Enclosing (E): Looking Upwards Through Cell Variables of Nested Lexical Scopes

The enclosing level exists when a function is defined inside another function.

```
def outer():
    quote = "Nobody expects the Spanish Inquisition!"

    def inner():
        print(f"inner sees quote: {quote}")

    inner()

outer()
```

Possible output:

`inner` sees `quote`: Nobody expects the Spanish Inquisition!

The name `quote` is not local to `inner()`. It is local to `outer()`. Since `inner()` is nested inside `outer()`, Python can search the enclosing function scope.

The lookup path inside `inner()` is:

1. Local scope of `inner()`
2. Enclosing scope of `outer()`
3. Global module scope
4. Built-in scope

This is lexical nesting. It depends on where the function is defined in the source, not merely where it is called.

```
quote = "global quote"

def outer():
    quote = "outer quote"

    def inner():
        print(f"quote: {quote}")

    inner()

outer()
```

Possible output:

```
quote: outer quote
```

The name `quote` is not found inside `inner()`. Python then checks the enclosing function scope and finds `outer`'s `quote`. It does not use the global `quote` because the enclosing binding was found first.

If the inner function has its own local name, that local name wins.

```
quote = "global quote"

def outer():
    quote = "outer quote"

    def inner():
        quote = "inner quote"
        print(f"quote: {quote}")

    inner()

outer()
```

Possible output:

```
quote: inner quote
```

The local level is still first.

A nested function that refers to an outer variable causes CPython to store that outer variable in a cell so the inner function can access it.

```
def outer():
    quote = "It is only a model."

    def inner():
        return quote

    return inner

fn = outer()

print(f"fn(): {fn()}")
print(f"fn.__closure__: {fn.__closure__}")
print(f"fn.__code__.co_freevars: {fn.__code__.co_freevars}")
```

Possible output:

```
fn(): It is only a model.
fn.__closure__: (<cell at 0x...: str object at 0x...>,)
fn.__code__.co_freevars: ('quote',)
```

The outer function has already finished, but the returned inner function still remembers the needed enclosing value. That preserved value lives through a closure cell.

The cell object can be inspected.

```
def outer():
    quote = "It is only a model."

    def inner():
        return quote

    return inner

fn = outer()
cell = fn.__closure__[0]

print(f"cell: {cell}")
print(f"type(cell): {type(cell)}")
print()

selected_names = [
    "cell_contents",
    "__class__",
    "__eq__",
    "__repr__",
]

for attr in selected_names:
    print(f"{attr:<16} {attr in dir(cell)}")

print()
print(f"cell.cell_contents: {cell.cell_contents!r}")
print(f"type(contents): {type(cell.cell_contents)}")
```

Possible output:

```
cell:          <cell at 0x...: str object at 0x...>
type(cell):    <class 'cell'>
```

```
cell_contents  True
__class__      True
__eq__         True
__repr__       True
```

```
cell.cell_contents: 'It is only a model.'
type(contents):     <class 'str'>
```

This is the mechanical basis of enclosing-scope lookup. The inner function does not search an arbitrary dead stack frame. CPython preserves the required enclosing references in closure cells.

The outer function's code object also records which local names become cell variables.

```
def outer():
    quote = "It is only a model."

    def inner():
        return quote

    return inner

print(f"outer co_cellvars: {outer.__code__.co_cellvars}")

fn = outer()
print(f"inner co_freevars: {fn.__code__.co_freevars}")
```

Possible output:

```
outer co_cellvars: ('quote',)
inner co_freevars: ('quote',)
```

The name `quote` is a cell variable from the outer function's point of view. It is a free variable from the inner function's point of view.

The full closure mechanism will be studied later in the section on first-class functions and closures. Here, the important lookup rule is:

If a name is not local, Python looks through enclosing lexical function scopes before checking the module namespace.

Global (G): Module-Level Dictionary Namespaces and Active Script Execution State (`globals()`)

The global namespace is the namespace of the current module. In a simple script, it is the top-level script namespace.

A name assigned at top level is global to that module.

```
answer = 42
quote = "No soup for you!"

def show_globals():
    print(f"answer: {answer}")
    print(f"quote: {quote}")

show_globals()
```

Possible output:

```
answer: 42
quote: No soup for you!
```

Inside `show_globals()`, the names `answer` and `quote` are not local. There is no enclosing function scope. Python therefore looks in the global module namespace.

The global namespace is visible through `globals()`.

```
answer = 42
quote = "No soup for you!"

global_map = globals()

print(f"type(global_map): {type(global_map)}")
print(f"'answer' in globals(): {'answer' in global_map}")
print(f"'quote' in globals(): {'quote' in global_map}")
print()
print(f"global_map['answer']: {global_map['answer']}")
print(f"global_map['quote']: {global_map['quote']!r}")
```

Possible output:

```
type(global_map): <class 'dict'>
'answer' in globals(): True
'quote' in globals(): True

global_map['answer']: 42
global_map['quote']: 'No soup for you!'
```

The object returned by `globals()` is a dictionary.

```
answer = 42

global_map = globals()

print(f"type(global_map): {type(global_map)}")
print()

selected_names = [
    "keys",
    "values",
    "items",
    "get",
    "__contains__",
    "__getitem__",
    "__setitem__",
]

for attr in selected_names:
    print(f"{attr:<14} {attr in dir(global_map)}")

print()
print(f"global_map.get('answer'): {global_map.get('answer')}")
```

Possible output:

```
type(global_map): <class 'dict'>
```

```
keys          True
values        True
items         True
get           True
__contains__  True
__getitem__   True
__setitem__   True
```

```
global_map.get('answer'): 42
```

This dictionary is the module's namespace dictionary. Top-level assignments place names into it.

```
answer = 42
```

```
print(f"before: globals()['answer'] = {globals()['answer']}")
```

```
globals()["answer"] = 100
```

```
print(f"after:  answer = {answer}")
```

Possible output:

```
before: globals()['answer'] = 42
```

```
after:  answer = 100
```

This works because `globals()` returns the actual module namespace dictionary.

That does not mean ordinary code should casually mutate `globals()`. It is usually clearer to use normal assignment at module level. The point here is architectural: the global namespace is a dictionary-like object used for module-level name resolution.

A function can read a global name without any special declaration.

```
answer = 42
```

```
def show_answer():
    print(f"answer inside function: {answer}")
```

```
show_answer()
```

Possible output:

```
answer inside function: 42
```

But assignment inside a function creates a local binding by default. It does not automatically mutate the global namespace.

```
answer = 42
```

```
def change_answer():
    answer = 100
    print(f"inside function: answer={answer}")
```

```
change_answer()
```

```
print(f"outside function: answer={answer}")
```

Possible output:

```
inside function: answer=100
outside function: answer=42
```

The assignment inside `change_answer()` created a local `answer`. The global `answer` remained unchanged.

The `global` declaration, which allows assignment to target the module namespace, belongs to the next subsection. For now, the lookup rule is:

If a name is not local and not found in an enclosing function scope, Python looks in the current module's global namespace.

Built-in (B): The Outer Built-In Namespace Boundary and the `builtins` Module

If Python does not find a name in the local, enclosing, or global namespace, it checks the built-in namespace.

This is why names such as `print`, `len`, `dict`, `list`, `int`, and `TypeError` are normally available without import statements.

```
text = "D'oh!"

print(f"len(text): {len(text)}")
print(f"type(text): {type(text)}")
```

Possible output:

```
len(text): 5
type(text): <class 'str'>
```

The names `print`, `len`, and `type` are not defined locally by this code. They are found in the built-in namespace.

The built-in namespace is exposed through the standard `builtins` module.

```
import builtins

print(f"builtins.len: {builtins.len}")
print(f"builtins.print: {builtins.print}")
print(f"builtins.dict: {builtins.dict}")
print()
print(f"type(builtins): {type(builtins)}")
```

Possible output:

```
builtins.len: <built-in function len>
builtins.print: <built-in function print>
builtins.dict: <class 'dict'>

type(builtins): <class 'module'>
```

The module object can be inspected.

```
import builtins

print(f"type(builtins): {type(builtins)}")
print()
```

```
selected_names = [
    "len",
    "print",
    "dict",
    "list",
    "int",
    "TypeError",
    "__dict__",
]

for attr in selected_names:
    print(f"{attr:<12} {attr in dir(builtins)}")
```

Possible output:

```
type(builtins): <class 'module'>
```

```
len          True
print        True
dict         True
list         True
int          True
TypeError    True
__dict__     True
```

A module has a namespace dictionary.

```
import builtins

builtins_map = builtins.__dict__

print(f"type(builtins_map): {type(builtins_map)}")
print()
print(f"'len' in builtins_map:  {'len' in builtins_map}")
print(f"'print' in builtins_map: {'print' in builtins_map}")
print(f"'dict' in builtins_map:  {'dict' in builtins_map}")
print()
print(f"builtins_map['len']: {builtins_map['len']}")
```

Possible output:

```
type(builtins_map): <class 'dict'>

'len' in builtins_map:  True
'print' in builtins_map: True
'dict' in builtins_map: True

builtins_map['len']: <built-in function len>
```

The built-in function object `len` can also be inspected.

```
import builtins

fn = builtins.len
```

```

print(f"fn:      {fn}")
print(f"type(fn): {type(fn)}")
print()

selected_names = [
    "__call__",
    "__name__",
    "__doc__",
    "__self__",
    "__text_signature__",
]

for attr in selected_names:
    print(f"{attr:<22} {attr in dir(fn)}")

print()
print(f"fn.__name__: {fn.__name__!r}")
print(f"fn('D\\'oh!'): {fn('D'oh!\\')}")

```

Possible output:

```

fn:      <built-in function len>
type(fn): <class 'builtin_function_or_method'>

__call__      True
__name__      True
__doc__       True
__self__      True
__text_signature__  True

fn.__name__: 'len'
fn('D\\'oh!'): 5

```

Built-in names can be shadowed by local or global names. This is usually a bad idea, but it clearly shows LEGB order.

```

len = 42

print(f"global len: {len}")
print(f"type(len): {type(len)}")
print()

try:
    print(len("D'oh!"))
except TypeError as err:
    print(f"error type: {type(err)}")
    print(f"message:    {err}")

```

Possible output:

```

global len: 42
type(len): <class 'int'>

error type: <class 'TypeError'>
message:    'int' object is not callable

```


The global name `len` now points to the integer object `42`. When Python sees `len("D'oh!")`, it finds the global `len` before it reaches the built-in namespace. It then tries to call the integer object, which fails.

The original built-in is still available through `builtins.len`.

```
import builtins

len = 42

print(f"global len:      {len}")
print(f"builtins.len(...): {builtins.len(\"D'oh!\")}")
```

Possible output:

```
global len:      42
builtins.len(...): 5
```

A local name can also shadow a built-in name.

```
def show_shadowing():
    list = "This is not the list class."

    print(f"local list:      {list!r}")
    print(f"type(local list): {type(list)}")

    try:
        values = list((1, 2, 3))
    except TypeError as err:
        print(f"error type: {type(err)}")
        print(f"message:      {err}")

show_shadowing()
```

Possible output:

```
local list:      'This is not the list class.'
type(local list): <class 'str'>
error type: <class 'TypeError'>
message:      'str' object is not callable
```

Inside the function, the local name `list` hides the built-in class `list`. Python finds the local string first and stops searching.

The compact rule is:

For an unqualified name, Python searches Local, then Enclosing, then Global, then Built-in. The first binding found is the one used.

A complete example can show all four levels.

```
import builtins

name = "global name"

def outer():
    name = "enclosing name"
```

```
def inner():
    name = "local name"

    print(f"name:          {name}")
    print(f"builtins.len: {builtins.len('D\\'oh!')}")

inner()

outer()
```

Possible output:

```
name:          local name
builtins.len:  5
```

If the local `name` is removed, the enclosing one is found. If the enclosing one is removed, the global one is found. If the global one is removed and no built-in exists with that name, lookup fails with `NameError`.

That is the LEGB invariant.

8.4.2 Mutating External Scopes

Reading a name from an outer scope and assigning to a name in an outer scope are different operations. Python allows ordinary read access through the LEGB chain, but assignment has stricter rules.

By default, assignment inside a function creates or updates a local binding in that function frame. It does not automatically rewrite a global name or an enclosing function's name.

There are two explicit declarations for overriding this default:

- `global` targets the module namespace;
- `nonlocal` targets an enclosing function scope.

These declarations do not change the object-passing model. They change which namespace receives the binding operation.

Local Read Access Boundaries vs. the Shadowing Consequence of Assignment

A function can read a global name without a declaration.

```
answer = 42

def show_answer():
    print(f"answer inside function: {answer}")

show_answer()
print(f"answer outside function: {answer}")
```

Possible output:

```
answer inside function: 42
answer outside function: 42
```

The function body does not define a local `answer`, so Python searches outward and finds the global `answer`.

Assignment is different.

```

answer = 42

def set_answer():
    answer = 100
    print(f"inside function: {answer}")

set_answer()
print(f"outside function: {answer}")

```

Possible output:

```

inside function: 100
outside function: 42

```

The assignment inside `set_answer()` creates a local name. It does not mutate the module-level name.

The structure is:

```

module namespace:
    answer -> 42

set_answer() local frame:
    answer -> 100

```

The two names are separate bindings.

This is shadowing: the local name hides the outer name during lookup inside the function.

```

message = "global message"

def show_shadowing():
    message = "local message"

    print(f"local lookup: {message}")
    print(f"'message' in locals(): {'message' in locals()}")
    print(f"'message' in globals(): {'message' in globals()}")

show_shadowing()

print(f"global lookup: {message}")

```

Possible output:

```

local lookup: local message
'message' in locals(): True
'message' in globals(): True
global lookup: global message

```

The name `message` exists in both namespaces. Inside the function, the local binding wins because Local comes before Global in LEGB.

The local namespace display is a dictionary-like object.

```

message = "global message"

def show_shadowing():
    message = "local message"

```

```

local_map = locals()

print(f"type(local_map): {type(local_map)}")
print()

selected_names = [
    "keys",
    "values",
    "items",
    "get",
    "__contains__",
    "__getitem__",
]

for name in selected_names:
    print(f"{name:<14} {name in dir(local_map)}")

print()
print(f"local_map: {local_map}")

show_shadowing()

```

Possible output:

```

type(local_map): <class 'dict'>

keys           True
values         True
items          True
get            True
__contains__   True
__getitem__    True

local_map: {'message': 'local message'}

```

Reading an outer mutable object and mutating that object in place does not require rebinding the outer name.

```

quotes = ["D'oh!"]

def add_quote():
    quotes.append("No soup for you!")

add_quote()

print(f"quotes: {quotes}")

```

Possible output:

```

quotes: ["D'oh!", 'No soup for you!']

```

The function did not assign to the name `quotes`. It read the global binding, obtained the list object, and called the list object's `append()` method.

The list object is mutable.

```

quotes = ["D'oh!"]

print(f"type(quotes): {type(quotes)}")
print()

selected_names = [
    "append",
    "extend",
    "insert",
    "pop",
    "clear",
    "__len__",
    "__getitem__",
    "__setitem__",
]

for name in selected_names:
    print(f"{name:<14} {name in dir(quotes)}")

print()
quotes.append("The answer is 42.")
print(f"after append: {quotes}")

```

Possible output:

```

type(quotes): <class 'list'>

append      True
extend      True
insert      True
pop          True
clear       True
__len__     True
__getitem__ True
__setitem__ True

after append: ["D'oh!", 'The answer is 42.']

```

This operation changes the shared list object. It does not reassign the global name.

This does reassign the local name:

```

quotes = ["D'oh!"]

def replace_quotes():
    quotes = ["This is an ex-list."]
    print(f"inside function: {quotes}")

replace_quotes()

print(f"outside function: {quotes}")

```

Possible output:

```

inside function: ['This is an ex-list.']
outside function: ["D'oh!"]

```

So the basic distinction is:

```
quotes.append(x)
    read outer name
    mutate shared list object

quotes = [...]
    bind local name unless global or nonlocal says otherwise
```

This distinction explains why some functions appear to “change globals” without `global`. They are not changing the global binding. They are mutating an object reached through that binding.

Overriding Module Scope: The `global` Declaration Syntax and Module Namespace Mutation

The `global` declaration tells Python that assignments to a name inside the function should target the module namespace.

```
answer = 42

def set_global_answer():
    global answer

    answer = 100
    print(f"inside function: {answer}")

print(f"before call: {answer}")

set_global_answer()

print(f"after call: {answer}")
```

Possible output:

```
before call: 42
inside function: 100
after call: 100
```

The assignment:

```
answer = 100
```

does not create a local binding here. Because of `global answer`, it updates the module namespace binding.

The module namespace can be inspected with `globals()`.

```
answer = 42

def set_global_answer():
    global answer

    print(f"before assignment in globals: {globals()['answer']}")
    answer = 100
    print(f"after assignment in globals: {globals()['answer']}")

set_global_answer()
```

Possible output:

```
before assignment in globals: 42
after assignment in globals: 100
```

The object returned by `globals()` is the actual module namespace dictionary.

```
answer = 42

global_map = globals()

print(f"type(global_map): {type(global_map)}")
print()

selected_names = [
    "keys",
    "values",
    "items",
    "get",
    "__contains__",
    "__getitem__",
    "__setitem__",
]

for name in selected_names:
    print(f"{name:<14} {name in dir(global_map)}")

print()
print(f"global_map['answer']: {global_map['answer']}")
```

Possible output:

```
type(global_map): <class 'dict'>

keys           True
values         True
items          True
get            True
__contains__   True
__getitem__    True
__setitem__    True

global_map['answer']: 42
```

The `global` declaration is needed only for rebinding the name. It is not needed for mutating an already reachable object.

```
quotes = ["D'oh!"]

def mutate_global_list():
    quotes.append("No soup for you!")

def rebind_global_list():
    global quotes
    quotes = ["It is only a model."]
```

```
print(f"start: {quotes}")

mutate_global_list()
print(f"after mutation: {quotes}")

rebind_global_list()
print(f"after rebinding: {quotes}")
```

Possible output:

```
start: ["D'oh!"]
after mutation: ["D'oh!", 'No soup for you!']
after rebinding: ['It is only a model.']
```

The first function changes the list object. The second function changes the module-level binding `quotes` itself.

The `global` declaration must appear before the name is used in a way that conflicts with the declaration inside the same function body.

This is valid:

```
score = 42

def set_score():
    global score
    score = 100

set_score()
print(score)
```

Possible output:

```
100
```

A `global` declaration should be used sparingly. It creates hidden coupling between the function body and the module state. When possible, it is usually clearer to return a value and let the caller decide whether to rebind the global name.

```
score = 42

def make_new_score(old_score):
    return old_score + 58

score = make_new_score(score)

print(score)
```

Possible output:

```
100
```

This version makes the state transfer explicit:

```
caller passes old value
function returns new value
caller rebinds module name
```

The `global` version directly mutates the module namespace from inside the function.

Overriding Nested Intermediary Scopes: The `nonlocal` Declaration Syntax and Cell Reference Mutation

The `nonlocal` declaration targets a name in an enclosing function scope. It does not target the module namespace.

Without `nonlocal`, assignment inside an inner function creates a new local binding.

```
def outer():
    count = 0

    def inner():
        count = 1
        print(f"inner count: {count}")

    inner()
    print(f"outer count: {count}")

outer()
```

Possible output:

```
inner count: 1
outer count: 0
```

The `count = 1` assignment creates a local `count` inside `inner()`. It does not modify `outer()`'s `count`.

With `nonlocal`, the assignment targets the nearest enclosing function scope that already has that name.

```
def outer():
    count = 0

    def inner():
        nonlocal count

        count = count + 1
        print(f"inner count: {count}")

    inner()
    inner()
    inner()

    print(f"outer count: {count}")

outer()
```

Possible output:

```
inner count: 1
inner count: 2
inner count: 3
outer count: 3
```

The inner function modifies the binding preserved in the enclosing scope. This is especially important for closures.

```
def make_counter():
    count = 0

    def next_count():
        nonlocal count

        count = count + 1
        return count

    return next_count

counter = make_counter()

print(counter())
print(counter())
print(counter())
```

Possible output:

```
1
2
3
```

The outer function has already returned, but the inner function still carries access to `count` through a closure cell.

The closure can be inspected.

```
def make_counter():
    count = 0

    def next_count():
        nonlocal count

        count = count + 1
        return count

    return next_count

counter = make_counter()

print(f"counter.__closure__: {counter.__closure__}")
print(f"counter.__code__.co_freevars: {counter.__code__.co_freevars}")
print()

cell = counter.__closure__[0]

print(f"cell: {cell}")
print(f"type(cell): {type(cell)}")
print(f"contents: {cell.cell_contents}")
```

Possible output:

```
counter.__closure__: (<cell at 0x...: int object at 0x...>,)
counter.__code__.co_freevars: ('count',)
```

```
cell:      <cell at 0x...: int object at 0x...>
type(cell): <class 'cell'>
contents:  0
```

After calls, the cell contents change.

```
def make_counter():
    count = 0

    def next_count():
        nonlocal count

        count = count + 1
        return count

    return next_count

counter = make_counter()
cell = counter.__closure__[0]

print(f"before: {cell.cell_contents}")

print(counter())
print(counter())

print(f"after:  {cell.cell_contents}")
```

Possible output:

```
before: 0
1
2
after:  2
```

The cell object can be inspected.

```
def make_counter():
    count = 0

    def next_count():
        nonlocal count

        count = count + 1
        return count

    return next_count

counter = make_counter()
cell = counter.__closure__[0]

print(f"type(cell): {type(cell)}")
print()

selected_names = [
    "cell_contents",
```

```

    "__class__",
    "__eq__",
    "__repr__",
]

for name in selected_names:
    print(f"{name:<16} {name in dir(cell)}")

print()
print(f"cell.cell_contents: {cell.cell_contents}")

```

Possible output:

```

type(cell): <class 'cell'>

cell_contents      True
__class__          True
__eq__             True
__repr__           True

cell.cell_contents: 0

```

The `nonlocal` declaration changes which binding is updated. It does not mean that integers become mutable. In the counter example, this line:

```
count = count + 1
```

creates or obtains a new integer object, then updates the enclosing cell so that `count` refers to the new object.

A mutable object in an enclosing scope can be mutated without `nonlocal`, just like a global mutable object can be mutated without `global`.

```

def outer():
    lines = []

    def inner():
        lines.append("Ni!")

    inner()
    inner()

    print(lines)

outer()

```

Possible output:

```
['Ni!', 'Ni!']
```

The inner function reads the enclosing binding `lines`, obtains the list object, and mutates the list. But rebinding the enclosing name requires `nonlocal`.

```

def outer():
    lines = []

```

```

def inner():
    nonlocal lines

    lines = ["This is an ex-list."]

    inner()

    print(lines)

outer()

```

Possible output:

```
['This is an ex-list.']
```

The difference is the same as before:

```

lines.append("Ni!")
    mutate the shared list object

lines = [...]
    rebind the enclosing name because nonlocal was declared

```

The `nonlocal` name must already exist in an enclosing function scope. It cannot create a new enclosing name, and it does not refer to the global namespace.

A useful practical rule is:

Use `nonlocal` only when an inner function must rebind a name from an enclosing function. For ordinary data flow, prefer returning values unless closure state is intentionally part of the design.

locals() Caveats: Why the Displayed Local Namespace Is Not Always a Directly Writable Control Surface

The function `locals()` returns a mapping that represents the current local namespace. At module level, local and global namespace are the same namespace.

```

answer = 42

print(f"locals() is globals(): {locals() is globals()}")

locals()["answer"] = 100

print(f"answer: {answer}")

```

Possible output at module level:

```

locals() is globals(): True
answer: 100

```

At module level, `locals()` and `globals()` usually refer to the same dictionary. Mutating that dictionary mutates the module namespace.

Inside an ordinary function, this should not be treated as a reliable way to create or rebind local variables.

```
def try_write_locals():
    x = 42

    local_map = locals()
    local_map["x"] = 100
    local_map["y"] = "D'oh!"

    print(f"local_map: {local_map}")
    print(f"x:         {x}")

    try:
        print(f"y:         {y}")
    except NameError as err:
        print(f"error type: {type(err)}")
        print(f"message:    {err}")

try_write_locals()
```

Possible output:

```
local_map: {'x': 100, 'y': "D'oh!"}
x:         42
error type: <class 'NameError'>
message:    name 'y' is not defined
```

The displayed dictionary changed, but the actual fast-local variable `x` used by the function body did not necessarily change. The name `y` did not become a normal local variable.

This is the caveat: inside optimized function frames, `locals()` is mainly an inspection surface, not a dependable write-control surface.

The map returned by `locals()` may look writable because it is dictionary-like.

```
def inspect_locals_map():
    x = 42
    local_map = locals()

    print(f"type(local_map): {type(local_map)}")
    print()

    selected_names = [
        "keys",
        "values",
        "items",
        "get",
        "__getitem__",
        "__setitem__",
    ]

    for name in selected_names:
        print(f"{name:<14} {name in dir(local_map)}")

inspect_locals_map()
```

Possible output:

```
type(local_map): <class 'dict'>
```

```
keys          True
values        True
items         True
get           True
__getitem__   True
__setitem__   True
```

But dictionary-like appearance does not guarantee that assigning into it rewires the optimized local slots of the running function.

The reason connects to the code object's local-variable layout.

```
def sample_func():
    x = 42
    y = "D'oh!"
    return x, y

code_obj = sample_func.__code__

print(f"co_varnames: {code_obj.co_varnames}")
print(f"co_nlocals:  {code_obj.co_nlocals}")
```

Possible output:

```
co_varnames: ('x', 'y')
co_nlocals:  2
```

CPython knows the ordinary local variable slots ahead of time. The actual execution of `x` and `y` uses that optimized local layout, not necessarily a live dictionary lookup for every access.

This is why writing into `locals()` inside a function is not a normal assignment substitute.

Use ordinary assignment for local names:

```
def correct_local_assignment():
    x = 42
    x = 100
    print(f"x: {x}")

correct_local_assignment()
```

Possible output:

```
x: 100
```

Use `global` for module-level rebinding:

```
answer = 42

def correct_global_assignment():
    global answer

    answer = 100

correct_global_assignment()
print(answer)
```

Possible output:

100

Use `nonlocal` for enclosing-function rebinding:

```
def outer():
    answer = 42

    def inner():
        nonlocal answer

        answer = 100

    inner()
    print(answer)
```

`outer()`

Possible output:

100

The rule is:

Use `locals()` to inspect local bindings. Do not use it as the normal mechanism for changing local variables inside an optimized function frame.

The scope-mutation summary is:

Goal	Correct mechanism	Typical effect
Create or update a local name	ordinary assignment	updates current function frame
Rebind a module-level name	<code>global</code> declaration	updates module namespace dictionary
Rebind an enclosing function name	<code>nonlocal</code> declaration	updates closure cell / enclosing binding
Mutate a reachable mutable object	method call or item assignment	changes the shared object, not the name binding
Inspect current locals	<code>locals()</code>	displays local namespace information

The central distinction remains unchanged:

Name rebinding changes a namespace entry. Object mutation changes an object reached through a name.

8.5 Runtime Scope Mechanics and Variable Binding Analysis

Scope errors are not exceptions to Python's name-resolution model. They are direct consequences of it. Once a function body is compiled, Python has already classified many names as local, global, free, or built-in lookups. Runtime execution then follows those classifications when loading and storing objects.

This section examines the failure cases that reveal the machinery most clearly. `UnboundLocalError`, conflicting local and global names, and ordinary `NameError` are not vague lookup problems. Each one

corresponds to a specific binding situation: a missing name, a local slot read before assignment, or a namespace selected differently than the programmer expected.

The section then moves from runtime symptoms to compile-time preparation. Python scans assignment targets, builds symbol information, chooses local-slot or global-lookup behavior, and preserves closure variables through cells. Understanding this separation between static name classification and dynamic value lookup is necessary before closures, decorators, generators, and coroutine state can be explained correctly.

8.5.1 Anatomy of Scope Failures

Scope failures are not random lookup accidents. They follow directly from Python's binding rules. The most important failure pattern appears when a name is classified as local because the function body assigns to it somewhere, but execution tries to read that local name before any value has been bound to it.

This is the source of `UnboundLocalError`.

The error is often misunderstood as "Python failed to find the global variable." That is not the real mechanism. In many cases, Python does not even try the global namespace, because the compiler has already classified the name as local for that function.

Runtime Execution Traces and Bytecode Analysis of `UnboundLocalError`

Start with a global name that can be read normally.

```
answer = 42

def show_answer():
    print(answer)

show_answer()
```

Possible output:

```
42
```

Inside `show_answer()`, there is no assignment to `answer`. Python treats `answer` as a nonlocal lookup, finds it in the global namespace, and prints it.

Now add an assignment inside the function.

```
answer = 42

def broken_answer():
    print(answer)
    answer = 100

try:
    broken_answer()
except UnboundLocalError as err:
    print(f"error type: {type(err)}")
    print(f"message:    {err}")
```

Possible output:

```
error type: <class 'UnboundLocalError'>
message:    cannot access local variable 'answer' where it is not associated with a value
```

The assignment:

```
answer = 100
```

makes `answer` a local name for the entire function body. This classification is not limited to the lines after the assignment. It applies to the function as compiled.

So the earlier read:

```
print(answer)
```

tries to read the local `answer`. At that moment, the local frame has no value bound to `answer`. Python raises `UnboundLocalError`.

The error object can be inspected.

```
answer = 42
```

```
def broken_answer():
    print(answer)
    answer = 100
```

```
try:
    broken_answer()
except UnboundLocalError as err:
    print(f"err: {err}")
    print(f"type(err): {type(err)}")
    print()

    selected_names = [
        "args",
        "__class__",
        "__str__",
        "__repr__",
        "with_traceback",
    ]

    for name in selected_names:
        print(f"{name:<18} {name in dir(err)}")

    print()
    print(f"err.args: {err.args}")
```

Possible output:

```
err:          cannot access local variable 'answer' where it is not associated with a value
type(err): <class 'UnboundLocalError'>
```

```
args          True
__class__     True
__str__       True
__repr__      True
with_traceback True
```

```
err.args: ("cannot access local variable 'answer' where it is not associated with a value",)
```

The exception object is a normal runtime object. It has a type, attributes, and a message stored in `args`.

The traceback shows where execution failed.

```
import traceback

answer = 42

def broken_answer():
    print(answer)
    answer = 100

try:
    broken_answer()
except UnboundLocalError:
    traceback.print_exc()
```

Possible output:

```
Traceback (most recent call last):
  File "scope_demo.py", line 8, in <module>
    broken_answer()
  File "scope_demo.py", line 5, in broken_answer
    print(answer)
    ~~~~~
```

UnboundLocalError: cannot access local variable 'answer' where it is not associated with a va

The traceback points to the read, not the later assignment. The assignment is still the reason the name was classified as local, but the runtime failure happens when Python tries to load the unbound local value.

The traceback object can also be inspected.

```
import sys

answer = 42

def broken_answer():
    print(answer)
    answer = 100

try:
    broken_answer()
except UnboundLocalError as err:
    err_type, err_obj, tb_obj = sys.exc_info()

    print(f"err_type:      {err_type}")
    print(f"err_obj:       {err_obj}")
    print(f"tb_obj:        {tb_obj}")
    print(f"type(tb_obj): {type(tb_obj)}")
    print()

    selected_names = [
        "tb_frame",
        "tb_lineno",
        "tb_next",
        "tb_lasti",
    ]

    for name in selected_names:
```

```
print(f"{name:<12} {name in dir(tb_obj)}")
```

Possible output:

```
err_type:      <class 'UnboundLocalError'>
err_obj:       cannot access local variable 'answer' where it is not associated with a value
tb_obj:        <traceback object at 0x...>
type(tb_obj):  <class 'traceback'>

tb_frame      True
tb_lineno     True
tb_next       True
tb_lasti      True
```

The traceback chain contains frame information. That frame information can point to the active code object and local namespace display.

```
import sys

answer = 42

def broken_answer():
    print(answer)
    answer = 100

try:
    broken_answer()
except UnboundLocalError:
    tb_obj = sys.exc_info()[2]

    while tb_obj is not None:
        frame = tb_obj.tb_frame
        code_obj = frame.f_code

        print(f"function: {code_obj.co_name}")
        print(f"line:      {tb_obj.tb_lineno}")
        print(f"locals:    {frame.f_locals}")
        print()

        tb_obj = tb_obj.tb_next
```

Possible output:

```
function: <module>
line:     8
locals:   {...}

function: broken_answer
line:     5
locals:   {}
```

Inside the failing `broken_answer()` frame, the local namespace display has no bound value for `answer`. The name was reserved as a local slot, but no runtime assignment has executed yet.

Bytecode inspection makes the distinction visible.

```

import dis

answer = 42

def show_answer():
    print(answer)

def broken_answer():
    print(answer)
    answer = 100

print("show_answer bytecode:")
dis.dis(show_answer)

print()
print("broken_answer bytecode:")
dis.dis(broken_answer)

```

Possible output on a modern CPython version:

```

show_answer bytecode:
 5           0 RESUME           0

 6           2 LOAD_GLOBAL      1 (NULL + print)
          14 LOAD_GLOBAL      2 (answer)
          26 CALL              1
          36 POP_TOP
          38 RETURN_CONST      0 (None)

broken_answer bytecode:
 8           0 RESUME           0

 9           2 LOAD_GLOBAL      1 (NULL + print)
          14 LOAD_FAST        0 (answer)
          16 CALL              1
          26 POP_TOP

10          28 LOAD_CONST       1 (100)
          30 STORE_FAST       0 (answer)
          32 RETURN_CONST      0 (None)

```

The exact instruction names vary across Python versions, but the critical difference is stable:

- `show_answer()` uses a global lookup for `answer`;
- `broken_answer()` uses a fast-local lookup for `answer`.

The assignment caused `answer` to become a local variable in `broken_answer()`. Therefore, the read uses the local-variable access path. Since no local value exists yet, the read fails.

The code object also records local variable names.

```

answer = 42

def show_answer():
    print(answer)

```

```
def broken_answer():
    print(answer)
    answer = 100

print(f"show_answer co_varnames: {show_answer.__code__.co_varnames}")
print(f"show_answer co_names: {show_answer.__code__.co_names}")
print()

print(f"broken_answer co_varnames: {broken_answer.__code__.co_varnames}")
print(f"broken_answer co_names: {broken_answer.__code__.co_names}")
```

Possible output:

```
show_answer co_varnames: ()
show_answer co_names: ('print', 'answer')

broken_answer co_varnames: ('answer',)
broken_answer co_names: ('print',)
```

In `show_answer()`, `answer` appears in `co_names`, which is used for nonlocal/global-style name lookup.

In `broken_answer()`, `answer` appears in `co_varnames`, which means it is a local variable of the function.

So the failure is not:

Python looked for the global `answer` and failed.

The correct description is:

Python classified `answer` as local in this function, tried to read the local slot before assignment, and raised `UnboundLocalError`.

Analyzing the Mechanics of Conflicting Local and Global Names

A conflict appears when the same name exists globally and is also assigned inside a function.

```
score = 42

def update_score():
    score = 100
    print(f"inside: {score}")

update_score()

print(f"outside: {score}")
```

Possible output:

```
inside: 100
outside: 42
```

There is no error here because the function does not read the local name before assigning it. It creates a local `score`, prints that local value, and exits. The global `score` remains unchanged.

The conflict becomes an error when the function tries to read the name before the local assignment executes.

```

score = 42

def update_score():
    print(f"before: {score}")
    score = 100
    print(f"after: {score}")

try:
    update_score()
except UnboundLocalError as err:
    print(f"error type: {type(err)}")
    print(f"message: {err}")

print(f"outside: {score}")

```

Possible output:

```

error type: <class 'UnboundLocalError'>
message:    cannot access local variable 'score' where it is not associated with a value
outside: 42

```

The global `score` exists. That does not help, because the function's own assignment makes `score` local inside `update_score()`.

The compiled function confirms this.

```

import dis

score = 42

def update_score():
    print(f"before: {score}")
    score = 100
    print(f"after: {score}")

print(f"co_varnames: {update_score.__code__.co_varnames}")
print(f"co_names: {update_score.__code__.co_names}")
print()

dis.dis(update_score)

```

Possible output excerpt:

```

co_varnames: ('score',)
co_names:    ('print',)

... LOAD_FAST          0 (score)
... STORE_FAST         0 (score)

```

The name `score` is a local variable in the function. It is not loaded from the module namespace.

The solution depends on the intended design.

If the function should only compute a new value, return it.

```

score = 42

def make_updated_score(old_score):

```

```

    new_score = old_score + 58
    return new_score

```

```
score = make_updated_score(score)
```

```
print(f"score: {score}")
```

Possible output:

```
score: 100
```

This is usually the clearest data flow:

```

caller gives old object
function returns new object
caller rebinds its own name

```

If the function is deliberately supposed to rebind the module-level name, use `global`.

```
score = 42
```

```

def update_global_score():
    global score

    print(f"before: {score}")
    score = 100
    print(f"after:  {score}")

```

```
update_global_score()
```

```
print(f"outside: {score}")
```

Possible output:

```

before: 42
after:  100
outside: 100

```

Now `score` is no longer local inside the function.

```
score = 42
```

```

def update_global_score():
    global score

    print(f"before: {score}")
    score = 100
    print(f"after:  {score}")

print(f"co_varnames: {update_global_score.__code__.co_varnames}")
print(f"co_names:    {update_global_score.__code__.co_names}")

```

Possible output:

```

co_varnames: ()
co_names:    ('print', 'score')

```


The name `score` is no longer in `co_varnames`. It is resolved through the global name mechanism. A frequent confusion appears with augmented assignment.

```
score = 42
```

```
def add_points():
    score += 1
    print(score)

try:
    add_points()
except UnboundLocalError as err:
    print(f"error type: {type(err)}")
    print(f"message: {err}")
```

Possible output:

```
error type: <class 'UnboundLocalError'>
message: cannot access local variable 'score' where it is not associated with a value
```

The statement:

```
score += 1
```

is still an assignment to `score`. It requires reading the current local `score`, adding 1, and storing the result back into the local `score`. But the local value does not exist yet.

A simplified expansion is:

```
score = score + 1
```

This explains the failure.

The same idea applies to lists, but mutation and rebinding must be separated carefully.

This works without `global` because it mutates the global list object:

```
items = ["D'oh!"]

def add_item():
    items.append("No soup for you!")

add_item()

print(items)
```

Possible output:

```
["D'oh!", 'No soup for you!']
```

This fails because augmented assignment to the name counts as rebinding that name inside the function:

```
items = ["D'oh!"]

def add_items_bad():
    items += ["No soup for you!"]

try:
    add_items_bad()
except UnboundLocalError as err:
    print(f"error type: {type(err)}")
    print(f"message: {err}")
```

Possible output:

```
error type: <class 'UnboundLocalError'>
message:    cannot access local variable 'items' where it is not associated with a value
```

The method call:

```
items.append(...)
```

reads the outer name and mutates the reached list object.

The augmented assignment:

```
items += [...]
```

is treated as assignment to the name `items`. That makes `items` local unless declared `global` or `nonlocal`.

The list object itself can be inspected.

```
items = ["D'oh!"]
```

```
print(f"type(items): {type(items)}")
print()
```

```
selected_names = [
    "append",
    "extend",
    "__iadd__",
    "__add__",
    "__len__",
    "__getitem__",
    "__setitem__",
]
```

```
for name in selected_names:
    print(f"{name:<14} {name in dir(items)}")
```

Possible output:

```
type(items): <class 'list'>
```

```
append      True
extend      True
__iadd__    True
__add__     True
__len__     True
__getitem__ True
__setitem__ True
```

Even though lists support in-place addition, the statement syntax still assigns to the target name. That target assignment is enough for Python to classify the name as local.

The safest rule is:

If a function assigns to a name anywhere in its body, that name is local by default throughout that function body.

The exceptions are explicit:

- `global name` means assignment targets the module namespace;
- `nonlocal name` means assignment targets an enclosing function scope.

Name Resolution Failure: `NameError` vs. `UnboundLocalError`

`NameError` and `UnboundLocalError` are related, but they are not the same situation.

A plain `NameError` happens when Python cannot find a name in the lookup chain.

```
try:
    print(missing_name)
except NameError as err:
    print(f"error type: {type(err)}")
    print(f"message:    {err}")
```

Possible output:

```
error type: <class 'NameError'>
message:    name 'missing_name' is not defined
```

Python searched the available namespaces and did not find a binding for `missing_name`.

An `UnboundLocalError` happens when Python has already decided that the name is local, but the local slot has no value at the moment of reading.

```
x = 42
```

```
def bad_local():
    print(x)
    x = 100
```

```
try:
    bad_local()
except UnboundLocalError as err:
    print(f"error type: {type(err)}")
    print(f"message:    {err}")
```

Possible output:

```
error type: <class 'UnboundLocalError'>
message:    cannot access local variable 'x' where it is not associated with a value
```

The difference is:

Error	Meaning	Typical cause
<code>NameError</code>	no binding found in the lookup chain	name was never defined in accessible scope
<code>UnboundLocalError</code>	local name exists as a compiled local slot, but no value is bound yet	function assigns to the name later or conditionally

The relationship can be inspected directly.

```
print(f"issubclass(UnboundLocalError, NameError): {issubclass(UnboundLocalError, NameError)}")
print(f"issubclass(NameError, Exception):         {issubclass(NameError, Exception)}")
```

Possible output:

```
issubclass(UnboundLocalError, NameError): True
issubclass(NameError, Exception):         True
```

`UnboundLocalError` is a specialized kind of `NameError`. All unbound local failures are name-resolution failures, but not all name-resolution failures are unbound-local failures.

The exception classes are objects too.

```
print(f"NameError:           {NameError}")
print(f"type(NameError):     {type(NameError)}")
print()
print(f"UnboundLocalError:    {UnboundLocalError}")
print(f"type(UnboundLocalError): {type(UnboundLocalError)}")
print()

selected_names = [
    "__name__",
    "__mro__",
    "__bases__",
    "__subclasses__",
]

for name in selected_names:
    print(f"{name:<18} {name in dir(UnboundLocalError)}")

print()
print(f"UnboundLocalError.__mro__: {UnboundLocalError.__mro__}")
```

Possible output:

```
NameError:           <class 'NameError'>
type(NameError):     <class 'type'>

UnboundLocalError:    <class 'UnboundLocalError'>
type(UnboundLocalError): <class 'type'>

__name__             True
__mro__               True
__bases__             True
__subclasses__        True

UnboundLocalError.__mro__: (<class 'UnboundLocalError'>, <class 'NameError'>, <class 'Exception'>,
```

The method resolution order shows the inheritance chain. `UnboundLocalError` is under `NameError`.

A conditional assignment can also cause `UnboundLocalError`.

```
def choose_value(flag):
    if flag:
        value = 42

    return value

print(choose_value(True))

try:
    print(choose_value(False))
except UnboundLocalError as err:
    print(f"error type: {type(err)}")
    print(f"message:     {err}")
```

Possible output:

```
42
error type: <class 'UnboundLocalError'>
message:    cannot access local variable 'value' where it is not associated with a value
```

The assignment target `value` exists syntactically inside the function, so `value` is a local variable. But when `flag` is `False`, the assignment line does not execute. The function later tries to return an unbound local name.

A safer version binds a default first.

```
def choose_value(flag):
    value = None

    if flag:
        value = 42

    return value

print(choose_value(True))
print(choose_value(False))
```

Possible output:

```
42
None
```

Now the local name `value` is always bound before it is returned. Another safe version returns directly from each branch.

```
def choose_value(flag):
    if flag:
        return 42

    return None

print(choose_value(True))
print(choose_value(False))
```

Possible output:

```
42
None
```

A missing global and an unbound local can be contrasted with bytecode.

```
import dis

def missing_global():
    return unknown_name

def unbound_local():
    print(value)
    value = 42

print("missing_global:")
```

```

print(f"co_varnames: {missing_global.__code__.co_varnames}")
print(f"co_names:     {missing_global.__code__.co_names}")
dis.dis(missing_global)

print()
print("unbound_local:")
print(f"co_varnames: {unbound_local.__code__.co_varnames}")
print(f"co_names:     {unbound_local.__code__.co_names}")
dis.dis(unbound_local)

```

Possible output excerpt:

```

missing_global:
co_varnames: ()
co_names:    ('unknown_name',)
... LOAD_GLOBAL             0 (unknown_name)

unbound_local:
co_varnames: ('value',)
co_names:    ('print',)
... LOAD_FAST               0 (value)
... STORE_FAST              0 (value)

```

The first function performs a global-style name lookup and fails with `NameError` if no accessible binding exists. The second function performs a local-slot load and fails with `UnboundLocalError` if the local slot has not yet been assigned.

The compact rule is:

`NameError` means no accessible binding was found. `UnboundLocalError` means the binding was classified as local, but no local value existed at the read point.

8.5.2 Compile-Time Scope Disambiguation

Python name lookup feels dynamic because values are resolved while the program runs. But the classification of many names is decided earlier, when Python compiles the function body.

Before a function runs, Python has already inspected its syntax and decided which names are local, which names are global-style lookups, which names are free variables from enclosing scopes, and which names must be stored in closure cells.

This distinction is essential:

- name **classification** is mostly static and syntax-driven;
- object **value lookup** is dynamic and happens at runtime.

That is why a function can fail with `UnboundLocalError` before it ever considers a global name with the same spelling. The compiler has already decided that the name is local.

How the Python Compiler Scans Syntax Trees for Assignment Targets

Python first parses source code into a syntax tree. That tree contains assignment targets, reads, function definitions, calls, literals, and other syntactic structures.

The standard `ast` module lets us inspect this representation.

```

import ast

source = """

```

```

answer = 42

def broken():
    print(answer)
    answer = 100
"""

tree = ast.parse(source)

print(f"type(tree): {type(tree)}")
print()
print(ast.dump(tree, indent=4))

```

The output is long, but the important part contains two different uses of `answer` inside `broken()`:

```

Name(id='answer', ctx=Load())
Name(id='answer', ctx=Store())

```

The `Load()` context means the name is being read. The `Store()` context means the name is an assignment target.

The AST object can be inspected.

```

import ast

tree = ast.parse("x = 42")

print(f"type(tree): {type(tree)}")
print()

selected_names = [
    "body",
    "type_ignores",
    "_fields",
    "__class__",
]

for name in selected_names:
    print(f"{name:<14} {name in dir(tree)}")

print()
print(f"tree.body: {tree.body}")

```

Possible output:

```

type(tree): <class 'ast.Module'>

body           True
type_ignores   True
_fields        True
__class__      True

tree.body: [<ast.Assign object at 0x...>]

```

The first statement is an `ast.Assign` object.

```

import ast

tree = ast.parse("x = 42")
assign_node = tree.body[0]

print(f"assign_node:      {assign_node}")
print(f"type(assign_node): {type(assign_node)}")
print()

selected_names = [
    "targets",
    "value",
    "lineno",
    "col_offset",
    "_fields",
]

for name in selected_names:
    print(f"{name:<14} {name in dir(assign_node)}")

print()
print(f"targets: {assign_node.targets}")
print(f"value:   {assign_node.value}")

```

Possible output:

```

assign_node:      <ast.Assign object at 0x...>
type(assign_node): <class 'ast.Assign'>

targets          True
value             True
lineno           True
col_offset       True
_fields          True

targets: [<ast.Name object at 0x...>]
value:   <ast.Constant object at 0x...>

```

The assignment target is an `ast.Name` object with `Store` context.

```

import ast

tree = ast.parse("x = 42")
target_node = tree.body[0].targets[0]

print(f"type(target_node): {type(target_node)}")
print()

selected_names = [
    "id",
    "ctx",
    "lineno",
    "col_offset",
    "_fields",
]

```



```

for name in selected_names:
    print(f"{name:<12} {name in dir(target_node)}")

print()
print(f"target_node.id: {target_node.id!r}")
print(f"target_node.ctx: {target_node.ctx}")
print(f"type(ctx):      {type(target_node.ctx)}")

```

Possible output:

```

type(target_node): <class 'ast.Name'>

id          True
ctx         True
lineno      True
col_offset  True
_fields     True

target_node.id: 'x'
target_node.ctx: <ast.Store object at 0x...>
type(ctx):      <class 'ast.Store'>

```

Now compare a name read.

```

import ast

tree = ast.parse("print(x)")
call_node = tree.body[0].value
name_node = call_node.args[0]

print(f"type(name_node): {type(name_node)}")
print(f"name_node.id:     {name_node.id!r}")
print(f"name_node.ctx:    {name_node.ctx}")
print(f"type(ctx):       {type(name_node.ctx)}")

```

Possible output:

```

type(name_node): <class 'ast.Name'>
name_node.id:    'x'
name_node.ctx:   <ast.Load object at 0x...>
type(ctx):       <class 'ast.Load'>

```

The compiler uses this kind of syntactic information to decide scope categories. A name that appears as an assignment target inside a function is local by default, unless a `global` or `nonlocal` declaration changes that rule.

This is true even if the assignment is textually after the read.

```

answer = 42

def broken():
    print(answer)
    answer = 100

try:

```

```

    broken()
except UnboundLocalError as err:
    print(f"error type: {type(err)}")
    print(f"message:    {err}")

```

Possible output:

```

error type: <class 'UnboundLocalError'>
message:    cannot access local variable 'answer' where it is not associated with a value

```

The compiler saw the assignment target `answer` in the function body. Therefore, `answer` became a local variable for that function. The earlier read tries to read the local slot before a value has been stored there.

A dead-looking assignment still affects classification.

```

answer = 42

def broken():
    if False:
        answer = 100

    return answer

try:
    print(broken())
except UnboundLocalError as err:
    print(f"error type: {type(err)}")
    print(f"message:    {err}")

```

Possible output:

```

error type: <class 'UnboundLocalError'>
message:    cannot access local variable 'answer' where it is not associated with a value

```

The branch does not execute, but the assignment target exists in the function syntax. The compiler does not need the branch to run before it classifies `answer` as local.

The code object reveals the decision.

```

answer = 42

def broken():
    if False:
        answer = 100

    return answer

print(f"co_varnames: {broken.__code__.co_varnames}")
print(f"co_names:    {broken.__code__.co_names}")

```

Possible output:

```

co_varnames: ('answer',)
co_names:    ()

```

The name `answer` is a local variable of `broken()`, even though the assignment never runs. The rule is:

For ordinary function bodies, assignment syntax classifies a name as local for the whole function unless `global` or `nonlocal` overrides that classification.

Pre-Determining Local Scope Allocation Flags via Symbol Tables Prior to Execution

The standard `symtable` module exposes Python's symbol-table analysis. This is closer to the compiler's scope reasoning than plain AST inspection.

```
import symtable

source = """
answer = 42

def broken():
    print(answer)
    answer = 100
"""

table = symtable.symtable(source, "demo.py", "exec")

print(f"table:      {table}")
print(f"type(table): {type(table)}")
```

Possible output:

```
table:      <SymbolTable for module demo.py>
type(table): <class 'symtable.SymbolTable'>
```

The symbol table object can be inspected.

```
import symtable

source = """
answer = 42

def broken():
    print(answer)
    answer = 100
"""

table = symtable.symtable(source, "demo.py", "exec")

print(f"type(table): {type(table)}")
print()

selected_names = [
    "get_name",
    "get_type",
    "get_symbols",
    "get_children",
    "lookup",
    "is_nested",
]

for name in selected_names:
    print(f"{name:<16} {name in dir(table)}")

print()
```

```
print(f"table name: {table.get_name()}")
print(f"table type: {table.get_type()}")
```

Possible output:

```
type(table): <class 'symtable.SymbolTable'>
```

```
get_name      True
get_type      True
get_symbols   True
get_children  True
lookup        True
is_nested     True
```

```
table name: top
table type: module
```

The module-level symbol table has a child table for the function.

```
import symtable

source = """
answer = 42

def broken():
    print(answer)
    answer = 100
"""

module_table = symtable.symtable(source, "demo.py", "exec")
children = module_table.get_children()

print(f"children:      {children}")
print(f"type(children): {type(children)}")
print()

func_table = children[0]

print(f"func name: {func_table.get_name()}")
print(f"func type: {func_table.get_type()}")
```

Possible output:

```
children:      [<Function SymbolTable for broken in demo.py>]
type(children): <class 'list'>

func name: broken
func type: function
```

The child list is a normal list.

```
import symtable

source = """
def broken():
```

```

        answer = 100
    """

module_table = symtable.symtable(source, "demo.py", "exec")
children = module_table.get_children()

print(f"type(children): {type(children)}")
print()

selected_names = [
    "append",
    "extend",
    "pop",
    "__len__",
    "__getitem__",
    "__iter__",
]

for name in selected_names:
    print(f"{name:<14} {name in dir(children)}")

print()
print(f"len(children): {len(children)}")

```

Possible output:

```

type(children): <class 'list'>

append          True
extend          True
pop             True
__len__          True
__getitem__      True
__iter__         True

len(children): 1

```

Now inspect the symbol for `answer` inside the function.

```

import symtable

source = """
answer = 42

def broken():
    print(answer)
    answer = 100
"""

module_table = symtable.symtable(source, "demo.py", "exec")
func_table = module_table.get_children()[0]

symbol = func_table.lookup("answer")

print(f"symbol:      {symbol}")

```

```

print(f"type(symbol): {type(symbol)}")
print()

selected_names = [
    "get_name",
    "is_local",
    "is_global",
    "is_referenced",
    "is_assigned",
    "is_free",
    "is_parameter",
]

for name in selected_names:
    print(f"{name:<18} {name in dir(symbol)}")

print()
print(f"name:           {symbol.get_name()}")
print(f"is_local:       {symbol.is_local()}")
print(f"is_global:      {symbol.is_global()}")
print(f"is_referenced: {symbol.is_referenced()}")
print(f"is_assigned:   {symbol.is_assigned()}")
print(f"is_free:       {symbol.is_free()}")

```

Possible output:

```

symbol:           <symbol 'answer'>
type(symbol):    <class 'symtable.Symbol'>

get_name         True
is_local         True
is_global        True
is_referenced    True
is_assigned      True
is_free          True
is_parameter     True

name:            answer
is_local:        True
is_global:       False
is_referenced:   True
is_assigned:     True
is_free:         False

```

The symbol table reports that `answer` is local and assigned inside the function. This happens before the function is ever called.

Compare a function that only reads `answer`.

```

import symtable

source = """
answer = 42

def show():
    print(answer)

```

```

"""

module_table = symtable.symtable(source, "demo.py", "exec")
func_table = module_table.get_children()[0]
symbol = func_table.lookup("answer")

print(f"name:           {symbol.get_name()}")
print(f"is_local:       {symbol.is_local()}")
print(f"is_global:      {symbol.is_global()}")
print(f"is_referenced: {symbol.is_referenced()}")
print(f"is_assigned:   {symbol.is_assigned()}")
print(f"is_free:       {symbol.is_free()}")

```

Possible output:

```

name:           answer
is_local:       False
is_global:      True
is_referenced:  True
is_assigned:    False
is_free:        False

```

Now `answer` is classified as global from the function's point of view.
The compiled code objects agree with the symbol table.

```

answer = 42

def show():
    print(answer)

def broken():
    print(answer)
    answer = 100

print("show")
print(f"co_varnames: {show.__code__.co_varnames}")
print(f"co_names:     {show.__code__.co_names}")
print()

print("broken")
print(f"co_varnames: {broken.__code__.co_varnames}")
print(f"co_names:     {broken.__code__.co_names}")

```

Possible output:

```

show
co_varnames: ()
co_names:    ('print', 'answer')

broken
co_varnames: ('answer',)
co_names:    ('print',)

```

The symbol-table decision affects the bytecode instruction choice.

```
import dis

answer = 42

def show():
    print(answer)

def broken():
    print(answer)
    answer = 100

print("show:")
dis.dis(show)

print()
print("broken:")
dis.dis(broken)
```

Possible output excerpt:

```
show:
... LOAD_GLOBAL              ... (answer)

broken:
... LOAD_FAST                ... (answer)
... STORE_FAST               ... (answer)
```

The exact bytecode format depends on Python version, but the difference remains:

- global-style names use global lookup instructions;
- local names use local-slot instructions.

The symbol table is therefore a compile-time planning artifact. It pre-determines how names will be accessed when the function eventually runs.

Static Name Binding Invariants vs. Dynamic Late-Binding Value Resolution

The classification of a name is static for a compiled function. The value found through that classification may still be dynamic.

A function that reads a global name does not freeze that global value when the function is defined.

```
message = "D'oh!"

def show_message():
    print(message)

show_message()

message = "No soup for you!"

show_message()
```

Possible output:

```
D'oh!
No soup for you!
```


The function `show_message()` performs a global lookup each time it is called. It does not store the original string as a hidden constant.

The code object shows that `message` is a global-style name.

```
message = "D'oh!"

def show_message():
    print(message)

print(f"co_varnames: {show_message.__code__.co_varnames}")
print(f"co_names:     {show_message.__code__.co_names}")
```

Possible output:

```
co_varnames: ()
co_names:    ('print', 'message')
```

The classification is fixed:

`message` is looked up as a global name

The value is not fixed:

whatever object the module namespace binds to `message` at call time is used

This is late binding of values.

The same idea appears with functions.

```
def action():
    return "first action"

def run_action():
    print(action())

run_action()

def action():
    return "second action"

run_action()
```

Possible output:

```
first action
second action
```

The function `run_action()` looks up the global name `action` when it runs. After the second `def action():` executes, the global name `action` refers to a different function object.

The function object can be inspected before and after rebinding.

```
def action():
    return "first action"

old_action = action

def action():
```

```

    return "second action"

print(f"old_action: {old_action}")
print(f"action:      {action}")
print(f"same object: {old_action is action}")
print()
print(f"old_action(): {old_action()}")
print(f"action():      {action()}")

```

Possible output:

```

old_action: <function action at 0x...>
action:      <function action at 0x...>
same object: False

```

```

old_action(): first action
action():      second action

```

The global name was rebound. Code that looks up the global name later sees the new object. This contrasts with local classification.

```

message = "global message"

def broken():
    print(message)
    message = "local message"

try:
    broken()
except UnboundLocalError as err:
    print(f"error type: {type(err)}")
    print(f"message:      {err}")

```

Possible output:

```

error type: <class 'UnboundLocalError'>
message:    cannot access local variable 'message' where it is not associated with a value

```

Even though the global `message` exists at runtime, the compiled function does not use the global lookup path for `message`. The local classification is fixed by syntax.

The invariant is:

The compiler decides the namespace access category. Runtime execution retrieves the current object from that category.

A default argument behaves differently from a global lookup. The default object is stored when the function is defined.

```

message = "D'oh!"

def show_default(text=message):
    print(text)

show_default()

```

```
message = "No soup for you!"
```

```
show_default()
show_default(message)
```

Possible output:

```
D'oh!
D'oh!
No soup for you!
```

The default value captured the old object at definition time. The global lookup in a function body would see the new object at call time.

Compare both in one example.

```
message = "D'oh!"

def show_both(text=message):
    print(f"default text: {text}")
    print(f"global name: {message}")

show_both()

message = "No soup for you!"

print()
show_both()
```

Possible output:

```
default text: D'oh!
global name: D'oh!

default text: D'oh!
global name: No soup for you!
```

The default `text` is stored in `__defaults__`. The name `message` inside the function body is looked up dynamically as a global.

```
message = "D'oh!"

def show_both(text=message):
    print(text)
    print(message)

print(f"defaults:      {show_both.__defaults__}")
print(f"co_varnames: {show_both.__code__.co_varnames}")
print(f"co_names:      {show_both.__code__.co_names}")
```

Possible output:

```
defaults:      ("D'oh!",)
co_varnames:   ('text',)
co_names:      ('print', 'message')
```

This example separates three ideas:

- `text` is a local parameter;
- its default object was stored at definition time;
- `message` in the body is a global lookup resolved at call time.

So Python is not simply “dynamic” or “static” in one flat sense. Its behavior is split:

Mechanism	When it is decided
Local/global/free/built-in classification	compile time / function creation path
Default argument object creation	definition time, when <code>def</code> executes
Global value lookup inside function body	call time, when instruction executes
Local value availability	call time, depending on executed assignments
Closure cell value lookup	call time, through preserved cell object

The important practical rule is:

The place Python will search is decided early. The object found in that place may still change before the search happens.

The Loop Variable Closure Trap: Why Nested Functions May See the Final Loop Value

A common closure trap appears when nested functions are created inside a loop.

```
funcs = []

for i in range(3):
    def show_i():
        return i

    funcs.append(show_i)

for fn in funcs:
    print(fn())
```

Possible output:

```
2
2
2
```

Many beginners expect:

```
0
1
2
```

But all three functions refer to the same name `i`. They do not each receive a frozen copy of the loop value.

At module level, `i` is a global name. After the loop finishes, `i` is bound to 2. Each function performs a global lookup of `i` when called.

This can be inspected.

```

funcs = []

for i in range(3):
    def show_i():
        return i

    funcs.append(show_i)

fn = funcs[0]

print(f"fn(): {fn()}")
print(f"i after loop: {i}")
print()
print(f"fn.__code__.co_varnames: {fn.__code__.co_varnames}")
print(f"fn.__code__.co_names:      {fn.__code__.co_names}")
print(f"fn.__closure__:           {fn.__closure__}")

```

Possible output:

```

fn(): 2
i after loop: 2

fn.__code__.co_varnames: ()
fn.__code__.co_names:    ('i',)
fn.__closure__:          None

```

At module level, the nested-looking function is not inside another function scope. It uses global lookup for `i`.

Inside an outer function, the trap uses closure cells.

```

def make_funcs():
    funcs = []

    for i in range(3):
        def show_i():
            return i

        funcs.append(show_i)

    return funcs

funcs = make_funcs()

for fn in funcs:
    print(fn())

```

Possible output:

```

2
2
2

```

Now `i` is not global. It is a local variable of `make_funcs()` that becomes a closure cell because `show_i()` refers to it.

All three inner functions share that same cell.

```

def make_funcs():
    funcs = []

    for i in range(3):
        def show_i():
            return i

        funcs.append(show_i)

    return funcs

funcs = make_funcs()

for index, fn in enumerate(funcs):
    cell = fn.__closure__[0]

    print(f"function {index}")
    print(f"  result:           {fn()}")
    print(f"  closure:           {fn.__closure__}")
    print(f"  freevars:          {fn.__code__.co_freevars}")
    print(f"  cell contents:     {cell.cell_contents}")
    print(f"  cell id:           {id(cell)}")
    print()

```

Possible output:

```

function 0
result:           2
closure:         (<cell at 0x...: int object at 0x...>,)
freevars:        ('i',)
cell contents:   2
cell id:         2293100811520

function 1
result:           2
closure:         (<cell at 0x...: int object at 0x...>,)
freevars:        ('i',)
cell contents:   2
cell id:         2293100811520

function 2
result:           2
closure:         (<cell at 0x...: int object at 0x...>,)
freevars:        ('i',)
cell contents:   2
cell id:         2293100811520

```

The same cell id appears for all three functions. They share one closure cell for `i`. After the loop ends, that cell contains the final loop value.

The cell object is a normal inspectable object.

```

def make_funcs():
    funcs = []

    for i in range(3):

```

```

    def show_i():
        return i

    funcs.append(show_i)

return funcs

cell = make_funcs()[0].__closure__[0]

print(f"type(cell): {type(cell)}")
print()

selected_names = [
    "cell_contents",
    "__class__",
    "__eq__",
    "__repr__",
]

for name in selected_names:
    print(f"{name:<16} {name in dir(cell)}")

print()
print(f"cell.cell_contents: {cell.cell_contents}")

```

Possible output:

```

type(cell): <class 'cell'>

cell_contents      True
__class__          True
__eq__             True
__repr__           True

cell.cell_contents: 2

```

The problem is not that the loop calls the functions too late in a vague sense. The exact mechanism is:

1. The loop repeatedly rebinds the same name `i`.
2. Each inner function refers to that same variable `cell`.
3. The functions are called after the loop has finished.
4. The `cell` now contains the final value, 2.
5. Every function reads that same `cell`.

A common fix is to capture the current value as a default argument.

```

def make_funcs():
    funcs = []

    for i in range(3):
        def show_i(i=i):
            return i

        funcs.append(show_i)

```

```

    return funcs

funcs = make_funcs()

for fn in funcs:
    print(fn())

```

Possible output:

```

0
1
2

```

This works because default argument objects are evaluated when the `def` statement executes. Each loop iteration executes a new `def` statement and stores the current object referenced by `i` as that function's default.

Inspect the functions.

```

def make_funcs():
    funcs = []

    for i in range(3):
        def show_i(i=i):
            return i

        funcs.append(show_i)

    return funcs

funcs = make_funcs()

for index, fn in enumerate(funcs):
    print(f"function {index}")
    print(f"  result:      {fn()}")
    print(f"  defaults:      {fn.__defaults__}")
    print(f"  varnames:       {fn.__code__.co_varnames}")
    print(f"  freevars:       {fn.__code__.co_freevars}")
    print(f"  closure:        {fn.__closure__}")
    print()

```

Possible output:

```

function 0
  result:      0
  defaults:    (0,)
  varnames:    ('i',)
  freevars:    ()
  closure:     None

function 1
  result:      1
  defaults:    (1,)
  varnames:    ('i',)
  freevars:    ()

```



```

closure:      None

function 2
  result:      2
  defaults:    (2,)
  varnames:    ('i',)
  freevars:    ()
  closure:     None

```

Now each function has its own local parameter `i` with its own stored default. There is no shared closure cell for `i`.

Another fix is to use a factory function. The factory creates a new enclosing scope for each value.

```

def make_one_func(value):
    def show_value():
        return value

    return show_value

funcs = []

for i in range(3):
    funcs.append(make_one_func(i))

for fn in funcs:
    print(fn())

```

Possible output:

```

0
1
2

```

Each call to `make_one_func(i)` creates a separate frame and a separate closure cell for `value`.

```

def make_one_func(value):
    def show_value():
        return value

    return show_value

funcs = []

for i in range(3):
    funcs.append(make_one_func(i))

for index, fn in enumerate(funcs):
    cell = fn.__closure__[0]

    print(f"function {index}")
    print(f"  result:      {fn()}")
    print(f"  freevars:     {fn.__code__.co_freevars}")
    print(f"  cell contents: {cell.cell_contents}")
    print(f"  cell id:      {id(cell)}")
    print()

```

Possible output:

```
function 0
  result:      0
  freevars:    ('value',)
  cell contents: 0
  cell id:     2293100811520

function 1
  result:      1
  freevars:    ('value',)
  cell contents: 1
  cell id:     2293100811392

function 2
  result:      2
  freevars:    ('value',)
  cell contents: 2
  cell id:     2293100811648
```

Now the cell ids are different. Each returned function closes over a different cell.

The loop variable closure trap is therefore not a special loop bug. It is a direct consequence of two ordinary rules:

- closures preserve variables through cells, not frozen textual values;
- the value inside a cell is read when the inner function runs, not when the inner function is created.

The compact rule is:

If a nested function refers to a loop variable, it usually refers to the variable itself, not a snapshot of its value. Use a default argument or a factory function when each generated function must remember a separate value.

8.6 First-Class Functions, Closures, and Function Transformation

Once functions are understood as runtime objects, they can no longer be treated as mere named blocks of code. A function object can be stored, passed, returned, wrapped, replaced, and inspected. This turns executable behavior into data that can move through the program.

This section studies that shift in three stages. First, it explains first-class functions and higher-order functions: functions that receive or return other functions. Then it explains closures, where returned inner functions preserve selected state from enclosing scopes after the outer function call has ended. Finally, it explains decorators as definition-time function transformations built from the same mechanisms.

The central idea is that Python does not need a separate special system for callbacks, closures, or decorators. They all follow from the same object model: a function is an object, a name is a binding, and rebinding a name can replace one executable object with another.

8.6.1 First-Class Citizens and Higher-Order Functions

A Python function is an object. Since it is an object, it can be stored in a variable, placed inside a list or dictionary, passed as an argument, returned from another function, and inspected at runtime.

This is what people usually mean when they say that functions are *first-class citizens*. The phrase sounds abstract, but the mechanism is concrete:

A function object can travel through the program like other objects.

A higher-order function is simply a function that receives another function, returns another function, or both.

Functions as In-Memory Objects: Passing, Returning, and Storing Subroutine References inside Variables

A function definition creates a function object and binds a name to it.

```
def say_doh():
    return "D'oh!"

print(f"say_doh:      {say_doh}")
print(f"type(say_doh): {type(say_doh)}")
print(f"say_doh():    {say_doh()}")
```

Possible output:

```
say_doh:      <function say_doh at 0x...>
type(say_doh): <class 'function'>
say_doh():    D'oh!
```

The name `say_doh` refers to a function object. Writing `say_doh` uses the object. Writing `say_doh()` calls the object.

This distinction is essential.

```
def say_doh():
    return "D'oh!"

fn = say_doh

print(f"fn:      {fn}")
print(f"type(fn): {type(fn)}")
print(f"fn():    {fn()}")
print()
print(f"fn is say_doh: {fn is say_doh}")
```

Possible output:

```
fn:      <function say_doh at 0x...>
type(fn): <class 'function'>
fn():    D'oh!
```

```
fn is say_doh: True
```

The assignment:

```
fn = say_doh
```

does not call the function. It copies the function reference into another name. Both names now refer to the same function object.

The function object can be inspected.

```

def say_doh():
    """Return one short line."""
    return "D'oh!"

fn = say_doh

print(f"type(fn): {type(fn)}")
print()

selected_names = [
    "__call__",
    "__name__",
    "__doc__",
    "__defaults__",
    "__kwdefaults__",
    "__annotations__",
    "__code__",
]

for name in selected_names:
    print(f"{name:<18} {name in dir(fn)}")

print()
print(f"fn.__name__: {fn.__name__!r}")
print(f"fn.__doc__: {fn.__doc__!r}")

```

Possible output:

```

type(fn): <class 'function'>

__call__      True
__name__      True
__doc__       True
__defaults__  True
__kwdefaults__ True
__annotations__ True
__code__      True

fn.__name__: 'say_doh'
fn.__doc__: 'Return one short line.'

```

The object remains the same function object even if several names point to it.
Functions can be stored inside a list.

```

def quote_one():
    return "D'oh!"

def quote_two():
    return "No soup for you!"

def quote_three():
    return "Nobody expects the Spanish Inquisition!"

actions = [quote_one, quote_two, quote_three]

```

```

print(f"actions:      {actions}")
print(f"type(actions): {type(actions)}")
print()

for fn in actions:
    print(f"{fn.__name__:<12} -> {fn()}")

```

Possible output:

```

actions:      [<function quote_one at 0x...>, <function quote_two at 0x...>, <function quote_three at 0x...>]
type(actions): <class 'list'>

quote_one     -> D'oh!
quote_two     -> No soup for you!
quote_three   -> Nobody expects the Spanish Inquisition!

```

The list contains references to function objects. The loop binds `fn` to each function object and then calls it with `fn()`.

The list itself is an ordinary list.

```

def quote_one():
    return "D'oh!"

def quote_two():
    return "No soup for you!"

actions = [quote_one, quote_two]

print(f"type(actions): {type(actions)}")
print()

selected_names = [
    "append",
    "extend",
    "pop",
    "__len__",
    "__getitem__",
    "__setitem__",
]

for name in selected_names:
    print(f"{name:<14} {name in dir(actions)}")

print()
print(f"len(actions): {len(actions)}")
print(f"actions[0]:    {actions[0]}")
print(f"actions[0]() : {actions[0]()}")

```

Possible output:

```

type(actions): <class 'list'>

append        True
extend        True
pop            True

```

```

__len__      True
__getitem__  True
__setitem__  True

len(actions): 2
actions[0]:   <function quote_one at 0x...>
actions[0]() : D'oh!

```

There is no special list of functions type. It is a normal list whose elements happen to be function objects.

Functions can also be stored inside dictionaries.

```

def small_quote():
    return "Ni!"

def big_quote():
    return "Chuck Norris can divide by zero."

def answer_quote():
    return "The answer is 42."

commands = {
    "small": small_quote,
    "big": big_quote,
    "answer": answer_quote,
}

for key in ["small", "answer", "big"]:
    fn = commands[key]
    print(f"{key:<8} -> {fn()}")

```

Possible output:

```

small    -> Ni!
answer   -> The answer is 42.
big      -> Chuck Norris can divide by zero.

```

The dictionary maps strings to function objects. Looking up a key returns a function object, and the returned function object can then be called.

The dictionary is ordinary too.

```

def small_quote():
    return "Ni!"

commands = {
    "small": small_quote,
}

print(f"type(commands): {type(commands)}")
print()

selected_names = [
    "keys",
    "values",
    "items",

```

```

    "get",
    "__getitem__",
    "__setitem__",
]

for name in selected_names:
    print(f"{name:<14} {name in dir(commands)}")

print()
fn = commands.get("small")

print(f"fn:          {fn}")
print(f"type(fn):    {type(fn)}")
print(f"fn():        {fn()}")

```

Possible output:

```

type(commands): <class 'dict'>

keys           True
values         True
items          True
get            True
__getitem__    True
__setitem__    True

fn:            <function small_quote at 0x...>
type(fn):      <class 'function'>
fn():          Ni!

```

A function can also return another function object.

```

def quote_one():
    return "D'oh!"

def quote_two():
    return "No soup for you!"

def choose_quote(kind):
    if kind == "simpsons":
        return quote_one

    return quote_two

selected_fn = choose_quote("simpsons")

print(f"selected_fn:      {selected_fn}")
print(f"type(selected_fn): {type(selected_fn)}")
print(f"selected_fn():     {selected_fn()}")

```

Possible output:

```

selected_fn:      <function quote_one at 0x...>
type(selected_fn): <class 'function'>
selected_fn():    D'oh!

```

The function `choose_quote()` does not return the result of calling `quote_one`. It returns the function object `quote_one` itself.

The difference is visible here:

```
def quote_one():
    return "D'oh!"

def return_function_object():
    return quote_one

def return_function_result():
    return quote_one()

a = return_function_object()
b = return_function_result()

print(f"a:      {a}")
print(f"type(a): {type(a)}")
print(f"a():    {a()}")
print()

print(f"b:      {b!r}")
print(f"type(b): {type(b)}")
```

Possible output:

```
a:      <function quote_one at 0x...>
type(a): <class 'function'>
a():    D'oh!

b:      "D'oh!"
type(b): <class 'str'>
```

The expression `quote_one` produces the function object. The expression `quote_one()` calls the function and produces its return value.

The first-class rule is therefore not philosophical. It is ordinary object-reference behavior:

```
name = function_object
list_item = function_object
dict_value = function_object
return function_object
call function_object later
```

Callbacks and Callback Chains: Decoupling Structural Execution Graphs

A callback is a function object passed into another function so the receiving function can call it later.

The callback function does not control the outer structure. It supplies a piece of behavior that the outer function uses at a chosen point.

```
def shout(text):
    return text.upper()

def whisper(text):
    return text.lower()
```



```
def process_line(text, callback):
    print(f"original: {text!r}")

    result = callback(text)

    print(f"processed: {result!r}")
    return result

process_line("No soup for you!", shout)
print()
process_line("No soup for you!", whisper)
```

Possible output:

```
original: 'No soup for you!'
processed: 'NO SOUP FOR YOU!'

original: 'No soup for you!'
processed: 'no soup for you!'
```

The function `process_line()` controls the execution frame:

```
print original text
call callback
print processed text
return processed text
```

The callback controls only the transformation step.
The callback is a normal function object.

```
def shout(text):
    return text.upper()

def process_line(text, callback):
    print(f"callback: {callback}")
    print(f"type(callback): {type(callback)}")
    print(f"callback name: {callback.__name__}")

    return callback(text)

result = process_line("D'oh!", shout)

print()
print(f"result: {result}")
```

Possible output:

```
callback: <function shout at 0x...>
type(callback): <class 'function'>
callback name: shout

result: D'OH!
```

The parameter name `callback` is just a local name bound to a function object supplied by the caller.

Callbacks are useful because they separate the stable structure from the replaceable behavior.

```

def format_plain(text):
    return text

def format_boxed(text):
    return f"[ {text} ]"

def format_numbered(text):
    return f"42. {text}"

def print_report_line(text, formatter):
    formatted = formatter(text)
    print(formatted)

print_report_line("It is only a model.", format_plain)
print_report_line("It is only a model.", format_boxed)
print_report_line("It is only a model.", format_numbered)

```

Possible output:

```

It is only a model.
[ It is only a model. ]
42. It is only a model.

```

The outer function does not need if branches for every formatting strategy. It receives the strategy as a function object.

A callback chain applies several callback functions in order.

```

def strip_text(text):
    return text.strip()

def upper_text(text):
    return text.upper()

def add_suffix(text):
    return text + "!"

def run_chain(text, callbacks):
    current = text

    for callback in callbacks:
        print(f"before {callback.__name__:<12}: {current!r}")
        current = callback(current)

    print(f"final result:      {current!r}")
    return current

steps = [strip_text, upper_text, add_suffix]

run_chain("  ni  ", steps)

```

Possible output:

```

before strip_text : '  ni  '
before upper_text : 'ni'
before add_suffix : 'NI'
final result:     'NI!'

```

The list `steps` stores function objects. The function `run_chain()` walks through that list and calls each function object in sequence.

This builds a small execution graph:

```
input text
  -> strip_text
  -> upper_text
  -> add_suffix
  -> final output
```

The chain itself is data. It can be changed without rewriting `run_chain()`.

```
def strip_text(text):
    return text.strip()

def upper_text(text):
    return text.upper()

def lower_text(text):
    return text.lower()

def add_suffix(text):
    return text + "!"

def run_chain(text, callbacks):
    current = text

    for callback in callbacks:
        current = callback(current)

    return current

loud_chain = [strip_text, upper_text, add_suffix]
quiet_chain = [strip_text, lower_text]

print(run_chain("  No Soup For You  ", loud_chain))
print(run_chain("  No Soup For You  ", quiet_chain))
```

Possible output:

```
NO SOUP FOR YOU!
no soup for you
```

The code that runs the chain is unchanged. Only the list of function objects changes. The chain list can be inspected.

```
def strip_text(text):
    return text.strip()

def upper_text(text):
    return text.upper()

def add_suffix(text):
    return text + "!"
```

```

steps = [strip_text, upper_text, add_suffix]

print(f"type(steps): {type(steps)}")
print()

for index, fn in enumerate(steps):
    print(f"index={index}")
    print(f"  object: {fn}")
    print(f"  type:   {type(fn)}")
    print(f"  name:   {fn.__name__}")

```

Possible output:

```

type(steps): <class 'list'>

index=0
  object: <function strip_text at 0x...>
  type:   <class 'function'>
  name:   strip_text
index=1
  object: <function upper_text at 0x...>
  type:   <class 'function'>
  name:   upper_text
index=2
  object: <function add_suffix at 0x...>
  type:   <class 'function'>
  name:   add_suffix

```

Callbacks can also be selected from a dictionary.

```

def as_upper(text):
    return text.upper()

def as_lower(text):
    return text.lower()

def as_title(text):
    return text.title()

formatters = {
    "upper": as_upper,
    "lower": as_lower,
    "title": as_title,
}

mode = "title"
text = "these pretzels are making me thirsty"

formatter = formatters[mode]
result = formatter(text)

print(f"mode:      {mode}")
print(f"formatter: {formatter.__name__}")
print(f"result:    {result}")

```

Possible output:

```
mode:      title
formatter: as_title
result:    These Pretzels Are Making Me Thirsty
```

This dictionary is a dispatch table. It replaces a chain of conditionals with a mapping from command names to function objects.

A simple conditional version would be:

```
if mode == "upper":
    result = as_upper(text)
elif mode == "lower":
    result = as_lower(text)
elif mode == "title":
    result = as_title(text)
```

The dispatch-table version is:

```
formatter = formatters[mode]
result = formatter(text)
```

Both are valid. The dispatch table makes the executable choices explicit as data.

A safer version checks whether the requested key exists.

```
def as_upper(text):
    return text.upper()

def as_lower(text):
    return text.lower()

formatters = {
    "upper": as_upper,
    "lower": as_lower,
}

mode = "reverse"
text = "D'oh!"

formatter = formatters.get(mode)

if formatter is None:
    print(f"unknown mode: {mode!r}")
else:
    print(formatter(text))
```

Possible output:

```
unknown mode: 'reverse'
```

The value returned by `dict.get()` is either a function object or `None`. The call is performed only after checking that a function object was actually found.

The core callback rule is:

A callback is a function object supplied as data so another function can call it at a controlled point in its own execution.

Higher-Order Functions: Functions That Receive or Return Other Functions

A higher-order function is a function that works with other functions as values.

It may receive a function:

```
def double(x):
    return x * 2

def apply_once(fn, value):
    return fn(value)

result = apply_once(double, 21)

print(f"result: {result}")
```

Possible output:

```
result: 42
```

The function `apply_once()` receives the function object `double`, binds it to its local name `fn`, and calls it.

The binding is ordinary parameter binding.

```
def double(x):
    return x * 2

def apply_once(fn, value):
    print(f"fn:          {fn}")
    print(f"type(fn): {type(fn)}")
    print(f"value:     {value}")
    print()

    return fn(value)

result = apply_once(double, 21)

print(f"result: {result}")
```

Possible output:

```
fn:          <function double at 0x...>
type(fn): <class 'function'>
value:      21

result: 42
```

A higher-order function may apply a function to several values.

```
def square(x):
    return x * x

def apply_to_all(fn, values):
    results = []

    for value in values:
        result = fn(value)
```

```

        results.append(result)

    return results

nums = [1, 2, 3, 4]

squares = apply_to_all(square, nums)

print(f"nums:    {nums}")
print(f"squares: {squares}")

```

Possible output:

```

nums:    [1, 2, 3, 4]
squares: [1, 4, 9, 16]

```

The result list is a normal list.

```

def square(x):
    return x * x

def apply_to_all(fn, values):
    results = []

    for value in values:
        results.append(fn(value))

    return results

squares = apply_to_all(square, [1, 2, 3, 4])

print(f"type(squares): {type(squares)}")
print()

selected_names = [
    "append",
    "extend",
    "pop",
    "__len__",
    "__getitem__",
]

for name in selected_names:
    print(f"{name:<14} {name in dir(squares)}")

print()
print(f"len(squares): {len(squares)}")
print(f"squares[2]:   {squares[2]}")

```

Possible output:

```

type(squares): <class 'list'>

append      True
extend      True

```

```
pop          True
__len__      True
__getitem__  True
```

```
len(squares): 4
squares[2]:   9
```

The higher-order structure is:

```
apply_to_all receives:
    fn      -> function object
    values  -> iterable object
```

```
inside the loop:
    call fn(value)
    collect returned object
```

A higher-order function may also return a function.

```
def make_multiplier(factor):
    def multiply(x):
        return x * factor

    return multiply

times_two = make_multiplier(2)
times_three = make_multiplier(3)

print(f"times_two:    {times_two}")
print(f"times_three: {times_three}")
print()
print(f"times_two(21):  {times_two(21)}")
print(f"times_three(14): {times_three(14)}")
```

Possible output:

```
times_two:    <function make_multiplier.<locals>.multiply at 0x...>
times_three:  <function make_multiplier.<locals>.multiply at 0x...>

times_two(21):  42
times_three(14): 42
```

The function `make_multiplier()` returns the inner function object `multiply`. The returned function can be stored and called later.

This example also involves a closure because the returned function uses `factor` after the outer function has returned. Closure internals will be studied in the next subsection. For now, the important first-class behavior is that a function can return another function object.

A simpler version can return one of several existing functions without introducing closure state.

```
def as_upper(text):
    return text.upper()

def as_lower(text):
    return text.lower()
```



```
def choose_formatter(mode):
    if mode == "upper":
        return as_upper

    return as_lower

formatter = choose_formatter("upper")

print(f"formatter:      {formatter}")
print(f"type(formatter): {type(formatter)}")
print(f"result:         {formatter('ni!')}")
```

Possible output:

```
formatter:      <function as_upper at 0x...>
type(formatter): <class 'function'>
result:         NI!
```

The returned object is selected at runtime. The caller receives a function object and can call it later.

A function can both receive and return functions.

```
def shout(text):
    return text.upper()

def make_logged(fn):
    def logged(text):
        print(f"calling {fn.__name__} with {text!r}")
        result = fn(text)
        print(f"result is {result!r}")
        return result

    return logged

logged_shout = make_logged(shout)

final_text = logged_shout("no soup for you!")

print(f"final_text: {final_text}")
```

Possible output:

```
calling shout with 'no soup for you!'
result is 'NO SOUP FOR YOU!'
final_text: NO SOUP FOR YOU!
```

The function `make_logged()` receives one function object and returns another function object. The returned function calls the original function and adds behavior around it.

This is the basic shape behind decorators, but the decorator syntax itself will be explained later. Here, the mechanism is still plain function-object passing and returning.

The returned function object can be inspected.

```
def shout(text):
    return text.upper()
```

```
def make_logged(fn):
    def logged(text):
        print(f"calling {fn.__name__}")
        return fn(text)

    return logged

logged_shout = make_logged(shout)

print(f"logged_shout:      {logged_shout}")
print(f"type(logged_shout): {type(logged_shout)}")
print(f"name:              {logged_shout.__name__}")
print(f"closure:           {logged_shout.__closure__}")
```

Possible output:

```
logged_shout:      <function make_logged.<locals>.logged at 0x...>
type(logged_shout): <class 'function'>
name:              logged
closure:           (<cell at 0x...: function object at 0x...>,)
```

The closure appears because `logged()` remembers `fn`. The detailed mechanics of `__closure__` and `cell` objects belong to the next subsection.

For now, the architectural point is enough:

Higher-order functions let a program pass executable behavior through ordinary object references.

This allows code structure to be separated from the specific operation performed at a given step. A compact summary:

Pattern	Meaning
<code>fn = some_function</code>	store a function object in another name
<code>items = [fn1, fn2]</code>	store function objects inside a list
<code>table["x"] = fn</code>	store a function object inside a dictionary
<code>outer(fn)</code>	pass a function object into another function
<code>return fn</code>	return a function object from a function
<code>fn()</code>	call the function object currently referenced by <code>fn</code>

The final rule is:

A function name without parentheses refers to the function object. A function name with parentheses calls the function object and produces its return value.

8.6.2 Lexical Closures

A lexical closure exists when an inner function keeps access to names from an enclosing function after the enclosing function has finished running.

This sounds strange only if a function frame is imagined as disappearing completely with all of its local state. In normal cases, a function frame does terminate after return. But when an inner function needs some of the outer function's local bindings, CPython preserves those specific bindings inside cell objects. The whole outer frame is not kept alive as an ordinary active call frame. The necessary referenced state is moved into closure storage.

The result is a function object that carries executable code plus references to preserved lexical state.

The Lifespan Shift: Preserving Enclosing Environments for Out-of-Scope Execution

Start with a nested function that is called while the outer function is still running.

```
def outer():
    message = "D'oh!"

    def inner():
        return message

    return inner()

result = outer()

print(f"result: {result}")
```

Possible output:

```
result: D'oh!
```

This example does not yet require a long-lived closure. The inner function runs before `outer()` returns. The enclosing frame still exists when `inner()` reads `message`.

Now return the inner function object itself.

```
def outer():
    message = "D'oh!"

    def inner():
        return message

    return inner

fn = outer()

print(f"fn:      {fn}")
print(f"type(fn): {type(fn)}")
print(f"fn():    {fn()}")
```

Possible output:

```
fn:      <function outer.<locals>.inner at 0x...>
type(fn): <class 'function'>
fn():    D'oh!
```

The function `outer()` has already returned. Its ordinary execution frame is no longer active. But `fn()` can still read `message`. That is the lifespan shift.

The local name `message` did not simply vanish, because the returned inner function still needs it. The preserved state is not global.

```
def outer():
    message = "D'oh!"

    def inner():
        return message

    return inner
```

```

fn = outer()

try:
    print(message)
except NameError as err:
    print(f"error type: {type(err)}")
    print(f"message: {err}")

print(f"fn(): {fn()}")

```

Possible output:

```

error type: <class 'NameError'>
message:    name 'message' is not defined
fn(): D'oh!

```

The name `message` is not available as a global name. It is available only through the closure carried by the returned function object.

A closure can preserve different values for different function objects.

```

def make_speaker(message):
    def speak():
        return message

    return speak

say_doh = make_speaker("D'oh!")
say_ni = make_speaker("Ni!")
say_answer = make_speaker("The answer is 42.")

print(say_doh())
print(say_ni())
print(say_answer())

```

Possible output:

```

D'oh!
Ni!
The answer is 42.

```

Each call to `make_speaker()` creates a new function call frame. Each returned `speak` function preserves the `message` value from its own call.

The function objects are separate.

```

def make_speaker(message):
    def speak():
        return message

    return speak

say_doh = make_speaker("D'oh!")
say_ni = make_speaker("Ni!")

print(f"say_doh: {say_doh}")

```

```

print(f"say_ni:      {say_ni}")
print(f"same object: {say_doh is say_ni}")
print()
print(f"say_doh():   {say_doh()}")
print(f"say_ni():    {say_ni()}")

```

Possible output:

```

say_doh:      <function make_speaker.<locals>.speak at 0x...>
say_ni:      <function make_speaker.<locals>.speak at 0x...>
same object:  False

say_doh():    D'oh!
say_ni():     Ni!

```

They were created from the same source code, but they are different function objects with different preserved environments.

The returned function object is still an ordinary function object.

```

def make_speaker(message):
    def speak():
        return message

    return speak

fn = make_speaker("No soup for you!")

print(f"type(fn): {type(fn)}")
print()

selected_names = [
    "__call__",
    "__name__",
    "__doc__",
    "__defaults__",
    "__code__",
    "__closure__",
]

for name in selected_names:
    print(f"{name:<16} {name in dir(fn)}")

print()
print(f"fn.__name__: {fn.__name__!r}")
print(f"fn():      {fn()}")

```

Possible output:

```

type(fn): <class 'function'>

__call__      True
__name__      True
__doc__       True
__defaults__  True
__code__      True

```

```
__closure__      True

fn.__name__: 'speak'
fn():            No soup for you!
```

The closure does not make it a different kind of callable. It is still a function object. It just has additional stored lexical state.

Closures are useful when a function must be configured once and executed later.

```
def make_prefixer(prefix):
    def add_prefix(text):
        return f"{prefix}: {text}"

    return add_prefix

warn = make_prefixer("WARNING")
info = make_prefixer("INFO")

print(warn("Nobody expects the Spanish Inquisition!"))
print(info("It is only a model."))
```

Possible output:

```
WARNING: Nobody expects the Spanish Inquisition!
INFO: It is only a model.
```

The function `make_prefixer()` builds a specialized function. The returned function remembers the selected `prefix`.

This is not class-based storage. It is not a global variable. It is a function object carrying references to preserved enclosing bindings.

How CPython Uses `__closure__` and Cell Objects to Store Lexical State After the Parent Frame Unwinds

The closure state of a function object is visible through `__closure__`.

```
def make_speaker(message):
    def speak():
        return message

    return speak

fn = make_speaker("D'oh!")

print(f"fn.__closure__: {fn.__closure__}")
print(f"type:           {type(fn.__closure__)}")
```

Possible output:

```
fn.__closure__: (<cell at 0x...: str object at 0x...>,)
type:           <class 'tuple'>
```

The `__closure__` attribute is either `None` or a tuple of cell objects.

Here it is a tuple containing one cell. That cell contains the preserved object referenced by the outer name `message`.

The tuple can be inspected.

```

def make_speaker(message):
    def speak():
        return message

    return speak

fn = make_speaker("D'oh!")
closure_tuple = fn.__closure__

print(f"type(closure_tuple): {type(closure_tuple)}")
print()

selected_names = [
    "count",
    "index",
    "__len__",
    "__getitem__",
    "__contains__",
]

for name in selected_names:
    print(f"{name:<14} {name in dir(closure_tuple)}")

print()
print(f"len(closure_tuple): {len(closure_tuple)}")
print(f"closure_tuple[0]: {closure_tuple[0]}")

```

Possible output:

```

type(closure_tuple): <class 'tuple'>

count          True
index          True
__len__        True
__getitem__    True
__contains__   True

len(closure_tuple): 1
closure_tuple[0]:  <cell at 0x...: str object at 0x...>

```

The cell object exposes its stored object through `cell_contents`.

```

def make_speaker(message):
    def speak():
        return message

    return speak

fn = make_speaker("D'oh!")
cell = fn.__closure__[0]

print(f"cell: {cell}")
print(f"type(cell): {type(cell)}")
print()

```

```

selected_names = [
    "cell_contents",
    "__class__",
    "__eq__",
    "__repr__",
]

for name in selected_names:
    print(f"{name:<16} {name in dir(cell)}")

print()
print(f"cell.cell_contents: {cell.cell_contents!r}")
print(f"type(contents):      {type(cell.cell_contents)}")

```

Possible output:

```

cell:          <cell at 0x...: str object at 0x...>
type(cell):    <class 'cell'>

cell_contents   True
__class__       True
__eq__          True
__repr__        True

cell.cell_contents: "D'oh!"
type(contents):    <class 'str'>

```

The closure cell stores a reference to the object. It does not store the source-code text "message". It stores runtime state needed by the inner function.

A function without preserved enclosing names has no closure.

```

def plain():
    return "D'oh!"

print(f"plain.__closure__: {plain.__closure__}")
print(f"plain():           {plain()}")

```

Possible output:

```

plain.__closure__: None
plain():           D'oh!

```

There is no enclosing function state to preserve.

A closure can contain several cells.

```

def make_card(name, score):
    mood = "calm"

    def card():
        return f"{name:<10} score={score:04d} mood={mood}"

    return card

fn = make_card("Homer", 42)

```



```

print(fn())
print()
print(f"fn.__closure__: {fn.__closure__}")

for index, cell in enumerate(fn.__closure__):
    print(f"{index}: {cell.cell_contents!r}")

```

Possible output:

```
Homer      score=0042 mood=calm
```

```

fn.__closure__: (<cell at 0x...: str object at 0x...>, <cell at 0x...: str object at 0x...>,
0: 'calm'
1: 'Homer'
2: 42

```

The cell order corresponds to the function code object's free-variable order, which will be inspected in the next subsection.

If the preserved object is mutable, the closure can mutate that object without rebinding the outer name.

```

def make_collector():
    items = []

    def add_item(text):
        items.append(text)
        return items

    return add_item

collector = make_collector()

print(collector("D'oh!"))
print(collector("No soup for you!"))
print(collector("The answer is 42."))

```

Possible output:

```

["D'oh!"]
["D'oh!", 'No soup for you!']
["D'oh!", 'No soup for you!', 'The answer is 42.']

```

The list object is preserved in the closure. Each call mutates the same list object. Inspect the preserved list.

```

def make_collector():
    items = []

    def add_item(text):
        items.append(text)
        return items

    return add_item

collector = make_collector()

```

```

cell = collector.__closure__[0]
items = cell.cell_contents

print(f"items before: {items}")
print(f"id(items):      {id(items)}")
print()

collector("D'oh!")
collector("Ni!")

print(f"items after:  {items}")
print(f"id(items):      {id(items)}")
print(f"cell value:     {cell.cell_contents}")

```

Possible output:

```

items before: []
id(items):    2298078960768

items after:  ["D'oh!", 'Ni!']
id(items):    2298078960768
cell value:   ["D'oh!", 'Ni!']

```

The same list object remains inside the cell. The list contents change.
The list object can be inspected.

```

def make_collector():
    items = []

    def add_item(text):
        items.append(text)
        return items

    return add_item

collector = make_collector()
items = collector.__closure__[0].cell_contents

print(f"type(items): {type(items)}")
print()

selected_names = [
    "append",
    "extend",
    "pop",
    "clear",
    "__len__",
    "__getitem__",
    "__setitem__",
]

for name in selected_names:
    print(f"{name:<14} {name in dir(items)}")

```

```
print()
items.append("It is only a model.")
print(f"items: {items}")
```

Possible output:

```
type(items): <class 'list'>
```

```
append      True
extend      True
pop         True
clear       True
__len__     True
__getitem__ True
__setitem__ True
```

```
items: ['It is only a model.']
```

If the inner function must rebind an outer name, it needs `nonlocal`.

```
def make_counter():
    count = 0

    def next_count():
        nonlocal count

        count = count + 1
        return count

    return next_count

counter = make_counter()

print(counter())
print(counter())
print(counter())
```

Possible output:

```
1
2
3
```

The integer object is immutable. The statement:

```
count = count + 1
```

does not mutate the integer object. It creates or obtains a new integer object and updates the closure cell so that `count` refers to the new object.

Inspect the cell before and after calls.

```
def make_counter():
    count = 0

    def next_count():
```

```

        nonlocal count

        count = count + 1
        return count

    return next_count

counter = make_counter()
cell = counter.__closure__[0]

print(f"before: {cell.cell_contents}")

counter()
counter()

print(f"after:  {cell.cell_contents}")

```

Possible output:

```

before: 0
after:  2

```

The closure cell, not a normal local frame slot, is where the preserved binding now lives. A compact closure structure is:

```

outer function call:
  creates local variable message
  creates inner function object
  inner function refers to message
  message is stored in a cell
  inner function receives reference to that cell
  outer function returns inner function

```

```

later:
  inner function reads message through the cell

```

Inspecting Free Variables via `co_freevars`, `co_cellvars`, and Closure Cells

Code objects record closure-related names.

From the outer function's point of view, a local variable that must be preserved for an inner function is a cell variable.

From the inner function's point of view, a variable coming from an enclosing scope is a free variable.

```

def outer():
    message = "D'oh!"

    def inner():
        return message

    return inner

fn = outer()

print(f"outer co_cellvars: {outer.__code__.co_cellvars}")
print(f"inner co_freevars: {fn.__code__.co_freevars}")

```

Possible output:

```
outer co_cellvars: ('message',)
inner co_freevars: ('message',)
```

The same lexical name appears from two perspectives:

```
outer:
    message is a cell variable

inner:
    message is a free variable
```

The inner function's closure tuple stores the cells for its free variables.

```
def outer():
    message = "D'oh!"

    def inner():
        return message

    return inner

fn = outer()

free_names = fn.__code__.co_freevars
closure_cells = fn.__closure__

for name, cell in zip(free_names, closure_cells):
    print(f"{name:<10} -> {cell.cell_contents!r}")
```

Possible output:

```
message    -> "D'oh!"
```

The `zip()` call pairs each free-variable name with its corresponding closure cell. For several free variables:

```
def make_card(name, score):
    mood = "calm"

    def card():
        return f"{name:<10} score={score:04d} mood={mood}"

    return card

fn = make_card("Homer", 42)

print(f"freevars: {fn.__code__.co_freevars}")
print()

for name, cell in zip(fn.__code__.co_freevars, fn.__closure__):
    print(f"{name:<8} -> {cell.cell_contents!r}")
```

Possible output:

```
freevars: ('mood', 'name', 'score')
```

```
mood      -> 'calm'
name      -> 'Homer'
score     -> 42
```

The exact ordering is the ordering recorded by the code object. Code should not guess it manually. When inspecting, pair `co_freevars` with `__closure__`.

The code object itself can be inspected.

```
def make_card(name, score):
    mood = "calm"

    def card():
        return f"{name:<10} score={score:04d} mood={mood}"

    return card

fn = make_card("Homer", 42)
code_obj = fn.__code__

print(f"type(code_obj): {type(code_obj)}")
print()

selected_names = [
    "co_freevars",
    "co_cellvars",
    "co_varnames",
    "co_names",
    "co_consts",
    "co_code",
]

for name in selected_names:
    print(f"{name:<14} {name in dir(code_obj)}")

print()
print(f"co_freevars: {code_obj.co_freevars}")
print(f"co_cellvars: {code_obj.co_cellvars}")
print(f"co_varnames: {code_obj.co_varnames}")
```

Possible output:

```
type(code_obj): <class 'code'>

co_freevars    True
co_cellvars    True
co_varnames    True
co_names       True
co_consts      True
co_code        True

co_freevars: ('mood', 'name', 'score')
co_cellvars: ()
co_varnames: ()
```

The inner function has no ordinary local variables in this example. Its useful names come from closure cells.

The outer function's code object has cell variables.

```
def make_card(name, score):
    mood = "calm"

    def card():
        return f"{name:<10} score={score:04d} mood={mood}"

    return card

outer_code = make_card.__code__

print(f"outer co_varnames: {outer_code.co_varnames}")
print(f"outer co_cellvars: {outer_code.co_cellvars}")
print(f"outer co_freevars: {outer_code.co_freevars}")
```

Possible output:

```
outer co_varnames: ('name', 'score', 'card')
outer co_cellvars: ('name', 'score', 'mood')
outer co_freevars: ()
```

The outer function has ordinary local names, but some of them also become cell variables because an inner function needs them.

A nested function can have ordinary local variables and free variables at the same time.

```
def make_builder(prefix):
    def build(text):
        result = f"{prefix}: {text}"
        return result

    return build

fn = make_builder("INFO")

print(fn("It is only a model."))
print()
print(f"co_varnames: {fn.__code__.co_varnames}")
print(f"co_freevars: {fn.__code__.co_freevars}")
print(f"closure:      {fn.__closure__}")
```

Possible output:

```
INFO: It is only a model.
```

```
co_varnames: ('text', 'result')
co_freevars: ('prefix',)
closure:      (<cell at 0x...: str object at 0x...>,)
```

Here:

- `text` is a parameter local to `build()`;
- `result` is a local variable inside `build()`;

- `prefix` is a free variable coming from the enclosing function;
- the actual `prefix` object is stored in a closure cell.

The bytecode can show closure reads.

```
import dis

def make_builder(prefix):
    def build(text):
        result = f"{prefix}: {text}"
        return result

    return build

fn = make_builder("INFO")

dis.dis(fn)
```

Possible output excerpt:

```
... LOAD_DEREF          2 (prefix)
... LOAD_FAST           0 (text)
... STORE_FAST          1 (result)
... LOAD_FAST           1 (result)
... RETURN_VALUE
```

The exact bytecode display varies by Python version. The important distinction is that closure variables are accessed differently from ordinary local variables. A local variable may use a fast-local access path. A closure variable is loaded through a cell.

A closure can be summarized as three connected objects:

```
function object
-> code object
    -> co_freevars names
-> __closure__ tuple
    -> cell objects
        -> preserved runtime objects
```

A final inspection example prints the complete mapping.

```
def make_counter(start):
    count = start

    def next_count(step=1):
        nonlocal count

        count = count + step
        return count

    return next_count

counter = make_counter(40)

print(counter())
```



```

print(counter())
print()

print(f"function object: {counter}")
print(f"code object:      {counter.__code__}")
print(f"freevars:        {counter.__code__.co_freevars}")
print(f"defaults:        {counter.__defaults__}")
print()

for name, cell in zip(counter.__code__.co_freevars, counter.__closure__):
    print(f"{name:<8} -> {cell.cell_contents!r}")

```

Possible output:

```

41
42

function object: <function make_counter.<locals>.next_count at 0x...>
code object:     <code object next_count at 0x..., file "...", line ...>
freevars:       ('count',)
defaults:       (1,)

count    -> 42

```

The default value `step=1` is stored in the function object's defaults. The preserved state `count` is stored through the closure cell. These are different storage mechanisms.

The final rule is:

A closure is not a copy of an outer function call. It is a function object whose code refers to preserved cell variables from its lexical enclosing scope.

8.6.3 Decorators

A decorator is a function transformation mechanism. It receives a function object, builds or selects another callable object, and returns that object to be bound under the original function name.

The important point is that decoration happens when the `def` statement executes, not when the decorated function is later called.

A decorator is not magic syntax around a function call. It is ordinary function-object passing, returning, and rebinding.

Decorators as Function Transformation at Definition Time

Start without decorator syntax.

```

def shout(text):
    return text.upper()

def plain(text):
    return text

print(f"shout:      {shout}")
print(f"type(shout): {type(shout)}")
print(f"shout('ni!'): {shout('ni!')}")

```

Possible output:

```
shout:          <function shout at 0x...>
type(shout): <class 'function'>
shout('ni!'): NI!
```

Now define a function that receives another function and returns a replacement function.

```
def make_logged(fn):
    def wrapper(text):
        print(f"calling {fn.__name__} with {text!r}")
        result = fn(text)
        print(f"result: {result!r}")
        return result

    return wrapper

def shout(text):
    return text.upper()

shout = make_logged(shout)

print(shout("no soup for you!"))
```

Possible output:

```
calling shout with 'no soup for you!'
result: 'NO SOUP FOR YOU!'
NO SOUP FOR YOU!
```

The critical line is:

```
shout = make_logged(shout)
```

This line:

1. reads the current function object bound to `shout`;
2. passes that function object into `make_logged`;
3. receives the returned `wrapper` function object;
4. rebinds the name `shout` to the wrapper.

After that assignment, the name `shout` no longer points directly to the original function object. It points to the wrapper function object.

We can inspect that.

```
def make_logged(fn):
    def wrapper(text):
        print(f"calling {fn.__name__}")
        return fn(text)

    return wrapper

def shout(text):
    return text.upper()

original_shout = shout
```

```

shout = make_logged(shout)

print(f"original_shout: {original_shout}")
print(f"shout:          {shout}")
print(f"same object:    {original_shout is shout}")
print()
print(f"original name:  {original_shout.__name__}")
print(f"new name:       {shout.__name__}")

```

Possible output:

```

original_shout: <function shout at 0x...>
shout:          <function make_logged.<locals>.wrapper at 0x...>
same object:    False

original name:  shout
new name:       wrapper

```

The original function still exists because `original_shout` and the wrapper closure still reference it. But the public name `shout` now refers to the wrapper.

The wrapper is a function object.

```

def make_logged(fn):
    def wrapper(text):
        print(f"calling {fn.__name__}")
        return fn(text)

    return wrapper

def shout(text):
    return text.upper()

shout = make_logged(shout)

print(f"type(shout): {type(shout)}")
print()

selected_names = [
    "__call__",
    "__name__",
    "__doc__",
    "__defaults__",
    "__code__",
    "__closure__",
]

for name in selected_names:
    print(f"{name:<16} {name in dir(shout)}")

print()
print(f"shout.__name__:    {shout.__name__!r}")
print(f"shout.__closure__: {shout.__closure__}")

```

Possible output:

```
type(shout): <class 'function'>
```

```
__call__      True
__name__      True
__doc__       True
__defaults__  True
__code__      True
__closure__   True
```

```
shout.__name__: 'wrapper'
```

```
shout.__closure__: (<cell at 0x...: function object at 0x...>,)

```

The wrapper has a closure because it remembers the original function object through the name `fn`. The decorator function itself is also a normal function object.

```
def make_logged(fn):
    def wrapper(text):
        return fn(text)

    return wrapper

print(f"make_logged:      {make_logged}")
print(f"type(make_logged): {type(make_logged)}")
print(f"make_logged name:  {make_logged.__name__}")
print(f"make_logged code:  {make_logged.__code__}")
```

Possible output:

```
make_logged:      <function make_logged at 0x...>
type(make_logged): <class 'function'>
make_logged name:  make_logged
make_logged code:  <code object make_logged at 0x..., file "...", line ...>
```

A decorator can add behavior before the original call.

```
def announce(fn):
    def wrapper(text):
        print("Prepare yourself.")
        return fn(text)

    return wrapper

def say_quote(text):
    return text

say_quote = announce(say_quote)

print(say_quote("Nobody expects the Spanish Inquisition!"))
```

Possible output:

```
Prepare yourself.
Nobody expects the Spanish Inquisition!
```

A decorator can add behavior after the original call.

```
def add_exclamation(fn):
    def wrapper(text):
        result = fn(text)
        return result + "!"

    return wrapper

def say_quote(text):
    return text

say_quote = add_exclamation(say_quote)

print(say_quote("Ni"))
```

Possible output:

Ni!

A decorator can also block or replace the original call.

```
def block_call(fn):
    def wrapper(text):
        return "This function has been blocked."

    return wrapper

def say_quote(text):
    return text

say_quote = block_call(say_quote)

print(say_quote("D'oh!"))
```

Possible output:

This function has been blocked.

The wrapper does not have to call the original function. A decorator receives the original function object, but the returned callable decides what to do with it.

The structural pattern is:

```
def decorator(original_function):
    def replacement_function(...):
        ...
        result = original_function(...)
        ...
        return result

    return replacement_function

target_function = decorator(target_function)
```

That is the basic decorator machine.

The @decorator Syntax as Rebinding Sugar

Python provides decorator syntax to write this transformation directly above the function definition.

This:

```
def make_logged(fn):
    def wrapper(text):
        print(f"calling {fn.__name__} with {text!r}")
        result = fn(text)
        print(f"result: {result!r}")
        return result

    return wrapper

@make_logged
def shout(text):
    return text.upper()

print(shout("no soup for you!"))
```

Possible output:

```
calling shout with 'no soup for you!'
result: 'NO SOUP FOR YOU!'
NO SOUP FOR YOU!
```

is equivalent to this:

```
def make_logged(fn):
    def wrapper(text):
        print(f"calling {fn.__name__} with {text!r}")
        result = fn(text)
        print(f"result: {result!r}")
        return result

    return wrapper

def shout(text):
    return text.upper()

shout = make_logged(shout)

print(shout("no soup for you!"))
```

The decorator syntax is rebinding sugar.

The execution order is:

1. create the original function object from `def shout(...)`
2. pass that function object to `make_logged`
3. receive the returned wrapper object
4. bind the name `shout` to the wrapper object

This happens when the `def` statement executes.

A print trace makes definition-time decoration visible.

```

def make_logged(fn):
    print(f"decorating {fn.__name__}")

    def wrapper(text):
        print(f"calling {fn.__name__}")
        return fn(text)

    return wrapper

print("before decorated def")

@make_logged
def shout(text):
    print("inside original shout")
    return text.upper()

print("after decorated def")
print()

print(shout("d'oh!"))

```

Possible output:

```

before decorated def
decorating shout
after decorated def

calling shout
inside original shout
D'OH!

```

The line:

```
decorating shout
```

appears before the function is called. It appears while the decorated `def` statement is being executed.

This proves that decorator application is a definition-time event. The wrapper's body runs later, when the decorated function name is called.

Multiple decorators stack from bottom to top.

```

def deco_a(fn):
    print("applying deco_a")

    def wrapper(text):
        print("inside deco_a wrapper")
        return fn(text)

    return wrapper

def deco_b(fn):
    print("applying deco_b")

    def wrapper(text):
        print("inside deco_b wrapper")
        return fn(text)

```

```

    return wrapper

@deco_a
@deco_b
def say(text):
    print("inside original say")
    return text

print()
print(say("It is only a model."))

```

Possible output:

```

applying deco_b
applying deco_a

inside deco_a wrapper
inside deco_b wrapper
inside original say
It is only a model.

```

This:

```

@deco_a
@deco_b
def say(text):
    return text

```

is equivalent to:

```

def say(text):
    return text

say = deco_a(deco_b(say))

```

The decorator closest to the function is applied first. The outer decorator receives the result of the inner decoration.

Decorator expressions can also be calls. This creates a decorator factory.

```

def make_prefix_decorator(prefix):
    print(f"creating decorator for prefix={prefix!r}")

    def decorator(fn):
        print(f"decorating {fn.__name__}")

        def wrapper(text):
            result = fn(text)
            return f"{prefix}: {result}"

        return wrapper

    return decorator

@make_prefix_decorator("INFO")

```



```
def say(text):
    return text

print()
print(say("The answer is 42."))
```

Possible output:

```
creating decorator for prefix='INFO'
decorating say

INFO: The answer is 42.
```

This is equivalent to:

```
def say(text):
    return text

say = make_prefix_decorator("INFO")(say)
```

There are two stages:

1. `make_prefix_decorator("INFO")` returns a decorator;
2. that returned decorator receives `say` and returns a wrapper.

The returned decorator and wrapper are both function objects.

```
def make_prefix_decorator(prefix):
    def decorator(fn):
        def wrapper(text):
            return f"{prefix}: {fn(text)}"

        return wrapper

    return decorator

decorator_obj = make_prefix_decorator("INFO")

print(f"decorator_obj:      {decorator_obj}")
print(f"type(decorator_obj): {type(decorator_obj)}")
print(f"name:                {decorator_obj.__name__}")
print(f"closure:              {decorator_obj.__closure__}")
```

Possible output:

```
decorator_obj:      <function make_prefix_decorator.<locals>.decorator at 0x...>
type(decorator_obj): <class 'function'>
name:               decorator
closure:            (<cell at 0x...: str object at 0x...>,)
```

The decorator object remembers `prefix`. The wrapper object later remembers both `prefix` and `fn`.

The key rule is:

The `@decorator` form is not a new kind of function. It is syntax for passing the just-created function object through another callable and rebinding the original name to the returned object.

Wrapper Functions, Closure State, and Metadata Preservation

A wrapper function often closes over the original function object.

```
def make_logged(fn):
    def wrapper(text):
        print(f"calling {fn.__name__}")
        return fn(text)

    return wrapper

@make_logged
def shout(text):
    """Return uppercase text."""
    return text.upper()

print(shout("ni!"))
print()
print(f"shout:           {shout}")
print(f"shout.__name__: {shout.__name__!r}")
print(f"shout.__doc__:   {shout.__doc__!r}")
print(f"closure:         {shout.__closure__}")
```

Possible output:

```
calling shout
NI!

shout:           <function make_logged.<locals>.wrapper at 0x...>
shout.__name__:  'wrapper'
shout.__doc__:   None
closure:         (<cell at 0x...: function object at 0x...>,)
```

The decorated name `shout` now points to `wrapper`. That is why `shout.__name__` is `'wrapper'` and the original docstring is not visible through `shout.__doc__`.

The original function object is preserved inside the wrapper's closure.

```
def make_logged(fn):
    def wrapper(text):
        print(f"calling {fn.__name__}")
        return fn(text)

    return wrapper

@make_logged
def shout(text):
    """Return uppercase text."""
    return text.upper()

cell = shout.__closure__[0]
original_fn = cell.cell_contents

print(f"cell:           {cell}")
print(f"type(cell):     {type(cell)}")
print(f"original_fn:      {original_fn}")
```

```

print(f"type(original):    {type(original_fn)}")
print()
print(f"original name:     {original_fn.__name__!r}")
print(f"original doc:      {original_fn.__doc__!r}")
print(f"original result:   {original_fn('doh')}")

```

Possible output:

```

cell:                <cell at 0x...: function object at 0x...>
type(cell):          <class 'cell'>
original_fn:         <function shout at 0x...>
type(original):      <class 'function'>

original name:       'shout'
original doc:        'Return uppercase text.'
original result:     DOH

```

The original function was not destroyed. It is just no longer directly bound to the public name `shout`. The wrapper closure still points to it.

The wrapper's code object records the captured free variable.

```

def make_logged(fn):
    def wrapper(text):
        print(f"calling {fn.__name__}")
        return fn(text)

    return wrapper

@make_logged
def shout(text):
    """Return uppercase text."""
    return text.upper()

print(f"freevars: {shout.__code__.co_freevars}")

for name, cell in zip(shout.__code__.co_freevars, shout.__closure__):
    print(f"{name:<8} -> {cell.cell_contents}")

```

Possible output:

```

freevars: ('fn',)
fn        -> <function shout at 0x...>

```

The wrapper function has a free variable named `fn`. Its closure cell contains the original function object.

A wrapper should usually accept flexible arguments so it can forward the same call shape to the original function.

A too-specific wrapper is fragile.

```

def bad_logged(fn):
    def wrapper(text):
        print(f"calling {fn.__name__}")
        return fn(text)

    return wrapper

```

```
@bad_logged
def build_line(name, score=42):
    return f"{name:<10} score={score:04d}"

print(build_line("Homer"))

try:
    print(build_line("Lisa", 100))
except TypeError as err:
    print(f"error type: {type(err)}")
    print(f"message:    {err}")
```

Possible output:

```
calling build_line
Homer      score=0042
error type: <class 'TypeError'>
message:    bad_logged.<locals>.wrapper() takes 1 positional argument but 2 were given
```

The original function could accept two arguments, but the wrapper could accept only one.
A flexible wrapper uses `*args` and `**kwargs`.

```
def logged(fn):
    def wrapper(*args, **kwargs):
        print(f"calling {fn.__name__}")
        print(f"args:    {args}")
        print(f"kwargs: {kwargs}")

        result = fn(*args, **kwargs)

        print(f"result: {result!r}")
        return result

    return wrapper

@logged
def build_line(name, score=42, *, mood="calm"):
    return f"{name:<10} score={score:04d} mood={mood}"

build_line("Homer")
print()
build_line("Lisa", 100, mood="focused")
```

Possible output:

```
calling build_line
args:    ('Homer',)
kwargs: {}
result: 'Homer      score=0042 mood=calm'

calling build_line
args:    ('Lisa', 100)
kwargs: {'mood': 'focused'}
result: 'Lisa      score=0100 mood=focused'
```

The wrapper receives arbitrary positional and keyword arguments, then forwards them to the original function with:

```
fn(*args, **kwargs)
```

This preserves the call boundary shape.

The `args` object is a tuple and `kwargs` is a dictionary.

```
def inspect_call(fn):
    def wrapper(*args, **kwargs):
        print(f"type(args): {type(args)}")
        print(f"type(kwargs): {type(kwargs)}")

        return fn(*args, **kwargs)

    return wrapper
```

```
@inspect_call
def say(text, *, times=1):
    return text * times
```

```
print(say("Ni!", times=2))
```

Possible output:

```
type(args): <class 'tuple'>
type(kwargs): <class 'dict'>
Ni!Ni!
```

The wrapper is still a normal function with a variadic signature.

The remaining issue is metadata. After decoration, the public function name points to the wrapper, so metadata such as `__name__`, `__doc__`, and annotations may describe the wrapper rather than the original function.

The standard solution is `functools.wraps`.

```
import functools
```

```
def logged(fn):
    @functools.wraps(fn)
    def wrapper(*args, **kwargs):
        print(f"calling {fn.__name__}")
        return fn(*args, **kwargs)

    return wrapper

@logged
def build_line(name: str, score: int = 42) -> str:
    """Build one formatted display line."""
    return f"{name:<10} score={score:04d}"

print(build_line("Homer"))
print()
print(f"build_line: {build_line}")
print(f"build_line.__name__: {build_line.__name__!r}")
print(f"build_line.__doc__: {build_line.__doc__!r}")
print(f"build_line.__annotations__: {build_line.__annotations__}")
print(f"build_line.__wrapped__: {build_line.__wrapped__}")
```

Possible output:

```
calling build_line
Homer      score=0042
```

```
build_line:          <function build_line at 0x...>
build_line.__name__:  'build_line'
build_line.__doc__:   'Build one formatted display line.'
build_line.__annotations__: {'name': <class 'str'>, 'score': <class 'int'>, 'return': <class 'str'>}
build_line.__wrapped__: <function build_line at 0x...>
```

The wrapper is still the object bound to `build_line`, but `functools.wraps` copies useful metadata from the original function onto the wrapper. It also stores the original function under `__wrapped__`.

The `functools.wraps` object can be inspected.

```
import functools

print(f"functools.wraps:      {functools.wraps}")
print(f"type(functools.wraps): {type(functools.wraps)}")
print()

selected_names = [
    "__call__",
    "__name__",
    "__doc__",
    "__module__",
]

for name in selected_names:
    print(f"{name:<14} {name in dir(functools.wraps)}")
```

Possible output:

```
functools.wraps:      <function wraps at 0x...>
type(functools.wraps): <class 'function'>

__call__      True
__name__      True
__doc__       True
__module__    True
```

`functools.wraps(fn)` is itself a decorator factory. It returns a decorator that modifies the wrapper function's metadata.

The structure is:

```
@functools.wraps(fn)
def wrapper(...):
    ...
```

which means:

```
def wrapper(...):
    ...
```

```
wrapper = functools.wraps(fn)(wrapper)
```

So even `wraps` follows the same decorator rule: create function object, pass it through a decorator, rebound the name to the returned object.

A minimal comparison:

```
import functools

def logged_without_wraps(fn):
    def wrapper(*args, **kwargs):
        return fn(*args, **kwargs)

    return wrapper

def logged_with_wraps(fn):
    @functools.wraps(fn)
    def wrapper(*args, **kwargs):
        return fn(*args, **kwargs)

    return wrapper

@logged_without_wraps
def first():
    """First docstring."""
    return "D'oh!"

@logged_with_wraps
def second():
    """Second docstring."""
    return "No soup for you!"

print(f"first.__name__: {first.__name__!r}")
print(f"first.__doc__: {first.__doc__!r}")
print()
print(f"second.__name__: {second.__name__!r}")
print(f"second.__doc__: {second.__doc__!r}")
print(f"has __wrapped__: {'__wrapped__' in dir(second)}")
```

Possible output:

```
first.__name__: 'wrapper'
first.__doc__: None

second.__name__: 'second'
second.__doc__: 'Second docstring.'
has __wrapped__: True
```

The wrapper still controls execution, but the metadata now reflects the original function more accurately.

This matters for documentation, debugging, stack traces, introspection, help output, decorators stacked on decorators, and tools that inspect function signatures.

The final decorator model is:

```
def decorator(fn):
    @functools.wraps(fn)
    def wrapper(*args, **kwargs):
        before behavior
```

```

    result = fn(*args, **kwargs)
    after behavior
    return result

```

```

return wrapper

```

```

@decorator
def target(...):
    original body

```

and the runtime meaning is:

```

target = decorator(target)

```

The final rule is:

A decorator is a definition-time function transformation. The `@` syntax performs function-object passing and name rebinding. A wrapper usually preserves access to the original function through a closure, and `functools.wraps` preserves the original function's public metadata on the wrapper.

8.7 Non-Preemptive Multitasking: Generators and Coroutines

Ordinary functions run until they return, raise an exception, or reach the end of their body. Generators and coroutines introduce a different execution shape: a computation can pause, keep its local state alive, return control to an outside driver, and later resume from the same point.

This section studies that suspended-frame model step by step. Generators begin the discussion because `yield` makes suspension visible: a generator can produce one value, stop without losing its frame, and continue later when the caller asks for the next value. From there, `yield from` shows how suspended execution can be delegated through chains of generators.

The section then moves to generator-based coroutines, where control is no longer only outward. Through `send()`, `throw()`, and `close()`, the caller can feed values, inject exceptions, and request shutdown. Finally, native `async def` coroutines and `await` show the modern structured form of the same general idea: execution advances cooperatively under an external scheduler, not through hidden preemptive interruption.

8.7.1 Lazy Stream Evaluation: Generators

A generator is a function-like object that produces values over time instead of computing and returning one final value immediately. The source-level difference is the presence of `yield`. Once a function body contains `yield`, calling that function does not execute the body immediately. It creates a generator object.

That generator object can later be resumed. Each resumption continues execution until the next `yield`, a `return`, or the end of the function body.

So a generator is not simply a function that returns a list slowly. It is a suspended execution mechanism.

The `yield` Keyword Architecture: Pausing Function Execution Without Destroying Local State

A normal function starts, runs, and terminates when it reaches `return` or the end of the body.

```

def normal_func():
    print("normal function starts")
    return "D'oh!"

```



```
result = normal_func()

print(f"result:      {result!r}")
print(f"type(result): {type(result)}")
```

Possible output:

```
normal function starts
result:      "D'oh!"
type(result): <class 'str'>
```

A generator function contains `yield`.

```
def generator_func():
    print("generator body starts")
    yield "D'oh!"

gen = generator_func()

print(f"gen:      {gen}")
print(f"type(gen): {type(gen)}")
```

Possible output:

```
gen:      <generator object generator_func at 0x...>
type(gen): <class 'generator'>
```

Notice that "generator body starts" was not printed yet. Calling `generator_func()` created a generator object. It did not run the function body immediately.

The body starts when the generator is advanced.

```
def generator_func():
    print("generator body starts")
    yield "D'oh!"

gen = generator_func()

print("before next()")
value = next(gen)
print("after next()")
print(f"value: {value!r}")
```

Possible output:

```
before next()
generator body starts
after next()
value: "D'oh!"
```

The `yield` statement sends one value outward and pauses the generator frame. It does not destroy the local state.

```
def count_quotes():
    print("step 1")
    yield "D'oh!"

    print("step 2")
    yield "No soup for you!"

    print("step 3")
    yield "The answer is 42."

gen = count_quotes()

print(next(gen))
print(next(gen))
print(next(gen))
```

Possible output:

```
step 1
D'oh!
step 2
No soup for you!
step 3
The answer is 42.
```

Each call to `next(gen)` resumes the same suspended execution. The function body does not restart from the top each time. It continues after the previous `yield`.

A generator can keep local variables alive between yields.

```
def counter():
    n = 40

    print(f"before first yield: n={n}")
    yield n

    n = n + 1
    print(f"before second yield: n={n}")
    yield n

    n = n + 1
    print(f"before third yield: n={n}")
    yield n

gen = counter()

print(f"first:  {next(gen)}")
print(f"second: {next(gen)}")
print(f"third:  {next(gen)}")
```

Possible output:

```
before first yield: n=40
first: 40
before second yield: n=41
second: 41
```

```
before third yield: n=42
third: 42
```

The local name `n` survives across suspension points. The generator frame is paused, not discarded. The important contrast is:

```
return:
    sends a value outward
    terminates the current function frame

yield:
    sends a value outward
    suspends the current generator frame
    allows later resumption
```

A generator object is inspectable.

```
def counter():
    yield 42

gen = counter()

print(f"type(gen): {type(gen)}")
print()

selected_names = [
    "__iter__",
    "__next__",
    "send",
    "throw",
    "close",
    "gi_frame",
    "gi_code",
    "gi_running",
    "gi_yieldfrom",
]

for name in selected_names:
    print(f"{name:<14} {name in dir(gen)}")
```

Possible output:

```
type(gen): <class 'generator'>

__iter__      True
__next__      True
send          True
throw         True
close         True
gi_frame      True
gi_code       True
gi_running    True
gi_yieldfrom  True
```

Some crucial attributes and methods are:

- `__next__`: resumes the generator until the next yielded value;
- `send`: resumes the generator and can send a value into it;
- `throw`: injects an exception into the suspended generator;
- `close`: requests generator shutdown;
- `gi_frame`: exposes the suspended frame, while it exists;
- `gi_code`: exposes the generator's code object.

Only `__next__`, `gi_frame`, and `gi_code` are needed for the current subsection. The bidirectional methods `send`, `throw`, and `close` will be studied later.

Generator Objects: Suspended Frames, Instruction Pointers, and Resumable Execution State

A generator object contains the machinery needed to resume execution later.

```
def small_gen():
    x = "D'oh!"
    yield x

gen = small_gen()

print(f"gen:          {gen}")
print(f"gen.gi_code:   {gen.gi_code}")
print(f"gen.gi_frame:   {gen.gi_frame}")
```

Possible output:

```
gen:          <generator object small_gen at 0x...>
gen.gi_code:   <code object small_gen at 0x..., file "...", line 1>
gen.gi_frame: <frame at 0x..., file "...", line 1, code small_gen>
```

The generator has a code object and a frame object. The code object describes the compiled instructions. The frame stores the current execution state.

The code object can be inspected.

```
def small_gen():
    x = "D'oh!"
    yield x

gen = small_gen()
code_obj = gen.gi_code

print(f"type(code_obj): {type(code_obj)}")
print()

selected_names = [
    "co_name",
    "co_varnames",
    "co_consts",
    "co_names",
    "co_code",
]
```

```

for name in selected_names:
    print(f"{name:<14} {name in dir(code_obj)}")

print()
print(f"co_name:      {code_obj.co_name}")
print(f"co_varnames: {code_obj.co_varnames}")
print(f"co_consts:   {code_obj.co_consts}")

```

Possible output:

```

type(code_obj): <class 'code'>

co_name      True
co_varnames  True
co_consts    True
co_names     True
co_code      True

co_name:      small_gen
co_varnames:  ('x',)
co_consts:    (None, "D'oh!")

```

The frame object can also be inspected.

```

def small_gen():
    x = "D'oh!"
    yield x

gen = small_gen()
frame = gen.gi_frame

print(f"type(frame): {type(frame)}")
print()

selected_names = [
    "f_code",
    "f_locals",
    "f_globals",
    "f_lineno",
    "f_lasti",
]

for name in selected_names:
    print(f"{name:<12} {name in dir(frame)}")

print()
print(f"frame.f_code.co_name: {frame.f_code.co_name}")
print(f"frame.f_locals:      {frame.f_locals}")
print(f"frame.f_lasti:        {frame.f_lasti}")

```

Possible output before first resumption:

```

type(frame): <class 'frame'>

```

```

f_code      True
f_locals    True
f_globals   True
f_lineno    True
f_lasti     True

frame.f_code.co_name: small_gen
frame.f_locals:      {}
frame.f_lasti:       0

```

Before the first `next()`, the generator body has not yet assigned `x`. The local namespace display is still empty.

After the first `yield`, the local state exists.

```

def small_gen():
    x = "D'oh!"
    yield x

gen = small_gen()

print("before next()")
print(f"locals: {gen.gi_frame.f_locals}")

value = next(gen)

print()
print("after next()")
print(f"value:  {value!r}")
print(f"locals: {gen.gi_frame.f_locals}")
print(f"lasti:  {gen.gi_frame.f_lasti}")

```

Possible output:

```

before next()
locals: {}

after next()
value:  "D'oh!"
locals: {'x': "D'oh!"}
lasti:  12

```

The exact value of `f_lasti` is version-dependent. It represents the current bytecode position. The important idea is that the frame remembers where execution stopped.

A generator can be disassembled.

```

import dis

def small_gen():
    x = "D'oh!"
    yield x
    y = "No soup for you!"
    yield y

dis.dis(small_gen)

```

Possible output excerpt:

```
... RETURN_GENERATOR
... RESUME
... LOAD_CONST          ... ("D'oh!")
... STORE_FAST          ... (x)
... LOAD_FAST           ... (x)
... YIELD_VALUE
... RESUME
... LOAD_CONST          ... ('No soup for you!')
... STORE_FAST          ... (y)
... LOAD_FAST           ... (y)
... YIELD_VALUE
...
```

The exact instruction list differs across Python versions, but `YIELD_VALUE` is the critical operation. It is the point where execution produces a value and suspends.

A completed generator has no active frame.

```
def one_value():
    yield 42

gen = one_value()

print(f"before first next: {gen.gi_frame}")

print(next(gen))

try:
    next(gen)
except StopIteration:
    pass

print(f"after completion: {gen.gi_frame}")
```

Possible output:

```
before first next: <frame at 0x..., file "...", line 1, code one_value>
42
after completion: None
```

After the generator is exhausted, `gi_frame` becomes `None`. There is no suspended execution state left to resume.

The generator object may still exist, but it is exhausted.

```
def one_value():
    yield 42

gen = one_value()

print(next(gen))

for attempt in [1, 2]:
    try:
        print(next(gen))
    except StopIteration as err:
        print(f"attempt {attempt}: {type(err).__name__}")
```

Possible output:

```
42
attempt 1: StopIteration
attempt 2: StopIteration
```

Once exhausted, repeated advancement raises `StopIteration` again.

Execution State Resumption: Re-Entering Suspended Function Frames Across Iteration Steps

A generator can be advanced manually with `next()` or automatically by a `for` loop.

Manual advancement shows the resumption points clearly.

```
def quotes():
    print("A")
    yield "D'oh!"

    print("B")
    yield "No soup for you!"

    print("C")
    yield "The answer is 42."

gen = quotes()

print("created generator")
print()

print(f"next 1 -> {next(gen)!r}")
print()
print(f"next 2 -> {next(gen)!r}")
print()
print(f"next 3 -> {next(gen)!r}")
```

Possible output:

```
created generator

A
next 1 -> "D'oh!"

B
next 2 -> 'No soup for you!'

C
next 3 -> 'The answer is 42.'
```

The generator resumes after the previous `yield`. It does not restart from the first line.

A `for` loop performs repeated `next()` calls and stops when `StopIteration` is raised.

```
def quotes():
    yield "D'oh!"
    yield "No soup for you!"
    yield "The answer is 42."
```



```
for quote in quotes():
    print(f"quote: {quote}")
```

Possible output:

```
quote: D'oh!
quote: No soup for you!
quote: The answer is 42.
```

The loop is roughly equivalent to this manual structure:

```
gen = quotes()

while True:
    try:
        quote = next(gen)
    except StopIteration:
        break

    print(f"quote: {quote}")
```

A generator is its own iterator.

```
def quotes():
    yield "D'oh!"

gen = quotes()
iterator = iter(gen)

print(f"gen: {gen}")
print(f"iterator: {iterator}")
print(f"gen is iterator: {gen is iterator}")
print(f"type(iterator): {type(iterator)}")
```

Possible output:

```
gen: <generator object quotes at 0x...>
iterator: <generator object quotes at 0x...>
gen is iterator: True
type(iterator): <class 'generator'>
```

The generator object implements both `__iter__` and `__next__`. This is why it can be used directly in a for loop.

The generator remembers local state across loop steps.

```
def running_total(values):
    total = 0

    for value in values:
        total = total + value
        print(f"inside generator: value={value}, total={total}")
        yield total

nums = [10, 20, 12]

for total in running_total(nums):
    print(f"outside loop: total={total}")
```

Possible output:

```
inside generator: value=10, total=10
outside loop:      total=10
inside generator: value=20, total=30
outside loop:      total=30
inside generator: value=12, total=42
outside loop:      total=42
```

The local variable `total` survives across yields. Each resumed step continues with the previous value of `total`.

A generator can produce an unbounded stream because it does not need to build a complete list before returning.

```
def count_up(start):
    n = start

    while True:
        yield n
        n = n + 1

gen = count_up(40)

print(next(gen))
print(next(gen))
print(next(gen))
```

Possible output:

```
40
41
42
```

This generator has no natural end. The caller decides how many values to request.

A list-building version would need to decide the full size in advance.

```
def make_three_numbers(start):
    numbers = []

    numbers.append(start)
    numbers.append(start + 1)
    numbers.append(start + 2)

    return numbers

def generate_numbers(start):
    yield start
    yield start + 1
    yield start + 2

list_result = make_three_numbers(40)
gen_result = generate_numbers(40)

print(f"list_result:      {list_result}")
print(f"type(list_result): {type(list_result)}")
```

```
print()
print(f"gen_result:      {gen_result}")
print(f"type(gen_result): {type(gen_result)}")
```

Possible output:

```
list_result:      [40, 41, 42]
type(list_result): <class 'list'>

gen_result:      <generator object generate_numbers at 0x...>
type(gen_result): <class 'generator'>
```

The list contains all values immediately. The generator contains suspended execution that can produce values later.

This is why generators are useful for lazy stream evaluation:

A generator produces the next value when asked, while preserving enough execution state to continue later.

Returning from Generators: StopIteration and Generator Completion Values

A generator ends when execution reaches the end of its body or when it executes `return`.

When a generator ends, advancing it raises `StopIteration`.

```
def one_quote():
    yield "D'oh!"

gen = one_quote()

print(next(gen))

try:
    next(gen)
except StopIteration as err:
    print(f"error type: {type(err)}")
    print(f"message:    {err}")
    print(f"err.args:    {err.args}")
```

Possible output:

```
D'oh!
error type: <class 'StopIteration'>
message:
err.args:  ()
```

The `StopIteration` object is the signal that the stream has no next value. It can be inspected.

```
err = StopIteration("finished")

print(f"err:      {err}")
print(f"type(err): {type(err)}")
print()

selected_names = [
    "args",
```

```

    "value",
    "__class__",
    "__str__",
    "__repr__",
    "with_traceback",
]

for name in selected_names:
    print(f"{name:<18} {name in dir(err)}")

print()
print(f"err.args: {err.args}")
print(f"err.value: {err.value!r}")

```

Possible output:

```

err:          finished
type(err): <class 'StopIteration'>

args          True
value         True
__class__     True
__str__       True
__repr__      True
with_traceback True

err.args: ('finished',)
err.value: 'finished'

```

A generator can use `return` with a value. That value does not appear as a normal yielded item. It becomes the `value` attribute of the `StopIteration` exception.

```

def gen_with_return():
    yield "D'oh!"
    return "finished with 42"

gen = gen_with_return()

print(next(gen))

try:
    next(gen)
except StopIteration as err:
    print(f"StopIteration.value: {err.value!r}")
    print(f"StopIteration.args: {err.args}")

```

Possible output:

```

D'oh!
StopIteration.value: 'finished with 42'
StopIteration.args: ('finished with 42',)

```

The return value is a completion value, not a stream value.

A for loop consumes `StopIteration` internally and does not expose the completion value.

```
def gen_with_return():
    yield "D'oh!"
    yield "No soup for you!"
    return "finished with 42"

for item in gen_with_return():
    print(f"item: {item}")
```

Possible output:

```
item: D'oh!
item: No soup for you!
```

The string "finished with 42" is not printed because it was not yielded. It was carried by `StopIteration` when the generator completed.

Manual iteration can observe it.

```
def gen_with_return():
    yield "D'oh!"
    yield "No soup for you!"
    return "finished with 42"

gen = gen_with_return()

while True:
    try:
        item = next(gen)
        print(f"yielded: {item!r}")
    except StopIteration as err:
        print(f"completion value: {err.value!r}")
        break
```

Possible output:

```
yielded: "D'oh!"
yielded: 'No soup for you!'
completion value: 'finished with 42'
```

A bare `return` in a generator is equivalent to completing with `None`.

```
def gen_bare_return():
    yield "Ni!"
    return

gen = gen_bare_return()

print(next(gen))

try:
    next(gen)
except StopIteration as err:
    print(f"value: {err.value!r}")
```

Possible output:

```
Ni!
value: None
```

Reaching the end of the generator body also completes with `None`.

```
def gen_end():
    yield "It is only a model."

gen = gen_end()

print(next(gen))

try:
    next(gen)
except StopIteration as err:
    print(f"value: {err.value!r}")
```

Possible output:

```
It is only a model.
value: None
```

The completion model is:

```
yield value:
    produce value to caller
    suspend generator

return final_value:
    terminate generator
    raise StopIteration(final_value) to the advancing mechanism

fall off end:
    terminate generator
    raise StopIteration(None)
```

A final example shows the full lifecycle.

```
def lifecycle():
    print("start")
    yield 40

    print("middle")
    yield 41

    print("end")
    return 42

gen = lifecycle()

print(f"created: {gen}")
print()

while True:
    try:
```

```

        value = next(gen)
        print(f"caller received yield: {value}")
        print(f"frame still exists:    {gen.gi_frame is not None}")
        print()
    except StopIteration as err:
        print(f"generator completed")
        print(f"completion value: {err.value}")
        print(f"frame still exists: {gen.gi_frame is not None}")
        break

```

Possible output:

```
created: <generator object lifecycle at 0x...>
```

```

start
caller received yield: 40
frame still exists:    True

```

```

middle
caller received yield: 41
frame still exists:    True

```

```

end
generator completed
completion value: 42
frame still exists: False

```

The generator frame exists while the generator is suspended. After completion, the frame is gone. The final rule is:

A generator call creates a generator object. Each `yield` pauses that object's frame and returns one stream value. A generator `return` terminates the frame and exposes its value only through `StopIteration.value`.

8.7.2 Generator Delegation

Generator delegation means that one generator temporarily hands part of its iteration work to another iterable or generator. Instead of writing an explicit loop that repeatedly yields values from another source, Python provides `yield from`.

At the simple level, `yield from` means:

```

for item in source:
    yield item

```

At the deeper level, when the delegated source is itself a generator, `yield from` also propagates completion values, sent values, thrown exceptions, and close requests through the generator chain.

This subsection starts with the ordinary iteration behavior and then moves to completion and exception propagation.

`yield from` as Delegated Iteration over Subgenerators and Iterables

Start with a generator that manually yields values from a list.

```

def manual_quotes():
    quotes = ["D'oh!", "No soup for you!", "The answer is 42."]

```

```

    for quote in quotes:
        yield quote

for item in manual_quotes():
    print(item)

```

Possible output:

```

D'oh!
No soup for you!
The answer is 42.

```

The generator receives values from the list and yields them one by one.
The same behavior can be written with `yield from`.

```

def delegated_quotes():
    quotes = ["D'oh!", "No soup for you!", "The answer is 42."]

    yield from quotes

for item in delegated_quotes():
    print(item)

```

Possible output:

```

D'oh!
No soup for you!
The answer is 42.

```

The line:

```
yield from quotes
```

means:

```

take values from quotes
yield each one outward through this generator

```

The delegated object can be any iterable, not only another generator.
A list works.

```

def from_list():
    yield from ["Homer", "Bart", "Lisa"]

for item in from_list():
    print(item)

```

Possible output:

```

Homer
Bart
Lisa

```

A tuple works.


```
def from_tuple():
    yield from ("Arthur", "Patsy", "Galahad")

for item in from_tuple():
    print(item)
```

Possible output:

```
Arthur
Patsy
Galahad
```

A string works because strings are iterable over characters.

```
def from_string():
    yield from "Ni!"

for item in from_string():
    print(repr(item))
```

Possible output:

```
'N'
'i'
'!'
```

A range works too.

```
def from_range():
    yield from range(40, 43)

for item in from_range():
    print(item)
```

Possible output:

```
40
41
42
```

The delegated object must be iterable.

```
def bad_delegate():
    yield from 42

gen = bad_delegate()

try:
    print(next(gen))
except TypeError as err:
    print(f"error type: {type(err)}")
    print(f"message: {err}")
```

Possible output:

```
error type: <class 'TypeError'>
message: 'int' object is not iterable
```

The integer object is not iterable, so `yield from` cannot pull values from it.
 The iterable object can be inspected before delegation.

```
items = ["D'oh!", "No soup for you!"]

print(f"type(items): {type(items)}")
print()

selected_names = [
    "__iter__",
    "__len__",
    "__getitem__",
    "append",
    "pop",
]

for name in selected_names:
    print(f"{name:<14} {name in dir(items)}")
```

Possible output:

```
type(items): <class 'list'>

__iter__      True
__len__       True
__getitem__   True
append        True
pop           True
```

The important method for `yield from` is `__iter__`. The object must be able to provide an iterator.
 A generator can also delegate to another generator.

```
def small_quotes():
    yield "D'oh!"
    yield "No soup for you!"

def all_quotes():
    yield "start"
    yield from small_quotes()
    yield "end"

for item in all_quotes():
    print(item)
```

Possible output:

```
start
D'oh!
No soup for you!
end
```

The outer generator `all_quotes()` yields `"start"`, then delegates to `small_quotes()`, then resumes after the delegated generator is exhausted.

The structure is:

```
all_quotes:
    yield "start"

    delegate to small_quotes:
        yield "D'oh!"
        yield "No soup for you!"

    yield "end"
```

This can be nested further.

```
def simpsons():
    yield "D'oh!"
    yield "Mmm... donuts."

def seinfeld():
    yield "No soup for you!"
    yield "These pretzels are making me thirsty."

def monty_python():
    yield "Ni!"
    yield "It is only a model."

def all_lines():
    yield from simpsons()
    yield from seinfeld()
    yield from monty_python()

for line in all_lines():
    print(line)
```

Possible output:

```
D'oh!
Mmm... donuts.
No soup for you!
These pretzels are making me thirsty.
Ni!
It is only a model.
```

The outer generator does not need three explicit loops. Each `yield from` delegates one segment of the stream.

The manual equivalent is:

```
def all_lines_manual():
    for line in simpsons():
        yield line

    for line in seinfeld():
        yield line

    for line in monty_python():
        yield line
```

For plain iteration, this is close to what `yield from` does. But `yield from` is deeper than loop shorthand when the delegated object is a generator.

We can inspect the outer generator while it is delegating.

```

def child():
    yield "D'oh!"
    yield "No soup for you!"

def parent():
    yield "before"
    yield from child()
    yield "after"

gen = parent()

print(f"created: gen.gi_yieldfrom = {gen.gi_yieldfrom}")

print(next(gen))
print(f"after first yield: gen.gi_yieldfrom = {gen.gi_yieldfrom}")

print(next(gen))
print(f"during delegation: gen.gi_yieldfrom = {gen.gi_yieldfrom}")

print(next(gen))
print(f"still during delegation: gen.gi_yieldfrom = {gen.gi_yieldfrom}")

print(next(gen))
print(f"after delegation: gen.gi_yieldfrom = {gen.gi_yieldfrom}")

```

Possible output:

```

created: gen.gi_yieldfrom = None
before
after first yield: gen.gi_yieldfrom = None
D'oh!
during delegation: gen.gi_yieldfrom = <generator object child at 0x...>
No soup for you!
still during delegation: gen.gi_yieldfrom = <generator object child at 0x...>
after
after delegation: gen.gi_yieldfrom = None

```

The attribute `gi_yieldfrom` shows the current delegated iterator while delegation is active. The generator object has that attribute.

```

def child():
    yield "D'oh!"

def parent():
    yield from child()

gen = parent()

print(f"type(gen): {type(gen)}")
print()

selected_names = [
    "__iter__",
    "__next__",
    "send",

```

```

    "throw",
    "close",
    "gi_frame",
    "gi_code",
    "gi_yieldfrom",
]

for name in selected_names:
    print(f"{name:<14} {name in dir(gen)}")

```

Possible output:

```

type(gen): <class 'generator'>

__iter__      True
__next__      True
send          True
throw         True
close         True
gi_frame      True
gi_code       True
gi_yieldfrom  True

```

The core delegation rule is:

`yield from iterable` exposes the delegated iterable's yielded values as if they were yielded by the outer generator.

Propagating Values, Exceptions, and Completion Through Delegated Generator Chains

The simple loop view of `yield from` explains ordinary yielded values, but it does not explain completion values.

A generator can return a value. That value is carried by `StopIteration.value`.

```

def child():
    yield "child value"
    return "child finished with 42"

gen = child()

print(next(gen))

try:
    next(gen)
except StopIteration as err:
    print(f"completion value: {err.value!r}")

```

Possible output:

```

child value
completion value: 'child finished with 42'

```

When `yield from` delegates to this generator, the outer generator can receive that completion value.

```
def child():
    yield "child value"
    return "child finished with 42"

def parent():
    result = yield from child()
    print(f"parent received completion: {result!r}")
    yield "parent after child"

for item in parent():
    print(f"caller received: {item!r}")
```

Possible output:

```
caller received: 'child value'
parent received completion: 'child finished with 42'
caller received: 'parent after child'
```

The line:

```
result = yield from child()
```

does two things:

1. yields all normal values produced by `child()`;
2. after `child()` completes, binds its return value to `result`.

So the child's `return` value does not go directly to the outside `for` loop. It becomes the value of the `yield from` expression inside the parent generator.

A plain iterable such as a list has no generator return value, so the `yield from` expression evaluates to `None` after the list is exhausted.

```
def parent():
    result = yield from ["D'oh!", "No soup for you!"]
    print(f"result after list delegation: {result!r}")
    yield "done"

for item in parent():
    print(f"caller received: {item!r}")
```

Possible output:

```
caller received: "D'oh!"
caller received: 'No soup for you!'
result after list delegation: None
caller received: 'done'
```

The list contributes yielded items, but no completion value.
Delegation can be chained.

```
def grandchild():
    yield "grandchild value"
    return "grandchild done"

def child():
```

```

    result = yield from grandchild()
    print(f"child received: {result!r}")
    return "child done"

def parent():
    result = yield from child()
    print(f"parent received: {result!r}")
    return "parent done"

gen = parent()

while True:
    try:
        item = next(gen)
        print(f"caller received: {item!r}")
    except StopIteration as err:
        print(f"caller sees completion: {err.value!r}")
        break

```

Possible output:

```

caller received: 'grandchild value'
child received: 'grandchild done'
parent received: 'child done'
caller sees completion: 'parent done'

```

The stream value flows outward:

```
grandchild yield -> child -> parent -> caller
```

The completion values flow one level upward at a time:

```

grandchild return -> child receives as yield from result
child return      -> parent receives as yield from result
parent return     -> caller sees as StopIteration.value

```

This is the main reason `yield from` is not only loop shorthand.

Exceptions can also move through a delegated chain. If the caller throws an exception into the outer generator while it is delegating, Python sends that exception into the currently delegated generator.

```

def child():
    try:
        yield "child ready"
        yield "child still running"
    except ValueError as err:
        print(f"child caught: {err}")
        yield "child recovered"

    return "child done"

def parent():
    result = yield from child()
    print(f"parent received completion: {result}")
    yield "parent done"

```

```

gen = parent()

print(next(gen))
print(gen.throw(ValueError("bad value")))
print(next(gen))

```

Possible output:

```

child ready
child caught: bad value
child recovered
parent received completion: child done
parent done

```

The call:

```
gen.throw(ValueError("bad value"))
```

throws the exception into `parent()`, but `parent()` is currently suspended at:

```
yield from child()
```

So the exception is forwarded into `child()`. The child generator catches it, yields a recovery value, then finishes normally.

The exception object is ordinary.

```

err = ValueError("bad value")

print(f"err:      {err}")
print(f"type(err): {type(err)}")
print()

```

```

selected_names = [
    "args",
    "__class__",
    "__str__",
    "__repr__",
    "with_traceback",
]

```

```

for name in selected_names:
    print(f"{name:<18} {name in dir(err)}")

print()
print(f"err.args: {err.args}")

```

Possible output:

```

err:      bad value
type(err): <class 'ValueError'>

```

```

args          True
__class__     True
__str__       True
__repr__      True
with_traceback True

```

```
err.args: ('bad value',)
```


If the delegated generator does not handle the thrown exception, the exception propagates outward.

```
def child():
    yield "child ready"
    yield "child still running"

def parent():
    yield from child()
    yield "parent done"

gen = parent()

print(next(gen))

try:
    gen.throw(ValueError("bad value"))
except ValueError as err:
    print(f"caller caught: {err}")
```

Possible output:

```
child ready
caller caught: bad value
```

The exception entered the delegated generator, was not caught there, was not caught by the parent generator, and finally reached the caller.

A parent can catch exceptions around the delegation.

```
def child():
    yield "child ready"
    yield "child still running"

def parent():
    try:
        yield from child()
    except ValueError as err:
        print(f"parent caught: {err}")
        yield "parent recovered"

    yield "parent done"

gen = parent()

print(next(gen))
print(gen.throw(ValueError("bad value")))
print(next(gen))
```

Possible output:

```
child ready
parent caught: bad value
parent recovered
parent done
```

Here the child did not handle the exception. The parent handled it after the delegated chain failed. The propagation structure is:

```

caller throws into parent
    parent is delegating to child
        exception is offered to child
        if child handles it:
            child continues
        else:
            exception returns to parent
            parent may handle it
        else:
            exception reaches caller

```

Completion and exceptions are therefore routed through the delegation chain in a structured way. A final example combines yielded values, completion values, and exception handling.

```

def child():
    try:
        yield "child: first"
        yield "child: second"
    except RuntimeError as err:
        print(f"child caught: {err}")
        yield "child: recovered"

    return "child completion value"

def parent():
    print("parent before delegation")

    result = yield from child()

    print(f"parent got completion: {result!r}")
    yield "parent: after child"

    return "parent completion value"

gen = parent()

print(next(gen))
print(next(gen))
print(gen.throw(RuntimeError("something happened")))

try:
    print(next(gen))
    print(next(gen))
except StopIteration as err:
    print(f"caller got completion: {err.value!r}")

```

Possible output:

```

parent before delegation
child: first
child: second
child caught: something happened
child: recovered
parent got completion: 'child completion value'
parent: after child

```

```
caller got completion: 'parent completion value'
```

The important events are:

1. `parent()` starts and delegates to `child()`;
2. `child()` yields values outward through `parent()`;
3. the caller throws an exception into the chain;
4. `child()` catches the exception and yields a recovery value;
5. `child()` returns a completion value;
6. `parent()` receives that value as the result of `yield from`;
7. `parent()` yields one more value;
8. `parent()` returns its own completion value;
9. the caller sees the final completion value through `StopIteration.value`.

The compact rule is:

`yield from` delegates a stream of yielded values outward and routes completion values, thrown exceptions, and generator control operations through the delegated iterator. It is a generator-chain control mechanism, not merely a shorter `for` loop.

8.7.3 Bidirectional Data Flow Pipelines: Generator-Based Coroutines

A generator normally sends values outward through `yield`. But a suspended generator can also receive values from the outside through `.send()`. This turns the generator into a simple coroutine: execution can pause, the caller can inject data, and the generator can continue using that injected data.

This is not preemptive multitasking. The generator is not interrupted by the operating system at an arbitrary instruction. It voluntarily suspends at `yield`, and the caller resumes it explicitly.

The basic direction changes from this:

```
generator -> caller
```

to this:

```
generator -> caller
generator <- caller
```

The same suspended frame is reused across both directions.

Consumers and Transformers: Feeding In-Flight Data via the `.send()` Interface

A `yield` expression can produce a value outward and later receive a value inward.

```
def receiver():
    print("receiver started")

    value = yield "ready"

    print(f"received: {value!r}")

gen = receiver()
```

```

first = next(gen)
print(f"first yielded value: {first!r}")

try:
    gen.send("D'oh!")
except StopIteration:
    print("receiver finished")

```

Possible output:

```

receiver started
first yielded value: 'ready'
received: "D'oh!"
receiver finished

```

The line:

```
value = yield "ready"
```

has two sides:

- `yield "ready"` sends `"ready"` outward and suspends;
- when the generator resumes, the value sent by `.send()` becomes the result of the `yield` expression and is assigned to `value`.

A generator must be started before it can receive a non-`None` value.

```

def receiver():
    value = yield "ready"
    print(f"received: {value!r}")

gen = receiver()

try:
    gen.send("D'oh!")
except TypeError as err:
    print(f"error type: {type(err)}")
    print(f"message: {err}")

```

Possible output:

```

error type: <class 'TypeError'>
message:    can't send non-None value to a just-started generator

```

A just-created generator is positioned before the first line of its body. There is no suspended `yield` expression ready to receive the value yet.

The usual start operation is called *priming* the generator.

```

def receiver():
    print("starting")
    value = yield "ready"
    print(f"received: {value!r}")

gen = receiver()

```

```

print("priming")
msg = next(gen)
print(f"priming result: {msg!r}")

print("sending")
try:
    gen.send("No soup for you!")
except StopIteration:
    print("done")

```

Possible output:

```

priming
starting
priming result: 'ready'
sending
received: 'No soup for you!'
done

```

Calling `next(gen)` is equivalent to `gen.send(None)` for this initial advance.

```

def receiver():
    print("starting")
    value = yield "ready"
    print(f"received: {value!r}")

gen = receiver()

msg = gen.send(None)
print(f"msg: {msg!r}")

try:
    gen.send("Ni!")
except StopIteration:
    print("done")

```

Possible output:

```

starting
msg: 'ready'
received: 'Ni!'
done

```

The generator object exposes the `send` method.

```

def receiver():
    value = yield "ready"
    print(value)

gen = receiver()

print(f"type(gen): {type(gen)}")
print()

selected_names = [

```

```

    "__next__",
    "send",
    "throw",
    "close",
    "gi_frame",
    "gi_code",
    "gi_running",
]

for name in selected_names:
    print(f"{name:<14} {name in dir(gen)}")

```

Possible output:

```

type(gen): <class 'generator'>

__next__      True
send          True
throw         True
close         True
gi_frame      True
gi_code       True
gi_running    True

```

A generator-based consumer can repeatedly receive values.

```

def printer():
    print("printer ready")

    while True:
        text = yield
        print(f"got: {text!r}")

gen = printer()

next(gen)

gen.send("D'oh!")
gen.send("No soup for you!")
gen.send("The answer is 42.")

```

Possible output:

```

printer ready
got: "D'oh!"
got: 'No soup for you!'
got: 'The answer is 42.'

```

The bare `yield` sends `None` outward and suspends. When resumed by `send(value)`, the `yield` expression evaluates to the sent value.

This consumer never naturally finishes because the loop is infinite. It waits at `yield`, receives a value, processes it, then loops back and waits again.

A transformer can receive a value and yield a transformed result.

```
def upper_transformer():
    print("transformer ready")

    while True:
        text = yield "waiting"
        result = text.upper()
        yield result

gen = upper_transformer()

print(next(gen))
print(gen.send("d'oh!"))
print(next(gen))
print(gen.send("no soup for you!"))
```

Possible output:

```
transformer ready
waiting
D'OH!
waiting
NO SOUP FOR YOU!
```

This generator has two suspension points inside the loop:

```
text = yield "waiting"
    sends "waiting"
    waits for input

yield result
    sends transformed output
    waits to be resumed again
```

The caller must follow that rhythm. After receiving a result, the caller advances once more to return the generator to the input-waiting point.

A slightly clearer design yields only after receiving input.

```
def upper_transformer():
    text = yield "ready"

    while True:
        result = text.upper()
        text = yield result

gen = upper_transformer()

print(next(gen))
print(gen.send("d'oh!"))
print(gen.send("ni!"))
print(gen.send("the answer is 42."))
```

Possible output:

```
ready
D'OH!
NI!
THE ANSWER IS 42.
```

Here, each `send()` both resumes the generator with input and receives the transformed output.
The statement:

```
text = yield result
```

means:

```
send result outward
```

```
pause
```

```
when resumed, store incoming value in text
```

The local state survives between sends.

```
def counted_receiver():
    count = 0

    while True:
        text = yield f"ready count={count}"
        count = count + 1
        print(f"{count:>2}. {text}")

gen = counted_receiver()

print(next(gen))
print(gen.send("D'oh!"))
print(gen.send("No soup for you!"))
print(gen.send("Ni!"))
```

Possible output:

```
ready count=0
1. D'oh!
ready count=1
2. No soup for you!
ready count=2
3. Ni!
ready count=3
```

The local variable `count` is preserved in the suspended generator frame.
The frame can be inspected.

```
def counted_receiver():
    count = 0

    while True:
        text = yield f"ready count={count}"
        count = count + 1
        print(f"{count:>2}. {text}")

gen = counted_receiver()

print(next(gen))
print(f"locals after prime: {gen.gi_frame.f_locals}")

print(gen.send("D'oh!"))
print(f"locals after send: {gen.gi_frame.f_locals}")
```


Possible output:

```
ready count=0
locals after prime: {'count': 0}
1. D'oh!
ready count=1
locals after send: {'count': 1, 'text': "D'oh!"}
```

The generator frame stores the live local names. The generator is not restarted for every sent value. A pipeline can connect a producer, transformer, and consumer manually.

```
def clean_transformer():
    text = yield "ready"

    while True:
        cleaned = text.strip().upper()
        text = yield cleaned

cleaner = clean_transformer()

next(cleaner)

items = [" d'oh! ", " no soup for you! ", " ni! "]

for item in items:
    result = cleaner.send(item)
    print(f"{item!r} -> {result!r}")
```

Possible output:

```
" d'oh! " -> "D'OH!"
' no soup for you! ' -> 'NO SOUP FOR YOU!'
' ni! ' -> 'NI!'
```

This is a bidirectional flow:

```
caller sends raw text into generator
generator sends cleaned text back to caller
```

The generator is a suspended transformation machine.

Exception Injection with `.throw()` and Controlled Shutdown with `.close()`

The `.throw()` method injects an exception into the generator at its current suspension point.

```
def receiver():
    try:
        while True:
            text = yield "ready"
            print(f"got: {text}")
    except ValueError as err:
        print(f"receiver caught: {err}")
        yield "recovered"

gen = receiver()
```

```
print(next(gen))
print(gen.send("D'oh!"))
print(gen.throw(ValueError("bad input")))
```

Possible output:

```
ready
got: D'oh!
ready
receiver caught: bad input
recovered
```

The exception is raised inside the generator at the suspended `yield`. The generator may catch it with `try/except`.

The exception object is ordinary.

```
err = ValueError("bad input")

print(f"err:          {err}")
print(f"type(err): {type(err)}")
print()

selected_names = [
    "args",
    "__class__",
    "__str__",
    "__repr__",
    "with_traceback",
]

for name in selected_names:
    print(f"{name:<18} {name in dir(err)}")

print()
print(f"err.args: {err.args}")
```

Possible output:

```
err:          bad input
type(err): <class 'ValueError'>

args          True
__class__     True
__str__       True
__repr__      True
with_traceback True

err.args: ('bad input',)
```

If the generator does not catch the injected exception, it propagates back to the caller.

```
def receiver():
    while True:
        text = yield "ready"
        print(f"got: {text}")
```

```

gen = receiver()

print(next(gen))

try:
    gen.throw(ValueError("bad input"))
except ValueError as err:
    print(f"caller caught: {err}")

```

Possible output:

```

ready
caller caught: bad input

```

The generator is no longer normally resumable after an uncaught exception terminates it.

```

def receiver():
    while True:
        text = yield "ready"
        print(f"got: {text}")

gen = receiver()

print(next(gen))

try:
    gen.throw(ValueError("bad input"))
except ValueError:
    print("uncaught exception terminated generator")

try:
    next(gen)
except StopIteration:
    print("generator is exhausted")

```

Possible output:

```

ready
uncaught exception terminated generator
generator is exhausted

```

The `.close()` method requests generator shutdown. Internally, it raises `GeneratorExit` at the suspension point.

```

def receiver():
    try:
        while True:
            text = yield "ready"
            print(f"got: {text}")
    finally:
        print("receiver cleanup")

gen = receiver()

```

```

print(next(gen))
print(gen.send("D'oh!"))

gen.close()

print("closed")

```

Possible output:

```

ready
got: D'oh!
ready
receiver cleanup
closed

```

The `finally` block runs during shutdown. This is the right place to release resources or print cleanup messages in educational examples.

After closing, the generator is exhausted.

```

def receiver():
    try:
        while True:
            text = yield "ready"
            print(f"got: {text}")
    finally:
        print("cleanup")

gen = receiver()

print(next(gen))
gen.close()

try:
    next(gen)
except StopIteration:
    print("cannot resume closed generator")

```

Possible output:

```

ready
cleanup
cannot resume closed generator

```

The generator object still exists as a Python object, but its suspended execution frame is gone.

```

def receiver():
    try:
        yield "ready"
    finally:
        print("cleanup")

gen = receiver()

print(f"before prime frame: {gen.gi_frame}")

```

```

print(next(gen))

print(f"before close frame: {gen.gi_frame}")

gen.close()

print(f"after close frame: {gen.gi_frame}")

```

Possible output:

```

before prime frame: <frame at 0x..., file "...", line 1, code receiver>
ready
before close frame: <frame at 0x..., file "...", line 3, code receiver>
cleanup
after close frame: None

```

Closing destroys the suspended frame state.

A generator should not yield a normal value while handling `GeneratorExit`. That violates the shutdown contract.

```

def bad_receiver():
    try:
        yield "ready"
    except GeneratorExit:
        yield "bad idea"

gen = bad_receiver()

print(next(gen))

try:
    gen.close()
except RuntimeError as err:
    print(f"error type: {type(err)}")
    print(f"message: {err}")

```

Possible output:

```

ready
error type: <class 'RuntimeError'>
message: generator ignored GeneratorExit

```

A generator may catch `GeneratorExit` for cleanup, but it must not yield another ordinary value as if execution were continuing.

A safer explicit version is:

```

def safe_receiver():
    try:
        yield "ready"
    except GeneratorExit:
        print("closing now")
        raise

gen = safe_receiver()

print(next(gen))
gen.close()

```

Possible output:

```
ready
closing now
```

The `raise` statement re-raises the `GeneratorExit` and allows shutdown to complete. The control methods can be summarized as:

Method	Meaning
<code>next(gen)</code>	resume with <code>None</code> ; receive next yielded value
<code>gen.send(x)</code>	resume and make the suspended <code>yield</code> expression evaluate to <code>x</code>
<code>gen.throw(exc)</code>	raise an exception inside the generator at the suspension point
<code>gen.close()</code>	request shutdown by injecting <code>GeneratorExit</code>

These are explicit control operations over a suspended frame.

Cooperative Multitasking: Voluntary Suspension Instead of Preemptive Scheduling

Generator-based coroutines are cooperative. They run until they reach a suspension point. They do not stop at arbitrary machine instructions.

The suspension point is explicit:

```
yield
```

A simple cooperative task can yield status messages.

```
def task(name, count):
    for i in range(1, count + 1):
        print(f"{name}: working step {i}")
        yield f"{name} paused at {i}"

    return f"{name} done"

a = task("A", 3)
b = task("B", 2)

print(next(a))
print(next(b))
print(next(a))
print(next(b))
print(next(a))
```

Possible output:

```
A: working step 1
A paused at 1
B: working step 1
B paused at 1
A: working step 2
A paused at 2
B: working step 2
B paused at 2
A: working step 3
A paused at 3
```

The caller controls which task resumes next. This is a very small scheduler.
A more systematic scheduler can keep a list of active generators.

```
def task(name, count):
    for i in range(1, count + 1):
        yield f"{name}: step {i}"

    return f"{name}: done"

tasks = [
    task("A", 3),
    task("B", 2),
    task("C", 4),
]

while tasks:
    current = tasks.pop(0)

    try:
        message = next(current)
        print(message)
        tasks.append(current)
    except StopIteration as err:
        print(err.value)
```

Possible output:

```
A: step 1
B: step 1
C: step 1
A: step 2
B: step 2
C: step 2
A: step 3
B: done
C: step 3
A: done
C: step 4
C: done
```

This scheduler performs round-robin execution:

```
take first generator
advance it until yield or completion
if it yielded:
    put it at the end of the task list
if it completed:
    remove it permanently
```

The list of tasks is ordinary.

```
def task(name, count):
    for i in range(count):
        yield f"{name}: {i}"
```

```

tasks = [
    task("A", 1),
    task("B", 1),
]

print(f"type(tasks): {type(tasks)}")
print()

selected_names = [
    "append",
    "pop",
    "__len__",
    "__getitem__",
    "__iter__",
]

for name in selected_names:
    print(f"{name:<14} {name in dir(tasks)}")

print()
print(f"type(tasks[0]): {type(tasks[0])}")

```

Possible output:

```

type(tasks): <class 'list'>

append      True
pop         True
__len__     True
__getitem__ True
__iter__    True

type(tasks[0]): <class 'generator'>

```

The scheduler is not using threads. It is not using multiple CPU cores. It is simply choosing which suspended generator frame to resume next.

The important difference from preemptive scheduling is control.

In preemptive scheduling, an external scheduler can interrupt execution without the function explicitly asking to stop.

In generator-based cooperative scheduling, the function must voluntarily yield.

This task cooperates:

```

def polite_task():
    print("step 1")
    yield

    print("step 2")
    yield

    print("step 3")
    yield

gen = polite_task()

next(gen)

```



```
print("caller regained control")

next(gen)
print("caller regained control")

next(gen)
print("caller regained control")
```

Possible output:

```
step 1
caller regained control
step 2
caller regained control
step 3
caller regained control
```

This task does not cooperate until the loop finishes.

```
def greedy_task():
    for i in range(3):
        print(f"working {i}")

        yield "finally paused"

gen = greedy_task()

print(next(gen))
```

Possible output:

```
working 0
working 1
working 2
finally paused
```

The caller gets control back only when the generator reaches `yield`. There is no automatic interruption during the loop.

A cooperative scheduler therefore depends on tasks yielding often enough.

```
def fast_task():
    for i in range(3):
        yield f"fast {i}"

def slow_to_yield_task():
    total = 0

    for i in range(1, 5):
        total = total + i
        print(f"slow internal work: {i}")

        yield f"slow total {total}"

tasks = [
    fast_task(),
```

```

        slow_to_yield_task(),
    ]

while tasks:
    current = tasks.pop(0)

    try:
        print(next(current))
        tasks.append(current)
    except StopIteration:
        pass

```

Possible output:

```

fast 0
slow internal work: 1
slow internal work: 2
slow internal work: 3
slow internal work: 4
slow total 10
fast 1
fast 2

```

The slow task monopolizes execution until it reaches its first `yield`. Cooperative multitasking requires explicit suspension points.

The architectural model is:

```

generator coroutine:
    owns a suspended frame
    runs when caller resumes it
    yields to give control back
    may receive data through send()
    may receive exceptions through throw()
    may be shut down through close()

```

```

scheduler:
    chooses which generator to resume
    has control only between yielded suspension points

```

This is the bridge toward native coroutines. Native `async def` coroutines use different objects and an event loop, but the conceptual boundary is similar: execution progresses until a structured suspension point, then control returns to an external scheduler.

The final rule is:

Generator-based coroutines implement cooperative control flow. They exchange values and control with the caller only at explicit suspension points. No hidden preemptive scheduler interrupts them in the middle of ordinary Python statements.

8.7.4 Native Coroutine Bridge

Generator-based coroutines show the basic idea of cooperative suspension: execution runs until a suspension point, then control returns to an external caller. Native coroutines use a newer syntax and a more structured execution model, but the core idea remains related.

The main syntax is:

```

async def func(...):
    ...

await something

```

An `async def` function does not behave like a normal function. Calling it creates a coroutine object. The body does not run immediately. The coroutine must be driven by an event loop or by another coroutine that awaits it.

`async def` Functions as Native Coroutine Object Factories

A normal function executes when called.

```

def normal_quote():
    print("normal body starts")
    return "D'oh!"

result = normal_quote()

print(f"result:      {result!r}")
print(f"type(result): {type(result)}")

```

Possible output:

```

normal body starts
result:      "D'oh!"
type(result): <class 'str'>

```

An `async def` function creates a coroutine object when called.

```

async def async_quote():
    print("async body starts")
    return "D'oh!"

coro = async_quote()

print(f"coro:      {coro}")
print(f"type(coro): {type(coro)}")

coro.close()

```

Possible output:

```

coro:      <coroutine object async_quote at 0x...>
type(coro): <class 'coroutine'>

```

The line "async body starts" was not printed. Calling `async_quote()` created the coroutine object, but did not start executing the coroutine body.

The `close()` call is included here because the coroutine was created only for inspection. In real code, a coroutine should normally be awaited or scheduled.

To actually run it in a simple script, use `asyncio.run()`.

```

import asyncio

async def async_quote():
    print("async body starts")

```

```

    return "D'oh!"

result = asyncio.run(async_quote())

print(f"result:      {result!r}")
print(f"type(result): {type(result)}")

```

Possible output:

```

async body starts
result:      "D'oh!"
type(result): <class 'str'>

```

The expression:

```
async_quote()
```

creates a coroutine object.

The expression:

```
asyncio.run(async_quote())
```

creates a coroutine object, gives it to an event loop, runs it to completion, and returns the coroutine's final result.

A coroutine object can be inspected.

```

async def async_quote():
    return "No soup for you!"

coro = async_quote()

print(f"type(coro): {type(coro)}")
print()

selected_names = [
    "__await__",
    "send",
    "throw",
    "close",
    "cr_frame",
    "cr_code",
    "cr_running",
    "cr_await",
]

for name in selected_names:
    print(f"{name:<14} {name in dir(coro)}")

coro.close()

```

Possible output:

```

type(coro): <class 'coroutine'>

__await__      True

```

```

send          True
throw         True
close         True
cr_frame      True
cr_code       True
cr_running    True
cr_await      True

```

Some important attributes are:

- `cr_frame`: the coroutine's frame while it exists;
- `cr_code`: the coroutine's code object;
- `cr_running`: whether it is currently executing;
- `cr_await`: what the coroutine is currently awaiting, if anything;
- `__await__`: the method that makes the object awaitable.

The coroutine's code object looks familiar.

```

async def async_quote():
    text = "The answer is 42."
    return text

coro = async_quote()
code_obj = coro.cr_code

print(f"type(code_obj): {type(code_obj)}")
print(f"co_name:         {code_obj.co_name}")
print(f"co_varnames:     {code_obj.co_varnames}")
print(f"co_consts:        {code_obj.co_consts}")

coro.close()

```

Possible output:

```

type(code_obj): <class 'code'>
co_name:         async_quote
co_varnames:     ('text',)
co_consts:       (None, 'The answer is 42.')

```

The coroutine object also has a frame before it completes.

```

async def async_quote():
    text = "Ni!"
    return text

coro = async_quote()

print(f"coro.cr_frame:      {coro.cr_frame}")
print(f"type(coro.cr_frame): {type(coro.cr_frame)}")

coro.close()

```

Possible output:

```
coro.cr_frame:      <frame at 0x..., file "...", line 1, code async_quote>
type(coro.cr_frame): <class 'frame'>
```

After the coroutine completes, its frame is gone.

```
import asyncio

saved_coro = None

async def async_quote():
    global saved_coro

    saved_coro = asyncio.current_task().get_coro()
    return "D'oh!"

result = asyncio.run(async_quote())

print(f"result: {result}")
print(f"saved_coro.cr_frame: {saved_coro.cr_frame}")
```

Possible output:

```
result: D'oh!
saved_coro.cr_frame: None
```

The completed coroutine no longer has an active frame.
The basic distinction is:

```
def function():
    calling it runs the body immediately

async def coroutine_function():
    calling it creates a coroutine object
    some scheduler must later drive that object
```

await as Structured Suspension Over Awaitable Objects

The `await` expression suspends the current coroutine until another awaitable object completes.

It can appear only inside an `async def` function.

```
import asyncio

async def wait_and_quote():
    print("before await")

    await asyncio.sleep(0)

    print("after await")
    return "Nobody expects the Spanish Inquisition!"

result = asyncio.run(wait_and_quote())

print(f"result: {result}")
```

Possible output:

```

before await
after await
result: Nobody expects the Spanish Inquisition!

```

The line:

```
await asyncio.sleep(0)
```

means:

```

create an awaitable sleep operation
suspend this coroutine until that operation completes
let the event loop run other ready work while this coroutine is suspended
resume this coroutine after the awaited operation completes

```

The object returned by `asyncio.sleep(0)` is a coroutine object.

```

import asyncio

sleep_coro = asyncio.sleep(0)

print(f"sleep_coro:      {sleep_coro}")
print(f"type(sleep_coro): {type(sleep_coro)}")
print(f"'__await__' in dir: {'__await__' in dir(sleep_coro)}")

sleep_coro.close()

```

Possible output:

```

sleep_coro:      <coroutine object sleep at 0x...>
type(sleep_coro): <class 'coroutine'>
'__await__' in dir: True

```

An object can be awaited if it is awaitable. One practical test is whether it can provide an await iterator through `__await__`.

A coroutine may await another coroutine.

```

import asyncio

async def get_name():
    print("get_name starts")
    await asyncio.sleep(0)
    print("get_name resumes")
    return "Homer"

async def build_line():
    print("build_line starts")

    name = await get_name()

    print("build_line resumes")
    return f"{name}: 42"

result = asyncio.run(build_line())

print(f"result: {result}")

```

Possible output:

```
build_line starts
get_name starts
get_name resumes
build_line resumes
result: Homer: 42
```

The outer coroutine `build_line()` suspends while waiting for `get_name()` to finish. When `get_name()` returns, its return value becomes the value of the `await` expression.

So this line:

```
name = await get_name()
```

means:

```
run get_name() as an awaitable operation
pause build_line() until get_name() completes
bind get_name()'s returned object to local name name
```

This resembles generator delegation, but it belongs to native coroutine scheduling rather than plain generator iteration.

A coroutine can keep local state across `await` suspension points.

```
import asyncio

async def counted_quote():
    count = 40

    print(f"before first await: count={count}")
    await asyncio.sleep(0)

    count = count + 1
    print(f"before second await: count={count}")
    await asyncio.sleep(0)

    count = count + 1
    print(f"before return: count={count}")
    return count

result = asyncio.run(counted_quote())

print(f"result: {result}")
```

Possible output:

```
before first await: count=40
before second await: count=41
before return: count=42
result: 42
```

The local variable `count` survives across suspension points. The coroutine frame is paused, not restarted.

The frame can be inspected while a coroutine is suspended by looking at task state from another coroutine.


```

import asyncio

async def worker():
    text = "D'oh!"

    await asyncio.sleep(0.01)

    return text

async def main():
    task = asyncio.create_task(worker())

    await asyncio.sleep(0)

    coro = task.get_coro()

    print(f"coro:      {coro}")
    print(f"type(coro): {type(coro)}")
    print(f"cr_frame:   {coro.cr_frame}")
    print(f"cr_await:   {coro.cr_await}")

    result = await task

    print(f"result:      {result}")
    print(f"cr_frame after completion: {coro.cr_frame}")

asyncio.run(main())

```

Possible output:

```

coro:      <coroutine object worker at 0x...>
type(coro): <class 'coroutine'>
cr_frame:  <frame at 0x..., file "...", line 6, code worker>
cr_await:  <coroutine object sleep at 0x...>
result:    D'oh!
cr_frame after completion: None

```

The `cr_await` attribute shows the awaitable object currently being awaited.

A coroutine that awaits nothing will run straight through once the event loop starts it.

```

import asyncio

async def immediate():
    print("immediate starts")
    return 42

result = asyncio.run(immediate())

print(f"result: {result}")

```

Possible output:

```

immediate starts
result: 42

```

The function is still an `async def` function. Calling it still creates a coroutine object. It simply has no internal suspension point once scheduled.

The compact rule is:

`await` is a structured suspension point. It pauses the current coroutine until the awaited object completes, then resumes the coroutine with the completed result.

Event Loops as External Schedulers for Coroutine Progress

A coroutine object does not advance itself. It needs a scheduler. In the standard library, the usual scheduler is an `asyncio` event loop.

The simplest script-level entry point is `asyncio.run()`.

```
import asyncio

async def main():
    print("main starts")
    await asyncio.sleep(0)
    print("main resumes")
    return "finished with 42"

result = asyncio.run(main())

print(f"result: {result}")
```

Possible output:

```
main starts
main resumes
result: finished with 42
```

The event loop drives the coroutine until completion. When the coroutine reaches `await`, the loop can run other ready work.

Two tasks can interleave cooperatively.

```
import asyncio

async def task(name, count):
    for i in range(1, count + 1):
        print(f"{name}: step {i}")
        await asyncio.sleep(0)

    return f"{name}: done"

async def main():
    task_a = asyncio.create_task(task("A", 3))
    task_b = asyncio.create_task(task("B", 2))

    result_a = await task_a
    result_b = await task_b

    print(result_a)
    print(result_b)

asyncio.run(main())
```

Possible output:

```
A: step 1
B: step 1
A: step 2
B: step 2
A: step 3
A: done
B: done
```

The exact interleaving can depend on scheduling details, but the important point is stable: each task gives control back to the event loop at `await asyncio.sleep(0)`.

The `Task` object wraps a coroutine and lets the event loop manage its progress.

```
import asyncio

async def worker():
    await asyncio.sleep(0)
    return "D'oh!"

async def main():
    task = asyncio.create_task(worker())

    print(f"task:      {task}")
    print(f"type(task): {type(task)}")
    print()

    selected_names = [
        "done",
        "result",
        "cancel",
        "get_coro",
        "add_done_callback",
        "__await__",
    ]

    for name in selected_names:
        print(f"{name:<20} {name in dir(task)}")

    result = await task

    print()
    print(f"result: {result}")
    print(f"task.done(): {task.done()}")

asyncio.run(main())
```

Possible output:

```
task:      <Task pending name='Task-2' coro=<worker() running at ...>>
type(task): <class '_asyncio.Task'>

done              True
result            True
cancel            True
```

```
get_coro          True
add_done_callback  True
__await__         True
```

```
result: D'oh!
task.done(): True
```

A task is awaitable. Awaiting a task suspends the current coroutine until that task completes. The coroutine inside the task can be inspected.

```
import asyncio

async def worker():
    await asyncio.sleep(0)
    return "No soup for you!"

async def main():
    task = asyncio.create_task(worker())
    coro = task.get_coro()

    print(f"coro:      {coro}")
    print(f"type(coro): {type(coro)}")
    print(f"cr_code:     {coro.cr_code}")
    print(f"cr_frame:     {coro.cr_frame}")

    result = await task

    print(f"result: {result}")

asyncio.run(main())
```

Possible output:

```
coro:      <coroutine object worker at 0x...>
type(coro): <class 'coroutine'>
cr_code:    <code object worker at 0x..., file "...", line 3>
cr_frame:   <frame at 0x..., file "...", line 3, code worker>
result: No soup for you!
```

The event loop does not make Python code run in parallel on multiple CPU cores. It coordinates progress between suspended coroutines.

A coroutine that never awaits will block other coroutines until it finishes.

```
import asyncio

async def greedy():
    print("greedy starts")

    for i in range(3):
        print(f"greedy working {i}")

    print("greedy ends")

async def polite():
    print("polite starts")
```

```
    await asyncio.sleep(0)
    print("polite resumes")

async def main():
    task_a = asyncio.create_task(greedy())
    task_b = asyncio.create_task(polite())

    await task_a
    await task_b

asyncio.run(main())
```

Possible output:

```
greedy starts
greedy working 0
greedy working 1
greedy working 2
greedy ends
polite starts
polite resumes
```

The `greedy()` coroutine does not suspend internally. Once it starts running, the event loop cannot move to another coroutine until `greedy()` returns.

This coroutine cooperates.

```
import asyncio

async def cooperative(name):
    for i in range(3):
        print(f"{name}: step {i}")
        await asyncio.sleep(0)

async def main():
    task_a = asyncio.create_task(cooperative("A"))
    task_b = asyncio.create_task(cooperative("B"))

    await task_a
    await task_b

asyncio.run(main())
```

Possible output:

```
A: step 0
B: step 0
A: step 1
B: step 1
A: step 2
B: step 2
```

Each `await asyncio.sleep(0)` gives control back to the event loop. The event loop can then resume another ready task.

The scheduling model is therefore still cooperative:

```

coroutine runs
    until it reaches await
        event loop regains control
        another ready coroutine may run
    awaited operation completes
    event loop resumes waiting coroutine
coroutine returns
    task becomes complete

```

The event loop itself can be accessed from inside a running coroutine.

```

import asyncio

async def main():
    loop = asyncio.get_running_loop()

    print(f"loop:      {loop}")
    print(f"type(loop): {type(loop)}")

asyncio.run(main())

```

Possible output:

```

loop:      <_UnixSelectorEventLoop running=True closed=False debug=False>
type(loop): <class 'asyncio.unix_events._UnixSelectorEventLoop'>

```

The exact event loop class can differ by operating system and Python configuration. The conceptual role is the same: it is the external scheduler that drives coroutine progress.

A simple final example shows the complete path.

```

import asyncio

async def get_score(name):
    print(f"start get_score({name})")

    await asyncio.sleep(0)

    print(f"resume get_score({name})")
    return 42

async def build_report():
    print("build_report starts")

    score = await get_score("Homer")

    print("build_report resumes")
    return f"Homer score={score}"

result = asyncio.run(build_report())

print(f"result: {result}")

```

Possible output:

```

build_report starts
start get_score(Homer)
resume get_score(Homer)
build_report resumes
result: Homer score=42

```

The execution path is:

```

asyncio.run starts event loop
  build_report coroutine starts
    awaits get_score coroutine
      get_score starts
        awaits sleep
          event loop regains control
        sleep completes
        get_score resumes
        get_score returns 42
      build_report receives 42
      build_report returns report string
    asyncio.run returns final result to normal synchronous code

```

The bridge from generator-based coroutines to native coroutines is:

Mechanism	Generator coroutine	Native coroutine
Function marker	<code>def</code> with <code>yield</code>	<code>async def</code>
Created object	generator object	coroutine object
Suspension syntax	<code>yield</code>	<code>await</code>
Manual driving	<code>next()</code> , <code>send()</code>	event loop / <code>task</code> / <code>await</code>
External scheduler	custom caller loop	<code>asyncio</code> event loop
Completion signal	<code>StopIteration.value</code>	returned result through <code>await</code> or <code>asyncio.run()</code>

The final rule is:

An `async def` call creates a coroutine object. `await` is the structured suspension point inside native coroutine code. An event loop is the external scheduler that resumes coroutines when their awaited operations are ready.

Chapter 9

Input/Output Architecture, File Streams, and External Data Boundaries

A Python program becomes practically useful when it crosses the boundary between internal runtime objects and external data systems. Values held in names, lists, dictionaries, strings, and byte sequences exist inside the running interpreter, but files, terminals, directories, streams, and serialized data formats belong to the operating-system environment outside that runtime.

This chapter explains that boundary from the ground up. It begins with standard streams and operating-system mediation, then moves through file construction, resource lifetime, text and binary access, filesystem paths, structured formats such as JSON and CSV, defensive error handling, and complete external data pipelines.

The central idea is that input/output is not just reading and writing. It is controlled translation between Python objects and external representations, managed through stream objects, encodings, buffers, paths, file handles, exceptions, and operating-system rules.

9.1 The I/O Boundary: From Runtime Objects to External Systems

A Python program does not live alone inside memory. While it runs, it creates names, objects, frames, strings, numbers, lists, and dictionaries, but useful programs must also communicate with the outside world. They receive input, produce output, read stored data, write new data, and exchange information with resources controlled by the operating system.

This section introduces that boundary. The central distinction is between runtime objects, which exist inside the interpreter process, and external data channels, which exist outside it. Variables disappear when the process ends, but files, terminal output, redirected streams, and operating-system resources belong to a larger execution environment.

For a C programmer, this is familiar territory under different names: standard streams, file descriptors, buffers, handles, and system calls. Python does not remove those layers. It wraps them in higher-level objects so that input and output can be handled through the same object-oriented runtime model used everywhere else in the language.

9.2 The I/O Boundary: From Runtime Objects to External Systems

Until this point, most Python objects have been treated as runtime entities: names refer to objects, objects live inside the interpreter process, and operations modify bindings or object state while the program is running. This is the internal world of the Python runtime. Input/output introduces a different boundary. A program no longer deals only with objects already present in memory; it exchanges data with something outside its own execution state.

This boundary is essential because useful programs rarely exist only to compute private internal values. They read commands, receive text, consume files, print reports, write logs, serialize data, communicate with terminals, and later may communicate with networks, databases, or other processes.

In every such case, Python objects must cross a boundary: internal runtime representation must be converted into some external form, or external data must be converted back into runtime objects.

The important point is that Python does not directly command the disk, keyboard, screen, or terminal device. It operates through stream objects, runtime wrappers, library calls, and ultimately operating-system services. This continues the execution model already established earlier: Python code runs inside the CPython interpreter process, and the interpreter mediates between Python-level objects and lower-level process resources.

9.2.1 Programs as Data Consumers and Data Producers

A running program can be understood as a temporary machine for transforming data. It consumes data from somewhere, holds and reshapes that data as runtime objects, and may produce data somewhere else. Sometimes the input is typed by a user. Sometimes it is stored in a file. Sometimes it comes from another process. Sometimes the output is printed to a terminal. Sometimes it is redirected into a file or piped into another program.

This means that input/output is not a decorative feature added after the language is understood. It is the mechanism through which a runtime computation becomes externally observable and externally useful.

Runtime Memory vs. Persistent Storage: Why Variables Disappear but Files Remain

A Python variable is not permanent storage. As explained earlier in the course, a Python name is a binding to an object inside the runtime environment. That object exists only while it remains reachable inside the process. When the process exits, its memory is reclaimed by the operating system. The names, objects, stack frames, temporary strings, lists, dictionaries, and other runtime structures disappear.

A file is different. A file is not a Python object living only inside the interpreter heap. A file is an operating-system-managed persistent storage object. Python can create Python-level file objects that represent access to such external resources, but the stored bytes themselves live outside the Python runtime. That is why a variable disappears when the program ends, while a saved file can still be opened tomorrow.

The following program does not write a file yet. It only shows the runtime side of the distinction.

```
msg = "Nobody expects the Spanish Inquisition!"
answer = 42

print("Runtime values currently exist inside this Python process.")
print(f"msg      = {msg}")
print(f"answer = {answer}")

print()
print("Object inspection:")
print(f"type(msg)      = {type(msg)}")
print(f"type(answer) = {type(answer)}")

print()
print("A few useful str methods visible through dir(msg):")
str_methods = ["upper", "lower", "replace", "split", "strip", "startswith"]
for name in str_methods:
    print(f"{name:12} -> {name in dir(msg)}")

print()
print("A few useful int methods visible through dir(answer):")
int_methods = ["bit_length", "to_bytes", "from_bytes", "as_integer_ratio"]
for name in int_methods:
```

```

    print(f"{name:18} -> {name in dir(answer)}")

print()
print("When this process ends, these bindings disappear.")
print("Nothing here has been stored persistently yet.")

```

The program creates two objects: a `str` object and an `int` object. The name `msg` is bound to the string object. The name `answer` is bound to the integer object. Both are runtime objects. They can be printed, inspected with `type()`, and queried with `dir()`, but they are still only part of the current execution.

The string object exposes methods such as `upper()`, `lower()`, `replace()`, `split()`, and `strip()` because strings are textual runtime objects. The integer object exposes methods such as `bit_length()` because integers are numeric runtime objects. These methods describe what can be done to the object while it exists in the interpreter.

Persistent storage will require crossing the I/O boundary. That will be done later through file objects. For now, the invariant is simple: variables are runtime bindings; files are external storage resources.

Standard Streams: `stdin`, `stdout`, and `stderr` as Process-Level Communication Channels

Every ordinary command-line process begins with a small set of standard communication channels. These are usually called standard input, standard output, and standard error.

`stdin` is the process-level channel from which ordinary input is read. In an interactive terminal, this usually means keyboard input. If a program is connected to a pipe, `stdin` may receive text produced by another program.

`stdout` is the process-level channel used for ordinary output. When Python code calls `print()`, the resulting text normally goes to `stdout`. In a terminal, this appears on the screen. If output is redirected, the same text may go into a file or into another process.

`stderr` is the process-level channel used for diagnostics, warnings, and error messages. It is separated from `stdout` so that normal program output and error reporting can be handled independently.

In Python, these channels are exposed through the `sys` module as `sys.stdin`, `sys.stdout`, and `sys.stderr`.

```

import sys

print("Standard stream objects exposed by Python:")
print(f"sys.stdin -> {sys.stdin}")
print(f"sys.stdout -> {sys.stdout}")
print(f"sys.stderr -> {sys.stderr}")

print()
print("Their runtime types:")
print(f"type(sys.stdin) = {type(sys.stdin)}")
print(f"type(sys.stdout) = {type(sys.stdout)}")
print(f"type(sys.stderr) = {type(sys.stderr)}")

print()
print("Selected names found through dir(sys.stdout):")
stream_names = ["write", "flush", "read", "readline", "isatty", "encoding"]
for name in stream_names:
    print(f"{name:10} -> {name in dir(sys.stdout)}")

print()
print("Writing through print():")

```

```

print("D'oh! This line goes to stdout by default.")

print()
print("Writing directly through sys.stdout.write():")
sys.stdout.write("Chuck Norris can divide by zero, but Python still should not.\n")
sys.stdout.flush()

print()
print("Writing directly through sys.stderr.write():")
sys.stderr.write("This is a diagnostic message sent to stderr.\n")
sys.stderr.flush()

```

The object returned by `type(sys.stdout)` is commonly a text stream wrapper. In CPython, this is usually an object of type `_io.TextIOWrapper`. That detail matters because `stdout` is not merely “the screen”. At Python level, it is an object with methods.

The method `write()` sends text to the stream. Unlike `print()`, it does not automatically add a newline. The method `flush()` asks the stream to push buffered output forward instead of keeping it temporarily in memory. The method `isatty()` can report whether the stream is connected to an interactive terminal-like device. The `encoding` attribute describes the text encoding used by the stream.

This is already a layered architecture. The Python program calls methods on Python objects. Those Python objects belong to the interpreter runtime. Beneath them, lower-level library and operating-system mechanisms eventually transfer bytes to the terminal, pipe, or redirected destination.

A small interactive example shows the input side:

```

import sys

print("Before input(), Python has access to sys.stdin:")
print(f"type(sys.stdin) = {type(sys.stdin)}")

print()
user_text = input("Write one short phrase: ")

print()
print("input() returned a Python object:")
print(f"user_text      = {user_text}")
print(f"type(user_text) = {type(user_text)}")

print()
print("A few names from dir(user_text):")
for name in ["upper", "lower", "replace", "split", "strip"]:
    print(f"{name:10} -> {name in dir(user_text)}")

print()
print(f"Uppercase version: {user_text.upper()}")
print(f"Length: {len(user_text)}")

```

The important boundary is visible here. The user’s typed characters arrive through `stdin`. Python’s `input()` function reads a line from that stream and returns a `str` object. From that point on, the data is no longer merely terminal input. It is a normal Python string, with normal string methods and normal runtime behavior.

C Comparison: `printf()`, `scanf()`, File Descriptors, and Python's Higher-Level Stream Objects

The C programmer already knows this territory in a lower-level form. In C, ordinary terminal output is often produced with `printf()`, and ordinary input may be consumed with `scanf()`, `fgets()`, or related functions. These functions operate through C's standard I/O library, commonly using `stdin`, `stdout`, and `stderr` as `FILE *` streams.

In a POSIX-style model, beneath those C streams are file descriptors: small integer handles owned by the process. Conventionally, descriptor 0 is standard input, descriptor 1 is standard output, and descriptor 2 is standard error. These integers are not the data itself. They are process-local references to open operating-system resources.

Python lives above that layer. Python exposes higher-level stream objects. These objects know about text encoding, buffering, newline handling, and methods such as `write()`, `readline()`, and `flush()`. The programmer usually works with these objects instead of manually operating on raw descriptors.

The following example shows the relationship without going deeply into descriptor-level programming yet.

```
import sys

print("Python-level standard streams:")
print(f"stdin object : {sys.stdin}")
print(f"stdout object: {sys.stdout}")
print(f"stderr object: {sys.stderr}")

print()
print("Selected stream attributes:")
print(f"stdout encoding = {sys.stdout.encoding}")
print(f"stdout isatty   = {sys.stdout.isatty()}")

print()
print("C-style idea:")
print("printf(...) writes formatted text to stdout.")
print("scanf(...) reads formatted input from stdin.")

print()
print("Python-style idea:")
print("print(...) formats objects and writes text to sys.stdout.")
print("input(...) reads text from sys.stdin and returns a str object.")

print()
print("Direct stream write with explicit formatting:")
name = "Kramer"
score = 42
sys.stdout.write(f"{name:10} | score = {score:04d}\n")
sys.stdout.flush()
```

The expression `f"{name:10} | score = {score:04d}"` is ordinary Python string formatting. It creates a `str` object before the stream receives anything. Then `sys.stdout.write()` sends that string into the output stream. The stream is responsible for taking Python text and pushing it toward the external destination.

So the comparison can be summarized precisely. In C, the programmer commonly thinks in terms of native process resources, C library streams, buffers, and format strings. In Python, the programmer usually thinks in terms of runtime objects and stream objects. The operating-system boundary still exists, but Python wraps it in managed objects that fit the rest of the object model.

This is the core idea of the I/O boundary: Python code manipulates objects inside the runtime, but useful input and output require those objects to cross into process-level communication channels or external storage systems.

9.2.2 The Operating System Mediation Layer

Input/output looks simple at Python level because the code often uses direct-looking operations such as `print()`, `input()`, or later `open()`. That simplicity hides an important fact: Python is not itself the owner of the keyboard, terminal, disk, or filesystem. Those resources are controlled by the operating system.

The Python interpreter runs as a normal process. When it needs external input or output, it must ask the operating system to perform work on its behalf. This is not a Python limitation. It is the normal protection boundary of modern operating systems. User programs are not allowed to freely touch hardware or shared kernel-managed resources. They request service through controlled operating-system interfaces.

This means that Python I/O is layered. At the top, Python code manipulates objects. Beneath that, CPython uses runtime wrappers and library machinery. Beneath that, the operating system manages handles, descriptors, buffers, devices, files, and permissions.

Why Python Does Not Read Disks Directly: System Calls, File Handles, and Kernel Buffers

A disk is not a Python object. It is a hardware device managed by the operating system. The filesystem is also not a Python structure. It is an operating-system-managed organization of directories, file names, metadata, permissions, and stored byte ranges.

When Python reads from or writes to an external resource, the operation eventually becomes an operating-system request. In lower-level language, such requests are often discussed as system calls. A system call is a controlled transition from ordinary user program execution into operating-system-managed service code.

Python normally hides the raw system-call boundary. The programmer does not usually call the kernel directly. Instead, Python exposes stream objects and library functions. Those Python-level operations eventually cause lower-level reads, writes, flushes, opens, closes, and seeks.

A simple standard-output example is enough to show the direction of travel.

```
import sys

msg = "These are not the droids you are looking for."
count = 42

print("Python object creation:")
print(f"msg      = {msg}")
print(f"count    = {count}")

print()
print("Object types:")
print(f"type(msg)      = {type(msg)}")
print(f"type(count)     = {type(count)}")
print(f"type(sys.stdout) = {type(sys.stdout)}")

print()
print("Selected names from dir(sys.stdout):")
for name in ["write", "flush", "fileno", "isatty", "encoding", "buffer"]:
    print(f"{{name:10}} -> {{name in dir(sys.stdout) }}")

print()
```

```
print("Now Python sends text toward an external destination.")
sys.stdout.write("Serenity now!\n")
sys.stdout.flush()
```

The variable `msg` refers to a Python `str` object. The variable `count` refers to a Python `int` object. These are ordinary runtime objects. By contrast, `sys.stdout` is a stream object connected to a process-level output channel.

The method `write()` accepts text. The method `flush()` forces buffered output to move forward. The method `fileno()` exposes the lower-level integer descriptor behind the stream, where available. The method `isatty()` asks whether the stream is connected to a terminal-like device.

The crucial idea is that the call is not magic. Python does not throw characters at the screen directly. It sends data through a stream object, and that stream object ultimately relies on lower-level operating-system machinery.

File Descriptors vs. Python File Objects: Native Resource Handles Wrapped in Managed Runtime Objects

At operating-system level, an open process resource is commonly represented by a small handle. On POSIX-like systems, this is often called a file descriptor. A file descriptor is usually just an integer inside the process. The traditional descriptors are:

- 0: standard input,
- 1: standard output,
- 2: standard error.

These integers are not the stream data. They are not strings, files, or buffers. They are process-local references to operating-system-managed resources.

Python usually wraps such low-level handles inside higher-level objects. For example, `sys.stdout` is not merely the integer 1. It is a Python object with methods, attributes, buffering behavior, encoding behavior, and error-handling behavior.

```
import sys

stdout_obj = sys.stdout
stdout_fd = sys.stdout.fileno()

print("Python stream object and lower-level descriptor:")
print(f"stdout_obj = {stdout_obj}")
print(f"stdout_fd = {stdout_fd}")

print()
print("Their types:")
print(f"type(stdout_obj) = {type(stdout_obj)}")
print(f"type(stdout_fd) = {type(stdout_fd)}")

print()
print("Selected names from dir(stdout_fd):")
for name in ["bit_length", "to_bytes", "from_bytes", "real", "imag"]:
    print(f"{name:12} -> {name in dir(stdout_fd)}")

print()
print("Selected names from dir(stdout_obj):")
for name in ["write", "flush", "fileno", "isatty", "encoding"]:
    print(f"{name:12} -> {name in dir(stdout_obj)}")
```

```
print()
print("The descriptor is a small integer handle.")
print("The Python stream object is the managed wrapper used by normal code.")
```

The distinction is architectural. The descriptor is closer to the operating-system process model. The Python stream object is closer to the Python object model.

The descriptor can identify the resource to the lower layers. The Python object provides a safer and more convenient interface for Python programs. It can translate strings to bytes using an encoding, apply buffering, expose methods, and participate in Python's normal object inspection mechanisms.

A more direct descriptor-level write can be performed through the standard `os` module. This is intentionally lower-level than normal `print()` usage.

```
import os
import sys

text = "Chuck Norris counted to infinity. Twice.\n"
data = text.encode("utf-8")

print("Text object before crossing the byte boundary:")
print(f"text          = {text.strip()}")
print(f"type(text) = {type(text)}")

print()
print("Encoded byte object:")
print(f"data          = {data}")
print(f"type(data) = {type(data)}")

print()
print("Selected names from dir(data):")
for name in ["decode", "hex", "startswith", "split", "replace"]:
    print(f"{name:12} -> {name in dir(data)}")

print()
print("Writing bytes directly to file descriptor 1:")
os.write(1, data)

sys.stdout.flush()
```

Here `text` is a Python `str`. The call to `encode("utf-8")` converts it into a `bytes` object. That matters because low-level operating-system writes operate on bytes, not on Python's abstract Unicode string objects.

The `bytes` object has methods such as `decode()`, `hex()`, `split()`, and `replace()`. It is not the same kind of object as `str`. A `str` represents text. A `bytes` object represents a sequence of byte values.

This anticipates a later distinction in the chapter: text streams perform encoding and decoding; binary streams transfer bytes directly.

Buffering Layers: Reducing Expensive Kernel Transitions Through Intermediate Memory Buffers

Crossing from Python execution into operating-system I/O is more expensive than ordinary object manipulation in memory. A normal arithmetic operation or string method call happens inside the process. An I/O operation may require interaction with the kernel, terminal driver, filesystem cache, disk subsystem, or another process.

Because of that cost, I/O systems often use buffering. A buffer is an intermediate memory area used to collect data before sending it onward, or to hold received data before the program consumes it. Instead of making a separate operating-system request for every tiny piece of output, the runtime can collect several pieces and transfer them in larger chunks.

This is why output sometimes does not appear exactly at the instant a program calls a write method. The data may be temporarily held in a buffer. The method `flush()` asks the stream to push buffered data forward immediately.

```
import sys

print("Normal print() usually writes through sys.stdout.")
print("The stream may use buffering internally.")

print()
print("Selected buffering-related names:")
for name in ["write", "flush", "buffer", "line_buffering", "write_through"]:
    print(f"{name:16} -> {name in dir(sys.stdout)}")

print()
print("Some stream attributes:")
print(f"encoding      = {sys.stdout.encoding}")
print(f"is terminal    = {sys.stdout.isatty()}")

if "line_buffering" in dir(sys.stdout):
    print(f"line buffering = {sys.stdout.line_buffering}")

if "write_through" in dir(sys.stdout):
    print(f"write through  = {sys.stdout.write_through}")

print()
sys.stdout.write("This text was sent with sys.stdout.write().\n")
sys.stdout.write("Now flush() is called explicitly.\n")
sys.stdout.flush()

print()
print("Buffering is not part of the logical text itself.")
print("It is part of the transfer machinery between runtime and outside world.")
```

Buffering should not be confused with persistence. A buffer is temporary memory used during transfer. A file is persistent external storage. A Python variable is a runtime binding. These are three different things.

A program may create a string in memory, send it to a stream, have that stream temporarily buffer the encoded bytes, and later have the operating system transfer those bytes to a terminal, pipe, or file. The same visible line of text may therefore pass through several representations and layers.

The practical rule is this: Python code works with objects, but external communication requires managed transfer. That transfer is mediated by stream objects, encoding rules, buffers, descriptors or handles, and operating-system services.

9.3 File Opening, Closing, and Resource Lifetime

Files are external resources, not ordinary in-memory values. Opening a file creates a Python object that gives the program temporary access to an operating-system-managed resource. That access has a lifetime: it begins when the file is opened and must end when the file is no longer needed.

This section explains how `open()` constructs file objects, how modes control reading, writing, appending, creation, text access, and binary access, and why closing files is a required resource-management rule. It also introduces the `with` statement as Python's normal way to bind file usage to a safe, visible block of code.

9.3.1 The `open()` Function and File Object Construction

The `open()` function is the ordinary entry point through which Python code obtains access to a file. It does not return the file contents directly. It returns a runtime object that represents an open communication channel between the Python process and an external filesystem resource.

That object is usually called a file object or stream object. It is a Python-level wrapper around lower-level operating-system access. The file object has methods such as `read()`, `write()`, `close()`, `flush()`, `seek()`, and `tell()`. Through these methods, Python code can move data across the boundary between runtime memory and persistent storage.

Path Argument, Mode Argument, Encoding Argument, and Runtime File Object Creation

The most common call shape is:

```
open(path, mode, encoding=...)
```

The `path` argument identifies the external file. The `mode` argument tells Python what kind of access is requested. The `encoding` argument matters in text mode because stored bytes must be decoded into `str` objects when reading, or encoded from `str` objects when writing.

```
path = "io_demo_1.txt"
text = "It's just a flesh wound.\n"

file_obj = open(path, "w", encoding="utf-8")

print("The file object was created by open().")
print(f"path      = {path}")
print(f"text      = {text.strip()}")
print(f"file_obj = {file_obj}")

print()
print("Object types:")
print(f"type(path)      = {type(path)}")
print(f"type(text)       = {type(text)}")
print(f"type(file_obj)    = {type(file_obj)}")

print()
print("Selected names from dir(file_obj):")
for name in ["write", "read", "close", "flush", "seek", "tell", "encoding", "mode"]:
    print(f"{name:10} -> {name in dir(file_obj)}")

print()
written_count = file_obj.write(text)
file_obj.close()

print(f"write() returned: {written_count}")
print("The returned number is the number of text characters written.")
print("The file now exists outside the Python runtime.")
```

The name `file_obj` does not contain the file itself. It refers to a Python object that manages access to the external file. The file object knows its mode, its encoding in text mode, and the operations allowed by that mode.

The call to `write()` receives a `str` object. Because the file was opened in text mode with `encoding="utf-8"`, Python encodes that string into bytes before they are stored externally. The call to `close()` releases the open file resource. Resource lifetime will be examined more carefully in the next subsection.

Read Modes, Write Modes, Append Modes, and Exclusive Creation Modes

The mode string controls the intended access pattern. The basic modes are:

- `"r"`: read an existing file;
- `"w"`: write a file, replacing existing contents;
- `"a"`: append to the end of a file;
- `"x"`: create a new file, failing if it already exists.

These modes are not cosmetic. They change what the file object is allowed to do and what happens to existing stored data.

```
path = "io_modes_demo.txt"

file_obj = open(path, "w", encoding="utf-8")
file_obj.write("First line: D'oh!\n")
file_obj.close()

print("After write mode:")
file_obj = open(path, "r", encoding="utf-8")
content = file_obj.read()
file_obj.close()
print(content)

file_obj = open(path, "a", encoding="utf-8")
file_obj.write("Second line: Serenity now!\n")
file_obj.close()

print("After append mode:")
file_obj = open(path, "r", encoding="utf-8")
content = file_obj.read()
file_obj.close()
print(content)

print("Type inspection:")
print(f"type(content) = {type(content)}")

print()
print("Selected names from dir(content):")
for name in ["splitlines", "upper", "replace", "strip", "count"]:
    print(f"{name:12} -> {name in dir(content)}")

print()
print("Trying exclusive creation on an existing file:")
try:
```

```

file_obj = open(path, "x", encoding="utf-8")
file_obj.write("This should not replace the old file.\n")
file_obj.close()
except FileExistsError as err:
    print(f"FileExistsError: {err}")

```

The first "w" opening creates or replaces the file. The later "a" opening preserves the existing data and adds new text at the end. The later "r" openings read the stored data back into Python as a `str` object.

The "x" mode is useful when overwriting would be a bug. It asks Python to create the file only if the path does not already exist. If the file already exists, Python raises `FileExistsError` instead of silently destroying previous contents.

Text Mode vs. Binary Mode: `str` Streams vs. `bytes` Streams

A file can be opened in text mode or binary mode.

Text mode works with `str` objects. Python performs encoding and decoding at the boundary. This is appropriate for ordinary textual data: source code, logs, CSV files, JSON files, configuration files, and plain text documents.

Binary mode works with `bytes` objects. Python does not interpret the bytes as text. This is appropriate for images, archives, compiled files, serialized binary formats, and any file where raw byte values matter.

The mode string controls this distinction. Modes such as "r", "w", and "a" are text modes by default. Modes such as "rb" and "wb" are binary modes.

```

text_path = "io_text_demo.txt"
bin_path = "io_binary_demo.bin"

text_msg = "The answer is 42, but the question is unstable.\n"

file_obj = open(text_path, "w", encoding="utf-8")
file_obj.write(text_msg)
file_obj.close()

file_obj = open(text_path, "r", encoding="utf-8")
text_data = file_obj.read()
file_obj.close()

print("Text mode result:")
print(f"text_data          = {text_data.strip()}")
print(f"type(text_data) = {type(text_data)}")

print()
byte_data = text_msg.encode("utf-8")

print("Manual encoding creates bytes:")
print(f"byte_data          = {byte_data}")
print(f"type(byte_data) = {type(byte_data)}")

print()
print("Selected names from dir(byte_data):")
for name in ["decode", "hex", "split", "replace", "startswith"]:
    print(f"{name:12} -> {name in dir(byte_data)}")

```

```

file_obj = open(bin_path, "wb")
file_obj.write(byte_data)
file_obj.close()

file_obj = open(bin_path, "rb")
raw_data = file_obj.read()
file_obj.close()

print()
print("Binary mode result:")
print(f"raw_data          = {raw_data}")
print(f"type(raw_data)    = {type(raw_data)}")

print()
decoded_again = raw_data.decode("utf-8")
print("Decoded back into text:")
print(f"decoded_again      = {decoded_again.strip()}")
print(f"type(decoded_again) = {type(decoded_again)}")

```

In text mode, `read()` returns a `str`. In binary mode, `read()` returns a `bytes` object. This is the central difference.

The text file example lets Python handle the conversion between `str` and stored bytes through the declared encoding. The binary file example performs the conversion manually with `encode()` and `decode()`. This makes the boundary visible: external storage ultimately contains bytes, while Python text is represented as `str` objects inside the runtime.

The practical rule is direct: use text mode when the file represents text, and always specify an encoding such as `"utf-8"`. Use binary mode when the file represents raw bytes and must not be decoded as text.

9.3.2 Resource Management Invariants

Opening a file creates a runtime object connected to an external operating-system resource. That connection must not be treated like an ordinary temporary string or integer. A file object has a lifetime that matters outside normal Python object manipulation, because the operating system has allocated a handle for the process.

The central invariant is simple: every successfully opened file should be closed when the program no longer needs it. Closing releases the external resource, flushes pending buffered output when appropriate, and marks the Python file object as no longer usable for ordinary read and write operations.

Manual Closing with `.close()` and the Risk of Leaked File Handles

The most direct way to close a file is to call its `close()` method manually. This makes the lifetime visible: first the program opens the file, then it uses the file, then it closes the file.

```

path = "manual_close_demo.txt"

file_obj = open(path, "w", encoding="utf-8")

print("File object after open():")
print(f"file_obj          = {file_obj}")
print(f"type(file_obj)    = {type(file_obj)}")
print(f"file_obj.closed    = {file_obj.closed}")

print()
print("Selected names from dir(file_obj):")

```

```

for name in ["write", "flush", "close", "closed", "read", "seek", "tell"]:
    print(f"{name:10} -> {name in dir(file_obj)}")

print()
file_obj.write("No soup for you!\n")
file_obj.write("The answer is still 42.\n")

print("Before close():")
print(f"file_obj.closed = {file_obj.closed}")

file_obj.close()

print()
print("After close():")
print(f"file_obj.closed = {file_obj.closed}")

```

The `closed` attribute reports whether the file object is already closed. Before `close()`, it is `False`. After `close()`, it is `True`. The object still exists as a Python object, but its connection to the external file resource has been shut down.

Trying to use a closed file for ordinary writing is an error.

```

path = "closed_file_demo.txt"

file_obj = open(path, "w", encoding="utf-8")
file_obj.write("I'm not dead yet.\n")
file_obj.close()

print("Closed file state:")
print(f"file_obj.closed = {file_obj.closed}")

print()
try:
    file_obj.write("This write happens too late.\n")
except ValueError as err:
    print("Python rejected the write.")
    print(f"type(err) = {type(err)}")
    print(f"err      = {err}")

print()
print("Selected names from dir(err):")
for name in ["args", "with_traceback", "add_note"]:
    print(f"{name:16} -> {name in dir(err)}")

```

Manual closing works, but it is fragile. If an error occurs between `open()` and `close()`, the closing call may be skipped unless the programmer has written explicit cleanup logic. This is the leaked-handle risk: the program has acquired an external resource but does not reliably release it.

Small scripts may terminate quickly enough that the operating system cleans up the process resources after exit. That is not a design rule. Long-running programs, servers, data pipelines, and repeated file operations must not rely on process termination as their cleanup mechanism.

Context Managers: The `with` Statement as Structured Resource Lifetime Control

The usual Python solution is the `with` statement. A `with` block gives the resource a visible lexical lifetime. The file is open inside the block. When execution leaves the block, Python closes the file automatically.

```

path = "with_demo.txt"

with open(path, "w", encoding="utf-8") as file_obj:
    print("Inside the with block:")
    print(f"file_obj          = {file_obj}")
    print(f"type(file_obj)    = {type(file_obj)}")
    print(f"file_obj.closed = {file_obj.closed}")

    file_obj.write("Spam, spam, spam, eggs, and spam.\n")
    file_obj.write("This file is managed by a with block.\n")

print()
print("After the with block:")
print(f"file_obj.closed = {file_obj.closed}")

```

The name `file_obj` may still exist after the block, because Python block statements do not create a new local scope. But the file itself is closed. The object remains inspectable, while the external resource connection has ended.

This is why `with open(...)` is usually preferred over manual `open()` followed by manual `close()`. The resource lifetime is attached to the structure of the code. The indentation shows exactly where the file is valid for normal operations.

A read example shows the same invariant from the input side.

```

path = "with_read_demo.txt"

with open(path, "w", encoding="utf-8") as file_obj:
    file_obj.write("Yada yada yada.\n")
    file_obj.write("Runtime objects cross into persistent storage.\n")

with open(path, "r", encoding="utf-8") as file_obj:
    content = file_obj.read()

    print("Inside read block:")
    print(f"type(file_obj) = {type(file_obj)}")
    print(f"type(content)  = {type(content)}")
    print(f"content          = {content.strip()}")

    print()
    print("Selected names from dir(content):")
    for name in ["splitlines", "upper", "replace", "strip", "count"]:
        print(f"{name:12} -> {name in dir(content)}")

print()
print("After read block:")
print(f"file_obj.closed = {file_obj.closed}")

```

The file object is temporary access machinery. The `content` variable is different. It refers to a `str` object created by reading from the file. After the file closes, the string remains available because it is now an ordinary runtime object.

The `__enter__()` / `__exit__()` Protocol Behind `with`

The `with` statement is not special only for files. It is built on a general protocol. An object used in a `with` statement must provide methods named `__enter__()` and `__exit__()`.

For file objects, `__enter__()` prepares the object for use and returns the object assigned after `as`. The `__exit__()` method performs cleanup when the block is left. For files, that cleanup includes closing the file.

The protocol can be inspected directly on a file object.

```
path = "protocol_demo.txt"

file_obj = open(path, "w", encoding="utf-8")

print("Protocol methods on a file object:")
print(f"__enter__ in dir(file_obj): {'__enter__' in dir(file_obj)}")
print(f"__exit__ in dir(file_obj): {'__exit__' in dir(file_obj)}")

print()
print("Selected names from dir(file_obj):")
for name in ["__enter__", "__exit__", "write", "close", "closed"]:
    print(f"{name:12} -> {name in dir(file_obj)}")

print()
entered_obj = file_obj.__enter__()

print("After explicit __enter__():")
print(f"entered_obj is file_obj = {entered_obj is file_obj}")
print(f"type(entered_obj)      = {type(entered_obj)}")
print(f"file_obj.closed          = {file_obj.closed}")

entered_obj.write("This line was written after __enter__().\n")
entered_obj.write("The Ministry of Silly Walks approves this protocol.\n")

file_obj.__exit__(None, None, None)

print()
print("After explicit __exit__():")
print(f"file_obj.closed = {file_obj.closed}")
```

This manual call style is shown only to reveal the mechanism. Normal code should use the `with` statement, not direct calls to `__enter__()` and `__exit__()`.

Conceptually, this block:

```
with open(path, "w", encoding="utf-8") as file_obj:
    file_obj.write("Hello.\n")
```

means roughly this:

```
file_obj = open(path, "w", encoding="utf-8")
managed_obj = file_obj.__enter__()

try:
    managed_obj.write("Hello.\n")
finally:
    file_obj.__exit__(None, None, None)
```

The real interpreter behavior also passes exception information into `__exit__()` when an error occurs. That is what makes the protocol useful for reliable cleanup.

Exception-Safe Cleanup: Why File Handles Close Even When Errors Occur Inside the Block

The strongest reason to use `with` is exception-safe cleanup. If an error happens inside the block, Python still leaves the block through the context-manager protocol. The file object's `__exit__()` method is still called, so the file is closed.

```
path = "exception_cleanup_demo.txt"

try:
    with open(path, "w", encoding="utf-8") as file_obj:
        print("Inside with block, before the error:")
        print(f"file_obj.closed = {file_obj.closed}")

        file_obj.write("Everything is fine so far.\n")
        file_obj.write("Then the program hits a deliberate error.\n")

        number = 42
        result = number / 0

        file_obj.write("This line is never reached.\n")

except ZeroDivisionError as err:
    print()
    print("The exception was caught outside the with block.")
    print(f"type(err) = {type(err)}")
    print(f"err      = {err}")

print()
print("After the failed with block:")
print(f"file_obj.closed = {file_obj.closed}")
```

The division by zero interrupts the normal flow of the block. The last write is not reached. Even so, the file is closed before control reaches the `except` block outside.

This is the resource-management invariant: cleanup must not depend on the happy path. A program that opens external resources must release them even when execution is interrupted by an exception. The `with` statement encodes that rule directly into the shape of the program.

For files, the practical rule is strict: use `with open(...)` unless there is a specific reason not to. It keeps acquisition and release in the same visible structure, prevents many leaked-handle mistakes, and matches Python's general object protocol for managed external resources.

9.4 Text File Reading and Writing

Text files are the most common external data form a beginner encounters, but they are not merely large strings stored somewhere else. A text file is stored as bytes outside the Python process, while Python code works with `str` objects inside the runtime. Reading and writing text therefore means crossing a translation boundary.

This section explains the practical mechanics of that boundary. It shows how complete files, individual lines, and batches of lines are read into runtime objects; how strings are written back to external storage; and why newline handling and character encoding are not minor details but part of the file format itself.

The central rule is simple: text-mode file operations let Python programs treat external textual data as `str` objects, while the file object manages the conversion between runtime text and stored byte sequences.

9.4.1 Reading Textual Data

Reading a text file means asking Python to pull stored bytes from an external file resource, decode those bytes into `str` objects, and return those strings to the running program. Once the text has been read, it behaves like ordinary Python text. It can be printed, sliced, split, counted, searched, and transformed with normal string methods.

The important boundary is still present: the file stores bytes outside the Python process, while the program receives `str` objects inside the Python runtime.

Full-File Loading with `.read()` and Memory Consumption Boundaries

The `.read()` method loads the remaining contents of a text file into one `str` object. This is convenient for small files because the program receives the whole text at once.

```
path = "read_full_demo.txt"

with open(path, "w", encoding="utf-8") as file_obj:
    file_obj.write("Line 1: Nobody expects the Spanish Inquisition!\n")
    file_obj.write("Line 2: The answer is 42.\n")
    file_obj.write("Line 3: D'oh!\n")

with open(path, "r", encoding="utf-8") as file_obj:
    content = file_obj.read()

    print("File object while still open:")
    print(f"file_obj      = {file_obj}")
    print(f"type(file_obj) = {type(file_obj)}")

    print()
    print("Selected names from dir(file_obj):")
    for name in ["read", "readline", "readlines", "close", "seek", "tell"]:
        print(f"{name:12} -> {name in dir(file_obj)}")

print()
print("Text returned by read():")
print(content)

print("Object returned by read():")
print(f"type(content) = {type(content)}")
print(f"len(content)    = {len(content)}")

print()
print("Selected names from dir(content):")
for name in ["splitlines", "upper", "replace", "strip", "count", "find"]:
    print(f"{name:12} -> {name in dir(content)}")

print()
print(f"Number of lines: {len(content.splitlines())}")
print(f"Count of letter e: {content.count('e')}")
```

The result of `read()` is one string. The file may contain several lines, but Python still returns one `str` object containing all of them.

This is simple, but it has a cost. If the file is large, `read()` attempts to place the entire file content into memory at once. For a small configuration file or a short note, that is fine. For a multi-gigabyte log file, it is a poor access pattern because the program may consume too much memory.

The practical rule is direct: use `read()` when the file is reasonably small and the program truly needs the complete content at once.

Line-Based Reading with `.readline()` and Iteration over File Objects

The `.readline()` method reads one line at a time. A line usually includes its final newline character if one exists. When there is nothing left to read, `readline()` returns the empty string `""`.

```
path = "readline_demo.txt"

with open(path, "w", encoding="utf-8") as file_obj:
    file_obj.write("First: Serenity now!\n")
    file_obj.write("Second: No soup for you!\n")
    file_obj.write("Third: 42 is still suspicious.\n")

with open(path, "r", encoding="utf-8") as file_obj:
    line1 = file_obj.readline()
    line2 = file_obj.readline()
    line3 = file_obj.readline()
    line4 = file_obj.readline()

print("Lines returned by readline():")
print(f"line1 = {line1!r}")
print(f"line2 = {line2!r}")
print(f"line3 = {line3!r}")
print(f"line4 = {line4!r}")

print()
print("Object types:")
print(f"type(line1) = {type(line1)}")
print(f"type(line4) = {type(line4)}")

print()
print("Selected names from dir(line1):")
for name in ["strip", "rstrip", "startswith", "split", "replace"]:
    print(f"{name:12} -> {name in dir(line1)}")

print()
print("Lines without trailing newline characters:")
print(line1.rstrip("\n"))
print(line2.rstrip("\n"))
print(line3.rstrip("\n"))
```

The representation marker `!r` is used in the formatted string so the newline character is visible as `\n`. Without that representation form, the newline would simply move the output to the next terminal line.

For many files, direct iteration over the file object is cleaner than repeatedly calling `readline()`.

```
path = "file_iteration_demo.txt"

with open(path, "w", encoding="utf-8") as file_obj:
    file_obj.write("Arthur: It is only a model.\n")
    file_obj.write("Guard: Shh!\n")
    file_obj.write("Narrator: The file object can be iterated line by line.\n")
```

```

with open(path, "r", encoding="utf-8") as file_obj:
    print("Iterating over the file object:")

    for line_number, line in enumerate(file_obj, start=1):
        clean_line = line.rstrip("\n")
        print(f"{line_number:02d}: {clean_line}")

    print(f"    type(line) = {type(line)}")

```

File iteration reads one line at a time instead of materializing the whole file as one string or one list. This makes it the normal pattern for processing large text files sequentially.

The file object acts as an iterable source of strings. Each loop step receives one `str` object representing one line.

Batch Line Loading with `.readlines()` and List Materialization Costs

The `.readlines()` method reads all remaining lines and returns them as a list of `str` objects. This is different from `read()`, which returns one large string.

```

path = "readlines_demo.txt"

with open(path, "w", encoding="utf-8") as file_obj:
    file_obj.write("Line A: To infinity, but not always 42.\n")
    file_obj.write("Line B: D'oh!\n")
    file_obj.write("Line C: Spam and eggs.\n")

with open(path, "r", encoding="utf-8") as file_obj:
    lines = file_obj.readlines()

print("Object returned by readlines():")
print(f"lines          = {lines}")
print(f"type(lines)    = {type(lines)}")
print(f"len(lines)     = {len(lines)}")

print()
print("Selected names from dir(lines):")
for name in ["append", "extend", "insert", "pop", "sort", "reverse"]:
    print(f"{name:10} -> {name in dir(lines)}")

print()
print("Inspecting each list element:")
for index, line in enumerate(lines):
    print(f"index {index}: {line!r}")
    print(f"    type(line) = {type(line)}")

print()
print("Clean printed version:")
for line in lines:
    print(line.rstrip("\n"))

```

The returned object is a list. Each element is a string. Because newline characters are part of the file content, each line string usually still contains its trailing `\n`. Methods such as `rstrip()` or `strip()` can remove that when needed.

The cost is materialization. `readlines()` builds a full list containing all lines. That means the program stores the complete file content and the list structure around it. This is useful when the program genuinely needs random access to the lines, repeated passes, indexing, slicing, or list operations.

For simple one-pass processing, direct file iteration is usually better:

```
path = "readlines_demo.txt"

print("One-pass processing without readlines():")

with open(path, "r", encoding="utf-8") as file_obj:
    for line_number, line in enumerate(file_obj, start=1):
        clean_line = line.strip()
        print(f"{line_number:02d} | {clean_line}")
```

The practical distinction is:

- `read()` returns one full `str`;
- `readline()` returns one `str` at a time;
- file iteration yields one `str` per loop step;
- `readlines()` returns one `list` containing many `str` objects.

The access method should match the size and purpose of the file. Small complete documents may be read with `read()`. Large sequential files should usually be processed by iteration. `readlines()` should be reserved for cases where a real list of all lines is useful.

9.4.2 Writing Textual Data

Writing textual data means sending Python `str` objects toward an external file resource. In text mode, Python does not store the `str` object itself. It encodes the string into bytes using the file's encoding, then sends those bytes through the file object to the operating system.

The visible operation is simple: call a method such as `write()` or `writelines()`. The architectural operation is layered: runtime text becomes encoded external data.

Writing Strings with `.write()`

The `.write()` method writes one string to a text file. It does not automatically add a newline. If a newline is needed, the string must contain `\n` explicitly.

```
path = "write_demo.txt"

line1 = "Line 1: It's just a flesh wound.\n"
line2 = "Line 2: The answer is 42.\n"

print("Objects before writing:")
print(f"path          = {path}")
print(f"type(path)    = {type(path)}")
print(f"type(line1)   = {type(line1)}")

print()
print("Selected names from dir(line1):")
for name in ["encode", "upper", "replace", "split", "strip", "endswith"]:
    print(f"{name:10} -> {name in dir(line1)}")

with open(path, "w", encoding="utf-8") as file_obj:
    print()
    print("File object inside the with block:")
```

```

print(f"file_obj      = {file_obj}")
print(f"type(file_obj) = {type(file_obj)}")

print()
print("Selected names from dir(file_obj):")
for name in ["write", "writelines", "flush", "close", "encoding", "mode"]:
    print(f"{name:12} -> {name in dir(file_obj)}")

count1 = file_obj.write(line1)
count2 = file_obj.write(line2)

print()
print("Return values from write():")
print(f"count1 = {count1}")
print(f"count2 = {count2}")
print("These counts are character counts in text mode.")

with open(path, "r", encoding="utf-8") as file_obj:
    content = file_obj.read()

print()
print("File content read back:")
print(content)

```

The method `write()` returns the number of characters written in text mode. This is not the same thing as the number of stored bytes in every case, because some characters may require more than one byte when encoded as UTF-8.

The file object receives only strings in text mode. Passing a number directly is an error. The number must first be converted to text.

```

path = "write_number_demo.txt"

answer = 42

with open(path, "w", encoding="utf-8") as file_obj:
    try:
        file_obj.write(answer)
    except TypeError as err:
        print("Direct integer writing failed.")
        print(f"type(err) = {type(err)}")
        print(f"err      = {err}")

    file_obj.write(f"The converted answer is {answer}.\n")

with open(path, "r", encoding="utf-8") as file_obj:
    content = file_obj.read()

print()
print("Correct file content:")
print(content)

```

The rule is strict: text files accept `str` objects. Formatting with an f-string, calling `str()`, or using `.format()` creates the string that can cross the text-output boundary.

Writing Multiple Lines with `.writelines()`

The `.writelines()` method writes several strings from an iterable object. The name can be misleading: it does not add newline characters. It writes exactly the strings it receives.

```
path = "writelines_demo.txt"

lines = [
    "First: D'oh!\n",
    "Second: Serenity now!\n",
    "Third: Nobody expects the Spanish Inquisition!\n",
]

print("The lines object:")
print(f"lines      = {lines}")
print(f"type(lines) = {type(lines)}")
print(f"len(lines)  = {len(lines)}")

print()
print("Selected names from dir(lines):")
for name in ["append", "extend", "insert", "pop", "sort", "reverse"]:
    print(f"{name:10} -> {name in dir(lines)}")

print()
print("Inspecting each element:")
for index, line in enumerate(lines):
    print(f"{index}: {line!r}")
    print(f"    type(line) = {type(line)}")

with open(path, "w", encoding="utf-8") as file_obj:
    result = file_obj.writelines(lines)

print()
print("Return value from writelines():")
print(f"result = {result}")

with open(path, "r", encoding="utf-8") as file_obj:
    content = file_obj.read()

print()
print("File content read back:")
print(content)
```

Unlike `write()`, `writelines()` returns `None`. Its purpose is not to report a character count. Its purpose is to consume an iterable of strings and send them to the file object.

The newline rule is visible if the strings do not contain `\n`.

```
path = "writelines_no_newlines_demo.txt"

parts = [
    "Spam",
    " eggs",
    " and",
    " spam",
    ".",
]
```

```

]

with open(path, "w", encoding="utf-8") as file_obj:
    file_obj.writelines(parts)

with open(path, "r", encoding="utf-8") as file_obj:
    content = file_obj.read()

print("Content written without explicit newline characters:")
print(repr(content))
print(content)

```

The output is one continuous text sequence because `writelines()` does not invent line separators. If the file must contain separate lines, the strings must already contain newline characters, or the program must add them before writing.

Newline Management: Explicit `\n`, Platform Differences, and Universal Newline Translation

A newline is not an invisible formatting wish. It is data. In Python source code, the escape sequence `\n` represents a line break inside a string.

```

path = "newline_demo.txt"

line1 = "George: It's not a lie if you believe it.\n"
line2 = "Elaine: Yada yada yada.\n"
line3 = "Kramer: Giddy up.\n"

with open(path, "w", encoding="utf-8") as file_obj:
    file_obj.write(line1)
    file_obj.write(line2)
    file_obj.write(line3)

with open(path, "r", encoding="utf-8") as file_obj:
    content = file_obj.read()

print("Normal printed content:")
print(content)

print("Representation with newline characters visible:")
print(repr(content))

print()
print("Split into logical lines:")
for line_number, line in enumerate(content.splitlines(), start=1):
    print(f"{line_number:02d}: {line}")

```

The printed version shows separate lines. The `repr()` version shows the actual `\n` markers inside the string representation. This is useful when debugging text files, because normal printing can hide where line breaks are stored.

Different operating systems historically used different newline byte sequences in text files. Unix-like systems use line feed. Windows text files commonly use carriage return plus line feed. Python text mode provides newline translation so code can usually work with `\n` as the ordinary in-memory newline representation.

The following program shows the usual text-mode behavior without relying on a specific platform.

```

path = "newline_translation_demo.txt"

lines = [
    "Line A: The runtime string uses \\n in source code.\\n",
    "Line B: Python text mode handles normal newline translation.\\n",
    "Line C: 42 remains portable enough.\\n",
]

with open(path, "w", encoding="utf-8") as file_obj:
    file_obj.writelines(lines)

with open(path, "r", encoding="utf-8") as file_obj:
    content = file_obj.read()

print("Text read back:")
print(content)

print("Representation read back:")
print(repr(content))

print()
print("Object type:")
print(f"type(content) = {type(content)}")

```

For ordinary text files, this is the desired behavior. The program writes and reads strings using `\n`. Python handles the platform-specific external representation in text mode.

When exact newline bytes matter, that is no longer ordinary text handling. The file should be treated more carefully, often in binary mode or with explicit newline settings. For normal text writing, the practical rule is simple: include `\n` where a new line is part of the intended file content, and remember that neither `write()` nor `writelines()` adds it automatically.

9.4.3 Encoding and Decoding Boundaries

A text file is not stored as Python `str` objects. A Python `str` object exists inside the runtime as Unicode text. A file stores bytes. Text file input/output therefore requires translation at the boundary.

When Python writes text, it encodes a `str` object into bytes. When Python reads text, it decodes stored bytes into a `str` object. Text mode performs this translation through the file object's encoding.

Text Encoding Revisited: Translating Between `str` Objects and Stored Byte Sequences

Encoding means converting text into bytes. Decoding means converting bytes back into text. The two operations are inverse only when the same compatible encoding rules are used.

```

text = "D'oh! The answer is 42. Știință."

print("Original runtime text:")
print(f"text          = {text}")
print(f"type(text) = {type(text)}")

print()
print("Selected names from dir(text):")
for name in ["encode", "upper", "replace", "split", "strip"]:
    print(f"{name:10} -> {name in dir(text)}")

data = text.encode("utf-8")

```



```

print()
print("Encoded byte sequence:")
print(f"data          = {data}")
print(f"type(data)    = {type(data)}")
print(f"len(text)     = {len(text)}")
print(f"len(data)      = {len(data)}")

print()
print("Selected names from dir(data):")
for name in ["decode", "hex", "split", "replace", "startswith"]:
    print(f"{name:10} -> {name in dir(data)}")

decoded_text = data.decode("utf-8")

print()
print("Decoded back into text:")
print(f"decoded_text      = {decoded_text}")
print(f"type(decoded_text) = {type(decoded_text)}")
print(f"text == decoded_text -> {text == decoded_text}")

```

The string and the byte sequence are different objects. The `str` object represents text. The `bytes` object represents byte values. Some visible characters require one byte in UTF-8, while others require several bytes. That is why `len(text)` and `len(data)` may differ.

The same boundary appears when writing and reading a text file.

```

path = "encoding_boundary_demo.txt"

text = "Nobody expects the Spanish Inquisition. Şi totuşi apare.\n"

with open(path, "w", encoding="utf-8") as file_obj:
    print("Text file object while writing:")
    print(f"type(file_obj) = {type(file_obj)}")
    print(f"encoding       = {file_obj.encoding}")

    file_obj.write(text)

with open(path, "r", encoding="utf-8") as file_obj:
    restored_text = file_obj.read()

print()
print("Text restored from file:")
print(restored_text)

print("Object inspection:")
print(f"type(restored_text) = {type(restored_text)}")
print(f"text == restored_text -> {text == restored_text}")

```

At Python level, the program writes and reads strings. At storage level, the file contains bytes. The text file object is the managed translation layer between those two forms.

Common Encoding Choices: UTF-8 as the Default Modern Interchange Encoding

UTF-8 is the normal modern choice for text interchange. It can represent ordinary ASCII text, Romanian characters, mathematical symbols, many writing systems, and emoji without switching encodings.

It is also compatible with ASCII for the first 128 character codes, which makes simple English text look byte-for-byte familiar.

For course code, the practical rule is simple: when opening a text file, specify `encoding="utf-8"` unless there is a concrete reason to use something else.

```
path = "utf8_demo.txt"

lines = [
    "English: The Ministry of Silly Walks.\n",
    "Romanian: Ș, ț, ă, î, â.\n",
    "Number: 42.\n",
]

with open(path, "w", encoding="utf-8") as file_obj:
    file_obj.writelines(lines)

with open(path, "r", encoding="utf-8") as file_obj:
    content = file_obj.read()

print("Content read with UTF-8:")
print(content)

print("Object inspection:")
print(f"type(content) = {type(content)}")
print(f"len(content) = {len(content)}")

print()
print("Visible lines:")
for line_number, line in enumerate(content.splitlines(), start=1):
    print(f"{line_number:02d}: {line}")
```

Different encodings store the same text using different byte rules. For plain ASCII characters, many encodings look identical. For non-ASCII characters, the difference becomes visible.

```
text = "Știință"

utf8_data = text.encode("utf-8")
utf16_data = text.encode("utf-16")

print("Same text, different byte encodings:")
print(f"text = {text}")
print(f"utf8_data = {utf8_data}")
print(f"utf16_data = {utf16_data}")

print()
print("Types:")
print(f"type(text) = {type(text)}")
print(f"type(utf8_data) = {type(utf8_data)}")
print(f"type(utf16_data) = {type(utf16_data)}")

print()
print("Lengths:")
print(f"len(text) = {len(text)}")
print(f"len(utf8_data) = {len(utf8_data)}")
print(f"len(utf16_data) = {len(utf16_data)}")
```

The text is logically the same, but the stored byte representation is different. Therefore, reading text correctly requires knowing the encoding used when the bytes were written.

Encoding Failure Modes: `UnicodeDecodeError`, `UnicodeEncodeError`, and Error Handling Strategies

Encoding and decoding can fail. A `UnicodeEncodeError` occurs when Python is asked to encode a character that cannot be represented in the chosen encoding. A `UnicodeDecodeError` occurs when Python is asked to decode bytes that do not form valid text under the chosen encoding.

```
text = "Știință and 42"

print("Trying to encode Romanian text as ASCII:")

try:
    data = text.encode("ascii")
except UnicodeEncodeError as err:
    print("Encoding failed.")
    print(f"type(err) = {type(err)}")
    print(f"err      = {err}")

    print()
    print("Selected names from dir(err):")
    for name in ["args", "encoding", "object", "start", "end", "reason"]:
        print(f"{name:10} -> {name in dir(err)}")
```

ASCII cannot represent characters such as Ș and ă. UTF-8 can.

```
text = "Știință and 42"

data = text.encode("utf-8")

print("Encoding with UTF-8 succeeds:")
print(f"data      = {data}")
print(f"type(data) = {type(data)}")

restored = data.decode("utf-8")

print()
print("Decoded text:")
print(f"restored      = {restored}")
print(f"type(restored) = {type(restored)}")
```

Decoding can fail when byte data is interpreted with the wrong encoding.

```
data = "Știință".encode("utf-8")

print("Valid UTF-8 bytes:")
print(data)

print()
print("Trying to decode those bytes as ASCII:")

try:
    text = data.decode("ascii")
```

```
except UnicodeDecodeError as err:
    print("Decoding failed.")
    print(f"type(err) = {type(err)}")
    print(f"err      = {err}")

    print()
    print("Selected names from dir(err):")
    for name in ["args", "encoding", "object", "start", "end", "reason"]:
        print(f"{name:10} -> {name in dir(err)}")
```

Python also allows explicit error-handling strategies. These should be used deliberately, not as a way to hide misunderstanding about the file's real encoding.

```
text = "Știință and 42"

print("Encoding with error handling:")

ascii_replace = text.encode("ascii", errors="replace")
ascii_ignore = text.encode("ascii", errors="ignore")
ascii_xml = text.encode("ascii", errors="xmlcharrefreplace")

print(f"replace      = {ascii_replace}")
print(f"ignore      = {ascii_ignore}")
print(f"xmlcharrefreplace = {ascii_xml}")

print()
print("Decoding invalid bytes with error handling:")

bad_data = b"Valid ASCII first. \xff\xfe Broken bytes later."

try:
    normal_text = bad_data.decode("utf-8")
except UnicodeDecodeError as err:
    print("Normal UTF-8 decoding failed.")
    print(f"err = {err}")

replace_text = bad_data.decode("utf-8", errors="replace")
ignore_text = bad_data.decode("utf-8", errors="ignore")

print()
print(f"replace_text = {replace_text}")
print(f"ignore_text  = {ignore_text}")
```

The common error strategies are:

- "strict": fail on invalid data; this is the default;
- "replace": replace invalid characters with a marker;
- "ignore": drop invalid data;
- "xmlcharrefreplace": when encoding, replace unsupported characters with XML-style character references.

The safest default for course programs is still explicit UTF-8 with strict error behavior:

```

path = "safe_utf8_text.txt"

text = "UTF-8 keeps the text boundary explicit. Și asta contează.\n"

with open(path, "w", encoding="utf-8") as file_obj:
    file_obj.write(text)

with open(path, "r", encoding="utf-8") as file_obj:
    content = file_obj.read()

print(content)
print(f"type(content) = {type(content)}")

```

The practical rule is this: `str` is runtime text, `bytes` is byte data, encoding moves from `str` to `bytes`, and decoding moves from `bytes` to `str`. Text files sit exactly on that boundary.

9.5 Binary File Reading and Writing

Binary files are handled as byte streams, not as decoded text. When a file is opened in binary mode, Python does not translate stored bytes into `str` objects and does not apply text newline handling. The program receives and writes byte-oriented objects such as `bytes` and `bytearray`.

This section explains how binary mode exposes the raw storage boundary more directly than text mode. It introduces immutable byte sequences, mutable byte buffers, chunk-based processing for large files, and byte-position navigation with `seek()` and `tell()`.

The central rule is simple: use text mode when the data is meant to be decoded as text; use binary mode when the exact byte values must be preserved and controlled directly.

9.5.1 Byte-Oriented Data Streams

Text files pass through an encoding and decoding boundary. Binary files do not. In binary mode, Python treats the file as a stream of bytes. The program does not ask Python to interpret those bytes as characters, lines, words, or encoded text.

This is the correct model for images, compressed archives, executable files, audio data, network packets, and any format where each byte value matters directly. A binary file is not “text with strange characters”. It is byte data.

Binary Mode as Raw Byte Transfer Without Text Decoding

Binary mode is selected by adding `b` to the mode string. The most common binary modes are `"rb"` for reading bytes and `"wb"` for writing bytes.

In binary mode, `write()` expects a bytes-like object, not a `str`. Likewise, `read()` returns `bytes`, not `str`.

```

path = "binary_mode_demo.bin"

data = b"Binary mode does not decode text. 42.\n"

print("Byte data before writing:")
print(f"data          = {data}")
print(f"type(data)    = {type(data)}")
print(f"len(data)     = {len(data)}")

print()
print("Selected names from dir(data):")

```

```

for name in ["decode", "hex", "split", "replace", "startswith", "count"]:
    print(f"{name:12} -> {name in dir(data)}")

with open(path, "wb") as file_obj:
    print()
    print("Binary file object while writing:")
    print(f"file_obj      = {file_obj}")
    print(f"type(file_obj) = {type(file_obj)}")
    print(f"file_obj.mode  = {file_obj.mode}")

    print()
    print("Selected names from dir(file_obj):")
    for name in ["write", "read", "close", "flush", "seek", "tell", "mode"]:
        print(f"{name:10} -> {name in dir(file_obj)}")

    written_count = file_obj.write(data)

print()
print(f"write() returned: {written_count}")
print("In binary mode, this is the number of bytes written.")

with open(path, "rb") as file_obj:
    restored_data = file_obj.read()

print()
print("Byte data read back:")
print(f"restored_data      = {restored_data}")
print(f"type(restored_data) = {type(restored_data)}")
print(f"data == restored_data -> {data == restored_data}")

```

The object written to the file is a `bytes` object. The object read back from the file is also a `bytes` object. Python does not decode the file content into text because the file was opened in binary mode.

Passing a string directly to a binary file is an error.

```

path = "binary_type_error_demo.bin"

text = "D'oh! This is a str, not bytes."

with open(path, "wb") as file_obj:
    try:
        file_obj.write(text)
    except TypeError as err:
        print("Writing str to a binary file failed.")
        print(f"type(err) = {type(err)}")
        print(f"err      = {err}")

    fixed_data = text.encode("utf-8")
    file_obj.write(fixed_data)

print()
print("Corrected by explicit encoding:")
print(f"fixed_data      = {fixed_data}")
print(f"type(fixed_data) = {type(fixed_data)}")

```

The failure is useful. It prevents the program from silently choosing an encoding. If the programmer wants text to become bytes, the conversion should be explicit.

The bytes Object: Immutable Byte Sequences Distinct from str

A `bytes` object is an immutable sequence of byte values. Each element is an integer from 0 to 255. This is different from a `str`, whose elements are Unicode characters.

```
text = "Spam and eggs."
data = b"Spam and eggs."

print("Text object:")
print(f"text          = {text}")
print(f"type(text)    = {type(text)}")
print(f"len(text)     = {len(text)}")

print()
print("Bytes object:")
print(f"data          = {data}")
print(f"type(data)    = {type(data)}")
print(f"len(data)     = {len(data)}")

print()
print("Indexing:")
first_char = text[0]
first_byte = data[0]

print(f"text[0]          = {first_char!r}")
print(f"type(text[0])    = {type(first_char)}")

print(f"data[0]          = {first_byte}")
print(f"type(data[0])    = {type(first_byte)}")

print()
print("Slicing:")
print(f"text[0:4]         = {text[0:4]!r}")
print(f"type(text[0:4])   = {type(text[0:4])}")

print(f"data[0:4]        = {data[0:4]}")
print(f"type(data[0:4])  = {type(data[0:4])}")
```

The difference is precise. Indexing a string returns a one-character `str`. Indexing a `bytes` object returns an `int`. Slicing a `bytes` object returns another `bytes` object.

A `bytes` object can be inspected and transformed without becoming text.

```
data = b"Nobody expects the Spanish Inquisition!"

print("Original bytes:")
print(data)

print()
print("Selected bytes methods:")
for name in ["hex", "decode", "replace", "split", "startswith", "endswith"]:
    print(f"{name:12} -> {name in dir(data)}")
```

```

print()
print("Byte-level operations:")
print(f"data.startswith(b'Nobody') -> {data.startswith(b'Nobody')}")
print(f"data.endswith(b'!') -> {data.endswith(b'!')}")
print(f"data.count(b'e') -> {data.count(b'e')}")
print(f"data.replace(b'Nobody', b'Everybody') -> {data.replace(b'Nobody', b'Everybody')}")

print()
hex_text = data.hex()
print("Hexadecimal view:")
print(f"hex_text = {hex_text}")
print(f"type(hex_text) = {type(hex_text)}")

```

The `hex()` method returns a textual hexadecimal representation of the byte sequence. This is useful for inspection because arbitrary bytes are not always printable as meaningful text.

The `decode()` method crosses from bytes back to text, but only if the bytes represent valid text under the chosen encoding.

```

data = b"Serenity now!"

text = data.decode("utf-8")

print("Decoded bytes:")
print(f"text = {text}")
print(f"type(text) = {type(text)}")

```

The practical rule is: `bytes` is not a rough synonym for `str`. It is a separate object type for byte sequences. Convert explicitly when crossing between text and byte data.

The bytearray Object: Mutable Byte Buffers for Incremental Modification

A `bytes` object is immutable. Once created, its byte values cannot be changed in place. A `bytearray` is the mutable counterpart. It stores byte values like `bytes`, but allows modification.

```

data = b"Chuck Norris fact: 41."

print("Immutable bytes object:")
print(f"data = {data}")
print(f"type(data) = {type(data)}")

print()
try:
    data[-2] = 50
except TypeError as err:
    print("Changing bytes in place failed.")
    print(f"type(err) = {type(err)}")
    print(f"err = {err}")

```

To modify byte data incrementally, create a `bytearray`.

```

buf = bytearray(b"Chuck Norris fact: 41.")

print("Mutable bytearray object:")
print(f"buf = {buf}")
print(f"type(buf) = {type(buf)}")

```



```

print(f"len(buf)  = {len(buf)}")

print()
print("Selected names from dir(buf):")
for name in ["append", "extend", "insert", "pop", "replace", "decode", "hex"]:
    print(f"{name:10} -> {name in dir(buf)}")

print()
print("Before modification:")
print(buf)

buf[-2] = ord("2")

print("After modification:")
print(buf)

text = buf.decode("utf-8")

print()
print("Decoded modified buffer:")
print(f"text          = {text}")
print(f"type(text) = {type(text)}")

```

The expression `ord("2")` returns the integer code value for the character "2". A `bytearray` position stores an integer byte value, not a one-character string.

A `bytearray` can also grow.

```

buf = bytearray()

print("Empty bytearray:")
print(f"buf          = {buf}")
print(f"type(buf) = {type(buf)}")

buf.extend(b"D'oh")
buf.append(33)
buf.extend(b" 42")

print()
print("After incremental changes:")
print(f"buf = {buf}")

print()
print("Element inspection:")
for index, value in enumerate(buf):
    print(f"index {index:02d} -> value {value:3d}")

print()
print("As decoded text:")
print(buf.decode("utf-8"))

```

The call `append(33)` adds one byte. The value 33 is the byte value for the exclamation mark in ASCII-compatible encodings. The call `extend(...)` adds several bytes.

For file output, both `bytes` and `bytearray` can be written in binary mode.

```

path = "bytearray_write_demo.bin"

```

```

buf = bytearray(b"The answer was ")
buf.extend(b"41")
buf[-1] = ord("2")
buf.extend(b".\n")

with open(path, "wb") as file_obj:
    file_obj.write(buf)

with open(path, "rb") as file_obj:
    restored_data = file_obj.read()

print("Restored binary data:")
print(f"restored_data      = {restored_data}")
print(f"type(restored_data) = {type(restored_data)}")

print()
print("Decoded for display:")
print(restored_data.decode("utf-8"))

```

The `bytearray` is useful when byte data must be built or modified step by step. The `bytes` object is useful when the byte sequence should be fixed and safe from in-place mutation.

The practical distinction is direct:

- `str`: immutable runtime text;
- `bytes`: immutable byte sequence;
- `bytearray`: mutable byte buffer.

Binary file mode works with byte-oriented objects. It does not perform text decoding or encoding unless the programmer explicitly asks for it.

9.5.2 Binary Access Patterns

Binary access usually cares less about lines and more about byte positions, byte counts, and raw transfer. The program may read a fixed number of bytes, write prepared byte buffers, or move to a specific byte offset inside the file.

This is different from text access. Text mode thinks in decoded characters and newline translation. Binary mode thinks in byte sequences.

Reading Fixed-Size Chunks for Large Files

The `read()` method accepts an optional size argument. In binary mode, that size means number of bytes. This allows a program to process a file gradually instead of loading the whole file into memory.

```

path = "chunk_read_demo.bin"

data = (
    b"Chunk 1: D'oh!\n"
    b"Chunk 2: Serenity now!\n"
    b"Chunk 3: The answer is 42.\n"
    b"Chunk 4: Nobody expects the Spanish Inquisition!\n"
)

with open(path, "wb") as file_obj:

```

```

    file_obj.write(data)

chunk_size = 12

with open(path, "rb") as file_obj:
    print("Binary file object:")
    print(f"file_obj      = {file_obj}")
    print(f"type(file_obj) = {type(file_obj)}")

    print()
    print("Selected names from dir(file_obj):")
    for name in ["read", "write", "seek", "tell", "close", "mode"]:
        print(f"{name:10} -> {name in dir(file_obj)}")

    print()
    print("Reading fixed-size chunks:")

    chunk_number = 1

    while True:
        chunk = file_obj.read(chunk_size)

        if chunk == b"":
            break

        print(f"chunk {chunk_number:02d}: {chunk!r}")
        print(f"    type(chunk) = {type(chunk)}")
        print(f"    len(chunk)  = {len(chunk)}")

        chunk_number += 1

```

The loop ends when `read(chunk_size)` returns the empty byte string `b""`. That is the binary-mode end-of-file marker for this pattern.

The important memory rule is that only one chunk is held at a time by the loop variable `chunk`. This is the normal pattern for large binary files, because the program does not need to materialize the whole file as one large `bytes` object.

Writing Byte Buffers to External Storage

Binary writing sends byte-oriented objects to a file opened in binary write mode. The most common objects are `bytes` and `bytearray`.

```

path = "byte_buffer_write_demo.bin"

header = b"HEADER:"
body = bytearray(b" fact number 41")
body[-1] = ord("2")
ending = b"\n"

print("Objects before writing:")
print(f"header      = {header}")
print(f"type(header) = {type(header)}")

print(f"body        = {body}")
print(f"type(body)  = {type(body)}")

```

```

print(f"ending          = {ending}")
print(f"type(ending) = {type(ending)}")

print()
print("Selected names from dir(body):")
for name in ["append", "extend", "insert", "pop", "decode", "hex"]:
    print(f"{name:10} -> {name in dir(body)}")

with open(path, "wb") as file_obj:
    count1 = file_obj.write(header)
    count2 = file_obj.write(body)
    count3 = file_obj.write(ending)

print()
print("Byte counts returned by write():")
print(f"count1 = {count1}")
print(f"count2 = {count2}")
print(f"count3 = {count3}")

with open(path, "rb") as file_obj:
    restored_data = file_obj.read()

print()
print("Restored byte data:")
print(f"restored_data          = {restored_data}")
print(f"type(restored_data) = {type(restored_data)}")

print()
print("Decoded only for display:")
print(restored_data.decode("utf-8"))

```

The file receives bytes. It does not receive Python strings, and it does not perform text encoding by itself. The `bytearray` object is accepted because it is a bytes-like object.

A list of byte chunks can also be written one by one.

```

path = "many_buffers_demo.bin"

chunks = [
    b"First block: Spam.\n",
    b"Second block: Eggs.\n",
    b"Third block: 42.\n",
]

print("Chunk container:")
print(f"chunks          = {chunks}")
print(f"type(chunks) = {type(chunks)}")
print(f"len(chunks) = {len(chunks)}")

print()
print("Selected names from dir(chunks):")
for name in ["append", "extend", "insert", "pop", "sort", "reverse"]:
    print(f"{name:10} -> {name in dir(chunks)}")

```

```

with open(path, "wb") as file_obj:
    total_count = 0

    for chunk in chunks:
        written_count = file_obj.write(chunk)
        total_count += written_count

        print(f"wrote {written_count:02d} bytes from {chunk!r}")

print()
print(f"total_count = {total_count}")

with open(path, "rb") as file_obj:
    data = file_obj.read()

print()
print(data)

```

This pattern is useful when a binary output is assembled from several pieces: a header, a payload, metadata, padding bytes, or generated byte buffers.

Random Access with `.seek()` and `.tell()`

Sequential reading consumes bytes in order. Random access means moving to a specific byte position and reading or writing from there.

The `tell()` method reports the current byte position. The `seek()` method moves the current position.

```

path = "seek_tell_demo.bin"

data = b"0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZ"

with open(path, "wb") as file_obj:
    file_obj.write(data)

with open(path, "rb") as file_obj:
    print("Initial position:")
    print(f"file_obj.tell() = {file_obj.tell()}")

    part1 = file_obj.read(5)

    print()
    print("After reading 5 bytes:")
    print(f"part1          = {part1}")
    print(f"type(part1)       = {type(part1)}")
    print(f"file_obj.tell() = {file_obj.tell()}")

    file_obj.seek(10)

    print()
    print("After seek(10):")
    print(f"file_obj.tell() = {file_obj.tell()}")

    part2 = file_obj.read(6)

```

```

print()
print("Read 6 bytes from position 10:")
print(f"part2          = {part2}")
print(f"file_obj.tell() = {file_obj.tell()}")

file_obj.seek(0)

print()
print("After seek(0), back to start:")
print(f"file_obj.tell() = {file_obj.tell()}")

part3 = file_obj.read(3)

print()
print("Read first 3 bytes again:")
print(f"part3          = {part3}")
print(f"file_obj.tell() = {file_obj.tell()}")

```

The byte positions are zero-based. Position 0 is the first byte. After reading five bytes, the current position is 5, meaning the next read begins at byte index 5.

Random access also works for overwriting bytes in an existing binary file.

```

path = "binary_overwrite_demo.bin"

with open(path, "wb") as file_obj:
    file_obj.write(b"Answer: 41\n")

with open(path, "r+b") as file_obj:
    print("Before overwrite:")
    file_obj.seek(0)
    before_data = file_obj.read()
    print(before_data)

    file_obj.seek(9)
    file_obj.write(b"2")

    print()
    print("After overwrite:")
    file_obj.seek(0)
    after_data = file_obj.read()
    print(after_data)

print()
print("Decoded for display:")
print(after_data.decode("utf-8"))

```

The mode "r+b" opens the file for both reading and writing in binary mode without erasing the existing file. The call `seek(9)` moves to the byte position containing `b"1"`. The write then replaces that one byte with `b"2"`.

This is not line-based editing. It is byte-position editing. If the replacement has a different length, the surrounding bytes are not automatically shifted like elements in a Python list. Binary random access should therefore be used with formats where byte positions and byte sizes are understood.

The practical binary-access rules are:

- use `read(size)` to process large files in bounded chunks;

- use `write()` with `bytes` or `bytearray` buffers;
- use `tell()` to inspect the current byte position;
- use `seek()` to move to a specific byte position.

9.6 Filesystem Path Architecture

A file operation always begins with a reference to a location. That reference is the path. It may look like a simple string, but it encodes filesystem structure: directory names, final file names, suffixes, roots, separators, and the difference between relative and absolute addressing.

This section explains how Python represents and manipulates paths before any actual file content is read or written. It contrasts plain string paths with `pathlib.Path` objects, shows how the current working directory affects relative paths, and introduces portable path construction across Windows and POSIX-style systems.

The central rule is simple: a path is not the file itself. It is a structured reference that Python passes to the operating system when the program wants to inspect, create, open, move, or delete filesystem entries.

9.6.1 Paths as Structured Filesystem References

A path is not the file itself. A path is a reference that tells the operating system where a filesystem object should be found or created. That object may be a file, a directory, a symbolic link, or some other platform-specific filesystem entry.

Earlier file examples used simple string paths such as `"demo.txt"`. That works, but a path has internal structure: parent directories, final names, suffixes, roots, drives, separators, and resolution rules. Python can represent paths as ordinary strings, but it also provides `pathlib.Path`, which exposes that structure through object methods and attributes.

String Paths vs. `pathlib.Path` Objects

A string path is just a `str` object whose content happens to describe a filesystem location. Python does not automatically understand its pieces as path components. It can still be passed to `open()`, but string methods such as `split()` and `replace()` are general text methods, not filesystem-aware path operations.

```
from pathlib import Path

str_path = "data/chuck_facts.txt"
path_obj = Path("data") / "chuck_facts.txt"

print("Two path representations:")
print(f"str_path = {str_path}")
print(f"path_obj = {path_obj}")

print()
print("Object types:")
print(f"type(str_path) = {type(str_path)}")
print(f"type(path_obj) = {type(path_obj)}")

print()
print("Selected names from dir(str_path):")
for name in ["split", "replace", "startswith", "endswith", "upper"]:
    print(f"{name:12} -> {name in dir(str_path)}")
```

```

print()
print("Selected names from dir(path_obj):")
for name in ["name", "parent", "suffix", "stem", "parts", "exists", "is_file"]:
    print(f"{name:12} -> {name in dir(path_obj)}")

print()
print("Path object components:")
print(f"path_obj.name    = {path_obj.name}")
print(f"path_obj.parent = {path_obj.parent}")
print(f"path_obj.stem   = {path_obj.stem}")
print(f"path_obj.suffix = {path_obj.suffix}")
print(f"path_obj.parts  = {path_obj.parts}")

```

The string path and the Path object may print similarly, but they are not the same kind of object. The string exposes text behavior. The Path object exposes filesystem-reference behavior.

The operator / is overloaded for Path objects. It does not mean division here. It means path composition. The expression `Path("data") / "chuck_facts.txt"` creates a new path by joining a directory-like component with a final filename component.

A Path object can still be converted to a string when necessary.

```

from pathlib import Path

path_obj = Path("quotes") / "monty_python.txt"
path_text = str(path_obj)

print("Path object:")
print(f"path_obj      = {path_obj}")
print(f"type(path_obj) = {type(path_obj)}")

print()
print("Converted string:")
print(f"path_text      = {path_text}")
print(f"type(path_text) = {type(path_text)}")

```

For modern Python code, Path objects are usually clearer than manually assembled string paths. They keep path construction separate from ordinary text manipulation.

Absolute Paths, Relative Paths, and Current Working Directory Resolution

A relative path is interpreted from some starting directory. For ordinary script execution, that starting point is usually the current working directory. An absolute path already contains enough information to identify a location from the filesystem root or drive root.

This distinction matters because the same relative path can point to different files when the program is started from different directories.

```

from pathlib import Path

cwd = Path.cwd()

rel_path = Path("data") / "answer.txt"
abs_path = rel_path.resolve()

print("Current working directory:")
print(f"cwd      = {cwd}")
print(f"type(cwd) = {type(cwd)}")

```



```

print()
print("Relative path:")
print(f"rel_path      = {rel_path}")
print(f"type(rel_path) = {type(rel_path)}")
print(f"rel_path.is_absolute() = {rel_path.is_absolute()}")

print()
print("Resolved absolute path:")
print(f"abs_path      = {abs_path}")
print(f"type(abs_path) = {type(abs_path)}")
print(f"abs_path.is_absolute() = {abs_path.is_absolute()}")

print()
print("Selected names from dir(rel_path):")
for name in ["resolve", "absolute", "is_absolute", "parent", "name", "parts"]:
    print(f"{name:14} -> {name in dir(rel_path)}")

```

The method `Path.cwd()` returns the current working directory as a `Path` object. The method `resolve()` returns an absolute version of a path. In normal use, this means the relative path is interpreted against the current working directory.

A relative path is not incomplete text. It is a valid reference whose meaning depends on where the process is currently positioned in the filesystem.

```

from pathlib import Path

rel_path = Path("logs") / "seinfeld.txt"

print("Relative path parts:")
print(f"rel_path.parts = {rel_path.parts}")

print()
print("If the current working directory changes,")
print("the same relative path may refer to another external location.")

print()
print(f"current directory = {Path.cwd()}")
print(f"relative path     = {rel_path}")
print(f"resolved path      = {rel_path.resolve()}")

```

The practical rule is simple: relative paths are convenient inside a known project directory; absolute paths are clearer when the program must identify one specific location regardless of where it was launched.

Platform Separators: Windows Backslashes, POSIX Slashes, and Portable Path Construction

Different operating systems display paths differently. POSIX-style systems, such as Linux and macOS, normally use forward slashes. Windows paths commonly use backslashes and may include drive names.

Hard-coding separators manually is fragile. Python can build portable paths with `pathlib.Path` instead.

```

from pathlib import Path, PureWindowsPath, PurePosixPath

portable_path = Path("course_data") / "quotes" / "simpsons.txt"

```

```

print("Portable path built with pathlib:")
print(f"portable_path      = {portable_path}")
print(f"type(portable_path) = {type(portable_path)}")
print(f"portable_path.parts = {portable_path.parts}")

print()
print("Pure path examples, without touching the real filesystem:")

win_path = PureWindowsPath("course_data") / "quotes" / "simpsons.txt"
posix_path = PurePosixPath("course_data") / "quotes" / "simpsons.txt"

print(f"win_path      = {win_path}")
print(f"type(win_path) = {type(win_path)}")
print(f"win_path.parts = {win_path.parts}")

print()
print(f"posix_path      = {posix_path}")
print(f"type(posix_path) = {type(posix_path)}")
print(f"posix_path.parts = {posix_path.parts}")

```

`PureWindowsPath` and `PurePosixPath` are useful for demonstrating path syntax rules without actually requiring the program to run on that operating system. They model path structure, but they do not access the filesystem.

Backslashes inside ordinary Python string literals also have a second problem: backslash begins escape sequences such as `\n` and `\t`. This can accidentally corrupt a Windows-style path if it is written carelessly.

```

bad_path = "C:\new\text.txt"
raw_path = r"C:\new\text.txt"

print("Careless Windows-style string:")
print(bad_path)
print(repr(bad_path))

print()
print("Raw string version:")
print(raw_path)
print(repr(raw_path))

```

In the first string, `\n` becomes a newline and `\t` becomes a tab. The string no longer contains the visible path the programmer probably intended. A raw string avoids that escape processing, but `pathlib` is usually cleaner because it reduces manual separator work.

```

from pathlib import Path

path_obj = Path("C:/new") / "text.txt"

print("Path constructed without manual backslash escaping:")
print(path_obj)
print(f"type(path_obj) = {type(path_obj)}")

```

The practical rule is direct: use `Path(...)` and the `/` composition operator for path construction. Let Python represent the path according to the current platform, and avoid treating filesystem references as ordinary string concatenation problems.

9.6.2 Filesystem Inspection and Manipulation

A path object does not only describe a possible location. It can also be used to ask the filesystem questions and perform filesystem operations. This moves the program beyond reading and writing file contents: the program can now inspect whether entries exist, create directories, list directory contents, rename entries, move entries, and delete entries.

These operations still cross the operating-system boundary. Python does not own the filesystem. It asks the operating system to inspect or change filesystem state.

Checking Existence, File Type, and Directory Type

The methods `exists()`, `is_file()`, and `is_dir()` are basic filesystem inspection methods. They do not inspect a Python variable. They ask whether the external path currently refers to something in the filesystem, and what kind of thing it appears to be.

```
from pathlib import Path

demo_dir = Path("fs_demo")
demo_file = demo_dir / "quote.txt"

demo_dir.mkdir(exist_ok=True)

with open(demo_file, "w", encoding="utf-8") as file_obj:
    file_obj.write("Filesystem fact: D'oh is persistent if written to disk.\n")

print("Path objects:")
print(f"demo_dir = {demo_dir}")
print(f"demo_file = {demo_file}")

print()
print("Object types:")
print(f"type(demo_dir) = {type(demo_dir)}")
print(f"type(demo_file) = {type(demo_file)}")

print()
print("Selected names from dir(demo_file):")
for name in ["exists", "is_file", "is_dir", "name", "parent", "suffix"]:
    print(f"{name:10} -> {name in dir(demo_file)}")

print()
print("Filesystem checks:")
print(f"demo_dir.exists() = {demo_dir.exists()}")
print(f"demo_dir.is_dir() = {demo_dir.is_dir()}")
print(f"demo_dir.is_file() = {demo_dir.is_file()}")

print()
print(f"demo_file.exists() = {demo_file.exists()}")
print(f"demo_file.is_dir() = {demo_file.is_dir()}")
print(f"demo_file.is_file() = {demo_file.is_file()}")

missing_path = demo_dir / "missing.txt"

print()
print("Missing path checks:")
print(f"missing_path.exists() = {missing_path.exists()}")
```

```

print(f"missing_path.is_file() = {missing_path.is_file()}")
print(f"missing_path.is_dir()  = {missing_path.is_dir()}")

exists_result = demo_file.exists()

print()
print("The result of exists() is a bool object:")
print(f"exists_result      = {exists_result}")
print(f"type(exists_result) = {type(exists_result)}")

```

The result of these checks is a Boolean value. The path object remains only a reference. The real question is answered by the filesystem at the moment the method is called.

This timing matters. A path may exist now and disappear later. Another process may create, delete, rename, or lock files between two operations. These checks are useful, but they are not permanent guarantees.

Creating Directories and Parent Directory Chains

The `mkdir()` method creates a directory. If the directory already exists, the default behavior is to raise an error. This is useful when accidental reuse of an existing directory would be suspicious.

```

from pathlib import Path

demo_root = Path("mkdir_demo")
simple_dir = demo_root / "simple"

demo_root.mkdir(exist_ok=True)

print("Before creating simple_dir:")
print(f"simple_dir.exists() = {simple_dir.exists()}")

simple_dir.mkdir()

print()
print("After creating simple_dir:")
print(f"simple_dir.exists() = {simple_dir.exists()}")
print(f"simple_dir.is_dir() = {simple_dir.is_dir()}")

print()
print("Trying to create the same directory again:")

try:
    simple_dir.mkdir()
except FileExistsError as err:
    print("Directory creation failed because it already exists.")
    print(f"type(err) = {type(err)}")
    print(f"err      = {err}")

```

When a chain of parent directories may not already exist, `parents=True` tells Python to create the missing parent directories too. The option `exist_ok=True` tells Python not to fail if the final directory already exists.

```

from pathlib import Path

nested_dir = Path("mkdir_demo") / "level1" / "level2" / "level3"

```

```

print("Nested directory path:")
print(f"nested_dir      = {nested_dir}")
print(f"type(nested_dir) = {type(nested_dir)}")
print(f"nested_dir.parts = {nested_dir.parts}")

print()
print("Before mkdir():")
print(f"nested_dir.exists() = {nested_dir.exists()}")

nested_dir.mkdir(parents=True, exist_ok=True)

print()
print("After mkdir(parents=True, exist_ok=True):")
print(f"nested_dir.exists() = {nested_dir.exists()}")
print(f"nested_dir.is_dir() = {nested_dir.is_dir()}")

print()
print("Selected names from dir(nested_dir):")
for name in ["mkdir", "exists", "is_dir", "parent", "parents", "parts"]:
    print(f"{name:10} -> {name in dir(nested_dir)}")

print()
print("Parent chain:")
for parent in nested_dir.parents:
    print(parent)

```

The method `mkdir()` changes filesystem state. It is not like creating a list or dictionary inside Python memory. The created directory remains after the Python process ends unless later removed.

The practical rule is to use `parents=True` when the whole directory chain should be created, and `exist_ok=True` when repeated execution of the same script should not fail merely because the directory is already present.

Listing Directory Contents and Iterating over Filesystem Entries

The method `iterdir()` returns an iterator over the entries directly inside a directory. Each yielded entry is a `Path` object.

```

from pathlib import Path

demo_dir = Path("listing_demo")
demo_dir.mkdir(exist_ok=True)

file1 = demo_dir / "alpha.txt"
file2 = demo_dir / "beta.txt"
file3 = demo_dir / "answer_42.txt"
subdir = demo_dir / "nested"

with open(file1, "w", encoding="utf-8") as file_obj:
    file_obj.write("Alpha: Nobody expects the Spanish Inquisition.\n")

with open(file2, "w", encoding="utf-8") as file_obj:
    file_obj.write("Beta: Serenity now.\n")

with open(file3, "w", encoding="utf-8") as file_obj:

```

```

file_obj.write("Answer: 42.\n")

subdir.mkdir(exist_ok=True)

entries = list(demo_dir.iterdir())

print("Directory entries as a list:")
print(f"entries      = {entries}")
print(f"type(entries) = {type(entries)}")
print(f"len(entries)   = {len(entries)}")

print()
print("Selected names from dir(entries):")
for name in ["append", "extend", "insert", "pop", "sort", "reverse"]:
    print(f"{name:10} -> {name in dir(entries)}")

print()
print("Inspecting entries:")
for entry in entries:
    print(f"entry = {entry}")
    print(f"    type(entry) = {type(entry)}")
    print(f"    name        = {entry.name}")
    print(f"    is_file     = {entry.is_file()}")
    print(f"    is_dir      = {entry.is_dir()}")

```

The call `list(demo_dir.iterdir())` materializes all immediate directory entries into a list. For a small directory this is convenient. For a very large directory, direct iteration may be preferable.

```

from pathlib import Path

demo_dir = Path("listing_demo")

print("Direct iteration over directory entries:")

for entry in demo_dir.iterdir():
    kind = "directory" if entry.is_dir() else "file"
    print(f"{entry.name:15} -> {kind}")

```

For filtered listing, `glob()` can match path patterns.

```

from pathlib import Path

demo_dir = Path("listing_demo")

txt_files = list(demo_dir.glob("*.txt"))

print("Only .txt files:")
for path in txt_files:
    print(f"{path.name:15} size unknown here, type = {type(path)}")

print()
print("txt_files object:")
print(f"type(txt_files) = {type(txt_files)}")
print(f"len(txt_files)   = {len(txt_files)}")

```

The pattern `*.txt` means entries whose names end with `.txt`. This is filesystem-oriented matching, not ordinary string splitting. The returned values are still `Path` objects.

Renaming, Moving, and Deleting Files Safely

Filesystem manipulation must be treated carefully because it changes external state. A rename, move, or delete operation affects files and directories outside Python memory. These changes are not undone automatically when the script ends.

The method `rename()` changes the path of a filesystem entry. In the same directory, this is a rename. Across directories, it can act as a move when the operating system permits it.

```
from pathlib import Path

demo_dir = Path("rename_demo")
demo_dir.mkdir(exist_ok=True)

old_path = demo_dir / "old_name.txt"
new_path = demo_dir / "new_name.txt"

with open(old_path, "w", encoding="utf-8") as file_obj:
    file_obj.write("Old name: These pretzels are making me thirsty.\n")

print("Before rename:")
print(f"old_path.exists() = {old_path.exists()}")
print(f"new_path.exists() = {new_path.exists()}")

old_path.rename(new_path)

print()
print("After rename:")
print(f"old_path.exists() = {old_path.exists()}")
print(f"new_path.exists() = {new_path.exists()}")

with open(new_path, "r", encoding="utf-8") as file_obj:
    content = file_obj.read()

print()
print("Content after rename:")
print(content)
```

A safer overwrite-aware pattern is to check whether the destination already exists before renaming. This does not remove all race conditions, but it prevents simple accidental replacement in ordinary scripts.

```
from pathlib import Path

demo_dir = Path("safe_rename_demo")
demo_dir.mkdir(exist_ok=True)

src_path = demo_dir / "source.txt"
dst_path = demo_dir / "destination.txt"

with open(src_path, "w", encoding="utf-8") as file_obj:
    file_obj.write("Source: It's only a model.\n")
```

```
with open(dst_path, "w", encoding="utf-8") as file_obj:
    file_obj.write("Destination already exists.\n")
```

```
print("Before safe rename attempt:")
print(f"src_path.exists() = {src_path.exists()}")
print(f"dst_path.exists() = {dst_path.exists()}")
```

```
if dst_path.exists():
    print("Rename skipped: destination already exists.")
else:
    src_path.rename(dst_path)
    print("Rename completed.")
```

Deleting a file is done with `unlink()`. Deleting an empty directory is done with `rmdir()`. These are different operations because files and directories are different filesystem entry types.

```
from pathlib import Path
```

```
demo_dir = Path("delete_demo")
demo_dir.mkdir(exist_ok=True)
```

```
file_path = demo_dir / "temporary.txt"
```

```
with open(file_path, "w", encoding="utf-8") as file_obj:
    file_obj.write("Temporary file: No soup for you.\n")
```

```
print("Before file deletion:")
print(f"file_path.exists() = {file_path.exists()}")
print(f"file_path.is_file() = {file_path.is_file()}")
```

```
file_path.unlink()
```

```
print()
print("After file deletion:")
print(f"file_path.exists() = {file_path.exists()}")
```

```
print()
print("Before directory deletion:")
print(f"demo_dir.exists() = {demo_dir.exists()}")
print(f"demo_dir.is_dir() = {demo_dir.is_dir()}")
```

```
demo_dir.rmdir()
```

```
print()
print("After directory deletion:")
print(f"demo_dir.exists() = {demo_dir.exists()}")
```

The method `rmdir()` removes only an empty directory. This is a useful safety property. It prevents a directory tree from being erased accidentally by a simple empty-directory removal call.

When deleting optional files, `missing_ok=True` can make repeated script execution safer.

```
from pathlib import Path
```

```
optional_path = Path("optional_delete_demo.txt")
```



```

with open(optional_path, "w", encoding="utf-8") as file_obj:
    file_obj.write("This file will be removed once.\n")

print("First unlink:")
optional_path.unlink(missing_ok=True)
print(f"optional_path.exists() = {optional_path.exists()}")

print()
print("Second unlink with missing_ok=True:")
optional_path.unlink(missing_ok=True)
print("No exception was raised.")

```

The practical rules are direct:

- use `exists()`, `is_file()`, and `is_dir()` to inspect current filesystem state;
- use `makedirs()` to create directories;
- use `iterdir()` or `glob()` to list entries;
- use `rename()` for renaming or moving;
- use `unlink()` for files;
- use `rmdir()` for empty directories.

All of these operations are external state operations. They should be written with more caution than ordinary in-memory object manipulation.

9.7 Structured Data Interchange

Many external files do not contain free-form text meant only for human reading. They contain structured data: records, lists, tables, nested fields, numbers, flags, and missing values. In those cases, reading the file as plain text is only the first step. The program must also interpret the structure encoded in that text.

This section introduces two common structured data boundaries: JSON and CSV. JSON stores tree-shaped data that maps naturally to Python dictionaries and lists. CSV stores row-oriented tabular data that maps naturally to sequences of fields or dictionaries keyed by column names.

The central rule is simple: structured data files are still external text files, but their usefulness comes from parsing them into meaningful Python objects and serializing Python objects back into agreed external formats.

9.7.1 JSON Data Boundaries

JSON is a text-based data interchange format. It is not Python syntax, even though many JSON structures look similar to Python dictionaries and lists. JSON exists as text outside the Python runtime. When Python reads JSON, it parses that text and creates Python objects. When Python writes JSON, it serializes Python objects back into JSON text.

This makes JSON another boundary-crossing mechanism. The program does not merely read characters; it reconstructs a structured tree of values.

JSON as Text-Based Tree Serialization: Objects, Arrays, Strings, Numbers, Booleans, and Null

A JSON document represents data as a tree. The main JSON value categories are objects, arrays, strings, numbers, booleans, and null.

A JSON object is written with braces and contains name-value pairs. A JSON array is written with square brackets and contains ordered values. JSON strings use double quotes. JSON booleans are written as `true` and `false`. JSON null is written as `null`.

```
json_text = """
{
    "name": "Chuck Norris",
    "level": 42,
    "active": true,
    "quote": "Chuck Norris can parse invalid JSON.",
    "tags": ["joke", "fact", "runtime"],
    "metadata": {
        "source": "course demo",
        "verified": false
    },
    "missing_value": null
}
"""

print("JSON text:")
print(json_text)

print("Runtime object before parsing:")
print(f"type(json_text) = {type(json_text)}")
print(f"len(json_text) = {len(json_text)}")

print()
print("Selected names from dir(json_text):")
for name in ["strip", "splitlines", "replace", "startswith", "encode"]:
    print(f"{name:12} -> {name in dir(json_text)}")
```

At this point, `json_text` is only a Python string. Python has not yet treated it as structured data. The braces, brackets, colons, commas, and quoted names are characters inside a `str` object.

The JSON parser is what converts that text into Python dictionaries, lists, strings, integers, booleans, and `None`.

```
import json

json_text = """
{
    "name": "Chuck Norris",
    "level": 42,
    "active": true,
    "quote": "Chuck Norris can parse invalid JSON.",
    "tags": ["joke", "fact", "runtime"],
    "metadata": {
        "source": "course demo",
        "verified": false
    },
    "missing_value": null
}
"""

data = json.loads(json_text)
```

```

print("Parsed Python object:")
print(data)

print()
print("Object types after parsing:")
print(f"type(data)                = {type(data)}")
print(f"type(data['name'])        = {type(data['name'])}")
print(f"type(data['level'])       = {type(data['level'])}")
print(f"type(data['active'])      = {type(data['active'])}")
print(f"type(data['tags'])        = {type(data['tags'])}")
print(f"type(data['metadata'])    = {type(data['metadata'])}")
print(f"type(data['missing_value']) = {type(data['missing_value'])}")

print()
print("Selected names from dir(data):")
for name in ["keys", "values", "items", "get", "update", "pop"]:
    print(f"{name:10} -> {name in dir(data)}")

print()
print("Dictionary access:")
print(f"name          = {data['name']}")
print(f"level         = {data['level']}")
print(f"first tag      = {data['tags'][0]}")
print(f"metadata src   = {data['metadata']['source']}")

```

The external JSON text has now become normal Python runtime data. The root object is a `dict`. The `tags` value is a `list`. The nested `metadata` value is another `dict`. JSON `true` became Python `True`. JSON `false` became Python `False`. JSON `null` became Python `None`.

This is serialization and deserialization. Serialization turns runtime structure into a stored or transferable textual representation. Deserialization parses that external representation and rebuilds runtime structure.

Loading JSON into Python Dictionaries and Lists with `json.load()` and `json.loads()`

The `json.loads()` function parses JSON from an existing Python string. The final `s` means string. The input is already loaded into memory as text.

```

import json

json_text = """
{
    "show": "The Simpsons",
    "line": "D'oh!",
    "episode_count": 42,
    "characters": ["Homer", "Marge", "Bart"],
    "still_running": true
}
"""

data = json.loads(json_text)

print("Loaded with json.loads():")
print(data)

```

```

print()
print("Object inspection:")
print(f"type(json_text) = {type(json_text)}")
print(f"type(data)      = {type(data)}")

print()
print("Selected names from dir(data):")
for name in ["keys", "items", "get", "setdefault", "pop"]:
    print(f"{name:12} -> {name in dir(data)}")

print()
print("Accessing parsed values:")
print(f"show          = {data['show']}")
print(f"line          = {data['line']}")
print(f"episode_count = {data['episode_count']}")
print(f"first character = {data['characters'][0]}")

```

The `json.load()` function parses JSON from an open file object. It reads from the file object and returns the corresponding Python structure.

```

import json

path = "json_load_demo.json"

json_text = """
{
    "title": "Monty Python record",
    "answer": 42,
    "quotes": [
        "It's just a flesh wound.",
        "Nobody expects the Spanish Inquisition!"
    ],
    "valid": true
}
"""

with open(path, "w", encoding="utf-8") as file_obj:
    file_obj.write(json_text)

with open(path, "r", encoding="utf-8") as file_obj:
    data = json.load(file_obj)

    print("File object used by json.load():")
    print(f"file_obj      = {file_obj}")
    print(f"type(file_obj) = {type(file_obj)}")

print()
print("Loaded Python object:")
print(data)

print()
print("Object types:")
print(f"type(data)      = {type(data)}")
print(f"type(data['quotes']) = {type(data['quotes'])}")

```

```

print(f"type(data['answer']) = {type(data['answer'])}")
print(f"type(data['valid']) = {type(data['valid'])}")

print()
print("Iterating through the quote list:")
for index, quote in enumerate(data["quotes"], start=1):
    print(f"{index:02d}: {quote}")
    print(f"    type(quote) = {type(quote)}")

```

The difference is only the source of the JSON text.

- `json.loads(json_text)` parses JSON from a `str`.
- `json.load(file_obj)` parses JSON from a text file object.

Both return Python objects. The root object may be a dictionary, a list, a string, a number, a Boolean, or None, depending on the JSON document.

A JSON document does not have to start with an object. It may also start with an array.

```

import json

json_text = """
[
    {"name": "Jerry", "score": 38},
    {"name": "Elaine", "score": 42},
    {"name": "George", "score": 17}
]
"""

rows = json.loads(json_text)

print("Root JSON value became a Python list:")
print(rows)

print()
print("Object inspection:")
print(f"type(rows) = {type(rows)}")
print(f"len(rows) = {len(rows)}")

print()
print("Selected names from dir(rows):")
for name in ["append", "extend", "insert", "pop", "sort", "reverse"]:
    print(f"{name:10} -> {name in dir(rows)}")

print()
print("Inspecting list elements:")
for row in rows:
    print(f"name = {row['name']:8} | score = {row['score']:02d}")
    print(f"    type(row) = {type(row)}")

```

Here the root Python object is a `list`, and each element of that list is a `dict`. JSON therefore represents nested trees, not only flat records.

Writing Python Structures Back to JSON with `json.dump()` and `json.dumps()`

The reverse operation converts Python structures into JSON text. The `json.dumps()` function serializes a Python object into a JSON string. The final `s` again means string.

```
import json

data = {
    "course": "Python I/O",
    "topic": "JSON",
    "answer": 42,
    "quotes": [
        "Serenity now!",
        "No soup for you!"
    ],
    "active": True,
    "missing": None
}

json_text = json.dumps(data, indent=2)

print("Original Python object:")
print(data)

print()
print("Serialized JSON text:")
print(json_text)

print()
print("Object types:")
print(f"type(data)          = {type(data)}")
print(f"type(json_text) = {type(json_text)}")

print()
print("Selected names from dir(json_text):")
for name in ["splitlines", "encode", "replace", "startswith", "count"]:
    print(f"{name:12} -> {name in dir(json_text)}")
```

The variable `data` refers to a Python dictionary. The variable `json_text` refers to a Python string containing JSON-formatted text. That string can be printed, stored in a file, sent over a network, or parsed later.

The `indent=2` argument asks the JSON serializer to format the output across multiple lines with indentation. Without it, the output is more compact.

```
import json

data = {
    "name": "Homer",
    "line": "D'oh!",
    "number": 42
}

compact_json = json.dumps(data)
pretty_json = json.dumps(data, indent=2)
```

```

print("Compact JSON:")
print(compact_json)

print()
print("Pretty JSON:")
print(pretty_json)

print()
print("Lengths:")
print(f"len(compact_json) = {len(compact_json)}")
print(f"len(pretty_json) = {len(pretty_json)}")

```

The `json.dump()` function serializes a Python object directly into an open text file object.

```

import json

path = "json_dump_demo.json"

data = {
    "name": "Arthur Dent",
    "answer": 42,
    "items": ["towel", "tea", "guide"],
    "panicking": False
}

with open(path, "w", encoding="utf-8") as file_obj:
    result = json.dump(data, file_obj, indent=2)

    print("File object used by json.dump():")
    print(f"file_obj      = {file_obj}")
    print(f"type(file_obj) = {type(file_obj)}")

print()
print("Return value from json.dump():")
print(f"result = {result}")

with open(path, "r", encoding="utf-8") as file_obj:
    content = file_obj.read()

print()
print("JSON file content read back as text:")
print(content)

print()
print("Type of file content:")
print(f"type(content) = {type(content)}")

```

The function `json.dump()` writes into the file and returns `None`. The JSON file is still a text file. Reading it with `file_obj.read()` returns a string. Reading it with `json.load()` returns parsed Python objects.

```

import json

path = "json_dump_demo.json"

```

```

with open(path, "r", encoding="utf-8") as file_obj:
    parsed_data = json.load(file_obj)

print("Parsed data:")
print(parsed_data)

print()
print("Object type:")
print(f"type(parsed_data) = {type(parsed_data)}")

print()
print("Accessing fields:")
print(f"name    = {parsed_data['name']}")
print(f"answer = {parsed_data['answer']}")
print(f"items  = {parsed_data['items']}")

```

The practical distinction is:

- `json.dumps(data)` returns a JSON `str`;
- `json.dump(data, file_obj)` writes JSON text to a file object;
- `json.loads(json_text)` parses a JSON `str`;
- `json.load(file_obj)` parses JSON text from a file object.

JSON Type Mapping Boundaries: None vs. null, dict vs. Object, list vs. Array

JSON and Python do not use the same names for every value category. The parser maps JSON types into Python types, and the serializer maps Python types back into JSON types.

The most common mappings are:

- JSON object maps to Python `dict`;
- JSON array maps to Python `list`;
- JSON string maps to Python `str`;
- JSON number maps to Python `int` or `float`;
- JSON `true` maps to Python `True`;
- JSON `false` maps to Python `False`;
- JSON `null` maps to Python `None`.

```

import json

json_text = """
{
  "dict_like": {"a": 1, "b": 2},
  "list_like": [10, 20, 42],
  "text_like": "D'oh!",
  "int_like": 42,
  "float_like": 3.14,
  "true_like": true,
  "false_like": false,
  "null_like": null
}

```



```

}
"""

data = json.loads(json_text)

print("Parsed values and Python types:")
for key, value in data.items():
    print(f"{key:12} -> {value!r:20} | type = {type(value)}")

```

The mapping is visible in the printed types. There is no Python value named `null`. Python uses `None`. There are no Python Boolean literals named `true` and `false`. Python uses `True` and `False`.

The reverse mapping appears during serialization.

```

import json

data = {
    "python_none": None,
    "python_true": True,
    "python_false": False,
    "python_list": [1, 2, 42],
    "python_dict": {"line": "No soup for you!"}
}

json_text = json.dumps(data, indent=2)

print("Python object:")
print(data)

print()
print("JSON text:")
print(json_text)

```

The Python value `None` is written as JSON `null`. Python `True` and `False` are written as JSON `true` and `false`. Python dictionaries become JSON objects, and Python lists become JSON arrays.

Not every Python object can be serialized to JSON automatically. JSON has a small type system. Arbitrary Python objects, file objects, sets, and many other runtime objects do not have a default JSON representation.

```

import json

data = {
    "normal_list": [1, 2, 3],
    "problem_set": {1, 2, 3}
}

print("Trying to serialize a set:")

try:
    json_text = json.dumps(data)
except TypeError as err:
    print("Serialization failed.")
    print(f"type(err) = {type(err)}")
    print(f"err      = {err}")

print()

```

```
print("Selected names from dir(err):")
for name in ["args", "with_traceback", "add_note"]:
    print(f"{name:16} -> {name in dir(err)}")
```

A `set` is a normal Python object, but JSON has no direct set type. The programmer must choose a representation, often a list.

```
import json

data = {
    "normal_list": [1, 2, 3],
    "set_as_list": list({1, 2, 3})
}

json_text = json.dumps(data, indent=2)

print("Serializable version:")
print(json_text)
```

There is also a syntax boundary. JSON is not Python literal syntax. JSON strings require double quotes. JSON booleans are lowercase. JSON null is lowercase. Trailing commas are not allowed.

```
import json

bad_json = """
{
    "name": "Homer",
    "line": "D'oh!",
}
"""

print("Trying to parse invalid JSON:")

try:
    data = json.loads(bad_json)
except json.JSONDecodeError as err:
    print("JSON parsing failed.")
    print(f"type(err) = {type(err)}")
    print(f"err      = {err}")

    print()
    print("Selected names from dir(err):")
    for name in ["msg", "doc", "pos", "lineno", "colno"]:
        print(f"{name:8} -> {name in dir(err)}")

    print()
    print(f"line number    = {err.lineno}")
    print(f"column number = {err.colno}")
    print(f"message       = {err.msg}")
```

The practical rule is strict: JSON is an external text format with its own grammar and type system. Python can map JSON values into Python objects and back again, but the boundary is real. A JSON document is not a dictionary until it is parsed, and a Python dictionary is not JSON until it is serialized.

9.7.2 CSV Tabular Data

CSV as Row-Oriented Text with Delimited Fields

Reading CSV Files with `csv.reader`

Dictionary-Based Row Access with `csv.DictReader`

Writing CSV Rows with `csv.writer` and `csv.DictWriter`

Quoting, Escaping, Delimiters, and Newline Handling

9.8 Error Handling in I/O Operations

I/O operations are less predictable than ordinary in-memory operations because they depend on external state controlled by the operating system. A variable lookup either finds a runtime binding or it does not, but a file operation may fail for many outside reasons: the file may be missing, access may be denied, the path may be invalid, or another process may change the filesystem at the same time.

This section explains the expected failure modes of file access and the defensive patterns used to handle them. The goal is not to prevent every failure in advance, because that is impossible at the I/O boundary. The goal is to write code that expects failure, catches the right exceptions, and keeps filesystem operations explicit and safe.

The central rule is simple: external resources are not under full program control, so serious I/O code must protect the operation itself, not only the assumptions made before the operation.

9.8.1 Expected Failure Modes

I/O code crosses from Python runtime objects into external operating-system state. That means failure is normal, not exceptional in the everyday sense. Files may be missing. Permissions may reject access. A path may refer to a directory instead of a file. A name may be invalid on the current platform. Another process may move, lock, or delete a file between two operations.

Good I/O code must therefore treat failure modes as part of the interface. Opening a file is not only a Python operation. It is a request sent to the operating system, and that request can be refused.

Missing Files and `FileNotFoundError`

The most direct failure is trying to read a file that does not exist. In read mode, Python expects the external file to already exist. If the path cannot be resolved to an existing file, Python raises `FileNotFoundError`.

```
from pathlib import Path

path = Path("missing_file_demo.txt")

print("Path before opening:")
print(f"path          = {path}")
print(f"type(path)    = {type(path)}")
print(f"path.exists() = {path.exists()}")

print()
print("Selected names from dir(path):")
for name in ["exists", "is_file", "is_dir", "name", "parent", "resolve"]:
    print(f"{name:10} -> {name in dir(path)}")

print()
print("Trying to read a missing file:")
```

```

try:
    with open(path, "r", encoding="utf-8") as file_obj:
        content = file_obj.read()
except FileNotFoundError as err:
    print("The file could not be opened because it does not exist.")
    print(f"type(err) = {type(err)}")
    print(f"err      = {err}")

    print()
    print("Selected names from dir(err):")
    for name in ["args", "filename", "errno", "strerror", "with_traceback"]:
        print(f"{name:16} -> {name in dir(err)}")

    print()
    print(f"err.filename = {err.filename}")
    print(f"err.errno   = {err.errno}")
    print(f"err.strerror = {err.strerror}")

```

The exception object contains information about the failed operation. The `filename` attribute can identify the path involved. The `errno` and `strerror` attributes expose lower-level operating-system error information when available.

The important rule is that a missing file is not represented by `None`. Python does not return an empty string, an empty list, or a false value. It raises an exception because the requested file operation could not be performed.

A successful read creates ordinary runtime text only after the file is actually opened and read.

```

from pathlib import Path

path = Path("existing_file_demo.txt")

with open(path, "w", encoding="utf-8") as file_obj:
    file_obj.write("D'oh! This file exists now.\n")
    file_obj.write("The answer is 42.\n")

with open(path, "r", encoding="utf-8") as file_obj:
    content = file_obj.read()

print("Successful read:")
print(content)

print("Object inspection:")
print(f"type(content) = {type(content)}")
print(f"len(content)    = {len(content)}")

```

The path alone does not provide the content. The content appears only after the I/O operation succeeds.

Permission Failures and PermissionError

A path may exist and still be inaccessible. The operating system may reject the operation because the current process does not have permission to read, write, create, or delete the target.

When access is denied, Python commonly raises `PermissionError`. The exact situations depend on the operating system, filesystem, user account, and security settings.

```

from pathlib import Path

```

```

path = Path("permission_pattern_demo.txt")

print("Permission failures are operating-system decisions.")
print("This example shows the normal handling pattern.")

try:
    with open(path, "w", encoding="utf-8") as file_obj:
        file_obj.write("This write may succeed in a normal directory.\n")

    print("The write succeeded here.")

except PermissionError as err:
    print("The operating system refused access.")
    print(f"type(err) = {type(err)}")
    print(f"err      = {err}")

    print()
    print("Selected names from dir(err):")
    for name in ["args", "filename", "errno", "strerror"]:
        print(f"{name:12} -> {name in dir(err)}")

```

This code may succeed because the current directory is usually writable. That is normal. Permission errors are not produced by the syntax of `open()`; they are produced by the operating system when the requested access violates the current filesystem rules.

A safer demonstration is to construct and inspect a `PermissionError` object directly. This does not pretend that the operating system denied the specific file operation. It only shows the exception type as a Python object.

```

err = PermissionError("Access denied by demonstration.")

print("Constructed PermissionError object:")
print(f"err      = {err}")
print(f"type(err) = {type(err)}")

print()
print("Selected names from dir(err):")
for name in ["args", "with_traceback", "add_note"]:
    print(f"{name:16} -> {name in dir(err)}")

print()
print(f"err.args = {err.args}")

```

In real file code, the exception is not usually constructed manually. It is raised by Python after the operating system refuses the operation.

Some permission-like failures can also appear through related exception classes. For example, trying to open a directory as if it were a file may fail with `IsADirectoryError` on many systems.

```

from pathlib import Path

dir_path = Path("directory_open_demo")
dir_path.mkdir(exist_ok=True)

print("Directory path:")
print(f"dir_path      = {dir_path}")

```

```

print(f"type(dir_path) = {type(dir_path)}")
print(f"dir_path.is_dir() = {dir_path.is_dir()}")

print()
print("Trying to open a directory as a text file:")

try:
    with open(dir_path, "r", encoding="utf-8") as file_obj:
        content = file_obj.read()
except OSError as err:
    print("The operating system rejected the operation.")
    print(f"type(err) = {type(err)}")
    print(f"err      = {err}")

```

Catching `OSError` here is intentional. Filesystem failures are often subclasses of `OSError`, and the exact subclass can vary with platform and operation.

Invalid Paths, Locked Files, and Platform-Specific Filesystem Constraints

Not every string or `Path` object can become a valid filesystem location. Some path forms are invalid. Some names are forbidden on particular systems. Some paths are too long. Some files are locked by other processes. Some operations are legal on one operating system but fail on another.

Python exposes many of these failures as `OSError` or one of its subclasses. Some invalid values can fail before the operating system operation is even attempted.

A null byte inside a path is invalid.

```

bad_path = "bad\0name.txt"

print("Bad path string:")
print(f"bad_path      = {bad_path!r}")
print(f"type(bad_path) = {type(bad_path)}")

print()
print("Trying to open a path containing a null byte:")

try:
    with open(bad_path, "w", encoding="utf-8") as file_obj:
        file_obj.write("This will not be written.\n")
except ValueError as err:
    print("Python rejected the path value.")
    print(f"type(err) = {type(err)}")
    print(f"err      = {err}")

    print()
    print("Selected names from dir(err):")
    for name in ["args", "with_traceback", "add_note"]:
        print(f"{name:16} -> {name in dir(err)}")

```

This is not a missing-file problem. The path value itself is invalid for a filesystem operation.

Other failures come from the operating system. A very long file name, a reserved name, an invalid character, or a locked file may produce different errors depending on the platform.

```

from pathlib import Path

base_dir = Path("path_error_demo")

```

```

base_dir.mkdir(exist_ok=True)

very_long_name = "x" * 300 + ".txt"
long_path = base_dir / very_long_name

print("Long path attempt:")
print(f"type(long_path) = {type(long_path)}")
print(f"name length      = {len(long_path.name)}")

print()
print("Trying to create a file with a very long name:")

try:
    with open(long_path, "w", encoding="utf-8") as file_obj:
        file_obj.write("This may fail on many filesystems.\n")

    print("The file was created on this filesystem.")

except OSError as err:
    print("The operating system rejected the path.")
    print(f"type(err) = {type(err)}")
    print(f"err       = {err}")

    print()
    print("Selected names from dir(err):")
    for name in ["args", "filename", "errno", "strerror"]:
        print(f"{name:12} -> {name in dir(err)}")

```

The exact result is intentionally not promised. Filesystem limits are external rules. Python asks for the operation; the operating system and filesystem decide whether it is valid.

Locked files are also platform-dependent. Some systems allow a file to be opened by several processes at the same time. Some operations may fail if another process holds an exclusive lock. Python code must be prepared for that failure even if the same code often works during testing.

The practical rule is this: I/O failure is part of the contract. A file path can be syntactically valid as a Python object and still fail as an operating-system operation. Therefore, serious I/O code should expect exceptions such as `FileNotFoundError`, `PermissionError`, `IsADirectoryError`, `FileExistsError`, and broader `OSError` failures.

9.8.2 Defensive I/O Patterns

Defensive I/O code accepts that the filesystem is external state. A path may exist, disappear, become inaccessible, become a directory, or be changed by another process. The program therefore needs patterns that handle failure cleanly instead of assuming that every file operation will succeed.

Python commonly uses two styles: try the operation and handle the exception, or check conditions before attempting the operation. Both are useful, but they do not provide the same guarantees.

EAFP File Access: Trying the Operation and Handling the Exception

EAFP means “easier to ask forgiveness than permission”. In file access, this means the program attempts the operation directly and catches the exception if the operating system refuses it.

```

from pathlib import Path

path = Path("missing_eafp_demo.txt")

```

```

print("Path object:")
print(f"path          = {path}")
print(f"type(path) = {type(path)}")

print()
print("Selected names from dir(path):")
for name in ["exists", "is_file", "is_dir", "name", "parent", "resolve"]:
    print(f"{name:10} -> {name in dir(path)}")

print()
print("EAFP pattern: try to read, then handle failure.")

try:
    with open(path, "r", encoding="utf-8") as file_obj:
        content = file_obj.read()

    print("File content:")
    print(content)

except FileNotFoundError as err:
    print("The file does not exist.")
    print(f"type(err) = {type(err)}")
    print(f"err      = {err}")

except PermissionError as err:
    print("The file exists, but access was denied.")
    print(f"type(err) = {type(err)}")
    print(f"err      = {err}")

except OSError as err:
    print("Another operating-system I/O error occurred.")
    print(f"type(err) = {type(err)}")
    print(f"err      = {err}")

```

This pattern is strong because the attempted operation is the same operation whose failure is being handled. The program does not merely ask whether the file looked usable a moment earlier. It asks the operating system to actually perform the read.

EAFP is especially useful when failure can happen for many external reasons. The path may be missing, permission may be denied, the path may point to a directory, the file may be locked, or the filesystem may reject the operation.

A successful EAFP example looks ordinary:

```

from pathlib import Path

path = Path("existing_eafp_demo.txt")

with open(path, "w", encoding="utf-8") as file_obj:
    file_obj.write("Serenity now!\n")
    file_obj.write("The answer is 42.\n")

try:
    with open(path, "r", encoding="utf-8") as file_obj:
        content = file_obj.read()

```



```

    print("Read succeeded:")
    print(content)

    print("Object inspection:")
    print(f"type(content) = {type(content)}")
    print(f"len(content)   = {len(content)}")

except OSError as err:
    print("Read failed.")
    print(f"type(err) = {type(err)}")
    print(f"err      = {err}")

```

The `except OSError` branch is broader than `FileNotFoundError` or `PermissionError`. This is useful when the program wants to handle general filesystem failure at one boundary.

LBYL File Access: Checking Conditions Before Opening

LBYL means “look before you leap”. In file access, this means the program checks conditions before attempting the operation.

```

from pathlib import Path

path = Path("lbyl_demo.txt")

with open(path, "w", encoding="utf-8") as file_obj:
    file_obj.write("No soup for you!\n")

print("LBYL pattern: check before opening.")

exists_result = path.exists()
file_result = path.is_file()

print(f"path.exists() = {exists_result}")
print(f"path.is_file() = {file_result}")

print()
print("Object types:")
print(f"type(exists_result) = {type(exists_result)}")
print(f"type(file_result)   = {type(file_result)}")

print()
print("Selected names from dir(exists_result):")
for name in ["bit_length", "to_bytes", "real", "imag"]:
    print(f"{name:12} -> {name in dir(exists_result)}")

if path.exists() and path.is_file():
    with open(path, "r", encoding="utf-8") as file_obj:
        content = file_obj.read()

    print()
    print("File content:")
    print(content)
else:
    print("The path is not currently a readable file candidate.")

```

The checks are clear and readable. They are useful when the program wants to choose different branches based on current filesystem state.

However, the checks do not open the file. They only report what appeared to be true when the checks were made. The later `open()` call is still a separate operation that can fail.

LBYL is also useful when producing clearer messages for a human user:

```
from pathlib import Path

path = Path("lbyl_missing_demo.txt")

if not path.exists():
    print(f"Cannot read {path}: the path does not exist.")
elif not path.is_file():
    print(f"Cannot read {path}: the path is not a file.")
else:
    with open(path, "r", encoding="utf-8") as file_obj:
        content = file_obj.read()

    print(content)
```

This style makes the decision tree visible. It is often helpful in scripts, teaching examples, command-line utilities, and validation code.

The weakness is that the checks and the operation are not one indivisible action.

Atomicity Concerns: Why Existence Checks Can Become Invalid Before Use

The filesystem can change between two Python statements. Another process can create, delete, rename, or replace a file after the program checks it but before the program opens it. This is often called a check-then-use problem.

The following example shows the shape of the issue without needing another process.

```
from pathlib import Path

path = Path("atomicity_demo.txt")

with open(path, "w", encoding="utf-8") as file_obj:
    file_obj.write("This file exists for the first check.\n")

print("First check:")
print(f"path.exists() = {path.exists()}")

print()
print("Simulating an external change by deleting the file before open().")
path.unlink()

print(f"path.exists() = {path.exists()}")

print()
print("Now the earlier check is no longer useful:")

try:
    with open(path, "r", encoding="utf-8") as file_obj:
        content = file_obj.read()

    print(content)
```

```
except FileNotFoundError as err:
    print("The file disappeared before it was opened.")
    print(f"type(err) = {type(err)}")
    print(f"err      = {err}")
```

The first existence check was true, but it did not reserve the file. It did not lock the file. It did not guarantee that a later read would succeed. The external state changed before the actual operation.

This is why EAFP is often preferred for the final file operation. The program still may use LBYL for user-friendly diagnostics, but the real operation should remain protected by exception handling.

A safer creation pattern uses exclusive creation mode "x". It asks the operating system to create the file only if it does not already exist.

```
from pathlib import Path

path = Path("exclusive_create_demo.txt")

print("Trying exclusive creation:")

try:
    with open(path, "x", encoding="utf-8") as file_obj:
        file_obj.write("Created exactly once. D'oh!\n")

    print("File was created.")

except FileExistsError as err:
    print("File already exists; it was not overwritten.")
    print(f"type(err) = {type(err)}")
    print(f"err      = {err}")

print()
print("Trying exclusive creation again:")

try:
    with open(path, "x", encoding="utf-8") as file_obj:
        file_obj.write("This should not replace the old file.\n")

    print("File was created.")

except FileExistsError as err:
    print("Second creation failed safely.")
    print(f"type(err) = {type(err)}")
    print(f"err      = {err}")
```

This is better than checking `exists()` and then opening with "w", because "w" would replace existing content. The mode "x" combines the existence condition with the creation operation.

The practical rule is:

- use EAFP when performing the actual file operation;
- use LBYL when clear pre-checks help produce better messages;
- do not treat existence checks as permanent guarantees;
- use operation-level safeguards such as "x" when overwriting would be unsafe.

Filesystem state belongs to the operating system and may be shared with other processes. Defensive I/O code must therefore protect the actual operation, not only the condition checked before it.

9.9 Practical External Data Pipelines

Real programs often use files as part of a larger data movement pattern. They read external data, convert it into runtime objects, transform those objects, and write the result back to another external form. This is the practical shape of many scripts, import tools, converters, filters, and report generators.

This section connects the earlier I/O mechanisms into complete pipelines. It explains why large inputs should be streamed instead of loaded all at once, how text and binary data can be processed incrementally, and how read-transform-write programs can be made safer with temporary files and clear separation between parsing, processing, and serialization.

The central rule is simple: external data should cross the runtime boundary in controlled stages, so the program remains memory-aware, failure-aware, and structurally readable.

9.9.1 Streaming Large Inputs

Large input files should not always be loaded into memory as one complete object. A small text file can be handled with `read()`, and a small binary file can be handled with `read()` as well. A large log file, dataset export, image archive, or media file should usually be processed incrementally.

Streaming means that the program reads one manageable piece at a time, processes that piece, and then moves to the next piece. The whole file does not need to exist as one giant Python object in memory.

Processing Files Line by Line Without Loading Entire Contents into Memory

For text files, the most common streaming pattern is direct iteration over the file object. Each loop step receives one line as a `str` object. The program processes that line and then continues.

```
from pathlib import Path

path = Path("large_text_stream_demo.txt")

with open(path, "w", encoding="utf-8") as file_obj:
    file_obj.write("001 | Homer | D'oh!\n")
    file_obj.write("002 | Jerry | Serenity now!\n")
    file_obj.write("003 | Arthur | Don't panic.\n")
    file_obj.write("004 | Chuck | Counted to 42 twice.\n")

line_count = 0
char_count = 0

with open(path, "r", encoding="utf-8") as file_obj:
    print("File object used as a line stream:")
    print(f"file_obj      = {file_obj}")
    print(f"type(file_obj) = {type(file_obj)}")

    print()
    print("Selected names from dir(file_obj):")
    for name in ["__iter__", "read", "readline", "readlines", "close", "mode"]:
        print(f"{name:12} -> {name in dir(file_obj)}")

    print()
    print("Streaming lines:")

    for line in file_obj:
        line_count += 1
        char_count += len(line)
```

```

        clean_line = line.rstrip("\n")

        print(f"line {line_count:02d}: {clean_line}")
        print(f"    type(line) = {type(line)}")
        print(f"    len(line)  = {len(line)}")

print()
print("Summary:")
print(f"line_count = {line_count}")
print(f"char_count  = {char_count}")

```

The loop variable `line` refers to one `str` object at a time. The program does not build a list of all lines. It does not create one large string containing the entire file. It advances through the file sequentially.

This is the practical difference between streaming and materialization:

- `file_obj.read()` materializes the whole remaining file as one string;
- `file_obj.readlines()` materializes all remaining lines as a list;
- `for line in file_obj` processes one line per loop step.

Line streaming is useful for filters, counters, searches, log processors, and text transformations.

```

from pathlib import Path

path = Path("large_text_filter_demo.txt")

with open(path, "w", encoding="utf-8") as file_obj:
    file_obj.write("INFO   | Startup complete.\n")
    file_obj.write("WARN   | Disk almost full.\n")
    file_obj.write("INFO   | User asked for 42.\n")
    file_obj.write("ERROR  | Nobody expects the Spanish Inquisition.\n")
    file_obj.write("INFO   | Shutdown complete.\n")

error_count = 0
warn_count = 0

with open(path, "r", encoding="utf-8") as file_obj:
    for line_number, line in enumerate(file_obj, start=1):
        clean_line = line.rstrip("\n")

        if clean_line.startswith("ERROR"):
            error_count += 1
            print(f"error at line {line_number:02d}: {clean_line}")

        if clean_line.startswith("WARN"):
            warn_count += 1
            print(f"warning at line {line_number:02d}: {clean_line}")

print()
print("Streaming summary:")
print(f"error_count = {error_count}")
print(f"warn_count  = {warn_count}")

```

The program only keeps counters and the current line. That is the important memory property. The file may contain five lines or five million lines; the loop shape remains the same.

Chunked Binary Processing for Large Media or Archive Files

Binary files usually do not have lines. A binary stream is more naturally processed in fixed-size chunks. The program reads a block of bytes, processes that block, and repeats until `read(size)` returns `b""`.

```
from pathlib import Path

path = Path("large_binary_stream_demo.bin")

data = bytearray()

for number in range(1, 65):
    data.append(number)

with open(path, "wb") as file_obj:
    file_obj.write(data)

chunk_size = 10
chunk_count = 0
byte_count = 0

with open(path, "rb") as file_obj:
    print("Binary file object used as a byte stream:")
    print(f"file_obj      = {file_obj}")
    print(f"type(file_obj) = {type(file_obj)}")

    print()
    print("Selected names from dir(file_obj):")
    for name in ["read", "write", "seek", "tell", "close", "mode"]:
        print(f"{{name:10}} -> {{name in dir(file_obj) }}")

    print()
    print("Reading binary chunks:")

    while True:
        chunk = file_obj.read(chunk_size)

        if chunk == b"":
            break

        chunk_count += 1
        byte_count += len(chunk)

        print(f"chunk {{chunk_count:02d}}: {{chunk}}")
        print(f"    type(chunk) = {{type(chunk)}}")
        print(f"    len(chunk)  = {{len(chunk)}}")

    print()
    print("Binary streaming summary:")
    print(f"chunk_count = {{chunk_count}}")
    print(f"byte_count  = {{byte_count}}")
```

The object returned by each `read(chunk_size)` call is a `bytes` object. It is immutable and contains at most `chunk_size` bytes. The final chunk may be smaller if the file ends before a full block is available.

Chunked processing is the normal shape for large binary transfers because it bounds memory use.

```
from pathlib import Path

src_path = Path("binary_source_demo.bin")
dst_path = Path("binary_copy_demo.bin")

with open(src_path, "wb") as file_obj:
    file_obj.write(b"Spam and eggs.\n")
    file_obj.write(b"D'oh!\n")
    file_obj.write(b"The answer is 42.\n")

chunk_size = 8
total_count = 0

with open(src_path, "rb") as src_obj:
    with open(dst_path, "wb") as dst_obj:
        while True:
            chunk = src_obj.read(chunk_size)

            if chunk == b"":
                break

            written_count = dst_obj.write(chunk)
            total_count += written_count

            print(f"copied {written_count:02d} bytes: {chunk!r}")

print()
print(f"total_count = {total_count}")

with open(dst_path, "rb") as file_obj:
    copied_data = file_obj.read()

print()
print("Copied data:")
print(copied_data)

print()
print("Object inspection:")
print(f"type(copied_data) = {type(copied_data)}")
print(f"len(copied_data) = {len(copied_data)}")
```

This example copies a binary file without reading the whole source file at once. The program repeatedly reads a small `bytes` chunk from the source and writes that same chunk to the destination.

A chunk can also be inspected or counted without being decoded as text.

```
from pathlib import Path

path = Path("binary_count_demo.bin")

with open(path, "wb") as file_obj:
```

```

file_obj.write(b"AAAABBBBCCCC")
file_obj.write(bytes([0, 1, 2, 3, 42, 255]))
file_obj.write(b"AAA")

chunk_size = 5
target_byte = ord("A")
target_count = 0
total_bytes = 0

with open(path, "rb") as file_obj:
    while True:
        chunk = file_obj.read(chunk_size)

        if chunk == b"":
            break

        total_bytes += len(chunk)
        target_count += chunk.count(target_byte)

        print(f"chunk = {chunk!r}")
        print(f"    count of byte {target_byte} = {chunk.count(target_byte)}")

print()
print("Final counts:")
print(f"total_bytes   = {total_bytes}")
print(f"target_count   = {target_count}")

```

The variable `target_byte` is an integer byte value. The `bytes.count()` method can count that value inside each chunk. No text decoding is involved.

The practical streaming rule is direct: text streams are usually processed line by line; binary streams are usually processed chunk by chunk. Both patterns avoid unnecessary full-file materialization and keep memory use tied to the current unit of work rather than the total file size.

9.9.2 Transforming External Data

Many useful I/O programs do not merely read data or write data. They read external data, convert it into runtime objects, modify or filter those objects, and then write a new external representation. This is a data pipeline.

The pipeline shape is simple:

```
external input -> Python objects -> transformed Python objects -> external output
```

The important point is that each stage has a different responsibility. Reading crosses into the runtime. Processing changes runtime objects. Writing crosses back out to external storage.

Read-Transform-Write Pipelines

A read-transform-write pipeline reads data from one file, changes it, and writes the result to another file. The simplest form is a text filter.

```

from pathlib import Path

src_path = Path("pipeline_input.txt")
dst_path = Path("pipeline_output.txt")

```



```

with open(src_path, "w", encoding="utf-8") as file_obj:
    file_obj.write("homer | d'oh! | 42\n")
    file_obj.write("jerry | serenity now! | 17\n")
    file_obj.write("arthur | don't panic | 23\n")

print("Input and output paths:")
print(f"src_path      = {src_path}")
print(f"dst_path      = {dst_path}")
print(f"type(src_path) = {type(src_path)}")

print()
print("Selected names from dir(src_path):")
for name in ["exists", "is_file", "name", "parent", "suffix", "replace"]:
    print(f"{name:10} -> {name in dir(src_path)}")

with open(src_path, "r", encoding="utf-8") as src_obj:
    with open(dst_path, "w", encoding="utf-8") as dst_obj:
        for line in src_obj:
            clean_line = line.strip()
            upper_line = clean_line.upper()

            dst_obj.write(upper_line)
            dst_obj.write("\n")

            print(f"read:  {clean_line}")
            print(f"wrote: {upper_line}")

print()
print("Output file content:")

with open(dst_path, "r", encoding="utf-8") as file_obj:
    content = file_obj.read()

print(content)
print(f"type(content) = {type(content)}")

```

This pipeline never loads the whole input file as one string. It processes one line at a time. Each input line becomes a `str`, the string is transformed, and the transformed string is written to the output file.

A slightly more structured pipeline can parse fields from each line.

```

from pathlib import Path

src_path = Path("scores_input.txt")
dst_path = Path("scores_output.txt")

with open(src_path, "w", encoding="utf-8") as file_obj:
    file_obj.write("Homer,38\n")
    file_obj.write("Elaine,42\n")
    file_obj.write("Kramer,29\n")

with open(src_path, "r", encoding="utf-8") as src_obj:
    with open(dst_path, "w", encoding="utf-8") as dst_obj:
        dst_obj.write("name,score,status\n")

```

```

for line in src_obj:
    clean_line = line.strip()
    parts = clean_line.split(",")

    name = parts[0]
    score = int(parts[1])

    if score >= 40:
        status = "high"
    else:
        status = "normal"

    out_line = f"{name},{score},{status}\n"
    dst_obj.write(out_line)

    print(f"{name:8} score={score:02d} status={status}")

print()
print("Transformed file content:")

with open(dst_path, "r", encoding="utf-8") as file_obj:
    content = file_obj.read()

print(content)

```

Here the program crosses several internal boundaries. The line is first a `str`. The call to `split()` creates a `list`. The score field is converted from `str` to `int`. Then the final output line is formatted back into a `str` and written to the destination file.

The pipeline is not only copying. It is interpreting, transforming, and serializing data.

Temporary Files and Safe Output Replacement

A direct output file can be risky when replacing an existing file. If the program opens the final output path with `"w"`, the old contents are erased immediately. If an error occurs halfway through writing, the program may leave behind a broken partial file.

A safer pattern writes the new content to a temporary file first. Only after the temporary file is complete does the program replace the real output file.

```

from pathlib import Path
import tempfile

target_path = Path("safe_replace_output.txt")

with open(target_path, "w", encoding="utf-8") as file_obj:
    file_obj.write("old line 1\n")
    file_obj.write("old line 2\n")

print("Original target content:")

with open(target_path, "r", encoding="utf-8") as file_obj:
    print(file_obj.read())

tmp_obj = tempfile.NamedTemporaryFile(
    "w",

```

```

        encoding="utf-8",
        delete=False,
        dir=target_path.parent,
        prefix="safe_replace_",
        suffix=".tmp"
    )

    tmp_path = Path(tmp_obj.name)

    print("Temporary file object:")
    print(f"tmp_obj          = {tmp_obj}")
    print(f"type(tmp_obj) = {type(tmp_obj)}")

    print()
    print("Temporary path:")
    print(f"tmp_path          = {tmp_path}")
    print(f"type(tmp_path) = {type(tmp_path)}")

    print()
    print("Selected names from dir(tmp_path):")
    for name in ["exists", "is_file", "replace", "unlink", "name", "parent"]:
        print(f"{name:10} -> {name in dir(tmp_path)}")

    try:
        with tmp_obj:
            tmp_obj.write("new line 1: Nobody expects the Spanish Inquisition.\n")
            tmp_obj.write("new line 2: The answer is 42.\n")

        tmp_path.replace(target_path)
        print()
        print("Replacement completed.")

    except OSError as err:
        print()
        print("Replacement failed.")
        print(f"type(err) = {type(err)}")
        print(f"err          = {err}")

        if tmp_path.exists():
            tmp_path.unlink()
            print("Temporary file removed.")

    print()
    print("Final target content:")

    with open(target_path, "r", encoding="utf-8") as file_obj:
        print(file_obj.read())

```

The temporary file is created in the same parent directory as the target. That is a useful habit because replacement is usually safest when both paths belong to the same filesystem location.

The call `tmp_path.replace(target_path)` asks the operating system to replace the target path with the temporary file. This is safer than destroying the original file before the new content has been completely written.

This pattern does not make every failure impossible. The disk may become full, permissions may

change, or the replacement itself may fail. But it reduces one common danger: leaving the final output file half-written after a transformation error.

Separating Parsing, Processing, and Serialization Functions

In larger pipelines, it is useful to separate three ideas:

- parsing: convert external text into Python objects;
- processing: transform Python objects;
- serialization: convert Python objects back into external text.

This separation prevents one loop from becoming an unreadable mixture of file access, string splitting, numeric conversion, business rules, and output formatting.

```
from pathlib import Path

src_path = Path("separated_pipeline_input.txt")
dst_path = Path("separated_pipeline_output.txt")

with open(src_path, "w", encoding="utf-8") as file_obj:
    file_obj.write("Homer|38|D'oh!\n")
    file_obj.write("Elaine|42|Yada yada yada.\n")
    file_obj.write("Arthur|23|Don't panic.\n")

def parse_line(line):
    clean_line = line.strip()
    parts = clean_line.split("|")

    record = {
        "name": parts[0],
        "score": int(parts[1]),
        "quote": parts[2]
    }

    return record

def process_record(record):
    new_record = record.copy()

    if new_record["score"] >= 40:
        new_record["status"] = "high"
    else:
        new_record["status"] = "normal"

    new_record["quote"] = new_record["quote"].upper()

    return new_record

def serialize_record(record):
    out_line = (
        f"{record['name']}|"
        f"{record['score']}|"
        f"{record['status']}|"
```

```

        f"{record['quote']}\n"
    )

    return out_line

with open(src_path, "r", encoding="utf-8") as src_obj:
    with open(dst_path, "w", encoding="utf-8") as dst_obj:
        for line in src_obj:
            record = parse_line(line)
            new_record = process_record(record)
            out_line = serialize_record(new_record)

            dst_obj.write(out_line)

            print("Pipeline step:")
            print(f"record          = {record}")
            print(f"type(record)    = {type(record)}")
            print(f"new_record      = {new_record}")
            print(f"out_line        = {out_line!r}")

print()
print("Selected names from dir(record):")
for name in ["keys", "values", "items", "get", "copy", "update"]:
    print(f"{name:10} -> {name in dir(record)}")

print()
print("Final output file:")

with open(dst_path, "r", encoding="utf-8") as file_obj:
    print(file_obj.read())

```

The functions are short because each one owns only one part of the pipeline. `parse_line()` knows how to read one external line into a dictionary. `process_record()` knows how to modify one dictionary. `serialize_record()` knows how to format one dictionary back into one output line.

This shape is especially useful when the input format changes. If the separator changes, the parsing function changes. If the output format changes, the serialization function changes. If the rule for `status` changes, the processing function changes.

The file loop stays simple:

```

for line in src_obj:
    record = parse_line(line)
    new_record = process_record(record)
    out_line = serialize_record(new_record)
    dst_obj.write(out_line)

```

That loop describes the pipeline without hiding the boundary crossings. External text enters the program, becomes Python objects, is transformed, and becomes external text again.

List of Figures

Bibliography

- [1] Python Software Foundation. Built-in functions: `id()`. <https://docs.python.org/3/library/functions.html#id>. Accessed 2026-05-29.
- [2] Python Software Foundation. Built-in functions: `type()`. <https://docs.python.org/3/library/functions.html#type>. Accessed 2026-05-29.
- [3] Python Software Foundation. The python language reference: Objects, values and types. <https://docs.python.org/3/reference/datamodel.html#objects-values-and-types>. Accessed 2026-05-29.